# Prism

A reference for the language
and its implementations

Working draft

# Contents

**Introduction** **1**
The three parts . . . 1
How to read it . . . 1
Conventions . . . 2
Summaries . . . 2

**1 A language whose only subject is time** **9**
The gaslighting checkbox . . . 9
The counter, with the declarations . . . 10
One bug with four names . . . 11
Who may write, and when . . . 12
Why it has to be a language . . . 15
What each paradigm fixed, and where the problem came back . . . 16
The goals . . . 21
The refusals are the language . . . 22
The boundary, and what the rest of the book has to show . . . 25
A pure language . . . 27
The file viewer . . . 29

**2 Worked examples** **31**
Conventions used in every walkthrough . . . 31
1. The counter . . . 33
2. A text input with a cursor . . . 35
3. A todo list . . . 36
4. A button that relabels itself . . . 38
5. A file viewer . . . 40
6. A stopwatch . . . 42
7. A sandbox . . . 44
8. A vertical box . . . 46
9. A cache . . . 48
10. A chat client . . . 50
11. A line counter . . . 52
12. A tiny HTTP server . . . 54
13. Testing the counter . . . 56
14. A shell . . . 58
What the fourteen have in common . . . 60

**3 The language: sigils, lexical structure, values and types, expressions** **63**
Sigils and where time lives . . . 63
Lexical structure and layout . . . 66

Values and types . . . 79
Expressions . . . 86
Diagnostics . . . 94
Layout corpus . . . 97
Negative space of the language core . . . 100
`&` references, `read` and `update` blocks . . . 102
User-defined traits and modules . . . 105
A package system . . . 107

**4 Events, dynamics, and time** **109**
Three questions and one clock . . . 109
Events . . . 110
Dynamics . . . 115
Time . . . 121
Laws as tests . . . 128
The negative space . . . 136
How this feels in practice . . . 137

**5 Widgets and switching** **143**
What a widget is . . . 143
Widget blocks . . . 146
`rebuild` . . . 150
`!Switch`, `$Join`, and why they differ . . . 156
Lifting over `@` . . . 158
`for` . . . 160
What switching may not do . . . 165
Patterns . . . 165
The invariants, and who keeps them . . . 173
The negative space . . . 174

**6 Elements and interception** **177**
What an element is . . . 177
Tags . . . 178
`@Element` . . . 183
The creation rule . . . 185
Identity . . . 186
The element list . . . 187
Two laws across the boundary . . . 188
Dynamic fields . . . 189
Why state has to be there in its first occurrence . . . 193
`intercept` . . . 194
Recapitulation and depth . . . 199
A GUI library as interceptors . . . 200
Worked interceptors . . . 211
A Prism-hosted World . . . 217
Negative space . . . 219
Recapitulation . . . 220

**7 Worlds** **221**

The World is the outermost interceptor, and it is small . . . 221
The run call . . . 222
Snapshots, the oracle and pushes . . . 225
Turns and `Defer` . . . 229
Obligations of a World, and who each protects . . . 233
The world core . . . 234
General components . . . 237
The CLI World . . . 244
GUI: one vocabulary, spoken by many Worlds . . . 247
Networks in depth . . . 253
Async runtimes . . . 255
System calls and blocking operations . . . 257
Cross-platform . . . 258
Servers . . . 259
Worlds that nest . . . 260
Worlds and parallelism . . . 262
Testing a World . . . 262
Negative space . . . 264
What a World is, in one paragraph . . . 265
Hot reload of a widget subtree . . . 265
Persistence of holds . . . 267
Real time: a tick World and a bounded occurrence . . . 269

**8 Use cases in depth** **271**
Conventions . . . 271

**9 Errors** **315**
What can fail, and what cannot . . . 315
The occurrence as the unit of failure . . . 316
Containment at a construction boundary . . . 318
Containment to an owning subtree . . . 319
Static causality and runtime detection . . . 322
Whether pure code may panic . . . 323
What the program sees . . . 324
What the World sees . . . 326
Error messages worth getting right . . . 329
Worked scenarios . . . 330
The invariants, collected . . . 337
Causality in types . . . 338

**10 Conformance and testing** **341**
Tests against what a World sees . . . 341
The Test World . . . 342
The script language . . . 344
The law tests . . . 349
The integration programs . . . 365
The layout corpus . . . 378
Typed-AST goldens and negative goldens . . . 381
Runtime unit tests and the invariant . . . 384

World tests against the world core . . . 386
CLI golden transcripts . . . 387
The differential test . . . 388
The quotable-test rule . . . 388

**11 The Rust runtime** **391**
What the runtime does not need . . . 391
The crate as it stands today . . . 392
Testing the runtime . . . 432
The gap . . . 435
Known issues . . . 437
Performance notes . . . 439
Parallelism . . . 440
Values representation . . . 441
Parallel runtimes . . . 451
Performance and constant factors . . . 453

**12 The Rust language implementation** **455**
The thesis . . . 455
The pipeline . . . 457
Lexer and layout . . . 458
Parser . . . 462
Resolver . . . 469
Elaboration . . . 472
Closure conversion . . . 482
The lint pass . . . 484
The closure compiler . . . 486
The typed-AST text form . . . 492
Elements and tags in the compiler . . . 497
Diagnostics . . . 500
Testing the compiler . . . 502
Deep in: a flat register listing as an alternative lowering . . . 504
What is deferred . . . 505
Native backends . . . 506
Incremental compilation . . . 509

**13 Rust Worlds and the world core** **513**
Where a World stands . . . 513
The `prism_world` crate . . . 515
The Test World . . . 529
The CLI World . . . 534
A tokio-based World skeleton . . . 545
The GUI World . . . 547
The X11-over-sockets World . . . 552
Conformance tests for the core . . . 554
CLI golden transcripts . . . 557

**14 The Haskell implementation** **561**
What the second implementation is for . . . 561

Interpret, do not compile . . . 562
The frontend . . . 563
The evaluator . . . 565
What must not be used . . . 579
The Test World and the transcript printer . . . 581
The reflex-dom World . . . 582
Differential testing . . . 584
What it cannot check, and what it alone can . . . 585
The smallest version that pays for itself . . . 586
What the Haskell implementation leaves out . . . 587

**15 Plan** **589**
How a milestone is cut . . . 589
The two tracks and where they meet . . . 590
Milestone 1: the runtime, single-threaded and transactional . . . 590
Milestone 2: the runtime boundary . . . 593
Milestone 3: interception and `for` . . . 596
Milestone 4: frontend to typed AST . . . 598
Milestone 5: the closure compiler . . . 601
Milestone 6: the conformance suite in full . . . 603
Milestone 7: the world core and the CLI World . . . 605
Milestone 8: the Haskell frontend . . . 607
Milestone 9: the Haskell evaluator and Test World . . . 608
Milestone 10: GUI Worlds . . . 609
Milestone 11: errors . . . 612
What is not on the plan . . . 613
Working with a model on a milestone . . . 614
Designs that wait on a fact . . . 616

**16 Appendices** **619**
A. The Reflex correspondence . . . 619
B. Prior art . . . 623

**17 A night's implementation: a report** **669**
What it is and is not . . . 669
Where I was wrong about the book . . . 669
What the book had wrong . . . 670
What just worked . . . 671
What I would tell the next implementer . . . 671

# Introduction

This book is a working reference for Prism, a functional reactive language, and for the programs that implement it. It is an internal document. Its readers are its author and the collaborators the author hands it to, and its job is memory: the design is too big to hold in one head, and every question of the form "what does the language do when…" should have an answer written down in one place, with the reason next to it. It is also the source the real specification and documentation will be written from later, by hand. That later writing will be a separate piece of work in a different voice, and nothing here is meant to be lifted out and published as it stands.

The book therefore favors completeness over polish. A chapter may repeat a rule that another chapter states, because the reader who arrives through a search should not have to follow a chain of references. A chapter may argue a decision at length, because the argument is what lets a later reader tell a choice from an accident. And where the design has taken a choice on thin evidence, the chapter says what the evidence is and what would change it, on the merits, in the section the choice lives in.

## The three parts

The first part is the language. Chapter 1 says why it exists and what it refuses to have. Chapter 2 shows programs working, occurrence by occurrence, before anything is defined. Chapters 3 to 5 define the language: the lexical structure, values and expressions; events, dynamics and time; widgets and switching. Chapters 6 and 7 define the boundary: elements and interception, and the Worlds that stand outside every program. Chapter 8 works larger use cases through the whole machine, and chapter 9 says what goes wrong and how it is reported.

The second part is the implementations. Chapter 10 is the conformance suite, which is the contract between them. Chapters 11 to 13 are the Rust implementation: the runtime, the frontend, and the Worlds. Chapter 14 is the Haskell implementation, which reads the same chapters independently and runs the typed AST on Reflex, so that a disagreement between the two points at a sentence of the book.

The third part is the plan and what lies beyond it. Chapter 15 cuts the work into milestones that a model or a person can be handed one at a time. The designs that wait on a fact (a value representation, references, native backends, modules, hot reload, persistence, real time) each sit at the end of their subject's chapter, and chapter 15 closes with an index of them. Chapter 16 collects the reference tables: the lexical summary, the prelude, the tag declarations.

## How to read it

A first read goes in order through chapters 1 to 7. A reader who wants only the model can stop after chapter 5 and one World. A reader implementing something starts from the chapter for that piece, reads chapter 3 or 4 as needed for the sentences it implements, and treats chapter 10's goldens as the arbiter. Chapter 16 is for lookup. The summaries at the end of the book give each chapter in a

page, and are the quickest way to find where an idea lives.

## Conventions

Code in Prism uses the sigils the language is built on: `!` for events, `$` for dynamics, `@` for blueprints and `?` for optionals. Chapter 3 defines them all; chapter 2 uses them before it does, deliberately, so that each one is first met doing work.

Some sections are marked as under-planned. That means they have had less effort, and nothing more: the design in them is as committed as anywhere else, the questions in them are listed with their merits, and a reader is more likely to find a gap there than elsewhere. The largest of these are the end-of-chapter designs that chapter 15 indexes. Where a mechanism's spelling is tentative, the section says why on the merits, for instance that the interaction of a comparison operator with the hold arrow has not been worked through, and says what would settle it.

The book does not track its own history. A chapter describes the design as it is. The one exception is the implementation chapters, which describe the current code where the plan changes it.

## Summaries

What follows are short summaries of each chapter, as an index to the argument.

### 1. A language whose only subject is time

The chapter opens on an afternoon every reader has lived: a "show archived" checkbox that unticks itself one time in five, because a setter notified its observers one line before it wrote its field, and a re-render read the model back half-updated. The thesis follows from that bug. The many bugs of interactive programs are one bug, a value whose set of writers is not a fact about its type. Functional reactive programming makes the writers a fact with two types, a value that happens and a value that holds, tied by one law: a held value changes only through the events named in its definition, and only when the moment ends. Prism is a language that keeps that law and no other, and the list of what it refuses is the evidence.

The chapter first names the four faces of the bug (uncontrolled mutation, action at a distance, "who can change this?", order within a moment) and shows one cause under all four. It then asks for a bigger version of `const`: a policy on who may write and when. A policy removes sentences from a language, and a library can only remove what its host lets it name, so the policy has to be a language. Each paradigm gets a picture, the constraint it adds, and the place its own documentation shows the problem returning: callbacks, objects and the Observer pattern's "make sure Subject is self-consistent", React's tearing and Suspense, signals as the behavior half of FRP found independently, Rx as the event half without behaviors as a law, Elm as the friendly proof that a language-level mutation policy works, Haskell's `dFoo`/`eFoo` naming and open door to `IO`, and the synchronous languages as allies built for fixed structure.

### 2. Worked examples

The chapter opens with the claim that the programs are the language and the walkthroughs are the laws applied to them. It offers fourteen programs, from a counter to a shell that hosts other programs, each followed by an occurrence-by-occurrence account of what the World sends, what fires

in dependency order, what commits, and what the element list looks like afterwards. A reader who has read the specification can predict each walkthrough and check; a reader coming cold meets every later rule here first.

The conventions fix the vocabulary for the book: occurrences numbered from zero with zero as the build, inputs addressed by path and simultaneous, the only promised order being dependency order, and a small shared set of declarations under `Gui`, `Ui`, `Fs`, `Time` and `Process`, each declared before first use. The first programs build up the core: the counter, a text input with a cursor, a todo list driven by `for` over a keyed map, a button that relabels itself (a dynamic field lifted into `rebuild` with a fresh slot per change), a file viewer over a state element, and a stopwatch reading `Clock` as state.

## 3. The language: sigils, lexical structure, values and types, expressions

The chapter opens on `total = price * quantity`, a line where you cannot tell without scrolling whether it ran once or on every keystroke, and on the Reflex habit of naming variables `dPrice` and `eQuantity` against the library author's advice. The thesis: a Prism program is a set of equations between things that are true, things that happen and things that exist, and the sigils on the names say which before a type checker runs. The chapter covers the part of the language that holds still, written so two parsers built from the text agree on every program.

Sigils come first. `!` is an event, `$` a dynamic, `@` a blueprint, `?` an optional, and bare is plain; they stack leftmost outermost, functor values nest inside plain aggregates, and a function's name takes the sigil of what it returns. The section argues against the alternatives: optional sigils would leave the boundary between styled and unstyled code unreadable, and inferred sigils shown by an editor would make review depend on tooling. The lexical section then goes down to the character, because the conformance suite fails the day the Rust and Haskell lexers disagree: comments, identifiers, keywords, literals (integer-spelled literals are `Nat | Int` by inference defaulting to `Int`, decimal-spelled are `Float`, `-1` is an `Int`), operators, the offside rule with `:` as block opener and more-indented lines continuing a statement, and paren-less calls where commas belong to the outermost call on the line. The chapter also names the kind of each sigil (event functor, dynamic functor, widget monad, optional) and argues for the offside rule at the top of its Layout section, and its last three sections cover `&` references with `read` and `update` blocks, user-defined traits and modules, and a package system.

## 4. Events, dynamics, and time

The chapter opens by saying that everything a Prism program does happens in an occurrence, and an occurrence is a transaction with a before, an after and no visible inside. Every interactive program asks three questions over and over: what is true now, what just happened, and what exists. Most languages answer with a variable, a callback and an object that disagree about time; Prism answers with three kinds of value whose sigils name their relation to one clock. The claim is that a small set of laws suffices, and the tag law alone turns a whole class of cyclic programs into legal ones.

Events come first, introduced by the handler that ran at the wrong time. An event is a thing that occurred, not an object with `.preventDefault()`. Simultaneity is defined, not raced: a program never observes two events in order inside an occurrence. `!never` is the identity for merge, and there is no `!always`. Merge is leftmost, with the reasoning for why the left wins. `on` gives an event a dynamic's value from before the occurrence. Lifting over one event fires exactly when that event fires, and `!Filter` is the one way to drop firings. `!Switch` gives a parent one event across every build of a `rebuild`. Events are values and need no subscription or disposal.

## 5. Widgets and switching

The chapter opens on the subtree that was removed while its listener lived on, and on the component remounted by a parent's render that fetched twice and let an old response overwrite a new one. Both bugs treat something with a lifetime as if it had none. The thesis: a widget is the one thing in Prism with a lifetime, so state, IO, identity, cycles and subscriptions are all owned by one, and three constructs (`rebuild`, `for`, and lifting over `@`) let a program change its own structure while two primitives, `!Switch` and `$Join`, bring results across the boundary.

The first part separates blueprint from widget: a blueprint is the value of type `@a`, a widget is what one bind builds, with identity, a path, holds, placeholders, slots and children, and widgets are never values. Building is the only doing; the World performs everything. The button is the archetype because it has every property at once, and the word stops meaning GUI once a socket is seen to have them too. Widget blocks are where "one way to change anything, written next to the thing" becomes syntax: holds appear only there, bindings are mutually recursive through delayed edges, every binding is marked build-time or occurrence-time, results can be bundles, blueprints can be arguments, `@main` takes nothing (arguments and environment are state elements), and `@started` is a pure `rebuild` over a constant that fires once at build.

## 6. Elements and interception

The chapter opens on the component that fetched on mount, fetched again on re-render, and showed a stale avatar after a race the test suite could not reach. The thesis: an element is the only thing a program can say to the outside and the only thing the outside can address in reply. The chapter defines that channel, the rule that nothing it asks for is answered in the occurrence it asked, the two identities of a slot, the element list as an ordinary dynamic, and `intercept`, the primitive that lets a widget stand where the World stands so that toolkits, sandboxes, caches and mocks are Prism code.

Tags declare elements: a name, plain fields, and any subset of `requests`, `responds` and `state`. A dozen declarations across GUI, files, network, time and process show why each part is what it is, and four rules follow from elements being plain data. `@Element` makes a slot and returns what the tag declares. The creation rule is quoted in full and argued: an `Fs.Load` that read on creation would save a line and cost the three things `@started` exists for. Its complement is that state has a value in its creation occurrence, an observation and not a firing. A slot has two identities: a path for conversations (bind positions, `rebuild` generations, `for` keys, never reused) and a value for facts (equal state elements share one dynamic).

## 7. Worlds

The chapter opens on the context object handed over at startup, stashed in a global, read after teardown, and eventually discovered to be the render loop, the clock and the filesystem at once. The thesis: a program cannot write a byte, read a clock or hand anyone a callback, so something outside must do all of that, and the design of the boundary lies in how little that thing has to know. A World is the outermost interceptor, written in the host language, the owner of the loop and the only starter of occurrences, driving the runtime through one synchronous call per occurrence with plain data both ways.

The run call is given whole. In: responses by path (at most one per path per occurrence), pushes by element value, and an oracle answering state from the snapshot as the occurrence opened. Out: the element list, request firings per slot, the result and errors. The list is a description the World reconciles to, never a trigger. Nothing else crosses, and occurrence zero builds `@main` with the oracle

live. The snapshot discipline comes next, opened by the Binance and Kubernetes snapshot-plus-changes procedure: the World keeps the last value supplied for every state element, the oracle cannot fail because unavailability is a case of the state type, and the World stitches native reads and watches before anything reaches the snapshot. A directory listing is traced through the race, and events-only and dynamics-across designs are argued against. Three sections appended after it cover hot reload of a widget subtree, persistence of holds, and real time through a tick World and a bounded occurrence.

## 8. Use cases in depth

The chapter opens on eight hard moments: the file that changes on disk as the user presses undo, the fill that lands with the book update that would have re-quoted it, the emergency stop that arrives with the tick, the member who leaves a room as someone speaks to it, the source that changes as its compile finishes, the window that quits under the click that raised it, the jump pressed on a frame boundary, the time range that changes as the old range's data arrives. The thesis: these programs are not toys, and each one, taken far enough, reaches a moment where two things happen in one instant and the answer is a line of the program.

Each deep dive follows one frame: the element vocabulary, the structure of the program, the hard moment walked occurrence by occurrence, what the goals buy, and where the language is awkward or wrong. The text editor has exactly the states a framework claims to manage (text, cursor, undo, dirty flag, search) and the outside events it does not, and giving each state one defining line decides which write wins. The market-data consumer shows what "act on exactly the state you saw" means and treats latency as an implementation question. The control loop gets a synchronous-language reading and admits where it is wrong.

## 9. Errors

The chapter opens on the stack trace that ends in a map lookup: a key present a moment ago, `null`, a dereference, a guard added forever. The thesis: a program's own failures are values, and the failures the language cannot make into values do the least damage the transaction allows. The first half is a discipline the type system enforces; the second covers logic errors and broken contracts, such as a cycle that exists only once two widgets are wired together at run time.

The first content section sorts what can fail: missing files, closed sockets, division by zero and out-of-range indexes are cases of a type, and leaving their arm out of a `match` is a compile error. The occurrence then becomes the unit of failure, opened by the half-updated model every reactive system has shown and by Spider's single exception for causality loops. A cycle, overflow or protocol error aborts the occurrence, and the transaction drops its buffered writes. Containment at a construction boundary is the one smaller unit: a `rebuild` whose new generation fails to build does not fire, keeps the tree consistent, and reports through `errors`. Containment to an owning subtree, quarantine, answers the dashboard with one broken panel among twelve: the World passes back the paths it wants quarantined on the next run call. A section appended after the invariants adds causality in types.

## 10. Conformance and testing

The chapter opens on the white-box test that read a private field, broke on a rename, and was fixed by exporting the field. The thesis: a specification only one implementation can check describes that implementation, so the language is tested from the one place two implementations must look identical, the World's side of the boundary. Every test sends plain data to elements and reads plain data back,

never naming a widget, hold or runtime object.

The Test World is a World with no effects, written once per host language on the world core, that accepts every tag and answers to a script. Everything it sees is printed turn by turn in one canonical transcript form that the harness compares and the differential test diffs. The script language follows: a test written `test name, @w:` with one line per turn (`build`, `idle`, `click`, `send`, `push` or `with` on the line, and `=> expect …` or `=> settles` checks indented under it), element patterns (named fields with `..`, `Ns.*(..)`, `ignore`), and every step (`expect`, `expect_top`, `expect_none`, `expect_top_none`, `expect_count`, `send`, `click`, `with`, `push`, `returns`, `fired`, `quiet`, `requested`, `unrequested`, `errors`), each with a script. Depth inside containers uses the GUI library's `children` field and `ChildInput` convention, and turns define when checks run: always between turns, against committed state.

## 11. The Rust runtime

The chapter opens on the list every runtime author knows: a scheduler, dirty tracking, re-entrancy guards, deferred effects, an unsubscribe story with its leaks. The thesis: the Prism runtime needs none of them. It holds events, behaviors and dynamics as nodes with heights, runs one occurrence per call by popping heights in order, commits every hold at the end, and owns the tree of widgets that is the program's whole state. A height-ordered action map, a commit list, weak subscriber lists and one synchronous call are the whole engine.

The first half describes `prism_runtime` as it stands on branch `on-fire-repros`, with 38 tests passing and no boundary yet. It covers values (`AnyValue` as a 24-byte enum behind `Value<T>`), the runtime core loop, lazy events that disconnect when unheld, behaviors as the pull half with the only dirty tracking, dynamics where a hold's change event is its input event, blueprints with `DynBlueprint` so a `rebuild` can rebuild into another kind, and the test backend that reads behaviors directly. The 38 tests are mapped to the laws they pin, and two occurrences are traced through the code. Three sections appended after them cover values representation, parallel runtimes, and performance and constant factors.

## 12. The Rust language implementation

The chapter opens on the compiler for a language that runs, whose back half was a fetch-execute loop, and on the reactive framework built over it that became a second runtime fighting the first for control of time. The thesis: a Prism compiler does not emit code that computes. It emits code that, run once, calls the runtime's constructors in order with the right captures, and the resulting graph does the computing. The frontend decides what every line means and writes it into one tree, the typed AST; a lint checks the tree after every pass; a closure compiler walks it once.

The pipeline is laid out and then each stage in order. The lexer and layout pass produce tokens with spans, implement the offside rule, treat inline `:` as a plain token, and keep only a column stack, bracket depth and a whitespace flag. The parser builds a parse tree from a grammar sketch, records spans, and leaves meaning undecided. The resolver handles scopes, qualified names and namespaces, intrinsics, dependency analysis into groups, and placement of placeholders for recursive groups. Two sections appended after the deferred list cover native backends and incremental compilation.

## 13. Rust Worlds and the world core

The chapter opens on the year the framework's context object became a door everything walked through, until a test could not run without a browser. The thesis: a World is the only part of a Prism system with effects, written in Rust against one synchronous call, and everything a World must

get right that is not specific to a screen, terminal or socket is written once in a crate, `prism_world`, so a World is that crate plus a handful of components.

The chapter first lists what the crate refuses to contain, each with its reason, then gives the crate as code: the types, the six-method `Component` trait with each method traced to an obligation, registration and the `World` struct, the turn loop (pushes applied after the first occurrence commits, re-queued on failure, `Defer` occurrences with no new pushes), `DeferTable` keyed by path with generations stripped, snapshot and pushes, slot tracking with discarded late results, `Unsupported`, error reporting, and a harness that drives a runtime from any event loop.

## 14. The Haskell implementation

The chapter opens on handing a specification to a second person who built something that passed every test and still disagreed on the first untested program, because both had read one sentence differently. The thesis: a specification with one implementation describes that implementation, so the Haskell implementation is a second, independent reading of the language chapters, in the language whose FRP library supplied most of those chapters' semantics. It shares nothing with the Rust side but the typed-AST text form and the transcript format.

The first decision is to interpret, not compile: the typed AST is interpreted at runtime against Reflex, never translated to Haskell source for GHC, so there is no second compiler to be wrong in. The frontend is a separate lexer, parser and elaborator written from the book and the goldens, whose output must match the Rust frontend's text form character for character, with `diff` as its test. A section appended after them, "The smallest version that pays for itself", orders the first Haskell binary by which law tests it can run.

## 15. Plan

The chapter opens on the plan that was a list of features, where the first week went to choosing a starting point because every feature needed a piece of every other. The thesis: cut milestones along the lines the design already has (the run call, the typed AST, the transcript) so each milestone can be handed to a model with the book and the previous code and come back with work judged by named tests and matching transcripts. Eleven milestones run in two tracks that share no code and meet when the compiled counter's transcript matches the hand-written one byte for byte.

Each milestone follows one frame: what it waits on, the work in steps, a done criterion, what it proves, and its risks. Milestone 1 makes the runtime single-threaded and transactional, with `Rc`/`RefCell` first and alone, a re-entry flag reporting `Cycle` instead of a double borrow, and `Dynamic::join`. Milestone 2 adds the boundary: element values, tags, slots, paths as values in tree order and never reused, and the run call, proven by a harness printing the counter's transcript. Milestone 3 adds `InterceptBlueprint`, state supply and `for`, with the risk that a supplied widget built mid-grandchild must sit under the right parent. The chapter closes with "Designs that wait on a fact", an index of the appended sections of chapters 3, 7, 9, 11 and 12 with what each must answer.

## 16. Appendices

The chapter opens as four reference tables for a reader who knows the design and wants one page to check against. Appendix A sets Prism beside Reflex construct by construct, opened by the afternoon a `Dynamic` read through `current` in one place and `updated` in another disagreed by a frame. The thesis is that almost every choice in the language is a choice to keep something of Reflex's or remove it, and the notes show both spellings side by side and point to the chapter that owns each argument.

The Reflex notes cover the hidden behavior half (no `current` except inside the Haskell evaluator's tag node), lifting in place of `fmap` and `zipDynWith`, one non-prompt switch in place of four variants, no `PostBuild` with `@started` in its place, elements instead of `Requester` and `MonadQuery`, a plain-data boundary in place of an improvised host, and `listWithKey` as `for` with prompt consolidation.

## Chapter 1

# A language whose only subject is time

Here is a counter in Prism, the whole of it:

```
@main:
    !increment <- @Element Ui.Button(label="Increment")
    !decrement <- @Element Ui.Button(label="Decrement")
    let !delta = -1 on !decrement | +1 on !increment
    $count <- 0 then $count + !delta
    @Element Ui.Label(text="Count: {$count}")
```

The fourth line is the subject of this chapter. `$count` starts at `0` and becomes `$count + !delta` whenever `!delta` fires. That line is the only place in the program, or in any other file, where `$count` can change, and the compiler knows it. Every bug in the next section is a program that lacked that line, and every section after it is about what you get when a language has it and refuses everything that would break it.

## The gaslighting checkbox

You have had the afternoon where a checkbox un-checks itself. The ticket says: tick "show archived", the box ticks, and then, about one time in five, it unticks. You reproduce it on the second try. You follow the click into the handler, which sets a flag on the model. The model's setter notifies its observers. One observer is the list, which re-renders. The re-render rebuilds the filter bar, and the checkbox in the filter bar reads its state back from the model. You put a breakpoint in the setter and read the stack. The notification went out from inside the setter, one line before the field was assigned. The list read the old value, drew an empty box, and then the field changed under a screen that had already been painted. You move a line. It works. You ship it.

Three months later the ticket comes back. A different checkbox, the same symptom. The setter is fine this time; the line you moved is still where you put it. This time the re-render wakes a second observer, added in the spring for a feature you never saw, which writes the flag from a copy it cached before the click. You fix that too. You know there will be a third.

Or the counter. A field holds a number. A button increments it. A label shows it. Then a keyboard shortcut increments it too, a reset button zeroes it, a message from the server sets it to the server's count, an undo stack restores an old value, and next week's feature will do something nobody has thought of yet. Six writers, each a callback written wherever its trigger lives: six files, in six modules with careful boundaries and private fields. Ask the counter two questions. What can change it? The answer is a search, and the search finds the writers it can find; it does not find the one that reaches the counter through a method three calls away. If the reset and the server's message land in the same frame, which wins? Whichever handler was registered first, typically, and that depends on the

order the modules loaded, which is nowhere in the program's text. The modules kept the writers tidy. They said nothing about when.

Or the row. You remove an item from a list. Its click listener was subscribed to a store, so the store still holds the listener, and the listener still holds the row. On the next update the store calls it, and it writes to a widget nobody can see. In the browser's memory tools the detached subtree is alive: reachable, so not garbage. The list removed it, and the removal meant nothing, because something else could keep it.

Or the dependency array. An effect runs when `a` or `b` changes, and `[a, b]` sits next to it, written by hand. Someone adds a read of `c` inside the effect and leaves the array alone. The code and its declared dependencies are two statements of one fact, and they have drifted apart. The linter noticed, in the spring, and someone suppressed the warning with a comment explaining why.

None of these was a stupid bug, and none was written by a careless person. Each was written by someone competent at six in the evening. In each one, nothing complained.

## The counter, with the declarations

Here is the program from the top of the chapter with everything it uses, so nothing is hidden.

```
type Element = choice ..
type Click = click

tag Ui.Button of Element:
    label: Text
    responds: Click

tag Ui.Label of Element:
    text: Text

@main:
    !increment <- @Element Ui.Button(label="Increment")
    !decrement <- @Element Ui.Button(label="Decrement")
    let !delta = -1 on !decrement | +1 on !increment
    $count <- 0 then $count + !delta
    @Element Ui.Label(text="Count: {$count}")
```

`Ui.Button` and `Ui.Label` are tags of the GUI library: a button has a label and responds with a click; a label has a text and does nothing. `@Element` makes a slot for an element and gives back what it responds with, so `!increment` is the Increment button's clicks. `on` takes a value at an event, so `-1 on !decrement` fires `-1` whenever Decrement is clicked. `|` merges two events, the left one winning when both fire in one occurrence. `then` holds: `$count` starts at `0` and becomes `$count + !delta` whenever `!delta` fires, with `$count` on the right read through a tag, the count before the click. Bindings in a block are recursive, so `$count` can appear on both sides. The label's text is a dynamic, and a dynamic where a plain field goes lifts the element into a `rebuild`: a fresh label slot each time the count changes. The block is a description. Nothing happens until a World builds it.

Here is the World's side. In the build occurrence it gets a list:

```
  0:      Ui.Button(label="Increment")
  1:      Ui.Button(label="Decrement")
  2/#0/0: Ui.Label(text="Count: 0")
```

The user clicks Increment. The World opens an occurrence with a click addressed to path `0`. `!increment` fires, `!delta` fires with `1`, `$count + !delta` fires with `0 + 1`, the label's text changes

promptly, the label's `rebuild` builds generation `#1`, and everything commits at once. The World reads the new list, sees `2/#1/0: Ui.Label(text="Count: 1")` where generation `#0` was, and redraws. Nothing told it to. Under a GUI World the library's interceptors turn these tags into a scene; under the Test World a script sends the same click and reads the same list:

```
test counter_counts, @main:
    build
        => expect Ui.Label(text="Count: 0")
    click Ui.Button(label="Increment")
        => expect Ui.Label(text="Count: 1")
        => expect_none Ui.Label(text="Count: 0")
    click Ui.Button(label="Increment"), click Ui.Button(label="Decrement")
        => expect Ui.Label(text="Count: 0")
```

The third turn delivers two clicks in one occurrence, and the label says `0`: the merge took the left operand, the decrement, and the program said so on its third line. Ask this program the questions from six in the evening. What can change `$count`? Read one line: `!delta`, and nothing else. If both buttons fire in one occurrence, which wins? Read another: `|` takes the left. Is there a setter somewhere else, in another file, registered by next week's feature? There cannot be. The language has no sentence for it.

## One bug with four names

The field has names for the afternoon's bugs. Uncontrolled mutation. Action at a distance. "Who can change this?" Order within a moment. Four names, and one thing underneath: a value whose set of writers is not a fact about its type. Because the set is open, you cannot answer who changes the value. Because the writers live elsewhere, a write here moves a value there, and the connection sits near neither end. Because the writers are unordered, two writes in one moment have no stated order, and the program's history decides.

A callback answers "what do we do when this happens". Keeping an invariant needs the other question, "what can happen to this", and a program made of callbacks has no place where that answer is written. Maier, Rompf and Odersky opened their case against the observer pattern with one sentence, "Programming interactive systems by means of the observer pattern is hard and error-prone", and two of their nine complaints are the two halves of the afternoon: "there is no guarantee for data consistency in the observer pattern", and the example "is hard to understand because the control flow is inverted".

So every reactive system of any size grows the same things inside it. Dirty flags, so a value knows to recompute. A batching layer, so two writes in one handler produce one redraw. An ordering rule, so a derived value runs after the values it derives from. And a page on the wiki titled "Conventions" that says who may call what, which is a type system written in a wiki. Together these are a runtime, written badly, inside the application.

Moving the runtime into a framework helps, and the rules still leak into your head. React's docs are exact about it: "React waits until *all* code in the event handlers has run before processing your state updates", and "React does not batch across *multiple* intentional events like clicks". Write `setNumber(number + 1)` three times in one handler and the number goes up by one. Write `setNumber(n => n + 1)` three times and it goes up by three. Same three lines, a different spelling of the setter, a different answer, and the rule that decides it is in the framework's documentation, not in your program. In Prism the same three increments are three events, and the question "how far does the count move" is answered by the merge you wrote: `!a | !b | !c` fires once per occurrence with the

leftmost value, so one; `!Union` over a map of three keyed events fires with all three, and a hold over `Size` of it moves by three. Either answer is a line in the program and a line in the transcript.

Two communities needed a word for seeing a value half-changed. React's is tearing: "In a user interface, by 'tearing' we mean that a UI has shown multiple values for the same state", which the React 18 working group called "a necessary consequence of concurrency". The FRP literature's is glitch: "such a momentary view of inconsistent data is known as a glitch". The Leptos book draws the standard diamond (one signal, two memos over it, an effect over both) and says a naive push implementation is "glitchy (`D` actually runs with the incorrect value for the second memo during its first run.)" Here is the diamond in Prism, and the script that says it does not glitch:

```
@diamond:
    !go <- @Element Ui.Button(label="Go")
    $a <- 0 then $a + 1 on !go
    let $b = $a * 2
    let $c = $a * 3
    let $d = $b + $c
    @Element Ui.Label(text="d={$d}")

test diamond_changes_once, @diamond:
    build
        => expect Ui.Label(text="d=0")
    click Ui.Button(label="Go")
        => expect Ui.Label(text="d=5")
        => expect_none Ui.Label(text="d=2")
        => expect_none Ui.Label(text="d=3")
```

`$d` never has the value `2` or `3`, not for a moment and not in any implementation, because a derived dynamic changes once per occurrence, with all its inputs settled. Every serious system solves this, each inside its own scheduler, and each teaches you the scheduler's rules. The checkbox afternoon was a glitch you saw with your own eyes.

## Who may write, and when

You already use one small subtraction and think well of it. `const` says this name will not be reassigned. It removes sentences from your language, and your programs got easier to read when you started using it. Hold on to that feeling; the rest of this chapter asks for bigger versions of it.

Every language answers three questions about a value: who may write it, from where, and in what order. C's answer is anyone, anywhere, any time. Rust's is in its book: "At any given time, you can have *either* one mutable reference *or* any number of immutable references", and "the benefit of having this restriction is that Rust can prevent data races at compile time". A data race, in the same chapter, is two pointers at the same data, one of them writing, and nothing to order them: a rule about who and when. The borrow checker enforces it on every expression, and no crate could have done the same job. Haskell's answer is: whatever the monad you are in says, and you choose the monad.

Prism's answer is the fourth line of the counter. A value changes only through the events named in its definition, and only when the moment ends. Take each bug from the afternoon and try to write it.

The second checkbox observer wants to write the flag from somewhere else:

```
$checked <- False then !toggle | !reset
# elsewhere, in the spring's feature:
$checked = True              # does not parse: there is no assignment
!spring_write <- ...
$checked <- $checked then !spring_write     # does not compile: $checked is already defined, once
```

The only way for `!spring_write` to move `$checked` is to appear in the one line that defines it, `False then !toggle | !reset | !spring_write`, and then it is not a surprise observer; it is in the list.

The counter's six writers want an order. In Prism they have one, and it is spelled:

```
@counter:
    !increment <- @Element Ui.Button(label="Increment")
    !reset <- @Element Ui.Button(label="Reset")
    !server <- @Element Net.Counts          # responds with the server's count
    $count <- 0 then $count + 1 on !increment | !server | 0 on !reset
    @Element Ui.Label(text="Count: {$count}")
```

If the server's message and the reset land in one occurrence, the server wins, because it is to the left. Swap them and the reset wins. There is no third answer, and a script can check which one you chose:

```
test server_beats_reset, @counter:
    build
    send Net.Counts, 9, click Ui.Button(label="Reset")
        => expect Ui.Label(text="Count: 9")
```

The detached row wants a reference to a widget that something else can keep. There is no such value. A row in a `for` is built when its key appears in the map and torn down when the key goes, and its slots leave the list in that occurrence:

```
for key, $item <- $items:
    !remove <- @Element Ui.Button(label="Remove")
    @ key on !remove
```

The block's result is the row's own key, fired when its button is clicked; the parent holds a map and removes the key; the row is gone, its button is gone from the list, and the World can never address it again, because the only name it ever had was a path in a list it is no longer in. There is no listener to leak because there is no listener: the parent reads an event, and an event is a value, not a subscription.

The dependency array wants you to say twice what depends on what. In Prism you say it once, because the dependency is the expression:

```
let $label = "Count: {$count}"
```

`$label` changes exactly when `$count` does. Add a read of `$unit` to the text and `$label` depends on `$unit`, with no array to forget, because there is nothing to write that could disagree with the expression.

A rule like this needs a noun, and the plain one is *mutation policy*: a rule about who may write a value, from where and when, which the language keeps on every expression, so a program that breaks it does not compile. A policy is in force whether or not anyone wrote it down, and every program in the afternoon had one: C's, in a framework's clothes.

The two types behind the policy are older than any framework you use. Elliott and Hudak's Fran gave them their names in 1997: a *behavior* is a value that can vary over time, and an *event* is a what-and-when pair. The Haskell wiki's one-line pitch is the checkbox's own: "A better way to

represent an edit widget's content is a time-varying value, called a behavior." Reflex, the library that made the idea practical, states the law that ties the two together in its `Dynamic` documentation: "a combination of a 'Behavior' and an 'Event', with a rule that the 'Behavior' will change if and only if the 'Event' fires". And its `hold` documentation states the rule for reading one: a sample taken in the moment that changes it "will see the **old** value of the 'Behavior', not the new one".

Prism keeps both and gives up one thing on purpose. Elliott's behaviors are continuous; a Prism dynamic is piecewise constant, a value that changes at discrete moments and holds between them. Elliott's own paper on implementing FRP begins "Functional reactive programming (FRP) has simple and powerful semantics, but has resisted efficient implementation." The restriction to piecewise-constant values is what can be implemented by pushing changes, and the part of FRP that survives it is the part that is a mutation policy: a value changes only when its event fires, and nothing observes the middle.

Prism writes the type in the name. `x` is plain, true now and forever. `!x` is an event, a value that occurs. `$x` is a dynamic, a value that is true now and changes at occurrences. `@x` is a blueprint, a description of something that, built, has a lifetime. `$count + !delta` tells you, before you know what either name means, that one of them is live and one of them happens, and that the sum happens; a compiler sees the same thing, which is why you never write `map`.

The moment needs a name too. Reflex says *frame*, which suggests a screen refresh and a loop that produces them. The synchronous languages say *instant* or *tick*, which suggest a clock. Prism says *occurrence*, because the word claims only that something occurred: there is no moment without an event, and an occurrence exists because the outside world delivered something. An occurrence has a before and an after and no inside. Within it every dynamic reads as it stood before; at its end every hold commits at once; nothing anywhere sees a half-finished one. The script that pins this is four lines:

```
@reads_before:
    !go <- @Element Ui.Button(label="Go")
    $n <- 0 then $n + 1 on !go
    $seen <- 0 then $n on !go
    @Element Ui.Label(text="n={$n} seen={$seen}")

test tag_reads_before, @reads_before:
    build
    click Ui.Button(label="Go")
        => expect Ui.Label(text="n=1 seen=0")
    click Ui.Button(label="Go")
        => expect Ui.Label(text="n=2 seen=1")
```

`$n on !go` reads the count before the click, in every implementation, every time; `$seen` is always one behind. A test that checks it is a test of the language, and the conformance suite has this one.

You know this already under another name. Nobody thinks ACID is an implementation detail of a database. It is what the database promises, and you would be angry at one that let a reader see half a write. The occurrence is a transaction in that sense, a promise to the program. The synchronous languages wrote the same promise for hardware controllers decades ago: "All writers run before any readers do." And Prism gets it for nothing, because there is no assignment. Every write is a hold's commit, every commit can wait for the end of the occurrence, and an occurrence that goes wrong can be dropped whole. You get transactions for free because you gave up the thing transactions protect you from.

## Why it has to be a language

A library adds sentences to a language. A policy removes them. So the question for any policy is how much a library can remove, and the answer is: whatever its host lets it name.

That is more than nothing, and the libraries have earned the credit. Reflex removes a great deal. `MonadHold` "is required to create any stateful computations with Reflex", and sampling, requesting and building networks each sit behind their own class, so a function whose signature leaves a class out cannot do what that class does. A signature without an IO class cannot touch the world. That is a library subtracting from its host, and doing it well. React removes things too, by another route. The Rules of Hooks ("Don't call Hooks inside loops, conditions, nested functions, or `try/catch/finally` blocks") are a policy about state identity, and the docs can say the rule ensures "all stateful logic in a component is clearly visible from its source code". JavaScript cannot express the rule, so a linter enforces it. And the docs describe the linter's off switch in their own words: "By suppressing the linter, you 'lie' to React about the values your Effect depends on." A subtraction you can turn off is a convention.

Reflex's IO hatch is the honest kind. `liftIO` shows in the signature, the way `unsafe` shows in Rust: a labelled place where the rule stops, and you can grep for it. The harder evidence is what Reflex keeps that is not a hatch. `switchPromptlyDyn`, `tagPromptlyDyn`, `coincidence` and `sample` need no IO and no special capability. They live in the same classes `hold` needs, so a program that uses them has the same signature as one that does not. The library cannot label them, because its own primitives build them. It cannot remove them, because the host can say them. `switchPromptly` was deprecated, and the deprecation is a rename: "Use 'switchHoldPromptly' instead."

What the library can do is warn, and it warns at length, in its own documentation. On `switchHold`: "Use this rather than 'switchHoldPromptly' and 'switchHoldPromptOnly' unless you are absolutely sure you need to act on the new event in the coincidental case", because the lax version avoids "many potential cyclic dependency / metastability failures". On `switchPromptlyDyn`: "The timing requirements that switching before imposes are likely to bring down your app unless you are very careful." On `coincidence`: "You maybe looking for '@switchHold@ @never@' instead." On `tagPromptlyDyn`: "When creating cyclic data flows, generally `tag (current d) e` is preferred." And on `networkHold`, a few files away, the advice that its result "would typically be flattened (via 'switchPromptlyDyn')", the function whose own page says it will bring down your app. Nobody on that project lacks discipline. The hole is in the host, and the warnings are the policy's border, drawn by people who know exactly where it is. A library cannot make the forbidden thing stop being a sentence. `unsafe` marks the place where Rust's rule stops; Reflex's prompt variants are unmarked places inside its rule. In Prism the word `sample` does not exist, and there is no spelling of "switch to the new event in this same occurrence" at all: `!Switch` is the only switch, and it adopts at commit. The refusals section shows the program where that matters.

"Remove the dangerous feature" proves too much, of course. Every removal closes some hole. A total language removes general recursion and gets termination; Esterel removes building structure at run time and gets a compiler that refuses contradictory programs; Elm removes local state and gets one model. Something has to say which removals earn their place. Here is the test this book uses: **a refusal earns its place when it closes a documented bug class, and the systems that keep the feature warn against it in their own docs.** The first half says the hole is real. The second says the people who kept the door open agree it is a hole.

Three systems meet the second half in writing. Reflex's warnings are above. React's are about dependencies and outside state: "**Notice that you can't 'choose' the dependencies of your**

**Effect.**", the lie sentence, and for state React cannot see, `useSyncExternalStore`, which is "intended to be used by libraries, not application code", with "When possible, we recommend using built-in React state" on its own page. The TC39 Signals proposal meets it by choice. It defines its design goal this way: "Computation is glitch-free, meaning no unnecessary calculations are ever performed", a weaker property than never observing an inconsistent value, and it says plainly that "The Signal API does not include any built-in function like `effect`", leaving scheduling to the `Signal.subtle` namespace, "better to leave for framework authors". That is a committee choosing the property everyone can ship and saying so in the open. It is also a standard marking the half where the problem lives as advanced and handing it to someone else.

The test refuses two removals as well as endorsing these. Nobody whose system builds structure at run time warns you off building structure at run time, so Esterel's fixed topology fails the second half, and Prism keeps runtime construction (`rebuild` and `for`) and pays for it with heights, a cycle check and a protocol at the boundary. Nobody whose system has local state warns you off local state (React recommends `useState`; Reflex attaches no warning to `hold`), so Elm's removal fails too, and Prism keeps it: the `for` row above has its own `!remove` and could have its own `$expanded`, and its parent never hears of either.

Haskell is the exception that shows the rule working. Haskell already subtracted effects from everything, and lets a library add them back, selectively, by monad. In practice it is a language for building separate silos with different models of mutation, `IO` and `ST` and the reactive monads, each with its own rules. That is why the best FRP lives in Haskell. It is also why FRP cannot leave Haskell by the same door: in JavaScript the library adds a `Signal` type and cannot subtract `let`.

Embedding has a cost of its own. Learning a paradigm is far more work than learning one more language within it; you have watched people who know four object-oriented languages complain that Rust has no inheritance, because habits travel with the programmer, not with the language. A paradigm embedded in a familiar host is a paradigm whose alternatives are always in reach, and a learner uses them, builds a worse version of a framework they already knew, and blames the paradigm. The paradigm cost is paid either way. Embedding adds unlearning the host, every day, at every keystroke where the host is available.

None of this is about villains. The second checkbox observer was written by someone competent at six in the evening, because the callback was there. Cheating, in the sense of writing state outside the policy, is not a moral failure to catch in review. In a language with one policy, it is a sentence that cannot be written. It should be easy to build an FRP program in such a language, and impossible to build any other sort.

## What each paradigm fixed, and where the problem came back

A von Neumann machine runs one instruction stream with a beginning and an end: read input, compute, write output, halt. That is a batch job, and it is the only thing the machine natively is. An interactive program has no end and no fixed order; what happens next is decided by the user, the network and the clock, not by the program. Every paradigm below is a way of making the one-stream machine behave like an agent, and each one was invented to kill one specific bug the previous way left behind. Here is each with the problem it set out to solve, the rule it added, the place its own documentation shows the problem coming back in another form, and the Prism line that stands where the problem was.

## Callbacks

The problem: flow control. The program's order is not the program's to choose, so the one instruction stream has to be given up. The answer: hand the stream to a loop, and hand the loop a set of fragments to run when things happen. The fragments share the variables, and each may read and write anything. That is inversion of control, and it is the whole trick that turns a batch machine into something that reacts; it is also the bargain every section of this chapter is about, because flow control was solved by giving up any statement of who writes what, and when. The constraint added is none, and that is their strength: callbacks are the substrate every other paradigm compiles to, and the thing you reach for at two in the morning because nothing stands in the way. Nothing stands in the way of the second checkbox observer either. Two handlers in one moment run in an order fixed by the program's history, and the question "what can happen to this" has no answer anywhere. Prism is the end of that line: there is no instruction stream left at all, the loop is the World, and the program is the set of reactions written as equations instead of fragments. The answer to "what can happen to this" is the hold's definition, and the order is `|`.

## Objects

The problem callbacks left: any function can write any variable, so nobody knows who writes the counter. The answer: the variables are gathered into objects, and only an object's own methods write its fields. That is a real constraint and it bought a lot: the writers of a piece of state at least live in the same file as the state. The problem comes back through the method call. A method reacting to a click calls a method on another object, which calls a third, which calls back into the first before the first has finished. The pattern literature's advice, "Make sure Subject is self-consistent before Notification", exists because the setter in the checkbox afternoon is the default. Encapsulation controls *who* writes. It says nothing about *when*, and in an interactive program when is the whole difficulty.

Objects got two things right, and Prism keeps both: interactive things have identity, and they have lifetimes. The `for` row is the thing an object was trying to be, minus the methods: built when its key appears, holding its own state, torn down when the key goes, and reachable from outside only through what it returns. What will not be added is inheritance, or anything else whose purpose is letting one object reach into another's state.

## Async and await

The problem callbacks left: the fragments are unreadable as a sequence. Read a file, then parse it, then fetch what it names, then draw: four callbacks nested four deep, each with its own error path, and the thing you meant, one sequence of steps, is spread across them. The answer: give each task its instruction stream back. `await` marks the points where a task yields to the loop, the loop interleaves tasks at those points, and the code between them reads top to bottom like the batch job it wishes it were. That is a real repair, and it is why every language that has a main loop has grown an `async`.

The problem comes back at the `await` itself, because an `await` is a callback with better syntax, and the variables are still shared. Everything the task read before the `await` may be stale after it, because another task ran in between: the checkbox afternoon again, with the setter and the re-render in two tasks. Two tasks awaiting two requests finish in an order the program's text does not state. A task that was cancelled half-way leaves whatever it had written. The names are different (a race, a stale closure, a leaked task) and the thing underneath is the one from the afternoon: a value whose writers are not a fact about its type, now with the writes interleaved at points the syntax hides.

Prism keeps the thing async got right, that a task should read in order, and moves the interleaving out of the language. A request goes out as an element, its response arrives as an input in a later occurrence, and the order between two responses in flight is the World's: the World's sequence is the input, a transcript records it, and the program's text says what it does under every order, so the same inputs give the same occurrences. Nondeterminism of arrival is not removed, because nothing can remove it; nondeterminism of response to arrival is. There is nothing to read stale, because nothing is read mid-occurrence, and nothing to cancel half-written, because a torn-down slot's response is discarded at the boundary. The four-deep chain is `let !parsed = Parse !contents, !fetched <- @Element Net.Get(requests=(Url !parsed))`, and a label over a hold of the result, each line waiting for the one before it in the only way Prism has: the next occurrence.

## Haskell's IO

The problem: a pure language has no instruction stream at all, and an interactive program needs one somewhere. The answer: make the stream a value. `IO a` is a description of an action, `>>=` sequences two descriptions into one, `main` is the whole description, and the runtime system performs it. The batch job and the agent are both expressible, and the type says which parts of a program are which: a function without `IO` in its type cannot be in the stream.

The problem comes back inside the `IO` block, which is an imperative language, with `IORef`, exceptions and threads, and with the ordering of its lines as the ordering of its effects. Every bug from the afternoon can be written in a `do` block, because a `do` block is the callback bargain with a type on it: the type says that effects happen here, and nothing about which ones, to what, or in what order relative to another block running at the same time. FRP in Haskell is the attempt to put a second, stricter stream inside the first, and the Haskell section below says where that stream leaks.

Prism takes the half of the answer that worked, a program is a value and something else performs it, and declines the other half: there is no `IO` type, no block whose lines are an instruction stream, and no monad to lift one into. A widget block's statements are unordered equations; the stream is the World's, and the program never sees it as a sequence, only as inputs arriving one occurrence at a time.

## React

The problem objects left: the screen is updated by hand, one widget at a time, from handlers that each know part of the state, so the screen drifts from the state (the checkbox drawn empty while the flag was true). The answer: the screen is a pure function of state, re-run when state changes and diffed. That fixes the render side completely. Nothing can draw stale state, because drawing is not a mutation, and every framework since has copied the idea for good reason. Prism's element list is the same idea at the boundary: the World reads a list of plain data and diffs it.

The problem comes back on the state side, and React's own documentation marks each place. State lives in hooks whose identity is their position in the call sequence: "React relies on the order in which Hooks are called." Updates are callbacks calling setters, and whether two of them land in one render depends on whether they share a handler, which is the `+3` that comes out 1. Derived work runs in effects whose dependencies you list by hand, and the docs say plainly what happens when the list is wrong. Effects run after the commit, and on timing the docs use the words they have to: React "will generally let the browser paint the updated screen first", and when a click caused the render, React "may run your Effect before the browser paints".

The later additions each name a problem in the docs' own words. `useSyncExternalStore` exists for

state that changes outside React, and its consistency rule is a retry loop: "React will call `getSnapshot` a second time just before applying changes to the DOM. If it returns a different value than when it was called originally, React will restart the update from scratch, this time applying it as a blocking update". That is a transaction's rollback built by hand, inside the framework, giving up concurrency to get consistency back. Suspense handles a value that is not there yet by interrupting the render: "React does not preserve any state for renders that got suspended before they were able to mount for the first time. When the component has loaded, React will retry rendering the suspended tree from scratch." The React Compiler exists because "manual memoization is tedious, easy to get wrong, and adds extra code to maintain", and it aims at what it calls "fine-grained reactivity". Strict Mode will "run one extra development-only setup+cleanup cycle" for every effect, because the framework cannot assume your mount effect is safe to run twice.

You can read each of these as growth: a measured problem, a shipped feature, documented limits. They are. They are also growth that keeps arriving at the same three places: consistency within a moment, a value that is not there yet, and values derived from other values. In Prism those three are the occurrence, the `Unavailable` arm of a `match`, and `let $label = "Count: {$count}"`. A pattern that persistent is information about the model underneath.

## Signals

The problem React left: re-running the whole screen function on every change costs too much, and the dependency array is the manual fix that drifts. The answer: every piece of state is a cell that records who read it, and a change re-runs exactly the readers, so dependencies are tracked instead of declared. Solid, Svelte, Vue, Angular and Preact all have them, the TC39 proposal lists those frameworks and more among its design inputs, and the convergence is good news: the industry found the behavior half of FRP on its own. A signal's derived values are their definitions, which is `$d = $b + $c` from the diamond above.

The problem comes back in two places. The first: there are no events as values. Solid's docs describe signals as "like mutable variables that can point to a value now and another in the future", and what happened is whichever callback wrote the cell. Two writes in one moment need help: "it is important that the subscriber is updated after both signals have been updated", and `batch` is how you arrange it. In Prism the two writes are `!a | !b`, and the batch is the occurrence. The second: consistency is each implementation's own work. The proposal promises that "A computed Signal always observes the Signal graph in a consistent state", defines glitch-free as doing no unnecessary computation, leaves effects out, and has a long thread arguing whether evaluation should be eager or lazy. That is a sound standard fixing what everyone agrees on, and the part everyone agrees on is the half without events.

## Rx and streams

The problem on the other side: the things that happen (clicks, packets, ticks) have no type, only handlers, so you cannot combine them, filter them or hand them around. The answer: everything that happens is a stream, and a program is a pipeline over streams. Rx has events as values and a rich set of operators over them, which is the other half of FRP, found from the other side. The problem comes back because there are no behaviors as a law. The current state is a stream you remember yourself, through a `BehaviorSubject` that has "a notion of 'the current value' ", built by hand. Each subscriber to a plain observable "owns an independent execution of the Observable", so two observers of one source can disagree about what happened. And "Observables are able to deliver values either synchronously or asynchronously", so time is whatever the subscription order makes it.

In Prism two readers of `!delta` see the same firing in the same occurrence, and the current value is `$count`, a type, not a subject you built.

At scale the missing half gets rediscovered in prose. Kafka's documentation has a section on the duality of streams and tables: "A stream is thus a table in disguise", and "A table is thus a stream in disguise". A table is a hold, `then`, and its changelog is the hold's change event, `!Updated $count`. Kafka had to write the documentation because its types did not say it.

## Elm

The problem every framework above left: the rule is enforced by convention or linter, and the language underneath still has assignment. The answer: one model, one update function, and a runtime that owns every effect, in a language with no assignment at all: "The Elm program produces HTML to show on screen, and then the computer sends back messages of what is going on." Elm is the existence proof that a mutation policy at the language level, with no escape, works for real interactive programs. Everything in the previous section agrees with Elm.

The policies differ, and the difference is who owns state. Elm has one model and no local state, and its guide is direct about it: "Actively trying to make components is a recipe for disaster in Elm", because "components = local state + methods" and "local state + methods = objects", and "There are no objects in Elm!" The advice is "**Just write the helper functions you need.**" A widget with its own state threads that state through its parent's model and update by hand.

Prism keeps widgets with lifetimes and holds inside them. A row in a list has its own expanded flag, defined in the row, listing the events that move it, and torn down with the row:

```
for key, $item <- $items:
    !toggle <- @Element Ui.Button(label="Expand")
    $expanded <- False then not $expanded on !toggle
    @Element Ui.Label(text=(if $expanded: Full $item else: Short $item))
    @ unit
```

"One way to change anything, written next to the thing" holds inside the row, and the parent never hears of `$expanded`. And Prism keeps something Elm's guide never offers: every widget is testable as it stands. Wrap a widget in an interceptor and the interceptor stands where the World stands; the boundary section below shows one that answers every file read with `"hello"`, and the widget cannot tell. No mocking framework, no seam added for the test, no change to the widget.

## Haskell, as FRP's host

The problem: a host language in which a mutation policy can be stated and checked, not just recommended; the IO section above said why Haskell is the candidate. The answer: a language in which a library may define its own mutation policy through its monads, so FRP can be built as one, and the best FRP has been. The seams show anyway. You keep three views of one thing apart by naming them `dFoo`, `eFoo` and `deFoo`, Hungarian notation for time, because the monads are recursive and shadowing does not work consistently. You order a transformer stack, and the order is a design decision. You write `lift . lift`. And the door to `IO` stays open: Reflex's `Adjustable` documentation warns that "there is some issue here with persistent side-effects: obviously, IO". Prism's sigils replace the naming convention outright: `$count`, `!count` and `@count` are one name with three checked spellings. And being tied to Haskell ties FRP's success to Haskell's. The community's unofficial motto, which Simon Peyton Jones gave a talk about under the title "(Failing to) avoid success at all costs", parses two ways; the community prefers "avoid (success at all costs)", and it has succeeded at the more

salient interpretation. FRP needs a language whose success it can depend on.

### The synchronous languages

The problem: a controller that reads sensors and drives actuators must never act on a half-updated reading, and must be provably free of such moments before it ships in an aircraft. The answer: Esterel, Lustre and their family, which have Prism's law, earlier, with a clock. Time is divided into discrete ticks, "Computations considered to take no time", "Each signal is only present or absent in a cycle, never both", "All writers run before any readers do". They refuse the static form of a prompt cycle: `present A else emit A end`, "if A is absent, make A present", is "an erroneous program", and the compiler will not compile it. These are allies. They are also designed for controllers whose structure is fixed when the program is compiled, and nobody who builds structure while running warns against it. Prism keeps the law and adds `rebuild` and `for`: a GUI is built and torn down while it runs, and so is a server's set of connections.

## The goals

Every goal below is a promise that some version of that afternoon cannot happen.

- **There is one way to change anything, and it is written next to the thing.** A hold's definition lists every event that can move it. Read `$checked <- False then !toggle | !reset` and you have read everything that will ever happen to `$checked`. The second checkbox observer has nowhere to live.
- **Two things in one moment are simultaneous, and the program says what that means.** Reset and the server's message in one occurrence? The merge says which wins: `|` takes the left. Not registration order, not luck, and if you want the other answer you swap the operands and the program says so.
- **You can see what moves.** `$count + !delta` tells you that one name is live and one happens, and so does the compiler, which lifts the addition for you. No `map`, no `zipWith`, no dependency array to drift from the code.
- **"Not yet" is a type, not a feeling.** The file that has not loaded is `Unavailable`, a case of its type, and the compiler makes you say what to draw then. Nothing is `null` while you wait, and no render is thrown away and retried.
- **Nothing is chosen for the implementer's comfort.** A rule that is here because it was easier to build is a bug in the specification. The reverse test: anything a convenience does, you can do by hand with the same primitives, and the hand version is the real one. Every refusal in the next section names the bug it closes before the machinery it saves, and the order of those two clauses is the policy.
- **The language is enough.** Layout, focus, text editing, sandboxes, mocks, caches, even a World that hosts other programs: Prism code over a handful of primitives. What is native is what touches hardware.
- **The spec is the thing.** Two implementations read it, one in Rust and one in Haskell over Reflex, and a differential test runs them side by side. When they disagree, the sentence gets fixed. A second implementation proves that the specification can be read twice to the same program, which is a claim about the document. It is not offered as evidence that the language is good; the programs are.

## The refusals are the language

A language is defined by the sentences you can write in it, and also by the ones you cannot. A library can only add to the first list; a language owns the second. So the list below is not a list of missing features. It is the part that makes Prism a language in the sense the previous sections need. Each refusal closes a door you have walked through into a bug, and the same refusal lets the implementation leave out a mechanism: a scheduler, a collector, a lock, a retry loop. Each entry shows the line you cannot write and the line you write instead.

Two refusals get the long treatment, because they are the two a reader who knows FRP will fight hardest, and because they meet both halves of the test in writing.

**No sampling.** You cannot ask a dynamic what it is right now.

```
let n = $count                  # does not compile: a dynamic is not a plain value
let n = $count.value            # no such thing
let !n = $count on !click       # the count before the click, at each click
rebuild $count:                 # a block that sees the count as a plain value, rebuilt when it changes
    @Element Ui.Label(text="{_}")
```

This one feels like a missing feature until you remember the checkbox: every "read it right now" in that story was a read of something half-updated. With no sampling there is no half to read, and the implementation may propagate in any order that respects dependencies, because no program can tell the orders apart. What replaces the peek covers every use the peek had. A widget that needs to act as soon as it exists uses `@started`, an event that fires once, in the occurrence the widget is built, and reads what it needs through a tag: `!first <- @started` then `$config on !first`. Reflex has `PostBuild` for the same job, documented as existing "to allow 'Behavior's and 'Dynamic's to be safely sampled, regardless of where they were created", and it fires after the build. Prism has no safe moment to wait for, because it has no unsafe one.

**No prompt switching.** When `!Switch` is handed a new inner event, it listens to it from the next occurrence, not this one. Here is the program where that matters: a button whose label counts its own clicks.

```
@clicker:
    let $label = "Clicked {$count} times"
    !!clicks <- rebuild $label:
        @Element Ui.Button(label=_)
    let !click = !Switch !!clicks
    $count <- 0 then $count + 1 on !click
    @ $count

test clicker_relabels, @clicker:
    build
        => expect Ui.Button(label="Clicked 0 times")
    click Ui.Button(label="Clicked 0 times")
        => expect Ui.Button(label="Clicked 1 times")
        => expect_none Ui.Button(label="Clicked 0 times")
    click Ui.Button(label="Clicked 1 times")
        => expect Ui.Button(label="Clicked 2 times")
```

`rebuild $label:` builds a button with the current label and rebuilds it whenever the label changes; each build is a new slot with a new path (`0/#0/0`, then `0/#1/0`), because an element's fields are fixed for the slot's life. The `rebuild` fires `!!clicks` in every build occurrence with the new slot's click event, and `!Switch` turns that event of events into the one `!click` the hold listens to. So every click relabels the button, and every click lands in the occurrence that replaces the button it was addressed to.

Prism has one answer for that occurrence. The World addresses the click to `0/#0/0`; routing looks the path up in the element list as the occurrence opened and finds the old button, the one the user saw. The old widget keeps propagating until the occurrence commits, so its click reaches `!Switch`, which still listens to the old generation. `$count + 1 on !click` reads the count before the click and fires; the label changes; the `rebuild` builds `0/#1/0`; at commit `!Switch` adopts the new button's click and the old generation is dropped. The program is a cycle (count, label, `rebuild`, switch, click, count), and the cycle is legal because it passes through two delayed edges: the tag on `$count`, and `!Switch`'s choice of which inner event to hear.

Reflex offers a choice at the switch, and its docs describe both options exactly. `switchDyn`: "Only the old event is considered the moment a new one is switched in", which is Prism's answer. `switchPromptlyDyn`: "Switching occurs **before** the inner 'Event' fires". Take the prompt one, and whether the click gets through depends on which button the switch hears; that depends on the new button; the new button depends on the label; the label depends on the count; and the count depends on whether the click got through. A loop inside one frame, the "cyclic dependency / metastability" failure the `switchHold` page names, waiting one word away at every switch in the program. And `networkHold`, the ordinary way to rebuild part of a Reflex program, suggests in its own documentation flattening its result "via 'switchPromptlyDyn' ".

The prompt forms are there to be reached for, because sometimes the coincidental case looks like the case you need. Prism's answer to each of those cases is design, not apology. A freshly built button cannot be clicked in the occurrence it was born, because the World addresses only paths it has seen, so no click is lost. A child that needs to act at its own birth uses `@started` inside itself, where no switch stands in the way. A parent that needs to know a rebuild happened has the `rebuild`'s own event, `!!clicks` above, which fires in every build occurrence including the first. The one remaining case is a new child reacting to the very input that caused it to be built, and that is Esterel's refused program in another notation: it asks for A to be present because A was absent. Prism refuses it by making `rebuild` a prompt edge and `!Switch` a delayed one.

What the implementation gets: every node's height is fixed by its sources, checking for a cycle is one comparison, and propagation never re-enters itself. Spider, Reflex's runtime, has special machinery for resetting coincidences in the middle of a frame; a runtime without `coincidence` needs none of it. If you have ever debugged a `coincidence`, you know which side of that trade you want.

The rest go faster.

**No assignment.** `count = count + 1` does not parse, anywhere. The only state is a hold, and a hold moves only when its event fires. This is the refusal most people flinch at, and it pays for everything else: transactions for free, as the occurrence section showed. It costs something too. A field written by six things has a merge of six events in its definition, `0 then !a | !b | !c | !d | !e | !f`, and the events come from elsewhere, so "next to the thing" means a list of names, some pointing away. The list is complete and the compiler checked it. The alternative is six setter calls in six files, and the answer to "what can change this" is the result of a search.

**No events from nothing.** No `emit`, no `setTimeout`, no event that fires because time passed. `!tick <- @Element Time.Every(period=100)` is an element, and its ticks are inputs the World delivers; under the Test World they arrive only when the script sends them. An event fires because an input led to it, or not at all. The World owns time and the runtime has one entry point, so the same inputs give the same occurrences every time, and a transcript of a run is a test.

**No effects, no callbacks, no handlers.** A program cannot write a byte, read a clock, or hand anyone a function to call later. It can put an element in a list: `@Element Cli.Stdout(requests=("hello\n"`

`on !go))` is how "print" is spelled, and the write happens after the occurrence, when the World reads the list. The World never holds a runtime object, so the runtime can be single-threaded, swapped for another, or driven by a test World with no screen that tells you exactly what the program would have drawn. `useSyncExternalStore`'s retry loop is what this saves: consistency with state the framework cannot see, recovered after the fact.

**No widget values.** You cannot keep a reference to a button. `let b = @Element Ui.Button(label="OK")` names a blueprint, a description; binding it with `<-` builds a widget, and what you get back is its click event, never the widget. A widget is a position in a tree, not a thing you can put in a map. That is the detached row: if a widget is a value, someone keeps it, and the parent's decision to remove it means nothing. Here ownership is a tree, teardown is recursive, and there is no collector.

**No request on creation.** Creating an element never makes anything happen by itself. `@Element Fs.File(path="notes.txt", requests=(Read on !started))` reads the file because `!started` fired and the program said `Read` then; drop the `requests` field and the slot sits in the list doing nothing. Strict Mode's extra setup-and-cleanup cycle exists because mount effects are not safe to repeat; here building is pure, a rebuild costs exactly what it asks for, and in the occurrence a widget is built nothing can depend on an effect having happened, because none has.

**No placeholders.** No `Pending`, no `undefined`, no `null`, no render thrown away to wait. The file you have not read is `Unavailable`, and the `match` that draws it has an arm for that case:

```
let Show: fn Contents -> Text = \|c| match c:
    Unavailable: "(nothing to show yet)"
    Known text: text
```

Leave the first arm out and the program does not compile. You have written the `if (data)` guard a thousand times and forgotten it once.

**No custom monads.** One block type per functor, and the set is fixed: `@widget:` builds, `rebuild $x:` rebuilds, `for k, $v <- $m:` builds per key. You never order a transformer stack, and the compiler never infers whether something is an event: the name says.

**No inheritance, no reaching into another widget.** A widget's results are what it returns. No protected fields, no `parent.setState`, no event bus. The `$expanded` in the Elm section's row is unreachable from outside the row, and the promise that a hold lists everything that can change it is only true because there is no back door.

**No exceptions as control flow.** If the program is meant to handle a failure, the failure is a value: `a / b` is `?Nat`, and `Unavailable Denied` is a file you may not read. If it cannot be a value (a cycle, an overflow), the occurrence aborts and nothing half-done survives. The transaction is the error mechanism, and it was already there.

**No ambient anything.** No globals, no implicit context, no environment variable read from the middle of a function. `$home <- @Element Process.Env(name="HOME")` is a state element in the list like any other, so a sandbox can answer it. A widget has its arguments and its elements, and a third-party widget has no channel that the interceptor around it does not see.

**No threads, no scheduling, no priorities, no phases.** The language has no idea of "later". Parallelism belongs to the implementation and can arrive without changing a program.

**No clock.** There is no `now()`. `$now <- @Element Clock` is a state element, like the contents of a file, and a test sets it with `with Clock = Instant 5000`. A debounce takes more lines than `setTimeout`, and it is a unit test. Anyone who has tried to test a debounce knows what that is worth.

**No dynamic fields in elements.** An element is born with its fields and keeps them. `Ui.Label(text="Count: 0")` at `2/#0/0` and `Ui.Label(text="Count: 1")` at `2/#1/0` are two slots, and the World sees the first leave and the second arrive. The boundary never sees a sigil, and a World diffs lists of plain data instead of subscribing to anything.

**No introspection.** A test cannot reach into a widget and poke `$count`. It clicks the button and reads the label, as every script in this chapter does, so a test that passes under the Test World passes under any World.

The pattern repeats: a feature you would have sworn was essential is removed, the removal closes a hole you have fallen into, and the same removal lets the implementation leave out the machinery the feature needed. A language with these refusals and these two types has no freedom left in the moment and at the boundary; the next section says what it leaves unproven.

## The boundary, and what the rest of the book has to show

The language has no effects. The World has all of them. A World is written in the host language, owns the event loop, and opens every occurrence. The only thing that crosses between it and the program is plain data. Going out: *elements*, records like `Ui.Button(label="Increment")` or `Fs.Contents(path="notes.txt")`, which a widget puts in its list to say what it wants. Coming in: responses addressed to an element's path (`click` to `0`), and state identified by an element's value (the contents of `Fs.Contents(path="notes.txt")`, the same answer for every widget that asks).

One rule governs time at the boundary, and it is the rule that keeps every ordering bug of the first section out:

**Elements never fire in the occurrence they were created. Firing within the same occurrence implies they could have been pure functions. Instead, elements with request-like semantics take an input event and fire their output event in a different occurrence.**

Nothing can depend on a side effect having happened during the moment it was requested. State elements are the complement: a state element has a value in the occurrence it is created, supplied by whoever handles it, because a dynamic without a value is not a dynamic. And the boundary is a diode. Inputs cause changes to the element list; changes to the element list never cause inputs. A resize is an input from the window; a text measurement is state. Neither is a slot appearing and thereby firing something.

You have seen this before, and you should say so. Elm's commands go out and its messages come in. Reflex's `Requester` lets an action "trigger requests … based on 'Event's, and receive responses … in return". Every actor system works this way. A command queue alone would not be a reason for a language.

The queue is not the point. The point is what comes with it. No sampling and no request on creation make building pure. State elements valued in their first occurrence mean a dynamic is never without a value; `Requester`'s signature, request out on an event and response back on an event, cannot give that, which is a reading of its interface rather than anything its docs say. And `intercept` nests the boundary. A widget can stand between its children and the World, answer some of what they ask for itself, and pass the rest on. A sandbox, a mock filesystem, a cache and a GUI toolkit are all interceptors written in Prism, and a World is the outermost one. Here is a mock, the one the Elm comparison promised:

```
@WithNotes: fn @a -> @a = \|@child| @widget:
    ((result, $slots)) <- intercept @child, !down, $ supply
    !down <- @Forward $slots
    let supply = \|e| match e:
        Fs.Contents(..): @ $ Known "hello"
        ignore: @Element e
    @ result

test viewer_under_mock, (@WithNotes @ShowFile "notes.txt"):
    build
        => expect Ui.Label(text="hello")
```

`intercept` builds the child and captures its element list as `$slots`. `@Forward`, a five-line widget from the worked examples, sends every slot outward unchanged and brings the responses back down. `supply` answers the child's state: any file it asks for contains `"hello"`, and every other state element goes to the real World. `@ShowFile` is the file viewer at the end of this chapter, unchanged, and it cannot tell. A library that types its output list gets the command queue. It does not get a child that cannot tell whether its World is real.

Everything above is about the moment and the boundary, and the moment is the part every shipping system already handles inside a task. The hard parts of a GUI toolkit are layout, text and asynchronous IO, and the competition's scars are there. Here is the design for each, and which part is not yet earned.

Layout and text editing are library code. The World does no geometry. A GUI World speaks five tags: `Gui.Window`, which delivers pointer, key and resize events; `Gui.Scene`, the draw list for a window as plain `Draw.*` values; `Gui.Focusable`, a named region that receives keys; `Gui.Metrics`, text measurement as a state element; and `Gui.Frames`, a tick per frame for whoever asks. Buttons, labels and text inputs are the library's `Ui.*` tags, and the library's interceptors turn them into scenes and hit-test pointer events against the layout they hold. Measurement is state, so it has its value in the occurrence it is asked for: measuring and laying out happen in one occurrence, and the scene that leaves it is consistent. No column flickers while a round trip completes. A text input's text and cursor are a hold in the input widget, outside any `rebuild`; keys arrive through a `Gui.Focusable` the library names; the World sees draw commands and never a cursor, so a re-render cannot lose it. The elements-and-interception chapter builds this library, and the use-cases chapter builds an editor and a compositor on it.

Asynchronous IO is the World's, and the async section of the paradigm tour said what the program sees of it: an order of arrival that is an input, recorded in the transcript. Cancellation is the World's obligation when a slot is torn down; a stream is an element that responds without being asked; a socket is one tag with requests, responses and state. The Worlds chapter has to make each of these work.

The unproven part, then: whether a constraint solver is pleasant to write as holds and events; whether an input method's composition survives a text input written in the library; whether a World on an asynchronous runtime keeps the transcript exact under load. The rest of the book is for these. This chapter claims the moment and the boundary, and claims that the toolkit is library code over five tags. It does not claim the toolkit is done.

The claim has edges in the other direction too. Prism is the right language for a program that is state, things happening, and responses: interactive interfaces, stateful network services, incremental computation like build systems and spreadsheets, simulations and games, controllers, shells and window managers. It is the wrong language for programs that are computations: numerics, compilers,

batch data work, an algorithm naturally written as a loop with a cursor, a request handler that keeps nothing between requests. Prism programs call out to that kind of code; they do not replace it. The first list is large, the tools for it are bad in a way that is structural, and a language with time in its types fixes the structure. The second list is where the claim stops.

## A pure language

A Haskell program is a value. `main :: IO ()` describes an action, and the runtime system performs it; nothing in the program performs anything. Elm's guide says the same of its own loop, "The Elm program produces HTML to show on screen, and then the computer sends back messages of what is going on", and neither sentence is an admission that the language is impure. A pure language that talks to a world draws one line, between a description and its performance, and lives on the description side. That line is not the test, though; a C binary is also a description the operating system runs. The tests are two, and they bite inside the language. Substitution: replace a name by its definition and you have the same program. Strategy: Sabry's criterion for a purely functional language, paraphrased, is that it extends the simply typed lambda calculus conservatively and that call-by-name, call-by-need and call-by-value give its programs the same answers, divergence aside. Here are the two tests run on the three things in Prism that look as if they would fail them.

The hold first. `$count <- 0 then $count + !delta` looks like a variable with an update rule, and the runtime keeps a cell for it and overwrites the cell at the end of each occurrence. Substitute the definition for the name and see whether the program changes:

```
$count <- 0 then $count + !delta
@Element Ui.Label(text="Count: {$count}")
# the same program:
@Element Ui.Label(text="Count: {0 then $count + !delta}")
```

Both labels print the same text at every occurrence of every script, and the reason is that no term in the language can read the cell's history except through the one-step delay that `then` is. The old value is unreadable after the commit and the new one is unreadable before it, so the cell is the latest value of a fold over the inputs, and a program that could tell a fold from a cell would need a sentence Prism does not have. The recursion, `$count` on both sides, is a stream equation with a delay in it, this occurrence's value defined from the previous one, which the synchronous languages write with an initialised delay operator and nobody there mistakes for a loop. The law that makes the delay real is the one Reflex states for `hold`: a sample taken in the occurrence that changes a value "will see the **old** value of the 'Behavior', not the new one". It is the `tag_reads_before` script above. A hold has no position and no identity, so two copies of one hold definition are one stream, equal at every occurrence.

The widget second, and here the test fails, on purpose, for the one thing the language refuses to hold. Bind one blueprint twice and look at the list:

```
@main:
    let @b = @Element Ui.Button(label="OK")
    !x <- @b
    !y <- @b
    @ unit
```

```
  0: Ui.Button(label="OK")
  1: Ui.Button(label="OK")
```

Two widgets, two paths, two click events. `@b` was one value and still is; substituting its definition for

its name in both lines gives the same list. What is not substitutable is `!x` for `@b`: `!x` is the clicks of the widget at `0`, the way `r` in `r <- newIORef 0` is a reference and not the action. A widget has a lifetime, it is wired to its neighbours, it is torn down, and two built from one blueprint are two; nothing with those properties is referentially transparent, so the language does not hold one. It holds the blueprint, and it hands back the results. The only thing binding a blueprint allocates is a position in a tree, and a position is data in the element list, not a reference to anything. The refusal of widget values is where the purity is kept, by putting the one thing with an identity on the far side of the type. And the block that describes a widget is not a sequence: `$count` can be used three lines above the line that defines it, and the order of the statements fixes the positions of the elements in the list and nothing else. A widget block is a set of simultaneous equations about one span of time.

The request third. `@Element Process.Exit(requests=(Code 0 on !quit))` looks like the program ending the process, and the objection writes itself: the program decides when the process ends, and calling the decision data changes when it takes effect, not the fact that it does. Deciding is pure; what makes an exit an effect is performing it, and the program cannot. It can put the record in its list, and it cannot depend on anything having followed from that, because a request whose result arrived in the occurrence it was made would be a function call, and this one's result never arrives in that occurrence. Under the Test World the record is a line in a transcript, `requested: Code 0`, and ends nothing; under the CLI World the process exits after the occurrence commits; the program is the same program. Reflex states the same of its own requester: "Semantically, the response event occurs at a later time than the request event." There is no IO type to lift and no class to leave out; Haskell's `unsafePerformIO` and `liftIO` into any `MonadIO` have no counterpart because there is nothing for them to reach.

Strategy is the second test, and Prism's version of it is a law the chapter has already stated: the implementation may propagate in any order that respects dependencies, because no program can tell the orders apart. A dynamic cannot be read outside an event; it is read at one, through a tag, and the hold law fixes what the tag sees whatever ran first. Two implementations of this book exist, one on Reflex in Haskell's monads and one as a hand-built graph in Rust with heights and a cycle check, and they pass the same conformance suite, whose laws are written as equations between programs, so an order that leaked would show up as a failed equation and not only as a difference between two runs. Two things look like order and are not. `|` takes the left, as Reflex's `leftmost` does: that is a function of two operands at an occurrence, the left value if there is one and the right otherwise, and the third turn of `counter_counts` tests it. The commit at the end of the occurrence is the meaning of `then`, that the new value is the value for the next occurrence, and a meaning is not a schedule.

Pure is not total. Indexing and division are total, each returning `?V` where there is no answer, but an overflow or a recursion without end is bottom, and a cycle closed by a switch is bottom too; a cycle inside one block is a compile error. The occurrence that reaches bottom is dropped whole, in every implementation, because the value layer is specified eager and the strategy the law leaves free is the propagation order, not the strictness of a plain expression. Sabry's criterion sets divergence aside, as every account of purity that admits bottom does. A reader who knows databases will say that atomic commit and abort are an effect discipline whether or not anything is undone. The discipline is the World's, not the program's: there is no sentence in the program about committing or aborting, and an aborted occurrence is bottom in the fold, exactly as an exception from pure code is in Haskell, where the runtime catches it in IO. Nothing was performed, so nothing is undone; the commits that would have happened are a list, and dropping the occurrence is forgetting the list. From inside an aborted occurrence nothing happened, so the program cannot see which of two faults the runtime met first, and order cannot leak in through the error either.

Two things purity gives for nothing, and the chapter has used both. A transcript is a test, because

no occurrence depends on anything but its inputs and the holds before it; every script above is that. A widget under an interceptor cannot tell that its World is a mock, because everything it ever had of the World was data in its slots; `viewer_under_mock` is that. An impure program can be made replayable by logging every effect and mockable by injecting every interface; here there is nothing to log and no interface to inject.

Elliott and Hudak defined a behavior as a value that varies with time. Prism keeps that definition, on clocks the inputs choose, and keeps nothing that performs. Its programs pass both tests: put a definition in place of its name and the program is the same; run propagation in any order and the answers are the same. The World is the only thing that runs, and time, the subject this chapter began with, is a value.

## The file viewer

The counter opened the chapter. Here is the second program, with "not yet" as a type and exit as a request.

```
type Contents = choice:
    Unavailable
    Known Text

type Code = Code Int

tag Fs.Contents of Element:
    path: Text
    state: Contents

tag Process.Exit of Element:
    requests: !Code

let Show: fn Contents -> Text = \|c| match c:
    Unavailable: "(nothing to show yet)"
    Known text: text

@ShowFile: fn Text -> @unit = \|path| @widget:
    $contents <- @Element Fs.Contents(path=path)
    @Element Ui.Label(text=(Show $contents))
    @ unit

@main:
    @ShowFile "notes.txt"
    !quit <- @Element Ui.Button(label="Quit")
    @Element Process.Exit(requests=(Code 0 on !quit))
```

`Fs.Contents` is a state element: the file's contents as a value over time. `$contents` has a value in the occurrence the slot is created, the World's answer as that occurrence opened: the text, if the World could read it then, or `Unavailable` if it could not or has not yet. When the file changes on disk, the World pushes the new contents in a fresh occurrence, `$contents` changes, and the label follows. The program has no code for that, because a dynamic that follows the file is what it asked for.

```
test viewer_follows_the_file, @main:
    with Fs.Contents(path="notes.txt") = Unavailable, build
        => expect Ui.Label(text="(nothing to show yet)")
    push Fs.Contents(path="notes.txt") = Known "hello"
        => expect Ui.Label(text="hello")
    click Ui.Button(label="Quit")
```

```
      => requested Process.Exit(..), Code 0
```

`Show` is a plain function, and the `match` in it has an arm for `Unavailable` because the type has that case and the compiler will not accept a `match` that leaves it out. There is no string standing in for a value, no `Pending`, no render thrown away and retried. The Quit button's clicks become `Code 0` requests on a `Process.Exit` element. The element sitting in the list does nothing; closing a window does nothing; dropping a widget does nothing. The run ends after the turn in which the program fires the request, because the program said so, in the occurrence after the click arrived at a button that existed when the occurrence opened.

Ask this program the six o'clock questions too. What can change the label? The file, through one state element, and nothing else. What does the user see before the file is read? Whatever the `Unavailable` arm says, chosen by you, checked by the compiler. What ends the program? One request, fired by one event, in one line.

Two short programs, and the questions that took an afternoon in the first section each have a one-line answer. Prism is a language whose only subject is time.

## Chapter 2

# Worked examples

Fourteen programs, from a counter to a shell that hosts other programs, each one followed by what happens occurrence by occurrence: what the World sends in, what fires and in what order, what commits, and what the element list looks like afterwards. The programs are the language; the walkthroughs are the laws applied to them. If you have read the specification once and want to know whether you understood it, read the programs, predict the walkthroughs, and check. If you are coming to the language cold, the walkthroughs are where every rule in the later chapters is first seen doing work, and each example ends by saying which afternoon it prevents: the checkbox that un-checks itself, the fetch on mount that fires twice, the native text box that loses its cursor, the click that lands on a button that has already been replaced.

## Conventions used in every walkthrough

Occurrences are numbered from zero. Occurrence 0 is the build of the program; nothing is delivered to it. From then on every occurrence is opened by the World with a set of inputs, each one addressed to a slot by its path, all simultaneous. A walkthrough says, for each occurrence: the inputs; the events that fire and the derived dynamics that change, in dependency order, which is the only order the language promises; the holds that commit at the end; and the element list the World reads afterwards.

Paths are opaque to a World and the chapter needs a way to write them down, so here is one. A path is the chain of bind positions from the root, written with `/`: the first widget bound in `@main` is `0`, the second is `1`; a `rebuild` adds its generation as `#n`, starting at `#0` for the first build; a `for` adds its key as `[k]`. The Increment button of the counter below ends up at `0/#0/0`: the first bind in `@counter`, generation zero of the `rebuild` inside `@Button`, the first slot in that generation. Holds and `let`s do not take a bind position; only widgets do, and `@started` is a widget, so it takes one even though it creates no slot. A widget block bound at position `p` numbers its own binds under `p`, which is why `@TextInput`'s two parts below are `3/0` and `3/1/...`; a `rebuild`, a `for` or an `@Element` bound at `p` is one segment, `p/#0/0` or `p/[k]/...` or `p`, not two, because the position is the thing itself and not a block around it. So a widget function's insides can never renumber its caller's slots, and an interceptor's routing by path prefix is stable across any change inside a child. A slot created by a widget that an interceptor's supplier built is written `s/n`, with `n` counting the supplier-built widgets in creation order in the interceptor's widget, followed by the supplied widget's own segments; its position in the widget is the runtime's business, and only its existence is a law.

Element lists are written one slot per line as `path: element`. A tag with no `responds` and no `state` is a presence element, there to be seen and nothing else. A slot whose request fired in the occurrence has `requested:` after it, then the fired values in firing order, each written as the Prism literal it is and separated by `;`, with no brackets of the transcript's own: `requested: Read`, `requested: [[Send "ok", Close]]`, `requested: "a"; "b"`. An array request is already an array literal, so wrapping the list in brackets would stack three of them; the separator keeps every bracket in the line meaning an

array.

The chapter assumes a small prelude of plain functions. None of them does anything with time; they are listed so that nothing in a program is a mystery:

```
!Filter: fn !?a -> !a                                  # fires x when the input fires Some x
!Union:  fn !(k ==> a), !(k ==> a) -> !(k ==> a)       # the union of the maps that fired; the left wins a
shared key
!Leftmost: fn [!a] -> !a
First:  fn ((a, b)) -> a
Second: fn ((a, b)) -> b
Len:    fn Text -> Nat
Prefix: fn Text, Nat -> Text                           # the first n characters
Suffix: fn Text, Nat -> Text                           # everything from character n on
Concat: fn [Text] -> Text
JoinText: fn Text, [Text] -> Text                      # separator first
Count:  fn Text, Char -> Nat
StartsWith: fn Text, Text -> Bool
ToText: fn a -> Text                                   # a: a primitive with a canonical rendering
Append: fn [a], a -> [a]
Insert: fn k ==> v, k, v -> k ==> v
Remove: fn k ==> v, k -> k ==> v
Size:   fn (k ==> v) -> Nat
Keys:   fn (k ==> v) -> [k]
Filter: fn k ==> v, (fn v -> Bool) -> k ==> v
MapValues: fn k ==> v, (fn v -> w) -> k ==> w
Fold:   fn k ==> v, a, (fn a, k, v -> a) -> a          # in key order
```

Every operation over these lifts by the rule of the sigils: a plain function applied to a dynamic is a dynamic, applied to a dynamic and an event is an event with the dynamic read through a tag, applied to two events is a compile error. Comparisons are the infix operators `==`, `!=`, `<`, `<=`, `>` and `>=`, through the equality and ordering traits, and lift the same way. Boolean logic is the keyword operators `not`, `and` and `or`, looser than comparison and tighter than `on` and `|`, and they lift the same way too. So `Append $log, !line` is an event of arrays, and `(Insert $items, $next, todo) on !add` is an event that fires with the click and reads both dynamics as they stood before it.

## Shared declarations

The GUI vocabulary in this chapter is deliberately small and deliberately not the language's. `Element` is the extensible variant every element is a value of; the tags under `Ui` are the GUI library's own, which the Test World accepts directly and a GUI World sees only through the library's interceptors, and later examples add tags as they need them, each declared before its first use.

```
type Element = choice ..

type Click = click

tag Ui.Label of Element:
    text: Text

tag Ui.Button of Element:
    label: Text
    responds: Click
```

A label has a text and does nothing; a button has a label and responds with a click. Both have plain fields, fixed for the life of the slot, which is the rule for every element: a label whose text changes is, to the World, a new label at the same spot. The two library widgets below are how a program with

a dynamic label gets one anyway. The dynamic lifts the element into a `rebuild`, which builds a fresh slot per change, and the click a button returns is a `!Switch` over the generations:

```
@Label: fn $Text -> @unit = \@Element Ui.Label(text=$_)

@Button: fn $Text -> @!Click = \@Element Ui.Button(label=$_)
```

A dynamic where a plain field goes lifts the element into a `rebuild`: a fresh slot per change, and for a button a `!Switch` over the generations' clicks. Written out, that is what each of these means:

```
@Label: fn $Text -> @unit = \@widget:
    rebuild $_:
        @Element Ui.Label(text=_)
    @ unit

@Button: fn $Text -> @!Click = \@widget:
    !!clicks <- rebuild $_:
        @Element Ui.Button(label=_)
    @ !Switch !!clicks
```

Inside the `rebuild` block `_` is the label as a plain value, because inside one build it never changes; a change rebuilds the block. The expansions are here so that nothing in the walkthroughs is hidden, and the library uses the one-line forms, which matters for paths: the shorthand is one bind and one position, so a `@Button` at position `0` prints `0/#0/0`, while the written-out block, bound yourself, would be a widget block at `0` with a `rebuild` as its first bind, `0/0/#0/0`. Example 4 is the same construction unrolled with a cycle through it.

# 1. The counter

```
@counter:
    !increment <- @Button $"Increment"
    !decrement <- @Button $"Decrement"
    let !delta = -1 on !decrement | +1 on !increment
    $count <- 0 then $count + !delta
    @Label "Count: {$count}"
```

You have written this program in every framework you have used and in each one the interesting question was never the increment; it was what happens when two things land at once. Here that question has an answer you can read off the third line, and it is the only thing the program has to say about it.

Nothing in the five lines says which numeric type `$count` is. `-1` is a signed literal, so `!delta` is an event of `Int`, so `$count + !delta` is `Int` addition, so the `0` that starts the hold is an `Int`. Had the deltas been `+1` and `+2`, nothing would pin the type, and an integer literal nothing constrains is an `Int`, so the program would mean the same thing; a `Nat` counter says so with an annotation.

**Occurrence 0 (build).** The World calls the run call with no inputs. `@counter` is built. The first `@Button` builds its `rebuild`, whose block builds generation `#0` with one slot, `Ui.Button(label="Increment")` at `0/#0/0`; the `rebuild`'s event `!!clicks` fires in this occurrence with the slot's click event, and `!Switch` records it as the inner event to listen to from occurrence 1. The second button does the same at `1/#0/0`. `$count` has its initial value, `0`. `"Count: {$count}"` is a derived dynamic whose value is `"Count: 0"`, and `@Label` builds generation `#0` of its `rebuild` with `Ui.Label(text="Count: 0")` at `2/#0/0`. Nothing fires at `!increment`, `!decrement` or `!delta`: a `!Switch` is subscribed to nothing in

the occurrence it is given its first inner event. The occurrence commits: `$count` is `0`, and the two `!Switch` holds now point at their slots' events.

```
0/#0/0: Ui.Button(label="Increment")
1/#0/0: Ui.Button(label="Decrement")
2/#0/0: Ui.Label(text="Count: 0")
```

**Occurrence 1.** The World sends `click` to `0/#0/0`. Routing looks the path up in the list as the occurrence opened and finds the Increment slot; its click event fires. `!Switch` is listening to exactly that event, so `!increment` fires. `!delta` is a merge; its left side is quiet and its right side fires with `+1`, so `!delta` fires with `+1`. `$count + !delta` is an operation over a dynamic and an event: it fires with `0 + 1`, where `0` is `$count` read through a tag, the value before this occurrence. That is the hold's change event, so the derived `"Count: {$count}"` changes promptly to `"Count: 1"`, which is the dynamic the label's `rebuild` watches, so the `rebuild` builds anew: generation `#0` of the label is torn down at commit and generation `#1` is built now, with a new slot. Commit: `$count` is `1`.

```
0/#0/0: Ui.Button(label="Increment")
1/#0/0: Ui.Button(label="Decrement")
2/#1/0: Ui.Label(text="Count: 1")
```

The World diffs the two lists, sees a label gone and a label arrived at the same tree position, and redraws. Nothing told it to. The list is a description of what the program wants on the screen; reading it is the World's whole job.

**Occurrence 2.** The World sends `click` to `0/#0/0` and `click` to `1/#0/0` in one occurrence, because the user mashed both and the World chose to deliver them together. Both slots' click events fire; both `!Switch`es deliver; `!increment` and `!decrement` both fire. Now `!delta` has both sides firing, and `|` takes the left: `!delta` fires with `-1`. `$count + !delta` fires with `1 + -1 = 0`. The label rebuilds to generation `#2`. Commit: `$count` is `0`.

```
0/#0/0: Ui.Button(label="Increment")
1/#0/0: Ui.Button(label="Decrement")
2/#2/0: Ui.Label(text="Count: 0")
```

Between occurrences 1 and 2 the World did what Worlds do: it read the list, drew it, and waited. Nothing in the program ran. There is no render loop inside a Prism program and no idle callback; an occurrence happens because the World opened one, and between them the program is a data structure. That is why the same program can be driven by a test World with no screen (example 13), and why the World can batch two clicks into one occurrence or deliver them as two, and the program's text says what each choice means.

Suppose the program wanted a count per button as well as the total. The hold is the only state, so there would be two more holds, `$ups <- 0 then $ups + 1 on !increment` and `$downs <- 0 then $downs + 1 on !decrement`, each listing its one event, and in occurrence 2 both would move while `$count` moved by the leftmost. Nothing about the first hold changes, and nothing anywhere else has to know the new ones exist.

The tag law decides that `$count` on the right of `then` means the count *before* the click, which is what turns a line that looks like an equation into a definition: there is no "which version of `$count` did the handler see" because there is only one version at an event. The merge decides simultaneity: two clicks in one moment are one `!delta`, the leftmost, and if you want the other answer you write `+1 on !increment | -1 on !decrement` and the program says so. And the hold's definition is the complete list of what can move the count. There is no setter, no second file, no reset handler registered somewhere else that wins or loses depending on registration order. That is the goal of one way to change anything, written next to the thing, in five lines.

## 2. A text input with a cursor

```
type Key = choice:
    Char Char
    Left
    Right
    Backspace

tag Ui.Focusable of Element:
    name: Text
    responds: Key

type Edit = record:
    text: Text
    cursor: Nat

let Apply: fn Edit, Key -> Edit = \|e, k| match k:
    Char c:
        Edit (Concat [[(Prefix e.text, e.cursor), (ToText c), (Suffix e.text, e.cursor)]]), e.cursor + 1
    Left:
        if (e.cursor == 0): e
        else: Edit e.text, e.cursor - 1
    Right:
        if (e.cursor == Len e.text): e
        else: Edit e.text, e.cursor + 1
    Backspace:
        if (e.cursor == 0): e
        else: Edit (Concat [[(Prefix e.text, e.cursor - 1), (Suffix e.text, e.cursor)]]), e.cursor - 1

let Render: fn Edit -> Text = \|e|
    Concat [[(Prefix e.text, e.cursor), "|", (Suffix e.text, e.cursor)]]

@TextInput: fn Text, Text -> @$Text = \|name, initial| @widget:
    !key <- @Element Ui.Focusable(name=name)
    $edit <- Edit initial, (Len initial) then Apply $edit, !key
    @Label (Render $edit)
    @ $edit.text
```

Everyone who has wrapped a native text box has lost the cursor. The box is a widget the toolkit owns; its value is a property you set; you set it from your model on every change; and the toolkit, seeing a new value, puts the cursor at the end. The fix is some version of "only set it if it differs", and that fix has a race in it. Here the text box is not native and the cursor is not the toolkit's. It is a hold, and it lives outside any `rebuild`.

The element that receives keys is `Ui.Focusable(name=name)`: a named region that can have focus and responds with keys. It has no dynamic fields, so its slot is created once and lives as long as the widget, and whoever delivers keystrokes has one stable path to send them to. Everything that changes per keystroke, the text and the cursor, is in `$edit`, and `Apply` is a plain function from the old state and a key to the new state. The label is a presence element: a fresh `Ui.Label` slot per keystroke, which is right, because nothing is in flight on a label and whatever draws the list diffs it.

**Occurrence 0 (build).** `@TextInput "input", "ab"` is built. The focusable's slot is at `0`; `$edit` is `Edit "ab", 2`; `Render $edit` is `"ab|"`; the label's `rebuild` builds generation `#0` at `1/#0/0`. The result dynamic `$edit.text` is a derived dynamic with value `"ab"`.

```
0:      Ui.Focusable(name="input")
1/#0/0: Ui.Label(text="ab|")
```

**Occurrence 1.** The user presses Left. The World sends `Left` to `0`. `!key` fires with `Left`; `Apply $edit, !key` fires with `Apply (Edit "ab", 2), Left`, with `$edit` read through a tag, so the cursor moves from 2 to 1. The hold's change event is that firing; `Render $edit` changes promptly to `"a|b"`; the label rebuilds to `#1`. Commit: `$edit` is `Edit "ab", 1`. The result dynamic `$edit.text` did not change in this occurrence, because `"ab"` is `"ab"`; a derived dynamic fires its change event when an operand changes, and the operand here is the record, so the change event fires with `"ab"` again. Whoever holds the result sees a change event with an equal value, which is fine: a dynamic's change event is allowed to fire with the value it already had, and nothing downstream can tell the difference except a tag, which gives the same answer either way.

```
0:      Ui.Focusable(name="input")
1/#1/0: Ui.Label(text="a|b")
```

**Occurrence 2.** The user types `x`. The World sends `Char 'x'` to `0`. `Apply` fires with `Edit "axb", 2`. The label rebuilds to `#2` with `"ax|b"`. Commit: `$edit` is `Edit "axb", 2`. The result dynamic changes to `"axb"`.

```
0:      Ui.Focusable(name="input")
1/#2/0: Ui.Label(text="ax|b")
```

**Occurrence 3.** The World sends `Backspace` to `0`. `Apply` deletes the character before the cursor: `Edit "ab", 1`. The label is `"a|b"` again at generation `#3`.

Nothing in these four occurrences involved a World knowing what a cursor is. Under a GUI World the library's window interceptor turns the label into a `Draw.Text` in the window's `Gui.Scene` and delivers the window's key events to the focusable's slot; the World sees draw commands and keys, never a cursor. Under the Test World a script sends `Left` to the focusable directly. If a World wraps a native toolkit and wants to use the toolkit's own text box, it can offer a `Gtk.Entry` tag and a different `@TextInput` targets it; but the one above runs under any World that can draw and deliver a key, and its cursor cannot be lost because there is no second copy of it anywhere. This is the goal that the language is enough: text editing is not a native capability, it is a hold and a function.

Focus is not in this program either. A form with three inputs has three `Ui.Focusable` slots, and something has to decide which one a keystroke is for. That is the library's window interceptor, not the World. It holds the focused name in a hold moved by Tab and by clicks (hit-tested against the layout the library computed), announces each focusable to the World as a `Gui.Focusable(name=...)`, so that a World with a notion of focus of its own, such as an input-method layer, knows the names, and delivers each key from `Gui.Window` to the slot whose name is focused. Under that interceptor the form above is unchanged. Focus is by name, which is why `@TextInput` takes one: two inputs in one form with the same name would be one focus target with two slots.

One thing the program does *not* do is react to its own label. The label is a fresh slot per keystroke, and if slots appearing were events, the program would have a loop in it. They are not; the diode holds: inputs cause list changes and list changes never cause inputs.

# 3. A todo list

```
type Todo = record:
    label: Text
    done: Bool

type Change = choice:
    Add Text
```

```
    Toggle
    Delete

let Step: fn Nat ==> Todo, Nat, Change -> Nat ==> Todo = \|items, id, change| match change:
    Add label: Insert items, id, (Todo label, False)
    Toggle: match items[id]:
        Some t: Insert items, id, (Todo t.label, not t.done)
        Nil: items
    Delete: Remove items, id

let StepAll: fn Nat ==> Todo, Nat ==> Change -> Nat ==> Todo = \|items, changes|
    Fold changes, items, \|acc, id, c| Step acc, id, c

let Describe: fn Todo, Bool -> Text = \|t, starred|
    Concat [[(if t.done: "[x] " else: "[ ] "), t.label, (if starred: " *" else: "")]]

@todos:
    !add <- @Button $"Add"
    $next <- 0 then $next + 1 on !add
    let !adds = {$next: Add "item {$next}"} on !add
    !rows <- for $items:
        !toggle <- @Button $"Toggle"
        !delete <- @Button $"Delete"
        !star <- @Button $"Star"
        $starred <- False then not $starred on !star
        @Label (Describe $_, $starred)
        @ Toggle on !toggle | Delete on !delete
    let !changes = !Union !adds, !rows
    $items <- {} then StepAll $items, !changes
```

The list is a map from an allocated id to a todo, not an array. The id is the identity: it decides which row survives a change, which paths stay stable, and which `$starred` keeps its value when the row above it is deleted. An array keyed by position would rebuild every row after an insertion at the front and reset every star; that is why `for` over an array is second class and why the ids are allocated from `$next` rather than taken from a position.

`$starred` is local to the row on purpose. It is not part of the model, and that is the demonstration: state inside a `for` block belongs to the key's widget and lives exactly as long as the key does, through every change to its neighbours.

Each row's block returns an event of `Change`, and `for` consolidates those into `!rows : !(Nat ==> Change)`, a map of the changes that fired in an occurrence, keyed by the row's id. The Add button is outside the `for`, so its change is made into a one-entry map by hand, `{$next: Add "item 3"} on !add`, reading `$next` through a tag. `!Union` merges the two maps when both fire. Had the program written `!adds | !rows` instead, an add and a toggle in one occurrence would lose the toggle, and that would be a bug the program chose; the union is the program saying that both count.

**Occurrence 0 (build).** `$items` is `{}`; `for $items` builds no rows; `$next` is `0`. The Add button is at `0/#0/0`.

```
  0/#0/0: Ui.Button(label="Add")
```

**Occurrence 1.** `click` to `0/#0/0`. `!add` fires. Two things read `$next` through a tag, and both read `0`: `$next + 1 on !add` fires with `1`, and `!adds` fires with `{0: Add "item 0"}`. `!Union` fires with the same map, since `!rows` is quiet. `StepAll $items, !changes` fires with `{0: Todo "item 0", False}`, reading `$items` as `{}`. That is `$items`'s change event, so `for $items` sees a key appear in this occurrence and builds row `[0]` now: three buttons at `1/[0]/0/#0/0`, `1/[0]/1/#0/0` and `1/[0]/2/#0/0`, a hold `$starred`

at `False`, and a label at `1/[0]/3/#0/0` with `"[ ] item 0"`. Commit: `$next` is `1`, `$items` has one entry.

```
0/#0/0:        Ui.Button(label="Add")
1/[0]/0/#0/0:  Ui.Button(label="Toggle")
1/[0]/1/#0/0:  Ui.Button(label="Delete")
1/[0]/2/#0/0:  Ui.Button(label="Star")
1/[0]/3/#0/0:  Ui.Label(text="[ ] item 0")
```

**Occurrence 2.** `click` to `0/#0/0` again. The same steps with `$next` read as `1`; row `[1]` is built. Keys are ordered, so the list has row `[0]` before row `[1]`.

**Occurrence 3.** `click` to `1/[0]/2/#0/0`, the Star button of row 0. Inside that row's block, `!star` fires, `not $starred on !star` fires with `True`, and `Describe $_, $starred` changes to `"[ ] item 0 *"`, rebuilding the row's label to `#1`. The row's result event is quiet, so `!rows` is quiet, `!changes` is quiet, and `$items` does not change. Commit: row 0's `$starred` is `True`.

**Occurrence 4.** `click` to `1/[0]/0/#0/0` and `click` to `0/#0/0` in one occurrence: toggle row 0 and add at once. Row 0's block fires `Toggle`; `!rows` fires with `{0: Toggle}`. `!adds` fires with `{2: Add "item 2"}`. `!Union` fires with `{0: Toggle, 2: Add "item 2"}`. `StepAll` folds in key order: row 0 becomes done, row 2 is inserted. `for` sees key 2 appear and builds its block; row 0's `$_` changes, which is the row's entry *as a dynamic*, so its label rebuilds to `"[x] item 0 *"` without the row being rebuilt, and `$starred` stays `True`. Commit.

```
0/#0/0:        Ui.Button(label="Add")
1/[0]/0/#0/0:  Ui.Button(label="Toggle")
1/[0]/1/#0/0:  Ui.Button(label="Delete")
1/[0]/2/#0/0:  Ui.Button(label="Star")
1/[0]/3/#2/0:  Ui.Label(text="[x] item 0 *")
1/[1]/...      (row 1, unchanged)
1/[2]/0/#0/0:  Ui.Button(label="Toggle")
1/[2]/1/#0/0:  Ui.Button(label="Delete")
1/[2]/2/#0/0:  Ui.Button(label="Star")
1/[2]/3/#0/0:  Ui.Label(text="[ ] item 2")
```

**Occurrence 5.** `click` to `1/[1]/1/#0/0`, Delete on row 1. `!rows` fires with `{1: Delete}`; `$items` loses key 1; `for` tears row `[1]` down at commit. Row 0's star is still set and row 2's paths are what they were. Nothing was renumbered.

The program is a cycle: `$items` is defined from `!changes`, `!changes` comes from `!rows`, and `!rows` is the result of `for $items`. It is legal because it passes through delayed edges. `StepAll $items, !changes` reads `$items` through a tag, and the consolidated event of a `for` merges the blocks that existed as the occurrence opened, the way `!Switch` picks its inner event: a block born in an occurrence has its own events heard from the next one. Nothing is lost by that, because a row's events are clicks, and a click cannot arrive at a slot in the occurrence the slot was created.

What this prevents: the afternoon where deleting item 3 cleared the expanded state of item 4, because the list was an array and the framework matched rows by position. Keys are identities, `for` keeps a widget per key, and the state that belongs to a key is torn down with it and only with it.

# 4. A button that relabels itself

```
@clicker:
    !!clicks <- rebuild $label:
        @Element Ui.Button(label=_)
    let !click = !Switch !!clicks
    !reset <- @Button $"Reset"
```

```
$count <- 0 then $count + 1 on !click | 0 on !reset
let $label = "Clicked {$count} times"
@Label $label
```

This is `@Button` unrolled, with the label depending on the click; it illustrates the fresh-slot rule and is not how a program normally labels a button, since the library's buttons take a dynamic label (`@Button $label`) and the fresh slot per change stays inside the library's interceptor. The `rebuild` watches `$label`; every change to the label tears the button's slot down and builds a new one; `!!clicks` fires in every build with the new slot's click event; and `!Switch` turns that into the one `!click` the rest of the program uses. The cycle is the thing to look at: `$count` depends on `!click`, `!click` on `!!clicks`, `!!clicks` on the `rebuild`, the `rebuild` on `$label`, and `$label` on `$count`. The `rebuild` is not a delay: its event fires in the occurrence of the rebuild. `!Switch` makes the cycle legal: it chooses its inner event from before the occurrence, and the tag on `$count` in `$count + 1` does the rest.

**Occurrence 0 (build).** `$count` is `0`; `$label` is `"Clicked 0 times"`; the `rebuild` builds generation `#0` with a button at `0/#0/0`; `!!clicks` fires with its click event; `!Switch` has it from occurrence 1. Reset is at `1/#0/0`; the label at `2/#0/0`.

```
0/#0/0: Ui.Button(label="Clicked 0 times")
1/#0/0: Ui.Button(label="Reset")
2/#0/0: Ui.Label(text="Clicked 0 times")
```

**Occurrence 1.** `click` to `0/#0/0`. The generation-`#0` slot's event fires; `!Switch` delivers it as `!click`; `$count + 1 on !click` fires with `1`; `$label` changes promptly to `"Clicked 1 times"`; the `rebuild` builds anew: generation `#1` is built with a slot at `0/#1/0`, and `!!clicks` fires with the new slot's click event. `!Switch` does not deliver anything from the new slot in this occurrence, and could not anyway, since no input was addressed to a slot that did not exist as the occurrence opened. At commit `!Switch` adopts the new inner event, generation `#0` is dropped, and `$count` is `1`. The label rebuilds too.

```
0/#1/0: Ui.Button(label="Clicked 1 times")
1/#0/0: Ui.Button(label="Reset")
2/#1/0: Ui.Label(text="Clicked 1 times")
```

**Occurrence 2.** `click` to `0/#1/0` and `click` to `1/#0/0` together: the user clicked the button and Reset in the same frame. Both slots' events fire. `$count + 1 on !click | 0 on !reset` takes the left: `$count` becomes `2`, and the reset is lost. The program said so; if reset should win, the merge is written the other way round. `$label` changes, the `rebuild` builds anew to `#2`.

**Occurrence 3.** Suppose the World has a click for `0/#1/0` queued from before it read the list after occurrence 2. It must not send it: `0/#1/0` is not in the list it last received, and sending to a path not in the opened list is an error of kind `UnknownPath`. A conforming World discards it. The click was for a button that no longer exists.

**Occurrence 4.** `click` to `1/#0/0` alone: Reset. `0 on !reset` fires; the merge's left side is quiet; `$count` becomes `0`; `$label` changes to `"Clicked 0 times"`; the `rebuild` builds anew to generation `#3` with a button whose label is the same text generation `#0` had. It is a different slot with a different path. A World that cached something about the first button by its path would find the cache useless, which is right: it is a new button. A World that keys native widgets by tree position and tag can reuse what it had, and the program cannot tell.

The question this example answers is what a click means in the occurrence the button is replaced. Routing says: the click is delivered to the slot at that path in the list as the occurrence opened, which is the old button, under the old label. The old widget keeps propagating until commit, so its click reaches `!Switch`, which is still listening to it. Then the new button exists, and from the next

occurrence its clicks are the ones heard. There is no moment in which the program has a button on screen that nothing is listening to, and no moment in which two buttons are listened to at once.

`!Switch` not being prompt costs nothing here. A freshly built button cannot be clicked in the occurrence it was built, because the World addresses only paths it has seen. The only thing a new generation could fire in its birth occurrence is something pure, like `@started`, and if a block wants to act on its own birth it uses `@started` inside the block, where no switch stands in the way. This is why the language can refuse prompt switching and lose nothing anyone wanted, while keeping every node's height fixed by its sources.

## 5. A file viewer

```
type FsRequest = choice Read | Write Text

type FsError = choice:
    NotFound
    Denied
    TooLarge
    Other Text

type FsResponse = choice:
    Contents Text
    Unavailable FsError

type Contents = choice:
    Unavailable FsError
    Known Text

tag Fs.File of Element:
    path: Text
    requests: !FsRequest
    responds: FsResponse

tag Fs.Contents of Element:
    path: Text
    state: Contents

let ShowResponse: fn ?FsResponse -> Text = \|r| match r:
    Nil: "(reading...)"
    Some (Contents t): t
    Some (Unavailable why): ShowError why

let ShowError: fn FsError -> Text = \|e| match e:
    NotFound: "(no such file)"
    Denied: "(permission denied)"
    TooLarge: "(too large)"
    Other t: "({t})"

let ShowContents: fn Contents -> Text = \|c| match c:
    Unavailable why: ShowError why
    Known t: t

@viewer:
    !started <- @started
    !motd <- @Element Fs.File(path="/etc/motd", requests=(Read on !started))
    $motd <- Nil then Some !motd
    @Label (ShowResponse $motd)
```

```
$path <- @TextInput "path", "/etc/hostname"
!$contents <- rebuild $path:
    @Element Fs.Contents(path=_)
$$current <- $ (Unavailable NotFound) then !$contents
@Label (ShowContents ($Join $$current))
```

Two ways to get a file, because there are two things you might mean. The top half wants the message of the day once, now, and is content to wait: it fires a `Read` request from `@started` and holds the response when it comes. The bottom half wants whatever file the user has typed the path of, as a value that is simply true: it asks for `Fs.Contents` state, which has a value in the occurrence it is created and is pushed when the file changes.

The top half is where "no request on creation" shows. Creating the `Fs.File` slot does nothing; the `Read` fires because `!started` fired, and `!started` fires exactly once, in the build occurrence, and it is pure. Compare the fetch-on-mount you have written, where every re-mount was a surprise network call and every test needed the mock before the first render. Here, if `@viewer` is rebuilt by some `rebuild` around it, the read happens again because `@started` fires again, and that is visible in the program's text rather than in a lifecycle table. Until the response arrives `$motd` is `Nil`, and its type says so: the `match` in `ShowResponse` has an arm for not-yet, and leaving it out is a compile error.

The bottom half is where dynamic fields show. `Fs.Contents(path=_)` has a plain path, but the `rebuild` over `$path` means a fresh slot per path: a new path is a new file, and the new slot's state is sampled in the occurrence of the change. The `rebuild`'s result is `!$Contents`, an event of dynamics, one per build; a hold with a constant initial dynamic and a `$Join` turn that into the current file's contents as one dynamic. The specification lets you write `@Element Fs.Contents(path=$path)` for exactly this, and this chapter spells it out once.

**Occurrence 0 (build).** `!started` fires. `Read on !started` fires, which is the `Fs.File` slot's request; the World will see it after the occurrence. `!motd` does not fire: nothing addressed to a slot arrives in the occurrence the slot was created. `$motd` is `Nil`, so the first label is `"(reading...)"`. `@TextInput` builds with name `"path"` and text `"/etc/hostname"`; the `rebuild` builds generation `#0` with an `Fs.Contents(path="/etc/hostname")` slot, and the runtime asks the World's oracle for that element's value as this occurrence opened. The World has never been asked about it; this is the first ask for a local file, so it reads the file once under its size cap, records the result in its snapshot, and answers `Known "box"`. `!$contents` fires with the new slot's dynamic; `$$current` changes to it in this same occurrence (a hold adopts a firing in its build occurrence); `$Join $$current` is `Known "box"`; the second label is `"box"`. Commit.

```
1:          Fs.File(path="/etc/motd")            requested: Read
2/#0/0:     Ui.Label(text="(reading...)")
3/0:        Ui.Focusable(name="path")
3/1/#0/0:   Ui.Label(text="/etc/hostname|")
4/#0/0:     Fs.Contents(path="/etc/hostname")
5/#0/0:     Ui.Label(text="box")
```

The World reads the list, sees a `Read` fired on slot `1`, and starts reading `/etc/motd`. It also sees an `Fs.Contents` slot for `/etc/hostname`, which it already holds, and starts watching that file.

**Occurrence 1.** The machine is renamed. The World's watch sees `/etc/hostname` change, so it has a new snapshot in which `Fs.Contents(path="/etc/hostname")` is `Known "crate"`, and the difference from the previous snapshot is one push. It opens an occurrence with that push and no responses. The slot at `4/#0/0` is identified for state by its element value, not its path, so the push reaches it: its dynamic changes to `Known "crate"`; `$Join $$current` changes promptly; the label at `5` rebuilds to `#1`

with `"crate"`.

**Occurrence 2.** The motd read finishes. The World sends `Contents "Welcome"` to path `1`. `!motd` fires; `Some !motd` is `$motd`'s change event; the label at `2` rebuilds to `"Welcome"`. Commit: `$motd` is `Some (Contents "Welcome")`.

**Occurrence 3.** The user presses Backspace in the path input: `Backspace` to `3/0`. `$path` changes to `"/etc/hostnam"`; the `rebuild` builds anew: generation `#1` is built with `Fs.Contents(path="/etc/hostnam")`, and the oracle is asked for it. The World has never seen that path, reads it once under its cap, and answers `Unavailable NotFound`. `!$contents` fires with the new slot's dynamic; `$$current` changes; the join changes to `Unavailable NotFound`; the label rebuilds to `"(no such file)"`. At commit generation `#0` is dropped, and with it the only slot for `/etc/hostname`; the World stops watching that file.

```
1:          Fs.File(path="/etc/motd")
2/#2/0:     Ui.Label(text="Welcome")
3/0:        Ui.Focusable(name="path")
3/1/#1/0:   Ui.Label(text="/etc/hostnam|")
4/#1/0:     Fs.Contents(path="/etc/hostnam")
5/#2/0:     Ui.Label(text="(no such file)")
```

**Occurrence 4.** Nothing arrives: the World's snapshot for `/etc/hostnam` is `Unavailable NotFound` and stays that way, so there is no push, and no occurrence happens at all. If the file ever appeared, the push would be `Known`. The program showed the truth the whole time.

If two widgets in a program ask for `Fs.Contents(path="/etc/hostname")`, there is one state, supplied once, pushed to both, and both see the same value in every occurrence: that is the equality law across the boundary, and it is why state is identified by value and responses by path. A response is a conversation with one slot; a state is a fact about the world.

This is the "not yet is a type" goal in both of its forms. The request form has a `?` that the program must match on. The state form has `Unavailable` with a reason as a case of the state's type, so a file that cannot be had says why in the type, and in no case is there a placeholder, a `Pending`, a retry loop, or a value that is wrong for one occurrence.

## 6. A stopwatch

```
type Instant = Instant Int                # milliseconds since the World's epoch

type Tick = tick

tag Clock of Element:
    state: Instant

tag Time.Every of Element:
    period: Int
    responds: Tick

type Run = choice:
    Stopped
    Running Instant                        # since when

type Watch = record:
    base: Int                             # milliseconds banked by earlier runs
    run: Run
```

```
type Action = choice Start | Stop | Reset

let Since: fn Run, Instant -> Int = \|run, now| match run:
    Stopped: 0
    Running from: now.value - from.value

let Step: fn Watch, Instant, Action -> Watch = \|w, now, a| match a:
    Start: match w.run:
        Running ignore: w
        Stopped: Watch w.base, (Running now)
    Stop: Watch (w.base + Since w.run, now), Stopped
    Reset: Watch 0, Stopped

let Total: fn Watch, Instant -> Int = \|w, now| w.base + Since w.run, now

@stopwatch:
    $clock <- @Element Clock
    @Element Time.Every(period=100)
    !start <- @Button $"Start"
    !stop <- @Button $"Stop"
    !reset <- @Button $"Reset"
    let !action = Start on !start | Stop on !stop | Reset on !reset
    $watch <- Watch 0, Stopped then Step $watch, $clock, !action
    @Label "{Total $watch, $clock} ms"
```

There is no `now()`. The clock is a state element like the contents of a file: it has a value in the occurrence the slot is created, and the World pushes a new one when it feels like it, which for a CLI or GUI World is at most once per turn. `Time.Every(period=100)` is bound as a statement and its tick event is discarded; the element exists to make the World come back every hundred milliseconds, and each time it comes back it opens an occurrence and pushes the clock. The label is derived from `$clock`, so it changes in that occurrence, and the stopwatch runs.

The state machine is one hold and one plain function. `Step` takes the watch, the clock, and what the user did, and returns the new watch; every input it needs is in its arguments, so it is testable without a World. `$clock` in `Step $watch, $clock, !action` is read through a tag: the time *before* the occurrence, which is the time the World pushed at the start of this turn or an earlier one. That is the right time to stamp a Start with, because it is the only one the program can know.

**Occurrence 0 (build).** `@Element Clock` creates a slot and asks the oracle for `Clock`'s value, say `Instant 5000`. `Time.Every` creates a slot. `$watch` is `Watch 0, Stopped`. `Total $watch, $clock` is `0`; the label is `"0 ms"`.

```
0:       Clock
1:       Time.Every(period=100)
2/#0/0:  Ui.Button(label="Start")
3/#0/0:  Ui.Button(label="Stop")
4/#0/0:  Ui.Button(label="Reset")
5/#0/0:  Ui.Label(text="0 ms")
```

**Occurrence 1.** A hundred milliseconds pass. The World opens an occurrence with a `tick` to `1` and a push for `Clock` of `Instant 5100`. The tick's event fires and nothing listens. `$clock` changes to `5100`; `Total $watch, $clock` is still `0`, since the watch is stopped; the derived label text changes, with the same value, and the label's `rebuild` builds anew to generation `#1` with the same text. A World that diffs lists sees no difference.

Rebuild a label every tick with the same text is wasteful and the program could avoid it, by deriving the label from `$watch` alone while stopped; it stays because correctness does not depend on it. The

rebuild is a description, not a draw call.

**Occurrence 2.** `click` to `2/#0/0`, together with the tick and a push of `5200`: the World delivered the click and the tick in one occurrence because both were waiting. `!start` fires; `!action` fires with `Start`; `Step $watch, $clock, !action` reads `$watch` as `Watch 0, Stopped` and `$clock` as `5100`, the value before the occurrence, and fires with `Watch 0, (Running (Instant 5100))`. `$clock` changes to `5200`. The label's text is `Total (Watch 0, Running 5100), 5200`, which is `"100 ms"`. Commit.

The stamp is `5100` and not `5200`, by the tag law, and the push in the same occurrence did not change that. If a program wanted the stamp to be the pushed value, it could not have it, because the push is the occurrence's input and the tag reads before the occurrence; the hundred milliseconds between the two is the price of a readable rule. A World that cares about it pushes the clock in an occurrence of its own before delivering the click, and the program sees no difference in its text.

**Occurrences 3 through 11.** Nine ticks, nine pushes, the label counts up by hundreds.

**Occurrence 12.** `click` to `3/#0/0` with the tick and a push of `6200`. `!action` fires with `Stop`; `Step` reads `$clock` as `6100` and banks `1000` ms: `Watch 1000, Stopped`. The label shows `"1000 ms"` and stays there through the following ticks.

**Occurrence 20.** `click` to `4/#0/0`: Reset. `Watch 0, Stopped`; `"0 ms"`.

To test this program, a script sets the clock: `with Clock = Instant 5000`, a click, `push Clock = Instant 5100`, an assertion. Anyone who has tried to unit-test a debounce with a real timer knows what that is worth, and it falls out of the clock being a state element rather than a function. The goal served is "no clock": time is data the World supplies, the program cannot ask for a finer one, and a transcript of pushes replays it exactly.

# 7. A sandbox

```
tag Defer of Element:
    requests: !unit
    responds: unit

# Map a path under the sandbox root, or refuse it.
let Confine: fn Text, Text -> ?Text = \|root, p|
    if (StartsWith p, "/home/"): Some (Concat [[root, p]])
    else: Nil

@Forward: fn $(Path ==> Slot) -> @!(Path ==> Response) = \|$slots| @widget:
    for $slots:
        rebuild $_:
            ((!resp, ignore)) <- @Element _.element
            @ resp

@Sandbox: fn Text, @a -> @a = \|root, @child| @widget:
    ((result, $slots)) <- intercept @child, !down, $ supply
    !down <- for $slots:
        rebuild $_:
            match _.element:
                Fs.File(path=p):
                    match Confine root, p:
                        Some q:
                            !resp <- @Element Fs.File(path=q, requests=_.!request)
                            @ resp
                        Nil:
```

```
                        !later <- @Element Defer(requests=(unit on _.!request))
                        @ Unavailable Denied on !later
            ignore:
                ((!resp, ignore)) <- @Element _.element
                @ resp
let supply = \|e| match e:
    Fs.Contents(path=p): match Confine root, p:
        Some q: @Element Fs.Contents(path=q)
        Nil: @ $ Unavailable Denied
    ignore: @Element e
@ result
```

You want to run a widget you did not write and be sure it cannot read your files. In most languages that is a question about the operating system, because the widget has the same ambient access you do. Here it is a question about the interceptor around it, because the widget has no ambient anything: every file it wants is an element in its list, and the only way that element reaches the World is if `@Sandbox` makes one.

`intercept @child, !down, $ supply` builds the child and captures its element list as `$slots`, a map from the child's paths to its slots. Nothing in `$slots` goes further out by itself. The two other arguments are the two directions down: `!down` is the responses the sandbox delivers to the child's slots, by path; `supply` is how the sandbox answers the child's state elements.

The response loop is a `for` over the child's slots. A slot never changes, so the `rebuild` inside builds once per slot and `_` is the plain slot. An `Fs.File` under `/home/` is forwarded with its path rewritten: the sandbox makes its *own* `Fs.File` element, with the child's request event wired straight into it, and the response the outer World sends back is this block's result, which `for` consolidates into `!(Path ==> Response)` keyed by the child's path, which is `!down`. Any other `Fs.File` is denied, and denial is the interesting case. The sandbox cannot answer a request in the occurrence it was made, and it cannot start an occurrence of its own. So it fires a `Defer`, which the World answers in a later occurrence of the same turn, and answers `Denied` then. The catch-all arm forwards everything else unchanged.

`supply` is a function from an element to a blueprint that returns the state's dynamic. When the child creates an `Fs.Contents` slot, the runtime applies `supply` to the element and builds the returned blueprint as a child of the sandbox's widget, in the same build pass, so the slot has its value in its creation occurrence. For a confined path the blueprint is the sandbox's own `Fs.Contents` element with the rewritten path, which asks the outer World; for a refused one it is the constant `$ Unavailable Denied`. A sandbox that wants to deny must say what denial is in the state's type, and `Unavailable FsError` is a case of it.

Take the child to be the viewer of example 5, with its motd path changed to `/home/me/motd` and its default input path to `/home/me/hostname`, under `@Sandbox "/jail", @viewer`.

**Occurrence 0 (build).** The sandbox's widget builds the child. The child's `Fs.File(path="/home/me/motd")` slot is created at child path `1`; the child's `Fs.Contents(path="/home/me/hostname")` slot at child path `4/#0/0`, and the runtime asks `supply` for it, which returns `@Element Fs.Contents(path="/jail/home/me/hostname")`, built in the sandbox's widget; the outer World's oracle answers `Unavailable NotFound`, since nothing is at that path under the jail. The child's `!started` fires, so its `Read` request fires. Now the sandbox's `for $slots` builds a block per child slot; the `Fs.File` block makes `Fs.File(path="/jail/home/me/motd", requests=...)` at a sandbox path, and since the child's request event fired in this occurrence and a widget built in an occurrence sees all of it, the rewritten element's request fires too. The child's other slots, the focusable and the labels, go through the catch-all arm and are re-emitted as elements of the sandbox. What the outer World sees:

```
1/[1]/#0/0:        Fs.File(path="/jail/home/me/motd")            requested: Read
1/[2/#0/0]/#0/0:   Ui.Label(text="(reading...)")
1/[3/0]/#0/0:      Ui.Focusable(name="path")
1/[3/1/#0/0]/#0/0: Ui.Label(text="/home/me/hostname|")
1/[5/#0/0]/#0/0:   Ui.Label(text="(no such file)")
s/0:               Fs.Contents(path="/jail/home/me/hostname")
```

The path prefix `1/[child path]/#0/0` is the sandbox's `for` (its second bind, after the intercept), keyed by the child's path, then the `rebuild` generation, then the slot. The child's own paths are not visible outside; the outer World addresses the sandbox's slots, and the sandbox routes inward.

**Occurrence 1.** The outer World answers the motd read: `Contents "hi"` to `1/[1]/#0/0`. That slot's response event fires; it is the `for` block's result for key `1`; `!down` fires with `{1: Contents "hi"}`; `intercept` routes it to the child's slot at path `1`, against the child's list as the occurrence opened. The child's `!motd` fires; its label rebuilds. The sandbox re-emits the new label in the same occurrence: the child's list changed, `$slots` changed, the `for` built a block for the new label's path and tore down the old one.

**Occurrence 2.** The user, inside the child, edits the path input to `/etc/passwd`. The child's `rebuild` builds anew its `Fs.Contents` slot with that path; the runtime applies `supply`, which returns `@ $ Unavailable Denied`, so the slot's dynamic is constantly `Unavailable Denied` and no element reaches the World. The old supplied widget is torn down at commit when its last slot goes.

**Occurrence 3.** Suppose the child, in some other part of it, builds `Fs.File(path="/etc/shadow", requests=(Read on !started))`. Its `@started` fires and so does its request, in the occurrence the slot is created. The sandbox's `for` builds a block for it: the `Nil` arm of `Confine. unit on _.!request` fires, so the `Defer` slot's request fires. Nothing else happens; from the child's view, it asked and is waiting, as it would be for a real read.

**Occurrence 4, same turn.** The World, after occurrence 3, found a fired `Defer` request and opened another occurrence delivering `unit` to it. `!later` fires; `Unavailable Denied on !later` fires; the `for` consolidates it as `{p: Unavailable Denied}`; `!down` fires; the child's slot receives `Unavailable Denied` and matches on it. From the outer World's view, occurrences 3 and 4 were one turn, and nothing it did was involved except answering a `Defer`.

The child cannot tell the difference between the sandbox and a World that happens to have a filesystem rooted at `/jail`. That is the test of the primitive: from inside, an interceptor is indistinguishable from the real thing. It is also why forwarding is manual. Suppose `intercept` took a table of tags to forward and did it itself; then the rewrite would be a mode of the table, the denial another, the `Defer` another, and before long the table is a second language. Here forwarding is the shortest program an interceptor can write, `@Forward` above, and rewriting, denying and holding back are longer programs in the same language. Nothing the child makes reaches the outside unless some Prism code chose to send it; there is no channel the sandbox does not see, because the language has no ambient anything.

# 8. A vertical box

```
type ChildInput = record:
    path: Path
    response: Response

tag Ui.VBox of Element:
    children: Path ==> Element
    responds: ChildInput
```

```
let IsUi: fn Element -> Bool = \|e| match e:
    Ui.*(..): True
    ignore: False

let UiOnly: fn (Path ==> Slot) -> Path ==> Element = \|slots|
    MapValues (Filter slots, \IsUi _.element), \_.element

let NonUi: fn (Path ==> Slot) -> Path ==> Slot = \|slots|
    Filter slots, \not IsUi _.element

let ToMap: fn ChildInput -> Path ==> Response = \|c|
    {c.path: c.response}

@VBox: fn @a -> @a = \|@child| @widget:
    ((result, $slots)) <- intercept @child, !down, $ \|e| @Element e
    !child_in <- @Element Ui.VBox(children=(UiOnly $slots))
    !fwd <- @Forward (NonUi $slots)
    let !down = !Union (ToMap !child_in), !fwd
    @ result
```

Layout is not something the language knows about, and this is the smallest demonstration of why it does not need to. A VBox is an interceptor: it captures its child's elements, holds back the library's (`Ui.*`), and puts out a single `Ui.VBox` whose `children` field holds them as data, in key order, which is tree order, which is top to bottom. The box does no geometry, and neither does any World. Further out, the library's window interceptor reads the `children` data, computes the layout, draws it as `Draw.*` values in a `Gui.Scene`, and hit-tests each pointer event from `Gui.Window` against the rectangles it computed. A pointer that lands on a row becomes a `ChildInput` holding that child's path and a `click`, delivered to the box's slot. The box turns it into a one-entry response map and `intercept` routes it. Everything the child made that is not a library element is forwarded untouched, by the `@Forward` of example 7.

The `children` field is a map that changes whenever the child's list changes, and a field is plain data. So the `Ui.VBox` element is rebuilt, a fresh slot per change of the child's list, and the input the box listens to is a `!Switch` over the generations: that is what binding an element with a dynamic field means. A layer that diffs lists sees a VBox replaced by a VBox at the same position with different children.

Routing falls out of the tag law. `intercept` delivers `!down` against the child's list as the occurrence opened. The user clicked a row of a layout drawn from the `children` field of the VBox slot that existed as the occurrence opened, and that field was built from exactly that list. The path in the `ChildInput` therefore names a slot in the list `intercept` routes against, and there is no way for a hit-test to pick a row the user has not seen.

Take the child to be the counter of example 1 wrapped as `@VBox @counter`. The counter's own paths are `0/#0/0`, `1/#0/0` and `2/#0/0`.

**Occurrence 0 (build).** The child is built under the intercept; `$slots` has three entries, all library elements. The box's bind builds generation `#0` with one `Ui.VBox` whose children are the three; `!child_in` fires with its input event, adopted by `!Switch` for occurrence 1. `NonUi $slots` is empty, so `@Forward` builds nothing. What the library above sees is one element:

```
1/#0/0: Ui.VBox(children={0/#0/0: Ui.Button(label="Increment"),
                          1/#0/0: Ui.Button(label="Decrement"),
                          2/#0/0: Ui.Label(text="Count: 0")})
```

**Occurrence 1.** The user clicks the first row. The library above hit-tests the pointer against the layout it drew from that list, finds the first child, and sends `ChildInput(path=0/#0/0, response=click)` to `1/#0/0`. (Under the Test World, `click Ui.Button(label="Increment")` produces the same input, by the convention at the end of this example.) The slot's event fires; `!Switch` delivers `!child_in`; `ToMap` makes `{0/#0/0: click}`; `!down` fires with the same, since `!fwd` is quiet. `intercept` routes `click` to the child's slot at `0/#0/0`; from there it is occurrence 1 of example 1: `!increment`, `!delta`, the count, the label rebuilt to `2/#1/0`. The child's list changes, so `$slots` changes promptly, the box's `rebuild` builds anew to generation `#1` with the new children, and the new slot's input event is adopted at commit.

```
1/#1/0: Ui.VBox(children={0/#0/0: Ui.Button(label="Increment"),
                          1/#0/0: Ui.Button(label="Decrement"),
                          2/#1/0: Ui.Label(text="Count: 1")})
```

**Occurrence 2.** The user clicks the third row. The library finds a label, which responds to nothing, and sends the box nothing. No occurrence happens, which is what clicking a label does.

**Occurrence 3.** The user clicks row 0 and, in the same frame, the child's list changes for some other reason (say, a second counter wrapped in the same VBox whose button was also clicked; or just suppose it). The `ChildInput` is delivered to the VBox slot that existed as the occurrence opened, and the click goes to the button at that path in the list as it opened. The new VBox slot built in this occurrence takes over from the next one. A click can never land on a row the user had not seen.

Had the child created a `Net.Listener` or an `Fs.File`, `NonUi $slots` would have had it, `@Forward` would have re-emitted it in the VBox's list beside the box, and its responses would have come back through `!fwd` under the child's path. A container and a sandbox are the same program with different arms.

The convention of a `children` field holding elements as data, and a `ChildInput` holding a path and a response, is the library's, not the language's. A Test World that wants `expect Ui.Button(label="Increment")` to find a button inside a box looks inside `children` fields, and its `send` wraps the response in one `ChildInput` per container, because the library and the test suite agree to; a different library could nest differently. The language fixes only three things: leaves are elements, containment is interception, and routing is against the opened list.

# 9. A cache

```
let Freeze: fn a -> $a = \|v| $ v

@Cached: fn @a -> @a = \|@child| @widget:
    ((result, $slots)) <- intercept @child, !down, $ supply
    !down <- @Forward $slots
    !refresh <- @Button $"Refresh"
    let supply = \|e| match e:
        Fs.Contents(path=p): @widget:
            $live <- @Element Fs.Contents(path=p)
            !started <- @started
            $$shown <- $live then (Freeze $live) on (!started | !refresh)
            @ $Join $$shown
        ignore: @Element e
    @ result
```

A child that watches a file sees every change the World pushes. Sometimes that is not what you want: a config file the program should read at start and again only when the user asks. This interceptor forwards every response and every element unchanged, and supplies `Fs.Contents` state through a hold

of dynamics.

The supplier returns a blueprint, and that is the point: a widget can have state. For an `Fs.Contents` element it builds, in the interceptor's widget, an `Fs.Contents` slot of its own asking the outer World for the live value, and a hold `$$shown` whose value is a dynamic: initially `$live` itself, then a frozen constant of the live value (`Freeze $live`, the constant dynamic of whatever `$live` holds) each time the supplied widget starts or the Refresh button is clicked. `(Freeze $live) on (!started | !refresh)` reads `$live` through a tag, the value before the occurrence. The child's slot gets `$Join $$shown`: the value of whichever dynamic the hold currently has.

Shared by value: the runtime builds a supplied widget once per distinct element value per interceptor. If the child has two slots for `Fs.Contents(path="/etc/app.conf")`, in two different widgets that have never heard of each other, there is one supplied widget, one live slot, one hold, and both child slots observe it. That is the equality law inside an interceptor for free, and it is also a cache in the ordinary sense: the outer World is asked once.

Take a child with two readers of `/etc/app.conf` and one of `/etc/motd`, each a label over the state.

**Occurrence 0 (build).** The child's first `Fs.Contents(path="/etc/app.conf")` slot is created; the runtime applies `supply` and builds the widget: a live slot in the cache's widget, whose oracle answer is, say, `Known "v1"`, and `$$shown`, whose initial value is `$live`. `!started` fires; `(Freeze $live) on !started` fires with the constant dynamic of `$live`'s value before the occurrence, which is what the oracle answered, `$ Known "v1"`; the hold adopts it in this occurrence. `$Join $$shown` has `Known "v1"` before the adoption, from `$live`, and `Known "v1"` after it, from the frozen constant. The child's slot's dynamic is that join, so the child's label is `"v1"`. The child's second `app.conf` slot is created; the runtime finds a supplied widget for that value already built and hands it the same join. The motd slot gets its own supplied widget. The World sees two `Fs.Contents` elements and a Refresh button, plus the forwarded labels.

**Occurrence 1.** The file changes on disk; the World pushes `Known "v2"` for `Fs.Contents(path="/etc/app.conf")`. The cache's live slot is identified by that value, so `$live` changes. `$$shown` holds the frozen constant and nothing fired at it, so the join does not change. Both child labels still say `"v1"`. Commit.

**Occurrence 2.** `click` on Refresh. `!refresh` fires; `(Freeze $live) on (!started | !refresh)` fires with the constant dynamic of `Known "v2"`; `$$shown` changes; the join changes promptly to `Known "v2"`; both child labels rebuild to `"v2"` in this occurrence.

**Occurrence 3.** The World pushes `Known "v3"` and, in the same occurrence, the user clicks Refresh. `$live` changes to `"v3"` in this occurrence, but `Freeze $live` on `!refresh` reads it through a tag and gets `"v2"`. The cache shows `"v2"`; the next refresh will show `"v3"`. This is the tag law doing what it always does, and a cache that wants the pushed value in the same occurrence as the click cannot have it, because the only window onto a dynamic at an event is the value before.

The child's slot has its value in its creation occurrence and nothing less, which is the law for state elements. Suppose the supplier held plain values instead, `$cached <- Unavailable NotFound then $live on (!started | !refresh)`. Then `$cached` would be `Unavailable NotFound` before the build occurrence and `Known "v1"` after it, and a tag in the child reading it in that occurrence would see `Unavailable NotFound`, a value the oracle never gave. The hold of dynamics has no such gap, because `$Join` switches to the new inner's current value in the occurrence of the switch instead of waiting for the new inner to fire, and the hold starts as `$live`, which has the oracle's value from the first instant.

The goal served is the one that says the language is enough. A cache is not a feature of the runtime, a flag on the element, or a mode of `intercept`. It is a hold, in a widget, returned by a function, and anything else you want between a child and the World is the same construction with a different body.

## 10. A chat client

```
type SocketEvent = choice:
    Received Text
    Closed

type Link = choice:
    Connecting
    Connected
    Disconnected Text

tag Net.Socket of Element:
    address: Text
    requests: !Text
    responds: SocketEvent
    state: Link

let WhenConnected: fn Link, Text -> ?Text = \|link, msg| match link:
    Connected: Some msg
    ignore: Nil

let Line: fn SocketEvent -> ?Text = \|ev| match ev:
    Received t: Some t
    Closed: Some "(closed)"

let Status: fn Link -> Text = \|link| match link:
    Connecting: "connecting..."
    Connected: "connected"
    Disconnected why: Concat [["disconnected: ", why]]

@chat:
    $address <- @TextInput "address", "chat.example:7000"
    $draft <- @TextInput "draft", ""
    !send <- @Button $"Send"
    !conns <- rebuild $address:
        let !out = !Filter (WhenConnected $link, $draft on !send)
        ((!rx, $link)) <- @Element Net.Socket(address=_, requests=!out)
        @ ((!rx, $link))
    let !rx = !Switch (First !conns)
    $$link <- $ Connecting then Second !conns
    let $link = $Join $$link
    $log <- [[]] then Append $log, !Filter (Line !rx)
    @Label (Status $link)
    @Label (JoinText "\n", $log)
```

One element, three parts. `Net.Socket` takes a request event, the text to send; responds with what arrives; and has state, whether the connection is up. The address is a dynamic, so the whole element lives inside a `rebuild`: a new address is a new socket, and reconnecting is typing. The `rebuild` returns a tuple, an event and a dynamic, and the two halves cross the `rebuild` boundary differently because they are different kinds of thing: the event through `!Switch`, which is not prompt, and the dynamic through a hold and `$Join`, which is.

`!out` is the send, filtered by the link state: `WhenConnected $link, $draft on !send` reads both dynamics through tags and fires only when the link was `Connected` before the occurrence. `$link` inside the `rebuild` block is the socket's own state, the one from this generation, and `!out` is bound before the element that produces it, which is fine in a widget block: bindings are recursive, and there is no prompt edge from a request event to a response event or to the state, since those come from outside

in later occurrences.

**Occurrence 0 (build).** Two text inputs and a button are built. The `rebuild` builds generation `#0`: an `Net.Socket(address="chat.example:7000")` slot. The oracle is asked for `Link` of that element; the World has no such connection and answers `Connecting`. `!conns` fires with the tuple; `$$link` changes to the new `$link` in this occurrence; the join is `Connecting`; the status label says `"connecting..."`. `!send` did not fire, so nothing was requested.

```
0/0:        Ui.Focusable(name="address")
0/1/#0/0:   Ui.Label(text="chat.example:7000|")
1/0:        Ui.Focusable(name="draft")
1/1/#0/0:   Ui.Label(text="|")
2/#0/0:     Ui.Button(label="Send")
3/#0/0:     Net.Socket(address="chat.example:7000")
4/#0/0:     Ui.Label(text="connecting...")
5/#0/0:     Ui.Label(text="")
```

The World reads the list, sees a socket element for an address it has no connection to, and opens one. Whether it keeps one OS socket per address or per slot is its business.

**Occurrence 1.** The connection comes up. The World pushes `Connected` for `Net.Socket(address="chat.example:7000")`. The slot's `$link` changes; the join changes; the status label rebuilds to `"connected"`.

**Occurrence 2.** The user has typed `hello` into the draft (several occurrences, each a keystroke to `1/0`, omitted) and clicks Send. `click` to `2/#0/0`. `!send` fires; `$draft on !send` fires with `"hello"`; `WhenConnected` reads `$link` as `Connected` and passes it; `!out` fires, which is the slot's request. The World sees `requested: "hello"` on `3/#0/0` after the occurrence and writes it to the connection.

**Occurrence 3.** A line arrives. The World sends `Received "hi back"` to `3/#0/0`. The slot's response event fires; `!Switch` delivers it as `!rx`; `Line` passes it; `Append $log, !line` fires with `[["hi back"]]`; the log label rebuilds.

**Occurrence 4.** The user edits the address to `chat.example:7001` and, in the same frame, clicks Send with `"bye"` in the draft. The keystroke goes to `0/0` and the click to `2/#0/0`. `$address` changes; the `rebuild` builds anew: generation `#1` is built with a slot for the new address, whose oracle answer is `Connecting`, since the World has no such connection. Now both generations propagate in this occurrence. In generation `#0`, `$draft on !send` fires with `"bye"`, `WhenConnected` reads that generation's `$link` as `Connected`, and the old slot's request fires. In generation `#1`, which sees all of the occurrence including the click, `$draft on !send` fires too, but `WhenConnected` reads the new slot's `$link` as `Connecting` and drops it. Exactly one send, on the old connection. `!conns` fires with the new tuple; `$$link` changes to the new `$link`; the join is `Connecting`; the status label says `"connecting..."`. At commit generation `#0` is dropped. `!Switch` adopts the new `!rx`.

```
3/#1/0:     Net.Socket(address="chat.example:7001")
```

The World sees the old socket element gone and the new one present. It closes the old connection, after writing `"bye"`, and opens the new one. The old slot's request fired in this occurrence and its path is not in the list after it; the World performs it or not by its own policy for an operation whose slot is gone, and whatever it does, no response for it will ever reach the program, because there is no slot to address.

**Occurrence 5.** Suppose the World had a `Received` queued for `3/#0/0`, read from the old connection before it closed. It discards it: that path is not in the list it last received.

A second `Net.Socket` element for `chat.example:7000` somewhere else in the program would share the first's `Link` state, because state is identified by value, and its sends and receives would be its own, because responses are identified by path. One connection, two conversations. If the user had typed

the old address back, a second slot with that value would keep the connection alive through the change, which is either a feature or a surprise depending on what the program wanted, and either way it is in the program's text.

What this prevents: the socket handler that writes to a connection object captured by a closure that outlived the reconnect. There is no connection object. There is an element with an address, a request event that is the program's only way to send, and a slot that lives exactly as long as the `rebuild` generation does.

## 11. A line counter

```
type StdinEvent = choice:
    Chunk Text
    End

type Code = Code Int

type WriteFailed = choice:
    Closed
    Failed Text

tag Cli.Stdin of Element:
    responds: StdinEvent

tag Cli.Stdout of Element:
    requests: !Text
    responds: WriteFailed

tag Process.Exit of Element:
    requests: !Code

let ChunkOf: fn StdinEvent -> ?Text = \|ev| match ev:
    Chunk t: Some t
    End: Nil

let IsEnd: fn StdinEvent -> ?unit = \|ev| match ev:
    End: Some unit
    ignore: Nil

@wc:
    !in <- @Element Cli.Stdin
    let !chunk = !Filter (ChunkOf !in)
    let !end = !Filter (IsEnd !in)
    $lines <- 0 then $lines + Count !chunk, '\n'
    !werr <- @Element Cli.Stdout(requests=("{$lines}\n" on !end))
    @Element Process.Exit(requests=(Code 0 on !end))
    @ unit
```

`wc -l`, under a World with no screen. Standard input is a stream element: it responds with chunks and then with an end, and the program never asks it for anything. Standard output and exit are request elements: nothing happens until their request events fire, and the firing is the whole instruction. There is no `print`, and there is no `exit()`; there is an element whose request fired, read by the World after the occurrence.

Exit is a request and not a presence. If exiting were "a `Process.Exit` element is in the list", the program would have to arrange for the element to appear at the right moment, with a `rebuild` over

some flag, and the World would have to watch for appearances, which is a trigger by existence and the language has none. Instead the element is always there and fires `Code 0` when the input ends.

**Occurrence 0 (build).** Three slots. `$lines` is `0`. Nothing fires.

```
0: Cli.Stdin
1: Cli.Stdout
2: Process.Exit
```

The World sees a `Cli.Stdin` slot and starts reading standard input.

**Occurrence 1.** The World sends `Chunk "one\ntwo\nthr"` to `0`. `!in` fires; `ChunkOf` passes it; `Count !chunk, '\n'` is `2`; `$lines + 2` fires with `2`. `IsEnd` gives `Nil` and `!end` is quiet. Commit: `$lines` is `2`.

**Occurrence 2.** `Chunk "ee\n"` to `0`. `$lines` becomes `3`.

**Occurrence 3.** `End` to `0`. `!end` fires; `"{$lines}\n" on !end` fires with `"3\n"`, reading `$lines` through a tag as `3`; `Code 0 on !end` fires. After the occurrence the World sees `requested: "3\n"` on `1` and `requested: Code 0` on `2`. It performs request firings in list order, so the write happens, then the exit.

`$lines on !end` is the count *before* the occurrence in which the end arrived. That is right because the end arrives alone: a slot receives at most one response per occurrence, so the last chunk and the end are always in different occurrences, and by the time `End` arrives every chunk has been counted and committed. A World that put the last chunk and the end into one occurrence could not, because they are both responses to the same slot. Had the program instead read a stream where the final chunk and the end were different slots, it would have had to count the chunk in the same expression as the output, and the type system would not have warned it; the walkthrough is the warning.

A real `wc` takes a file name on the command line, and that is where `@main` taking no arguments shows its reason. Arguments are a state element:

```
tag Process.Args of Element:
    state: [Text]

@wc_file:
    $args <- @Element Process.Args
    !!resps <- rebuild $args:
        !started <- @started
        match _[0]:
            Some path:
                !r <- @Element Fs.File(path=path, requests=(Read on !started))
                @ r
            Nil:
                @ !never
    let !resp = !Switch !!resps
    ...
```

The `match` inside the `rebuild` chooses between two blueprints by a plain value, which is the one form a `match` with blueprint arms may take.

`$args` has a value in occurrence 0, from the oracle, and the `rebuild` over it builds once with that value; a World that cannot observe a change to the arguments never pushes one and the `rebuild` never rebuilds. Arguments are not parameters of `@main` because of the sandbox of example 7: an interceptor can supply a child a different `Process.Args`, and a test can set them with `with Process.Args = [["notes.txt"]]`, and neither needs a second way to start a program. A `Process.Env` is the same again.

The World that runs this is the same one that runs example 5's viewer, and `Fs.File` is not a CLI tag: a GUI program reads files the same way. The CLI World adds only standard input, standard

output, standard error and signals, four tags; exit, like the arguments, is a process tag.

## 12. A tiny HTTP server

```
type Accepted = Accepted Nat

type ConnEvent = choice:
    Received Text
    Closed

type ConnRequest = choice:
    Send Text
    Close

tag Net.Listener of Element:
    port: Nat
    responds: Accepted

tag Net.Conn of Element:
    id: Nat
    requests: ![ConnRequest]
    responds: ConnEvent

type Delta = choice Opened | Shut

let IdOf: fn Accepted -> Nat = \|a| a.value

let RequestLine: fn ConnEvent -> ?Text = \|ev| match ev:
    Received t: Some t
    Closed: Nil

let IsClosed: fn ConnEvent -> ?unit = \|ev| match ev:
    Closed: Some unit
    ignore: Nil

let Respond: fn Text -> Text = \|request_line|
    Concat [["HTTP/1.1 200 OK\r\nContent-Length: 13\r\n\r\nHello, world\n"]]

let ApplyDeltas: fn Nat ==> unit, Nat ==> Delta -> Nat ==> unit = \|conns, deltas|
    Fold deltas, conns, \|acc, id, d| match d:
        Opened: Insert acc, id, unit
        Shut: Remove acc, id

@server:
    !accepted <- @Element Net.Listener(port=8080)
    let !opens = {IdOf !accepted: Opened}
    !closes <- for $conns:
        !ev <- @Element Net.Conn(id=key, requests=!reply)
        let !reply = [[Send (Respond !Filter (RequestLine !ev)), Close]]
        @ Shut on !Filter (IsClosed !ev)
    $conns <- {} then ApplyDeltas $conns, (!Union !opens, !closes)
    @ unit
```

A listener is a stream element: it responds with accepted connections, each with an id the World allocated, and asks for nothing. A connection is a request element keyed by that id: the program sends on it and hears from it. The program's model is the set of open connections, `$conns`, a map from id to nothing; the `for` over it builds one `Net.Conn` slot per open connection, and the connection's

block answers the first thing it receives with a fixed response and a close. When the World reports the connection closed, the block fires `Shut`, the consolidated event removes the key, and the block is torn down.

`!reply` is an array literal with an event inside it, so it is an event of arrays, firing when the request line arrives, with two requests in order. A request event fires at most once per occurrence, which is why a request type that wants two things in one moment is an array.

The cycle is the todo list's again: `$conns` is held from `!closes`, which is the result of `for $conns`. It passes through the tag on `$conns` in `ApplyDeltas` and through the delayed membership of the `for`'s merge, and a connection's events are responses, which never arrive in the occurrence the slot was created.

**Occurrence 0 (build).** One slot, the listener; `$conns` is `{}`; the `for` builds nothing.

```
0: Net.Listener(port=8080)
```

The World binds port 8080 because a listener element exists in the list, and starts accepting.

**Occurrence 1.** A client connects. The World sends `Accepted 7` to `0`. `!accepted` fires; `!opens` fires with `{7: Opened}`; `!Union` fires with it; `ApplyDeltas` fires with `{7: unit}`; `$conns` changes; the `for` builds block `[7]` in this occurrence, creating `Net.Conn(id=7)` at `1/[7]/0`. Nothing else fires: the new slot's `!ev` cannot fire in the occurrence it was created. Commit.

```
0:       Net.Listener(port=8080)
1/[7]/0: Net.Conn(id=7)
```

The World sees a connection element for id 7, which it has, and will deliver its bytes to `1/[7]/0`.

**Occurrence 2.** The request arrives. `Received "GET / HTTP/1.1\r\n..."` to `1/[7]/0`. `!ev` fires in block `[7]`; `RequestLine` passes it; `Respond` builds the response; `!reply` fires with `[[Send "HTTP/1.1 200 OK...", Close]]`, which is the slot's request. `IsClosed` is `Nil`; the block's result is quiet; `$conns` is unchanged. After the occurrence the World sees `requested: [[Send "...", Close]]` on `1/[7]/0`, writes the response and closes the connection.

**Occurrence 3.** Two things at once: the World has closed connection 7 and reports it, and a new client has connected. `Closed` to `1/[7]/0` and `Accepted 8` to `0`, together. Block `[7]` fires `Shut`; `!closes` fires with `{7: Shut}`. `!opens` fires with `{8: Opened}`. `!Union` fires with `{7: Shut, 8: Opened}`; `ApplyDeltas` removes 7 and inserts 8; `$conns` changes; the `for` builds `[8]` now and tears `[7]` down at commit. The World sees one connection element replaced by another and, because it already closed 7, has nothing to cancel.

```
0:       Net.Listener(port=8080)
1/[8]/0: Net.Conn(id=8)
```

**Occurrence 4.** Suppose the World has bytes queued for `1/[7]/0` that arrived before the close. It must discard them: the path is not in the list it last received. A late result for a slot that is gone is the World's problem, and the rule that it must drop them is what lets the program never have a slot it did not ask for.

A server that read real requests would need to collect the bytes until the blank line that ends the headers, and that is a hold per connection, inside the `for` block, next to the slot it serves:

```
!closes <- for $conns:
    !ev <- @Element Net.Conn(id=key, requests=!reply)
    $buf <- "" then Concat [[$buf, !Filter (RequestLine !ev)]]
    let !head = !Filter (CompleteHead (Concat [[$buf, !Filter (RequestLine !ev)]]))
    let !reply = [[Send (Respond !head), Close]]
```

```
        @ Shut on !Filter (IsClosed !ev)
```

`$buf` lives as long as key `key` does and is torn down with it; two connections have two buffers that cannot see each other; and `Concat [[$buf, !chunk]]` reads the buffer before the chunk by the tag law, which is the right buffer to append to. `CompleteHead: fn Text -> ?Text` is a plain function that returns the head once the blank line is in. The second `Concat` repeats the first rather than reading `$buf` after it changed, because there is no "after" inside an occurrence: a program that wants the new buffer and the test on it in the same occurrence writes the expression once and uses it twice, or writes a derived `let`.

What this prevents: the server where a connection object is kept in a map by hand, removed in a close callback that sometimes runs after the next read callback, so the map has a dead entry that the next write crashes on. Here the map is the only model, its definition lists every event that changes it, the slot's life is the key's life, and a write to a dead connection is impossible because a dead connection has no slot.

## 13. Testing the counter

```
test counter_counts, @counter:
    build
        => expect Ui.Label(text="Count: 0")
    click Ui.Button(label="Increment")
        => settles
    click Ui.Button(label="Increment")
        => expect Ui.Label(text="Count: 2")
    click Ui.Button(label="Decrement")
        => expect Ui.Label(text="Count: 1")
        => expect_none Ui.Label(text="Count: 2")

test counter_left_wins, @counter:
    build
        => settles
    click Ui.Button(label="Increment"), click Ui.Button(label="Decrement")
        => expect Ui.Label(text="Count: -1")

test counter_is_quiet, @counter:
    build
        => settles
    click Ui.Button(label="Increment")
        => quiet
        => returns unit

test stopwatch_banks_time, @stopwatch:
    with Clock = Instant 5000, build
        => expect Ui.Label(text="0 ms")
    click Ui.Button(label="Start")
        => settles
    push Clock = Instant 5100
        => expect Ui.Label(text="100 ms")
    push Clock = Instant 5600
        => settles
    click Ui.Button(label="Stop"), send Time.Every(period=100), tick
        => expect Ui.Label(text="600 ms")
    push Clock = Instant 9000
        => expect Ui.Label(text="600 ms")
```

```
        => unrequested Time.Every(..)
```

A test is a World. The Test World accepts every tag, has no effects, is the only source of inputs, and its snapshot is whatever the script says. A script drives the program the way a user would, by sending inputs to elements, and reads what a user would read, the element list. It cannot reach into `@counter` and poke `$count`, because nothing can: there are no reified names, and a test that passes under this World passes under any World that delivers the same inputs.

Each step is a turn or a check between turns. The build is turn zero, so the first `expect` sees the built program. `send` finds exactly one slot matching the pattern, delivers the value, and runs a turn, including any `Defer` occurrences the program causes. Several deliveries on one line go in one occurrence; two to the same slot fail the step, because a slot receives one response per occurrence. `with` sets the snapshot value a state element will see, from the next turn's oracle and push; `push` is `with` followed by a turn with nothing else in it. `quiet` and `returns` look at the widget's result over the last turn.

Here is the first script as the Test World sees it, turn by turn.

**Turn 0 (build).** The list is example 1's after occurrence 0. `expect Ui.Label(text="Count: 0")`: some slot at any depth matches; `2/#0/0` does. Pass.

**Turn 1.** `click Ui.Button(label="Increment")`: one slot matches, `0/#0/0`; the World runs an occurrence with `{0/#0/0: click}` as its responses. Occurrence 1 of example 1. The list now has `2/#1/0: Ui.Label(text="Count: 1")`.

**Turn 2.** The same. `"Count: 2"`. The `expect` passes.

**Turn 3.** `send` to the Decrement slot. `"Count: 1"`. `expect` passes; `expect_none Ui.Label(text="Count: 2")` passes because the generation-`#2` label was dropped at commit and is not in the list.

The second script is the tag law and the leftmost merge in four lines. the comma makes one occurrence with two responses to two slots; `!delta` fires with `-1`; the label says `"Count: -1"`. A test that wanted the other answer would be testing a different program.

The third script says something about the result. `@counter`'s last statement is `@Label ...`, whose result is `unit`, so the widget's result is `unit` and `returns unit` holds; `quiet` holds trivially because the result is not an event. A widget that returned `!delta` would make `fired +1` the interesting step.

The fourth script is example 6 under a clock the script owns. `with Clock = Instant 5000` sets the snapshot before the build, so the oracle answers it in occurrence 0. `push Clock = Instant 5100` is a turn with one push and no responses; the label counts. The line with both delivers a Stop click and a tick in one occurrence, as a real World might: `Step` reads `$clock` as `5600` through a tag and banks `600`. A later push does not move the label, because the watch is stopped. One more script, for the sandbox of example 7, because it shows a turn that is more than one occurrence:

```
@probe:
    !started <- @started
    !r <- @Element Fs.File(path="/etc/shadow", requests=(Read on !started))
    $seen <- Nil then Some !r
    @Label (ShowResponse $seen)

test sandbox_denies_outside_root, (@Sandbox "/jail", @probe):
    build
        => expect_none Fs.File(..)
        => expect Ui.Label(text="(reading...)")
        => requested Defer, unit
```

```
    push Clock = Instant 0
        => expect Ui.Label(text="(permission denied)")
```

**Turn 0 (build).** The probe's `Fs.File` slot is created and its request fires; the sandbox's `for` builds the `Nil` arm for it, which fires the `Defer`. The outer list has no `Fs.File` in it, which `expect_none` checks: the sandbox held it back. The label says `"(reading...)"`. But the turn is not over: the Test World, like every World, checks for fired `Defer` requests after each occurrence and runs another occurrence delivering the answer. In that second occurrence, still turn 0, the sandbox fires `Unavailable Denied` into the probe, and the probe's label rebuilds. So by the time the script's first step runs, the label already says `"(permission denied)"`, and the second `expect` above fails.

That is the point of the script as written: a reader who expects `"(reading...)"` has the turn wrong. The build turn is the build occurrence plus any `Defer` occurrences it causes, and a check between turns sees the end of the turn. The corrected script deletes the second `expect` and the `push`, which was only there to make a turn pass, and keeps `requested Defer, unit`, which holds because the `Defer` slot fired in turn 0, and the final `expect`. A test that wanted to see the request before the denial would need a World that delivered `Defer` answers lazily, and there is no such World, because the `Defer` rule is what bounds a turn.

The transcript the Test World prints for each turn, the element list, the request firings per slot, the result and any error, is what the conformance suite compares between the two implementations. A difference is a bug in one or an ambiguity in the specification, and the second kind is the more valuable find.

The goal served is "the spec is the thing", in its testable form. The script never names a path, a widget, or a runtime object; it names elements and values, which are the only things a World can name, and the transcript it checks is the only thing a World can see.

## 14. A shell

```
type App = choice A | B

let Other: fn App -> App = \|app| match app:
    A: B
    B: A

let Pick: fn App, Path ==> Slot, Path ==> Slot -> Path ==> Slot = \|app, sa, sb| match app:
    A: sa
    B: sb

let OnlyFor: fn App, App, Path ==> Response -> ?(Path ==> Response) = \|want, have, m|
    if (want == have): Some m
    else: Nil

@Shell: fn @unit, @unit -> @unit = \|@a, @b| @widget:
    ((ignore, $sa)) <- intercept @a, !down_a, $ \|e| @Element e
    ((ignore, $sb)) <- intercept @b, !down_b, $ \|e| @Element e
    !next <- @Button $"Next app"
    $active <- A then Other $active on !next
    let $shown = Pick $active, $sa, $sb
    !child_in <- @Element Ui.VBox(children=(UiOnly $shown))
    !fwd_a <- @Forward (NonUi $sa)
    !fwd_b <- @Forward (NonUi $sb)
```

```
    let !click = ToMap !child_in
    let !down_a = !Union (!Filter (OnlyFor A, $active, !click)), !fwd_a
    let !down_b = !Union (!Filter (OnlyFor B, $active, !click)), !fwd_b
    @ unit

@main:
    @Shell @counter, @stopwatch
```

A World can be written in Prism, and this is one: a program that runs two other programs as its children, shows one of them on the screen at a time, and routes the user's clicks to the one that is showing. From the children's side, `@Shell` is their World. The counter asks for buttons and a label and gets clicks; the stopwatch asks for a clock, a timer and buttons and gets pushes, ticks and clicks; neither knows the other exists or that the screen is shared. From the outer World's side, the shell is one program with a box, a button, a clock and a timer.

Two intercepts, one per child, each with its own response event and the forwarding supplier. The screen is example 8's VBox over whichever child is active, with `Pick` lifted over three dynamics choosing the list; the `ChildInput` the box receives names a path in `$shown` as the occurrence opened. The click goes down to the active child only, with `$active` read through a tag so that a click in the same occurrence as a switch of apps goes to the app the user was looking at. Everything that is not a library element, from either child, is forwarded unchanged, and the responses come back under the child's own paths through `!fwd_a` and `!fwd_b`. The inactive child keeps running: its holds persist, its `Time.Every` is still in the outer list, its clock still gets pushes. It is simply not drawn and not clickable.

**Occurrence 0 (build).** Both children are built. `$sa` has the counter's three slots; `$sb` has the stopwatch's six, of which `Clock` and `Time.Every` are not library elements. The oracle for `Clock` is asked through the shell's supplier, which makes an outer `Clock` element in the shell's widget, built once because there is one value, and the outer World answers `Instant 5000`. `$active` is `A`; `$shown` is `$sa`; the box's element is built as generation `#0` with the counter's three library elements as children. `@Forward (NonGui $sa)` builds nothing; `@Forward (NonGui $sb)` builds a `Clock` and a `Time.Every(period=100)` in the shell's widget. The outer World sees:

```
  2/#0/0:              Ui.Button(label="Next app")
  3/#0/0:              Ui.VBox(children={0/#0/0: Ui.Button(label="Increment"),
                                         1/#0/0: Ui.Button(label="Decrement"),
                                         2/#0/0: Ui.Label(text="Count: 0")})
  5/0/[0]/#0/0:        Clock
  5/0/[1]/#0/0:        Time.Every(period=100)
  s/0:                 Clock
```

Two `Clock` elements, the supplier's and the forwarder's, and that is fine: state is identified by value, so the outer World has one clock and both slots observe it. The stopwatch's own `Clock` slot gets its dynamic from the supplied widget, and the forwarded `Clock` slot's result is bound to nothing. A shell that minded the duplicate would not forward state elements, since the supplier already answers them; this one forwards everything that is not GUI because it is the shortest correct program, and the duplicate costs nothing.

**Occurrence 1.** A tick: `tick` to `5/0/[1]/#0/0` and a push of `Instant 5100` for `Clock`. The forwarded `Time.Every` slot's response is routed by `!fwd_b` to the stopwatch's slot at its path `1`; nothing listens there. The push reaches the supplied `Clock` widget's slot, whose dynamic is the stopwatch's `$clock`; the stopwatch's label, derived from it, rebuilds with the same text. `$sb` changes, but `$shown` is `$sa`, so nothing on screen changes.

**Occurrence 2.** The user clicks the first row of the box. The library above hit-tests the pointer against the layout it drew from `$shown`, the counter's list as it opened, finds the button, and sends `ChildInput(path=0/#0/0, response=click)` to `3/#0/0`. `!click` fires with `{0/#0/0: click}`; `OnlyFor A, $active, !click` reads `$active` as `A` and passes it; `!down_a` fires; the counter's Increment slot gets its click. The counter's label rebuilds; `$sa` changes; `$shown` and `$kids` change; the box rebuilds to generation `#1`. `OnlyFor B` drops the click and `!down_b` is quiet, apart from `!fwd_b`, which is also quiet.

**Occurrence 3.** `click` to `2/#0/0`, Next app, together with a tick and a push. `$active` changes to `B`; `Pick` changes `$shown` to `$sb`; the box rebuilds with the stopwatch's buttons and label. In the same occurrence the tick and push go where they always go.

```
2/#0/0:              Ui.Button(label="Next app")
3/#2/0:              Ui.VBox(children={2/#0/0: Ui.Button(label="Start"),
                                       3/#0/0: Ui.Button(label="Stop"),
                                       4/#0/0: Ui.Button(label="Reset"),
                                       5/#2/0: Ui.Label(text="0 ms")})
5/0/[0]/#0/0:        Clock
5/0/[1]/#0/0:        Time.Every(period=100)
s/0:                 Clock
```

**Occurrence 4.** The user clicks the first row, and in the same frame hits Next app again. `ChildInput(path=2/#0/0, response=click)` to `3/#2/0` and `click` to `2/#0/0`. The library hit-tested against the layout of `$shown` as it opened, the stopwatch's list, and found the Start button at `2/#0/0`. `OnlyFor A, $active, !click` reads `$active` as `B`, before the occurrence, and drops it; `OnlyFor B` passes it; the stopwatch starts. `$active` becomes `A` at commit and the screen shows the counter from the next occurrence. The click went to the app the user was looking at when they clicked, which is the only answer routing allows and the one the user meant.

**Occurrences 5 and on.** Ticks keep coming; the stopwatch keeps counting behind the counter; when the user switches back, the box shows the elapsed time it reached while hidden.

A shell, a window manager and a compositor are this program with more arms: a process table is a `for` over a keyed collection of programs, a window is a region of a bigger box, focus is which child gets the keys. None of it is in the language. The language supplies two things: an interceptor that re-emits everything and supplies by forwarding is observably not there, so each layer can be put in and taken out without the children knowing; and routing is always against the opened list, so no layer can send an input to something the user has not seen. The outer World below the shell is small, because the shell did the multiplexing; it could be the software rasteriser of a Prism-native system, or a Test World that says `click Ui.Button(label="Increment")`, which reaches the button through the box's `children` field by the library's convention, and checks what the box contains afterwards.

## What the fourteen have in common

Every program above is a set of equations over three kinds of name, built once, and driven by a World that sends plain data to paths it has seen and reads plain data back. None of them registers a callback, none of them reads a value "right now", none of them asks for an effect except by firing a request, and none of them has a moment in which some of its values are new and some are old, except through the tag law, which is the one deliberate window onto before. The hard cases a framework usually handles with a lifecycle table, a batching heuristic or a scheduler were each handled above by a sentence: a click in the occurrence of a rebuild goes to the old slot; a widget built mid-occurrence sees the whole occurrence; a state element has its value in the occurrence it is created; a request is answered later or not at all; the leftmost wins. The chapters that follow define those sentences one

at a time, and each one points back here to the occurrence where it did its work.

## Chapter 3

# The language: sigils, lexical structure, values and types, expressions

A Prism program is a set of equations between things that are true, things that happen and things that exist, and the sigils on their names say which is which before a type checker runs. This chapter is the part of the language that holds still: how a line is cut into tokens and lines into blocks, what the plain values under the sigils are, and how an expression over mixed sigils finds its meaning without anyone writing `map`. Every rule here is written so that two parsers built from this text agree on every program, and every rule is argued from the goals it serves, so that a reader who wants a different rule knows what it would cost.

## Sigils and where time lives

You have read a line like `total = price * quantity` in an interactive program and not known, without scrolling, whether it ran once or every time the quantity box changed. You found out in production, because the label showed the old total. In a Reflex program you knew, because the variables were called `dPrice` and `eQuantity`, and you also knew that the author of Reflex asked you not to name them that way, and that everybody did anyway, because a line of code with no hint about which operand is live is a line you cannot review.

Prism makes the hint mandatory, uniform, and checked. A name's first character says what the name is:

- `x` is a plain value. It is true, now and always. It cannot change, it cannot happen, it cannot be torn down.
- `!x` is an event. It happens at some occurrences, with a value each time, and between them there is nothing to look at.
- `$x` is a dynamic. It is true now, and it will be different later, at some occurrence.
- `@x` is a blueprint. You would expect a widget function to return a widget, and it cannot, because a widget is not a value: it has a lifetime, it is wired to other widgets, it is torn down, and two of them built from the same source are two different things, which is to say it is not referentially transparent. A widget is closer to an object than to data, and it belongs to the structure of the running program, which is beyond the reach of a pure language. What a pure language can hold is the description, and that is what `@x` is: a blueprint, from which the runtime builds a widget each time the blueprint is bound. The book says "widget" for the built thing, the way everyone does, and "blueprint" for the value of type `@a` whenever the difference matters.
- `?x` is a plain value that may be absent.

The four have names, for when the book has to talk about them as a kind rather than as a value: `!` is the *event functor*, `$` the *dynamic functor*, `@` the *widget monad*, `?` the *optional*. The first two are functors because an operation lifts over them (`$count + 1`) and that is all; flattening them is `!Switch`

and `$Join`, each with laws of its own, so neither is a monad in the sense that would let you treat `!!a` as an `!a`. The third is a monad in exactly that sense: `<-` is its bind, `@ v` is its return, and a widget block is a `do` block. The fourth lifts at the plain layer, inside the other three.

The sigil is part of the name, the compiler checks it against the type, and that is the whole idea. `$count + !delta` tells you, before you know what a count or a delta is, that one of them is live and one of them happens, and so it tells you that the expression is an event, firing when the delta does, with the count read as it stood before. No inference over functors is needed to know that; a lexer could classify it. This serves the goal that you can see what moves. The other goal it serves is less obvious: because the time structure of an expression is readable off the names, the compiler never has to guess it, so the lifting rule in the last section of this chapter is a lookup table rather than a search, and the two implementations cannot disagree about it.

## Stacking

The sigils stack, leftmost outermost, in the order you would say the thing out loud:

- `?$user` is an optional dynamic: maybe there is a live user, and if there is, it changes.
- `$?user` is a dynamic optional: there is always a value here, and sometimes it is "nobody".
- `!!rendered` is an event of events: what a `rebuild` returns, one inner event per build.
- `!$contents` is an event of dynamics: what a `rebuild` returns when its block returns state.
- `$$current` is a dynamic of dynamics: what you hold an `!$contents` into, before `$Join` flattens it.
- `@!click` is a blueprint whose result is an event: the type of a button.
- `@$text` is a blueprint whose result is a dynamic: the type of a text input.
- `$@a` is a dynamic of blueprints: what a `rebuild` consumes.
- `@@a` is a blueprint that returns a blueprint, unbuilt: a widget factory.

Two of these deserve a second look, because the difference between them is the kind of difference the sigils exist to make visible. `?$user` and `$?user` are not two spellings of one thing. The first says that the *existence* of the dynamic is in question and fixed: you match on it once, and either you have a dynamic forever or you never did. The second says the dynamic exists and its *value* is sometimes absent: a logged-in user who may log out. A chat client's current connection is the second; a widget's optional parent callback is the first. Writing one when you meant the other is a type error at the first use, which is where you want to find out.

## Nesting in plain aggregates

A sigil names a kind, not a place where the value is permitted. Functor values sit inside plain values freely:

- `[!Click]` is a plain array of events. The array does not change; its elements each happen.
- `((!a, $b))` is a plain tuple holding an event and a dynamic.
- A record with fields `!changed` and `$value` is a plain record and is the ordinary way a widget returns two things at once.
- `Path ==> Slot`, where a `Slot` holds an `!request`, is a plain map of records each holding an event.

The array and the record are plain in the sense that *they* do not move: their membership and their field set are fixed. That is why `!Leftmost [[!a, !b, !c]]` is an ordinary function from a plain array to an event, and why a widget's result bundle is a record and not some special "result" construct.

So the sigil's *position* says what changes, and the three ways to write a list of items mean three different things:

- `$[Item]` is a list that changes as a whole. When any item changes, there is a new list.
- `[$Item]` is a fixed list of items that each change on their own. The list never grows or shrinks.
- `$(Key ==> $Item)` is a map whose membership changes and whose entries also change on their own. This is what a `for` block sees: `$_` inside it is one entry's `$Item`, and a change to the entry does not rebuild the block.

These are different types with different operations. You cannot append to `[$Item]`, because the list is plain and plain values do not change; you can only build a new plain list, which is a new value, not a change to the old one. You cannot read one item of `$[Item]` as its own dynamic without a lifted index, `$items[3]`, which is `$?Item`. The compiler keeps them apart and the names keep them apart for the reader. In other languages this distinction exists just as much and is written nowhere: a `List<Observable<Item>>` and an `Observable<List<Item>>` are both "the items" to the person reading the call site.

## Where time lives

`!` and `$` are defined against *occurrences*: the discrete, atomic moments at which anything happens. An occurrence has a before and an after and no inside anyone can see. An event fires at an occurrence or does not; a dynamic has one value before an occurrence and one after. A widget's lifetime is measured in occurrences too: built in one, alive through some, torn down in one. The laws are in the chapter on the functors; this chapter needs only that the sigils name those three relations to time, and that plain values and optionals have no relation to time at all, which is what makes them plain.

That last point makes the aggregate rule above work. A plain array of events is plain because arrays have no relation to time: the array was built once and is what it is. The events inside have their own relation to time, each on its own. Nothing about "an array" needs a notion of "now". If it did, if arrays were somehow live, then every container would be a dynamic and every lifted operation would have to say which layer it lifted over. Instead there is one plain layer, which is timeless, and the event functor, the dynamic functor and the widget monad sitting on top of it or inside it, and lifting only ever looks at the outermost layer of each operand.

## Functions and return sigils

Functions and types are `UpperCamelCase`; values are `snake_case`. A function's name takes the sigil of what it *returns*: `@Button` returns a blueprint, `!Leftmost` returns an event, `$Join` returns a dynamic, `IsClick` returns a plain value. The sigil on a function is a promise about its result, not about its arguments, and it is checked against the annotation.

A function is a plain value when it is passed around. `$(fn Element -> @$State)` is a dynamic whose value is a function; the function itself is plain, it is the dynamic that changes. A function held in a record field, an array or a map is plain. There is no "function that moves": if you want a computation that changes over time, you have a dynamic of a function, and the sigil on the dynamic says so. This is the same rule as for every other value, and it means that `\|e| @Element e` passed as the supplier to `intercept` is a plain lambda, wrapped in `$` by the caller because the parameter wants a dynamic.

A function takes at least one argument. A function of nothing is spelled as taking `unit`, and a widget function of nothing is not a function: it is a blueprint. `@counter` is a value you bind, not something you call. The distinction matters for the reader more than for the checker: `@Button $"OK"` is an application and builds nothing until bound, and `@counter` is a description you bind as it is. Both are plain values until a `<-` builds them.

### What the sigils are not

They are not Hungarian notation. Hungarian notation is a convention a reviewer enforces; a sigil is a token a compiler enforces, and the difference is the difference between `dCount` being wrong in a code review and `count` bound to a dynamic being a compile error with a fix in the message. Suppose the sigils were optional, a style you could turn on. Then a library written without them would be used from a program written with them, and at the boundary a reader would be back to reading types to learn what moves. Suppose instead the sigils were inferred and shown by the editor. Then the program's text, the thing that goes in version control and in a diff, would say nothing, and a reviewer reading the diff would be where the Reflex reviewer was. The sigils are mandatory because the one place they have to be right is the one place that is not an editor: the diff.

They are also not types. `$count` says that `count` is a dynamic; its type, `$Int`, says of what. The sigil is the part of the type that governs lifting and the part a reader needs at every use; the rest is for the checker. A program that annotates every binding with its full type is allowed and not expected.

Consider the alternative, where a function's name said nothing about its result and `Button $"OK"` returned a blueprint by inference. Every call site would then need the type to be read before the line could be understood: does `Label text` draw a label or return the text of one? Because a name says what it returns, a call site can be read without the callee. It costs one character per function, and the character is one the reader was going to want anyway.

## Lexical structure and layout

Two parsers, one in Rust and one in Haskell, read every Prism program, and the conformance suite fails the day they disagree about one. That is the reason this section is written at the level of individual characters: every place where a reasonable lexer could go two ways gets one answer here, with the reasoning, so that the sentence can be changed instead of one of the lexers. The goals in play are the two about reading: you can see what moves, which is why sigils are tokens and not a convention; and the spec is the thing, which is why the offside rule is spelled out below with its ugly cases instead of being described as "like Python".

### Source and comments

Source is UTF-8. A comment runs from `#` to the end of the line and is whitespace for every purpose below, except inside a text literal, where `#` is a character. There are no block comments; a program that wants to comment out a block indents the `#` to taste, since comment-only lines never take part in layout.

Indentation is counted in spaces from column 0. A tab in leading whitespace is an error, with the message naming the line and column; a tab after the first non-space character of a line is ordinary whitespace. The alternative, giving a tab a width, means two editors can show one program with two layouts and a reader cannot tell which the lexer saw; the error costs nothing anyone wants.

### Identifiers and sigils

An identifier is an optional sigil prefix followed by a name. There are two name conventions and they are not interchangeable:

- Lowercase names are `snake_case` and denote values: `count`, `next_id`, `is_done`. The pronoun `_` is a lowercase name.

- Uppercase names are `UpperCamelCase` and denote functions, constructors and types: `Button`, `IsClick`, `TodoItem`.

The first character alone decides which kind a name is, so a name that strays from its convention is still understood. A lowercase name with a capital in it, `isClick`, is a value, and the compiler reports a lint warning suggesting `is_click`; the program compiles and runs. An uppercase name with an underscore in it, `Todo_item`, is an error, because a capital first letter promises a function, constructor or type, and `Todo_item` follows neither convention. The first case is a warning because nothing downstream depends on the spelling: the lexer, the checker and the reader all classify `isClick` correctly from its first letter, and refusing to compile a program over a style the compiler sees through would cost a build for a rename. Digits are allowed after the first character in both forms: `item2`, `Vec2`.

A qualified name is an uppercase name followed by `.` and another uppercase name, with no whitespace anywhere in it, repeated as needed: `Ui.Button`, `Fs.File`, `Net.Socket`. It is one token. The namespace part is how extensible variants (next section) spread their tags across modules. A `.` followed by a lowercase name is never part of a qualified name; it is field access, and `Ui.Button.label` would be field access on the constructor, which is a type error, not a lexing question.

The sigils are `!` `$` `@` `?`. (`&` is reserved for references, later in this chapter, and is rejected with a message saying so; `%` does not exist.) The meaning of a sigil depends on what it is attached to:

- On a value's name, it states the value's kind: `!delta` is an event.
- On a function's name, it states the kind of the function's result: `@Button` returns a blueprint.
- On a type, it is the type constructor: `!Text`, `$[Item]`, `@!Click`; sigils stack leftmost-outermost here as elsewhere.

**Attachment.** The lexer has to decide whether a sigil is part of the next token or a prefix operator standing on its own. The rule: a sigil is *attached*, and part of the token, when the character after it is a name character (letter, digit), `_`, `"`, or another sigil. Otherwise it is *free*: a prefix operator that applies to the whole expression after it, to the end of the enclosing call.

```
!delta                      # attached: the name of an event
$"Count: {$count}"          # attached to a text literal: a constant-or-lifted dynamic
!$contents                  # attached twice: an event of dynamics
$_                          # attached to the pronoun
@ unit                      # free: the widget returning unit
@ ChecklistUpdate ix, !checked   # free: lifts the whole constructed record into a widget
$ supply                    # free: the constant dynamic of the function supply
@ ((!a, $b))                # free, before a bracket: a widget returning a pair
@((!a, $b))                 # same tokens: `((` is not a name character, so the sigil is free
$ Unavailable               # free: the constant dynamic of a unit value
```

The attached and free forms mean the same thing when both are possible: `$"Increment"` and `$ "Increment"` are the same expression. The attachment rule exists for the token stream, not for the semantics, and it is chosen so that the common spellings are the attached ones and the free form reads as an operator with a space after it. A sigil followed by whitespace is always free. A sigil followed by `(`, `[`, `{` or a double bracket is free and applies to the bracketed expression, which is the same thing as if it were attached, so `@((!a, $b))` and `@ ((!a, $b))` have identical token streams: a free sigil then a bracket.

Suppose instead the sigil were always attached to whatever token followed it. Then `@ ChecklistUpdate ix, !checked` would be `(@ChecklistUpdate) ix, !checked`: a call to a function named `@ChecklistUpdate`, which does not exist. The free form lets a sigil act as `return` on a

whole expression, and the space says so. The alternative of a separate keyword for lifting, `pure x` or `return x`, would spell the same thing with a word that does not say which functor is meant; `@ x` and `$ x` say it.

## Keywords

`let type tag of fn match if else for rebuild intercept widget record choice map array unit test ignore on then not and or key ix jx kx empty`.

Reserved for the `update` blocks later in this chapter and rejected with a message saying so: `read update` and the sigil `&`.

Not keywords, though they look like them: `never`, `started`, `main`, `Switch`, `Leftmost`, `Join`, `HoldFirst`, `Element`, `Some`, `Nil`, `True`, `False`, and the optional default `Or`. These are ordinary names in the prelude, written with their sigils where they have one (`!never`, `@started`, `@main`, `!Switch`, `!Leftmost`, `$Join`, `@Element`, `@HoldFirst`). A program may shadow them, which is a bad idea the compiler does not stop. `unit` is a keyword because it is both a type and the only value of that type, and letting it be shadowed would let `@ unit` mean something else.

`key` is a keyword rather than a name because the shorthand `for` binds it without it appearing in the loop's syntax, and `ix`, `jx` and `kx` are keywords because the loops of an `update` block bind them without their appearing in its syntax (the first for the innermost loop, `jx` and `kx` for the next two levels of nesting); a keyword cannot be shadowed by an outer binding by accident. They may be used as record field names (`ChecklistUpdate ix, !checked` is a record whose first field is named `ix`; `Entry(key=k, value=v)`) because a field name is never an expression on its own. Keywords are reserved everywhere; no word is a keyword in one position and a name in another.

## Literals

**Numbers.** A decimal digit string is an integer-spelled literal; a digit string with one `.` and digits on both sides is a decimal-spelled literal. `1_000_000` is allowed, with underscores ignored. `0xFF`, `0o17` and `0b1010` are integer-spelled literals in hexadecimal, octal and binary; `1.5e3` and `2e-3` are decimal-spelled, since an exponent makes a number a `Float` the way a point does. No name starts with a digit, so none of these collides with anything. Numeric literals are polymorphic over the numeric types and take their type from inference, as the next section describes: an integer-spelled literal is a `Nat` or an `Int`, a decimal-spelled one is a `Float`. `-1` is the negation of a literal and is therefore not a `Nat`. A literal is one token; the sign is a prefix operator, told apart from subtraction by the spacing rule below.

**Text.** `"..."` with the escapes `\\ \" \n \t \{ \}` and `\u{hex}`. A text literal may not contain a raw newline; a program that wants one writes `\n`. Inside a text literal, `{expression}` interpolates an expression, and `{name}` is the common case: `"hi {name}"`, `"count: {$count}"`, `"{$!line}"`. The braces nest like any brackets: the lexer lexes their contents as ordinary tokens, including nested text literals, until the matching `}`. A `$` in text is always a literal `$`, so `"cost: $5"` and `"$count"` are plain text; a literal brace is written `\{` or `\}`. The kind of the whole literal follows lifting: a literal that interpolates a dynamic is a dynamic, so `"Count: {$count}"` and `$"Count: {$count}"` are one expression, and `$"Increment"` is a constant dynamic because there is nothing to lift over.

The lexer produces a text literal as one token with a list of parts: plain text segments and interpolation token streams. Which parts there are is fixed at lex time, and a `{` that opens an interpolation is the only character in text that the lexer treats as more than itself.

**Char.** `'c'` with the same escapes. A char literal holds exactly one code point.

**Unit.** `unit`. **Empty collection.** `[[]]` or `empty` for an empty array; `{}` for an empty map. `empty` is a keyword so that an empty array can be written without brackets where the brackets would be noise: `Fold xs, empty, F`.

**Bool.** `True` and `False` are constructors of the prelude `choice`, not literals.

## Operators and punctuation

The operator tokens, grouped by what they are for:

- Statement: `<-` bind, `=` definition, `then` hold (a keyword used as an operator: `0 then $count + !delta`).
- Event: `|` merge, `on` sample (a keyword used as an operator).
- Boolean: `not` (prefix), `and`, `or`, keywords used as operators like `on`, over `Bool`. `and` binds tighter than `or`, `not` binds tighter than `and`, and comparison binds tighter than `not`; all of them are tighter than `on` and `|`.
- Comparison: `< <= > >=`, resolved through `Ord`, and `== !=`, resolved through `Eq`.
- Arithmetic: `+ - * /`, which resolve by inference, and the suffixed spellings `+~ -~ *~ /~` for `Int` and `+. -. *. /.` for `Float`, each one token, for when inference cannot decide. `/` and `/~` return `?` on the integers and `/.` returns a plain `Float`.
- Conversion: prefix `+?`, one token, converting an `Int` to a `?Nat`.
- Types: `->` function, `==>` map, `:` ascription.
- Structure: `, . .. *` (in `Ns.*`), the brackets `( ) [ ] { }`, and the double brackets `(( ))` for tuples and `[[ ]]` for array literals, each pair written with nothing between its two characters.
- Lambda: `\` and the argument bars `|a, b|`.
- Layout: `:` as a block opener, and `;` between the statements of an inline block.

`;` lets an inline block hold more than one statement: `type Pair<a, b> = record: first: a; second: b`, `let s = if c: let t = x + 1; t * t else: 0`. The lexer emits a source `;` as the same statement separator the layout rules emit at a line's start, so the parser sees one kind of separator. A layout block never needs one, and a `;` at the end of a line is an error rather than a style, so that a program has one way to end a statement.

**Double brackets.** Brackets are matched one character at a time, as in any language, and the lexer marks each bracket with whether it touches the bracket before it. Two opens written with no space between them, whose matching closes are also written with no space between them, are one double bracket; the parser, which pairs brackets anyway, reads them so. Pairing goes outermost first.

```
((a, b))            # a tuple: the opens touch and so do their closes
((x))               # no comma inside: x itself, so redundant parentheses never change a program
((a + b) * c)       # a group inside a group: the inner ( closes after b, apart from the outer )
( (a, b) )          # spaced: single parens around `a, b`, which is not a call: an error
[[1, 2, 3]]         # an array literal
xs[ys[0]]           # two indexings: the opens are apart, though the closes touch
[[ys[0]]]           # an array literal holding one indexing
(((G b, c), d))     # a tuple; inside it, a single-paren call G(b, c)
```

Adjacency is the whole test, and it is the same test the spacing rule already applies: what touches what. One case costs something: an index whose index expression is itself a pronoun index, `xs[_[0]]`, cannot drop the `_` to `xs[[0]]`, since that is `xs` followed by an array literal. Writing the `_` is the fix, and the case is rare enough that the price is right.

`a < b` is less-than, `<=`, `>` and `>=` follow, and `a == b` and `a != b` are equality; every program compares things, and `if e.cursor == 0:` is what a reader expects to find there. The lexer tells `<`, `<-`, `<=`, `->` and `==>` apart by taking the longest token. Hold is the keyword `then`, so that `$count <= 3` means what it means in every other language and the line that defines a program's state reads as a timeline: `$count <- 0 then $count + !delta` is zero, then the sum whenever the event fires. Suppose instead hold were spelled `<=`, as the arrow suggests. Then `0 <= !e` would be a hold and `n <= !e` a lifted comparison, two different types from one spelling, told apart only by whether the left operand's type matches the event's payload; the one line a reader most needs to read aloud would be the one line that needed a type checker to read.

**Boolean operators.** You have written `!(a && b) || c` and counted parentheses, or written `or(and(a, b), c)` in a language whose booleans are functions and read the line from the inside out. `not`, `and` and `or` are words, as `<` and `on` are, so a condition reads as it is said: `if not done and $count > limit:`. They take `Bool` operands and lift like every other operation: `$a and $b` is a `$Bool`, and `not !flag` is an event. On plain operands, `and` and `or` short-circuit: when the left operand decides the answer, the right one is not evaluated. The reason is overflow, which aborts the occurrence. A guard like `a < 3_000_000 and a * a * a < limit` is only a guard if the cube is never computed for a large `a`; without short-circuit the left test would protect nothing, and the program would need a nested `if` to say what the operator says. Over dynamics and events, `and` and `or` are the same function lifted by the lifting rule below, so `$a and $b` is a dynamic that is recomputed when either changes and an `and` with one event fires when the event does; the short-circuit applies to each evaluation of the plain function inside. There are no prelude functions with these names, and the one prelude `Or` is the optional default (`Or xs[i], 0`), which takes a `?T` and a `T` and has nothing to do with `Bool`.

**The argument bars.** A `|` immediately after `\` opens an argument list, which closes at the next `|`. The lexer distinguishes the bars from the merge operator by position, not by spacing: `\|a, b| a | b` is a lambda of two arguments whose body merges them. Inside the bars only patterns appear, so there is no merge to confuse them with.

**The spacing rule.** Several tokens can be prefix or binary, and the lexer decides which from the whitespace around them: a token that can be either is *prefix* when it is preceded by whitespace (or by an open bracket, a comma, or the start of a line) and followed immediately by a non-space; it is *binary* otherwise.

```
F -1          # prefix: F applied to the Int -1
F - 1         # binary: F minus one (a type error unless F is a number)
F-1           # binary
\F .x         # prefix, with _ in scope: F applied to _.x
\F [0]        # prefix: F applied to _[0]
for xs: F .x  # prefix, in a for body: F applied to _.x
a.x           # binary: field x of a
xs[0]         # binary: index into xs
@ x           # free sigil (always prefix; a sigil is never binary)
Size $elems   # the sigil is attached to the name
```

The tokens the rule applies to are `+`, `-`, `.`, `[` and `+?`. A prefix `.` or `[` applies to the pronoun `_` wherever `_` is in scope: a lambda body, a `for` body, any construct that binds it (the lambda section below). Where no `_` is in scope, a prefix `.` or `[` is an error. A `.` that is first on a continuation line is a method-chain step, not a pronoun use; the line break tells the two apart (the paren-less call section). The sigils are not subject to it because they are never binary; `*` and `/` are never prefix. When `a -1` parses as a call and `a` turns out not to be a function, the type error says: "`a` is not a function; did you mean `a - 1`?" That diagnostic exists because the spacing rule is the one rule in the lexer that a newcomer will trip on, and the trip has to be a soft landing.

Why a spacing rule at all, rather than Haskell's approach (from memory), where `-` after an expression is always binary and a negative argument is written `F (-1)`? Because paren-less calls (below) make argument lists the most common thing in a program, and an argument list where a negative number needs parentheses while a positive one does not reads as a trap. The counter writes `-1 on !decrement | +1 on !increment`, and both literals look alike. The cost is that `F -1` and `F - 1` differ by a space, and the diagnostic above is the price paid for that cost. A second alternative, making spacing matter only for sigils and requiring parentheses around negative arguments, would make the counter's third line `(-1) on !decrement | (+1) on !increment`, which is the trap again with extra characters.

## Layout

You have read a widget block before you knew it was one. Here is the counter with braces, the way a C-family language would have it:

```
@counter {
    !inc <- @Button($"Increment");
    !dec <- @Button($"Decrement");
    $count <- 0 then $count + (-1 on !dec | +1 on !inc);
    @Label("Count: {$count}");
}
```

and here it is as Prism writes it:

```
@counter:
    !inc <- @Button $"Increment"
    !dec <- @Button $"Decrement"
    $count <- 0 then $count + (-1 on !dec | +1 on !inc)
    @Label "Count: {$count}"
```

Nothing is lost, and what is gone was never information. A widget block is a list of equations, one per line, and the thing that tells you where one equation ends and the next begins is the line; a `;` says it again, and a brace says a third time what the indentation already said. The habit of reading indentation as structure is one every programmer already has, because every program is indented whether or not its language reads the indentation, and the languages that do not read it have to lint for the case where the braces and the indentation disagree, which is the one case a reader is guaranteed to get wrong. A language in which the two cannot disagree has removed a whole class of bug from the reader rather than from the compiler, which is where the bug lived.

The real reason is the one about seeing what moves. The counter's block is four lines of definitions, and a reader is meant to take it in as a table: left column the names, right column what they are. Braces and semicolons put punctuation in the table; a `rebuild` or a `for` with a nested block puts a closing brace on a line by itself, which is a line the reader has to skip. The nested forms are where the difference is biggest: `rebuild $path:` followed by its block reads as "while the path is this, these," and the block is over when the indentation returns, which is exactly when the reader's attention does.

Rust made the same decision from the other side: an `if` body must have braces, with no brace-less form, so that there is one way to write a block and no dangling `else`. One channel, chosen in the other direction. Suppose instead the language took braces and ignored whitespace, as Rust does. The cost would be the separators on every line and the dangling closers, and the benefit would be that a formatter could reflow code and that a line could be split anywhere. Prism gets the second from rule 3 below (an indented continuation line continues the statement) and does not want the first, since a block whose lines are equations has no reason to reflow. The price of the offside rule is that it has to be specified exactly, with its ugly cases, so that two lexers agree; that specification is the rest of this section, and it is short.

Layout is the offside rule, with `:` as the block opener. Everything below is the contract. The intuition is Python's: a colon at the end of a line opens a block, the block is the lines indented under it, and the block ends when the indentation comes back. The difference from Python is in what a line *inside* a block means when it is indented more than its neighbours: it continues the statement above it, as in Haskell, rather than being an error.

1. The lexer keeps a stack of block columns, initially `[0]`. The top of the stack is the current block's column.
2. A `:` that is the last token on its line (comments ignored) opens a *layout block*. The next line that is not blank and not comment-only must be indented strictly more than the current block's column; its column is pushed and becomes the new block's column. Anything else, including end of file, is an error: "expected an indented block after `:`".
3. Within a block, a line at exactly the block's column begins a new statement; the lexer emits a statement separator before it. A line indented more than the block's column *continues* the current statement. A line indented less closes the block: the lexer emits a block close and pops, and the comparison repeats against the new top. After popping, the line's column must equal the top of the stack; an unindent that matches no open block is an error: "unindent does not match any open block".
4. A `:` with more tokens after it on the same line is an *inline block*: the rest of the line, up to an `else` keyword or the end of the line, is the block, and the block closes there. It holds one statement, or several separated by `;`. `True: 3` in a `match`, `"foo": 3` in a `map:` block, `if some_bool: 3` and `x: Float` in a `record:` block are all this form. The lexer emits the inline `:` as an ordinary colon token; the parser treats what follows as a one-statement block. If the line ends with another `:`, that one opens a layout block by rule 2, and the layout block belongs to the statement inside the inline block.
5. Inside `(`, `[`, `{`, the double brackets, and the `{...}` of a text interpolation, layout is suspended: line breaks and indentation are whitespace until the matching close bracket (the lexer still marks a `.` that begins a line, for method chains), and `:` inside brackets never opens a block of either kind; it is the key separator of a one-line map or a type ascription. A bracket opened on one line and closed on a later one leaves the stack untouched.
6. A block opened in the middle of a statement (`!events <- for $_:`, `$x <- match y:`) belongs to that statement; when the block closes, the statement ends. A following line indented more than the enclosing block's column but less than the closed block's is an error by rule 3; there is no continuation of a statement after its block.
7. `else` at the column of its `if` begins a statement that the parser attaches to the preceding `if` statement. An `else` after an inline `if` on the same line ends the inline block, by rule 4.
8. Top level is a block at column 0. A top-level definition is a statement of that block. A top-level line indented more than column 0 that is not a continuation is an error by rule 3, since nothing is open to match it.
9. Blank and comment-only lines never open, close or continue anything. A statement may contain blank lines between its continuation lines.
10. End of file closes every open block.

There is no explicit-brace form of a layout block. Programs that need one line use the inline form or brackets. The alternative, Haskell's `{ ; }` fallback (from memory), exists there so that generated code and one-liners can avoid layout; here generated code writes newlines, and one-liners have the inline form, and the saving is that no parser has to merge two block grammars.

**The ugly cases, each with its answer.**

*A statement continued by a more-indented line.* The continuation is any line indented more than the

block column; it does not have to align with anything.

```
@form:
    $edit <- Edit initial, (Len initial)
        then Apply $edit, !key
    @Label (Render $edit)
```

The second line is indented more than the block's column (4), so it continues the hold statement. The third line is at column 4, so it begins a new statement.

*Two blocks closed by one dedent.*

```
@outer:
    !x <- rebuild $a:
        !y <- @Button $"B"
        @ !y
    @Label $"done"
```

After `@ !y` the next line is at column 4. The stack is `[0, 4, 8]`; 4 is less than 8, so the `rebuild` block closes and the stack is `[0, 4]`; 4 equals the top, so the line begins a statement of the outer block. Had the line been at column 0, two closes would have been emitted before the comparison matched. Had it been at column 6, the error in rule 3 fires: 6 is less than 8 and does not equal 4.

*An inline arm followed by a layout arm in one match.*

```
match shape:
    Circle r: Pi *. r *. r
    Square s:
        let side = s
        side *. side
```

Both arms are statements of the match block at column 4. The first is an inline block; the second opens a layout block at column 8 that belongs to the arm statement. Mixing is allowed because each arm decides for itself, and the lexer sees nothing special about a match.

*`:` inside brackets.*

```
let m = {"a": 1, "b": 2}
let n = Insert m, "c", (if c: 3 else: 4)
let t = Pair (x: Nat), y
```

None of these colons opens a block. The first two are map key separators; the `if` inside parentheses uses an inline block whose end is the `else`, not the line end, and the second inline block ends at the close paren, since brackets close everything inside them. The third is a type ascription on an expression.

*A trailing comment after the opening colon.*

```
@counter:    # the counter from chapter 2
    !increment <- @Button $"Increment"
```

Comments are whitespace, so the `:` is still the last token on its line and opens a layout block.

*A block that is the whole of a statement.* `@counter:` at top level, `type Vector = record:`, `test counter, @counter:`. In the last one the block holds one statement per turn, and each turn's `=>` lines are continuations of it by rule 3.

*A bracket spanning lines inside a block.*

```
@main:
    let names = [[
        "a",
        "b"
    ]]
    @Label $"ok"
```

Inside `[[` the lines are whitespace, so the indentation of `"a"` and of `]]` is irrelevant and the statement is one statement. The next line at column 4 begins a new statement. A close bracket at column 0 inside a block is still fine; the bracket's own position is never compared to the stack.

*A `for` with a block followed by a merge.* This one is an error, on purpose:

```
@bad:
    !x <- for $m:
        @ !click
      | !other              # error: unindent does not match any open block
```

Rule 6: the statement ended when the block closed, and column 6 matches nothing. The program wanted `!x <- (for $m: ...) | !other`, which cannot be written with layout, and is written by binding the `for` to a name on its own line and merging on the next. This refusal is deliberate: a continuation after a dedent would mean the reader has to find the end of a block by looking past it.

*Nested inline forms.*

```
if c: match x:
    Some v: v
    Nil: 0
else: 1
```

`if c:` opens an inline block; its statement is `match x:` whose line-final `:` opens a layout block belonging to the match; the arms are in it; the dedent to column 0 closes it and ends the match statement, which ends the inline block; `else` at the column of `if` attaches. Legal, defined, and not recommended.

*The lambda bars across a line.* `\|a, b|` must open and close on one line. A lambda's body may continue onto later lines by the continuation rule, or be a layout block if the body is `@widget:` or `match a:` or anything else ending in `:`.

*A text literal with an interpolation containing a colon.* `"{if c: "yes" else: "no"}"` is one token; the interpolation is lexed as bracket contents, where the inline `if` ends at `else` and the second inline block ends at the `}`. The nested text literals inside are complete literals of their own.

## Paren-less calls

You have read `render(layout(items, width), theme)` and counted parentheses to learn which call `theme` belongs to, and you have read `f x y + 1` in Haskell and stopped to remember that application binds tighter than `+`. Prism's call syntax is built so that neither question comes up: which call a comma belongs to is decided by the nearest bracket around it, and where an argument ends is decided by the next comma, never by a function's arity or a precedence you have to recall.

A function is applied by writing its name and then its arguments, separated by commas: `Insert items, id, todo`. The rules, all of them:

- **A paren-less call takes the rest of its expression.** Its arguments are whole expressions separated by commas, and the last one runs to the end of the expression the call sits in. A

paren-less call inside an argument takes no commas, so commas belong to the outermost call: `F a, G b, c` is `F(a, G(b), c)`, and `F a + 1, b` is `F(a + 1, b)`.

- **A lambda takes the rest of its expression too, so the call heading its body takes the commas after it.** `Fold changes, items, \|acc, id, c\| Step acc, id, c` gives `Step` three arguments and `Fold` three; a lambda that is not the last argument takes parentheses.
- **Single parentheses group, and a group whose head is a call is that call with all its commas.** `F a, (G b, c)` gives `G` two arguments; `(Insert $items, $next, todo) on !add` is one three-argument call, sampled at `!add`.
- **Double parentheses make a tuple.** `((a, b))` is a pair, and the commas inside belong to the tuple: `((G b, c))` is the pair of `G b` and `c`. A multi-argument call inside a tuple takes its own single parentheses: `(((G b, c), d))`.
- **Double brackets make an array literal.** `[[F a, G b]]` is a two-element array. Single brackets are always indexing: `xs[i]`.
- **Braces make a map**, and the commas inside separate entries: `{k: F a, j: b}`.

```
Insert items, id, (Todo label, False)            # Insert gets three arguments; Todo gets two
intercept @child, !down, $ supply                # three arguments; the free $ takes supply
intercept @_, (!Union !kid_ins, !fwd_ins)        # !Union gets two arguments
Concat [[(Prefix e.text, e.cursor), ToText c]]    # two array elements; the first is a two-argument call
((Len xs, ys))                                    # a pair: Len xs and ys
(Len xs, ys)                                      # a call: Len with two arguments
Describe $_, $starred                             # two arguments, lifted over the dynamics
@Label (Render $edit)                             # one argument, parenthesised for clarity, not necessity
Fold changes, items, \|acc, id, c| Step acc, id, c   # the trailing lambda's body takes the rest
```

A reader therefore never needs a function's arity to parse a line, and the parser never needs it either: every comma has a bracket or an outermost call to belong to, and which one is visible in the characters.

**The case to watch.** Because a paren-less call takes the rest of its expression, `Size $elems + 1` is `Size($elems + 1)`, and the program that wanted one more than the size writes `(Size $elems) + 1`. This is the one place the rule surprises, and it is kept on purpose. Arguments are whole expressions, so a reader looking at a call never weighs operator precedence against application; the only question is where the next comma is. Suppose instead application bound tighter than every operator, as in Haskell (from memory). Then `Size $elems + 1` would mean `(Size $elems) + 1`, the usual intent in arithmetic, and every argument that is itself an expression would need parentheses: `@Label ($"Count: {$count}")`, `Len ($name)`, `@Button ($"OK")` on the first line anyone writes, and `F (a + 1), b` where the rule above writes `F a + 1, b`. This language's programs are mostly calls with structured arguments and rarely arithmetic over a call's result, and the rule follows the programs. When arithmetic over a result is wanted, `let` it or parenthesise it; a lint flags a paren-less call on the left of an arithmetic operator, which catches the case cheaply.

**Why the doubled brackets.** Tuples and array literals get the doubled brackets so that the single ones mean one thing each: single parentheses group (and a group headed by a call is the call), single brackets index. Suppose tuples were written with single parentheses, as in Python and Rust. Then `(Len xs, ys)` is either the pair of `Len xs` and `ys` or `Len` called with two arguments, and only `Len`'s arity says which; a parser that wants to stay out of the type checker has to pick one meaning and give the other a second spelling anyway. Suppose arrays were written with single brackets. Then `xs[i]` and `xs [i]` differ by one space, between "index `xs`" and "call `xs` with a one-element array", and inside a lambda body `\F [0]` is either `F` applied to `[0]` or `F` applied to the pronoun's first element. With the doubled forms, the parse of every bracket is readable from the bracket alone, and the two literal forms follow one rule, so there is one thing to learn rather than two. The cost is two characters per literal. In these programs tuples are mostly bundle patterns and short pairs, and most collections

are maps built by functions rather than written out, so the cost lands on the rarer forms and the common ones, calls, grouping and indexing, stay single.

**Method chains.** The method form `x.F y, z` is `F x, y, z`: the receiver becomes the first argument. There is no distinction between functions and methods; the form exists because `slots.Filter IsGui` reads better than `Filter slots, IsGui` when steps follow one another. Chains associate left, so `x.F.G` is `G (F x)`, and the receiver of a `.F` written against an expression is that expression up to the `.`: `a.b.F c` is `F (a.b), c`.

A step that takes arguments starts its own line. A `.Method` that begins a continuation line closes the call above it and takes it as its receiver:

```
let done = items.Filter IsDone
    .MapValues Label            # MapValues (Filter items, IsDone), Label
    .Values                    # Values (MapValues ..., Label)
```

The receiver runs back to the start of the statement's right-hand side, after its `=`, `<-` or `then`; inside brackets, where lines are whitespace, it runs back to the open bracket or the nearest comma at that bracket's depth. On one line, `x.F y` is method form and a spaced `.F` is not a chain step: `items.Filter IsDone .MapValues Label` is an error whose message says to put the step on its own line. Indentation decides because the alternative rules each give the parse to something a reader cannot see. Suppose a spaced `.F` on one line closed the call before it: then `F a .G b` and `F a.G b` would be different programs, distinguished by a space in the middle of a line, and the paren-less call would end somewhere other than at a comma or the end of the expression. A line break is the one boundary that is obvious from across the room, and one step per line is how a chain is written in every language that has them.

## Parse corpus

The programs on which the two parsers must produce identical trees, sketched the way the layout corpus sketches token streams. The sketch writes every application with explicit parentheses, `F(a, b)`; a tuple as `tuple(a, b)`; an array literal as `array(a, b)`; indexing as `index(xs, i)`; field access as `field(x, f)`; a lambda as `λ_. body`; and keeps operators infix with explicit grouping. Both implementations keep it as a golden test beside the token streams.

**1. Nested calls, with and without parentheses.**

```
let a = F x, G y, z
let b = F x, (G y, z)
let c = F G H x
```

```
a = F(x, G(y), z)
b = F(x, G(y, z))
c = F(G(H(x)))
```

`G y` in the first line takes no commas because it is not the outermost call. A chain of one-argument calls needs no parentheses at all.

**2. Tuple against call.**

```
let t = ((G y, z))
let u = (G y, z)
let v = (((G y, z), w))
```

```
t = tuple(G(y), z)
u = G(y, z)
v = tuple(G(y, z), w)
```

**3. Array literal against index.**

```
let xs = [[1, 2, 3]]
let x = xs[0]
let y = grid[r][c]
let z = [[xs[0], ys[i]]]
```

```
xs = array(1, 2, 3)
x = index(xs, 0)
y = index(index(grid, r), c)
z = array(index(xs, 0), index(ys, i))
```

**4. A method chain on continuation lines.**

```
let names = todos.Filter IsDone
    .MapValues Label
    .Values
```

```
names = Values(MapValues(Filter(todos, IsDone), Label))
```

**5. The case to watch.**

```
let a = Size $elems + 1
let b = (Size $elems) + 1
let c = 1 + Size $elems
```

```
a = Size($elems + 1)
b = (Size($elems)) + 1
c = 1 + Size($elems)
```

Nothing precedes a call's name that the call could absorb, so `1 + Size $elems` adds one to the size.

**6. A higher-order function type.**

```
let Map<a, b>: fn (fn a -> b), [a] -> [b]
let Curry<a, b, c>: fn (fn ((a, b)) -> c) -> (fn a -> (fn b -> c))
```

```
Map : fn(fn(a) -> b, [a]) -> [b]
Curry : fn(fn(tuple(a, b)) -> c) -> fn(a) -> fn(b) -> c
```

The parentheses around a function-typed argument or result are required, so the commas of an inner `fn` never join the outer one's argument list.

**7. A trailing lambda with the pronoun shorthand.**

```
let open = Keep $todos, \not .done
let firsts = for rows: Map _, \[0]
```

```
open = Keep($todos, λ_. not(field(_, done)))
firsts = for(rows, Map(_, λ_. index(_, 0)))
```

The lambda is the last argument and its body takes the rest of the line. Inside the `\` body a prefix `.done` is `_.done` and a prefix `[0]` is `_[0]`; the `_` of the `for` body is the argument `Map` receives, and the lambda's own `_` shadows it inside the lambda.

**8. A lambda body that is a call, and a bare projection.**

```
let total = Fold changes, 0, \|acc, c| Add acc, c.amount
let ids = Map todos, \.id
```

```
total = Fold(changes, 0, λ(acc, c). Add(acc, field(c, amount)))
ids = Map(todos, λ_. field(_, id))
```

`Add` takes both commas because the lambda body takes the rest of the expression; `Fold` still has three arguments.

### 9. Redundant parentheses, a free sigil on a call, and an inline block.

```
let a = ((x))
let b = @ F x, y
let c = F x + 1, y
type Pair<a, b> = record: first: a; second: b
```

```
a = x
b = @(F(x, y))
c = F(x + 1, y)
Pair<a, b> = record(first: a, second: b)
```

`((x))` has no comma and is `x`. The free `@` wraps the whole call, and the `;` separates the two statements of the inline `record` block.

### 10. A spaced `.Method` on one line is an error.

```
let bad = items.Filter IsDone .MapValues Label
```

```
ERROR(a .Method step must start its own line)
```

The program writes the step on its own line, as in case 4.

### 11. Boolean operators against comparison, `on` and arithmetic.

```
let a = x or y and z
let b = not done and $n > limit
let c = $done or $n > limit on !click
let d = not a == b
let e = not $n + 1 > limit
let f = not done and ready
```

```
a = x or (y and z)
b = (not done) and ($n > limit)
c = ($done or ($n > limit)) on !click
d = not (a == b)
e = not (($n + 1) > limit)
f = (not done) and ready
```

`and` binds tighter than `or`, both bind looser than comparison and tighter than `on`, and `not` sits between `and` and comparison, so its operand is a whole comparison: cases `d` and `e` negate the comparison, and case `f` negates `done` alone because `and` is looser than `not`.

### 12. Guards and array patterns.

```
let a = match items[id]:
    Some t if t.done: 1
    Some t: 2
    Nil: 3
let b = match xs:
    []: 0
    [head, ..rest]: head
```

```
a = match(index(items, id);
      arm(Some(t) if field(t, done) => 1),
      arm(Some(t) => 2),
      arm(Nil => 3))
b = match(xs;
      arm(array() => 0),
```

```
        arm(array(head, ..rest) => head))
```

The guard is part of the arm and runs from `if` to the colon. `a` is exhaustive by its second and third arms alone; `b` is exhaustive by its two arms, the empty array and the non-empty one.

**13. `for` with named binders.**

```
let b = for k, item <- xs: F k, item
let c = for xs: F key, _
```

```
  b = for(k, item <- xs, F(k, item))
  c = for(key, _ <- xs, F(key, _))
```

`c` is the shorthand and the same tree as `for key, _ <- xs:`, so the two spellings are one program.

### What a conforming lexer must get right

The layout corpus at the end of this chapter lists the programs on which both implementations must produce identical token streams. The shortest statement of the contract is this: every decision in this section is made by the lexer or by the parser's treatment of colon tokens, `else`, `;` and bracket adjacency, never by the type checker; the token stream for a program is a pure function of its characters; and the only context the lexer keeps is the column stack, the bracket stack, and whether the previous character was whitespace. Two marks on tokens come from that context: each bracket says whether it touches the bracket before it, which is all the parser needs to pair double brackets, and a `.` that is the first token on its line is marked as a chain step, since the line break that makes it one is otherwise gone from the stream.

## Values and types

The value layer is meant to be boring, and that is a decision. Everything interesting about Prism is in the functors; the plain values under them are the ones any typed functional language has, so that nothing here competes with time for your attention. Two things in this section are not boring, and both are goals. Absence is in the type, `?` and the cases of every element's state, because you have shipped the `if (user)` guard that was missing. And there is no way to mutate a plain value, anywhere, because the only state is a hold and that is the promise the whole language rests on.

A plain value is immutable, eager, and cheap to copy or alias; the program cannot tell copying from aliasing and the implementation is free to do either. Functions are the one lazy thing. Every type in the language is either a plain value or a plain value under one or more of the functors.

### Primitives

- `Nat`: unsigned, 64-bit. Indexes arrays. Has no negation.
- `Int`: signed, 64-bit.
- `Float`: IEEE double.
- `Text`: Unicode text. Indexed by character position through prelude functions, never by byte.
- `Char`: one code point.
- `unit`: the type with one value, also spelled `unit`.
- `Bool`: not a primitive. `type Bool = choice True | False` in the prelude, and `if` is `match` on it.

Overflow in `Nat` or `Int` arithmetic is a runtime error that aborts the occurrence with the error kind `Overflow`; the error chapter says what that means, and the prelude's `Checked.*` functions (which return `?`) and `Wrap.*` functions (which wrap around) are the spellings for a program that wants a

different answer. Division on `Nat` and `Int` returns an optional: `a / b` is `?Nat` for `Nat` operands and `?Int` for `Int` operands, `Nil` when `b` is zero. Division on `Float` follows IEEE and never fails: `1.0 / 0.0` is infinity, `0.0 / 0.0` is NaN, and `a / b` is a `Float`. The two rules differ because the two failures differ. An integer has no value to answer with when the divisor is zero, and an integer division that panics is the one that bites in a running program, so the `?` marks the one place a divide-by-zero can be and the type makes the program say what it does then. A `Float` has infinities and NaN to answer with, and they are the answers every numeric library and every GPU already gives; wrapping each `Float` division in a `?` would make a drawing program match on `Some` at every scale factor to guard against a case the representation already covers.

## Numbers by inference, and the three traits

Three things are overloaded in Prism and nothing else is: equality, ordering, and arithmetic. They are traits, in the Rust sense: a named set of operations that a type implements, with the operation resolved by inference from the operand types. The three are `Eq`, `Ord` and `Num`.

- `Eq`: every plain value whose type contains no function and no functor value. Structural: two records are equal when their fields are, two maps when their entries are, two values of an extensible variant when their tags and plain fields are (an element's `requests` event is excluded).
- `Ord`: primitives (`Float` included, by the total order described below), text, chars, tuples and arrays of ordered types (lexicographic), a `choice` whose payloads are ordered (declaration order of the alternatives, then payload; a closed `choice` of bare alternatives is ordered by declaration order alone, so `type Size = choice Small | Medium | Large` has `Small < Large`), a record whose fields are all ordered (lexicographic in field declaration order, like Rust's derived `Ord`), and an extensible variant whose plain fields are ordered (tag name, then plain fields in declaration order, `requests` excluded). Map keys must be `Ord`, so a record or an element can key a sorted map.
- `Num`: `Nat`, `Int` and `Float`, with `+ - * /`. Prefix `-` is `Int` and `Float` only. `/` is `?Nat` and `?Int` on the integers and plain `Float` on `Float`.

Values that hold a function, an event, a dynamic or a blueprint have none of these. Equality on events would have to mean something, and the only thing it could mean is identity, which the language does not give events; equality on functions is undecidable; equality on blueprints would be equality on descriptions, which is close to structural but would make two different closures compare unequal in a way the program cannot predict. Ruling them out keeps `Eq` meaning "the same value" and nothing subtler.

Arithmetic is written `+ - * /` and the type of each operation is found by inference from its operands:

```
let !delta = -1 on !decrement | +1 on !increment     # -1 is signed, so !delta : !Int
$count <- 0 then $count + !delta                       # Int addition; the 0 is an Int because the hold is
let area = w *. h                                     # Float, said directly
let third = total / 3                                 # ?Nat, if total is a Nat
let half = width / 2.0                                # Float, if width is a Float; never Nil
```

Numeric literals are polymorphic over `Nat`, `Int` and `Float` and take their type the same way. An integer-spelled literal is a `Nat` or an `Int`; a decimal-spelled literal is a `Float`; the negation of a literal is not a `Nat`. So `-1` is an `Int` by its spelling alone, `1.5` is a `Float`, and `0` is whatever the context says. When nothing constrains an integer-spelled literal, it is an `Int`: `$n <- 0 then $n + 1 on !tick` is an `Int` counter with no annotation anywhere. The suffixed spellings `+~ -~ *~ /~` (`Int`) and `+. -. *. /.` (`Float`) pin the type at the operator when a program wants to say it there: `0 then $count +~ !delta` says `Int` outright. `Nat` has no suffixed spelling; a program that needs to pin a `Nat` annotates it, `let n: Nat = 0`. Prefix `+` is the identity on any numeric type and exists so that `+1` and `-1` can sit next to

each other in a merge. `+?` converts an `Int` to a `?Nat`, and `Abs` to a `Nat`.

Why inference rather than one operator per type, as OCaml does (from memory)? A program full of `+~` reads as a program about its own numeric representation, and the counter, the first program anyone writes, would have one on its most important line. Why `Int` when nothing decides? An integer literal that nothing pins is almost always a count, an offset or a step, and `Int` is the type under which all three behave: it goes below zero without an overflow abort, and it never turns `n / 2` into a fraction the way a `Float` default would. Suppose instead an unpinned literal were an error. Then the counter without its `!delta` line, `$n <- 0 then $n + 1 on !tick`, would be rejected, and the first program anyone writes would end in an annotation it does not need. The suffixed forms exist for the program that wants the type stated at the operator rather than in an annotation three lines away.

Equality and ordering in use look like this:

```
if e.cursor == 0: e else: Edit e.text, e.cursor - 1
let sorted = Sort names                    # Text is Ord
let by_id = Insert items, id, todo         # Nat is Ord, so Nat ==> Todo is a map
let same = slot_a.element == slot_b.element    # structural, on an extensible variant
let later = a.at > b.at                    # Ord, through the > operator
```

`Eq` on `Float` is IEEE equality, so a NaN is not equal to itself and `-0.0 == 0.0`. `Ord` on `Float` is the IEEE totalOrder: negative NaN, then negative infinity, the negative numbers, `-0.0`, `0.0`, the positive numbers, positive infinity, positive NaN. `< <= > >=`, `Sort` and map keys all use it, so a sort of floats terminates, a map with `Float` keys has a defined order and can hold a NaN key, and `-0.0 < 0.0` is `True` while `-0.0 == 0.0` is also `True`. The two relations disagree only on zeros and NaN, and a total order that agreed with `==` there would have to call a NaN equal to itself; a program that wants something else wraps the float in a newtype and compares through a function. Equality on `Text` is by code points, not by any normalisation; two texts that look the same and are encoded differently are different, and a program that cares normalises first. These are the dull answers, chosen because the alternatives are a choice of locale or a choice of normalisation form, and either one is a dependency on something outside the program.

Generic code may be bounded by a trait, `let Max<a: Ord>: fn a, a -> a`, and is still erased: the trait implementation is passed where the bound is, not looked up at runtime. It is not a general trait system. There are no user-defined traits, no associated types, no instances declared away from the type. The checker is written so that those can be added, and the chapter on the compiler says how; for this specification the three are the whole of it, and they are enough for sorting, map keys, element identity, and arithmetic that reads as arithmetic.

## Aggregates

- **Arrays** `[T]`: homogeneous, written `[[a, b, c]]`, `[[]]` or `empty`, or as an `array:` block with one element per statement. Indexed by `Nat` with `a[i]`, which returns `?T`. `Len`, `Append`, `Concat` and friends are prelude functions.
- **Maps** `K ==> V`: sorted by key, and the key type must be `Ord`. Written `{k: v, ...}` on one line, `{}` when empty, or as a `map:` block with one `k: v` per statement. `m[k]` returns `?V`. Iteration order is key order, always.
- **Tuples** `((A, B))`: heterogeneous, comma-separated, in double parentheses as values, types and patterns alike. A one-tuple is `((a,))`. The empty tuple is `unit`. Arrays are written in double brackets as values and as patterns alike (the `match` section gives the array patterns).

```
let some_map = map:
    "foo": 3
    "bar": 4
let other_map = {"foo": 3, "bar": 4}
let xs = array:
    First pair
    Second pair
```

This language prefers maps wherever identity matters. A map's key is an identity: in a `for`, it decides which block survives a change, which element path stays stable, and which hold keeps its state. An array has only positions, and a position is not an identity: insert at the front and every later item is "a different item" to anything keyed on position. The todo list in the worked examples allocates ids and keeps `Nat ==> Todo` for exactly this reason. Arrays are for positional data: the characters of a line, the rows of a matrix, the arguments of a program.

Maps are sorted rather than hashed because a sorted map has a defined iteration order and a hashed one has an implementation-defined one, and the element list a World reads is "in tree order" only because the paths that key it are ordered. A program whose output depended on hash order would be a program whose transcript changed with the runtime, which is what the determinism goal forbids.

Indexing returns `?T` rather than failing. You have written the bounds check, and you have forgotten it once. With `?` on the index the forgetting is a type error, and the cost, a `match` or an `Or` default at each index, is the cost of saying what happens when the index is bad, which the program had to decide anyway.

## Declared types

```
type Vector = record:
    x: Float
    y: Float

type Bool = choice True | False

type Shape = choice:
    Circle Float
    Square Float

type Meters = Meters Float

type Click = click

type TextInput = record:
    !changed: Text
    $value: Text
```

**Records.** A `record` names its fields and their types. The type name is also its constructor, callable three ways: positionally, `Vector 1.0, 2.0`; with named arguments in parentheses, `Vector(x=1.0, y=2.0)`, in any order, all of them given; or as a block of `field: value` statements:

```
let origin = Vector:
    x: 0.0
    y: 0.0
```

Fields are read with `.name`. A field holding a functor value is declared and read with its sigil: `.!changed`, `.$value`. The sigil is on the field name rather than only on the type so that `input.!changed`

reads as an event at the point of use, the same as any other name; a field `changed: !Text` would be a name without a sigil denoting an event, which is the one thing the naming rule forbids.

Records are the one place a widget returns several things at once, and the `TextInput` above is the pattern: a plain record with sigiled fields is a bundle, and a widget `@TextInput: fn Text -> @TextInput` returns one. The parent binds it with a record pattern, `TextInput(!changed, $value) <- @TextInput ""`, or keeps the record and reads the fields.

**Choices.** A `choice` lists alternatives, inline with `|` or one per statement in a block. An alternative with a payload declares a newtype on the spot: after `type Shape = choice: Circle Float ...`, `Circle` is both a type and a one-argument constructor, and `Circle 2.0` is a `Shape`. An alternative without a payload is a unit type (below). Two choices may not declare the same alternative name in one scope; `Circle` means one thing.

**Newtypes.** `type Meters = Meters Float` declares a type and a one-argument constructor of the same name, unwrapped with `.value`. `Meters 3.0` is not a `Float` and `(Meters 3.0) +. 1.0` is a type error, which is what a newtype is for. Newtypes over tuples make lightweight records, `type Pair = Pair ((Nat, Nat))`, but records are preferred because fields have names.

**Unit types.** `type Click = click` declares a type with exactly one value, spelled in lowercase. A unit type is the type of a response that has no data: a click, a tick, an end-of-input. The lowercase value is a plain name and follows the value naming rule; the uppercase type follows the type rule. A `choice` alternative with no payload, `Stopped` in `choice: Stopped | Running Instant`, is the same construction, with the uppercase name serving as both the alternative and its value, because inside a choice the alternative name is the thing you match on.

**Generics.** Generic types and functions take parameters in angle brackets: `type Pair<a, b> = record: first: a; second: b`, `let Swap<a, b>: fn Pair<a, b> -> Pair<b, a>`. A lowercase name in a type is a type variable. In a `let` annotation, free type variables are quantified implicitly, `!Filter: fn !?a -> !a`, and the angle-bracket list is needed only to state a bound: `let Max<a: Ord>: fn a, a -> a`.

Generic code is erased. It runs on values whose types it does not know and never needs to, because nothing in the language inspects a value's type at runtime: there is no `typeof`, no downcast, no runtime type descriptor and no monomorphisation. A generic `Swap` is one function, run on every `Pair`. The one place where erasure needs a rule of its own is extensible variants, below, where the rule is that a tag's fields have concrete types so that a value built in generic code never needs a type it does not have.

## Optionals

`?T` is `Nil` or `Some x`. It is a plain-value type with a sigil, not a time functor: a `?Nat` has no relation to occurrences. It is a sigil rather than a type constructor spelled `Option` for the same reason the others are: so that a name says what it is. `?user` reads as "maybe a user" at the point of use, and a function that returns one says so in its name.

`?` lifts, at the plain layer. A plain operation applied to a `?T` where it wants a `T` gives a `?` result, `Nil` if any operand was `Nil` and `Some` of the operation otherwise, which is the applicative `Maybe` you know. So `a / b + 1` is `?Nat` when `a` and `b` are `Nat`, `Len ?name` is `?Nat`, and `?x == 3` is `?Bool`. `match` is the eliminator: the only way to get from `?T` back to `T` is a `match` with a `Nil` arm, or a prelude function that is one, `Or ?x, default`.

The worry about lifting `?` is the integer division case: `a / b + 1` quietly becomes optional, and the

reader of the line might not notice. Look at where the result goes. It is bound to a name, and that name must be `?total`, because the sigil is part of the name and checked against the type; it is passed to a function wanting a `Nat`, and the function gets a `?Nat` instead, so that application lifts too and its result is optional, and so on until some binding is forced to show the `?` or some `match` eliminates it. The `?` cannot be lost; it can only move outward, and it stops at the first name. The alternative, no lifting, so that `a / b + 1` is a type error and every optional needs a `match` at the site of its first use, puts the `match` where the division is rather than where the decision about absence is made, and in the todo list that is the difference between one `match` in `Describe` and five in the arithmetic above it. Lifting `?` is the same decision as lifting `!` and `$`: the structure is in the names, so the operations can be plain.

Lifting over `?` happens at the plain layer, which means *inside* the event functor, the dynamic functor and the widget monad. `$a / $b` is `$?Nat`: a dynamic whose value is sometimes `Nil`. `?$x + 1` does not lift and is a type error: an optional dynamic is a value you match on, once, to find out whether there is a dynamic, and only then can you lift over the dynamic. The sigil order says which it is, and the lifting rule in the next section states the precedence between `?` and the time functors in one line.

## Extensible variants

```
type Element = choice ..

tag Ui.Button of Element:
    label: Text
    responds: Click

tag Fs.Contents of Element:
    path: Text
    state: Contents
```

An extensible variant is a `choice` whose alternatives are declared across modules rather than in one place: `type Element = choice ..` opens it, and each `tag` declaration adds an alternative. A tag's name is qualified, `Ui.Button`, and the namespace is the module's way of saying whose tag it is. A tag declaration states its fields with their types, and, for the one extensible variant the language relies on, up to three parts that say what the element does: `requests`, `responds` and `state`. The parts are specified in the elements chapter, where they mean something; here the variant is a type like any other.

The rules:

- **Field types are concrete.** A tag declaration may not mention a type variable. The reason is erasure: generic code never knows the type of a value it holds, and a tag whose field could be "whatever `a` is" would need a runtime type descriptor to match on. With concrete fields, every value of an extensible variant has fully known field types where it is built, and generic code cannot build one except at a type it knows. The cost: a generic library cannot wrap its own type parameter in an element. It takes elements already built, or is written at a concrete type. A generic function that tries, `let Wrap: fn a -> Element = \Test.Box(value=_)`, is rejected with a message naming the field and the type variable.
- **Construction** is the record forms: `Ui.Button(label="OK")` with named arguments, `Ui.Button "OK"` positionally, or a block. A tag with no fields is bare: `Clock`.
- **Matching** is by tag, with the fields bound by name or position as for a record: `Ui.Button(label=l)` binds `l : Text`. There is no matching on field *types*, because every field's type is fixed by its tag; what you match on is which tag, and the fields follow.

- `Ns.*` in a pattern matches any tag declared under that namespace: `Ui.*(..)` is every element of the GUI library. The `*` is the pattern's and appears nowhere else; a namespace alone, `Ui`, is not a pattern, so that adding a tag `Ui.Window` to a library cannot change which arm an existing `Ui` match takes. Nested namespaces match by prefix: `Ui.*` includes `Ui.Text.Input` if someone declares one.
- **The catch-all rule.** A `match` on an extensible variant must end in a catch-all arm, `ignore: ...` or a bare name, because the set of alternatives is open and no match can be exhaustive by listing. In an interceptor the catch-all is the forwarding case, and the rule makes forgetting to forward a compile error rather than a silently dropped element.
- **Equality** is structural, tag then plain fields, with the `requests` event excluded, and is `Eq` when every plain field is. **Ordering** is by tag name, then plain fields in declaration order, and is `Ord` when every plain field is, so elements can key a sorted map and a list of them sorts the same way under every implementation. State identity across the World boundary uses this equality, so two `Fs.Contents(path="/etc/motd")` values are one state wherever they are built.

Suppose instead that elements were ordinary records with a tag field, `Element(tag="Ui.Button", fields=...)`, and matching compared tag strings. Then a misspelled tag would match nothing and fall to the catch-all, silently, and a field's type would be whatever the record said that time. Tags as declared alternatives of a declared type get the checker involved: an undeclared tag is an error at the construction, a field of the wrong type is an error there too, and a `match` arm for a tag that does not exist is an error rather than dead code.

## Type annotations and inference

Types are inferred by Hindley–Milner over the value layer, with the functor layer read off the sigils. Inside a definition, nothing needs an annotation: `let !delta = -1 on !decrement | +1 on !increment` is checked from its parts. Top-level definitions take a full annotation:

```
let Magnitude: fn Vector -> Float = \Sqrt (_.x *. _.x +. _.y *. _.y)
@Button: fn $Text -> @!Click = \@widget:
    !!clicks <- rebuild $_:
        @Element Ui.Button(label=_)
    @ !Switch !!clicks
```

Function types are written `fn A, B -> C`, with the arguments comma-separated the way a call is; a function takes at least one argument, and a function of nothing takes `unit`. A function-typed argument or result is parenthesised, `fn (fn A -> B), [A] -> [B]` and `fn A -> (fn B -> C)`, so the commas and arrow of the inner type never run into the outer one's. A function of one pair is `fn ((A, B)) -> C`, written with the tuple's double parentheses, and differs from `fn A, B -> C` in the way `F ((a, b))` differs from `F a, b`. The `intercept` signature reads with all three rules at once: `fn @a, !(Path ==> Response), $(fn Element -> @$State) -> @((a, $(Path ==> Slot)))`, a function of three arguments, the third a dynamic of functions, returning a blueprint whose result is a pair. The annotation requirement at top level keeps every error local to one definition, since the checker never has to guess a signature from uses in another module, and it makes the typed form of a module readable at its boundary. It is kept rather than relaxed because the cost is one line per definition, and the line is the one a reader of the module wants first. If experience shows that small helper definitions make the cost real, the relaxation is to infer unannotated top-level `let`s that are used only within their module.

Local annotations are allowed anywhere a pattern is: `let n: Nat = 0`, `\|e: Element| ...`, `$x: $Int <- 0 then ...`. On a sigiled name the annotation gives the plain type under the sigil or the full type

with it; both spellings are accepted and mean the same thing.

Inference here splits into a value layer and a functor layer. Hindley–Milner runs on plain types and stays principal: the type of `Insert items, id, todo` is found by unification over `k ==> v`, `k`, `v`, exactly as it would be in ML. The functor layer is not unified at all; it is *read*. `$items` is a dynamic because its name says so, and the application `Insert $items, $next, todo` is a dynamic because the operands say so and the lifting rule says what a plain function over dynamics is. No unification variable ever stands for "some functor"; the sigils have already answered the question. That is why a value's kind is readable from its spelling and why the lifting rule can be a table.

# Expressions

The sigils buy lifting. You have written `zipWith (+) counts deltas` and `fmap render model` and `combineLatest([a$, b$]).pipe(map(([a, b]) => a + b))` (RxJS, from memory), and in each case the interesting part was the `+`, which was buried under a word that said how to apply it. Because every operand in Prism says what kind it is, the compiler can apply a plain function across events and dynamics without being told, and the common case, `$count + !delta`, is written as arithmetic. The goal is that the program reads as what it computes and the time structure is inferred from the names, never the other way round: nothing in this section requires inference to decide whether something is an event. The second goal is one way to do each thing, so that two readers reach one parse; the precedence and application rules exist for that and nothing else.

## Lifting

Every operation and every function application is lifted over the event functor, the dynamic functor and the widget monad. The rule is read off the operands' outermost sigils, not inferred:

- **All operands plain:** the result is plain.
- **Some dynamics, no event:** the result is a dynamic, recomputed whenever any operand changes, in the occurrence of the change. It is prompt: there is no occurrence in which an operand has changed and the result has not.
- **Some dynamics and exactly one event:** the result is an event that fires when the event fires, with the value of the operation applied to the event's value and the dynamics' *old* values. Each dynamic is read through a tag, which is the one window onto "before".
- **Two or more events in one operation:** a compile error naming both and pointing at `|` and `!Leftmost`.
- **A blueprint operand:** not lifted by this rule. Widgets are built, not computed over. What happens to a widget-returning application with a dynamic argument is below.
- **An optional operand** where a plain one is wanted: lifted at the plain layer, giving `?` of the result, as the previous section describes. When an operand has both, `$?x`, the time functor is outermost and lifts first; `?$x` does not lift and must be matched.

Small examples, each the whole of what it says:

```
1 + x                    # plain
1 + $x                   # $Nat (or $Int): changes when $x does
$a + $b                  # a dynamic; changes when either does, once per occurrence
$count + !delta          # an event: fires with !delta, reads $count before the occurrence
!a + !b                  # error: two events in one operation
!a | !b                  # the merge: the right way to say "either"
Len $name                 # $Nat: a plain function over a dynamic
IsClick !in              # !Bool: a plain predicate over an event
```

```
Keep IsGui, $kids          # a two-argument plain function with one dynamic argument: a dynamic
(Insert $items, $next, todo) on !add    # an event; both dynamics read through tags
Describe $_, $starred     # a dynamic text, inside a for block
Append $log, !line         # an event of arrays
a / b + 1                 # ?Nat: the ? from division lifts through the +
$a / $b                   # $?Nat
?x == 3                   # ?Bool
$count > limit           # $Bool: a comparison, lifted like any operation
"Count: {$count}"         # $Text
"Got {!line}"            # !Text
$"Increment"              # a constant dynamic: nothing to lift over
v on !e                   # an event with v's value when !e fires
$v on !e                  # an event with $v's old value when !e fires
```

`on` has no semantics of its own. It is a binary operator that returns its left operand, and the rule above makes `v on !e` an event that fires with `e` and has the value `v`, and `$v on !e` an event whose value is the tag read of `$v`. Reflex spells that one operator plus the lifting rule as `tag`, `tagPromptlyDyn`, `attachWith`, and (from memory) `attach` and `<@`: the dynamics are always read as they stood before, so there is one `on` and no choice to make.

The two-events error is the rule most worth defending, because the alternative looks friendly. Suppose `!a + !b` meant "fire when either fires, using the other's most recent value". Then every event would need a most-recent value, which is a hold, which is state nobody declared, with an initial value nobody chose. Suppose instead it meant "fire when both fire in one occurrence". That is a real operation, and it is spelled by hand: `!a` and `!b` both fire in some occurrence exactly when both sides of a merge of them fire, and the program that wants that writes it with `|` and a tuple, or holds one and samples it at the other. The error exists because there are three things `!a + !b` could mean and the program has to say which.

The tag read in the third case is the whole of the language's mutation policy in one rule. `$count + !delta` reads the count *before* the click. That turns the counter's `$count <- 0 then $count + !delta` from an equation into a definition: the cycle passes through a read of the old value. It is also why there is no "which version of `$count` did the handler see": at an event there is one version, the one before. If you have debugged a Reflex program where `tagPromptlyDyn` and `tag . current` were both in use, you know which afternoon this prevents.

**Lifting never crosses a sigil boundary.** A function on `A` lifts over `$A` but not over the inner layer of `$$A`; `$$a + 1` is a type error, and `$Join $$a + 1` is what the program meant. Likewise a function on `!A` does not lift over `!!A`. Flattening is `!Switch` and `$Join`, in the switching chapter, and it has laws of its own that a lifted `+` could not keep.

**Lifting over `@`.** A blueprint operand is not computed over, but a widget-returning function applied to a dynamic where it wanted a plain value is a common thing to write, and it has one meaning. `@Element Ui.Button(label=$l)` applies `@Element` to a dynamic, and by the rule above the result is `$@!Click`: a dynamic of blueprints, one per value of `$l`. Binding a dynamic of blueprints with `<-` lifts to `rebuild`:

```
!click <- @Element Ui.Button(label=$l)
```

means

```
!!clicks <- rebuild $l:
    @Element Ui.Button(label=_)
let !click = !Switch !!clicks
```

The bound name is the `!Switch` of the `rebuild`'s event when the widget's result is an event; for a result that is a dynamic, it is `$Join` of `@HoldFirst` of the `rebuild`'s event, and for a plain result it is `@HoldFirst` of that event, so the bound name is a `$`; for a `unit` result, nothing is bound; for a bundle, each field is treated by its own kind. `@HoldFirst` is the prelude's answer to a hold with nothing to start from. A hold needs an initial value, and an event has none until it fires; `@HoldFirst: fn !a -> @$a` is a dynamic that holds its event's value, with the value of the first firing as its initial value, and the first firing of a `rebuild`'s event is in the occurrence of the first build, so the dynamic always has a value when anything reads it:

```
$label <- @HoldFirst !labels        # $label is the latest label, starting with the first one
```

The shorthand is used freely from here on and means exactly the expansion; a program that wants the `rebuild`'s event or a different initial value writes the expansion.

## Application

Application is juxtaposition with comma-separated arguments, under the rules of the paren-less call section in the lexical part of this chapter; this section places it among the operators. A paren-less call takes the rest of its expression, so in the precedence table it sits just above the statement forms: `F a | G b` is `F(a | G(b))`, and `1 + F a` is `1 + F(a)`, because nothing precedes a call's name that the call could absorb. Parentheses make any other grouping, and `Size $elems + 1` against `(Size $elems) + 1` is the case to watch, argued in that section.

```
@Label $"Count: {$count}"                    # one argument
Len $name                                     # one argument, lifted
!Filter (HitTest $slots, !pointer)           # the group is a two-argument call
$live on (!started | !refresh)               # on binds tighter than |, so the parentheses are needed
{!Filter (HitTest $slots, !pointer): Click}  # a one-entry map with a lifted key
```

Method form and chains follow the same section: `x.F y, z` is `F x, y, z`, `a.b.F c` is `F (a.b), c`, and a chain step that takes arguments starts its own line.

## Precedence

Loosest to tightest:

| | Operator | Reads as |
|---|---|---|
| 1 | `<-`, `let ... =` | statement level, not expressions |
| 2 | `then` | hold |
| 3 | paren-less application | to the right, to the end of the expression |
| 4 | `\|` | merge |
| 5 | `on` | sample |
| 6 | `or` | boolean or |
| 7 | `and` | boolean and |
| 8 | prefix `not` | boolean negation |
| 9 | `< <= > >= == !=` | comparison |
| 10 | `+ -` and `+~ -~ +. -.` | additive |
| 11 | `* /` and `*~ /~ *. /.` | multiplicative |
| 12 | prefix `+ - +?` and free sigils `! $ @ ?` | prefix |

| | Operator | Reads as |
|---|---|---|
| 13 | `.field`, `[index]`, `.Method`, attached sigils | postfix |

Binary operators at one level associate to the left, except `then`, which does not associate: `a then b then c` is an error. The counter's lines are the test of the table: `$count <- 0 then $count + !delta` is `$count <- (0 then ($count + !delta))`; `-1 on !decrement | +1 on !increment` is `(-1 on !decrement) | (+1 on !increment)`; `$count + 1 on !click | 0 on !reset` is `(($count + 1) on !click) | (0 on !reset)`.

`on` tighter than `|` lets a merge of samples read without parentheses, which is the form of every "what happened" event in the worked examples. `on` looser than arithmetic makes `$count + 1 on !click` mean "the count plus one, at the click" rather than "the count, plus (one at the click)", which would be a two-event error waiting to happen. Under both choices the first program anyone writes needs no parentheses, and a rule that made the counter uglier would need a better reason than any of these. Comparison sits between `on` and arithmetic, so `$count + 1 > limit on !click` compares the sum and samples the answer at the click. The boolean operators sit between `on` and comparison, so a condition reads the way it is said: `$done or $count > limit on !click` is `($done or ($count > limit)) on !click`, `a < b and c < d` is `(a < b) and (c < d)`, and `x or y and z` is `x or (y and z)`. Sampling still wraps the whole condition without parentheses, which is what `on` being looser than every value operator buys. `not` sits between `and` and comparison, so it negates a whole comparison: `not a == b` is `not (a == b)`, `not $n + 1 > limit` is `not (($n + 1) > limit)`, and `not done and ready` is `(not done) and ready`, because `and` is looser. The operand of `not` is everything tighter than it, which is why `not $done` and `not a.ready` also read as expected. Suppose `not` bound tighter than comparison, as the unary minus does. Then `not a == b` would be `(not a) == b`, a type error in nearly every program that writes it, since the writer meant the negated comparison; the precedence that makes the line mean what it says is the one to choose. `not` is a prefix operator, so it cannot be the right operand of a tighter operator: `a == not b` is a parse error whose message asks for `a == (not b)`.

A free sigil at level 12 applies to everything to its right up to the end of the enclosing call or bracket, because it is prefix and nothing at level 12 or looser follows it: `@ ChecklistUpdate ix, !checked` is `@ (ChecklistUpdate ix, !checked)` since the call at level 3 is looser and extends under it. The free sigil and the paren-less call therefore nest the obvious way: the sigil wraps the call. `$ x + 1` is `$ (x + 1)` by the same reading.

A `.Method` that starts a continuation line stands outside the table. It closes everything open on the lines above, back to the statement's `=`, `<-` or `then`, and takes that whole expression as its receiver; level 13 covers only a `.` written against the expression before it.

## Statements

Three statement forms exist, and they are the only places where the sigil-level structure of a program is built rather than computed. They are statements, not expressions, because each one does something an expression cannot: a bind builds a widget, a hold owns state, and a `let` introduces a name that later statements in the same block can see.

```
!click <- @Button $"OK"                  # bind: build the widget, name its result
@Element Time.Every(period=100)           # bind with the result discarded
((!lines, !eof)) <- @Lines               # bind with a tuple pattern over a bundle
TextInput(!changed, $value) <- @TextInput ""   # bind with a record pattern
```

```
let !delta = -1 on !decrement | +1 on !increment   # let: name a value, build nothing
let @b = @Button $"OK"                  # let of a widget: a description, unbuilt
$count <- 0 then $count + !delta          # hold
$$current <- $ Unavailable then !$contents # hold of dynamics, for a $Join
```

A bind's left side is a pattern, and every name in it takes the sigil of what it is bound to. A `let` is the same, with no building: `let @b = @Button $"OK"` followed by `!x <- @b` and `!y <- @b` builds two buttons, because a blueprint is a description and each bind of it is a widget. A hold's left side is a single sigiled name, because a hold is one piece of state with one definition; a program that wants two holds writes two.

Outside a widget block, only `let` exists, and a sequence of `let`s ending in an expression is how a lambda body or a `for` body with intermediate names is written:

```
let Magnitude: fn Vector -> Float = \match _:
    Vector(x, y):
        let xx = x *. x
        let yy = y *. y
        Sqrt (xx +. yy)
```

Statements in a widget block are mutually recursive and may be written in any order; the resolver finds the order. Statements outside one are sequential: a `let` sees the names above it and not below, since plain values cannot be cyclic and there is nothing to be gained by pretending they can.

## Lambdas and the pronoun

`\body` is a one-argument function whose argument is `_`:

```
\IsGui _.element                      # a predicate on slots
\_.element                            # a projection
\not IsGui _.element                  # the complement; the call takes the rest of the body
\Test.Box(value=_)                    # a constructor
\Sqrt (_.x *. _.x +. _.y *. _.y)      # a magnitude
```

Wherever `_` is in scope, a `.` or `[` in prefix position (by the spacing rule) applies to it: `.x` means `_.x` and `[i]` means `_[i]`. This holds in a lambda body, in a `for` body and in any other construct that binds the pronoun. So `\.element` is the projection above, `\[0]` indexes the argument, and `for todos: .id` projects each todo. Only one `_` is in scope at a time; an inner lambda's `_` shadows the outer one, and a program that needs both names one of them. A `.Method` that starts a continuation line is a chain step on the call above it, not a pronoun use.

`\|a, b| body` names the arguments; one of them may still be the pronoun, `\|_, k| ...`, and any of them may be a pattern, `\|((x, y)), k| ...`, or `ignore`:

```
let Step: fn Watch, Instant, Action -> Watch = \|w, now, a| match a: ...
Fold changes, items, \|acc, id, c| Step acc, id, c
\|ignore, v| v
```

`\@widget:` opens a widget block as the body, which is how widget functions are written, and `\|$slots| @widget:` names a dynamic argument for one:

```
@Button: fn $Text -> @!Click = \@widget:
    !!clicks <- rebuild $_:
        @Element Ui.Button(label=_)
    @ !Switch !!clicks
```

```
@Forward: fn $(Path ==> Slot) -> @!(Path ==> Response) = \|$slots| @widget:
    for $slots:
        rebuild $_:
            ((!resp, ignore)) <- @Element _.element
            @ !resp
```

**The argument's kind.** A lambda's argument has a sigil like any name, and the rule is this: a body that mentions a sigiled pronoun, `$_`, `!_` or `@_`, takes an argument of that kind and is not lifted over it. `@Button`'s body says `rebuild $_:`, so it takes a dynamic; a lambda whose body says `intercept @_, ...` takes a blueprint. A named argument has the sigil it is written with. A body with no sigiled pronoun and no annotation takes a plain argument, and the lambda is a plain function.

A plain function is lifted afresh at each application: one definition serves a plain argument and a dynamic one, so `IsGui` written once over a plain `Element` is applied to `$kids` and to `!in` without a second definition. That is why the prelude's `Len` and `Insert` are written at plain types and used over everything, and the operators `not`, `and`, `or` and `+` lift the same way. The alternative, inferring a lambda's binder sigil from its uses, would mean a lambda applied to a dynamic takes `$_` and cannot also take a plain value, so the prelude would need two of everything.

When a lambda with no sigiled pronoun is written where the expected type says the argument is a dynamic, as in `rebuild` blocks and the `intercept` supplier, the argument takes the kind the expected type says; with no expected type it is plain. This is bidirectional checking in the one place it is needed. It is tentative in exactly one respect: the alternative, inferring functor variables alongside Hindley–Milner so that a lambda's kind is unified from its uses, would make the rule uniform at the cost of the property that a value's kind is readable from its spelling, and if that cost turns out to be lower than it looks, this is the rule that would change.

### `match` and `if`

```
match shape:
    Circle r: Pi *. r *. r
    Square s: s *. s

match todo:
    TodoItem(label, checked=True): Some label
    TodoItem(..): Nil

match items[id]:
    Some t: Insert items, id, (Todo t.label, not t.done)
    Nil: items

match element:
    Ui.Button(label=l): ...
    Ui.*(..): ...
    ignore: ...
```

A `match` has a scrutinee and a block of arms, each `pattern: expression` or `pattern:` followed by a layout block, and optionally a guard between the pattern and the colon (below). Arms are tried in order and the first that matches is taken. A `match` on a closed `choice` must be exhaustive; one on an extensible variant must end in a catch-all arm; one on `?T` is the eliminator for optionals and must have both a `Some` and a `Nil` arm (or a catch-all); one on an array must cover every length (the array patterns, below). A guarded arm does not count toward any of these.

**Patterns:**

- A literal: `0`, `"quit"`, `'x'`, `unit`.
- A lowercase name, which binds. `_` binds the pronoun.
- `ignore`, which matches anything and binds nothing.
- `((p, q))` for tuples; `((p,))` for a one-tuple.
- `Some p` and `Nil`.
- `Type p` for a newtype or a choice alternative with a payload: `Circle r`, `Running from`.
- `Type` alone for a unit alternative: `Stopped`, `Backspace`.
- `[[]]` for the empty array; `[[p, q]]` for an array of exactly two elements; `[[head, ..rest]]` for a non-empty array, with `head` the first element and `rest` the array of the others. The patterns mirror the literals: `[[p1, ..., pn]]` matches exactly `n` elements, and `[[p1, ..., pn, ..rest]]` matches `n` or more, binding the remainder (`..` alone leaves it unbound). The `..rest` comes last.
- `Type(p1, p2)` for a record, matched positionally; `Type(f=p, g=q)` by field name; `Type(f, g)` as shorthand for `Type(f=f, g=g)`, binding each field to a name of its own; `..` allowing the remaining fields to go unmatched. A record pattern that names every field needs no `...`
- `Ns.Tag(...)` for a tag of an extensible variant, with the same record forms; `Ns.*(..)` for any tag under a namespace; a bare `Ns.Tag` for a tag with no fields.
- `p: Type` ascribes a type to a pattern.

**Array patterns.** You have walked an array by hand: check the length, index the front with `xs[0]`, get an `?T` back for an element you just proved was there, and match it anyway. An array pattern asks the question once. The empty case and the non-empty case are the two arms that cover every array, and the non-empty one hands over the first element and the rest:

```
let Summary: fn [Text] -> Text = \match _:
    [[]]: "nothing"
    [[only]]: only
    [[first, ..rest]]: "{first} and {Len rest} more"
```

The arms cover every length: none, exactly one, and one or more. The middle arm only picks out a case of the last one, and a program with just the first and the last is exhaustive too. A `match` on an array is exhaustive when its unguarded arms cover every length; `[[a, b]]` alone leaves out every other length, and the missing-case message names a length it does not cover.

**Guards.** You have written an arm that matches a pattern and then immediately checks a condition on what it bound, and had to repeat the fall-through case because the nested `if` cannot send control back to the next arm. A guard sits between the pattern and the colon, `pattern if condition:`, and the arm is taken only when the pattern matches and the condition, with the pattern's bindings in scope, is `True`. Otherwise the next arm is tried:

```
match items[id]:
    Some t if t.done: Insert items, id, (Todo t.label, False)
    Some t: items
    Nil: items
```

Without the guard, the second arm's work would be inside the first as an `if` and the `Some t: items` case would be written twice. The condition runs from `if` to the arm's colon, so an `if` expression inside a guard takes parentheses. A guarded arm does not count toward exhaustiveness: the compiler does not reason about conditions, so `Some t if t.done` covers nothing, and the `match` above is exhaustive only because the unguarded `Some t` and `Nil` follow. A guarded `ignore if c:` is not a catch-all for an extensible variant. A guard on a dynamic scrutinee is lifted with the rest of the arm, and dynamics it mentions are read as the arm body's are.

**`match` lifts like application.** With all arms plain, `match $x: ...` is a dynamic of the arms' results,

changing when `$x` does, and `match !e: ...` is an event firing when `!e` does, with the taken arm's value. Dynamics mentioned in the arms are lifted by the same rule as any operation: in `match !e: Some v: v + $offset`, the `$offset` is read through a tag. When any arm is a blueprint, the scrutinee must be plain, because choosing a blueprint by a changing value is construction and is spelled `rebuild`; the error says so and points at it.

```
let $text = match $contents:
    Unavailable: "(unavailable)"
    Known t: t

let !line = match !ev:
    Received t: Some t
    Closed: Nil
```

`if c: a` then `else: b`, on one line or on two lines with `else` at the column of `if`, is `match c:` on `Bool`, with `True: a` and `False: b`. It lifts the same way: `if $done: "x" else: " "` is a dynamic text. An `if` without an `else` is an error, because every expression has a value and there is no `unit` to fall back on silently; a program that wants "do nothing" says `unit` in the `else`, and one that wants "nothing" says `Nil` in it: `if n == 1: Some unit else: Nil` is a `?unit`.

## Text interpolation

`"... {expr} ..."` builds text, lifted by the rule above. The braces hold an expression, usually a name, and the name keeps its sigil: `{$count}` interpolates a dynamic, `{!line}` an event, `{name}` a plain value. The interpolated value must be `Text` or a primitive with a canonical rendering (`Nat`, `Int`, `Float`, `Char`); anything else is a type error, not an implicit `ToText`. A record interpolated into a label would print as something, and the something would be a format nobody chose; the error costs a call to a rendering function the program had to write anyway.

```
$"Count: {$count}"                    # $Text over a $Int
"{Total $watch, $clock} ms"           # $Text over two dynamics, through a plain function
"{name} has {n} items"                # plain, if both are
"Line: {!line}"                       # !Text
"Got {?maybe}"                        # ?Text: the ? lifts at the plain layer
"cost: $5"                            # plain text; the $ is only a character
"set \{x\}"                          # the characters {x}
```

Text has one interpolation marker, `{`, and an escape for it. A `$` in text is never special, so a price, a shell variable or a currency sign needs no escape, and `"cost: $5"` is the text it looks like. Suppose `$name` also interpolated. Then every dollar amount in a label would be a question about what follows the `$`, and a name written next to a digit or a space would silently turn into text or an error depending on the next character. With one marker the reader asks one question of any literal: is there a brace? The braces are also what a reader already knows from other languages' format strings (from memory: Python's f-strings and Rust's `format!` both use `{}`), and the cost is that a literal brace is written `\{` or `\}`.

## `for` as an expression

Outside a widget block, `for xs: body` maps `body` over a plain array or map. In the shorthand, `_` is the element and `key` is the key of the entry: the map key for a map, and the position, a `Nat`, for an array:

```
let squares = for [[1, 2, 3]]: _ * _
let ids = for todos: .id                        # prefix .id is _.id
let doubled = for prices: _ *. 2.0              # a map, keyed as the input
let labelled = for names: "{key}: {_}"          # key is the position in an array
```

You have nested two loops and needed the outer index inside the inner one, and found the inner loop had taken the name. The shorthand binds the same `_` and `key` at every level, so an inner loop hides the outer one. Named binders say which is which: `for k, item <- xs: body` binds `k` to the key and `item` to the element, and binds nothing else, so an enclosing loop's `_` and `key` stay visible:

```
let cells = for r, row <- grid:
    for c, cell <- row:
        "{r},{c}: {cell}"
```

The binder before `<-` is a pattern, so it takes the sigil of what it is bound to. Over a plain collection both are plain, `for k, item <- xs:`. Over a dynamic collection in a widget block the key is plain, since it is the identity of the block, and the element is the per-key dynamic, written `for k, $item <- $items:`; its body sees `$item` in the way the shorthand's body sees `$_`, and the key `k` where the shorthand sees `key`. `for $items:` is the shorthand for `for key, $_ <- $items:`. Writing `for k, item <- $items:` is an error that asks for `$item`, because the per-key value is a dynamic. There is no binder for a third thing: a loop that wants more than the key and the element takes them from the element.

It evaluates to an array when the input is an array and a map when the input is a map, in key order. The body is one expression or a layout block of `let`s ending in a value. Over a dynamic collection it is the construct of the switching chapter and is only allowed in a widget block, because it builds widgets; the error for `for $m:` outside a widget block says so.

## Diagnostics

A language whose syntax leans on spacing, sigils and inference is a language whose error messages are part of its design, because the mistakes a newcomer makes are predictable and each one has a right answer the compiler can name. Every node of the parse tree and of the typed tree has a source span from the first commit, since spans are the one thing a compiler cannot retrofit cheaply. The errors below are the ones the conformance suite pins with negative goldens: a rejected program and the exact message, spans included. Each is given with the form of its message; the wording is the implementation's, the content is not.

**Two events in one operation.**

```
error: two events in one operation
  --> counter.prism:4:17
   |
 4 |     let !delta = !increment + !decrement
   |                  ^^^^^^^^^^   ^^^^^^^^^^ both are events
   = an operation may lift over any number of dynamics and at most one event
   = to fire on either, merge them: `!increment | !decrement`
   = to fire on both in one occurrence, hold one and sample it at the other
```

Names both events, points at `|` and `!Leftmost`, and says what the two meanings of "both" would be spelled as.

**A cycle with no delayed edge.**

```
error: these bindings depend on each other in the same occurrence
  --> loop.prism:3:5
```

```
  |
3 |     let $a = $b + 1
  |         ^^ depends on $b
4 |     let $b = $a * 2
  |         ^^ depends on $a
  = a cycle in a widget block must pass through a hold's value, a tag, or a !Switch
  = a `rebuild` is not a delay: its event fires in the occurrence of the rebuild
```

Lists every binding in the cycle, in order, with spans. A cycle through `rebuild` alone gets the same message with the last line as its reason.

**A negative number where a subtraction was meant.**

```
error: `a` is not a function
  --> math.prism:7:12
   |
 7 |     let n = a -1
   |             ^ ^^ applied to -1
   = did you mean `a - 1`?
```

**A suffixed operator against known types.** A program using `+~` on two values already known to be `Float`, or `+.` on `Int`s, gets "operator `+~` is Int addition, but both operands are Float; use `+.` or `+`".

**A name whose sigil does not match its type.**

```
error: `count` is bound to a dynamic, but its name has no sigil
  --> counter.prism:5:5
   |
 5 |     let count = $total + 1
   |         ^^^^^ a dynamic must be named with `$`
   = write `$count`
```

The reverse, `let $x = 3` with a plain right side, says "a plain value must be named without a sigil, or lifted: `$ 3`". A name with the wrong sigil, `!x` bound to a dynamic, says which it is and which it should be.

**A name that strays from its convention.**

```
warning: `isClick` is a value name that is not snake_case
  --> gui.prism:2:5
   = values are snake_case (`is_click`); functions and types are UpperCamelCase (`IsClick`)
```

The lint is a warning. `Todo_item`, an uppercase name with an underscore, is an error with the same two lines of explanation.

**A match on an extensible variant without a catch-all.**

```
error: this match on `Element` has no catch-all arm
  --> fwd.prism:9:13
   |
 9 |             match _.element:
   |             ^^^^^ `Element` is an extensible variant; tags may be declared elsewhere
   = add a final arm: `ignore: ...`
```

**A non-exhaustive match on a closed choice.**

```
error: match does not cover `Backspace`
  --> edit.prism:12:5
   = `Key` has alternatives Char, Left, Right, Backspace; add an arm or `ignore:`
```

**A match on a dynamic with a blueprint arm.**

```
error: choosing a widget by a changing value
  --> view.prism:4:5
   |
```

```
 4 |     match $mode:
   |           ^^^^^ the scrutinee is a dynamic and an arm is a widget
   = building depends on the value; write `rebuild $mode:` and match inside it
```

**A hold outside a widget block.**

```
error: a hold must be inside a widget block
  --> helper.prism:3:5
   |
 3 |     $n <- 0 then $n + !e
   |     ^^^^^^^^^^^^^^^^^^ holds belong to a widget
```

**Lifting across a sigil boundary.**

```
error: `+` cannot lift into the inner layer of `$$a`
  --> join.prism:6:13
   = lifting covers the outermost sigil only; flatten first: `$Join $$a + 1`
```

**An optional where a plain value is required at a boundary.**

```
error: `?Nat` where `Nat` is required
  --> idx.prism:4:21
   |
 4 |     @Element Ui.Row(height=xs[i])
   |                             ^^^^^ indexing returns ?Nat
   = element fields are plain; match on it, or give a default: `Or xs[i], 0`
```

Lifting over `?` stops at any position whose type is fixed by a declaration: an element's field, an annotated binding, a tag's `requests` event. The message says where the `?` came from.

**A type variable in a tag's field.**

```
error: tag field `value` mentions the type variable `a`
  --> lib.prism:8:5
   |
 8 |     let Wrap: fn a -> Element = \Test.Box(value=_)
   |                                           ^ `a` is not a concrete type
   = a tag's fields have concrete types; build the element at a known type
```

**Interpolating something without a rendering.**

```
error: cannot interpolate a `Todo` into text
  --> list.prism:7:22
   = text interpolation takes Text, Nat, Int, Float and Char; render it: `{Describe t}`
```

**Layout.** "expected an indented block after `:`", with the span of the colon and the line that was found instead; "unindent does not match any open block", with the line's column and the columns currently open (`open blocks are at columns 0, 4, 8; this line is at 6`); "tab character in indentation", with position. The second message's list of open columns turns a layout error from a puzzle into a one-keystroke fix.

**Reserved words.** `read`, `update` and `&` each say "reserved for a future version of the language; not available", so that a reader who tries one learns that it is intended rather than misspelled. `%` is reported as an unknown character.

**`for` over a dynamic outside a widget block.** "`for` over a dynamic collection builds widgets and must be inside a widget block"; with a plain collection inside a widget block it is the plain mapping and needs no message.

**`ix` outside a `for`.** "`ix` is bound only inside a `for` in an `update` block"; likewise `jx` and `kx`, and `jx` inside a single loop says "`jx` is the index of the second-nested `for`; this loop's index is `ix`".

**`key` outside a shorthand `for`.** "`key` is bound only inside a `for` written without `<-`; name the key

in `for k, x <- xs:`". The same message applies inside a loop with named binders, where `key` is not bound.

**An `if` without `else`.** "every `if` has an `else`; this one has no value when the condition is False".

**A guarded arm in an exhaustiveness check.** "match does not cover `Nil`; the arm `Some t if t.done` has a guard and does not count". The message names the guard so that a program that thought the guarded arm covered its case finds out why it did not.

**A match on an array that misses a length.** "match does not cover arrays of length 0; add `[[]]` or `ignore:`".

**Unknown path and other runtime errors** belong to the error chapter; the compiler's job is the ones above, and the test for each is that the message contains the fix.

## Layout corpus

The programs on which the two lexers must produce identical token streams. Each is given with its stream sketched in a compact notation: `⟨` opens a layout block, `;` is a statement separator, `⟩` closes a block; `:` is an inline colon token; names are written as they appear, with attached sigils part of the token; `$free` marks a free sigil; `TEXT(...)` is a text literal with its parts, a plain segment as a string and an interpolation as its tokens in braces; `-pre` and `-bin` mark the prefix and binary readings; `\|` opens lambda bars and `|)` closes them, as distinct from the merge `|`. The streams omit nothing.

**1. The counter.** A statement continued by nothing; every line a statement.

```
@counter:
    !increment <- @Button $"Increment"
    let !delta = -1 on !decrement | +1 on !increment
    $count <- 0 then $count + !delta
    @Label $"Count: {$count}"
```

```
  @counter ⟨ !increment <- @Button $TEXT("Increment")
  ; let !delta = -pre 1 on !decrement | +pre 1 on !increment
  ; $count <- 0 then $count +bin !delta
  ; @Label $TEXT("Count: ", { $count }) ⟩
```

Note `$"Increment"` is an attached sigil on the text token, and `$"Count: {$count}"` is likewise attached with an interpolation part inside. `-1` is the prefix reading because the `-` follows whitespace and precedes a non-space.

**2. A continued statement inside a block.**

```
@form:
    $edit <- Edit initial, (Len initial)
        then Apply $edit, !key
    @Label (Render $edit)
```

```
  @form ⟨ $edit <- Edit initial , ( Len initial ) then Apply $edit , !key
  ; @Label ( Render $edit ) ⟩
```

The continuation line produces no separator and no block tokens.

**3. Two blocks closed by one dedent.**

```
@outer:
    !x <- rebuild $a:
        !y <- @Button $"B"
```

```
        @ !y
    @Label $"done"
```

```
  @outer ( !x <- rebuild $a ( !y <- @Button $TEXT("B") ; @free !y )
  ; @Label $TEXT("done") )
```

**4. Dedent of two levels at once, to column 0.**

```
@a:
    !x <- rebuild $b:
        @ unit
@c:
    @ unit
```

```
  @a ( !x <- rebuild $b ( @free unit ) ) ; @c ( @free unit )
```

Two closes, then the separator for the new top-level statement.

**5. Inline arm then layout arm.**

```
let Area: fn Shape -> Float = \match _:
    Circle r: Pi *. r *. r
    Square s:
        let side = s
        side *. side
```

```
  let Area : fn Shape -> Float = \ match _ ( Circle r : Pi *. r *. r
  ; Square s ( let side = s ; side *. side ) )
```

The first `:` after `Area` is an inline colon, not a block opener, because tokens follow it on the line; the parser reads an annotation there because the statement began with `let`. The `:` after `_` is line-final and opens the layout block.

**6. `:` inside brackets, across lines.**

```
let m = {
    "a": 1,
    "b": if c: 2 else: 3
}
```

```
  let m = { TEXT("a") : 1 , TEXT("b") : if c : 2 else : 3 }
```

No block tokens anywhere; the braces suspend layout and the colons are plain colon tokens.

**7. A trailing comment after the opening colon, and comment-only lines inside the block.**

```
@counter:    # the counter
    # the first button
    !increment <- @Button $"Increment"

    # a blank line above, a comment here
    @Label $"ok"
```

```
  @counter ( !increment <- @Button $TEXT("Increment") ; @Label $TEXT("ok") )
```

**8. `F -1` against `F - 1` against `F-1`.**

```
let a = F -1
let b = F - 1
let c = F-1
let d = F - -1
```

```
  let a = F -pre 1 ; let b = F -bin 1 ; let c = F -bin 1 ; let d = F -bin -pre 1
```

**9. Merge against lambda bars.**

```
let m = \|a, b| a | b
let n = !x | !y
```

```
let m = \| a , b |) a | b ; let n = !x | !y
```

**10. A sigil before a bracket, both spellings.**

```
@ ((!a, $b))
@((!a, $b))
```

```
@free (( !a , $b )) ; @free (( !a , $b ))
```

Identical streams.

**11. A test definition: the `=>` continuation lines.**

```
test counter, @counter:
    build
        => expect Ui.Label(text="Count: 0")
    click Ui.Button(label="Increment")
        => expect Ui.Label(text="Count: 1")
```

```
test counter , @counter ⟨ build => expect Ui.Label ( text = TEXT("Count: 0") )
; click Ui.Button ( label = TEXT("Increment") ) => expect Ui.Label ( text = TEXT("Count: 1") ) ⟩
```

The line-final colon opens the block; the `=>` lines are indented more than the block's column, so by rule 3 they continue the turn line above them, and the `test` form reads each `=>` as one check of that turn. Two `=>` lines under one turn are two continuations of one statement.

**12. The `for`-then-merge error.**

```
@bad:
    !x <- for $m:
        @ !click
      | !other
```

```
@bad ⟨ !x <- for $m ⟨ @free !click ⟩ ERROR(unindent to column 6 matches no open block: 0, 4)
```

Both lexers must report the error at the `|` line with those columns.

**13. A method chain with the spacing rule.**

```
let a = items.Filter IsDone
    .MapValues Label
let b = Filter items, \.done
```

```
let a = items .bin Filter IsDone .chain MapValues Label ; let b = Filter items , \ .pre done
```

`.bin` after `items` is method form; `.chain` marks a `.` that is the first token of a continuation line, which the parser reads as a step whose receiver is everything back to the `=`; `.pre` after `\` is field access on `_`. The one-line spelling `items.Filter IsDone .MapValues Label` is an error from the parser, since its `.` is neither first on its line nor attached.

**14. Interpolation containing text and a colon.**

```
let s = "{if c: "yes" else: "no"}!"
```

```
let s = TEXT( { if c : TEXT("yes") else : TEXT("no") } , "!" )
```

One token, with one interpolation part holding seven tokens and one text part.

**15. A record block as an argument.**

```
let origin = Vector:
    x: 0.0
    y: 0.0
```

```
  let origin = Vector ⟨ x : 0.0 ; y : 0.0 ⟩
```

**16. `else` at the column of `if`, with layout blocks in both branches.**

```
let v = if c:
    let t = 1
    t + 1
else:
    0
```

```
  let v = if c ⟨ let t = 1 ; t +bin 1 ⟩ ; else ⟨ 0 ⟩
```

The separator before `else` is emitted by the lexer, since `else` is at the block column; the parser joins the two statements. The whole thing is one `let`.

**17. A tab in leading whitespace.**

```
  @a:
  →   @ unit
```

```
  @a ERROR(tab in indentation at 2:1)
```

**18. Nested namespaces and the wildcard.**

```
match e:
    Ui.Text.Input(..): 1
    Ui.*(..): 2
    ignore: 3
```

```
  match e ⟨ Ui.Text.Input ( .. ) : 1 ; Ui . * ( .. ) : 2 ; ignore : 3 ⟩
```

`Ui.Text.Input` is one qualified-name token. `Ui.*` lexes as the name `Ui`, a binary `.`, and `*`; the parser recognises the three as the wildcard pattern.

**19. A dollar sign and an escaped brace in text.**

```
let s = "cost: $5, {n} items, \{literal\}, $count"
```

```
  let s = TEXT("cost: $5, ", { n }, " items, {literal}, $count")
```

One token with three parts: a plain segment, an interpolation holding one token, and a plain segment. Every `$` in text is a character, and the escapes `\{` and `\}` are decoded by the lexer into the plain segment, so the parser never sees a brace that is not an interpolation.

Both implementations keep this corpus as a golden test: the source on one side, the stream on the other, diffed. A new ugly case found in the field is added here first and to the lexers second.

## Negative space of the language core

The language's refusals are listed with the whole design in the first part of the book; these are the ones that belong to the core, each with what its exclusion buys.

**No assignment.** `count = count + 1` cannot be written. Not in a handler, not in a helper, not in a `let`. The only state is a hold, and a hold moves only when its event fires. What this buys the core: every binding is an equation, so a widget block can be read in any order and the resolver can

build it in dependency order; `let` is a definition and never an update, so two readers of a block agree on what a name means without tracing control flow; and because every write is a hold's commit, an occurrence is a transaction for free.

**No sampling.** There is no `.current`, `.value` or `sample` on a dynamic. The only reads are a tag (in a lifted operation with an event), a derived dynamic (prompt), and `rebuild` (rebuild). What this buys: there is no "which version" question, since each of the three reads has one defined answer, and the implementation is free to propagate in any dependency-respecting order because nothing can observe the difference.

**No custom monads, no effect system.** One block type per functor, and the set is fixed: widget blocks, `rebuild`, `for`, `match`. There is no `do`, no transformer stack, no way to define a fourth functor with its own bind. What this buys: the lifting rule is a table over four sigils; a name's kind is readable from its spelling because the set of kinds is closed; and the compiler's elaboration is a lookup, not a search through instances. A program that wants a `Reader` passes an argument; one that wants a `with` has a hold; one that wants `IO` has an element.

**No overloading beyond the three traits.** `Eq`, `Ord` and `Num` are resolved by inference; nothing else is. There are no user-defined traits, no instances, no operator definitions, no implicit coercions through a class. What this buys: a call site names one function, and the reader can find it; inference stays Hindley–Milner on the value layer with three well-understood exceptions, and the checker's error messages stay local. The three are the ones without which sorting, map keys, element identity and arithmetic cannot be written; everything else has a name.

**No implicit conversions.** `Nat` does not become `Int`, `Int` does not become `Float`, `Text` does not become anything, a newtype does not become its payload and a `?T` does not become a `T`. Conversions are functions (`+?`, `Abs`, `ToText`, `.value`) and the one lifting over `?` is visible in the result's sigil. What this buys: the type of every expression is the type of its parts, and a count never becomes a float because it was divided; a line of arithmetic has one numeric type and the reader can find it at the nearest annotation or suffixed operator.

**No reified identifiers, no introspection.** There is no way to name a binding as data, reach into a widget, list a block's bindings, or ask a value for its type. A test drives a program through its elements, the way a World does, and reads its element list and its result. What this buys: a test that passes under the Test World passes under any World, since it used only the boundary; a name can be renamed without breaking anything but the program; and generic code can be erased, because nothing can ask a value what it is.

**No symbol with two meanings.** `<=` is less-or-equal and nothing else, `then` is hold and nothing else, and `<-` is bind and nothing else. What this buys: the line that defines a program's state never looks like a comparison, and a comparison never looks like state.

**No explicit-brace layout, no block comments, no tabs, no `else`-less `if`.** Each of these removes a second way to write something, and each second way is one more case on which two parsers can disagree. The spec is the thing, and the thing is shorter for it.

**No `read` and `update` blocks, no `&` references, no lenses.** These are described at the end of this chapter, and no transcript depends on them. Plain values are built, never modified; a function that "updates" a record builds a new one. What this buys now: the value layer has one operation on a record, construction, and the question of how mutation inside a plain value interacts with sharing does not arise. If they come, they come as sugar over construction, and nothing in this chapter changes meaning.

## `&` references, `read` and `update` blocks

You have a record with a record in it with a map in it, and you need to add one to a counter three levels down. In an imperative language it is a line. In a pure language it is a `let` per level, each one rebuilding the enclosing record with one field replaced, and the line that does the work is at the bottom of a staircase of plumbing. This is the one place a pure language is worse to write than an imperative one, and it is the one `update` blocks exist to fix. An `update` block sits beside the book's statements that the language has no assignment without contradicting the language the rest of the book specifies: it is a separate block form inside plain code, its value is an updated copy of a value, nothing outside the block sees a write, and nothing else in the book depends on it. Nothing in the other chapters needs them, and the semantics of references under erasure and copy-on-write are the part that has to be designed before the Rust implementation can commit to a representation.

### What the feature is

Three pieces, each small on its own.

A `read` block gives the pronoun over a value. `\read _:` opens a block in which `_` is the lambda's argument, `.x` reads a field of it, and `[k]` indexes it, as in any lambda or `for` body; `read v:` gives the same prefix `.x` and `[k]` over any value `v`, and an `update` block gives them over the value being updated, and `read p <- v:` binds a pattern. Only `let` statements and one result statement are allowed. The point is not saving characters. A function over a large value should read like a description of the value:

```
type Vector = record:
    x: Float
    y: Float
    z: Float

let Magnitude: fn Vector -> Float = \read _:
    let x = .x
    let y = .y
    let z = .z
    Sqrt (x * x + y * y + z * z)
```

An `update` block is an expression that evaluates to an updated copy of a value. Inside it, `&_` is a reference to the value being updated and `&_.x`, `&_[k]` are references into it. A reference can be set with `<=`, or combined with an operator: `+=`, `-=`, and so on for every operator the type has:

```
let Add: fn Vector, Vector -> Vector = \|a, b| update a:
    &_.x += b.x
    &_.y <= .y + b.y      # the same thing, spelled out
    .z += b.z             # the pronoun is implicit on the left too
```

The original `a` is unchanged. The block's value is the updated copy. Statements run in sequence, and a later statement reads the value as earlier statements left it, which is what makes the block worth having: `&_.x += 1` followed by `let t = .x` sees the new `x`.

References, `&T`, exist only inside an `update` block. A function may take a reference and return one, `fn &Vector -> &Float`, and that is a lens: it says where inside a `Vector` a `Float` lives, and applied inside an `update` it gives a reference you can set. A function may not take two references, return a reference it did not derive from its argument, or store one anywhere; a reference is a path into the value being updated, and a path only means something while the update is happening. `*r` dereferences for reading and gives the current value at the path.

Two conveniences fall out. A function whose result type is its first argument's type, `fn Vector, Vector -> Vector`, is an *update function*, and inside an `update` block it may be used as a statement with its first argument implied: `Add b` inside `update a:` means `&_ <= Add _, b`. And `for` inside an `update` block over a reference to an array or map yields a reference per element:

```
let AddByElement: fn [Float], [Float] -> [Float] = \|a, b| update a:
    for &_:                # `&_` inside the loop is a reference to a[ix]
        &_ += b[ix]
```

`ix` is the index of the current iteration; nested loops get `jx` and `kx`, and past that a helper function is better than another keyword.

## Why it is wanted

The use cases are the ones where a program keeps a large value in a hold and folds small changes into it. A text editor's buffer, a spreadsheet's cell map, a drawing's list of strokes, a game's entity table, a form with forty fields. Each one is a hold whose event is a merge of small edits, and the fold is a function from the old value and an edit to the new value. Today that function is written with functional update at every level:

```
type Doc = record:
    lines: Nat ==> Text
    cursor: Cursor
    dirty: Bool

let Insert: fn Doc, Edit -> Doc = \|doc, e|
    Doc(
        lines = Set doc.lines, e.line, (Splice doc.lines[e.line], e.col, e.text),
        cursor = Cursor(line = e.line, col = e.col + Len e.text),
        dirty = True)
```

Every field has to be named even when two of them are unchanged, the nested `Splice` is hidden inside the `Set`, and adding a field to `Doc` touches every function that builds one. The same function as an update:

```
let Insert: fn Doc, Edit -> Doc = \|doc, e| update doc:
    &_.lines[e.line] <= Splice .lines[e.line], e.col, e.text
    &_.cursor <= Cursor(line = e.line, col = e.col + Len e.text)
    &_.dirty <= True
```

The goal this serves is the one about seeing what moves. The hold's definition still lists every event that can change the document; nothing about that changes. What changes is that the function the hold applies says what it changes and nothing else, so reading it is reading the diff, which is what you wanted to read.

The second reason is performance, and it is the same reason reuse is in the optimisations list above. A functional update that rebuilds a record of forty fields to change one copies forty words; a map insertion copies the map unless the map is persistent. An `update` block that knows which path it changed can, when the value's count is 1, change exactly those words in place. The feature and the optimisation are one design seen from two sides.

## The semantics as far as they go

Everything about references can be said without mutation, and should be, because that is what makes them safe to add to a language with no assignment. An `update` block is a function from a

value to a value. A reference is a lens, a pair of a getter and a setter over the root value. A statement `&p <= v` is `root <- set_p root v`. A statement `&p += v` is `root <- set_p root (get_p root + v)`. A `let` inside the block reads the root as it is after the statements before it. The block's value is the root after the last statement. Under this reading an `update` block is a fold of lens applications over the root, every value is immutable, and the block could be elaborated into exactly the staircase of functional updates it replaces. That is the reference semantics, and the Haskell implementation runs it directly.

Lenses nest. `&_.lines[e.line]` is the lens for `lines` followed by the lens for key `e.line`. A user function `fn &Doc -> &Text` is a lens written by hand, and inside an `update` it is applied like any lens: `&(CurrentLine _) <= "..."`. The map indexing lens has the insertion rule from the aggregate types: setting `&m[k]` on a map inserts if the key is absent, and reading `*&m[k]` gives a `?V`. The array indexing lens is a plain index, and setting it out of range is the same error as reading out of range would be, a `?` on the read side and an abort on the write side, which is the checked-arithmetic discipline applied to indexing.

Then the borrowing rules, which exist for one reason: so that an implementation may mutate in place. If two live references could alias, an implementation that wrote through one would have to know whether the other would see it, and the only safe answer would be to copy. So, inside one block: a reference is live from its creation until its last use; two live references must be disjoint, and the compiler must be able to see that they are. Two different fields of a record are disjoint. Two different literal keys or indices are disjoint; two keys that are variables are not known to be, and the second reference is an error while the first is live. A reference obtained through a user lens function borrows the lens's whole argument for its life, because the compiler cannot see inside the function. These are Rust's rules for `&mut` reduced to a block with no loops but `for`, no conditionals that keep a reference across arms, and no escape, which is a small enough fragment that the check is a pass over the block and not a borrow checker.

Copy-on-write is the implementation side, and the thing the Rust implementation has to get right. When the block starts, the root value has some reference count. If it is 1, which it is when the caller passed the block its last reference, the block owns it and every `<=` through a path with count 1 at each level writes in place. If any level's count is greater than 1, that level is copied once, the copy is written, and the parent is pointed at the copy; later writes through the same level find a count of 1. This is `Arc::make_mut` applied along a path. The observable behaviour is identical to the functional reading because no one else can see the write, and the borrowing rules are what guarantee that "no one else" includes the block itself.

The hold case is the one the reuse entry above describes: one copy per occurrence per level touched, which is already far better than a functional update's copy per field, and the runtime hook that lets a hold hand over its value when its only reader this occurrence is its own update removes the last copy. Nothing about `update` depends on that hook existing.

## What it must not change

An `update` block is an expression with a value. No reference escapes it; a reference is not a value and has no type outside a block. Nothing about occurrences is involved: an `update` block that appears inside a lifted expression runs when that expression's node runs, once per occurrence, over the value it was given, and the block cannot see or touch a hold, an event or a dynamic. Widget blocks do not get mutation; the only state in a program is a hold and the only way to change one is its event. A program that compiles today compiles unchanged, because `&`, `read` and `update` are new syntax and no old program has them.

The erasure question is the one that is not yet answered and is why the item is research: a lens through a type variable, `fn &[a] -> &a`, needs to write into a value whose layout generic code does not know. Under the one-word representation, where generic code clones and drops by tag bits and a record points at its layout, the setter can write a word into a field slot without knowing its type, so erasure and `update` are compatible, but the proof is a working implementation and not this paragraph.

### Why it belongs in the Reflex implementation first

The Haskell implementation interprets the typed AST, and a lens in Haskell is a first-class value with exactly the getter-setter semantics above: two lenses nest the way two functions do, and reading through one, writing through one and mapping through one are the three operations an `update` block needs (the `lens` library is the implementation choice; the form of its API is from memory). Adding `update` to the Haskell evaluator is adding an elaboration case that turns a block into a fold of lens applications, and the library does the rest. That makes the experiment cheap: the syntax, the pronoun rules, the typing of `&`, the borrowing check and the `for` form can all be tried on real programs and tested through the conformance suite before the Rust implementation commits to copy-on-write along paths and the compiler to a borrow pass. If the semantics change during the experiment, nothing in Rust has to change back. Once the suite has `update` programs with golden transcripts, the Rust implementation has an oracle, and the differential test says when its in-place version is right.

### The fact it waits on

A program in the suite, or a library, whose fold functions are more plumbing than logic. The text editor is the likeliest first case, because its buffer is a map of lines and every keystroke is a nested update. Before that, functional update is adequate and the feature would be a guess about ergonomics.

### What the check can refuse

The borrowing check has a question of its own. The sketch rejects two references whose keys are variables, a reference held across the arms of an `if`, and a function taking two references. All three have a meaning in the functional reading of the block, since swapping `a[i]` and `a[j]` is a well-defined fold of two lens applications. The rules are there so that the in-place implementation is safe and so that the check stays one pass over the block. A static check for the cases it can prove, with a runtime comparison of keys for the cases it cannot, gives programs the meaning the functional reading gives them: when the two keys are equal, the statements run in sequence on a copy at that level, or the occurrence aborts the way an out-of-range write does. Which of the two is right depends on whether a program that swaps an element with itself should be a no-op or a mistake, and the text editor's buffer is the first place the question will come up.

## User-defined traits and modules

Two pieces of ordinary language work, with no bearing on the semantics of time, deferred for exactly that reason: they can be added when a program needs them without any law changing, and a design that lands them early is a design that spends its attention on the wrong thing while the laws are still being tested. What follows is what each would look like, so that when the need arrives the work is known.

## Traits

You have a `Money` type, or a `Vector`, and every sum in the program is spelled `Add a, b` where you would write `a + b`, because the numeric trait reaches only the built-in number types. Comparison is already there: a record is ordered by its fields in declaration order, so `a < b` on two `Money` values works today, with the derived order rather than one the type's author chose. The language has three built-in traits, equality, ordering and numeric, which the checker resolves at each use to the implementation for the operand type and passes as a hidden argument where a generic bound needs one. That mechanism is a trait system with no syntax for declaring a trait or adding a type to one. User-defined traits are the syntax.

```
trait Show a:
    Show: fn a -> Text

impl Show Vector:
    Show = \read _: "({.x}, {.y}, {.z})"

impl <a: Show> Show [a]:
    Show = \|xs| Concat [["[", (JoinText ", ", (for xs: Show _)), "]"]]
```

The elaborator treats a trait as a record of functions, an `impl` as a value of that record, and a bound `<A: Show>` on a generic function as a hidden record argument, resolved at each call site by the operand's type. That is dictionary passing, the same thing the three built-ins do, and it is why generic code stays erased under the one-word representation: a trait method on a type variable is a call through the dictionary, which is a plain record of closures, and the value itself needs no descriptor.

What traits do not do, and the exclusion is the design: no traits over the functors. There is no `Functor` or `Monad` trait, no user-defined block kind, no instance that says how `!` lifts. The sigils fix the set of functors and the lifting rule is the language's; a trait that could change how `$a + !b` elaborates would make the sentence "you can see what moves" false, because what moves would depend on an instance in scope. Traits are over plain types only, and a sigiled type (`!Int`) may not be an instance. Coherence is the simple rule: one `impl` per trait per type in a program, and overlap is an error at the point both are in scope, which the module system below makes a concrete place.

What traits buy, in order of how soon it will be wanted: `Show` for debugging and labels, `Default` for a record with many fields, a `Paint` trait in the GUI library so that a container can take a list of any types with a `Paint` instance, without a closed choice of what can be drawn, and the user-defined numeric types (a `Money`, a `Px`) that make `+` mean something for them without a suffix. The last is also the one place the deferral has a visible cost today: a program with a `Vector` type writes `Add a, b` where it would write `a + b`, which is where this section began.

## Modules

One file plus the prelude is the whole of the current module story. Anything larger than one file needs a way to split, and anything meant to be reused by another program needs a way to say what it exports.

```
module Ui.Layout

import Draw (Rect, Color)
import Ui.Focus as Focus

export @HBox, @VBox, Extent, Pos
```

A file is a module and names it on its first line. Imports bring in a module's exports, optionally renamed; exports list what is visible outside. Tags are already namespaced, `Ui.Button`, and a module's tag declarations live in its namespace, so `tag Button of Element` inside `module Ui` declares `Ui.Button`, which makes the tag namespaces and the module namespaces one thing. Instances are exported with their trait or their type, whichever the module defines, and the overlap check runs at import.

What the typed AST needs: qualified names. Every binding in the text form is already named; a module system makes the name `Gui.Layout.HBox` instead of `HBox`, and nothing else in the form changes. That is the test for whether modules are ordinary language work: if the typed-AST text form had to gain a construct, the deferral would be hiding a semantic question. It does not.

The thing deliberately left out: no module-level state and no module initialisation. A module is a set of definitions, a program is `@main`, and the first thing that happens is occurrence zero building it. A module that wanted to "run something at load" would be a widget being built by nothing, which the language does not have, and this is the same rule as "nothing happens because an element appears" applied to source structure.

### The fact it waits on

A program that does not fit in one file, or a library (the GUI library is the candidate) that a second program wants to import. Both will arrive during the conformance work, because the GUI library in Prism is a module the twelve programs share, and the moment it is shared it needs a name. Traits come when the second user-defined numeric type appears.

## A package system

A second repository wants the GUI library. Someone copies the directory. Three weeks later the two copies disagree on `Ui.Button`'s fields, a program that imports one of them is handed elements by a widget written against the other, and the first sign is a `Build` error in a tag the author never touched. A module system says how one program is split into files. A package system says how code written by someone else gets into a program, with a version, and how a program says which versions it is compatible with, so that the disagreement is a version mismatch the loader reports and not an afternoon. It comes after modules and is the last item here because it is the one with the least to do with the language and the most to do with everything around it.

### What a package is

A named set of modules with a version, a manifest, and its own conformance scripts:

```
name    = "gui"
version = "0.3.0"
prism   = ">=1.0"          # the specification version the package is written against
depends = { "draw" = "^0.2" }
```

Modules in a package live under its namespace, so a package `gui` provides `Gui.Layout`, `Gui.Focus`, and so on, and its tag declarations are in the `Gui` namespace. That makes the tag namespaces, the module namespaces and the package namespaces one thing, and it makes "who owns `Gui.Button`" a question with one answer. A World's components are packages too: the CLI World's tags are the `cli` package's declarations, and a program that imports `Cli` is declaring which World components it needs, which a World can check at load.

Two things a package may not contain, and the loader is what enforces both. No native code: a

package is Prism source and the manifest and nothing else, so that the Rust and Haskell implementations read the same package and the differential test covers it. Native code lives in Worlds, which are host-language programs, and a package that needs something native declares a tag and a World provides it. And no build scripts: a package is read, not run, which is the "nothing happens because something exists" rule applied once more. Each implementation's loader rejects a package containing any file that is not Prism source or the manifest, before any of it is parsed, and both loaders run the same check from the same rule, so a package that one implementation accepts and the other refuses is a differential-test failure like any other. The package author is bound by the rule; the loader checks it; nothing downstream of the loader needs to.

## Versions and the conformance suite

A package's version is a claim about its exported signatures and its behaviour, and the second half of that claim is testable. A package ships its own test scripts, written in the conformance suite's script language against the Test World, and a dependent program's build runs them under the implementation it uses. A package that passes its own scripts under both implementations is a package the differential test has checked; that is a stronger guarantee than most ecosystems offer, and it costs nothing beyond what the suite already is.

Resolution is the ordinary thing: a lockfile with exact versions, semantic versioning on the manifest, one version of a package per program (no diamond with two versions, because tag declarations would collide and because the one-impl-per-trait-per-type rule has to hold program-wide). The compatibility question that is Prism's own is the `prism` field: the specification version. The typed-AST text form has a version, and a package compiled against one specification version has a dump that a later frontend can read or reject, which is the same mechanism incremental compilation's cache uses.

## What it must not change

The language, the typed AST beyond qualified names, the laws. A program with packages is a program with more modules.

## The fact it waits on

The second program that wants the GUI library, from a different repository. Before that, a module system and a path on disk are enough.

## Chapter 4

# Events, dynamics, and time

Everything a Prism program does happens in an occurrence, and an occurrence is a transaction: it has a before, an after, and no inside that anything can see. This chapter is the definition of the two kinds of value that live against that clock, events and dynamics, and of the clock itself. Each is given the same way: the afternoon you lost to the problem it solves, the ordinary-language meaning, the denotation, the laws, and a few lines of code that show the law doing work. The laws are small and there are not many of them, and the claim is that they are enough: the tag law alone turns a cyclic definition into a definition, the change law alone rules out glitches, and the rule for what a widget built in the middle of an occurrence sees is one sentence with five consequences. The chapter ends with each law restated as a Test World script, the things deliberately left out of this part of the language and what each absence buys, and four idioms that are what writing against these laws feels like day to day.

## Three questions and one clock

Every interactive program is three questions asked over and over: what is true now, what just happened, and what exists. Most languages answer the first with a variable, the second with a callback, and the third with an object, and the three answers do not agree about time. The variable has whatever value the last writer left; the callback runs whenever the framework gets to it; the object was constructed at some point and will be destroyed at some other point, and neither point is visible from the code that uses it. Prism answers the three questions with three kinds of value whose sigils are part of their names, `$`, `!` and `@`, and defines all three against one clock.

The clock is a sequence of occurrences, numbered from zero. An occurrence is discrete and atomic. Something either happens in occurrence 7 or it does not; there is no "during 7" at which half of it has happened. A dynamic has one value before 7 and one value after 7. An event fires at 7 or is silent at 7. A widget is built in some occurrence, lives through some, and is torn down in one. Occurrence zero is the one in which the program is built, and it exists so that everything built at the start has an occurrence to have been built in. Plain values and optionals have no relation to occurrences at all, which is what makes them plain.

Widgets are the subject of their own chapter. Events and dynamics are the subject of this one, with the clock they are defined against. What follows holds for every event and every dynamic in a program, whether it came from a button, a socket, a `for`, or a line of arithmetic, because none of those is a special case: `!click` is an event in exactly the sense `!delta` is, and the laws do not know which one was native.

# Events

## The handler that ran at the wrong time

You have a text field with a change handler, and a form with a submit handler, and the submit handler reads the field's model. One day a user types a character and presses Enter in the same instant, which on a slow machine means the same frame, and the submit handler runs before the change handler, because the submit handler was registered first, and the form posts the old text. You fix it by reordering two `addEventListener` calls. Nothing in the program's text says that the order matters; the fix is invisible in the diff except as a swap, and the next person to touch the file will put them back in the order that reads better.

That is the whole of what is wrong with callbacks, and the ordering is only the visible part. A callback is code that runs *when* something happens, and "when" was never defined. It runs after some other callbacks and before others, reading state that some of them have changed and some have not. Two things that happen "at once" happen in registration order, which is a fact about the program's history and not about its text. And because a callback is code, it can do anything: read any state, write any state, call anything, including the code that registers more callbacks. The question "what happens when the user presses Enter" has an answer spread across every handler that might run in that frame, in an order nothing records.

Prism's answer is to take "when" away from the code and give it to the language. Nothing runs when something happens. Instead, a thing happening is a fact about a moment, and the program is a set of statements about how facts at a moment relate. Enter was pressed at occurrence 12; the text at occurrence 12 is whatever it was before 12; the submission at 12 is the text before 12, if Enter was pressed at 12. There is no handler to order because there is no handler.

## What an event is

An event is something that happens, with data about the specifics of this happening. A click, with where it was. A chunk of stdin, with its bytes. A response from a file read, with the contents or the error. A timer going off. The common sense of the word is the right one, and it is the one to hold on to against the other sense the word has acquired, in which an "event" is an object with a `.preventDefault()` method that a handler receives. A Prism event is not a value that is true, and it is not an object that is delivered. It is a thing that occurred, and between occurrences there is nothing there to look at.

**Definition.** An event `!A` is a value of type `A` at some occurrences and absent at the others. As a function, it is a partial function from occurrence numbers to `A`.

That is the entire denotation, and it is deliberately the simplest thing that could be written down. It has two consequences that everything else in this section follows from. An event is total over time: `!click` is not "the click that just happened", it is the whole history and future of clicks on that button, and asking "did it fire at 12" is a question with an answer. And an event has no value between firings. There is no `!click.last`, no "the most recent click"; if you want that, you hold it into a dynamic and say what it was before the first click, and the hold is where the "most recent" lives.

## Simultaneity

Two events that fire at the same occurrence are simultaneous, and simultaneity is a thing the language defines rather than a race the implementation resolves. A program never observes two events in an

order within an occurrence. There is no "`!a` fired, then `!b` fired" at occurrence 12; there is "`!a` and `!b` both fired at 12", and every operation that takes more than one event says what that means for it.

This is why `!a + !b` is a compile error and not an event of sums. At an occurrence where only `!a` fires, what is the sum? There is no answer, and rather than invent one (zero? skip it? the last `!b`?), the language refuses. The rule is that an operation may have any number of dynamics and at most one event among its operands, and the two or more case is an error naming both events and pointing at the two operations that do know what simultaneity means: `|`, which takes one of them, and `!Leftmost`, which takes one of several. The message has the form "two events in one operation: `!a` and `!b`; merge them with `|` or `!Leftmost`", with both spans, and it is a compile error, never a runtime one.

This is liveable because simultaneity is rare, and when it happens the program has already said what it wants. A click and a keystroke in one frame; a network response and a timer in one turn; two buttons the user mashed together. In every framework you have used, these cases were resolved by registration order and you found out which order when a bug report arrived. Here they are resolved by the line that merges the two events, and reading that line is reading the resolution.

The refusal is about two *events*. An optional inside an event is a different matter and raises no error: `!?a` is an event that sometimes fires with `Nil`, and `?` lifts inside it at the plain layer, so `!?a + 1` is an `!?Nat` that fires whenever `!?a` fires, with `Nil` when it did. The same holds under `$`: `$a / $b` over `Nat`s is a `$?Nat`, because `/` on `Nat` and `Int` returns `?`; on `Float` it follows IEEE and never fails, so the same line over `Float`s is a plain `$Float`. The time functor is always outermost and lifts first; `?` lifts second, inside it; and `match` on the optional is how you get back out. `!Filter` is that `match` packaged for events: it takes an `!?A` and gives the `!A` that fires only on `Some`. The precedence between `?` and the time sigils, and `?$x` as the thing that does not lift, are chapter 3's; here it is enough that a `?` never makes two events coincide, because it never adds a second event.

### `!never`, and why there is no `!always`

`!never` fires at no occurrence. It has every type, and there is one of it. It is the identity for merge, the event you return from a `match` arm that has nothing to say, and the initial inner event of every `!Switch`.

There is no `!always`. It is the first place the language says no to something that seems harmless. An `!always` would fire at every occurrence, and the obvious use is "do something every time anything happens": log the occurrence, recompute a layout, poll a value. But "every time anything happens" is a statement about the program as a whole, and no part of the program is entitled to it. If an occurrence has nothing to do with you, you do not get to know about it. A widget that could observe every occurrence could observe that a sibling it has never heard of received a response, which is exactly the ambient channel the goal of no global anything forbids. It would also break the implementation's one license: the runtime may skip a node whose inputs did not fire, and `!always` would make every node's inputs fire.

Everything you wanted `!always` for is available some other way. A value that should be recomputed when anything it depends on changes is a derived dynamic, and the change law (below) says it changes exactly then. A log of what happened is a merge of the events you care about. A poll is a clock element, which is a dynamic pushed by the World, and a program that depends on one says so in its element list.

## Merge, and why the left wins

`a | b` fires at n if either fires at n; its value is `a`'s if `a` fires, else `b`'s. It is associative, and `!never` is its identity on both sides. `!Leftmost [[e1, e2, ...]]` is the same over an array, taking the first in array order that fires.

The left bias is the answer to "what if both buttons are clicked at once", and it is an answer the program gives by choosing which one to write on the left. In the counter,

```
let !delta = -1 on !decrement | +1 on !increment
```

two clicks in one occurrence are a decrement, and if that is wrong the fix is to swap the operands, which is a change you can read.

Suppose instead the merge produced both. `a | b` would be an event of a small list, or of a pair of optionals, and every consumer would have to say what it does with two. For the counter that is `-1 + 1 = 0`, which is a defensible answer, and it is also an answer the program can write by hand: `$count + (Sum !deltas)` over a `for` that consolidates into a map. The question is what the *operator* should mean, and an operator that returns a collection is an operator everyone has to unpack. The common case by a wide margin is that one of the two should win, and the uncommon case has `for`'s consolidation and the library's `!Union` of maps, where both really are wanted and the result really is a collection.

Suppose instead simultaneity in a merge were an error. Then every merge of two events that could ever coincide would need a guard, and most pairs of events can coincide: the World is allowed to batch two inputs into one occurrence, and a program cannot know which pairs it will batch. A merge that could fail at runtime would make every program's correctness depend on the World's batching policy, which is the registration-order bug with a new name.

Suppose instead the merge were right-biased, or took "the most recent". The first is the same design with a different convention. The second is the registration-order bug itself: there is no "most recent" within an occurrence because there is no order within an occurrence.

So the left wins, the program says what is on the left, and when the program wants both it says that too, with a different operator whose result type admits both. A few passing examples of the left bias chosen on purpose:

```
let !action = Reset on !reset | Step on !tick        # a reset in the same frame as a tick is a reset
let !key = !shortcut | !typed                         # the shortcut layer sees the key first
let !close = unit on !escape | unit on !close_button  # either closes; it does not matter which
```

The third one is the common case in which the bias is irrelevant because both sides have the same value, and the merge is simply "or".

## `on`: how an event gets its data

`v on !e` fires exactly when `!e` fires, with the value `v`. When `v` is a dynamic, the value is the dynamic's value *before* the occurrence, which is the tag law of the next section applied here. `on` is how a click becomes "a click while the label was X" and a keystroke becomes "Enter, with the text as it stood".

`on` has no semantics of its own. It is a plain function that returns its left operand, `\|v, e| v`, and the lifting rule gives it its meaning: an operation with one event operand fires when that event fires, with the operation applied to the event's value and the dynamics' old values. The whole of `on` is that lifting rule plus a function that ignores its second argument. This matters because it means there is nothing to learn: `$text on !enter` and `(Length $text) on !enter` and `(Submit $text, $user) on`

`!enter` are all the same mechanism, and in the last one the name `on` could be dropped in favour of a three-argument function and nothing would change.

Three uses you will write constantly:

```
let !submitted = $text on !enter              # the value at the moment
let !tick_at = $clock on !tick                # the time before the tick
let !selected = $items[$cursor] on !choose     # a lookup, lifted over two dynamics and one event
```

One thing `on` does not do is pair an event's own value with a dynamic; it discards the event's value. To keep both, apply a function: `(\|k, n| ((k, n))) !key, $count` is an event of pairs, the key that fired and the count before it. Everyone reaches for `(($count, !key))` once and finds that a tuple with an event inside it is a plain tuple, not an event; the sigil's position says what moves, and in `((!a, $b))` nothing does.

## Lifting over one event

A lifted operation with exactly one event operand fires exactly when that operand fires, with the operation applied to the event's value and the dynamics' values before the occurrence. `IsClick !in` filters nothing; it is an event of booleans that fires with every `!in`. `!Filter` turns an `!?A` into an `!A` that fires only on `Some`, and it is the one plain combinator a program needs to drop firings; everything else is a function applied to an event.

The rule has a consequence that is easy to miss: nothing inside the language makes an event fire from nothing. A lifted operation fires when its event operand fires; a merge fires when a side fires; `!Switch` fires when its chosen inner fires; `rebuild`'s event fires when its block is built, which happens when its dynamic changes or when the `rebuild` itself is built, which happens because something above it was built, and so on up to the root in occurrence zero. Every firing in a program is traceable to an input the World delivered or to the build. The World owns time, the runtime has one entry point, and the same inputs give the same occurrences every time, which is why a transcript is a test.

### `!Switch`

A button whose label is dynamic is rebuilt when the label changes, and each build has its own click event. The parent does not want a new `!click` per label; it wants one `!click`, for the button, whichever label it happens to show. A view that is replaced when a mode changes has a submit event per view, and the model wants one `!submit`. A connection that is reopened has a received-bytes event per socket, and the protocol wants one `!received`. In every case there is a `rebuild` whose block returns an event, the `rebuild`'s result is an event of events, one per build, and the rest of the program wants the one event that is "whichever is current".

`!Switch: fn !!A -> !A` is that event. At occurrence n it delivers the firing, if any, of the inner event that the outer event most recently delivered *before* n. It starts subscribed to nothing. An inner event delivered at n is heard from n+1.

```
@DynLabelButton: fn $Text -> @!Text = \@widget:
    !!rendered <- rebuild $_:
        !click <- @Button _
        @ _ on !click
    @ !Switch !!rendered
```

Inside the `rebuild` block, `_` is the label as a plain value, because within one build it never changes; a change rebuilds the block. `!!rendered` fires in every build with the new block's click event, tagged

with the label. `!Switch` turns the generations into one event.

This pattern is so common that binding over `@` does it for you. `!click <- @Button $label`, with a dynamic where the widget wants a plain label, means exactly the five lines above: a `rebuild $label:` around the build, and `!click` is the `!Switch` of the `rebuild`'s event. A widget function applied to a dynamic argument lifts to `rebuild`, and the bound name is the switch of the generations. When the widget's result is a dynamic rather than an event, the bound name is a hold of the generations' dynamics, `$Join`ed, with the first build's result as the hold's initial value; when the result is plain, the bound name is a `$` holding the first build's value. The rule is stated here once; every later example in this book writes `!a <- @Button $"A"` and means the switch. One consequence to keep in mind: a `!click` bound this way is non-prompt in the label, so a relabel and a click in one occurrence deliver the click from the old button, as the next paragraphs explain.

**Why it is not prompt.** Suppose it were: an inner event delivered at n would be heard at n. Then a block built at n could fire at its parent at n, through the switch, and the parent could react, and the reaction could change the dynamic the `rebuild` watches, which would rebuild the block at n, which would fire again. Reflex has `coincidence` and `switchPromptlyDyn` for exactly this, and anyone who has used them has debugged them. The height of a prompt switch's output depends on which inner event it is currently listening to, and the inner event can be chosen *in the same occurrence* by something downstream of the output, which is a cycle that only shows up when a particular inner is chosen. The runtime has to re-enter propagation to handle it, and the program has to reason about what a freshly built thing can and cannot do in its first occurrence.

With a non-prompt switch, every node's height is fixed by its sources as they stood when the occurrence opened. A `!Switch` reads which inner to listen to from a hold, and the hold's value side is a read of before-state, so the switch's output is below nothing that fires at n through it. A cycle through a switch is one comparison at the moment the switch changes inners, not a re-entrant search. And propagation never has to re-enter itself: once a node has fired at n, nothing at n can make it fire again.

**What it costs.** A freshly built block cannot fire at its parent in the occurrence it was born, through the switch. The switch swallows whatever the new block fired at n before it was adopted. Under the rule that elements never fire in the occurrence they were created, the only things a new block can fire at n are pure consequences of its own construction, `@started` and the events of any `rebuild` inside it, and a parent that wants to know the block was built already knows: the `rebuild`'s own event fired at n with the block's result. If a block needs to act on its own birth, it uses `@started` inside the block, where there is no switch in the way:

```
!!loads <- rebuild $path:
    !started <- @started
    !resp <- @Element Fs.File(path=_, requests=(Read on !started))
    @ !resp
let !loaded = !Switch !!loads
```

`Fs.File` is chapter 7's one-shot file conversation, `requests: !FsRequest` and `responds: FsResponse`. Here `@started` fires in the block's build occurrence, the `Read` fires in that occurrence, the response comes in a later one, and by then `!Switch` is listening to the new block's `!resp`. Nothing is lost, because nothing a block could want to say to its parent on the occurrence of its birth is something the parent did not already know from building it.

In the button above, a click in the same occurrence as a relabel is delivered from the old button, under the old label. That is the only answer consistent with routing, since the click was addressed to a slot in the element list as the occurrence opened, and the old button is what was there. The old

block keeps propagating until the occurrence commits, so its click reaches the switch, which is still listening to it; at commit the switch adopts the new inner, and from n+1 the new button's clicks are the ones heard. There is no moment at which a button is on screen and nothing is listening to it, and no moment at which two are listened to at once.

### Events are values, not callbacks

An event is a value. It can be stored in a record, put in an array, passed to a function, returned from a widget, merged with another, used twice. `[!Click]` is a plain array of events and `!Leftmost` takes one. A record with fields `!changed` and `$value` is how a text input returns two things. None of this requires a subscription, a listener, or a disposal; an event used twice is the same event, and an event nobody uses costs nothing.

Compare the callback. It is not a value you can merge: to say "either of these" you write a third function that both call. It is not a value you can pass without also passing the thing it will be called on. It cannot be used twice without being called twice. Its "subscription" is an effect with a lifetime that someone has to end, and the someone is usually nobody, which is where leaks come from. And it can do anything, so "what can happen when this fires" is not a question with an answer in the type.

The consequence that matters most for reading a program: in Prism, the flow of what-happens is in the data, not in the control. `!delta` is defined from `!increment` and `!decrement`; `$count` is defined from `!delta`; the label is defined from `$count`. Each line names its inputs. To find out what a click does, you follow the uses of `!click`, and every use is a definition of something else. There is no handler body to read, because there is no handler.

## Dynamics

### The variable read in the middle

You have had the afternoon where a checkbox un-checks itself. You click it, the handler sets the model, the model notifies the list, the list re-renders, the re-render reads the model to decide whether the box is checked, and the model, it turns out, was updated *after* the list read it, because the notification went out from inside the setter before the assignment. You move a line and it works. Three months later someone adds a second subscriber that also reads the model during notification, and the box un-checks itself from a different direction.

The variable in that story was never wrong. At every instant it held exactly what the last assignment put there. What was wrong was that "now" had no definition: the list read the model at a moment in the middle of a cascade, and at that moment some things had been updated and some had not, and nothing in the program could say which. Every scheduler, dirty flag and batching layer in every reactive framework exists to paper over this one fact, and the papering is never finished because the fact is structural. A variable that can be read at any instant can be read at the wrong one.

Prism fixes "now" to the occurrence. Before an occurrence a dynamic has one value. After, it has one value. During, the only question anyone can ask is "what was it before", and that question has exactly one answer. There is no instant in the middle of the cascade at which a read can land, because reads do not happen at instants; they happen at occurrences, and an occurrence has no inside.

## What a dynamic is

A dynamic is a value that is true now and will be different later: the count, the text in the box, the contents of a file, the set of connected users, the position of the cursor. It is the thing every program has and most languages call a variable, and it is the only mutable thing in Prism. It is mutable in exactly one way: it changes when its change event fires, to that event's value. That sentence is the whole of the language's mutation policy. Everything else, the World boundary included, is arranged so that this stays the only way anything changes.

**Definition.** A dynamic `$A` has a value of type `A` at every occurrence, and changes only at occurrences. It is a pair: a *current value* and a *change event*. At occurrence n the current value is the value *before* n, and the change event, if it fires at n, has the value *after* n. After n, the current value is the change event's value if it fired, else unchanged.

The current value is *before* n, always; there is no reading of a dynamic at n that gives the after-value except the change event itself, which is an event and is consumed as one. The change event has a name: `!Updated $d` is `$d`'s change event, a plain prelude function of type `fn $a -> !a`, firing with the new value in every occurrence `$d` changes and never at build. The current value has no name: there is no operator that returns it, no way to ask for it except through the laws below. Reflex exposes it as `current`, the behavior half of a `Dynamic`, and lets you `sample` it. Prism does not, and the reasons are the subject of the last part of this section.

## The tag law

In any lifted operation with an event operand, a dynamic contributes its value before the occurrence. `$count + !delta` at n is `(count before n) + (delta at n)`. `$text on !enter` at n is the text before n. `Step $state, $clock, !action` at n is `Step` applied to the state before n, the clock before n, and the action at n.

This is the one window onto *before*, and it is deliberately the only one.

The tag law makes a cyclic definition a definition. Look at the counter's hold:

```
$count <- 0 then $count + !delta
```

`$count` appears on both sides. In a language with assignment this is `count = count + delta`, an equation whose solution is "whatever the last write was". Here it is not an equation: the `$count` on the right is a read through a tag, which by the law is the count *before* the click, and the hold on the left is the count *after*. The line reads "the count after a click is the count before it plus the delta", which is a definition, and the cycle in the graph passes through a read of before-state, so nothing in the occurrence depends on itself.

The tag law makes propagation deterministic. At an event there is one version of every dynamic, the one before. There is no "which version did I see", because within an occurrence nothing has a second version to see. Two reads of `$count` in two places in one occurrence, at two different events or at the same event through two different operations, give the same number. Reflex lets you choose: `tag (current d) e` gives the old value and `attachPromptlyDyn d e` gives the new one, and which you wanted depends on whether `d` is updated by something upstream of `e` in this frame, which is a question about the graph's structure that the program's text does not show. The `tagPromptlyDyn` documentation says outright that `tag (current d) e` is preferred when creating cyclic data flows, which is the library telling you that the choice is a trap. Prism removes it. If you want the new value, you want the change event, and you say so by using the change event.

The tag law leaves the implementation free. Because every read at an event is a read of before-state, the implementation may propagate the occurrence in any order that respects prompt dependencies and nothing can tell. It may compute a value early, late, or on demand; it may skip a node whose inputs did not fire; it may, later, run independent nodes in parallel. None of that is observable, because the only observable reads are of a state that was fixed when the occurrence opened.

The tag law is also where the "before" in a stopwatch comes from. A Start click stamped with `$clock on !start` gets the clock as it stood when the occurrence opened, even if the same occurrence also pushes a new clock value. The hundred milliseconds between the two is the price of a readable rule, and a World that cares pushes the clock in an occurrence of its own first.

## The change law, and promptness

A lifted operation over dynamics and no event is a dynamic whose value at every occurrence is the operation applied to the operands' values at that occurrence, and whose change event fires at n, once, with the new value, if any operand's change event fires at n.

So `1 + $x` is a dynamic that is one more than `$x` at every occurrence, before and after. `"Count: {$count}"` is a dynamic text. `$items[$cursor]` is a dynamic optional, changing when either changes. `Total $watch, $clock` changes with the clock while the watch runs. A derived dynamic is never stale: in the occurrence its input changes, it changes.

That is promptness, and it is the point. Suppose derived dynamics lagged: `$x` changes at n, `1 + $x` changes at n+1. Then at the end of n, after commit, `$x` is 5 and `1 + $x` is 5, and anything that reads both, a label showing "x is 5 and x+1 is 5", sees a state that was never true. That is a glitch, and it is the checkbox afternoon in miniature. A derived dynamic that changes in the occurrence its input changes cannot show such a state, because there is no occurrence in which the input has changed and the derivation has not.

The change law and the tag law meet at a lifted operation with both kinds of operand. `$count + !delta` is an event, and its dynamics are read through tags. `$a + $b` is a dynamic, and its change event is prompt. The sigils decide which law applies, which is why you can read it off the line: an event in the operands means a tag; no event means promptness.

A derived dynamic fires its change event when an operand's change event fires, with the new value, *even if the new value equals the old one.* `$edit.text` changes when `$edit` changes, and if only the cursor moved the text is the same text. A dynamic's change event is allowed to fire with the value it already had. Nothing downstream can tell the difference except by comparing, and the comparison is a plain function the program can apply if it wants to; a derived `rebuild` over `$edit.text` will rebuild on a cursor move, and a program that minds writes `rebuild ($Distinct $edit.text):`. `$Distinct` is a prelude function that needs equality on the value and passes on only the changes that differ from the value before; it is the one place equal values are suppressed, and it is opt-in. The alternative, in which the change law compared values and suppressed equal ones, would need equality on every type a dynamic can hold, including types that have none, and would make the number of firings depend on the representation of values, which the implementation chapter wants to be free to choose.

## Hold

```
$checked <- False then !toggle | !reset
```

There is the whole story of `$checked`. It starts false. A toggle sets it to the toggle's value, a reset sets it to the reset's, a toggle and a reset together are a toggle, and nothing else in the program can

touch it, because nothing else in the program has a way to. You have searched a codebase for every assignment to a flag, found eleven, and still missed the one behind a reflection call; here the search is one line long and it is the line that declares the thing. That is the goal of one way to change anything, written next to the thing, and the hold statement is where it is syntax.

Formally: `$x <- v then !e`, allowed only as a statement in a widget block, defines `$x` whose value before the first firing of `!e` is `v`, whose change event is `!e`, and whose value after any occurrence where `!e` fired is `!e`'s value there. Hold is the only way to make a dynamic that is not derived from another. Every piece of state in a Prism program is a hold, so every piece of state has a line that names its initial value and the one event that can move it, and reading that line is reading everything that will ever happen to it.

The value `v` is plain, or a dynamic if `$x` is a dynamic of dynamics. It is not an expression re-evaluated on each firing; it is the value before the first firing and nothing else. A hold is a statement rather than an expression because it has an owner, the widget the block built, and there is no hold outside a widget block because that would be state nothing owns. Owner is the wrong word for what the widget does; the widget *coordinates* the hold, and it has to for two concrete reasons. The first is birth. A widget built in occurrence n sees all of n (the construction rule in the time section), so a hold born in n after its event fired in n has to adopt that firing at commit, and to do that it has to ask the occurrence in progress whether the event fired. That is the one thing a hold needs from the moment it is made, and it is the only thing: two holds of the same event made in different occurrences differ until the event's next firing, and that difference needs no bookkeeping, it falls out of subscribing when made. The second is rollback. An occurrence that aborts has to drop everything it built, and the widget's journal is the list of what that was; a hold made outside any widget would survive an abort that should have unmade it. What the widget does not do is kill the hold: a hold keeps holding as long as anything refers to its dynamic, which may be longer than the widget that made it lives, since a block's result can be passed up and kept. A `rebuild` makes a fresh hold because the rebuilt block's code runs again, not because the old one was destroyed. A hold outside a widget block would have no occurrence to ask and no journal to be in. The Haskell implementation runs on Reflex, where both are spelled as the `MonadHold` constraint: the monad is how the frame reaches `hold` for the same-frame read, and a `hold` outside it does not type-check. The runtime spells it as a parameter: `Behavior::hold` takes the runtime, which only a build or a World holds. Which widget owns which hold, what a `rebuild` firing and a `for` key's departure do to the holds inside them, and where state that must survive a rebuild lives are chapter 5's; the one fact this chapter needs is that a hold inside a rebuilt block is a new hold, starting at `v`, and it obeys the construction rule below like any widget built mid-occurrence.

In the occurrence in which the widget is built, `$x`'s value is `v`, and if `!e` fires in that same occurrence, `$x` changes to it then. This is the build occurrence behaving like every other occurrence: the hold has a before-value, `v`, and a change event, which fires or does not. It is how `@started` kicks things off:

```
!started <- @started
$phase <- Starting then Loading on !started | Loaded on !resp
```

`$phase` is `Starting` before occurrence zero and `Loading` after it, because `!started` fired in zero. A tag reading `$phase` in occurrence zero reads `Starting`; a label derived from it shows `Loading` from the end of zero. There is no special first-occurrence rule, and no special "initial render" during which holds are frozen.

## Glitch freedom

No operation observes a derived dynamic with some operands updated and others not. Within one occurrence a dynamic's change event fires at most once.

The diamond is the test:

```
$a <- 0 then !set
let $b = $a + 1
let $c = $a * 2
let $d = $b + $c
```

When `$a` changes from 0 to 5 at n, `$d` changes from 1 to 16, once. It does not pass through 6 (new `$b`, old `$c`) or 11 (old `$b`, new `$c`). Nothing that reads `$d`, a label, a `rebuild`, a tag at some event, can see either intermediate, because there is no occurrence in which either is `$d`'s value. A signals library has to work for this property, with a topological scheduler or a two-phase mark-and-sweep, and the schedulers are where its edge cases live. Here it is a law: the change law says `$d`'s value at every occurrence is the sum at that occurrence, and "at that occurrence" has no sub-steps. An implementation that fires each node once, after all of its prompt sources, satisfies it; heights (below) are the standard way.

The at-most-once half is the half that keeps the first half honest. If `$d`'s change event could fire twice at n, once per input, then "the value at n" would be ambiguous, and a hold fed by `$d`'s change event would have to pick. One firing, one value, one commit.

Who is bound: the runtime, for every derived dynamic, and the World, for every state element. A state element is a dynamic the World feeds, and a World that pushed two values for one element value in one occurrence would be making that dynamic change twice at n, which is the at-most-once half broken from outside. The runtime enforces its own half by firing each node once after its prompt sources (heights, below), and enforces the World's half at the boundary: a second push for the same element value in one run call is rejected as a `Component` error, like a duplicate response to one path, and the occurrence does not open. The script form of the law is the diamond test in "Laws as tests"; the boundary half has its test in chapter 10, where the run call is specified.

## Equality of readings

Two widgets that have never met both ask for `Fs.Contents(path="/etc/app.conf")`. One shows the file, the other parses it for a setting. The file changes on disk. You have seen the version of this where the parser ran on the new file and the viewer still showed the old one, for a frame or for a minute, because two caches with two expiry policies were between the disk and the two readers. Here the two slots are one dynamic, because state is identified by element value, and two results of it in one occurrence must match: the viewer and the parser change in the same occurrence or in neither.

The law: two readings of one dynamic in one occurrence, by any means, give the same value. A tag at `!click` and a tag at `!key`, both reading `$count` in occurrence n, read the same `$count`. Two labels derived from `$count` show the same number in every occurrence. A `rebuild` over `$count` and a tag on `$count` in the same occurrence agree about what the value was before and what it is after. This is the tag law and the change law stated as a property of the reader rather than of the operation, and it is listed separately because it is the law that extends across the World boundary. The occurrence is the version, for everything. The program is bound to read only through the laws, which the language enforces by having no other way to read. The World is bound to answer one value per element value per occurrence, through its oracle and its pushes, and the runtime enforces that by delivering one snapshot value to every slot that asks and one push to all of them; a World that answered the same

value differently to two slots in one occurrence would have broken the program, not merely the World, and the `clock_readings_agree` script below is the check.

## No sampling, and what it buys

You cannot ask a dynamic what it is right now. There is no `sample`, no `.current`, no `.value`, no way to write a plain function that takes a dynamic and returns its contents. You can read it through a tag, at an event, and get the value before the occurrence. You can rebuild on it with `rebuild` and get the value as a plain `_` inside a block that is replaced when it changes. You can derive from it and get another dynamic. That is the whole list.

This is the restriction that feels like a missing feature until the checkbox. Every "read it right now" in that story was a read of something half-updated, and the reason it was half-updated is that "right now" was an instant in the middle of a cascade. With no sampling there is no instant. A read is either at an event (and gets before-state, defined) or is a derivation (and gets the current value at every occurrence, prompt and glitch-free). There is nothing in between to get wrong.

The program gains this: every read of state is visible as such. `Step $state, !input` has `$state` in it; you can see that `Step` reads the state, and you can see at which event. A function that could sample would read state invisibly, inside its body, at a moment the call site does not show, and "what does this function read" would again be a question about its implementation.

The implementation gains freedom of order. Since no read can land in the middle, the middle can be anything. Nodes can fire in any order consistent with prompt dependencies. A value nobody reads need not be computed. Later, independent nodes at one height can run in parallel, and nothing in any program has to change, because no program could tell the difference. A language with sampling has to define what a sample in the middle of propagation returns, and whatever it defines becomes a constraint on every optimisation.

The thing you reach for sampling to do is almost always one of two things. "Get the value when this happens" is a tag: `$x on !e`. "Do something different depending on the value" is `rebuild`: build the block for this value, and rebuild when it changes. The one thing neither covers is "get the value *after* this occurrence's changes, at this event", and that is the thing the design refuses, because it is the thing that depends on propagation order.

## The hidden behavior

Reflex's `Dynamic` is a pair of a `Behavior` and an `Event`, and both halves are first-class: `current d` is the behavior, `updated d` is the event, and a program can combine them freely. The flexibility is real and so is the cost. `sample`, `tag`, `attach`, `attachWith`, `attachPromptlyDyn`, `tagPromptlyDyn`: the library has two of most things, one reading the behavior (old value) and one reading the event (new value), and the one you want depends on whether the dynamic is updated upstream of your event in this frame, which depends on the graph you cannot see. Pick wrong and you get a stale read or a cycle, and both are quiet.

Prism keeps the pair and hides the behavior half. Every operation that would read it, a tag, `!Switch`'s choice of inner, a hold's own value, `$Join`'s choice of inner before the swap, does so non-promptly, as before-state, and no operation in the language can do otherwise. There is one `tag`, not two, and it is spelled by putting a dynamic in an operation with an event.

The one place promptness remains is construction, and it is deliberate. `rebuild $x:` sees the new value of `$x` in the occurrence it changes, and the block it builds sees all of that occurrence (the

construction rule, below). Derived dynamics are prompt. `$Join` is prompt. You can always trust that a dynamic's change event has the new value in any occurrence it fires, and that the value after the occurrence is exactly that. What you cannot do is read that new value at an event in the same occurrence, and that is the whole restriction.

### Small things

`$ v` is the constant dynamic: `$ "Increment"` is a dynamic text that never changes, and `$"Increment"` is the same with the sigil attached. A function that takes `$Text` can be given a constant this way, and since a constant dynamic's change event is `!never`, deriving from it costs nothing.

`{$x}` inside a text literal interpolates by lifting: `"Count: {$count}"` is a dynamic text, changing with `$count`, with no sigil on the literal, because a literal that interpolates a dynamic is a dynamic. An event interpolated with `{!line}` makes the literal an event, firing once per line. A `$` in text is always an ordinary character, so `"costs $5"` says what it looks like, and a literal brace is written `\{` or `\}`.

`$$x` is a dynamic of dynamics, and `$Join` flattens it: `$Join $$x` at n is the value after n of the inner that `$$x` holds after n, so it is prompt in the outer and in the inner, including when both change at n. Chapter 5 owns the rest of switching; the law has its script in "Laws as tests" below. `?$x` is an optional dynamic, a plain value that is `Nil` or holds a dynamic; `$?x` is a dynamic whose value is sometimes `Nil`. They are different types, and the order of the sigils is the order you would say it.

## Time

### The occurrence as a transaction

You have a save button and an autosave timer, and one day they fire in the same frame. The button handler serialises the document and starts a write; the timer handler, running second, sees the dirty flag still set (the button handler clears it when the write completes, asynchronously), serialises again, and starts a second write, and the two writes race on disk. The fix is a lock, or an `isSaving` flag, or a queue, and whichever you pick is a transaction you are writing by hand around two handlers that each believed it was alone.

An occurrence is that transaction, written once, for everything, and the simplest way to hold it in your head is as an instant. Nothing in it takes time. Everything that changes in it changes at once, and the program never sees two of those changes in an order, because an instant has no inside. The alternative, a timestamp on every change and an ordering argued out from the timestamps, is what every event system ends up with, and it is where the save button and the timer came from. It is opened by the World with a set of inputs, each addressed to an element slot, all simultaneous. The program propagates: every event that fires as a consequence fires, every derived dynamic changes, every `rebuild` and `for` whose input changed builds anew. Then the occurrence commits: every hold takes its new value, every element list and every result takes its new value, and the World reads the outcome. Then it is closed. Between open and commit nothing outside the program observes anything, and nothing inside the program observes a state that mixes before and after except through the tag law, which is the one deliberate window onto *before*.

The two handlers above become one occurrence with two inputs, a click and a tick. The program's text says what a click and a tick together mean, through a merge, and the write request is one event firing once. There is no flag because there is no second handler to see it.

Holds committing at the end makes the rest possible. Because no write happens until commit, all

writes in an occurrence can be buffered, and an occurrence can be aborted by dropping the buffer: a cycle found halfway through, an overflow in arithmetic (kind `Overflow`; programs that want a different answer use the `Checked.*` and `Wrap.*` prelude functions), a protocol error from a World, and the occurrence simply did not happen. Nothing half-done survives, because nothing is done until the end. The error chapter builds on this; here it is enough that the transaction was already there, and it was there because the language has no assignment.

Occurrence zero is the build of `@main`. It has no inputs; it exists so that everything built at the start has an occurrence to have been built in, so that holds have a before-value and a build occurrence like any other hold, and so that `@started` fires at the start the way it fires in any rebuild. There is nothing else special about it. A program that behaves differently in occurrence zero than in a rebuild of its root is a program with a bug, not a program using a feature.

## The dependency graph

The program defines a graph over its events and dynamics. Each event or dynamic is a node; an edge runs from a source to a target when the target's firing or change at an occurrence is determined by the source's. The graph is what the program's text is: `let !delta = -1 on !decrement | +1 on !increment` draws two edges into `!delta`; `$count <- 0 then $count + !delta` draws one from `!delta` into the hold's change event and one, of a different kind, from the hold's value back into the addition. The whole program is this graph, the World feeds inputs into its leaves, and an occurrence is a wave through it.

Every edge is one of two kinds, and the distinction is the single most useful thing to be able to see in a Prism program.

## Prompt and delayed edges

An edge is *prompt* when the target's firing or change at n depends on the source's firing or change *at n*. These are the edges that make the wave: lifting over an event (`F !e` fires at n because `!e` did), lifting over dynamics (the change law: `$a + $b` changes at n because `$a` did), merge (`!a | !b` fires at n because a side did), `for`'s consolidation (the map fires at n because a block's event did), `$Join` (changes at n because the outer or the current inner did), and construction (`rebuild $x:` rebuilds at n because `$x` changed at n, and its event fires at n with the new block's result).

An edge is *delayed* when the target reads the source's value *before* n, which is to say its behavior side. There are three such edges, and they are the same thing seen from three places:

- A **tag**: a dynamic operand in an operation that has an event operand. `$count` in `$count + !delta`.
- **`!Switch`'s choice of inner**: which inner event the switch listens to at n is the one delivered before n. It is a hold of inner events, read before the occurrence.
- **A hold's own value side**: the value a hold has at n is the value it had after n−1.

All three are a read of a dynamic as it stood when the occurrence opened. That is what "delayed" means and all it means: not "later", not "next occurrence", not "scheduled", but "before". A delayed edge does not make anything happen later. It reads the past.

The distinction matters because only prompt edges constrain propagation order within an occurrence, and only prompt edges can form a cycle that means anything. A delayed edge reads a value that was fixed before the occurrence opened, so it imposes no order and closes no loop. The counter's cycle is `!delta → (change event) → $count → (tag) → $count + !delta → !delta`, and the tag edge is delayed, so the prompt graph of the counter is a tree.

Three ways to recognise the two kinds in source, since everything comes down to recognising them:

```
let !next = $count + 1 on !tick        # delayed into $count (tag), prompt from !tick
let $double = $count * 2               # prompt from $count (change law)
let !heard = !Switch !!generations     # delayed into !!generations' hold, prompt from whichever inner was
chosen
```

The rule of thumb: an event among the operands means every dynamic in the operation is read through a delayed edge; no event means every dynamic feeds a prompt one.

## Heights

The prompt graph must be acyclic. Given that, propagation in dependency order is well defined, and an implementation that fires each node once, after all of its prompt sources have settled, is correct.

Heights are the standard way to do it, and both implementations use them, but the specification does not require them. Each node sits at a height strictly above every prompt source; nodes fire in height order, lowest first; a node at height h fires only when every node below h that could fire at n has. A `!Switch` whose chosen inner changes recomputes its height from the new inner and invalidates every cached height downstream, so that the next occurrence's order is right for the new graph. A node built mid-occurrence gets a height from its sources as they are at that moment. Heights are cheap, they make "has everything below me fired" a single comparison, and they make a cycle detectable the moment a node tries to schedule below the current height.

The specification requires only the observable consequences: each event fires at most once per occurrence, each dynamic changes at most once, and no node observes a prompt source before that source has settled. An implementation that met those three by sorting the graph topologically at the start of every occurrence, or by pulling values on demand with memoisation, or by running a fixpoint iteration that happens to converge in one pass for an acyclic graph, would conform. The reason to say so is that the conformance suite tests transcripts and not heights, and the Haskell implementation runs on Reflex, whose heights are Spider's and not the Rust runtime's.

One consequence of "fires each node once, after all of its prompt sources": a node whose prompt sources did not fire at n does not fire at n and is not visited. An occurrence costs work in proportion to what happened in it, not in proportion to the program's size. A program with ten thousand holds and one click does the work of one click. That is the push model, and the reason the language can afford to make every derived value a live node.

## Cycles

A cycle in the prompt graph is a definition that depends on itself in one occurrence, and it has no meaning. `let !a = !b` with `let !b = !a` is the simplest: at n, does `!a` fire? Only if `!b` does, which only if `!a` does. There is no answer, not even "no", because `!never` is only one of the fixed points and nothing in the program chose it.

**Inside one widget block**, cycles are found at compile time. The rule: a binding cycle within a block is legal if and only if it passes through at least one delayed edge. A cycle made of prompt edges alone is an error that lists every binding in it, with spans. The compiler has every binding of the block in front of it and knows for each operation which of its operands are dynamics and which are events, so it knows which edges are tags. The counter passes: its cycle has a tag in it. This does not:

```
@broken:
    let !a = 1 + !b
```

```
    let !b = 2 + !a
    @ unit
```

and the error names `!a` and `!b` and points at both lines.

`rebuild` is not a delay. Its event fires in the occurrence of the rebuild, with the new block's result, and the block it builds sees the occurrence it was built in. So a cycle through a `rebuild` alone, in which the dynamic a `rebuild` watches is derived from the `rebuild`'s own event, is a prompt cycle and is rejected:

```
@also_broken:
    !gen <- rebuild $n:
        @ _ + 1                          # the block's result is plain, so the rebuild's event is !Nat
    $n <- 0 then !gen                      # the hold's change event is !gen
    @ unit
```

`$n` changes at n because `!gen` fired; the `rebuild` builds anew at n because `$n` changed; `!gen` fires at n because the `rebuild` built anew. If this were allowed it would rebuild forever within one occurrence. The fix, if something like it is wanted, is to read `$n` through a tag at some event that is not the `rebuild`'s own, or to put a `!Switch` in the loop, which reads its inner from before.

The structure that looks like the cycle above is also the structure of every list program, so `for` needs its own rule. The consolidated event of a `for` over `!A` blocks fires at n with the entries of every block that fired at n, including a block built at n. The edge from a block's event to the consolidated event is prompt. The edge from the collection's change event to a *new* block's event exists only through whatever that block fires in the occurrence it is born, and the only things a block can fire then are pure consequences of its own construction: `@started`, and the events of `rebuild`s inside it. Elements cannot fire in the occurrence they are created, so a button inside the block cannot. The compiler has the block's body and can see whether its result depends promptly on anything that fires at build; if it does not, the collection's change is not promptly upstream of the consolidated event, and the usual list program is legal:

```
@todos:
    !add <- @Button $"Add"
    $next <- 0 then $next + 1 on !add
    let !adds = {$next: Add "item {$next}"} on !add
    !rows <- for $items:
        !delete <- @Button $"Delete"
        @ Delete on !delete
    let !changes = !Union !adds, !rows
    $items <- {} then StepAll $items, !changes
```

(`!Union` merges two maps of firings, the left winning a shared key; `StepAll` folds a map of changes into the items in key order. Both are plain library functions.) `$items` is defined from `!changes`, which comes from `!rows`, which is the consolidation of `for $items`. The row's result is `Delete on !delete`, and `!delete` is a `!Switch` inside `@Button`, which reads before. A row born at n fires nothing at n. The cycle is legal, and would be a compile error with `@ Delete on !started` as the row's result, which is right: that program adds a row and immediately deletes it, in the same occurrence, in which the deletion rebuilds the `for`, which builds nothing, which changes `$items` again. For a `for` whose blocks return plain values or dynamics, the consolidated `$(K ==> A)` changes at n whenever a key appears at n, so the edge from the collection to the consolidation is always prompt and a hold fed from it that feeds the collection is always a cycle.

The check can only see what is in front of it. The todo list is legal because the row's body is written

in the same block as the hold it feeds; the compiler reads `@ Delete on !delete`, finds a switch, and knows the row is silent at birth. Write the same list with the row passed in, `for $items: @Row _` where `@Row` is a parameter of the enclosing widget function, and the body is not visible, so the compiler assumes the worst: a row might fire at birth, the edge from `$items` to `!rows` is taken as prompt, and `$items <- {} then StepAll $items, !changes` is rejected as a cycle even if every `@Row` ever passed is silent. The elaborator records "can fire at birth" on every `for` node, from the body when it can see one and conservatively when it cannot, and the typed AST prints the flag; the Haskell implementation reads it to decide whether the consolidation needs Reflex's `coincidence` over the membership change, which is the one place that combinator is permitted. A program that wants a parameterised row and the closed loop breaks the loop with a delayed edge the compiler can see: the rows feed a hold, and the collection is stepped from that hold through a tag at some other event, so the cycle passes through a read of before-state wherever the rows came from.

**Across widget functions and through switching**, not every cycle can be found statically. `!out <- @Echo !out` is a cycle if `@Echo`'s result depends promptly on its argument and not if `@Echo` holds the argument first, and the block that binds it does not see `@Echo`'s body. A `rebuild` may choose, at runtime, between a block whose result is prompt in some input and one whose result is not, and the cycle exists only when the first is chosen. An implementation must detect these when they occur and treat them as an error: it must not deadlock, must not loop, and must not fire a node twice. In a height-ordered runtime the moment is precise: a node that tries to schedule at or below the current height is a node that was already supposed to have fired, and that is the cycle, reported with the chain of nodes that led to it. What happens next, abort of the occurrence or quarantine of the subtree, is the error chapter's subject; what this chapter promises is that the occurrence in which a cycle is found commits nothing.

A note on what is *not* a cycle, because the form fools people. An interceptor's response event is defined from a `for` over its child's element list, and the child's element list is produced by the `intercept` that consumes the response event. That is a loop in the text. It is legal because routing reads the list as the occurrence opened, which is a tag, and the slots a new input could reach are the ones that existed before it. Hit-testing is the same: a VBox tags a click with its layout as it stood when the occurrence opened, and no cycle exists in the heights. Anything that reads the list to route an input reads before, by the routing rule, and the routing rule is the tag law applied to the list.

## Construction in the middle of an occurrence

`rebuild` and `for` build widgets while an occurrence is in progress. The new widget subscribes to events that may already have fired at n, derives from dynamics that may already have changed at n, and contains holds, `rebuild`s and `@started`s of its own. The question is what it sees, and the answer is one sentence:

**A widget built in occurrence n behaves as though it had been built immediately before n's inputs arrived.**

Everything else is a consequence. The rule binds the runtime and nothing else: a program cannot break it and a World cannot see it being broken except through the transcript. The Haskell implementation gets it from Spider, whose subscriptions read the current frame's value on connection; the Rust runtime must build the catch-up cache sketched at the end of this section. The scripts that check it are `nested_dyn_builds_once` and `hold_resets_and_catches_up` in "Laws as tests", and the differential test between the two implementations catches the half the scripts cannot see, the doubled request firing.

**It sees every firing of n.** An event that fired at n, at any point in the order, is seen as having fired by anything the new widget subscribes to it. A lifted operation over such an event, inside the new widget, fires at n. A hold built at n whose source fired at n adopts that value at commit. If the new block contains `$seen <- Nil then Some !input` and `!input` fired at n before the block was built, `$seen` is `Some` after n. This holds at any depth: an event three combinators down from an input that fired is seen as fired, exactly as it would be by a subscriber that had been there all along.

**Its tags read before n.** A tag inside the new widget reads values before n, like every tag. The new widget is not a privileged observer of the middle of the occurrence; it is an ordinary widget that happens to have been built at a particular moment, and the moment is "before the inputs", so before-state is what it reads.

**A `rebuild` built at n over a dynamic that changed at n builds once, with the new value.** This is the consequence with teeth. Picture a page whose title is derived from the page number, with an outer `rebuild` over the page and an inner `rebuild` over the title:

```
@pages:
    !next <- @Button $"Next"
    $page <- 0 then $page + 1 on !next
    let $title = "Page {$page}"
    rebuild $page:
        @Label $"body of {_}"
        rebuild $title:
            @Element Ui.Label(text=_)
    @ unit
```

A click at n changes `$page` and, promptly, `$title`. The outer `rebuild` builds anew at n. Its block builds an inner `rebuild` over `$title`, which has already changed at n. The inner `rebuild` builds its block once, with `"Page 1"`, because it sees `$title`'s change event as having fired at n. It does not build `"Page 0"` and then rebuild. The label the World sees after n is `"Page 1"` and there was never a `"Page 0"` slot in this generation.

**Every computation runs at most once per occurrence**, regardless of how many late subscribers read it. If the inner `rebuild` and a tag and a hold all read the same derived event at n, the closure behind that event ran once, and all three got its one value.

**Its elements exist from n, and nothing can reach them at n.** A slot the new widget creates is in the element list after n, but no input addressed to it can arrive at n (routing is against the list as n opened), and nothing it requests can answer at n (the creation rule). A state element it creates has, at n, the value before n, which is what the World's snapshot for n holds, and receives n's push like every other slot; so it ends n holding the same value a slot that had existed before n would hold.

**Why the alternatives are wrong.** Each rule an implementation might have adopted instead fails in a way you would otherwise meet in production.

Suppose a late subscriber were hidden from earlier firings: a widget built at n sees only what fires *after* it is built. The nested `rebuild` above is then built after `$title` changed, sees no change, builds with the value its dynamic had before n, `"Page 0"`, and stays wrong until the next time the title changes. The outer page says 1 and the inner title says 0, and nothing in the program is wrong except the runtime. This is the stale nested `rebuild`, and it is the first bug anyone writing this runtime meets, because it is what the obvious implementation does.

Suppose instead that each late subscriber re-ran the closures it needed: the new widget, on subscribing, computes the event's value afresh from its inputs. Now a closure runs once for the original subscribers and once more for each late one. A closure that is pure gives the same value each time,

which hides the problem until the value is used for something that counts, a request fired from a lifted operation that now fires twice, or a cost that is now proportional to subscribers. At-most-once is a law, and this breaks it.

Suppose instead the `rebuild` built with the old value and then, hearing the change, rebuilt in the same occurrence. The World sees one generation after n, so the transcript looks right. But the first build happened: its `@started` fired, and any request it fired from `@started` fired, so the World is handed two requests, one for a page that never existed. Its holds were created and discarded. Its elements were in the list for a moment nobody saw, and its paths were minted and dropped, which chapter 5's path rules make visible in every later path. Two builds in one occurrence cost twice the work and leave a visible trace in the request log, and the request log is what the conformance suite compares.

Suppose instead that construction were deferred: the `rebuild` notes the change at n and builds at n+1, in an occurrence the runtime opens itself. Now the parent's `rebuild` event fires one occurrence late, so a hold on the block's result lags its cause; the element list's change event fires one occurrence after the dynamic that caused it; a World that batched inputs sees an occurrence it did not open; and the runtime has gained a scheduler, a queue, and a budget for how many self-opened occurrences a turn may contain. Every one of these is a mechanism the language was designed not to need, and the only thing bought is a simpler rebuild path.

The rule as stated is the only one consistent with the change law for a widget that would have existed from the start of n, and the change law is not negotiable. It is also the rule Reflex's Spider implements, under the name `subscribeAndRead`: subscribing to an event returns its value for the current frame if it has one, and a `hold` subscribed mid-frame adopts that value at the frame's end. Both halves are in `Spider.hs`: `subscribeAndRead` returns a `SubscribeResult` with the occurrence alongside the subscription, and `hold` queues a `SomeHoldInit` whose initialisation calls `subscribeAndRead` and, when an occurrence comes back, defers a `SomeAssignment` to the end of the frame.

**How an implementation honours it.** The realisation is simpler than the consequences suggest. First, a build is whole before anything in it hears anything: a firing raised while a block is being built, `@started`'s or a `rebuild`'s own, is queued and delivered when propagation runs, in the build occurrence as in any other, so the block cannot observe its own half-built state. Then, every event caches its value for the current occurrence; whoever computes it first records it, and the cache is cleared at commit. Subscribing to an event mid-occurrence returns its cached value if it already fired. A derived event subscribed mid-occurrence connects to its sources, takes any already-fired values, computes its own value once, caches it, and delivers it to its new subscriber. A hold subscribed late defers its adoption to commit. A `rebuild` built late reads its dynamic's value *as of now*: the change event's value if it has fired at n, else the current value, and if the change fires later at n the `rebuild` hears it by the ordinary push. Because every closure is pure and every delayed read is of before-state, a value pulled early is the same value the push would have delivered, and the cache ensures it is computed once however it was asked for. Heights still give the push order; the cache lets a late arrival catch up without disturbing it.

## Determinism

A bug report arrives with a transcript attached: the inputs the World sent, turn by turn, and the element lists it got back. You replay it and the bug replays, on your machine, on the first try, with the same paths and the same requests in the same order. You have never had that. Every bug report you have ever had came with "steps to reproduce" that reproduced on the reporter's machine and not on yours, because the thing that went wrong depended on a timer, a thread, an iteration order,

or which handler was registered first, and none of those was in the steps. Here the transcript of what the World sent is the program's entire input, and replaying it replays the bug.

The property: given the same sequence of input sets and the same state-element snapshots, a program's sequence of element lists and outputs is fully determined. There is no scheduling, no ordering between simultaneous inputs, no clock the program can read except as a state element, no randomness, no identity of widgets that a program could observe. Two runs with the same inputs are the same run.

This is the property the conformance suite tests and the differential test between the two implementations relies on. Every rule in this chapter contributes to it. The tag law removes "which version did I see"; the left bias removes "which handler ran first"; no sampling removes "what instant did I read at"; the construction rule removes "was it built before or after the firing"; no events from nothing removes "when did the timer go off". A function from inputs to outputs is left, and functions are testable.

The one thing determinism does not promise is that two *implementations* compute the same thing the same way. They may differ in order, in what they skip, in how they represent a list. They must not differ in a transcript, and when they do, the sentence in this document that they read differently is what gets fixed.

### What an occurrence is not

An occurrence does not correspond one-to-one to anything outside the program. It is not a GUI callback: a World may put a click and a key that arrived in the same frame into one occurrence, and the program sees them as simultaneous. It is not an iteration of a run loop: a World may run several occurrences for one iteration, because a `Defer` was requested, and the program sees them as several. It is not a network packet or a tick of a clock: a packet becomes a response to a slot and a tick becomes a push to a state element, and each becomes an occurrence only when the World opens one. A World that batched two packets into one occurrence and one that delivered them as two would both be conforming, and the program's text says what each choice means.

An occurrence is also not a thread. There is no occurrence in which nothing happens; the World opens one because it has inputs or because a `Defer` is owed. The program cannot ask for one. There is no idle, no `requestAnimationFrame`, no "next tick". A program that wants to do something later asks an element, and the element answers in an occurrence the World opens when it is time.

And an occurrence is not a unit of cost. Nothing says an occurrence is fast or slow, and nothing limits what one may rebuild. It is a unit of *meaning*: the smallest thing that either happened or did not.

## Laws as tests

A law that only one implementation can check is a description of that implementation. Each law above is restated here as a short program and a Test World script, written against what a World can see: elements sent, elements expected, requests fired, errors reported. Nothing reaches into a widget, because nothing can. The scripts are the form in which the laws enter the conformance suite, and both implementations run every one of them.

The programs share a small vocabulary, declared once. `Ui.Label` and `Ui.Button` are the GUI library's tags, which the Test World accepts directly, as it accepts every tag. The two library widgets are

written out so that nothing is hidden; a program may write `@Element Ui.Button(label=$x)` directly and mean exactly what `@Button $x` means.

```
type Element = choice ..
type Click = click
type Instant = Instant Int            # milliseconds since the World's epoch

tag Ui.Label of Element:
    text: Text

tag Ui.Button of Element:
    label: Text
    responds: Click

tag Clock of Element:                  # state the World pushes; one clock per program
    state: Instant

tag Test.Log of Element:               # a request the Test World records and never answers
    requests: !Text

tag Echo of Element:                   # a request the script answers by hand
    requests: !Text
    responds: Text

@Label: fn $Text -> @unit = \@widget:
    rebuild $_:
        @Element Ui.Label(text=_)
    @ unit

@Button: fn $Text -> @!Click = \@widget:
    !!clicks <- rebuild $_:
        @Element Ui.Button(label=_)
    @ !Switch !!clicks

let Ms: fn Instant -> Int = \|i| match i:
    Instant n: n
```

`!Updated $d` is `$d`'s change event, a plain prelude function; it never fires at build. Two scripts below count its firings, and the count starts at zero.

### The left wins

```
@merge_law:
    !a <- @Button $"A"
    !b <- @Button $"B"
    $last <- "none" then "a" on !a | "b" on !b | !never
    @Label $last

test merge_left_wins, @merge_law:
    build
        => expect Ui.Label(text="none")
    click Ui.Button(label="B")
        => expect Ui.Label(text="b")
    click Ui.Button(label="A"), click Ui.Button(label="B")
        => expect Ui.Label(text="a")
```

Two clicks in one occurrence are the left one. The `!never` on the right is there to pin its identity law in passing: merging it changes nothing.

### The tag reads before

```
@tag_law:
    !bump <- @Button $"Bump"
    $n <- 0 then $n + 1 on !bump
    $seen <- -1 then $n on !bump
    @Label "n={$n} seen={$seen}"

test tag_reads_before, @tag_law:
    build
        => expect Ui.Label(text="n=0 seen=-1")
    click Ui.Button(label="Bump")
        => expect Ui.Label(text="n=1 seen=0")
    click Ui.Button(label="Bump")
        => expect Ui.Label(text="n=2 seen=1")
```

`$n on !bump` and `$n + 1 on !bump` read the same `$n` at the same event, and both read the value before the click. An implementation that read the new value for either would show `seen=1` after the first click.

### A hold has its initial value in its build occurrence, and adopts a firing there

```
@hold_law:
    !started <- @started
    $phase <- "starting" then "started" on !started
    $before <- "?" then $phase on !started
    @Label "{$phase} / {$before}"

test hold_in_build_occurrence, @hold_law:
    build
        => expect Ui.Label(text="started / starting")
```

One step, two facts. `$phase` adopted the firing of `!started` in occurrence zero, so the label shows `started` after the build. The tag on `$phase` at that same firing read `starting`, the initial value, because a tag reads before the occurrence even in occurrence zero.

### Derived dynamics are prompt, change once, and the diamond does not glitch

```
@diamond:
    !set <- @Button $"Set"
    $a <- 0 then 5 on !set
    let $b = $a + 1
    let $c = $a * 2
    let $d = $b + $c
    let !d_changed = !Updated $d
    $fires <- 0 then $fires + 1 on !d_changed
    @Label "d={$d} fires={$fires}"

test diamond_changes_once, @diamond:
    build
        => expect Ui.Label(text="d=1 fires=0")
    click Ui.Button(label="Set")
        => expect Ui.Label(text="d=16 fires=1")
```

`fires` is 0 after the build: `!Updated` never fires at build, and `$d` has not changed. After the click

`$fires` is 1: `$d` changed exactly once at that occurrence, from 1 to 16. An implementation that let `$d` change per input would show `fires=2`; one that lagged `$d` by an occurrence would show `d=1` after the click. The script does not and cannot see the intermediate values 6 and 11. The count is what makes their absence checkable.

## Two readings agree

```
@equal_readings:
    !bump <- @Button $"Bump"
    !other <- @Button $"Other"
    $n <- 0 then $n + 1 on !bump
    $x <- -1 then $n on (!bump | !other)
    $y <- -1 then $n on (!other | !bump)
    @Label "{$n} {$n}"
    @Label "{$x} {$y}"

test readings_agree, @equal_readings:
    build
        => settles
    click Ui.Button(label="Bump"), click Ui.Button(label="Other")
        => expect Ui.Label(text="1 1")
        => expect Ui.Label(text="0 0")
```

Two tags on `$n` at two different events in one occurrence, and two interpolations of `$n` in one label, agree. The second label pins that the order of the merge does not change what the tag reads.

The cross-World half of the law needs a state element, and the script needs a `with` step before the build, because the program creates a `Clock` slot in occurrence zero and the oracle must have something to answer; a state tag the program creates at build with no `with` step before it is a script error, not a default.

```
@Stamp: fn Text -> @unit = \|name| @widget:
    $clock <- @Element Clock
    @Label "{name}={Ms $clock}"
    @ unit

@clock_readings:
    !bump <- @Button $"Bump"
    @Stamp "a"
    @Stamp "b"
    $clock <- @Element Clock
    $stamped <- -1 then Ms $clock on !bump
    @Label "stamped={$stamped}"

test clock_readings_agree, @clock_readings:
    with Clock = Instant 0, build
        => expect Ui.Label(text="a=0")
        => expect Ui.Label(text="b=0")
    push Clock = Instant 5000
        => expect Ui.Label(text="a=5000")
        => expect Ui.Label(text="b=5000")
        => expect_none Ui.Label(text="a=0")
    with Clock = Instant 5100, click Ui.Button(label="Bump")
        => expect Ui.Label(text="a=5100")
        => expect Ui.Label(text="b=5100")
        => expect Ui.Label(text="stamped=5000")
```

Three slots in three widgets that have never met bind `Clock`, and `Clock` has no fields, so the three

are one dynamic; the Test World pushes one value for it and all three labels move in the same occurrence, which the `expect_none` pins. The last three steps are the stopwatch's "before" from the tag law section: a `with` step runs no turn, so the click's turn opens with the new clock as its push and the click as its response, in one occurrence. The two stamp labels show `5100`, because a state element changes promptly like any dynamic, and `stamped` shows `5000`, because a tag reads before the occurrence even when the same occurrence pushes a new value. A World that answered `a` and `b` differently in one occurrence would have broken this program, and this is the script that says so.

## `!Switch` is not prompt, and swallows a block's birth

```
@switch_law:
    !rebuild <- @Button $"Rebuild"
    $gen <- 0 then $gen + 1 on !rebuild
    !!born <- rebuild $gen:
        !started <- @started
        @ _ on !started
    $heard <- -1 then !Switch !!born
    let !built = !Updated $gen
    $built <- -1 then !built
    @Label "heard={$heard} built={$built}"

test switch_swallows_birth, @switch_law:
    build
        => expect Ui.Label(text="heard=-1 built=-1")
    click Ui.Button(label="Rebuild")
        => expect Ui.Label(text="heard=-1 built=1")
```

Each generation fires `@started` in the occurrence it is born, and `!Switch` never hears it, because the switch adopts the new inner at commit and the firing was before commit. The parent still knows the block was built: `!!born`, the `rebuild`'s own event, fired in occurrence zero and again at the click, with each generation's `@started`-tagged event as its value, and a parent that wants to count generations counts that. `built` is `-1` after the build, since `!Updated $gen` does not fire at build, and `1` after the click. `heard` never moves in this program, and that is the law.

```
@switch_follows:
    !rebuild <- @Button $"Rebuild"
    $gen <- 0 then $gen + 1 on !rebuild
    !!clicks <- rebuild $gen:
        !c <- @Element Ui.Button(label="gen {_}")
        @ _ on !c
    $clicked <- -1 then !Switch !!clicks
    @Label "clicked={$clicked}"

test switch_follows_generations, @switch_follows:
    build
        => settles
    click Ui.Button(label="gen 0")
        => expect Ui.Label(text="clicked=0")
    click Ui.Button(label="Rebuild")
        => expect_none Ui.Button(label="gen 0")
    click Ui.Button(label="Rebuild"), click Ui.Button(label="gen 1")
        => expect Ui.Label(text="clicked=1")
        => expect Ui.Button(label="gen 2")
    click Ui.Button(label="gen 2")
        => expect Ui.Label(text="clicked=2")
```

The two-input line is the case that matters: a click on generation 1 in the occurrence generation 1 is replaced is delivered from generation 1, under its own number, because routing is against the list as the occurrence opened and the switch is still listening to the old inner. From the next occurrence the new button is the one heard.

### `$Join` is prompt on the outer and on the inner, including both at once

```
@join_law:
    !swap <- @Button $"Swap"
    !bump <- @Button $"Bump"
    $a <- 0 then $a + 1 on !bump
    $b <- 100 then $b + 1 on !bump
    $use_b <- False then not $use_b on !swap
    !$chosen <- rebuild $use_b:
        @ (if _: $b else: $a)
    $$current <- $ $a then !$chosen
    @Label "{$Join $$current}"

test join_is_prompt, @join_law:
    build
        => expect Ui.Label(text="0")
    click Ui.Button(label="Bump")
        => expect Ui.Label(text="1")
    click Ui.Button(label="Swap")
        => expect Ui.Label(text="101")
    click Ui.Button(label="Swap"), click Ui.Button(label="Bump")
        => expect Ui.Label(text="2")
```

The last step is the hard case. The outer changes (back to `$a`) and the new inner changes (`$a` goes to 2) in the same occurrence, and the join shows 2: the new inner's value *after* the occurrence. An implementation built from a non-prompt switch plus a sample at swap time shows 1, the new inner's value before, and its label disagrees with its holds from then on.

The third case is the outer switching to an inner that does not change in that occurrence, and it has its own script so that a failure names the case:

```
@join_quiet_inner:
    !swap <- @Button $"Swap"
    !bump_a <- @Button $"Bump A"
    $a <- 0 then $a + 1 on !bump_a
    $b <- 100 then !never
    $use_b <- False then not $use_b on !swap
    !$chosen <- rebuild $use_b:
        @ (if _: $b else: $a)
    $$current <- $ $a then !$chosen
    @Label "{$Join $$current}"

test join_follows_quiet_inner, @join_quiet_inner:
    build
        => settles
    click Ui.Button(label="Bump A")
        => expect Ui.Label(text="1")
    click Ui.Button(label="Swap")
        => expect Ui.Label(text="100")
    click Ui.Button(label="Bump A")
        => expect Ui.Label(text="100")
    click Ui.Button(label="Swap")
        => expect Ui.Label(text="2")
```

`$b` never changes: its hold is fed by `!never`. Swapping to it shows its current value, 100, in the swap's occurrence, with no change event from the inner to help; an implementation that only updated the join on the inner's change would keep showing 1. Bumping `$a` while `$b` is chosen changes nothing visible, and swapping back shows the 2 that `$a` reached while it was not being watched: the join reads the inner's current value when it adopts it, and that value is whatever the inner's own history made it. Chapter 5 says how `$Join` is built from a hold of the inner's change events and a prompt read of the inner at the swap; that construction has to pass this script.

## A nested `rebuild` built in the occurrence its dynamic changed builds once, with the new value

```
@nested_law:
    !next <- @Button $"Next"
    $page <- 0 then $page + 1 on !next
    let $title = "Page {$page}"
    rebuild $page:
        rebuild $title:
            !started <- @started
            @Element Test.Log(requests=(_ on !started))
            @Element Ui.Label(text=_)
    @ unit

test nested_dyn_builds_once, @nested_law:
    build
        => expect Ui.Label(text="Page 0")
        => requested Test.Log, "Page 0"
    click Ui.Button(label="Next")
        => expect Ui.Label(text="Page 1")
        => expect_none Ui.Label(text="Page 0")
        => requested Test.Log, "Page 1"
```

The inner `rebuild` is built partway through the click's occurrence, after `$title` has changed. An implementation that hid the earlier change from it would leave `Page 0` in the list, and the `expect_none` catches that. An implementation that built `Page 0` and then rebuilt would fire `Test.Log` twice in that turn, and the turn's transcript, which lists every request firing per slot, shows two where the law says one; the differential test compares transcripts, so the doubled firing is visible even though the script's `requested` step only asks whether the right one fired.

## A hold inside a rebuilt block starts over, and sees the occurrence it was born in

```
@reset_law:
    !rebuild <- @Button $"Rebuild"
    !bump <- @Button $"Bump"
    $gen <- 0 then $gen + 1 on !rebuild
    rebuild $gen:
        $n <- 0 then $n + 1 on !bump
        @Label "n={$n}"
    @ unit

test hold_resets_and_catches_up, @reset_law:
    build
```

```
        => settles
    click Ui.Button(label="Bump")
        => settles
    click Ui.Button(label="Bump")
        => expect Ui.Label(text="n=2")
    click Ui.Button(label="Rebuild")
        => expect Ui.Label(text="n=0")
    click Ui.Button(label="Rebuild"), click Ui.Button(label="Bump")
        => expect Ui.Label(text="n=1")
```

`!bump` is captured from outside the `rebuild`; the hold is inside it. A rebuild makes a new hold at 0. The two-input line builds a new block in an occurrence in which `!bump` has already fired, and the new hold adopts it: `n=1`, exactly what a hold that had existed from the start of the occurrence would show.

### Nothing answers in the occurrence it was asked

```
@creation_law:
    !started <- @started
    !back <- @Element Echo(requests=("hi" on !started))
    $got <- "nothing" then !back
    @Label $got

test no_answer_in_the_request_occurrence, @creation_law:
    build
        => expect Ui.Label(text="nothing")
        => requested Echo, "hi"
    send Echo, "hi"
        => expect Ui.Label(text="hi")
```

The request fired in the build occurrence, from `@started`, which is pure; the label after the build still says `nothing`, because no response can arrive in the occurrence of its request. The script answers by hand in the next turn. This is also the test that there are no events from nothing: without the `send`, the label never changes, however many turns the script runs.

### A prompt cycle formed through a widget function is reported, not run

```
@Echo: fn !Int -> @!Int = \|!in| @widget:
    !started <- @started
    @ (!in + 1 | 0 on !started)

@runtime_cycle:
    !out <- @Echo !out
    @ unit

test cycle_is_an_error, @runtime_cycle:
    build
        => errors Error(kind=Cycle, ..)
```

The block that binds `!out` cannot see that `@Echo`'s result is prompt in its argument, so the compiler lets it through. In occurrence zero `@started` fires, `!out` fires with 0, `!in + 1` fires with 1, which is `!out` again, in the same occurrence. The runtime reports a cycle and the build turn ends in an error: one `Error` value with `kind=Cycle`, the spans of the nodes in the chain, and the occurrence number. What the World does with it, an aborted occurrence or a subtree in quarantine, is the error chapter's.

The step says only that the turn ended in a cycle error and that the implementation did not hang.

A cycle of the same kind inside one block is not a test in this suite, because it never runs: `let !a = 1 + !b` with `let !b = 2 + !a` is rejected by the compiler with both bindings named, and the conformance suite's compiler corpus holds it as a program that must not compile.

### Determinism

Every script above is a determinism test twice over. Each is run under both implementations and the transcripts compared turn by turn, and each is run more than once under each, which is cheap to do and catches the implementation that depends on an iteration order of a hash map or on the address of a widget. A script that passes once and fails once has found a bug in an implementation; a script whose two implementations disagree has found a bug in one of them or an ambiguity in a sentence, and the second kind of find is the more valuable.

## The negative space

Four things are missing from this part of the language, and each absence is a feature. The pattern is the same each time: the thing you would have sworn was essential is removed, the removal closes a hole you have fallen into, and the same removal lets an implementation leave out a mechanism it would otherwise need. The four are stated together here because they are the four that people ask for first, and because their reasons overlap.

### No sampling

No `sample`, no `.current`, no plain function from `$A` to `A`; a dynamic is read through a tag, through `rebuild`, or by derivation, and "No sampling, and what it buys" above has the reasoning. What it closes: the read in the middle of the cascade. What it buys the program: every read of state is visible as a tag. What it buys the implementation: freedom of order, and no definition of "a sample in the middle" that every later optimisation would have to honour.

### No prompt switching

`!Switch` hears a new inner from the next occurrence; there is no `coincidence`, no `switchPromptly`, and no way to write one, for the reasons given under "`!Switch`" above. What it closes: the cycle that exists only when a particular inner is chosen, and the re-entrant propagation that handling it needs. What it costs: a block cannot fire at its parent through the switch in the occurrence of its birth, and the parent already knows of the birth from the `rebuild`'s event. What it buys the implementation: heights fixed as the occurrence opened, one pass, no re-entry, no second node type for switching.

### No events from nothing

No `!always`, no `emit`, no `setTimeout`, no `requestAnimationFrame`, no way for a Prism program to cause an occurrence or to fire an event that is not a consequence of an input or of construction.

What it closes: the ambient channel. A widget that could fire on its own could observe or affect things it was never handed. A widget that could observe every occurrence could learn about siblings it has never met. A program that could schedule its own occurrences would have a scheduler, with priorities and starvation and a cap, written somewhere in the runtime, and the program's behaviour would depend on it.

What it buys the program: a transcript is a test. The World's inputs are the program's entire input, and replaying them replays the run.

What it buys the implementation: one entry point. The runtime runs one occurrence when asked and returns; it never opens one on its own, keeps nothing queued, and has no thread, no timer, and no wakeup. A World that wants something to happen later owns the timer, and that is the World's business, in the host language, where timers belong. The runtime is a function from "this input arrived" to "this occurrence happened", and functions are the thing both implementations know how to test.

### No clock

There is no `now()`. Time is a state element, `Clock`, like the contents of a file: it has a value in the occurrence its slot is created and the World pushes a new one when it chooses. A program that wants to be woken asks a `Time.After` or `Time.Every` element, and is woken in an occurrence the World opens.

What it closes: the untestable debounce. Anyone who has tried to unit-test a timeout with a real timer knows what a clock that a script can set is worth. It also closes the program whose behaviour depends on how long an occurrence took to compute, which is a program whose bugs appear only under load.

What it buys the program: `Step $watch, $clock, !action` has every input it needs in its arguments. The stopwatch's state machine is a plain function, and it runs identically under a GUI World, a server World and a test that says `push Clock = Instant 5100`.

What it buys the implementation: nothing in the runtime knows what time it is, so nothing in the runtime has to be told, and two implementations cannot disagree about it. Determinism would be false the moment a program could read a wall clock; with the clock as data, it is a theorem.

### What the four have in common

Each is a refusal to let a program observe something about *when* it is running: the instant within an occurrence, the occurrence after the one it is in, an occurrence it did not cause, the wall-clock time. Only what the laws define remains observable: before-values, firings at this occurrence, and the values the World pushed in. The laws are small because the observables are few, and the observables are few because these four doors are shut.

## How this feels in practice

The laws are the whole language, and after a few programs they stop being laws and become habits. Four of the habits, each small, each something you will write in the first hour, with the law it leans on named once.

### Fold with hold

All state has the form of the counter's line:

```
$count <- 0 then $count + !delta
```

A fold is a hold whose change event reads the hold through a tag. The initial value on the left, the step on the right, the dynamic on both sides. Anything a `reduce` can express is this line with a

different function:

```
$total <- 0 then $total + !amount
$history <- [[]] then Append $history, !line
$state <- Idle then Step $state, !action
$best <- Nil then Better $best, !score            # Better: fn ?Int, Int -> ?Int
```

When more than one event moves the state, the merge picks who wins when they coincide, and the hold is still one line:

```
$count <- 0 then 0 on !reset | $count + !delta
```

A reset in the same occurrence as an increment is a reset, and the next person to read the line knows it. When two events should *both* apply, the merge is wrong and the right tool is a map through `for`'s consolidation or a library `!Union`, folded in key order, as in the todo list; the hold is still one line, and the step function says what it means to apply several changes at once.

When the step needs another dynamic, it goes in the operation and is read through a tag:

```
$watch <- (Watch 0, Stopped) then Step $watch, $clock, !action
```

The clock is the clock before the occurrence. If the same occurrence pushes a new clock, the stamp is still the old one; that is the tag law, and a program that wants the newer time cannot have it, because at an event there is one version.

The habit to acquire: when you want state, write the hold first and let its line be the complete answer to "what can change this". If the line is getting long, the step is a function with a name; if the merge is getting long, the events want a `choice` type and one merge of tagged actions, like `Start on !start | Stop on !stop | Reset on !reset`.

## Gating with `on`

`$x on !e` is "the value when this happens", and most of what people do with `sample` is this:

```
let !submitted = $text on !enter              # the text, as it stood, when Enter was pressed
let !chosen = $items[$cursor] on !choose      # an optional, since the index might be out of range
let !stamped = (\|k, t| ((k, t))) !key, $clock  # the key that fired, paired with the clock before it
```

The third line is the one everyone first writes as `(($clock, !key))`, which is a plain tuple holding a dynamic and an event, and does not fire. The sigil's position says what moves, and in a tuple nothing does.

To drop firings on a condition, lift a function that returns an optional and filter:

```
let !When = \|!e, $open| !Filter ((\|x, g| if g: Some x else: Nil) !e, $open)
let !accepted = !When !submitted, not $busy
```

`!When` takes an event and a dynamic and returns an event, so its name has the `!`: a function's sigil is a promise about its result, and this one's result is the `!Filter`. It is not a plain function lifted over its arguments; a plain function in that position would be lifted over `!e` and would build an `!Filter` inside every firing, which is an event of events and not `!accepted`.

`$busy` is read through a tag, so a submit in the same occurrence as the response that clears `$busy` is dropped: the program was busy before that occurrence, and the submit sees before. If that is the wrong answer for your program, the fix is visible: fold `$busy` from a merge in which the clearing wins, and gate the *next* submit, or make the gate a dynamic the response sets promptly and read it one

occurrence later. Either way the choice is in the text.

A gate that depends on the event's own value is just a function: `!Filter (\|n| if n < 10: Some n else: Nil) !n` keeps the small ones. A gate on a dynamic and the value together is one lifted function with both as arguments. There is no separate `when`, `gate` or `filterWhen` to learn; there is lifting, `on`, and `!Filter`.

## Debouncing with a Time element

You want to search as the user types, but not on every keystroke: wait until they have stopped for three hundred milliseconds. In every framework this is a timer you cancel and restart, and the cancel is where the bug lives, because the callback that was already in flight when you cancelled it still runs. Here there is no cancel, because there is no callback. A timer is an element that answers later, and "later" is an occurrence in which the program checks whether the answer is still wanted.

```
tag Ui.TextInput of Element:        # the GUI library's text input; the Test World sends it the whole text
    name: Text
    responds: Text

tag Time.After of Element:
    delay: Int                      # milliseconds
    requests: !Nat                  # a token; the World answers with the same token after `delay`
    responds: Nat

@Debounce: fn Int, !a -> @!a = \|delay, !in| @widget:
    $gen <- 0 then $gen + 1 on !in
    !back <- @Element Time.After(delay=delay, requests=($gen + 1 on !in))
    $latest <- Nil then Some !in
    let Settle = \|token, gen, latest| if token == gen: latest else: Nil
    @ !Filter (Settle !back, $gen, $latest)
```

Every input increments a generation and asks the timer for that generation back. When a token comes back, it is compared with the generation as it stands; if a newer input has moved the generation on, the token is stale and is dropped; if not, the latest input is released. `$gen + 1 on !in` reads the old generation and requests the new one, which is the number the hold will have after the occurrence; `Settle !back, $gen, $latest` reads both holds before the occurrence the token arrives in. There is no cancellation. Stale timers go off and are ignored: every token requested is answered, the World owes an answer to each one it accepted, and the program's three lines decide which answers matter. The only tokens the World does not answer are the ones addressed to a slot that no longer exists, because the `@Debounce` that asked was torn down; the World performs the timer anyway and discards the result, under its obligation to discard late results for gone slots. Nothing is cancelled anywhere, and nothing leaks, because a timer is a request the World owns and a slot that is gone is a slot the World knows is gone.

`Time.Every` is the other timer, and it is not a debounce. It is a stream element that ticks once per period for as long as the slot exists; it takes no request and answers no token, so there is nothing to compare a tick against, and a tick that lands while the program is busy is skipped rather than queued. A debounce needs "later than *this* input", which is a request, and a request is `Time.After`.

Occurrence by occurrence, with a 300 ms delay: a keystroke at occurrence 4 requests token 1 and `$gen` commits 1. A keystroke at 5 requests token 2, `$gen` commits 2. The World answers token 1 at occurrence 6: `$gen` before 6 is 2, the token is 1, `Settle` returns `Nil`, nothing is released. The World answers token 2 at occurrence 9: `$gen` is 2, the token is 2, `$latest` is released. If a keystroke and a token arrive in the same occurrence, the token is compared against the generation before the

keystroke and the keystroke's own input is in `$latest` only from commit, so the release, if any, is the previous input and a fresh timer is requested for the new one. Every case is decided by the tag law and nothing else.

```
@debounced_search:
    !typed <- @Element Ui.TextInput(name="query")        # the text as it stands after each edit
    $query <- "" then !typed
    !settled <- @Debounce 300, !typed
    $shown <- "" then !settled
    @Label "searching: {$shown}"

test debounce_waits_for_the_last, @debounced_search:
    build
        => settles
    send Ui.TextInput(name="query"), "a"
        => requested Time.After(delay=300), 1
    send Ui.TextInput(name="query"), "ab"
        => requested Time.After(delay=300), 2
    send Time.After(delay=300), 1
        => expect Ui.Label(text="searching: ")
    send Time.After(delay=300), 2
        => expect Ui.Label(text="searching: ab")
```

The test owns the timer. It can answer the tokens in any order, together, or never, and each choice is a case the program handles by the same three lines. Under a real World the text input is the library's `@TextInput` over `Ui.TextInput`, fed by `Gui.Window`'s key events through the library's focus routing; the debounce does not know and does not need to. This is the "no clock" absence paying for itself: a debounce is testable without waiting, and the program that was tested is the program that ships.

## Accumulating a map

A stream of words, and you want a count per word, shown as a list that updates in place:

```
tag Words of Element:
    responds: Text

let Bump: fn Text ==> Nat, Text -> Text ==> Nat = \|m, w| match m[w]:
    Some n: Insert m, w, (n + 1)
    Nil: Insert m, w, 1

@word_counts:
    !word <- @Element Words
    $counts <- {} then Bump $counts, !word
    for $counts:
        @Label "{key}: {$_}"
    @ unit
```

The hold is the fold-with-hold habit over a map: one line, one event, one step function. The `for` builds one label per key. Inside the block, `key` is the word and `$_` is the count *as a dynamic*: when a word that already has a row is counted again, the row's `$_` changes and its label rebuilds, and the row itself, its widget, its path, any hold it might have, stays. A new word is a new key and a new row, built in the occurrence the word arrived. A word never disappears here, but if `Bump` removed keys at zero, the rows would be torn down at that occurrence's commit.

This program uses a map because the key is an identity. An array of pairs would make every row's

position its identity, and a new word sorting into the middle would rebuild every row after it. Keys are ordered, so the labels appear in word order, which is what a map is for.

Two refinements that are both one line. Several words in one occurrence, if the element delivered chunks: make the response a `[Text]` and the step a fold over it, `$counts <- {} then Fold !words, $counts, Bump`, where `Fold` is the plain function and `$counts` is read once, before. And a per-row piece of state that is not in the model, a highlight toggled by clicking the row: a hold inside the `for` block, owned by the key, which survives every other key's arrival and departure and dies with its own:

```
let Mark: fn Bool -> Text = \|m| if m: " *" else: ""

    for $counts:
        !click <- @Button $"{key}"
        $marked <- False then not $marked on !click
        @Label "{key}: {$_}{Mark $marked}"
```

`$"{key}"` is a constant dynamic of the key, which `@Button` takes as its label. Nothing about `$counts` changed to make room for it. That is the whole experience of the language in miniature: state is a hold, it lives where it is written, and the structure of the program is the structure of the data.

## Chapter 5

# Widgets and switching

A widget is the one thing in Prism with a lifetime, and this chapter is about what that buys. Events and dynamics describe the whole of time and never need to be born or die; a widget is built in one occurrence, lives through some, and is torn down in another, and so every piece of the language that needs a beginning and an end (state, IO, identity, cycles, subscriptions) is owned by one. Three constructs let a program change its own structure while it runs, `rebuild`, `for` and the lifting that quietly spells `rebuild` for you, and two primitives, `!Switch` and `$Join`, bring results across the boundary of a part that was replaced. The underlying claim is that one ownership tree, with rebuilds as ordinary consequences of dynamics changing in the occurrence they change, replaces reconciliation, lifecycle and garbage collection at once.

## What a widget is

You have had the afternoon where the subtree was gone but its listener was not. A panel was removed from the screen; a timer it had registered kept firing; the handler read a model that no longer existed and threw, or worse, wrote to one that did. Or the other afternoon: a component was unmounted and remounted by a parent's re-render, its `componentDidMount` ran again, and the server got a second request for the same data, which arrived after the first one and overwrote a newer value with an older one. Both are the same bug. Something with a lifetime was treated as though it did not have one. The listener outlived its panel; the request outlived its component; nobody owned either, so nothing could end them.

Prism's answer is that exactly one kind of thing has a lifetime, and everything that needs one is owned by it. An event is a partial function from occurrences to values. A dynamic is a value at every occurrence. Neither is created and neither is destroyed; they are descriptions of all of time, and when a runtime drops one it is pretending for efficiency, not changing the meaning of a program. A widget is different in kind. It is built, at a position in a tree, in some occurrence; it exists for a while; and it is torn down when its parent replaces or removes it. The sigil `@` marks the things that are built, and the list of what a widget does is the list of everything in the language that is not pure computation.

**Widgets do IO.** All IO is elements, and an element is created by a widget with `@Element`. The slot lives as long as the widget that created it and not one occurrence longer. When the widget is torn down the element leaves the list, the World sees it go, and whatever the World was doing on its behalf is cancelled or discarded. A file read, a socket, a button, a timer: each is a widget, because each has a beginning and an end, and the thing you reached for in every other framework, a handle you could stash in a field and forget to close, does not exist.

**Widgets own holds.** `$x <- v then !e` is a statement in a widget block and nowhere else, because the hold's value has to belong to something that can be torn down. When a `rebuild` builds anew its block the state inside it resets; when a `for` removes a key the state of that key's block is freed. A hold outside any widget would be state that nothing owns, which is a leak by construction, and the language refuses to write one.

**Widgets own their cycles.** `$count <- 0 then $count + !delta` refers to itself. Something has to hold the placeholder that stands in for `$count` before its definition is complete and the closure that completes it, and that something is the widget. A cyclic subgraph is then reachable only through its widget and dies with it, rather than keeping itself alive by referring to itself, which is the one kind of leak reference counting cannot collect and the one the ownership tree makes impossible.

**Widgets own subscriptions.** Every occurrence-time binding in a block, every `$count + !delta`, is a node in the propagation graph subscribed to its inputs. The subscription is held weakly from the input's side and owned by the widget from the subscriber's side. When the widget is torn down its graph nodes stop being reachable and the inputs stop delivering to them. There is no `unsubscribe` in the language and no `removeEventListener` to forget; there is teardown.

**Widgets have identity, and blueprints do not.** Building the same blueprint twice gives two widgets, two paths, two sets of slots, two sets of holds. That is what makes two buttons with the same label two buttons. Events and dynamics have no identity at all: an event used in two places is one event, and asking whether two dynamics are "the same one" is not a question the language can pose.

**Widgets are the GC story.** The whole graph hangs off the widget tree. The tree is owned top-down, from the World's root widget through every `<-` bind, every `rebuild` and every `for`. A widget is torn down when its parent replaces or removes it; teardown is recursive; and because every graph object is owned by exactly one widget, there is nothing to collect that the tree does not reach and nothing the tree reaches that is not owned. A result event of a torn-down widget, still held by some parent, simply never fires again; a result dynamic keeps its last value. The replaced widget is kept until the end of the occurrence in which it was replaced, so that anything it fired in that occurrence is delivered, and is dropped at commit. The Rust implementation needs no tracing collector: ownership of widgets is a tree, and reference counting handles the sharing of plain values.

## Blueprint and widget

Two words, for two things that every UI framework blurs. A *blueprint* is what you write: `@counter:` and the block under it, or `@Button $"OK"`, or whatever a widget function returns. It has the type `@a`, and it is a value, so you can store it with `let`, hand it to a function, get it back from another blueprint, and bind it as often as you like. A *widget* is what a bind builds from it. Bind the same blueprint twice and you get two widgets, each with its own path, its own holds, its own slots, and no way to be confused with the other. Widgets never alias.

Why not "recipe"? Because once a dish is cooked, the recipe's steps are gone from it, and that is not how a widget relates to its blueprint. The bindings in the blueprint are the widget's wiring for as long as the widget stands, and the closures in it are the widget's working parts. A blueprint stays in correspondence with what was built from it, the way a building's blueprint does; that is the word.

The runtime chapters add a third word, *node*, for the widget seen from inside: the data structure with a path, a child list and a lifetime. Same thing, other side of the wall.

```
@two_buttons:
    let @b = @Button $"OK"
    !x <- @b
    !y <- @b
    @ ((!x, !y))
```

Two buttons, two paths, two click events, and `@b` is neither of them. If you have used a UI framework where a component instance was a value you could hand around, this is the thing to unlearn: `@b`

cannot be clicked, because `@b` is not on the screen. Only its widgets are, and a widget is not something you can name.

A widget is not a value, and the reason is not a rule but a fact about what it is. It has a lifetime, it is wired to its neighbours, it is torn down, and two built from one blueprint are two; nothing with those properties is referentially transparent, so a pure language cannot hold one. You cannot put a widget in a variable, pass one to a function, put one in a map, or compare two. The only way to get a widget is to bind a blueprint in a hierarchy, and what you get is not the widget but its *result*, the event or dynamic or bundle that the widget block's last statement produced. The widget itself is a position in the tree, known to the runtime, and the program talks about it only implicitly, by being written at that position.

Suppose widgets were values. Then a program could put one in a hold, and the parent's decision to remove it would mean nothing: the widget would live on, its slots would stay in the element list, its subscriptions would keep delivering, and ownership would be a graph rather than a tree. Every runtime whose widgets are values ends up with a collector, because a tree with back-references is not a tree. Every program in such a runtime ends up with the leaked subtree. Refusing widget values is what makes teardown a `drop` and the GC story a sentence.

So, in this book: a hold belongs to a widget, a path names a widget, teardown happens to a widget, and what gets bound is a blueprint. "Widget block" and "widget function" keep their names, because the block describes a widget and the function is about one, the way a cookie recipe is about cookies; what the function returns is a blueprint, and the sentences that need to say so, say so.

## Building is the only doing

A widget block does not run. It is built, once per widget, and reading it top to bottom is reading a blueprint: these children, these holds, these graph edges, these slots. The recipe's only effects are on the tree, never on the world. The world is told about the tree afterwards, through the element list, and does what it does with that. In this sense a Prism program is pure even though widgets do IO: a widget does not perform IO, it *is* IO, in the way a function type is a computation rather than the performing of one, and the World is the only thing that performs.

This is what makes the question "what happens when this widget is built" always have the same answer. Nothing happens. Slots appear in a list. If the program wants something to happen, it fires a request event, and `@started` exists so that a widget can fire one in its build occurrence; the response arrives later, from outside. There is no code path by which construction reaches the world, and so there is no ordering problem between construction and the world, no fetch-on-mount that fires twice, and no test that needs a mock installed before the first render.

## Why the archetype is a GUI, and why the word stops meaning that

A button is the smallest thing that has every property above at once. It has a lifetime (it is on the screen or it is not). It has a little state (pressed). It has an element in the world (something to draw, something to click). It has an identity distinct from its twin (two OK buttons are two buttons). And it has a parent that can remove it. Once a button is understood as "the thing with all of these", the next step is noticing that a socket has all of them too: it is opened and closed, it has state (connecting, connected), it has an element (an address the World dials), it has identity (two connections to the same host are two connections), and it has an owner that can drop it. So does a file read in flight. So does a timer. So does an entire application hosted by a shell.

That is why the sigil is called "widget" and why, a few pages in, the word has nothing to do with

screens. Anything with a lifetime is a widget; everything without one is a value. The claim of the language is that this division is universal, and the test is whether anything ever needs to be neither. Nothing in this book is.

### What a widget owns

A widget holds its children, its holds, its cyclic placeholders, its element slots (each with a path, an element value, a response trigger, a state dynamic and a request event) and the graph nodes its occurrence-time bindings compile to. The target widget has no backend channel: no public event table, no `backend_data`, no delegate. The run call is the only boundary, and a widget is told nothing and tells nothing except through the element list.

Ownership points one way, parent to child, through `Rc`. Subscriber lists in the graph hold `Weak` references. A closure compiled from an occurrence-time expression captures the graph objects it reads through the widget that owns it, as indices into that widget's environment, never as a strong pointer to a sibling's shared object. Those three rules together are the whole reason there is no collector: the tree owns the widgets, the widgets own their closures, the closures reach the graph through the tree, and nothing points back up. The runtime is bound by all three and the compiler by the third; chapter 11 has the `WidgetBuilder` that makes the first two true by construction and the test, `dropping_a_widget_frees_its_cyclic_loops`, that checks the one place a widget could own itself.

## Widget blocks

Widget blocks are where "one way to change anything, written next to the thing" becomes syntax. A hold is a statement that may appear only here, so every piece of state in a program lives inside some widget, owned by its widget, with its definition naming every event that can move it. When you want to know what can happen to `$count`, you read the line that defines it and you are done.

A widget block is the body of a widget. `@counter:` at top level defines a widget named `@counter`; `let @counter = @widget:` is the same thing; `\@widget:` is a lambda whose body is a widget block, which is how widget functions are written:

```
@Button: fn $Text -> @!Click = \@widget:
    !click <- @Element Ui.Button(label=$_)
    @ !click
```

A widget block is a Reflex `do` block with the monad left out: the statements are the sequence, `<-` is bind, `@` is `return`, and there is no `m` to name because there is only one. `@Button` takes a `$Text`, and the dynamic in the `label` field is what makes the button follow its label: the library's `@Button` is where the `rebuild` lives, so a caller writing `@Button "Count: {$count}"` has no lift at the call site. The section on lifting over `@` spells out what that `dyn` is.

### Every statement form

The forms, in the order a reader meets them, each with what it builds:

`pattern <- blueprint` builds the widget as a child of this block's widget and binds its result to the pattern. The pattern is usually a sigiled name. It may be a tuple or record pattern over a bundle:

```
((!lines, !eof)) <- @Lines
TextInput(!changed, $value) <- @TextInput "hello"
```

`blueprint` alone, as a statement, builds it and discards the result. `@Label "Count: {$count}"` on a line by itself is the common case; `@Element Time.Every(period=100)` is another, where the element exists to make the World come back and nobody wants the tick.

`pattern <- @Element e` is the bind form of the one widget that reaches outside: it creates a slot in this block's widget, with a path under the widget's, and its result is whatever the tag declares, `!Response`, `$State`, a pair of both, or `unit` for a presence element. `!click <- @Element Ui.Button(label="OK")` and `$args <- @Element Process.Args` are the two usual spellings; in a catch-all arm of an interceptor, where the tag is not statically known, the result is the generic `((!resp, ignore)) <- @Element _.element` and the types are checked per slot at runtime. The slot lives as long as the widget and the element's fields are fixed for the slot's life; a dynamic in a field lifts the whole `@Element` into a `rebuild`, which the lifting section below covers.

`pattern <- intercept @child, !responses, $supplier` binds an interceptor, which is a widget like any other: it builds `@child` in its own widget and its result is the pair of the child's result and the child's element list, `((result, $list)) <- intercept @inner, !down, $supply`. Everything about what an interceptor can do with the list is chapter 6's; here it is one more bound statement whose child is owned by this widget.

`let pattern = expression` binds a value of any kind without building anything. `let !delta = -1 on !decrement | +1 on !increment` names an event; `let $label = "Clicked {$count} times"` names a derived dynamic; `let @b = @Button $"OK"` names a blueprint, which is a value until bound. A `let`-bound blueprint can be bound later any number of times, and builds that many widgets.

`$x <- v then !e` is a hold. Its value before the first firing of `!e` is `v`; its change event is `!e`; its value after any occurrence in which `!e` fired is that firing's value. In the occurrence the block is built, `$x` is `v`, and if `!e` fires in that same occurrence `$x` changes to it then, which is what makes `@started`-driven initialisation behave the way it looks. Holds are statements, not expressions, because a hold is owned, and the `<-` makes the owner visible: the widget whose block this is.

The last statement is the block's result. If it is a widget, the block's result is that widget's result. `@ v` makes a plain value the result; `@ ((!a, $b))` returns a bundle; `@ !never` is a block that returns an event which never fires, which the modal pattern below uses.

`match v:` with a blueprint in its arms is a statement when `v` is plain and every arm is a widget block, and it chooses which of several blueprints this widget builds. It is legal only where the scrutinee is plain, which inside a `rebuild` or `for` block means the value the block was built for: `match mode: Browse: @BrowsePane … Edit: @EditPane …` under `rebuild $state:` is the common form, and `match step:` in the wizard below is another. A `match` over a dynamic with a blueprint in an arm is rejected, for the reason the lifting section gives: choosing a blueprint by a changing value is construction, and construction over a dynamic is spelled `rebuild`. `if c: @a else: @b` with plain `c` is the two-arm case of the same statement.

Two more things are statements in a widget block and are the subject of the rest of the chapter: `rebuild $x:` and `for $m:`, each with a block of its own, each bound with `<-` like any blueprint, because each is one.

## Recursive bindings and groups

Bindings in a widget block are mutually recursive: a name may be used before the statement that binds it. The counter uses `$count` on the right of its own definition, and the todo list of the worked examples defines `$items` from `!changes`, `!changes` from `!rows`, and `!rows` from `for $items`. The restriction is the one time imposes: the cycle has to pass through a delayed edge, a tag, a `!Switch`'s

hold of inner events, or a hold's value side. A cycle of prompt edges alone is a compile error that lists every binding in it. A `let`-bound function may call itself or its neighbours, because a call is not a cyclic value. Other plain values cannot be cyclic at all; a cycle among `let`s of plain non-function values is an error, because there is no occurrence for the cycle to be resolved across.

An implementation resolves the block into strongly connected groups and builds them in dependency order. A group with a cycle is built with placeholders for the forward references, which are closed once the group is built. The runtime's builder exposes this directly: `add_cyclic_event(name)` returns a forward event that fires when the event it is later closed with fires, and `close_cyclic_event_by_name(name, event)` closes it; `add_cyclic_dynamic` and its close are the same for a dynamic whose behavior is a computation that reads through the placeholder once set. The runtime's own `fold_dyn` is written this way and is a good picture of what the compiler emits for a self-referential hold:

```
(handle, both_in) = add_cyclic_event(name)
new_value         = both_in.filter_map(|(old, update)| f(old, update))
dynamic           = Dynamic::hold(initial, new_value)
both_out          = update.tag_map(|update, old| (old, update), dynamic)
close_cyclic_event_by_index(handle, both_out)
```

The hold reads its own old value through a tag, the tag is a delayed edge, and the placeholder is closed before the builder returns. All of this is invisible to the program: a widget block is a set of equations, and reading it top to bottom is one valid order.

The placeholder is why the widget owns its cycles rather than the graph owning itself. The `ForwardEvent` and the set-once cell live in the widget's `cyclic_events` table; the closed event refers to the placeholder, the placeholder refers to the closed event, and if the widget did not hold both the pair would hold each other forever. The test `dropping_a_widget_frees_its_cyclic_loops` is the check that it does not.

## Build time and occurrence time

Every binding in a widget block is one of two things, and the compiler marks which. A *build-time* binding runs once, when the widget is built: binding a child widget, a hold's initial value, a `let` of a plain value, the construction of an element's fields. An *occurrence-time* binding is a node in the propagation graph and runs whenever its inputs fire: anything lifted over an event or a dynamic. The sigils decide:

```
@report: fn Data -> @unit = \|data| @widget:
    let summary = Summarise data            # once, at build
    let $live = Summarise $feed             # once per change of $feed
    let !each = Summarise !chunk            # once per firing of !chunk
    @Label "{summary} / {$live}"
```

`Summarise data` with plain `data` runs once per build of `@report`, and never again, because nothing in it can change. `Summarise $feed` is a derived dynamic, recomputed in the occurrence `$feed` changes. `Summarise !chunk` runs per firing. There is no annotation to get wrong and no dependency array to maintain: the kinds of the operands are the dependency list.

This matters because people expect `let` to do something it does not. A `let` in a widget block is not a statement that executes in sequence each time "the component renders"; there is no render. If `data` is expensive to summarise, it is summarised once per widget, and if you want it summarised again you need a new widget, which is what `rebuild` makes.

## Results and bundles

A widget's result is any value: an event, a dynamic, a plain value, a function, a blueprint, or a record of several. A record with sigiled fields is the bundle type:

```
type TextInput = record:
    !changed: Text
    $value: Text

@TextInput: fn Text -> @TextInput = \|initial| @widget:
    ...
    @ TextInput !changed, $value
```

Returning a blueprint from a widget (`@@A`) is legal and means the inner one is not built; the caller gets a blueprint and may bind it, which is how a widget function can return a blueprint for its caller to place. Returning a function is legal and plain. Results are read by the parent in the occurrence the child is built, and the rules of time say what they hold then: a result dynamic has the value the child's block gives it in that occurrence, a result event fires in that occurrence if the block's definition makes it fire, which under the creation rule means only if something pure like `@started` drives it.

## Blueprints as arguments

A widget function can take a blueprint, and the sigil on the parameter says so:

```
@Framed: fn Text, @a -> @a = \|title, @child| @widget:
    @Label $title
    result <- @child
    @Label $"end of {title}"
    @ result
```

`@child` is a blueprint handed in; `result <- @child` binds it, which builds it as a child of `@Framed`'s widget, between the two labels, with a path under `@Framed`'s. The caller wrote the recipe and `@Framed` chose where the dish goes. This is how containers are written, and it is the whole mechanism behind `intercept`, whose first argument is a blueprint for the same reason: the interceptor builds it, in its own widget, and so owns it.

Two things follow from a blueprint argument being a value. `@Framed "A", @counter` and `@Framed "B", @counter` build two counters, two widgets, two sets of holds; the blueprint was shared and the dishes are not. And a widget function that takes a blueprint and never binds it builds nothing of it: `@Discard: fn @a -> @unit = \|@child| @ unit` is a legal program in which the child never exists. A blueprint passed to a function is not a child until it is bound; the bind is the ownership.

A widget function cannot bind the same argument twice and get one thing. `!a <- @child` followed by `!b <- @child` is two children. If the function wants one child and two views of its result, it binds once and names the result. If you find yourself wanting to pass a *widget* rather than a blueprint, so that two containers could share one built child, that is the widget value the language refuses, and the reason it refuses is that neither container would own it.

### `@main`

The program is a widget `@main` taking no arguments. The World builds it in occurrence zero. Its result type is the World's business: a test World may require `@unit` or a bundle it can assert on; a CLI World ignores it. Arguments and environment are not parameters of `@main`. They are state elements, `Process.Args` and `Process.Env`, with a value in the occurrence they are created, and the reason is the

same reason the clock is a state element: they can change while the program runs, and an interceptor can confine or fake them for a subtree. A sandbox that gives its child a different argument list is a supplier function; a `@main` with parameters could not be sandboxed that way without a second mechanism.

### `@started`

`@started: @!unit` is a widget whose result fires once, in the occurrence in which it is built, and never again. It is pure, and it is definable in the language:

```
@started: @!unit = @widget:
    !once <- rebuild $ unit:
        @ unit
    @ !once
```

A `rebuild` over a constant dynamic builds its block once and fires its result event in that build occurrence; the block returns `unit`; the `rebuild`'s event is an `!unit` that fires exactly then. The prelude provides `@started` as a primitive because it is used everywhere, and the runtime implements it as a filter over the builder's done event rather than by building a `rebuild`, but it needs no semantics of its own.

The conformance suite checks the law, not the implementation: `@started` fires once, in the occurrence the widget is built, after the block is built, and never again. The two spellings have to agree on the one observable that follows from "after the block is built": the firing lands at `height + 1` of the build, so a hold in the same block driven by it, `$x <- v then w on !started`, changes in the build occurrence, and anything in the block that reads `$x` through a tag in that occurrence sees `v`, while anything derived from `$x` sees `w`. A runtime whose done event fired at the build's own height, or in the next occurrence, would pass a test that only asked "did it fire once" and fail the one that holds on it; chapter 10's law test for `@started` holds on it.

Its use is kicking off a request as soon as a widget exists:

```
!started <- @started
!resp <- @Element Fs.File(path=path, requests=(Read on !started))
```

The request fires in the build occurrence; the response arrives in a later one. That is the whole of "load on mount" in Prism, and the difference from the hook you know is that it is visible: the read happens because this line says a request fires at start, and if the widget is rebuilt by a `rebuild` the read happens again because `@started` fires again, which is in the text rather than in a lifecycle table. What `@started` is not: a primitive that makes creating a widget cause an event by itself. Its firing is construction, nothing outside the program observes it, and the creation rule stands. A `rebuild` block that needs to act on its own birth uses `@started` inside the block, where no `!Switch` stands between it and the hold it feeds.

## `rebuild`

You have written the view that is replaced when the mode changes, and you have written the bug that comes with it. The detail pane is rebuilt when the selection changes; the old pane had a subscription to a store; the subscription closed over the old item's id; the new pane subscribed too; now two handlers fire on every update and one of them writes a stale id into the model. The fix in every framework is a cleanup function, called from a lifecycle hook, in an order the documentation tells you not to depend on. The problem is that "replace this part of the program" was not a thing the

language could say, so it was said with construction plus a convention for destruction.

`rebuild` is the construct that says it. It takes a dynamic and a block, builds the block for the dynamic's current value, and rebuilds it whenever the dynamic changes, in the occurrence of the change. The old widget is torn down at that occurrence's commit, and everything it owned, holds, slots, subscriptions, placeholders, goes with it. There is no cleanup function because there is nothing to clean up by hand; there is no hook because there is no moment at which code runs; there is a tree, and a part of it is replaced.

```
@DynLabelButton: fn $Text -> @!Text = \@widget:
    !!rendered <- rebuild $_:
        !click <- @Element Ui.Button(label=_)
        @ _ on !click
    @ !Switch !!rendered
```

`rebuild $x:` takes a dynamic and a block in which `_` (or a pattern, `rebuild pattern <- $x:`) is the *plain* value of `$x`. Inside the block the label is `Text`, not `$Text`, because inside one build it never changes; a change makes a new build. Other dynamics in scope stay dynamic inside the block, and reading them there is ordinary lifting; only the dynamic named at the top triggers a rebuild. That distinction is the one to keep in your head: `rebuild` is not "re-render when anything changes," it is "re-build when *this* changes," and everything else is live inside.

The pattern form is for structure:

```
!!views <- rebuild ((mode, selected)) <- $state:
    match mode:
        Browse: @BrowsePane selected
        Edit: @EditPane selected
```

Both fields are plain inside; a change to either, since they are one dynamic, rebuilds.

## The result event

The result of a `rebuild` is an event of the block's result: `!A` for a block returning `A`. It fires in every occurrence in which the block is built, the first build included, with that build's result. In the occurrence the `rebuild` itself is built, its event fires in that occurrence; in every rebuild occurrence it fires again. The event goes to the caller. The block cannot see it unless the caller passes it back in, which, since the block's result is being defined, would be a cycle the block would have to close through a delayed edge like any other.

So `!!rendered` above is an event of events: each firing is a new build's `!Text`. In occurrence zero it fires with generation `#0`'s click-with-label; when the label changes in occurrence n it fires with generation `#1`'s. The parent needs one click event for all generations, and that is what `!Switch` is for, below.

Why does the result go up rather than being something the block owns? Because the parent is what persists across rebuilds and the block is what does not. A block that could see its own `rebuild`'s event would be seeing an event whose firings include its own death; the parent, which outlives every generation, is the only thing in a position to hold the stream of generations as one value. Reflex's `dyn` and `networkView` return an `Event t a` to the caller for the same reason.

## What a rebuild resets

A rebuild replaces the widget. Everything the old widget owned is new in the new one:

Holds inside the block are new holds with their initial values. A `$count <- 0 then ...` in a `rebuild` block starts at `0` again on every rebuild; this is either what you want (the pane's scroll position should reset when the item changes) or it is not, and the next subsection is about the second case.

Slots are new slots with fresh paths. A `rebuild` adds a generation to the path of everything under it, `#0`, `#1`, `#2`, and a path is never reused: the generation counter belongs to the `rebuild` widget, only goes up, and is not reset by anything short of the `rebuild` widget itself being torn down and rebuilt by its own parent, at which point every path under it has a new generation of the parent's in front of it anyway. A World that was doing something on behalf of an old slot stops, because the slot is gone from the list; a response it had in flight for the old path is discarded, because the path is not in the list it last received.

Subscriptions inside the block are gone. An occurrence-time binding in the old block was a graph node owned by the old widget; when the old widget drops, its subscribers are dropped, and the inputs it listened to, which hold their subscribers weakly, deliver to it no more. Nothing was unsubscribed; something was dropped.

Cyclic placeholders are gone with the widget that owned them.

A rebuild does not cut the old widget off early. The old widget keeps propagating until the occurrence commits, so an event that fired inside it in the rebuild occurrence is delivered to whoever was subscribed. A click on the old button in the occurrence its label changes is delivered from the old button, under the old label, to the `!Switch` that was still listening to it. Then commit, and the old widget is dropped.

A timeline makes the rule concrete. Take `@clicker` from the worked examples, a button whose label counts its own clicks:

```
@clicker:
    !!clicks <- rebuild $label:
        @Element Ui.Button(label=_)
    let !click = !Switch !!clicks
    $count <- 0 then $count + 1 on !click
    let $label = "Clicked {$count} times"
    @ unit
```

| Occurrence | Input | Inside | List after |
|---|---|---|---|
| 0 | build | generation `#0` built; `!!clicks` fires with its click; `!Switch` records it | `0/#0/0: Button("Clicked 0 times")` |
| 1 | click to `0/#0/0` | `#0`'s click fires; `!Switch` delivers; `$count + 1` fires with 1; `$label` changes; `rebuild` builds anew to `#1`; `!!clicks` fires with `#1`'s click; `!Switch` ignores it this occurrence; commit: `$count` is 1, `!Switch` adopts `#1`, `#0` dropped | `0/#1/0: Button("Clicked 1 times")` |

| Occurrence | Input | Inside | List after |
|---|---|---|---|
| 2 | click to `0/#1/0` | as above, to `#2` | `0/#2/0: Button("Clicked 2 times")` |

In occurrence 1 both generations exist for the length of the occurrence: `#0` because it has not committed out yet, `#1` because it was just built. Only `#0` can be clicked, because only `#0`'s path was in the list the World addressed. Only `#1` will be in the list the World reads next. There is no occurrence in which a button is on the screen that nothing listens to, and none in which two are listened to at once.

## When the new generation fails to build

You have seen the page that went blank. The new view threw halfway through its constructor; the framework had already unmounted the old one; the user got a white rectangle and a console full of red. The question a rebuild has to answer before anything else is what the tree looks like when the block it was asked to build cannot be built.

A `rebuild` block can fail to build in two ways, and they are reported differently because they are different failures. The first is a cycle found during the build: the new block's prompt graph, together with what it subscribes to outside, has an action scheduled at or below the current height. That is kind `Cycle`, found `AtBuild`, and it aborts the whole occurrence: nothing commits, the dynamic that would have rebuilt the `rebuild` keeps its old value, the old generation is still there, and the World is told, through its `errors` stream element if the program subscribed and through the run call's error return either way, which occurrence failed and which bindings were in the cycle. A cycle is a defect in the program's structure, not in its data, and a program with a cycle in one of its generations will have it on every rebuild to that generation, so there is nothing to be gained by containing it.

The second is a failure inside the construction that is not a cycle: a native component the block builds panics, a `for` key whose block fails, an `@Element` whose state tag no supplier and no World accepts. That is kind `Build`, with the owning widget's path, and it is contained: the occurrence commits, the dynamic changes, the old generation is torn down at commit like any other, and *there is no new generation.* The `rebuild` widget has no child until its dynamic changes again. Its event does not fire for the failed build, so a `!Switch` over it keeps listening to the old generation's event, which never fires again because the old generation is gone, and a hold-of-dynamics joined over it keeps the old generation's last dynamic, frozen at its last value. The World sees the old generation's slots leave the list and nothing arrive. The error goes out through `errors` with kind `Build` and the `rebuild` widget's path as `owner`, in the same occurrence, so a program that subscribes to `errors` learns of the hole in the occurrence after the one that made it, which is the earliest anything from outside the program can be learned.

The `for` case is the same rule per key: a key whose block fails to build is torn down (nothing of it survives) and the consolidated result has no entry for it; the other keys built in the same occurrence are unaffected; the error names the `for` widget and the key.

Why keep the old generation on a cycle and drop it on a build failure, rather than one rule for both? Suppose the old generation stayed on a `Build` failure. Then its slots would be in the list under a path whose generation the program has moved past, with holds reading a dynamic that has changed, drawing a value that is no longer the one the `rebuild` is over: a screen showing yesterday's selection with today's model behind it. The block that failed was the program's statement of what should be

there for the new value, and nothing else can be put there truthfully; a hole is the truthful thing. Suppose instead a cycle were contained. Then the occurrence would have to commit with half its propagation done, since the cycle is found mid-propagation and the actions already run cannot be unrun without rollback, which is exactly what aborting is. The two rules are one rule, "commit what is true or commit nothing," applied to a failure that can be isolated to a widget and one that cannot.

Bound by this: the runtime, which must not fire the `rebuild`'s event for a failed build and must tear the old generation down anyway; the World, which must report what the run call returns and may pass back `quarantine` paths on its next call if it wants a failing subtree held out of further rebuilds; and the program, which cannot observe a failed build except through `errors` and the list, because there is no event for it. Chapter 9 has the error type and the quarantine protocol in full.

## State that must survive goes outside

The mistake every newcomer makes once is putting a hold inside a `rebuild` and expecting it to persist. `Styled: fn Theme, Text -> Text` here is a plain function that decorates a label for a theme:

```
@wrong: fn $Theme -> @unit = \@widget:
    rebuild $_:
        !inc <- @Button $(Styled _, "+")
        $count <- 0 then $count + 1 on !inc          # resets on every theme change
        @Label (Styled _, "{$count}")
    @ unit
```

Change the theme and the count is zero. The `rebuild` is over the theme; the block is rebuilt for a new theme; the hold is in the block; the hold is new. The program says exactly this, if you read it as a description of ownership rather than as code that runs: `$count` belongs to the generation. The fix is to move the hold to the widget that persists:

```
@right: fn $Theme -> @unit = \@widget:
    !!incs <- rebuild $_:
        !inc <- @Button $(Styled _, "+")
        @ !inc
    let !inc = !Switch !!incs
    $count <- 0 then $count + 1 on !inc
    rebuild $_:
        @Label (Styled _, "{$count}")
    @ unit
```

Now the button is rebuilt per theme and the count is not. `$count` is read inside the second `rebuild` as a dynamic, because it is one from outside, and the label's `rebuild` builds anew on the theme, while the label's text, derived from `$count`, changes live inside each generation (and, since a label's text is a plain field, lifts into a `rebuild` of its own per the rule below; the walkthroughs write that out). The rule of thumb: a `rebuild` block owns what should die with the value it was built for, and nothing else. If you find yourself wanting a hold to survive a rebuild, it is not the rebuild's hold.

## What a rebuild costs, and what to keep out of one

A rebuild costs what the block builds, and nothing else: no diff against the old generation, no reconciliation, no patching of a retained tree. The old widget is dropped and the new one is built. For a label this is one slot; for a pane with a hundred rows it is a hundred blocks, and a `rebuild` around that pane over a dynamic that changes per keystroke is a hundred blocks per keystroke. The program says what the `rebuild` is over, so the program says what the cost is, and the way to make

it smaller is always the same: narrow the dynamic. A `rebuild` over `$state` rebuilds on every change of any field; a `rebuild` over `$state.mode` rebuilds on mode changes and reads the rest live.

Fresh paths are the other cost, and the one a World notices. Every slot under a rebuilt generation has a new path, so a World that keeps anything by path, a native toolkit widget or an in-flight request, loses it. Focus is the exception by design: a `Gui.Focusable` has a `name` and no other field, and the GUI library holds which name is focused, so a button whose label is a `rebuild` is a new slot per label change and keeps focus, provided its name does not change with the label. Where a native widget really must keep its identity, the same split applies: the element that must keep its identity is given no dynamic fields and is created once, and the thing whose fields change is a separate element that nobody needs to address. That split is available for every element and it is the reason dynamic fields are allowed to mean "a fresh slot" rather than something more complicated.

Library widgets take dynamic arguments, so a program normally writes `@Button $label` and never spells the `rebuild` or the fresh slot. The fresh slot per change belongs to the library: `@Button` builds a `Ui.Button` element, the library's interceptor answers it, and a relabelled button is a new slot in that interceptor's list, which a World that sees `Gui.*` only never observes. The fresh-slot-per-change rule holds for elements that reach a World; `Gui.Window` or `Fs.Contents` with a dynamic field is a new slot in the World's list, and a `Ui.Button` with one is a new slot in the interceptor's. The button that relabels itself in this chapter's examples illustrates the rule and is not how a program normally labels a button.

## A `rebuild` built in the middle of an occurrence

A widget built in occurrence n behaves as though it had been built immediately before n's inputs arrived. For a nested `rebuild` this has one consequence that an implementation can get wrong and the specification pins: a `rebuild` built in n, over a dynamic that changed in n, builds *once*, with the new value. It does not build with the old value and then rebuild.

```
@outer: fn $Mode, $Text -> @unit = \|$mode, $text| @widget:
    rebuild $mode:
        rebuild $text:
            @Label $(Render _, mode)       # mode is plain here; _ is the text; Render: fn Text, Mode -> Text
    @ unit
```

Suppose `$mode` and `$text` both change in occurrence n, because one input moved both (a hold over a record, say, that the `rebuild`s project). The outer `rebuild` builds anew in n. Its new block binds an inner `rebuild` over `$text`. That inner `rebuild`, at the moment it is built, subscribes to `$text`'s change event and finds that the event has already fired in this occurrence, with the new text; so it builds its block with the new text, once, and its result event fires in n with that build. Had it read `$text`'s value through its behavior side it would have seen the old text, built a label with it, and been wrong until the next change of `$text`, which might be never.

The runtime does this with the Spider-style catch-up: every event caches its current-occurrence value, `subscribe` returns it if the event has already fired, and the `Rebuild` takes the initial widget from that cached firing if there is one and from the behavior otherwise. The test `a_nested_dyn_built_in_the_occurrence_its_dynamic_changes_shows_the_new_value` pins it. Its companion, `a_closure_runs_once_per_occurrence_even_when_late_subscribers_read_it`, pins the other half of the rule: a late subscriber reads the cached value, and the closure that produced it is not run a second time, so the at-most-once law holds for everything a mid-occurrence build touches.

### What the runtime promises for `rebuild`

Three promises, each the runtime's, each with the test that pins it; chapter 11 has the `Rebuild` that keeps them.

The rebuild is an ordinary action at an ordinary height, one above the dynamic's change event, and the new generation is built at a height above that, so that everything the new block fires at birth fires after the block exists. A cycle through a `rebuild` is therefore caught by the same comparison as any other cycle, an action scheduled at or below the current height, and no second detector exists for rebuilds.

The old widget is dropped by an action at the commit height, not by a destructor in the middle of propagation, so nothing it owned can vanish while something at a lower height is still delivering to it. Nothing is notified when a widget goes: the only way anything outside the program learns of it is the list it reads afterwards, so the drop action has no callback. `a_replaced_child_keeps_working_until_its_last_occurrence_ends` is the check.

The `rebuild`'s event is the leftmost of a rebuild event and the first build's done event, and the done event fires at `height + 1` of the build, which is why the first firing lands in the build occurrence and after the build has finished. A `rebuild` built after its dynamic has already changed in the occurrence rebuilds once and in order, by deferring the same action to `max(height, current + 1)`; the nested-`rebuild` test above is the check.

### `rebuild` is not a delay

A cycle through `rebuild` alone is rejected. The `rebuild`'s event fires in the occurrence of the rebuild, promptly, so it is a prompt edge, and a program like

```
@bad:
    !r <- rebuild $x:
        @ _ + 1
    $x <- 0 then !r                  # error: the cycle through rebuild has no delayed edge
```

has no delayed edge anywhere: `$x` changes, the `rebuild` builds anew in that occurrence, its event fires in that occurrence, the hold's change event fires in that occurrence, which is `$x` changing again, in the same occurrence. The compiler rejects it with both bindings named. The reason `rebuild` is not made a delay, which would make this legal and a tick behind, is that every other rebuild would then be a tick behind too, and the new widget would see the occurrence it was born in as the past. The delayed edges are reads of a dynamic as it stood when the occurrence opened, and there are three of them; a rebuild is not a read.

## `!Switch`, `$Join`, and why they differ

A `rebuild` returns an event per generation. The parent wants one event, or one dynamic, across all of them. Two primitives do that, and they are not symmetric, and the asymmetry is one of the few places where the language chooses something that looks like an implementation convenience and is not.

### `!Switch`

The clicker has `!!clicks`, one click event per generation of its button, and it wants to count clicks. It cannot subscribe to a generation's click event by name, because the generation is not a value and

the next one will have a different event; it cannot wait for the stream of generations to end, because it never does. It wants one event that is "the current button's click, whichever button that is," and `!Switch` is that event.

`!Switch: fn !!A -> !A`. At occurrence n it delivers the firing, if any, of the inner event that the outer event most recently delivered *before* n. It starts subscribed to nothing. It is not prompt: an inner event delivered at n is heard from n+1.

Which inner event `!Switch` is listening to is a hold, a hold of events, and like every hold it is read on its value side, before the occurrence. That is the delayed edge that makes the clicker's cycle legal: `$count` depends on `!click`, `!click` is the switch, the switch's choice of inner event was made before the occurrence, and so the cycle `$count → $label → rebuild → !!clicks → !Switch → !click → $count` passes through a read of something as it stood when the occurrence opened.

The consequence for `rebuild` is that whatever a newly built block fires in its creation occurrence is swallowed at the `!Switch`. The parent does not hear it. Under the creation rule this costs nothing anyone wanted: a slot cannot be addressed in the occurrence it was created, so no click, no response, no key can fire inside a fresh block in its birth occurrence. The only things that can fire there are pure, which means `@started` and things derived from it, and a block that needs to act on its own start uses `@started` *inside* the block, where no switch is in the way. The parent, for its part, already knows about the birth, because the `rebuild`'s own event fired.

Suppose `!Switch` were prompt. Then in the clicker, in occurrence 1, the new generation's click event would be listened to from the moment it was built, in that same occurrence. Its height would have to be known before the rebuild action that creates it has run, because `!click`'s height is what `$count + 1 on !click` sits above; so either heights are dynamic in the middle of propagation and every node above a switch re-sorts itself mid-occurrence, or the switch's height is an upper bound guessed in advance and wrong when the inner event turns out to be taller. Reflex has `coincidence` for exactly this, and its documentation points you at `switchHold never` instead. A non-prompt switch has a fixed height, one above the outer event, and the comparison that catches a cycle is one comparison.

### $Join

A file viewer has a path the user edits and wants to show the file's contents. `Fs.Contents(path=p)` is a state element, so one slot is one dynamic of contents; the path changes, so the slot is inside a `rebuild`; and the `rebuild` returns an event of dynamics, `!$Contents`, one per path. The label wants one dynamic, "the contents of whatever file the path names now," and it wants it with no gap: in the occurrence the path changes, the new slot has a value (the oracle answered as the occurrence opened), and the label should show it then, not an occurrence later.

```
!$contents <- rebuild $path:
    @Element Fs.Contents(path=_)
$$current <- @HoldFirst !$contents
let $contents = $Join $$current
```

`$Join: fn $$A -> $A`. At occurrence n its value before n is the value before n of the inner dynamic current before n. Its change event fires at n, with the value after n, if the outer dynamic changes at n (to a new inner, whose value after n is taken) or if the current inner changes at n. It is prompt, like every derived dynamic.

The hold needs an initial dynamic, and the first build's result is the one to use, because the first build happens in the occurrence the `rebuild` is built: the `rebuild`'s event fires then, the hold adopts the firing in its build occurrence, and `$Join` sees the adopted inner in that same occurrence. `@HoldFirst:`

`fn !a -> @$a` is the prelude's name for that hold, and it is the idiom: a hold written by hand with a placeholder initial dynamic, `$ Unavailable then !$contents`, has the placeholder as its value for the length of one tag read in the build occurrence, and nothing is gained by having a value there that no build produced. `@HoldFirst` has no semantics beyond what it expands to, a hold whose initial value is the first firing of its event in the occurrence the hold is built, and the lifting section below uses it for exactly that.

This is also the canonical cache idiom: a hold of dynamics, chosen between a live one and a frozen one, joined. A hold of plain values copied out of the live dynamic is the simpler construction, and it has a one-occurrence gap at creation that the join version does not, because a join switches to the new inner's *current* value in the occurrence of the switch rather than waiting for the new inner to fire. Chapter 2's cache is the join form, with the hold of dynamics around the forwarded query.

### The asymmetry

Why is one prompt and the other not? Because a dynamic always has a value to switch to, and an event does not. When `$$current` changes at n to a new inner dynamic, that inner has a value after n (every dynamic does, at every occurrence), and `$Join` can fire with it promptly without asking anything to fire first. When `!!clicks` changes at n to a new inner event, the question "did the new inner fire at n" would have to be answered by an event that was being created during n, at a height nobody knew when the switch's height was fixed. The join never needs a firing from the new inner; it needs a value, and values are always there. The switch would need a firing, and firings have heights.

The two together, then: results that are events cross a `rebuild` with a one-occurrence lag that the creation rule makes unobservable, and results that are dynamics cross it with no lag at all. A derived dynamic that lagged would be a glitch, and ruling out glitches is why `$Join` must be prompt; an event that was switched promptly would be a height cycle in every interesting case, and ruling out height cycles is why `!Switch` must not be.

### What the runtime promises for the two

`!Switch` reads its inner event from a hold's value side, before the occurrence; its height is fixed at one above the outer's. `$Join`'s change event is the leftmost of the outer's change mapped to the new inner's value after n and the non-prompt switch over the current inner's change event; its height is dynamic, since inner events have arbitrary heights. Both are the runtime's to keep, and chapter 11 has `switch_hold` and `Dynamic::join`.

The harness tests three cases for join: the inner changes alone; the outer swaps to an inner that changed in the same occurrence, where the change event must deliver the new inner's *new* value and the behavior must agree afterwards; and the outer swaps to an inner that does not change, where the change event must still fire, with the new inner's current value, because the joined dynamic changed even though nothing inside it did. The third is the one an implementation that only listens to the inner's change event gets wrong. For the switch, the law test is the swallowed birth: a block that fires at its parent through `!Switch` in its creation occurrence is not heard, and is heard from the next occurrence on.

## Lifting over `@`

The lifting rule says that an operation over dynamics is a dynamic. What is a blueprint over a dynamic? `@PlainButton $label`, where `@PlainButton: fn Text -> @!Click` takes a plain label, applies

a widget function to a dynamic; `@Element Ui.Label(text=$t)` puts a dynamic where a plain field goes. Both are applications of something that builds to something that changes, and the answer is the one thing the language has for "build again when this changes."

**Binding a dynamic of blueprints lifts to `rebuild`.** `!x <- @PlainButton $label` means `rebuild $label:` around the build, with the bound name being the `!Switch` of the `rebuild`'s event:

```
!x <- @PlainButton $label

# is

!!xs <- rebuild $label:
    @PlainButton _
let !x = !Switch !!xs
```

The lift happens only where the parameter wants a plain value and gets a dynamic. The library's `@Button: fn $Text -> @!Click` takes the dynamic itself and holds the `rebuild` inside its body, which is why `@Button $label` has no lift at the call site and the same widget structure as the expansion above; the two spellings differ only in which widget's block the `rebuild` is written in, and chapter 11 states the canonical widget structure both compile to.

For a widget whose result is a dynamic, the bound name is the hold-of-dynamics joined, with the first build's result as the hold's initial value:

```
$s <- @Status $url

# is

!$ss <- rebuild $url:
    @Status _
$$cur <- @HoldFirst !$ss
let $s = $Join $$cur
```

`@HoldFirst` is a hold whose initial value is the firing of `!$ss` in the occurrence the hold is built, which exists because a `rebuild` fires in its build occurrence and a widget built in n sees all of n. No hold written with `then` can be seeded that way, since a hold's initial value is an expression and the first build's result is not in scope until the `rebuild` has been bound; this is why the prelude has it, and why it is the idiom wherever a program holds a `rebuild`'s event. It expands to the runtime's `hold_first` over the event, and chapter 12 prints it as `HoldFirst` in the typed AST.

For a plain result, binding gives a dynamic: `$x <- @W $d` is `@HoldFirst` over the `rebuild`'s `!A`, so `$x` is the current generation's result, held from the first build's value and changing in every rebuild occurrence. A plain result of a rebuilt widget is a value per generation, and a value per generation held is a dynamic. For a bundle, field by field by the kind of each field: an event field switches, a dynamic field joins, a plain field holds.

An element with a dynamic where a field goes is the same thing one level down: `@Element Fs.Contents(path=$p)` is `$(@Element Fs.Contents(path=p))`, a dynamic of blueprints, which is a `rebuild` with a fresh slot per change of `$p`. This is why the element list never contains a sigil and why the World never diffs anything but plain data: a label whose text changes is, to the World, a new label at the same tree position, and the generation in its path says so.

Walkthroughs write the expansion out only when the generations are the point.

One thing that does not lift on its own is a tuple. Functor values nest inside plain aggregates, so `(($step, $answers))` is a plain pair holding two dynamics, not a `$((Step, Answers))`, and `rebuild`

over it is a type error. To make one dynamic of two, lift a plain function over them: `Pair: fn a, b -> ((a, b))` is in the prelude, and `Pair $step, $answers` is a `$((Step, Answers))` that changes when either does, once per occurrence however many of its operands moved. The wizard and the tabs interceptor below both write it.

What does *not* lift. A blueprint over an event is not a `rebuild` and not anything else: `@Button !label` is a type error. A widget per firing of an event would be a widget with no owner's decision to replace it, built in the occurrence of the firing and torn down when? Never, or at the next firing, and both answers are a lifecycle rule the language does not have. The program that wants a widget per event holds the event into a dynamic first, which says when the old one goes. Likewise `match $x:` with a blueprint in any arm is rejected: choosing a blueprint by a changing value is construction, and construction over a dynamic is spelled `rebuild`, so that the rebuild is visible where it happens.

## for

You have had the afternoon where deleting item 3 cleared the expanded state of item 4. The list was an array; the framework matched rows to data by position; when item 3 went, item 4 was now at position 3, and the row that had been at position 3, with its expanded flag set, was handed item 4's data. The fix was a `key` prop, and the fix after that was a lint rule telling you not to use the index as the key, and the one after that was a conversation about where keys come from. The language's answer is that there is one collection for anything that matters, it is a map, and its keys are the identities.

```
!updates <- for id, $item <- $items:
    !checked <- @Checkbox $item.checked
    @ ItemUpdate id, !checked
```

`for k, $v <- $m:` over a dynamic map `$(K ==> V)` builds one copy of the block per key. Inside, `k` is the key, plain, because a key never changes for the life of its block; and `$v` is the entry's value *as a dynamic*: a change to a value that keeps its key updates `$v` and does not rebuild the block. `$item.checked` is ordinary lifting over that dynamic. The binders are ordinary patterns, named by the loop. (The loops inside `update` blocks, at the end of chapter 3, bind the reserved indices `ix`, `jx` and `kx` instead.) Most blocks need no names, so the shorthand `for $m:` binds `key` and `$_`, and the examples below use it unless the names help. A key that appears in occurrence n has its block built in n, under the rule for mid-occurrence construction; a key that disappears in n has its block torn down at n's commit, like a `rebuild`'s old generation. The parent sees the new key's result in n; the new block does not see itself.

That last clause is the same fact as `!Switch` swallowing a block's birth, seen from the collection side. The consolidated event of a `for` is a merge over the blocks; a block born in n contributes to the merge from n, and so its parent can hear it in n if it fires in n, which only pure code can make it do; but nothing inside the block is downstream of the consolidation, because the consolidation is the block's own result.

### Keys as identity

A key decides three things. Which block survives a change: an entry whose key is still present after a change keeps its widget, and an entry whose key is gone loses it. Which element paths stay stable: a `for` adds its key to the path of everything under it, `1/[7]/0/#0/0`, so a button in row 7 keeps its path while rows 3 and 12 come and go, and the World keeps addressing it. And which hold keeps its

state: a `$expanded` inside the block belongs to the key's widget and lives exactly as long as the key does, through every change to its neighbours.

Keys are ordered, because maps are, so the element list has a defined order, which is tree order, which under a `for` is key order. A list that should display in some other order sorts by making a map whose key sorts that way.

## Consolidation by result kind

Every row has a Toggle button and a Delete button, and the program above the list wants to apply what the user did. Two rows can be clicked in one occurrence (the World batched two pointer events into one turn, or a test put them on one line), and the program above the `for` wants to hear about both at once, with no order between them that it did not ask for:

```
!rows <- for $items:
    !toggle <- @Button $"Toggle"
    !delete <- @Button $"Delete"
    @ Toggle on !toggle | Delete on !delete
```

`!rows : !(K ==> Change)`. When row 3's Toggle and row 9's Delete are clicked in one occurrence, `!rows` fires once, with `{3: Toggle, 9: Delete}`. When nothing in any row fires, `!rows` is quiet. The program above the `for` gets to say what simultaneous changes to several rows mean, in one place, by folding the map; it never sees an order between rows that the language did not promise, because there is none.

A `for` has one result, not a map of results, and what it is depends on what the block returns:

| Block returns | `for` returns | Meaning |
|---|---|---|
| plain `A` | `$(K ==> A)` | a dynamic map of the blocks' values, changing when a key appears or disappears |
| `!A` | `!(K ==> A)` | fires in any occurrence in which at least one block's event fires, with exactly the entries that fired |
| `$A` | `$(K ==> A)` | a dynamic map whose value is every block's current value, changing when any block's dynamic changes or the key set changes |
| `@A` | by the inner kind | the blocks are built as children; their results are consolidated by these rules |
| `unit` | `$(K ==> unit)` | the key set as a dynamic; the compiler elides it when nothing binds the `for` |

Consolidation pushes the map as deep into the result as these rules allow, and a record result is consolidated field by field: a block returning `record: !changed: Text; $value: Text` gives a `record: !changed: K ==> Text; $value: K ==> Text`, one map in each field, because a map of records of two functors is less useful than a record of two maps and the rules can always get there.

The `@A` row is the one that looks like a special case and is not. A block whose last statement is a bound widget has that widget's result as its own, so a `for` whose rows end in `@Row $_`, with `@Row: fn $Item -> @TextInput` returning the bundle `record: !changed: Text; $value: Text`, consolidates exactly as if the block had returned the bundle itself:

```
TextInput(!changes, $values) <- for $items:
    @Row $_
# !changes : !(K ==> Text)   fires with the rows whose inputs changed this occurrence
# $values  : $(K ==> Text)   every row's current text
```

The row is a child of the `for` widget per key, its slots sit under `[k]` in the path, and its two result fields are consolidated field by field into two maps. Nothing about the inner widget being a function application changes what consolidation does; the kind of the result is what is consolidated, wherever the result came from.

The remaining kinds, with an example each. A plain result is one that depends on nothing live, which inside a `for` means on `key` alone:

```
$ids <- for $items:
    @ key                        # $(K ==> K): changes only when the key set does
```

A `unit` result is the common case of a row that only draws, `for $items: @Label $_.title`, and consolidates to the key set, `$(K ==> unit)`, which is a dynamic nobody reads; a `for` statement that is not bound has its result elided by the compiler, so a display-only list costs no consolidation node.

A dynamic result:

```
$texts <- for $fields:                        # $fields : $(((Nat, Text)) ==> Field): id and initial text in the
key
    let ((id, initial)) = key                   # plain, for the life of the block
    TextInput(ignore, $v) <- @TextInput initial
    @ $v
```

`$texts : $(((Nat, Text)) ==> Text)`, the current text of every input, changing when any input's text changes or a field is added or removed.

The mistake to make once is in that last example. Had the initial text been in the entry, `@TextInput $_.initial` would apply a widget function that takes plain text to a dynamic, which lifts to a `rebuild`, and the input would be rebuilt, cursor and all, every time the entry changed. The compiler accepts it, because it is a meaningful program; it is just not the one you wanted. The question "is this plain, dynamic or an event inside my `for`" is the one the sigils answer: `key` is plain for the life of the block, `$_` is live, and anything that should be fixed at the block's birth belongs in the key or is read once through a hold that nothing else moves.

## Consolidation is prompt

A key that appears in occurrence n is built in n, and anything its block fires in n is in the consolidated event in n. Only pure code can fire in a block's birth occurrence, and the thing a block fires at birth is a request from `@started`:

```
!requests <- for $watched:
    !started <- @started
    @ (Watch key) on !started
```

A key added in n makes `!requests` fire in n with `{new_key: Watch new_key}`, consolidated with whatever other blocks fired, and the parent, which perhaps feeds `!requests` into an element's `requests`

field, sends it out in n. The new block, had it been able to see `!requests`, would see its own birth; it cannot, because nothing in the block is downstream of the block's result, and a program that passes `!requests` back into the `for` has to close that cycle through a delayed edge like every other.

## A walkthrough

Take the todo list from the worked examples, reduced to what matters for `for`. `Describe: fn Item, Bool -> Text` renders a row's label from the item and its star, and `StepAll: fn Nat ==> Item, Nat ==> Change -> Nat ==> Item` applies a map of changes to the items; `!Union: fn !(k ==> v), !(k ==> v) -> !(k ==> v)` merges two map events entry-wise, firing when either does:

```
@todos:
    !add <- @Button $"Add"
    $next <- 0 then $next + 1 on !add
    let !adds = {$next: Add "item {$next}"} on !add
    !rows <- for $items:
        !delete <- @Button $"Delete"
        !star <- @Button $"Star"
        $starred <- False then not $starred on !star
        @Label (Describe $_, $starred)
        @ Delete on !delete
    let !changes = !Union !adds, !rows
    $items <- {} then StepAll $items, !changes
```

**Occurrence 0.** `$items` is `{}`; `for $items` builds no blocks. The list has one button.

**Occurrence 1, Add.** `!adds` fires with `{0: Add "item 0"}`; `$items`'s change event fires with a one-entry map; `for` sees key `0` appear in this occurrence and builds its block now: two buttons at `1/[0]/0/#0/0` and `1/[0]/1/#0/0`, a hold `$starred` at `False`, a label at `1/[0]/2/#0/0`. Commit.

**Occurrence 2, Add.** Key `1` appears; its block is built. Key `0`'s block is untouched: not rebuilt, not re-anything. Its `$starred` is whatever it was.

**Occurrence 3, Star on row 0.** Inside row 0's block, `!star` fires and `$starred` becomes `True`; the label's text changes, so its `rebuild` builds anew the label slot to `1/[0]/2/#1/0`. The row's result event, `Delete on !delete`, is quiet; `!rows` is quiet; `$items` does not change; the `for` does nothing at all.

**Occurrence 4, an edit arrives for item 0 from elsewhere** (a push, say, or another button outside the `for` that changes the label). `$items` changes; key `0` is still present; so row 0's `$_` changes, its label text changes, the label rebuilds, and `$starred` is still `True`, because the block was not rebuilt. The row's buttons keep their paths.

**Occurrence 5, Delete on row 0.** `!rows` fires with `{0: Delete}`; `StepAll` removes key `0`; `for` tears row `[0]`'s block down at commit. Row `[1]` is where it was, with its paths and its star.

Nothing in this walkthrough involved the program telling the `for` which rows had changed. It changed the map. The `for` compared keys.

## Arrays are second class

`for` over a dynamic array `$[V]` is keyed by index, and the index is the key in every sense: the block's `key` is a `Nat`, the path segment prints as `[3]` exactly as a `Nat ==> V` map's key `3` would, and a transcript does not distinguish the two, because there is nothing to distinguish; a `for` over an array is a `for` over the map from index to element, and the paths say so. It is allowed, and it is the wrong tool for anything with identity: an insertion at the front changes every later key, which rebuilds every

later block and resets its state, and the World sees every later path vanish and reappear. That is the array-index-as-key bug, built in, and the language keeps it only because positional data exists, a row of three fixed columns, a list of coordinates, where the index is the identity and nothing else is.

Programs that need positional display of identified items keep a map with allocated ids and sort for display. The idiom, with `Insert: fn k ==> v, k, v -> k ==> v` and `NewItem: Item` a blank item:

```
@list:
    !add <- @Button $"Add"
    $next <- 0 then $next + 1 on !add                        # the id allocator
    $items <- {} then (Insert $items, $next, NewItem) on !add  # Nat ==> Item, keyed by id
    !rows <- for $items:
        ...
    @ unit
```

`$next` is a counter that only goes up; an id is never reused, so a key that returns is a new key, which is right. That is the program's half of an invariant whose other half is the runtime's: a key that leaves a `for` and comes back gets a new widget, with fresh holds and fresh paths under `[k]`, however short the absence, because the old widget was torn down at the commit of the occurrence the key left, and the runtime keeps no table of departed keys to revive. The program's allocator makes the two halves agree: with ids that never repeat, "the same key came back" cannot happen, and the question of whether a returning key should keep its state never arises. A program that reuses keys gets a new widget per return anyway, and the runtime's promise is what makes that predictable rather than dependent on how long the key was gone.

When the display order is not id order, make a map whose keys sort the way the display should, with `ReKey: fn k ==> v, (fn k, v -> j) -> j ==> v`:

```
let $ordered = ReKey $items, \|id, item| ((item.rank, id))    # $(((Nat, Nat)) ==> Item)
!rows <- for $ordered:
    let ((rank, id)) = key
    ...
```

The key is a tuple of rank and id, so two items with the same rank stay distinct and sort by id, and an item whose rank changes gets a new key and a new block, which is what moving it is. An item whose rank does not change keeps its block through everything else. If re-ranking an item should keep its block, the rank does not belong in the key, and the display order comes from the World or an interceptor reading `rank` out of the elements it lays out; both are ordinary programs, and which is right depends on whether the thing with the state is "the item" or "the item at this rank."

Suppose instead the language gave arrays an opaque per-element identity, minted when an element is created and following it through inserts and removals. That is what `for` over an array would need to be first class. The identity would have to be minted somewhere, which means every array operation in the plain-value layer would have to preserve a hidden field that plain values do not have, or the language would need a second array type with identities, which is a map with allocated keys under a different name. The allocated-id map is that second type written in the language, with the allocation visible, and so it is the one the language recommends and the one the todo list uses. Reflex offers both: `listWithKey` over a `Map` and `simpleList` over a list.

## Nested `for`

A `for` inside a `for` written with the shorthand has its own `key` and its own `$_`, and the outer ones are shadowed, like `_` in a nested lambda. Nesting reads better with named binders, `for row, $r <- $rows:` around `for col, $cell <- $r.cells:`, because both keys stay in scope and the path says

`[row]/[col]`. With the shorthand, name the outer key with `let` before the inner `for`.

The runtime's promise for `for` is the `rebuild`'s promise per key: a key that appears is built in the occurrence of the change, at a height above the map's change event, and a key that disappears is torn down at commit by the same keep-until-the-end action; `$_` is a per-key dynamic projected from the map, so a change to an entry that keeps its key reaches its block without rebuilding it; and the consolidated event's membership is the blocks as the occurrence opened plus the blocks built in it. Chapter 11 has the keyed list widget that keeps all three.

## What switching may not do

No switching construct creates an occurrence or defers to a later one. A rebuild happens in the occurrence of the change. A key is built in the occurrence it appears. A `rebuild`'s event fires in the occurrence of the build. If what the new widget wants is something from the World, it asks through an element, in the build occurrence if it likes, and is answered in a later one, and there is no way for a rebuild to "wait for" that answer other than the way every program waits: by holding the response when it arrives and typing the gap.

This is why `rebuild` is not a delayed edge and why a cycle through `rebuild` alone is an error. It is also why there is no `Defer` in the language: `Defer` is an element, the World's, and the only thing it means is "answer this in a later occurrence of this turn," which a rebuild can ask for like anything else and cannot do by itself.

And nothing is ever triggered by a widget appearing or disappearing. A `rebuild` that tears a generation down does not fire a "destroyed" event; a `for` whose key vanished does not fire "removed." The parent knows, because the parent made the change to the dynamic that caused it. The World knows, because the element list it reads is different. If a program wants something done when a part of it goes, the part fires a request before it goes, from the same event that causes it to go, and the request is answered later; a draw list a GUI World reads is description, not a trigger. Exit is a request element with a request event, and a shell that wants to know a hosted program has ended reads the list and sees its elements are not in it.

## Patterns

Five constructions that every interactive program has some version of, each written in the language and then run occurrence by occurrence with attention to one question: what rebuilds when. The answer is always readable from where the holds are and what the `rebuild`s and `for`s are over. The GUI vocabulary and the path notation are those of the worked examples: `Ui.Button(label)` responds with `click`, `Ui.Label(text)` is a library tag the GUI library turns into `Draw.Text` in a `Gui.Scene`, a dynamic in a field is a `rebuild` with a generation in the path, and `@Button`/`@Label` are the library widgets that write that `rebuild` for you. A few plain helpers are assumed and given a signature where they are used.

### Master-detail

A list on the left, the selected item's detail on the right. The detail should rebuild when the *selection* changes and only then; an edit to the selected item should update the detail in place.

```
type Item = record:
    title: Text
    body: Text

Keys: fn k ==> v -> [k]                          # in key order
FirstKey: fn k ==> v -> ?k = \|m| (Keys m)[0]    # Nil on an empty map
LastKey: fn k ==> v -> ?k
Gone: fn k ==> v, ?k -> ?unit                    # Some unit when the key is Some and absent from the map

let ShowBody: fn ?Item -> Text = \|i| match i:
    Some item: item.body
    Nil: "(no such item)"

@Detail: fn $?Item -> @unit = \@widget:
    @Label (ShowBody $_)
    @ unit

@master_detail: fn $(Nat ==> Item) -> @unit = \|$items| @widget:
    !picks <- for $items:
        !click <- @Button $_.title
        @ key on !click
    $selected <- Nil then Some (!Filter (FirstKey !picks))
    rebuild $selected:
        match _:
            Nil: @Label $"Select an item"
            Some id: @Detail $items[id]
    @ unit
```

Three things are live and they are at three different places. `$items` is the model and comes from outside. `$selected` is a hold in `@master_detail`'s own widget, outside both the `for` and the `rebuild`, so it survives everything. The `rebuild` is over `$selected` alone; inside it, `id` is plain and `$items[id]` is a lifted lookup, a `$?Item` that changes whenever `$items` changes, read live by `@Detail` without rebuilding anything but the label's slot. `FirstKey` picks the lowest key when two rows are clicked in one occurrence, and that choice is written down.

**Occurrence 0.** `$items` has, say, keys 1, 2, 3. The `for` builds three blocks, a button each. `$selected` is `Nil`; the `rebuild` builds generation `#0`, the "Select an item" label.

**Occurrence 1, click row 2.** Row 2's block fires `2`; `!picks` fires with `{2: 2}`; `FirstKey` gives `Some 2`; `!Filter` passes `2`; `$selected`'s change event fires with `Some 2`. The `rebuild` builds anew: generation `#1` is built with `id = 2`, `$items[2]` is `Some (Item "Two", "...")`, `@Detail` builds a label with the body. Generation `#0` is dropped at commit. What rebuilt: the detail, once. What did not: the three rows.

**Occurrence 2, the model changes item 2's body.** `$items` changes; key 2 is still there, so row 2's `$_` changes and its button's label slot rebuilds (the derived title fires its change event even when the text is equal; the World diffs and draws nothing new). No other row is touched. Inside the detail's generation `#1`, `$items[2]` changes promptly, `ShowBody` changes, and the label's slot rebuilds to the new body. The detail widget itself, generation `#1`, is untouched: the `rebuild` is over `$selected`, and `$selected` did not change.

**Occurrence 3, a new item 4 arrives.** The `for` builds row 4's block in this occurrence. The detail is untouched.

**Occurrence 4, item 2 is deleted while selected.** Row 2's block is torn down at commit. In the detail, `$items[2]` becomes `Nil` and the label says "(no such item)". `$selected` is still `Some 2`, because nothing fired at it; a program that wants the selection cleared adds `| Nil on !Filter (Gone`

`(!Updated $items), $selected)` to the hold, reading the selection through a tag as the items change, and that is a decision the hold's line shows.

**Occurrence 5, click row 1 and row 3 at once.** `!picks` fires with `{1: 1, 3: 3}`; `FirstKey` says 1. The detail rebuilds to item 1. If the program would rather the higher key won, it says `LastKey`.

The alternative to compare against is a `rebuild` over the pair `($items, $selected)`, which is what you get in frameworks where the detail is a function of all its props. That version rebuilds the detail on every model change, which for a detail with its own state (a scroll position, an expanded section) resets that state whenever anything anywhere in the list changes. Here the thing that owns the detail's generation is the selection, and the model flows through it live.

## Tabs

Tabs are the master-detail pattern with the twist that the user expects each tab to remember where it was. There are two honest versions and the language makes you pick one.

```
type Tab = choice Inbox | Drafts | Settings

@tabs_simple: @unit = @widget:
    !to_inbox <- @Button $"Inbox"
    !to_drafts <- @Button $"Drafts"
    !to_settings <- @Button $"Settings"
    $tab <- Inbox then Inbox on !to_inbox | Drafts on !to_drafts | Settings on !to_settings
    rebuild $tab:
        match _:
            Inbox: @inbox_pane
            Drafts: @drafts_pane
            Settings: @settings_pane
    @ unit
```

**What rebuilds when.** Clicking Drafts in occurrence n changes `$tab`; the `rebuild` tears down the inbox pane at n's commit and builds the drafts pane in n. Everything the inbox pane owned goes: its scroll position hold, its selected-message hold, its `Fs.File` slots, any request in flight (the World discards the response, since the slot is gone). Come back to Inbox and it is built fresh, at the top, with nothing selected, and its `@started` fires again, so whatever it loads at start is loaded again. For a settings pane this is usually what you want. For an inbox it is usually not.

The second version lifts the state that should survive out of the `rebuild`, into the widget that persists, and hands each pane its state as a dynamic. The pane returns an event of the changes it wants made, and the hold outside applies them. Since only one pane exists at a time and the `rebuild`'s result has to have one type across all generations, the block returns a record with one event field per pane, `!never` in the fields for the panes that are not showing:

```
type TabEvents = record:
    !inbox: InboxState
    !drafts: DraftsState

@tabs_kept: @unit = @widget:
    !to_inbox <- @Button $"Inbox"
    !to_drafts <- @Button $"Drafts"
    $tab <- Inbox then Inbox on !to_inbox | Drafts on !to_drafts
    $inbox <- (InboxState 0, Nil) then switched.!inbox
    $drafts <- DraftsState "" then switched.!drafts
    !!panes <- rebuild $tab:
        match _:
```

```
            Inbox:
                !ch <- @InboxPane $inbox
                @ TabEvents !ch, !never
            Drafts:
                !ch <- @DraftsPane $drafts
                @ TabEvents !never, !ch
    let switched = TabEvents (!Switch !!panes.!inbox), (!Switch !!panes.!drafts)
    @ unit
```

`!!panes.!inbox` is field access lifted over the outer event: an `!!InboxState`, the inbox's change event per generation, `!never` in generations where the drafts pane is showing. `!Switch` over it is the one `!inbox` change event across all generations, which feeds the hold that lives outside; `switched` is a plain record of two events, and the holds read its fields. The pane takes `$inbox` as a dynamic and returns the changes it wants made; it does not own the state, it edits it, and the hold's line in `@tabs_kept` lists the one event that can move it.

**What rebuilds when, in this version.** Clicking Drafts rebuilds the pane, as before. The inbox pane's widget goes, with its slots and its subscriptions. Its *state* does not, because it never had any: `$inbox` is the parent's. Come back to Inbox and `@InboxPane $inbox` is built with the scroll and selection it had. The cycle `$inbox → rebuild → !!panes → !Switch → switched → $inbox` is legal through the switch's hold, and the pane's changes in the occurrence it is built (there are none, by the creation rule, unless it fires one from `@started`) are swallowed, which is fine, because the parent's state is what it was.

There is a third version, which keeps every pane built and shows only one. The language has no "hidden" flag, because visibility is a World's or a library's idea, not the language's; the construction is an interceptor around all three panes that holds back the `Ui.*` elements of the inactive ones and forwards everything else. Each pane is wrapped in a `@Tagged` that puts its elements under a known path prefix, and the interceptor's `for` over the captured list keeps or drops each slot by whether its prefix is the active tab's:

```
PaneOf: fn Path -> ?Tab                      # which pane's subtree a path is under, by its first segment
IsUi: fn Element -> Bool                     # chapter 6's: True for every Ui.* tag

@tabs_all: @unit = @widget:
    !to_inbox <- @Button $"Inbox"
    !to_drafts <- @Button $"Drafts"
    !to_settings <- @Button $"Settings"
    $tab <- Inbox then Inbox on !to_inbox | Drafts on !to_drafts | Settings on !to_settings
    ((ignore, $slots)) <- intercept @all_panes, !down, $ \|e| @Element e
    !down <- for $slots:                                  # key is the child's path
        let $shown = not (IsUi $_.element) or (PaneOf key) == Some $tab
        !!resps <- rebuild ((slot, shown)) <- Pair $_, $shown:
            if shown:
                ((!resp, ignore)) <- @Element slot.element
                @ !resp
            else: @ !never
        @ !Switch !!resps
    @ unit

@all_panes: @unit = @widget:
    @inbox_pane
    @drafts_pane
    @settings_pane
    @ unit
```

`$shown` is true for every non-`Ui.*` slot and for the `Ui.*` slots of the active pane, so a pane's sockets, file reads and timers reach the World whether the pane is showing or not, and its buttons and labels reach it only while it is. A slot never changes, so `Pair $_, $shown` changes only when `$shown` does, and the `rebuild` over it rebuilds a row exactly when its slot becomes shown or hidden: the hidden row's element leaves the interceptor's list, the shown row's arrives, and the World sees one pane's buttons go and another's come. Each forwarded element's response is the row's result through `!Switch`, consolidated into `!down : !(Path ==> Response)`, which `intercept` routes to the child by path; the one-occurrence lag of the switch costs nothing, since a freshly forwarded slot cannot be answered in the occurrence it was created anyway. The supplier is the transparent one, since no pane here has a state element the interceptor wants to answer. This is chapter 6's `@Forward` with one `rebuild` added per row, and chapter 6's library containers are the same construction with layout as the policy instead of "shown". In the `$shown` line, `(PaneOf key)` and `(IsUi $_.element)` are parenthesised because a paren-less call takes the rest of its expression: `PaneOf key == Some $tab` would hand `PaneOf` the comparison as its argument.

Every pane's slots, requests and state then exist all the time; the World sees only the active pane's drawing; an inbox whose socket should stay open while you edit settings is this version. The cost is that every pane exists, which is a cost you pay knowingly, in a line that says `intercept`.

## A modal dialog

A dialog is a widget that exists for a while and whose result is the answer. The thing to get right is that the dialog's existence is a hold's value, the dialog's answer is what closes it, and the two form a cycle that is legal.

```
type Answer = choice Confirmed | Cancelled

@delete_with_confirm: fn Text -> @!unit = \|what| @widget:
    !open <- @Button $"Delete {what}"
    $dialog <- Nil then (Some unit) on !open | Nil on !answer
    !!answers <- rebuild $dialog:
        match _:
            Nil: @ !never
            Some ignore:
                @Label $"Really delete {what}?"
                !yes <- @Button $"Confirm"
                !no <- @Button $"Cancel"
                @ (Confirmed on !yes | Cancelled on !no)
    let !answer = !Switch !!answers
    @ unit on !Filter (IsConfirmed !answer)
```

`IsConfirmed: fn Answer -> ?unit` is `Some unit` for `Confirmed` and `Nil` otherwise, so `!Filter` over it is a unit event that fires on confirmation.

`$dialog : $?unit` is open or closed. The `rebuild` over it builds nothing but a never-firing result while closed, and the dialog while open. `!answer` is the `!Switch` over generations; `$dialog`'s hold lists both the thing that opens it and the thing that closes it, and the widget's result is a unit event that fires on confirmation.

**Occurrence 0.** `$dialog` is `Nil`; generation `#0` is the `!never` block, which has no slots; `!!answers` fires with `!never`; `!Switch` adopts it. The list has one button.

**Occurrence 1, click Delete.** `!open` fires; `(Some unit) on !open` fires; the merge's left wins (its right is quiet); `$dialog`'s change event fires with `Some unit`. The `rebuild` builds anew: generation `#1` is built with a label and two buttons, at `1/#1/0/#0/0`, `1/#1/1/#0/0`, `1/#1/2/#0/0`; `!!answers` fires

with the new block's answer event. `!Switch` is still on `!never` for this occurrence. Commit: `$dialog` is `Some unit`, the switch adopts generation `#1`'s event. The World draws the dialog.

**Occurrence 2, click Cancel.** The World sends `click` to `1/#1/2/#0/0`. `!no` fires; the answer event fires with `Cancelled`; `!Switch` delivers it as `!answer`; `Nil on !answer` fires; `$dialog`'s merge: `Some unit on !open` is quiet, so `Nil` wins; the change event fires with `Nil`; the `rebuild` builds anew to generation `#2`, the empty block; `!!answers` fires with `!never`. Generation `#1` keeps propagating until commit, which is how its `Cancelled` reached the switch at all, and is dropped at commit. `IsConfirmed` says no, the result is quiet.

**Occurrence 3, click Delete again.** As in occurrence 1: generation `#3` is built, with fresh paths `1/#3/...`, and the switch adopts its answer event at commit. The dialog that was cancelled is not "reopened"; a new one exists.

**Occurrence 4, Delete and Cancel together.** The user clicks the Delete button behind the dialog and Cancel in the same frame, and the World delivers both. `!open` and `!answer` both fire. The hold's merge takes the left, `Some unit`, and the dialog stays open: generation `#4` is built because the change event fired, even with the same value as before, and generation `#3` is dropped. The user sees a dialog where there was a dialog. A program that finds that rebuild wasteful filters the hold's event so that opening an open dialog is not a change, and a program that wants Cancel to win writes the merge the other way round; either way the answer is on the hold's line.

**Occurrence 5, Confirm.** `Confirmed` reaches the switch, the hold closes the dialog, the `rebuild` builds anew to the empty block, and the widget's result fires `unit`. Whoever bound `@delete_with_confirm` gets one event, in the occurrence of the click, and the dialog is gone by commit.

What the program does not have: a dialog manager, a z-order stack, a `showModal()` that returns a promise, or any notion of the dialog "closing itself." The dialog's block returns an event; the parent's hold decides what the event means for the dialog's existence; and the cycle between them is the same cycle as the clicker's, closed through the switch. Where the dialog is drawn, over everything else, is a layout question for an interceptor that reads the element list and puts the dialog's elements last.

## A list with selection

Master-detail had the selection outside the list and the detail outside the list. The variant where the rows themselves show the selection is the one that tests whether you have the ownership right, because the easy version rebuilds every row on every click.

```
let Mark: fn Text, Bool -> Text = \|t, sel| if sel: "> {t}" else: "  {t}"

@selectable: fn $(Nat ==> Item) -> @$?Nat = \|$items| @widget:
    !picks <- for $items:
        let $is_selected = $selected == Some key
        !click <- @Button (Mark $_.title, $is_selected)
        @ key on !click
    $selected <- Nil then Some (!Filter (FirstKey !picks))
    @ $selected
```

`$selected` is outside the `for`, and each row reads it through a lifted `==`. The button's label is a dynamic, so inside each row the button is a `rebuild` over its label.

**Occurrence 1, click row 2.** `$selected` changes to `Some 2`. In row 2, `$is_selected` changes to `True`, the label changes, and the button's `rebuild` builds anew its slot to generation `#1`. In every other row `$is_selected` is recomputed, since `$selected` changed, and is `False` as before; its change event

fires with the same value; the label is recomputed to the same text; the label's change event fires with the same text; and the button's `rebuild` builds anew, because a `rebuild` builds anew when its dynamic's change event fires, and equality of the value is not something a derived dynamic checks for you. Every row's button gets a new slot.

That is correct and it is wasteful. Correct, because the World diffs plain data and will draw nothing differently for the rows that did not change. Wasteful, because every row rebuilt a slot, and because every row's button lost its path, which a World keying native widgets by path would notice. The fix is a prelude function that stops change events with equal values, applied where the fan-out is:

```
        let $is_selected = $Distinct ($selected == Some key)
```

`$Distinct: fn $a -> $a` for `a: Eq`, a derived dynamic whose change event fires only when the value differs from the value before. With it, clicking row 2 changes `$is_selected` in exactly two rows, the one that was selected and the one that now is, and two buttons rebuild their slots. The rest of the rows see no change event and do nothing.

Why is `$Distinct` not the default? Because the change law says a derived dynamic fires its change event when an operand's does, and making that conditional on equality would require every derived dynamic's value type to be comparable, which functions, events and blueprints are not. The dynamic that fires with an equal value is a correct dynamic; it is only an expensive one when a `rebuild` sits downstream, and the program says where the de-duplication goes.

**Occurrence 2, item 2's title changes.** Row 2's `$_` changes, its label changes, its button rebuilds. No other row is touched, and `$selected` is `Some 2` still.

**Occurrence 3, row 2 is deleted.** Its block is torn down at commit. `$selected` is `Some 2` and nothing is selected on screen; the widget's result still says `Some 2`, which is the truth about the hold, and the parent decides what a selection of a missing key means.

The contrast case is putting the selection inside: a `$is_selected` hold per row, toggled by the row's own click. Then clicking row 3 does not deselect row 2, because row 2's hold did not hear about it, and making it hear means broadcasting an event into every row, which is the consolidated event upside down. The selection is a fact about the list, so it lives in the list's widget, and the rows read it.

## A wizard

A sequence of pages, each with inputs, with Next and Back, and the answers collected at the end. The trap is where the answers live: in the pages, they die with the page; in the parent, each page needs its answer as a plain initial value and returns the edited one.

```
type Step = choice Name | Address | Review

type Answers = record:
    name: Text
    address: Text

type Nav = record:
    step: Step
    answers: Answers
    done: Bool

let IfDone: fn Nav -> ?Answers = \|n| if n.done: Some n.answers else: Nil
```

```
@wizard: @!Answers = @widget:
    let $page = Pair $step, $answers
    !!navs <- rebuild ((step, answers)) <- $page:
        match step:
            Name:
                TextInput(ignore, $name) <- @TextInput answers.name
                !next <- @Button $"Next"
                let $forward = Nav Address, (Answers $name, answers.address), False
                @ $forward on !next
            Address:
                TextInput(ignore, $address) <- @TextInput answers.address
                !back <- @Button $"Back"
                !next <- @Button $"Next"
                let $edited = Answers answers.name, $address
                @ (Nav Name, $edited, False) on !back | (Nav Review, $edited, False) on !next
            Review:
                @Label $"{answers.name}, {answers.address}"
                !back <- @Button $"Back"
                !finish <- @Button $"Finish"
                @ (Nav Address, answers, False) on !back | (Nav Review, answers, True) on !finish
    let !nav = !Switch !!navs
    $step <- Name then !nav.step
    $answers <- (Answers "", "") then !nav.answers
    @ !Filter (IfDone !nav)
```

The `rebuild` is over `Pair $step, $answers`, one dynamic of the pair, and the pattern form takes it apart into two plain values. Inside a page, `answers` is plain, so `@TextInput answers.name` gets a plain initial and is built once per page visit; `$name` is the input's live value; `$forward` is a lifted record over it; `$forward on !next` is an event with the record's value as it stood before the click, which is the text as typed.

**Occurrence 0.** `$step` is `Name`, `$answers` is empty, `$page` is the pair, the `rebuild` builds the Name page.

**Occurrences 1 through 5, typing.** Each keystroke changes `$name` inside the page: the text input's own label slot rebuilds, `$forward` changes, nothing else. `$page` does not change, because neither `$step` nor `$answers` did. The page is not rebuilt.

**Occurrence 6, Next.** `!next` fires; `$forward on !next` fires with `Nav Address, (Answers "Ada", ""), False`; the switch delivers it as `!nav`; `!nav.step` fires with `Address` and `!nav.answers` with the record; both holds' change events fire in this occurrence; `$page`, a derived dynamic over both, changes once (glitch freedom: one change event per occurrence however many operands moved); the `rebuild` builds anew to the Address page with `answers.name = "Ada"` plain, and an address input initialised from `""`. The Name page is dropped at commit, and `$name` with it; the name is safe in `$answers`, which is outside.

**Occurrence 7, Back.** The Address page's `!back` fires with `Nav Name, (Answers "Ada", "..."), False`, whatever had been typed so far; both holds move; the `rebuild` builds anew the Name page with `answers.name = "Ada"` as the input's initial. The text is there. It did not survive in the page; it went out and came back in, and the line that says so is the hold for `$answers`.

**Occurrence 8, Finish.** `done` is `True`; `!Filter (IfDone !nav)` fires with the answers; the widget's result fires. The holds also move (to the same step and the same answers), so `$page` changes with an equal value and the Review page is rebuilt, which is harmless and could be stopped with `$Distinct` if the review page were expensive.

What this pattern shows about `rebuild` over a tuple: the rebuild happens when either part changes, and the design makes both parts change only together, in a navigation occurrence, so one rebuild per navigation. Had the answers been saved live, on every keystroke, `$page` would change on every keystroke and the page would rebuild under the user's cursor; that program is also writable, and it is wrong, and it is wrong in a way you can see from the line `let $page = Pair $step, $answers` together with the line that moves `$answers`. Where the state lives and what the `rebuild` is over are the whole design of a wizard, and they are two lines.

## The invariants, and who keeps them

Everything above rests on a short list of promises, each with a party bound by it and a mechanism that enforces it. They are gathered here because a reader changing the design should know which sentence breaks when a rule does.

*Every graph object is owned by exactly one widget, and widgets form a tree.* Bound: the runtime and the compiler. Enforced: widgets are not values in the language (the compiler has no construct that yields one), `WidgetBuilder::bind` is the only way to make a child and it pushes the child onto the parent, subscriber lists are weak, and closure conversion makes captures indices into the owning widget's environment. The test `dropping_a_widget_frees_its_cyclic_loops` checks the one place a widget could own itself.

*A replaced widget propagates until the commit of the occurrence that replaced it, and is dropped then.* Bound: the runtime. Enforced: the rebuild action replaces the child and schedules the old widget's drop at the commit height; nothing drops a widget from inside propagation. Checked by `a_replaced_child_keeps_working_until_its_last_occurrence_ends`.

*A widget built in occurrence n sees all of n.* Bound: the runtime. Enforced: every event caches its current-occurrence value and `subscribe` returns it; holds, switches and `rebuild` builds anew triggered by an already-fired event defer to `max(height, current + 1)`; closures run at most once per occurrence. Checked by the `_mid_occurrence_` tests and the nested-`rebuild` test.

*A `rebuild`'s event fires in every build occurrence, the first included, and goes to the caller.* Bound: the runtime. Enforced: the result is the leftmost of the rebuild event and the done-event-mapped first build, and the block has no name for it.

*`!Switch` is non-prompt; `$Join` is prompt.* Bound: the runtime. Enforced: `switch_hold` reads its inner event from a hold's value side; `Dynamic::join` fires on the outer's change with the new inner's value after n. The conformance suite has a law test for the swallowed creation-occurrence firing of the switch and three for join: inner changes alone, swap to an inner that changed in the same occurrence, swap to an inner that did not.

*A path is never reused.* Bound: the runtime. Enforced: each `rebuild` widget keeps a generation counter that only goes up, incremented on every rebuild and never reset while the widget lives; each `for` widget's key segment is the key itself, and a key's widget is dropped when the key leaves, so a returning key is a new widget under the same segment with fresh generations beneath it. A quarantine does not touch the counters: a quarantined subtree is held out of rebuilds, and when the World lifts the quarantine the next rebuild takes the next generation, not `#0`. A World may therefore remember anything by path and be sure that a path it has seen leave the list will not come back meaning something else.

*A `for` key that returns is a new key.* Bound: the runtime, which has no table of departed keys and builds a returning key's block from scratch; and the program, whose allocator decides whether keys

can return at all. Enforced on the runtime's side by the keyed list widget owning exactly the widgets of the keys currently present; on the program's side by nothing but the allocator's line, `$next <- 0 then $next + 1 on !add`, which is why the idiom is written that way.

*A failed build fires no event and leaves a hole.* Bound: the runtime and the World. Enforced: the rebuild action, on a contained failure, tears the old generation down at commit, installs no child, and does not notify the `rebuild`'s subscribers; the error is returned from the run call as kind `Build` with the `rebuild` widget's path as its owner, and offered through `errors` if the program subscribed. A cycle found during the build is kind `Cycle` and aborts the occurrence instead, which keeps the old generation by rolling everything back. The World is bound to report what it is returned and may send `quarantine` paths on its next call. Chapter 9 has the type and the protocol; chapter 10's `errors` step is the check.

*A `for` keeps a widget per key; a key that appears is built in its occurrence and a key that disappears is torn down at commit; consolidation is by result kind and is prompt.* Bound: the runtime and the compiler (which states the result kind). Enforced: the keyed list widget projects a per-key dynamic and defers builds like a `rebuild`; the merge for an event result includes blocks built in the occurrence.

*Holds, cycles and `@Element` appear only in widget blocks.* Bound: the program; enforced by the compiler, which has no other place to put them.

*A cycle inside one widget block passes through a delayed edge.* Bound: the program; enforced statically by the compiler over the block's groups, with `rebuild` edges marked prompt. Cycles formed through widget functions or by switching are found by the runtime's height comparison and abort the occurrence or the construction.

*Nothing fires because a widget appeared or was dropped.* Bound: the runtime, Worlds and interceptors. Enforced in the runtime by there being no such event; in Worlds by the obligation list, as a norm the Test World checks with `quiet`.

*Building has no effects.* Bound: the compiler and the runtime. Enforced: `build` is handed a builder and nothing else; the only thing it can do to the world is put a slot in a widget, which the World reads after the occurrence.

## The negative space

Each of the things widgets cannot do is a hole you have fallen through, and each is also a mechanism the implementation does not have to build. The specification lists the refusals for the whole language; these are the four that belong to widgets, with what each buys.

**No widget values.** You cannot keep a reference to a button, a pane, a row. A widget is a position in the hierarchy and nothing else, and the only way to get one is to bind a widget at that position. What this closes: the detached subtree kept alive by a stale listener, the parent whose decision to remove a child means nothing because someone else still holds it, the ownership graph that needs a collector because it is not a tree. What it buys the implementation: ownership is a tree of `Rc`s pointing down, subscriber lists are `Weak`, closures reach the graph through the widget that owns them by environment index, and teardown is `drop`. The Rust runtime has no tracing collector and will not need one; the one threat, a closure capturing a sibling's widget strongly, is closed by closure conversion making captures indices rather than pointers, and cyclic placeholders inside one block are the explicit exception, owned by the widget as a unit.

**No lifecycle hooks.** There is no `onMount`, `onUnmount`, `componentDidUpdate`, `useEffect`, `ngOnDestroy`.

A widget is built, which has no effects, and torn down, which has no effects. What this closes: the cleanup function you forgot, the effect that ran in an order the documentation told you not to rely on, the hook that fired twice in development mode and once in production, the unmount that raced the fetch. What it buys: build is a pure function of the arguments and the occurrence, so two builds with equal inputs make equal widgets; there is no second ordering system beside propagation; and teardown is a drop at commit with nothing to run. The thing you reached for a mount hook to do, kick something off when the widget exists, is `@started` plus a request event, and it is in the block, where the widget's other events are, with the same laws.

**No request on creation.** Creating an element does nothing. A slot appears in the list and the World reads the list; if the program wants something performed it fires the slot's request event, and `@started` exists so it can do that in the build occurrence. What this closes: the fetch-on-mount that fires on every rebuild, the test that cannot run without a mock installed before the first render, the rebuild whose cost is a surprise. What it buys: the diode. Inputs cause list changes and list changes never cause inputs, so there is no loop across turns where a rebuild requests, the response rebuilds, and the cycle check never sees it, and a rebuild costs exactly what its block asks for, which is visible in the block.

**No mutation.** The only state is a hold; the only way to change one is its event; there is no assignment anywhere, not in a block, not in a helper, not in a closure. What this closes: the counter with six updaters in six files, the handler that writes the model before the list reads it, the "which version did I see." What it buys: every write in an occurrence is a hold's commit, so all writes are buffered to the end and an occurrence can be aborted by dropping the buffer. The transaction is not a feature the runtime adds; it is what is left when there is nothing to protect against. Rollback on error is the same drop. And the ownership story above depends on it: a widget owns its holds because holds are the only state, and if there were another kind of state, something would have to own that too.

The pattern is the one the specification promises for the language as a whole. A feature you would have sworn a widget system needs is absent; the absence closes a bug you have shipped; and the same absence lets the runtime leave out the machinery the feature would have needed. Widgets are positions, so no collector. No hooks, so no lifecycle scheduler. No request on creation, so no reconciliation of effects against renders. No mutation, so no dirty tracking and free transactions. What remains is a tree, a graph hanging off it, one entry point, and the three constructs of this chapter for changing the tree in the occurrence the program says to.

## Chapter 6

# Elements and interception

An element is the only thing a Prism program can say to the outside, and the only thing the outside can address when it speaks back. This chapter defines that one channel in full: what an element is and how its tag declares what it does; the rule that nothing an element asks for can be answered in the occurrence it asked; the two identities a slot has and why they are different; the element list as an ordinary dynamic that an interceptor or a World reads; and `intercept`, the one primitive that lets a Prism widget stand where the World stands, so that a GUI toolkit, a sandbox, a cache, a mock, a tracer and a program that hosts other programs are all written in the language rather than under it. The chapter is long because everything about the boundary is here, and because the boundary is where every other framework keeps a second set of rules.

## What an element is

You have shipped the component that fetched on mount. It was fine until the list it lived in re-rendered, and then it fetched again, and the second response landed after the third mount's, and the row showed a stale avatar for a user who had been scrolled away. You fixed it with a cancellation token in a ref, and the fix had a race in it that the test suite could not reach because the test suite needed a mock server before the first render just to get past `componentDidMount`. Somewhere in that afternoon you wrote the sentence "the network layer has its own lifecycle," and that sentence is the problem this chapter removes.

Three promises meet at the boundary and pull in different directions.

The first is that IO must be impossible from inside the language. A program may not write a byte, open a socket or read a clock; the promise that a hold's definition lists everything that can change it is only true if nothing can reach around it. So whatever a program says to the outside has to be data, with no function and no object in it, and whatever the outside says back has to arrive the way every other value arrives, as an event firing or a dynamic changing in an occurrence.

The second is that the toolkit, the sandbox and the mock must be writable in Prism. Layout, focus, text editing, confinement, caching, tracing: each is something a program wants between itself and the world, and if the language cannot express "stand between a child and the world", every one of them is native code, with native code's rules. So the boundary has to be a thing a Prism program can stand on either side of, and it has to look the same from below whether a World or a widget is above it.

The third is that a widget which asks for state must get a type that tells the truth. The file that has not been read is `Unavailable`, as a case of the type, and the `match` that draws it has an arm for that. But a dynamic has a value at every occurrence, so a widget that asks for the contents of a file as a dynamic must get a value in the occurrence it asks. That forces a question most systems never put to themselves: when does outside state exist for a program? The answer here, in the occurrence the slot is created, is the only one under which a widget built in the middle of an occurrence is consistent

with everything else built then, and the second half of this chapter is the reasoning for it.

An element is what satisfies all three. It is a plain value of the extensible variant `Element`: a tag and some fields, `Ui.Button { label: "Increment" }`, `Fs.File { path: "/etc/motd" }`, nothing else. A widget puts one in its element list by calling `@Element`, which creates a *slot*: a position in the widget tree where that element sits for as long as the widget lives. Whoever is outside reads the list, does whatever the element means to it, and sends back responses addressed to the slot and state keyed by the element's value. The book calls that outside a *context*: the World is the external context, written in the host language, and an interceptor is an inner context, written in Prism, and a subtree cannot tell which one is answering it. That is the entire protocol. There is no handle, no callback, no promise, no context object; there is a list of plain data going out and a map of plain data coming in, once per occurrence.

The consequence that takes getting used to is that an element does nothing. A `Ui.Button` is not a button; it is a request to whoever is outside to show one and to say when it is clicked. An `Fs.File` is not an open file; it is a declaration that this widget may ask for reads and writes of that path and wants the answers. Whether anything happens is up to the outside, and the program's only way to find out is a response arriving. This is what makes the same program run under a test World with no screen and no disk: the program never did anything, it only ever said what it wanted, and the World that says "done" without doing is as conforming as the one that does.

## Tags

A tag is the declaration of one kind of element: its name, its fields, and which of three things it does. Every element in the language is built from a declared tag, and the declaration is where the types that cross the boundary are fixed.

```
type Element = choice ..

tag Ui.Label of Element:
    text: Text

tag Ui.Button of Element:
    label: Text
    responds: Click

tag Fs.File of Element:
    path: Text
    requests: !FsRequest
    responds: FsResponse

tag Fs.Contents of Element:
    path: Text
    state: Contents

tag Clock of Element:
    state: Instant
```

`type Element = choice ..` declares an extensible variant: a choice whose alternatives are added by `tag` declarations across modules rather than listed in one place. `Ui.Label` and `Fs.File` are constructors of it, namespaced, and a value built from one is a plain value with structural equality like any record. The constructor is called the way a record is, positionally or with named fields: `Ui.Button(label="OK")`, `Fs.File(path=p, requests=!reqs)`.

The three parts a tag may declare, in the order a reader meets them:

- **Fields** are plain data, fixed for the life of the slot. `label: Text`, `path: Text`, `period: Int`. A field's type is concrete; a tag may not mention a type variable, so generic code cannot build an element except at a type it knows, because there is no generic tag to build. Fields are what the outside reads to know what the element is.
- `requests: !R` declares an event field. The program supplies the event when it creates the element, and its firings go outward: after the occurrence in which the event fired, the outside sees the values fired, in order. A request event fires at most once per occurrence like every event, so a request type that wants to say two things at once is an array, `requests: ![ConnRequest]`.
- `responds: R` declares that the element returns an event `!R`. Its firings come from outside, addressed to this slot by path, in occurrences after the one in which the slot was created.
- `state: S` declares that the element returns a dynamic `$S`, supplied from outside, with a value in the occurrence the slot is created and a change whenever the outside pushes one. State is keyed by the element's value, not its path, which is the subject of the section on identity.

A tag declares any subset, including none. The subsets that occur are the kinds of element you have met under other names elsewhere, and they are combinations of three parts, not five different mechanisms:

| requests | responds | state | what it is | examples |
|---|---|---|---|---|
| | | | presence: the element being in the list is the whole message | `Ui.Label`, `Gui.Scene`, `Gui.Focusable` |
| ✓ | | | sink: firings go out, nothing comes back | `Process.Exit`, `Log.Line` |
| | ✓ | | stream: things arrive for as long as the slot lives | `Cli.Stdin`, `Net.Listener`, `Time.Every`, `Ui.Button`, `Gui.Window` |
| ✓ | ✓ | | request: one firing in, some number of answers later | `Fs.File`, `Net.Http`, `Time.After`, `Cli.Stdout`, `Defer` |
| | | ✓ | state: a value that is simply true, pushed when it changes | `Clock`, `Process.Args`, `Fs.Contents` |
| | ✓ | ✓ | a watched thing that also talks | a feed that is up or down and delivers messages: `Net.Socket` without the send |

| requests | responds | state | what it is | examples |
|---|---|---|---|---|
| ✓ | | ✓ | a thing you drive and observe | `Process.Child` without its output |
| ✓ | ✓ | ✓ | the full conversation | `Net.Socket`, `Process.Child` |

A button is a stream in this table, and that is right: a click is something that arrives, and the button asked for nothing. A request element is a stream with an input. A state element is neither; it is the outside's view of a fact, offered as a dynamic. Nothing in the language treats the rows differently except by what `@Element` returns.

The alternative worth a paragraph is one part per tag, so that a socket is three elements: a `Net.Send` sink, a `Net.Receive` stream and a `Net.Link` state, each with its own slot. It is cleaner in the declaration and it costs a path per part. A program then has three slots to keep together, a World has three slots to correlate into one connection, and an interceptor that wants to forward a socket has to forward three things that are only a socket if they travel together. A tag with three parts says in one place that these three things are one conversation, and the slot has one path for its responses and one value for its state. The cost is a declaration syntax with three optional lines in it, and that is cheap.

## A dozen declarations

Here are the tags the rest of this chapter uses, each with the one sentence that says why its parts are what they are. These are the GUI, filesystem, network, time and process vocabularies a World is assumed to accept; a World that accepts more declares more, and a World that accepts fewer answers `Unsupported` to the rest.

```
type Click = click
type Tick = tick
type Instant = Instant Int                  # milliseconds since the World's epoch

tag Ui.Label of Element:                     # presence: a draw command
    text: Text

tag Ui.Button of Element:                    # stream: clicks arrive
    label: Text
    responds: Click

tag Gui.Window of Element:                     # stream: pointer, keys and resizes arrive
    title: Text
    size: Extent                            # the size the program asks for
    responds: WindowInput

tag Gui.Scene of Element:                    # presence: the picture, as data
    window: Text
    draw: [Draw]

tag Gui.Focusable of Element:                # presence: the name of the region that has focus
    name: Text
```

A label asks for nothing and hears nothing; it is on the screen because it is in the list and off it because it is not. A button hears clicks. The three `Gui` tags are three of the five that a World drawing pixels accepts; the GUI section declares the other two, text measurement and frame ticks. A window is a stream: the pointer, the keys and the platform's resizes arrive as responses, and nothing in the

window decides who gets them. A scene is a presence element like a label, the list of draw commands for that window, replaced whenever the picture changes. A focusable is a presence element too: it names the region the library considers focused, so that a World can show the platform's focus ring, place an input method's candidate window or tell a screen reader, and it hears nothing, because which region gets a key is decided in Prism by name. Everything between a program's buttons and those tags is Prism, and the long section on the GUI library builds it.

```
type FsRequest = choice Read | Write Text
type FsResponse = choice:
    Contents Text
    Missing
    Denied
type Contents = choice:
    Unavailable
    Known Text
type Listing = choice:
    Unlisted
    Entries [Text]

tag Fs.File of Element:                      # request: reads and writes, answered later
    path: Text
    requests: !FsRequest
    responds: FsResponse

tag Fs.Contents of Element:                   # state: the file as a value that is true now
    path: Text
    state: Contents

tag Fs.Dir of Element:                        # state: the listing as a value that is true now
    path: Text
    state: Listing
```

Two ways to get at a file because there are two things you might mean. `Fs.File` is a conversation: ask, and be answered some occurrences later, once per ask, with a response that can say the file is missing or the read was denied. `Fs.Contents` is a fact: the contents of the path as a dynamic, `Unavailable` when the World does not have them, `Known` when it does, pushed when they change. The request form is for writes and for one-shot reads where the program wants to know that *this* ask was answered; the state form is for anything a program wants to show.

```
type SocketEvent = choice:
    Received Text
    Closed
type Link = choice:
    Connecting
    Connected
    Disconnected Text
type Accepted = Accepted Nat
type HttpRequest = record:
    method: Text
    url: Text
    body: Text
type HttpResponse = record:
    request: HttpRequest
    status: Nat
    body: Text

tag Net.Socket of Element:                    # the full conversation
```

```
    address: Text
    requests: !Text
    responds: SocketEvent
    state: Link

tag Net.Listener of Element:                # stream: accepted connections, with ids the World allocates
    port: Nat
    responds: Accepted

tag Net.Http of Element:                    # request: one firing, one answer, later
    requests: !HttpRequest
    responds: HttpResponse
```

A socket has all three parts because a connection is all three things: you send on it, you receive from it, and it is up or not. `Net.Http` has no fields at all: the URL is in the request, so one slot can fetch many things over its life, and the response says which request it answers so that two in flight can be told apart. A listener is the stream of accepted connections; each accepted connection is its own `Net.Conn` element keyed by the id the World allocated, built under a `for`, which is the server of the worked examples.

```
tag Time.After of Element:                  # request: a delay in milliseconds, echoed back once it has passed
    requests: !Nat
    responds: Nat

tag Time.Every of Element:                  # stream: a tick per period
    period: Int
    responds: Tick

tag Clock of Element:                       # state: the time, as the World last pushed it
    state: Instant
```

There is no `now()`. The clock is a fact the World pushes, at most once per turn, and a program that wants to be woken asks `Time.After` or keeps a `Time.Every` in its list. The split between the three is the split between asking once, subscribing, and reading, and it is the same split as between `Fs.File`, `Net.Listener` and `Fs.Contents`.

```
type ChildRequest = choice:
    Stdin Text
    Signal Nat
type ChildEvent = choice:
    Stdout Text
    Stderr Text
type Status = choice:
    Running
    Exited Int

tag Process.Child of Element:               # the full conversation again: a subprocess
    command: [Text]
    requests: !ChildRequest
    responds: ChildEvent
    state: Status

tag Process.Args of Element:                # state: the arguments, which an interceptor may replace
    state: [Text]

tag Cli.Stdin of Element:                   # stream
    responds: StdinEvent
```

```
tag Cli.Stdout of Element:                  # request: a write that fails comes back
    requests: !Text
    responds: WriteFailed

tag Process.Exit of Element:                # sink: the firing is the exit
    requests: !Code

tag Defer of Element:                       # request: answered in a later occurrence of the same turn
    requests: !unit
    responds: unit
```

A subprocess is a socket with a command instead of an address: the World starts it because the element is in the list, stops it when the element leaves, and its exit status is a fact. Arguments are state rather than parameters of `@main` so that an interceptor can give a subtree different ones and a test can set them. Exit is a sink whose firing is the instruction, never a presence whose appearance is, because nothing in this language is triggered by an element appearing. `Defer` is the one request element every World must provide, and it is what lets an interceptor answer a request at all; it is the subject of its own section below.

### The rules a tag obeys

Four rules, each a consequence of elements being plain data that crosses a boundary.

Field types are concrete. The boundary is between a typed program and an untyped outside, and the outside has to be able to read a field without knowing what type variable it was instantiated at. A tag that said `value: a` would be a tag no World could accept.

A `match` on `Element` ends in a catch-all arm, because the set of alternatives is open: a module this program has never seen may declare a tag, and an interceptor that forgot about it must still do something with it. In an interceptor the catch-all is the forwarding case, and leaving it out is the compile error that stops a sandbox from silently dropping an element it did not anticipate.

Matching is by tag, with fields bound by name as for a record, and `Ns.*(..)` matches any tag under a namespace. There is no matching on a field's type, because every field's type is fixed by its tag; `Fs.File(path=p)` binds `p : Text` and nothing else is possible.

Equality is structural, tag then fields. Two `Fs.Contents(path="/etc/motd")` values are equal wherever they were built, and that equality is what identifies state across the boundary: it is the key of the push map and the argument of the oracle. A field containing an event, which only a `requests` field does, is excluded from the comparison, because events have no equality; two elements that differ only in their request event are the same element to the outside, which is right, since the outside never sees the event, only its firings.

## `@Element`

`@Element e` is a blueprint. Binding it in a widget block creates a slot for `e` in the enclosing widget and returns what `e`'s tag declares:

| tag declares | `@Element e` has type |
|---|---|
| nothing | `@unit` |
| `responds: R` | `@!R` |

| tag declares | `@Element e` has type |
|---|---|
| `state: S` | `@$S` |
| `responds: R` and `state: S` | `@((!R, $S))` |

The slot exists from the occurrence in which the widget is built until its widget is torn down, and its element never changes: a field is read once, when the slot is created, and anything that looks like changing a field is creating another slot. `@Element` is therefore the only place where the program's text meets the outside, and a reader who wants to know everything a widget asks of the world greps for one word.

```
@Button: fn Text -> @!Click = \|label| @widget:
    !click <- @Element Ui.Button(label=label)
    @ !click

@viewer: fn Text -> @unit = \|path| @widget:
    !started <- @started
    !resp <- @Element Fs.File(path=path, requests=(Read on !started))
    $text <- "(reading...)" then ShowResponse !resp
    @Label $text
    @ unit

((!rx, $link)) <- @Element Net.Socket(address="chat.example:7000", requests=!out)
```

The three statements show the three return forms. A button returns its click. A file read returns its response event, and the request is wired in at creation: `Read on !started` is an event that fires once, in the build occurrence, and that firing is the one read this widget will ever make unless it fires the event again. A socket returns a tuple, and the pattern on the left takes it apart.

When the tag of `e` is not statically known, as in a catch-all arm of a `match` inside an interceptor, `@Element e` has the generic type `@((!Response, $State))`, and the response and state types are checked at the slot at runtime, per tag. In a tag-specific arm the types are the tag's, and the checker knows them. This is the one place in the language where a type is refined by a `match` on a tag, and it is what lets an interceptor forward an element it has never heard of while still being typed when it handles one it has.

One thing `@Element` does not take is a dynamic where a field goes. `@Element Ui.Label(text=$text)` is not a label whose text changes; it is an application to a dynamic, which lifts, so it is `$(@Element Ui.Label(text=text))`, a dynamic of blueprints. Binding a dynamic of blueprints lifts to `rebuild`: `@Element Ui.Label(text=$text)` as a statement means `rebuild $text: @Element Ui.Label(text=_)`, a fresh slot per change of the text, and a bound name on the left is the `!Switch` of the `rebuild`'s event when the element responds, or a hold of dynamics joined with `$Join` when it has state, with the first build's result as the hold's initial value. The chapter says this once here and uses the short form from now on; the section on dynamic fields says what it costs and why it is the right cost.

A program rarely spells this out for library widgets. Library widgets take dynamic arguments: `@Button` takes a `$Text` label, and a button whose label changes is written `@Button $label`. The fresh slot per change belongs to the library's interceptor, which builds a new `Ui.Button` slot when the label changes and answers to the program through the one `!Click` event it returns; a World never sees it, because a World sees only the `Gui.*` elements the interceptor emits. The rule that a field change is a fresh slot holds in full for every element that reaches a World, and the `rebuild` forms above are that rule written out, not the way a program normally labels a button.

# The creation rule

> **Elements never fire in the occurrence they were created. Firing within the same occurrence implies they could have been pure functions. Instead, elements with request-like semantics take an input event and fire their output event in a different occurrence.**

Read the second sentence twice, because it is the argument and not just the rule. Suppose an element could be created and answered in one occurrence. Then from the program's side, `@Element Fs.File(...)` with a `Read` fired from `@started` would be a widget that returns the file's contents in its build occurrence, which is to say a function from a path to text. But it is not a function from a path to text: it is a conversation with something outside that may take time, may fail, may be denied, and may never answer. A rule that let it answer at once would let a program depend on an effect having happened in the moment it was asked for, and every ordering bug at a boundary is some version of that dependency. The rule says the dependency cannot be written.

The practical statement is in three parts, one per party.

**The program** may assume that nothing it asked for has happened yet in the occurrence it asked. A widget that fires a `Read` from `@started` knows that its response event is quiet in the build occurrence and that its hold is at the initial value then. A widget that wants something as soon as it exists fires its request from `@started`, which fires in the build occurrence and is pure; the request firing in that occurrence is allowed and is the whole point, and the response arrives in a later one.

**Worlds and interceptors** are bound not to answer in the occurrence of the request, and not to offer any element that responds merely because it was created. The first is a norm a World cannot break by construction, since request firings come out of the run call after the occurrence commits and the earliest an answer can go in is the next call. The second is a norm a World could break, by starting a download because an `Fs.File` slot appeared and delivering the contents next turn; a conforming World does not, and the next subsection is the reason.

**The runtime** enforces the half it can. Responses enter the program only at the start of an occurrence, through the run call's response map, or through an interceptor's response event, which is routed against the element list as the occurrence opened. A slot created in occurrence n is not in that list, so nothing can be delivered to it in n: an interceptor that fires a response at a path created this occurrence has sent to an unknown path, which is an error that aborts the occurrence. That one check is what turns the rule from a convention into a law. An interceptor cannot cheat it, because the only way down to a child's slot is the response argument of `intercept`, and that argument is routed.

## No request on creation

A World could offer an element that acts because it exists: an `Fs.Load(path)` that reads the file when the slot appears and responds with the contents next turn. It would save a line in every file viewer, and it is what most toolkits do, and it is not offered, for three reasons that are the same three reasons `@started` exists.

Building stops being pure. Today building a widget adds widgets and slots and nothing else; with act-on-creation, every rebuild of a `rebuild` and every new key of a `for` is also an effect, at a moment the program's text does not show. The fetch-on-mount that fired twice was exactly this.

The trigger stops being a value. With `Read on !started` the program can merge the start with a refresh button, `Read on (!started | !refresh)`, gate it on a setting, or replace it with a click,

without touching the element. A test sees the request fire as an ordinary event and can assert `requested Fs.File(..), Read`. With act-on-creation the trigger is the slot's existence, which is not an event and cannot be merged with anything.

There would be two kinds of request element, the ones that act on build and the ones that act on events, and every interceptor between a widget and the World would have to handle both: a sandbox would deny some requests by answering their events and others by refusing to forward their elements, and a cache would memoise two protocols. With one kind, an interceptor answers requests and nothing else.

The objection is that `@started` is act-on-creation with an extra step. It is one step, and the step is the point: the program wrote down when it asks.

### The complement: state has a value

The rule has one complement, and it is not an exception. A `state` element has a value in the occurrence it is created. That is not a firing; it is the dynamic being a dynamic. The value it has in that occurrence is the value the outside held as the occurrence opened, from the World's snapshot or an interceptor's supplied dynamic, and if the outside pushes a change in that same occurrence the new slot sees the change like every slot with that value. A state element is an observation of something that already exists, the way reading a dynamic is, and creating the slot asks for nothing. The section on why state has to be there in its first occurrence argues that no other arrangement keeps a widget consistent with itself.

## Identity

A slot has two identities, and they answer different questions.

### By path

Every slot has a path: its position in the widget tree, made of the bind positions, `for` keys and `rebuild` generations from the root down to it. The worked examples write a path as `0/#1/[7]/2`: the first bind in `@main`, generation one of the `rebuild` there, key 7 of the `for` inside it, the third bind in that block. A path is unique for the life of its slot and is never reused: a `rebuild` that rebuilds gives the new slot a new generation number, a `for` key that leaves and comes back gets a new path, and a World that keeps a stale path in a queue will find it refused rather than redirected. An implementation may represent paths as it likes provided the element list is in tree order and the outside can treat a path as an opaque key.

Responses are addressed by path. A click is for *this* button, not for every button with this label; the answer to a read is for the slot that asked, not for every slot reading that file. A path is the identity of a conversation.

### By value

A `state` element's dynamic is identified by the element's value, tag and fields compared structurally. Two slots whose elements are equal observe one and the same state: whoever supplies it does so once per distinct value, however many slots share it, and every push keyed by that value reaches every one of them, including a slot created in the occurrence of the push. A value is the identity of a fact.

Two examples show the distinction, and they are the two the worked examples walk through.

**The socket.** `Net.Socket(address="chat.example:7000")` has a request, a response and a state. Its sends and receives are its own: addressed by path, a conversation between this slot and the connection. Its link state is shared with every other socket element of the same address in the program, because there is one connection, and whether it is up is a fact about the world and not about any slot. Two chat panes to one server are two conversations on one link. If the user types the old address back while a second slot still has it, the connection stays up through the change, which is either a feature or a surprise depending on what the program wanted, and either way it is in the program's text. Whether the World keeps one OS socket per address or per slot is the World's business and is invisible so long as it keeps the equality law.

**The clock.** `Clock` has no fields, so every clock element in the program is one clock. A stopwatch, a status bar and a log stamp each bind `@Element Clock` without knowing about each other, and in every occurrence all three read the same instant, because there is one value and the World pushes it once. A test sets it once, `with Clock = Instant 5000`, and all three see it. Had state been identified by path, the three would be three subscriptions the World had to answer identically, and the equality law would be a promise the World made three times per occurrence instead of a fact about one key.

The two identities are not in tension. A slot with all three parts has a path for its responses and a value for its state, and the two never cross: a response never reaches a slot because its value matched, and a push never reaches a slot because its path matched. The alternative, state by path, is worth imagining for a paragraph. Each `Fs.Contents` slot would be its own watch; a hundred rows reading one file would be a hundred watches at the World; the World would be obliged to push the same value to all hundred in one occurrence and to answer the oracle identically for each; and an interceptor that wanted to share one outer query among many inner slots would need a table from value to supplied dynamic, which is the table the runtime keeps anyway under value identity. Identity by value puts the sharing where it is free.

## The element list

The element list of a widget is an ordinary dynamic, `$(Path ==> Slot)`: a map from path to slot, in tree order because paths are ordered that way. Its value before occurrence n is the list as n opened; its change event, if the list changed in n, fires with the list after n. A slot created in n is in the list after n; a slot torn down in n is in the list before n and not after. Interceptors read it through `intercept`, and at the root the World reads it out of the run call.

```
type Slot = record:
    element: Element
    !request: Request          # typed by the tag, once `element` has been matched
```

A slot is its element and, if the tag declares one, its request event. The request event is in the slot so that whoever reads the list can wire it into an element of their own; the response event and the state dynamic are not in the slot, because they are what the reader supplies, not what the reader sees.

Because the list is a dynamic, everything the language says about dynamics applies to it. It changes promptly in the occurrence a slot is created or torn down. Reading it through a tag gives the list as the occurrence opened. Deriving from it is a derived dynamic: `GuiOnly $slots` is a dynamic of the GUI elements, kept current for free. And a `for` over it builds one block per slot, with the slot's path as the key, which is how every interceptor in this chapter is written.

### Routing

An input addressed to a path in occurrence n is delivered to the slot at that path in the list *as n opened*. Clicks go to what was there before the click. A slot created in n is addressable from n+1. An input addressed to a path not in that list is an error that aborts the occurrence: the sender had the list and sent to something not in it.

This is the tag law at the boundary. The list as the occurrence opened is the list's value before the occurrence, and reading a dynamic at an event gives its value before the occurrence. There is no separate routing rule to learn; routing reads the list the way every other read at an event reads a dynamic, and the consequences follow from that. A button clicked in the occurrence it is replaced delivers its click to the old slot, under the old label, because the old slot is what the user saw and what the World's list named. A hit-test in a layout interceptor reads the child's list through a tag and so hit-tests against the list the user was looking at, with no extra rule. A World that queued a click for a path and then received a list without that path must drop the click, and the obligation is stated in the chapter on Worlds; an interceptor that does the same thing gets an error, because an interceptor is Prism code and Prism code that sends to an unknown path has a bug.

### The diode

Inputs cause changes to the element list; changes to the element list never cause inputs. A new occurrence comes only from outside, with inputs addressed to slots that already existed. Nothing in the language observes its own element list and reacts to it except an interceptor, and an interceptor reads it through a delayed edge: the `for` over the list builds blocks for new slots in the occurrence they appear, but nothing a new block fires can be delivered to the new slot in that occurrence, by routing.

The diode is why there is no feedback loop in a program whose label is a fresh slot per keystroke. If a slot appearing were an event, a text input whose label rebuilds on every edit would fire an event on every edit and the World would have to decide whether that event causes anything, and some World would decide yes. Here the list is a description, the World reads it after the occurrence, and the only things that start occurrences are the things the World chooses to deliver. A label appearing is not a thing that happened; it is a thing that is.

It is also why a World can be as simple as it is. The run call takes inputs and returns a list. The World never subscribes to anything, never registers interest, never receives a notification that the list changed; it reads the list when the call returns and diffs it against the last one if it wants to. Everything a GUI framework does with observers, dirty flags and invalidation queues is replaced by the World looking at a map.

## Two laws across the boundary

Everything in this chapter rests on two laws, and each binds a named party.

**Equality.** Two readings of one state element's dynamic in one occurrence, by any two slots, anywhere in the program, give the same value. The occurrence is the version. This binds whoever supplies state: a World, which must answer its oracle from one snapshot per occurrence and push one value per key; and an interceptor, which gets the law for free because the runtime builds one supplied widget per distinct value and hands its dynamic to every slot with that value. The law is what the language's own equality-of-readings law (two readings of one dynamic in one occurrence agree) becomes when the dynamic's source is outside, and it is stated separately because the source is outside: a World

that answers two slots differently in one occurrence has broken the program, not the World, and the Test World checks it every turn.

**Monotone turns.** A response reaches a slot in an occurrence strictly after the request's, and the outside reads a request only after its occurrence commits. There is no round trip inside an occurrence. This binds Worlds and interceptors, and the runtime enforces it for interceptors by routing and for Worlds by the structure of the run call. Its corollary is the creation rule: a slot created in n fired at the earliest in n, so nothing can answer it before n+1.

Together they say what an occurrence is at the boundary: a snapshot in and a list out, with every value in agreeing with every other value in, and nothing out visible until the snapshot is closed. That is a transaction, and the rest of the chapter is what a Prism program can build on top of one.

## Dynamic fields

Fields are plain data. A label's text, a file's path, a socket's address: each is read once when the slot is created and never again. The question every reader asks next is what happens to the label whose text changes, and the answer is that the language already has a construct for building something afresh when a dynamic changes, and uses it.

An element with a dynamic where a field goes is an application to a dynamic, which lifts: `Ui.Label(text=$text)` is `$(Ui.Label(text=text))`, a dynamic of elements, and `@Element` of it is a dynamic of blueprints. Binding a dynamic of blueprints is `rebuild`. So

```
@Element Ui.Label(text=$text)
```

means

```
rebuild $text:
    @Element Ui.Label(text=_)
```

a fresh slot per change of `$text`, with the old slot torn down at the commit of the occurrence in which the text changed and the new one in the list from that occurrence. For an element that responds, the bound name is the `!Switch` over the generations:

```
!click <- @Element Ui.Button(label=$label)
# is
!!clicks <- rebuild $label:
    @Element Ui.Button(label=_)
let !click = !Switch !!clicks
```

and for an element with state it is a hold of dynamics joined:

```
$contents <- @Element Fs.Contents(path=$path)
# is
!$each <- rebuild $path:
    @Element Fs.Contents(path=_)
$$current <- $first then !$each        # $first: the dynamic the first build returned
let $contents = $Join $$current
```

where the first build's result is available as the hold's initial value because the first build happens in the occurrence the `rebuild` is built; the runtime has it in hand when it builds the hold, and a library spells the three lines as one word. An event where a field goes is a type error unless the field is the declared `requests` event, which is the one field whose type is an event.

The consequences, each of which some other part of the design depends on: the element list never contains a sigil, so a World and an interceptor read plain data; value identity never has to pick a moment, because a value is fixed for its slot's life; nothing about an element changes after creation, so "the same element with a new field" is not a case any reader has to handle; a response to the old slot is for the old slot, and the World drops it because the slot is gone; and the new slot's state is sampled in the occurrence of the change, under the rule for widgets built mid-occurrence, so a file viewer whose path changes shows the new file's state in the occurrence the path changed.

## Fresh slot per change, case by case

The worry about this design is that a slot is destroyed and recreated for every change of a field, and some things want to persist across the change. Take the cases one at a time.

| Element | Dynamic field | Fresh slot per change is | Why |
|---|---|---|---|
| `Ui.Label(text)` | the text | right | A draw command. The World diffs lists; nothing is in flight; a new label at the same position is what a redraw is. |
| `Ui.Button(label)` | the label | right | A click in the occurrence of the relabel goes to the old slot, under the old label, which is what the user clicked. From the next occurrence the new slot's clicks are heard. There is no moment with two buttons listened to or none. |
| `Fs.Contents(path)` | the path | right | A new path is a new file. The new slot samples the new file's state in the occurrence of the change; the old slot's watch is released when the World sees it gone. |

| Element | Dynamic field | Fresh slot per change is | Why |
|---|---|---|---|
| `Fs.File(path, requests)` | the path | right | Responses to the old path's requests are for the old file. The World drops them because the slot is gone, and the program could not have told them apart from the new file's anyway. |
| `Net.Http` | none | n/a | The URL is in the request, so the element has no field to change and one slot serves many fetches. This is the design to reach for when a field would change per request. |
| `Net.Socket(address)` | the address | right | Reconnect. The old generation's send in the occurrence of the change goes on the old connection, which is the only connection that is up. |
| `Time.Every(period)` | the period | acceptable | The phase restarts. A World that wants phase continuity keys its timer on tree position, which the path exposes; nothing in the program can tell. |
| `Process.Child(command)` | the command | right | A different command is a different process. The old one is stopped when its element leaves the list. |
| `Gtk.Entry(value)` native | the value | the hard case | A native text box has cursor and selection state. Per keystroke, a fresh slot loses it. |

| Element | Dynamic field | Fresh slot per change is | Why |
|---|---|---|---|
| `Gui.Window(title)` as a container | the title | a non-problem | Containment is by interception, not by fields. The window's children are widgets under an interceptor; the window element is a leaf with no children field, so its rebuild touches nothing else. |

The hard case is the native text input, and the answer is that in Prism the text input is not native. The GUI library is Prism, so the cursor is a hold in `@TextInput` outside any `rebuild`; the keystrokes arrive at the window, and the library routes each one to the input whose name is the focus, a name that does not change when you type into it; and what the World sees is a scene of draw commands and, for the focused input, a `Gui.Focusable` with that name. The worked example with a cursor is four occurrences of exactly this, and no cursor is lost because there is no second copy of it anywhere. A World that wraps a native toolkit and wants continuity for a native widget keys it by tree position and tag, which the path exposes; that is a World's policy, and the language promises nothing about it.

## The declared-dynamic-field alternative

Suppose a tag could declare a field as `$Text`:

```
tag Gtk.TextBox of Element:
    value: $Text
```

meaning the slot persists across changes of the value and the outside sees the list's value change at the same path. This is the sample-plus-update form: the World reads the field at creation and is told each change. It would make the native text box a language-level non-problem, and it is the first thing a reader who has written a toolkit binding asks for.

What it costs. Elements would contain a sigil, so the list would no longer be plain data: a World would hold a dynamic, which is a runtime object, and the rule that no runtime object crosses the boundary would have an exception. Value identity would have to pick a moment, because two `Gtk.TextBox` elements with equal values now may differ next occurrence, and the oracle, the push map and the supplier would each need a rule for which moment. A response to a slot whose field changed would need a second rule for what it answers: the slot as it was when the request fired, or as it is now. And interceptors would have two kinds of element to forward, the ones whose fields are fixed and the ones whose fields move, and forwarding the second kind means forwarding a dynamic, which is a subscription, which is the thing the boundary was designed not to have.

The table above is the argument that the cost is not yet paid for. Every case but one is right or acceptable with fresh slots, and the one hard case has a better answer in the library than in the language. If a use case appears that the leaf-plus-interceptor pattern cannot express, a declared dynamic field is the escape hatch, and the costs above are what it would cost; until then the boundary never sees a sigil, and that single fact is what makes a World diff lists of plain data instead of subscribing to anything.

# Why state has to be there in its first occurrence

A dynamic has a value at every occurrence. A widget that binds `$contents <- @Element Fs.Contents(path="/etc/motd")` therefore has, in the occurrence it is built, a value for `$contents`, and the question is where it comes from. The answer is the oracle: a World answers synchronously from its snapshot as the occurrence opened, and an interceptor wires in a dynamic it already has. This section is the reasoning for why nothing less will do, written as the designs one would try first.

## Suppose state were a request

The first design to try builds state out of the request element the language already has. Ask for the current value with a request fired from `@started`; get it back next occurrence; subscribe to changes thereafter; hold the lot into a dynamic. The library widget would look like this:

```
tag Fs.Query of Element:                      # not the design
    path: Text
    requests: !unit                          # "tell me the current contents"
    responds: Contents                        # the answer, and every later change

@Contents: fn Text -> @$?Contents = \|path| @widget:
    !started <- @started
    !msg <- @Element Fs.Query(path=path, requests=(unit on !started))
    $contents <- Nil then Some !msg
    @ $contents
```

The program holds a `?Contents` that is `Nil` for one occurrence. That is honest, in the sense that a type says the value may be absent, and it is cheap, in the sense that the World needs no oracle. The trouble is not the one occurrence of waiting. The trouble is that the wait is visible, and visible in a way no library can hide.

Take a widget built in occurrence n by a `rebuild` over `$selected`, which changed in n, and which also reads a dynamic `$theme` captured from its enclosing scope. Under the rule for widgets built mid-occurrence, the new widget sees all of n: it sees `$selected`'s new value (that is why it was built) and it sees `$theme`'s value as of n, including a change to `$theme` in n if there was one. Now give it a state request. Its `$contents` is `Nil` in n and `Some` in n+1. So there is an occurrence, n, in which the widget has the new `$selected`, the new `$theme`, and no contents; and if `$theme` changes again in n+1, there is no occurrence at all in which the widget has the contents and the `$theme` it was built under. The widget was supposed to be a function of three dynamics and it has been shown a combination of their values that never existed.

## Suppose an interceptor hid the delay

The second design keeps state as a request but puts an interceptor around the child that shows it one occurrence for the runtime's two: the child's build occurrence n is held back, the request goes out, the answer comes in n+1, and the interceptor presents n+1 to the child as if it were n, with the answer already there. Everything downstream of the child is delayed by one occurrence to keep the story consistent.

The residue is the captured dynamic. `$theme` is not the child's; it is lexically in scope from outside the interceptor, and the child reads it directly, not through the interceptor's response argument. If `$theme` changed in n and again in n+1, the child sees both changes, and nothing the interceptor does to the child's own slots can hide that two occurrences passed. The interceptor could try to freeze every dynamic the child captures, but the child captures whatever its author wrote, and the

interceptor does not know which names those are; an interceptor sees elements, not scope. The only arrangement under which a widget built in n is consistent in n is the one where its state is there in n, and the only party that can put it there is whoever holds the state, answering synchronously as the occurrence opens.

### Suppose a placeholder

The third design admits the wait and gives it a name: the state type gets a `NotYet` case, the World answers next occurrence, and every `match` on the state has a `NotYet` arm. It is the first design with a nicer type, and it fails the same way; its failure looks like a feature, which is why it gets its own paragraph. A `NotYet` arm in every consumer is the `if (data)` guard you have written a thousand times and forgotten once, except that here forgetting it is a compile error, so the design feels safe. What it hides is that `NotYet` is not a value the World ever held. `Unavailable` is: the World looked and the file was not there, or had not been read. `NotYet` is the runtime's constant before anyone looked, and a program that draws it is drawing something that was never true of the world. The language has a goal that "not yet" is a type, and this is the case that shows the goal needs the *World's* not-yet, not the runtime's.

### What the oracle is

So the World, as it opens an occurrence, answers a function from state-element value to state value, synchronously, from its snapshot; and an interceptor, when its child creates a state slot, builds a widget that returns a dynamic, then and there, in the same build pass, so the slot's value is the dynamic's value before the occurrence. Both give a widget built in n a state value in n that agrees with every other reading of that value in n, which is the equality law, and both let the widget see a push in n to that value, which is the rule for widgets built mid-occurrence. The cost is that the World answers a question in the middle of a run call rather than only at its start, and that cost is paid by Worlds, not programs. A World with a fact in memory answers it; a World that would have to go and look answers `Unavailable` and pushes `Known` when it has looked, which is what it would have had to do anyway, one occurrence later, with the honest difference that the program never shows a value the World did not hold.

### What this subsumes

Two protocols one might design for live state fall out of this one. A *query plus subscription*, where an element answers "what is it now" on request and reports changes as a stream, with a rule that an answer and a change in one occurrence agree, is what the World does internally to maintain its snapshot; the program never sees the two halves because the oracle and the push map are the snapshot's interface. A *versioned channel*, where every message is a complete state or a delta against a version number, is what a World keeps toward a remote source that does not share its clock; the program never sees the version because the occurrence is the version. In both cases the matching is the World's job, and it is the World's job because only the World sees the race between a read and a watch. The program gets a dynamic.

## `intercept`

You have written the code that wraps a third-party widget so that its network calls go through your proxy. In most frameworks that is a global: you replace `fetch` for the whole process, and the widget

beside it that you did not want to proxy goes through the proxy too, and the test that ran before yours left `fetch` replaced. Or it is a context object the widget was kind enough to accept, in which case the widget you did not write ignores it. Interception here is per subtree, written in the program, and there is no channel a child can use that the interceptor around it does not see, because the language has no ambient anything.

An interceptor stands between a child and everything outside it. From the child's side it is indistinguishable from a World: the child's slots get their responses and their state from it, and the child cannot ask who is answering. It does exactly two things, and the two directions never mix.

*Downward*, it supplies what the child's slots return: responses for elements that respond, dynamics for elements with state.

*Upward*, it makes elements of its own, with `@Element` like any widget, or does not. Nothing the child creates goes further out by itself. Whether the interceptor re-emits a child's element unchanged, rewrites it, holds it back, merges it with another, answers it itself, or decides at runtime, is ordinary code.

That last sentence is the test the design has to pass. Every policy an interceptor could have must be writable by hand in the language, because if some policy needs a mode of the primitive, the semantics are incomplete and the mode is hiding it. `intercept` is a low-level call for framework writers; it is not meant to be convenient, and the library wraps it.

```
intercept: fn @a, !(Path ==> Response), $(fn Element -> @$State) -> @((a, $(Path ==> Slot)))

type Slot = record:
    element: Element
    !request: Request          # typed by the tag, once `element` has been matched
```

`intercept @child, !responses, $supplier` builds `@child` as a child widget of the interceptor's widget and captures its element list. It returns the child's result and that list, keyed by path. The two other arguments are the two downward supplies: responses by path, and state by value through the supplier.

## Downward: responses by path

`!responses` is an event of maps from the child's paths to response values. When it fires in occurrence n, each entry is delivered to the slot at that path in the child's list *as n opened*, under the routing rule, and that slot's response event fires with the value. An entry whose path is not in that list is an error that aborts the occurrence. The interceptor makes this event however it likes; the usual form is a `for` over the list that does something per slot and lets consolidation build the map:

```
@Forward: fn $(Path ==> Slot) -> @!(Path ==> Response) = \|$slots| @widget:
    for $slots:                      # key is the child's path
        rebuild $_:                      # a slot never changes, so this builds once
            ((!resp, ignore)) <- @Element _.element
            @ !resp

@Transparent: fn @a -> @a = \|@child| @widget:
    ((result, $slots)) <- intercept @child, !down, $ \|e| @Element e
    !down <- @Forward $slots
    @ result
```

Read `@Forward` slowly, because every interceptor in this chapter is a variation on it. `for $slots` builds one block per slot in the child's list, keyed by the child's path; a block is built in the occurrence its slot appears and torn down at the commit of the occurrence its slot leaves. Inside, `$_` is the slot

as a dynamic, and since a slot never changes, `rebuild $_:` builds exactly once and gives `_` as the plain slot. `@Element _.element` makes an element of the interceptor's own with the same value, in the interceptor's widget; its tag is not statically known, so it returns the generic pair of a response event and a state dynamic, and the pattern keeps the event and discards the dynamic, which nothing needs because the child's state slots are answered by the supplier, not by this loop. The block returns the event, and `for` consolidates the blocks' events into `!(Path ==> Response)`, firing in any occurrence in which at least one outer response arrived, with exactly the entries that did. That map is `!down`, and `intercept` delivers each entry to the child's slot at that path.

The request side forwards itself. The child's request event is in the slot, `_.!request`, and `@Element _.element` makes an element whose `requests` field is that very event, so when the child's request fires, the interceptor's element's request fires in the same occurrence, and the outside sees it after the occurrence with no delay. A widget built in occurrence n sees all of n, so even a request fired from `@started` in a slot's creation occurrence is seen by the block built for that slot in the same occurrence and goes out that turn.

`!down` is used before it is defined, and its definition reads `$slots`, which the `intercept` that consumes `!down` produces. That cycle is legal because routing reads the list as the occurrence opened: the edge from `!down` into the child is through a tag on the list. At runtime the only way round the loop in one occurrence would be a block, built for a slot created in n, firing something into `!down` in n; by the creation rule only pure code could do that, and routing refuses it as an unknown path. An interceptor that writes `@ click on !started` in a block has written a bug, and the runtime names the path.

`@Transparent` is the identity. A child under it has the same list at the outside, one level deeper, with its responses coming back under its own paths and its state supplied by an element of the same value. The law the language promises is that an interceptor which re-emits everything and supplies by forwarding is observably not there; the conformance suite tests it by running every program under `@Transparent` and comparing transcripts.

## Downward: state by value, through the supplier

The third argument is a dynamic of a function from an element to *a blueprint returning the state's dynamic*. When the child creates a state slot for element `e`, and no supplied widget for a value equal to `e` is alive in this interceptor, the runtime reads the function (the supplier dynamic's current value at that instant), applies it to `e`, and builds the blueprint it returns, then and there, as a child of the interceptor's widget, in the same build pass as the child's slot. The widget's result dynamic is the slot's dynamic from that instant on: its value before the occurrence is the slot's value in its creation occurrence, and its changes are the slot's changes. The slot therefore has its value in its creation occurrence, exactly as it would from a World's oracle.

The widget runs in the interceptor's context, so its own elements go into the interceptor's list. It can ask the outer context for the same element, a rewritten one, two of them, or nothing:

```
\|e| @Element e                              # transparent: ask outside for the same thing
\|e| @Element (Rewrite e)                    # ask outside for something else
\|e| @ $ Unavailable                         # sandbox: a constant, no element at all
\|e| @widget:                                # cache: a hold around a forwarded query
    $live <- @Element e
    ...
    @ $cached
```

There is no second mechanism and no table. Forwarding a state element *is* making an element, with

the one call the language has for that, and supplying a constant is returning the constant dynamic, with the one call the language has for that.

The supplied widget is built once per distinct element value per interceptor, and it lives as long as some child slot with that value lives. A second child slot with an equal value joins the existing supplied widget and gets the same dynamic, whatever the supplier dynamic says at that moment; the supplier is consulted only for a value no live supplied widget serves. That is value identity holding inside an interceptor for free, and it is also the equality law: two child slots with equal values read one dynamic, so they cannot disagree. When the last child slot with that value is torn down, the supplied widget is torn down with it at the same commit, and its elements leave the interceptor's list, so the outer context sees the interest go.

The supplier is a dynamic so that policy can change over time. A slot's dynamic is fixed at creation; if the supplier changes, values first seen afterwards get the new policy and values already being served keep theirs. Most interceptors pass a constant, `$ \|e| @Element e`. The migration interceptor below is the one that does not.

The World works the same way, and the parallel is deliberate: its supplier is a synchronous function from element to value, the oracle, with pushes for the changes. An interceptor's supplier is that function with a widget around it, because an interceptor, unlike a World, has an outside to ask, and asking is building an element.

## Upward: elements, or nothing

An interceptor's own elements are made with `@Element`, and those are what the next interceptor out, or the World, sees. There is no third way. The child's list is a dynamic the interceptor holds; nothing in the runtime reads it on the interceptor's behalf and re-emits it. If the interceptor's code does not make an element for a child's slot, that slot is invisible outside, and the child waits for an answer that never comes unless the interceptor gives one. Holding back is therefore the default and forwarding is a choice, which is the right way round for a sandbox and the only way round under which "nothing reaches past an interceptor unless Prism code sends it" can be a law.

## Why forwarding is manual

Suppose `intercept` took a table instead: a list of tag patterns to forward unchanged and a map from tag to supplier, and did the forwarding itself.

```
# not the design
intercept @child, forward=[Gui.*, Time.*], supply={Fs.Contents: ..., Clock: ...}
```

It is convenient, and it covers the sandbox and the logger. Now a library wants to forward a file read with its path rewritten under a root; the table grows a `rewrite` entry with a function per tag. A cache wants to forward one query for many identical children and hold the answer; the table grows a `memo` mode with a policy for when the memo is stale. A tracing layer wants to forward and also log each request; the table grows an `observe` hook. A quota layer wants to forward until a count is reached and deny after; the table grows a condition, and the condition has to read a hold, so the table is now a dynamic of tables and the question of when it is re-read has to be answered. A migration layer wants to forward to one of two backends depending on a dynamic; that is a second condition with a different scope. Each is a new mode of the primitive, each mode has its own rules for when it runs and what it sees, and before long the table is a second language for things the first language already says, with its own ordering rules at the one place in the program where ordering rules matter most.

The alternative is a primitive that does nothing but expose the two directions, so that forwarding is the shortest program an interceptor can write, `@Forward` above, and every other policy is a longer program in the same language, with the same laws about tags, holds and occurrences that the rest of the program has. That is the goal that anything a convenience does, you can do by hand with the same primitives, applied where it matters most. It also keeps the invariant a sandbox is made of: with a table, the question "does anything the child makes reach the outside" is answered by the table's semantics; with manual forwarding, it is answered by reading the interceptor's `match`, and the catch-all arm is where the answer is.

## Typing

Inside a `match` arm on tag `T`, `_.!request` has `T`'s declared request type, `@Element` of an element built from `T` returns `T`'s declared types, and a blueprint the supplier returns for `T` must return `T`'s declared state type. In the catch-all arm, `_.!request` is the generic `Request`, `@Element _.element` is the generic `@((!Response, $State))`, and the runtime checks the per-slot types at the slot. The refinement is a rule of the checker and not a runtime structure: the extensible variant with per-tag payload types is what makes it natural, and the alternative, a runtime check everywhere, is what elements get in the generic arm anyway.

An interceptor that wants to be typed therefore writes one arm per tag it handles and a catch-all for the rest. An interceptor that handles a namespace, `Gui.*(..)`, is in the generic case inside that arm, which is right, since it does not know which tag it has.

## Answering a request

An interceptor that answers a request itself, a sandbox denying a read, a cache serving a hit, a mock returning canned data, does so the only way anything does: in a later occurrence. Prism cannot start an occurrence, so the interceptor fires a `Defer`, which every World provides, and answers when the `Defer` comes back:

```
Fs.File(path=p):
    !later <- @Element Defer(requests=(unit on _.!request))
    @ Denied on !later
```

`unit on _.!request` fires in the occurrence the child's request fires; the `Defer`'s request goes out with the turn; the World, after each occurrence, checks for fired `Defer` requests and, finding one, runs another occurrence of the same turn delivering `unit` to it; `!later` fires; `Denied on !later` fires; the block's result is consolidated into `!down`; the child's slot receives `Denied`. From the World's view it was one turn. From the child's view it asked and was answered later, exactly as it would have been by a World with a filesystem that refused.

Why not answer at once? The block for the child's slot sees the request fire, and could in principle compute `Denied on _.!request` and put that into `!down` in the same occurrence. Two things stop it. The creation rule is one: a response in the occurrence of the request means the element could have been a pure function, and the child is entitled to assume it was not. Routing is the other: if the child's slot was created in this occurrence, which is the common case for a request fired from `@started`, the path is not in the list as the occurrence opened, and the delivery is an unknown-path error. `Defer` is the one occurrence of delay that makes both go away, and it is why the rule can be a law for interceptors and not only for Worlds.

A `Defer` slot is answered at most once per turn. A slot that fires again after being answered in the same turn is an error, deferring backwards, and that is what bounds a turn: a child that retries at

once on `Denied` makes the sandbox's `Defer` fire a second time in the same turn, and the World reports it rather than running forever. A retry belongs behind `Time.After`, where the World's clock separates the attempts. The error is a feature: it catches the tight retry loop in the occurrence it is written, with the path of the slot that caused it.

### What an interceptor may not do

The supplier's type says it returns a blueprint for every element. An interceptor that wants to deny a state element cannot return nothing; it must say what denial is, in the state's type, `@ $ Unavailable` for a file, `@ $ Disconnected "sandboxed"` for a socket. That is not an inconvenience. It is the rule that unavailability lives in the type, enforced at the one place it could be dodged: a child under a sandbox is shown a value the World could have shown it, and the child's `match` already has the arm.

A state tag no supplier and no World accepts is an error at the creation of the slot, because nothing later can give it a value. A request tag no World accepts gets `Unsupported` as its response in the next turn, so a `Ui.Button` under a CLI World is a program-visible fact rather than a silent nothing; an interceptor that forwards it unchanged forwards the `Unsupported` back down.

### Invariants

Stated once, with who is bound and what enforces each:

- *Nothing reaches past an interceptor unless Prism code sends it.* Bound: the runtime. Enforced by the structure of `intercept`: the child's list is returned as a value and read by nothing else. A scoped interceptor, one written as `intercept Ui.*` that claims a namespace, is this law's special case and not an exception to it: it is sugar for a full interceptor whose body forwards every element outside the namespace with `@Element e`, and forwarding is Prism code sending it. The runtime may route the forwarded namespaces without materialising the forwarding slots, because nothing can tell the difference from doing it the long way, which is what makes it an optimisation and not a hole.
- *Responses reach a child's slot only through the interceptor's response argument, routed against the opened list.* Bound: the interceptor. Enforced by the runtime's unknown-path check.
- *A state slot has its value in its creation occurrence, equal for every slot of that value under this interceptor.* Bound: the interceptor's supplier, which must return a blueprint for every element. Enforced by the runtime building one supplied widget per value and handing its dynamic to every slot.
- *No response in the occurrence of its request.* Bound: the interceptor. Enforced by routing for slots created this occurrence, and by convention, with `Defer` as the only tool, for older ones; the conformance suite's `mock_timing` test is the check.
- *An interceptor that forwards everything and supplies by forwarding is observably not there.* Bound: the runtime. Enforced by the differential test against `@Transparent`.
- *A supplied widget lives exactly as long as some child slot with its value.* Bound: the runtime. Enforced by ownership: the supplied widget is a child of the interceptor's widget, keyed by value.

## Recapitulation and depth

An interceptor's own elements are what the next layer out sees, and a layer that wants to present several of its child's elements as one thing puts them inside a field of its own element, as data. A container's element then holds its children's elements; the outside addresses the container; and the

container routes inward through its response argument. This is how depth works, and the convention a library adopts for it, a `children` field holding a `Path ==> Element` and a `ChildInput` response naming a path and a response for the slot there, is the library's, not the language's. The language guarantees only the two laws, routing against the opened list, and transparency of the forwarding interceptor; what a container looks like is up to whoever writes containers.

The convention has a consequence for tests. A Test World that wants `expect Ui.Button(label="OK")` to find a button inside a box looks inside `children` fields because the library and the suite agree to, and `send` to a nested slot wraps the input in one `ChildInput` per level, so that delivery takes the path a real World's input would. The suite knows the convention because the library is part of the suite, not because the language does. The next section builds that library.

## A GUI library as interceptors

Every toolkit you have used had layout, focus and hit-testing inside it, in the host language, next to the pixels, and the consequence was that a button was a native object: it had a position the toolkit assigned, a focus flag the toolkit set, and a click handler the toolkit called, and the three were visible to your program only through whatever accessors the toolkit chose to expose. The text box of the worked examples lost its cursor because the cursor was the toolkit's. This section builds the whole of a small toolkit, labels, buttons, text inputs, vertical and horizontal boxes, scrolling, focus and hit-testing, as Prism interceptors over five World-facing tags, so that there is nothing native between a program's button and the pixel except the drawing of the pixel. The library is the demonstration that the language is enough, and it is also the convention the conformance suite's Test World uses to look inside containers.

### The five tags

A World that draws accepts a window, a scene, a focusable, a text measurer and a frame clock. The first three are declared in the section on tags above. The other two, and the plain values all five use, are these:

```
type Point = record:
    x: Float
    y: Float
type Extent = record:
    w: Float
    h: Float
type Rect = record:
    x: Float
    y: Float
    w: Float
    h: Float
type Color = choice White | Grey | Blue | Black

type Draw = choice:
    Rect ((Rect, Color))
    Text ((Point, Text))
    Image ((Rect, Text))
    Clip ((Rect, [Draw]))

type Key = choice:
    Char Char
    Left
```

```
    Right
    Backspace
    Tab
    Enter

type WindowInput = choice:
    Pointer Point
    Pressed Key
    Wheel ((Point, Float))
    Resized Extent

type Frame = record:
    now: Instant

tag Gui.Metrics of Element:                 # state: the size of a text, answered synchronously
    text: Text
    font: Font                             # a plain value; Default unless the program names one
    state: Extent

tag Gui.Frames of Element:                   # stream: one Frame per displayed frame while a slot exists
    responds: Frame
```

The window has no dynamic field, so its slot is stable for the life of the program: the World has one path to send pointer, key, wheel and resize events to. Its `size` field is the size the program asks for at the start; the size the platform actually gives arrives as `Resized`, like any other input. The scene is a presence element whose `draw` field is the picture. A change to the picture is a fresh `Gui.Scene` slot, which the World draws, and nothing is in flight on a picture, so a fresh slot per change is right. Keeping the window and the scene apart is what lets the window have one path while a scene that changes sixty times a second is sixty presence elements.

Draw commands are values inside the scene, not elements. Suppose each rectangle and each run of text were a slot of its own: a window of two thousand glyph runs would be two thousand paths, each with an identity nobody addresses, and a redraw would be two thousand teardowns and builds. As one field, the scene is one slot, the World reads the list in order, which is paint order, and `Draw.Clip` nests a list inside a rectangle. A command has no response and no state, so nothing about it needs a slot.

The measurer is a state element for the reason the whole second half of this chapter gives. A layout needs the width of a text in the occurrence it lays out. As a request, the width would arrive a turn later, and the scene would be drawn once with the wrong width and once with the right one. As state, `Gui.Metrics(text="Ada", font=Default)` has its `Extent` in the occurrence the slot is created, and every slot asking about the same text and font shares one answer. `Font` is a plain value naming a family and size, declared with the other values in Appendix D, and `Default` is the font a program gets when it names none. The World owns the fonts and the library never sees them.

A World that offers these five can run every program in this section. A software rasteriser, a GTK-backed World that paints into a drawing area, a web World that turns `Draw` values into canvas calls, and a native Prism system that speaks X11 over a socket are all the same World as far as the library is concerned. A World that would rather offer native widgets accepts `Ui.*` tags or `Gtk.Button` and friends directly, and a different library targets it; the program above both libraries is the same program.

## The library's own tags

Between a program and the window, the library has elements of its own, declared under `Ui` to keep visible whose they are. `Ui.Label` and `Ui.Button` are declared above. The rest:

```
type ChildInput = record:
    path: Path
    response: Response

tag Ui.TextInput of Element:
    name: Text
    text: Text
    cursor: Nat
    responds: Key

tag Ui.VBox of Element:
    children: Path ==> Element
    responds: ChildInput

tag Ui.HBox of Element:
    children: Path ==> Element
    responds: ChildInput

tag Ui.Scroll of Element:
    name: Text
    height: Float
    children: Path ==> Element
    responds: ChildInput
```

The window interceptor claims every one of these. A World that accepts them natively could take them directly, and the Test World accepts every tag, which is how the suite looks inside the library.

Containers hold their children as data in a `children` field, keyed by the child's path, which is the *children convention*. A `Ui.VBox`'s `children` is the map of its children's elements, in tree order, and a container two levels up sees the whole subtree as one value. That is what the window needs to lay the tree out, and it is why the convention is a field and not an interceptor-to-interceptor protocol: the tree is plain data, and layout is a plain function over it.

Every container responds with `ChildInput`: a path in the child's list and a response for the slot there. `Response` is the generic type of `intercept`'s response argument, so a container delivers whatever it is given without knowing what the child is, and a value of any tag's response type is a `Response`, checked against the slot's tag when it arrives. A button's slot gets a `Click`, a text input's a `Key`, another container's a `ChildInput`. Suppose instead a container named the response types of its children: a union of every leaf's type, or one type for all leaves that each leaf filters. A container written before a new leaf tag exists could not hold it, and every leaf would see inputs meant for others. With the generic `Response`, routing is one line per container, and the leaf is the one place that knows what it accepts.

## Leaves

```
@Label: fn $Text -> @unit = \@widget:
    @Element Ui.Label(text=$_)
    @ unit

@Button: fn $Text -> @!Click = \@widget:
    !click <- @Element Ui.Button(label=$_)
```

```
    @ !click
```

A label is one element with a dynamic text, which is a `rebuild` with a fresh slot per change. A button is the same with a response, which is a `!Switch` over the generations. There is nothing else in a leaf; everything a leaf does belongs to the layers above it.

## Containers

A container is an interceptor. It captures its child's list, puts the library elements into a `children` field of one element of its own, forwards everything else unchanged, and routes `ChildInput` down to the child at the named path:

```
let IsUi: fn Element -> Bool = \|e| match e:
    Ui.*(..): True
    ignore: False

let UiOnly: fn Path ==> Slot -> Path ==> Element = \|slots|
    MapValues (Filter slots, \IsUi _.element), \_.element

let NonUi: fn Path ==> Slot -> Path ==> Slot = \|slots|
    Filter slots, \not IsUi _.element

let Inward: fn ChildInput -> Path ==> Response = \|c| {c.path: c.response}

@VBox: fn @a -> @a = \|@child| @widget:
    ((result, $slots)) <- intercept @child, !down, $ \|e| @Element e
    !in <- @Element Ui.VBox(children=(UiOnly $slots))
    !fwd <- @Forward (NonUi $slots)
    let !down = !Union (Inward !in), !fwd
    @ result

@Scroll: fn Text, Float, @a -> @a = \|name, height, @child| @widget:
    ((result, $slots)) <- intercept @child, !down, $ \|e| @Element e
    !in <- @Element Ui.Scroll(name=name, height=height, children=(UiOnly $slots))
    !fwd <- @Forward (NonUi $slots)
    let !down = !Union (Inward !in), !fwd
    @ result
```

`@HBox` is `@VBox` with `Ui.HBox`. `UiOnly $slots` is a derived dynamic of the child's library elements, kept current by the change law. `Ui.VBox(children=...)` with a dynamic field is a `rebuild`, a fresh box per change of the children, and `!in` is the `!Switch` over its generations. When the window sends a `ChildInput` to the box, `Inward !in` is the one-entry map from the named path to the response, `!down` is that map merged with whatever the forwarded non-library elements received, and `intercept` delivers the response to the child's slot at that path, against the child's list as the occurrence opened. The child at that path is a leaf, which fires, or another container, which peels the next `ChildInput`.

The two maps never share a key, because `Inward !in` has library paths and `!fwd` has the others; `!Union` is there so that a click into the box and a file response to a forwarded `Fs.File` can land in one occurrence without one losing the other.

The supplier is `\|e| @Element e`: a box has no opinion about state and forwards a child's `Clock` or `Fs.Contents` by asking the outside for the same thing. A box that wanted to give its children a different clock would be a different interceptor with the same first line and a different third argument.

## Text input and focus

A text input is a leaf with a name, and the editing state sits in a hold outside the element:

```
@TextInput: fn Text, Text -> @$Text = \|name, initial| @widget:
    !key <- @Element Ui.TextInput(name=name, text=$edit.text, cursor=$edit.cursor)
    $edit <- (Edit initial, (Len initial)) then Apply $edit, !key
    @ $edit.text
```

`Apply` is the plain function of the worked example with a cursor: it takes the old edit state and a key and returns the new state. The hold `$edit` is referenced by the element's fields before the line that defines it, which a widget block allows, and the cycle passes through the hold's value side.

Look at what changes per keystroke and what does not. The element's text and cursor change, so the element is a fresh slot; the hold `$edit`, with the cursor in it, is outside the `rebuild`, so it persists. The slot being fresh per keystroke is why focus, in the window below, is keyed by the input's `name` and not by its path: the name is the one thing about a text input that does not change when you type into it. That is the same move as identifying state by value rather than by path, for the same reason. A path names a slot; a name names the thing the user is looking at.

Focus is a hold of an optional name in the window, and only there. Tab has to walk every input in tree order, and a click has to move focus from one input to another, so one owner sees all of them; an input that asked "am I focused" would need the window's answer anyway.

## Scrolling

A scroll region needs an offset, which wheel events move, and the layout needs the offset to place the children. The offset is state that no child reads, so the window holds it, in a map keyed by the region's `name` for the reason focus is: a `Ui.Scroll` is a fresh slot whenever its children change, and a list whose items come and go must not jump back to the top. `Ui.Scroll` is therefore a container like `Ui.VBox` with a name and a height. The next sections show where the offset is applied and where it is moved.

## Layout and hit-testing as plain functions

Layout is a function from an element tree, a rectangle and what the layout needs to know to a draw list and a list of hit regions. It needs the measured size of each text, the scroll offsets and the focus, bundled as `Env`:

```
type Env = record:
    sizes: Text ==> Extent       # one Gui.Metrics answer per distinct text
    offsets: Text ==> Float       # scroll offsets, by region name
    focus: ?Text

type HitKind = choice:
    Clickable
    Focus Text
    Scrollable ((Text, Float))     # region name, largest offset

type Hit = record:
    rect: Rect
    chain: [Path]                # from the window's child down to this element
    kind: HitKind

type Placed = record:
    draw: [Draw]
```

```
    hits: [Hit]

let TextExtent: fn Env, Text -> Extent = \|env, t|
    Or env.sizes[t], (Extent 0.0, 0.0)

let Measure: fn Env, Element -> Extent = \|env, e| match e:
    Ui.Label(text=t): Padded (TextExtent env, t), 4.0
    Ui.Button(label=l): Padded (TextExtent env, l), 8.0
    Ui.TextInput(text=t, ..): Padded (TextExtent env, t), 4.0
    Ui.VBox(children=cs): Fold cs, (Extent 0.0, 0.0), \|acc, ignore, c| Stacked acc, (Measure env, c)
    Ui.HBox(children=cs): Fold cs, (Extent 0.0, 0.0), \|acc, ignore, c| Beside acc, (Measure env, c)
    Ui.Scroll(children=cs, height=h, ..): Extent (Measure env, Ui.VBox(children=cs)).w, h
    ignore: Extent 0.0, 0.0

let Place: fn Env, Element, Rect, [Path] -> Placed = \|env, e, r, chain| match e:
    Ui.Label(text=t):
        Placed [[(Draw.Text (((Point r.x + 4.0, r.y + 4.0), t)))]], [[]]
    Ui.Button(label=l):
        Placed [[(Draw.Rect ((r, Grey))), (Draw.Text (((Point r.x + 8.0, r.y + 8.0), l)))]], [[(Hit r, chain,
        Clickable)]]
    Ui.TextInput(name=n, text=t, cursor=c):
        Placed (InputDraw env, r, n, t, c), [[(Hit r, chain, (Focus n))]]
    Ui.VBox(children=cs):
        PlaceStacked env, cs, r, chain, True
    Ui.HBox(children=cs):
        PlaceStacked env, cs, r, chain, False
    Ui.Scroll(name=n, children=cs, height=h):
        Clipped (PlaceStacked env, cs, (Up r, (Offset env, n)), chain, True), r, [[(Hit r, chain, (Scrollable
        ((n, (MaxOffset env, cs, h)))))]]
    ignore:
        Placed [[]], [[]]

# TextsIn: fn Path ==> Element -> Text ==> unit      every text the layout measures: labels, button labels, input
texts, and each input's text up to its cursor
# PlaceStacked: fn Env, Path ==> Element, Rect, [Path], Bool -> Placed    children in key order, down or across
# InputDraw: fn Env, Rect, Text, Text, Nat -> [Draw]  a white box, the text, and a caret bar when the name is the
focus
# Clipped: fn Placed, Rect, [Hit] -> Placed           a Draw.Clip around the draw list; hits cut to the rect, the
region's own last
# Offset, MaxOffset, Up, Stacked, Beside, Padded: the arithmetic on offsets, sizes and rectangles their names say
```

`Or ?x, d`, as in `TextExtent`, is the optional default: `x` if there is one and `d` otherwise. Boolean or is the keyword `or`.

`PlaceStacked` walks the children in key order, which is tree order, which is top to bottom or left to right, allotting each its measured extent, placing it with its path appended to the chain, and concatenating the draw lists and hits with the children's hits *before* the container's own, so that the innermost region at a point is found first. `InputDraw` finds the caret's x by measuring the text up to the cursor, `TextExtent env, (Take t, c)`, which is why `TextsIn` lists that prefix: the library asks the World for the width of every text it will need, and `Gui.Metrics` answers all of them in the occurrence of the question. `Clipped` wraps the draw list in a `Draw.Clip` and discards hits outside the clip rect. `Stacked`, `Beside`, `Padded`, `Up` and the rest are the arithmetic their names say. None of this touches time: `Measure` and `Place` are functions on plain values, testable without a World, and when they are applied to a dynamic tree they lift to a dynamic picture.

Hit-testing is a lookup in the hit list, and turning a hit into an input is a fold over its chain:

```
let Contains: fn Rect, Point -> Bool = \|r, p|
    r.x <= p.x and p.x < r.x + r.w and r.y <= p.y and p.y < r.y + r.h

let HitAt: fn [Hit], Point -> ?Hit = \|hits, p|
    FirstWhere hits, \Contains _.rect, p

let Nest: fn [Path], Response -> Response = \|chain, r| match chain[0]:
    Some head: ChildInput(path=head, response=(Nest (Rest chain), r))     # Rest: the array without its first
    element
    Nil: r

let Deliver: fn [Path], Response -> Path ==> Response = \|chain, r| match chain[0]:
    Some head: {head: (Nest (Rest chain), r)}
    Nil: {}
```

`Deliver` turns a chain `[[box, row, button]]` and a `click` into `{box: ChildInput(path=row, response=ChildInput(path=button, response=click))}`: a response to the box, which the box peels and sends to the row, which the row peels and sends to the button. The map's one key is the first element of the chain, a path in the window's child's list, which is the key `!down` needs.

## The window

The window interceptor claims every library element, lays the tree out, draws it, hit-tests the pointer, keeps focus and scroll offsets, and routes keys and wheel events:

```
type Wheeled = record:
    name: Text
    most: Float
    by: Float

let PointerOf: fn WindowInput -> ?Point = \|w| match w:
    Pointer p: Some p
    ignore: Nil

let PressedOf: fn WindowInput -> ?Key = \|w| match w:
    Pressed Tab: Nil
    Pressed k: Some k
    ignore: Nil

let IsTab: fn WindowInput -> ?unit = \|w| match w:
    Pressed Tab: Some unit
    ignore: Nil

let WheelOf: fn WindowInput -> ?((Point, Float)) = \|w| match w:
    Wheel ((p, d)): Some ((p, d))
    ignore: Nil

let ResizeOf: fn WindowInput -> ?Extent = \|w| match w:
    Resized x: Some x
    ignore: Nil

let ClickFor: fn Hit -> ?(Path ==> Response) = \|h| match h.kind:
    Clickable: Some (Deliver h.chain, click)
    Focus ignore: Nil
    Scrollable ignore: Nil

let FocusOf: fn Hit -> ?Text = \|h| match h.kind:
    Focus n: Some n
```

```
    ignore: Nil

let NextFocus: fn [Hit], ?Text -> ?Text = \|hits, current|
    After (Collect hits, FocusOf), current   # the name after current in hit order, wrapping; Nil if none

let RouteKey: fn [Hit], ?Text, Key -> ?(Path ==> Response) = \|hits, focus, k|
    match focus:
        Nil: Nil
        Some n: match (FirstWhere hits, \(FocusOf _) == Some n):
            Some h: Some (Deliver h.chain, k)
            Nil: Nil

let WheelTarget: fn Hit, Point, Float -> ?Wheeled = \|h, p, d| match h.kind:
    Scrollable ((n, most)): if Contains h.rect, p: Some (Wheeled n, most, d) else: Nil
    ignore: Nil

let WheelAt: fn [Hit], ((Point, Float)) -> ?Wheeled = \|hits, ((p, d))|
    (Collect hits, \WheelTarget _, p, d)[0]  # hits are in tree order, children first: the innermost region
under the pointer

let Clamp: fn Float, Float, Float -> Float = \|lo, hi, v|
    if v < lo: lo else: if hi < v: hi else: v

let Scrolled: fn Text ==> Float, Wheeled -> Text ==> Float = \|offsets, w|
    Insert offsets, w.name, (Clamp 0.0, w.most, (Or offsets[w.name], 0.0) + w.by)

@Window: fn Text, Extent, @a -> @a = \|title, size0, @app| @widget:
    ((result, $slots)) <- intercept @app, !down, $ \|e| @Element e
    !raw <- @Element Gui.Window(title=title, size=size0)
    let $kids = UiOnly $slots
    $sizes <- for (TextsIn $kids):              # a Gui.Metrics slot per distinct text
        $m <- @Element Gui.Metrics(text=key, font=Default)
        @ $m
    $size <- size0 then !Filter (ResizeOf !raw)
    let $root = Ui.VBox(children=$kids)
    let $placed = Place (Env $sizes, $offsets, $focus), $root, (Rect 0.0, 0.0, $size.w, $size.h), [[]]
    @Element Gui.Scene(window=title, draw=$placed.draw)

    let !hit = !Filter (HitAt $placed.hits, !Filter (PointerOf !raw))
    $focus <- Nil then (Some !Filter (FocusOf !hit)) | (NextFocus $placed.hits, $focus) on !Filter (IsTab
    !raw)
    $offsets <- {} then Scrolled $offsets, !Filter (WheelAt $placed.hits, !Filter (WheelOf !raw))
    rebuild $focus:
        match _:
            Some n: @Element Gui.Focusable(name=n)
            Nil: @ unit

    let !clicks = !Filter (ClickFor !hit)
    let !keys = !Filter (RouteKey $placed.hits, $focus, !Filter (PressedOf !raw))
    !fwd <- @Forward (NonUi $slots)
    let !down = !Union (!clicks | !keys), !fwd
    @ result
```

Walk the lines. `$kids` is the app's library elements, and `$root` is those elements as one tree, a derived dynamic. The `for` over the texts in the tree builds one `Gui.Metrics` slot per distinct text and consolidates their answers into `$sizes`; a text that appears in this occurrence has its slot built in this occurrence, and its answer is there too, so `$placed`, the layout of the tree at the window's current size with the current sizes, offsets and focus, is consistent in every occurrence in which anything changed.

The scene element draws its list, a fresh slot per change. The window's raw input is split by plain filters. A pointer is hit-tested against `$placed.hits` *read through a tag*, the layout as the occurrence opened, which is the layout the user was looking at when they clicked. A hit on a button becomes a `Deliver` addressed to the app's slot at the head of the chain; a hit on an input moves focus and delivers nothing, since an input responds only to keys. Focus is a hold of an optional name, moved by a click on an input or by Tab, which steps through the input names in tree order. A key is routed to the input whose name matches the focus as the occurrence opened, again through tags. A wheel event moves the offset of the innermost scroll region under the pointer, clamped to the largest offset that region's content allows. The `Gui.Focusable` is the window telling the World which name has focus, rebuilt whenever the hold changes. The window delivers one `WindowInput` per occurrence, so `!clicks | !keys` loses nothing. Everything that is not a library element is forwarded, and `!down` is the union.

There is no second pass, no layout phase, no deferred focus. Hit-testing is a lifted operation over a dynamic and an event, so it runs in the input's occurrence, against the list the user saw, which is the routing rule. The scene is derived, so it changes promptly in any occurrence in which anything on the screen changed, and the World draws it after the turn. Focus moving to an input built in this occurrence cannot happen in this occurrence, because that input's name is not in `$placed.hits` as the occurrence opened; it happens the next time the user presses Tab or clicks, which is the only time focus moves anyway. A program that wants an input focused as soon as it appears puts the name into the window's focus through a request element, which the window answers with a `Defer`, and that is a few more lines in the same language.

## A form, occurrence by occurrence

Take a small form under the window:

```
@form:
    $name <- @TextInput "name", "Ada"
    !ok <- @HBox @widget:
        !ok <- @Button $ "OK"
        !cancel <- @Button $ "Cancel"
        @ !ok
    @Label "Hello, {$name}"
    @ unit

@main:
    @Window "Form", (Extent 400.0, 300.0), (@VBox @form)
```

Paths, from the window's child down. The `@VBox` is the window's child, and it holds everything back, so the app's list has one library element, the box's `Ui.VBox`, at `1/#0/0`. Inside the box, the form's elements: the text input at `0/#0/0`, the HBox's element at `1/#0/0`, the greeting label at `2/#0/0`. Inside the HBox, the two buttons at `0/#0/0` and `1/#0/0`. A path is only a key in the list of the interceptor that reads it, so the same string names different slots at different levels.

The World's text measurement is fixed-width in this walk: eight pixels per character and sixteen per line. A label pads its text by 4 on each side, a button by 8, an input by 4, which puts the input at the top left with width 32 and height 24, the two buttons below it from y = 24, `OK` from x = 0 to 32 and `Cancel` from x = 32 to 96, each 32 high, and the greeting at y = 56, 88 wide.

**Occurrence 0 (build).** Everything is built. The `Gui.Window` slot is created. `TextsIn` finds four distinct texts, `Ada` (the input's text and its text up to the cursor), `Cancel`, `Hello, Ada` and `OK`, and the `for` builds a `Gui.Metrics` slot for each; the oracle answers `Extent 24.0, 16.0` for `Ada` and so on,

in this occurrence. `Place` lays the tree out with those sizes, and the scene slot is created with the draw list. `$focus` is `Nil`, so no focusable. The World sees:

```
1:              Gui.Window(title="Form", size=Extent(w=400.0, h=300.0))
2/[Ada]/0:      Gui.Metrics(text="Ada", font=Default)
2/[Cancel]/0:   Gui.Metrics(text="Cancel", font=Default)
2/[Hello, Ada]/0: Gui.Metrics(text="Hello, Ada", font=Default)
2/[OK]/0:       Gui.Metrics(text="OK", font=Default)
4/#0/0:         Gui.Scene(window="Form", draw=[[...]])
```

It draws the scene and waits.

**Occurrence 1.** The user clicks on the OK button. The World sends `Pointer (Point 12.0, 40.0)` to `1`. `PointerOf` passes the point; `HitAt` reads `$placed.hits` through a tag and finds the button's region first (leaves before containers), with chain `[[1/#0/0, 1/#0/0, 0/#0/0]]`: the box in the app's list, the HBox in the box's list, the OK button in the HBox's list. `ClickFor` turns it into `{1/#0/0: ChildInput(path=1/#0/0, response=ChildInput(path=0/#0/0, response=click))}`. `!down` fires with it; `intercept` delivers to the box's slot at `1/#0/0` in the app's list as opened. The box's `!in` fires; `Inward` turns the `ChildInput` into `{1/#0/0: ChildInput(path=0/#0/0, response=click)}`; the box's `!down` delivers it to the HBox's slot; the HBox peels again and delivers `{0/#0/0: click}`; the OK button's `!in` fires with `click`; `!ok` fires in the form. Nothing on the screen changes, so no scene is rebuilt. Three interceptors and one leaf, in one occurrence, each one line of routing.

**Occurrence 2.** The user clicks in the text field. `Pointer (Point 12.0, 10.0)` to `1`. `HitAt` finds the input's own region, with kind `Focus "name"`. `ClickFor` gives `Nil`, since an input responds only to keys, and `FocusOf` fires `Some "name"`; `$focus` changes to `Some "name"`. The layout depends on the focus, so `$placed` changes promptly and the scene rebuilds to `4/#1/0`, now with a caret bar after `Ada`; and the focus rebuild builds `Gui.Focusable(name="name")` at `7/#1/0`. Commit: focus is the input, and the World can show the platform's focus for it.

**Occurrence 3.** The user types `!`. `Pressed (Char '!')` to `1`. `PressedOf` passes it; `RouteKey` reads `$placed.hits` and `$focus` through tags, finds the hit whose kind is `Focus "name"`, and `Deliver` addresses `Char '!'` down its chain `[[1/#0/0, 0/#0/0]]`. The box peels; the input's `!key` fires; `Apply $edit, !key` fires with `Edit "Ada!", 4`; `$edit` changes. The input element is a fresh slot at `0/#1/0`; the box's `children` changes and the box is a fresh slot at `1/#1/0`; `$kids` changes; the greeting label changes too, since `$name` is `$edit.text`. Now the measurer: `TextsIn` finds `Ada!`, `Cancel`, `Hello, Ada!` and `OK`. The `for` builds `Gui.Metrics` slots for `Ada!` and `Hello, Ada!` in this occurrence and the oracle answers them in this occurrence; `Ada` and `Hello, Ada` are torn down at commit. `$sizes` has the new widths before the occurrence ends, `$placed` is laid out with them, and the scene is rebuilt at `4/#2/0`. There is no occurrence in which the text has changed and its width has not. Commit: `$edit` is `Edit "Ada!", 4`, and `$focus` is still `Some "name"`, which names the new input slot as well as it named the old one.

**Occurrence 4.** The user presses Tab. `IsTab` passes; `NextFocus` reads the hit list and the focus through tags; there is one input, so the name after `"name"`, wrapping, is `"name"` again. A hold set to the value it already has still changes, so the scene and the focusable are rebuilt with equal contents; the World's diff finds nothing different, and `$Distinct` on `$focus` would suppress the rebuild. A form with two inputs would move to the second here, and Tab on the last would wrap to the first, which is what `After` does; a library that wants Shift-Tab adds a key and a `Before`.

**Occurrence 5.** The user resizes the window. The World sends `Resized (Extent 600.0, 300.0)` to `1`. `$size` changes; `$placed` changes promptly; the scene rebuilds with the new rectangles. Nothing in the form ran, and the next click is hit-tested against the new layout because the next occurrence opens with it.

This prevents the afternoon where the click landed on the button that had just moved, because layout ran after the event was dispatched and the hit-test used the new positions while the user had seen the old ones. Here the hit-test reads the layout the user saw, by the tag law, and no layout pass can get between a click and the thing it was aimed at.

## Tests that look inside

A `Ui.VBox` holds its children as data, so the Test World can look inside it. `expect Ui.Button(label="OK")` matches a slot at any depth because the suite's matcher descends into `children` fields, and `click Ui.Button(label="OK")` wraps `click` in one `ChildInput` per level and delivers it to the outermost slot on the chain, which is the same delivery a window would make after a hit-test. The Test World knows the convention because the library is part of the suite, not because the language does; a library that nested differently would ship a different matcher.

A library test of the form above, with no window at all:

```
test form_greets, (@VBox @form):
    build
        => expect Ui.Label(text="Hello, Ada")
    send Ui.TextInput(name="name", ..), Char '!'
        => expect Ui.Label(text="Hello, Ada!")
    click Ui.Button(label="OK")
        => settles
```

The test drives the library below the window, so it checks routing through the boxes and the input without any layout. A test of the window adds the measurer, which the Test World has no answers for. The window creates its `Gui.Metrics` slots at build, so the script gives each one a value before the build:

```
test window_focus, (@Window "Form", (Extent 400.0, 300.0), (@VBox @form)):
    with Gui.Metrics(text="Ada", font=Default), Extent 24.0 = 16.0, with Gui.Metrics(text="Cancel",
font=Default), Extent 48.0 = 16.0, with Gui.Metrics(text="Hello, Ada", font=Default), Extent 80.0 = 16.0, with
Gui.Metrics(text="OK", font=Default), Extent 16.0 = 16.0, build
        => settles
    send Gui.Window(..), Pointer (Point 12.0, 10.0)
        => expect Gui.Focusable(name="name")
```

`Place` is a plain function, so most of the window's behaviour is a unit test of `Place` and `HitAt` against a tree and a map of sizes.

## What the library does not do, yet

Text longer than its box needs wrapping, which is line breaking over widths: the library asks `Gui.Metrics` for the prefixes it needs, as `InputDraw` does for the caret, and breaks where the widths say. A library with more than one font passes the font along: `Gui.Metrics` already has the `font` field, the draw commands gain one, and `$sizes` is keyed by text and font. Size feedback, where a child wants to be a different size because of the size it was given, is a cycle through the layout and needs a `Defer` to break it; the library above has no such child. An accessibility tree is a derived dynamic over the same element tree the layout uses, with the input names as node ids, emitted as an element of its own for a World that speaks to a screen reader. Animation is a `Gui.Frames` slot inside a `rebuild` over whether anything is moving, so that a window at rest asks the World for no frames, and each `Frame` is an input to a hold. Each is more Prism over the same five tags, and none is a new primitive.

# Worked interceptors

Each of the following is a complete interceptor with a short walkthrough. They are presented in order of how much they do beyond forwarding, and every one of them is `@Forward` with a different `match`. The point of the sequence is that nothing in it needed a mode of `intercept`.

## Transparent forwarding

`@Transparent` from the section on `intercept` is the identity: every slot re-emitted with its request wired in, every response routed back under the child's path, every state element supplied by asking outside for the same value. A child under it has the same transcript it would have without it, one level deeper in the outer list. It is in the conformance suite because it is the law every other interceptor is measured against, and it is in the library because a wrapper that wants to observe without altering starts from it.

## A path-rewriting sandbox

```
let Confine: fn Text, Text -> ?Text = \|root, p|
    if (StartsWith p, "/home/"): Some (Concat [[root, p]])
    else: Nil

@Jail: fn Text, @a -> @a = \|root, @child| @widget:
    ((result, $slots)) <- intercept @child, !down, $ supply
    !down <- for $slots:
        rebuild $_:
            match _.element:
                Fs.File(path=p):
                    match Confine root, p:
                        Some q:
                            !resp <- @Element Fs.File(path=q, requests=_.!request)
                            @ !resp
                        Nil:
                            !later <- @Element Defer(requests=(unit on _.!request))
                            @ Denied on !later
                Fs.Dir(..):
                    @ !never                            # a state element: supplied below, not forwarded here
                ignore:
                    ((!resp, ignore)) <- @Element _.element
                    @ !resp
    let supply = \|e| match e:
        Fs.Contents(path=p): match Confine root, p:
            Some q: @Element Fs.Contents(path=q)
            Nil: @ $ Unavailable
        Fs.Dir(path=p): match Confine root, p:
            Some q: @Element Fs.Dir(path=q)
            Nil: @ $ Unlisted
        ignore: @Element e
    @ result
```

Two directions, two policies. Requests to files under `/home/` are forwarded with the path rewritten under the root, and the response comes back under the child's path; requests to anything else are denied, one occurrence later, through a `Defer`. State for files under `/home/` is forwarded with the same rewrite; state for anything else is a constant, `Unavailable` for contents and `Unlisted` for a listing, each a case of the type the child already matches on. The `Fs.Dir` arm in the response loop returns `!never` because a state element has no responses; it is there so that the catch-all does not put a second,

unconfined `Fs.Dir` element in the outer list beside the one the supplier makes. Everything else goes through untouched.

The child cannot tell the difference between this and a World whose filesystem is rooted at `root` and refuses everything outside `/home/`. That is the test of the primitive. The worked example that runs a file viewer under a jail walks the build occurrence, a forwarded read's answer coming back, a confined path getting a constant, and a denial arriving in the second occurrence of a turn.

## A denial sandbox

A sandbox that denies a whole namespace needs no rewrite:

```
let Forbidden: fn ?HttpRequest -> ?HttpResponse = \|asked| match asked:
    Some r: Some (HttpResponse r, 403, "sandboxed")
    Nil: Nil

@NoNetwork: fn @a -> @a = \|@child| @widget:
    ((result, $slots)) <- intercept @child, !down, $ supply
    !down <- for $slots:
        rebuild $_:
            match _.element:
                Net.Http(..):
                    !later <- @Element Defer(requests=(unit on _.!request))
                    $asked <- Nil then Some _.!request
                    @ !Filter (Forbidden $asked on !later)
                Net.Socket(..):
                    @ !never
                ignore:
                    ((!resp, ignore)) <- @Element _.element
                    @ !resp
    let supply = \|e| match e:
        Net.Socket(..): @ $ Disconnected "sandboxed"
        ignore: @Element e
    @ result
```

Nothing under `Net` reaches the outside. An HTTP request is answered with a 403 in the next occurrence of the turn; the response has to name the request it answers, so the block remembers the request in a hold across the `Defer` and reads it back through a tag when the answer comes. A socket's state is `Disconnected "sandboxed"` from its first occurrence, which is what a socket to an unreachable host looks like; its sends go nowhere and it never receives, which is also what that socket looks like. The child's `match` on `Link` already has the arm.

The strong objection to this whole arrangement is that forwarding by default gives every widget the World's full access unless something above it says otherwise. That is true, and it is the same as `std::fs` in any language. What the language adds is confinement per subtree, in the program, without a process boundary, so a parent can confine a child it did not write; and the guarantee that there is nothing else, no global, no context, no ambient channel, that the confinement could miss. The standard library should ship a sandbox, and the guidance for third-party widgets should be to wrap them. As in any capability system, an interceptor above the sandbox must not turn the child's data into authority: a sandbox that forwards `Fs.File(path=p)` for any `p` the child chooses has confined nothing.

## A cache for state

A child that watches a file sees every change the World pushes. A cache shows it the value at start and at a refresh, and nothing in between:

```
let Freeze: fn a -> $a = \|v| $ v

@Cached: fn !unit, @a -> @a = \|!refresh, @child| @widget:
    ((result, $slots)) <- intercept @child, !down, $ supply
    !down <- @Forward $slots
    let supply = \|e| match e:
        Fs.Contents(path=p): @widget:
            $live <- @Element Fs.Contents(path=p)
            !started <- @started
            $$shown <- $live then (Freeze $live) on (!started | !refresh)
            @ $Join $$shown
        ignore: @Element e
    @ result
```

The supplier returns a blueprint, and the widget has state. For an `Fs.Contents` element it builds, in the cache's widget, a live slot for the same path, asking the outer context; and a hold of dynamics `$$shown`, initially the live dynamic itself, replaced by a frozen constant of the live value at start and at each refresh. `(Freeze $live) on !started` fires in the supplied widget's build occurrence with the constant dynamic of `$live`'s value before that occurrence, which is the oracle's answer; the hold adopts it in that occurrence; `$Join $$shown` has, before the occurrence, `$live`'s value, and after it, the frozen constant with the same value. The child's slot sees one value in its creation occurrence and no hiccup. From then on pushes move `$live` and not `$$shown`, until a refresh freezes the live value again.

The cache is also a cache in the ordinary sense. The supplied widget is built once per distinct value, so two child slots for `/etc/app.conf`, in two widgets that have never heard of each other, read one `$Join $$shown` from one live slot, and the outer World is asked once. That is the equality law inside an interceptor and the sharing of interest in one mechanism.

The tag law shows up when a push and a refresh land in one occurrence: `(Freeze $live) on !refresh` reads `$live` before the occurrence, so the cache shows the value before the push and the next refresh shows the pushed one. A cache that wanted the pushed value in the same occurrence as the click cannot have it, because the only window onto a dynamic at an event is the value before, and that is the same rule everywhere else.

## A cache for requests

A request cache remembers answers and serves repeats without going outside:

```
let Lookup: fn Text ==> HttpResponse, HttpRequest -> ?HttpResponse = \|memo, r|
    if r.method == "GET": memo[r.url] else: Nil

let Missing: fn Text ==> HttpResponse, HttpRequest -> ?HttpRequest = \|memo, r|
    match Lookup memo, r:
        Some ignore: Nil
        Nil: Some r

let Remember: fn Text ==> HttpResponse, Path ==> HttpResponse -> Text ==> HttpResponse = \|memo,
answers|
    Fold answers, memo, \|acc, ignore, a| Insert acc, a.request.url, a
```

```
type CacheOut = record:
    !resp: Response                              # what goes down to the child
    !learned: HttpResponse                       # what came back from outside

@HttpCache: fn @a -> @a = \|@child| @widget:
    ((result, $slots)) <- intercept @child, !down, $ \|e| @Element e
    out <- for $slots:
        rebuild $_:
            match _.element:
                Net.Http(..):
                    let !hit = !Filter (Lookup $memo, _.!request)
                    let !miss = !Filter (Missing $memo, _.!request)
                    !fetched <- @Element Net.Http(requests=!miss)
                    !later <- @Element Defer(requests=(unit on !hit))
                    $pending <- Nil then Some !hit
                    @ CacheOut (!fetched | !Filter ($pending on !later)), !fetched
                ignore:
                    ((!resp, ignore)) <- @Element _.element
                    @ CacheOut !resp, !never
    let !down = out.!resp
    $memo <- {} then Remember $memo, out.!learned
    @ result
```

Each child `Net.Http` slot gets a block. When its request fires, the block reads `$memo` through a tag, the cache as the occurrence opened. A miss goes out on the block's own `Net.Http` element and comes back as `!fetched`, under the child's path. A hit is remembered in `$pending` and answered through a `Defer`, so that it arrives in a later occurrence like a real answer would. The block returns a record, which `for` consolidates field by field: `out.!resp` is the map of responses going down, and `out.!learned` the map of answers that came from outside, from which `$memo` is held.

`$memo` is read by the blocks whose answers define it, and that is a legal cycle because the read is a tag. Two identical GETs in one occurrence from two child slots both miss, because neither sees the other's answer before the occurrence, and both go out. A cache that wanted to coalesce them would key an outstanding-requests hold by URL and attach the second slot to the first's answer, which is a longer program in the same language.

The conformance suite's `cache_hit_no_request` test puts two identical GETs in two turns under this interceptor and checks that the second turn has `unrequested Net.Http(..)` at the top and that the child still received an answer, in a later occurrence than its request.

## A mock for tests

```
@MockHttp: fn (fn HttpRequest -> HttpResponse), @a -> @a = \|answer, @child| @widget:
    ((result, $slots)) <- intercept @child, !down, $ \|e| @Element e
    !down <- for $slots:
        rebuild $_:
            match _.element:
                Net.Http(..):
                    !later <- @Element Defer(requests=(unit on _.!request))
                    $asked <- Nil then Some _.!request
                    @ !Filter (answer $asked on !later)
                ignore:
                    ((!resp, ignore)) <- @Element _.element
                    @ !resp
    @ result
```

Every HTTP request in the subtree is answered from a pure function, through a `Defer`, and nothing in `Net` reaches the outside. `answer $asked on !later` lifts the plain function over a dynamic and an event: it fires when the `Defer` answers, with the function applied to the remembered request. The mock is for previews and for tests of a widget's request logic that should not depend on a Test World's fake network. Because the answer goes through a `Defer`, a widget under the mock sees the timing it would see in production: never in the occurrence it asked. The suite's `mock_timing` test checks exactly that; it catches the mocks that hide a timing bug the real World would have shown.

A mock for state is the supplier alone:

```
@MockFs: fn Text ==> Text, @a -> @a = \|files, @child| @widget:
    ((result, $slots)) <- intercept @child, !down, $ supply
    !down <- @Forward $slots
    let supply = \|e| match e:
        Fs.Contents(path=p): match files[p]:
            Some t: @ $ Known t
            Nil: @ $ Unavailable
        ignore: @Element e
    @ result
```

A child under `@MockFs {"/etc/motd": "hi"}` reads `Known "hi"` in the occurrence it asks and `Unavailable` for anything else, with no World involved. A test that wants the file to change mid-test wraps the map in a dynamic and reads it in the supplied widget; the map's change is then a push, as far as the child can tell.

## A logger

```
tag Log.Line of Element:
    requests: !Text

@Traced: fn @a -> @a = \|@child| @widget:
    ((result, $slots)) <- intercept @child, !down, $ \|e| @Element e
    !down <- for $slots:
        rebuild $_:
            ((!resp, ignore)) <- @Element _.element
            @Element Log.Line(requests=("{ShowPath key} asked {ToText _.!request}" | "{ShowPath key} got
            {ToText !resp}"))
            @ !resp
    @ result
```

Forward everything and, per slot, fire a log line for each request and each response. `key` is the child's path, so the line says which slot spoke; interpolating an event into a text literal lifts the literal to an event of text, so each side of the `|` fires when its event does. The two cannot coincide for one slot, since a response is never in the occurrence of its request, so `|` is exact. The log lines are `Log.Line` requests in the tracer's own list, which the World performs in list order after the occurrence, so a transcript reads in the order the program spoke. Nothing about the child's timing changes, because the tracer added no occurrences.

## A quota

```
let IfUnder: fn Nat, Nat, HttpRequest -> ?HttpRequest = \|used, limit, r|
    if used < limit: Some r else: Nil

let IfOver: fn Nat, Nat, HttpRequest -> ?HttpRequest = \|used, limit, r|
```

```
    if used < limit: Nil else: Some r

let Refusal: fn ?HttpRequest -> ?HttpResponse = \|asked| match asked:
    Some r: Some (HttpResponse r, 429, "quota")
    Nil: Nil

@Quota: fn Nat, @a -> @a = \|limit, @child| @widget:
    ((result, $slots)) <- intercept @child, !down, $ \|e| @Element e
    !asked <- for $slots:                        # every HTTP request fired this occurrence, by path
        rebuild $_:
            match _.element:
                Net.Http(..): @ _.!request
                ignore: @ !never
    $used <- 0 then $used + Size !asked          # Size: the prelude's count of a map's entries
    !down <- for $slots:
        rebuild $_:
            match _.element:
                Net.Http(..):
                    let !allowed = !Filter (IfUnder $used, limit, _.!request)
                    let !refused = !Filter (IfOver $used, limit, _.!request)
                    !fetched <- @Element Net.Http(requests=!allowed)
                    !later <- @Element Defer(requests=(unit on !refused))
                    $held <- Nil then Some !refused
                    @ !fetched | !Filter (Refusal $held on !later)
                ignore:
                    ((!resp, ignore)) <- @Element _.element
                    @ !resp
    @ result
```

Forward until a count is reached and refuse after. Two `for`s over the same list: the first collects every HTTP request fired in an occurrence into one map, whose size moves the count; the second does the routing, with each block reading the count through a tag. Two requests in one occurrence both see the count as it opened and both pass if it was under the limit, so the quota can overshoot by the number of simultaneous requests; a quota that must be exact folds over the consolidated map and lets the leftmost through, which is a change to the first `for` and not to the primitive. A refusal is a 429 through a `Defer`, so the child sees it in a later occurrence like any other answer.

## Migration between two outer contexts

An interceptor has one outer context, its own widget's list, but that list can hold elements of two kinds, and a dynamic can choose between them:

```
type Backend = choice Local | Remote

tag Kv.Get of Element:
    requests: !Text
    responds: ?Text

tag Kv.Store of Element:
    key: Text
    state: ?Text

tag Remote.Get of Element:
    host: Text
    requests: !Text
    responds: ?Text
```

```
tag Remote.Store of Element:
    host: Text
    key: Text
    state: ?Text

let IfLocal: fn Backend, Text -> ?Text = \|b, r| if b == Local: Some r else: Nil
let IfRemote: fn Backend, Text -> ?Text = \|b, r| if b == Remote: Some r else: Nil

@Migrate: fn $Backend, Text, @a -> @a = \|$backend, host, @child| @widget:
    ((result, $slots)) <- intercept @child, !down, $supply
    !down <- for $slots:
        rebuild $_:
            match _.element:
                Kv.Get(..):
                    let !local = !Filter (IfLocal $backend, _.!request)
                    let !remote = !Filter (IfRemote $backend, _.!request)
                    !a <- @Element Kv.Get(requests=!local)
                    !b <- @Element Remote.Get(host=host, requests=!remote)
                    @ !a | !b
                ignore:
                    ((!resp, ignore)) <- @Element _.element
                    @ !resp
    let $supply = Supplier $backend, host
    @ result

let Supplier: fn Backend, Text -> (fn Element -> @$State) = \|backend, host| \|e| match e:
    Kv.Store(key=k): match backend:
        Local: @Element Kv.Store(key=k)
        Remote: @Element Remote.Store(host=host, key=k)
    ignore: @Element e
```

Requests are routed per firing: each `Kv.Get` request reads `$backend` through a tag and goes out on one of two elements, both of which exist in the interceptor's list for the life of the child's slot, and the answer comes back from whichever was asked. State is routed per slot: the supplier is a *dynamic* of a function, `Supplier $backend, host` lifted over `$backend`, so a `Kv.Store` slot created while the backend is `Local` is supplied by a local store element and keeps it, while a slot created after the backend changes to `Remote` is supplied by a remote one. A value already being served keeps its supplied widget whatever the backend says, by the equality law, so two child slots for one key never disagree; and when the last slot for a key goes and a new one comes under the new backend, the key migrates. That is the behaviour a migration wants: existing readers see a consistent store, new readers see the new one, and the cutover is per key rather than per program.

This is the one interceptor in the chapter that passes a non-constant supplier, and it shows what the dynamic is for. Policy over time is a dynamic like any other; the runtime reads it at the instant a value first needs a supplier and never again for that value.

## A Prism-hosted World

A World can be written in Prism: a program whose children are other programs, each under its own interceptor, with the host deciding what each one sees and what reaches the World below. The shell of the worked examples shows two apps sharing one screen. Here is a host with a process table, so that the set of programs is a keyed collection that changes while the host runs:

```
type Launch = choice:
    Start Text
    Stop Text

let ApplyLaunch: fn Text ==> unit, Launch -> Text ==> unit = \|table, l| match l:
    Start name: Insert table, name, unit
    Stop name: Remove table, name

@Host: fn Text ==> @unit -> @unit = \|programs| @widget:
    $clock <- @Element Clock
    let supply = \|e| match e:
        Clock: @ $clock
        Process.Args(..): @ $ [[]]
        ignore: @Element e
    !launch <- @Launcher (Keys programs)              # a menu returning Start name or Stop name
    $table <- {} then ApplyLaunch $table, !launch
    $lists <- for $table:                             # key is the program's name
        match programs[key]:
            Some @app:
                ((ignore, $slots)) <- intercept @app, !down, $ supply
                !down <- @Forward (NonUi $slots)
                @ $slots
            Nil:
                @ $ {}
    @Window "Host", (@Tiled $lists)
    @ unit
```

Each entry in the process table is a `for` block, keyed by the program's name, that builds the named program under an `intercept`, forwards everything that is not GUI, and returns the program's element list as a dynamic; the `for` consolidates those into `$lists : $(Text ==> (Path ==> Slot))`, one list per running program, which a tiling interceptor lays out side by side and routes clicks into by the same `Into` convention as a box. The `match` inside the block chooses between two blueprints by a plain value, `programs[key]`, which is the one form a `match` with blueprint arms may take. The supplier gives every program the host's own clock, so there is one clock across the host and all its guests, and an empty argument list; everything else is forwarded. A `Stop` removes the key, the block is torn down, the program's slots leave the outer list, and the World below releases whatever it was doing for them.

From a guest's side, the host is its World: it sees buttons and labels answered, a clock with a value, files it asked for. From the World below, the host is one program with a window, a clock and some forwarded file reads. A guest that wants to escape has no way to: it cannot name the host, cannot reach another guest, cannot find a global, and every element it makes passes through `supply` or the `for` block's `match` before anything outside knows it exists. That is what a World is, and the only difference between this one and the Rust one below it is that this one has an outside to forward to.

The clock deserves a second look because it is the host's one piece of policy. `supply` returns `@ $clock` for every guest's `Clock`: not an element, the host's own dynamic, so the guests' clocks are not even forwarded, they are the host's clock read directly. A host that wanted to run a guest at half speed, or frozen for a debugger, would return a different dynamic for that guest, and the guest would have no way to know. That is the sandbox argument applied to time, and it is why there is no `now()`.

## Negative space

The boundary is defined as much by what it refuses as by what it offers. Each refusal below closes a hole, and each one also lets the implementation, and every World, leave out a mechanism.

**No dynamic fields in elements.** An element is born with its fields and keeps them; a label whose text changes is a new label at the same spot. What it buys: the element list is plain data, so a World diffs maps instead of subscribing to anything, value identity never has to pick a moment, a response is always to the slot that asked, and the runtime has no subscription object that crosses the boundary and no notification that a field moved. The cost is a fresh slot per change, and the table in the section on dynamic fields is the argument that the cost is right in every case but one, and that the one has a better answer in the library.

**No request on creation.** Creating an element never, by itself, makes anything happen. What it buys: building is pure, so a rebuild costs exactly what it asks for and a `for` key appearing is not an effect; the trigger is a value the program can merge, gate and mock; there is one kind of request element, so an interceptor answers requests and nothing else; and in the occurrence a widget is built nothing can depend on an effect having already happened, because none has. The World core never has to watch for slots appearing, only for request events firing.

**No implicit forwarding.** An interceptor's child's elements reach the outside only if the interceptor makes elements for them. What it buys: a sandbox is a `match` whose catch-all arm is readable, and the question "can this child reach the network" is answered by reading it; a container holds back what it lays out and forwards the rest with one line; and the primitive has no table, no modes, and no second set of rules for when the table is consulted. The cost is that the shortest interceptor is seven lines rather than one, and the library ships the seven.

**No reaching past an interceptor.** Nothing a child creates goes further out by itself; nothing further out can address a child's slot except through the interceptor's response argument; a child cannot name its interceptor, its siblings, or anything above it. What it buys: the interceptor is a capability boundary with no back door, the Test World can stand in for any World because a program cannot tell who is answering, and a program that hosts programs is an ordinary program.

**No ambient context.** No globals, no "the current World", no environment read from the middle of a function, no clock. A widget has its arguments, its scope, and its elements. What it buys: there is no channel an interceptor does not see, so confinement is complete; arguments, environment and time are state elements, so a sandbox can replace them and a test can set them; and there is no implicit parameter for the compiler to thread or the runtime to look up.

**No round trip inside an occurrence.** A response is in a later occurrence than its request, always, from a World and from an interceptor. What it buys: nothing a program does can depend on an effect having happened in the moment it was asked for; routing is always against a list the sender has seen; and `Defer` is the only form of "later" the language needs, which bounds a turn without a cap.

**No placeholder for state.** A state element has the value the outside held, from its first occurrence. What it buys: a widget built mid-occurrence is consistent with everything it captured; `NotYet` does not exist, so no `match` has an arm for a value the world never held; and the equality law is a fact about one key rather than a promise made per slot.

**No widget values, no reified paths in the program.** A program cannot hold a path, compare two, or address a slot by one; only the outside addresses paths, and the program sees them only as the keys of an interceptor's list. What it buys: ownership is a tree, a path is never reused, and a program cannot keep a stale reference to a slot that its parent chose to remove.

The pattern repeats from the rest of the language: a feature that looks essential is removed, the removal closes a hole, and the same removal lets the World be a loop that reads a map and writes a map.

## Recapitulation

An element is plain data in a list. A tag says what it is and which of three things it does. `@Element` makes a slot and returns what the tag declares. Nothing the slot asks for is answered in the occurrence it asked, and that is a law bound on the World by the structure of the run call and on interceptors by routing against the opened list. A slot has a path for its conversations and a value for its facts. The list is a dynamic, read by whoever is outside, and the diode says the outside reads it and never reacts to it. State is there in its first occurrence because nothing else keeps a widget built mid-occurrence consistent with what it captured, and two readings of one value in one occurrence agree because there is one value. `intercept` exposes the two directions and nothing else, so that forwarding is the shortest program and every policy is a longer one, and a World written in Prism is a program with an outside to forward to.

Everything above the two World-facing GUI tags, and everything between a program's file read and the disk, is a `for` over a list of slots with a `match` inside it. The chapter on Worlds says what the loop at the bottom has to do, and it is short, because this chapter took everything else.

## Chapter 7

# Worlds

A Prism program cannot write a byte, read a clock, or hand anyone a function to call later, so something outside it must do all of that, and the whole design of the boundary is in how little that something has to know. A World is the outermost interceptor: written in the host language, the owner of the main loop, the only thing that starts an occurrence, and the only thing with effects. It drives the runtime through one synchronous call per occurrence, sending plain data in and reading plain data out, and nothing else ever crosses: no callbacks, no runtime objects, no threads, no second set of rules for "later." This chapter is the run call in full; the snapshot discipline that makes state elements honest and why every system that delivers state to subscribers ends up rediscovering it; turns and `Defer`; the obligations a World takes on and who each one protects; the world core that every World shares; the general components for files, time, processes, networks and errors, each with its tag declaration and an example; the CLI World and the GUI Worlds, including the one where the World is nothing but a socket and the display protocol is Prism; async runtimes, blocking calls, platforms, servers, Worlds written in Prism that host other programs, and parallelism. Throughout, the negative space: what a World may not do, and what each refusal buys.

## The World is the outermost interceptor, and it is small

You know the failure mode. The framework hands you a context object on startup. You stash it in a global, because the socket handler needs it and the socket handler has no other way to get it. Three months later a timer callback reads the context after it was torn down; a year later a test cannot run without a browser, because the context is the browser; and when you finally try to move the application onto a server you discover that the context is also the render loop, the clock and the filesystem, and that forty callbacks hold it. The program and its World grew into each other, and now neither can be replaced.

Prism's answer is that the program and the World never touch. A program is pure construction and pure propagation: it builds a tree of widgets, some of which put *elements* in a list, plain records like `Ui.Button(label="Increment")` or `Fs.File(path=path)`. The World reads that list. What the World sends back is also plain data: responses addressed to elements by path, and the state that state elements asked for. There is no object in the program that refers to the World and no object in the World that refers to the program. The program can run under a World that has no screen, no clock and no filesystem, and be told exactly what it would have done; and the World can be rewritten without the program noticing, which is the same promise seen from the other side.

From the program's side a World is an interceptor. `intercept` stands between a child and everything outside it, supplying the child's responses by path and its state by a supplier function, and making elements of its own upward or not. A World is the interceptor with nothing above it. Its responses come from sockets and keyboards rather than from a Prism event; its supplier is a synchronous host-language function rather than a widget; and it has no "upward," because it is where upward ends. A child cannot tell whether its parent is a sandbox written in Prism or the real World, and that is the

test the whole chapter is held to: every policy an interceptor could have, a World could have, and the other way round.

A World is small because the language can do nothing to it. It never registers a callback, so it never has to protect itself from re-entry. It never holds a runtime object, so it never has to worry about a lifetime. It never has to decide what a widget means, because a widget is the program's business and only elements reach the World. Everything it must get right is a short list of obligations, the same for every World, written once in a library the chapter calls the world core. What differs between Worlds is which tags they accept, and that is a list of components.

The goals this serves are the two that pull hardest against each other. The language has no effects, so IO must be impossible from inside it; and the language must be enough, so layout, focus, text editing, sandboxes, mocks and even a World that hosts other programs must be writable in Prism. A fat World would satisfy the first by taking everything; a leaky boundary would satisfy the second by letting the program reach out. The thin World satisfies both: it does only what touches hardware, and everything above the hardware is Prism, under the same laws as the rest of the program.

What a World is not: a renderer, a widget toolkit, a scheduler, a thread pool. A GUI World may contain a renderer, and a server World may contain a thread pool, but those are components inside it, and the language sees only tags. In particular a World never reads `Gui.*` elements in order to lay them out or decide focus. It reads a scene and delivers pointer events and keys to the window; which field is focused, what is under the pointer, where the caret is and what a tab key does are holds in a Prism library.

## The run call

The World drives the runtime through one synchronous call per occurrence. Here is the whole interface, in Rust-flavoured pseudocode. Everything in it is plain data except the oracle, which is a function from plain data to plain data, called only during the call and never retained.

```
/// Plain data: an interned tag plus field values, with structural
/// equality and ordering. This is what state identity compares.
pub struct ElementValue { tag: TagId, fields: Vec<Value> }

/// A slot's position in the widget tree, assigned at build and never
/// reused. Opaque to Worlds; ordered so that lists are in tree order.
pub struct Path(Vec<Segment>);
pub enum Segment { Bind(u32), Key(Value), Gen(u64) }

pub struct Inputs<'a> {
    /// Responses addressed to slots, by path. At most one per path.
    pub responses: Vec<(Path, Value)>,
    /// New values for state elements, by element value. At most one per value.
    pub pushes: Vec<(ElementValue, Value)>,
    /// The World's snapshot as this occurrence opens, as a function.
    /// Must answer synchronously for every state tag the World accepts.
    pub oracle: &'a dyn Fn(&ElementValue) -> Value,
}

pub struct Outcome {
    /// The element list after the occurrence, in tree order.
    pub list: Vec<(Path, ElementValue)>,
    /// For each slot whose tag declares `requests`, what it fired this
    /// occurrence, in firing order. Includes slots torn down this occurrence.
    pub requests: Vec<(Path, Vec<Value>)>,
```

```
    /// The program's result, for Worlds that read it (a test World does).
    pub result: Value,
}

pub enum RunError {
    UnknownPath(Path),                    // a response to a path not in the opened list
    WrongResponseType { path: Path, .. }, // a response the slot's tag does not declare
    UnsuppliedState(ElementValue, Span),  // a state tag the oracle cannot answer
    DuplicateResponse(Path),              // two responses to one path in one occurrence
    Cycle(Vec<Span>),                     // a prompt cycle found during propagation
    Panic { span: Span, message: String }, // overflow or a native bug, caught
}

impl Runtime {
    /// Run exactly one occurrence. Holds commit at its end. On `Err`,
    /// the graph is as it was before the call.
    pub fn run(&mut self, inputs: Inputs) -> Result<Outcome, RunError>;
}
```

That is the entire surface a World needs. The runtime's graph types exist and are public, because compiled widgets are built from them, but a World never names one. `Runtime` is `send` and not `Sync`: it can be moved onto a thread and driven there by one owner, and never shared.

## What goes in

*Responses* are addressed by path. A slot with a `responds` field gets at most one response per occurrence; two to one path is an error, because a response event fires at most once per occurrence and the World would be asking the runtime to pick. If the World has two chunks of standard input waiting, they are two occurrences, and the program sees them in order. If it has a click and a tick waiting for different slots, they may be one occurrence, and the program sees them as simultaneous, which is a claim the World is entitled to make about things that were waiting together.

*Pushes* are addressed by element value, not path, because state is identified by value: every slot whose element equals `Fs.Contents(path="/etc/hostname")` observes one and the same dynamic, and a push for that value reaches all of them, including a slot created in the push's occurrence. One push per value per occurrence, for the same reason as responses.

*The oracle* is the World's snapshot as a function. When the runtime creates a slot for a state element during the occurrence, whether in the build occurrence or in a `rebuild` firing halfway through propagation, it asks the oracle for that element's value and the slot's dynamic starts there. The oracle is a function rather than a map because the World cannot know in advance which values will be asked: a `rebuild` over a text input creates `Fs.Contents(path=whatever the user typed)` and the World learns the path by being asked. The answer must come from the World's snapshot as the occurrence opened and nowhere else; the next section is about why.

## What comes out

*The list* is every slot in the program after the occurrence, with its path and its element, in tree order. It is a description of what the program wants from the world, complete each time, and the World reads it the way a compositor reads a scene graph: it reconciles reality to the description. A draw list is drawn. A `Net.Listener(port=8080)` in the list means a socket should be bound; the same element in the next list means it should still be bound; its absence means it should not be. Reading the list is idempotent, and that is what separates a description from a trigger. Nothing is *done because* an

element appeared; the World makes the world match the list, and a list that is the same twice means nothing twice.

*The request firings* are the one-shot instructions: a write to standard output, a `Read` on a file, bytes on a socket, a `Code 0` on `Process.Exit`. Each is listed under its slot's path in the order it fired. The World performs them after the turn, in occurrence order and, within an occurrence, in list order. A firing from a slot that was torn down in the same occurrence is still reported, with its path; the World performs it if it can and discards any response, since there is no slot to address. The program asked, and the World does not second-guess that the asker is gone; what it cannot do is answer.

*The result* is the root widget's result, which most Worlds ignore (a CLI World runs `@main : @unit`) and a test World reads for `returns` and `fired`.

*The error* says the occurrence did not happen. Because every write during an occurrence is buffered to commit time, the error path is a rollback: the hold values, the switch targets, the widgets built during the occurrence are dropped, and the graph is as it was before the call. The inputs that opened the occurrence are lost, and the World decides what that means for it: a CLI World prints the error and exits nonzero, a GUI World might show it and keep going, a test World records it for an `errors` step.

## What never crosses

Nothing in `Inputs` or `Outcome` is an event, a dynamic, a blueprint, a widget, a closure over the graph, or a reference into the runtime. The oracle is the nearest thing to an exception and it is not one: it is a function the World owns, over the World's own data, that the runtime calls and forgets. The runtime never calls the World at any other time, never calls it after `run` returns, and never calls it from a thread the World did not call `run` on.

Suppose instead the runtime called the World back during propagation, "a response just fired, do your thing." The World's code would then run inside an occurrence, where holds have not committed and the element list is half built, and it would have to be told what it may and may not read. Every framework with a `componentDidUpdate` (from memory) has that table of rules, and every table has a footnote. Here there is nothing to put in the table because there is no call. Or suppose the World held a handle to the socket slot's response event and fired it directly. Then the World would have to know the event's lifetime, and when the `rebuild` around the socket rebuilt, the World's handle would be stale, and the stale handle is exactly the leaked DOM subtree the language was built to make impossible. Addressing by path, against a list the World was given, means the World can only ever name what the program told it exists.

The payoff is the test World. A test is a script of inputs and expectations, and because the World interface is data, a script can be a World. The program you test under a transcript is the program you ship under a screen, with the same run call and the same laws, and a transcript of inputs replays a session exactly.

## Occurrence zero

The first call is occurrence zero, with no responses and no pushes: it builds `@main` and returns the initial list. That is how a World learns what the program wants before anything has happened. The oracle is live during occurrence zero, because the build creates state slots and they need values. A `Clock` slot built in occurrence zero gets the World's clock as of the start; `Process.Args` gets the arguments; `Fs.Contents` of a small readable file gets its contents, and of a file the World cannot give (missing, unreadable, over its size cap) gets `Unavailable` with the reason; the World, reading the list

afterwards, sees that someone wants that file and starts watching it.

Here is the file viewer of chapter 2 as one run call, so that the pieces have names. The program fires a `Read` on `/etc/motd` from `@started` and watches `Fs.Contents` of whatever path is in a text input, initially `/etc/hostname`.

```
run(Inputs { responses: [], pushes: [], oracle: |e| snapshot.get(e).unwrap_or_else(|| first_answer(e)) })
→ Outcome {
    list: [
      (1,        Fs.File(path="/etc/motd")),
      (2/#0/0,   Ui.Label(text="(reading...)")),
      (3/0,      Gui.Focusable(name="input")),
      (3/1/#0/0, Ui.Label(text="/etc/hostname|")),
      (4/#0/0,   Fs.Contents(path="/etc/hostname")),
      (5/#0/0,   Ui.Label(text="box")),
    ],
    requests: [(1, [Read])],
    result: unit,
  }
```

The World now knows three things: there is a `Read` to perform on `/etc/motd`, addressed back to path `1` when it completes; someone is watching `/etc/hostname`, which `first_answer` read for the oracle (`Known "box"`, so the label already shows it) and which the World should now keep watching; and there is a screen to draw. It performs the read on a thread, sets the watch, draws, and waits. When the motd read completes, the next call has one response, `(1, Contents "Welcome")`. When someone renames the host, the call after has one push, `(Fs.Contents(path="/etc/hostname"), Known "crate")`, and no responses. The order in which they happen is the order of the calls, and the program's text says what each order means.

## Snapshots, the oracle and pushes

Picture a book builder joining an exchange feed after the open. It needs the book as it stands, and every change from then on, and it needs to know which of those changes the book it fetched already includes. Binance's documentation tells you the procedure (from memory): open the stream and buffer it, fetch a snapshot stamped with a sequence number, drop every buffered update at or below that number, apply the rest, and if an update's first id ever skips past your last, throw the book away and start over. CME's recovery feed says the same thing in different tags (from memory). Kubernetes clients list, note the `resourceVersion`, and watch from it; when the server has forgotten that version they get `410 Gone`, clear the cache, and list again (from memory). Debezium reads the transaction-log position, snapshots the tables, then streams from the position it noted (from memory). ZooKeeper watches are one-shot and the documentation says outright that you cannot reliably see every change; what it does promise is that you see the watch event before you see the data it is about, so a read after the event is current (from memory). inotify's queue can overflow, and the man page's recovery advice is to rebuild the cache from a fresh scan (from memory).

Every one of these is the same design discovered independently: a *snapshot*, a stream of *changes*, and a *version* that says which changes the snapshot already includes. The edge says when; the level says what; the version says they agree. Edges alone lose state, silently: the epoll man page walks through the edge-triggered reader that takes one kilobyte of two and then blocks forever, because the edge for the write was used up and nobody will announce the leftover (from memory). Levels alone lose moments and cost polling: Kubernetes makes a virtue of it ("the system is not required to touch base at 5 before changing to 3", quoted from memory), which is fine for converging on a desired state and useless for anything that needs to know 5 happened. So the robust systems pair the two, and because

the two come from different clocks (a REST call and a websocket, a scan and an inotify queue), they bolt on a sequence number to line them up.

Prism already has the clock those systems bolt on. The occurrence is the version. The World numbers the occurrences, it answers each from one snapshot of its own state, and anything stamped with the same occurrence describes the same state. That is the whole of the discipline, and the rest of this section is working it out.

## The snapshot discipline

The World keeps, for every state-element value it has ever been asked about, the value it last supplied. Call that the snapshot. As occurrence n opens, the World has gathered whatever changed since n−1 (a read completed, a file changed on disk, the clock moved, a socket connected) into a new snapshot $S_n$. It then:

- answers the oracle from the *previous* snapshot $S_{n-1}$, and
- passes the difference between $S_{n-1}$ and $S_n$ as the pushes.

So a slot that existed before n holds $S_{n-1}$'s value as n opens and receives n's push, ending n with $S_n$'s value. A slot *created* in n reads the oracle, getting $S_{n-1}$'s value, and then receives n's push, and ends n with $S_n$'s value. The two kinds of slot end in the same place by different routes, and nothing in the program can tell which route a slot took. That is the rebuild rule of the language applied at the boundary: a widget built mid-occurrence behaves as if built just before the occurrence's inputs, sees all of n, and a `rebuild` built in n over a dynamic that changed in n builds once with the new value.

Why the previous snapshot and not the new one? Suppose the oracle answered from $S_n$. A slot created in n would start at $S_n$'s value, and then the push for n, which also says $S_n$'s value, would arrive and fire its change event with the value it already has. Harmless for the value, but now the slot's dynamic "changed" in n and the one that existed before n also changed in n, and anything derived from either fires in n. Still consistent. The real trouble is a tag read. A widget built in n that reads the new slot's state through a tag in the same occurrence should see the value *before* n, by the tag law, because that is what every other dynamic gives at a tag. $S_{n-1}$ is the value before n. Answering from $S_n$ would make a freshly created state element the one dynamic in the language whose tag reads the after-value. Answering from $S_{n-1}$ keeps the law uniform: every dynamic, including one born this occurrence, has a before-value and a change event, and the before-value of a state element is what the World held as the occurrence opened.

Why pushes by value rather than by path? Because two slots asking for the same state must agree in every occurrence, and the cheapest way to guarantee that is to give them one dynamic. A push keyed by path would reach one slot; the World would have to find every slot with an equal element and push to each, and a slot created in n would need a push the World could not have known to send. Keyed by value, the runtime fans the push out, including to slots born in n, and the World's bookkeeping is one map from value to last-supplied value.

## The oracle cannot fail

A state tag the World accepts is always answerable, because the state's type says what unavailable looks like: `Unavailable FsError | Known Text`, `Connecting | Connected | Disconnected Reason`, `NoStamp | Modified Instant`. A World that cannot give a file's contents says why, in the type (`Unavailable NotFound`, `Unavailable Denied`, `Unavailable TooLarge`), and goes on watching it; a World asked for a socket it has not opened answers `Connecting` and opens it. There is no `Pending`, no

`null`, no "ask again later," because a dynamic without a value is not a dynamic, and the program's `match` on the state type has an arm for the not-yet case or it does not compile.

A state tag the World does *not* accept is an error in the occurrence the slot is created, reported with the element, the span and the interceptor chain it came through, because nothing later can give it a value. This is deliberately harsher than the `Unsupported` response an unknown request tag gets. A request with no answer is a program-visible fact in a later occurrence: the program fired and got `Unsupported` back and can match on it. A state element with no value has nowhere to put the fact; its dynamic would have to hold something in the occurrence it was born, and the World has nothing to offer. The build error is the right answer, and the way for a portable program to avoid it is below, under cross-platform.

## Where the stitch happens

The World does the stitching, because the World is the only party that sees both clocks. A native read and a native watch are separate system calls that interleave: the watch reports a change while the read is in flight, and whether the read saw the change is unknowable from either alone. The World sets the watch first, reads, reconciles anything the watch reported during the read (a reread, or a check of the modification stamp), and only then puts the result in its snapshot. From the program's side there is one value per occurrence and it is right. The Fs component's inner loop is the Binance procedure with the occurrence as `lastUpdateId` (the field name from memory).

What the World does *not* do is pick a policy the program might want to choose. A book has policy in it: top ten or full depth, what to show during a resync, whether a gap means "stale" or "empty." Policy of that kind belongs in Prism, in an interceptor the program chooses, with the gap in the state's type, and the World offers the feed's edges (`Md.Feed`, a stream of sequenced increments) and snapshot (`Md.Snapshot`, a request) rather than one book for everyone. A World that happens to have a native book builder next to the exchange socket can offer `Md.Book` as a state element beside the feed, and a program that wants its own policy ignores it. The line is: the World stitches where the stitch is mechanical and the source's own protocol dictates it; Prism stitches where there is a choice.

```
type BookState = choice:
    Buffering [Delta]          # no snapshot yet; increments wait
    Live Book, Nat               # the book and the last applied sequence number
    Resyncing Book, [Delta]    # a gap; the last good book stays visible

@BookBuilder: fn Symbol, @a -> @a = \|sym, @strategy| @widget:
    !incr <- @Element Md.Feed(symbol=sym)
    !started <- @started
    !snap <- @Element Md.Snapshot(symbol=sym, requests=(unit on (!started | !gap)))
    let !msg = Snapshot !snap | Update !incr
    $book <- Buffering [[]] then StepBook $book, !msg
    let !gap = !Filter (NewGap $book, !msg)
    ((result, $slots)) <- intercept @strategy, !down, $ supply
    !down <- @Forward $slots
    let supply = \|e| match e:
        Md.Book(symbol=s): if s == sym: @ $book else: @Element e
        ignore: @Element e
    @ result
```

`StepBook` is the exchange's procedure as a fold: buffer until the snapshot answers, drop increments at or below its sequence number, apply the rest, go `Resyncing` on a skipped number and ask again. The strategy under it asks for `Md.Book(symbol=sym)` as a state element and gets `$book`, in its creation occurrence, with the policy visible in the type. Under a World with a native book, the same strategy

runs with `@BookBuilder` removed. That is the substitutability test from the other direction: the interceptor supplied state in exactly the form the World would have.

## What a push is, and how many

A push is the World saying "as occurrence n opens, this value is now this." It is not an event in the program's sense: a state element's dynamic has a change event, and a push fires it, but the program cannot distinguish a push from any other change of a dynamic, and two pushes for one value in one occurrence are an error, because a dynamic changes at most once per occurrence.

How many pushes per occurrence, and whether a change on disk gets its own occurrence or rides along with a click, is the World's business. A CLI World pushes `Clock` at most once per turn. A GUI World coalesces three resize reports that arrived between frames into one push of the last size, which promises the program no less than the platform promised the World. A World must never coalesce two responses to one slot, because responses are edges and the program may be counting them; it may coalesce state, because state is a level and the last value is the truth.

Every push must satisfy the equality law: in one occurrence, every slot with equal elements sees one value. The runtime guarantees it structurally, by giving equal elements one dynamic; the World guarantees it by answering the oracle and computing the pushes from one snapshot. A World that answered the oracle by calling `stat` would break it, because two calls could straddle a write; that is why the oracle answers from the snapshot map, and why a value not yet in the map is filled once, by the component, and recorded before the answer is given twice.

## The directory listing, stitched

Here is the Fs component's inner loop for `Fs.Dir`, occurrence by occurrence, with the race in it. A program builds `@Element Fs.Dir(path="/home/me")` in occurrence 7.

*Occurrence 7.* The runtime asks the oracle for `Fs.Dir(path="/home/me")`. The World has never been asked; `initial` returns `Unavailable`; the slot's dynamic starts there. The program draws "(unavailable)". After the turn, the component sees a new `Fs.Dir` slot in the list and starts: it registers a watch on `/home/me` *first*, then starts a scan on a thread.

*Between turns.* The scan is running. The watch reports `created b.txt`. The component holds that report, because the scan is not done and it cannot know whether the scan saw `b.txt`.

*The scan completes* with `[[a.txt]]`. The component now reconciles: the watch said `b.txt` was created during the scan, and the scan did not list it, so the component rescans (or, where the platform gives a stable modification stamp, compares stamps and patches). The rescan gives `[[a.txt, b.txt]]`. Only now does the component call `changed(Fs.Dir("/home/me"), Known [[a.txt, b.txt]])`, and the core puts it in `incoming`.

*Occurrence 8*, opened by whatever arrives next (a click, say). The core computes the pushes as `incoming`: one push, `Known [[a.txt, b.txt]]`. The oracle still answers `Unavailable` for the value, which is $S_7$. The push fires the slot's change event in 8. A second widget that happens to be built in 8 asking for the same directory gets `Unavailable` from the oracle and the push in the same occurrence, and ends 8 holding the same listing as the first. The program draws both.

*Between turns.* The watch reports `deleted a.txt`, then `created c.txt`. The component applies both to its copy and reports `changed(.., Known [[b.txt, c.txt]])` twice; the core keeps the last. One push in occurrence 9, not two: the intermediate listing `[[b.txt]]` is a state the program never sees, which is what level semantics means and what Kubernetes calls not touching base at 5 (from memory).

*The watch overflows.* inotify says events were lost. The component rescans from scratch and reports the result. The program sees an ordinary change. Nothing about overflow crosses the boundary, because the program asked for the listing as a level and the World's job is to keep the level true.

Every occurrence in that sequence answered everything about `/home/me` from one snapshot, and the version that lined up the scan with the watch events was the component's own bookkeeping, which is the part Binance makes every client write. The program wrote none of it.

### Why not events only, and why not dynamics across

Two alternatives deserve a hearing, because each is what you would build first.

*Events only.* The World sends a one-shot answer and a stream of changes, both as responses, and a library widget holds them into a dynamic: drop changes until the answer arrives, let an answer win over a change in the same occurrence, apply changes afterwards. Everything is edges; the boundary is minimal. The cost is that the dynamic has no value in the occurrence its slot is created. A widget built in n sees all of n, including a lexically captured dynamic that changed in n; if its own state arrives in n+1, there is an occurrence in which it has a new value of one thing and a stale value of another, and nothing can hide that from it. Every wrapper also has to apply the stitching rules identically or two readers of one file disagree, which is the equality law reduced to a convention. State elements with a value in their first occurrence are what remove the hiccup, and the oracle is the one mechanism that can supply such a value.

*Dynamics across.* The World hands the program a dynamic. A dynamic is not plain data; it is a runtime object with a lifetime, and a World holding one is a World coupled to the runtime. Written out as plain data, "a dynamic from the World" is an initial value plus a sequence of changes keyed to occurrences, which is a snapshot plus pushes, which is this design. The oracle is how the initial value gets in without being a firing; the pushes are the changes; the occurrence is the key.

An interceptor, which shares the program's clock, can hand a child a dynamic it already holds, and the supplier mechanism is exactly that: a widget returning the state's dynamic, built in the interceptor's widget. The World cannot, so its supplier is a function over data, and the two look identical from below.

## Turns and `Defer`

A *turn* is the World's unit: the occurrences it runs before it acts on the result. The World acts on the state at the end of the turn, so it reads the list and performs the request firings after the last occurrence of the turn, not after each one. Most turns are one occurrence: an input arrives, the World opens an occurrence with it, reads the outcome, acts. The exception is `Defer`, an element every World must provide and the only way a turn is ever longer than one occurrence.

```
tag Defer of Element:
    requests: !unit
    responds: unit
```

A `Defer` slot that fires its request in an occurrence is answered in a later occurrence *of the same turn.* After each occurrence the world core looks at the request firings; if any `Defer` fired, it runs another occurrence delivering `unit` to each of them, reads the firings again, and repeats until no `Defer` fired. Then the turn is over.

## Why it exists

An interceptor is Prism and cannot start an occurrence. Yet it must sometimes answer a request, and a response is always in a later occurrence than its request; that is the creation rule, and an interceptor is bound by it exactly as a World is. A sandbox denying a file read cannot deliver `Denied` in the occurrence of the `Read`, both because the rule forbids it and because the requesting slot may have been born in that occurrence and is not yet routable. So the sandbox fires a `Defer` and, in the occurrence that answers it, fires `Denied` into its child. From the World's view none of this happened: the turn was one input and one list, and the only thing it did between was answer a `Defer`.

```
Nil:
    !later <- @Element Defer(requests=(unit on _.!request))
    @ Denied on !later
```

That arm from the sandbox of chapter 2 is the whole pattern. The child asked in occurrence k; the `Defer` fired in k; the world core opened k+1 with `unit` to the `Defer`'s path; `!later` fired; `Denied on !later` fired; the `for` consolidated it under the child's path; `intercept` routed it against the child's list as k+1 opened, in which the child's slot now exists. The child sees a denial one occurrence after it asked, which is also the soonest a real World could have answered.

Everything that needs "a moment later, before the World acts" is this: a cache answering from what it holds, a mock, a focus manager that wants to move focus to a field built this occurrence (the field is in the list next occurrence, so the move goes through a `Defer`), a layout that feeds a measured size back to a child. None of them is a feature of the language. Each is an ordinary element with the ordinary rule that a response is later than its request.

## Bounded without a cap

Each `Defer` slot is answered at most once per turn. A slot that fires again after being answered in the same turn is an error, and the error has a name: *deferring backwards*. Its request was answered; it is asking to be answered again in a turn that has already given it its moment.

That rule bounds a turn without anyone picking a number. A turn runs at most as many occurrences as there are `Defer` slots that fire, plus one for the input. There is no budget, no timeout, no runaway report to design, and no configuration knob whose value is wrong for somebody. The objection is that a budget would let a deep but finite cascade through one `Defer` finish, where this rule stops it on the first repeat. That cascade is iteration to a fixpoint, and it belongs in a pure function (where it is a loop), or in the next turn (where the platform gets to run in between). A loop through one `Defer` that does not stop is a bug, and finding it on the first turn beats freezing the application's output while the loop spins.

The classic version of the bug is caught early: a child that retries immediately on `Denied` makes its sandbox's `Defer` fire again in the same turn, which is deferring backwards, which the world core reports with the `Defer`'s path and the interceptor chain. A retry belongs behind `Time.After`, where the World's clock separates the attempts and each is its own turn.

A `Defer` slot created in occurrence k of a turn, firing in k (from `@started`, say, or because the request it stands in for fired in k), is answered in k+1 of the same turn. The sandbox needs this: the child's slot and the sandbox's `Defer` for it are both born in k. The bound is then the number of `Defer` slots that exist at any point during the turn, which a program can inflate only by building a fresh `Defer` in every occurrence, with a `rebuild` that rebuilds on the previous one's answer and fires the new one from `@started`. That is a loop the program wrote, through a rebuild instead of through one slot, and the world core treats it as deferring backwards too: a `Defer` at a path that differs from an

already-answered `Defer`'s path only in its `rebuild` generation is the same `Defer` reborn. The rule is stated this way because it makes the bound structural rather than counted; if a program turns up that needs a `Defer` per generation for a finite, legitimate cascade, the rule would loosen to counting generations, and the test would be that program.

## Deferring backwards, precisely

Two things are errors, both reported with the path and the interceptor chain: a `Defer` slot firing after it was answered in this turn, and a `Defer` slot whose path matches an answered one up to `rebuild` generation, as above. One thing that looks like a third is not: a `Defer` whose slot was torn down in the occurrence it fired. There is nothing to answer, so the world core answers nothing, and the turn ends if that was the last fired `Defer`.

## Why no phases

Suppose the World ran named stages after the input occurrence: layout, then focus, then frame, each at most once per turn, with an element `Phase(name)` that defers an event until its stage. A GUI library would declare which it uses; size feedback would go to the layout stage, autofocus to the focus stage, animation to the frame stage before paint. It is a real design and it solves real problems, and it is not in the language.

What it costs. There is now a second notion of "later" with an ordering table: which stage runs before which, what happens when a stage defers into an earlier stage, whether an empty stage costs an occurrence, what a stage means under a test World, whether an interceptor may add stages. Each World picks its stages, so a library written against one World's stages runs differently under another. And every stage is expressible without it: a layout feedback is a `Defer` whose answer the layout reads in the next occurrence; ordering between two deferred things is a dynamic one of them reads through a tag; a frame tick is a `Gui.Frames` stream element the World fires when it is about to paint. The language has one "later," which is "the next occurrence of this turn," and nothing finer. Priorities, stages and scheduling belong to the implementation if they belong anywhere, and the implementation does not need them either.

## What a turn looks like from each side

From the World's side: one input (or a batch it chose to deliver together), one call to the world core's `turn`, one outcome. It performs the request firings of every occurrence of the turn in order, reads the final list, reconciles, and waits for the next input.

From the program's side: an occurrence with inputs from outside, possibly followed by occurrences with inputs only from `Defer` slots, each a full occurrence under the ordinary laws. The program cannot tell where a turn ends, and must not care; a program that behaves differently depending on whether two inputs were one turn or two is depending on the World's batching, which the language leaves to the World. What the program *can* rely on is that a `Defer` fired in occurrence k is answered before any new outside input arrives, which is what makes a sandbox's denial land before the user's next keystroke.

From a test's side: `send` runs a whole turn, including the `Defer` occurrences, and `expect` sees the end of it. A script that expects `"(reading...)"` after a sandbox denial would fail: by the end of turn zero the denial has landed.

## A turn with a cache, occurrence by occurrence

An HTTP cache is the interceptor that shows every part of a turn, because it answers some requests itself and forwards the rest, and the two take different numbers of occurrences.

```
@HttpCache: fn @a -> @a = \|@child| @widget:
    ((result, $slots)) <- intercept @child, !down, $ \|e| @Element e
    !down <- for $slots:
        rebuild $_:
            match _.element:
                Http.Fetch(..):
                    let !get = !Filter (CacheKey _.!request)            # Some url for a GET, else Nil
                    let !hit = !Filter (Lookup $cache, !get)             # Some answer if cached
                    let !miss = !Filter (NotCached $cache, _.!request)
                    $hit <- Nil then Some !hit
                    !later <- @Element Defer(requests=(unit on !hit))
                    !fetched <- @Element Http.Fetch(requests=!miss)
                    @ !Filter ($hit on !later) | !fetched
                ignore:
                    ((!resp, ignore)) <- @Element _.element
                    @ !resp
    let !answers = !Filter (Cacheable !down)        # ((url, answer)) for GETs that passed through; an answer
    names its request
    $cache <- {} then InsertPair $cache, !answers
    @ result
```

For each child `Http.Fetch` slot, a request that the cache can answer fires a `Defer`; one it cannot is forwarded on the cache's own `Http.Fetch` element, and the response comes back to this block and is consolidated under the child's path. `$cache` is read through a tag, so a request sees the cache as the occurrence opened.

Take a child that fires `GET /a` in one turn and, later, `GET /a` again, under a World that answers `/a` in one turn.

**Turn 1, occurrence 1.** Outside input: a click, delivered to a button in the child. The child's widget fires `GET /a` on its `Http.Fetch` slot. The cache's block for that slot sees `_.!request` fire; `$cache` is `{}` as the occurrence opened, so `!hit` is quiet and `!miss` fires; the cache's own `Http.Fetch` slot fires its request. No `Defer` fired. `run` returns with `requests: [(cache/.., [GET /a])]`. The world core sees no `Defer` in the firings; the turn is one occurrence. The World performs the fetch.

**Turn 2, occurrence 1.** The fetch completes. The World delivers `Answered ..` to the cache's `Http.Fetch` slot at its path. `!fetched` fires in the block; the block's result fires; `!down` fires with `{child_path: Answered ..}`; `intercept` routes it to the child's slot, which existed as the occurrence opened; the child's label updates. `!answers` sees a cacheable GET pass through and `$cache` gains `/a`. One occurrence.

**Turn 3, occurrence 1.** Another click. The child fires `GET /a` again. The cache's block reads `$cache` through a tag: it has `/a`. `!hit` fires with the answer; the `Defer` slot's request fires. `!miss` is quiet. `run` returns with `requests: [(cache/../defer, [unit])]`.

**Turn 3, occurrence 2.** The world core sees the fired `Defer` in the list it just received, records its path as answered, and calls `run` again with `responses: [(defer_path, unit)]`, no pushes, the same oracle. `!later` fires; `$hit on !later` reads the hold through a tag and gets the answer committed in occurrence 1; `!Filter` passes it; `!down` fires; the child's slot gets its answer. No `Defer` fired in this occurrence. The turn is over: two occurrences, one input. The World performed no fetch, and the transcript shows `requested Http.Fetch(..)` for turn 1 and nothing for turn 3, which is what

`cache_hit_no_request` asserts.

What the child saw in turn 3 is a response one occurrence after its request, which is the earliest any World could have answered, so a child tested under the cache sees the same timing as one tested without it. A mock built the same way has the same property, and that is why mocks go through `Defer` rather than answering "immediately": there is no immediately, and a mock that pretended there was would hide timing bugs production would show.

## Obligations of a World, and who each protects

A conforming World keeps the list below. Every item is stated with the party it protects, because an obligation nobody would miss is a rule for its own sake, and there are none of those here.

**Send responses only to paths in the list it last received.** *Protects the program.* A slot that is gone has no widget to receive, and a response to it is a response to nothing; the runtime reports it as `UnknownPath`. The World is the only party that knows an operation is still in flight after its slot was torn down, so the World discards the late result and cancels the operation if the transport allows. The program never has to defend against an answer to a question it stopped asking.

**Never deliver a response in the occurrence of its request.** *Protects the program's reasoning and the creation rule.* Request firings come out of `run` after the occurrence commits; the earliest they can be answered is the next call. A widget may therefore assume that in the occurrence it asked, nothing it asked for has happened, which is what lets it fire from `@started` without a guard. The runtime enforces this by construction; the obligation is that a World not offer an element that "responds merely because it was created," which it could do by sending a response in the first occurrence after a slot appears with no request having fired. A World may do that only for a tag that declares `responds` without `requests`, a stream, where the response is an observation and not an answer.

**Answer the oracle synchronously and consistently with the pushes.** *Protects the equality law.* One snapshot per occurrence, answered the same to everyone, with the pushes as the diff from the previous one. A World that answers the oracle from a live source instead of its snapshot can give two slots two values in one occurrence and has broken the program, not itself.

**Provide `Defer` with the turn semantics above.** *Protects interceptors.* Every interceptor that answers anything needs it, and a World without it cannot host a sandbox, a cache or a mock.

**Treat the element list as the effect stream, and perform request firings in order.** *Protects determinism, and so the transcript.* The list after the turn is what the World reconciles to; the firings are performed in occurrence order then list order. Two Worlds reading the same outcome do the same things in the same order, which is what makes a golden transcript of a CLI run a test.

**Answer a request on a tag it does not accept with `Unsupported` in the next turn.** *Protects portable programs.* A `Ui.Button` under a CLI World, a `Clipboard` under a World without one: the program fired, and it gets a program-visible fact back rather than silence. A presence element it does not accept (one with neither `responds` nor `state`) is an error it reports and otherwise ignores, since there is nothing to answer.

**Report a state tag it does not accept as an error in the creation occurrence.** *Protects the type.* A dynamic that cannot have a value cannot exist, and the error says so where the slot was made.

**At most one response per path and one push per value per occurrence.** *Protects the*

*runtime's one-firing-per-occurrence law.* A World with two chunks for one slot has two occurrences.

**Discard the responses of a turn that errored, keep its pushes, and report.** *Protects the transaction.* An occurrence that returned `Err` did not happen; the responses that opened it are gone, since the program's slots never received them; the pushes are state the world holds and the program has not seen, so they go again with the next turn, as the diff from whatever the program last held.

What a World is free to do: batch inputs that arrived together into one occurrence or not, coalesce state (never responses), run on whatever thread the platform dictates, multiplex sockets and timers however it likes, keep one OS socket per address or per slot, key native widgets by tree position for continuity across rebuilds. The language says nothing about any of it, and a program that could tell the difference is depending on something the language did not promise.

## The world core

Everything in the obligations that is not about a particular tag is written once, in the host language, as a library every World in that language uses. A World is the core plus its components. This is not a language rule; it is what makes a World small enough that there will be several, and it is also where the obligations are tested, once, instead of per World.

### The turn loop

```
pub struct Core {
    runtime: Runtime,
    snapshot: BTreeMap<ElementValue, Value>,   // S_{n-1}: what the program last saw
    incoming: BTreeMap<ElementValue, Value>,   // changes components reported since
    last_list: BTreeMap<Path, ElementValue>,   // the list the World last received
    components: Vec<Box<dyn Component>>,
    turn_no: u64,
}

pub struct TurnOutcome {
    pub list: Vec<(Path, ElementValue)>,
    pub requests: Vec<(Path, Vec<Value>)>,     // every occurrence of the turn, in order
    pub occurrences: u32,
    pub error: Option<RunError>,
}

impl Core {
    pub fn turn(&mut self, responses: Vec<(Path, Value)>) -> TurnOutcome {
        // 1. Check the World's own addressing before the runtime does.
        let responses: Vec<_> = responses.into_iter()
            .filter(|(p, _)| self.last_list.contains_key(p))   // obligation 1
            .collect();

        // 2. Build this turn's snapshot and pushes.
        let previous = self.snapshot.clone_shallow();            // S_{n-1}, for the oracle
        let pushes: Vec<_> = self.incoming.drain().collect();   // S_n - S_{n-1}
        self.snapshot.extend(pushes.iter().cloned());            // S_n
        let oracle = |e: &ElementValue| previous.get(e).cloned()
            .unwrap_or_else(|| self.unavailable_for(e));          // never fails for accepted tags

        let mut all_requests = Vec::new();
        let mut inputs = Inputs { responses, pushes: pushes.clone(), oracle: &oracle };
        let mut answered: BTreeSet<Path> = BTreeSet::new();
```

```
        let mut occurrences = 0;

        loop {
            occurrences += 1;
            let outcome = match self.runtime.run(inputs) {
                Ok(o) => o,
                Err(e) => {
                    // The program saw none of this turn: put the pushes back so the
                    // next turn's diff is from what the program last held.
                    self.snapshot = previous;
                    for (ev, v) in pushes { self.incoming.entry(ev).or_insert(v); }
                    return self.failed_turn(e, occurrences);
                }
            };
            all_requests.extend(outcome.requests.iter().cloned());
            self.last_list = outcome.list.iter().cloned().collect();

            // 3. Defer: collect fired Defer slots that are still in the list.
            let defers: Vec<Path> = outcome.requests.iter()
                .filter(|(p, _)| self.last_list.get(p).map_or(false, |e| e.tag == DEFER))
                .map(|(p, _)| p.clone())
                .collect();
            if defers.is_empty() {
                return TurnOutcome { list: outcome.list, requests: all_requests, occurrences, error: None };
            }
            for p in &defers {
                if answered.contains(p) || answered.iter().any(|q| same_up_to_generation(p, q)) {
                    self.snapshot = previous;
                    for (ev, v) in pushes { self.incoming.entry(ev).or_insert(v); }
                    return self.failed_turn(RunError::DeferBackwards(p.clone()), occurrences);
                }
                answered.insert(p.clone());
            }
            // 4. The next occurrence of the same turn: Defer answers, no pushes, same oracle.
            inputs = Inputs {
                responses: defers.into_iter().map(|p| (p, Value::unit())).collect(),
                pushes: Vec::new(),
                oracle: &oracle,
            };
        }
    }
}
```

Four things to read off it. The oracle is the same closure for every occurrence of the turn, over the same `previous`, and the pushes go in only with the first occurrence: the snapshot moves once per turn, as the turn opens, so every occurrence of a turn sees one world. The pushes are the diff the components reported since the last turn; the core does not compute diffs, it collects what components told it changed. Deferring backwards is detected here, by path, before the runtime sees a second answer. And the list the World keeps for addressing is replaced after every occurrence, not every turn, because a slot torn down in a `Defer` occurrence must be gone before the next outside input is addressed.

A `Defer` occurrence could take fresh pushes, letting a file change land in the middle of a turn. Nothing would break, but the sandbox that answered in occurrence k+1 would then be answering against a world that moved under it between k and k+1, and the program could observe the move by comparing a state read in k with one in k+1 of the same turn. Holding the snapshot still for the turn makes the turn what it claims to be: one step, as the outside sees it.

## Bookkeeping the core does for every World

*The last list.* Kept for addressing (responses only to known paths) and for diffing: after a turn, the core tells each component which of its slots are new and which are gone, so a component can open what is newly described and cancel what is no longer. The diff is the component's input, not a trigger in the program's sense: a component that opens a socket for a `Net.Socket` slot is reconciling the world to the description, and will reconcile the same list to the same world without opening a second socket.

*Late results.* A component that finishes an operation for a slot that is gone hands the core a result the core cannot address. The core drops it and records it in diagnostics, with the path and the tag, so that a World developer can see that a response was discarded rather than wondering why it never arrived. Cancellation is the component's job because only the component holds the handle.

*Snapshot and pushes.* Components report `changed(value, new)` into `incoming`; the core commits `incoming` into `snapshot` as the next turn opens and hands it to the runtime as pushes. A component that reports the same value twice before a turn opens has reported one push, the last, which is coalescing done right: state is a level.

***`Unsupported`.*** A request firing on a tag no component claims gets a response `Unsupported` in the next turn, queued by the core as an ordinary response. A presence tag no component claims is logged once per path. A state tag no component claims makes `unavailable_for` return an error marker, and the runtime reports `UnsuppliedState` in the creation occurrence.

*Errors.* `failed_turn` discards the turn's responses, puts the turn's pushes back into `incoming` so that the next turn's diff is from what the program last held (the world moved, and the program has not seen it), and returns the error with the occurrence number of the turn in which it happened. How the error is printed is one format, shared, so that every World in one host language prints a cycle the same way.

## The component trait

```
pub trait Component {
    /// The tags this component claims. Tags are claimed by exactly one component.
    fn tags(&self) -> &[TagId];

    /// The World's snapshot, for a value no component has reported yet:
    /// the tag's unavailable case, never a syscall.
    fn initial(&self, e: &ElementValue) -> Value;

    /// After a turn: slots of this component's tags that appeared and vanished.
    /// Reconcile: open, close, cancel. Never answer here.
    fn reconcile(&mut self, appeared: &[(Path, ElementValue)], vanished: &[(Path, ElementValue)], ctx: &mut
    Ctx);

    /// After a turn: request firings on this component's slots, in order.
    fn perform(&mut self, path: &Path, e: &ElementValue, fired: &[Value], ctx: &mut Ctx);
}

/// What a component may say to the core. Everything here is queued for a
/// later turn; nothing reaches the runtime during a turn.
pub struct Ctx {
    pub fn respond(&mut self, path: Path, value: Value);          // next turn, if the path still exists
    pub fn changed(&mut self, e: ElementValue, value: Value);     // next turn's push
    pub fn wake(&self) -> Waker;                                   // for threads: "there is input"
}
```

A component never sees the runtime. It sees paths and element values, which are data; it reports responses and changes, which are data; and it is called by the core at two moments, after a turn to reconcile and perform. Its own threads, futures and file descriptors are its business, and the only way they reach the program is by putting a response or a change into the core's queue and waking the loop.

Why a component is *called* rather than subscribed: there are no callbacks across the boundary, and that rule holds inside the World too, between core and component. The core could instead hand each component a stream of list changes and have the component react; that is the same information with the component in charge of timing, and the core exists to own timing. A component that performs between occurrences of a turn, say, would make the turn observable from outside, and the core calling `perform` once after the turn is what prevents it.

### What the core does not do

It does not diff GUI trees, lay anything out, or know what a window is. It does not retry anything, because a retry is a policy and policies are Prism. It does not run occurrences on its own initiative: a World with no input has no turn, and a program under it is a data structure until something arrives. And it does not hold a reference to anything in the runtime but the runtime itself, which it owns.

## General components

A World's components are the tags it accepts. The ones below are general: a CLI World, a GUI World and a server World all accept them, each with its own native implementation, and a program written against them runs under any of the three. Each gets its tag declaration and an example of use. The language fixes the kinds: which parts a tag declares and what each part means. A World may spell its own tags differently.

Three forms recur. A *request* element declares `requests` and `responds`: the program fires, the World answers later, one answer per firing. A *stream* element declares `responds` alone: observations arrive on their own, each in an occurrence after the slot's creation, and the program never asks. A *state* element declares `state`: a value in the creation occurrence from the oracle, pushed afterwards. A tag may declare any combination, and `Net.Socket` declares all three.

### Fs

```
type FsRequest = choice:
    Read
    Write Text
    Append Text
    Delete

type FsError = choice:
    NotFound
    Denied
    TooLarge
    Other Text

type FsResponse = choice:
    Contents Text
```

```
    Written
    Unavailable FsError

tag Fs.File of Element:
    path: Text
    requests: !FsRequest
    responds: FsResponse

type Contents = choice:
    Unavailable FsError          # the reason the contents cannot be had
    Known Text

tag Fs.Contents of Element:
    path: Text
    state: Contents

type Entry = record:
    name: Text
    kind: EntryKind

type EntryKind = choice:
    File
    Dir
    Link
    Other

type Listing = choice:
    Unavailable
    Known [Entry]

tag Fs.Dir of Element:
    path: Text
    state: Listing

type Stamp = choice:
    NoStamp
    Modified Instant

tag Fs.Watch of Element:
    path: Text
    state: Stamp
```

`Fs.File` is the one-shot conversation: fire a `Read`, get `Contents`, `Written` or `Unavailable` with an `FsError` reason back, addressed to this slot, in a later occurrence. The path is a field, fixed for the slot's life; a program with a dynamic path has a `rebuild`, a fresh slot per path, and the World drops any response to the old slot because the slot is gone. One slot can read now and write later, and a program that wants two reads in flight at once has one slot and two firings, or two slots.

```
@SaveButton: fn Text, $Text -> @$Text = \|path, $draft| @widget:
    !save <- @Button $"Save"
    !r <- @Element Fs.File(path=path, requests=(Write $draft on !save))
    $status <- "" then match !r:
        Written: "saved"
        Unavailable Denied: "permission denied"
        ignore: "failed"
    @ $status
```

One click, one write, and `$draft on !save` reads the draft as it stood before the click, which is the

draft the user was looking at.

`Fs.Contents` is the file as a value that is simply true: the World answers the oracle from its snapshot, filling an entry it has never held by reading the file once under a size cap, and then watches and pushes each time the file changes. A file that cannot be had is `Unavailable` with one of four reasons from a closed choice: `NotFound`, `Denied`, `TooLarge` (past the cap, where a program that wants the file asks `Fs.File`), or `Other` with the operating system's text. The reason is part of the level, so a file that appears is a push from `Unavailable NotFound` to `Known`, and one that vanishes is the push back. Two widgets watching one path share one dynamic. The World stops watching when the last slot for that path is gone.

`Fs.Dir` is the same for a directory listing. A directory is where the stitch earns its keep: the World sets the watch, scans, reconciles what the watch reported during the scan, and only then puts the listing in its snapshot; on an inotify overflow it rescans and the program sees one ordinary change. A program sees `Known [[..]]` and never a partial listing.

`Fs.Watch` is a watch *as state*: its value is the file's modification stamp, pushed when the file changes. A program that wants to do something on change derives it from the dynamic's change, and the idiom is a `rebuild` whose block fires from `@started`:

```
@ReloadOnChange: fn Text -> @!FsResponse = \|path| @widget:
    $stamp <- @Element Fs.Watch(path=path)
    !!reads <- rebuild $stamp:
        !started <- @started
        !r <- @Element Fs.File(path=path, requests=(Read on !started))
        @ !r
    @ !Switch !!reads
```

Every change of the stamp rebuilds the block, whose `@started` fires a fresh `Read`. That is a watch delivered as a level and consumed as edges, by the program's choice, and it is why the watch is not a stream. A stream element `Fs.Changed(path)` responding with `unit` would be the same information as an event, with one difference: a slot created in the same occurrence as a change would miss it, because a response cannot reach a slot born this occurrence, while a state slot created in the push's occurrence sees the push. Watches as state lose nothing at the edges; watches as streams can.

The negative space in Fs: no `Fs.Exists`, because `Fs.Contents` answers it with `Unavailable NotFound` against `Unavailable Denied` or `Known`, and the reason is in the type, so a program that wants to tell absent from unreadable matches on it. No streaming reads; a World that wants to offer them offers `Fs.Stream(path)` responding with chunks, like standard input, and the first version does not.

## Time and Clock

```
type Instant = Instant Int        # milliseconds since the World's epoch
type Duration = Duration Int      # milliseconds
type Tick = tick

tag Clock of Element:
    state: Instant

tag Time.After of Element:
    requests: !Nat                # a delay in milliseconds
    responds: Nat                 # the same number, echoed back

tag Time.Every of Element:
    period: Duration
```

```
    responds: Tick

type Frame = record:
    now: Instant

tag Gui.Frames of Element:
    responds: Frame                # a GUI World fires this before it paints
```

There is no `now()`. `Clock` is a state element: a value in the creation occurrence, pushed when the World chooses, at most once per turn for the Worlds in this chapter. `Clock` has no fields, so every clock in a program is one clock and every reading in one occurrence agrees. A test sets it with `with Clock = Instant 5000` and moves it with `push`, which is why a debounce is testable here and nowhere else.

`Time.After` is a request: fire a delay in milliseconds, get the same number back after at least that long. Each firing is its own timer, and a slot with two firings in flight gets two answers. A debounce is this element and a counter:

```
let IsOne: fn Int -> ?unit = \|n| if n == 1: Some unit else: Nil

@Debounced: fn Nat, !a -> @!a = \|d, !in| @widget:
    $last <- Nil then Some !in
    !done <- @Element Time.After(requests=(d on !in))
    let !delta = +1 on !in | -1 on !done
    $pending <- 0 then $pending + !delta
    let !settled = !Filter (IsOne $pending on !done)
    @ !Filter ($last on !settled)
```

Every input starts a timer and bumps a pending count; every expiry decrements it; the output fires with the last input when an expiry finds the count at one, meaning the timer that just expired was the only one alive, so no input arrived during its life. Under a test World the timer is `send Time.After(..), d`, and the program's text is the whole specification of what "quiet for d" means, including the case the merge decides: an input and an expiry in one occurrence count as an input, and the lost decrement makes the next expiry too early by one. A program that minds sums the two with a `!Union` over a map instead of a merge, and the point is that it had to say so.

`Time.Every` is a stream: a tick per period while the slot exists. Its period is a field, so a dynamic period is a `rebuild` and a fresh slot, and the phase restarts; a World that wants phase continuity across the rebuild keys its timer on tree position and tag, and no program can tell. Ticks are responses, so they never coalesce: a World that fell behind skips the missed ticks and delivers the next one; it never delivers two in one occurrence.

`Gui.Frames` is a GUI World's "about to paint" stream, for animation. A program animating a transition puts the slot inside a `rebuild` over whether it is animating, so that when nothing wants frames the World stops asking for them.

The stopwatch of chapter 2 is the worked example: `Clock` for the time, `Time.Every` to make the World come back, and a hold that stamps `Start` with the clock as it stood before the click, by the tag law.

## Process

```
type Command = record:
    program: Text
    args: [Text]
```

```
    env: Text ==> Text
    cwd: ?Text

type SpawnResponse = choice:
    Spawned ProcId
    NotFound
    Denied

tag Process.Spawn of Element:
    requests: !Command
    responds: SpawnResponse

type ProcInput = choice:
    Stdin Text
    CloseStdin
    Signal Sig

type ProcOutput = choice:
    Stdout Text
    Stderr Text
    Exited Code

type ProcStatus = choice:
    Running
    Exited Code
    Gone                              # no such child; the id is stale

tag Process.Child of Element:
    id: ProcId
    requests: !ProcInput
    responds: ProcOutput
    state: ProcStatus
```

Spawning is a request, because starting a process is a one-shot effect with a result (`Spawned id` or `NotFound`), and because the alternative, a `Process.Child(command=..)` whose presence in the list means "a child like this should be running," runs into identity. State is identified by value: two slots with equal elements share one state, so two `Process.Child(command=[["ls"]])` slots would share one `ProcStatus`, while their responses, identified by path, would belong to two conversations. One process or two? The element's fields cannot say. With an id the World allocated, two children are two ids, two elements, two states, and the program keys a `for` on the ids it was given, exactly as a server keys connections on the ids its listener handed out.

```
@RunOnce: fn Command -> @$ProcStatus = \|cmd| @widget:
    !started <- @started
    !sp <- @Element Process.Spawn(requests=(cmd on !started))
    $id <- Nil then match !sp:
        Spawned i: Some i
        ignore: Nil
    !$status <- rebuild $id:
        match _:
            Some i:
                ((!out, $st)) <- @Element Process.Child(id=i, requests=!never)
                @ $st
            Nil:
                @ $ Gone
    $$status <- $ Gone then !$status
    @ $Join $$status
```

The child's output arrives as responses, each chunk in its own occurrence; its exit is both a response (`Exited`, an edge the program can count) and a state (`Exited`, a level a late-built widget can read). Standard input to the child is a request firing. When the last `Process.Child(id)` slot for an id is gone, the World kills the child: that is its cancel-on-teardown obligation, the same one that aborts an in-flight read. A program that wants a graceful stop fires `Signal Term` first and keeps the slot until `Exited`.

Arguments and environment are state elements, and they belong to the process, so a GUI World offers them as readily as a CLI World does:

```
tag Process.Args of Element:
    state: [Text]

tag Process.Env of Element:
    state: Text ==> Text
```

They are state rather than parameters of `@main` for the sake of the sandbox: an interceptor can supply a child a different `Process.Args`, a test can `with Process.Args = [["notes.txt"]]`, and neither needs a second way to start a program. A World that cannot observe a change to the environment never pushes one; a World that can (a shell that lets the user set a variable) pushes it, and the program's `rebuild $args` rebuilds, which is what you want.

## Net

```
type Link = choice:
    Connecting
    Connected Nat                  # bytes queued and not yet written: backpressure
    Disconnected Text

type SocketEvent = choice:
    Received Bytes
    Closed

tag Net.Socket of Element:
    address: Text                  # "tcp://host:port", "tls://host:port", "unix:/path"
    requests: !Bytes
    responds: SocketEvent
    state: Link

type Accepted = Accepted ConnId

tag Net.Listener of Element:
    address: Text
    responds: Accepted

tag Net.Conn of Element:
    id: ConnId
    requests: ![ConnRequest]       # Send Bytes | Close
    responds: SocketEvent
    state: Link

type HttpRequest = record:
    method: Text
    url: Text
    headers: Text ==> Text
    body: Bytes
```

```
type HttpResult = choice:
    Answered HttpAnswer            # status, headers, body
    Failed Text

tag Http.Fetch of Element:
    requests: !HttpRequest
    responds: HttpResult

type Incoming = record:
    id: ReqId
    request: HttpRequest

tag Http.Server of Element:
    address: Text
    responds: Incoming

tag Http.Reply of Element:
    id: ReqId
    requests: !HttpAnswer
    responds: Delivered            # Delivered | Gone
```

`Net.Socket` is the three-part element. The address is a field; a reconnect is a new address and a fresh slot, and the chat client of chapter 2 reconnects by typing. Its request is the bytes to send; its responses are what arrives and the close; its state is the link, shared by every slot with the same address, because there is one connection, while sends and receives are per slot, because each is its own conversation on it. Whether a World keeps one OS socket per address or per slot is invisible if it keeps the equality law.

Backpressure is state. `Connected n` says how many bytes the World has accepted and not yet written, and a program that cares reads it through a tag at the moment it would send:

```
    let !out = !Filter (WhenRoom $link, $draft on !send)

let WhenRoom: fn Link, Text -> ?Text = \|link, msg| match link:
    Connected queued: if queued < 65536: Some msg else: Nil
    ignore: Nil
```

A program that ignores it still works; the World buffers. A program that respects it never has a buffer grow without bound, and it did so by reading a dynamic, not by waiting on a writable callback.

`Net.Listener` is a stream of accepted connections, each with a World-allocated id, and `Net.Conn(id)` is the connection, keyed by that id in a `for`, as in chapter 2's server. `Http.Fetch` is the client: fire a request, get a result. `Http.Server` is a stream of incoming requests with ids, and `Http.Reply(id)` is how a reply goes back: a request element keyed by the request's id, fired once with the answer. A proxy that answers each incoming request by making an outgoing one is a `for` over the pending ids:

```
type Change = choice:
    Open HttpRequest
    Close

let OneOpen: fn Incoming -> ReqId ==> Change = \|i| {i.id: Open i.request}

let Apply: fn ReqId ==> HttpRequest, ReqId ==> Change -> ReqId ==> HttpRequest = \|m, cs|
    Fold cs, m, \|acc, id, c| match c:
        Open r: Insert acc, id, r
        Close: Remove acc, id
```

```
@Proxy: fn Text, Text -> @unit = \|listen, upstream| @widget:
    !in <- @Element Http.Server(address=listen)
    let !opens = OneOpen !in
    !dones <- for $pending:
        !started <- @started
        !res <- @Element Http.Fetch(requests=(Rewrite upstream, $_ on !started))
        let !answer = match !res:
            Answered a: a
            Failed why: BadGateway why
        !sent <- @Element Http.Reply(id=key, requests=!answer)
        @ Close on !sent
    $pending <- {} then Apply $pending, (!Union !opens, !dones)
    @ unit
```

Each pending request gets its own `Http.Fetch` slot under its own key, so there is no correlation table: the response comes back to the slot that asked. The `for` item is built in the occurrence the request arrives, `@started` fires there, and the outgoing request is performed after that turn. When the reply is delivered, the block fires `Close`, the consolidated `!dones` removes the id from `$pending`, and the slots go at commit. An incoming request and a delivered reply in one occurrence are both counted, because `!Union` merges the maps rather than choosing one.

### Errors as a stream

```
type RuntimeError = record:
    kind: ErrorKind                # Cycle | Overflow | Protocol | Panic
    spans: [Span]
    message: Text

tag Errors of Element:
    responds: RuntimeError
```

A World may offer `errors`: a stream element that fires, in a later turn, with each runtime error the World received. A program subscribes to it like any stream, and what it does with the information is its own business: show it, log it through `Cli.Stderr`, count it. A program cannot *catch* an error, because the occurrence it happened in did not happen and nothing in the language can observe a rollback from inside; what it can do is learn, afterwards, that one occurred. For a World that quarantines a failing subtree (replacing it with an `Error(span, message)` presence element in the list), the stream and the element say the same thing from two sides: the stream is the edge, the element is the level.

The alternative is a `catch` that lets a parent widget react to a child's runtime error in the occurrence it happens. That is an effect handler in disguise, and it would make a hold's value depend on whether some unrelated subtree threw, which is the back door the language closed. Errors a program is meant to handle are values with types; errors the language cannot make into values are visible as elements and nowhere else.

## The CLI World

The smallest World that is not a test, and the first place a Prism program touches a real process. Its components are the general ones above plus four `Cli.*` tags of its own, declared here with `Process.Exit`:

```
type StdinEvent = choice:
    Chunk Text
    End

tag Cli.Stdin of Element:
    responds: StdinEvent

type WriteFailed = choice:
    Closed
    Failed Text

tag Cli.Stdout of Element:
    requests: !Text
    responds: WriteFailed

tag Cli.Stderr of Element:
    requests: !Text
    responds: WriteFailed

tag Process.Exit of Element:
    requests: !Code

type Signal = choice:
    Interrupt
    Terminate
    Hangup
    WindowChange

tag Cli.Signals of Element:
    responds: Signal
```

Standard input is a stream: chunks, then `End`, delivered only while a `Cli.Stdin` slot exists, each in its own occurrence. The stream belongs to the World, not to the slot. A read that is already blocked when the slot goes cannot be taken back, so when it returns its chunk is not delivered to the dropped slot or anywhere else; the World keeps it with the stream, and the next `Cli.Stdin` slot receives it first, ahead of anything read afterwards. A chunk nobody asks for by the time the process ends is discarded with it. Standard output and standard error are requests: a firing is a write, and nothing is written for any other reason. Exit is a request. Signals are a stream, and the World installs a handler only while a `Cli.Signals` slot exists; without one the process keeps the operating system's defaults, so Ctrl-C still kills a program that never asked to be told about it.

## One turn per input

Each chunk, each completed read, each timer expiry, each signal is its own turn. The World's loop is a plain blocking loop on an input queue: block until something is there, take one thing, run a turn, perform the firings, reconcile, block again. Two chunks that arrived while the program was busy are two turns in order, never one occurrence, because they are two responses to one slot. A completed file read and a chunk that arrived together may be one occurrence, since they address different slots, and the World that batches them is making the honest claim that both were waiting.

## Output ordering

After a turn the World performs the turn's request firings in occurrence order, and within an occurrence in list order. For writes that means: everything occurrence 1 wrote, in the order the slots

appear in the list, then everything occurrence 2 wrote. A `Cli.Stdout` slot and a `Cli.Stderr` slot that both fired in one occurrence write in list order, which is the order the program bound them. Two firings from one slot in one occurrence are impossible (a request event fires once per occurrence), so a program that wants two lines in one moment fires a text with a newline in it, or a `[Text]` if the tag's request type is an array. Output is flushed after each turn, so a program that writes and then waits on a timer has its output visible while it waits.

## Exit as a request

```
@Element Process.Exit(requests=(Code 0 on !end))
```

When `Process.Exit` fires in a turn, the World finishes the turn, performs every firing of that turn in order (so a write fired in the same occurrence as the exit lands first, and a write fired in a later occurrence of the same turn also lands first), and exits with the code. Exit is a request and not a presence for the reason chapter 2 gives: a presence would have to appear at the right moment, which means a `rebuild` over a flag, and the World would have to act on an appearance, which is a trigger by existence. The element is always there, and firing it is the instruction.

A program that never fires `Process.Exit` exits with code 0 when nothing can ever cause another occurrence: the input queue is empty, no stream slot is live (no `Cli.Stdin` or it has delivered `End`, no `Cli.Signals`, no `Time.Every`, no `Net.Listener` or socket), and no request is in flight. That is not a policy so much as an observation: a graph with no sources left cannot change, and a process waiting on it would wait forever. The alternative, requiring an explicit exit, turns every forgotten exit into a hang. `Process.Args`, `Process.Env`, `Clock`, `Fs.Watch` and `Fs.Contents` watches do not keep a program alive; a live `Cli.Stdin`, a `Cli.Signals` slot, a `Time.Every` slot, an in-flight request or timer, and a listener or socket do. Args, env and the clock do not, because the CLI World never pushes them on its own; a `Clock` push arrives with some other input, and when no other input can come, neither can the clock.

## A worked run

`wc -l` from chapter 2, driven by the CLI World:

```
  turn 0:  run(no inputs)                 → list [Cli.Stdin, Cli.Stdout, Process.Exit]; no firings
           reconcile: a Cli.Stdin slot exists → start reading stdin on a thread
  turn 1:  chunk "one\ntwo\nthr"          → responses [(0, Chunk "one\ntwo\nthr")]; no firings
  turn 2:  chunk "ee\n"                   → responses [(0, Chunk "ee\n")]; no firings
  turn 3:  end of input                   → responses [(0, End)]
                                          → firings [(1, ["3\n"]), (2, [Code 0])]
           perform in list order: write "3\n" to stdout; exit 0
```

Three turns, three occurrences, and the exit code is the last thing the World did. A golden test for the CLI World is exactly this transcript: program, arguments, environment and standard input in; standard output, standard error and exit code out. Anything involving timers is tested under the Test World instead, with `push Clock` and `send Time.After(..), 30000`, because a golden test with a real clock is a flaky test.

## A REPL with a timeout

```
@Repl: @widget:
    !in <- @Element Cli.Stdin
    !idle <- @Element Time.After(requests=(30000 on (!started | !line)))
```

```
    !started <- @started
    let !line = !Filter (ChunkOf !in)
    let !end = !Filter (IsEnd !in)
    $n <- 0 then $n + 1 on !line
    @Element Cli.Stdout(requests=((Reply $n, !line) | "bye\n" on (!end | !idle)))
    @Element Process.Exit(requests=(Code 0 on (!end | !idle)))
    @ unit
```

The timeout restarts on every line, since each firing is a new timer. An old timer's response still arrives, and the program exits on it, which is a bug: the fix is the debounce pattern, a pending count, and the point of writing it this way first is that the bug is in the program's text, where a test under the Test World (`send Time.After(..), 30000` after a line) finds it, and not in a race between a signal handler and a read loop.

### Where tokio goes

The CLI World runs its turn loop on the main thread and uses tokio, or plain threads, for reading standard input, waiting on signals, performing file reads and timers. Each of those tasks ends by pushing plain data onto the queue and waking the loop. No task ever touches the runtime, and no task holds anything but a path, a tag and some bytes. The async-runtime section below says why this split is the right one for every World, including the ones with a window.

## GUI: one vocabulary, spoken by many Worlds

GUI is not a language concept, and not a World concept either. Some Worlds can put pixels on a screen and read a pointer and a keyboard, and they offer tags for that. Layout, hit-testing, focus traversal, text editing, buttons, lists and scrolling are Prism, written as interceptors over those tags, in a library that runs unchanged under any World that offers them and under the Test World, which offers everything. The language has an opinion only about structure: leaves are elements, containment is interception, a World draws what it is given and delivers what it sees.

You have lost a cursor to a native text box, and you have fought a toolkit's focus manager for a field it insisted on focusing. Both happened because the toolkit owned state the program also had, and the two copies disagreed. The vocabulary below has no state in it that a program could also hold. A `Gui.Focusable` is a name and nothing else, so a World can show a focus ring; where the caret is, which field is focused and what Tab does are holds in Prism, and there is no second copy.

### The vocabulary

Five tags. A window, the picture in it, the name of the focused region, a way to measure text, and a tick per frame.

```
type Pointer = choice:
    Moved Point
    Down Point, Button
    Up Point, Button
    Scroll Point, Vector

type Key = choice:
    Char Char
    Named NamedKey                  # Left | Right | Backspace | Tab | Enter | ...
    Composition CompositionEvent    # IME: Start | Update Text, Nat | Commit Text | End
```

```
type WindowEvent = choice:
    Resized Extent
    CloseRequested
    Pointer Pointer
    Pressed Key

type Extent = record:
    w: Float
    h: Float

tag Gui.Window of Element:
    title: Text
    size: Extent                    # the size the program asks for when the window opens
    responds: WindowEvent

tag Gui.Scene of Element:
    window: Text                    # the title of the window it paints
    draw: [Draw]

tag Gui.Focusable of Element:   # presence: the name of the focused region
    name: Text

tag Gui.Metrics of Element:
    text: Text
    font: Font
    state: Extent                   # width and height of the text; answered synchronously

tag Gui.Frames of Element:
    responds: Frame                 # `Frame` is the record declared in the Time section
```

The draw commands are not tags. They are plain values in the scene's `draw` list, and the list order is the paint order.

```
type Draw.Rect = record:
    rect: Rect
    color: Color

type Draw.Text = record:
    origin: Point
    text: Text
    font: Font
    color: Color
    ime_at: ?Nat                    # an offset into text where an input method's candidate window goes

type Draw.Image = record:
    rect: Rect
    image: ImageId

type Draw.Clip = record:
    rect: Rect
    draw: [Draw]

type Draw = Draw.Rect | Draw.Text | Draw.Image | Draw.Clip
```

A `Draw.Rect` is a rectangle on the screen and nothing else: it has no events, no name, no identity. A button's background is a `Draw.Rect` and the button is a library widget that knows where that rectangle is. The last line is a union of the four records and has no constructors of its own, which is why a draw command is never an element and never needs a path.

A window has a title and a size, both plain fields that rarely change, so its slot is stable for the life of the program. The World has one path to send pointer events and resizes to. The picture is a separate presence element, `Gui.Scene`, which is replaced whenever the picture changes: a fresh slot per frame is right for something nobody is waiting on, and keeping the picture out of the window is what lets `Gui.Window(title="Counter", ..)` stay one fact. The World diffs the new scene against the committed one, or redraws it whole. It decides which, because the language cannot see the difference.

The window answers with four things: the size the platform gave it, a close request, every pointer event over the whole window, and every key. The World does no hit-testing, and the reason is that a World that did would need to know what is clickable. Suppose it hit-tested against rectangles the program declared. Then every clickable thing would exist twice, once as the pixels the library drew and once as the rectangle the World matches, and the two would disagree on the frame where a button moves and its rectangle has not, which is the toolkit bug from the start of this section in a new place. It would also need an answer for overlap, for a scrolled list clipping a half-visible row, for a disabled button and for a pointer capture. The library has the layout and the clip regions and every one of those answers already, so the library hit-tests: the pointer position goes in, the widget path comes out, and the World needs only to know where the pointer is, which only it knows.

Keys go to the window, like pointer events, and the World does not choose who gets them. The library routes each `Pressed` key to the input whose name is the focus, so the World has one place to send keys and no focus order to keep. Which field is focused, what Tab does and where the caret is stay holds in the library, and focus is by name: the library's focus hold is a `Text`, and a rebuild that keeps the name keeps the focus. `Gui.Focusable` is presence only, a name and nothing else, and the library emits at most one at a time, the one it holds as focused. It hears nothing. It exists so that a World can show the platform's focus ring, place an input method's candidate window and tell a screen reader what has focus. Composition events from an input method arrive as keys, at the window. The World puts the candidate window at the `ime_at` offset of the focused `Draw.Text` run, which it finds in the committed scene when the platform asks; the World shapes text itself and so can turn an offset into a position, and the library does not tell it twice.

`Gui.Metrics` is the one state element in the vocabulary, and it answers the question every layout asks: how wide is this text? Its answer is an `Extent`, the width and height of the text in the font. You cannot place a caret or centre a label without it, and only the thing that has the fonts knows. As a request it would answer next turn and the text field would paint one frame with stale positions per keystroke. As state, its oracle is synchronous, the slot has its metrics in the occurrence that creates it, and the layout that depends on them is right in that occurrence. Value identity keeps it cheap: every slot asking about `("Start", Default)` shares one measurement. The font is a plain `Font` value in `Draw.Text` and in `Gui.Metrics`, and the World resolves it.

`Gui.Frames` responds with the time once per frame while a slot exists, for animation, and the World requests continuous redraw only while there is one.

Two things a GUI program seems to need are not in the vocabulary, a clipboard and a font set. A clipboard is a request and a response, `Clipboard` with `Read` and `Write Text`, offered as a bare tag by any World that has one. It is separate from `Gui.*` because it has nothing to do with drawing: a terminal World with access to the system clipboard offers it, and a GUI World on a platform without one does not, and `Unsupported` already says so. Installed fonts are not a tag at all. A program names a font in the value it draws, the World resolves the name, and anything that wants to enumerate fonts for a picker asks a separate state tag, which a World may offer and the five-tag vocabulary does not need. Both stay out for the same reason: each tag in `Gui.*` is something every World that draws pixels must implement, and these two are not.

## The GUI library

Above those tags, in Prism. The library has its own tags under `Ui:` `Ui.Label`, `Ui.Button` (responds `Click`), `Ui.TextInput`, `Ui.VBox`, and so on. They never reach a World; the library's interceptors claim them. `@Button` is a `Ui.Button` that the enclosing box turns into a `Draw.Rect` and a `Draw.Text` at the place it chose; `@TextInput` is a hold with the text and cursor, `Draw.Text` runs and a caret rectangle placed with a `Gui.Metrics` reading (chapter 2's version is the whole thing); `@VBox` is an interceptor that captures its children's `Ui.*` elements, assigns them rows, and re-emits one `Ui.VBox` with the layout; `@Window` is the interceptor at the top. It captures everything, lays it out using the metrics it asked for, emits one `Gui.Window` and one `Gui.Scene` with the whole draw list, hit-tests the window's pointer events against the layout it holds, and routes each hit inward as a `Click`, and each key to the input whose name is the focus. Focus is a hold in `@Window` moved by Tab and by clicks, and the library emits only the focused child's `Gui.Focusable` upward. Scrolling is a `Draw.Clip` and an offset. An accessibility tree, where a World can use one, is a field the library builds from the same element list it already holds, with node identities taken from `for` keys so that focus survives a rebuild that keeps the data's identity.

None of this reaches the World except as `Gui.Window`, `Gui.Scene`, `Gui.Focusable` and `Gui.Metrics` elements, and that is the division of labour: the World rasterises and delivers; the library decides.

## A GUI turn, end to end

The counter of chapter 2 under `@Window @VBox @counter`. `@Window` lays out the box, emits the window and the scene, and hit-tests pointer events by the layout it computed. The World is the software rasteriser, and its font has 8-pixel advances.

**Occurrence 0.** The counter builds its two buttons and a label; `@VBox` captures them and emits a `Ui.VBox` with the three as children; `@Window` captures that and asks for the width of the label's text with a `Gui.Metrics` slot, which has its value in this occurrence. It computes a layout (three rows of 24 pixels, the label centred) and emits:

```
0/0:        Gui.Window(title="Counter", size=Extent(w=200.0, h=72.0))
0/1/#0/0:   Gui.Metrics(text="Count: 0", font=Default)
0/2/#0/0:   Gui.Scene(window="Counter", draw=[[
              Draw.Rect(rect=((0, 0, 200, 24)), color=grey),
              Draw.Text(origin=((4, 4)), text="Increment", font=Default, color=black, ime_at=Nil),
              Draw.Rect(rect=((0, 24, 200, 24)), color=grey),
              Draw.Text(origin=((4, 28)), text="Decrement", font=Default, color=black, ime_at=Nil),
              Draw.Text(origin=((68, 52)), text="Count: 0", font=Default, color=black, ime_at=Nil)]])
```

The World reads the list, sees a window it has not opened, opens a 200×72 window, answers the metrics slot from its font (64 pixels wide), rasterises the scene into the window, presents, and waits. There are no regions in the list and no button anywhere in it: the World has a window to deliver pointers to and a picture to paint.

**Turn 1.** The user clicks at (50, 10). The World delivers `Pointer (Down ((50, 10)), Left)` as a response to `0/0`. `@Window`'s slot fires; the window interceptor hit-tests the point against the layout it holds, read through a tag, which is the layout as the occurrence opened, the one the user saw; row 0 is the `Ui.VBox`'s first child, the Increment button's path; it fires `{vbox_path: Pointer(row=0)}` down; `@VBox` hit-tests the row against its children's list as the occurrence opened and fires `{0/#0/0: Click}` down; the counter's `!increment` fires; `$count` becomes 1; the label rebuilds; the VBox's children change; the VBox rebuilds its element; `@Window` relays out. The label text is a plain field of the metrics slot, so the metrics are a fresh slot at generation `#1`, and its value (still 64) is in the oracle in this occurrence, so the new scene at generation `#1` has the right label position. One occurrence,

no `Defer`. The window slot is untouched. The World diffs the scene, rasterises the new draw list, presents.

**Turn 2.** The user drags the window edge; the platform reports a new size. The World delivers `Resized Extent(w=300.0, h=72.0)` to `0/0`. `@Window` holds the size, relays out with wider rows and the label at x=118, and emits a scene at generation `#2`; the metrics are unchanged, so their slot stays. The World resizes its buffer and presents. On a platform that expects the paint before the resize callback returns, the World runs this turn inside the callback and presents before returning; one turn per input makes that a non-event.

**Turn 3.** The user presses a key. The World delivers `Pressed (Char 'x')` to the window at `0/0`, like any other input. There is no `Gui.Focusable` in the list, because the counter has nothing to type into, and `@Window` has no focused input to route the key to, so it drops it. Had there been a text input, `@Window` would have emitted `Gui.Focusable(name="input")` upward, the focused name it holds, so the World could show a focus ring; the key would still have arrived at `0/0`; `@Window` would have routed it to the input named `input`; and the input's hold would have moved its cursor, asked `Gui.Metrics` for the width of the text up to the cursor, and drawn the caret there. The World knows nothing about cursors.

Three turns, three occurrences, and the World's whole job was: open a window, rasterise a list, deliver a point, a key and a size to a path. Everything that looked like a toolkit was Prism, and ran unchanged under the Test World when `send Gui.Window(..), Pointer (Down ((50, 10)), Left)` drove it.

## Software rasteriser

The smallest GUI World: one window, one scene, a pointer and a keyboard. The World opens a platform window, and after each turn takes the `Gui.Scene` element's draw list, rasterises it into a buffer with its own text rasteriser, and presents it. It answers `Gui.Metrics` from the same font, delivers every pointer event and every key to the window's slot. It shows a focus ring for the `Gui.Focusable` in the list, if any. It is a few hundred lines over a windowing library and a rasteriser, and it is the World the conformance suite's GUI programs run under first, because it has nothing in it that could be wrong except the obligations.

## A native toolkit wrapper

A World that wraps GTK, AppKit or Win32 offers higher tags: `Gtk.Button(label)` responding with `Clicked`, `Gtk.Entry(text)` responding with `Changed Text`, `Gtk.Box(children)`. The World diffs the list against the widget tree it keeps, creating, updating and removing native widgets. The same Prism program targets it through a different library: `@Button` becomes `@Element Gtk.Button(label=$_)` inside a `rebuild`.

The hard case is the native text entry. Its cursor and selection live in the toolkit, and a fresh slot per keystroke (which is what a dynamic `text` field means) would lose them. The World's answer is continuity by tree position: a `Gtk.Entry` slot replaced by a `Gtk.Entry` slot at the same bind position and `for` key is the same native widget with its text updated, and the World says so in its documentation. The program cannot tell, and the language promises nothing about it. The alternative, a tag field declared as `$Text` so that the slot persists and the World sees the field change, would make the boundary see a sigil, and the leaf-plus-interceptor pattern has not yet produced a case that needs it.

Which is better, native widgets or a Prism library over draw primitives, is a library question with a platform answer: native widgets get the platform's own accessibility, input methods and look; a

Prism library gets one behaviour everywhere and a text box whose cursor cannot be lost. A program can be written for either and some will be written for both.

## HTML and the web

The browser is the proof that the vocabulary works, because the DOM is it. A DOM node is a plain record with a tag and attributes; the browser diffs what you give it against what it has, lays it out, paints it, runs the input method and the screen reader, and delivers events addressed to nodes. A web World offers `Dom.Node(tag, attrs, text, children)` as its one container element, responding with `DomEvent(name, detail)`, and the whole GUI library of the previous sections becomes a different library that emits nodes instead of a `Gui.Scene`. Text editing keeps its state in Prism and puts out an editable node as its bridge: the browser's `beforeinput` and composition events (from memory) arrive as responses, Prism decides what they do to the text, and the World writes the node's text and selection back only when Prism's state differs from what the node last reported, and never during a composition, so there are no echo writes and no caret fights.

There are no threads in a browser page, and no loop to own. Each DOM listener, each `requestAnimationFrame` tick, each completed `fetch` is a callback, and each callback runs one turn. The World's input queue is the browser's event loop; the World's "block for input" is returning from the callback. Nothing about the run call changes, because it was synchronous and single-threaded from the World's view all along.

## A native Prism system: X11 over a socket

Take the vocabulary to its end. The World is a socket system: it accepts `Net.Socket` and nothing else, and it has never heard of a window. The program connects to `unix:/tmp/.X11-unix/X0`, and everything from there up is Prism: an interceptor that captures the GUI library's `Gui.Window`, `Gui.Scene` and `Gui.Focusable` elements, answers its `Gui.Metrics` slots, and emits bytes.

```
@X11: fn @a -> @a = \|@app| @widget:
    ((result, $slots)) <- intercept @app, !down, $ \|e| @Element e
    ((!rx, $link)) <- @Element Net.Socket(address="unix:/tmp/.X11-unix/X0", requests=!tx)
    $conn <- Handshaking then StepX11 $conn, !rx              # parse replies and events
    let !tx = Encode $conn, $slots, !rx                      # CreateWindow, PutImage, ...
    let !events = DecodeEvents $conn, $slots, !rx            # ButtonPress → Pointer to the window's path
    !fwd <- @Forward (NonGui $slots)
    let !down = !Union !events, !fwd
    @ result
```

`Encode` is a plain function from the connection state, the current element list and the latest bytes to the bytes to send: on the first list with a `Gui.Window`, a `CreateWindow` and `MapWindow`; on each change of the scene's draw list, a `PutImage` of the rasterised buffer, where the rasteriser is a pure native function `Rasterise: fn [Draw] -> Image` with its own font data and no time in it. `DecodeEvents` turns a `ButtonPress` at a point into a `Pointer` response addressed to the window's slot, and a key press into a `Pressed` response addressed to the same window slot, both read against `$slots` as the occurrence opened, which is the same routing law every interceptor obeys. The point is not hit-tested here, because the library above does that. The interceptor also supplies the `Gui.Metrics` slots the application asked for, from the same font data the rasteriser uses, so a label is measured with the glyphs it will be painted with. The World below knows it is writing bytes to a socket and reading bytes back.

This is not the way to build a desktop; it is the demonstration that the GUI vocabulary does not

depend on the World having one, and that "the things that are native are the things that touch hardware" means what it says. A compositor, a terminal emulator and a window manager are each this interceptor with more state, and each can be tested under the Test World by sending it the bytes an X server would.

### IME and accessibility queries

A platform sometimes asks a question and blocks on the answer: where is the caret, so the candidate window can be placed; what is the text of this node, for a screen reader. The World answers from the committed list, where the GUI library keeps the answer current: the `ime_at` offset of the focused text run in the scene, the accessibility tree. That is the right answer for a query about inputs the platform already delivered, because each of those ran its own turn and the committed list reflects all of them. A World that wants the answer to include an input it has not yet delivered may run a turn inside the callback, delivering the query as a response to the window slot and reading the reply field afterwards; nothing forbids a World from running a turn whenever it likes, and the program sees an ordinary occurrence. A callback that arrives *during* a turn (the platform calling back from inside a frame change) is answered from the last committed list without starting anything, because the World never runs a turn inside a turn.

## Networks in depth

A socket is where every abstraction over IO shows its seams, because a socket has all the problems at once: bytes with no boundaries, a peer that can stall, a connection that can drop, and a handshake nobody wants to write. Here is each, and where it lives.

### Framing

`Net.Socket` moves bytes. `Received Bytes` is whatever the operating system handed the World, and the World does not promise that one send on the far side is one `Received` here. Framing is the program's: a hold that accumulates bytes and a plain function that splits off complete frames, in the `for` block or widget that owns the socket, as chapter 2's server accumulates a request head.

```
let SplitLines: fn Text -> (([Text], Text)) = ...    # complete lines, and the remainder

@Lines: fn Text, !Text -> @![Text] = \|address, !send| @widget:
    ((!rx, $link)) <- @Element Net.Socket(address=address, requests=(Encode !send))
    let !chunk = !Filter (TextOf !rx)
    let !split = SplitLines (Concat [[$rest, !chunk]])
    $rest <- "" then Second !split
    @ First !split
```

`Concat [[$rest, !chunk]]` reads the remainder through a tag, before the chunk, which is the remainder to append to; `SplitLines` returns the complete lines and the new remainder; the hold keeps the remainder and the widget returns the lines. The remainder and the lines come from one expression because there is no "after" inside an occurrence. A World could offer a `Net.Lines(address)` variant that responds with whole lines, and should, since it is a few lines of host code and saves every program the fold; it is a convenience, and the hand version is the real one.

One frame per occurrence or many? A chunk may hold three lines, and the widget above returns all three in one firing of `![Text]`. A program that wants each line in its own occurrence cannot have it from one response; it holds the lines in a queue and uses `Defer` to drain one per occurrence, or it asks

the World for a `Net.Lines` element whose responses are one line each, in which case three lines are three turns. The second is what a World should offer, because the World can make the claim "these arrived separately" and the program cannot.

## Backpressure as state

`Connected n` in the link state says how many bytes are queued. A program that reads it at the moment of sending, through a tag, and declines to send when the queue is deep, has implemented flow control without a writable callback, a promise, or a `drain` event. The World updates the count as it writes, which is a change to a state element, which is a push at most once per turn. That granularity is right: a program does not need to know about every kilobyte, it needs to know whether to send now, and "now" is the occurrence.

The incoming side has no backpressure in the tag, and that is deliberate. A `Received` is a response, and responses are performed by the World in order and never dropped; a World that read faster than a program could consume would queue occurrences, which is the correct place for the queue, since each occurrence is cheap and the program's holds are the buffer. A World that wants to stop reading when a program's queue is deep can do so, and the program never knows.

## Reconnect by address change

The address is a field, so changing it is a `rebuild` and a fresh slot, and the chat client of chapter 2 reconnects by typing. A reconnect *to the same address* is the interesting case: the state is shared by value, so a second slot with the old address keeps the old connection alive, and a program that wants a clean reconnect to the same host needs the element to differ. The field to add is a generation: `Net.Socket(address=a, attempt=n)`, with `n` bumped by the program on each retry, which is also where a retry policy lives, behind `Time.After`, in the program's text.

A World *may* reconnect on its own behind one address, since the link state has `Disconnected` and `Connecting` cases to tell the program about it, and the program's sends while `Disconnected` get `Closed` back. Whether it does is World policy; the tag's type makes either policy expressible to the program.

## TLS is a World concern

The address scheme says `tls://`, and the World does the handshake, the certificate check and the encryption, reporting `Connecting` until the handshake completes and `Disconnected "certificate ..."` if it fails. Nothing about TLS is in Prism, for two reasons that both come from the negative space. The language has no effects, and a TLS implementation wants randomness, which is an effect; it could be a state element the World pushes, but then the key material would be a value in a Prism hold, which is the wrong place for a secret, and the handshake would be a state machine over occurrences, which is a slow place for one. And the language is for programs that are "state, things happen, respond," not for computations; a cipher is a computation. So TLS is on the socket's far side of the boundary, with the kernel's TCP stack, and a program that needs to know which certificate it got asks a state element for it.

## One socket per address, or per slot

Two `Net.Socket(address="chat.example:7000")` slots in one program share one `Link` and have separate request and response streams. The World may keep one OS socket and multiplex the two slots' sends onto it in firing order, delivering every `Received` to both, which is what "one connection, two

conversations" means; or it may keep two OS sockets and lie about the shared state, which it may not, since the equality law says the two slots agree on `Link` in every occurrence. One socket per address is the implementation that keeps the law for free, and it is the one the first World uses.

### A listener and its connections

`Net.Listener(address)` is a stream of `Accepted id`, and `Net.Conn(id)` is the connection, created by the program under a `for` keyed by the ids it was given. The World binds the address when a listener slot exists after a turn and unbinds it when none does, and it accepts only while a listener slot exists, so a program that wants to stop accepting removes the element (a `rebuild` over a flag) and the World stops. That is reconciliation, not a trigger: a listener in the list twice is one bound socket.

A connection the World accepted for which the program never creates a `Net.Conn` slot is the World's to close, after a grace period it documents; a `Net.Conn(id)` slot for an id the World never allocated gets `Gone` as its state and `Closed` as its first response. The ids are the World's and the program only ever echoes them.

### HTTP

`Http.Fetch` is a request element and nothing more: fire a request record, get a result in a later turn, with non-2xx statuses as `Answered` (they are answers) and transport failures as `Failed`. Redirects, compression and connection pooling are inside the World. A program that wants a streaming body asks for `Http.Stream`, which responds with chunks and an end, and a World offers it when it can.

`Http.Server(address)` responds with `Incoming(id, request)`, and `Http.Reply(id)` is a request element the program fires once per id. A reply to an id the World is not waiting on is answered `Gone`; a second `Http.Reply(id)` slot for one id is a second conversation the World answers `Gone` too. Concurrent requests are several ids under one `for`, each with whatever outgoing calls it needs as its own slots, and the proxy above is the whole pattern.

## Async runtimes

A World built on tokio has a main loop that looks like this, and the point of showing it is where the run call sits in it.

```
enum Input {
    Response(Path, Value),             // a completion addressed to a slot
    Changed(ElementValue, Value),      // a state element's new value
}

async fn main_loop(mut core: Core, mut rx: mpsc::Receiver<Input>) {
    loop {
        // 1. Wait for the first input; then take everything else already queued.
        let first = match rx.recv().await { Some(i) => i, None => break };
        let mut batch = vec![first];
        while let Ok(more) = rx.try_recv() { batch.push(more); }

        // 2. One turn per response-bearing input (responses to one slot never share
        //    an occurrence); state changes ride along with the next turn.
        for inputs in core.partition(batch) {
            let outcome = core.turn(inputs);        // synchronous; never awaits
            core.dispatch(outcome).await;           // perform + reconcile: spawn tasks
        }
```

```
    }
}
```

The run call is synchronous and single-threaded from the World's view, and it is inside `core.turn`, which never awaits. Everything asynchronous happens in `dispatch`, after the turn: a `Read` firing becomes a `tokio::fs::read` task that ends by sending `Input::Response(path, Contents ..)` on the channel; a `Net.Socket` slot appearing becomes a connect task that sends `Input::Changed(socket, Connected 0)` and then a read loop that sends `Input::Response(path, Received ..)` per chunk; a `Time.After` firing becomes a `sleep` that sends `Input::Response(path, delay)`, echoing the number it was fired with. The tasks hold a path, an element value and a channel sender. None holds the runtime, the core, or anything that could reach either.

## Why synchronous, and why one thread

Suppose the run call were `async`, awaiting inside propagation when a response was needed. Then an occurrence could be suspended with holds uncommitted while another input arrived, and either the second input waits (so the await bought nothing) or it opens a second occurrence inside the first (which the transaction forbids). An occurrence is a transaction, and a transaction that yields halfway is the checkbox bug with a scheduler attached. And suppose the runtime were driven from several tokio worker threads. Then two occurrences could run at once, which is the parallelism the model allows across occurrences, but as transactional concurrency with read-set checking and retry, not as "whichever worker got there first." That is a runtime variant, described in the last section, and nothing a World does.

So the runtime is `send + !Sync`: one owner, one thread, and the World's choice of which thread. A GUI World's main thread, because the platform says so. A CLI World's main thread, because there is no reason not to. A server World's one thread per shard, because that is the parallelism that costs nothing. tokio is the IO side of every World, never the propagation side. A World grown with tokio as its propagation loop would learn to await inside a turn and hold graph state across an await, and no GUI main thread allows either, so the World that was developed would not be the one users got.

## One occurrence per readiness event, or a batch

A readiness event (a socket readable, a timer expired, a read completed) becomes one input on the channel. When the loop wakes, several may be queued, and the World chooses. Responses to different slots that were waiting together may be one occurrence: the program sees them as simultaneous, which the World is entitled to claim, and a program whose text has a merge in it has already said what simultaneity means. Responses to one slot are always separate occurrences, in arrival order. State changes are coalesced to the last value per element and ride with the next turn, because state is a level.

Two policies are both conforming and feel different. *One turn per input* gives the program the finest view of order and the most occurrences; it is right for a CLI World and for anything with a transcript, because the transcript is unambiguous. *Batch what is queued* gives fewer occurrences and lower latency under load; it is right for a GUI World between frames, and for a server. A test World does what the script says, and two deliveries on one line is how a script says "batch these."

### Completions become responses, and vanished slots cancel

A task that completes sends `Input::Response(path, value)`. The core, at the start of the next turn, checks the path against the list it last received; if the slot is gone, the response is dropped and logged. The task itself does not know the slot is gone, so the World keeps, per in-flight operation, an abort handle keyed by path, and `reconcile` aborts the handle when the slot vanishes: an in-flight `fetch` is cancelled, a `sleep` is dropped, a socket is closed after its queued writes drain. A write that already landed stays landed; the World cannot un-write.

The same mechanism is why a `rebuild` firing "reloads": the old slot's read is cancelled with the old slot and the new slot's `@started` fires a new one. Guessing that the two are the same read, by position, is how a half-typed draft ends up on the wrong row; the World does not guess.

### The browser, where there is no loop

A web World has no `main_loop`, because the page's event loop is not the World's to own. Every callback runs one turn: a DOM listener, a `requestAnimationFrame` tick, a resolved `fetch`. The channel is replaced by the browser's own queue, and "spawn a task" is `spawn_local` (from memory) of a future that ends by running a turn with its result. Nothing about `Core::turn` changes, and nothing about the obligations. A World that cannot block is still a World, because a turn never blocked.

## System calls and blocking operations

Some things block and have no asynchronous form worth using: `getaddrinfo` on some platforms, a synchronous filesystem on others, `fork`/`exec`, a database driver. The rule is that blocking happens on a thread inside the World, never inside the runtime, and never inside the oracle.

*Not inside the runtime*, because the run call is one thread doing one transaction, and a block there stalls every input behind it; in a GUI World it stalls the frame. A component that needs to block spawns a thread (tokio's `spawn_blocking`, from memory, or a plain `std::thread`) and sends the result on the channel like any completion.

*Not inside the oracle*, with one bounded exception. The oracle is the snapshot as a function, and it must answer the same value to every slot that asks in one occurrence. A World that implemented `Fs.Contents`'s oracle by reading the file on every ask would give two slots created in one occurrence two different values if the file changed between the reads, which breaks the equality law. So the oracle reads the snapshot map, and the one exception is the first ask for a local file: the component reads it then, under a size cap, and records the result in the snapshot before answering, so every later ask in every occurrence gets the recorded value and the change arrives as a push. Past the cap the answer is `Unavailable TooLarge`. Anything that waits on something outside the machine (a socket, a DNS lookup, a database) is never read in the oracle: its answer is the tag's unavailable case, and the component starts the work after the turn, when it sees the slot in the list.

The cap keeps the exception honest. Without it, the oracle would block on whatever size of file a program named, and the latency would land inside the occurrence; with it, the read is the kind of blocking a main thread does anyway, and a file past the cap gets the true frame, `Unavailable TooLarge`, and a program that minds draws the previous state instead, with a hold. The Test World has no disk, so it models the oracle with `with` steps, including `with Fs.Contents(path="/big"), Unavailable TooLarge`.

The same discipline applies to `Process.Spawn`: the spawn happens on a thread after the turn, the `Spawned id` comes back as a response, and the program that fired from `@started` sees it a turn later,

never in the build.

## Cross-platform

A World is per platform; a program is not, if it stays inside the general components and the GUI vocabulary. The line between the two is drawn by tag namespace.

### Which components are per platform

The general components, `Fs`, `Time`, `Clock`, `Process`, `Net`, `Http`, `errors`, `Defer`, and the GUI vocabulary, are offered by every World that can, with one meaning. Raw access is per platform and named so: `Sys.Posix` on Linux and macOS, `Sys.Win32` on Windows, `Sys.Wasi` on a WASI host, nothing on the web. A shared `Sys` that meant different things in different places would only pretend to port, and a program that uses `Sys.Posix` has said in its text that it runs on POSIX.

### What a program sees

A request on a tag the World lacks gets `Unsupported` in the next turn, which the program matches on: a `Clipboard` under a World with no system clipboard, a `Process.Spawn` in a browser. A state tag the World lacks is a build error in the creation occurrence, because a dynamic cannot exist without a value. That asymmetry makes a portable program's job clear: it may fire optimistically and handle `Unsupported`, but it must not create a state element it is not sure of. The way to be sure is a state element every World provides, alongside `Defer`:

```
tag World.Tags of Element:
    state: [Text]                  # the tags this World accepts, by name
```

A program reads it in occurrence zero (it is state, so it has a value then) and builds under a `rebuild` over it, choosing a `Clipboard` path or a fallback by a plain `match`. An interceptor can supply a child a narrower `World.Tags`, which is what a sandbox should do so that the child never tries. The Test World's `World.Tags` is everything, and a test that wants to exercise the fallback sets it narrower with a `with` step. This is the one component that is about the World itself, and it is state rather than a request because the question "what can I ask" has to be answerable before anything is asked.

### Platforms

*Linux and Windows.* A main thread for the turn loop (the platform's event loop for a GUI World, a plain loop for a CLI World), tokio threads for IO, every general component native. Linux is X11 and Wayland under one windowing library; Wayland wants the IME cursor rectangle pushed rather than queried (from memory), which is the focusable's `rect` field written out at reconcile when it changes.

*macOS.* The event loop must be created on the main thread and the window system allows interface calls only there (from memory), so the turn loop lives on the main thread and the World's IO tasks wake it through the platform's proxy. Live resize puts the main loop into a tracking mode that expects synchronous layout and paint per size change (from memory); a resize is a response to the window slot, a turn, and a paint before the callback returns, which the one-turn-per-input loop does without special casing.

*Web.* No threads, no filesystem, no sockets; `Fs` is absent or offered over the origin-private filesystem, `Net.Socket` is absent or a WebSocket, `Http.Fetch` is `fetch`, the GUI vocabulary is the DOM. One turn per callback. `World.Tags` is how a program finds out.

*Mobile.* A GUI World with two more things: a lifecycle state element (`App.Lifecycle: Foreground | Background | Suspended`) the program reads and, since it is state, a late-built widget reads correctly; and a World that may be killed between turns, so that any state worth keeping goes through a store component (below). The turn loop runs on the platform's main thread, as on macOS.

*A server.* No screen; the general components plus `Http.Server`, `Net.Listener` and a store; and the parallelism question, next.

## Servers

A stateful service is "state, things happen, respond" with the screen removed, and the only new question is how many runtimes there are. A runtime is single-threaded and a run call is cheap, so the answer is: as many as there are independent things, and a World that routes between them.

### One runtime per connection, or per shard

A chat server has rooms. Two rooms share nothing: a message in one changes nothing in the other, and if they were one program, a `for` over rooms would build two subtrees connected only through the listener. So they can be two runtimes, each running the same program with a different `Process.Args`, on two threads, with a World that owns the listener and routes each accepted connection to the runtime for its room. From each runtime's side the World is ordinary: a `Net.Conn(id)` for each connection it was given, a `Store` for persistence, and `Defer`. From the World's side there are N cores, each with one runtime, each driven by its own thread, and a routing table from connection id to core.

```
struct Shard { core: Core, tx: mpsc::Sender<Input> }

async fn router(listener: TcpListener, shards: Vec<Shard>) {
    loop {
        let (sock, _) = listener.accept().await?;
        let room = read_room_header(&sock).await?;          // which program this belongs to
        let shard = &shards[hash(room) % shards.len()];
        shard.tx.send(Input::Response(listener_path(shard), Accepted(alloc_id()))).await;
        spawn_socket_tasks(sock, shard.tx.clone());         // bytes → Response(conn_path, Received ..)
    }
}
```

Per connection instead of per shard is the same design with N equal to the number of connections, and it is right when connections share nothing at all (a request-response protocol with per-connection state). Per shard is right when a shard's connections share a model, which is most servers. Nothing in the program changes between the two; the World's routing table does.

### Load balancing

A shard that is busy is a thread that is busy, and the World sees it as a channel that backs up. Rebalancing means moving a room from one runtime to another, which means tearing down its subtree in one program and building it in another, and that is a program-level question: the room's state lives in holds, and holds do not cross runtimes. So the room's state has to be in a store that both runtimes can reach, and the World tells the old runtime the room is gone (its `Process.Args`, or a `Shard.Rooms` state element, changes; the `for` tears the room down) and the new runtime that it has arrived. The model's state comes back from the store in the new runtime's first occurrence for that room, as state, because the store is a state element.

### Persistence via state elements

```
tag Store.Value of Element:
    key: Text
    state: Stored                  # Absent | Present Bytes

tag Store.Put of Element:
    key: Text
    requests: !Bytes
    responds: PutResult            # Stored | Failed Text
```

A room's model is `Store.Value(key="room:lobby")`, a state element with a value in its first occurrence (the World's snapshot, loaded from the store when the World started or when the key was first asked) and pushed when any runtime writes it. Writes are `Store.Put` firings. A model held in Prism is then a hold whose initial value is a `rebuild` over the stored value's first reading, and whose changes are written back by firing `Store.Put` on them, which is the cache pattern of chapter 2 with the direction reversed. Two runtimes watching one key see one value per occurrence, each, because each World core keeps the equality law within its own runtime; across runtimes, "the same occurrence" has no meaning and the store's own version is the clock.

A host that instantiates a runtime per request (a WASI HTTP handler, from memory) is this with N equal to the number of requests and a lifetime of one turn. The program puts out `Http.Server`; the host delivers the request as the first response after the build; the program fires `Http.Reply`; the World sets the host's out-parameter and drops the runtime. State that outlives the request is in the store. A World whose life is one turn is still a World, because the turn was always the unit and the loop was always the platform's.

## Worlds that nest

A World can be written in Prism, and the proof is the shell of chapter 2: a program that runs two programs as its children, shows one, routes clicks to it, and forwards everything else. From each child's side, the shell is its World. From the real World's side, the shell is one program. The interface is the same at every level, and that is the point of `intercept` having exactly the World's two downward supplies and the World's one upward channel: a World and an interceptor are the same thing with a different host language.

### A shell

A process table is a `for` over a keyed map of programs. Each entry is an `intercept` of a child, with the child's `Cli.Stdin`, `Cli.Stdout` and `Process.Exit` held back and answered by the shell, and everything else forwarded. The shell's own `Cli.Stdin` is the terminal; it parses commands, builds children, pipes one child's `Cli.Stdout` firings into the next child's `Cli.Stdin` as responses (through a `Defer`, because a response is always later than the request it answers), and removes a child from the table when its `Process.Exit` fires. The outer World below the shell is a CLI World that knows nothing about the children: it sees one program with one standard input.

```
@Pipe: fn @unit, @unit -> @unit = \|@producer, @consumer| @widget:
    ((ignore, $ps)) <- intercept @producer, !down_p, $ \|e| @Element e
    ((ignore, $cs)) <- intercept @consumer, !down_c, $ \|e| @Element e
    !writes <- for $ps:                        # the producer's stdout firings, by its path
        rebuild $_:
            match _.element:
```

```
            Cli.Stdout(..): @ _.!request
            ignore: @ !never
let !out = JoinText "", (Values !writes)
$seq <- 0 then $seq + 1 on !out
!delivered <- for $queue:                    # one Defer slot per queued chunk
    !started <- @started
    !later <- @Element Defer(requests=(unit on !started))
    @ $_ on !later
let !changes = !Union {$seq: Enqueue !out}, (MapValues !delivered, \|ignore| Dequeue)
$queue <- {} then StepQueue $queue, !changes
let !down_c = {StdinPath $cs: Chunk (JoinText "", (Values !delivered))}
!fwd_p <- @Forward (Except (\|e| match e:
    Cli.Stdout(..): True
    Process.Exit(..): True
    ignore: False), $ps)
!fwd_c <- @Forward (Except (\|e| match e:
    Cli.Stdin(..): True
    ignore: False), $cs)
let !down_p = !fwd_p
@ unit
```

The producer's write is captured by the first `for` (it never reaches the World) and queued under a sequence number. The second `for` builds a block per queued chunk, and each block has its own `Defer`, fired from `@started` in the occurrence the chunk was queued and answered in the next; the block then fires the chunk, the consolidated `!delivered` goes to the consumer's stdin slot, and the key leaves the queue. One `Defer` per delivery keeps the pipe inside the turn rule: a producer that writes in two occurrences of one turn (possible if it is itself fed by a `Defer`) makes two blocks and two `Defer` slots, each answered once, instead of one slot asked to answer twice. `StepQueue` folds `Enqueue` and `Dequeue` changes into the map, and the `!Union` means a write and a delivery landing in one occurrence are both applied. `StdinPath` is a plain function over the consumer's list, `Except` is `@Forward` with a filter, and the consumer cannot tell that its standard input is another program rather than a terminal, which is the test.

## A window manager

The shell with a screen. Each application is an `intercept` whose `Gui.Window` and `Gui.Scene` are held back and replaced by a part of one big scene: the manager wraps the application's draw list in a `Draw.Clip` at the application's offset, and the manager's own hit-test routes pointer events whose point falls in that frame to the application's window slot with the point translated. Focus is a hold in the manager moved by clicks and a key chord; keys go to the focused application's `Gui.Window` slot, and the application's own library routes them by focus name. The manager emits one `Gui.Window` and one `Gui.Scene` to the World below, and the World below is the software rasteriser, or the X11 interceptor over a socket World, or a native toolkit wrapper showing one native window. None of them knows there are several applications.

## Sandboxing third-party widgets

A date picker you downloaded has, in most languages, the same ambient access you do: `std::fs` is an import away. Here it has nothing: every file it wants is an element in its list, and that element reaches the World only if some interceptor above it makes one. Wrapping it in `@Sandbox` (chapter 2) confines its `Fs` to a directory and denies the rest, one occurrence after each request, and the widget's code does not change and cannot tell. The guidance for third-party widgets is: wrap them.

The standard library ships `@Sandbox`, `@Confine` for a namespace list, and `@Deny` for everything, and a program that forgets is no worse off than any program in any language, while a program that remembers has something no other language gives: confinement per subtree, in the program, with no process boundary.

The thing a capability system has to get right is that nothing below the sandbox can reach above it except through the sandbox. Here that is structural: there is no global, no ambient World, no event bus, and the only upward channel is the element list, which the sandbox reads in full. What the sandbox must not do, as in any capability system, is turn the child's data into authority: a sandbox that forwards a file request with the child's path unchanged has given the child the filesystem.

### Which World is at the bottom

The outer World under a nested stack is smaller than the one you would have written without the stack, because the stack did the multiplexing. A shell needs a CLI World; a window manager needs a World with one window; a test needs the Test World. The language promises that an interceptor which re-emits everything and supplies by forwarding is observably not there, so each layer can be put in and taken out, and the children run under the Test World with the layer or without it, which is how a window manager is tested without a screen.

## Worlds and parallelism

The model has three kinds of parallelism, and a World is involved in exactly one of them.

*Within an occurrence.* Every node at one height is independent of every other at that height, so propagation can fork-join per height level. That is a runtime variant with `Sync` graph types, and a World drives it through the same run call on the same one thread. The World cannot tell.

*Across occurrences.* An occurrence is a transaction with buffered writes, so two occurrences can run optimistically and the one whose read set overlaps the other's write set retries. That is a runtime variant too, and from the World's view the run call still returns outcomes in order. The World cannot tell either, except that it may be asked to supply two occurrences' inputs before the first has returned, which changes the call's signature to a submit-and-collect pair and nothing else.

*Across the program.* Independent subtrees connected only through elements can be separate runtimes, each on its own thread, with a World that routes between them. This is the server above, and it is the only parallelism a World sees, because the World is the one doing it. It needs no runtime support beyond the run call being cheap, and the no-callback rule makes it free: a World with N runtimes is N single-threaded run calls and a routing table, and nothing crosses between runtimes but plain data through the World.

The World's own parallelism, its IO threads and tasks, is also free for the same reason. A World in which the runtime called back into World code would have to decide which thread the callback ran on and what it may touch; a World that is only ever called, by its own loop, after a turn, has no such question.

## Testing a World

The conformance suite tests the language under the Test World and says nothing about any other World. A World is tested separately, in its host language, against the world core, and the tests are

the obligations turned into assertions. Most of them run against the core with fake components and no platform at all, which is why the core is a library: the loop, `Defer`, the snapshot and the discard rule are tested once, and a platform World tests only what the platform adds.

## The core, with fakes

| Test | What it catches |
|---|---|
| A response to a path not in the last list is dropped and logged, never passed to `run`. | A World addressing something the program does not have. |
| A `Defer` fired in occurrence k is answered in k+1 of the same turn; `fired` on the child's side shows the answer in the turn. | `Defer` answered in its own occurrence, or deferred to the next turn. |
| A `Defer` that fires again after being answered ends the turn with an error naming its path. | Deferring backwards passing as a loop. |
| A `rebuild` built anew by a `Defer` answer that fires a new `Defer` from `@started`, three times. | The generation rule not holding; a turn that never ends. |
| Random programs with D `Defer` slots: every turn runs at most 1 + D occurrences. | Any path to an unbounded turn. |
| A turn with two `Defer`s that fire in occurrence 1 runs three occurrences, and both answers land in occurrence 2. | Deliveries split that should land together. |
| Two components report changes to one value before a turn; the turn has one push with the last value. | State pushed twice in an occurrence; pushes not coalesced. |
| A slot created in the occurrence of a push ends the occurrence with the pushed value, and a tag read in that occurrence sees the oracle's value. | The oracle answering from $S_n$; the push not reaching a new slot. |
| Two slots with equal elements created in different occurrences hold equal values in every occurrence. | The equality law broken by the core's bookkeeping. |
| The oracle is called for a value no component owns: `UnsuppliedState` in the creation occurrence, with the span. | A state tag silently defaulting. |
| A request on an unclaimed tag gets `Unsupported` next turn; a presence element on an unclaimed tag is logged once. | Portable code waiting forever. |
| A request fired by a slot torn down in the same occurrence is handed to `perform` with its path; its response, if any, is discarded. | Firings lost because the slot went; responses to nothing. |
| A component's `reconcile` sees a slot appear once however many occurrences the turn had, and never sees one that appeared and vanished inside a turn. | Resources opened per occurrence; resources opened for slots the outside never saw. |
| `run` returns `Err`; the turn's inputs are dropped; the next turn's pushes are the diff from the last committed snapshot. | A failed turn's inputs replayed; a snapshot that lost the world's movement. |

| Test | What it catches |
|---|---|
| The same inputs, twice, from a fresh core: identical outcomes. | Anything in the core that depends on time or thread order. |

### A platform World

Each World adds the tests only its platform can run. For the CLI World: golden runs with fixed arguments and standard input (an echo, the line counter, a program that writes to both streams in one occurrence with the order kept, `Process.Exit` fired in the same turn as writes with the writes landing first, `SIGINT` with and without a `Cli.Signals` slot, a closed standard-output pipe producing `WriteFailed`, a `Cli.Stdin` slot dropped mid-read and rebuilt, with the chunk read in between delivered to the new slot first, a program with no live sources exiting zero). For a GUI World: a scripted click at a point reaching the right widget through the library's hit-test, a resize turn that paints before the callback returns, a composition sequence reaching the window as `Composition` keys with the candidate window placed at the `ime_at` offset read from the committed scene, a `Gui.Frames` slot inside a `rebuild` starting and stopping frame requests. For a server World: concurrent requests under one `for`, one of them waiting on an outgoing call, with the replies landing under the right ids, and a routing World with two shards that never deliver each other's connections.

What none of them tests: the language. A World test never asserts on a hold or a widget's result; it asserts on what crossed the boundary, in both directions, and on what the World did with it.

## Negative space

What a World may not do, and what each refusal buys. The list is the design.

**No callbacks across the boundary.** The runtime never calls the World, and the World never hands the runtime a function (the oracle is called during one run call and forgotten). What it buys: the World runs on whatever thread the platform dictates, with IO wherever it likes, because nothing can reach the runtime from the wrong thread; a World is replaceable without the program noticing; a session is a transcript of plain data and replays exactly; the Test World exists.

**No runtime objects in a World.** No event handle, no dynamic, no widget, no slot. What it buys: a World written in an afternoon, with no lifetime to track and nothing to leak; a World that cannot address what the program has not told it exists, because it has only paths from a list it was given; the sharded server, where a runtime moves to a thread and nothing follows it.

**No phases.** One "later," the next occurrence of this turn, through `Defer`. What it buys: a turn bounded by the program's own structure rather than a configured cap; no ordering table for stages; no stage-dependent behaviour that differs between Worlds; a GUI library that runs the same under a test.

**No presence triggers.** Nothing happens because an element appeared or a widget was dropped. The list is a description the World reconciles to; one-shot effects are request firings. What it buys: reconciliation is idempotent, so a rebuild costs exactly what it asks for and never a surprise fetch; the World never watches for appearances; exit, writes and spawns are events a program can merge, gate and mock.

**No World-side sampling of the program.** The World reads the list after a turn and nothing during one; there is no "what is `$count` right now" for a World any more than for a widget. What

it buys: the occurrence is atomic from outside, so a half-finished turn is never drawn, written or answered.

**No dynamic fields in elements.** An element is born with its fields and keeps them; a change is a fresh slot. What it buys: the boundary never sees a sigil, value identity means what it says, and the World diffs lists of plain data instead of subscribing to anything.

**No policy in the World that a program might want to choose.** Retries, caches, book-building, framing, focus, layout: Prism. What it buys: every policy is in the program's text, testable under the Test World, and replaceable per subtree by an interceptor; the World stays the size of its hardware.

**No scheduler, no priorities, no budget.** The World performs firings in the order the program fired them and runs occurrences in the order inputs arrived. What it buys: two Worlds reading one outcome do the same things; the transcript is the specification of the run.

**No clock inside.** Time is a state element the World pushes. What it buys: a debounce is a test with a `push`; a transcript replays with the same timestamps; a program cannot ask for a finer time than the World gave it and so cannot depend on one.

The pattern is the language's own: each thing a World cannot do is a thing that would have made the boundary a second set of rules, and each removal is also a mechanism the World never has to build. A World is small because the list above is long.

## What a World is, in one paragraph

A World is a host-language program that owns a runtime and a loop. It builds `@main` in occurrence zero. It keeps a snapshot of every state value it has supplied and answers the oracle from it. It turns what the outside does into responses by path and pushes by value, runs one synchronous occurrence per turn plus one per fired `Defer`, performs the request firings in order, reconciles the world to the element list, and waits. It discards what it cannot address and reports what it cannot run. It accepts some tags and answers `Unsupported` for the rest. Everything else, from a button to a window manager, is Prism, and would be the same Prism under any other World that accepted the same tags.

## Hot reload of a widget subtree

You are tuning the padding on a dialog that takes six clicks to reach. Each change is edit, rebuild, restart, six clicks. You want to change the dialog's code and see the running program pick it up with its state intact. Prism has most of this already, because `rebuild` replaces a subtree when a dynamic changes and nothing in the language cares what the new subtree's code is. Hot reload is making the code a dynamic.

### The design

Two parts, neither of which touches the language.

A World component. A development World offers a state tag whose value is compiled code:

```
tag Dev.Code of Element:
    name: Text
    state: Unavailable | Known Code
```

`Code` is an opaque plain value, a handle to a compiled `IrBlueprint` constructor, and `Dev.Code(name="Dialog")`

is the current compiled form of the top-level widget named `Dialog`. When the file changes the World recompiles the definition (incrementally, if the frontend above exists; from scratch otherwise), and pushes the new `Code` as a state change in the next turn. It is a state element because it is a value that changes and whose current value every slot asking for it must see the same: two `@Reloadable "Dialog"` on screen reload together.

A library widget, in two layers. The inner one is the reload itself:

```
@Live: fn Text, @a -> @!a = \|name, @fallback| @widget:
    $code <- @Element Dev.Code(name=name)
    !built <- rebuild $code:
        match _:
            Known c: @Build c
            Unavailable: @fallback
    @ !built
```

`@Build: fn Code -> @a` is a primitive that builds the compiled widget the handle names, and is the only new primitive, a native in the closure compiler's table. Under the development World, every save of the file rebuilds the subtree under the `rebuild`, in the occurrence of the push, with the new code, and `!built` hands the parent the new subtree's result each time; a parent that wants a continuous event or dynamic across reloads puts a `!Switch` or the hold-and-`$Join` idiom over it, as it would over any `rebuild`.

The outer layer is what makes the same program run under a production World, and it cannot be left to tolerance. A World that does not accept a state tag reports an unknown state tag as an error at the slot's creation, which is the obligation that keeps a misspelled tag from being quietly `Unavailable` forever; so a production World handed `Dev.Code` would abort the build, and `@Live` on its own is a development-only widget. The library therefore asks first. `World.Tags` is the core-provided state element listing the tags the World accepts, valued in its first occurrence like any state element, and `@Reloadable` reads it before it creates anything:

```
@Reloadable: fn Text, @a -> @!a = \|name, @fallback| @widget:
    $tags <- @Element World.Tags
    !gen <- rebuild $tags:
        match Accepts _, "Dev.Code":
            True: @Live name, @fallback
            False: @Once @fallback
    @ !Switch !gen

@Once: fn @a -> @!a = \|@w| @widget:
    !built <- rebuild (Const unit): @w       # the construction @started is: a rebuild over a constant, built
    once
    @ !built
```

`Accepts: fn [Text], Text -> Bool` is a plain function over the tag list. Under a production World `$tags` lacks `Dev.Code`, the `rebuild` builds the `False` arm, and `Dev.Code` is never created, so no obligation is tested and nothing is tolerated: the production path is the fallback compiled into the program, built once and never rebuilt. Under the development World the `True` arm builds `@Live`, and the `!Switch` over the outer `rebuild`'s event flattens the two layers so the parent sees one `!a` either way. The tag list does not change during a run, so the outer `rebuild` builds once; it is a `rebuild` and not a `match` on a plain value only because `World.Tags` is a state element and its value arrives as a dynamic.

The typing is the part that needs a sentence. `@Build c` returns `@a` for whatever `a` the handle's widget returns, and the handle is opaque, so the check is at runtime: the World knows the compiled widget's result type from the typed AST, the `Reloadable` site knows the `a` it expects, and a mismatch is a

build error in the occurrence of the push, reported like any build error, with the program continuing to run the old subtree. That is the construction abort from the errors chapter doing its job for a subtree that was being reborn.

### What survives and what does not

Holds inside the reloaded subtree reset, because a `rebuild` firing makes new holds with their initial values. That is the semantics, and it is the honest answer: the new code may have changed what the holds mean. State that should survive a reload lives outside the `rebuild`, which is where state that should survive any rebuild already lives, and a program written for reload keeps its model above its views for the same reason a program written for `rebuild` does. A `$Join` over a hold of the subtree's result dynamic gives the parent a continuous value across reloads.

Suppose instead the runtime tried to migrate holds by name across the rebuild, matching the old widget's holds to the new widget's by binding name and copying values where the types agree. It would make reload feel better in the common case, and it would make it lie in the uncommon one: a hold whose meaning changed but whose name and type did not would start the new code in a state the new code never produced. Reload is `rebuild`, nothing more, and that is a feature: the behaviour is the one the program already has and the test suite already checks.

### What it must not change

Nothing in the language; a program with `@Reloadable` in it is an ordinary program with a `rebuild` over `World.Tags` and, under a World that lists `Dev.Code`, a second `rebuild` over a state element; under any other World it is the fallback, built through `@Once`, and `Dev.Code` is never created. The production World core need not know `Dev.Code` exists, because the library never asks it to; a World that wanted to accept the tag and always answer `Unavailable` could, and `@Live` would then run under it with the fallback, but nothing requires that and the library does not rely on it. Paths are the other thing to say once: a reload is a `rebuild` firing, so every slot under it gets a fresh generation segment and a fresh path, and a World holding a native widget keyed by path (a GTK button for a `Ui.Button` it accepted directly) sees a teardown of the old path and a creation at the new one, exactly as it does for any rebuild. The World discards late responses for the old paths under its obligation for gone slots, and nothing about reload asks it to recognise the new slot as the old one.

### The fact it waits on

A GUI World and a program big enough that the restart is noticed. The whole thing is a World component, a prelude widget and one native, so it can be done the week the GUI World works.

## Persistence of holds

Close the program and open it again and everything is gone: the document, the scroll position, the half-filled form. Every application solves this and most solve it twice, once for the real model and once for the dozen pieces of UI state nobody thought of until a user complained. In Prism all state is holds and every hold is owned by a widget with a path, so there is a temptation to snapshot the whole tree, write it out, and read it back. The right design is smaller than that and goes through the element list; the whole-tree snapshot is a different thing, a World feature for hosting programs, and is taken up after it.

## Persistence through elements

A hold that should persist is a hold whose initial value comes from outside and whose changes go outside. Both halves are elements, and the worlds chapter declares them: `Store.Value(key)` is a state element whose state is `Stored, Absent | Present Bytes`, and `Store.Put(key, requests=!Bytes)` is a request element responding `PutResult`. A `Store.Value` slot has, in the occurrence it is created, the value the store holds for that key, because it is a state element and state is there in its first occurrence; a `Store.Put` firing writes a new value and is answered in a later occurrence. A library widget makes a persistent hold out of the pair:

```
@Persisted: fn Text, a, !a -> @$a = \|key, default, !change| @widget:
    $stored <- @Element Store.Value(key=key)
    !written <- @Element Store.Put(key=key, requests=!write)
    let $loaded = match $stored:            # lifts over $; valued in occurrence zero
        Present b: Decode b
        Absent: default
    $$current <- $loaded then Const !change   # a hold of dynamics
    let !write = Encode !change
    @ $Join $$current
```

The hold is the cache idiom, a hold of dynamics joined with `$Join`, and the reason is occurrence zero. Suppose instead the widget held a plain value, `$value <- default then !loaded | !change`, with `!loaded` read from `$stored on !first` through a tag at `@started`. That reads the oracle's answer correctly, but the hold commits at the end of occurrence zero, so inside occurrence zero `$value` is `default`: a `rebuild` over `$value` in the parent builds under `default`, that slot is in the element list the World reads after occurrence zero, and the `rebuild` builds anew in occurrence one with a fresh path. A one-occurrence hiccup, visible in the transcript as a slot that exists for one turn under the wrong value. With a hold of dynamics the initial value is `$loaded` itself, a derived dynamic that is prompt on `$stored` and so has the decoded stored value from the first instant of occurrence zero; `$Join` follows it, and nothing anywhere sees `default` unless the store was empty. `Const: fn a -> $a` is the prelude's constant dynamic, lifted over `!change`, so each change replaces the inner dynamic with a constant one holding the new value. The widget's contract is "start from what was saved, then follow the program's edits": before the first edit it follows the store, which is right for a document another program may be writing; after the first edit it is pinned to this program's values. A widget that wants to follow the store after edits too is a different widget, a `rebuild` over `$stored`, and a program that wants the one-shot stored value as an event still has `$stored on !first`, the tag read at `@started`. A program then writes `$doc <- @Persisted "doc", EmptyDoc, !edit` and the document survives a restart. `!written` is the `PutResult` stream; this widget ignores it, and a stricter one surfaces it. `Encode` and `Decode` are the program's; a `Show`-like trait pair would make them derivable once traits exist.

Why this and not a tree snapshot. Three reasons, each a law.

A hold's identity is its path, and paths change when code changes: insert a bind above the hold and its path moves. A snapshot keyed by path is invalid after any edit, and a program that persists state across versions of itself needs a stable key, which is what `key` is. The program names what persists, which is the goal about one way to change anything written next to the thing: the hold's line says it is persisted, under what key, and nothing else in the program can make a hold persist.

Nothing happens because a widget exists. A tree snapshot taken by the World "when the program exits" is an effect caused by the program's structure, which is the thing the language refuses. A `Store.Put` write is a request fired by an event the program named.

Persistence is a World policy. Whether `Store` is a file, a browser's local storage, a database or a test

fixture is the World's business, and a sandbox interceptor can give a subtree a different store or no store by supplying `Store.Value` itself, which is how a test runs a program against a known saved state: the Test World's script sets the oracle's answer for `Store.Value(key="doc")` with a `with` step before the build, the program starts from it, and a `requested` step checks the `Store.Put` firing after an edit.

### Whole-program snapshots

There is a second thing the word means, and it is a World feature rather than a program one. A World that hosts programs, a shell, a server that migrates sessions between machines, a debugger, wants to freeze a running program and resume it later, code and all. That is a snapshot of the runtime: every node's state, every hold's value, every switch's target, the element list, keyed by path, plus the fingerprint of the typed AST that built it.

It is possible because of what the runtime does not have. No threads, no pending effects, no closures with hidden state: between turns a runtime is a tree of nodes and plain values, and a hold's value is a plain value, an event or a dynamic, each of which is either data or a node reference that the tree also owns. Serialising the tree is a walk. Resuming is building the program's `@main` under a fresh runtime and overwriting every hold's value and every switch's target from the snapshot before the first occurrence, which is legal because occurrence zero has no inputs and the snapshot was taken between turns, when every commit was done.

Its constraints are the ones the path-keyed design has: the typed AST must be the same, since paths are bind positions, and the snapshot is a World's artefact and not a program's, invisible to the program and not part of any transcript. Closures in values are the one representation question: a closure is a code id plus captures, and the code id is stable across runs of the same typed AST, which is one more reason the fingerprint is part of the snapshot.

### What it must not change

The creation rule: a `Store.Put` slot does not write in the occurrence it is created, and a `Store.Value` slot's state is the oracle's answer, never a value the program pushed in that occurrence. No hold persists without a line saying so. A whole-program snapshot is a World feature that produces a runtime indistinguishable, to the program, from one that never stopped.

### The fact it waits on

Element-level persistence: the first program with a document. It needs a `Store` component in the CLI World and the GUI World and a prelude widget, which is an afternoon once the world core exists. Whole-program snapshots: a World that hosts programs and wants to migrate or debug them, which is far off and is listed so that nothing done before it makes it impossible.

## Real time: a tick World and a bounded occurrence

A motor controller reads three sensors every millisecond and writes one duty cycle, and the one question its author has is whether the step can ever take longer than a millisecond. In Lustre or Esterel the answer is a number: the program is a fixed graph, a step is one pass over it, and the compiler can add up the pass. Prism's occurrence is the same pass with the same order, heights within a block, and the question is what has to be true of a program for the same number to exist.

Three things, each a restriction a World can impose rather than a change to the language. Closures are total: no general recursion in plain code, or recursion with a fuel the type states, so that every leaf has a bound. `Defer` is off: a turn is one occurrence, and the World refuses a program that creates a `Defer` slot, so a turn's work is one pass. The tree is fixed: no `rebuild` and no `for`, or a `for` whose key count has a stated maximum, so that the set of nodes an occurrence can touch is known before it runs. Under those three the work of an occurrence is the sum over the prompt cone of its inputs of the leaves' bounds, which a tool can compute from the typed AST, because the typed AST already has every node's height and every lift explicit.

The World that fits is a tick World. It pushes a snapshot of its state elements every tick (the sensors as `state`, read through the oracle) and delivers no responses at all, because a controller has no edges, only levels: it does not react to a sensor changing, it reads every sensor every time. That is the level-versus-edge story from the Worlds chapter with the edge half removed, and it is how every control loop already works. Outputs are requests on a sink element, fired every tick, and the World performs them after the pass. There is no `Clock` to push, since the tick is the clock.

What this buys is the synchronous languages' guarantee with Prism's type system, and one thing they do not have: the same program, unrestricted, runs under the Test World and the GUI World, so a controller's logic is tested by script and shown on a screen without a second implementation. What it costs is that the program cannot build structure while running, which is the thing Prism adds to the synchronous family and the thing a hard-real-time program does not want anyway.

It needs work in two places. A fuel-bounded recursion form, or a total-functions lint, does not exist, and without it "closures are total" is a promise the author makes rather than one the compiler checks. And the bound itself needs a tool that walks the typed AST and prints the worst-case node count per input, which is small once the restrictions hold and does not exist.

## Chapter 8

# Use cases in depth

Eight programs that are not toys, each taken far enough that the hard moment shows up: the file that changes on disk in the occurrence the user presses undo, the fill that lands with the book update that would have re-quoted it, the emergency stop that arrives with the tick, the member who leaves the room in the moment someone speaks to it, the source file that changes while its compile is finishing, the window that quits under the click that raised it, the jump pressed on the frame boundary, the time range that changes while the old range's data is arriving. For each: the element vocabulary it needs, declared as tags; the widget structure, as code that could compile; the walkthrough of that one moment at the occurrence level; what the goals buy that you would otherwise pay for by hand; and the part of the problem where Prism is slow, awkward, or the wrong tool, said plainly. The chapter ends with the programs Prism is the wrong language for, and what a Prism program does with them instead of being them.

## Conventions

Occurrences are numbered from zero; occurrence zero is the build. Paths are written as in the worked examples: bind positions from the root separated by `/`, `#n` for a `rebuild` generation, `[k]` for a `for` key, and `s/n` for a slot that an interceptor's supplier built. Element lists are written one slot per line as `path: element`, with `requested:` after a slot that fired its request in that occurrence, then the fired values bare, in firing order, separated by `;`.

The GUI vocabulary is the two-level one. `Gui.*` is what a World that draws pixels accepts, five tags: `Gui.Window(title)` responds with pointer, key and resize events and has the window's size as state; `Gui.Scene(window, draw)` is the picture for a window, a presence element whose `draw` field is a plain list of `Draw.*` values (`Draw.Rect`, `Draw.Text`, `Draw.Image`, `Draw.Clip`), replaced whenever the picture changes; `Gui.Focusable(name)` responds with keys; `Gui.Metrics(text, font)` is text measurement as state; `Gui.Frames` responds `Frame(now: Instant)` once per frame while a slot exists. The World does no geometry and no hit-testing. `Ui.*` is the Prism GUI library's own vocabulary (`Ui.Button`, `Ui.Label`, `Ui.TextInput`, `Ui.VBox`, …), claimed by the library's interceptors on the way out, which hit-test against their own layout and focus by name, and reaching a World only when that World accepts them, which the Test World does. Programs in this chapter are written against `Ui.*` and assume `@Window` from the GUI library around them where a screen is involved; `@Button`, `@Label` and `@TextInput` are the library widgets over `Ui.Button`, `Ui.Label` and `Gui.Focusable`, and `Ui.Button` responds `Click`, a unit type.

```
type Click = click
type Tick = tick

tag Ui.Label of Element:
    text: Text

tag Ui.Button of Element:
    label: Text
    responds: Click
```

```
tag Gui.Focusable of Element:
    name: Text
    responds: Key

@Label: fn $Text -> @unit = \@widget:
    @Element Ui.Label(text=$_)
    @ unit

@Button: fn $Text -> @!Click = \@widget:
    !click <- @Element Ui.Button(label=$_)
    @ !click
```

A dynamic where a field goes lifts the element into a `rebuild`, a fresh slot per change, and a bound event is the `!Switch` over the generations; a bound state is a hold of dynamics joined with `$Join`, starting from the first build's value. The chapter uses that shorthand throughout and spells the `rebuild` out only where the generations matter to the walkthrough.

The prelude of plain functions from the worked examples is assumed (`!Filter`, `!Union`, `!Leftmost`, `Insert`, `Remove`, `Fold`, `MapValues`, `Keys`, `Values`, `Concat`, `Len`, and the rest; comparisons are the infix operators `<`, `<=`, `>`, `>=`, `==` and `!=`), plus two that this chapter leans on: `!Updated: fn $a -> !a`, the change event of a dynamic, firing with the new value in every occurrence the dynamic changes and never at build; and `$Distinct: fn $a -> $a`, which suppresses a change to an equal value, since derived dynamics and holds never do that on their own. Every other function a program uses is defined before its first use, or is so plainly a pure function of its arguments (`Mid`, `Pid`, `Render`) that its body would say nothing about time.

Every deep dive below hits one rule from the worked examples: an operation over one event and any number of dynamics fires with the event and reads each dynamic through a tag, as it stood before the occurrence; an operation over two events is a compile error. When a program wants two things that can land together to both count, it says so, with `!Union` over maps or with one event of a record, and the walkthroughs below are mostly about moments where that choice is the whole bug. ## 1. A text editor

You have used an editor that lost an undo step because the file changed on disk at the wrong moment, and one that marked a buffer clean after a save that had not finished, and one that asked "reload from disk?" about a change it had made itself. Every one of those is a state with more than one writer and no rule about which write wins. An editor is a good first deep dive because it has exactly the states a GUI framework claims to handle for you (the text, the cursor, the undo stack, the dirty flag, the search) and exactly the outside events that it does not (a file changing under you, a save completing), and because the hard moment is where the two meet.

## The vocabulary

```
type Key = choice:
    Char Char
    Left
    Right
    Backspace
    Enter

type Instant = Instant Int
```

```
type Stamp = choice:
    NoStamp
    Modified Instant

type FsRequest = choice:
    Read
    Write Text

type FsError = choice:
    NotFound
    Denied
    TooLarge
    Other Text

type FsResponse = choice:
    Contents Text
    Written
    Unavailable FsError

tag Fs.Watch of Element:          # state: the file's modification stamp
    path: Text
    state: Stamp

tag Fs.File of Element:           # request: reads and writes, answered later
    path: Text
    requests: !FsRequest
    responds: FsResponse
```

Three elements talk to the outside for each buffer. `Gui.Focusable(name)` hears keys when it is focused; which buffer is focused is the window interceptor's business, and the buffer never knows. `Fs.Watch(path)` is the watcher, as *state*: its value is the file's modification stamp, pushed when the file changes. `Fs.File(path)` is the one-shot conversation: a `Read` or a `Write` fires, and `Contents`, `Written` or `Unavailable` with a reason comes back addressed to this slot, later. The watcher is state and not a stream for the reason the Worlds chapter gives: a slot built in the occurrence of a change sees the push, while a slot built in the occurrence of a stream's firing would miss it. An editor that opens a file in the moment it changes wants the change.

## The structure

A document is a plain record, and editing is a plain function from a document and a key to a document. The undo stack is a plain record too, and everything the buffer can do to it is one closed choice:

```
type Doc = record:
    text: Text
    cursor: Nat

type History = record:
    past: [Doc]
    now: Doc
    future: [Doc]

type Cmd = choice:
    Reload Text
    Undo
    Redo
```

```
    FindNext
    Edit Key

let Apply: fn Doc, Key -> Doc = ...          # the text input of chapter 2, plus Enter

let Step: fn History, Text, Cmd -> History = \|h, query, c| match c:
    Reload t: History [[]], (Doc t, (Min h.now.cursor, (Len t))), [[]]
    Undo: match (Last h.past):
        Some d: History (Init h.past), d, (Prepend h.now, h.future)
        Nil: h
    Redo: match h.future[0]:
        Some d: History (Append h.past, h.now), d, (Tail h.future)
        Nil: h
    FindNext: match (NextMatch h.now.text, query, h.now.cursor):
        Some i: History h.past, (Doc h.now.text, i), h.future
        Nil: h
    Edit k: History (Append h.past, h.now), (Apply h.now, k), [[]]
```

`Step` is where every decision about the document lives, and it has no time in it: it can be tested with a list of commands and nothing else. The buffer widget is the time around it:

```
let ContentsOf: fn FsResponse -> ?Text = \|r| match r:
    Contents t: Some t
    ignore: Nil

let ReasonOf: fn FsResponse -> ?FsError = \|r| match r:
    Unavailable why: Some why
    ignore: Nil

let Problem: fn ?FsError -> Text = \|p| match p:
    Some NotFound: "no such file"
    Some Denied: "not allowed to read it"
    Some TooLarge: "too large to open"
    Some (Other t): t
    Nil: ""

let Unless: fn Text, Text -> ?Text = \|saved, t|
    if saved == t: Nil else: Some t

let Title: fn Text, Bool -> Text = \|path, dirty|
    if dirty: Concat [[path, " *"]] else: path

@Buffer: fn Text -> @!unit = \|path| @widget:
    !started <- @started
    !key <- @Element Gui.Focusable(name=path)
    !undo <- @Button $"Undo"
    !redo <- @Button $"Redo"
    !save <- @Button $"Save"
    !close <- @Button $"Close"
    $query <- @TextInput ""
    !next <- @Button $"Find next"

    $stamp <- @Element Fs.Watch(path=path)
    !resp <- @Element Fs.File(path=path, requests=!io)
    let !io = (Read on (!started | !Updated $stamp)) | (Write $doc.text on !save)
    let !loaded = !Filter (ContentsOf !resp)
    let !reload = !Filter (Unless $saved, !loaded)
    let !failed = !Filter (ReasonOf !resp)
    $problem <- Nil then (Some !failed) | Nil on !loaded
```

```
    $saved <- "" then ($doc.text on !save) | !reload

    let !cmd = (Reload !reload) | Undo on !undo | Redo on !redo | FindNext on !next | (Edit !key)
    $hist <- History [[]], (Doc "", 0), [[]] then Step $hist, $query, !cmd
    let $doc = $hist.now
    $dirty <- False then True on !key | False on (!save | !reload)

    @Label (Title path, $dirty)
    @Label (Problem $problem)
    @Label (Render $doc)
    @ !close
```

The holds are the whole specification of what can happen to a buffer. `$hist` moves on `!cmd` and nothing else, and `!cmd` is a merge whose order is the program's answer to every "what if two of these land together": a reload beats an undo, an undo beats a redo, a search beats a keystroke. `$saved` is the text the buffer last knows to be on disk: what it wrote, or what it loaded. `$dirty` is set by a keystroke and cleared by a save or a reload, and the merge there matters: a keystroke and a save in one occurrence leave the buffer dirty, which is right, because `Write $doc.text on !save` wrote the text *before* the keystroke by the tag law, and the document after the occurrence has one more character than the disk.

`!io` is the only way the buffer talks to the file, and it has three sources. `@started` reads the file once when the buffer is built; a change of the stamp reads it again; a click on Save writes it. The watcher's value at creation comes from a real stat, so `@started` reads the file once and the stamp's first push is a real change.

`Unless` is the line that stops the editor asking about its own saves. A `Write` lands on disk; the watcher pushes a new stamp; the buffer reads; the contents come back equal to `$saved`; the reload is dropped. A change by another program comes back different and is applied. There is no "ignore the next watch event" flag, which is the fix you have written and the fix that has a race in it, because the next event is not always yours. Comparing contents is the test that cannot be wrong about whose change it is.

Buffers are a map keyed by path, so opening a path twice is one buffer, and closing one tears its widget down with its history, its watcher and its pending reads:

```
type Change = choice Open | Close

let ApplyChanges: fn Text ==> unit, Text ==> Change -> Text ==> unit = \|bs, cs|
    Fold cs, bs, \|acc, p, c| match c:
        Open: Insert acc, p, unit
        Close: Remove acc, p

@editor:
    $path <- @TextInput ""
    !open <- @Button $"Open"
    let !opens = {$path: Open} on !open
    !closes <- for $buffers:
        !closed <- @Buffer key
        @ Close on !closed
    $buffers <- {} then ApplyChanges $buffers, (!Union !opens, !closes)
    @ unit
```

The cycle through `$buffers` is the todo list's: `ApplyChanges` reads the map through a tag, and a row's close is a click, which cannot arrive in the occurrence the row was born. `!Union` rather than `|` so that opening one file and closing another in the same occurrence both count, and if the same path is opened and closed at once the left side, the open, wins, which the program chose by writing `!opens`

first.

## The hard moment

The buffer has `"v1"` loaded and saved. The user types `x`, so the document is `"v1x"`, dirty, with one step of history. Another program writes `"v2"` to the file.

**Occurrence 10.** The World pushes `Modified (Instant 7000)` for `Fs.Watch(path="notes.txt")`. `$stamp` changes; `!Updated $stamp` fires; `Read on ...` fires, which is the `Fs.File` slot's request. Nothing else happens. The World reads the file after the turn.

**Occurrence 11.** The read completes, and the user presses Undo in the same frame; the World delivers both: `Contents "v2"` to the `Fs.File` slot and `Click` to the Undo button. The two are responses to different slots, so the World is entitled to put them in one occurrence, and it has. `!loaded` fires with `"v2"`; `$saved` is `"v1"` through a tag; `Unless` passes it; `!reload` fires. `!undo` fires too. Now `!cmd`: its leftmost firing operand is `Reload "v2"`, so `!cmd` fires with that, and the undo is gone. `Step` replaces the history: no past, `Doc "v2", 2`, no future. `$saved` fires with `"v2"` (its left side, the save, is quiet). `$dirty` clears. Commit: the buffer shows `"v2"`, clean, with nothing to undo.

That is the program's answer and it is defensible: a change on disk is a fact, and the local history was about a text that no longer exists. Three other answers are each one line away, and this is the point of writing them down.

Swap the merge, `Undo on !undo | Reload !reload`, and in occurrence 11 the undo applies to `"v1x"`, giving `"v1"`, and the reload is lost from `!cmd`. But `$saved` has its own definition and still takes `"v2"`, and `$dirty` still clears on `!reload`. The buffer now shows `"v1"`, marked clean, with `"v2"` on disk. Two holds listened to overlapping events and each decided alone; the program said so, and the bug is in its text, not in a race.

Use `!Union` instead, so both count: make `!cmd` a map and fold it, `{0: Reload !reload}` unioned with `{1: Undo on !undo}`, applied in key order. The reload runs first and empties the past, so the undo is a no-op. Same screen as the first answer, by a route that generalises: a program with many simultaneous commands folds them all and never loses one.

Or decide that a reload should not throw history away: `Reload t` keeps `h.past` and makes the reload one more undoable step. Then an undo after a reload restores `"v1x"`, dirty, and a save writes it over `"v2"`. Some editors do exactly this. It is a product decision, and the language's contribution is that it is one `match` arm, in the function with no time in it, rather than a flag checked in two handlers.

## What the goals buy

One way to change anything, written next to the thing: `$hist`'s line is the complete list of what can move the document, and the merge on `!cmd` is the complete ordering of simultaneous commands. There is no handler for the watcher that reaches into the history and no handler for undo that checks a reloading flag.

Simultaneity defined by the program: occurrence 11 is the whole of this section, and its outcome is read off one line.

"Not yet" as a type: a buffer under a slow disk shows an empty document until the first `Contents` arrives, and the arm for `Unavailable` and its reasons exists because the response type made the program write them. The watcher is not part of that wait: it is answered from a real stat as its slot is created, so it says `NoStamp` only for a file that is not there. There is no spinner state that has to be cleared on three different paths.

The language is enough: the undo stack, the dirty flag, the echo suppression and the search are plain data and plain functions; the only things native are the focusable region, the file and the watcher.

The spec is the thing: the Test World runs this editor with `push Fs.Watch(path="notes.txt"), Modified (Instant 7000)` and `send Fs.File(path="notes.txt"), Contents "v2"` on one line with the Undo click, and the test for occurrence 11 is four lines that a second implementation must agree with.

## Where it is awkward

The text is a `Text`, a plain value, and `Apply` builds a new one per keystroke. For a file of a few thousand lines that is a copy per keystroke and nobody notices; for a hundred-megabyte log it is the whole problem. The answer is a rope or a piece table, which is a plain data structure the prelude can provide and `Apply` can be written over, with no change to anything that has time in it. Until there is an `update` block (chapter 3), writing that structure in Prism is the same functional-update ceremony as in any pure language, and the rope is native code the program calls, like text shaping.

Search is a derived dynamic over the whole text and the whole query, recomputed on every change of either. That is right for a query box and wrong for a hundred-megabyte log, and the fix is the same: an index that is a plain value, updated incrementally by a plain function, with the derived dynamic reading the index. Nothing about occurrences changes.

Syntax highlighting is the same story told the other way: `Highlight $doc.text` is a derived dynamic, prompt, glitch-free, and the whole document is re-highlighted per keystroke. The incremental version is a plain function with a memo keyed on unchanged regions, which is a computation, which is the kind of thing Prism calls out to.

And the cursor blink. A blink is a `Time.Every` and a hold that toggles, and a buffer that is not focused should not blink, so the timer is inside a `rebuild` over whether it is focused, which the buffer learns from the window interceptor as a state element or a response. That is right and it is slightly more ceremony than a toolkit's `cursor_visible` property; the ceremony is what makes the blink testable with `push`. ## 2. A market-data consumer

A trading system is the program that builds the occurrence-as-transaction rule by hand, every time, badly. It holds a book, applies updates in sequence order, decides on the book it has, sends an order, and then has to answer a question no framework helps with: when the fill comes back, which book was the decision made on, and has anything happened to the position since? The teams that get this right keep a single-threaded loop, a sequence number on everything, and a rule that nothing reads the book while it is being updated. That is an occurrence with a sequence number, written in C++. Here it is as the language's own rule, with a plain paragraph about speed at the end.

## The vocabulary

Three things come from outside and one goes out. The feed is a stream of sequenced increments; the snapshot is a request answered with a full book and the sequence number it is good through; the book is a state element whose type says whether it is live; and an order is a request element keyed by a client-side id.

```
type Side = choice Bid | Ask

type Increment = record:
    seq: Nat
    side: Side
    price: Int                  # ticks
```

```
    qty: Nat                    # 0 removes the level

type Snapshot = record:
    seq: Nat
    bids: Int ==> Nat
    asks: Int ==> Nat

type Book = record:
    seq: Nat
    bids: Int ==> Nat
    asks: Int ==> Nat

type BookState = choice:
    Buffering [Increment]       # no snapshot yet
    Live Book
    Resyncing ((Book, [Increment]))   # a gap was seen; the last good book stays visible

tag Md.Feed of Element:
    symbol: Text
    responds: Increment

tag Md.Snapshot of Element:
    symbol: Text
    requests: !unit
    responds: Snapshot

tag Md.Book of Element:
    symbol: Text
    state: BookState

type NewOrder = record:
    side: Side
    price: Int
    qty: Nat
    on_seq: Nat                 # the book sequence the decision was made on

type OrderRequest = choice:
    Submit NewOrder
    Cancel

type OrderEvent = choice:
    Acked
    Rejected Text
    Filled ((Nat, Int))         # quantity, price
    Cancelled

tag Om.Order of Element:
    id: Nat
    requests: !OrderRequest
    responds: OrderEvent
```

`Md.Book` is supplied either natively, by a World with a book builder next to the exchange socket, or by the `@BookBuilder` interceptor of the Worlds chapter, which holds `Md.Feed` increments, asks `Md.Snapshot` from `@started` and again on a gap, folds the exchange's own recovery procedure into a `BookState`, and supplies that hold to every `Md.Book(symbol)` slot beneath it. The strategy below cannot tell which it is under, and that is the substitutability test from the other direction. It can tell whether the book is `Live`, because the type makes it say so; a strategy with no arm for `Resyncing` does not compile.

## The structure

The whole strategy is one state machine, one hold, and one `for`. Everything that can happen to it arrives as one event of a map, so that a book update and a fill landing together are both applied, in an order the program wrote down:

```
type Msg = choice:
    BookChanged BookState
    OrderEvents (Nat ==> OrderEvent)

type Action = choice:
    Place ((Nat, NewOrder))      # a fresh id and the order
    Pull Nat                     # cancel by id

type Strategy = record:
    position: Int
    working: Nat ==> NewOrder    # live orders by id
    next_id: Nat
    book: BookState

let Desired: fn BookState, Int, Params -> ?((NewOrder, NewOrder)) = ...   # the bid and ask we want, if
the book is Live
let Reconcile: fn Strategy, Params -> ((Strategy, [Action])) = ...        # diff desired against working

let ApplyMsg: fn Strategy, Msg -> Strategy = \|s, m| match m:
    BookChanged b: Strategy s.position, s.working, s.next_id, b
    OrderEvents evs: Fold evs, s, \|acc, id, ev| match ev:
        Acked: acc
        Rejected ignore: Strategy acc.position, (Remove acc.working, id), acc.next_id, acc.book
        Cancelled: Strategy acc.position, (Remove acc.working, id), acc.next_id, acc.book
        Filled ((q, ignore)): match acc.working[id]:
            Some o: Strategy (acc.position + (Signed o.side, q)), (Remove acc.working, id), acc.next_id,
            acc.book
            Nil: acc

let Step: fn Strategy, Params, Nat ==> Msg -> ((Strategy, [Action])) = \|s, params, msgs|
    Reconcile (Fold msgs, s, \|acc, ignore, m| ApplyMsg acc, m), params

@MarketMaker: fn Text, Params -> @$Int = \|sym, params| @widget:
    $book <- @Element Md.Book(symbol=sym)
    let !msgs = !Union {0: BookChanged (!Updated $book)}, {1: OrderEvents !events}
    let !stepped = Step $state, params, !msgs
    $state <- Strategy 0, {}, 0, Buffering [[]] then First !stepped
    let !actions = Second !stepped

    !events <- for $state.working:
        !started <- @started
        let !place = Submit $_ on !started
        let !pull = Cancel on !Filter (PullOf key, !actions)
        !ev <- @Element Om.Order(id=key, requests=(!place | !pull))
        @ !ev

    @ $state.position
```

`!msgs` is a `!Union` of two one-entry maps, so a book change and a batch of order events in one occurrence make one firing with both entries, folded in key order: book first, then order events, then `Reconcile`. Had it been `(BookChanged (!Updated $book)) | (OrderEvents !events)`, a fill landing with a book update would be lost, and the position would be wrong for the rest of the session; the

compiler does not catch that, but it does refuse `Step $state, params, (!Updated $book), !events`, two events in one operation, and the refusal is what makes you write the map.

`!stepped` is one event whose value is a pair, and the hold and the actions are its two halves. The alternative, a hold for the state and a separate lifted call for the actions, would read the state twice through tags and could not see the fold's result in the same occurrence. One expression, used twice: there is no "after" inside an occurrence.

`for $state.working` keys an `Om.Order` slot on each live order's id. The order is submitted at the block's birth, from `@started`, in the occurrence the state decided it; the slot lives while the id is in `working`; a partial fill that leaves the id in the map with a smaller quantity changes `$_` and rebuilds nothing; a terminal event removes the id and the slot goes at commit. A late `Cancelled` for an order the program already saw filled is addressed to a path that is gone, and the World discards it, which is right: the conversation ended.

`!pull` reads `!actions` from the enclosing scope, and that is a prompt edge from `!actions` into the block. There is no cycle, because the block's result is `!ev`, the slot's response, and a slot's response has no edge from its request: the creation rule is also a fact the compiler knows. The only prompt edge from a `for`'s collection to its consolidated event is what a new block fires at birth, and here a block fires only its request at birth, never its result.

## What "act on exactly the state you saw" means

An order fired in occurrence n is a pure function of the state before n and the inputs of n, and nothing else. `on_seq` in the order is the book's sequence as `Reconcile` saw it, and `Reconcile` saw a book that was whole, because derived dynamics are glitch-free and the fold inside `Step` runs to completion before anything reads its result. The World reads the request after n commits. Whatever comes back, an ack, a fill, a reject, lands in an occurrence after n, addressed to the slot that asked, and anything that changed the program's view of the world in between was itself an occurrence the program applied. There is no window in which the strategy reads a book with three of five increments applied, no window in which the position is updated but the working set is not, and no way for a response to be handled against a state the request did not see, because the response is a `Msg` folded into the same state machine.

That is the thing the hand-built loop is for, and it is cheaper here because the language refuses the alternatives: no sampling means no reading a half-built book; no callbacks means a fill cannot run code that mutates the working set while a book update is halfway through; one firing per event per occurrence means the fold sees each thing once.

## The hard moment

The book is `Live` at sequence 1041. The strategy has one working bid, id 7, for 100 at 9995, placed on sequence 1040. The exchange fills it, and in the same instant the book moves.

**Occurrence 20.** The World delivers two things it had waiting: a push for `Md.Book(symbol="XYZ")` with `Live (Book 1042, ..)`, and `Filled ((100, 9995))` to `Om.Order(id=7)` at path `1/[7]/1`. They address different slots, so one occurrence is legal, and under a World whose book builder and order gateway share a loop it is the common case.

`$book` changes; `!Updated $book` fires with the new state; `{0: BookChanged ..}` fires. Block `[7]`'s `!ev` fires with the fill; `!events` fires with `{7: Filled ((100, 9995))}`; `{1: OrderEvents ..}` fires. `!Union` fires with both entries. `Step` reads `$state` through a tag, the state before 20: position 0, working `{7:`

`..}`, book at 1041. It folds: the book becomes 1042; then the fill: position becomes +100 and 7 leaves `working`. `Reconcile` now wants a bid at the new best less one tick, say 9993, and an ask at 9998, and finds nothing working, so it returns `[[Place ((8, (NewOrder Bid, 9993, 100, 1042))), Place ((9, (NewOrder Ask, 9998, 100, 1042)))]]` and a state with `next_id` 10 and both orders in `working`.

`$state`'s change event fires with that state. `$state.working` changes promptly, so the `for` sees key 7 gone and keys 8 and 9 appear. Blocks `[8]` and `[9]` are built now, in 20; each `@started` fires; each `Submit` fires at birth. Block `[7]` is torn down at commit. `!actions` fires with the two `Place`s, which nothing in `[8]` or `[9]` listens to (`PullOf` wants `Pull`). Commit. After the turn the World sees:

```
1/[8]/1:  Om.Order(id=8)    requested: Submit (NewOrder Bid, 9993, 100, 1042)
1/[9]/1:  Om.Order(id=9)    requested: Submit (NewOrder Ask, 9998, 100, 1042)
```

and sends both, each stamped with the sequence it was decided on.

**Occurrence 21.** `Acked` to `1/[8]/1` and `Acked` to `1/[9]/1`. `ApplyMsg` leaves the state alone; `Reconcile` finds desired and working equal and returns no actions. `!stepped` fires, the hold changes to an equal value, `$state.working` changes to an equal map, the `for` rebuilds nothing because no key appeared or vanished. The position is 100, from occurrence 20, and nothing was ever 100 with order 7 still working.

Now suppose the World had delivered the fill and the book update as two occurrences, fill first. In 20 the fill removes 7, position goes to 100, `Reconcile` places replacements on the 1041 book. In 21 the book moves to 1042, `Reconcile` finds the working bid one tick off from desired, returns `[[Pull 8, Place ((10, ..))]]`, and block `[8]`'s `!pull` fires `Cancel` while block `[10]` is born. One more cancel than before, on a book the program fully applied. The program did not depend on the batching; the World's policy decided how many occurrences the instant took and the program's text said what each one meant. A test under the Test World writes both orders, on one line and on two, and the second implementation must agree on each.

The merge version is what gets written first, and the walkthrough of 20 under it is short: `(BookChanged ..) | (OrderEvents ..)` fires with the book, the fill is never applied, `Reconcile` sees 7 still working at 9995 on a book whose best bid has moved to 9994, pulls it, and places a new bid. The cancel reaches the exchange for an order that no longer exists and is rejected; the position is 0 in the program and 100 at the clearing house. Nothing crashed. That is the bug every one of these systems has had once, and the reason the `!Union` line is the most important one in the program.

## What the goals buy

You can see what moves: `Step $state, params, !msgs` says, before you know what any of them are, that one thing happens and two things are read as they stood. The risk reviewer reads the sigils.

Simultaneity is the program's: the `!Union` and the fold order are the specification of what a fill-and-update instant means, and the two orders of folding (book first, then events) are a visible choice that could be reversed in one line.

"Not yet" as a type: `Buffering`, `Live` and `Resyncing` are arms the strategy must write, so there is no strategy that quotes on a book that is being rebuilt, and nothing that returns an empty book while waiting for the snapshot.

The spec is the thing: the entire session is a transcript of pushes and responses, and a transcript replays exactly. A bad fill from last Tuesday is a test, with the inputs on one line where the World batched.

## Latency, as an implementation question

The specification does not claim a latency, and here is what the Rust runtime as described costs. An occurrence is one synchronous run call: fire the inputs, propagate by height through a height-ordered action map, commit the holds, rebuild the element list, return. Values are boxed except small integers; the state record is cloned on every step (a persistent map inside it makes that cheap, a plain array does not); the element list is recomputed in full after each occurrence in the first implementation; and before any of that, the feed's bytes crossed from the socket thread to the World's loop through a queue. For a graph the size of the one above that is on the order of microseconds to tens of microseconds per increment on ordinary hardware, which is three orders of magnitude slower than a tuned kernel-bypass loop and one or two orders faster than it needs to be for a strategy that thinks in milliseconds (an estimate, not a measurement; the runtime chapter's measurements replace it). Every increment is its own occurrence, because increments are responses and responses to one slot never share an occurrence; a World that offers the book as native state instead coalesces, and a strategy that reads `Md.Book` gets one occurrence per batch the World chose. The knobs that would move the number are all in the implementation chapters and none in the language: values representation, an incremental element list, arena allocation per widget, a runtime per symbol on its own thread. A system whose whole budget is five microseconds from packet to wire is not this program; it is a loop in C++ that this program supervises, through a state element for the loop's status and a request element for its parameters, with the strategy's decisions taken out of the hot path on purpose. ## 3. A control loop

A controller is the oldest reactive program there is: read the sensors, compute, drive the actuators, wait for the next tick. The synchronous languages (Esterel, Lustre, Lucid Synchrone) were built for exactly this and got the law right decades ago: an instant is atomic, every signal has one value per instant, and `pre` is the only way to see the previous one. Prism has the same law with a different clock: the tick is an occurrence the World opens, `pre` is the tag law, and the controller is a hold whose change event is the control law applied on the tick. Prism adds that the structure can change while the loop runs, and it owes the synchronous tradition a plain statement about timing, which the last part of this section gives.

## The vocabulary

Sensors are state: the latest reading as the occurrence opened. Actuators are requests: a firing is a command, answered with whether it was applied. The tick is a stream, and the emergency stop is state, because a late-built widget that asks "are we stopped" needs an answer in the occurrence it asks.

```
type Imu = record:
    gyro: ((Float, Float, Float))
    accel: ((Float, Float, Float))

type ImuState = choice:
    NoReading
    Reading ((Instant, Imu))

tag Sensor.Imu of Element:
    bus: Text
    state: ImuState

tag Sensor.Encoder of Element:
    wheel: Text
    state: Nat                  # ticks since power-on; wraps, and the program knows it
```

```
type MotorEvent = choice:
    Applied
    Fault Text

tag Actuator.Motor of Element:
    id: Text
    requests: !Float            # torque, normalised
    responds: MotorEvent

tag Safety.Stop of Element:
    state: Bool                 # True while the stop is asserted

tag Time.Every of Element:
    period: Nat                 # milliseconds
    responds: Tick

tag Clock of Element:
    state: Instant
```

Why the sensor is state and not a stream of samples: a stream delivers at the sensor's rate, each sample its own occurrence, and a 1 kHz IMU under a 100 Hz loop would run ten occurrences per tick, nine of which do nothing. As state, the World coalesces (state is a level, and the last value is the truth), the loop reads the latest reading through a tag on the tick, and the sample-and-hold every controller does by hand is the tag law. A program that wants every sample, for logging or for a filter that needs them all, asks for a stream element beside it, and a World offers both.

## The structure

```
type Gains = record:
    kp: Float
    ki: Float
    kd: Float

type Loop = record:
    integral: Float
    last_err: Float
    last_t: Int

type Sample = record:
    setpoint: Float
    measured: Float
    now: Int

let Pitch: fn ImuState -> ?Float = ...        # Nil while NoReading

let Control: fn Gains, Loop, Sample -> ((Loop, Float)) = \|g, l, s|
    let err = s.setpoint - s.measured
    let first = l.last_t == 0 or s.now <= l.last_t
    let dt = if first: 0.01 else: (ToFloat (s.now - l.last_t)) / 1000.0
    let integral = l.integral + err * dt
    let deriv = if first: 0.0 else: (err - l.last_err) / dt
    (((Loop integral, err, s.now), g.kp * err + g.ki * integral + g.kd * deriv))

let IfTrue: fn Bool -> ?unit = \|b| if b: Some unit else: Nil

let Unless: fn Bool, Float -> ?Float = \|stopped, t| if stopped: Nil else: Some t
```

```
@Balance: fn Gains, $Float -> @$Bool = \|gains, $setpoint| @widget:
    !tick <- @Element Time.Every(period=10)
    $clock <- @Element Clock
    $imu <- @Element Sensor.Imu(bus="i2c-1")
    $stop <- @Element Safety.Stop

    let !sample = !Filter (SampleOf $setpoint, (Pitch $imu), $clock) on !tick
    let !stepped = Control gains, $loop, !sample
    $loop <- Loop 0.0, 0.0, 0 then First !stepped
    let !torque = Second !stepped

    let !halt = !Filter (IfTrue (!Updated $stop))
    let !fault = !Filter (IsFault (!left | !right))
    let !cmd = 0.0 on (!halt | !fault) | !Filter (Unless $stop, !torque)
    !left <- @Element Actuator.Motor(id="left", requests=!cmd)
    !right <- @Element Actuator.Motor(id="right", requests=!cmd)
    @ $stop
```

`SampleOf` is a plain function that returns `Nil` while the IMU has no reading, so `!sample` fires on a tick only when there is something to control on, and the loop's state is untouched on the ticks before the first reading. `Control` is the whole control law and has no time in it beyond the timestamps it is handed: `dt` comes from the clock the World pushed, not from the nominal period, because a World skips missed ticks rather than delivering two at once, and a loop that assumed ten milliseconds per tick would integrate wrongly after every hiccup. `Float` division follows IEEE and never fails, so nothing would warn about the first tick: with `last_t` zero, `dt` would be the whole clock reading since power-on, and the derivative would be an error divided by seconds-since-epoch. `first` says what the first tick is. It is true when `last_t` is zero, and also when the clock has not advanced, because a zero `dt` would make the derivative an infinity that drives a motor. On those ticks `dt` is the nominal period and the derivative is zero.

`!cmd` is the only thing that reaches the motors and its merge is the safety policy in one line: a stop asserted or a fault reported drives zero torque, and that wins over the control law in the same occurrence; otherwise the law's output goes out, unless the stop was already asserted as the occurrence opened. Read it as three cases and you have read the program's answer to every ordering of tick, stop and fault.

## The synchronous view

Here is the Lustre you would write for the integral term (Lustre syntax from memory), with `inc` the increment computed on the same clock, and the Prism beside it:

```
integral = 0.0 -> pre integral + inc

$integral <- 0.0 then $integral + !inc
```

`->` is the hold's initial value; `pre` is the tag; the clock on which the equation runs is the event the hold's change is lifted over. A Lustre node is a widget; a Lustre program's static causality analysis, which rejects `x = x + 1` with no `pre` (from memory), is the static cycle check, which rejects a hold whose change event reads the hold promptly. The two are the same model with two differences.

The first is the clock. In Lustre the base clock is the program's and every instant is a tick. In Prism an occurrence is whatever the World opened, and a push with no tick, say the stop being asserted between ticks, is an instant in which `on !tick` expressions are silent and everything else runs. That is why `!halt` is derived from `!Updated $stop` rather than from reading `$stop` on the tick: the stop

acts in the occurrence it arrives, tick or no tick, and under a World that gives it its own turn the motors get their zero before the next sample. A synchronous program would have to poll the stop on its clock and wait up to a period.

The second is structure. A Lustre program's topology is fixed at compile time. Here the gain schedule can be a `rebuild` over a mode, and a mode change tears down one controller's widget and builds another, in the occurrence of the change, with the new one seeing all of it:

```
!$halted <- rebuild $mode:
    match _:
        Hover: @Balance hover_gains, $setpoint
        Carry: @Balance carry_gains, $setpoint
```

The new widget's `$loop` starts from zero, which is the bumpless-transfer problem in a sentence: a controller that must keep its integral across a mode change keeps the hold outside the `rebuild` and passes it in, and the program's text says which it does.

## The hard moment

The loop is running. The stop button is pressed, and the World, which batches what arrived together between turns, delivers the stop's push in the same occurrence as a tick.

**Occurrence 500.** Inputs: `Tick` to the `Time.Every` slot, a push of `Instant 5000` for `Clock`, a push of `Reading (..)` for `Sensor.Imu(bus="i2c-1")`, and a push of `True` for `Safety.Stop`. `!tick` fires. `SampleOf` reads `$setpoint`, `$imu` and `$clock` through tags: the setpoint, the IMU reading and the clock *before* the occurrence, the ones the previous tick also saw as "latest" plus whatever was pushed in occurrences since. `Control` fires with a torque, say `0.31`. `$stop` changes to `True`; `!Updated $stop` fires with `True`; `!halt` fires. Now `!cmd`: the left side, `0.0 on (!halt | !fault)`, fires, so `!cmd` fires with `0.0`, and the `0.31` from the right side is not heard. Both motor slots fire `0.0`. Commit: `$loop` has integrated the error (the law ran; its output was overridden), `$stop` is `True`.

```
4:  Actuator.Motor(id="left")     requested: 0.0
5:  Actuator.Motor(id="right")    requested: 0.0
```

**Occurrence 501.** The next tick, with the usual pushes. `Control` fires `0.29`. `Unless $stop, !torque` reads `$stop` through a tag, `True`, and returns `Nil`; `!Filter` drops it; the left side of the merge is quiet (`!Updated $stop` did not fire; the stop did not change). `!cmd` is quiet. Nothing is sent, and the motors hold the zero they were given. The loop keeps integrating, which is the wind-up problem and a product decision: a program that wants the integral frozen while stopped puts `$stop` into `Sample` and lets `Control` see it.

**Occurrence 530.** The stop is released: a push of `False` for `Safety.Stop`, with a tick. `!Updated $stop` fires with `False`; `IfTrue` drops it; `!halt` is quiet. `Unless $stop, !torque` reads `$stop` through a tag, still `True` before this occurrence, and drops the torque. `!cmd` is quiet. The motors stay at zero for this tick, and from 531 on the tag reads `False` and torque flows. One tick of delay on release, by the tag law, and none on assertion, by `!Updated`. A program that wants release to act in its own occurrence derives the torque from `!Updated $stop` as well, and the asymmetry is the right default: stopping is urgent and starting is not.

Now the same stop under a World that delivers each input in its own turn. Occurrence 500 has the push alone: `!tick` is quiet, `!torque` is quiet, `!halt` fires, `!cmd` fires `0.0`. The motors stop one turn earlier than the next tick. Occurrence 501 has the tick: as above. The program's text covered both without a case split.

Had the program written `!cmd = !Filter (Unless $stop, !torque)` alone, trusting the tag, the stop would act one tick late in both Worlds, and under the batching World the torque computed in 500 would go out with the stop already asserted. Ten milliseconds, in a loop that may be the only thing between a motor and a person. The merge with `!Updated` is one line, and it is readable as "a change of the stop wins in the moment it changes."

## What the goals buy

One way to change anything: `$loop` moves on `!stepped` and nothing else; `!cmd` is the complete list of what can reach a motor, with its priority order. There is no interrupt handler that writes a torque behind the loop's back.

Simultaneity: the stop-and-tick instant is a line of the program, under either World policy.

No clock: `dt` is from a state element the test sets. The step response of the controller is a test with `push Clock` and `push Sensor.Imu(..)` per tick and `requested Actuator.Motor(id="left"), 0.31` as the assertion, and it runs in no time at all, because there is no timer in it.

The language is enough: the control law, the safety policy, the mode switch and the anti-windup are all Prism. The native things are the bus, the motor driver and the timer, which is the list of things that touch hardware.

## Where it is awkward, and where it is wrong

Prism promises nothing about *when* an occurrence runs. The tick is a `Time.Every` the World delivers when its loop gets to it; the run call takes as long as the graph takes; a `rebuild` firing in the tick path (the mode switch above, in the same occurrence as a tick) takes as long as the build takes. A controller with a deadline needs the World to run the loop on a thread with real-time priority, to keep its own IO off that thread, and to measure the occurrence; the language gives it a deterministic amount of work per occurrence and nothing more. Structural changes belong outside the tick path or inside the jitter budget, and the advice is to measure the rebuild under the Test World and decide.

A World skips missed ticks rather than delivering them late, which is the right policy for a controller and the wrong one for a program counting ticks; such a program counts the clock instead.

Estimation is a computation. A Kalman filter over a six-by-six covariance is a plain function called on the tick, and writing it in Prism means writing matrix code in a language with no mutable arrays and no `update` block yet; it is native code the prelude exposes, like text shaping, and the Prism program decides when to call it and what to do with the result.

And the innermost loop is not here. A motor driver's current loop at twenty kilohertz, with a deadline measured in microseconds, lives in firmware, and `Actuator.Motor` is the interface to it: the Prism program sets a torque target at a hundred hertz and the driver closes the fast loop itself. The synchronous-language view is the whole justification for Prism in this domain, and the synchronous languages themselves were never used for the fastest loop either; they were used for the layer that decides. ## 4. A chat server

You have written the server where a connection object is kept in a map by hand, removed in a close callback that sometimes runs after the next read callback, so the map has a dead entry that the next broadcast crashes on. Or the one where a user who left the room still got three messages because the membership set was read before the leave was applied, by a handler that ran first for no reason anyone wrote down. A chat server is "maintain a model, fan out its changes" with nothing else in it, which makes it the cleanest place to see what a `for` keyed on connection ids and a single state

machine buy, and the right place to see what happens to a message addressed to someone who is leaving in the same moment.

## The vocabulary

```
type Accepted = Accepted Nat

type ConnRequest = choice:
    Send Text
    Close

type SocketEvent = choice:
    Received Text
    Closed

type Link = choice:
    Connected Nat                # bytes queued and not yet written
    Disconnected Text

tag Net.Listener of Element:
    address: Text
    responds: Accepted

tag Net.Conn of Element:
    id: Nat
    requests: ![ConnRequest]
    responds: SocketEvent
    state: Link

type Stored = choice:
    Absent
    Present [Text]

type PutResult = choice:
    Stored
    Failed Text

tag Store.Value of Element:
    key: Text
    state: Stored

tag Store.Append of Element:
    key: Text
    requests: ![Text]
    responds: PutResult
```

The listener is a stream of accepted connections with ids the World allocated. A connection is the three-part element: requests are what to send, responses are what arrived and the close, and the link is state shared by value, which here means nothing, since each id is one connection. The store is a key-value component: a value as state, with `Absent` in the type, and an append as a request answered later. A World that has no store answers `Unsupported` to the append and errors at the creation of the state slot, and a program that must run without one checks `World.Tags` first; this one assumes a store.

## The structure

The model is one record, and every command any connection can issue is one closed choice:

```
type Cmd = choice:
    Nick Text
    Join Text
    Say Text
    Who
    Leave                       # synthesised from Closed

type Model = record:
    nicks: Nat ==> Text
    rooms: Text ==> (Nat ==> unit)      # room name to its members
    member_of: Nat ==> Text             # one room per connection

# Apply every command that landed in this occurrence, in key order, and say what
# each connection should be sent as a result. One function, one result, no "after".
let StepAndRoute: fn Model, Nat ==> [Cmd] -> ((Model, Nat ==> [Text], Text ==> [Text])) = ...

let Presence: fn Model -> Text ==> [Text] = \|m|
    MapValues m.rooms, \|members| Sorted (Names m.nicks, members)
```

`StepAndRoute` is the server. It folds the commands of an occurrence into the model and, as it goes, accumulates the lines each connection is owed: the room's members get `"<nick> hi"` on a `Say`, `"nick joined"` on a `Join`, `"nick left"` on a `Leave`; the asker gets the member list on `Who`. It is a pure function over a map of command lists, and every policy question (does a leaver see their own leave notice, does a join to a full room fail, is `Who` answered before or after the joins in the same occurrence) is an arm of it, testable with no sockets.

The widget around it:

```
let IdOf: fn Accepted -> Nat = \|a| match a: Accepted n: n

let TextOf: fn SocketEvent -> ?Text = \|ev| match ev:
    Received t: Some t
    Closed: Nil

let IsClosed: fn SocketEvent -> ?unit = \|ev| match ev:
    Closed: Some unit
    ignore: Nil

let SendAll: fn [Text] -> [ConnRequest] = \|lines| Map lines, \Send (Concat [[_, "\r\n"]])

let HasLeave: fn [Cmd] -> Bool = ...

@server:
    !accepted <- @Element Net.Listener(address="tcp://0.0.0.0:6667")
    let !opens = {IdOf !accepted: Opened}

    !cmds <- for $conns:
        ((!ev, $link)) <- @Element Net.Conn(id=key, requests=!out)
        let !split = SplitLines (Concat [[$rest, !Filter (TextOf !ev)]])
        $rest <- "" then Second !split
        let !parsed = ParseAll (First !split)
        let !gone = [[Leave]] on !Filter (IsClosed !ev)
        let !out = SendAll (!Filter (!owed[key]))
        @ !parsed | !gone
```

```
let !stepped = StepAndRoute $model, !cmds
$model <- Model {}, {}, {} then First !stepped
let !owed = Second !stepped
let !said = Third !stepped

let !closes = MapValues (Filter !cmds, HasLeave), \|ignore| Shut
$conns <- {} then ApplyDeltas $conns, (!Union !opens, !closes)

let $presence = Presence $model
@ $presence
```

Each connection's block does framing (the `$rest` hold and `SplitLines`, as in the Worlds chapter), parses complete lines into commands, turns a close into a `Leave`, and sends whatever the model owes it. `!owed[key]` is an event of a map indexed by a plain key, which lifts to an event of an optional; `!Filter` keeps the occurrences in which this connection is owed something. The block's result is its commands, consolidated by `for` into `!(Nat ==> [Cmd])`, keyed by connection id, with exactly the connections that said something in this occurrence.

The cycle runs through the whole program and is legal at every edge. `$conns` is held from `!closes`, which comes from `!cmds`, the result of `for $conns`; that passes through the tag on `$conns` in `ApplyDeltas` and through the fact that a block's commands are responses, which never arrive in the occurrence the block was born. `!out` inside a block reads `!owed`, which is derived from `!cmds`, the consolidated result of the very `for` the block is in; that is a prompt edge from the collection's result into every block, and it is fine because the edge ends at a request field, and a slot's response has no edge from its request. The compiler sees a `for` whose blocks fire nothing at birth, which is the only prompt edge from a collection to its consolidated event it has to worry about.

A `Say` and a `Leave` from two different connections in one occurrence are two entries of `!cmds`. A `Say` and a `Leave` from *one* connection cannot share an occurrence, because they are both responses to one slot (`Received` and `Closed`), and a World delivers at most one response per path per occurrence. That is the one place the model is simpler than the sockets it describes, and it is simpler because the World promised it.

## Presence as a derived dynamic

`$presence` is the room-to-members map, derived from `$model`, prompt and glitch-free: in any occurrence it is the model after that occurrence's commands, never a membership set with the join applied and the nick change not. It is the widget's result, so a program that wraps `@server` in an admin page shows it, and a `Who` command is answered inside `StepAndRoute` from the same model, so the two can never disagree. A derived dynamic changes whenever its operand changes, including to an equal value, so a dashboard that redraws on `!Updated $presence` wraps it in `$Distinct` first; a message said in a room changes `$model` and therefore fires `$presence`'s change event with the same membership, and the dashboard would redraw per message otherwise. Equality on a map of arrays of text exists, so `$Distinct` is available.

## Persistence

A room's history is a state element per room, and an append is a request per line said:

```
$histories <- for $model.rooms:
    $h <- @Element Store.Value(key=(Concat [["room:", key]]))
    @Element Store.Append(key=(Concat [["room:", key]]), requests=(!Filter (!said[key])))
    @ $h
```

`for $model.rooms` is keyed by room name, so a room's history slot and append slot live as long as the room has members and go when it empties; `!said` is the third output of `StepAndRoute`, the lines to persist per room. `$histories` is `$(Text ==> Stored)`, with a value in the occurrence each room appears, from the oracle, which is `Absent` until the World has loaded the key and `Present` after; a join that wants to send the last twenty lines reads `$histories` through a tag in `StepAndRoute` and gets what was known as the occurrence opened. A `Failed` from the append is a response the block can hold and retry behind a `Time.After`, and the retry policy is the block's text.

The store is also the sharding seam. Two rooms share nothing but the listener, so a World that owns the listener can route each accepted connection to one of several runtimes, each running this program, with the store as the only thing they share; the Worlds chapter has the router. Nothing in `@server` changes between one runtime and ten.

## The hard moment

Connections 5, 9 and 12 are in `lobby`. Connection 5's client disconnects, and in the same instant connection 9 says "hi".

**Occurrence 300.** The World delivers `Closed` to `1/[5]/0` and `Received "hi\r\n"` to `1/[9]/0`: two slots, one occurrence, which the World may do and a busy server often will. In block `[5]`, `!gone` fires with `[[Leave]]`. In block `[9]`, the chunk is split, `ParseAll` gives `[[Say "hi"]]`, `!parsed` fires. `!cmds` fires with `{5: [[Leave]], 9: [[Say "hi"]]}`. `StepAndRoute` reads `$model` through a tag and folds in key order: 5 first. `Leave` removes 5 from `lobby` and from `member_of`, and owes 9 and 12 the line `"five left"`. Then 9: `Say "hi"` is routed to `lobby`'s members *as the fold now has them*, 9 and 12, so each is owed `"<nine> hi"` as well. `!owed` fires with `{9: [["five left", "<nine> hi"]], 12: [["five left", "<nine> hi"]]}`. Block `[9]`'s `!out` fires with two `send`s; block `[12]`'s likewise; block `[5]`'s `!owed[5]` is `Nil` and nothing fires there. `!closes` fires with `{5: Shut}`; `$conns` loses key 5; block `[5]` is torn down at commit. After the turn the World writes two lines to 9 and two to 12 and sees the `Net.Conn(id=5)` element gone, which it already knew.

Nothing was sent to 5, and the reason is that the leave and the say were folded by one function in one pass with one model, so the say saw the leave. Now the other way to write it, which is the way the first version always gets written: a `Step` that updates the model and a separate `Route` lifted over `$model` and `!cmds`:

```
$model <- ... then Step $model, !cmds
let !owed = Route $model, !cmds
```

`Route` reads `$model` through a tag, the model before 300, in which 5 is still in `lobby`. `!owed` fires with 5 owed `"<nine> hi"`. Block `[5]`'s `!out` fires `[[Send "<nine> hi\r\n"]]` in the occurrence block `[5]` is torn down. The runtime reports that firing to the World with its path, `1/[5]/0`, because the program asked; the World performs it if it can, which for a closed socket it cannot, and discards any response, since the slot is gone. The user never sees the message and nothing crashes. The program paid one wasted send and one line in the World's diagnostics; the callback version would have paid a write on a closed descriptor from a handler that still held the connection object, and the exception that goes with it.

Key order is doing work in 300. Within an occurrence there is no order of arrival; the leave and the say are simultaneous, and the program chose to apply them in key order, which here means 5 before 9. Had the leaver been 12, the fold would have routed the say to 12 first and removed 12 after, and

12's block would have fired a send on its own teardown. A program that wants leaves applied before says regardless of id sorts the occurrence's commands by kind inside `StepAndRoute` before folding, and that is one line and a stated policy. The point is that the policy is in the program's text, and that in the callback version it was in the order two sockets happened to become readable.

## What the goals buy

One way to change anything: `$model` has one change event, and `StepAndRoute` is the complete list of what any command does to it. Membership cannot drift from `member_of`, because both are written by one function in one place.

Simultaneity: occurrence 300 is a `!Union` and a fold order, both visible.

No widget values: there is no connection object to keep in a map and forget to remove. A dead connection has no slot, and a write to it is impossible to express.

The spec is the thing: the Test World accepts `Net.Listener` and `Net.Conn`, so the whole server, including the race above, is a script of `send Net.Conn(id=5), Closed` on one line with `send Net.Conn(id=9), Received "hi\r\n"` and a `requested` step per recipient. The server you tested is the server you shipped.

## Where it is awkward

Every message is an occurrence, and every occurrence recomputes the element list in the first implementation, which with a hundred thousand connections is a hundred thousand entries per message. The incremental element list is the fix and it is an implementation item, not a language one; until it exists, the server that needs that scale is several runtimes with fewer connections each, which the sharding seam allows anyway.

Fan-out to a room of five thousand is a map with five thousand entries in `!owed`, five thousand request firings, and five thousand performs by the World after the turn. That is the right amount of work and it is done on the World's side, where the write buffers are; the program cannot stream it, because a request firing is atomic per occurrence. A room that large wants its fan-out in the World, as a `Net.Multicast(group)` element the program sends to once, and a World can offer that without the language knowing.

Backpressure is `$link`'s `Connected n`, readable through a tag at the moment of sending, and a block that drops or queues when `n` is large has a policy in its text. A block that ignores it works until a slow client fills the World's buffer, which is the same as every other server, and the difference is that the fix is one `match` in the block rather than a writable-callback dance.

Parsing a wire protocol is string work in a pure language with no mutable buffers. It is fine for lines and tedious for anything with length prefixes and binary fields; the prelude should offer the usual codec functions natively, and the program should call them.

TLS is the World's: `tls://` in the address and a `Disconnected "certificate .."` in the link type is all the program sees. ## 5. A build system

Every build tool is an incremental computation with a file watcher bolted on afterwards, and the bolt is where it leaks: the rebuild that ran twice because the editor wrote the file in two syscalls, the stale object that survived because the watcher fired while the compiler was still writing, the dependency the tool did not know about because it was discovered inside a compile. A dynamic *is* an incrementally maintained value, and a `for` over a map *is* an incrementally maintained collection, so a build system in Prism is the thing itself rather than a graph the tool maintains by hand. It is not a fast compiler, and the end of this section says where the line is.

## The vocabulary

```
type Stamp = choice:
    NoStamp
    Modified Instant

tag Fs.Watch of Element:
    path: Text
    state: Stamp

type Command = record:
    program: Text
    args: [Text]
    cwd: ?Text

type SpawnResponse = choice:
    Spawned Nat
    NotFound
    Denied

tag Process.Spawn of Element:
    requests: !Command
    responds: SpawnResponse

type ChildInput = choice:
    Stdin Text
    CloseStdin
    Signal Sig

type ChildOutput = choice:
    Stdout Text
    Stderr Text
    Exited Int

type ChildStatus = choice:
    Running
    Exited Int
    Gone

tag Process.Child of Element:
    id: Nat
    requests: !ChildInput
    responds: ChildOutput
    state: ChildStatus

tag Cli.Stdout of Element:
    requests: !Text
    responds: WriteFailed
```

Files are state: a target's inputs are `Fs.Watch` slots whose stamps are the facts the build is a function of. A compiler is a process: spawning is a request (a one-shot effect with a result, an id the World allocated), and the child is a state-and-stream element keyed by that id, whose `Exited` arrives both as a response (an edge the target counts) and as state (a level a late-built widget can read). When a `Process.Child` slot is torn down, the World's cancel-on-teardown obligation kills the child; a target that stops wanting a compile stops having the slot, and the compile stops.

## The structure

The dependency graph is a map from target to rule. Each target is a `for` block over that map, and a target's dependencies on other targets are a read of the `for`'s own consolidated result:

```
type Rule = record:
    inputs: [Text]              # source files
    deps: [Text]                # other targets
    output: Text
    command: Command

type Job = record:
    id: Nat
    on: Text ==> Stamp          # input stamps as they stood when the job was spawned

let AllKnown: fn Text ==> Stamp -> Bool = ...     # no input NoStamp
let Newer: fn Text ==> Stamp, Text ==> Stamp -> Bool = ...

@Target: fn Rule, $(Text ==> Nat) -> @$Nat = \|rule, $versions| @widget:
    $stamps <- for $ (KeyBy rule.inputs):
        $s <- @Element Fs.Watch(path=key)
        @ $s
    let $deps = Project $versions, rule.deps           # dependency versions, as a map
    let $want = $Distinct (Wanted $stamps, $deps)      # the (stamps, dep versions) this build is a function of

    !sp <- @Element Process.Spawn(requests=(rule.command on !Filter (ReadyFor $job, (!Updated $want))))
    $job <- Nil then (Some (Job (!Filter (IdOf !sp)), $stamps)) | Nil on !done
    !child <- rebuild $job:
        match _:
            Some j:
                ((!out, $st)) <- @Element Process.Child(id=j.id, requests=!never)
                @ ((!Filter (ExitOf !out), $st))
            Nil:
                @ ((!never, $ Gone))
    let !done = !Switch (First !child)
    $version <- 0 then $version + 1 on !Filter (IsZero !done)
    @ $version

@build: fn Text ==> Rule -> @unit = \|graph| @widget:
    $versions <- for $ graph:
        $v <- match graph[key]:
            Some rule: @Target rule, $versions
            Nil: @ $ 0
        @ $v
    @Element Cli.Stdout(requests=(Report (!Updated $versions)))
    @ unit
```

Take it from the bottom. `$versions` is `$(Text ==> Nat)`, one entry per target, each target's count of successful builds, and it is passed back into every `@Target` as a dynamic. A target derives `$want` from its input stamps and its dependencies' versions; when `$want` changes to a new value (`$Distinct`, because a stamp push with an equal value is not a reason to rebuild), the target spawns its command. The spawn's response is an id; the `rebuild` over `$job` builds a `Process.Child(id)` slot for it; the child's `Exited 0` bumps `$version`; and the bump reaches every target that depends on this one through `$versions`, promptly, in the same occurrence. A chain of three targets whose root input changed rebuilds in three turns separated by two compiles, because each spawn is a request answered later, and never rebuilds a target before its dependency has finished, because `$deps` cannot change until the dependency's `$version` does.

The read of `$versions` inside the `for` that produces `$versions` is a cycle, and it is legal. `$version` in a target is a hold; its change event fires on `!done`, which is a response from a `Process.Child` slot, and a slot's response has no edge from anything the program did. So the loop from `$versions` into a target and back out to `$versions` passes through the World, which is the delay that makes every build system's dependency graph acyclic in time even when the tool's own data structure is not: a dependency's new version arrives in an occurrence after the dependency was spawned, which is after the dependent read the old version. Had `$version` been a derived dynamic over `$versions` with no response in between, the cycle would be prompt and the compiler would refuse it, which is the spreadsheet case below.

`Job` records the input stamps as they stood when the job was spawned. That is the line that makes the hard moment come out right.

## The hard moment

Target `lib` has one input, `lib.c`, and target `app` depends on `lib`. The user saves `lib.c`; `lib` is compiling; the user saves `lib.c` again just as the compile finishes.

**Occurrence 40.** The World pushes `Modified (Instant 1000)` for `Fs.Watch(path="lib.c")`. In `lib`, `$stamps` changes, `$want` changes, `!Updated $want` fires, `ReadyFor` passes it, `rule.command on ..` fires: the `Process.Spawn` slot's request. The World spawns after the turn.

**Occurrence 41.** `Spawned 77` to the spawn slot. `$job` becomes `Some (Job 77, {lib.c: Modified 1000})`; the `rebuild` builds anew with a `Process.Child(id=77)` slot, whose oracle answer is `Running`.

**Occurrence 42.** The compile finishes and the user's second save lands in the same instant. The World delivers `Exited 0` to the child slot at `0/[lib]/2/#1/0` and pushes `Modified (Instant 1300)` for `lib.c`. In `lib`: `!out` fires; `ExitOf` passes `0`; `!child`'s current generation fires its exit; `!Switch` delivers `!done` with `0`; `$version + 1 on ..` fires with `1`; `$version` changes. In the same occurrence `$stamps` changes to `{lib.c: Modified 1300}`, `$want` changes, `!Updated $want` fires. `ReadyFor` is the function that decides whether to spawn now, and it reads `$job` through a tag: `Some (Job 77, {lib.c: Modified 1000})`, a job still running as the occurrence opened. The program has to say what a change during a running job means, and this program's `ReadyFor` says: spawn, since the stamp the job was spawned on is older than the stamp now. So the spawn request fires again in 42. `$job`'s change event has two sides: the left is lifted over `!sp`, the spawn's *response*, which did not fire in 42 (the request did; the answer comes later), and the right is `Nil on !done`, which did. `$job` becomes `Nil`; the `rebuild` builds anew to the `Nil` arm; the child slot for 77 is torn down at commit, which is fine because 77 has exited.

Meanwhile `$versions` changed, so `app`'s `$deps` changed, `app`'s `$want` changed, and `app` fires its own spawn in 42, against the `lib` that was just built from the 1000 stamp. That is correct and wasteful: `app` will be built from the old `lib`, then `lib` will be rebuilt from the 1300 stamp, then `app` again. The program could avoid the first `app` build by having `lib` not bump `$version` when its stamps have moved past the job's: `IsZero !done` becomes `Fresh $job, $stamps, !done`, reading both through tags, and the job's recorded stamps are exactly what that test needs. The decision to record the stamps at spawn time, by the tag law in `Some (Job .., $stamps) on !sp`, is what makes either policy expressible; a build tool that records nothing cannot tell a compile that finished on time from one that finished stale.

**Occurrence 43.** `Spawned 78` to `lib`'s spawn slot. `$job` becomes `Some (Job 78, {lib.c: Modified 1300})`. Everything proceeds. Had the second save and the exit been separate occurrences, the walkthrough is the same with the steps in two places, and the program never cared which.

The version with a watcher-as-stream and a mutable dirty set does, in occurrence 42, the bug from the first paragraph: the exit handler clears the dirty flag, the watch handler sets it, and whichever ran second decides whether `lib` is rebuilt. Here there is no second; there is one occurrence, and `ReadyFor` saw both facts.

## A spreadsheet is the same thing, until it is not

A spreadsheet is cells with formulas that reference other cells, and a change to one cell recomputes the cells that depend on it. Write it as the build system above with cells for targets and formulas for commands and it fails to compile, and the failure is instructive. A cell's value would be a derived dynamic over the consolidated `$values` of the `for`, with no hold and no World response in the loop; that is a prompt cycle through a `for`'s result, and the static check refuses it. It is right to refuse it. The build system's loop was legal because every dependency crosses the World and comes back later; a spreadsheet's dependencies are between values in one occurrence, and a cycle among them is the thing the spreadsheet must detect, not the thing the language can wave through.

So the sheet's values are one derived dynamic over the sheet's formulas:

```
@sheet:
    !edits <- for $ (Grid 26, 100):       # one block per cell reference, "A1" to "Z100"
        $text <- @TextInput ""
        @ !Updated $text
    $formulas <- {} then Union $formulas, !edits
    let $values = Evaluate $formulas      # topological order, cycle detection, all inside
    for $values:
        @Label (Show $_)
    @ unit
```

`Evaluate` is a plain function from the whole map of formulas to the whole map of values. It sorts, detects cycles, and reports them as a value (`Cycle [Ref]` in the cell's result type) rather than as an abort. Every keystroke in any cell recomputes the whole sheet, and for a thousand cells that is nothing and for a million it is the problem. The incremental evaluator, which recomputes only the cells downstream of the edit, is a plain function too: it takes the old values and the edit and returns the new values, and a hold that keeps the old values gives it its argument through a tag. That is incremental computation written as a fold, which is what a build system's dependency graph is once you stop pretending the tool's data structure is the graph.

The division of labour is the one this chapter keeps finding. Between files and processes, where the edges cross the World and the facts arrive at different times, the dynamics and the `for` are the dependency tracker and the program writes none of it. Between cells in one occurrence, where all the facts are present at once, the dependencies are data, and data is a plain function's business.

## What the goals buy

One way to change anything: a target's `$version` moves on one event, its child's exit, and nothing can bump it because a file was touched or a sibling finished.

No request on creation: a target built in occurrence zero spawns nothing until `$want` changes, and a `$Distinct` over it makes "changes" mean what the build means by it. The build that runs on startup because the tool was restarted is not a thing here, and the one that runs because an input is `NoStamp` is a line the program wrote (`AllKnown`).

"Not yet" as a type: a target whose input is `NoStamp` is a target the program had to write an arm for; there is no rebuild loop against a file that does not exist.

The language is enough: the dependency graph, the dirty tracking, the stale-job detection and the scheduling are all the hold-and-`for` structure of the program. The native things are the watcher, the spawn and the child.

The spec is the thing: the build system's race in occurrence 42 is one turn line with a `send Process.Child(id=77), Exited 0` and a `push Fs.Watch(path="lib.c"), Modified (Instant 1300)`, and the expected `requested Process.Spawn, ..` after it.

## Where it is awkward, and where it is wrong

The compile is a computation and it lives in a process, which is right, and it means the fine-grained incrementality everyone wants from a build system (rebuild this function, not this file) is not something the build system can offer; it is something the compiler must. Prism tracks dependencies at the granularity of what crosses the World: files and processes. A compiler that reports the headers it actually included (a depfile) is one more `Fs.File` read after the exit and a `$discovered` hold that joins the rule's inputs, which is a few lines, and dynamic dependencies turn out to be ordinary.

A build with ten thousand targets is a `for` with ten thousand blocks, each with its own `Fs.Watch` slots, and the World has ten thousand watches, which on most platforms is fine and on some is not; aggregating them is an interceptor (one watch per directory, fanned out by value) and an afternoon.

`Evaluate` for a million cells, or any evaluator that wants a mutable worklist, is the kind of code Prism is wrong for, and the right answer is the same as for the rope: a native function the program calls. A spreadsheet in Prism is a thin layer of time over an evaluator written in something else, and the thin layer is where the bugs were. ## 6. A shell and a window compositor

A shell is a World written in Prism that hosts programs; a compositor is the same thing with a screen. The worked examples have the small version: two programs, one shown at a time, clicks routed to the visible one. This section takes it to a desktop: any number of programs launched and quit at runtime, overlapping windows with a stacking order, focus, and the one invariant a compositor must keep that a toolkit never states, which is that a click lands on what the user saw. The hard moment is the window that quits under the click that raised it.

## The vocabulary

The compositor speaks the primitive GUI vocabulary downward to the World and, upward to the applications it hosts, the same vocabulary again. From an application's side the compositor *is* a World that accepts `Gui.Window` and `Gui.Scene`; from the real World's side the compositor is one program with one window.

```
type Extent = record:
    w: Float
    h: Float

type Point = record:
    x: Int
    y: Int

type Rect = record:
    x: Int
    y: Int
    w: Int
    h: Int
```

```
type WindowInput = choice:
    Pointer Point
    Pressed Key
    Released Key

tag Gui.Window of Element:
    title: Text
    responds: WindowInput
    state: Extent

tag Gui.Scene of Element:
    window: Text
    draw: [Draw]

type Code = Code Int

tag Process.Exit of Element:
    requests: !Code

tag Process.Spawn of Element:
    requests: !Command
    responds: SpawnResponse

tag Process.Child of Element:
    id: Nat
    requests: !ChildInput
    responds: ChildOutput
    state: ChildStatus
```

`Process.Exit` is a request. An application that wants to end fires `Code 0` on it; nothing ends because a window closed or a widget was dropped. That is the rule the compositor leans on hardest, because it is what lets the compositor decide what an application's exit means (remove its frame) rather than having the World decide (end the process the compositor is running in).

## Processes as state: the shell half

Before the screen, the shell. A process table is a keyed collection of state elements, and the shell's own job list is a derived dynamic over it:

```
type Entry = record:
    id: Nat
    command: Command

@Shell: @widget:
    !line <- @Lines                                  # standard input, one line per firing
    let !spawn = !Filter (ParseCommand !line)
    !sp <- @Element Process.Spawn(requests=!spawn)
    let !opens = {!Filter (IdOf !sp): Opened}
    $table <- for $jobs:
        ((!out, $status)) <- @Element Process.Child(id=key, requests=!never)
        @Element Cli.Stdout(requests=(!Filter (StdoutOf !out)))
        @ $status
    let !closes = MapValues (Filter (!Updated $table), IsExited), \|ignore| Shut
    $jobs <- {} then ApplyDeltas $jobs, (!Union !opens, !closes)
    @Element Cli.Stdout(requests=(Render (!Updated $table)))
    @ unit
```

`$table` is `$(Nat ==> ChildStatus)`: one entry per child, each a state element the World answers `Running` for from the oracle and pushes `Exited n` on. `jobs` is a rendering of `$table` on each change; a child leaves the table when its status says it exited, which is a `for` whose membership is a hold moved by the consolidated state's own change, legal because `ApplyDeltas` reads `$jobs` through a tag and a child's exit is a push. Piping one child's output into another's input is the `Defer`-per-chunk pattern in the Worlds chapter, and a `Signal Term` on a `Process.Child` is a request like any other. A shell that spawns a child and is itself torn down kills the child, because the World's cancel-on-teardown obligation covers `Process.Child` slots.

## The compositor

Applications are a catalogue, so that a frame can be keyed by an id and hold its application as plain data:

```
type App = choice Counter | Stopwatch | Editor

let Launch: fn App -> @unit = \|a| match a:
    Counter: @counter
    Stopwatch: @stopwatch
    Editor: @editor

type Frame = record:
    app: App
    rect: Rect

type Desk = record:
    frames: Nat ==> Frame
    order: [Nat]                # bottom to top
    focus: ?Nat
    next: Nat

type DeskCmd = choice:
    Open App
    Raise Nat
    Quit Nat
    Move ((Nat, Rect))

let StepDesk: fn Desk, Text ==> DeskCmd -> Desk = ...      # fold in key order; a Raise or Move of a
missing id is a no-op

type FrameOut = record:
    !quit: DeskCmd
    $draw: [Draw]
```

A frame is an interceptor around one application. It supplies the application's window its size from the frame's rectangle, holds the application's scene back and offers it upward as a dynamic, holds `Process.Exit` back and reports the exit as a result, delivers the pointer and key events the compositor addresses to it, and forwards everything else:

```
let IsFramed: fn Element -> Bool = \|e| match e:
    Gui.Window(..): True
    Gui.Scene(..): True
    Process.Exit(..): True
    ignore: False

@Frame: fn App, $Rect, !WindowInput -> @((!Code, $[Draw])) = \|app, $rect, !input| @widget:
    ((ignore, $slots)) <- intercept (Launch app), !down, $ supply
```

```
    let supply = \|e| match e:
        Gui.Window(..): @ (SizeOf $rect)
        ignore: @Element e
    !held <- for $slots:
        rebuild $_:
            match _.element:
                Process.Exit(..): @ _.!request
                ignore: @ !never
    let !exit = !Filter (AnyValue !held)
    let $draw = SceneOf $slots
    !fwd <- @Forward (Filter $slots, \not IsFramed _.element)
    let !to_window = !Filter (Addressed $slots, !input)
    let !down = !Union !to_window, !fwd
    @ ((!exit, $draw))
```

`Addressed` is the plain function that turns an input into a one-entry map keyed by the application's `Gui.Window` slot path, read from `$slots` through a tag, or `Nil` if the application has no window yet; `SceneOf` is the plain function that finds the application's `Gui.Scene` element in the list and takes its `draw` list, or `[[]]`; both read the application's list, and an application that draws two windows is an application the frame draws as one, which is a policy the frame could change. `AnyValue` takes the first value of a map, if there is one, so `!exit` fires with the code of whichever exit slot fired; the frame does not know its own id, so it reports a code and the desktop turns it into a `Quit` of the right frame.

The desktop is a `for` over frames, a hit-test, and a fold:

```
@Desktop: @widget:
    ((!raw, $size)) <- @Element Gui.Window(title="Desktop")
    let !hit = !Filter (HitFrame $desk, (!Filter (PointerOf !raw)))      # ((id, local point)), topmost first
    let !keys = !Filter (KeyOf !raw)

    frames <- for $desk.frames:
        let !own = (Pointer (!Filter (ForFrame key, !hit))) | (Pressed (!Filter (WhenFocused $desk, key,
        !keys)))
        !gen <- rebuild $_.app:
            out <- @Frame _, $_.rect, !own
            @ out
        let !quit = (Quit key) on (!Switch (First !gen))
        $$draw <- $ [[]] then Second !gen
        @ FrameOut !quit, ($Join $$draw)

    let !system = !Union ({"open": Open Counter} on !Filter (IsLaunchChord !keys)), {"raise": RaiseOf !hit}
    let !cmds = !Union !system, (Prefixed "quit:", frames.!quit)
    $desk <- Desk {}, [[]], Nil, 0 then StepDesk $desk, !cmds

    let $scene = Compose $desk, frames.$draw
    @Element Gui.Scene(window="Desktop", draw=$scene)
    @ unit
```

`rebuild $_.app` builds the frame once per key, since an application never changes for a key, and is there only to get `_` as a plain `App`; its result crosses the `rebuild` by the usual two routes, `!Switch` for the quit and a hold joined for the draw lists. `frames` is a plain record of consolidated results: `frames.!quit` is `!(Nat ==> DeskCmd)`, firing with exactly the frames that quit in an occurrence, and `frames.$draw` is `$(Nat ==> [Draw])`, every frame's picture. And `!cmds` is a `!Union` of everything that can change the desk, keyed by text so that a frame's quit and a raise from the same click are two entries folded in key order, never one winning.

Focus and layering are nothing but `$desk.focus` and `$desk.order`, read through tags. `HitFrame` walks

`order` from the top and finds the first frame whose rectangle contains the point, against the desk as the occurrence opened: the user clicked on the stacking they saw. `WhenFocused` passes a key to the frame whose id is `focus` as the occurrence opened. `Compose` wraps each frame's draw list in a `Draw.Clip` at its rectangle and concatenates them in `order`, bottom to top, so the scene the World draws after the occurrence is the stacking after the occurrence. There is no z-buffer, no focus manager and no event capture: three fields of one record, one function that moves them, and the tag law.

## The hard moment

Frames 5 and 9 overlap, 9 on top, 5 focused. The user clicks the visible corner of 5, on the spot where 5's application has drawn its Quit button.

**Occurrence 70.** The World delivers `Pointer (Point 40, 300)` to `0`, the desktop window. `HitFrame` reads `$desk` through a tag: `order` is `[[5, 9]]`, 9's rectangle does not contain the point, 5's does, so `!hit` fires with `((5, (Point 12, 20)))`, the point in 5's coordinates. Two things follow from one firing. `RaiseOf !hit` fires `Raise 5`, so `!system` fires with `{"raise": Raise 5}`. And block `[5]`'s `!own` fires with `Pointer (Point 12, 20)`, so `@Frame` for 5 fires `!to_window` with `{w: Pointer (Point 12, 20)}` where `w` is the application's window path, and `intercept` delivers it to the application's `Gui.Window` slot against the application's list as the occurrence opened.

Inside application 5, the library's `@Window` hit-tests the point against its layout as the occurrence opened, finds the Quit button, and fires its click; the application's `Process.Exit(requests=(Code 0 on !quit))` fires `Code 0`. Back in the frame, `!held`'s block for the exit slot sees `_.!request` fire; `!held` fires with `{p: Code 0}`; `!exit` fires; it crosses the `rebuild` through `!Switch`, which has been listening to this generation since the frame was built; block `[5]`'s `!quit` fires with `Quit 5`; in the desktop, `frames.!quit` fires with `{5: Quit 5}`, and `!cmds` fires with `{"quit:5": Quit 5, "raise": Raise 5}`.

`StepDesk` reads `$desk` through a tag and folds in key order. `"quit:5"` first: frame 5 leaves `frames`, `order` becomes `[[9]]`, `focus` becomes `Nil`. Then `"raise": Raise 5` on a desk with no frame 5 is a no-op, as `StepDesk` was written. `$desk` changes; `$desk.frames` changes promptly; the `for` sees key 5 gone and tears block `[5]` down at commit, with the whole application under it: its holds, its timers, its sockets, every slot, each cancelled by the World under its obligations when the next list arrives without them. `frames.$draw` changes promptly to a map without 5; `Compose $desk, frames.$draw` is a derived dynamic over the new desk and the new ops, and it is glitch-free: there is no instant in which the scene has 5's draw list with 5 missing from `order`, or `order` naming a frame with no draw list. The scene rebuilds, the World draws a desktop with one window. The application's own `Gui.Scene` also rebuilt in 70 (its button drew pressed), in a widget that is dropped at commit, and nobody saw it. Nothing about the exit reached the World below, because the frame held `Process.Exit` back and it was never in the compositor's list.

```
0:        Gui.Window(title="Desktop")
2/#41/0:  Gui.Scene(window="Desktop", draw=[[... frame 9 only ...]])
```

Now the two ways to get this wrong, each a line. If `!cmds` were `!system | (Prefixed ..)`, the raise would win and the quit would be lost: application 5 fired its exit, nothing happened, and it cannot fire again, since the firing was a click's consequence and the click is over. A program with a dead application in a frame that cannot quit. If `StepDesk` had applied `"raise"` before `"quit:5"`, by sorting its commands the other way, the result would be the same desk, because the fold applies both; the only order that matters is that both are applied, and that is what `!Union` promises.

And the click that *raised* 9 while 9's application was quitting on a timer: `Raise 9` and `Quit 9` in one occurrence, from different sources, folded the same way, same answer. The compositor never has a

frame that was raised and then found missing, because it never has a "then."

## What the goals buy

The language is enough: a window manager is an interceptor, a process table is a `for`, focus is a field. The World under the desktop is the software rasteriser, or the X11-over-a-socket interceptor from the Worlds chapter, and it knows nothing about frames.

Nothing happens because an element appeared: an application appears on the desk because `Open` was folded into `$desk`, and leaves because `Quit` was. The World below never sees an application's exit, and the compositor's own run ends only if the compositor fires `Process.Exit` itself.

No widget values: an application cannot keep a reference to its frame, and the compositor cannot keep a reference to an application. When frame 5 is removed from the map, everything under it is gone, and there is no listener somewhere holding the dead subtree alive.

The spec is the thing: the hard moment is a Test World script with `send Gui.Window(title="Desktop"), Pointer (Point 40, 300)` and an `expect_top_none` for frame 5's label afterwards, and the same script runs with the desktop under a `@VBox` or under nothing.

## Where it is awkward, and where it is wrong

Every application runs in the compositor's runtime, in the same occurrence. One application's prompt cycle or overflow aborts the occurrence for all of them, and the Errors chapter's quarantine is the answer within one runtime: the World quarantines the failing subtree on its next run call, and the desk shows a dead frame. That is containment, not isolation. Real isolation means real processes, and then the application is a `Process.Spawn`ed program speaking the GUI vocabulary over a socket, with the compositor as the X11 interceptor's mirror image: an interceptor that turns `Net.Conn` bytes into a `Gui.Scene` and pointer events back into bytes. That is the design for untrusted applications, and this section's design is for trusted ones, which is most of what a desktop runs.

`Compose` concatenates every frame's draw list on every change of any frame, and the World rasterises the whole scene. There are no damage regions, no retained layers, and no GPU textures the compositor could blit instead of redrawing. A scene of ten thousand draw values at sixty frames a second is fine; a browser window's worth of content behind a terminal is not, and the fix is in the World (retained layers keyed by frame id, which the draw values can name) and in the vocabulary (a `Draw` value that references an image the World already holds), not in the language.

Images and video are opaque handles the World allocates (`ImageId`), which means an application that decodes a video frame does so natively and hands the compositor a handle, and the compositor never sees pixels. That is right, and it means the compositor cannot do a pixel effect without asking the World for one. ## 7. A game

A game loop is the one program every programmer has written as a mutable loop and knows is wrong in the same place: input is polled at the top, physics is integrated in the middle, rendering reads the result at the bottom, and the press that arrived between the poll and the render is either dropped, applied a frame late, or applied to half-integrated state, depending on where the input callback happened to run. A tick is an occurrence, the world state is a dynamic, and the press on the frame boundary is the hard moment. A game also shows, better than anything else, that rendering is a description: the element list is the draw list, and the World draws it after the turn, so there is no render call anywhere in the program.

## The vocabulary

```
type Frame = record:
    now: Instant

tag Gui.Window of Element:
    title: Text
    responds: WindowInput              # Pointer, Pressed, Released
    state: Extent

tag Gui.Scene of Element:
    window: Text
    draw: [Draw]

tag Gui.Frames of Element:
    responds: Frame                    # Frame(now: Instant), once per frame the World is about to draw, while
    a slot exists

type Sound = choice Jump | Land | Hit

tag Audio.Play of Element:
    requests: !Sound
    responds: Played
```

`Gui.Frames` is a stream that exists only while a slot for it does; a paused game puts the slot inside a `rebuild` over the pause flag and the World stops asking for frames. `Audio.Play` is a request: a sound happens because the program fired it, after the turn, which is the right time, since the turn is the frame.

## The structure

```
type Entity = record:
    kind: Kind
    pos: ((Float, Float))
    vel: ((Float, Float))
    grounded: Bool

type Held = record:
    left: Bool
    right: Bool
    jump: Bool

type Game = record:
    entities: Nat ==> Entity
    held: Held
    score: Int
    last_t: Int

type Msg = choice:
    Down Key
    Up Key
    Tick Instant

type Effect = choice:
    Play Sound

let Step: fn Game, Nat ==> Msg -> ((Game, [Effect])) = \|g, msgs|
```

```
    Fold msgs, ((g, [[]])), \|st, ignore, m| match m:
        Down k: (((Press (First st), k), Second st))
        Up k: (((Release (First st), k), Second st))
        Tick now: Integrate (First st), now, (Second st)     # physics, collisions, scoring; appends effects

@game:
    ((!raw, $size)) <- @Element Gui.Window(title="Jump")
    !!frames <- rebuild $paused:
        if _: @ !never
        else: @Element Gui.Frames
    let !frame = !Switch !!frames
    let !key = !Filter (KeyOf !raw)
    $paused <- False then not $paused on !Filter (IsPause !key)

    let !msgs = !Union {0: !Filter (KeyMsg !key)}, {1: Tick !frame.now}
    let !stepped = Step $game, !msgs
    $game <- Initial then First !stepped
    @Element Audio.Play(requests=(!Filter (FirstSound (Second !stepped))))

    $sprites <- for $game.entities:
        $phase <- 0 then ($phase + 1) on !frame
        @ Sprite $_, $phase
    @Element Gui.Scene(window="Jump", draw=(Concat (Values $sprites)))
    @Element Ui.Label(text="Score: {$game.score}")
    @ unit
```

`Step` is the game. It folds the occurrence's messages in key order: key edges first (key `0`), then the tick (key `1`), so a press that lands with a frame is applied before the frame integrates. `Press` and `Release` set the held flags; `Integrate` advances every entity by the time since `last_t`, resolves collisions, scores, and appends an `Effect` for each thing that should make a sound. It has no time in it beyond the timestamps it is handed, and it is the function a replay runs.

The tick's timestamp is the frame's own: the program asks `Gui.Frames` to respond with the time, and `Frame(now: Instant)` says when the World is about to draw. Nothing reads a clock through a tag, so there is no lag to state, and the differences between consecutive ticks are the frame intervals `Integrate` needs. A World that stalls delivers a late `now`, not two frames.

`$sprites` is a `for` over the entities, which looks like more than a draw loop needs, and is: `Sprite` over `$game.entities` as one derived dynamic would do for a picture. The `for` earns its keep when an entity has *widgets*: a `$phase` of its own that survives other entities spawning and dying, a `Time.After` for a death animation that must outlive the entity's removal from physics, a particle emitter with holds. A key is an identity, and an entity's local state belongs to its key. The consolidated `$(Nat ==> [Draw])` is concatenated in key order into one `Gui.Scene`, and that scene is the entire output of the program: a presence element, replaced every frame, drawn by the World after the turn. There is no `render()`; there is a dynamic whose value is the picture.

Sounds are requests. `Integrate` cannot play a sound, because nothing in Prism can; it returns `Play Jump` in its effects, and the `Audio.Play` slot fires it. One request per occurrence per slot, so a frame with two sounds either fires an array (make the request type `[Sound]`) or picks the first, as `FirstSound` does here; the program said which.

## The hard moment

The player is on the ground. They press Space in the instant the next frame is due.

**Occurrence 1200.** The World, which batches between frames, delivers `Pressed Space` to the window slot and `Frame` to the frames slot in one occurrence. `!key` fires with `Space`; `KeyMsg` turns it into `Down Space`; `{0: Down Space}` fires. `!frame` fires with `Frame(now: Instant 20000)`; `Tick !frame.now` fires with that time; `{1: Tick ..}` fires. `!Union` fires with both. `Step` reads `$game` through a tag and folds: `Down Space` sets `held.jump`; then `Tick` integrates, and `Integrate` sees `held.jump` true with the player grounded, sets a vertical velocity, clears `grounded`, and appends `Play Jump`. `$game` changes; the player's `$_` in the `for` changes; `Sprite` draws the player one step into the air; the scene rebuilds; `Audio.Play` fires `Jump`. After the turn the World draws the frame with the player already rising and plays the sound. The press was applied in the frame it arrived in.

Here is the version everyone writes first:

```
$held <- Held False, False, False then (Press $held, (!Filter (DownOf !key))) | (Release $held, (!Filter
(UpOf !key)))
let !stepped = Integrate $game, $held, !frame.now
```

A hold for the keys, updated by key events, read by the frame step through a tag. In occurrence 1200, `$held` changes to `jump = True` and `Integrate` reads `$held` through a tag, *before* the occurrence: `jump = False`. The player does not jump this frame. In 1201 the tag reads `True` and the jump happens, one frame late, sixteen milliseconds after the press. That is the input lag that game programmers measure with high-speed cameras (from memory), and here it is a consequence of one line that could be read: the step reads the keys as they stood before this frame. The `!Union` version reads them as the fold has them, with this occurrence's edges applied, and the difference between the two programs is visible in their text rather than in a frame counter.

And the version with `|`: `(KeyMsg !key) | (Tick ..)` loses the tick when a key lands with it, which is a dropped frame on every keypress, or loses the key, which is a stuck or missed input depending on whether it was a press or a release. Neither is a race; both are the program saying the wrong thing, and the two-events-in-one-operation error is what stops the third version, `Integrate $game, !key, !frame`, from ever being written.

Now the same press under a World that gives every input its own turn. Occurrence 1200 has the key alone: `!msgs` fires with `{0: Down Space}`, `Step` sets `held.jump`, nothing integrates, the scene rebuilds with the same picture (derived dynamics do not dedupe, and the World diffs or redraws; a `$Distinct` on the draw list would spare it). Occurrence 1201 has the frame: `Integrate` sees `jump` held and the player rises. Same result, one occurrence later in numbering and the same frame on screen. The program covered both policies because it said what a key means and what a frame means, separately, and let `!Union` say what both together mean.

## Replay, for free

The sequence of inputs the World delivered, with their occurrence numbers and the frame times, is the entire session. Given the same transcript, the program's sequence of scenes is bit-identical, because there is no `now()`, no random source that is not a state element, no scheduling, and no float that depends on which thread ran first. A replay file is a transcript; a demo is a replay; a test of a level is a replay with an `expect` at the end. Lockstep networking is two peers running the same `Step` over the same transcript, exchanging inputs rather than state, and the thing lockstep netcode spends its life fighting, nondeterminism between the two simulations, cannot happen between two Prism runtimes running one program, and the differential test between the two implementations is the guarantee that it cannot happen between a Rust client and a Haskell server either.

## What the goals buy

Simultaneity is the program's: a key and a frame in one instant is the `!Union` and the fold order, which is the first thing a game programmer wants to be able to state and the last thing a loop lets them.

No clock: the step is a function of the frame times the World delivered, so a slow-motion mode is a World that delivers slower times, and a test runs a thousand frames in no time.

You can see what moves: `Step $game, !msgs` is the whole loop, and `$sprites` reads as a picture derived from state, which is what rendering is.

Nothing is chosen for the implementer's comfort: the `for` over entities with per-key state is the same construct as the todo list, and a dying entity's animation outliving its physics is a `Time.After` in its block, not a "pending removals" list.

## Where it is awkward, and where it is wrong

`Integrate` is a computation over every entity every frame, and it builds a new map each time. A persistent map makes an unchanged entity free and a changed one a path copy, which for a few thousand entities at sixty frames a second is fine. Fifty thousand bullets is not this; it is an array of structs updated in place by a loop, which is the kind of code that wants an `update` block and, until then, native code the program calls with the map and gets a map back. The physics engine, the navigation mesh, the particle system: computations, called out to, with the Prism program owning when they run and what their results mean.

Fifty thousand entities is also fifty thousand `for` blocks, each rebuilding a sprite per frame, and the first implementation recomputes the element list per occurrence. For a picture alone, do not use the `for`; derive the draw list from the map in one plain function and keep the `for` for the entities that have widgets, which is a few.

The GPU is the World's. A `Draw` value can name a mesh or a texture by a handle the World allocated, and a shader is a World component with a request element for its parameters; a program that wants to write a shader in Prism is writing a computation in the wrong language. The `Draw` vocabulary for a real game is a retained scene graph with handles, not a list of rectangles, and that vocabulary is a World's to offer; the language only fixes that it is plain data and that it is drawn after the turn.

Fixed-timestep integration with an accumulator, the standard advice for stable physics, is a loop inside `Integrate`, in the plain function, where it belongs. Audio latency is the turn: a sound requested in a frame plays after it, which at sixty frames a second is under a frame of lag, and a rhythm game that wants better schedules sounds ahead through a request with a timestamp, which is a World policy.

And a thousand-player simulation is a server, and the sharding seam is the one from the Worlds chapter: one runtime per region, the World routing players between them, and the handoff of an entity between regions a teardown in one `for` and a birth in another, through a store. ## 8. A dashboard over HTTP

Twelve charts on a page, each asking a server for a time series every few seconds. You have built this, and you remember the three things that went wrong: the chart that flashed the old range's data under the new range's title because the fetch for the old range landed after the user dragged; the twelve identical requests per tick because each chart had its own timer; and the cache that was "simple" until it had a TTL, a stale indicator and a race with the in-flight request. All three are about where state lives and who may change it, and the answer here is that a chart asks for a query

*as state*, something between the chart and the network supplies it, and whether that something polls, pushes or caches is a choice of interceptor that the chart never sees.

## The vocabulary

```
type HttpRequest = record:
    method: Text
    url: Text
    headers: Text ==> Text
    body: Text

type HttpAnswer = record:
    status: Nat
    headers: Text ==> Text
    body: Text

type HttpResult = choice:
    Answered HttpAnswer
    Failed Text

tag Http.Fetch of Element:
    requests: !HttpRequest
    responds: HttpResult

type Range = record:
    from: Int
    to: Int

type Query = record:
    metric: Text
    range: Range
    step: Int

type QueryState = choice:
    Unavailable
    Known ((Instant, [(Int, Float)]))   # fetched when, and the rows
    Errored Text

tag Query.Result of Element:
    query: Query
    state: QueryState

tag Time.Every of Element:
    period: Nat
    responds: Tick

tag Clock of Element:
    state: Instant
```

`Http.Fetch` is the World's: fire a request, get an answer later. `Query.Result` is the dashboard's own tag, a state element whose value is the latest result of a query, and the point of the section is that nothing in the chart says who supplies it. A World with a push channel to the server can accept it natively and push each new result; a World that has only HTTP needs an interceptor to supply it by polling; a test supplies it with `with`. State is identified by value, so two charts with equal queries observe one dynamic wherever it comes from.

## The chart

```
let IsKnown: fn QueryState -> ?QueryState = \|s| match s:
    Known ignore: Some s
    ignore: Nil

let Loading: fn QueryState -> Bool = \|s| match s:
    Unavailable: True
    ignore: False

@Chart: fn Text, $Query -> @unit = \|title, $query| @widget:
    $live <- @Element Query.Result(query=$query)
    $shown <- Unavailable then !Filter (IsKnown (!Updated $live))
    @Element Gui.Scene(window=title, draw=(Plot title, $shown, (Loading $live)))
    @ unit
```

`$query` is a dynamic and `query` is a field, so the element is inside a `rebuild`: a new query is a new slot, and `$live` is the hold of the generations' dynamics joined, starting from the first build's value. `$shown` is the last `Known` the chart has seen, across every change of query: it moves only when `$live` changes to a `Known`, so a fresh slot's `Unavailable` leaves the old rows on screen with a loading badge drawn from `Loading $live`. That is the whole "show stale data while loading" policy, two lines, and it is in the chart because it is the chart's policy; a chart that would rather go blank omits `$shown`.

A dashboard is charts over a shared range:

```
@dashboard:
    $range <- @RangePicker
    let q = \|m| Query m, $range, 60
    @Chart "cpu", (q "cpu")
    @Chart "cpu again", (q "cpu")
    @Chart "mem", (q "mem")
    @Chart "disk", (q "disk")
    @ unit
```

`q "cpu"` applies a lambda whose body reads a dynamic, which lifts: a `$Query` that changes with the range. The first two charts ask for equal elements and get one state. Four charts, three queries, however many suppliers there are below.

## Polling as an interceptor

```
let Encode: fn Query -> HttpRequest = ...
let Decode: fn Instant, HttpResult -> QueryState = \|now, r| match r:
    Answered a: if a.status == 200: Known ((now, ParseRows a.body)) else: Errored (ToText a.status)
    Failed why: Errored why

@Poller: fn Nat, @a -> @a = \|period, @dash| @widget:
    ((result, $slots)) <- intercept @dash, !down, $ supply
    !down <- @Forward $slots
    let supply = \|e| match e:
        Query.Result(query=q): @widget:
            !started <- @started
            !tick <- @Element Time.Every(period=period)
            $clock <- @Element Clock
            !res <- @Element Http.Fetch(requests=((Encode q) on (!started | !tick)))
            $latest <- Unavailable then Decode $clock, !res
            @ $latest
        ignore: @Element e
```

```
    @ result
```

The supplier returns a blueprint for each `Query.Result` element, and the runtime builds it once per distinct element value, in the poller's widget, when the first slot with that value is created, and keeps it until the last such slot is gone. So the two `cpu` charts share one timer and one fetch per period, by value identity, without the poller counting anything. The widget fires a fetch at birth and on every tick, decodes the answer with the clock as it stood, and holds the result; the hold is the slot's dynamic, with `Unavailable` as its value in the occurrence the first slot was created, which is the truth: nobody has asked yet.

Under a World that accepts `Query.Result` natively, pushing a new value whenever the server sends one, the program is `@dashboard` with no `@Poller` around it, and nothing in `@Chart` changes. Polling versus pushes is, from the chart's side, no difference at all; from the World's side it is the difference between a request element and a state element, and the interceptor is the adapter between them.

## Caching as an interceptor

The poller's supplied widget dies when the last slot for its query goes, so dragging the range away and back refetches. A cache that survives that lives outside the poller, where the poller's `Http.Fetch` elements are visible, and it is the HTTP cache of the Worlds chapter with a TTL in its lookup:

```
@main:
    @HttpCache (@Poller 5000, @dashboard)
```

The order matters and is readable: the cache is outermost, so it sees the poller's fetches, and the poller sees the dashboard's queries. A GET the cache has answered within the TTL is answered from the cache's hold through a `Defer`, in the next occurrence of the same turn; anything else is forwarded. The cache does not know what a query is, and the poller does not know there is a cache. Each is a few lines and each is tested alone.

## The hard moment

The dashboard shows the last hour of `cpu`. A poll for that range is in flight. The user drags the range picker to the last day, and the poll's answer arrives in the same instant.

**Occurrence 90.** The World delivers `Answered ..` to the poller's `Http.Fetch` slot for the hour query, at `s/0/3`, and a key to the range picker's focusable, in one occurrence. The range picker's hold changes; `$range` changes; both `cpu` charts' `q "cpu"` change promptly to the day query; each chart's `rebuild` builds anew: generation `#1` is built with a `Query.Result(query=(Query "cpu", (Range day), 60))` slot. The runtime asks the poller's supplier for that element, finds no supplied widget for it, applies `supply`, and builds one now: `@started` fires, the fetch for the day query fires at birth, `$latest` is `Unavailable`. Both charts' new slots get that `$latest`, since the second chart's slot, created in the same occurrence with an equal element, joins the widget already built. In each chart, `$live`'s hold of dynamics changes to the new generation's dynamic, and `$Join` changes to `Unavailable`; `!Updated $live` fires with `Unavailable`; `IsKnown` drops it; `$shown` keeps the hour's rows. The scenes rebuild with the hour's rows and the loading badge.

Meanwhile the answer. It was addressed to `s/0/3`, the hour widget's fetch slot, which existed as the occurrence opened, so it is routed: `!res` fires, `Decode` fires, the hour widget's `$latest` changes to `Known`. Whose dynamic is that? The two `#0` generations' slots, which are being torn down at commit, and nothing else. `$Join` in each chart has already switched to the `#1` inner; its change event fires

on the outer change with the new inner's value, and the old inner's change in the same occurrence is not heard, because the outer changed. So `!Updated $live` fired once, with `Unavailable`, and the hour's fresh rows reached no chart. At commit, the `#0` generations go, the hour's slots go, the hour's supplied widget has no slot left and goes too, with its fetch slot and its timer.

The dashboard's list, as the poller sees it, with the poller's own supplied slot beside it:

```
1/0/#1/0:   Query.Result(query=(Query "cpu", (Range day), 60))
2/0/#1/0:   Query.Result(query=(Query "cpu", (Range day), 60))
s/1/3:      Http.Fetch                                  requested: GET .../cpu?range=day
```

The old range's fresh data did not flash under the new range's title, and the reason is not a guard; it is that the chart's `rebuild` built anew in the occurrence the query changed, and a widget built in an occurrence sees the occurrence from the start, so the new generation's `$live` was never the old slot's dynamic. There is no instant in which the chart has the day's query and the hour's rows as its live value. It has the hour's rows as `$shown`, which it chose to keep, labelled as loading.

**Occurrence 91.** The cache. The day fetch fired at `s/1/3` in 90, and the `@HttpCache` around the poller has a block for that slot, built in 90 when the slot appeared, which saw the request fire in 90 under the mid-occurrence construction rule. If the user had looked at the day range in the last TTL, the cache's `$cache` has it: `!hit` fired in 90, the cache's `Defer` fired in 90, and 91 is the second occurrence of the same turn, delivering the `Defer`'s answer; the cache fires the cached answer down to `s/1/3`; `!res` fires; `$latest` becomes `Known`; both charts' `$Join` change; `!Updated $live` fires with a `Known`; `$shown` moves to the day's rows; the badge clears. The user sees the day's chart before the World has drawn the loading state, because the turn was not over. If the cache had nothing, 91 does not happen; the World performs the fetch after the turn, and the answer lands in a later turn by the ordinary route.

Had the answer for the hour arrived one turn later instead of in 90, its path `s/0/3` would not be in the list the World last received, and the World would discard it. Same screen either way. The hour's rows are gone from the poller and present in the cache, which saw them pass through in 90 as an answer to a GET, and a drag back to the hour is a cache hit.

## What the goals buy

The boundary nests: polling, caching and the push World are three suppliers of one tag, stacked or swapped without touching a chart. The chart you tested with `with Query.Result(..) = Known ..` is the chart you ship under the poller, and the poller you tested with `send Http.Fetch, Answered ..` is the poller you ship under the cache.

"Not yet" as a type: `Unavailable` is a case the chart matched on, and `$shown` is the chart's explicit decision about what to draw then. There is no undefined series and no chart that renders an empty array as a flat line.

Identity by value: twelve charts of one query are one fetch, with no registry of subscribers and no reference count, because the runtime builds a supplier's widget once per distinct element.

Routing against the opened list: the answer that arrived with the drag reached the slot it was for and nothing else, and the slot's teardown at commit is what made the rest of the walkthrough unnecessary.

## Where it is awkward

A poll's phase restarts whenever the supplied widget is rebuilt, which happens whenever a query's last slot goes and a slot with that value comes back; a dashboard whose charts toggle in and out

polls on an irregular schedule, and a World that wanted phase continuity for `Time.Every` could key it by tree position, which under a supplier it cannot see. The cache is the answer: rebuilds are cheap when the data is a hit.

Decoding a large body is a computation on the runtime's thread, inside the occurrence: `ParseRows` over ten megabytes of JSON is ten megabytes of JSON parsed between the response arriving and the scene being drawn, with the whole program waiting. A World that decodes on its IO thread and hands the program rows instead of text has a request type to match, and should; the language offers no way to parse in the background, because there is no background.

Two hundred charts rebuilding two hundred `Gui.Scene` slots on every push of a shared clock, because `Decode` stamps the result with the clock and the stamp changed: `$Distinct` on the rows, not on the whole `QueryState`, or the stamp out of the equality. Derived dynamics do not dedupe and the program has to say where equality matters.

Authentication is a header, and the right place for it is an interceptor around the fetches that rewrites the request with the current token from a state element, which is the path-rewriting sandbox with a different `match` arm. Token refresh on a `401` is a response the interceptor matches, a request to the token endpoint, and a replay of the original, all later, through the ordinary rules; it is a page of Prism and the page is where the bugs in every auth layer have lived.

A websocket for live updates is the World's component, and under it `Query.Result` is native state. The poller is the thing you write when the backend is a plain HTTP API, and the chapter's point is that it is written once, outside the charts, and deleted when the backend grows up. ## Where Prism is the wrong language

Eight programs, and in every one of them the same seam: the part that is "state, things happen, respond" was shorter and safer than you have ever written it, and the part that was a computation was called out to, awkward, or sent elsewhere. The claim of the language is narrow and this section draws its edge, by kind of program, with what a Prism program does with each instead of being it. None of this is a weakness to be fixed later; it is the price of a language whose only subject is time, and a language that tried to be good at these too would stop being good at the thing it is for.

## Batch and throughput work

A compiler, a numerical solver, a log cruncher, a video encoder. There is no time in these: the input is all there at the start, the output is all there at the end, and the only question is how fast. Every mechanism Prism has is overhead here. An occurrence is a transaction with buffered commits, which is pointless when nothing is observing; a dynamic is a value with a change event nobody subscribes to; the dependency graph is built once and used once. The build system of this chapter said this plainly: the compile is a process, the Prism program is the thing that decides when to run it and what its exit means.

What a Prism program does with batch work: it fires it as a request on a process or a native component and receives the result as a response, or it calls a plain function that is native. `Shape` for text, `Rasterise` for a draw list, `ParseRows` for a body, `Evaluate` for a spreadsheet: each is a plain function from plain data to plain data with no occurrence between asking and knowing, and that is the foreign function interface, which is also the whole of it. A plain function has no way to touch time and so needs no rules about it; the program is still deterministic, still a transcript, and the function is a black box the transcript records the results of.

The cost is latency inside the occurrence: a plain function runs on the runtime's thread, between the inputs arriving and the holds committing, and a slow one stalls every input behind it. The dashboard's

ten-megabyte parse and the game's fifty thousand entities were the examples. A computation that takes longer than the program can afford to pause belongs behind a request, in a process or a World thread, with "not yet" in the response type and a turn of latency.

## Sequential algorithms

Anything naturally written as a loop with a cursor: a parser over a token stream, a graph search, a sort you wanted to write yourself, a state machine that walks through a protocol one step at a time with no outside input between steps. These can be written in Prism and the ways of writing them are instructive, because each is wrong in a way that shows what occurrences are for.

The first way is as a recursive plain function, which is fine and is not Prism; it is the functional language Prism has as its value layer, and the closure compiler's recursion depth guard is the only thing the implementation adds. A backtracking search with a mutable visited set wants an `update` block (chapter 3), and until then it is the same ceremony as in any pure language. Use the plain function when the algorithm fits in one; it has no time in it and should not pretend to.

The second way is as a chain of `Defer`s: each step of the loop an occurrence, the cursor a hold, the loop body the hold's change event lifted over the `Defer`'s answer. It runs, one occurrence per step, inside one turn, and it is wrong twice over. Each `Defer` slot is answered at most once per turn, so the loop needs a fresh `Defer` per step, which means a `rebuild` rebuilding per step, which the world core treats as the same `Defer` reborn, so the second firing in the turn exceeds the once-per-turn bound and is an error. That rule exists to stop exactly this, because a loop through the turn mechanism is a loop that freezes the program's output while it spins, and the right place for an iteration to a fixed point is a plain function.

The third way is as a chain of `Time.After`s, one step per turn, so that the World gets to run in between. That is not wrong; it is an animation, or a protocol handshake, or a retry schedule, and those are things that happen over time and belong here. The test is whether a step waits for something outside the program. A handshake waits for the peer; an animation waits for the frame; a sort waits for nothing, and a sort written this way is a sort that takes a frame per comparison.

## Data-parallel pipelines

Map a function over a billion elements, reduce, repeat. The parallelism Prism has is between occurrences and across heights within one, and neither is across the elements of one big array. A `for` over a map with a million keys builds a million widgets, and the runtime visits each one per occurrence that touches it; a lifted `MapValues` over a million-entry dynamic recomputes the map in one closure on one thread. The game's bullets and the chat server's fan-out were the small versions of this. The place for data parallelism is a native function that uses the machine's cores, or a World component that runs the pipeline and offers its progress as a state element; the Prism program is the thing that knows when the pipeline should start and what to show while it runs.

## Programs that are one pure function

A request-response handler with no state between requests: a function from an HTTP request to an HTTP response, a lambda in a cloud, a stateless RPC. The Worlds chapter shows that a WASI handler can be a World whose life is one turn, and it can, and the exercise shows why it should not be: the program builds `@main`, receives one response, fires one request, and is dropped, and every mechanism of the language ran once to compute what a function would have computed with none of them. If there is no state between requests, there is no "what can change this," and the question

Prism answers has no subject. A language that has only the request is the right one, and if the handler grows state (a session, a cache, a rate limit), it has grown into the first bucket and the chat server is the model.

## Hard real time

The control loop said this plainly. Prism gives a deterministic amount of work per occurrence and no bound on how long it takes; a `rebuild` firing in the tick path is a build of unknown size, an occurrence runs on whatever thread the World chose at whatever priority the platform gave it, and the runtime has no notion of a deadline. A controller with a deadline measured in milliseconds runs under a World that pins the loop to a real-time thread and measures the occurrence, and keeps structural changes out of the tick. A controller with a deadline measured in microseconds is firmware, and Prism is the layer that sets its targets.

## Cryptography and secrets

TLS is on the World's side of the boundary, and so is anything else that wants randomness, constant-time operations, or a secret that should not sit in a value. Prism has no effects, and randomness is an effect; a random number as a state element the World pushes is a value in a hold, visible to the transcript, which is the wrong place for key material and the right place for a dice roll in a game. A cipher is a computation. The program sees `tls://` in an address and `Disconnected "certificate .."` in a link type and nothing else, and that is the correct amount of cryptography for a program whose subject is time.

## Large mutable structures

A rope, a matrix, a spatial index, a B-tree. The value layer is immutable and eager, and functional update of a large structure is the one place a pure language is worse to write than an imperative one, which is why `update` blocks and `&` references are in chapter 3. Until they exist, a large structure that changes a little per occurrence is either a persistent data structure the prelude provides (maps already are) or native code the program calls with the old value and gets the new value back. The editor's text, the controller's covariance and the game's entities were the three examples, and in each the answer was the same: the structure is a plain value from the program's side, whatever it is inside.

## Programs whose structure is a call stack

Recursive descent, backtracking, coroutines, anything where "where am I" is a position in code rather than a value in state. Prism has no call stack at occurrence time; a widget block is built once, and what runs afterwards is a graph. An algorithm whose natural state is a stack frame has to make the stack a value, which is the continuation-passing transformation done by hand, and the result is correct and unreadable. Write it as a plain function, where the value layer's ordinary recursion gives the stack back.

## The first bucket

Interactive interfaces, market data and order flow, control, stateful services, incremental computation over things that arrive at different times, simulations and games, shells and compositors. Each of the eight deep dives found the same structure under its problem: a model, a closed set of things that can happen to it, and a boundary where the outside sends and receives plain data. In each, the hard moment was two of those things in one instant, and the answer was a line of the program. And

in each, the parts that were not that structure were sent out through a plain function or a request, which is the foreign function interface and the only one the language needs.

The industry-changing claim is that the first bucket is most of the programs that get written, that the tools for it are bad in a way that is structural, and that a language with time in its types fixes the structural problem. The second bucket is where the claim stops, and the right thing to do at its edge is to call out, cleanly, with "not yet" in the type and the transcript still true. A Prism program is a thin layer of time over whatever computations it needs, and the thin layer is where the bugs were.

## Chapter 9

# Errors

A program's own failures are values, and the failures the language cannot make into values do the least damage the transaction allows. That sentence is the whole chapter. The first half is a discipline the type system enforces: a file that is not there, a socket that closed, an integer divided by zero, an index past the end are all cases of a type, and the `match` that handles them is a compile error to leave out. The second half is about what is left after that discipline has done its work, which is logic errors and broken contracts: a cycle that only exists once two widgets are wired together at runtime, an overflow, a World that answers a path that no longer exists. None of these can be a value, because a value would be a lie about what happened, and so each one has to be contained. The occurrence is a transaction, so the cheapest unit of containment is already there: drop the buffer and the occurrence never happened. The chapter reasons about when a smaller unit is possible (a construction), when a larger one is wanted (quarantine of an owning subtree), what the program and the World each get to see, and why there is no `catch`.

## What can fail, and what cannot

You have had the stack trace that ends in a map lookup. Some callback, three layers down from the click, asked a map for a key that was there a moment ago, got `null` back, and dereferenced it. The fix was a guard, and the guard is now in the code forever, next to a comment nobody will read, and the thing that removed the key is still somewhere else entirely. Most of what a runtime calls an error is this: a value that was allowed to be absent, in a place that assumed it was not, discovered at the worst moment.

Prism's answer to that whole class is to refuse it at the type. `m[k]` returns `?V`, and so does `a[i]`, and so does `a / b` on `Nat` and `Int`. A `match` on a closed choice is exhaustive or it does not compile. There is no null, so there is nothing to dereference. The state a widget asked for has a type with an `Unavailable` case, and the `match` that draws it has an arm for that case. A response from the World is a value of the tag's declared type, and that type says what failure looks like, `Contents Text | Failed Fs.Error`, so a file that is not there is an arm in the program and not an exception in the runtime. Nothing a World component does can throw across the boundary, because the only thing that crosses it is plain data: a component that fails produces a response or does not, and the program's type already allows for a response that never comes.

That removal is the negative space of this chapter, and each piece of it buys something for the implementation as well as the program. No partial matches means the propagation loop never has to unwind through a closure that failed to match. No out-of-range panics means indexing and division are total, each for its own reason. An index past the end has no element, so `a[i]` says `?V`. An integer divided by zero has no value either: no `Nat` or `Int` is the quotient, any one the language picked would be a plausible wrong answer, and the denominator is often data the program has something to say about, so `/` on `Nat` and `Int` returns `?`. `Float` is different on the merits, because the type already has values for the answer. `x / 0.0` is an infinity, `0.0 / 0.0` is NaN, and IEEE says what every operation

does with them, so `/` on `Float` follows IEEE and never fails; a ratio of two sensor readings needs no `match` around a quotient that is already a number, and a program that dislikes infinities tests for them where it cares. Neither division can abort an occurrence. No null means no value representation needs a sentinel. No component exceptions crossing means the runtime's only caller is a World that sends plain data, and the only way a World can hurt the runtime is by sending the wrong plain data, which is a protocol error and is listed below. The runtime does not need a general exception mechanism because the language gave it nothing general to catch.

What is left is short. Grouped by who caused it:

**Logic errors in the program.** A cycle in the prompt graph that the compiler could not see: one through a widget function's argument and result, or one formed at runtime by switching. Arithmetic overflow in `Nat` or `Int`. A plain function that recurses without end. These are bugs in the program, and they are still errors, because no type can make "this program's dependency graph has a loop in it" into a value the program could handle.

**Protocol errors at the boundary.** A response addressed to a path that was not in the list as the occurrence opened. A response of the wrong type for the slot's tag. Two responses to one slot in one occurrence. A `Defer` deferring backwards. A state tag that reaches the World with no component that supplies it. A quarantine instruction for a path no error named. Each is a bug in a World or in an interceptor, not in the program whose slots were addressed, and each is detectable at the moment it happens.

**Failures in the world.** A file that is not there, a socket that closes, a timer the host could not set, a request the World does not support. These are not errors in this chapter's sense at all. They are responses and state values, the tag's types have cases for them, and the oracle cannot fail. The discipline that keeps them out of this chapter, unavailability in the type and never a placeholder, is the main reason the chapter is short.

**Exhaustion.** Memory, stack depth in a plain function, a turn that will not end. The last two have language answers. The `Defer` rule bounds a turn to one occurrence per `Defer` slot plus one, without a budget or a timer. A plain function that recurses without end is stopped by a depth guard in the closure compiler and reported as an error with the function's span, below under pure code. Memory is the host's.

Everything below is about the first two groups and the two exhaustions the language answers. The third group is the type system's and memory is the operating system's.

## The occurrence as the unit of failure

Every reactive system you have debugged had a moment where it was half done. The model was updated and the view was not; the first subscriber had been told and the second had not; a listener had been removed from one list and was still in another. An exception in that moment left the system there, and the next event found it there. Reflex's engine, Spider, throws one exception of its own, `EventLoopException`, when it detects a causality loop, and catches nothing inside propagation; what an exception thrown from inside a `push` or `pull` leaves behind is whatever it leaves behind, and the rule that nothing in the network may throw is kept by discipline rather than by the library (from memory). Prism has the same rule and a different reason it holds.

An occurrence is a transaction. Holds commit at the end. The element list's new value is the list after the occurrence, read by the World after the run call returns. Request firings are read by the World after the occurrence commits. `!Switch` adopts its new inner event at commit. A widget replaced by

a `rebuild` firing is kept until commit and dropped then. Every write an occurrence makes, in other words, is buffered until the occurrence is over, because the language has no assignment: the only state is a hold, and a hold's update is a value waiting for commit. The buffer exists so that nothing can observe a half-updated graph. It is also, with no further work, a rollback mechanism: an occurrence that cannot finish is abandoned by dropping the buffer.

Dropping the buffer means, concretely:

- the pending hold updates are discarded, so every hold keeps the value it had before the occurrence;
- the pending `!Switch` adoptions are discarded, so every switch keeps listening to the inner it listened to before;
- widgets built during the occurrence by `rebuild` or `for` are dropped, and widgets they were replacing are kept, so the widget tree is the tree as the occurrence opened;
- the per-occurrence firing records are cleared, so no event is marked as having fired;
- the action queue is emptied and the current height is reset, so the runtime is ready for the next occurrence;
- the element list is the list as the occurrence opened, which is also the list the World last received.

The run call returns an error instead of a result. Occurrence n did not happen. The inputs that opened it are lost: a click, a response, a push, a `Defer` answer, all of it. The World decides what the loss means, and the policies are below.

The invariant, stated for each party. The *runtime* promises that after an aborted occurrence the program is observably the program as the occurrence opened: every hold, every switch, every slot, every path. The *program* is bound by nothing, because from inside, nothing happened; this is the weakness of the occurrence as the unit, taken up under what the program sees. The *World* is bound to treat the inputs of an aborted occurrence as never delivered: a response it sent was not received, a push it made did not reach the snapshot the program holds, and its own snapshot bookkeeping must say so, which the world core handles by advancing its snapshot only when a run call returns a list. An *interceptor* is bound by nothing special, since an interceptor is Prism and an aborted occurrence rolls it back with everything else.

The one thing that cannot be taken back is work already done: a closure that ran, ran. That is why it matters that closures are pure. A lifted operation that computed a value nobody will commit wasted its time and nothing else. There is no file half-written, because writing files is not something a closure can do; there is no listener half-registered, because subscriptions made during the occurrence belong to widgets that are being dropped. The transaction is cheap precisely because the language already forbade everything a transaction would have had to undo.

Consider the coarser unit, for comparison. Any error ends the process with a message. It is honest, and for a single-program CLI it is the right policy anyway. But a World with many programs in it, a shell or a server that keys a `for` over connections, is then hostage to its worst widget: one client's cycle ends the service for every client. The occurrence as the unit lets such a World drop one occurrence and keep running, and the subtree as the unit, below, lets it drop the one client.

Now the finer unit. Could a failure be contained to a single node, so that the rest of the occurrence completes? Not in general. When a cycle is found mid-propagation, every node below the current height has fired and its value has been read by its subscribers; the nodes above have not. A node that failed in the middle has subscribers that were waiting for it and may have already consumed its old value through a tag; completing the occurrence without it means some subscribers saw an occurrence in which it fired and others saw one in which it did not, which is the half-updated graph the whole language exists to rule out. A smaller unit is coherent only where something already has

a boundary of its own, and exactly one thing does.

The Rust runtime's `catch_unwind` around every action in `propagate` is the backstop under all of this: a panic anywhere in an occurrence clears the action map, resets the height and rethrows, and holds and switches are untouched because they commit only in the final batch; the run call turns that path into a returned error and adds the dropping of widgets built in the occurrence (chapter 11). The Haskell implementation has no buffer to drop and rolls back instead by replaying the occurrence on a fresh local timeline (chapter 14); the observable promise is the same.

## Containment at a construction boundary

A `rebuild` building its block in the middle of an occurrence is the one place the unit of failure can be smaller than the occurrence, and it is the common case for runtime cycles, because a cycle the compiler could not see is almost always one that only exists once some generation of some `rebuild` has been built.

A build has a boundary. It is a closure the runtime calls, which creates a widget, builds children into it, closes its cyclic placeholders, creates its slots, and returns a result. Until it returns, nothing outside the widget refers to anything inside it: subscriptions from inside to outside events exist, but those are weak from the outside's side and die with the widget; the slots it created are not yet in any element list the World has read; the `rebuild`'s event has not yet fired with its result. So a build that fails can be abandoned by dropping the half-built widget, and the occurrence around it continues as if the `rebuild` had built something else.

What it builds instead is an error widget: a widget with one slot, whose element is the presence element `Error`, and no result. The slot sits where the block's slots would have sat, so its path is the generation's path with one slot segment, `0`: a `rebuild` at position `0` in `@main` whose generation `#1` failed has its `Error` element at `0/#1/0`, and a `for` at position `1` whose key `[7]` failed has it at `1/[7]/0`. That rule is the whole of the error widget's address and it is the same one quarantine uses below, so a transcript that expects `Error` after either knows where to look. The `rebuild`'s event does not fire in that occurrence, because there is no result of the block's type to fire with. The parent is left with whatever it had: a `!Switch` keeps the old generation's event, which belongs to a widget that is being torn down and so never fires again; a hold of dynamics under `$Join` keeps the old generation's dynamic, which keeps its last value. The old generation is still torn down at commit, because the dynamic it was built for has changed and the old block is not the right one for the new value either. The `Error` element is in the list after the occurrence, where an interceptor sees it in its child's list and the World sees it at the root.

The failure itself leaves the runtime in the outcome of the run call that contained it, not as the call's error: `Outcome.errors` is the list of every build failure the occurrence contained, each a `RuntimeError` of kind `Build`, with the cause (a cycle, an unsupplied state tag, a native failure in a build-time closure) in its message and the cause's spans, and the failed widget's path as its owner. The world core takes the list from there to its `errors` element, so a program subscribed to its own errors hears of a contained failure in the next turn exactly as it hears of an abort. A World that offers no `errors` still sees the `Error` element in the list, since the list is the one thing every World reads; a World that reads neither learns nothing, which is a World's right.

This is tentative in one respect: a block whose result is plain data, `rebuild $x: @ Compute _`, gives the parent nothing when it fails, and a parent that holds the `rebuild`'s event into a dynamic keeps a stale plain value with no signal in the language that it is stale. If that turns out to be a trap in practice, the alternative is for the `rebuild`'s result type to acquire an error case, `!(Built a | Failed`

`Error)`, which every caller would then have to match on. The cost of that is paid at every `rebuild` in every program for a case that is a bug; the cost of the current answer is paid once, by whoever reads `Outcome.errors` and the `errors` stream. The second seems right until a program shows otherwise.

The same boundary serves `for`. A key appearing in occurrence n builds its block in n; if the build fails, that key's block is the error widget, the consolidated map has no entry for the key's result, and the other keys are untouched. For a `for` over connections in a server, this is one client failing to connect and the rest unaffected, which is the behavior a server wants and the behavior the occurrence as the unit would not give. The last worked scenario walks through it.

And the same boundary serves the first build of all, occurrence zero. The World builds `@main`; the construction is the whole program; a cycle found there means the program cannot start. There is no outside for the build to continue into, so the run call for occurrence zero returns an error, of the cause's own kind rather than `Build`, and no list, and a CLI World prints it and exits nonzero before it has done anything else. There is no partial program to quarantine and nothing to roll back; it is a compile error that arrived late, and the message, below, is written to read like one.

The build-time abort does not need to abort the occurrence, and the reason is sound rather than convenient. The build happened at some height, when the `rebuild`'s dynamic changed. Everything below that height had fired. The new widget read some of those firings through its subscriptions, which is the mid-occurrence construction rule doing its ordinary work. None of those reads changed anything outside the widget, because reads are pure and the values were cached by the firing event, not computed for the new subscriber. Dropping the widget leaves every event's cached value as it was and every subscriber list without entries that were weak anyway. The occurrence continues at the next height with a tree in which the `rebuild`'s slot holds an error widget, which is a legal tree. Nothing has been half done.

What a World does with a contained failure is its policy, and the policy is lighter than for an abort because nothing was lost: the occurrence committed, its requests were performed, and the program is consistent with a visible hole in it. The CLI World prints the error to stderr and continues. Exiting would be the honest choice for a logic error if the error were the whole program's, but a contained failure is by construction one generation of one block, and a server under the CLI World whose seventh connection failed to build should keep serving the other six. The GUI World draws the `Error` element where the block's slots were, which its layout interceptor already knows how to do. The Test World records every entry of `Outcome.errors` for the `errors` step and prints the `Error` element in the transcript at its path.

## Containment to an owning subtree

Here is the afternoon this section is about. You have a dashboard with twelve panels, each a widget from a different author, and one of them, after an update, has a cycle in it. Under the occurrence as the unit, every occurrence in which that panel's cycle fires is aborted, which means every click anywhere on the dashboard that happens to also tick that panel's clock is lost. The other eleven panels are fine, and nobody can use them. What you want is for the bad panel to become a red box that says what went wrong, and for the other eleven to keep working.

Every node in the graph is owned by a widget, because ownership is a tree and the tree is the GC story. A cycle, found during propagation, runs through some set of nodes, and that set has a lowest common owning widget: walk up from each node on the loop until the paths meet. Not every widget has a path, since a plain widget-function application makes a widget but adds no segment; so the owner is the lowest common owning widget that does have one, a `rebuild` generation, a `for` key's

block, an intercept child, or the root. Call it the owner. Quarantine replaces the owner's widget with an error widget, the same one-slot widget the construction boundary builds, at the owner's position in the tree, and tears the subtree down. The `Error` element is then at the owner's path with one slot segment: an owner at `1/[relay]` leaves `1/[relay]/0: Error(..)` in the list, and an owner that is a generation `0/#3` leaves `0/#3/0`.

Quarantine is the Rust runtime's. The Haskell implementation rolls an aborted occurrence back by replaying on a fresh local timeline and takes no `quarantine` input, so the conformance scripts that exercise it are Rust-only, and the differential test compares the two implementations up to the abort and not past it.

It cannot happen in the aborted occurrence, because that occurrence is being rolled back and a rollback that also changes the tree is not a rollback. It happens in the next occurrence the World runs, as an instruction in that run call: `quarantine: [path]`, the owner's path as the error reported it. The runtime performs the quarantine before routing that occurrence's inputs: the owner's widget is torn down, its slots leave the list, the error widget takes its place, and inputs addressed to paths under the owner, which were legal addresses because those paths were in the list the World last received, are delivered to nothing. The runtime reports each one in the outcome with its path, in the same place as request firings from a slot torn down in its occurrence (chapter 11 names the field), so the World can discard the late result it was waiting to deliver and the Test World can check that it was reported rather than lost. The instruction is the World's to give because the policy is the World's: a CLI World that exits on any error will never send it, a dashboard World will send it every time, a server World will send it when the owner is under the connection `for` and exit when it is above.

The owner is the runtime's to name, per kind, from the widget whose closure was running when the error was found. For a cycle, it is the lowest common owner of the loop, as above. For an overflow, or a native failure during propagation, it is the owner of the node whose lifted operation failed, which is the smallest subtree whose removal removes the failing operation. For a contained build failure it is the widget that was being built, which is also where the `Error` element now sits, so a World never needs to quarantine it: the runtime already did the equivalent. For an unknown path, a wrong-typed or duplicate response, a state tag with no supplier at the root, and deferring backwards, there is no owner, because nothing in the tree is at fault, and the field is `Nil`. The runtime remembers every owner it named in its last error until the next run call completes; a `quarantine` path it did not name is refused, before anything else in that call, as a `Component` error naming the path, because the World is allowed to quarantine only what it was told to and a World that sends some other path has a bug the runtime should not perform. The Test World's quarantine script sends the path from the recorded error and nothing else.

What quarantine costs, and the costs are the argument against it:

- The owner's parent gets its result cut off without being told. A `!Switch` over the owner keeps a dead inner and never fires again; a `$Join` keeps the last value; a plain `<-` bind of the owner has an event that is now `!never` and a dynamic that is now constant. The parent's program text says nothing about this. The `errors` stream is where it is said instead, and an interceptor that wants to react sees the `Error` element in its child's list.
- Every hold in the subtree is gone. The panel's state is lost, not paused.
- The owner may be large. A cycle through the root's `!Switch` and a leaf's event has the root as its owner, and the root has no path a World could send; quarantining it would be ending the program with a slower message, and the instruction is not available for it. The error names the owner so the World can see this coming, and a World may reasonably have a policy of quarantining only

below some depth, or only under a `for`.
- The occurrence that found the cycle was still aborted, and its inputs still lost. Quarantine does not recover them; it stops the next occurrence from being lost the same way.

Whether a quarantined subtree can come back follows from what the error widget is: a generation of whatever built it. An owner that is a `rebuild` generation is replaced when the `rebuild`'s dynamic next changes, by an ordinary rebuild, with fresh holds and fresh paths; nothing in the language knows the previous generation was an error widget. An owner that is a `for` key's block is replaced if the key leaves and returns. An owner that is an intercept child in a block that is never rebuilt never comes back, because nothing ever rebuilds it. This is the `rebuild`'s own meaning doing the work, and a retry policy is a `rebuild` over a counter that an `errors` subscriber increments, written in Prism where it can be read. The alternative, a runtime that retries on its own schedule, would be an event from nothing, and the language has none.

Whether the `Error` element is one tag or several, `Error.Cycle`, `Error.Overflow`, decides what an interceptor can match on. One tag, with the kind as a closed choice inside it, is the design: matching on an element's tag is for routing it to the code that handles that kind of element, and nothing handles a cycle differently from an overflow at the element level. A GUI library draws every error the same way; a shell lists every error the same way. The kind is a field, and a `match` on the field is available to anyone who wants it. If a World turns out to want a component per error kind, the tags can split; nothing else depends on it.

```
type ErrorKind = choice:
    Cycle
    Overflow
    UnknownPath
    UnsuppliedState
    DeferBackwards
    Component
    Build

tag Error of Element:
    kind: ErrorKind
    spans: [Span]
    occurrence: Nat
    turn: Nat
    message: Text
    owner: ?Path
    path: ?Path
```

The seven kinds are the closed list, and three of them hold more than their names say. `Component` is every failure that is the World's or an interceptor's and not the program's: a World component that crashed, a response of a type the slot's tag does not declare, two responses to one slot in one occurrence, a quarantine of a path no error named, and a panic in native code that a World or a library put inside a closure; `message` says which. `Build` is a failure contained at a construction boundary, whatever its cause, with the cause named in `message` and its bindings in `spans`. `Overflow` is arithmetic past the width of `Nat` or `Int` and also recursion past the depth guard. `spans` is the loop in order for a cycle and the one binding otherwise; `path` is the path involved for an unknown path or a backwards `Defer`; `occurrence` and `turn` say when; `owner` is as above.

`Span` is a source location: file, line, column, and the binding's name. `Path` is the opaque path type an interceptor already handles in `$(Path ==> Slot)`; a program cannot make one, but it can hold one, compare it, look it up in a map it was given, and ask whether one lies under another with the prelude's `Under: fn Path, Path -> Bool`, which is the one operation on paths beyond equality and

ordering and exists for the shell below, which needs to know whose subtree an owner is in.

## Static causality and runtime detection

A button that relabels itself is legal and useful: `!!clicks <- rebuild $label: @Element Ui.Button(label=_)` with `$label` derived from the clicks is the worked examples' counter, and nobody wants it rejected. `!!inner <- rebuild $mode: match _: Double: @Double !out` with `let !out = !Switch !!inner` has the same text, bind for bind, and one mode over it is a loop: the generation's result feeds the switch that feeds the generation. Nothing in either block's text tells the two apart, because what makes the second a loop is which generation gets built, and that is a runtime fact. So the compiler finds every cycle it can see, and the question is what it can see.

Within one widget block, the bindings form a graph whose edges the compiler knows: every lifted operation is a prompt edge from its operands, every `|` and `for` consolidation and `rebuild` firing is prompt, and every tag, `!Switch` choice and hold value side is delayed. A cycle made of prompt edges alone is a compile error that lists every binding in it, with spans, and `rebuild` is not a delay because its event fires in the occurrence of the rebuild. That check is complete for the block.

What it cannot see is through a function. `!a <- @Plus !tick, !b` followed by `!b <- @Double !a` is a prompt cycle if `@Plus`'s result depends promptly on its second argument and `@Double`'s on its first, and the function's type, `fn !Int, !Int -> @!Int`, does not say. The compiler has to assume either that every function is prompt in every argument, which rejects the `@clicker` of the worked examples and every supervisor pattern, or that none is, which is what it does, and leaves the rest to the runtime. And it cannot see through switching at all, which is the relabelling button and the doubling relay above: whether some generation's inner event depends promptly on `!out` depends on which generation is built.

A function type could say more. Alongside `fn !Int, !Int -> @!Int` the checker could record which arguments the result promptly depends on, an effect in the type, inferred for every function the way the block check infers it for bindings and checked at every call. Then `@Plus` would be known prompt in both arguments and the cycle above would be a compile error at the call site. This is the right kind of answer and a research project: it has to pass through `rebuild`, `for`, `intercept` and higher-order arguments, and it has to be erased, since generic code is. It is not in this specification; its effect, if it comes, is to make the runtime check below a backstop rather than the mechanism, and nothing else in the design moves.

The runtime finds a prompt cycle by the one property a prompt cycle has: no consistent assignment of heights. Each node's height is one more than the greatest of its prompt sources' heights, and a node on a cycle would need to be above itself. Computing a height is a walk upstream, and a walk that reaches a node it is already computing has found the cycle. The runtime has two places to run that walk, and the choice between them decides which unit of failure a cycle lands in.

The first is at the moment an edge is created. A cyclic placeholder is closed during a build; a widget function's result is connected to its arguments' sources during a build; a `!Switch` adopts a new inner at commit. If the height walk runs then, eagerly, a cycle is found at the construction that closed it or at the commit that adopted it, and never in the middle of propagation. The second is lazily, on the first firing through the new edge, which is what a cache-and-invalidate scheme does on its own: the switch adoption clears every cached height downstream, and the next firing recomputes them and finds the loop then, mid-occurrence, in the occurrence after the switch.

Eager is right, and the reason is which unit it lands in. A cycle found at a `rebuild`'s build is contained

to the build. A cycle found at a `!Switch`'s adoption is found at commit, and the walk is the first step of commit: it runs after propagation is complete and before the final batch in which holds and switches commit (chapter 11 fixes that batch as the last thing propagation runs), so aborting there discards a buffer nothing has yet been written from; the occurrence is lost, but cleanly. If the walk ran after holds had committed, a rollback would need to undo them, which is the undo log the design does not have. A cycle found lazily is found in the following occurrence, which may be a click on an unrelated button, aborted for a bug it had nothing to do with, and the report names the occurrence that tripped over the cycle rather than the one that made it. The cost of eager is a walk upstream per edge creation, which the height cache bounds: the walk stops at any node whose height is cached, and a node's cached height is valid exactly as long as nothing below it has been invalidated, which is the rule every event type in the runtime already keeps. What would change this is a measurement showing the walk dominates build time in programs with deep fan-in; nothing suggests it, since the same walk runs lazily anyway on the first firing.

The remaining runtime check, that a node never schedules at or below the current height, stays as what it is in the runtime today: an invariant assertion. With eager detection it should never fire for a program's cycle, and if it fires the bug is in the runtime, which is the one case where a panic that ends the process is the right report.

So the units line up with where cycles are found. A cycle in one block is a compile error. A cycle through functions closed at the build of `@main` is an error in occurrence zero and the program does not start. A cycle closed at a `rebuild` or `for` build is contained to that build. A cycle closed by a switch adoption aborts the switching occurrence and is the one case where quarantine has work to do. The invariants a runtime must keep in every case: it never deadlocks, it never loops, it never fires a node twice, and after the error the program is the program as the occurrence opened. The current runtime's `try_lock` on the height mutex is the detection in its crudest form, a re-entry check that panics with the kind of cycle but not its nodes; the walk it replaces will keep the nodes on the loop as it goes, so the report can list them.

## Whether pure code may panic

The counter you shipped has been running for four years and `$count` is a `Nat`. Nobody has clicked it eighteen quintillion times. But somewhere a `$total` is a sum of file sizes in bytes, and someone pointed it at a directory of disk images, and `+` on `Nat` wrapped, and the label now says the directory is three kilobytes. That bug is worse than a crash, because it is a plausible wrong answer, and the discipline here is chosen against it.

Three disciplines are possible, and the choice is between what the language promises and what it can keep.

*Panic and abort.* Overflow is detected and the occurrence aborts. Native code the runtime calls may fail, and the occurrence aborts. The transaction gives this for free and it is honest about logic errors.

*Checked everywhere.* Every operation that can fail returns a value that says so: overflow returns `?` like integer division does, or arithmetic wraps and never fails, and native code returns results. The runtime then never sees a failure from user code at all, and the only aborts are protocol errors, which are World bugs. This is the Reflex rule, nothing in the network may throw, made into a type.

*Both.* The default spelling of arithmetic detects overflow and aborts the occurrence. An explicit spelling exists for code that wants overflow as a value or wants wrapping. The transaction stays as the backstop for native code.

The third is the discipline here, for two reasons that pull in opposite directions and meet in the middle. Against checked-everywhere: `$count + !delta` has to read as arithmetic, because "you can see what moves" is a goal, and `$count + !delta` returning `?Int` would put a `match` around every sum in every program for a case that is a bug when it happens. Integer division returns `?` because integer division by zero is an ordinary value's ordinary behavior, the denominator is often data, and the program usually has something to say about it; `Float` division has infinities to answer with and returns a `Float`. Overflow is not like that: a counter past the machine's width is a bug, and a bug should stop the occurrence rather than be matched on. Against panic-and-abort alone: a hash function wants wrapping multiplication, a checksum wants wrapping addition, and a program that reads sizes from a file wants to know, as a value, that the sum did not fit. So the explicit spellings exist, as prelude functions rather than operators: `Wrap.Add`, `Wrap.Mul` and their siblings wrap; `Checked.Add`, `Checked.Mul` return `?`. They are functions because the hot loop that needs them can afford a name and because an operator spelling is one more token every reader would have to know for every sum they read. If hashing and PRNG code turn out common enough to make `Wrap.Mul a, (Wrap.Add b, c)` painful, an operator spelling can be added without changing anything else.

Native code is why the backstop stays. The runtime calls host functions for text, for maps, for arithmetic, and eventually for whatever a native backend does, and a language that promised its pure code never fails would be making a promise about Rust code it cannot keep. A backstop that is never used costs nothing: it is the same `catch_unwind` the runtime already has. What it buys is that a bug in a native function is reported as an aborted occurrence with the span of the Prism operation that called it, rather than as a process that died. Its kind is `Component`, because the code that failed is code a World or a library put inside a closure and not the program's own; during a build the same failure is contained and reported as `Build`.

Recursion without end in a plain function is the one exhaustion the language turns into an error. A host stack overflow is not a panic in Rust and ends the process, so the closure compiler counts call depth and aborts the occurrence past a limit, as an `Overflow` error with the function's span and the owner of the node whose closure made the call. The limit is the implementation's; the specification says only that a conforming implementation does not hang and does not die. The guard is there to catch recursion without end, not to ration recursion with one: a recursive-descent parser over a long input is an ordinary program and has to work. So the limit is set by memory, not by the host thread's default stack size, which means the closure compiler keeps Prism call frames off the host stack (or runs evaluation on a stack it grows), and a depth a program reaches on purpose is a depth the guard allows.

## What the program sees

From inside an aborted occurrence, nothing. That is the weakness of the occurrence as the unit. A program whose click was lost cannot tell that it was lost, because from the program's side the click did not happen, and a program that could tell would be a program that observed an occurrence that was rolled back. The transaction is doing exactly what it should, and the consequence is that the program's own text is silent about runtime errors, always.

What the program can see is what a World tells it, and a World tells a program things through elements. So the program's view of runtime errors is an element:

```
tag Errors of Element:
    responds: Error
```

The response is an `Error` element value, the same value the presence element holds when a build fails or a subtree is quarantined, so there is one error value in Prism code and one at the boundary, and the `match` that reads one reads the other. `errors` declares `responds` alone and nothing else: it is a stream element in the sense of chapter 7, observations arriving on their own, and `Kind Errors` is `Kind(requests=false, responds=true, state=false)`. A World that offers it lists it in `World.Tags`, and the obligation that comes with offering it is the world core's to meet: every `errors` slot in the list as the delivery occurrence opened receives every error from the last turn, as a response, in the first occurrence of the next turn, one firing per error. Aborts and contained build failures travel the same way, the first from the run call's error and the second from `Outcome.errors`, so a program cannot tell from the element which it was except by the `kind`. A slot created in the delivery occurrence hears nothing about those errors, under the creation rule, and no error is replayed to a slot that did not exist when it was reported; a program that wants to know about errors subscribes before they happen, like a program that wants to know about clicks. The Test World is built on the core and delivers the same way, and because it accepts every tag, a script may also `send` an `Error` value of its own to an `errors` slot, which is how a program's error bar is tested without a program that fails. The tag and chapter 10's `errors` step share a name on purpose: the step asserts that the last turn ended in a matching error, and the tag is how a program under test hears of the same error one turn later.

A program subscribes to its own runtime errors the way it subscribes to anything else, binds the event, holds it, draws it, and the discipline is the ordinary one: a response arrives in a later occurrence than the thing it is about, and nothing in the aborted occurrence survives to receive it, which is right because nothing in it existed.

```
@with_error_bar: fn @a -> @a = \|@child| @widget:
    !err <- @Element Errors
    $last <- Nil then Some !err
    result <- @child
    @Label (ShowError $last)
    @ result
```

Because it is an element, every property elements have applies. An interceptor can offer `errors` to its children itself, so a shell that hosts other programs can give each program its own errors and keep the others' private: it makes its own `errors` element at the root, and for each error it receives it looks at the `owner`, finds the child whose subtree the owner is under, and answers only that child's `errors` slots. The lookup is `Under`, from the section above, against the paths the shell already holds in each child's `$slots`; an error with no owner, a protocol error, goes to the shell's own log and to no child, since no child caused it. A sandbox can refuse the element, by supplying nothing and never answering, or answer with a redacted value whose `spans` are empty. A test can assert on it. A program that does not create one hears nothing, which is the default and is right: most programs have nothing useful to do with "a cycle was found in a widget you did not write".

The second thing a program can see is the `Error` presence element, in two places. In its own list, where the quarantine or the failed build put it, visible to the GUI library's layout interceptor, which draws it, and to the World, which sees it at the root. And in a child's list, from an interceptor's side, where `$slots` holds a slot whose element is `Error(kind=Cycle, ..)`, and the interceptor may do with it what it does with any element: re-emit it, replace it with a friendlier element, log it, or count it. A supervisor is this:

```
IsError: fn Element -> Bool = \|e| match e:
    Error(..): true
    _: false
```

```
@Supervise: fn @a -> @unit = \|@child| @widget:
    !restart <- @Button $"Restart"
    $gen <- 0 then $gen + 1 on !restart
    rebuild $gen:
        ((ignore, $slots)) <- intercept @child, !down, $ (\|e| @Element e)
        !down <- @Forward $slots
        let $crashed = Any $slots, IsError
        @Label (if $crashed: $"crashed" else: $"running")
    @ unit
```

`@Forward` is the forwarding loop from the sandbox example, so the child's elements reach the outside unchanged, the `Error` element among them. `Any` is a plain function over the map's slots' elements; `$crashed` is a dynamic by lifting. A restart is a change of `$gen`, which rebuilds the `rebuild`, which builds a fresh child with fresh holds and fresh paths: the retry policy is a counter and a `rebuild`, in the program, where it can be read.

Why there is no `catch`. A `catch` would let a parent react to a child's failure in the occurrence it happened, by running an arm that was not otherwise going to run. That is an effect handler: control transferred from the point of failure to a handler chosen by dynamic extent, with the parent's hold updated by something other than its declared event. The language has none of those, on purpose, and the promise that a hold's definition lists everything that can change it is only true because there is no back door. The supervisor above handles a child's failure in the only way the language handles anything, by reading what the child emits and rebuilding on an event, and it handles it in a later occurrence, as every reaction to the outside is. Suppose `catch` existed anyway, as a widget combinator `@Catch @child, @fallback` that built the fallback when the child failed. It would have to decide what the child's result is in the occurrence of the failure, which is the question the construction boundary answered by not firing; it would have to decide whether the fallback is built in the aborted occurrence, which cannot be, since that occurrence is being rolled back; so it would build the fallback next occurrence, which is `rebuild` over an `errors` subscription, which the program can write. Nothing the combinator could do is beyond the primitives, and the hand version is the real one.

## What the World sees

A differential test has failed. The Rust transcript's last line says the turn ended in a cycle and the Haskell transcript's does not, and what you need, from that one line, is which Prism binding closed the loop, without opening either runtime. The error value out of the run call is designed for that moment: a value with everything a human or a test needs to find the bug, and nothing that names a runtime's own objects.

An aborted occurrence returns an error and no list. A completed occurrence returns the list and, in `Outcome.errors`, every build failure it contained. The two never mix: an error that aborted is the `Err` of the call, and an error that was contained is an entry in the `Ok`. Both are the one type:

```
#[derive(Clone, Copy, PartialEq, Eq)]
pub enum ErrorKind { Cycle, Overflow, UnknownPath, UnsuppliedState, DeferBackwards, Component, Build }

pub struct RuntimeError {
    pub kind: ErrorKind,
    pub message: String,                  // the human form of what the fields do not hold
    pub occurrence: u64,
    pub spans: Vec<Span>,                 // the loop in order, or the one binding
    pub owner: Option<Path>,              // the lowest owning addressable node, for quarantine
```

```
    pub path: Option<Path>,                // the path involved, for UnknownPath and DeferBackwards
    pub element: Option<ElementValue>,     // the element involved, for UnsuppliedState
}

pub struct Outcome {
    pub list: Vec<(Path, ElementValue)>,
    pub requests: Vec<(Path, Vec<Value>)>,
    pub result: Value,
    pub errors: Vec<RuntimeError>,         // contained build failures, this occurrence
}
```

One record with a kind, rather than one Rust variant per kind, because every reader of it prints it the same way and the `Error` element a program matches on has a closed `kind` field; a variant per kind would be unpacked into this record at the boundary anyway. What a kind needs beyond the common fields goes in `message` and `spans`: the loop's edges are `spans` in order with the edge kind in the message, the overflow's operands and operation are in the message, the chain of interceptors a bad response passed through is in the message with each one's `intercept` span in `spans`. The runtime does not know what a turn is, so the turn is not here; the world core pairs the error with the turn it was in when it reports (chapter 13), and that pair is what becomes the `Error` element's `turn` field and the transcript line.

The chain matters for protocol errors because the culprit is whichever interceptor sent the bad map or failed to supply, and the chain says who was in a position to. The world core, not the runtime, constructs the `DeferBackwards` value, because the runtime does not know what `Defer` is; it is the same `RuntimeError` type, with the slot's path and the chain, reported through the same channel. The core also constructs `Component` for its own components' failures. The runtime raises the other five, and `Component` for one thing of its own: a panic inside propagation whose payload is not the runtime's, which can only come from native code a World or a library put in a closure, and the two protocol failures it refuses before scheduling anything, a wrong-typed response and a duplicate response, which are the World's bugs and are named as such.

The compiler's part is that every cyclic placeholder, every lifted operation and every `intercept` is named after its source span, so that a runtime cycle lists Prism bindings and not runtime node ids. The typed AST (Part III) keeps spans through closure conversion for exactly this. A runtime that reported `SwitchHold#4412 -> FilterMap#4419` would be reporting its own implementation.

World policies, each the World's own:

- A *CLI World* prints an abort to stderr, in the format below, and exits nonzero. Nothing else is right for a single program that is a logic error; quarantine would leave a program running with a hole in it, and a user of a command-line tool wants the tool to fail rather than produce output from a damaged program. A contained build failure it prints and continues past, as above, since the occurrence committed and nothing was lost.
- A *GUI World* shows the error and keeps running, and sends the quarantine instruction on its next run call when the owner is below the root, so that the panel becomes a red box and the rest of the window keeps working. Its `errors` element delivers the error to any program subscribed. What it does when the owner is the root, or when there is none, is a question of taste; ending the run with a dialog is honest, and so is keeping the window up with the error drawn over it and every further occurrence lost the same way.
- A *server World* logs the error, quarantines when the owner is under the connection `for`, which is one client, and exits when it is above, which is the service.
- The *Test World* records the error for the `errors` step and the transcript. A turn that ends in an

error is a turn whose transcript line says so, and the differential test compares that line between implementations like any other; two implementations that disagree about whether a program has a cycle have found an ambiguity in the specification, which is the find the suite exists for.

## Errors inside `Defer` turns

The sandbox denied a request in occurrence 1 of a turn, and that occurrence committed. The child reacted to the denial in occurrence 2 of the same turn, and its reaction overflowed. The user clicked once. Half of what that click did is committed and the other half is rolled back, and the question is what the turn means now.

A turn is several occurrences when `Defer` slots fire: the input occurrence, then one per round of answers. The occurrence is what rolls back and the turn is what stops. The alternative, rolling back the whole turn, would need the runtime to keep every occurrence's buffer until the World acts, which is a multi-occurrence undo log the runtime does not have; it is the same machinery that optimistic parallelism across occurrences would need (Part III), and if that machinery arrives, whole-turn rollback can be offered with it. Until then, the committed occurrences stand, which is coherent because each was a complete transaction on its own, and the turn ends at the abort.

The `Defer` answers the aborted occurrence was delivering are consumed: the slot counts as answered for this turn. Re-delivering what just failed would, in a deterministic program, fail the same way, and would also break the bound on a turn's length, which depends on each slot being answered at most once. `Defer` slots that fired in committed occurrences and were not yet answered are left waiting, as a program is always left when a response does not come, in a state its type allows; the world core does not run further occurrences of a stopped turn. The World acts on the turn's committed state: it performs the requests of the occurrences that committed, drops the aborted occurrence's, and reports. The error is on the `errors` stream in the first occurrence of the next turn. The core's loop for this is in chapter 13; the rule here is the one it implements, and the Test World's `errors` step sees the list as it stood after the last committed occurrence.

## Errors in World component code

The host's `read` panics on a path with a NUL byte in it, because the program interpolated a text that came off a socket into an `Fs.File` field and nobody validated it. Or a toolkit call returns an error code the component's author never checked. Either way the failure is in the World, in host code, and the program whose slot was involved did nothing a type could have refused.

The world core, which calls every component, contains that failure, and the policy follows from what a request already means to a program. A request component that fails leaves the slot unanswered; a request may always go unanswered, so no lie is told, and `Unsupported` would be one, since the World supports the tag and failed. A state component that fails leaves the previous snapshot value standing, with no push, so every slot still agrees and the program sees a state that did not change; in occurrence zero, with no previous value, it is the same unsupplied-state error a missing component gives. A presence component that fails, a window the host could not open, is reported and otherwise ignored, as an unsupported presence element is. In every case the failure is reported as a `Component` error with the tag, the path and the host's message, through the same channel as runtime errors, so a program subscribed to `errors` and a user reading stderr both learn that the file read did not fail in the filesystem but in the World. Chapter 7 holds the normative text for these obligations and chapter 13 the code that meets them; this chapter only places `Component` among the kinds. None of it is a language rule: a World that bypasses the core gets the host language's own behavior, which is a crash, and deserves it.

## Error messages worth getting right

A message is read by someone who has just lost a click and wants to know which line to look at. The forms below are what the CLI World prints and the Test World records, rendered from the error's `message` and its fields; they are examples of the rendering, not a format the fields must reproduce, and chapter 10's transcript line is the fields themselves, which is what the differential test compares. Each names Prism source, lists what was involved in order, and says what was done about it.

A cycle found at a switch:

```
error: cycle found when a switch adopted a new inner event
  occurrence 7 of turn 4 was aborted; nothing from it was committed

  the loop, in order:
    relay.prism:9   !out      = !Switch !!inner | !tick       (switch, adopted this occurrence)
    relay.prism:6   !!inner   = rebuild $mode: ...                 (generation #3, built this occurrence)
    relay.prism:8             @ !out * 2                        (lifted over !out: prompt)
    relay.prism:9   !out                                        (back to the start)

  the cycle passes through no delayed edge once the switch has adopted generation #3
  owner: @relay at path 1/[relay] (quarantine would replace this node)
```

A cycle found at a build:

```
error: cycle found while building a rebuild block
  the build was abandoned; the rebuild's slot holds an Error element; occurrence 7 of turn 4 continued

  the loop, in order:
    plus.prism:4    !a        = @Plus !tick, !b                 (result depends promptly on !b)
    double.prism:3  !b        = @Double !a                      (result depends promptly on !a)
    plus.prism:4    !a                                          (back to the start)

  @Plus and @Double are widget functions; the block check cannot see through them
  at: view.prism:12, rebuild $page: ..., generation #1 (path 0/#1)
```

An overflow:

```
error: arithmetic overflow
  occurrence 3 of turn 3 was aborted; nothing from it was committed

  totals.prism:4    let $doubled = $count * 2
                    Nat multiplication: 9223372036854775808 * 2 does not fit in a Nat
  $count changed in this occurrence from 4611686018427387904 (totals.prism:3, $count + image_size on !add)
  owner: @totals at path 0/#0 (quarantine would replace this node)

  to keep the overflow as a value, use Checked.Mul; to wrap, use Wrap.Mul
```

A response to a vanished path:

```
error: response sent to a path not in the element list
  occurrence 5 of turn 5 was aborted; nothing from it was delivered

  path 0/#1/0 is not in the list as this occurrence opened
  (it was in the list after occurrence 3; a `rebuild` generation change at clicker.prism:3 replaced it in
  occurrence 4)
  sent by: the World
```

The same error from inside an interceptor:

```
error: response sent to a path not in the element list
  ...
  sent by: @Rewrite at rewrite.prism:5, through its !down argument (rewrite.prism:6)
  the child's list as this occurrence opened does not contain 3/#2/0
```

An unsupplied state tag:

```
  error: no supplier for a state element
    the build was abandoned; the rebuild's slot holds an Error element; occurrence 2 of turn 2 continued

    element: Game.Score(player="ada")       declared at game.prism:4 with state: Nat
    created at: hud.prism:7, @Element Game.Score(player=p)
    reached: @Sandbox (sandbox.prism:12) which forwarded it, then the World
    the CLI World has no component for Game.Score
```

Deferring backwards:

```
  error: a Defer slot fired again after it was answered in this turn
    turn 6 ended; the second request was discarded and will not be answered

    Defer at path 1/[4]/#0/1, created at sandbox.prism:19
    answered in occurrence 9; fired again in occurrence 9
    the request event: unit on _.!request (sandbox.prism:19)
    a retry in the occurrence of a denial is a loop; put the retry behind Time.After
```

Two responses to one slot:

```
  error: two responses to one slot in one occurrence
    occurrence 2 of turn 2 was aborted; nothing from it was delivered

    path 0/#0/0 (Ui.Button(label="Increment"))
    a slot receives at most one response per occurrence; merge them with | where they are made
    sent by: the Test World (a line with two deliveries to one slot)
    this is a Component error: the fault is the sender's, not the program's
```

The things every message has: which occurrence and turn, what was done (aborted, abandoned, discarded), the Prism spans in the order a reader would follow them, the owner when there is one, and for a protocol error, who sent it. The last line is advice when the fix is known, and most of the time it is.

## Worked scenarios

Six programs, each with one error in it, each followed by what happens occurrence by occurrence. The conventions are the worked examples': paths written as bind positions with `#n` for a `rebuild` generation and `[k]` for a `for` key, and the prelude's plain functions.

### A runtime cycle through two widget functions

```
@Plus: fn !Int, !Int -> @!Int = \|!x, !y| @widget:
    @ !x | !y + 1

@Double: fn !Int -> @!Int = \|!in| @widget:
    @ !in * 2

@relay:
    !tick <- @Element Time.Every(period=1000)
    $mode <- Plain then Toggle $mode on !tick
    !!inner <- rebuild $mode:
        match _:
            Plain: @ !never
            Double: @Double !out
    let !out = !Switch !!inner | !tick
    $last <- 0 then !out
```

```
    @Label $"last: $last"
```

Two cycles are in here and they are found in different places. Take the simpler one first, in a different `@main`:

```
@loop:
    !tick <- @Element Time.Every(period=1000)
    !a <- @Plus !tick, !b
    !b <- @Double !a
    @Label $"a"
```

`!a` and `!b` are a recursive group. The block check sees `!a` bound from `@Plus !tick, !b` and `!b` from `@Double !a`, and because it does not know whether `@Plus`'s result depends promptly on its arguments, it does not reject the group. The build of `@loop` in occurrence zero builds the group with placeholders: `@Plus` subscribes its `|` to `!tick` and to the placeholder for `!b`; `@Double` subscribes its `* 2` to `!a`'s placeholder; the placeholders are closed. Closing `!a`'s placeholder creates the edge from the `|` node to the placeholder's forward, and the eager height walk runs: forward of `!a` → `|` → `!b + 1` → forward of `!b` → `!a * 2` → forward of `!a`, which is being computed. The cycle is found during the build of `@main`. Occurrence zero returns an error of kind `Cycle` and no list, the CLI World prints the build-form message with the loop listed through `plus.prism` and `double.prism`, and exits nonzero. The program never started, which is what should happen to a program whose structure is wrong before any input exists.

Now `@relay`, where the cycle only exists in one mode. The block check sees `!!inner` depends on `$mode`, the `Double` arm depends on `!out`, `!out` depends on `!!inner` through `!Switch`, and `!Switch` is a delayed edge, so the block is legal. It has to be: this is exactly the structure of a button that relabels itself, and that is legal and useful. For the scenario, `@relay` is one panel of a dashboard, built by a `for` over the panel names at position `1` in `@main`, so its widget is at `1/[relay]`, its timer slot at `1/[relay]/0` and its label at `1/[relay]/1`.

**Occurrence 0 (build).** `$mode` is `Plain`; the `rebuild` builds generation `#0`, whose block is `@ !never`; `!!inner` fires with `!never`; `!Switch` has it from occurrence 1. `!out` is `!never | !tick`. The list has a `Time.Every` and a label under `1/[relay]`.

**Occurrence 1.** `Tick` to the timer slot. `!tick` fires; `Toggle $mode on !tick` fires with `Double`; `$mode` changes promptly; the `rebuild` builds anew: generation `#1` builds `@Double !out`, which subscribes `!out * 2` to `!out`. This edge, from `!out` into the new generation, is not a cycle yet: nothing from the generation feeds back into `!out` until the switch adopts it. The build succeeds. `!!inner` fires with the new inner event. `!out` fires with the tick's value from the `|`'s right side; `$last` is updated in the buffer; the label rebuilds in the buffer. Propagation finishes. At commit, before the batch in which holds and switches commit, the eager walk runs for the switch's adoption of generation `#1`'s event: switch → `!out * 2` → `!out` → switch. Found. The commit is abandoned: `$mode` stays `Plain`, `$last` stays `0`, generation `#1` is dropped and generation `#0` is kept, the switch keeps `!never`. The run call returns the switch-form message, with `owner 1/[relay]`. Occurrence 1 did not happen and the tick was lost.

**Occurrence 2, under a CLI World.** There is none; the World exited.

**Occurrence 2, under a GUI World that quarantines.** The error named the owner: the lowest common owning widget of the switch (bound in `@relay`'s block) and the generation's `* 2` node (owned by the generation, owned by the `rebuild`, owned by `@relay`'s widget) is `@relay`'s widget, which has no segment of its own, so the owner is the key's block at `1/[relay]`. The World's next run call says `quarantine: [1/[relay]]`. The runtime finds the path among the owners it named last, tears the

subtree down before routing, and the list has `1/[relay]/0: Error(kind=Cycle, ..)` where the timer and the label were. A tick queued for `1/[relay]/0` under the old list is delivered to nothing and reported with its path, and the World discards it. The dashboard around it keeps working, and its layout interceptor draws the error. Had the World sent `quarantine: [1/[clock]]`, a panel no error named, the run call would have returned a `Component` error naming the path and run nothing.

**Occurrence 2, under a GUI World that does not quarantine.** Every tick toggles `$mode`, every toggle is aborted, every tick is lost, and the program sits at `last: 0` forever with an error on stderr per second. Nothing is inconsistent; the program is stuck in an honest way. The `errors` stream tells any subscriber so.

The fix in the program is to say what the program meant. If `Double` mode was meant to double the tick, `@Double !tick` instead of `@Double !out` has no cycle. If it was meant to feed back, the feedback has to pass through a delayed edge that is not the switch's choice, which means a hold: `$prev <- 0 then !out` and `@Double ($prev on !tick)`, which doubles the previous value at each tick and is what feedback means in a language without prompt loops.

## An overflow in a derived dynamic

```
@totals:
    !add <- @Button $"Add a disk image"
    let image_size: Nat = 4611686018427387904
    $count <- 0 then $count + image_size on !add
    let $doubled = $count * 2
    @Label $"bytes: $doubled"
```

`$count` is a `Nat`, because `image_size` is annotated as one and inference takes the literal `0` and the `2` along with it. The increment is large so the scenario fits in three clicks.

**Occurrence 0 (build).** `$count` is `0`, `$doubled` is `0`. The list is a button and a label.

**Occurrence 1.** `Click`. `$count + ... on !add` fires with `4611686018427387904`; `$count`'s update is buffered; `$doubled` recomputes promptly to `9223372036854775808`, which fits; the label rebuilds. Commit. `bytes: 9223372036854775808`.

**Occurrence 2.** `Click`. `$count` would become `9223372036854775808`, which fits in a `Nat`; `$doubled` recomputes: `9223372036854775808 * 2` does not. The multiplication is the lifted operation at `totals.prism:4`, and it detects the overflow and aborts. The buffer is dropped: `$count` stays at its occurrence-1 value, `$doubled` stays, the label stays. The run call returns the overflow message, naming the operation, both operands, the binding whose change led there, and the owner, which is the widget that owns the `* 2` closure's binding: `@totals`'s widget if it has a segment, or its nearest ancestor that does. The click is lost.

**Occurrence 3.** `Click` again. The same, every time. The program is stuck one click before the overflow, which is the honest outcome: the alternative, wrapping, would have shown `bytes: 0` and been believed.

Quarantine is possible here but pointless: the owner is the whole counter, and replacing it with an error box to stop a click from being lost is no better than losing the click. A World's quarantine policy should be about keeping the rest of a program running, not about every error.

The fix is in the type or in the spelling. If the total can legitimately exceed a `Nat`, it is not a `Nat`; `Float` or a bignum from the prelude says so. If the program wants to show the user that the sum did not fit, `Checked.Mul $count, 2` is a `$?Nat` by lifting, and the label's text comes from a `match` with an

arm for `Nil` that says `bytes: too many`. That arm is the program handling its own failure as a value, which is what this chapter says should happen to every failure that can be one.

## A World sending to a vanished path

The relabelling button of the worked examples:

```
@clicker:
    !!clicks <- rebuild $label:
        @Element Ui.Button(label=_)
    let !click = !Switch !!clicks
    $count <- 0 then $count + 1 on !click
    let $label = $"Clicked $count times"
    @Label $label
```

After occurrence 1 the button is at `0/#1/0` and the list the World received says so; `0/#0/0` is gone. Suppose the World has a click for `0/#0/0` queued from before it read the list after occurrence 1, and, in violation of its obligations, sends it.

**Occurrence 2.** The run call's responses are `{0/#0/0: Click}`. Routing is against the list as occurrence 2 opened, which is the list after occurrence 1. `0/#0/0` is not in it. The runtime does not deliver the click to nothing, and does not deliver it to `0/#1/0` on the grounds that it is "the same button", because it is not: the user clicked a button labelled `Clicked 0 times` and that button is gone. It aborts, before any propagation, and returns `UnknownPath` with the path in `path`, no owner, and the sender in the message, which is the World.

Why abort the whole occurrence rather than drop the one stray input and deliver the rest? If the click had come on one line with a legitimate response to `0/#1/0`, the legitimate one is lost too. The argument for dropping only the stray is that the program is being punished for the World's bug. The argument for aborting is that the run call's contract is all-or-nothing and the obligation was cheap to meet: the world core checks every path against the list it last received before it calls the runtime, so a World built on the core never trips this error, and a World that bypassed the core and got the list wrong may have got the rest of the occurrence wrong too. The runtime check exists for Worlds that bypass the core and for interceptors, whose `!down` maps are Prism and can have stale keys through an ordinary bug, which is the next form.

**The same error from an interceptor.** `@Rewrite` of the specification holds its child's `$slots` and routes `!down` by the child's paths. Suppose a version of it holds responses in a hold across occurrences, so that a response for a child slot that was torn down in occurrence 4 is still in the map it fires in occurrence 5. `intercept` routes the map against the child's list as occurrence 5 opened; the path is not there; occurrence 5 is aborted with `UnknownPath`, the sender being `@Rewrite` by its `intercept` span, which is the one entry in `spans`. This is an interceptor bug and the abort is the runtime finding it; a World's own stale-response discard does not apply inside the program, where the interceptor is the World from the child's point of view and has the same obligation.

## A state tag with no supplier

```
tag Game.Score of Element:
    player: Text
    state: Nat

@hud: fn $Text -> @unit = \|$player| @widget:
    $score <- @Element Game.Score(player=$player)
```

```
    @Label $"score: $score"
```

`Game.Score` is a state tag the program's author declared. A game World has a component for it. The CLI World does not. The element has a dynamic field, so it lifts into a `rebuild`: a fresh `Game.Score` slot per change of `$player`.

**Occurrence 0 (build), under the CLI World.** `@hud` is built under `@main`; the `rebuild` builds generation `#0`; `@Element Game.Score(player="ada")` creates a state slot and asks for its value, in this occurrence, because a dynamic without a value is not a dynamic. The request goes outward: no interceptor between here and the root supplies it, so it reaches the World's oracle, and the CLI World has no component for `Game.Score`. The oracle cannot fail, so this is not the oracle failing; it is a tag the World does not accept, which is a build error in the occurrence the slot is created. The build that was in progress is `@main`'s, so occurrence zero returns `UnsuppliedState` with the element in `element`, the chain (empty here) in the message, and the World's name, and the CLI exits with the message above. The program could not have run: there is no later occurrence in which a value could appear.

**Occurrence 3, under a sandbox that forwards.** Put `@hud` under `@Sandbox` under the CLI World, and have `$player` change in occurrence 3. The `rebuild` builds anew mid-occurrence: generation `#1` creates `Game.Score(player="bob")`; the sandbox's supplier is applied to it and returns `@Element Game.Score(player="bob")`, the forwarding case, which creates a slot in the sandbox's widget, which asks the World, which has no component. The build in progress is generation `#1`'s. It is abandoned: the half-built generation is dropped, the sandbox's supplied widget with it, the `rebuild`'s slot holds an `Error` element at `0/#1/0`, the `rebuild`'s event does not fire, and `$score`, which is a hold of dynamics joined with `$Join`, keeps generation `#0`'s dynamic and its last value. Occurrence 3 continues and commits; `Outcome.errors` has one entry, kind `Build`, with the unsupplied tag in its message and owner `0/#1`. The label still says ada's score, which is stale, and the `Error` element is in the list beside it, and the `errors` stream says why in the next turn. The program is wrong in a visible way, and nothing else in it was touched.

**Under a sandbox that supplies.** A sandbox whose supplier has an arm `Game.Score(..): @ $ 0` never lets the tag reach the World, and no error occurs anywhere; the hud shows zero. That is the rule that unavailability lives in the type at work: a supplier's type says it returns a blueprint for every element, so an interceptor cannot fail to supply, and the only place a state tag can go unsupplied is the World, whose set of components is finite and known.

**Under the Test World.** The Test World accepts every tag, so the question is what value it gives. A `with Game.Score(player="ada"), 7` step before the build sets the snapshot and the oracle answers `7`. With no `with` step, the test fails at the slot's creation with the unsupplied-state error, which is the test being told it forgot to set up a state it used, before any step runs.

## Deferring backwards

The sandbox of the worked examples, with a child that does the thing the error exists to catch:

```
@impatient:
    !started <- @started
    !r <- @Element Fs.File(path="/etc/shadow", requests=(Read on !started | Read on !denied))
    let !denied = !Filter (IsDenied !r)
    @Label $"trying"
```

On denial, the child asks again, in the occurrence of the denial. Under a real filesystem that denies at once, the World's response would arrive a turn later each time and the program would be a slow

loop the user could stop. Under the sandbox, the denial comes through `Defer`, and the loop closes inside one turn.

**Occurrence 0 (build).** The child's `!started` fires; `Read` fires on the `Fs.File` slot; the sandbox's `for` builds the `Nil` arm for it, because `/etc/shadow` is outside the root; `unit on _.!request` fires, so the sandbox's `Defer` slot's request fires. The run call returns; the world core sees a fired `Defer` and runs another occurrence of the same turn.

**Occurrence 1, same turn.** `unit` is delivered to the `Defer` slot. `!later` fires; `Denied on !later` fires; the `for` consolidates it; `!down` fires with `{p: Denied}`; the child's slot receives `Denied`; `!r` fires; `!denied` fires; `Read on !denied` fires, so the child's request fires again, in this occurrence; the sandbox's block for that slot is still there, so `unit on _.!request` fires again, and the `Defer` slot's request fires again. Propagation completes and the occurrence commits, because the runtime does not know what `Defer` is and nothing in the occurrence was wrong by the runtime's rules. The run call returns with the `Defer` slot's request firings listed.

The world core now sees that a `Defer` slot it answered in this turn has fired again. That is deferring backwards: answering it again would be a second answer in one turn, which the bound on a turn forbids. The core ends the turn, reports `DeferBackwards` with the slot's path in `path`, the chain in the message, and the two occurrences, and discards the second request. It does not answer it in the next turn, because that would turn the loop into one denial per turn forever, with an error per turn, which is a hang with extra steps. The child is left waiting for a response that will not come, which its type permits; the label says `trying`, which is true.

Occurrence 1 was committed, not aborted, and that is right: the occurrence was consistent, and the error is a protocol error between the interceptor and the World that the runtime could not see. The World's policy applies at the turn: the CLI exits nonzero, the GUI shows the message and continues, the Test World records it for an `errors Error(kind=DeferBackwards, ..)` step, which is how the suite pins this rule.

The message says what to do instead, and it is the general rule for retries: a retry in the occurrence of a denial is a loop, and a retry belongs behind `Time.After`, where the World's clock separates the attempts and the user has a chance to stop them. The sandbox is not at fault and needs no change; it answered once, as it must, and the child asked again at once, which no interceptor can answer without starting an occurrence of its own, which nothing in Prism can.

## A `for` key whose block fails to build

The server of the worked examples, with a handler chosen per connection, and one handler that has the cycle of the first scenario in it:

```
@Join: fn !Bytes, !Bytes -> @!Bytes = \|!x, !y| @widget:
    @ !x | !y

@Rev: fn !Bytes -> @!Bytes = \|!in| @widget:
    @ Reverse !in

@Echo: fn !Bytes -> @!Bytes = \|!in| @widget:
    @ !in

@Mirror: fn !Bytes -> @!Bytes = \|!in| @widget:
    !a <- @Join !in, !b
    !b <- @Rev !a
    @ !b
```

```
@server:
    !accepted <- @Element Net.Listener(address=":7000")
    $n <- 0 then $n + 1 on !accepted
    !closes <- for $conns:
        !ev <- @Element Net.Conn(id=key, requests=!reply)
        !out <- match _:
            Echo: @Echo (!Filter (Received !ev))
            Mirror: @Mirror (!Filter (Received !ev))
        let !reply = [[Send !out]]
        @ Shut on !Filter (IsClosed !ev)
    $conns <- {} then ApplyDeltas $conns, (!Union (Open !accepted, ModeFor $n), !closes)
    @ unit
```

`ModeFor` is a plain function that picks a handler by connection number, `Echo` for most and `Mirror` for every seventh, which is the kind of thing a load experiment does. `@Mirror` has a prompt cycle through two widget functions, `!a` from `@Join !in, !b` and `!b` from `@Rev !a`, that the block check cannot see, because it does not know that `@Join`'s `|` is prompt in its second argument. `$conns` is a map from connection id to its mode. The listener is at `0`, the `for` at `1`, and a key's `Net.Conn` slot is at `1/[id]/0`.

**Occurrence 0 (build).** One slot, the listener; `$conns` is `{}`; the `for` builds nothing.

**Occurrences 1 to 6.** Six `Accepted` responses, one per turn. Each adds a key to `$conns` with mode `Echo`; the `for` builds the key's block mid-occurrence; `@Echo` has no cycle; the block creates its `Net.Conn` slot at `1/[id]/0`. The list after occurrence 6 has the listener and six connections. Each client's bytes come back reversed by nobody and unchanged by everybody, which is what an echo server is.

**Occurrence 7.** `Accepted 7`. `$n` becomes 7; `ModeFor 7` is `Mirror`; `$conns` gains `7: Mirror`; the `for` builds the block for `[7]` at the height where `$conns` changed. Inside the block, `@Element Net.Conn(id=7, ..)` creates the slot at `1/[7]/0`, and `@Mirror` builds its recursive group with placeholders; closing `!a`'s placeholder creates the edge from `@Join`'s `|` to the forward of `!a`, and the eager walk runs: forward of `!a` → `|` → forward of `!b` → `Reverse` → forward of `!a`, which is being computed. Found, inside the build of the key's block. The build is abandoned: the half-built widget, its `Net.Conn` slot and `@Mirror`'s widgets are dropped, and an error widget takes the block's place at `1/[7]`, with `1/[7]/0: Error(kind=Build, ..)` as its one slot. The consolidated `!closes` has no entry for `7`, and the other six keys were not touched: no hold in them changed, no subscription moved, and their slots are where they were. The occurrence continues at the next height and commits. The run call returns the list, with the six connections and the error element, and `Outcome.errors` with one entry: kind `Build`, message naming the cycle, `spans` the loop through `mirror.prism` and `join.prism`, owner `1/[7]`.

**What the World does.** The CLI World, which includes `Net`, prints the build-form message to stderr and continues; the six clients are being served and nothing justifies dropping them. Connection 7 has no `Net.Conn` slot and never will, since the error widget is what the key built; under the World's own rule for a connection the program never claimed, the World closes it after its grace period. Nothing in the program asked for that close; the World is reconciling an accepted socket to a list that does not mention it. Key `7` stays in `$conns` with its error widget until the program removes it, which it never will, since `!closes` never fires for it; a server that wants the key gone subscribes to `errors`, reads the owner, and removes the key whose path it is, which is three lines and an `Under`. A server World that quarantines has nothing to do here: the failure was contained at the build and the error widget is already in place.

**Occurrence 8.** `Accepted 8`. `ModeFor 8` is `Echo`; the key builds and serves. The seventh connection's failure left no trace in the eighth's build, which is the property the scenario exists to pin: a `for`

block's build failure is that key's and no other's.

The fix is in `@Mirror`, as in the first scenario: the feedback needs a delayed edge, `$prev <- Empty then !b` and `@Join !in, ($prev on !in)`, which mirrors the previous reply into the next one and is what the handler presumably meant.

## The invariants, collected

What the runtime promises:

- An occurrence that cannot complete is abandoned, and afterwards every hold, switch, node, slot and path is as it was when the occurrence opened. Nothing is half done, ever.
- A build that cannot complete, inside `rebuild` or `for`, is abandoned without aborting the occurrence; the slot at the block's path with segment `0` holds an `Error` element; the failure is in `Outcome.errors` with kind `Build`.
- A prompt cycle is found when the edge that closes it is created, during a build or as the first step of the commit that adopts it, never by looping, deadlocking, or firing a node twice. The scheduling-height check remains as an assertion against the runtime's own bugs.
- Overflow in the default arithmetic and recursion past the depth guard abort the occurrence as `Overflow`; a failure in native code aborts the occurrence as `Component`; the error names Prism spans.
- A response to a path not in the opened list aborts as `UnknownPath`; a response of the wrong type and two responses to one slot abort as `Component` with the sender named; a state tag that reaches the root with no component abandons the build or, at the root, aborts as `UnsuppliedState`.
- A quarantine instruction names only an owner the last error named, or the run call returns a `Component` error and runs nothing; a performed quarantine puts the `Error` element at the owner's path with segment `0` and reports every input addressed under the owner, with its path, in the outcome.
- The error out of the run call, and every entry of `Outcome.errors`, is a value with the occurrence, the kind, the spans, and the owner where there is one, and nothing that names a runtime object.

What the World promises:

- To treat the inputs of an aborted occurrence as undelivered and keep its snapshot bookkeeping consistent with that.
- To pair every error with its turn, and, if it lists `errors` in `World.Tags`, to deliver each error of a turn to every `errors` slot in the list as the next turn's first occurrence opened, as a response, one firing per error.
- To send `quarantine` only for an owner the last error named, and to discard the inputs the outcome reports as delivered to nothing under it.
- To stop a turn at an abort, perform the requests of the occurrences that committed, leave unanswered `Defer` slots waiting, and report.
- To report deferring backwards, discard the stray request, and end the turn.
- To contain its own components' failures to a withheld response or an unchanged snapshot, reported as `Component`, never a crash and never `Unsupported`, if it is built on the core.

What an interceptor promises: nothing beyond what every program promises, because an interceptor is Prism. Its stale `!down` key is a program bug found by the runtime; its supplier cannot fail by type; its `Defer` misuse is found by the World.

What the program promises: nothing. From inside, nothing happened.

The negative space, as a list, since it is what makes the rest cheap. There is no `catch`, no `try`, no error type the runtime hands to Prism code except through an element, no retry the runtime performs on its own, no partial commit, no whole-turn rollback, no error that a World sees as an exception, and no error whose report names a runtime node instead of a Prism span. Each absence is a mechanism the implementation does not have to build: no unwinding through closures, no handler lookup, no multi-occurrence undo log, no scheduler for retries, and one error channel, the run call's return value and its outcome's error list, for everything. The transaction was already there, and this chapter is the list of what it turns out to cover.

## Causality in types

A cycle in the prompt graph inside one widget block is a compile error with every binding in it listed, and that check has been enough for every program in the suite. Then you write a layout library. `@Pack` takes a list of children's size dynamics and returns a dynamic of positions; a child takes its position and returns its size; both are widget functions, each is acyclic inside its own block, and the first time a program wires a child's size from its own position through both of them, the cycle exists only at the call site, where neither function's body is visible. The compiler says nothing. The runtime finds it the first time the occurrence runs, a node scheduling below the current height, and aborts the occurrence with the spans of the two bindings at the call site and no explanation of why they form a cycle. This item is a type-level record of which arguments a function's result depends on promptly, so that the check inside a block can cross a function boundary and the runtime's detection becomes a backstop. It is an effect system in a small way, and it is research.

### The check as it stands

Inside a widget block, every binding is a node and every use is an edge, labelled prompt or delayed by the construct that makes it: lifting over an event or a dynamic, `|`, `for` consolidation and `rebuild`'s rebuild are prompt; a tag, `!Switch`'s choice of inner event and a hold's value side are delayed. The resolver finds the strongly connected groups; the elaborator rejects any group whose cycle has no delayed edge. A widget function's body is one such block, so its own cycles are caught. What the check cannot see is an edge from an argument to the result: `@F !a` returns `!b`, and whether `!b` fires promptly on `!a` is a fact about `@F`'s body that its type `fn !A -> @!B` does not state. At the call site the elaborator must assume either that every result depends promptly on every argument, which rejects the layout library and most of the GUI corpus, or that none does, which is what it assumes now and why the cycle above is found at runtime.

### The annotation

A function type gains, for each sigiled result, the set of arguments it depends on promptly. Written out, in a form a signature could use:

```
@Button: fn $Text -> @!Click                      # result depends promptly on nothing
Scale:   fn !Int, $Int -> !Int    prompt !_1      # result fires promptly on the first argument, reads the
second through a tag
@Pack:   fn [$Extent] -> @$[Pos]  prompt $_1        # positions change promptly when any size changes
```

The default, with no annotation, is that the compiler infers the set from the body: a function's result depends promptly on an argument if there is a path of prompt edges from the argument to the result inside the body. Inference is a reachability pass over the same labelled graph the cycle check already builds, so it costs nothing new. Written annotations are for signatures the author wants to pin, the

way a type annotation pins an inferred type, and the body is checked against them.

At a call site the elaborator then adds, for each argument the result depends on promptly, a prompt edge from that argument to the binding of the result, and the group check runs as before. The layout cycle becomes:

```
$positions <- @Pack [[$size]]        # prompt on $size, by @Pack's type
$size <- @Child (First $positions)   # prompt on $positions, by @Child's type
```

`First: fn [Pos] -> Pos` is a plain library function, lifted over `$positions`, and the lifted application is prompt like any lift. Two prompt edges and no delayed one, rejected with both bindings' spans and the two signatures that make the edges prompt. The fix is the one the author would have reached for after the runtime abort, a tag or a hold between the two, and now the message says so before the program runs.

The same cycle through an element is the case the annotation has to be exact about, because it is the case the GUI library actually has. The library's `@HBox` is `fn @a -> @a`: an interceptor over its child, with the child's `Ui.*` elements in a `Ui.HBox(children=...)` element it puts out. A layout interceptor above it supplies positions as state to the children, so a child that wants its position asks for it through a tag:

```
tag Ui.Placed of Element:
    size: Extent
    state: Pos
```

A child writes `$pos <- @Element Ui.Placed(size=$size)`, and a dynamic in a field lifts into a `rebuild`: a fresh slot per size change, with the slot's state read from the supplier the layout interceptor built. If the child then computes `$size` from `$pos`, the loop runs through the element list: `$size` changes, the `rebuild` builds anew the slot promptly, the interceptor's `$slots` changes promptly, the supplier's state dynamic for the new slot is built from `$slots` and read into `$pos`, and `$pos` feeds `$size`. Every edge in that loop is prompt, including the one that looks as if it might not be, inside `intercept`: the state a slot sees is the supplier widget's result, and the supplier is built in the interceptor's widget at slot creation, as a widget built mid-occurrence that sees the occurrence's inputs, so it is prompt on `$slots`. So the loop is a real cycle and the runtime finds it today. The annotation that catches it at compile time is on `@Element`: for a state tag, the returned `$State` is prompt on the slot's element value, and the element value is prompt on every dynamic that went into a field, because each one is a `rebuild`. With that one rule the elaborator sees `$pos` prompt on `$size` across the `@Element` boundary, and the message names the field. The two-function form above and the element form are the same cycle; the element form is the one where the type has to say something at a boundary the program did not write.

## What makes it research

Three places where the small version stops being small.

*Higher-order functions.* `Map: fn (fn a -> b), !a -> !b` is prompt on its event argument, and whether it is prompt on anything else depends on the function passed in, which has no sigiled arguments and so no dependency set of its own. The clean answer is that a plain function's dependency is empty, since a plain value has no time in it, and the only prompt dependencies come from sigiled arguments. That holds for every prelude function. It breaks when a function argument is a widget function, `fn (fn $A -> @!B), $A -> @!B`, whose dependency set is a variable the caller fills in. That is dependency polymorphism, and it is the point at which the annotation becomes an

effect system with variables and inference becomes unification over sets. It is tractable; it is also where the design has to stop being a reachability pass and become a type system, and the payoff should be measured against how often widget functions take widget functions in real libraries.

*Switching.* A `rebuild` over `$x` returns an event that fires promptly on `$x`'s change; the block's result is built fresh each time, and what the result depends on inside the block is the block's business and acyclic by the block check. So `rebuild` contributes exactly one prompt edge, from its dynamic to its event, and the block's dependency on captured outer dynamics is prompt through them as well. `!Switch` is delayed by definition and contributes nothing. `for` is prompt on its map and its consolidated result is prompt on whatever the block's result is prompt on, through the per-key `$_`. None of these needs the annotation to say anything new, but the inference pass has to understand them, and the construction-in-the-middle rule means a cycle formed by a rebuild is a cycle the new block's result makes at the `rebuild`'s call site, which the annotation on the block's captured names covers.

*Interceptors.* The recursive binding every interceptor has, `!down` defined from `$slots` which `intercept` produces from `!down`, is legal because routing reads the list as the occurrence opened, which is a tag. `intercept`'s type has to say so: its result list is prompt on the child and delayed on the responses argument. That is one annotation on one primitive and it is exact.

## What it must not change

No program that was legal and never hit a runtime cycle becomes illegal, which is soundness in the direction that matters, and it holds wherever the inference can see a body: there the annotation only adds prompt edges that exist, so a rejected program is one that would have aborted on the first occurrence to exercise the cycle. Where the inference cannot see a body it over-approximates, and the two places are named so a reader knows where a rejection may be of a program that would never have aborted. A blueprint parameter, `@child` in `fn @a -> @a`, has no body at the call site, so its result is treated as prompt on every sigiled argument the caller passed it. A `for` over a blueprint parameter whose body is not visible is treated as able to fire at birth, which is the rule the static cycle check has already and this pass inherits: that adds a prompt edge from the collection's change to the consolidated result that may not exist. In both cases the written annotation on the parameter is what the author writes to get the program accepted: `fn (@a prompt nothing) -> @a` says the child's result depends promptly on nothing, the elaborator checks the argument's inferred set against it at the call site, and the body of the function is checked under that assumption. That is the same bargain a type annotation on a higher-order parameter makes, and it keeps the default conservative without making it final. The runtime's detection stays, because a cycle formed through a path the type system does not track, a function value stored in a record and applied later, for instance, still has to be found and still has to abort the occurrence rather than loop. The typed AST gains a field on function types and nothing else changes in it; the Haskell implementation can ignore the field, because Reflex finds the cycle at runtime the same way the Rust runtime does, and the differential test is unaffected because an accepted program means the same under both.

## The fact it waits on

Runtime cycle errors in practice: a library whose users hit the abort at call sites and have to read two function bodies to understand it. The GUI library's layout widgets are where it will happen first if it happens. Until then the block check plus the runtime backstop is the design, and the annotation is a note about where the type system would go next.

## Chapter 10

# Conformance and testing

A specification that only one implementation can check is a description of that implementation, and the way out is to test the language from the only place where two implementations must look identical: the World's side of the boundary. Every test in this chapter sends plain data to elements and reads plain data back, the way a real World does, and never names a widget, a path, a hold or a runtime object. That choice decides everything else here: the Test World that accepts every tag and has no effects; the script language that drives a program one turn at a time; the law tests, one short script per sentence of the chapters on the functors, on time, on widgets and switching, and on elements and Worlds; the integration programs that each pin one piece of meaning; the layout corpus and the typed-AST goldens that pin the frontends; the runtime's own unit tests and the invariant they check after every occurrence; the World tests against the world core; and the differential test that runs all of it under both implementations and diffs the output. The chapter ends with the one rule every test obeys: it is quotable, and if it cannot be stated as a sentence about the language, it is deleted.

## Tests against what a World sees

You have written the test that reaches into a component and reads its private field. It passed. Then someone renamed the field, or moved the state up a level, or replaced the component with one that computes the same thing differently, and the test broke without the program being wrong. Later someone else made the test pass again by exporting the field, and now the field is part of the interface because a test wanted it to be. That is the ordinary life of a white-box test, and a conformance suite cannot live that way, because it has two implementations to be true for at once, and their insides have nothing in common: one is a graph of `Rc` nodes with heights and the other is a Reflex `Spider` timeline. The only thing they share is what comes out of the run call.

So the suite is written against the run call. A test is a program and a script; the script delivers responses and pushes, which is what a World does, and checks the element list, the request firings, the result and the errors, which is what a World sees. The spec promises that these are fully determined by the inputs and the snapshots, which is the determinism law of the chapter on time, and that promise is what makes a script a test: the same script gives the same transcript under every conforming implementation, and a transcript that differs is a bug or an ambiguity, never a matter of taste.

This has a consequence the language already chose for other reasons. There are no reified identifiers, no `Inspect $count`, no way to poke a hold. Suppose there were: a step `Peek $count, 2` that read a named dynamic inside the blueprint under test. Convenient for the counter. Then the counter is rewritten with the count held inside a `rebuild`, and `$count` is now a name in a block the step cannot reach; or the count becomes a field of a record held under another name; or the implementation under test is the Haskell one, in which a name is a `Dynamic t Int` in an evaluator environment, and the step needs a second evaluator feature to find it. Each of those is work, and none of it tests the language. Worse, every test that peeks is a test that passes under the Test World and says nothing about any

other World, because no other World can peek. A test that clicks the button and reads the label passes under the Test World and under the GUI World and under a shell written in Prism, because every one of them clicks and reads the same way. The suite tests the language, not a program's private names.

The exclusions are as deliberate as the inclusions. The suite does not test performance or memory; a slow implementation that produces the right transcripts conforms. It does not test any World other than the Test World; a CLI World's obligations are tested per World, in the host language, in a later section. It does not test the order in which an implementation fires nodes within an occurrence, because the language only promises the observable consequences of that order: each event at most once, each dynamic changed at most once, no node seeing a prompt source before it settles. And it does not test diagnostics' wording except in the negative goldens, which are a frontend concern and say so.

## The Test World

The Test World is a World with no effects, written once per host language on top of the world core, with one component that answers to a script instead of to an operating system. Three properties make it the suite's World.

**It accepts every tag at the root.** A program that creates a `Net.Listener` is tested without a network; one that creates a `Ui.Button` is tested without a screen; one that creates a tag nobody has heard of is tested all the same. The Test World does not know what any tag means. It knows that a `requests` field fires values it records, that a `responds` field is an event it can deliver to by path, and that a `state` field is a dynamic it answers from a snapshot the script sets. That is everything a World does with an element, and it is why the Test World can be written before any real World exists: it is the obligations of a World and nothing else. Suppose instead the Test World had components, a fake filesystem with files the script writes, a fake clock that ticks. The tests would then be about those fakes: a fake filesystem that answers reads has an opinion about what a read is, and a program that expects `Unavailable(NotFound)` for an absent file is testing the fake's opinion. With the script as the only source of inputs, the program's expectation is the script's expectation, written next to the program, and a reader can see both.

**Its snapshot is whatever the script says.** A state element's value in its creation occurrence comes from the oracle, and the oracle answers from the previous snapshot; the script's `with` steps are that snapshot. There is no clock in the Test World except the one the script pushes, which is the goal that there is no clock, applied to testing: a debounce, a timeout, a stopwatch are tested by setting an `Instant` and pushing another, and a transcript replays them exactly.

**It holds no runtime object.** The Test World reads the run call's outcome and nothing else. It does not read behaviors, does not hold event handles, does not receive callbacks. The whole of what it knows about the program after a turn is the element list, the request firings per slot, the result value and any error, which is the same four things a GUI World knows. This is the property the differential test depends on, because the Haskell Test World cannot hold a Rust `Rc`, and it is the property that keeps the suite honest: anything the Test World could learn by holding a runtime object would be something no real World could learn.

A test builds a widget under this World and runs steps in order; the first failing step fails the test, and the failure message prints the script up to that step and the transcript of the turn it failed in.

## The transcript

Everything the Test World sees is printed, turn by turn, in one canonical text form that both implementations produce. The conformance harness compares transcripts; the differential test diffs them; a failing step prints the one it failed in. Here is the counter's first two turns:

```
turn 0
  list:
    0/#0/0: Ui.Button(label="Increment")
    1/#0/0: Ui.Button(label="Decrement")
    2/#0/0: Ui.Label(text="Count: 0")
  result: unit
turn 1  responses: {0/#0/0: click}
  list:
    0/#0/0: Ui.Button(label="Increment")
    1/#0/0: Ui.Button(label="Decrement")
    2/#1/0: Ui.Label(text="Count: 1")
  result: unit
```

A turn header names the turn and its inputs: `responses` by path, `pushes` by element value. The list is one slot per line, `path: element`, in tree order. A slot that fired its request in the turn has `requested:` after its element, then the fired values bare, in firing order, separated by `;`: `requested: Read`, `requested: "a"; "b"`, `requested: [[Send "ok", Close]]`. An array request is already an array literal, so the transcript adds no brackets of its own and every bracket in the line means an array; an occurrence boundary inside a turn, which only `Defer` causes, is marked `occurrence 2` between the parts of a turn so that a reader can see which occurrence a request fired in. The result is printed when the widget's result is a plain value or a dynamic (by its current value) and as `fired: v` or `quiet` when it is an event. A slot that fired its request in the turn and was torn down before the turn's end prints after the list, under `torn down:`, as `path: element requested: ...`, because the World still has to perform what it fired; a program that sends on a socket and changes its address in one occurrence is the ordinary case, and a transcript that dropped the send would be hiding an effect. An aborted occurrence prints `error: Error(kind=UnknownPath, path=0/#1/0, occurrence=1)` and no list, since the list did not change.

Paths are opaque to Worlds, and the transcript still has to print them, so the transcript fixes a spelling that both implementations use: bind positions from the root joined by `/`, a `rebuild` generation as `#n` from `#0`, a `for` key as `[k]` with the key printed as a value. That spelling is the chapter-2 convention and it is part of the interchange format. An implementation may represent paths however it likes inside, and the Haskell runtime's paths are nothing like the Rust runtime's, but what they print must match character for character or the differential test fails. This is a real constraint, deliberately taken: two implementations that agree on every element but number bind positions differently would be two readings of the sentence "its position in the widget tree, made of the bind positions, `for` keys and `rebuild` generations", and the point of the differential test is to find exactly that kind of difference. Slots created by widgets that an interceptor's supplier built print as `s/n`, numbered in creation order within the interceptor's widget, because their position is the runtime's business and only their existence is a law.

Element values print as their constructor with named fields in declaration order; `Path ==> Element` fields print with their keys as paths, which is how a `Ui.VBox`'s children appear. Plain values print as Prism literals. Nothing in the transcript is implementation-specific, which is the test of whether it belongs there.

## The script language

A test is a top-level statement:

```
test counter, @counter:
    build
        => expect Ui.Label(text="Count: 0")
    click Ui.Button(label="Increment")
        => expect Ui.Label(text="Count: 1")
```

`test name, blueprint:` opens the form. The blueprint is any blueprint expression; most tests name a top-level one, and an interceptor test wraps one: `test jailed, (@Sandbox "/jail", @probe):`. Inside, every line is one turn, and every turn has two sides: what the script delivers, on the line, and what it checks afterwards, on indented `=>` lines under it. So the number of lines at the block's indentation is the number of turns, which is the thing a reader of a script most needs to count. The first line is `build`, turn zero. An input line is one or more deliveries separated by commas, all in one occurrence: `click p` delivers a `click` to the one slot matching `p`, `send p, v` delivers any other response, and either may address a slot by its path instead of a pattern, `click at 1/[1]/2/#0/0`, for the case two slots match one pattern, a list of rows with identical buttons being the usual one, since a pattern that matches two slots is a script error and a script should not have to star a row before it can add a second; `push p = v` sets a state entry and runs the turn on that push alone, and `with p = v` before an input sets a state entry for that turn without being an input. A turn with no inputs at all is `idle`. Each `=>` line is one check against the state after the turn; a turn the script only needs to have happened says `=> settles`, which asserts that it ran, every `Defer` it caused was answered, and no error ended it. No side is ever empty: a line with nothing to deliver says `build` or `idle`, and a turn with nothing to check says `=> settles`.

`test` is a form of its own and not a function call, for two reasons. The patterns in it, `Ui.Button(label="OK", ..)` and `ignore`, are not values, so the frontend checks them against the tag declarations in scope the way it checks a `match`, and a `send` of a `Tick` to a button is a compile error in the test file rather than a runtime surprise. And the build system treats the two differently: `prism test` collects every `test` form and `prism build` strips them, which is the same reason Rust marks tests with an attribute rather than a naming convention.

### Patterns

A pattern is what `match` accepts for an element, written where the step wants one: a tag with named fields and `..` for the rest, `Ui.Button(label="OK", ..)`; a tag with every field, `Ui.Label(text="Count: 0")`; a bare tag for a tag with no fields, `Clock`; `Ns.*(..)` for any tag in a namespace; `ignore` for anything. Field patterns are value patterns: a literal, a nested element pattern, `..` inside a record, `ignore`. Because element fields are plain data fixed for the slot's life, there is no "current value" of a field to compare; a label whose text changed is a different slot with a different element, and a pattern that matched the old one does not match the new one. That is why `expect Ui.Label(text="Count: 1")` after a click is a complete check: the old label is gone from the list and the new one is in it.

Response and state values in a step are plain Prism expressions of the tag's declared type: `click`, `Contents "hi"`, `Instant 5100`, `{3: Toggle}`. The frontend checks them against the declaration of the tag the pattern names, so a `send` of a `Tick` to a button is a compile error in the test file rather than a runtime surprise. When the pattern is `ignore` or `Ns.*(..)`, the value is checked at the slot when the step runs.

## The steps

Here is every input and every check, with its meaning and a script that uses it. The counter, the viewer and the stopwatch are chapter 2's; `@probe` and the rest are defined where they appear.

**`expect pattern`.** Some slot in the current list, at any depth, matches. Depth means inside container elements by the GUI library's convention, below.

```
test expect_sees_the_built_program, @counter:
    build
        => expect Ui.Label(text="Count: 0")
        => expect Ui.Button(label="Increment")
```

**`expect_top pattern`.** Some root-level slot matches: one the Test World itself received, not one found inside a container's `children` field. The difference is what a container test is about:

```
test vbox_holds_its_children, (@VBox @counter):
    build
        => expect_top Ui.VBox(..)
        => expect_top_none Ui.Button(..)
        => expect Ui.Button(label="Increment")
```

**`expect_none pattern`** and **`expect_top_none pattern`.** No slot at any depth, or no root-level slot, matches. `expect_none` is how a test says that something is gone, which is the half of a rebuild that a positive check cannot see:

```
test rebuild_removes_the_old_label, @counter:
    build
        => settles
    click Ui.Button(label="Increment")
        => expect Ui.Label(text="Count: 1")
        => expect_none Ui.Label(text="Count: 0")
```

**`expect_count pattern, n`.** Exactly `n` slots at any depth match. `expect_count p, 0` is `expect_none p`; `expect_count p, 1` is the step to use when the test needs there to be one of something, which `expect` does not say.

```
test one_row_per_key, @tallies:
    build
        => expect_count Ui.Label(text="a: 0"), 1
        => expect_count Ui.Label(text="b: 0"), 1
        => expect_count Ui.Button(..), 4
```

**`send pattern, response`.** Exactly one slot must match the pattern; otherwise the step fails before anything is delivered, with the count it found. The response is delivered to that slot's path as the sole input of a new occurrence, and a turn is run: that occurrence plus any `Defer` occurrences it causes. The requirement that exactly one slot match is what keeps a test from sending a click into an ambiguity: two buttons labelled "OK" need a pattern that tells them apart, or one input line that delivers both.

```
test send_runs_a_turn, @counter:
    build
        => settles
    click Ui.Button(label="Increment")
        => expect Ui.Label(text="Count: 1")
```

**Several deliveries on one line.** `click p, click q` delivers both in one occurrence, which the program sees as simultaneous. Two deliveries that resolve to the same slot fail the line, because a

slot receives at most one response per occurrence; a World that has two things for one slot delivers them in two occurrences, and a script says that by writing two lines. The comma is the only way a script makes two inputs simultaneous, and it is how every simultaneity law is tested:

```
test together_is_one_occurrence, @counter:
    build
        => settles
    click Ui.Button(label="Increment"), click Ui.Button(label="Decrement")
        => expect Ui.Label(text="Count: -1")
```

**`with pattern = value.`** Sets the snapshot value for the state element whose *value* matches the pattern. The pattern here is matched against element values, not slots: `with Clock = Instant 5000` sets the snapshot entry for the element `Clock` whether or not any slot has asked for it yet, and `with Fs.Contents(path="/etc/motd") = Known "hi"` sets the entry for that one path. A pattern with `..` or `ignore` in it is a script error, because a snapshot entry is keyed by a whole value. `with` is not an input; it is written on the turn's line before the inputs, and its effect is that turn's: as the occurrence opens, the oracle answers from the snapshot *before* this `with`, and the difference between that and the snapshot after it is the push. A `with` on the `build` line is the snapshot the build occurrence's oracle answers from, which is how a test gives a clock a starting value:

```
test state_before_build_is_the_oracle, @stopwatch:
    with Clock = Instant 5000, build
        => expect Ui.Label(text="0 ms")
```

Two `with`s for one element on one line: the second replaces the first, and the push is the difference between the previous turn's snapshot and the final one, since a World gathers everything that changed into one snapshot per occurrence.

**`push pattern = value.`** A `with` that is the turn's only input. The turn's only input is the push; a program with no slot for that element runs an occurrence in which nothing fires, which is still a turn and still prints.

```
test push_moves_the_clock, @stopwatch:
    with Clock = Instant 5000, build
        => settles
    click Ui.Button(label="Start")
        => settles
    push Clock = Instant 5100
        => expect Ui.Label(text="100 ms")
```

A `with` and a `click` on one line make a turn with both a push and a response, which is the only way a script puts a state change and an input in one occurrence, and it is what the law about a push reaching a slot created in the push's occurrence needs.

**`returns pattern.`** The widget's result matches. A plain result is compared as a value; a dynamic result is compared by its value after the last turn; an event result makes `returns` a script error, since an event has no value between occurrences, and the step to use is `fired`. A widget returning a bundle is matched with a record or tuple pattern, with `ignore` for the parts the test does not care about.

```
test returns_reads_a_dynamic, @accumulator:
    build
        => returns 0
    send Test.Accumulator(..), Add 3
        => returns 3
```

**`fired pattern`** and **`quiet.`** The widget's result is an event; `fired` passes if it fired at some occurrence

of the last turn with a value matching the pattern, and `quiet` passes if it fired at no occurrence of the last turn. Both look at the whole turn, not only its first occurrence, because a World sees turns: a value that came back through a `Defer` is, to the World, a consequence of the input it sent. Both are script errors on a widget whose result is not an event.

```
@ok_button:
    !b <- @Button $"OK"
    @ "ok" on !b

test fired_and_quiet, @ok_button:
    build
        => settles
    click Ui.Button(label="OK")
        => fired "ok"
    push Clock = Instant 0
        => quiet
```

Before any turn, `quiet` holds and `fired` fails: turn 0 is the build, and the result event of a widget cannot fire in turn 0 through a `!Switch`, though a widget whose result is `@started` directly fires in turn 0 and `fired unit` holds then. The law test for `@started` uses exactly that.

`requested pattern, value.` Some slot matching the pattern fired its request during the last turn with a value matching the value pattern. `ignore` as the value matches any firing. A slot whose tag has no `requests` field fails the step with a message saying so, rather than quietly never matching. The counterpart is `unrequested pattern`: no slot matching the pattern fired its request during the last turn. The negative form is in the language because the most important law in the suite, that creating an element requests nothing, cannot be stated without it.

```
@viewer_with_refresh:
    !started <- @started
    !refresh <- @Button $"Refresh"
    !motd <- @Element Fs.File(path="/etc/motd", requests=(Read on !started))
    $motd <- Nil then Some !motd
    @Label (ShowResponse $motd)

test started_requests_in_turn_zero, @viewer_with_refresh:
    build
        => requested Fs.File(path="/etc/motd"), Read
    click Ui.Button(label="Refresh")
        => unrequested Fs.File(..)
```

The button is wired to nothing on purpose. The read is requested on `!started` and on nothing else, so the test is that a click which no request listens to requests nothing, which `unrequested` is the only way to say.

`errors pattern.` The last turn ended in a runtime error matching the pattern. Errors are `Error` values with a `kind`, so the patterns are record patterns on the kind and the fields the test cares about: `Error(kind=UnknownPath, ..)`, `Error(kind=DeferBackwards, ..)`, `Error(kind=Cycle, ..)`, `Error(kind=UnsuppliedState, element=Clock, ..)`, `Error(kind=Overflow, ..)`. When an occurrence aborts, its holds do not commit, its new widgets are dropped, the element list is what it was before the occurrence, and the turn ends there; a check after an `errors` step sees the list as it stood before the failed turn. A turn that ends in an error with no `errors` step following fails the script, so a test cannot pass by accident through a crash. The tags and fields of `Error` are tentative in the way the whole error story is: if the unit of failure becomes a quarantined subtree rather than the occurrence, `errors` will still match the error and a later `expect Error(..)` will see the quarantine

element in the list, and nothing else in the script language changes.

```
tag Secret of Element:
    state: Text

@needs_secret:
    $s <- @Element Secret
    @Label $s

test unsupplied_state_fails_at_creation, @needs_secret:
    build
        => errors Error(kind=UnsuppliedState, element=Secret, ..)
```

That script has no `with Secret` before the build, so the slot's creation in occurrence 0 is the error and turn 0 is the turn that ends in it. The same script with `with Secret = "open"` as its first line builds a label showing `open`.

## Every state element needs a `with` first

A tag declaration says what an element means and nothing else, so a state element has no value until someone supplies one. In the Test World the supplier is the script: a program that creates a `Clock` at build has `with Clock = Instant 0` as the first line of its test, and a program that creates one later, say on a click, has the step before the `send`. Without it the slot's creation is the unsupplied-state error above.

```
test a_clock_needs_a_value_before_the_build, @stopwatch:
    build
        => errors Error(kind=UnsuppliedState, element=Clock, ..)

test and_has_one_after_it, @stopwatch:
    with Clock = Instant 0, build
        => expect Ui.Label(text="0 ms")
```

The line is noise in a test that is not about the clock, and it is also the honest line: a real World answers its oracle from its own knowledge, a World without the component cannot answer at all, and the script says which of those it is pretending to be. Suppose instead each tag could declare a default value. Every test would then pass without a line that a real World has to earn, and a program that works under the Test World would fail on the first World without the component, which is a bug the suite exists to catch before it ships.

## Depth: the GUI library's convention

`expect` finds a button inside a box, and `send` clicks it, by a convention that belongs to the suite's GUI library and not to the language. The library's container tags hold their children's elements as data in a field named `children` of type `Path ==> Element`, keyed by the child's path in the container's own child list, and respond with a `ChildInput`:

```
type ChildInput = record:
    path: Path
    response: Response

tag Ui.VBox of Element:
    children: Path ==> Element
    responds: ChildInput
```

`expect` searches every `children` field recursively, so a pattern matches a slot at any depth. When `send` resolves to a slot inside one or more containers, the Test World wraps the response in one `ChildInput` per level, innermost first, and delivers the result to the outermost container's path, which is the only path it has. The delivery then takes the route a real World's input would: the container's `@VBox` receives a `ChildInput`, routes its response to the child path through its `intercept`, and if that child is itself a container, the same again. A test cannot bypass the library's routing, which is the point; the law that routing is against the opened list applies at every level and a test of a nested button tests all of them.

The library is part of the suite, which is why the Test World can know its convention without the language knowing it. A different library with a different convention, say children in a `rows` field, would ship its own depth rule with it, and a Test World knows the convention of the library it was built with. The language fixes only that leaves are elements and containment is interception; the spelling of a container's children belongs to the library.

The suite's library is small and written out in full later in this chapter, because the integration programs are built on it and because its own typed-AST dump is one of the goldens.

### Turns, and when checks run

The build is turn 0. It is a real turn: occurrence 0 plus any `Defer` occurrences the build causes, and a check before the first `send` sees the end of it. Every input line is one turn: the occurrence it opens plus every `Defer` occurrence that follows until no `Defer` request is outstanding. `with` and the checks run no turn. A check always runs between turns, against committed state, which is the only state a World ever reads.

Those sentences have consequences a reader gets wrong the first time, and the suite has a test for each one. A check never sees the inside of a turn: there is no step that observes occurrence 1 of a two-occurrence turn, because no World can. A `Defer` answered in a turn is answered before the next check, so a sandbox's denial is visible in the first `expect` after the request, not one step later. `fired` and `quiet` are about the whole turn, so a result event that fires in a `Defer` occurrence counts. And the number of occurrences in a turn is bounded by the number of `Defer` slots plus one, so a script never hangs: a turn that would run forever is a deferring-backwards error, which `errors` catches.

Chapter 2's sandbox script is the worked case: a reader who expects to see `"(reading...)"` after the build has the turn wrong, because the denial came back through a `Defer` inside turn 0. The corrected script is in the law tests below, under the `Defer` laws.

One more rule about what a turn contains. A `send` whose response reaches a slot that was torn down in the same occurrence, because the input rebuilt the `rebuild` around it, is delivered: the slot existed as the occurrence opened, which is what routing requires, and the old widget keeps propagating until commit. The script sees the consequences in the list after the turn, and chapter 2's relabelling button is the program that shows it.

## The law tests

One short program and one short script per law, each pinning one sentence of the chapters on the functors, on time, on widgets and switching, and on elements and Worlds. The scripts are the suite's spine: when an implementation fails one, the failure names the sentence, and when a sentence changes, the script that pins it changes with it. They are deliberately tiny. A law test that needs a hundred-line program is testing a program, and the integration programs are for that.

Every script below assumes the suite's shared declarations, which are also the integration programs' and the GUI library's:

```
type Element = choice ..
type Click = click
type Tick = tick
type Instant = Instant Int

tag Ui.Label of Element:
    text: Text

tag Ui.Button of Element:
    label: Text
    responds: Click

tag Clock of Element:
    state: Instant

tag Defer of Element:
    requests: !unit
    responds: unit

type FsRequest = choice Read | Write Text

type FsError = choice:
    NotFound
    Denied
    TooLarge
    Other Text

type FsResponse = choice:
    Contents Text
    Unavailable FsError

type Contents = choice:
    Unavailable FsError
    Known Text

tag Fs.File of Element:
    path: Text
    requests: !FsRequest
    responds: FsResponse

tag Fs.Contents of Element:
    path: Text
    state: Contents

@Label: fn $Text -> @unit = \@widget:
    @Element Ui.Label(text=$_)
    @ unit

@Button: fn $Text -> @!Click = \@widget:
    @Element Ui.Button(label=$_)
```

`@Label` and `@Button` use the shorthand: a dynamic where a field goes lifts the element into a `rebuild`, one fresh slot per change, with the button's click the `!Switch` over the generations. Chapter 2 unrolls both.

## Events

**The leftmost wins.** `a | b` fires at n if either fires at n; its value is a's if a fires, else b's.

```
@left_wins:
    !inc <- @Button $"Inc"
    !dec <- @Button $"Dec"
    $n <- 0 then $n + (-1 on !dec | +1 on !inc)
    @Label "n={$n}"

@right_wins:
    !inc <- @Button $"Inc"
    !dec <- @Button $"Dec"
    $n <- 0 then $n + (+1 on !inc | -1 on !dec)
    @Label "n={$n}"

test leftmost_takes_the_left, @left_wins:
    build
        => settles
    click Ui.Button(label="Inc"), click Ui.Button(label="Dec")
        => expect Ui.Label(text="n=-1")

test leftmost_is_the_program_choice, @right_wins:
    build
        => settles
    click Ui.Button(label="Inc"), click Ui.Button(label="Dec")
        => expect Ui.Label(text="n=1")
```

Two programs that differ in the order of two operands give two answers, and the test says which is which. The second test is there so that nobody can make the first pass by always taking the decrement.

**`!never` is the identity of `|` on both sides.**

```
@never_identity:
    !b <- @Button $"Go"
    $left <- 0 then $left + 1 on (!never | !b)
    $right <- 0 then $right + 1 on (!b | !never)
    @Label "{$left} {$right}"

test never_is_the_identity, @never_identity:
    build
        => settles
    click Ui.Button(label="Go")
        => expect Ui.Label(text="1 1")
```

**`v on !e` fires exactly when `e` fires, with the value `v`.**

```
@on_button:
    !b <- @Button $"OK"
    @ "ok" on !b

test on_fires_with_its_event, @on_button:
    build
        => quiet
    click Ui.Button(label="OK")
        => fired "ok"
    push Clock = Instant 1
        => quiet
```

The `push` is a turn with no click in it, which is the only way a script gets a turn in which the button

was not pressed, and `quiet` is the claim that nothing fires from nothing.

**The tag law.** In a lifted operation with an event operand, a dynamic contributes its value before the occurrence.

```
@tag_reads_before:
    !b <- @Button $"Go"
    $n <- 0 then $n + 1 on !b
    $seen <- 0 then $n on !b
    @Label "n={$n} seen={$seen}"

test a_tag_reads_the_value_before, @tag_reads_before:
    build
        => settles
    click Ui.Button(label="Go")
        => expect Ui.Label(text="n=1 seen=0")
    click Ui.Button(label="Go")
        => expect Ui.Label(text="n=2 seen=1")
```

`$n on !b` and `$n + 1 on !b` fire in the same occurrence and both read `$n` through a tag; the first sees the count before the click, the second computes the count after. Any implementation that let `$seen` see `1` after one click has let a tag read a half-finished occurrence.

## Dynamics

**The change law.** A derived dynamic changes in the occurrence its operands change, and a derived dynamic of a derived dynamic does too.

```
@derived_prompt:
    !b <- @Button $"Go"
    $n <- 0 then $n + 1 on !b
    let $a = $n + 1
    let $b = $a * 2
    @Label "{$a} {$b}"

test derived_dynamics_are_prompt, @derived_prompt:
    build
        => expect Ui.Label(text="1 2")
    click Ui.Button(label="Go")
        => expect Ui.Label(text="2 4")
```

**At most once.** Within one occurrence a dynamic's change event fires at most once, however many paths lead to it.

```
@diamond:
    !b <- @Button $"Go"
    $n <- 0 then $n + 1 on !b
    let $sum = ($n + 1) + ($n * 2)
    !built <- rebuild $sum:
        @ unit
    $k <- 0 then $k + 1 on !built
    @Label "builds={$k}"

test a_diamond_changes_once, @diamond:
    build
        => expect Ui.Label(text="builds=1")
    click Ui.Button(label="Go")
        => expect Ui.Label(text="builds=2")
```

`$sum` depends on `$n` by two routes. A `rebuild` over it rebuilds once per change event, and the `rebuild`'s own event counts the builds: one for the first build, one per click. An implementation with a glitch, where `$sum` changed once per route, would show `builds=3` after one click. This is the only way a script can count a dynamic's changes, and it is deliberately indirect: there is no step that observes a change event, because no World can.

**A hold's initial value, and adoption in the build occurrence.** `$x <- v then !e` has the value `v` before the first firing of `e`; if `e` fires in the occurrence the widget is built, `$x` changes to it then.

```
@hold_adopts:
    !started <- @started
    $x <- 0 then 1 on !started
    $y <- 0 then $x on !started
    @Label "x={$x} y={$y}"

test a_hold_adopts_a_build_occurrence_firing, @hold_adopts:
    build
        => expect Ui.Label(text="x=1 y=0")
```

`$x` adopts `1` in occurrence 0 because its source fired there. `$y` reads `$x` through a tag in the same occurrence and gets the initial value, `0`, which is the tag law and the hold's "value before the first firing" in one line.

**A hold set to the value it already has still changes.** The change law says a derived dynamic's change event fires when an operand's change event fires; a hold's change event is its event, and nothing compares values.

```
@same_value:
    !b <- @Button $"Same"
    $v <- 7 then 7 on !b
    !built <- rebuild $v:
        @ unit
    $k <- 0 then $k + 1 on !built
    @Label "builds={$k}"

test an_equal_value_is_still_a_change, @same_value:
    build
        => settles
    click Ui.Button(label="Same")
        => expect Ui.Label(text="builds=2")
```

This pins a sentence that is easy to get the wrong way round when an implementation adds a short-circuit on equality as an optimisation. A `rebuild` over a hold rebuilds when the hold's event fires, and a program that does not want that filters the event.

## Time

**A widget built in occurrence n sees all of n.** A hold built in n whose source fired in n adopts that value at commit.

```
@sees_all_of_n:
    !add <- @Button $"Add"
    $keys <- {} then (Insert $keys, (Size $keys), unit) on !add
    for $keys:
        $hits <- 0 then $hits + 1 on !add
        @Label "{$key}: {$hits}"
```

```
test a_new_block_sees_the_event_that_made_it, @sees_all_of_n:
    build
        => settles
    click Ui.Button(label="Add")
        => expect Ui.Label(text="0: 1")
    click Ui.Button(label="Add")
        => expect Ui.Label(text="0: 2")
        => expect Ui.Label(text="1: 1")
```

The click that inserts key 0 is the click that key 0's block counts: the block behaves as though it had been built just before the click arrived. The alternative, hiding earlier firings from late subscribers, would show `0: 0` and the block would be one click behind for its whole life.

**A `rebuild` built in n over a dynamic that changed in n builds once, with the new value.**

```
@nested_dyn:
    !b <- @Button $"Go"
    $n <- 0 then $n + 1 on !b
    let $tens = $n * 10
    rebuild $n:
        let outer = _
        rebuild $tens:
            @Label "outer={$outer} inner={$_}"
        @ unit
    @ unit

test a_nested_dyn_builds_once_with_the_new_value, @nested_dyn:
    build
        => expect Ui.Label(text="outer=0 inner=0")
    click Ui.Button(label="Go")
        => expect Ui.Label(text="outer=1 inner=10")
        => expect_none Ui.Label(text="outer=1 inner=0")
        => expect_count Ui.Label(..), 1
```

The outer `rebuild` builds anew on the click; the inner one is built inside the new block, in the same occurrence, over `$tens`, which changed in that occurrence. It sees the change event and builds with `10`. An implementation that built it with the old value and then rebuilt it would leave a `Ui.Label(text="outer=1 inner=0")` in some intermediate list, and although no script can see an intermediate list, it would also have fired the inner `rebuild`'s event twice in one occurrence, which the diamond test's technique catches; this test keeps the direct form because the sentence is about the value.

**Routing against the opened list.** An input addressed to a path in occurrence n is delivered to the slot at that path in the list as n opened; the old widget keeps propagating until commit.

```
@clicker:
    !c <- @Button $label
    $count <- 0 then $count + 1 on !c
    let $label = "Clicked {$count} times"

test a_button_is_clicked_under_its_old_label, @clicker:
    build
        => settles
    click Ui.Button(label="Clicked 0 times")
        => expect Ui.Button(label="Clicked 1 times")
        => expect_none Ui.Button(label="Clicked 0 times")
    click Ui.Button(label="Clicked 1 times")
        => expect Ui.Button(label="Clicked 2 times")
```

The button the script clicks is the one in the list as the turn opened. Its click moves the count, the label changes, the `rebuild` inside `@Button` rebuilds, and the new slot is the one the next `send` finds. The cycle through `$label`, the `rebuild`, `!Switch` and `$count` is legal because `!Switch` chooses its inner event from before the occurrence and `$count + 1` reads through a tag.

## Widgets

**`@ v` builds nothing and returns `v`.**

```
@pure_five:
    @ 5

test a_pure_widget_builds_nothing, @pure_five:
    build
        => returns 5
        => expect_none ignore
```

**Each bind builds a separate widget.** `let @b = ...` followed by two binds gives two widgets, two paths, two holds.

```
@twice:
    let @counted = @widget:
        !c <- @Button $"Hit"
        $n <- 0 then $n + 1 on !c
        @Label "n={$n}"
    @counted
    @counted

test a_widget_bound_twice_is_two_nodes, @twice:
    build
        => expect_count Ui.Button(label="Hit"), 2
        => expect_count Ui.Label(text="n=0"), 2
```

A `send Ui.Button(label="Hit")` here would fail, since two slots match, and that failure is itself a check that there are two. The independence of the two holds is the `for` test below, where keys tell the twins apart.

**Holds are owned by the widget and torn down with it.** A rebuild replaces the widget; holds inside the block are new holds with their initial values.

```
@reset_on_rebuild:
    !flip <- @Button $"Flip"
    $gen <- 0 then $gen + 1 on !flip
    rebuild $gen:
        !hit <- @Button $"Hit"
        $n <- 0 then $n + 1 on !hit
        @Label "gen {$_}: {$n}"
    @ unit

test a_rebuild_resets_the_holds_inside, @reset_on_rebuild:
    build
        => settles
    click Ui.Button(label="Hit")
        => settles
    click Ui.Button(label="Hit")
        => expect Ui.Label(text="gen 0: 2")
    click Ui.Button(label="Flip")
        => expect Ui.Label(text="gen 1: 0")
```

```
        => expect_none Ui.Label(text="gen 0: 2")
```

**`@started` fires once, in the build occurrence.**

```
@starter:
    !started <- @started
    $n <- 0 then $n + 1 on !started
    @Label "started {$n}"

@started_result:
    @started

test started_fires_once_at_build, @starter:
    build
        => expect Ui.Label(text="started 1")
    push Clock = Instant 1
        => expect Ui.Label(text="started 1")

test started_is_visible_as_a_result, @started_result:
    build
        => fired unit
    push Clock = Instant 1
        => quiet
```

The second script is the one case where `fired` holds before any `send`: the widget's result is `@started`'s event directly, with no `!Switch` between, so turn 0 is a turn in which it fired.

**`@started` inside a `rebuild` fires again on every rebuild.** A rebuilt widget that loads on start loads again, visibly, in the program's text.

```
@restarter:
    !flip <- @Button $"Flip"
    $gen <- 0 then $gen + 1 on !flip
    rebuild $gen:
        !started <- @started
        @Element Fs.File(path="/f", requests=(Read on !started))
    @ unit

test a_rebuild_restarts, @restarter:
    build
        => requested Fs.File(path="/f"), Read
    click Ui.Button(label="Flip")
        => requested Fs.File(path="/f"), Read
    push Clock = Instant 1
        => unrequested Fs.File(..)
```

## Switching

**`rebuild` fires in every build occurrence, including the first.** The diamond test above counts those firings; this one checks the value.

```
@dyn_values:
    !b <- @Button $"Go"
    $n <- 0 then $n + 1 on !b
    !vals <- rebuild $n:
        @ _ * 100
    $last <- -1 then !vals
    @Label "last={$last}"
```

```
test dyn_fires_with_each_build_result, @dyn_values:
    build
        => expect Ui.Label(text="last=0")
    click Ui.Button(label="Go")
        => expect Ui.Label(text="last=100")
```

**`!Switch` is not prompt: a creation-occurrence firing is swallowed.**

```
@swallowed:
    !flip <- @Button $"Flip"
    $gen <- 0 then $gen + 1 on !flip
    !!starts <- rebuild $gen:
        !started <- @started
        @ !started
    $heard <- 0 then $heard + 1 on !Switch !!starts
    @Label "heard={$heard}"

test a_switch_does_not_hear_the_birth, @swallowed:
    build
        => expect Ui.Label(text="heard=0")
    click Ui.Button(label="Flip")
        => expect Ui.Label(text="heard=0")
```

Each generation fires `!started` in the occurrence it is born, and `!Switch` adopts the generation's event at commit, so it never hears it. A block that needs its own start uses `@started` inside the block, as `@restarter` does.

**`!Switch` follows the new inner event from the next occurrence.** The `@clicker` script above shows it: the second `send` reaches the rebuilt button and its click is heard. This one shows the same through a `rebuild` whose generations are different widgets:

```
@toggle:
    !flip <- @Button $"Flip"
    $on <- False then not $on on !flip
    !!pressed <- rebuild $on:
        if _:
            @Label $"On"
            @ !never
        else:
            @Button $"Press me"
    $presses <- 0 then $presses + 1 on !Switch !!pressed
    @Label "presses={$presses}"

test a_rebuilt_slot_is_heard_from_the_next_occurrence, @toggle:
    build
        => settles
    click Ui.Button(label="Press me")
        => expect Ui.Label(text="presses=1")
    click Ui.Button(label="Flip")
        => expect_none Ui.Button(label="Press me")
    click Ui.Button(label="Flip")
        => settles
    click Ui.Button(label="Press me")
        => expect Ui.Label(text="presses=2")
```

**`$Join` is prompt on both the outer and the inner change.**

```
@join_prompt:
    !flip <- @Button $"Flip"
    $gen <- 0 then $gen + 1 on !flip
    !$inner <- rebuild $gen:
        !hit <- @Button $"Hit"
        $n <- _ * 100 then $n + 1 on !hit
        @ $n
    $$cur <- $ 0 then !$inner
    @Label "cur={$Join $$cur}"

test join_follows_inner_and_outer_changes, @join_prompt:
    build
        => expect Ui.Label(text="cur=0")
    click Ui.Button(label="Hit")
        => expect Ui.Label(text="cur=1")
    click Ui.Button(label="Flip")
        => expect Ui.Label(text="cur=100")
    click Ui.Button(label="Hit")
        => expect Ui.Label(text="cur=101")
```

The hit changes the inner dynamic; the join changes in that occurrence. The flip changes the outer: a new block, a new inner with initial `100`, and the join shows `100` in the occurrence of the flip, not one later. This is the asymmetry with `!Switch` written as two `expect`s.

**`for` consolidates events into one firing with exactly the entries that fired.**

```
@for_events:
    let $m = $ {"a": unit, "b": unit}
    !hits <- for $m:
        !h <- @Button ($ key)
        @ key on !h
    $last <- 0 then Size !hits
    @Label "fired {$last}"

test for_consolidates_simultaneous_firings, @for_events:
    build
        => settles
    click Ui.Button(label="a"), click Ui.Button(label="b")
        => expect Ui.Label(text="fired 2")
    click Ui.Button(label="a")
        => expect Ui.Label(text="fired 1")
```

**`for` consolidates dynamics into a dynamic map, and plain values into a dynamic map.**

```
@for_dynamics:
    let $m = $ {"a": unit, "b": unit}
    $counts <- for $m:
        !h <- @Button ($ key)
        $n <- 0 then $n + 1 on !h
        @ $n
    $caps <- for $m:
        @ Upper key
    @Label "total={Fold $counts, 0, \|acc, k, v| acc + v} {Fold $caps, "", \|acc, k, v| Concat [[acc, v]]}"

test for_consolidates_dynamics_and_values, @for_dynamics:
    build
        => expect Ui.Label(text="total=0 AB")
    click Ui.Button(label="a")
        => settles
    click Ui.Button(label="a")
```

```
        => settles
    click Ui.Button(label="b")
        => expect Ui.Label(text="total=3 AB")
```

**A key's block keeps its state across an insert and a neighbour's removal.**

```
@Tally: fn $Text -> @unit = \@widget:
    !hit <- @Button $_
    $n <- 0 then $n + 1 on !hit
    @Label "{$_}: {$n}"

@tallies:
    !add <- @Button $"Add c"
    !drop <- @Button $"Drop a"
    $names <- {"a": "a", "b": "b"} then ((Insert $names, "c", "c") on !add) | ((Remove $names, "a") on !drop)
    for $names:
        @Tally $_

test state_survives_an_insert, @tallies:
    build
        => settles
    click Ui.Button(label="a")
        => settles
    click Ui.Button(label="a")
        => settles
    click Ui.Button(label="Add c")
        => expect Ui.Label(text="a: 2")
        => expect Ui.Label(text="c: 0")

test state_survives_a_neighbours_removal, @tallies:
    build
        => settles
    click Ui.Button(label="b")
        => settles
    click Ui.Button(label="Drop a")
        => expect_none Ui.Button(label="a")
        => expect Ui.Label(text="b: 1")
```

**The parent sees a new key's result in the creation occurrence; the new block does not see itself.**

```
@new_key_seen:
    !add <- @Button $"Add"
    $keys <- {} then (Insert $keys, (Size $keys), unit) on !add
    !starts <- for $keys:
        !started <- @started
        @ key on !started
    $seen <- 0 then $seen + Size !starts
    @Label "seen={$seen}"

test a_parent_sees_a_new_key_at_once, @new_key_seen:
    build
        => expect Ui.Label(text="seen=0")
    click Ui.Button(label="Add")
        => expect Ui.Label(text="seen=1")
    click Ui.Button(label="Add")
        => expect Ui.Label(text="seen=2")
```

`for` consolidation is prompt where `!Switch` is not: a block born in n is in the consolidated event in

n. Only pure code can fire in a block's birth occurrence, and `@started` is the pure code that does.

**`for` over an array is keyed by index, and state follows the index.** This is the sentence that makes arrays second class, and it is pinned so that nobody improves it by accident.

```
@indexed:
    !front <- @Button $"Push front"
    $xs <- [["a", "b"]] then (Prepend $xs, "z") on !front
    for $xs:
        !h <- @Button $_
        $n <- 0 then $n + 1 on !h
        @Label "{$key}: {$n}"

test an_array_block_keeps_state_by_position, @indexed:
    build
        => settles
    click Ui.Button(label="b")
        => expect Ui.Label(text="1: 1")
    click Ui.Button(label="Push front")
        => expect Ui.Button(label="z")
        => expect Ui.Label(text="1: 1")
        => expect Ui.Label(text="2: 0")
```

After the insertion at the front, the block at index 1 holds `"a"` and still has the count that was given to `"b"`. A program that wants the count to follow the item keeps a map with allocated ids, which is chapter 2's todo list. If arrays ever get a stable per-element identity, this test is the one that changes, and it would then say `expect Ui.Label(text="2: 1")`.

## Elements and interception

**Elements never fire in the occurrence they were created, and `@started` fires the request in the build occurrence.**

```
@requester:
    !started <- @started
    !r <- @Element Fs.File(path="/f", requests=(Read on !started))
    $got <- "none" then "some" on !r
    @Label "got={$got}"

test a_request_is_answered_later, @requester:
    build
        => requested Fs.File(path="/f"), Read
        => expect Ui.Label(text="got=none")
    send Fs.File(path="/f"), Contents "x"
        => expect Ui.Label(text="got=some")
```

**Creating an element requests nothing.**

```
@silent:
    @Element Fs.File(path="/f", requests=!never)
    @ unit

test creation_requests_nothing, @silent:
    build
        => unrequested Fs.File(..)
    push Clock = Instant 1
        => unrequested Fs.File(..)
```

This is the shortest test in the suite and the one that guards the most. A World component that

performed a read because an `Fs.File` appeared would not fail this test, since the Test World has no components; this test checks the Test World's own script component, and the real Worlds are checked against the same sentence in their own tests.

**A response to a path not in the opened list is an error.** No script can send to an unknown path, because `send` resolves against the current list; the only way to make the error is an interceptor that answers a slot in the occurrence the slot was created.

```
@Eager: fn @a -> @a = \|@child| @widget:
    ((result, $slots)) <- intercept @child, !down, $ \|e| @Element e
    !down <- for $slots:
        rebuild $_:
            match _.element:
                Ui.Button(..):
                    !started <- @started
                    !resp <- @Element _.element
                    @ !resp | click on !started
                ignore:
                    ((!resp, ignore)) <- @Element _.element
                    @ resp
    @ result

test answering_at_creation_is_an_unknown_path, (@Eager @left_wins):
    build
        => errors Error(kind=UnknownPath, ..)
```

The child's buttons are created in occurrence 0; `@Eager`'s blocks for them are built in the same occurrence, fire `click` from `@started`, and `!down` fires with paths that are not in the list as the occurrence opened, which for occurrence 0 is empty. The build aborts. This is also the law that an interceptor answers requests only in a later occurrence, from the other side: the one way to answer a slot is a `Defer`, and `@Eager` did not use one.

**A state element has, in its creation occurrence, the value from the snapshot.**

```
@clock_label:
    $t <- @Element Clock
    @Label "t={$t.value}"

test state_is_there_at_creation, @clock_label:
    with Clock = Instant 5, build
        => expect Ui.Label(text="t=5")
```

**A push reaches a slot created in the push's occurrence.**

```
@late_clock:
    !show <- @Button $"Show"
    $shown <- False then True on !show
    rebuild $shown:
        if _:
            $t <- @Element Clock
            @Label "t={$t.value}"
        else:
            @Label $"hidden"
    @ unit

test a_push_reaches_a_slot_born_in_its_occurrence, @late_clock:
    with Clock = Instant 1, build
        => expect Ui.Label(text="hidden")
    with Clock = Instant 2, click Ui.Button(label="Show")
```

```
        => expect Ui.Label(text="t=2")
        => expect_none Ui.Label(text="t=1")
```

The `with` before the `send` makes the `send`'s turn a turn with a push and a response. The `Clock` slot is created by the click, reads the oracle, which answers from the previous snapshot, `1`, and then receives the push to `2` in the same occurrence, ending it holding what every other slot holds. A test that showed `t=1` would be an implementation that sampled the oracle and forgot the push, which is a slot one occurrence behind the world for its whole life.

**Two slots of equal value observe one state.**

```
@two_clocks:
    $a <- @Element Clock
    $b <- @Element Clock
    @Label "{$a.value} {$b.value}"

test equal_elements_share_one_state, @two_clocks:
    with Clock = Instant 1, build
        => expect Ui.Label(text="1 1")
    push Clock = Instant 2
        => expect Ui.Label(text="2 2")
```

**An interceptor that forwards everything is observably not there.**

```
@Transparent: fn @a -> @a = \|@child| @widget:
    ((result, $slots)) <- intercept @child, !down, $ \|e| @Element e
    !down <- for $slots:
        rebuild $_:
            ((!resp, ignore)) <- @Element _.element
            @ resp
    @ result

@counter_with_clock:
    $t <- @Element Clock
    !inc <- @Button $"Inc"
    $n <- 0 then $n + 1 on !inc
    @Label "n={$n} t={$t.value}"

test forwarding_is_the_identity, (@Transparent (@Transparent @counter_with_clock)):
    with Clock = Instant 1, build
        => expect_count Ui.*(..), 2
        => expect Ui.Label(text="n=0 t=1")
    click Ui.Button(label="Inc")
        => settles
    push Clock = Instant 2
        => expect Ui.Label(text="n=1 t=2")
```

The transcript of this test is compared, by the harness, with the transcript of the same script run on `@counter_with_clock` alone, element by element. Paths differ, because the forwarder's slots are at the forwarder's paths; everything else must be identical. The `Clock` appears more than once in the outer list, once per forwarder's supplier and once per forwarded slot, and each is the same state by value. A forwarder that minded the duplicates would not forward state slots; this one is the shortest correct program.

**A sandbox supplies `Unavailable(Denied)`, and nothing confined reaches the World.** `@Sandbox` and `ShowContents` are chapter 2's, with a refusal spelled `Unavailable(Denied)` and `ShowContents` matching `Unavailable why`.

```
@peek:
    $c <- @Element Fs.Contents(path="/etc/secret")
    @Label (ShowContents $c)

test a_sandbox_denies_state, (@Sandbox "/jail", @peek):
    with Fs.Contents(path="/etc/secret") = Known "hunter2", build
        => expect Ui.Label(text="(unavailable)")
        => expect_none Fs.Contents(path="/etc/secret")
```

The script sets the real value and the child never sees it: the supplier returned `@ $ Unavailable(Denied)`, which is a blueprint with no elements, so nothing about `/etc/secret` is in the outer list.

**A supplied widget's own elements appear in the interceptor's list.**

```
@peek_home:
    $c <- @Element Fs.Contents(path="/home/me/notes")
    @Label (ShowContents $c)

test a_supplier_asks_the_outer_world, (@Sandbox "/jail", @peek_home):
    with Fs.Contents(path="/jail/home/me/notes") = Known "hi", build
        => expect_top Fs.Contents(path="/jail/home/me/notes")
        => expect_none Fs.Contents(path="/home/me/notes")
        => expect Ui.Label(text="hi")
    push Fs.Contents(path="/jail/home/me/notes") = Known "bye"
        => expect Ui.Label(text="bye")
```

**Two child slots of equal value share one supplied widget.**

```
@Supplied: fn @a -> @a = \|@child| @widget:
    ((result, ignore)) <- intercept @child, !never, $ \|e| @Element e
    @ result

@two_readers:
    $a <- @Element Fs.Contents(path="/x")
    $b <- @Element Fs.Contents(path="/x")
    @Label "{ShowContents $a} {ShowContents $b}"

test equal_slots_share_a_supplier, (@Supplied @two_readers):
    with Fs.Contents(path="/x") = Known "v1", build
        => expect_count Fs.Contents(path="/x"), 1
        => expect_none Ui.*(..)
    push Fs.Contents(path="/x") = Known "v2"
        => returns unit
```

`@Supplied` forwards nothing and supplies by forwarding, so the only elements in the outer list are the supplied widgets'. Two child slots, one supplied `Fs.Contents`. The labels are held back, so the test cannot read them, and the harness checks the sharing through the count; the integration program for suppliers, later, reads the labels through a forwarder.

**`Defer` is answered later in the same turn.** `@Sandbox` denies a request through a `Defer`, and the denial is visible by the first check after the build.

```
let ShowResponse: fn ?FsResponse -> Text = \|r| match r:
    Nil: "(reading...)"
    Some (Contents t): t
    Some Unavailable(NotFound): "(no such file)"
    Some Unavailable(Denied): "(permission denied)"
    Some Unavailable(why): "(unreadable)"
```

```
@probe:
    !started <- @started
    !r <- @Element Fs.File(path="/etc/shadow", requests=(Read on !started))
    $seen <- Nil then Some !r
    @Label (ShowResponse $seen)

test a_sandbox_denies_within_the_turn, (@Sandbox "/jail", @probe):
    build
        => expect_none Fs.File(..)
        => requested Defer, unit
        => expect Ui.Label(text="(permission denied)")
```

Turn 0 is two occurrences: the build, in which the probe requests and the sandbox fires its `Defer`, and the answer, in which the sandbox fires `Denied` into the probe. The script's first check runs after both. A reader who writes `expect Ui.Label(text="(reading...)")` here has the turn wrong, which is why chapter 2 shows that version failing.

**Deferring backwards is an error.** A `Defer` slot that fires again after being answered in the same turn.

```
@backwards:
    !started <- @started
    !back <- @Element Defer(requests=(unit on !started | unit on !back))
    @ unit

test deferring_backwards_is_an_error, @backwards:
    build
        => errors Error(kind=DeferBackwards, ..)
```

Occurrence 0 fires the request; occurrence 1 of the same turn answers it, and the answer fires the request again. The World refuses to open a third occurrence for it, the turn ends in the error, and there is no turn that runs forever. This is the only bound on a turn and it needs no cap.

**A turn with chained `Defer`s runs one occurrence per link; independent `Defer`s are answered together.**

```
@chain:
    !started <- @started
    !r1 <- @Element Defer(requests=(unit on !started))
    !r2 <- @Element Defer(requests=(unit on !r1))
    $log <- [[]] then (Append $log, "a" on !r1) | (Append $log, "b" on !r2)
    @Label (JoinText ",", $log)

@fan:
    !started <- @started
    !r1 <- @Element Defer(requests=(unit on !started))
    !r2 <- @Element Defer(requests=(unit on !started))
    $log <- [[]] then (Append $log, "a" on !r1) | (Append $log, "b" on !r2)
    @Label (JoinText ",", $log)

test chained_defers_are_three_occurrences, @chain:
    build
        => expect Ui.Label(text="a,b")

test independent_defers_are_answered_together, @fan:
    build
        => expect Ui.Label(text="a")
```

In `@chain`, `r2`'s request fires only when `r1` is answered, so the turn is three occurrences and the log has both entries. In `@fan`, both requests fire in occurrence 0, the World answers both in occurrence 1, both `Append`s fire in that one occurrence, and the leftmost wins: the log has one entry. The second script is also a test that the World does not answer `Defer`s one per occurrence, which would be a turn length that depended on the World's mood.

**A state tag with no supplier is an error at the slot's creation.** Shown with the script language's own rule, since the Test World's supplier is the script; `Secret` and `@needs_secret` are the ones declared with the `errors` step above.

```
test an_unsupplied_state_fails_at_creation, @needs_secret:
    build
        => errors Error(kind=UnsuppliedState, element=Secret, ..)

test a_supplied_state_is_fine, @needs_secret:
    with Secret = "shh", build
        => expect Ui.Label(text="shh")
```

That is forty-four scripts. Each one is short enough to appear whole in a failure message, each is named by its sentence, and each would be the first thing to change if its sentence changed.

## The integration programs

Where a law test pins one sentence, an integration program pins one piece of meaning as a user of the language meets it: a whole widget with several tests, built on the suite's GUI library, driven through containers, with the laws interacting. Twelve of them exist and nine more join them. Each program file holds its widgets and its tests, and each is also a typed-AST golden and a differential fixture, so a program earns its place three times.

### The GUI library

The library is Prism over `@Element` and `intercept`, and it is the convention the Test World's depth rule depends on. Here it is in full, with the shared declarations of the law tests assumed.

```
type ChildInput = record:
    path: Path
    response: Response

tag Ui.VBox of Element:
    children: Path ==> Element
    responds: ChildInput

let IsUi: fn Element -> Bool = \|e| match e:
    Ui.*(..): True
    ignore: False

let UiOnly: fn (Path ==> Slot) -> Path ==> Element = \|slots|
    MapValues (Filter slots, \IsUi _.element), \_.element

let NonUi: fn (Path ==> Slot) -> Path ==> Slot = \|slots|
    Filter slots, \not (IsUi _.element)

let ToMap: fn ChildInput -> Path ==> Response = \|c|
    {c.path: c.response}
```

```
# Re-emit every slot unchanged; route each response back under the child's path.
@Forward: fn $(Path ==> Slot) -> @!(Path ==> Response) = \|$slots| @widget:
    for $slots:
        rebuild $_:
            ((!resp, ignore)) <- @Element _.element
            @ resp

# Hold the GUI children back as data in one VBox; forward everything else.
@VBox: fn @a -> @a = \|@child| @widget:
    ((result, $slots)) <- intercept @child, !down, $ \|e| @Element e
    !child_in <- @Element Ui.VBox(children=(UiOnly $slots))
    !fwd <- @Forward (NonUi $slots)
    let !down = !Union (ToMap !child_in), !fwd
    @ result
```

`@Element Ui.VBox(children=(UiOnly $slots))` has a dynamic where a field goes, so it is a `rebuild` with a fresh `Ui.VBox` slot per change of the child's `Ui.*` list, and `!child_in` is the `!Switch` over the generations. The supplier `\|e| @Element e` forwards every state element to the outer context; the `for` in `@Forward` builds one block per non-`Ui.*` child slot, each re-emitting the element and returning the response the outer context sends to it, consolidated under the child's path. `ToMap` turns a `ChildInput` into a one-entry response map, and `!Union` merges it with the forwarded responses, so a click routed through the box and a file response for a forwarded slot can land in one occurrence.

The library's own typed-AST dump is a golden, and `@VBox` under the Test World, with a counter inside, is program 06.

## The twelve

| Program | What it pins |
|---|---|
| `01_element_roundtrip` | An element goes out and a response comes back to the same slot; a response the widget does not want is ignored by the widget, not the World. |
| `02_counter` | The counter, and the leftmost merge when both buttons fire in one occurrence. |
| `03_hold_cycle` | A hold whose dynamic appears in its own update, read through a tag; `returns` on a dynamic result. |
| `04_intercept_consumes` | What an interceptor does not re-emit never reaches the World, while the child stays live. |
| `05_forward_transparent` | Wrapping in a forwarder twice is observably the identity. |
| `06_vbox_routing` | A click routed down through a container; a non-`Ui.*` response bypassing it; both in one occurrence. |
| `07_dyn_rebuild` | A rebuild swaps slots, new slots get fresh paths, and a rebuilt slot receives input from the next occurrence. |
| `08_for_keyed` | One child per key, with state kept across an insert and a neighbour's removal. |

| Program | What it pins |
|---|---|
| `09_non_prompt` | Routing against the list as the occurrence opened: a button clicked under its old label. |
| `10_tag_matching` | `match` on elements: by tag, by namespace, by field value, with the catch-all; a nested element in a field. |
| `11_simultaneous` | Several responses to different slots in one occurrence are all delivered. |
| `12_widget_output_events` | `fired` and `quiet` on a widget's result event. |

The programs as written here are updated to the current syntax: numbers by inference with `-1` and `+1` where a literal needs a sign, tags declared with their fields and response types, `click` as the click, the three-argument `intercept`, and the `ChildInput` convention. Where a program's test coincides with a law test above, the program keeps it, because the program is also a golden and a differential fixture and the duplication costs nothing.

### 01. An element goes out and comes back.

```
type Poke = choice:
    Set Nat
    Other

tag Test.Probe of Element:
    name: Text
    value: Nat
    responds: Poke

let SetValue: fn Poke -> ?Nat = \|p| match p:
    Set n: Some n
    Other: Nil

@echo:
    !in <- @Element Test.Probe(name="echo", value=$v)
    $v <- 0 then !Filter (SetValue !in)

test element_roundtrip, @echo:
    build
        => expect_top Test.Probe(name="echo", value=0)
    send Test.Probe(name="echo", ..), Set 5
        => expect_top Test.Probe(name="echo", value=5)
    send Test.Probe(name="echo", ..), Other
        => expect_top Test.Probe(name="echo", value=5)
```

`value=$v` lifts the element into a `rebuild`, so `expect_top Test.Probe(name="echo", value=5)` after the `send` is a fresh slot at generation `#1`, and the `send` of `Other` is addressed to that one. The response type is declared, so a `send` of something that is not a `Poke` is a compile error in the test file; what the program ignores is a `Poke` it has no use for.

### 02. The counter.

```
@counter:
    !increment <- @Button $"Increment"
    !decrement <- @Button $"Decrement"
    let !delta = -1 on !decrement | +1 on !increment
    $count <- 0 then $count + !delta
    @Label "Count: {$count}"
```

```
test counter, @counter:
    build
        => expect Ui.Label(text="Count: 0")
    click Ui.Button(label="Increment")
        => settles
    click Ui.Button(label="Increment")
        => expect Ui.Label(text="Count: 2")
    click Ui.Button(label="Decrement")
        => settles
    click Ui.Button(label="Decrement")
        => expect Ui.Label(text="Count: 0")

test counter_same_occurrence_left_wins, @counter:
    build
        => settles
    click Ui.Button(label="Increment"), click Ui.Button(label="Decrement")
        => expect Ui.Label(text="Count: -1")
```

### 03. A hold in its own update.

```
type Add = Add Int

tag Test.Accumulator of Element:
    total: Int
    responds: Add

@accumulator:
    !inc <- @Element Test.Accumulator(total=$total)
    $total <- 0 then $total + !inc.value
    @ $total

test accumulator, @accumulator:
    build
        => returns 0
    send Test.Accumulator(..), Add 3
        => returns 3
    send Test.Accumulator(..), Add -4
        => returns -1
    send Test.Accumulator(..), Add 1
        => returns 0
        => expect_top Test.Accumulator(total=0)
```

`!inc.value` is `.value` lifted over the event, unwrapping the newtype; `$total + !inc.value` reads `$total` through a tag. The hold is `Int` because `Add` holds an `Int`, and the `0` follows.

### 04. An interceptor that swallows.

```
@counted:
    !increment <- @Button $"Increment"
    $count <- 0 then $count + 1 on !increment
    @Label "Count: {$count}"
    @ $count

@Swallow: fn @a -> @a = \|@child| @widget:
    ((result, ignore)) <- intercept @child, !never, $ \|e| @Element e
    @ result

test swallow_hides_elements, (@Swallow @counted):
    build
```

```
        => expect_none Ui.*(..)
        => returns 0

@CountSlots: fn @a -> @a = \|@child| @widget:
    ((result, $slots)) <- intercept @child, !down, $ \|e| @Element e
    !down <- @Forward $slots
    @Element Test.SlotCount(count=(Size $slots))
    @ result

tag Test.SlotCount of Element:
    count: Nat

test count_and_forward, (@CountSlots @counted):
    build
        => expect_top Test.SlotCount(count=2)
    click Ui.Button(label="Increment")
        => expect Ui.Label(text="Count: 1")
        => returns 1
```

The child under `@Swallow` is built and its result is live, and nothing it made is in the outer list. Under `@CountSlots` the count is a dynamic field, so `Test.SlotCount` is a `rebuild`, one slot per change of the child's list; the count stays `2` across the click because the child replaces a label and keeps two slots.

**05. Forwarding twice is the identity.**

```
@Forwarded: fn @a -> @a = \|@child| @widget:
    ((result, $slots)) <- intercept @child, !down, $ \|e| @Element e
    !down <- @Forward $slots
    @ result

test forward_is_identity, (@Forwarded (@Forwarded @counted)):
    build
        => expect_count Ui.*(..), 2
    click Ui.Button(label="Increment")
        => settles
    click Ui.Button(label="Increment")
        => expect_top Ui.Label(text="Count: 2")
        => returns 2
```

The harness also compares this transcript with `@counted`'s under the same script, paths aside.

**06. A container routes down.**

```
type Accepted = Accepted Nat

tag Net.Listener of Element:
    port: Nat
    responds: Accepted

@panel:
    ((!ok, !conn)) <- @VBox @widget:
        @Label $"Hello"
        !b <- @Button $"OK"
        !l <- @Element Net.Listener(port=8080)
        @ ((!b, !l))
    $oks <- 0 then $oks + 1 on !ok
    $conns <- 0 then $conns + 1 on !conn
    @Label "ok={$oks} conns={$conns}"

test vbox_wraps_gui, @panel:
```

```
    build
        => expect_top Ui.VBox(..)
        => expect_top_none Ui.Button(..)
        => expect Ui.Button(label="OK")
        => expect Ui.Label(text="Hello")
        => expect_top Ui.Label(text="ok=0 conns=0")

test vbox_forwards_non_gui, @panel:
    build
        => expect_top Net.Listener(port=8080)
        => expect_count Net.Listener(..), 1

test click_routes_through_vbox, @panel:
    build
        => settles
    click Ui.Button(label="OK")
        => expect_top Ui.Label(text="ok=1 conns=0")

test forwarded_response_bypasses_vbox, @panel:
    build
        => settles
    send Net.Listener(port=8080), Accepted 7
        => expect_top Ui.Label(text="ok=0 conns=1")

test both_in_one_occurrence, @panel:
    build
        => settles
    click Ui.Button(label="OK"), send Net.Listener(port=8080), Accepted 7
        => expect_top Ui.Label(text="ok=1 conns=1")
```

The `send` to the button is delivered by the Test World as `ChildInput(path=p, response=click)` to the `Ui.VBox` slot, where `p` is the button's key in the box's `children`; `@VBox` turns it into `{p: click}` and `intercept` routes it. The listener's `Accepted` goes to the forwarded slot at the top and comes back under the child's path through `!fwd`. In the last test both arrive in one occurrence and `!Union` delivers both.

**07. A rebuild swaps slots.** The `@toggle` widget of the law tests, with the original two scripts:

```
test dyn_swaps_slots, @toggle:
    build
        => expect Ui.Button(label="Press me")
        => expect_none Ui.Label(text="On")
    click Ui.Button(label="Flip")
        => expect_none Ui.Button(label="Press me")
        => expect Ui.Label(text="On")
    click Ui.Button(label="Flip")
        => expect Ui.Button(label="Press me")

test rebuilt_slot_receives_input, @toggle:
    build
        => settles
    click Ui.Button(label="Flip")
        => settles
    click Ui.Button(label="Flip")
        => settles
    click Ui.Button(label="Press me")
        => expect Ui.Label(text="presses=1")
```

The transcript shows the "Press me" button at `2/#0/0` in turn 0 and at `2/#2/0` after the second flip:

a fresh path per generation, never reused, which is the sentence the transcript pins and the script cannot.

**08. One child per key.** `@tallies` of the law tests, with its third script:

```
test one_child_per_key, @tallies:
    build
        => expect_count Ui.Label(text="a: 0"), 1
        => expect_count Ui.Label(text="b: 0"), 1
        => expect_none Ui.Label(text="c: 0")
```

**09. Clicked under the old label.** `@clicker` of the law tests, as written there.

**10. Matching on elements.**

```
type Inner = Inner Text

tag Test.Box of Element:
    value: Nat
    inner: Inner

let Describe: fn Element -> Text = \|e| match e:
    Ui.Button(..): "button"
    Ui.*(..): "other ui"
    Test.Box(value=3, ..): "box with 3"
    Test.Box(..): "some box"
    ignore: "unknown"

@describe_button:
    @ Describe Ui.Button(label="x")

@describe_label:
    @ Describe Ui.Label(text="x")

@describe_box_3:
    @ Describe Test.Box(value=3, inner=(Inner "i"))

@describe_box_4:
    @ Describe Test.Box(value=4, inner=(Inner "i"))

@describe_other:
    @ Describe Fs.Contents(path="/tmp/x")

test tag_match, @describe_button:
    build
        => returns "button"

test namespace_match, @describe_label:
    build
        => returns "other ui"

test field_value_match, @describe_box_3:
    build
        => returns "box with 3"

test field_value_fall_through, @describe_box_4:
    build
        => returns "some box"

test catch_all, @describe_other:
    build
```

```
        => returns "unknown"

@nested:
    !in <- @Element Test.Probe(name="outer", value=$n)
    $n <- 0 then 1 on !in
    @Element Test.Box(value=$n, inner=(Inner "i"))
    @ unit

test a_nested_element_field_is_matched, @nested:
    build
        => expect_top Test.Box(value=0, inner=Inner "i")
    send Test.Probe(name="outer", ..), Other
        => expect_top Test.Box(value=1, inner=Inner "i")
```

There is no test here for a field of the wrong type falling through to the next arm, because a tag's field types are fixed by its declaration and a `Test.Box(value="3")` cannot be built; that case is a negative golden.

**11. Simultaneous responses.**

```
@pair:
    !a <- @Button $"A"
    !b <- @Button $"B"
    $as <- 0 then $as + 1 on !a
    $bs <- 0 then $bs + 1 on !b
    $last <- "" then "A" on !a | "B" on !b
    @Label "{$as} {$bs} {$last}"

test together_delivers_all, @pair:
    build
        => settles
    click Ui.Button(label="A"), click Ui.Button(label="B")
        => expect Ui.Label(text="1 1 A")

test separate_occurrences, @pair:
    build
        => settles
    click Ui.Button(label="B")
        => settles
    click Ui.Button(label="A")
        => expect Ui.Label(text="1 1 A")
    click Ui.Button(label="B")
        => expect Ui.Label(text="1 2 B")
```

**12. A result event.** `@on_button` of the law tests, with `fired "ok"` after a click and `quiet` after a push.

## The successors

Nine programs for the sentences the first twelve do not reach. Each is a program a user might write, not a minimal law script, and the tests say what the program means at the points where a reader guesses wrong.

**13. `@started` kicks a request whose answer arrives in a later turn.**

```
@motd:
    !started <- @started
    !refresh <- @Button $"Refresh"
```

```
    !resp <- @Element Fs.File(path="/etc/motd", requests=(Read on (!started | !refresh)))
    $shown <- Nil then Some !resp
    @Label (ShowResponse $shown)

test started_requests_and_waits, @motd:
    build
        => requested Fs.File(path="/etc/motd"), Read
        => expect Ui.Label(text="(reading...)")
    send Fs.File(path="/etc/motd"), Contents "Welcome"
        => expect Ui.Label(text="Welcome")
        => unrequested Fs.File(..)

test refresh_requests_again, @motd:
    build
        => settles
    send Fs.File(path="/etc/motd"), Contents "Welcome"
        => settles
    click Ui.Button(label="Refresh")
        => requested Fs.File(path="/etc/motd"), Read
        => expect Ui.Label(text="Welcome")
    send Fs.File(path="/etc/motd"), Unavailable(NotFound)
        => expect Ui.Label(text="(no such file)")

test a_response_in_the_request_turn_is_impossible, @motd:
    build
        => settles
    click Ui.Button(label="Refresh")
        => expect Ui.Label(text="(reading...)")
```

The last script looks trivial and is the creation rule in its everyday form. The refresh turn fires a request; the label is still `"(reading...)"` after it because no World can answer in the request's occurrence and the script has not answered yet. The trigger is a value, `!started | !refresh`, which a test, a mock or a gate can replace without touching the element; that is what `@started` buys over an element that loaded itself.

**14. A `rebuild` child's creation-occurrence output is swallowed, and a request from inside is not.**

```
@reloader:
    !flip <- @Button $"Flip"
    $gen <- 0 then $gen + 1 on !flip
    !!births <- rebuild $gen:
        !started <- @started
        !resp <- @Element Fs.File(path="/data", requests=(Read on !started))
        $got <- "none" then "some" on !resp
        @Label "gen {$_}: {$got}"
        @ !started
    $heard <- 0 then $heard + 1 on !Switch !!births
    @Label "heard={$heard}"

test birth_is_swallowed_but_the_request_is_not, @reloader:
    build
        => expect Ui.Label(text="heard=0")
        => requested Fs.File(path="/data"), Read
        => expect Ui.Label(text="gen 0: none")
    send Fs.File(path="/data"), Contents "x"
        => expect Ui.Label(text="gen 0: some")
    click Ui.Button(label="Flip")
        => expect Ui.Label(text="heard=0")
```

```
        => expect Ui.Label(text="gen 1: none")
        => expect_none Ui.Label(text="gen 0: some")
        => requested Fs.File(path="/data"), Read
```

The parent never hears `!started` through the `!Switch`, in either generation. The request fired by that same `!started` reaches the World every time, because it goes out through the element list and not through the switch. The two paths out of a block are different in kind, and this program is the place a reader sees both at once.

**15. A `for` parent sees a new key's output in the creation occurrence.**

```
type Change = choice:
    Opened
    Toggled

@registry_count:
    !add <- @Button $"Add"
    $next <- 0 then $next + 1 on !add
    $items <- {} then (Insert $items, $next, False) on !add
    !events <- for $items:
        !started <- @started
        !toggle <- @Button "Toggle {$key}"
        @ Opened on !started | Toggled on !toggle
    $opened <- 0 then $opened + Size (Filter !events, \_ == Opened)
    $toggled <- 0 then $toggled + Size (Filter !events, \_ == Toggled)
    @Label "opened={$opened} toggled={$toggled}"

test a_new_key_reports_in_its_birth_occurrence, @registry_count:
    build
        => settles
    click Ui.Button(label="Add")
        => expect Ui.Label(text="opened=1 toggled=0")
    click Ui.Button(label="Toggle 0")
        => expect Ui.Label(text="opened=1 toggled=1")
    click Ui.Button(label="Add"), click Ui.Button(label="Toggle 0")
        => expect Ui.Label(text="opened=2 toggled=2")
```

On the two-input line, key 1 is born and reports `Opened` while key 0 reports `Toggled`, and the consolidated event has both: a map with two entries, one from a block that existed as the occurrence opened and one from a block that did not. The parent's two holds each count what they count. The new block does not see itself, which is why its own `Opened` is in the parent's event and not in anything inside key 1.

**16. A state element is sampled at creation.**

```
let Show: fn Contents -> Text = \|c| match c:
    Unavailable why: "-"
    Known t: t

@watcher:
    !open <- @Button $"Open"
    $open <- False then True on !open
    rebuild $open:
        if _:
            $c <- @Element Fs.Contents(path="/cfg")
            @Label (Show $c)
        else:
            @Label $"closed"
```

```
    @ unit

test state_is_sampled_when_the_slot_is_made, @watcher:
    with Fs.Contents(path="/cfg") = Known "v1", build
        => expect Ui.Label(text="closed")
    click Ui.Button(label="Open")
        => expect Ui.Label(text="v1")

test an_unavailable_state_has_its_reason, @watcher:
    with Fs.Contents(path="/cfg") = Unavailable(NotFound), build
        => settles
    click Ui.Button(label="Open")
        => expect Ui.Label(text="-")
    push Fs.Contents(path="/cfg") = Known "v1"
        => expect Ui.Label(text="v1")
    push Fs.Contents(path="/cfg") = Unavailable(TooLarge)
        => expect Ui.Label(text="-")

test a_slot_made_with_no_value_set_is_the_error, @watcher:
    build
        => expect Ui.Label(text="closed")
    click Ui.Button(label="Open")
        => errors Error(kind=UnsuppliedState, element=Fs.Contents(path="/cfg"), ..)
        => expect Ui.Label(text="closed")

test a_snapshot_set_while_closed_is_seen_on_open, @watcher:
    with Fs.Contents(path="/cfg") = Known "v1", with Fs.Contents(path="/cfg") = Known "v2", build
        => settles
    click Ui.Button(label="Open")
        => expect Ui.Label(text="v2")
```

The second script is the negative space: the build needs no `with` because no `Fs.Contents` exists yet, and the click that creates one fails with the unsupplied-state error, the turn aborts, and the list is the closed one it was before. The third script sets the snapshot twice before one turn. The oracle answers `v1`, the previous snapshot; the push in the same occurrence is to `v2`; the slot ends the turn at `v2`. A reader who expected `v1` has the order of oracle and push backwards, and a reader who expected the second `with` to be lost has forgotten that a World gathers everything that changed into one snapshot per occurrence.

### 17. A push reaches a slot created in the push's occurrence, and every slot alike.

```
@gauges:
    !more <- @Button $"More"
    $n <- 1 then $n + 1 on !more
    let $keys = Range $n
    $readings <- for $keys:
        $t <- @Element Clock
        @ $t.value
    @Label (JoinText ",", (MapValues $readings, ToText))

test every_gauge_reads_one_clock, @gauges:
    with Clock = Instant 10, build
        => expect Ui.Label(text="10")
    with Clock = Instant 11, click Ui.Button(label="More")
        => expect Ui.Label(text="11,11")
    push Clock = Instant 12
        => expect Ui.Label(text="12,12")
    click Ui.Button(label="More")
```

```
        => expect Ui.Label(text="12,12,12")
```

`Range n` is the map `{0: unit, ..., n-1: unit}`. The second gauge is created in the occurrence of the `11` push and shows `11` with the first. The third is created with no push in its occurrence and shows the snapshot. Every reading in every occurrence is the same, which is the equality law stated as a label.

**18. A supplied widget's elements appear in the interceptor's list.** `@Sandbox` of chapter 2, which re-emits the child's GUI elements so that the labels are readable at the top, with a child that reads a confined path and an unconfined one:

```
@two_files:
    $home <- @Element Fs.Contents(path="/home/me/a")
    $etc <- @Element Fs.Contents(path="/etc/b")
    @Label "home={Show $home} etc={Show $etc}"

test a_sandbox_rewrites_state_paths, (@Sandbox "/jail", @two_files):
    with Fs.Contents(path="/jail/home/me/a") = Known "A", with Fs.Contents(path="/etc/b") = Known "B", build
        => expect_top Fs.Contents(path="/jail/home/me/a")
        => expect_none Fs.Contents(path="/home/me/a")
        => expect_none Fs.Contents(path="/etc/b")
        => expect Ui.Label(text="home=A etc=-")
    push Fs.Contents(path="/jail/home/me/a") = Known "A2"
        => expect Ui.Label(text="home=A2 etc=-")
    push Fs.Contents(path="/etc/b") = Known "B2"
        => expect Ui.Label(text="home=A2 etc=-")
```

The supplied widget for the confined path made an `Fs.Contents` with the rewritten path in the sandbox's widget, and that is what the top sees. The supplied widget for the refused path is `@ $ Unavailable(Denied)`, with no element, and a push to the real `/etc/b` goes nowhere because nothing in the list asked for it.

**19. Two equal slots share one supplier.** Chapter 2's `@Cached`, which supplies `Fs.Contents` through a hold refreshed by a button, with a child that has two readers of one path and one of another:

```
@readers:
    $a <- @Element Fs.Contents(path="/cfg")
    $b <- @Element Fs.Contents(path="/cfg")
    $c <- @Element Fs.Contents(path="/other")
    @Label "{Show $a} {Show $b} {Show $c}"

test equal_readers_share_one_cache_entry, (@Cached @readers):
    with Fs.Contents(path="/cfg") = Known "v1", with Fs.Contents(path="/other") = Known "o1", build
        => expect_count Fs.Contents(path="/cfg"), 1
        => expect_count Fs.Contents(path="/other"), 1
        => expect Ui.Label(text="v1 v1 o1")
    push Fs.Contents(path="/cfg") = Known "v2"
        => expect Ui.Label(text="v1 v1 o1")
    click Ui.Button(label="Refresh")
        => expect Ui.Label(text="v2 v2 o1")
```

`@Cached` forwards every slot, so the child's own `Fs.Contents` slots are re-emitted by `@Forward` and would count too; the `expect_count` of `1` therefore says that `@Cached` here is the variant that forwards responses for request slots only, `@Forward (NonState $slots)`, with `NonState` filtering by the tag's declared kind. That variant is the one in the suite, because the count is the point. Both readers show `v1` after the push and `v2` after the refresh, in the same occurrence, because they observe one hold.

**20. The `Defer` bound.** A chain of five, and a fan of five, with an occurrence counter the program keeps itself:

```
@chain5:
    !started <- @started
    !r1 <- @Element Defer(requests=(unit on !started))
    !r2 <- @Element Defer(requests=(unit on !r1))
    !r3 <- @Element Defer(requests=(unit on !r2))
    !r4 <- @Element Defer(requests=(unit on !r3))
    !r5 <- @Element Defer(requests=(unit on !r4))
    $steps <- 0 then $steps + 1 on (!r1 | !r2 | !r3 | !r4 | !r5)
    @Label "steps={$steps}"

@fan5:
    !started <- @started
    !r1 <- @Element Defer(requests=(unit on !started))
    !r2 <- @Element Defer(requests=(unit on !started))
    !r3 <- @Element Defer(requests=(unit on !started))
    !r4 <- @Element Defer(requests=(unit on !started))
    !r5 <- @Element Defer(requests=(unit on !started))
    $steps <- 0 then $steps + 1 on (!r1 | !r2 | !r3 | !r4 | !r5)
    @Label "steps={$steps}"

test a_chain_of_five_is_six_occurrences, @chain5:
    build
        => expect Ui.Label(text="steps=5")

test a_fan_of_five_is_two_occurrences, @fan5:
    build
        => expect Ui.Label(text="steps=1")
```

`$steps + 1 on (...)` moves once per occurrence in which any `Defer` answered, so it counts occurrences after the first. Five in the chain, one in the fan. The transcript marks each occurrence inside turn 0, and the differential test checks that both implementations agree on how many there were, which is a check on the world core as much as on the runtime.

**21. Deferring backwards, as the bug it catches.** A child that retries at once when denied, under the sandbox:

```
@retrier:
    !started <- @started
    !resp <- @Element Fs.File(path="/etc/shadow", requests=(Read on (!started | !again)))
    let !again = !Filter (IsDenied !resp)
    @Label $"asked"

let IsDenied: fn FsResponse -> ?unit = \|r| match r:
    Denied: Some unit
    ignore: Nil

test an_immediate_retry_is_deferring_backwards, (@Sandbox "/jail", @retrier):
    build
        => errors Error(kind=DeferBackwards, ..)
```

Occurrence 0: the request; the sandbox fires its `Defer`. Occurrence 1: the denial; the child retries; the sandbox's `for` block for that slot fires `unit on _.!request` again, and the `Defer` slot it owns has already been answered this turn. The error names the `Defer`'s path and the interceptor it belongs to. This is a program that in a callback framework would spin forever, and here it is a reported error in turn 0 with no turn that ran long. A retry belongs behind a `Time.After`, where the World's clock

separates the attempts, and a second version of `@retrier` in the same file does that and passes.

## The layout corpus

Two lexers written by two people from one description of the offside rule will disagree on something, and the place they disagree will be a line nobody thought to write down: a statement continued onto a more-indented line, a dedent that closes two blocks at once, a colon inside braces. The corpus is those lines, written down, each with the token stream both lexers must produce. It is the first thing a frontend must pass, before parsing, because every later disagreement would otherwise be blamed on the parser.

A corpus entry is a source file and a token file. Tokens are printed one per line as `kind(text)` with layout tokens as `open`, `sep` and `close`; here they are shown on one line each with the layout tokens in angle brackets, which is how a failure message prints them.

**A statement continued by a more-indented line.**

```
@w:
    $count <- 0 then $count
        + !delta
```

```
  ident(@w) colon <open> ident($count) bind nat(0) hold ident($count) plus ident(!delta) <close>
```

The second line of the statement is indented more than the block's column, so it continues the statement and no `sep` is emitted.

**A dedent of two levels at once.**

```
@w:
    rebuild $x:
        @Label $"a"
@v:
    @ unit
```

```
  ident(@w) colon <open> kw(rebuild) ident($x) colon <open> ident(@Label) sigil($) text("a") <close> <close>
  <sep> ident(@v) colon <open> sigil(@) kw(unit) <close>
```

Two `close`s, then a `sep` at column 0 because the top level is itself a block, then `@v`. A lexer that emitted one `close` would leave the `rebuild` block open across a top-level definition, and the parser's error would point at the wrong line.

**An inline block and a layout block in one `match`.**

```
let f = \match _:
    True: 3
    False:
        4
```

```
  kw(let) ident(f) eq lambda kw(match) ident(_) colon <open> ident(True) colon <open> nat(3) <close> <sep>
  ident(False) colon <open> nat(4) <close> <close>
```

The inline form `True: 3` opens a block that is the rest of the line and closes at the end of it; the layout form `False:` opens a block on the next line. Both arms are statements of the `match` block and both are blocks, which is what makes an arm's body able to be either form.

**A colon inside brackets, and a trailing comment after the opening colon.**

```
@w:   # the colon above opens a block, this comment is not a token
    let m = {"a": 1,
             "b": 2}
    @ m
```

```
  ident(@w) colon <open> kw(let) ident(m) eq lbrace text("a") colon nat(1) comma text("b") colon nat(2) rbrace
  <sep> sigil(@) ident(m) <close>
```

Layout is suspended inside the braces: the line break and the indentation before `"b"` are whitespace, and the colons are key separators, not block openers. The comment after `@w:` is whitespace too, so the colon is still the last token on its line and opens the block.

**Interpolation, nested braces and a literal `$`.** A text literal is one token whose parts are literal text and `interp` groups; each group holds the tokens of its expression, lexed with layout suspended up to the matching `}`.

```
@w:
    @Label "n={Size {"a": 1}}, \{ok\}"
    let p = "cost: $5, {$n} left"
```

```
  ident(@w) colon <open> ident(@Label) text("n=", interp(ident(Size) lbrace text("a") colon nat(1) rbrace), ",
  {ok}") <sep> kw(let) ident(p) eq text("cost: $5, ", interp(ident($n)), " left") <close>
```

The braces inside the interpolation nest, so the first `}` after `1` closes the map and the second closes the interpolation; the colon inside them is a key separator, not a block opener; the nested `"a"` is a text token inside the group. `\{` and `\}` are literal braces and join the surrounding literal part, which is why the last part is `", {ok}"`. The `$` in `$5` is text, and the `$` before `n` belongs to the name inside the group. A lexer that treated `$` in text as the start of an interpolation would split the first literal of the second line at the dollar sign.

**`F -1` against `F - 1`, and `\F .x` against `a.x`.**

```
let a = F -1
let b = F - 1
let c = \F .x
let d = a.x
```

```
  kw(let) ident(a) eq ident(F) int(-1) <sep>
  kw(let) ident(b) eq ident(F) minus nat(1) <sep>
  kw(let) ident(c) eq lambda ident(F) dot ident(x) <sep>
  kw(let) ident(d) eq ident(a) field(x)
```

The spacing rule decides at the lexer: a `-` preceded by whitespace and followed immediately by a non-space is a prefix sign and the literal is signed; `-` with space on both sides is binary. A `.` in prefix position is printed as `dot` then the name, and a `.` attached to a name is a `field` token. The lexer is where this lives because it needs the whitespace. The parser then gives a prefix `.x` its meaning: wherever `_` is in scope it is the pronoun's field, `_.x`, so `c` is `\F _.x`; at the start of a continuation line it is a method-chain step. A sibling entry pins that `F .x` is a call of `F` on `_.x` when `_` is in scope, and an unbound-pronoun error, with that error's span, when it is not.

**`|` as merge against `|a, b|` as lambda bars.**

```
let m = !a | !b
let f = \|a, b| a | b
```

```
  kw(let) ident(m) eq ident(!a) merge ident(!b) <sep>
  kw(let) ident(f) eq lambda lbar ident(a) comma ident(b) rbar ident(a) merge ident(b)
```

A `|` immediately after `\` opens an argument list and the next `|` closes it; every other `|` is merge. The

second `|` inside the lambda body is merge even though an argument list was just closed.

**A sigil before an open bracket.**

```
let s = $ (x + 1)
let t = $(x + 1)
```

```
  kw(let) ident(s) eq sigil($) lparen ident(x) plus nat(1) rparen <sep>
  kw(let) ident(t) eq sigil($) lparen ident(x) plus nat(1) rparen
```

Both are the free sigil: a sigil is attached only when the next character is a name character, an underscore, a quote or another sigil, and `(` is none of those. The two lines produce the same tokens, which is the entry's point.

**A guard on a `match` arm.**

```
let f = \match _:
    n if n > 3: "big"
    ignore: "small"
```

```
  kw(let) ident(f) eq lambda kw(match) ident(_) colon <open> ident(n) kw(if) ident(n) op(>) nat(3) colon
  <open> text("big") <close> <sep> ident(ignore) colon <open> text("small") <close> <close>
```

The `if` in an arm's head is a keyword token and opens nothing; the one colon after the guard is the arm's, and its block is the rest of the line. A lexer that opened a block at `if` would have two `open`s before `"big"`. Chapter 12's grammar for arms is what reads `pattern if cond:` out of this stream.

**A `for` with named binders.**

```
@w:
    !done <- for k, $item <- $items:
        @ unit
```

```
  ident(@w) colon <open> ident(!done) bind kw(for) ident(k) comma ident($item) bind ident($items) colon <open>
  sigil(@) kw(unit) <close> <close>
```

Both `<-` are `bind` tokens, the comma is a `comma`, and only the final colon opens a block. The entry pins that the binder list does not end the statement and that the second `bind` is not read as starting a new one.

**An array pattern.**

```
let g = \match _:
    [[head, ..rest]]: head
    []: 0
```

```
  kw(let) ident(g) eq lambda kw(match) ident(_) colon <open> arrayopen ident(head) comma dotdot ident(rest)
  arrayclose colon <open> ident(head) <close> <sep> arrayopen arrayclose colon <open> nat(0) <close> <close>
```

`[[` and `]]` are one token each, as in the array literal, and `[[]]` is an `arrayopen` and an `arrayclose` with nothing between. `..rest` is `dotdot` and a name, not a field access and not two `dot`s. A sibling entry pins that `a[b[0]]` ends in two `rbracket`s: its two opens do not touch, so by chapter 3's pairing rule its two closes are two closes, however they are spaced.

The corpus has about forty entries, the eight kinds above and their variants: a block opened mid-statement by `for $x:` and closed by the next statement; an `else` at the column of its `if`; a `match` whose arms are all inline; a text literal with an interpolated `{expr}` containing a colon, and one with an interpolation that holds another text literal; a line with only a comment inside a block, which neither opens nor closes nor continues anything; a tab in leading whitespace, which is an error with a span. The corpus is also the first differential test: the Rust and Haskell lexers are run over every

entry and the token files diffed, before either implementation has a parser.

## Typed-AST goldens and negative goldens

The frontends are tested by what they decide. Elaboration's output is the typed AST, a tree in which every meaning has been fixed: every lift is explicit, every literal and operator has its numeric type, every binding is marked build-time or occurrence-time and placed in its group, every recursive group has its placeholder and close, every element construction names its tag. The typed AST has a text form, printed and parsed from one definition, and a golden test is a Prism file and its dump. When elaboration changes, the diff is something a person reads, and the question "did this change meaning" is answered by whether a golden changed.

### The text form

One binding per line: the name with its sigil, its type, its mark (`[build]` or `[occ]`), its group, and the expression with every lift explicit, followed by the span. The format is chapter 12's, spelled the same here because the goldens are the same files. Here is the counter, with the two library widgets it uses, which is the first golden and the one every reader of the compiler should be able to write by hand:

```
@Button: fn $Text -> @!Click = \|$label| @widget:
    !!clicks <- rebuild $label:
        @Element Ui.Button(label=_)
    @ !Switch !!clicks

@Label: fn $Text -> @unit = \|$text| @widget:
    rebuild $text:
        @Element Ui.Label(text=_)
    @ unit

@counter:
    !increment <- @Button "Increment"
    !decrement <- @Button "Decrement"
    let !delta = -1 on !decrement | +1 on !increment
    $count <- 0 then $count + !delta
    @Label "Count: {$count}"
```

```
  @Button: fn $Text -> @!Click = \|$label| @widget:                                  @1:1
    !!clicks: !!Click [build] g1 = rebuild $label \|label| @widget:                      @2:5
      _: !Click [build] g1 = element Ui.Button{label=label} responds                   @3:9
      @ _
    @ switch !!clicks

  @Label: fn $Text -> @unit = \|$text| @widget:                                      @6:1
    _: unit [build] g1 = rebuild $text \|text| @widget:                                  @7:5
      _: unit [build] g1 = element Ui.Label{text=text}                                 @8:9
      @ unit
    @ unit

  @counter: @unit                                                                    @11:1
    !increment: !Click [build] g1 = bind @Button(const "Increment")                  @12:5
    !decrement: !Click [build] g2 = bind @Button(const "Decrement")                  @13:5
    !delta: !Int [occ] g3 = leftmost [map !decrement \|ignore| -1:Int, map !increment \|ignore| +1:Int]   @14:9
    $count: $Int [build] g4 = placeholder                                            @15:5
    $count.hold: $Int [build] g4 = hold 0:Int (tag !delta [$count] \|d, c| Num.Add[Int] c, d)   @15:5
    close $count $count.hold                                                         @15:5
```

```
_: unit [build] g5 = bind @Label(dmap [$count] \|c| cat ["Count: ", ToText[Int] c])   @16:5
@ _
```

Everything the compiler chose is on the page. `-1 on !decrement` became a `map` over the event yielding an `Int` literal, because `on` returns its left operand and lifting did the rest. `|` became `leftmost`. `$count + !delta` became a `tag`, which is the one window onto the value before the occurrence, applied with `Num.Add[Int]`, because `!delta` is `Int` and inference flowed from the literal `-1` through the merge into the hold. `$count` is a cycle through that tag, so its group has a placeholder, the hold under the name `$count.hold`, and a close. The two binds, the hold and the label bind are `[build]`; the merge is `[occ]`, and so are the closures inside `map`, `tag` and `dmap`, whose marks are implied by their position. The label's interpolation is a `cat` of parts over a `dmap` of `$count`, with `ToText[Int]` rendering the number. `@Button` and `@Label` are library widgets whose single `rebuild` is the programmer's; the `element` lines are the only place a tag appears. Nothing below this line decides anything about meaning, which is the sentence the golden pins.

The spellings, `leftmost`, `tag`, `hold`, `map`, `dmap`, `const`, `bind`, `element`, `placeholder`, `close`, `cat`, `Num.Add[Int]`, `ToText[Int]` and literals as `0:Int`, are the printer's and are frozen by the goldens; the Haskell frontend must print the same characters for the same source. Elements print as `element Ui.Button{label=label}` followed by `responds` when the tag responds, `rebuild` as `rebuild $x \|x| @widget:` with the block's bindings indented beneath it, `for` with its kind and whether the block can fire at birth, and `intercept` as `intercept child responses supplier`. A closure after conversion prints its capture list, `\[$total]|d| ...`, and that form is the one the lint checks.

## What the goldens are

The counter. The GUI library, whose dump shows the `rebuild` under a dynamic field, the `for` in `@Forward` consolidating to an event, and the `intercept` in `@VBox`. Every integration program. And one small file per rule of the chapters on the lexical structure and on expressions: the comma rule, including `intercept @_, !Union !kid_ins, !fwd_ins` with its golden showing it parsing as `intercept(@_, Union(!kid_ins), !fwd_ins)`, which is the reason the library writes the parentheses; the precedence table, one file per adjacent pair of levels; the spacing rule; a lambda with a sigiled underscore taking that kind; an unannotated `let` lambda lifted at two call sites with different kinds; a `match` on a dynamic becoming a dynamic of the arms; `for` as an expression over a plain map; text interpolation of each primitive; a hold whose literal type is pinned only by an operand three lines away; and `not a == b`, which parses as `not (a == b)` because comparison binds tighter than `not`, so the dump is a `Bool.Not` around an `Eq` and a reader who expected `(not a) == b` sees at once that no such tree exists:

```
let same: fn Int, Int -> Bool = \|a, b| not a == b
```

```
  same: fn Int, Int -> Bool [build] g1 = \|a, b| Bool.Not (Eq[Int] a, b)              @1:1
```

The guard on a `match` arm has a golden too, as a node of the dump: the guard is a field of the arm.

What a golden catches, in the order these have bitten: a lift that changed kind, say a dynamic read promptly where it should be a tag, which is a glitch that no script may notice for months; a literal or operator that resolved to a different numeric type; a binding that moved between build-time and occurrence-time, which changes how often an expensive function runs; a recursive group split or placed differently, which changes build order; a `for` consolidating to the wrong kind.

## Negative goldens

A rejected program and its exact error, spans included; a lint warning has a golden of the same kind, with the program accepted. These exist because an error can get worse silently: the program is still rejected, the test that checks "rejected" still passes, and the message that used to name both events now says "type error". Each negative golden is the message, so a worse message is a diff.

```
error: two events in one operation
  --> two_events.prism:3:16
   |
 3 |     let !sum = !a + !b
   |                ^^   ^^
   |
   = both `!a` and `!b` fire; an operation lifts over at most one event
   = help: merge them with `!a | !b` or `!Leftmost [[!a, !b]]`, or hold one and read it through a tag

error: cycle without a delay
  --> prompt_cycle.prism:2:5
   |
 2 |     let $a = $b + 1
   |         ^^ `$a` depends on `$b`
 3 |     let $b = $a * 2
   |         ^^ `$b` depends on `$a`
   |
   = every edge in this cycle is prompt; a cycle must pass through a tag, a `!Switch`, or a hold's value

error: cycle without a delay
  --> dyn_cycle.prism:2:5
   |
 2 |     !!r <- rebuild $x:
   |     ^^^ `!!r` depends on `$x`
 4 |     $x <- 0 then !Switch !!r
   |     ^^ `$x` depends on `!!r`
   |
   = `rebuild` is not a delay: its event fires in the occurrence of the rebuild
```

The third is the sentence "rebuild is not a delay" as a negative golden; the cycle is legal with `!Switch` between `!!r` and the hold, and a sibling positive golden has exactly that program with the switch.

```
error: `+~` is `Int` addition
  --> suffix.prism:3:23
   |
 3 |     let $mean = $sum +~ 0.5
   |                       ^^ both operands are `Float`
   |
   = help: use `+.` or `+`
```

There is deliberately no golden for an integer literal nothing pins: `$n <- 0 then $n + 1 on !tick` is an `Int` counter, because an unconstrained integer-spelled literal defaults to `Int`, and a positive golden pins that.

```
error: `a -1` is a call
  --> spacing.prism:2:13
   |
 2 |     let b = a -1
   |             ^ `a` is a `Nat`, not a function
   |
   = help: did you mean `a - 1`?

error: a tag's field has a concrete type
  --> generic_box.prism:1:40
   |
 1 | let Wrap: fn a -> Element = \|x| Test.Box(value=x)
```

```
  |                                              ^ `value` is a `Nat`; `x` is `a`

error: `match` on an extensible variant needs a catch-all
  --> no_catch_all.prism:1:30
  |
1 | let IsButton = \|e| match e:
  |                      ^^^^^
2 |     Ui.Button(..): True
  |
  = `Element` is open; add an `ignore:` arm

error: sigil disagrees with the kind
  --> wrong_sigil.prism:2:5
  |
2 |     let $sum = $a + !b
  |         ^^^^   ^^^^^^^ this is an event
  |
  = help: name it `!sum`

warning: a lowercase name is snake_case
  --> naming.prism:1:5
  |
1 | let myValue = 3
  |     ^^^^^^^ help: rename to `my_value`
```

This one is a warning, so the golden's expected output also records that the program compiled: the lint pass reports camelCase and the build goes on. The sibling positive golden is the same file with the name fixed and no diagnostics at all.

```
error: `update` is reserved; see the chapter on references
  --> future.prism:3:5
  |
3 |     update $items:
  |     ^^^^^^
  |
  = `read`, `update` and `&` are not part of this version of the language
```

Runtime errors have negative goldens too, through the `errors` step, and their text form is the error value as the transcript prints it: `Error(kind=UnknownPath, path=0/#1/0, turn=1)`, `Error(kind=DeferBackwards, path=1/[3]/#0/1, turn=0, occurrence=2)`, `Error(kind=Cycle, spans=[[...]])` with the spans of every binding in the cycle in order, `Error(kind=UnsuppliedState, element=Secret, path=0)`. The compiler names every cyclic slot after its source span so that a runtime cycle through two widget functions, which no static check can see, reports Prism source and not runtime internals; the golden for that case is a program with two widget functions whose results feed each other's arguments promptly, and its expected error lists both.

## Runtime unit tests and the invariant

Under the conformance suite sits the Rust runtime's own test suite, written in Rust, against the graph directly, with no Prism in it. It exists because a combinator that drifts from its law should fail before any Prism program can reach it, and because some laws, like a closure running at most once per occurrence, are not observable from a World at all and have to be checked from inside.

Each test is one sentence about the runtime. The ones that pin the laws of the chapters on the functors and on time, with the names they have or will have:

- `leftmost_takes_the_left_when_both_fire`
- `a_tag_reads_the_value_before_the_occurrence`

- `a_hold_adopts_a_firing_in_its_build_occurrence`
- `a_derived_dynamic_changes_once_however_many_routes_lead_to_it`
- `switch_hold_hears_a_new_inner_from_the_next_occurrence`
- `a_child_under_dyn_and_switch_is_not_heard_in_its_build_occurrence`
- `dyn_fires_in_its_first_build_and_in_every_rebuild`
- `a_nested_dyn_built_in_the_occurrence_its_dynamic_changes_shows_the_new_value`
- `nested_dyn_rebuilds_in_the_same_occurrence_as_its_parent`
- `a_widget_built_mid_occurrence_sees_the_same_events_at_any_depth`
- `a_widget_built_mid_occurrence_gets_the_mapped_value_of_an_event_that_already_fired`
- `a_closure_runs_once_per_occurrence_even_when_late_subscribers_read_it`
- `a_replaced_child_keeps_working_until_its_last_occurrence_ends`
- `dropping_a_widget_frees_its_cyclic_loops`
- `joined_dynamic_takes_the_new_inner_value_when_it_changes_in_the_switch_occurrence`
- `joined_dynamic_ignores_the_old_inner_changing_in_the_switch_occurrence`
- `joined_dynamic_fires_once_when_outer_and_inner_change_together`
- `a_same_occurrence_cycle_through_a_cyclic_event_is_reported_instead_of_hanging`
- `a_dynamic_defined_as_a_function_of_itself_is_reported_instead_of_hanging`
- `a_cycle_through_intercept_is_reported`
- `a_failed_occurrence_leaves_the_graph_as_it_was`
- `a_runtime_moved_to_another_thread_keeps_working`
- `one_call_is_one_occurrence`
- `a_slot_removed_in_n_still_receives_input_in_n_and_is_gone_in_n_plus_1`
- `input_to_a_path_not_in_the_opened_list_is_an_error`
- `a_request_fired_by_a_slot_torn_down_in_the_same_occurrence_is_reported_with_its_path`
- `a_keyed_list_keeps_a_childs_state_across_an_insert_and_a_removal`
- `an_insert_at_the_front_of_an_array_list_rebuilds_every_later_child`
- `a_state_slot_reads_the_oracle_then_the_push_in_its_creation_occurrence`
- `equal_state_elements_share_one_supplied_widget_until_the_last_slot_goes`
- `a_supplied_widgets_elements_join_the_interceptors_list`

The join tests are the `Dynamic::join` of the chapter on the Rust runtime, and the last seven are the boundary; they are written against the run call with a hand-built widget, which is the harness the Test World replaces for Prism programs.

## The invariant, checked after every occurrence

Every runtime test runs with an invariant check after every occurrence, failed or not, and the check is the list of things the transaction promises:

- Every event's fired mark is clear, and every event's cached current-occurrence value is gone. A mark left set would make the next occurrence see a firing that did not happen.
- No hold holds a value that was committed before `usize::MAX`: during the occurrence, every hold's visible value is its value before the occurrence, and after commit, its value is the last value its event fired with. The check keeps a shadow copy per hold and compares.
- No node fired twice in the occurrence; the check counts firings per node and reports the first node that reached two, with its height and the heights of its sources.
- The action map is empty and the current height is reset. A leftover action would run at the start of the next occurrence as if it were part of it.
- The routing table the occurrence used equals the element list as the occurrence opened, and every input delivered went to a path in that table. This is the routing law stated as a check, and it

catches both routing against the updated list mid-occurrence and tearing a slot down too early.
- Every node built during the occurrence is owned by exactly one widget, and every node torn down during it is owned by none and has no strong references into it from any subscriber list. The second half is what makes "no collector" a fact rather than a hope; the check walks the tree and the subscriber lists.
- After a failed occurrence: the element list, every hold's value, every switch's current inner and every `rebuild`'s current generation are what they were before the occurrence opened. The test that injects a panic into a closure in the middle of propagation and then checks this is the rollback test, and it runs the same check as every other test; nothing special is needed because the invariant already says what "as it was" means.

The invariant is a function in the runtime crate, compiled in tests only, and it is the one place where the runtime's internals are tested as internals. That is allowed because the invariant's items are each a sentence of the chapter on time; the quotable-test rule, below, is satisfied by the list rather than by the function's name.

## World tests against the world core

A World is the world core plus its components, and the core's obligations are the same for every World, so they are tested once, in the host language, with a fake component. The fake component accepts a handful of tags, records every call the core makes to it, performs requests by appending them to a log, answers the oracle from a map the test sets, and produces responses and pushes when the test tells it to. The runtime under it is real; the program is a small hand-built widget, or the compiled counter once the closure compiler exists.

What the core tests check, each a sentence from the chapter on Worlds:

- *The turn loop.* One input, one occurrence, one outcome returned; the test counts run calls and they match the inputs delivered exactly.
- *The `Defer` bound.* A program with `d Defer` slots in a chain runs `d + 1` occurrences in one turn and the core returns after the last; a fan of `d` runs two; a random program with `d Defer`s never runs more than `d + 1`.
- *Deferring backwards.* A `Defer` answered this turn that fires again is reported as `DeferBackwards` with the slot's path and the occurrence, and the core does not open another occurrence for it.
- *Snapshot and push consistency.* The test changes the fake's state between turns and during a turn; for every occurrence, the oracle's answers equal the previous snapshot and the pushes equal the difference to the current one; the core never answers an oracle from the snapshot it is pushing. A slot created in an occurrence with a push to its value ends that occurrence holding the pushed value.
- *Discarding late responses.* The fake produces a response for a slot that left the list in the previous turn; the core drops it, logs it to diagnostics, and asks the fake to cancel; nothing reaches the runtime, and the runtime's unknown-path error is not raised because the core caught it first.
- *Requests from torn-down slots.* A slot that fires a request and is torn down in the same occurrence has its request reported with its path; the core performs it through the component and discards any response.
- *`Unsupported`.* A request on a tag no component accepts gets an `Unsupported` response in the next turn, not the same occurrence; a presence tag nobody accepts is reported once; a state tag nobody accepts is a build error in the occurrence the slot is created, with the slot's path.
- *Effects after the turn.* The fake's log shows requests performed after the run call returned, in

occurrence order and, within an occurrence, in list order, never during propagation.
- *Error reporting.* An aborted occurrence comes out of the core as an error with the kind, the spans, the turn and the occurrence, in the one format every World prints.

The Test World is a World, so it runs under these tests too, with the script component standing in for the fake. The CLI World's tests are separate, below, because its components touch the operating system and need a process around them.

## CLI golden transcripts

The CLI World is the first World that runs Prism against a real operating system, and its tests are the only ones in the suite that run a real binary. Each test is a program, its arguments and environment, its standard input (as a list of chunks, because chunking is the thing to pin), and the expected standard output, standard error and exit code:

```
program: wc.prism
args: []
env: {}
stdin: ["one\ntwo\nthr", "ee\n"]
stdout: "3\n"
stderr: ""
exit: 0
```

The harness runs the binary with the given stdin delivered chunk by chunk with a pause between chunks, so that each chunk is its own turn, and compares. Timers are not in these transcripts: a program with a `Time.After` is tested under the Test World with a `push Clock`, so that no golden depends on wall-clock time.

The transcripts, each with what it pins:

- *Line counting*, chapter 2's `@wc`, on empty input, one line, many lines, and lines split across chunks. Pins one turn per chunk, the partial line held to the next chunk, output written after the turn, and exit by `Process.Exit`'s request.
- *Falling off the end*: a program that prints and never fires `Process.Exit`. Pins exit 0 when nothing can cause another occurrence: no stdin slot, no signal slot, no request in flight, no timer; args, env, clock and watch slots do not keep a run alive.
- *Exit codes*: a computed nonzero code; two `Process.Exit` slots firing in one turn, where the first in occurrence-then-list order is the code the process exits with; a code outside 0–255, which is an error.
- *Output order*: two `Cli.Stdout` slots firing in one occurrence, written in list order; stdout and stderr interleaved across turns, in turn order; writes and an exit in one turn, with the writes landing first.
- *Args and env*: a program that prints `Process.Args` and `Process.Env` from their state elements, with no arguments, several arguments, and an empty environment. Pins that both have their value in occurrence 0 from the oracle and that nothing is requested to get them.
- *File first, then stdin*: a program that reads a file from `@started` and builds its `Cli.Stdin` slot inside a `rebuild` after the response. Pins that stdin is not read until the slot exists, which also keeps the transcript free of a race.
- *Unsupported*: a `Ui.Button` at the root. Pins the World's error on stderr naming the interceptors the element came through, the `Unsupported` response reaching the program's slot in the next turn, and the program continuing.
- *Signals*: with and without a `Cli.Signals` slot, by sending the process a signal from the harness.

Pins OS defaults without a slot and the signal arriving as a response with one.
- *A closed pipe*: stdout closed by the harness before a write. Pins that the write's failure comes back as a response rather than killing the process.

The CLI World's transcripts are also the only tests in which an implementation's behaviour against time is checked at all, and only by order: a chunk's turn completes before the next chunk is delivered. Anything finer is the World's business and not tested.

## The differential test

Everything above runs under both implementations, and the differential test is `diff`. Two commands produce the two interchange formats: `prism dump foo.prism` prints the typed AST in the text form, and `prism test foo.prism --transcript` runs every test in the file under the Test World and prints the transcript of each, turn by turn. The differential harness runs both commands under both implementations over every file in the suite, the layout corpus's token files included, and reports every byte that differs.

An empty diff is the claim that both implementations read the specification the same way, for every program in the suite, at the level of what a World sees and at the level of what the compiler chose. It is not a claim that either implementation is right; the conformance scripts make that claim, and the differential test adds the second reader.

A divergence means one of two things, and telling them apart is the whole value of the exercise. Either one implementation has a bug, which the law test that pins the sentence will usually have caught already and the diff localises to a construct; or two careful readers read a sentence of the specification differently, and the sentence is ambiguous. The second kind is the more valuable find, and the procedure for it is fixed: the sentence gets rewritten so that it admits one reading, a law test is added that pins that reading, and both implementations are brought to it. A divergence is never resolved by making one implementation copy the other's behaviour without a sentence to point at, because that turns the second reading back into the first and the specification into a description of the Rust runtime.

Three kinds of divergence have been anticipated and have a place to land. A path printed differently is a reading of "its position in the widget tree" and is fixed in the transcript format's definition, not in either runtime. A typed-AST dump that differs in a lift or a numeric type is a reading of the lifting rule or the inference rule, and the goldens, which both frontends must match, decide it before the transcripts can. A transcript that differs in which occurrence of a turn a request fired in is a reading of the `Defer` rule or the creation rule, and the world core tests decide it. Anything else is new, and new is what the test is for.

The differential test runs on every change to either implementation and on every change to the specification, since a change to a sentence is a change to what the goldens and transcripts should say. When a sentence changes and the diff stays empty, both implementations have the same bug, and the law test is the only thing that can find it; that is why there are both.

## The quotable-test rule

Every test in every layer obeys one rule: it is quotable. Its name is a sentence from the specification, or a sentence that could be added to it, and its body is short enough to read whole in the failure message. `a_push_reaches_a_slot_born_in_its_occurrence`, `leftmost_takes_the_left`,

`an_immediate_retry_is_deferring_backwards`: each is a claim about the language that a reader can check against the text without knowing anything about the implementation that failed it.

The rule has teeth in both directions. A test that cannot be stated as a sentence about the language is testing the implementation's structure, and those tests are deleted when the structure changes, which is the only honest thing to do with a test whose subject has gone. A test that the runtime is `send + Sync` would be a fact about locks the design does not want; the test the suite has is that a runtime can be moved to another thread, which is the sentence the design does want. And a sentence in the specification with no test is a sentence that two implementations may read differently without anyone noticing, so when a reader of the specification finds a sentence they cannot write a test for, the sentence is suspect: either it is not observable, in which case it is not a rule and should say so, or it is observable and the suite is missing a script.

You have read a test suite that was a pile of regression cases named after bug numbers, and you have had to work out, from the body, what each one was for. The rule above is the refusal to build that pile. A conformance suite is the specification, said a second time in a form a machine can check, and every one of its tests should read like a line of the first saying.

## Chapter 11

# The Rust runtime

The Rust runtime turns a built graph into occurrences. It holds events, behaviors and dynamics as nodes with heights; it runs one occurrence per call by popping heights in order; it commits every hold at the end; and it owns the tree of nodes that is the whole of a program's state. The whole of it is a height-ordered action map, a commit list, weak subscriber lists and one synchronous call; everything else here is either the code that exists or the consequences of that boundary.

## What the runtime does not need

You have written one of these, or read one, and you know the list. There is a scheduler, with a queue and a policy about what runs next and a worry about starvation. There is a dirty-tracking protocol, because derived values have to find out that their inputs moved. There are re-entrancy guards, because a setter notifies a listener that calls a setter, and sooner or later one of those calls arrives while the first is half done. There is a way to defer effects until the batch is finished, because an effect run in the middle of a batch sees a mixture. There is an unsubscribe story, with its leaks and its double frees. And at the end of the list there is either a tracing collector or a long conversation about weak references, because listeners point at the things they listen to and the things point back. Almost all of this is absent from the Prism runtime, and every absence is bought by a refusal the language already made. The refusals are in the goals as the language's negative space; this section is the same negative space seen from the implementation: each item is a mechanism that does not have to be written, tested, or debugged at three in the morning.

**No scheduler.** A scheduler decides what runs next when several things could. Here nothing could. There is no sampling, so a node cannot ask for a value before its source has fired or declined to, and there is no prompt switching, so a node's set of sources is fixed for the whole of an occurrence. Together these mean every node's position in the order is a function of its prompt sources and nothing else: a height, one more than the greatest height among the events it is computed from. Propagation is "pop the lowest height, run everything at it, repeat." There is no priority, no fairness and no policy. Two programs that differ only in the order their bindings were written propagate identically.

**No dirty tracking.** A dirty-tracking protocol watches writes. There are no writes. A dynamic changes only when its change event fires, and the change event is a node in the same graph as everything else, at a height, run by the same loop. A derived dynamic is a function of its inputs' values and a leftmost of their change events, so "has my input moved" is answered by "did its change event fire in this occurrence," which the graph already knows. The pull side of a dynamic, its behavior, does keep a cache and does invalidate it, but that invalidation is driven by the commit of a hold at the end of the occurrence and is the only kind there is.

**No re-entrancy guards, except one.** Nothing fires an event from nothing and nothing samples, so the only way propagation can arrive back at a node it is already inside is a cycle in the prompt graph. A cycle shows up as a node whose height is being computed in terms of itself, or as an action scheduled at or below the height currently running. Both are one comparison, and both are errors.

The current code makes the comparison with `try_lock` on a mutex the single propagating thread already holds, which reads as a concurrency primitive but is a re-entry flag; the target makes it a `Cell<bool>` per node with the same effect and a message that names the loop.

**No deferred effects.** There are no effects to defer. The only outputs are request firings, which are facts about the occurrence collected into the run call's outcome, and the element list, which is a value the World reads after the call returns. The World acts after the occurrence has committed, in the order the outcome gives. There is no "after this batch" hook because no closure inside propagation can touch anything outside the graph.

**No unsubscribe.** A subscription is a `Weak` pointer in the source's subscriber list. The subscriber is owned by the node that built it, the node by its parent in the widget tree, and when the parent drops the node the subscriber's strong count reaches zero and the weak pointer stops upgrading. The source finds the dead entries the next time it consolidates its list. Teardown is `drop`.

**No collector.** Ownership is a tree because nodes are not values: a program cannot hold a reference to a node, so nothing a program does can make two nodes own each other. The one real threat is a closure. An occurrence-time closure compiled from `$count + !delta` has to reach `$count`'s behavior, and `$count`'s hold has to reach the event that updates it, which is the closure's node; if both links were strong, that is a cycle through the heap that reference counting never frees. The rule that prevents it is that a node owns its closures and its placeholders, and every loop is closed through a placeholder that holds only a `Weak` to its target. The strong ends live in the node; the loop points back at them weakly; dropping the node frees the loop. Closure conversion in the compiler makes this mechanical, because a compiled closure captures by index into its own node's values and never by pointer into a sibling's. Cyclic bindings inside one block are the explicit exception and are owned by the node as a unit, which is why the node has slots for them.

**No locks.** The runtime is single-threaded by design. The current code is written over `Arc` and `Mutex`, and every lock in it is either taken by the one thread that is propagating or is there to detect re-entry, which the height comparison already does. There is no other thread that could contend. The target replaces `Arc` with `Rc`, `Mutex` with `RefCell` and `Cell`, keeps `Runtime: Send` so that a World can move it to a thread of its choosing, and gives up `Sync`, which nothing uses.

**No transaction log, no phases, no runtime-started occurrences.** The occurrence is a transaction, but the undo log is not a separate structure: every write a program can observe is already buffered to commit time for the tag law's sake, so rollback is "drop the buffer." There are no phases inside an occurrence; `Defer` is a World feature built on the run call, and the runtime has no branch for it. And the runtime never opens an occurrence of its own; a rebuild finishing, a slot appearing, a request firing cause nothing until the World calls again. These three absences are what make the run call one synchronous function from inputs to an outcome.

The remainder is small. A graph of nodes with heights; one per-occurrence map from height to a list of actions; a commit list of buffered writes; weak subscriber lists; a tree of nodes holding the strong ends of its loops; and one entry point. That is the whole runtime. It is small enough that writing it twice, once in Rust and once as a mapping onto Reflex, is a reasonable plan rather than a heroic one, and small enough that the rest of this chapter can go through all of it.

## The crate as it stands today

`prism_runtime` is one crate in the workspace, branch `on-fire-repros`, tip `c03a6fd`, edition 2024, with three dependencies (`log`, `serde_json`, `thiserror`) and one dev-dependency (`test-log`). `cargo test`

`-p prism_runtime` runs 37 library tests and one integration test, 38 in all, passing. The crate has the graph and the transaction and does not yet have the boundary: no elements, slots, paths or interceptors, and what it exposes to the outside is a test backend that reaches into nodes by name. This section is the crate as a reference, in the order the modules depend on each other.

```
src/lib.rs
src/value/        Value<T>, AnyValue, PrismValueInner, HeapValue, TypeMismatch
src/runtime/      Runtime, Action, schedule, propagate, the occurrence's state
src/event/        Event<T>, EventImpl, EventCallback, SubscriptionManager,
                  external, filter_map, tag, combine, leftmost, switch_hold,
                  forward, never, trace
src/behavior/     Behavior<T>, BehaviorImpl, hold, compute (map, map2, join), constant
src/dynamic/      Dynamic<T>
src/widget/       Blueprint, DynBlueprint, WidgetBuilder, Widget, Slots,
                  WidgetReadyEvent, Rebuild, fold_dyn, AnyEvent/AnyDynamic,
                  WidgetDelegate
src/backends/     test_backend
tests/            runtime_send.rs
```

## Values

Everything that flows through the graph is a `Value<T>`, a typed view onto an untyped `AnyValue`:

```
pub trait ValueType: PrismValueInner + Clone + Debug + Sync + Send + 'static {}

#[derive(Clone)]
pub struct Value<T: ValueType> { value: AnyValue, phantom: PhantomData<T> }

#[derive(Clone)]
pub struct AnyValue(AnyValueInner);

#[derive(Clone)]
enum AnyValueInner {
    Arc(Arc<dyn HeapValue>),
    U32(u32),
    I32(i32),
    Unit,
}

pub type PrismArc<T> = Value<Arc<T>>;
```

`Value<T>` derefs to `T::Ref` (the `T` itself for immediates, the pointee for `Arc<T>`); `Value::get(self) -> T` unwraps; `From<T>` wraps; `PrismArc::new(t)` and `PrismArc::extract(self) -> T` are the heap conveniences, the second being `Arc::unwrap_or_clone`. `AnyValue::downcast::<T>()` panics on a mismatch; `try_downcast` returns `TypeMismatch { expected, actual }` with two type strings and exists for the test backend. The module doc states the position that matters: Prism is statically typed, the runtime trusts that, and the checked downcast is a debugging aid a release build may one day skip. There is no introspection and no fallible path meant for programs.

`PrismValueInner` says how a Rust type lives inside `AnyValue`:

```
pub trait PrismValueInner: Debug + Clone + Sync + Send + 'static {
    type Ref: ?Sized;
    fn get_any(value: &AnyValue) -> &Self::Ref;         // panics on mismatch
    fn try_get_any(value: &AnyValue) -> Result<&Self::Ref, TypeMismatch>;
    fn from_any(value: AnyValue) -> Self;
    fn to_any(self) -> AnyValue;
    fn type_string() -> String;
```

```
}
```

Implementations exist for `()`, `u32`, `i32`, `AnyValue` itself (so an erased value is a value), `Arc<T>` for any `T: HeapValue` (`HeapValue` is `Any + Sync + Send + Debug + 'static`, blanket-implemented, with a `type_string` for diagnostics), and three runtime objects as heap values: `Event<T>`, `Dynamic<T>` and `AnyEvent`. The last three make an event of events or a dynamic of blueprints expressible as ordinary values, which `switch_hold` and `rebuild` need. A `Value<Event<T>>` is an `Arc<Event<T>>` underneath and `Event<T>` is itself an `Arc<dyn EventImpl<T>>`, so there is a double indirection there that the one-word representation of the values chapter removes. The enum is two words plus a tag (the crate has no `size_of` assertion pinning it), and the representation is deliberately behind the trait so that the rest of the runtime does not notice when it changes.

## The runtime core

This is the whole engine: a map from height to the actions scheduled at it, and a loop that pops the lowest height, runs everything at it, and repeats. Nothing else decides what runs next.

```
pub struct Runtime(pub(crate) Mutex<RuntimeInner>);

pub(crate) struct RuntimeInner {
    actions: BTreeMap<usize, Vec<Box<dyn Action>>>,
    pub(crate) height: Option<usize>,
}

pub(crate) trait Action: Send {
    fn act(self: Box<Self>, runtime: &Runtime);
}

pub(crate) struct RuntimeInner {
    actions: BTreeMap<usize, Vec<Box<dyn Action>>>,
    pub(crate) height: Option<usize>,
    clear_at_end: Vec<Box<dyn FnOnce() + Send>>,
}

impl Runtime {
    pub(crate) fn current_height(&self) -> Option<usize>;
    pub(crate) fn clear_at_end(&self, clear: impl FnOnce() + Send + 'static);
}

impl Runtime {
    pub fn new() -> Self;
    pub(crate) fn schedule(&self, height: usize, action: impl Action + 'static);
    pub fn propagate(&mut self);
}
```

The action map is the height-ordered queue: a `BTreeMap` from height to the actions scheduled at it. An occurrence is `propagate`: pop the lowest height, record it as the current height, run the actions at it, repeat until the map is empty. Each action is a boxed object: a `MainAction` that makes a derived event compute and fire, a `TriggerAction` that fires an external event, a `HoldAction` that writes a hold, a `SwitchHoldOuterAction` that retargets a switch, a `DWAction` that rebuilds a `rebuild`, a `KeepUntilOccurrenceEnds` that drops a replaced node, a `WidgetReadyTrigger` that schedules done events, and the `FireCombine` and `FireLeftmost` actions of the catch-up path.

`schedule` is where the cycle check lives:

```
if let Some(current_height) = this.height && height <= current_height {
    panic!("Scheduling action at height={height} when already at height={current_height}");
}
```

Outside propagation `height` is `None` and anything may be scheduled; inside it, an action may be scheduled only strictly above the height being run. Every derived event schedules itself at its own height, which is above all of its sources, so a scheduling at or below the current height means a node has been reached from above itself: a cycle. `propagate` makes the same check when it pops a height that is not greater than the last, which cannot happen if `schedule` holds and is kept as belt to the braces.

The occurrence's state lives in `RuntimeInner` beside the action map: the current height, and a `clear_at_end` list of closures run when the occurrence ends, one per event that cached a value for this occurrence, so the next occurrence starts clean. Everything that needs that state takes `&Runtime`. `EventImpl::subscribe(runtime, cb)` and `EventCallback::already_fired(runtime, value)` take it because subscribing mid-occurrence has to know which occurrence it is in and schedule its catch-up into it; `Behavior::hold(runtime, initial, event)`, `Dynamic::hold`, `switch_hold` and `trace` take it because they subscribe when they are made. `map`, `filter_map`, `tag`, `leftmost` and `combine` do not take it: they subscribe lazily, when something subscribes to them, and until then they are descriptions with no creation time. That split is Reflex's split exactly. The operations whose meaning depends on when they are made are the ones in `MonadHold` (`hold`, `holdDyn`, `switchHold`, `headE`, `now`); the ones that are pure functions of their inputs are plain functions there too. In Haskell the monad is forced, because a `hold` written as a pure function would be floated and shared by the optimizer and would then mean the wrong thing; in Rust a call happens where it is written, so the parameter is only there to say which runtime and which occurrence. The parameter is also what makes `Runtime` honestly `send`: an occurrence's state travels with the runtime, not with the thread that happens to be driving it.

Widgets get the handle from the builder's `runtime` field, which is why, from the language's side, a hold is written only in a widget block. A hold has a creation occurrence; the only Prism construct with one is a build; and the Haskell implementation runs the typed AST on Reflex, where a hold outside `MonadHold` cannot be typed at all. So `$count <- 0 then $count + !delta` is a statement of a widget block and not an expression that could appear in a `let` at top level, and the runtime API is now shaped so that the Rust side can make the same mistake only by holding a `&Runtime`, which a build or a World has and a leaf closure does not.

Holds commit at `usize::MAX`. Every hold, every `switch_hold` retarget, and the drop of a replaced node schedules its action at `usize::MAX`, so the commit batch is the last thing `propagate` runs and runs only after every event that could fire has fired. Nothing samples a new value early because nothing is new until then. The exit path, when the map is empty, sets the height back to `None` and runs the clear-at-end list.

The panic path is the transaction's abort. Each action runs under `catch_unwind`; on a panic the map is cleared, the height reset, the clear-at-end list drained, and the panic resumed:

```
if let Err(panic) = outcome {
    self.0.lock().unwrap().actions.clear();
    self.finish_occurrence();          // height = None; run and drain clear_at_end
    resume_unwind(panic);
}
```

Because holds and switches commit only in the `usize::MAX` batch, a panic before that batch leaves

every hold at its value from before the occurrence, which is why this is already a rollback for the values a program can observe. `runtime_keeps_working_after_a_closure_panics` pins it: a mapped closure panics on the value 1, the caller catches the panic, the next value is 2, and the hold ends at 2. A trigger scheduled in the aborted occurrence is released by `TriggerAction`'s `Drop` (below), which `a_trigger_dropped_by_a_panic_accepts_values_again` checks. The panic path does not turn the panic into a value and does not drop the nodes a `rebuild` built before the panic; the target's `run` does both.

## Events

An event is a handle to an implementation with exactly two obligations: accept a subscriber, and say its height. It is lazy: it computes whether it fires, and its value, only while someone is subscribed, and an event nobody holds disconnects from its source. Everything else about events is what a subscriber can be told.

```
pub struct Event<T: ValueType>(pub(crate) Arc<dyn EventImpl<T>>);

pub(crate) trait EventImpl<T: ValueType>: Send + Sync + 'static {
    fn subscribe(&self, cb: Weak<dyn EventCallback<T>>) -> Option<Value<T>>;
    fn height(&self) -> usize;
}

pub(crate) trait EventCallback<T: ValueType>: Send + Sync {
    fn event_fired(&self, runtime: &Runtime, value: Value<T>);
    fn already_fired(&self, value: Value<T>);
    fn invalidate_height(&self);
}
```

A subscriber has three obligations: be told when the event fires during propagation; be told that it already fired this occurrence, which is the catch-up path; and be told that the event's height may have changed. Subscribers are held weakly. Whoever subscribes is responsible for keeping both the callback and the event alive, and the module doc says why in as many words: this is exactly the bookkeeping FRP exists to take out of user code, and it is kept inside the runtime so that user code never sees it.

`subscribe` returns `Option<Value<T>>`: the event's value in the current occurrence if it has already fired, else `None`. This one line implements the construction rule, that a widget built in the middle of occurrence n behaves as if built just before n's inputs, and it is modelled on Spider's `subscribeAndRead`.

Laziness is why a `rebuild`'s output event needs a keep-alive (below) and why the test backend holds the events it watches.

**The subscription manager.** Every derived event is built around a `SubscriptionManager`, which holds the subscriber list, the upstream connection, the per-occurrence cache, and the logic that makes catch-up work:

```
pub struct SubscriptionManager<T: ValueType, E: SubscriptionEvent<T>> {
    subscribers: Mutex<Vec<Weak<dyn EventCallback<T>>>>,
    subscription: Mutex<Option<Arc<MainSubscription<T, E>>>>,
    tag: E::Tag,
    occurrence: Arc<Mutex<Option<Option<Value<T>>>>>,
}

pub trait SubscriptionEvent<T: ValueType>: EventImpl<T> + 'static {
    type Inner: ValueType;            // the upstream event's type
    type Tag: Clone + Send + Sync;    // which upstream, for multi-source events
```

```
    fn invalidate_height(&self);
    fn handle_main_subscription(&self, runtime: &Runtime, value: Value<Self::Inner>, tag: Self::Tag);
    fn handle_early_subscription(&self, _: &Runtime, _: Value<Self::Inner>, _: Self::Tag) {}
    fn handle_already_fired(&self, value: Value<Self::Inner>, tag: Self::Tag);
    fn finish_already_fired(&self) {}
}
```

A derived event `E` implements `SubscriptionEvent` to say what to do with an upstream firing (`handle_main_subscription`, run at `E`'s height as a `MainAction`), what to do with an upstream value that had already fired when `E` subscribed (`handle_already_fired`), and, for events with several sources, what to do with a source's value that arrives before `E`'s turn (`handle_early_subscription`) and how to finish once every source has been consulted (`finish_already_fired`). The manager does the rest. Its pieces, in the order a firing meets them:

`fire(runtime, value)` is what a source calls: record the value for this occurrence, then call `event_fired` on every live subscriber. `notify(this, event, runtime, compute)` is what a derived event calls from its `MainAction`: if there are no live subscribers, disconnect from upstream and return `false`, which is laziness; otherwise compute the value once, record it, deliver it, and make sure the upstream connection exists.

`record_once(compute)` is the per-occurrence cache. If the manager already has a value for this occurrence, or no occurrence is in progress, it returns `None` without calling `compute`; otherwise it calls `compute`, stores the result (which may itself be `None`, for a filter that dropped the value), and registers a clear-at-end closure. The double `Option` is "has this event been computed this occurrence" outside and "did it fire" inside. The effect is that a closure runs at most once per occurrence however many subscribers read its result and whenever they arrive, which `a_closure_runs_once_per_occurrence_even_when_late_subscribers_read_it` pins with a counter.

`add_subscriber(this, event, cb)` is `subscribe`: connect to upstream if not connected, push the weak callback, return `current()`, the cached value if there is one. `connect` is where catch-up starts. When a derived event connects to its upstream mid-occurrence and the upstream has already fired, `resubscribe` returns that value and the manager hands it to `handle_already_fired`, which computes the derived value through `record_once` and delivers it with `deliver_already_fired`. A chain of maps built in the middle of an occurrence therefore catches up link by link, each computing once and caching, and a hold at the end of the chain receives the value it would have received had it existed from the start. `a_widget_built_mid_occurrence_gets_the_mapped_value_of_an_event_that_already_fired` is that sentence as a test, with a `*10` map and a hold that ends at 70.

`MainSubscription` is the callback the manager installs upstream, and it is where height decides the delivery path:

```
fn event_fired(&self, runtime: &Runtime, value: Value<E::Inner>) {
    let Some(this) = self.this.upgrade() else { return };
    let height = this.height();
    let tag = self.tag.clone();
    if runtime.current_height().is_some_and(|current| height <= current) {
        this.handle_already_fired(runtime, value, tag);
        this.finish_already_fired(runtime);
        return;
    }
    this.handle_early_subscription(runtime, value.clone(), tag.clone());
    runtime.schedule(height, MainAction { this, value, tag })
}
```

If the derived event's height is above the current height, the firing is scheduled as a `MainAction` at that height, which is the ordinary push path. If its height is at or below the current height, its turn has already passed in this occurrence: it was subscribed late, and the firing is delivered through the catch-up path at once. The invariant that makes this correct is the one heights give: every firing below the current height has already happened and is cached, so a late subscriber reads caches instead of waiting for pushes that have gone by, and a firing that has not happened yet still arrives by push. `a_widget_built_mid_occurrence_sees_the_same_events_at_any_depth` builds a child from a rebuild at height 2 with one hold on the raw input (height 0) and one on a five-deep map chain (height 5), fires 42, and checks both holds end at 42.

`consolidate()` walks the subscriber list, drops dead weak pointers, and returns the live callbacks upgraded. `push_subscriber` consolidates every sixteen additions so that a list that gains and loses subscribers does not grow without bound; the comment admits this stands in for a real weak bag.

**Height invalidation.** Every derived event caches its height in a `Mutex<Option<usize>>` and computes it as one more than its upstream's `height()`, which caches upstream's in turn. When a `switch_hold` moves to a new inner event, every height computed through it is stale, and the rule that keeps invalidation cheap is in the doc of `EventImpl::height`: invalidation stops early at any node whose cache is already empty, and stopping early is safe only while every node that caches a height also caches whatever it computed that height from. A node can only have a cached height if it asked its upstream, which cached upstream's; so if a node's cache is empty, nothing below it holds a height computed through it, and the walk can stop. Every event type follows the rule: `invalidate_height` clears its own cache and, if the cache was not already empty, passes the call to every subscriber. `switching_to_a_deeper_event_keeps_downstream_heights_valid` is the deterministic check: switch from a height-0 inner to a height-4 inner, fire through two more maps, and the hold at the end gets the value instead of a scheduling panic. A randomised version of the same check, random graphs of switches and maps driven for many occurrences under the invariant checker, is not in the crate and belongs with the checker of the target.

**The combinators.** Each is one file and one struct implementing `SubscriptionEvent`.

```
impl<T: ValueType> Event<T> {
    pub fn external() -> (Event<T>, EventTrigger<T>);
    pub fn never() -> Self;
    pub fn filter_map<O: ValueType>(&self, f: impl Send + Sync + Fn(Value<T>) -> Option<Value<O>> + 'static)
    -> Event<O>;
    pub fn map<O: ValueType>(&self, f: impl Send + Sync + Fn(Value<T>) -> Value<O> + 'static) -> Event<O>;
    pub fn tag<B: ValueType>(&self, behavior: impl Into<Behavior<B>>) -> Event<Arc<(Value<T>, Value<B>)>>;
    pub fn tag_map<B: ValueType, O: ValueType>(
        &self,
        function: impl Fn(Value<T>, Value<B>) -> Option<Value<O>> + 'static + Send + Sync,
        behavior: impl Into<Behavior<B>>,
    ) -> Event<O>;
    pub fn combine<A: ValueType, B: ValueType>(
        function: impl Send + Sync + Fn(OneOrBoth<A, B>) -> Option<Value<T>> + 'static,
        a: Event<A>, b: Event<B>,
    ) -> Event<T>;
    pub fn leftmost(events: Vec<Event<T>>) -> Event<T>;
    pub fn trace(&self, label: String);
}
impl<T: ValueType> Event<Event<T>> {
    pub fn switch_hold(&self) -> Event<T>;
}
```

*External.* `Event::external()` returns the event and an `EventTrigger<T>`, a `Weak` handle to the trigger's

inner. The inner has a `SubscriptionManager` with no upstream (its `Inner` type is `()` and its `handle_*` methods are `unreachable!()`), a `scheduled: Mutex<bool>` flag, and height 0. Triggers are scheduled through the runtime:

```
impl Runtime {
    pub fn schedule_trigger<T: ValueType>(&self, trigger: &EventTrigger<T>, value: Value<T>)
        -> Result<(), ScheduleError>;
}
pub enum ScheduleError { DuringPropagation, AlreadyScheduled }
```

It returns an error rather than panicking in the two cases a caller can plausibly produce: a call from inside a subscriber while the runtime is propagating, and a second value for one trigger before the next `propagate`. The first value wins. `TriggerAction` clears the `scheduled` flag in `Drop`, not in `act`, so the flag is cleared whether the action ran or was thrown away by the panic path; and it releases the trigger's lock before notifying, so a subscriber that reschedules its own trigger gets `Err(DuringPropagation)` instead of a deadlock. Three tests pin these: `scheduling_a_trigger_during_propagation_is_refused_without_a_panic`, `a_subscriber_rescheduling_its_own_trigger_does_not_hang`, `a_second_value_on_one_trigger_before_propagating_is`. In the target the whole trigger API is internal, since the run call is the only thing that fires one.

*Never.* An implementation whose `subscribe` returns `None` and keeps nothing, height 0, one `Arc` per call.

*Filter-map and map.* `FilterMapEvent` holds the upstream event, the closure and a height cache. `handle_main_subscription` is `notify` with the closure; `handle_already_fired` is `record_once` with the closure followed by `deliver_already_fired`. `map` is `filter_map` with `Some`.

*Tag and tag-map.* `Tag` holds the event, a `Behavior<B>` and the closure. On a firing it reads the behavior with `query_for_tag`, which is the value before the occurrence because holds commit at `usize::MAX`, and applies the function. The doc says it in three words: not prompt, never prompt. `tag` is `tag_map` pairing the two values in an `Arc`. This is the tag law as a combinator, and the last steps of `hold_map2_test` pin it: an update and a query in one occurrence, and the query sees the old sum.

*Combine.* `CombineEvent` is the two-source event. Each source is wrapped by `filter_map` into an `Arc<OneOrBoth<A, B>>` so both arrive with one type; there are two managers, one per source, with the first owning the subscriber list that delivers; `one_or_both: Mutex<Option<OneOrBoth<A, B>>>` accumulates what has arrived this occurrence. A source's firing is `accumulate`d early, through `handle_early_subscription`, and the combined firing happens once, at the combine's own height, through `fire`, which takes the accumulator and `notify`s with the function. The catch-up version, `finish_already_fired`, handles a subscription made mid-occurrence when one source has fired and the other might still: if the combine's height is above the current height it defers a `FireCombine` action to its height, since the other source may yet fire; otherwise both sources are behind it and it computes at once. Its `height` is the maximum of the two sources plus one, recomputed on every call; it writes a cache it never reads, a known issue.

*Leftmost.* `LeftmostEvent` is `combine` over a vector with a winner instead of a pair: one manager per source, each tagged with its index, and `compete` keeps the value with the lowest index seen so far. Every manager connects on subscribe, and the subscriber is pushed onto every manager's list, but only the first manager's list is used to deliver (the others' copies are consulted only by `invalidate_height`, so a subscriber is told of a height change once per source, which is harmless and wasteful). This is `|` and `!Leftmost`; `leftmost_test` checks that left wins in both orders of arrival.

*Switch-hold.* `SwitchHold` is `!Switch`:

```
struct SwitchHold<T: ValueType> {
    subscriber_list: SubscriptionManager<T, Self>,
    inner_event: Mutex<Option<Event<T>>>,
    _outer_event: Event<Event<T>>,
    _outer_sub: Arc<SwitchHoldOuterCallback<T>>,
    weak_self: Weak<Self>,
    height: Mutex<Option<usize>>,
}
```

It subscribes to the outer event with a callback that, when the outer fires, schedules a `SwitchHoldOuterAction` at `usize::MAX`: the inner is replaced only at commit, so an inner delivered in n is heard from n+1. The action installs the new inner, clears the switch's height cache, resubscribes the manager to the new inner if anyone is listening, and calls `invalidate_height` on every subscriber, which is the walk above. The switch's height is the current inner's height plus one, or 0 with no inner. The `already_fired` path of the outer callback, for a switch built mid-occurrence over an outer that already fired, defers the same action to `usize::MAX`, so a switch built in n adopts an inner delivered in n at n's commit, exactly as one that existed before n would. `switch_hold_test2` is the law test: outer and inner firing together produce nothing, the next inner firing produces the sum, and two outer firings in a row replace the inner twice.

*Forward.* `ForwardEvent` is the placeholder behind cyclic bindings:

```
pub(crate) struct ForwardEvent<T: ValueType> {
    inner: OnceLock<Weak<dyn EventImpl<T>>>,
    subscriber_list: SubscriptionManager<T, Self>,
    weak_self: Weak<Self>,
    height: Mutex<Option<usize>>,
}
impl<T: ValueType> ForwardEvent<T> {
    pub(crate) fn new() -> Arc<Self>;
    pub(crate) fn close(&self, event: &Event<T>);
}
```

Before `close` it has no target and height 0. `close` stores a `Weak` to the target and, if anyone already subscribed, resubscribes to the target and delivers the target's already-fired value if there is one, so a cyclic event closed after its subscribers attached hands them what the closing event fired in this occurrence. Its height is the target's plus one, cached and invalidated like any other. The target is held weakly for the sake of loop ownership, under widgets below.

*Trace.* `Event::trace(label)` subscribes a logging callback at `log::trace!` level and `mem::forget`s it. The doc says it leaks on purpose and violates many design principles; it is a debugging aid and a known issue.

## Behaviors

A behavior is the pull half of push-pull FRP: a value you ask for, computed on demand and cached until something it depends on changes. There are two ways to ask, and the difference between them is the only dirty-tracking in the runtime.

```
pub struct Behavior<T: ValueType>(pub(crate) Arc<dyn BehaviorImpl<T>>);

pub(crate) trait BehaviorImpl<T: ValueType>: Send + Sync {
    fn query_for_behavior(&self, dependent: Weak<dyn BehaviorDependent>) -> Value<T>;
    fn query_for_tag(&self) -> Value<T>;
}
```

```
pub(crate) trait BehaviorDependent: Send + Sync { fn invalidate(&self); }
```

`query_for_tag` is what a `tag` does: give me the value, I will not be told when it changes. `query_for_behavior(dependent)` is what a derived behavior does: give me the value and remember me, so that when you are invalidated you invalidate me. The dependents list tracks one thing, the commit of a hold.

```
impl<T: ValueType> Behavior<T> {
    pub fn constant(value: Value<T>) -> Self;
    pub fn hold(initial: Value<T>, event: Event<T>) -> Self;
    pub fn map<O>(&self, f: impl Send + Sync + Fn(Value<T>) -> Value<O> + 'static) -> Behavior<O>;
    pub fn map2<A, B>(f: impl Fn(Value<A>, Value<B>) -> Value<T> + 'static + Sync + Send,
                      a: Behavior<A>, b: Behavior<B>) -> Self;
    pub fn computation_behavior(computation: impl BehaviorComputation<T> + 'static) -> Behavior<T>;
    pub fn query_for_computation(&self, dep: BehaviorDependencyTracker) -> Value<T>;
}
impl<T: ValueType> Behavior<Arc<Behavior<T>>> {
    pub fn join(&self) -> Behavior<T>;
}
pub trait BehaviorComputation<O: ValueType>: Send + Sync {
    fn compute(&self, dependent: BehaviorDependencyTracker) -> Value<O>;
}
```

*Hold* is the transaction:

```
struct HoldBehavior<T: ValueType> {
    value: Mutex<Value<T>>,                // the held value; not a cache
    _event: Event<T>,                      // keeps the event alive
    weak_self: Weak<Self>,
    dependents: Mutex<Vec<Weak<dyn BehaviorDependent>>>,
}
impl<T: ValueType> EventCallback<T> for HoldBehavior<T> {
    fn event_fired(&self, runtime: &Runtime, value: Value<T>) {
        runtime.schedule(usize::MAX, HoldAction { hold: this, value })
    }
    fn already_fired(&self, runtime: &Runtime, value: Value<T>) {
        runtime.schedule(usize::MAX, HoldAction { hold: this, value })
    }
    fn invalidate_height(&self) {}         // a behavior's height is always usize::MAX
}
```

`Behavior::hold(runtime, initial, event)` subscribes the hold to its event; if `subscribe` returns an already-fired value (the hold was built mid-occurrence after its event fired) it schedules a `HoldAction` at `usize::MAX` at once. Either way the write happens in the commit batch: `HoldAction::act` stores the value and invalidates every dependent, which drops their caches so that the next query recomputes. Between the firing and the commit every `query_for_tag` sees the old value, which is the tag law; after the commit every query sees the new one, which is the change law on the behavior side. A hold built in the occurrence its event fires adopts the value at that occurrence's commit, which is the last clause of the hold law and the test `fold_dyn_counts_an_event_from_the_build_occurrence`: a count that starts at 0, folds the widget's done event, and reads 1 after occurrence zero.

*Computed* behaviors, `DependentBehavior<O, C>`, hold a `BehaviorComputation` and a `cache: Mutex<Option<(Value<O>, Dependents)>>`. On a query with an empty cache they run `compute`, which queries the computation's own behaviors with a tracker pointing back at this one, so each upstream records this behavior as a dependent; the result and the dependents that asked for it are cached together. `invalidate` takes the cache and invalidates the cached dependents. `map` and `map2`

are two small computations; `join` is a third, which queries the outer for the current inner and then the inner. The seven `joined_behavior_*` tests pin the behavior half of `$Join`: it starts at the initial inner's value; follows a new inner and ignores the old one afterwards; takes the new inner's new value when the swap and the change land in one occurrence; ignores the old inner changing in the swap occurrence; shows the previous value when sampled mid-occurrence; follows a computed inner; and keeps its dependents bounded across five hundred swaps.

Two details keep the dependents lists honest. Each `query_for_behavior` first retains only entries that are alive and are not the dependent being added, then pushes, so a live dependent appears at most once (`a_behavior_that_never_changes_does_not_collect_dependents`: a `map2` whose one side never changes stays under ten weak entries over a thousand occurrences). And the cache lock is a `try_lock` that panics with "behavior cycle: a behavior depends on itself" when a computation reaches back into itself, which is the re-entry check on the pull side (`a_dynamic_defined_as_a_function_of_itself_is_reported_instead_of_hanging`).

*Constant* is a value and nothing else; it records no dependents because it never invalidates.

## Dynamics

A dynamic is the pair, with the invariant that the event fires exactly when the behavior changes and with the value the behavior then holds. The one fact that makes the pair cheap is that a hold's change event *is* its input event: nothing is built to make a hold a dynamic.

```
pub struct Dynamic<T: ValueType> { event: Event<T>, behavior: Behavior<T> }

impl<T: ValueType> Dynamic<T> {
    pub unsafe fn new(behavior: Behavior<T>, event: Event<T>) -> Dynamic<T>;
    pub fn event(&self) -> Event<T>;
    pub fn behavior(&self) -> Behavior<T>;
    pub fn hold(initial: Value<T>, event: Event<T>) -> Dynamic<T>;
    pub fn constant(val: Value<T>) -> Dynamic<T>;
    pub fn map<O>(&self, f: impl Send + Sync + 'static + Clone + Fn(Value<T>) -> Value<O>) -> Dynamic<O>;
    pub fn map2<A, B>(a: Dynamic<A>, b: Dynamic<B>,
                      f: impl 'static + Clone + Send + Sync + Fn(Value<A>, Value<B>) -> Value<T>) -> Self;
}
```

`new` is marked `unsafe` to guard the invariant, which is logical rather than memory-related, a known issue. `hold` pairs `Behavior::hold` with the event itself, which is why a hold's change is prompt in the occurrence its event fires even though its value side commits at the end. `constant` pairs a constant behavior with `never`. `map` maps both halves. `map2` is the change law for two operands: the behavior is `Behavior::map2`, and the event is `combine` over `a.event.tag(b.behavior)` and `b.event.tag(a.behavior)` with the `Both` case taking both new values; so the derived dynamic changes once in an occurrence where either or both operands change, with both new values, which is glitch freedom. `event()` is already the accessor that `!Updated` compiles to. There is no `Dynamic::join`; its design is in the target section.

## Widgets

A blueprint is a description with a `build` method; a node is what `build` produces through the builder. One blueprint value bound twice is two nodes, and a `rebuild` that rebuilds into a widget of another kind needs the two kinds to be one Rust type to the runtime, which is what the polymorphic form is for.

```
pub trait Blueprint: Send + Sync {
    type Output: ValueType;
    fn build(&self, builder: &mut WidgetBuilder) -> Self::Output;
}

pub trait DynBlueprint<O: ValueType>: Sync + Send {
    fn build(&self, builder: &mut WidgetBuilder) -> O;
}
pub struct DynBlueprintAdapter<O: ValueType, W: Blueprint<Output = O>>(W);
impl<O: ValueType> Blueprint for Arc<dyn DynBlueprint<O>> { type Output = O; ... }
```

`DynBlueprint` is the `dyn`-compatible form, and `Arc<dyn DynBlueprint<O>>` implementing `Blueprint` is what lets every compiled widget block be one Rust type to the runtime and lets a `rebuild` firing into a widget of a different kind. The adapter has a private field and no constructor, so nothing outside its module can make one; the blanket `impl<W: Blueprint> DynBlueprint<W::Output> for W` replaces it and compiles.

### The builder.

```
pub struct WidgetBuilder<'a> {
    node: Widget,
    children: Vec<Arc<Widget>>,
    pub(super) done_event: Arc<WidgetReadyEvent>,
    runtime: &'a Runtime,
}

impl WidgetBuilder<'_> {
    pub fn build_root<W: Blueprint>(runtime: &Runtime, widget: &W, height: usize) -> (Arc<Widget>, W::Output);
    pub fn bind<W: Blueprint>(&mut self, widget: &W) -> W::Output;
    pub fn done_event(&self) -> Event<Arc<Widget>>;
    pub fn add_cyclic_dynamic<T: ValueType>(&mut self, name: String) -> (usize, Dynamic<T>);
    pub fn close_cyclic_dynamic_by_index<T: ValueType>(&mut self, ix: usize, dynamic: Dynamic<T>);
    pub fn close_cyclic_dynamic_by_name<T: ValueType>(&mut self, name: &str, dynamic: Dynamic<T>);
    pub fn add_cyclic_event<T: ValueType>(&mut self, name: String) -> (usize, Event<T>);
    pub fn close_cyclic_event_by_index<T: ValueType>(&mut self, index: usize, event: Event<T>);
    pub fn close_cyclic_event_by_name<T: ValueType>(&mut self, name: &str, event: Event<T>);
    pub fn add_external_event<T: ValueType>(&mut self, name: String) -> Event<T>;
    pub fn add_public_event<T: ValueType>(&mut self, name: String, event: Event<T>);
    pub fn add_public_dynamic<T: ValueType>(&mut self, name: String, dynamic: Dynamic<T>);
    pub fn fold_dyn<A: ValueType, B: ValueType>(&mut self, name: String,
        f: impl 'static + Sync + Send + Fn(Value<A>, Value<B>) -> Value<A>,
        initial: Value<A>, update: Event<B>) -> Dynamic<A>;
}
```

`build_root` makes a node for `widget` with no parent: it creates a `WidgetReadyEvent` with a trigger at `height + 1`, calls `build`, wraps the node in an `Arc`, tells the ready event its node, installs the children, and schedules the trigger at `height`. A World calls it with height 0 before the first `propagate`; a `rebuild` firing calls it from inside an action with the action's height plus one. `bind` is the same for a child: a sub-builder whose ready event shares the parent's trigger, `build`, the child node pushed onto the parent's children. In both, `set_node` on the ready event happens as soon as `build` returns, before any occurrence could reach the node, which is what the rebuild path relies on.

`done_event()` is the event that fires once, with the node, when the node's build has landed. The `WidgetReadyTrigger` is one per `build_root`, scheduled at the build height; when it runs it schedules every `WidgetReadyEvent` registered with it at the trigger's height, which is build height plus one. For the root that is occurrence zero, height 1, which is where `@started` fires and where a `rebuild`'s

first-build output fires. For a rebuild at height h the trigger is at h+1 and the done events at h+2.

**Cyclic bindings.** A widget block's recursive bindings are built with a placeholder that is closed once the group is built. The builder offers two, typed, and both are stored in the node:

```
pub fn add_cyclic_event<T: ValueType>(&mut self, name: String) -> (usize, Event<T>) {
    let forward = ForwardEvent::<T>::new();
    let ix = self.node.cyclic_events
        .add(name, Arc::new((forward.clone(), OnceLock::<Event<T>>::new())));
    (ix, Event(forward))
}
pub fn close_cyclic_event_by_index<T: ValueType>(&mut self, index: usize, event: Event<T>) {
    let (forward, target) = self.node.cyclic_events[index]
        .downcast_ref::<(Arc<ForwardEvent<T>>, OnceLock<Event<T>>)>()
        .expect("cyclic event closed with the wrong type");
    forward.close(&event);
    target.set(event).expect("cyclic event closed twice");
}
```

The slot in the node is typed by `T` at creation, so a close with the wrong `T` fails the downcast while the widget is building (`closing_a_cyclic_dynamic_with_the_wrong_type_fails_while_building`), not in the first occurrence. A cyclic dynamic is a `ForwardEvent` for the change event paired with a `OnceLock<Dynamic<T>>` for the value side, and a computed behavior that reaches the slot through a `Weak` and queries the closed dynamic's behavior:

```
struct Computation<T: ValueType> { slot: Weak<(OnceLock<Dynamic<T>>, Arc<ForwardEvent<T>>)> }
impl<T: ValueType> BehaviorComputation<T> for Computation<T> {
    fn compute(&self, dep: BehaviorDependencyTracker) -> Value<T> {
        let slot = self.slot.upgrade().expect("cyclic dynamic sampled after its widget was dropped");
        let d = slot.0.get().expect("cyclic dynamic never closed");
        d.behavior().query_for_computation(dep)
    }
}
```

Here is the ownership arrangement in full, because it does the collector's job. The node holds the strong ends: the slot `Arc` with the `ForwardEvent` and the closing event (or dynamic). The loop points back at the node's slot weakly: the forward holds a `Weak` to its target, the cyclic dynamic's behavior holds a `Weak` to the slot. Everything built from the placeholder (`increment.tag_map(.., accumulator)`, the hold over it, the event that closes the loop) is held strongly by whatever the widget returns or by the node, and strongly along the edges that go *up* to things built earlier. When the node and the widget's outputs are dropped, nothing outside holds the loop strongly and the whole of it is freed; `dropping_a_widget_frees_its_cyclic_loops` takes weak pointers into a loop, drops the node and the outputs, and checks that neither upgrades. A cyclic binding is always closed before `build` returns, which is before any occurrence that could fire through it, so the forward is connected from the first moment its closing event can fire: `cyclic_dynamic_event_fires_when_its_value_changes_in_the_build_occurrence` closes a cyclic dynamic with a hold of the done event and reads 7 from both halves after occurrence zero.

`fold_dyn` is the first client of all this and the model for a compiled hold with a cycle: a cyclic event for the "old value and update" pair, a `filter_map` applying the fold, a `Dynamic::hold` of that, a `tag_map` of the update over the hold producing the pair, and a close. Two tests drive it through a script of increments, one by hand and one through `fold_dyn`, and check both halves after every step.

`add_external_event(name)` makes an external event and registers its trigger in the node by name; `add_public_event` and `add_public_dynamic` register outputs by name. These three are the backend channel, and they go when the run call arrives.

**The node.**

```
pub struct Widget {
    pub children: Mutex<Vec<Arc<Widget>>>,
    pub public_dynamics: Slots<AnyDynamic>,
    pub public_events: Slots<AnyEvent>,
    pub _public_event_callbacks: OnceLock<Vec<Arc<DelegateCallback>>>,
    pub cyclic_events: Slots<Arc<dyn Any + Send + Sync>>,
    pub cyclic_dynamics: Slots<Arc<dyn Any + Send + Sync>>,
    pub triggers: Slots<AnyEventTrigger>,
    pub backend_data: JsonValue,
    pub delegate: OnceLock<Arc<dyn WidgetDelegate>>,
}
```

A node owns its children, its cyclic loops, and (today) its public outputs, triggers and delegate. `Slots<T>` is a `Vec<T>` with a `BTreeMap<String, usize>` of names, indexable by either, panicking on a duplicate name; its own comment says it is an `IndexMap`. `AnyEvent`, `AnyDynamic` and `AnyEventTrigger` are type-erased handles with a checked `get::<T>()` and, for the first two, a converted `Event<AnyValue>` or `Dynamic<AnyValue>` so a backend can subscribe without knowing `T`. `WidgetDelegate` is the trait a backend installs on a node to be told of new children, of destruction, and of public event firings; `set_delegate` subscribes a `DelegateCallback` to every public event and recurses into the children.

**`rebuild`.**

```
impl<T: Blueprint + 'static + Send + Sync + Debug> Dynamic<Arc<T>> {
    pub fn rebuild(&self) -> impl Blueprint<Output = Arc<Event<Arc<T::Output>>>>;
}
```

`rebuild` on a dynamic of blueprints is the language's `rebuild`: a blueprint whose node holds one child, the build of the dynamic's current widget, replaced whenever the dynamic changes, with an output event that fires in every build occurrence with the built widget's output. Its `build`:

```
fn build(&self, builder: &mut WidgetBuilder) -> Self::Output {
    let next_event = Arc::new_cyclic(|weak_self| RebuildEvent {
        widget: self.clone(), weak_self: weak_self.clone(),
        ready: builder.done_event.clone(),
        subscribers: SubscriptionManager::new(()),
    });
    let initial_widget = self.dynamic.event().0
        .subscribe(Arc::downgrade(&(next_event.clone() as Arc<dyn EventCallback<Arc<T>>>)))
        .unwrap_or_else(|| self.dynamic.behavior().0.query_for_tag());
    let output = Mutex::new(Some(builder.bind(&*initial_widget)));
    let first_time_event = builder.done_event().filter_map(move |_| {
        Some(PrismArc::new(output.lock().unwrap().take().unwrap()))
    });
    let next_event = Event(next_event);
    builder.add_public_event("__rebuilt__".to_string(), next_event.clone());
    Arc::new(Event::leftmost(vec![next_event, first_time_event]))
}
```

Four things happen here. The `RebuildEvent` subscribes to the dynamic's change event; if the dynamic already changed earlier in this occurrence, `subscribe` returns the new blueprint and the first build uses it, which is the "builds once with the new value" clause of the construction rule and the test `a_nested_dyn_built_in_the_occurrence_its_dynamic_changes_shows_the_new_value` (a page `rebuild` with a title `rebuild` inside it and one input changing both; the rebuilt page's title shows 7, not 0). Otherwise the first build samples the behavior. The initial blueprint is bound as child 0. The first build's output is delivered through a `filter_map` on the done event, so it fires at the done height

of the build occurrence. And the output is `leftmost` of the rebuild event and the first-build event, with the rebuild listed first: in an occurrence where both fire (occurrence zero with a trigger that changes the dynamic), the rebuild's output wins, which is the output of the widget that is in the tree at commit.

The `add_public_event("__rebuilt__", ..)` line is a keep-alive. `RebuildEvent` is the subscriber to the dynamic's change; if nobody held the `rebuild`'s output event, the only strong reference to it would be gone and the `rebuild` would stop rebuilding. Registering it as a public event stores an `Arc` in the node. A private keep-alive list on the node is the right home, a known issue.

The rebuild is `DWAction`, scheduled at the `RebuildEvent`'s height, which is the dynamic's change height plus one:

```
impl<T: 'static + Blueprint + Debug> Action for DWAction<T> {
    fn act(self: Box<Self>, runtime: &Runtime) {
        let parent_node = self.event.ready.node.get().unwrap().clone();
        let (child_node, output) = WidgetBuilder::build_root(runtime, &*self.value, self.height + 1);
        if let Some(delegate) = parent_node.delegate.get() { /* tell it about child 0 */ }
        self.event.subscribers.notify(Arc::downgrade(&self.event), None, runtime,
                                      || Some(PrismArc::new(output)));
        let old_node = {
            let mut children = parent_node.children.lock().unwrap();
            std::mem::replace(&mut children[0], child_node)
        };
        old_node.prepare_destruction(runtime);
        runtime.schedule(usize::MAX, KeepUntilOccurrenceEnds(old_node));
    }
}
```

The parent node comes from the ready event's `OnceLock`, set when the `rebuild`'s own build returned, so a rebuild in any occurrence, including the one the `rebuild` was built in and including a nested `rebuild` built anew in the same occurrence as its parent, finds its node (`dyn_rebuilds_when_its_input_arrives_in_occurrence_zero`, `nested_dyn_rebuilds_in_the_same_occurrence_as_its_paren
The delegate is optional (`dyn_rebuilds_without_a_backend_delegate`). The new child is built with `build_root` at the action's height plus one, so its done events land above everything the action depends on. The output fires through `notify` at the action's height. The old node is swapped out at child index 0 and, rather than dropped, handed to a `KeepUntilOccurrenceEnds` action at `usize::MAX`. This is the clause that says a replaced node keeps propagating until the occurrence commits: a parent's `switch_hold` still follows the replaced child's output for the rest of the occurrence, and a tick that reaches the old child in the rebuild occurrence must still reach the old child's cyclic dynamic, whose slot the old node owns. `a_replaced_child_keeps_working_until_its_last_occurrence_ends` fires a tick and a rebuild together and checks that the old counter's output, under the non-prompt switch, delivered the tick.

`already_fired` on the `RebuildEvent`, the path taken when a `rebuild` subscribes mid-occurrence to a change that has already happened and is not the initial build, defers its `DWAction` to `max(height, current + 1)`, so a rebuild is never scheduled at or below the running height.

The `RebuildEvent` implements `SubscriptionEvent` with `Inner = ()` and `unreachable!()` bodies, because it has no upstream in the manager's sense: its upstream is the dynamic's change event, which it subscribes to as a plain `EventCallback`.

## The test backend

```
pub struct TestBackend { node: Arc<Widget>, runtime: Runtime, delegate: Arc<TestDelegate>, event_rx:
mpsc::Receiver<EventRecord> }
impl TestBackend {
    pub fn build_tree<W: Blueprint>(widget: &W) -> (W::Output, TestBackend);
    pub fn main_loop(&mut self, script: TestScript) -> Result<(), ScriptError>;
}
pub fn script<const N: usize>(steps: [TestStep; N]) -> TestScript;
pub fn trigger<T: TestValue, const N: usize>(node_path: [usize; N], name: &str, v: T) -> TestStep;
pub fn delay_trigger<..>(..) -> TestStep;   // trigger without propagating yet
pub fn event<..>(node_path, name, v) -> TestStep;      // expect the next recorded firing
pub fn dynamic<..>(node_path, name, v) -> TestStep;    // read a public dynamic now
pub fn crickets() -> TestStep;                          // nothing recorded
impl TestScript {
    pub fn run<W: Blueprint>(self, widget: &W) -> Result<W::Output, ScriptError>;
    pub fn test<W: Blueprint>(self, widget: &W) -> W::Output;
}
```

`build_tree` makes a runtime, builds the root with `build_root` at height 0, installs a `TestDelegate` that clones the node's `public_dynamics` and `triggers` and records every public event firing onto a channel with its node path and name, recurses into children, and propagates once so occurrence zero lands. A script is a list of steps addressed by `[child indexes]` and a name: `trigger` schedules a named trigger and propagates; `delay_trigger` schedules without propagating, so several inputs land in one occurrence; `event` pops the next recorded firing and compares path and value; `dynamic` reads a public dynamic's behavior with `query_for_tag` and compares; `crickets` asserts the channel is empty. `TestValue` is blanket-implemented for any `ValueType + PartialEq` and does the typed trigger and comparison. Eight of the suite's tests are scripts; the rest drive the runtime directly with `Event::external`, `schedule_trigger`, `propagate` and `query_for_tag`.

What the backend does that the target forbids: it holds live runtime objects outside the runtime (the delegate's cloned `Slots`), it reads behaviors directly, and it addresses nodes by child index and name. All three are replaced by the run call, under which a Test World reads only the outcome.

## The 38 tests as the laws they pin

Each test's name is a sentence, and together they are the runtime half of the conformance suite. By law:

- *Leftmost merge and simultaneity*: `leftmost_test`, `combine_test`, `event_filter_test`.
- *The tag law*: `hold_map2_test` (update and query together: the query sees the old value).
- *Non-prompt switching*: `switch_hold_test2`, `switching_to_a_deeper_event_keeps_downstream_heights_valid`.
- *Holds and cycles*: `fold_dyn`, `fold_dyn_cyclic_dyn`, `cyclic_dynamic_event_fires_when_its_value_changes_in_the_bu`
  `fold_dyn_counts_an_event_from_the_build_occurrence`, `closing_a_cyclic_dynamic_with_the_wrong_type_fails_wh`
  `dropping_a_widget_frees_its_cyclic_loops`.
- *Cycles are errors, not hangs*: `a_same_occurrence_cycle_through_a_cyclic_event_is_reported_instead_of_hanging`,
  `a_dynamic_defined_as_a_function_of_itself_is_reported_instead_of_hanging` (each on a thread with a five-second timeout).
- *`rebuild`*: `rebuild`, `dyn_rebuilds_without_a_backend_delegate`, `dyn_rebuilds_when_its_input_arrives_in_occurren`
  `nested_dyn_rebuilds_in_the_same_occurrence_as_its_parent`, `a_replaced_child_keeps_working_until_its_last_c`
- *Construction mid-occurrence*: `a_widget_built_mid_occurrence_sees_the_same_events_at_any_depth`,
  `a_nested_dyn_built_in_the_occurrence_its_dynamic_changes_shows_the_new_value`, `a_widget_built_mid_occurren`
  `a_closure_runs_once_per_occurrence_even_when_late_subscribers_read_it`.

- *Abort and recovery*: `runtime_keeps_working_after_a_closure_panics`, `a_trigger_dropped_by_a_panic_accepts_values_a`
- *The entry points*: `scheduling_a_trigger_during_propagation_is_refused_without_a_panic`, `a_subscriber_rescheduling_its_own_trigger_does_not_hang`, `a_second_value_on_one_trigger_before_propagating_is_`, `runtime_can_move_to_another_thread`.
- *Behaviors*: `a_behavior_that_never_changes_does_not_collect_dependents`, the seven `joined_behavior_*` tests.
- *Values*: `round_trips`.

The gaps in this list are the gaps in the crate: no test of `Dynamic::join` because there is none, no routing test because there are no paths, and no law test that reads only what a World could read.

## Two occurrences traced through the code

The pieces above are easier to hold together as one trace than as a list, so here are two, with the heights written down. The first is the `fold_dyn` test's widget, an accumulator over an external `increment` event; the second is the mid-occurrence construction test, where catch-up does its work.

**An increment.** At build, `build_root(&runtime, &TestWidget, 0)` makes the node and a `WidgetReadyTrigger` at height 1, scheduled at height 0, and calls `build`. `add_external_event("increment")` makes the external event `E` (height 0) and stores its trigger. `fold_dyn` then makes, in order: a `ForwardEvent` `F` for the "old value and update" pair, stored in `cyclic_events`; `FM = F.filter_map(f)`; the hold `H = Dynamic::hold(0, FM)`, whose `Behavior::hold` subscribes to `FM`, which connects to `F` (which has no target yet, so the connection reaches nothing); `T = E.tag_map(pair, H)`; and `close_cyclic_event(F, T)`, which stores a `Weak` to `T` in `F` and, since `F` already has a subscriber, resubscribes it to `T`, which connects `T` to `E`. Heights come out as `E=0`, `T=1`, `F=2`, `FM=3`, and `H` commits at the top. Occurrence zero is `propagate`: height 0 runs the ready trigger, which schedules the node's done event at 1; height 1 fires it to nobody; the map is empty; clear-at-end has nothing to do.

The script's `trigger([], "increment", 3)` calls `schedule_trigger`, which checks the runtime is idle, sets the trigger's `scheduled` flag and schedules a `TriggerAction` at 0, then `propagate`. Height 0: the trigger action calls `E.fire(3)`; `record_once` stores `Some(3)` in `E`'s occurrence cache and registers its clear; `E`'s one live subscriber is `T`'s `MainSubscription`, whose `event_fired` finds `T.height() = 1 > 0` and schedules a `MainAction` for `T` at 1. Height 1: `T.handle_main_subscription` calls `notify`: there is a subscriber (`F`'s subscription), so it computes once, and the computation reads `H.query_for_tag()`, which is 0, the value before the occurrence, pairs it with 3, records `(0, 3)` and delivers it to `F`'s subscription, which schedules `F` at 2. Height 2: `F` forwards `(0, 3)` to `FM`'s subscription, scheduled at 3. Height 3: `FM` computes `f(0, 3) = 3`, records it, and delivers to two subscribers: `H`, whose `event_fired` schedules a `HoldAction` at `usize::MAX`, and the test backend's `DelegateCallback` on the public event `new_value`, which puts `3` on the channel. Height `usize::MAX`: the hold action writes 3 and invalidates its dependents, of which there are none. The map is empty: height becomes `None`, and the four clear closures empty the caches of `E`, `T`, `F` and `FM`. The script's `event([], "new_value", 3)` pops the channel and `dynamic([], "accumulator", 3)` reads `H.query_for_tag()`, which is now 3.

Every hold in a Prism program commits this way, and the whole of the tag law is the one line at height 1 that read `H` before height `usize::MAX` wrote it.

**A widget built in the middle.** The test builds an external `E`, a five-deep chain `D1..D5 = E.map(id)` five times (heights 1 to 5), a hold on `D5` so the chain is subscribed, and a `rebuild` over `children = Dynamic::hold(new_child(), E.map(new_child))` whose block makes a child with two holds, one on `E` and one on `D5`. The `RebuildEvent` subscribes to `children`' change event `M = E.map(..)` (height 1), so its own height is 2. Occurrence zero builds the first child, whose holds subscribe to `E` and `D5` and

get `None` back, since nothing has fired.

Occurrence one is `42` on `E`. Height 0: `E` fires; its subscribers are `M` (scheduled at 1), `D1` (at 1), and the first child's shallow hold (a hold action at the top). Height 1: `M` computes a new child blueprint and delivers it to the `RebuildEvent`, which schedules a `DWAction` at 2; `D1` fires and schedules `D2` at 2. Height 2: `D2` fires, scheduling `D3` at 3; the `DWAction` runs `build_root(child, 3)`. Inside that build, `Dynamic::hold(0, E)` subscribes to `E` and `subscribe` returns `Some(42)` from `E`'s cache, so `Behavior::hold` defers a `HoldAction` with 42 to the top: this hold has caught up. `Dynamic::hold(0, D5)` subscribes to `D5`, which is already connected and has no cached value yet, because height 5 has not run; `subscribe` returns `None`, and the hold waits like any subscriber. The action schedules the new node's ready trigger at 3, delivers the new child's output, swaps the node in at child 0, and hands the old node to a `KeepUntilOccurrenceEnds` at the top; then `propagate` drains the deferred list and schedules the caught-up hold action. Heights 3 and 4: `D3`, `D4`, and the done events. Height 5: `D5` fires 42 to three holds: the test's own, the old child's, and the new child's, each scheduling at the top. Height `usize::MAX`: five hold actions write 42, and the old node is dropped. Both of the new child's holds read 42.

There is a third branch the test does not take and the mechanism covers. Had nobody subscribed to `D5` before the new child did, `D5` would have had no connection and no cache; the child's `subscribe` would run `connect`, which resubscribes up the chain to `E`, finds `E`'s cached 42, and hands it to `D1.handle_already_fired`, which computes `id(42)` once through `record_once` and `deliver_already_fired`s it to `D2`, and so on to `D5`, which returns it to the new hold. The chain catches up on demand, each link computing once, and a later subscriber reads the caches. That is the whole of Spider's `subscribeAndRead`, and it is why a closure runs at most once per occurrence whoever asks and whenever. ## The target

The runtime is a library a World calls. Its public surface is the run call and the handful of plain-data types that cross it; the graph types are public for compiled blueprints, which are the only other client, and for nothing else. Everything in this section is a consequence of the boundary (one synchronous call per occurrence, plain data both ways, nothing shared) applied to the core that exists.

## Single-threaded

Every lock in the current code is taken by the one thread that is propagating; the locks are uncontended by construction, and the only work they do beyond locking is the re-entry check that the height comparison already makes. So the pointer and cell types become `Rc`, `Weak`, `RefCell` and `Cell`. The `send + Sync` bounds on `Blueprint`, `DynBlueprint`, `EventImpl`, `EventCallback`, `BehaviorImpl`, `HeapValue`, `Action` and every closure parameter go, which also lets a compiled closure capture an `Rc` without a wrapper. The occurrence state moves into one shared core that every node points at:

```
pub struct Runtime {
    core: Rc<Core>,
    root: Option<Rc<Widget>>,          // the root InterceptBlueprint's node
    tags: Rc<TagTable>,
}

pub(crate) struct Core {
    actions: RefCell<BTreeMap<usize, Vec<Box<dyn Action>>>>,
    commits: RefCell<Vec<Box<dyn Commit>>>,
    deferred: RefCell<Vec<(usize, Box<dyn Action>)>>,
    height: Cell<Option<usize>>,
    occurrence: Cell<u64>,
    built: RefCell<Vec<Built>>,            // the build journal
```

```
    requests: RefCell<Vec<(Path, Value)>>, // the request collector
    errors: RefCell<Vec<RuntimeError>>,    // contained errors to report in the outcome
    pending_quarantine: RefCell<BTreeMap<Path, RuntimeError>>,   // owners the last error named
    depth: Cell<u32>,                      // the recursion depth guard
}

unsafe impl Send for Runtime {}            // sound: see below
```

The runtime is a parameter of every constructor that subscribes when it is made and of `subscribe` itself, so an occurrence's state never leaves the runtime. What a parameter cannot do is stop a caller from passing the wrong runtime: an event built against one runtime and subscribed in another is a bug nothing catches. The target closes that by giving every graph object an `Rc<Core>`, taken from the upstream it is built from: `filter_map` copies its source's, `hold` copies its event's, `combine` and `leftmost` check that their sources agree. The only constructors with no upstream are `never` and `constant`, and they schedule nothing, so they need none. The root of every chain is an event the builder made, and the builder has the core. A `subscribe` then asserts that the runtime it was handed is the one its source belongs to, and the mix is a panic at the call rather than a silent misdelivery. The cost is one pointer per node.

`Runtime` is `send` and not `Sync`. It cannot be `send` by derivation, since it holds `Rc`s, so the impl is written by hand, and it is sound because the runtime is the unique owner of every `Rc` in its graph: no handle to any of them crosses the boundary, by the rule that nothing in `Inputs` or `Outcome` is a runtime object, and compiled blueprints live inside the graph they are built into. Moving all the clones of an `Rc` to one other thread together is the one case `Rc`'s non-`send`-ness does not cover, and it is this case. A World may build the runtime on one thread, move it to another and drive it there; no two threads can touch it at once because `run` takes `&mut self` and nothing shares it. The `runtime_send` test becomes "build a graph on one thread, move the runtime, run an occurrence on the other."

The `try_lock` re-entry checks become a `Cell<bool>` per cached height and per behavior cache, set while computing and cleared after, with the same effect and a message that names the node: "event cycle: `name` depends on itself in one occurrence", where `name` is the source span the compiler attaches to every placeholder and slot. The eager detection of the error chapter, below, means this flag is reached only by a runtime bug once the walks at edge creation are in; it stays as the assertion it is.

Why not keep the locks and gain threads for free? Because they buy nothing. The parallelism that would use them is a different runtime variant with a different loop, described at the end of the chapter. Keeping `Mutex` as a down payment on that variant costs a lock operation on every subscriber list and every height read, and makes every closure `send + Sync`, which pushes `Arc` into compiled code that would otherwise use `Rc`.

## The run call

A World could keep today's form: hold triggers for the slots it cares about, call `schedule_trigger` then `propagate`, read behaviors. That is a World holding runtime objects, and a World that holds a trigger for a slot that has since been torn down holds a pointer into a tree that has moved on; the rules about discarding late results become the World's to enforce against its own data structures. With paths as the only addresses, a late result is a path not in the opened list, and the runtime says so. So the boundary is one call, plain data in, plain data out:

```
pub struct Inputs<'a> {
    pub responses: Vec<(Path, Value)>,
    pub pushes: Vec<(ElementValue, Value)>,
```

```
    pub quarantine: Vec<Path>,
    pub oracle: &'a dyn Fn(&ElementValue) -> Value,
}

pub struct Outcome {
    pub patch: ListPatch,                  // the root list's change in this occurrence; empty if none
    pub requests: Vec<(Path, Vec<Value>)>,
    pub result: Value,
    pub errors: Vec<RuntimeError>,         // contained build failures, this occurrence
}

#[derive(Clone, Copy, PartialEq, Eq)]
pub enum ErrorKind { Cycle, Overflow, UnknownPath, UnsuppliedState, DeferBackwards, Component, Build }

pub struct RuntimeError {
    pub kind: ErrorKind,
    pub message: String,
    pub occurrence: u64,
    pub spans: Vec<Span>,                  // the loop in order, or the one binding
    pub owner: Option<Path>,               // the lowest owning node, for quarantine
    pub path: Option<Path>,                // the path involved, for UnknownPath
    pub element: Option<ElementValue>,     // the element involved, for UnsuppliedState
}

impl Runtime {
    pub fn build(main: Rc<dyn DynBlueprint<Value>>, tags: Rc<TagTable>, oracle: &dyn Fn(&ElementValue) ->
    Value)
        -> Result<(Runtime, Outcome), RuntimeError>;
    pub fn run(&mut self, inputs: Inputs<'_>) -> Result<Outcome, RuntimeError>;
    pub fn list(&self) -> ElementList;     // the committed root list: an Rc clone, constant time
}
```

The outcome reports the list as a patch because a GUI changes a label and keeps a thousand rows, and a `Defer` occurrence often changes nothing at all; a full list per call would charge every turn the program's size. A World that wants the whole list, at startup or to rebuild its own tables, asks `list()`, which hands out the persistent map the runtime already holds.

`Value` at the boundary is the runtime's untyped value, today's `AnyValue`, restricted to plain data: no event, dynamic or blueprint inside, which the compiler guarantees for element fields, responses, pushes and results and which a debug build asserts at the slot. The typed wrapper that is `Value<T>` today is renamed `Typed<T>` so that the boundary can use the plain name; nothing else about it changes. `Value` holds `I64` and `U64` immediates, since `Int` is 64-bit signed and `Nat` 64-bit unsigned; the 32-bit variants go in step 1. `Path`, `ElementValue` and `TagTable` are below. `build` is occurrence zero: it constructs `@main` under a root node and returns the runtime with the initial list as a patch of insertions; a cycle found there, or a state tag the oracle cannot answer, is an `Err` and no runtime. `run` is every occurrence after that. Nothing else a World needs is public; `Event::external`, `schedule_trigger`, `build_root` and `propagate` become `pub(crate)`.

`RuntimeError` is one record with a kind rather than one variant per kind because the World core prints every error the same way and the `Error` element a program can match on has a closed `kind` field; a Rust enum per kind would be unpacked into this record at the boundary anyway. Of the seven kinds, the runtime raises five: `Cycle`, `Overflow`, `UnknownPath`, `UnsuppliedState` and `Build`. `DeferBackwards` is raised by the World core, which is the only thing that knows what a turn is. `Component` is raised by the World for its own components' failures, and by the runtime for one thing: a panic inside propagation whose payload is not the runtime's own, which can only come from native code a World

or a library put inside a closure. Two further protocol failures, a response of a type the slot's tag does not declare and two responses to one path in one occurrence, are the World's bugs rather than the program's; `run` refuses them before scheduling anything, as `Component` errors naming the path, and the core treats them as its own fault.

`quarantine` is the World's instruction, taken from an error the runtime reported: the `owner` path of a `Cycle` found at a switch adoption, which is the one error where the occurrence is lost and the next one would be lost the same way. The occurrence section says what the runtime does with it. A path in `quarantine` that no error named is a `Component` error, since the World is allowed to quarantine only what it was told to.

## The occurrence

Two things go wrong in a run call that has no internal order. An input delivered against a list that changed under it reaches a slot that did not exist when the World chose the path, or misses one that did; and a hold written as soon as its event fires is read, by a tag later in the same occurrence, at its new value. The second needs every write held until every read is done, and holding the list's own writes to commit answers the first too: every routing table is an element list's value side, which does not move until commit, so a lookup at any height reads the list the World or the interceptor last received. `run` therefore has five phases, each short.

*Open.* Increment the occurrence counter, which clears every per-occurrence cache at once (the caches are stamped, below). Perform the quarantine, if any. Nothing else: there is no table to copy, because each list's committed value is already the table.

*Fire.* For each response `(path, value)`: look the path up in the root list's committed value, the list the previous call returned; if absent, return `Err` with kind `UnknownPath` before anything has been scheduled; if the slot's tag does not respond, or the value's type is not the tag's response type, or the path was already given a response in this call, return the protocol error; otherwise schedule the root fan at height 0 with the entry. For each push `(element, value)`: find the root state cell for that element value (a cell exists for every distinct element value some live state slot has asked for) and schedule its trigger at height 0. A push for a value no slot holds is ignored; it is stale bookkeeping on the World's side and harms nothing. All inputs are scheduled before any runs, so they are simultaneous.

*Propagate.* The loop of today's `propagate`, with the error path changed and with the journal: every node built inside the loop, by a `rebuild`, a list widget or a supplier, is pushed onto `built` with its parent and its position, so that rollback can find it.

*Commit.* The commit list runs: hold writes, switch retargets (each preceded by the eager cycle walk, below), drops of replaced nodes, list-widget removals, supplied-widget removals. In the target the commit list is its own `Vec<Box<dyn Commit>>` on the core rather than the `usize::MAX` entry of the action map. Two reasons. The current encoding makes "after everything" and "the greatest height" the same number, so an action in the commit batch that needs to defer something to commit has nowhere to put it (scheduling at `usize::MAX` while at `usize::MAX` fails the cycle check, and `current + 1` overflows there). And rollback is "drop the commit list", which is one line when the list is its own field.

*Close.* Collect the outcome: the root list's patch for this occurrence, the request firings grouped per slot, the result, and the contained errors. The grouping order is the order of the *opened* list, with slots created in this occurrence after it in creation order, whether or not they survived to commit; a slot torn down in this occurrence that was in the opened list keeps its opened

position. This is the order the conformance chapter's transcript prints `requested` lines in, and `a_request_fired_by_a_slot_torn_down_in_the_same_occurrence_is_reported_with_its_path` pins the torn-down case. Within one slot, firings are in firing order, which for one event is at most one per occurrence. Return `Ok`.

**Rollback.** If any action panics, or any check returns an error from inside propagation, `run` does, in order: drop the action map and the deferred list; drop the commit list, which is every hold write, switch retarget and node drop that was going to happen; unwind the journal, which detaches every node built in this occurrence from its parent (restoring the previous child at that position, which is still alive because replaced nodes are dropped only at commit) and drops it, including supplied widgets, since those are journaled under the interceptor's node like any other; reset the height and the depth counter; clear the request collector and the contained-error list; and return `Err`. The caches need nothing: they are stamped with the occurrence number and the next `run` increments it. Nothing resumes the panic across the boundary; a panic is caught at the action and becomes a `RuntimeError`: kind `Overflow` when the payload is the runtime's own `PrismPanic` (which checked arithmetic and the depth guard raise, with the span), kind `Component` for any other payload. After `Err`, the graph is as it was after the previous `Ok`, and the next `run` works. The invariant checker, below, is what tests this: after a failed occurrence, every hold at its previous value, every switch on its previous inner, every node that existed before still existing, every node built during the failure gone, no cache stamped with the failed occurrence's number.

Why rollback is cheap rather than merely possible: every write a program can observe is already buffered to commit time, for the tag law's sake. The only writes during propagation are to per-occurrence caches, which the stamp handles, and to the tree, which is append-only during an occurrence: new nodes are built and journaled, old nodes are replaced only at commit, and until then stay in the tree as the payload of a commit entry. So the undo log is the commit list plus the journal, and both are kept for other reasons. Rollback cannot undo a closure's side effect outside the graph, and compiled code has none, because it is pure by construction.

**Contained failures: the error node.** A `rebuild` build, a `for` key's build and a supplier's build each have a boundary of their own: a closure the runtime calls, which creates a node, builds into it, closes its placeholders, creates its slots, and returns. Until it returns, nothing outside the node refers to anything inside it, so a build that fails can be abandoned by dropping the half-built node, and the occurrence continues. The builder implements this as a nested transaction. On entry it records the lengths of the commit list, the deferred list, the journal and the request collector; a failure (a cycle found when a placeholder is closed, an unsupplied state tag, a panic in a build-time closure) truncates all four to the recorded lengths, drops the node, and builds the *error node* in its place: a node with one slot whose element is `Error(error=the report)` and no result. The `rebuild`'s event does not fire in that occurrence, because there is no result of the block's type to fire with; a `!Switch` over it keeps the old generation's event, which never fires again once the old generation is dropped at commit, and a hold of dynamics under `$Join` keeps the old generation's dynamic at its last value. The old generation is still torn down at commit, because the dynamic it was built for has changed. The report goes on `errors` in the outcome with kind `Build` wrapping the cause's kind in its message and spans. The root build has no outside to continue into, so a failure there is the `Err` of `build`.

The truncation is why the four lists are plain vectors with push at the end: a build is synchronous and nothing else appends while it runs, so "everything after the recorded length belongs to this build" is exact. A hold built inside the failed build may have deferred its catch-up write onto the commit list; the truncation removes it, and with it the only strong reference to the half-built hold.

**Eager cycle detection.** A prompt cycle has one property: no consistent assignment of heights.

The runtime finds one by the height walk, and the choice that decides which unit of failure a cycle lands in is *when* the walk runs. The target runs it at the moment an edge is created, in three places: when a placeholder is closed (`ForwardEvent::close` computes the target's height through `height_checked`, which walks upstream with a per-node in-progress flag and returns the loop's spans when it meets a node already in progress); when a switch adopts a new inner at commit (the retarget computes the new inner's height the same way, and the walk reaches the switch itself exactly when the inner depends on it, because every cached height below the switch was invalidated by the retarget and the walk stops only at cached heights, which are the nodes not downstream of it); and when `Dynamic::join` or the list widget retargets, which are the same operation. A cycle found at a close during a build is a build failure, contained as above. A cycle found at a switch adoption is found in the commit batch, after propagation and before any hold write, and aborts the occurrence with the loop's spans and the owner's path. The owner is the lowest common owning node of the nodes on the loop, found by walking each node's owner chain, which every graph object has as a `Weak<Widget>` set by the builder. The cost is a walk upstream per edge creation, bounded by the cache: the walk stops at any node whose height is cached, and a cached height is valid exactly as long as nothing below it has been invalidated, which is the rule every event type keeps. The runtime check that a node never schedules at or below the current height stays as an assertion against the runtime's own bugs; with eager detection it should never fire for a program's cycle.

**Quarantine.** At open, for each path in `quarantine`: look it up among the owners the runtime named in its last error (it keeps them in `pending_quarantine: BTreeMap<Path, RuntimeError>` until the next successful `run`); find the node at that path; replace it in its parent with an error node holding `Error(error=that report)` at the same position; drop the subtree, which drops its slots, its supplied widgets' interest and its loops. The replacement is in the parent's children map under the same segment, so the parent's list changes in this occurrence like any other child replacement and the World sees the `Error` element in the outcome. Responses in this same call addressed to paths under the quarantined node were legal addresses, since those paths were in the list the World last received; they are delivered to nothing and reported with their paths in `requests` as firings-to-nothing are, so the World can discard them. What the parent held of the quarantined node's results stays as handles to a dropped subtree: an event that never fires again, a dynamic at its last value.

## The graph API

The compiler emits one node per lift. `$a + $b + $c` is one derived dynamic, not a tree of `map2`s allocating pairs, and `$a + $b + !e` is one tag reading two behaviors, because the typed-AST text form prints one node per lift and the graph should not be a lie about the text form. So the target's combinators are n-ary where the current ones are binary, and three constructors exist for the compiler's sake: `hold_first`, for a dynamic-of-blueprints bind whose initial value is the first build's result; `map_n`, a derived dynamic over any number of dynamics; and `tag_map_n`, a tag reading any number of behaviors. The whole of the target's graph surface, with the fate of today's public entry points:

```
impl<T: ValueType> Event<T> {
    pub(crate) fn external(core: &Rc<Core>) -> (Event<T>, EventTrigger<T>);   // the run call's inputs only
    pub fn never() -> Self;
    pub fn filter_map<O: ValueType>(&self, f: impl Fn(Typed<T>) -> Option<Typed<O>> + 'static) -> Event<O>;
    pub fn map<O: ValueType>(&self, f: impl Fn(Typed<T>) -> Typed<O> + 'static) -> Event<O>;
    pub fn tag_map_n<O: ValueType>(&self, behaviors: Vec<Behavior<Value>>,
                                   f: impl Fn(Typed<T>, &[Value]) -> Option<Typed<O>> + 'static) -> Event<O>;
    pub fn tag_map<B: ValueType, O: ValueType>(&self, behavior: Behavior<B>,
                                   f: impl Fn(Typed<T>, Typed<B>) -> Option<Typed<O>> + 'static) -> Event<O>;
// tag_map_n over one
```

```
    pub fn combine<A: ValueType, B: ValueType>(f: impl Fn(OneOrBoth<A, B>) -> Option<Typed<T>> + 'static,
                                               a: Event<A>, b: Event<B>) -> Event<T>;
    pub fn leftmost(events: Vec<Event<T>>) -> Event<T>;
}
impl<T: ValueType> Event<Event<T>> {
    pub fn switch_hold(&self) -> Event<T>;                       // no inner until the first outer firing
    commits
    pub fn switch_hold_seeded(&self, initial: Event<T>) -> Event<T>;   // follows `initial` until then
}
impl<T: ValueType> Dynamic<T> {
    pub(crate) fn new(behavior: Behavior<T>, event: Event<T>) -> Dynamic<T>;   // checked by the invariant
    checker
    pub fn constant(value: Typed<T>) -> Dynamic<T>;
    pub fn hold(initial: Typed<T>, event: Event<T>) -> Dynamic<T>;
    pub fn hold_first(event: Event<T>) -> Dynamic<T>;            // initial value: the event's catch-up value in
    the build occurrence
    pub fn map<O: ValueType>(&self, f: impl Fn(Typed<T>) -> Typed<O> + 'static) -> Dynamic<O>;
    pub fn map_n<O: ValueType>(dynamics: Vec<Dynamic<Value>>, f: impl Fn(&[Value]) -> Typed<O> + 'static) ->
    Dynamic<O>;
    pub fn map2<A: ValueType, B: ValueType>(a: Dynamic<A>, b: Dynamic<B>,
                                            f: impl Fn(Typed<A>, Typed<B>) -> Typed<T> + 'static) -> Self;   //
map_n over two
    pub fn distinct(&self, eq: impl Fn(&Value, &Value) -> bool + 'static) -> Dynamic<T>;   // $Distinct
    pub fn event(&self) -> Event<T>;                             // !Updated
    pub fn behavior(&self) -> Behavior<T>;
}
impl<A: ValueType> Dynamic<Dynamic<A>> {
    pub fn join(&self) -> Dynamic<A>;                            // $Join, prompt on both
}
impl<W: DynBlueprint<O>, O: ValueType> Dynamic<Rc<W>> {
    pub fn rebuild(&self) -> Rebuild<W, O>;         // rebuild: Output = (Event<O>, O), the event and the first
    build's result
}
impl<K: ValueType + Ord, V: ValueType> Dynamic<PMap<K, V>> {
    pub fn list_blueprint<W: Blueprint>(&self, kind: Consolidation, block: impl Fn(&K, Dynamic<V>) -> W +
    'static) -> ListWidget<K, V, W>;
}
impl WidgetBuilder<'_> {
    pub fn bind<W: Blueprint>(&mut self, widget: &W) -> W::Output;
    pub fn started(&mut self) -> Event<()>;                      // @started: a rebuild over a constant, takes a
    position
    pub fn element(&mut self, element: ElementValue, request: Option<Event<Value>>) -> Result<ElementResult,
    RuntimeError>;
    pub fn add_cyclic_event<T: ValueType>(&mut self, span: Span) -> (CyclicEvent<T>, Event<T>);
    pub fn add_cyclic_dynamic<T: ValueType>(&mut self, span: Span) -> (CyclicDynamic<T>, Dynamic<T>);
    pub(crate) fn build_root<W: Blueprint>(..) -> ..;              // called by rebuild, for and the supplier
    only
}
impl<T: ValueType> CyclicEvent<T>   { pub fn close(self, event: Event<T>) -> Result<(), RuntimeError>; }
impl<T: ValueType> CyclicDynamic<T> { pub fn close(self, dynamic: Dynamic<T>) -> Result<(), RuntimeError>; }
impl Runtime {
    pub(crate) fn schedule_trigger<T: ValueType>(&self, trigger: &EventTrigger<T>, value: Typed<T>);   // from
    run only
    pub(crate) fn run_occurrence(&mut self) -> Result<(), RuntimeError>;                               //
    today's propagate
}
```

`hold_first` is a hold with no initial argument: it subscribes to its event in the build occurrence, takes the catch-up value `subscribe` returns as the initial value, and from then on is an ordinary hold. The

compiler emits it for one thing only, the plain-result lift under the canonical node structure, below, where the event is a `rebuild`'s output and has always fired by the time the hold is built (the first build is inside the `rebuild`'s own build, and its output is the `rebuild`'s second result, so the catch-up value exists by construction); a `hold_first` over an event that has not fired is a build error, since there is no value to hold, and the compiler never emits one. `map_n` is the change law for any arity: its behavior is a computation over every operand's behavior, and its event is a single merge node that fires at n with every operand's value after n if any operand changed at n, once however many did, which is glitch freedom with no pair allocation in the middle. `tag_map_n` reads every behavior with `query_for_tag` at the event's firing, which is the tag law; the behaviors are untyped at the signature because the compiler has already checked them and the closure reads them by position. `map2` and `tag_map` stay as the two-operand cases that hand-written blueprints and the join use. `distinct` is `$Distinct`: a derived dynamic whose event is the source's filtered by `eq old, new` against the value before the occurrence, one node, which is why dedupe is a prelude function and not a runtime policy.

The one thing the signatures hide is `Rc<Core>`. Every constructor takes it from its first operand, `never` and `constant` take none because they schedule nothing, and the builder hands it to `external`, `started` and the placeholders, which are the roots of every chain.

### `Dynamic::join`

The outer switches to `d2` in the same occurrence that `d2` changes. Whether `d2`'s change sits above or below the switch in height order depends on where the two were built, and both orders are common: a `rebuild` over a block that returns a dynamic produces an event of dynamics whose inner change events sit above the `rebuild`'s rebuild height, while a hold of two pre-existing dynamics switched by a click has the inner below. A `switch_hold` over the inner events plus a `tag` of the new inner's behavior at the swap gives the *old* value of `d2` when `d2` changes later in the occurrence, and the joined behavior ends at `d2`'s new value, so the two halves disagree.

For `outer: Dynamic<Dynamic<A>>`, `join` is `$Join`: at occurrence n its value before n is the value before n of the inner current before n; its change event fires at n, with the value after n, if the outer changes at n (to a new inner, whose value after n is taken) or if the current inner changes at n. It is prompt on both. The behavior half exists and passes its seven tests. The event half is new, and the correct event is the leftmost of two:

```
impl<A: ValueType> Dynamic<Dynamic<A>> {
    pub fn join(&self) -> Dynamic<A> {
        let behavior = self.behavior().map(|d| d.behavior()).join();   // exists today
        let on_outer = JoinOuterEvent::new(self.clone());               // outer change -> new inner's value
        after n
        let on_inner = self.event().map(|d|
        d.event()).switch_hold_seeded(self.behavior().query_for_tag().event());
        let event = Event::leftmost(vec![on_outer, on_inner]);
        Dynamic::new(behavior, event)                                   // pub(crate), checked
    }
}
```

`on_inner` is a `switch_hold` over the inners' change events, seeded with the initial inner's: it follows the inner that was current before n and moves to the new one at commit. `on_outer` is the new piece:

```
struct JoinOuterEvent<A: ValueType> {
    outer: Dynamic<Dynamic<A>>,
    current: RefCell<Event<A>>,          // the inner's change event as of the last commit
    pending: RefCell<Option<Pending<A>>>, // set when the outer fires in this occurrence
    subscribers: SubscriptionManager<A, Self>,
```

```
    height: Cell<Option<usize>>, ...
}
struct Pending<A: ValueType> { inner: Dynamic<A>, heard: Cell<Option<Value<A>>>, sub: Rc<dyn EventCallback<A>>
}
```

When the outer fires at n with inner `d`, the joined value after n is `d`'s change value at n if `d` changes at n, else `d`'s current value. Whether `d` changes at n is not known at the outer's height; it is known once `d`'s change height has passed. So `JoinOuterEvent` does not fire at the outer's height plus one. On the outer's firing it subscribes a fresh callback to `d.event()` with catch-up: if `subscribe` returns a value, `d` already changed below and `heard` is set; otherwise `d` will push to the callback if it changes above. It then invalidates its own height and its subscribers' and schedules its firing at its new height, `max(outer.height(), d.event().height()) + 1`, where it reads `heard` if set, else `d.behavior().query_for_tag()`. If `d`'s height is at or below the current height, `d` has either fired (and was caught up) or will not fire this occurrence, and the firing can be scheduled at the current height plus one. At commit, `current` becomes `d`'s event and `pending` is cleared. The height is dynamic, like a switch's, and is recomputed through the same invalidation walk.

With `leftmost` putting `on_outer` first, an occurrence where the outer changes and the *old* inner also changes gives the new inner's value, which is right: the old inner is no longer current after n. An occurrence where only the current inner changes fires `on_inner` with its value. An occurrence where the outer changes and the new inner changes fires only `on_outer`, since `on_inner` is still on the old inner. Both halves agree at every commit, which the invariant checker asserts for every dynamic.

**The swap-and-change case, traced in the "above" order.** Take an external `E` at height 0, `d2 = Dynamic::hold(0, E.map(id).map(id).map(id))` so that `d2`'s change event `C2` is at height 3, `outer = Dynamic::hold(d1, E.map(|_| d2))` whose change event `O` is at height 1, and `j = outer.join()`. Before the occurrence `j` is on `d1`, at value 5, and `JoinOuterEvent` has `current = d1.event()`. The World fires `E = 9`.

```
height 0   E fires 9.  Subscribers: O's map (schedule at 1), d2's first map (schedule at 1).
height 1   O fires d2.  JoinOuterEvent: pending = { inner: d2, heard: None }; subscribe to C2 with
           catch-up -> None (height 3 has not run); invalidate own height; new height is
           max(1, 3) + 1 = 4; schedule firing at 4.
           d2's first map fires 9 to the second (schedule at 2).
height 2   second map fires 9 to the third (schedule at 3).
height 3   C2 fires 9.  Subscribers: d2's hold (commit entry: write 9), the pending callback
           (heard = Some(9)).  on_inner's switch_hold is still on d1.event(), which is quiet.
height 4   JoinOuterEvent fires: heard is Some(9), so the value is 9.  leftmost picks on_outer; j's
           event fires 9.  A map over j fires at 5 and reads 9: the join is prompt.
commit     d2's hold writes 9; outer's hold writes d2; the joined behavior now reads d2 = 9;
           JoinOuterEvent: current = d2.event(), pending = None; on_inner's switch adopts d2.event().
```

The "below" order is the same trace with `C2` at height 1 and `O` at height 3: at height 3 the fresh subscription to `C2` returns `Some(9)` by catch-up, `heard` is set at once, `d2`'s height is at or below the current height, and the firing is scheduled at the current height plus one, 4, where it reads `heard`. The split that an implementer has to get right is in those two lines: below the outer's height, catch-up fills `heard`; above it, the push fills `heard`; and the firing is scheduled above whichever height is greater, never at the outer's height plus one.

**The three law tests.** Each is a statement of the join law with a widget around it, and each has the behavior-only twin among the seven that exist.

- *The outer switches to an inner that does not change.* `join_follows_the_outer_switch_and_reads_the_new_inner`: `d1` at 0, `d2` at 3, the outer switches to `d2` in occurrence 1 and nothing else fires. The joined event fires in occurrence 1 with 3, once, and does not fire in occurrence 2. This is the test that catches

a join built from `switch_hold` alone, which waits for `d2` to change before saying anything.

- *The outer switches and the new inner changes in the same occurrence, in both height orders.* `join_follows_the_new_inner_when_both_change_in_one_occurrence`: once with `d2`'s change below the outer's (both external, `d2` listed first in the input) and once with `d2`'s change above it (`d2` derived from the outer's source through three maps). In both, the joined event fires once with `d2`'s new value, and the behavior agrees after commit. This is the case that forces `on_outer` to wait for the inner's height.
- *The old inner is ignored after the switch, and in the switch occurrence.* `join_ignores_the_old_inner_after_a_switch`: the outer switches to `d2` and `d1` changes in the same occurrence; the joined event fires with `d2`'s current value and not `d1`'s new one; in the next occurrence `d1` changes again and the joined event is quiet.

A fourth, `join_is_prompt`, is the change law applied to the join: a dynamic derived from the join by `map` changes in the same occurrence as the join and reads the joined value.

This is the only join, and it is enough for everything the specification needs: `rebuild` over a block that returns a dynamic (a hold of the `rebuild`'s event initialised with the first build's result, then joined), the canonical cache idiom, and the dynamic consolidation of `for`, which is the same mechanism over a map of inners rather than one.

### `!Updated`

`!Updated: fn $a -> !a` is the prelude's name for a dynamic's change event, and it compiles to `Dynamic::event()`, which is a clone of a handle. It fires with the new value in every occurrence the dynamic changes and never at build: a hold's event is its input event, which did not fire because the hold was built; a derived dynamic's event is a `filter_map` or `combine` over its operands' events, which did not fire because the operand was read; a constant's event is `never`. Nothing is constructed and no widget is involved, which is why it is a plain function and not a widget. A program that wants a firing at build binds `@started`, which is a `rebuild` over a constant and takes a bind position, and merges it in.

The one thing worth saying about `!Updated` in the runtime is what it does not do: it does not dedupe. A hold set to the value it already holds changes, its event fires, and everything downstream recomputes; `$Distinct` is a `filter_map` that drops a firing equal to the last one it let through, with equality from the trait table, and a hold of the filtered event. Deduping in the runtime would need equality on every type, which `Value` does not have, and would make `!Updated` on a hold differ from the hold's input event, which the hold law says they are one and the same. ### The keyed list widget

A todo list with a half-typed edit in row three must keep that edit when a row is inserted above it. That is the whole requirement on `for`: a key that stays is never torn down, whatever happens to its neighbours, and a key that appears is built in the occurrence it appears and is part of the result in that same occurrence, birth firings included. `for $m:` is a widget over a dynamic map:

```
impl<K: ValueType + Ord, V: ValueType> Dynamic<PMap<K, V>> {
    pub fn list_blueprint<W: Blueprint>(&self, kind: Consolidation,
                                    block: impl Fn(&K, Dynamic<V>) -> W + 'static) -> ListWidget<K, V, W>;
}
pub enum Consolidation { Plain, Event, Dynamic }   // the compiler states the result kind
```

Its node holds one child per key, in key order, and the child for key `k` is built from `block(k, per_key)`, where `per_key: Dynamic<V>` is projected from the map: value `m[k]`, with a change event that fires

when the map changes and `m[k]` differs from before. A change to an entry's value therefore updates `$_` inside the block and does not rebuild it. (Whether `m[k]` differs is decided by the diff below; an entry whose subtree is shared is unchanged by identity, and an entry the diff reaches is compared with `V`'s equality from the trait table, or counted as changed for a `V` without equality, which is still correct.)

`PMap<K, V>` is the runtime's map: a persistent B-tree whose nodes are `Rc`s, with structural sharing between versions, and it is the representation of every Prism map value, not only the ones `for` reads. Two things force it. A map in a hold changes by `Insert $todos, k, v` and the like, and on an `Rc<BTreeMap>` that is a copy of the whole map whenever anyone else holds the old one, which the hold's own value side always does until commit; on a persistent B-tree it is O(log n) new nodes and the rest shared. And the list widget has to know which keys appeared, which went and which changed, and on two unrelated maps that is a walk over both, O(n) per change however small the change. Between two versions of one persistent map, the walk descends both trees together and skips every pair of subtrees that are the same `Rc`, so the diff costs O(changed · log n). The list widget runs that diff once per change of the map and drives the per-key projections from it, so a key the change did not touch costs nothing, not even a comparison.

That is as far as a value-only map goes. Derived functions over maps (`Filter`, `MapValues`, `Values`) still compute their result from the whole input on each change, O(n) per change, and their result shares nothing with the previous result, so the diff downstream of them is O(n) too. The full answer is a dynamic whose change event fires with a patch rather than a value, with the value side maintained by applying the patch: Reflex's `Incremental` (`holdIncremental`, `updatedIncremental`, `mergeIncremental` in the Spider sources) is that idea, and the element list, below, is the runtime's one instance of it. Making `Filter` and `MapValues` patch-to-patch functions, so that a `for` over a filtered map pays for what changed, is the next step when a benchmark asks for it; nothing a program can observe changes, because the value side is the same map.

On the map's change event at n, at the change's height h: keys that appear are built in n, each as a journaled build at h+1, under the construction rule, so a new block sees all of n (catch-up does this; the list widget does nothing special); keys that disappear are scheduled for teardown at commit, like a replaced `rebuild` child, so an event that fired inside a disappearing block earlier in n is still delivered; keys that stay are untouched. The node's children map is keyed by `Segment::Key(k)`, so a block's slots keep their paths across any change that keeps the key.

The result is consolidated by the kind the compiler states, and all three consolidations are prompt: the consolidated value in n reflects the keys after n.

- `Plain`: a `Dynamic<PMap<K, O>>` whose value is the map of block results and whose change event fires at n, with the new map, when the key set changes at n. The new map is computed at h+2, above the new blocks' builds.
- `Dynamic`: each block returns `Dynamic<O>`; the result is `Dynamic<PMap<K, O>>`, a join over a map. Its behavior queries every current inner; its change event fires at n when the key set changes at n or any current inner changes at n, with the whole map after n. The implementation is `JoinOuterEvent` with a map of inners in place of one: on a key-set change it subscribes to the new inners with catch-up and schedules its firing above all of their change heights; on an inner's change it fires with the previous map updated at that key, one persistent insert. Prompt on both, which is what makes `for` over a list of counters a dynamic of their counts with no lag.
- `Event`: each block returns `Event<O>`; the result is `Event<PMap<K, O>>`, firing at n with exactly the entries whose events fired at n. This is a merge over a dynamic set of members, and the membership is prompt: a key that appears at n is a member at n, and anything its block fires in

its creation occurrence is in the consolidated event at n.

The event consolidation is where the static cycle check and the runtime's height check have to agree, and they do. The merge's height is one more than the greatest among its current members, invalidated and recomputed when the membership changes, exactly as a switch's is. The only thing a block built at n can fire at n is pure code: `@started`, an inner `rebuild`'s event, a hold's change from one of those. A block that fires nothing at birth, which is every block whose result is a response from a slot it created (the slot was created at n and cannot receive anything at n), adds a member whose event will not fire this occurrence, and the merge's recomputed height is irrelevant to n. A block that does fire at birth adds a member that fires at a height above h; if the merge had not yet fired at n, its recomputed height is above that member's and it fires once with every entry, old and new; if the merge had already fired at n, because its old height was below h, the newborn's firing asks it to fire twice in one occurrence, which the manager's `record_once` refuses, and the eager walk at the membership change reports it as a cycle: the merge's new height is at or below the current height. That is the runtime's reading of the rule that `@ Delete on !started` as a row result is an error while the ordinary todo list is legal: in the todo list, `!deletes` feeds `$todos` through a hold, `$todos`' change rebuilds the `for`, and the new rows fire nothing at birth, so no edge runs from `$todos` to `!deletes` and the heights are consistent; with `!started` in the row, there is such an edge, and the merge would have to sit above the hold's change, which sits above the merge.

This is also what makes the interceptor's loop legal. An interceptor writes `!down <- for $slots: ... @ resp` and feeds `!down` back into `intercept`. The fan delivers `!down`'s entries to the child's slots at `!down`'s height plus one; the child reacts, perhaps rebuilding a `rebuild`; the child's list changes; `$slots` changes, at some height above the fan; the `for` builds new blocks for the new paths, each of which creates an outer slot with `@Element` and returns that slot's response event, a fan output whose height is one above the *outer* responses event and has nothing to do with `$slots`. The new member cannot fire at n, the merge's height does not depend on `$slots`' height, and there is no cycle. The `for` over `$slots` is a membership edge, not a data edge, and a membership edge becomes a data edge only when a newborn speaks.

Arrays under `for` are the same widget keyed by index, with the specification's warning attached: an insertion at the front changes every later key, which rebuilds every later block and resets its state.

## Elements, slots and paths

An `ElementValue` is plain data, a tag and its field values, so that two slots with equal elements are equal for state identity and so that a World can key a map by it. Elements are ordered by tag name, then by plain fields in declaration order, which is the language's ordering on an extensible variant. A path is the address a World sends to, and the counter's Increment button is `0/#0/0` under every implementation: the `rebuild` at position 0, its generation 0, the slot at position 0 inside it.

Both are keys in every hot map of the runtime (the memo, the element list, the fan's outputs, the World's tables), and both are compared far more often than they are built. Compared structurally, an element with a large field (a `Gui.Scene` whose draw list is the whole window, an `Fs.Contents` keyed by a long path) costs the field's size per comparison, and a `BTreeMap` lookup makes log n of those; a path costs its depth. So both are interned: built once through a weak table on the core, with the hash computed at construction and cached, so that two equal values are one allocation. Equality is then a pointer comparison. Ordering compares pointers first and answers `Equal` at once on a match, and otherwise compares structurally, which for two different paths stops at the first differing segment.

```
#[derive(Clone)]
pub struct ElementValue(Rc<ElementData>);      // interned: equal values share one Rc
pub struct ElementData { pub tag: TagId, pub fields: Box<[Value]>, hash: u64 }   // fields in declaration order
// PartialEq: Rc::ptr_eq. Ord: ptr_eq first, then tag order, then fields. Hash: the cached hash.

#[derive(Clone, Copy, PartialEq, Eq, PartialOrd, Ord, Hash)]
pub struct TagId(u32);                          // assigned in tag-name order, so TagId order is name order

pub struct TagInfo {
    pub name: &'static str,                // "Ui.Button"
    pub fields: &'static [&'static str],
    pub requests: Option<TypeId>,
    pub responds: Option<TypeId>,
    pub state: Option<TypeId>,
}
pub struct TagTable { infos: Vec<TagInfo>, by_name: BTreeMap<&'static str, TagId> }

#[derive(Clone, PartialEq, Eq, PartialOrd, Ord, Hash)]
pub enum Segment { Bind(u32), Gen(u64), Key(Value), Supplied(u32) }

#[derive(Clone)]
pub struct Path(Rc<PathNode>);                  // interned, like ElementValue
pub struct PathNode { parent: Option<Path>, last: Segment, depth: u32, hash: u64 }
```

Equality on `ElementValue` excludes the request event because the request event is not a field of the value: it is held by the slot, below. `TagTable` is the compiler's table, handed to the runtime at `build`; it numbers tags in name order, so the derived order on `TagId` is the order by tag name and an element comparison never looks the names up; the runtime consults it for which of the three parts a tag has and for the response and state types to check at the boundary. `Kind: fn Element -> Kind` in the prelude is a lookup in this table.

A `Path` is the chain of segments from the root, and the canonical spelling in transcripts is fixed by the conformance suite: segments joined by `/`, `Bind(i)` as `i`, `Gen(g)` as `#g`, `Key(k)` as `[k]` with the key printed as a Prism literal, `Supplied(n)` as `s/n` followed by the supplied widget's own segments, so a supplied widget whose body is one slot prints `s/0/0`. `Display` on `Path` prints exactly that, and both implementations print the same characters.

What takes a position. Only things that give the tree structure visible from outside take a `Bind` position: a slot, a `rebuild`, a `for`, an `intercept`'s captured child, and `@started` (which is a `rebuild`). They are numbered in build order within the nearest *addressable* node, which is the root, a `rebuild` generation, a `for` key's block, a captured child or a supplied widget. A plain `<-` of a widget block or of a widget function application makes a node, for ownership, but the node adds no segment: its contents are numbered in the enclosing addressable node's sequence, because a plain child lives exactly as long as its parent and is not a unit of teardown. This is the rule that gives the counter its paths: `@Button $"Increment"` at the first statement of `@main` is a lifted application, which is a `rebuild` taking position 0, whose generation `#0` holds the button's one slot at position 0, so `0/#0/0`; `@Label` the same at position 2; and `@Element Fs.File(path="/etc/motd")` with plain fields at the second statement of a file viewer is `1` with no generation, because no `rebuild` is involved. Holds and `let`s take nothing.

`Gen(g)` counts from zero per `rebuild` node and never reuses a number, so a rebuilt widget's slots get fresh paths and nothing addressed to the old generation can reach the new one. `Supplied(n)` numbers, per interceptor node, each widget the supplier built, in creation order, never reset; the slots inside it take their own segments under that prefix, so a supplied widget with a `rebuild` prints `s/n/0/#g/0`

and gets fresh paths per generation like any other. Paths compare lexicographically by segment, with `Bind` by index, `Gen` by number, `Key` by key order, `Supplied` by number, and the variants in that order; since at most one generation of a `rebuild` is in any committed list, the order between generations never matters to a World, and tree order is path order. A path is an interned node pointing at its parent's path, so every slot under one node shares that node's prefix, minting a slot's path is one allocation, and comparing two sibling paths compares their parents by pointer and then one segment each.

A slot is a record in its node:

```
pub struct SlotRec {
    pub path: Path,
    pub element: ElementValue,
    pub response: Option<Rc<FanOutput>>,        // if the tag responds: this slot's output of the enclosing fan
    pub state: Option<Rc<Supplied>>,            // if the tag has state: keeps the supplied widget alive
    pub request: Option<(Event<Value>, Rc<RequestSink>)>,   // if the tag requests: the event and its collector
    sink
}
impl WidgetBuilder<'_> {
    pub fn element(&mut self, element: ElementValue, request: Option<Event<Value>>) -> Result<ElementResult,
    RuntimeError>;
}
pub struct ElementResult { pub response: Option<Event<Value>>, pub state: Option<Dynamic<Value>> }
```

`element` is what `@Element` compiles to. It mints the slot's path from the node's prefix and the position counter, creates the response output if the tag responds (a selection on the enclosing fan, below), asks for the state dynamic if the tag has state (state supply, below, which is where the `Err` can come from), subscribes the request event to the collector if the tag requests, and pushes the record. It returns what the tag declares; in a catch-all arm the compiler asks for both and checks the per-slot types at the slot. The slot lives exactly as long as its node.

## The element list

A list that waited for commit to change would be a list one occurrence stale, and the counter's transcript would show the old label for a turn after every click. So a node's element list is a derived dynamic, changing in the occurrence a child is built, and its value side, which lags to commit like any dynamic's, is exactly the delayed edge that routing reads.

```
pub type ElementList = PMap<Path, ListEntry>;   // the persistent B-tree of the list widget section
pub struct ListEntry { pub element: ElementValue, pub slot: Weak<SlotRec> }
pub struct ListPatch { pub inserted: PMap<Path, ListEntry>, pub removed: PSet<Path> }
```

A node's own slots are fixed for its life. The compiler is bound by that: an `Element` step runs only inside `build`, never in an occurrence-time closure, and the lint's mark check refuses the other placement. The builder enforces it: `element` is a method on `WidgetBuilder`, and a builder exists only for the duration of one `build`. So the only sources of change in a node's list are its children that can change: a `rebuild`, whose list is the current generation's; a list widget, whose list is the union over keys; and an interceptor, whose supplied widgets come and go. A node's list is the union of its own entries and its children's lists (except a captured child's, which goes to the interceptor instead), and because every child's paths extend that child's segment, the union is disjoint and the map's order is tree order. For a node with fixed children this is a constant; under a `rebuild` it is the generations joined; under a list widget it is a map-join over keys.

The list's change event fires in the occurrence a child is built, which is the construction rule applied

to the list: the `rebuild`'s generation is built at h+1 and the `rebuild` node's list changes at h+2 with the new generation's entries, and each ancestor's list changes one height above its changing child's. The list's value stays the old list until commit while its change event has the new one; that is the ordinary dynamic contract, and routing reads exactly this delayed edge.

The change event fires with a *patch*, not a whole map. Rebuild the union at every ancestor on any change below would cost the subtree's size per ancestor, and a GUI pays that on its first click: one label deep in a window of a thousand rows would copy a thousand entries at every level up to the root. So a `rebuild` firing's patch is "remove the old generation's entries, insert the new generation's"; a list widget's is the union of the appearing keys' lists and the disappearing keys' paths; a parent's is the union of its changing children's patches, which is cheap because the paths are already distinct. Each node's list value is a `PMap` maintained by applying its patch at commit, which shares every untouched subtree with the previous value, so no World holding an older list forces a copy. The run call reports the root's patch; `Runtime::list` hands out the root's map. A program observes nothing of this: `$slots` in an interceptor is the map, and where Prism code reads its change event, the runtime applies the patch to the old map persistently, in time proportional to the patch, and the value is that new map.

## Routing

A button relabels itself on click: the label changes in occurrence n, the generation is rebuilt, and a click the World sent in that same occurrence was aimed at the old slot, because the old slot was in the list the World had. That click has to land on the old slot, which is still in the tree until commit, and a click aimed at the new slot in n has to be refused, because no World could have known its path. Routing therefore consults the list as the occurrence opened and nothing else, through a keyed fan, one per interceptor and one at the root:

```
pub struct ResponseFan {
    source: Event<Rc<[(Path, Value)]>>,          // the responses event, or the run call's inputs
    table: Behavior<ElementList>,                // the captured child's list, value side
    born: u64,                                   // the occurrence the fan was built in
    outputs: RefCell<BTreeMap<Path, Weak<FanOutput>>>,
    owner: Weak<Widget>,                     // the interceptor's node, named in errors
}
```

A `FanOutput` is an event with the fan as its upstream and height one more than the source's. When the source fires, the fan reads `table` with `query_for_tag` and looks each `(path, value)` up in it. A list's value side changes only at commit, so the read is the captured child's list *as the occurrence opened*, whatever has been built since; a path not in it is `UnknownPath`, which aborts the occurrence and names the interceptor; a path in it whose output has no subscribers is skipped at no cost; a path with a live output gets its value, once. The lookup against the opened list is the routing rule, and because the fan reads the list through a tag, the lookup is a delayed edge: `!down` may depend on `$slots` through a `for` and `$slots` may change because of what `!down` delivered, and no height runs backwards. Cost is per fired path, not per slot: a slot that receives nothing is never touched, and an occurrence that delivers nothing to a fan costs that fan nothing.

One case needs the stamp. A fan built in occurrence n belongs to an interceptor built in n, so every slot under it was created in n, and none of them was in any list a sender could have read. The list's value as a tag reads it in n is the list as built, since the node did not exist before n, and would route to those slots. So a fan whose `born` equals the current occurrence treats every lookup as `UnknownPath`, which is the rule that pure code firing at a path created in the same occurrence is an error. From n+1 on, the value side is the committed list and the stamp no longer matters.

A copy of every fan's table taken at open, by a walk over all fans, would give the same answers at a cost proportional to the number of interceptors in every occurrence, including occurrences that deliver to none of them. The value side already holds the opened list, so there is nothing to copy.

`outputs` is bounded by the live slots under the fan, not by every slot ever created there. The `FanOutput` is owned by its `SlotRec`, and `SlotRec`'s `Drop` removes its path from the fan's map, so an entry leaves the map in the same commit that drops the slot; the fan is reached from the slot through a `Weak`, and a fan already gone means nothing to remove. The runtime is bound by this and the checker asserts it: after every occurrence, the fan's map has exactly one entry per slot in the captured child's list that responds. Consolidating on the next lookup, the way a subscriber list does, would also keep the count bounded, but it would leave dead entries in the map between lookups and make the checker's count a lie for an occurrence.

## Request collection

A slot with a `requests` field holds the program's event. The run call has to report, per slot, the values it fired in this occurrence, in order. The collector is the `requests` vector on the core, and a sink per slot feeds it. This replaces the delegate callback in today's crate, through which the test backend hears event firings and which hands it the node's live slots; with the collector, nothing outside the runtime ever holds a runtime object.

```
struct RequestSink { path: Path, core: Rc<Core> }
impl EventCallback<Value> for RequestSink {
    fn event_fired(&self, _: &Runtime, v: Value) { self.core.requests.borrow_mut().push((self.path.clone(), v))
    }
    fn already_fired(&self, v: Value)          { self.core.requests.borrow_mut().push((self.path.clone(), v)) }
    fn invalidate_height(&self) {}
}
```

At slot creation, if the tag requests, `element` subscribes the sink to the request event, with catch-up, so a request event that fired earlier in the creation occurrence (`Read on !started` in a block built mid-occurrence; `@started` fires at the done height, which is above the build) is recorded too. The sink is owned by the slot record, so it dies with the slot; the request event is held strongly by the slot so that laziness does not disconnect it when nothing else listens. At close, `run` groups the vector by path in the order given under the occurrence's close phase and clears it.

A request from a slot that was torn down in the same occurrence is still in the vector, with its path, and is reported. There are two ways this happens: the slot's node was replaced by a `rebuild` firing or a key's removal later in the occurrence, in which case the node was alive when the request fired and is dropped at commit; or the slot was created and torn down in one occurrence, a `rebuild` built at h+1 and rebuilt again at some h' > h because its dynamic changed twice in n through two different paths, in which case the first generation's `@started` request was collected before the second build. In both the World performs the request if it can and discards any response, because the slot is gone and the next list says so. The collector never filters by liveness, which is why it needs no reference to the tree.

The World never sees an event; it sees a list of what fired.

## State supply

A `state` element has a value in the occurrence it is created, and the value comes from whoever stands outside: an interceptor's supplier, or the World's oracle at the root. The mechanism is the same at every level and has no table and no chain to walk.

Every node knows its nearest enclosing *supply context*, a field set by the builder when an `InterceptBlueprint` builds its child and otherwise inherited from the parent:

```
pub struct SupplyContext {
    supplier: Supplier,
    memo: RefCell<BTreeMap<ElementValue, Weak<Supplied>>>,
    owner: Weak<Widget>,                      // the interceptor's node, or the root
    next_supplied: Cell<u32>,                     // the Supplied(n) counter
}
enum Supplier {
    Widget(Dynamic<Rc<dyn Fn(&ElementValue) -> Rc<dyn DynBlueprint<Dynamic<Value>>>>>),
    Oracle,
}
pub struct Supplied { node: Option<Rc<Widget>>, dynamic: Dynamic<Value>, trigger: Option<EventTrigger<Value>>
}
```

When `element` is called for a tag with state, it asks the context for the slot's dynamic by element value. The context looks in its memo first: if a `Supplied` for an equal value is alive, it returns it, the slot takes a strong reference, and *the supplier is not consulted*. A new slot whose value already has a live supplied widget joins that widget whatever the supplier dynamic says at that moment; the supplier is read only for a value no live supplied widget serves. This is value identity inside an interceptor held by construction, and it is what the migration interceptor relies on: two child slots for one key never disagree, and a key migrates to a new backend only when its last reader is gone and a new one arrives.

On a miss under an interceptor, supplying means reading the supplier function from its dynamic right now (`query_for_tag`, the value as the occurrence opened, which is what "a slot's policy is fixed at creation" means), applying it to the element, and building the returned blueprint as a child of the interceptor's node, then and there, in the same build pass, as a journaled build with the nested-transaction boundary. The widget takes the next `Supplied(n)` position in the interceptor's node, and its slots take their own segments under it. The order is build, then memoise: the supplied widget is built to completion under the nested-transaction boundary, and only when its build returns with a result dynamic is the `Supplied` record made, memoised weakly, and handed to the asking slot strongly. A supplier build that fails (a cycle at a placeholder close inside it, a state tag it asks for that the next context out cannot answer, a panic in one of its build-time closures) is truncated like any failed build, leaves nothing in the memo, and surfaces to the asking slot as the `Err` of `element`, with the supplier's error as the cause: the asking slot's own build then fails, and the failure is contained or fatal by the ordinary rule, under a `rebuild` as an error node and in occurrence zero as the `Err` of `build`. The slot never sees a half-built `Supplied`, and a later slot with the same value asks the supplier again, since the memo has no entry for it. Because the supplied widget is built in the interceptor's node, its own elements go into the interceptor's list and its own state slots ask the *next* context out, which is how `\|e\| @Element e` forwards a state element one level: the supplied widget is one `@Element` with the same value, whose slot asks the outer context, and so on up to the root. A supplied widget's result dynamic is read by the child's slot in the child's creation occurrence, and it has a value then by the same rule applied one level up.

At the root the supplier is the oracle. Supplying is: call the oracle with the element value to get the value before this occurrence; make a *root state cell*, an external trigger and a hold of it initialised from the oracle's answer; memoise it by element value in the same memo. The run call's pushes are deliveries to these triggers at height 0. A slot created in n reads the oracle (the value before n, from the World's previous snapshot) and then, if n's pushes include its value, receives the push in n and ends n holding the new snapshot, exactly like a slot that existed before n. Two slots with equal

values share one cell and therefore one dynamic. A state tag the World does not answer is a build error, `UnsuppliedState`, raised at the slot's creation with the element and the slot's span; the oracle's signature cannot express refusal because the specification says it cannot fail, so the world core checks the tag against its table before the call and the runtime checks the answer's type after it, and either failure is the error in the occurrence the slot is created, contained to a `rebuild` build when it happens in one and fatal to `build` when it happens in occurrence zero.

The memo holds `Weak<Supplied>` and the slots hold strong references, so the teardown of the last slot with a value frees the supplied widget at the same commit; nothing scans the memo. A supplied widget torn down this way is an ordinary node drop under the interceptor's node, so its elements leave the interceptor's list in the same patch, and the outer context sees the interest go the same way.

### `InterceptBlueprint`

Forwarding is manual, so the interceptor has no policy to implement: `@Forward`, the sandbox, the VBox and the cache are Prism programs over `intercept`, `for` and `@Element`, and the runtime does not know they exist. The runtime keeps a fan, a supply context, a captured child and its list, and nothing else.

```
pub struct InterceptBlueprint<W: Blueprint> {
    child: W,
    responses: Event<Rc<[(Path, Value)]>>,
    supplier: Dynamic<Rc<dyn Fn(&ElementValue) -> Rc<dyn DynBlueprint<Dynamic<Value>>>>>,
}
impl<W: Blueprint> Blueprint for InterceptBlueprint<W> {
    type Output = (W::Output, Dynamic<ElementList>);
    fn build(&self, builder: &mut WidgetBuilder) -> Self::Output;
}
```

Its `build` does four things and nothing else. It makes a `ResponseFan` with `responses` as the source, the captured child's list behavior as the table, and the current occurrence as its birth stamp. It makes a `SupplyContext` with its supplier and its own node as owner. It binds the child, at a `Bind` position of its own node, with a builder whose fan and context are these, so that every slot the child creates, at any depth that does not pass another interceptor, gets its response output from this fan and its state from this context. And it captures the child's element list: the child node's list dynamic is the interceptor's second result and does *not* join the interceptor's own list. The interceptor's own list holds only the slots its node makes with `element`, which includes those made by supplied widgets.

That is the entire mechanism.

The root is an `InterceptBlueprint` the runtime makes itself in `build`: its child is `@main`, its responses event is the run call's input trigger, its supply context is the oracle, and its captured list is the one the run call returns.

Two alternatives show what this buys. Suppose state supply were a table: the runtime keeps, per interceptor, a map from tag to supplier, and a slot walks outward through the interceptors until a table answers. Then an interceptor that wants to forward a state element with a rewritten path needs a rewrite entry, one that wants to cache needs a memo entry, one that wants to supply from a dynamic needs a dynamic entry, and the table has become the second language the specification refuses. With the supplier being a function to a blueprint, the rewrite is `@Element (Rewrite e)`, the cache is a hold around `@Element e`, the dynamic supplier is the dynamic the interceptor was given, and the runtime implements one thing: build the widget here, now, and memoise by value. Suppose, instead, that responses were routed by the runtime walking the tree: an interceptor registers a filter,

the runtime applies filters as it descends. Then an input the interceptor wanted to rewrite or hold back would need a mode, and simultaneity between the interceptor's own responses and the forwarded ones would need a rule the runtime chose. With one responses event per interceptor, the merge is `|` in the interceptor's own code, the rule is the one the language already has, and the runtime routes against one snapshot and does nothing else.

## The canonical node structure of a lifted application

A hand-written widget in the harness and the compiled form of the same Prism must produce the same paths and the same transcript, so the structure that a lifted widget application builds is fixed here, once.

`!x <- @Button $label`, where `@Button` takes a plain label, is `rebuild $label: @Button _` with `!x` the `!Switch` of the `rebuild`'s event. It builds, in the enclosing node:

1. A `Rebuild` node at the next `Bind` position, over the dynamic `label.map(|l| button_widget(l))`, where `button_widget` is the widget function's body closed over the plain label. The generation `#g` is the body's node; the body's own position-takers are numbered inside it from 0.
2. In the enclosing node's values, `x = dyn_event.map(|out| out.event).switch_hold()`, owned by the enclosing node through its environment vector.

For a widget whose result is a dynamic, step 2 is `Dynamic::hold(first, dyn_event.map(|out| out.dynamic)).join()`, where `first` is the first build's result, which the builder has in hand because the first build happens inside the `Rebuild`'s own build and its output is returned from it (the `rebuild` returns both its event and the first build's output, as a pair, for exactly this use). For a widget whose result is plain, `$x <- @Label $text` binds a `$` holding the first build's value, and step 2 is `Dynamic::hold_first(dyn_event.map(|out| out.value))`: a hold whose initial value is the first firing, which has happened by the time the hold is built, and which changes to each rebuild's value in the rebuild's occurrence. A hold built from `first` and the event would be the same thing with a second copy of the value; `hold_first` exists so that the compiler emits one step and the text form prints one node. For a bundle, field by field by the kind of each field. For an element with a dynamic where a field goes, the same with `@Element` inside the generation: `@Element Ui.Label(text=$t)` is a `Rebuild` over `t.map(|t| element_widget(Ui.Label { text: t }))`, and its generation's node has one slot at position 0, which is why the label is `2/#g/0`.

The path consequence is the same in all three cases, because the `rebuild` is the same: a lifted application takes one position, its generation holds the body's slots from 0, and the bound name, whether `!x`, `$x` or a plain-valued `$x`, takes none. A harness fixture for `@Label $text` with a plain result is a `Rebuild` over the label whose generation makes one slot and returns `unit`, and the transcript shows `n/#g/0` with a fresh `g` on every text change.

A hand-written `Button` in the harness that wants to match this must therefore not be "a node with a slot and a `rebuild` inside it"; it must be the `Rebuild` itself, built with `rebuild` over the label, with the slot made inside the generation. The harness's fixtures are written that way, and the compiled `@Button` from the GUI library must match their transcript byte for byte.

## Compiled widgets

Three places build widgets at occurrence time: a `rebuild` rebuilding, a `for` key appearing, and a supplier answering a state slot. Each hands the runtime a blueprint it has never seen, and the rebuild path must not care whether it got a block of `@main`, a widget function's body, or something a hand-written Rust World made. So every compiled widget block is one Rust type, `IrBlueprint`,

implementing `DynBlueprint<Value>` (or `DynBlueprint<O>` for the typed result the block declares), and `Rc<dyn DynBlueprint<Value>>` is what all three places take. An `IrBlueprint` is a reference into the compiled program plus the values it captured:

```
pub struct IrBlueprint {
    pub program: Rc<Program>,      // Program { functions, blocks, tags, constants }, shared by every widget of
    one program
    pub block: BlockId,            // which of program.blocks this widget builds
    pub captures: Rc<[Value]>,     // the values the block captured from its enclosing node
    pub span: Span,
}
```

Its `build` walks `program.blocks[block]`'s steps in the group order the resolver computed, with an environment vector for the block's values. The steps are the block's, not the widget's: two `IrBlueprint`s over one block with different captures are two nodes from one description, which is what "a blueprint is a description" means at the Rust level.

```
enum Step {
    Let { dst: usize, expr: PlainExpr },                      // build-time plain value
    Bind { dst: usize, widget: BlueprintExpr },                  // child widget: builder.bind (no segment)
    Hold { dst: usize, initial: usize, event: usize },        // Dynamic::hold
    HoldFirst { dst: usize, event: usize },                   // Dynamic::hold_first
    Map { dst: usize, src: usize, f: ClosureId },             // filter_map / map
    Tag { dst: usize, event: usize, dynamics: Vec<usize>, f: ClosureId },   // tag_map_n
    DMap { dst: usize, dynamics: Vec<usize>, f: ClosureId },  // map_n
    Leftmost { dst: usize, srcs: Vec<usize> },
    SwitchHold { dst: usize, src: usize },
    Join { dst: usize, src: usize },
    Updated { dst: usize, src: usize },                       // Dynamic::event
    Distinct { dst: usize, src: usize, eq: ClosureId },
    Started { dst: usize },                                   // takes a position
    Dyn { dst: usize, dynamic: usize, block: BlockId, captures: Vec<usize> },  // takes a position
    For { dst: usize, map: usize, block: BlockId, captures: Vec<usize>, kind: Consolidation },   // takes a
position
    Element { dst: usize, tag: TagId, fields: Vec<usize>, request: Option<usize> },   // takes a position
    Intercept { dst: usize, child: usize, responses: usize, supplier: usize },        // takes a position
    Placeholder { dst: usize, kind: PlaceholderKind, span: Span },   // add_cyclic_*
    Close { placeholder: usize, src: usize },                        // CyclicEvent::close / CyclicDynamic::close
    Return { src: usize },
}
```

Every step is one call on the builder or one constructor on an event or dynamic, with operands resolved to environment indices. Occurrence-time expressions compile to closures that the corresponding graph node calls, and a closure is a function id into `program.functions` plus a capture list resolved to indices into `captures`, which is copied into the node at build. A `Dyn` or `For` step makes its child widgets as `IrBlueprint { program, block, captures: this node's values at the listed indices plus the rebuild value, span }`, so the three occurrence-time build sites and the compiler agree on one constructor. Typed-result blocks (a supplier's, returning `Dynamic<Value>`) are `DynBlueprint<Dynamic<Value>>` and are downcast nowhere, because the compiler chose the type at the call site. The adapter goes; the blanket impl stands in for it.

## Closures and ownership

A compiled closure captures by index into its own node's values, never by pointer into a sibling's node. The values it captures are either plain data (cloned) or handles to events and dynamics (an `Rc` clone). The handle edges run in one direction: a closure captures things bound *earlier* in group order,

which are things built earlier, and the graph edge from a source event to the node that subscribes to it is a `Weak` in the source's subscriber list. So every strong edge points from later to earlier in build order, every reverse edge is weak, and there is no strong cycle through the heap. The widget tree holds every node strongly from the root down, so everything reachable is owned, and when a node is dropped everything it built is dropped unless a parent holds a result handle, in which case the result keeps working as the specification says: an event that never fires again, a dynamic at its last value.

The exception is a cyclic group, where a later binding is referenced by an earlier one. The group is closed through placeholders, and the placeholder is the one edge that points from earlier to later, and it is weak. The node owns the strong ends of every loop it closes in its `cyclic_events` and `cyclic_dynamics` slots, and it owns them as a unit: a group's placeholders are added before the group is built and closed after, and the compiler's lint checks that every placeholder is closed exactly once and that nothing samples one before its close. Dropping the node drops the slots, the loop's only strong roots, and the loop goes with it. A cyclic dynamic whose node is gone but whose handle a parent still holds keeps its last value, because teardown writes the dynamic's current value into a cell the computation falls back to when the slot no longer upgrades.

Two more rules keep this true across the constructs that build at occurrence time. A `rebuild` block or `for` block compiled as an `IrBlueprint` captures from its *enclosing* block's environment, which is the enclosing node's values; the child node then holds handles up into its parent, which is the direction that is allowed. And a supplied widget is built in the interceptor's node and captures from the supplier closure, which was built in the interceptor's block; again handles point up. A child never holds a strong handle to anything in a sibling or in its own descendants, because nothing in the language can name those.

**The recursion depth guard.** A plain function in Prism may recurse without bound, and a host stack overflow is not a panic in Rust; it ends the process. The closure compiler emits, at the entry of every compiled function body, an increment of the core's `depth` counter and a check against a limit, and at exit a decrement; past the limit it raises the runtime's `PrismPanic` with kind `Overflow` and the function's span, which the action loop catches and turns into the occurrence's `RuntimeError`. The limit is the implementation's (a few thousand frames, sized to the host stack with margin) and is a field on the core a World may set; the specification requires only that a conforming implementation neither hangs nor dies. Build-time functions run inside `build`, where the same counter guards them and a failure is a build error, contained or fatal by the usual rule.

## An occurrence traced through the target

Take the counter from the worked examples, built as `@main`, and take occurrence 1, in which the World delivers a click to the Increment button. The list the World received after occurrence 0 was

```
0/#0/0: Ui.Button(label="Increment")
1/#0/0: Ui.Button(label="Decrement")
2/#0/0: Ui.Label(text="Count: 0")
```

*Open.* `run` increments the counter to 1. No quarantine. The root list's committed value is the three paths above, each with a weak pointer to its slot.

*Fire.* `responses = [(0/#0/0, click)]`. The path is in the committed root list; the tag `Ui.Button` responds with `Click`; the value is a `Click`; the path has not been given a response yet. The root fan is scheduled at height 0 with the one entry. No pushes.

*Propagate.* Height 0: the fan fires; its output for `0/#0/0` is the response event of generation 0 of the `rebuild` that is `@Button $"Increment"`, and it has a subscriber, the `switch_hold` that is `!increment`,

which adopted this inner at occurrence 0's commit; the output fires at 1 and the switch at 2. `!delta` is `leftmost` over two maps of the two buttons' clicks, at 3, and fires with `+1`. `$count + !delta` is a `tag_map` over `!delta` reading `$count`'s behavior, which is 0, the value before the occurrence; it fires `1` at 4. `$count` is `Dynamic::hold(0, that)`, so its change event is that event, and the derived `"Count: {$count}"` is a `map` of it at 5, firing `"Count: 1"`. The `@Label`'s `Rebuild` subscribes to that map; its `RebuildEvent` schedules a rebuild at 6. Height 6: the rebuild builds generation 1 of the label block at 7, journaled under the `rebuild` node at `Gen(1)`: the block's one step is `element(Ui.Label { text: "Count: 1" }, None)`, which mints path `2/#1/0`, creates no response output and no state, and pushes the slot. The `rebuild` node's list recomputes at 8 and emits the patch: remove `2/#0/0`, insert `2/#1/0`. `@main`'s node is the root interceptor's captured child; its list emits the same patch, and that list is the one the root captured. The old generation's node goes onto the commit list. The request collector has recorded nothing, because no slot in this program requests.

*Commit.* The hold writes `1`. The old label node is dropped, which drops its slot. The list patch is applied at every node it passed through: the root list's value is now the three entries with `2/#1/0: Ui.Label(text="Count: 1")`, and `2/#0/0` is gone from it, which is why a later response to that path is `UnknownPath`.

*Close.* The outcome is the patch (remove `2/#0/0`, insert `2/#1/0`), an empty request vector, `unit`, and no errors. The World applies the patch to its own copy, redraws the label, and waits for the next click.

Had the World sent the click to `2/#0/0` in occurrence 2 instead, it would have been sending to a path not in the list it received, and `run` would have returned `UnknownPath` before scheduling anything. Had a closure in the chain panicked at height 4, the commit list (the hold write) and the journal (empty at that point) would have been dropped, the occurrence number left behind the stale caches, and the hold would still read 0. ## Invariants

A runtime with no scheduler, no dirty tracking and no collector is held together by invariants instead, and the point of this section is to write them down with who is bound by each and what enforces it. The rule for every one is the same: it is either enforced by construction, in which case there is a sentence saying what construction, or it is checked, in which case the debug checker below asserts it after every occurrence and the conformance suite has a test that would fail if it broke.

**At-most-once.** Every event fires at most once per occurrence; every dynamic's change event fires at most once; every closure runs at most once per occurrence. *Bound:* the runtime. *Enforced:* `record_once` on every subscription manager. The cache is written once per occurrence and every later reader, by push or by catch-up, reads it. A second firing of a node in one occurrence is impossible because the manager will not compute twice; the only way to ask for one is a node scheduled twice at different heights, which `schedule`'s assertion turns into a runtime bug report if the second is at or below the current height and `record_once` turns into a no-op otherwise.

**Height order.** An action runs only at a height strictly above every height already run in this occurrence, and every derived event's height is strictly above its prompt sources'. *Bound:* the runtime and every event implementation. *Enforced:* `schedule`'s comparison; the `height` method's `+ 1`; the invalidation rule, which keeps cached heights true after a retarget; the eager walk at every edge creation, which refuses an edge that would make a height inconsistent. The checker verifies that the heights actually run were strictly increasing.

**Delayed edges run at commit.** A hold's value changes only in the commit batch; a switch's inner changes only in the commit batch; a replaced node is dropped only in the commit batch; a list-widget key and a supplied widget are removed only in the commit batch. *Bound:* the runtime. *Enforced:*

every one of these is a `Commit`, and nothing else writes to a hold's value, a switch's inner or a node's children during propagation. This is the tag law, the non-promptness of `!Switch`, and the "replaced node keeps working" clause, all from one rule. The checker samples every hold before and after the occurrence and asserts the value changed if and only if its event fired, and to the event's value.

**Dynamic agreement.** For every dynamic, after every occurrence, the behavior's value equals the change event's value if it fired, else the previous value. *Bound:* every constructor of a `Dynamic`. *Enforced:* by construction for `hold`, `constant`, `map`, `map2`; by the `on_outer` design for `join` and the map-join; and by the checker, which registers every dynamic built under it and asserts the equation after every occurrence. This is the invariant `Dynamic::new` is marked `unsafe` to protect, and the checker is what makes it safe to unmark.

**Construction sees all of n.** A node built at any point in occurrence n ends n in the state it would have been in had it been built before n's inputs. *Bound:* the runtime. *Enforced:* `subscribe` returning the already-fired value; `MainSubscription` routing a firing to the catch-up path when the subscriber's height has passed; holds and switches deferring to commit on catch-up; `rebuild` building the already-fired value first; the collector's sink subscribing with catch-up. The checker cannot state this directly; the law tests state it, one per subscriber kind.

**Routing against the opened list.** A response addressed to a path is delivered to the slot at that path in the list as the occurrence opened; a path not in that list is an error that aborts the occurrence. *Bound:* the runtime, and the World and every interceptor, which may only send to paths in the list they last received. *Enforced:* every fan looks paths up in its child's list through a tag, whose value side changes only at commit, and a fan built in the current occurrence routes nothing; the run call checks every path against the committed root list before scheduling anything. The checker copies each fan's committed list before the occurrence and asserts after it that every delivered path was in the copy.

**The diode.** Inputs cause list changes; list changes never cause inputs. *Bound:* the runtime, the World, and every interceptor. *Enforced:* for the runtime, by the fan's lookup being a tag read of the list, a delayed edge, so that no prompt path leads from a list change to a response event; for the World, by the structure of the world core's turn loop, which has no code path from an outcome's patch to a run call's inputs except through a component's own event or a `Defer` the program fired, so a World built on the core cannot answer a list change with an input, and by the Test World's `send` steps, which are script lines a list cannot write; the harness's two boundary checks are the test of it for a World written by hand; for an interceptor, by the cycle check, which rejects any prompt path from a child's list change to the interceptor's responses event, and which admits the membership edge of a `for` over the list because a newborn block fires nothing a response could cause.

**The creation rule.** No slot receives a response in the occurrence it was created, and no state slot lacks a value in that occurrence. *Bound:* the runtime, the World, every interceptor. *Enforced:* the first by routing against the opened list, since a slot created in n is not in it; the second by state supply at slot creation, which cannot return without a dynamic or an error. A World that answers in the request's occurrence is impossible, since the request leaves the run call only after commit.

**Equality of state.** Two slots with equal element values under one supply context hold the same dynamic. *Bound:* the runtime and every supplier. *Enforced:* memoisation by element value in every supply context, consulted before the supplier, with the root cell shared the same way. The checker walks the list and asserts that equal-valued state slots under one context share a dynamic by pointer.

**Ownership is a tree.** Every event, behavior, dynamic and node is owned, directly or through handles, by exactly one node, and the root owns everything. *Bound:* the runtime, the compiler and

every blueprint written in Rust. *Enforced:* the direction rules on captures, weak subscriber lists, weak placeholders, the compiler's lint on capture lists. The checker, in tests that drop a subtree, takes weak pointers into it beforehand and asserts none upgrades afterwards.

**Stale caches are unreadable.** No per-occurrence cache written in occurrence n is read in any later occurrence, on success or on error. *Bound:* the runtime. *Enforced:* the occurrence stamp on every cache, compared against the core's counter on every read; the counter is incremented at every open, including after a failed occurrence. The checker asserts after every occurrence that no manager's stamp is ahead of the counter.

**Rollback leaves no trace.** After `Err`, the graph equals the graph after the previous `Ok`. *Bound:* the runtime. *Enforced:* buffered commits, the append-only tree during propagation, the journal, the nested-transaction truncation in every build. The checker, after an injected failure, compares every hold, every switch's inner, every node's children and the root list against the snapshot taken before the occurrence.

**One call, one occurrence.** The runtime opens an occurrence only inside `run` or `build`, and exactly one per call. *Bound:* the runtime. *Enforced:* there is no other code path to the loop; `schedule_trigger` is `pub(crate)` and called only from `run`. The checker counts occurrences and compares with calls.

**The checker** is a debug-only `Invariants` struct that a test attaches to a core. It registers every dynamic, hold, switch and fan at construction (through a hook the constructors call under `cfg(debug_assertions)`), snapshots their observable state before `run`, and asserts the invariants above after. It runs inside every runtime unit test and inside the Test World under the conformance suite in debug builds, so that any sentence of the specification an implementation drifts from fails a test that names the sentence. It is the one piece of test scaffolding kept forever.

## Testing the runtime

The runtime is tested at two levels, and the levels test different things. The unit tests drive the graph directly in Rust, the way the 38 do today, because a law about `switch_hold` or `join` is easiest to state against the combinators with no program in the way. The harness tests drive the run call with Rust-defined tags, the way a World does, because routing, paths, state supply and interception are statements about what crosses the boundary, and a test that reads anything else is testing the implementation's structure rather than the specification. Neither level touches Prism source; that is the conformance suite's layer, where `fired` and `quiet` catch an input that went nowhere because a pattern failed to match, and the twelve integration programs wait for it.

**The harness.** A small Rust World over `Runtime::build` and `Runtime::run`, with a tag table of a dozen Rust-defined tags (a button that responds, a label, a file that requests and responds, a contents element with state, `Defer`), an oracle backed by a `BTreeMap<ElementValue, Value>` the test sets, a turn loop that answers `Defer` and nothing else, and fixtures that build widgets with the same builder calls the compiler emits, in the canonical node structure. After every occurrence the harness runs the invariant checker and two boundary checks of its own: every path it delivered to was in the list it had received before the occurrence, and every path the outcome's patch inserts has a prefix that changed in this occurrence (a `Gen`, `Key` or `Supplied` segment), which catches paths drifting under an unrelated rebuild; and the previous list with the patch applied equals `Runtime::list()`. It drives only through plain data: no triggers, no `query_for_tag`, no node handles. A test that needs a value reads it from an element. Its transcript printer is the one the world core takes over, kept in one small file for that reason.

**The scenario tests.** Each names a sentence and what it would catch if it failed. The third column says where the test lives: *suite* means the conformance chapter owns it, as a law test, an integration program or an entry in its runtime-test list, and this chapter only names it; *harness* means it needs something a Prism program cannot write (a Rust-defined tag, an injected panic, a count of internal deliveries, a quarantine input) and lives only here.

| Scenario | What it catches | Where |
|---|---|---|
| A nested `rebuild` whose inner rebuilds in the same occurrence as its parent | The rebuild path losing its node; a rebuild needing anything but the ready event | suite |
| A cyclic event closed in terms of itself promptly, on a thread with a timeout | A cycle hanging instead of returning kind `Cycle` with the loop's spans, at the close | suite |
| A switch adopting an inner that depends on the switch | The cycle found lazily in the next occurrence instead of at the commit of the switching one; the owner path missing | harness |
| An interceptor whose responses event depends promptly on its child's result event | A prompt loop through `intercept` slipping past the cycle check | suite |
| An interceptor whose responses are a `for` over its own child's list, forwarding everything | A false cycle from the membership edge; the sandbox and the VBox rejected | suite (`@Forward`) |
| A `for` whose row result is `@Delete on !started`, fed back to the map | The membership edge turning into a data edge without a cycle report | harness (the compiler refuses the source; the runtime's report needs a hand-built widget) |
| VBox-style hit-testing: tag a pointer with the opened list, route by the result | A false cycle on legal layout code | suite (the GUI library) |
| The relabelling button: label change in n, click in n, click in n+1 (timeline below) | Routing against the live list; the old generation torn down early; the switch turning prompt | suite (chapter 2's program) |
| A slot removed in n still receives a response addressed in n, and is gone in n+1 | Teardown at the wrong point of the occurrence | suite |
| Rebuild an unrelated sibling; insert at the front of a `for` over a map; rebuild a `rebuild` twice | Paths drifting under a sibling's change; keys losing their paths when entries shift; a generation reused | suite for the insert; harness for the sibling and the double rebuild |
| A thousand slots, one response; count fan deliveries | Per-slot filtering coming back in place of the fan | harness |
| The three `join` law tests and `join_is_prompt` | Event and behavior disagreeing; the join waiting for a change instead of taking the current value | suite |

| Scenario | What it catches | Where |
|---|---|---|
| Root build: count the `rebuild` event's and `@started`'s firings over several occurrences | Occurrence zero missing; either firing twice | suite |
| `@started` inside a `rebuild` block rebuilt in n | `@started` not firing in the rebuild occurrence | suite |
| A child under `rebuild` and `!Switch` whose block fires in its build occurrence | The switch turning prompt; the parent must not see it | suite |
| A `for` that gains a key in n, with the block returning a plain value, a dynamic, and an event that fires at birth | Any consolidation lagging; the newborn's birth firing missing from the event consolidation | suite for the event kind; harness for all three in one fixture |
| A state slot created in the occurrence of a push to its value | The push missing the new slot; the oracle answering after the push instead of before | suite |
| Two child slots of equal value under one interceptor, the supplier changed between them, then one torn down | Two supplied widgets; the second slot consulting the supplier; teardown with the first slot instead of the last | suite |
| A request fired by `@started` in a block that is torn down in the same occurrence | The request missing from the outcome; the outcome without its path | suite |
| An injected panic at height 4 of a six-height occurrence, then a normal occurrence | Rollback leaving a hold written, a node built, or a cache readable | suite's runtime-test list; injected from Rust |
| A `rebuild` build that fails at the close of a placeholder, inside an occurrence that otherwise succeeds | The occurrence aborted instead of continuing; the `rebuild`'s event firing; the `Error` element missing; a commit or a collector entry from the failed build surviving | harness |
| A quarantine instruction after a switch cycle, with a response to a path under the owner in the same call | The owner not replaced; the error element absent; the response reported as unknown instead of discarded | harness (quarantine scripts are Rust-only) |
| A plain `Blueprint` boxed as `Rc<dyn DynBlueprint<_>>`, and a `rebuild` that rebuilds into a different blueprint type | The erased blueprint path | harness |
| Several `run` calls in a row, counting occurrences | The runtime opening an occurrence on its own | suite |
| A function recursing past the depth limit in an occurrence-time closure, and in a build-time one | The process dying instead of an `Overflow` error; a build failure not contained | harness |

**The timeline.** The relabelling button is the one scenario worth writing out, because it exercises

routing, the switch and teardown at once. `@Button $label` is a `rebuild` over the label with an element inside and a `!Switch` over the generations' responses. The label changes in occurrence n; call the old slot `k1` (generation 0) and the new one `k2` (generation 1).

| | list's change event | list's value (opened) | routable | `!Switch` follows |
|---|---|---|---|---|
| n | `k2` | `k1` | `k1` | `k1`'s response |
| n+1 | none | `k2` | `k2` | `k2`'s response |

A click the World sends in n was aimed at `k1`, because the World read the list after n−1, and it lands on `k1`: the old generation is still in the tree in n, its response output is the switch's current inner, and the click reaches `!click`. A response sent to `k2` in n is `UnknownPath`. At n's commit the old node is dropped and the switch adopts `k2`'s response. In n+1 the World has the list with `k2`, sends there, and the click arrives through the new generation. The harness asserts all four cells of both rows.

## The gap

In the order worth closing. Each step keeps the suite green and adds the law tests it enables; none is sized larger than a model can be handed with this chapter and the previous step's code.

1. **`Rc`/`RefCell`, the shared core, re-entry flags.** Mechanical, touching every file, so done first and alone, with no behaviour change. Every graph object gains its `Rc<Core>`; `Runtime` gets its hand-written `send`; `Value` gains `I64` and `U64` and loses the 32-bit variants. Closes the known issues "`Arc`/`Mutex` everywhere", "Events are not tied to a runtime" and "`Dynamic::new` is `unsafe` for a logical invariant". *Done when:* no `Arc`, `Mutex`, `send` bound or `Sync` bound remains in `src/` outside the one `unsafe impl`; the cycle tests assert the new messages; a graph built on one thread runs on another after the runtime is moved; `Dynamic::new` is `pub(crate)`.
2. **The commit list, the journal, stamped caches, and rollback.** `propagate` becomes `pub(crate) fn run_occurrence(&mut self) -> Result<(), RuntimeError>` with the panic path returning `Err` and the stale-cache clear replaced by the stamp; nodes built during propagation are journaled. Closes "Scheduling at `usize::MAX` while at `usize::MAX`". The invariant checker lands here and runs in every test from now on. *Done when:* an injected panic returns `Err` with the graph unchanged, as the checker asserts; a `rebuild` firing in a failing occurrence leaves the old child in place and the new one gone; `a_trigger_dropped_by_a_panic_accepts_values_again` still passes with no trigger flag involved; no clear-at-end list exists.
3. **Eager cycle detection and the build boundary.** `height_checked`, the in-progress flags collecting spans, the walk at placeholder close and at switch retarget, the nested-transaction truncation in `bind` and `build_root`, the error node. Closes "Cyclic handles are bare `usize` plus a name", "Unclosed placeholders are found at the first propagate" and "The `RebuildEvent` has no height cache and no owner". *Done when:* the two cycle tests assert `Cycle` with the loop's spans; a cycle closed inside a `rebuild` build is contained and the `rebuild`'s event does not fire; a cycle at a switch adoption aborts the switching occurrence with an owner path; the `schedule` check is reached by no test.
4. **`Dynamic::join`, `hold_first`, `map_n`, `tag_map_n`.** `JoinOuterEvent` with its dynamic height; `switch_hold_seeded`; the three law tests and `join_is_prompt`; the n-ary constructors with one test each that a three-operand lift is one node. *Done when:* the seven behavior tests have event twins; the swap-and-change case in both height orders agrees in both halves under the checker; a hold of dynamics plus join builds the cache idiom with no hiccup.

5. **Elements, slots, paths, the element list, request collection,** `build` **and** `run`**.** `ElementValue`, `TagId`, `TagTable`, `Path`, `Segment`, `SlotRec`, `element`, the position rule, `PMap` and `PSet`, interned elements and paths, the per-node list with patches on its change event, `Runtime::list`, the root fan, the collector, `Inputs` without `quarantine`, `Outcome`, `RuntimeError`, `Runtime::build` and `Runtime::run`. The test backend is deleted and the harness written at this step, with the counter as the first fixture in the canonical structure. Closes "`WidgetDelegate` exposes live slots", "The test backend reads behaviors directly", "`backend_data` cannot be set" and "`__rebuilt__` as a keep-alive". *Done when:* the eight script tests run as harness scripts reading only the outcome; the counter's transcript matches the conformance chapter's golden byte for byte; routing against the opened list, unknown path as error, a slot created in n addressable from n+1, request firings from a slot torn down in the same occurrence reported with its path, `rebuild` generations giving fresh paths, and one call per occurrence each have a test; a benchmark with a thousand slots shows a label rebuild costing time independent of the thousand.
6. **State at the root.** The oracle supply context, root cells, pushes, the `UnsuppliedState` error. *Done when:* a state slot has its value in its creation occurrence from the oracle; a push reaches a slot created in the push's occurrence; two slots of equal value share one cell and cost one oracle call; an unsupplied tag is an error at creation, contained under a `rebuild` and fatal in occurrence zero.
7. `InterceptBlueprint`**.** The fan per interceptor, the supply context per interceptor, `Supplied(n)` positions, the captured list, `Defer` in the harness turn loop (as a seed of the world core, in a function with no harness state in it). *Done when:* an interceptor that forwards everything is observably absent (transcript identical with and without it, and with two); a sandbox supplying `Unavailable` works; a supplied widget's elements appear in the interceptor's list under `s/n/`; two child slots of equal value share one supplied widget regardless of a supplier change between them; a supplied widget lives until the last slot with its value goes; the interceptor's loop over its own child's list builds without a cycle error; pure code firing at a path under an interceptor built in the same occurrence is `UnknownPath`.
8. **The list widget.** `list_blueprint` with the three prompt consolidations, sharing the deferred-build code with `Rebuild`, and the key diff by shared subtrees. Closes "`DWAction` replaces child index 0" and "`rebuild` builds twice when its dynamic changes later in the occurrence". *Done when:* each consolidation kind has a test; state survives an insert and a neighbour's removal; a value change updates in place; a key's block is torn down at commit; a key born in n is in all three consolidations in n; `@ Delete on !started` as a row result is reported as a cycle at the membership change.
9. **Quarantine.** `pending_quarantine`, the `quarantine` input, the replacement, the discarded responses. *Done when:* the quarantine scenario passes; a quarantine for a path no error named is refused.
10. **Benchmarks for the costs that are not semantics.** Three programs, each pinned to the cost the performance notes give it: a window of nested boxes with a thousand labels under `Gui.Frames`, one label changing per frame; chapter 5's selectable list with a thousand rows and `$Distinct` on the selection test, one click; a thousand-key map with one `Insert` under `for`. The changes they call for land here, each with no transcript change: shared draw lists with pointer-equality diffs and damage rectangles in the GUI library and the world core, the `$Distinct` warning in the compiler, and native versions of the prelude functions the benchmarks show hot. *Done when:* every transcript is unchanged and each benchmark's cost grows with what changed, not with the thousand.
11. **Small debts.** Every known issue no step above names: `trace`, the `unreachable!()` stubs, the cyclic dynamic sampled after its node is dropped, the `combine`/`leftmost` height caches, `leftmost`'s

subscriber lists, the dependents scan, the two source events that record only when subscribed, the copied doc comments, the misnamed file. *Done when:* the list below is empty.

## Known issues

Every loose end in the current code, concretely, with its fix. Most are small; they are listed because a reader executing the plan should not have to rediscover them.

**`trace` leaks its tracer.** `Event::trace` subscribes a logging callback and `mem::forget`s the `Arc`, because subscribers are weak and nothing else would hold it. *Fix:* make `trace` a builder method, `WidgetBuilder::trace(&mut self, event, label)`, which stores the tracer in the node's keep-alive list; or keep the free function behind a `debug-trace` feature with the leak documented as the price of using it outside a node.

**`unreachable!()` stubs in `SubscriptionEvent` implementations.** `EventTriggerInner`, `WidgetReadyEvent` and `RebuildEvent` are sources, not derived events, yet they implement `SubscriptionEvent` because `SubscriptionManager` requires it, with `Inner = ()` and bodies that panic. *Fix:* split the manager's upstream handling into a separate trait, `Derived<T>`, implemented only by events with an upstream, and give sources a `SubscriptionManager<T, NoSource>` whose connect and notify paths have no upstream to consult. The three stubs disappear and the type system says which events have an upstream.

**Scheduling at `usize::MAX` while at `usize::MAX`.** The commit batch runs at the greatest height, so an action in it that needs to defer to commit has no height above it, and `RebuildEvent::already_fired`'s `current + 1` overflows there. Nothing in the commit batch builds or subscribes today, so the case is not reached, but it is one new commit action away. *Fix:* the separate commit list of step 2; `defer_commit(entry)` pushes onto it, and the action map never contains the commit height.

**`DWAction` replaces child index 0.** A `rebuild`'s node has exactly one child, so the index is right for `rebuild`, but the replacement is by position, the delegate is told index 0, and the same code cannot serve a list widget. *Fix:* `Widget::children` becomes `BTreeMap<Segment, Rc<Widget>>` for addressable children plus a `Vec` for plain ones; a `rebuild` replaces at `Gen(g)` and a list widget at `Key(k)`, and the element list's prefixes come from the same keys.

**`WidgetDelegate` exposes live slots.** `TestDelegate::for_node` clones `node.public_dynamics` and `node.triggers`, so the backend holds `AnyDynamic` and `AnyEventTrigger` handles outside the runtime for the life of the test, reads behaviors through them with `query_for_tag`, and fires triggers through them by name. A World holding runtime objects is what the boundary forbids, and it is also why the delegate has to be told about every new child. *Fix:* delete `WidgetDelegate`, `WidgetDelegateContext`, `DelegateCallback`, `set_delegate`, `prepare_destruction`, `_public_event_callbacks`, `public_events`, `public_dynamics`, `triggers`, `add_external_event`, `add_public_event`, `add_public_dynamic`, `AnyEvent`, `AnyDynamic`, `AnyEventTrigger` and `backends/test_backend`; the harness over `run` replaces all of it, and the eight script tests are rewritten against the outcome.

**`backend_data` cannot be set.** `Widget::backend_data: serde_json::Value` is initialised to `Null`, no builder method writes it, and the node is behind an `Arc` by the time anyone could. It is the only use of `serde_json`. *Fix:* delete the field and the dependency. Its purpose, telling a backend what a node means, is served by the element list.

**`Arc`/`Mutex` everywhere.** Covered by step 1. The places where the locks do more than lock: the `try_lock` re-entry checks in `FilterMapEvent::height`, `Tag::height`, `SwitchHold::height`, `ForwardEvent::height` and `DependentBehavior::query_for_*`; `TriggerAction` holding and releasing

the trigger's `scheduled` lock around notification. *Fix:* `Cell<bool>` re-entry flags; the `scheduled` flag disappears with public triggers.

**The test backend reads behaviors directly.** `TestAction::ExpectDynamic` calls `query_for_tag` on a public dynamic from outside the runtime, a read a World cannot make. *Fix:* the harness reads the outcome; a test that wants a dynamic's value makes the program put it in an element.

**`__rebuilt__` as a keep-alive.** The `RebuildEvent` is kept alive by registering it as a public event, which also means a backend sees a firing it was not meant to see (the comment notes the delegate attaches after the first one). *Fix:* a private `keep_alive: Vec<Rc<dyn Any>>` on `Widget`, written by the builder.

**Cyclic handles are bare `usize` plus a name.** `add_cyclic_*` returns an index and takes a name; closing by the wrong index with the right type is accepted; two `fold_dyn`s with the same name in one node panic on the duplicate; a wrong-typed close is a runtime panic rather than a compile error. *Fix:* `CyclicEvent<T>` and `CyclicDynamic<T>` handles returned by `add_cyclic_*`, closed by a method that consumes the handle; no names; a `compile_fail` doc test. The compiler, which already names every placeholder by span for diagnostics, keeps the span in the handle, and the span is what the cycle report prints.

**Unclosed placeholders are found at the first propagate.** A cyclic dynamic never closed panics with "cyclic dynamic never closed" the first time it is sampled, far from the widget that forgot. *Fix:* `bind` and `build_root` check every slot in the node is closed before returning, and fail the build with the placeholder's span.

**A cyclic dynamic sampled after its node is dropped panics.** The behavior reaches its slot through a `Weak`; after the node is gone the upgrade fails with "cyclic dynamic sampled after its widget was dropped". A parent that kept the dynamic as a result would hit this. *Fix:* on teardown, before the slot goes, write the dynamic's last value into a cell the computation falls back to, so the dynamic keeps its last value as the specification says.

**`combine` and `leftmost` recompute their height on every call.** Both write a height cache they never read, so every `height()` call walks every source. *Fix:* read the cache; invalidation already reaches them.

**`leftmost` pushes each subscriber onto every manager.** Only the first manager delivers, so the other copies serve only `invalidate_height`, which then tells each subscriber once per source. *Fix:* one subscriber list, on the first manager; the others connect and compete only.

**The dependents list is scanned on every add.** `query_for_behavior` does a `retain` over the whole list before each push to keep it deduplicated. *Fix:* a generation-stamped weak set keyed by the dependent's pointer, which is the `WeakBag` the subscriber list's comment also asks for.

**`Dynamic::new` is `unsafe` for a logical invariant.** *Fix:* `pub(crate)`, backed by the checker.

**Events are not tied to a runtime.** An event built against one runtime can be scheduled in another and nothing notices. *Fix:* the `Rc<Core>` every graph object holds after step 1, which makes the question unaskable.

**`rebuild` builds twice when its dynamic changes later in the occurrence.** A `rebuild` built mid-occurrence samples the behavior if the change has not happened yet, binds that, and rebuilds by push when the change arrives above it; the end state is right and the first node is replaced before commit, but the first build ran, and any request it fired is reported to the World with a path that is already gone. *Fix:* when `subscribe` returns `None` and the dynamic's change height is above the current height, defer the first build to just above that height, so that it builds once with the value the

occurrence ends with. A counter inside the block in the nested-`rebuild` test, in both height orders, pins it.

**`rebuild` and done events record a firing only if subscribed.** A `rebuild`'s output event and a done event fire through a manager that records only when it has subscribers, so a widget built after the firing, in the same occurrence, cannot catch up on it. Nothing today needs to. *Fix:* record unconditionally in those two sources, which costs one clone.

**The `RebuildEvent` has no height cache and no owner.** Its height is recomputed from the dynamic's event on every call, which is fine, but it has nothing the eager walk can hang an in-progress flag or an owner on. *Fix:* the same `height: Cell<Option<usize>>`, flag and `owner: Weak<Widget>` every other node gets in step 3.

**Copied doc comments.** `Event::combine` and `Event::leftmost` have `filter_map`'s doc, with a "Some(vague)" typo. *Fix:* write them.

**`widget/runtime.rs` is misnamed.** The file holds `DynBlueprint` and reads like the crate's runtime module. *Fix:* `widget/polymorphic.rs`.

## Performance notes

What is O(what), where allocation happens, and where a program pays for more than what changed. The numbers are structural; the crate has no benchmarks yet, and the first ones belong with step 10. The runtime's own machinery is proportional to what fired. The places where a GUI pays more than that are the draw list, equal values that still fire, derived functions over maps, comparisons of large keys, and plain compute in closures; each has a paragraph below, and none of the fixes changes what a program means.

**Propagation.** An occurrence costs the sum over fired nodes of their closure plus O(log H) for the `BTreeMap` insertion at scheduling time, where H is the number of distinct heights with actions, plus O(subscribers) per firing to consolidate the list and deliver. Nodes that do not fire cost nothing: a quiet subtree is never visited. A subtree that fires with equal values is not quiet, and the paragraph on equal values says how often that happens. The map is cleared and refilled every occurrence, so its allocation is per height with actions; a `Vec<Box<dyn Action>>` per height and a `Box` per action. A ring of pre-sized vectors indexed by height, reused across occurrences, removes both when it matters.

**Catch-up.** `record_once` costs one `Cell` read, one clone of the value and, today, one boxed clear-at-end closure per fired event per occurrence. The closure is the avoidable part, and the stamp removes it: each manager's cache holds `(u64, Option<Value>)` and a read compares the stamp against the core's counter, so the cache clears itself and the clear-at-end list is gone. Catch-up for a late subscriber is one walk up its sources until a cached value is found, each step constant.

**Heights.** Cached per node after the first computation, invalidated transitively on a retarget with the walk stopping at empty caches. The cost of a retarget is proportional to the number of nodes downstream of it that had computed heights since the last retarget, which is bounded by the subgraph below it and is usually small. The eager walk at an edge creation is bounded the same way and stops at cached heights.

**Behaviors.** Pull with a cache; a query is constant when cached and proportional to the dependency subgraph when not. Invalidation on a hold's commit walks the dependents that have queried since the last invalidation. The retain-on-add scan is O(dependents) per query until the weak set lands.

**Values.** Every value in the graph is an enum of two words plus a tag; heap values are `Arc<dyn`

`HeapValue>` (an `Rc` after step 1) with one allocation per construction and a refcount bump per clone. `Event<T>` and `Dynamic<T>` stored as values pay a second allocation for the outer `Arc`. The one-word representation of the values chapter removes the second indirection and shrinks the enum to a word; nothing in the runtime depends on the current layout, which is behind `PrismValueInner` for exactly this reason.

**Plain compute.** An occurrence-time closure is a boxed Rust closure over two-word `Value`s: every operation matches on the value's tag, every call goes through an `Rc<dyn Fn>`, and every intermediate is a `Value`. For a tight loop (an arithmetic fold, a scan over text) that is roughly an order of magnitude slower than the same loop written in Rust; the figure is an estimate from the structure, not a measurement. Most of a program is not tight loops, but layout, hit-testing and text editing live in Prism, in the GUI library, and run on every frame that changes anything. So the prelude functions that library leans on (traversals of maps and arrays, text slicing and measurement, rectangle arithmetic, sorting) are native Rust in the program's function table, called from compiled closures like any other function, and the library's own hot loops are the first clients of a native backend.

**Routing.** Per delivered response, one `query_for_tag` of the fan's list (constant once cached; the cache is invalidated only when the list commits a change) and one lookup, O(log S) in the number of slots under that fan, plus the output's delivery. A slot that receives nothing costs nothing, and a fan whose responses event is quiet costs nothing: there is no per-occurrence work per interceptor. Depth adds one hop per interceptor an input passes through, which is the price of every level seeing its children's inputs and is per input, not per slot.

**State supply.** One memo lookup per state slot created, O(log V) in the number of distinct element values under the interceptor; one supplied widget per distinct value, however many slots.

**The element list.** Naive: a node's list is rebuilt as the union of its children's on any change below it, which is O(total slots in the subtree) per change and O(depth) such rebuilds per change up to the root, so a label rebuild deep in a tree of N slots costs O(N) per occurrence it changes. Incremental: the patch for a label rebuild is two entries, and applying it at each ancestor is O(log N); the root's map is updated in place behind the `Rc` or cloned on write if a World still holds the previous one. This is the one change in the chapter that turns a per-occurrence cost proportional to the program's size into one proportional to what changed, which is what a GUI with a thousand rows needs.

**Builds.** A `rebuild` firing costs the block's build plus the journal entry plus the list patch; a failed build costs the same plus the truncation, which is a `Vec::truncate` on four vectors. A supplied widget costs its build once per distinct value. Nothing is allocated for a key that stays or a generation that is not rebuilt.

**Allocation summary.** Per occurrence: the action map's vectors and boxes, one value clone per fired event, the request collector's vector, the outcome's vectors. Per build: nodes, managers, closures, paths (one `Rc<[Segment]>` per slot and per addressable node). Nothing allocates between occurrences, and nothing allocates for a node that does not fire.

## Parallelism

Single-threaded is a decision about the first implementation, not a claim about the model, and the structure the runtime already has is the structure each of three kinds of parallelism needs. None requires a change to a program, and none is needed to make the language exist.

**Fork-join per height.** Within an occurrence, every action at one height is independent of every other at that height: each reads only values below its height, which have already fired or not, and

writes only its own node's cache. So the inner loop of `propagate`, "run the actions at this height", can hand them to a pool and join before popping the next height. The one thing that is not independent is a build: a `rebuild` firing at height h mutates the tree and the journal and subscribes to nodes at many heights, so builds at one height run under a *build lock*, one at a time, while the pure actions at that height run in parallel around them; or, simpler, a height with any build in it runs serially. The variant needs `Sync` graph types, a concurrent subscriber-list consolidation and a concurrent action map: a second runtime compiled from the same source with the pointer and cell types swapped, not a change to this one. The single-threaded variant stays the reference the other is tested against. The win is bounded by the width of the graph at each height, which for a GUI with many independent rows is wide and for a chain of maps is one.

**Optimistic across occurrences.** An occurrence is a transaction with buffered writes and a known read set (every `query_for_tag` and every hold read). Two occurrences whose inputs are addressed to disjoint subtrees can be run speculatively against two copies of the commit list and committed in turn if the second's read set does not intersect the first's write set, else retried. This is ordinary transactional concurrency, and the runtime's rollback path is its first half: a retry is a rollback followed by a rerun. It needs read-set tracking, which is one push per `query_for_tag`, and a World willing to batch inputs into candidate occurrences. It is the variant for a server with many independent clients under one program.

**Sharded runtimes.** Independent subtrees connected only through elements can be separate runtimes, each on its own thread, with a World that routes between them: a server with one runtime per connection, or a GUI with one runtime per window. This needs nothing from the runtime beyond the run call being cheap and the boundary being plain data, both of which the target already is; it is a World design, and a World written in Prism that hosts other programs is the same thing with the routing in Prism.

What none of them changes: the specification's order within an occurrence, which is the height order and is the same under all three; the transcript a program produces, which the differential test checks across variants as it checks across implementations; and the program, which cannot tell.

# Values representation

You have profiled the runtime and the top of the profile is not propagation. It is `clone` and `drop` on a 24-byte enum, an atomic increment for every event payload that passes through a map, and an allocation every time a float crosses from one node to the next. The graph is doing what it should; the values riding on it are the cost. This item is the design of the value representation that replaces the enum, in full, with the small part that should be done first and the longer list of things that become possible once the base is in place.

## What the runtime has and what it must keep

Every value the runtime touches goes through one module. The rest of the runtime sees a typed wrapper and an erased form, and reaches the inside only through checked downcasts. The source today spells those `Value<T>` and `AnyValue`; the boundary's names, and the names this chapter uses for the design, are `Typed<T>` for the wrapper and `Value` for the erased word. Inside is an enum over an `Arc<dyn HeapValue>`, a `u32`, an `i32` and a unit. It is 24 bytes on x86_64. It was written to be replaced: the module's own comment says the implementation is meant to change in a way the rest of the runtime adapts to, and the API it exposes has no introspection and no fallible downcast, so nothing outside the module depends on how a value is laid out.

Three properties of that API are the contract the replacement keeps. A downcast to the wrong type is an error, not a reinterpretation; events and dynamics are values like any other and can sit inside records and element fields; and generic code in the runtime works on `Value` without knowing what it holds. The replacement is free to make all three cheaper. It is not free to weaken the first in a release build, because the producers of values include the frontend, the closure compiler, native components and whatever comes back from a World, and the first one to get a type wrong should hit a check, not a wild pointer.

## One word, three tag bits

A Prism value is one machine word. Its low three bits say what it is: zero means the rest of the word is the address of a heap object; the other seven patterns are immediate kinds with the payload in the high bits.

| low bits | kind | payload |
|---|---|---|
| 0 | heap pointer | address of a header |
| 1 | `Int` | signed integer |
| 2 | `Nat` | unsigned integer |
| 3 | `Char` | code point |
| 4 | `Bool` | 0 or 1 |
| 5 | tag | unit, or a payload-free choice variant's index |
| 6 | `Nil` | none |
| 7 | `Float` on 64-bit targets (below); unassigned on wasm32 | |

Heap objects are allocated with eight-byte alignment on every target, because they all start with one header type declared `align(8)`; wasm32 would otherwise give four and leave only two bits. So a heap pointer ends in three zero bits on both targets and the seven other patterns are free. The zero word is never a valid value, so `Option<Value>` is one word through the same niche `NonZeroUsize` has, which matters because optionals are everywhere in node state.

```
pub struct Value {
    word: NonNull<()>,
    #[cfg(debug_assertions)]
    shadow: &'static TypeInfo,
}
```

The word is a `NonNull<()>` and not a `usize` so that heap pointers keep their provenance. Heap words come straight from the allocation. Immediates are built from an integer address and retagged with the pointer's own provenance, the strict-provenance style (`NonNull::without_provenance` and `map_addr`, from memory) that Miri checks, and Miri is the test that stands between this module and the rest of the runtime.

`Bool` has a kind of its own because it is the most common choice type and a `match` on it should be a mask and a compare. `Nil` has a kind of its own for the optional rule below.

Why three bits and not one. A one-bit scheme, the kind OCaml uses (from memory), lets the static type say which immediate a word is. Three is better here because a release build never decodes a word as the wrong kind, whatever produced it. For an immediate a kind check is a mask and a compare on a word already in a register, so there is no speed case for dropping it, and three bits also let a field of an element record say its own kind without a type beside it. What three bits cost is two bits of integer range, and the integer scheme below makes that a matter of when an integer goes to

the heap, not of what integers the language has.

## Integers keep their meaning

`Nat` is `u64` and `Int` is `i64` on every target, overflow aborts the occurrence with kind `Overflow` (in the runtime a panic caught at the occurrence boundary; the language offers `Checked.*` and `Wrap.*` for programs that want another outcome), and none of that changes. Tag bits belong to the representation, and the language does not shrink to make room for them.

The runtime keeps an `Int` in the word when it fits beside the tag: $-2\hat{}60$ through $2\hat{}60 - 1$ on 64-bit targets, $-2\hat{}28$ through $2\hat{}28 - 1$ on wasm32. Otherwise the `Int` is a heap object with an `Int` descriptor holding the full `i64`. `Nat` works the same way on its own tag. The form is canonical: a value that fits is always immediate, never boxed. So two immediates are equal exactly when their words are, a boxed integer never equals an immediate one, and equality on integers is a word compare followed, only when both words are heap pointers, by a descriptor call.

The worry is that every arithmetic operation now needs its own overflow logic, since Rust's checked operations know nothing about a 61-bit bound. They do not, because the payload sits in the high bits, so the tagged word overflows exactly when the payload does. If `a = 8x + 1` and `b = 8y + 1` then `(a ^ 1) + b = 8(x + y) + 1`, and that overflows the machine word exactly when `x + y` leaves the immediate range:

```
const INT: isize = 0b001;

fn int_add(a: &Value, b: &Value) -> Value {
    if let (Some(x), Some(y)) = (a.small_int_word(), b.small_int_word()) {
        if let Some(sum) = (x ^ INT).checked_add(y) {
            return Value::from_int_word(sum);
        }
    }
    let sum = a.int().checked_add(b.int()).expect("Int overflow");
    Value::from_int(sum) // immediate if it fits, boxed otherwise
}
```

Subtraction is `a.checked_sub(b ^ INT)`; multiplication is `(a >> 3).checked_mul(b ^ INT)` with the tag put back. When the fast path overflows, or either side is boxed, the slow path works at the full 64 bits and panics only where the specification says to. Dividing two immediates always gives an immediate except $-2\hat{}60 / -1$, which boxes like any other large result; the only overflow left in division is `i64::MIN / -1`, as in Rust.

The cost is allocation, and it is worth being exact about where. On 64-bit, an integer allocates only past $2\hat{}60$, which is hashes and little else. On wasm32, anything past about 268 million allocates, and that includes Unix time in seconds, so in a browser World those values are heap objects. One tag bit would not save them: $2\hat{}30$ is about 1.07 billion, and current Unix time is past 1.79 billion. `Int` is 64 bits on every World, so a program means the same thing in a browser as on a desktop; on wasm32 the values past $2\hat{}28$ box, and the cost is time, never meaning.

## Heap objects

A heap object begins with a reference count and a `&'static Descriptor`. The descriptor holds a type id, a debug name, drop, eq, hash and debug functions, and a layout saying which of the object's words are values to be counted and which are raw data.

Clone never calls through the descriptor. It tests the low bits: nonzero means copy the word; zero

means increment the count at a fixed offset from the pointer, inline. That is the increment `Arc` does, with the same release decrement and acquire fence on drop (from memory). The descriptor is called only when a count reaches zero and for eq, hash and debug. Whether the count has to be atomic at all is the first entry under the optimisations below, and the answer is the same for one word, two words or an enum.

Text, arrays, maps, records, choice variants with payloads, closures (a code id plus captures, one descriptor per closure site), large integers and boxed floats are heap kinds. Events and dynamics are boxed today and stay boxed, with a header and descriptor where `Arc<dyn HeapValue>` is now. Text is one allocation: header, length, bytes. Not a header pointing at a separate buffer. Text literals become static objects with no allocation at all, which is most of the text in a user interface.

Where a `&'static Descriptor` comes from depends on who made the value, and the first reader implementing this hits the question at closures. The runtime's own kinds (text, arrays, maps, events, dynamics, boxed numbers) have descriptors written as Rust statics. A closure site, a record type or an element layout from a compiled program needs a descriptor the Rust source never saw. The closure compiler builds those at load, one per closure site and one per interned layout, into a table it leaks for the life of the program; leaked memory is `'static`, the program is loaded once per process, and a World that loads many programs leaks one table per program, which is bounded by the number of programs and not by the number of values. A bytecode-loaded program is no different: its load step is the same build of the same table. The emitted-Rust backend below writes the same descriptors as statics in the generated source, so under that backend nothing is leaked and nothing is built at load. Either way, every descriptor pointer a value holds outlives every value that holds it, which is the invariant the drop path relies on and the loader is bound to.

## Records keep their floats raw

Floats are the strongest case against one word, and the design has to meet it head on. In a UI language the numbers that change most are coordinates, sizes, colors, opacities and animation progress, all `f64`. An `f64` cannot sit beside a tag in a 64-bit word and does not fit a wasm32 word at all, so a float stored as a bare word is a heap object. If every UI float went through a word, every frame would allocate all over.

Most of them do not go through a word, because they live in records. A record's descriptor already needs a layout so that clone and drop know which fields to count, and that layout can mark a `Float` field as eight raw bytes (two words on wasm32). That is data, not code: one clone and one drop routine per record type, walking a list, which is the struct layout any compiled language has. A rectangle, point or color is one heap object with raw floats inside, cloned with one increment and no allocation.

Elements and the responses a World sends get the same treatment. An element value is a heap object: header, interned tag, a pointer to its layout, then the fields. The layout lists each field's name, type and storage, and `Float` fields are raw. The layout comes from the tag declaration, so a `Draw.Rect(x=rx, y=ry, w=rw, h=rh)` in a `Gui.Scene`'s draw list, with its fields statically `Float`, has a layout known at compile time, and so does the pointer record `Gui.Window` responds with. A `match` arm that names the pointer record and reads `x` as a `Float` checks the layout pointer and loads the raw bits, with no box anywhere between the World's input and the arithmetic. Layouts are interned for the life of the program; records with the same tag and fields share one wherever they were built; matching compares layout pointers and the result can be cached per pattern site. There is no second representation at the boundary: `@Element` stores the same words and raw floats the rest of the code uses, and an event or an element list held in a field is the same heap pointer it was outside, which is

how `requests: !R` fields and the `children` field of `Ui.VBox` come for free.

## Where floats still box, and the immediate float

A bare `Float` travelling alone as the payload of `!Float` or `$Float`, or as the element of a generic array, is a heap object. An animated `$Float` allocates and frees once per firing. That is acceptable as a base, for three reasons, and then fixed on 64-bit by the last tag.

The first reason is proportion: a firing already propagates through the graph, runs subscribers and updates a held value; one allocation is in scale with that. The second is where floats usually live: a uniform representation can box a scalar as long as the places the scalar usually sits do not, and in Prism a float in a typed register inside a leaf or in a record field is not boxed, so the lone float is the exception and not the rule. The third is what the alternatives charge, below.

The fix: an `f64` has 11 exponent bits. Drop three, and any double whose exponent falls in an 8-bit range, which is roughly the range of an `f32` with the full 52-bit mantissa, fits beside tag 7 exactly, with no rounding. Zero gets its own encoding. Every other double (huge, tiny, infinities, NaN) stays boxed, and the form is canonical the same way integers are: a float that fits is always immediate. Coordinates, sizes, colors, opacities and animation progress all fit. Smalltalk-family 64-bit VMs have immediate floats on this principle (from memory). Decoding is a shift and a rotate; float equality goes through the decoded value, with NaN and $-0.0$ handled the way they have to be handled anywhere. It does nothing for wasm32, where 29 bits cannot hold a useful double, so browser Worlds keep boxing lone floats; and it uses the last free tag, which is why short strings do not get it (a 7-byte inline string on 64-bit and a 3-byte one on wasm32 is worth less than every animated value in the program not allocating).

The alternatives are two other layouts, and both are worth taking seriously before rejecting. Two words per value, a descriptor pointer and an 8-byte payload, is 16 bytes on both targets, never allocates an integer, never boxes a float, knows its own full type, and makes debug and release the same program. What it charges is 8 bytes on every value in every node's state, every event payload and every record field, which doubles a record of integers, and the second word repeats what a heap object's header already says. NaN-boxing makes every value 8 bytes on both targets, which is twice the word on wasm32; it narrows integers to fewer than 64 bits and ties pointers to the address bits a NaN payload has room for (48 or so, from memory), and it builds the whole encoding around doubles, which fits a language whose numbers are all doubles and not one with `Nat` and `Int` beside `Float`. Immediate floats on tag 7 get most of what either would give, every UI-range double on 64-bit, while keeping one tag scheme on both targets. NaN-boxing stays as the comparison point in the benchmark, so the choice is measured and not argued.

## Where generic code gets its types

Layouts need a type for each field, and a single language rule makes sure they have one: a field of an element may not have a type that mentions a type variable (bare `a`, `[a]`, `?a`, `fn a -> b`), and breaking that is a compile error. Every tag declaration therefore gives complete field types, and every element layout is known where the element is built. The case that would have needed help, a function generic in `a` putting an `a` into an element where the word cannot say `?Nat` from `Nat` or which choice type a tag index belongs to, cannot be written.

So generic code is erased. A register whose type is a type variable holds the one-word value and clones and drops it by its tag bits; the kind bits and a heap value's descriptor are all that clone, drop and downcast need. There are no hidden descriptor arguments and no monomorphization. The cost

is that generic library code cannot wrap its own type parameter into an element; it takes elements already built, or is written at a concrete type. Elements are strongly typed by their tags, so handing a value of the wrong type to a tag's field is a runtime error at the element.

Suppose instead the language allowed `a` in an element field and had the compiler pass a type descriptor for `a` as a hidden argument to any function where a value of a type variable reaches an element constructor, spreading the requirement to callers the way a type-class constraint spreads. That is Haskell's dictionary passing, it works, and it does not reintroduce two words: the descriptor is an argument used once to pick a layout, not a second word on every value. What it costs is a second calling convention in the closure compiler and a requirement that spreads through signatures for the benefit of a use (a generic function building elements out of its type parameter) that no library in the GUI corpus has needed. If one appears, this is the design to reach for, and the rule above is the line it would move.

## Debug builds are two words

In a debug build a value is the release word, unchanged, plus a pointer to its full static type. Nothing needs an ABI yet, so there is no reason to squeeze the shadow into the word. Debug runs every check release does and also checks the full type at every downcast.

That makes debug and release slightly different programs, and the gap should be named exactly. The first word is bit-identical in both. Release checks every kind and, through the descriptor's type id, the full type of every heap value. What release cannot catch is confusion within an immediate kind, and the whole list is short:

| Within-kind confusion | Debug | Release |
|---|---|---|
| Two payload-free choice types swapped (a `Color` index read as a `Side`) | Shadow mismatch, panic at the downcast | Wrong answer: the index is read as the other type's variant; `match` bounds-checks it, so an index past the other type's last variant is a panic, not a jump |
| A newtype read as its base (`Px` as `Nat`) | Panic at the downcast | Wrong answer with the right bits |
| `?Nat` read as `Nat` where the value is `Some n` | Panic at the downcast | Wrong answer: `Some n` is `n`'s word, so the read succeeds; a `Nil` where a `Nat` is expected is a cross-kind confusion and panics in both |
| A boxed `Int` read as a boxed `Nat` | Panic at the downcast | Panic in release too: both are heap objects and the descriptor's type id differs |

Every entry is a wrong answer or a panic, never a memory error, because the kind bits say what may be done with a word and the one place an immediate indexes anything, a payload-free `match`, is bounds-checked with one compare. CI runs the suite in both profiles, and the tests that pin each row are in the table at the end of this section.

## Optionals and choices

`Nil` is its own kind. `Some x` is `x`'s word, unless `x` is `Nil` or an already boxed `Some`, in which case it is a

one-field box. The rule depends on the value and not the type, so generic code over `?a` works without knowing `a`, and only nested optionals with a `Nil` inside, `Some Nil` in a `??T`, ever allocate. When `x`'s static type is neither an optional nor a type variable, `Some x` is the word with no check at all. Integer division returns `?Int` and allocates nothing on immediates, which matters because division is in every layout computation.

Units and payload-free variants are tag immediates holding the variant index. Variants with payloads are heap objects whose descriptor names the choice type. The kind check catches `Nil` where a value belongs and a value where `Nil` belongs; telling `?Nat` from `Nat` is the static type's job, the debug shadow's, or an element layout's.

## The urgent subset

The full design is worth doing only with a benchmark in hand, and the trigger below says what that benchmark is. Two parts of it pay before any benchmark, because they remove costs the current enum pays on every program and need no unsafe code:

- Inline `i64` and `f64` in the enum in place of the `u32` and `i32`. The enum grows to no more than its current 24 bytes, since the `Arc` variant already sets the size, and every integer and float stops allocating.
- Text as one allocation, with literals static.

Both are changes inside the value module behind the API the runtime already uses. Both are also steps one and two of the plan for the full design, so nothing done for them is thrown away.

## The plan for the full design

1. **Enum as reference.** Extend the enum to every kind: full-width `Int` and `Nat`, `Char`, `Bool`, tags, `Nil`, `Float`, `Text`. Put it behind a cargo feature. Write property tests against the public API only, so they run unchanged against the word later. Done when the runtime's suite passes on it.
2. **Header and descriptor.** Replace `Arc<dyn HeapValue>` with the header-and-descriptor object, still inside the enum; move events and dynamics onto it without changing their APIs; text becomes one allocation. Done when no `Arc<rebuild ...>` is left in the value path.
3. **The word.** The `NonNull<()>` implementation: tag table, inline clone, drop through the descriptor, integers with the tagged fast path, the full-width slow path and canonical boxing. Done when the suite passes on both implementations, every `i64` and `u64` edge value round-trips boxed exactly when out of range, and Miri is clean with strict provenance.
4. **Debug shadow.** CI in both profiles from here. Done when a within-kind mismatch panics in debug and a cross-kind one panics in both.
5. **Optionals.** `Nil`, the `Some`-box rule, division returning a word. Done when nested-optional round trips pass and dividing immediates allocates nothing under a counting allocator.
6. **Records and choices.** Raw `Float` fields, payload-free variants as immediates, payload variants on the heap. Done when a record of four floats clones with one increment and no allocation.
7. **Element layouts in the compiler.** Interned layouts with field types and storage, interned symbols for tags and field names, matching by layout with a per-site cache, and the compile error for a type variable in a tag's field. Done when a pointer element goes from World to `match` arm without boxing a float.
8. **Measure, then switch.** Benchmark clone, drop, allocations and time per occurrence on the runtime harness for both implementations, with a float-heavy animation, a large keyed `for` list and a wasm32 build. The word becomes the default when its time per occurrence is no worse than the enum's on every case; the enum stays as the oracle.

9. **Immediate floats on tag 7**, benchmarked against a NaN-boxing prototype on the animation case.

The enum is the safe version. The unsafe code sits in one module behind the API the runtime already uses, and the enum stays as the thing the word is checked against. That is the normal order, ship safe and measure before going unsafe, and the plan keeps it.

## Other optimisations in the same bucket

The one-word design is a base. Each item below says what it buys, what it costs, and what has to exist first. None of them changes what a program means, because Prism values have no identity and nothing a program can do distinguishes a value that was reused from one that was copied. None of them should land before the measurement step above.

**Non-atomic reference counts.** Clone and drop of a heap value become a plain add and subtract instead of locked atomics; they are the commonest operations on heap values, so this touches everything. The cost is that a non-atomic count makes values `!Send`, which cannot be a quiet switch inside the value module. Two ways to do it: a cargo feature making the whole runtime single-threaded, with two builds to test; or a per-object sharing mark, where an object uses a plain count until it is shared with another thread and is marked for atomic counting then, which is one build and one branch per count update (Lean 4 does something of this form, from memory). The runtime is single-threaded, and the parallel variants later in this chapter are separate runtimes, so the feature is the fit, and it comes right after the `Rc`/`RefCell` conversion the runtime plan already has.

**An incremental element list.** The element list of a widget is the concatenation of its own slots and its children's lists in tree order, and the first implementation rebuilds the whole thing when any slot changes. That is linear in the program's element count per occurrence that touches the list. The incremental version keeps the list as a persistent ordered map keyed by path, so that a slot created or removed in a `for` or `rebuild` is one insertion or deletion, and the interceptor and World above see a map whose unchanged subtrees are pointer-equal to last occurrence's. The dynamic's value and change event are the same as before; only the cost changes. Paths are already ordered in tree order, so the map's order is the list order with no sort. This pays as soon as any program has a list of a few hundred items and a label that changes per keystroke, which is any text editor.

**Interned tags and field names.** `Ui.Button` and `label` become pointers to interned symbols, so matching a tag is a pointer compare and finding a field by name happens once per layout rather than per access. Tags written in source are interned at load; if code ever builds a tag from runtime text, that needs a runtime intern step with a table that grows. Step 7 assumes it.

**Arena allocation of nodes per widget.** Graph nodes are `Rc<dyn Impl>` with weak subscriber lists, and a `rebuild` firing frees a subtree by dropping counts one at a time. Nodes held in a per-node-tree arena, addressed by generational index, drop the per-node count traffic and give locality, and node lifetime follows build structure exactly: a rebuild frees its subtree by freeing its arena block. Reference-counted graphs with feedback also have to be careful about cycles, which is why a widget owns its cyclic placeholders as a unit; an arena has no such care. The cost is a large refactor of the runtime's core, and event and dynamic values in Prism still have to be one word, so the value becomes a heap object holding a node index. Of everything here this is the biggest change and the furthest out; it waits on a benchmark showing node churn from `rebuild` and `for` is a cost.

**A per-occurrence arena for temporaries.** Values allocated during an occurrence that do not outlive it, a lone float consumed by the next node, could come from a bump allocator reset at commit. Anything that escapes into a hold, a switch target or node state must be copied out, so every store

into persistent state needs a check, and holds commit at exactly the moment the arena would reset, so the order has to be right. This is the item most likely to turn out unnecessary once immediate floats and fusion are in.

**A fast path for `for` over a map that changed in one key.** The keyed list widget diffs the old map against the new one on every change, which is linear in the map. With persistent maps the diff can skip any subtree that is the same pointer in both, which makes the diff proportional to what changed. The common case, a hold folding one insertion or one update into a map, then costs one path from root to the changed key. This needs the collection representation below and the runtime's keyed list to use pointer equality on subtrees; it changes nothing about which blocks are built or torn down.

**Persistent maps and arrays.** Persistent maps and vectors (the `im` crate is the implementation choice; its structures give logarithmic updates with the rest shared, from memory) cover the hold case that in-place reuse cannot (below), and give the `for` diff above. Small maps should stay flat and switch representation past a size threshold, since persistent structures lose on point reads and small sizes. This is the one representation choice here that a World can observe indirectly, through the time a turn takes with a large list, which is to say not at all in a transcript.

**Copy-on-write in library collection operations, then compiler reuse.** An array push that checks the count and mutates when it is 1 is what `Arc::make_mut` does and needs no compiler help. Its limit is the hold: a fold over an event runs during the occurrence while the old value is still readable through the hold's tag, so the hold's own reference keeps the count at 2 and nothing is reused; the first write through the root copies the top level, one copy per occurrence per level touched, and every later write through the same level finds a count of 1. Reuse past that needs the runtime to know that the hold's only reader this occurrence is its own update, which is knowable from the graph in many cases, and a hook letting the hold hand its value to the update. The general form, compiler-inserted ownership so that a last use hands its value on instead of copying, is what Koka's Perceus and Lean 4 do (from memory); it needs the closure compiler to know which uses are last. The `update` blocks later in this chapter are the language feature this optimisation exists to make cheap.

**Fused map chains.** `f` mapped over `g` mapped over an event becomes one node running `f . g`: one subscription, one propagation step, and no intermediate payload, which removes the box when the intermediate is a lone float. It is safe only when the intermediate has no other subscriber, which the compiler sees at elaboration in most cases and the runtime's builder sees when a map subscribes to a map with one subscriber. The compiler already lowers one lifted expression to one leaf, so `$a * 2 + 1` is one map with one closure; fusion extends that across bindings.

**Typed numeric arrays.** An array whose element type is statically `Float` stores raw `f64`s with a descriptor saying so, the way records do. Vertex lists, plot data and audio buffers stop being arrays of boxes. Generic code is erased, so an array it builds gets the general layout; raw arrays come from code where the element type is static. Waits on the record layout machinery and a program that holds a large numeric array, which a form does not and a chart does.

**Layout-keyed field access caches.** Each field-access site remembers the last layout it saw and the field's offset, so a repeat read is one pointer compare and a load, which is the inline cache of the Self and JavaScript VMs (from memory). A small mutable cache per site in compiled code, which is fine on a single-threaded runtime and one more reason the threading decision comes first.

## Testing the representation

The enum is the oracle: every test runs against both implementations through the public API, so the word is checked against something simple enough to trust.

| Test | What it catches |
|---|---|
| Property tests on clone and drop, counted with a test descriptor recording drops | Leaks and double frees in the inline clone and the descriptor drop path; a heap value dropped while an immediate copy of the word is still around |
| Integer round trip for every `i64` and `u64` edge value (0, ±1, the immediate bounds and one past, MIN, MAX) | Off-by-one in the immediate range; a value that fits but was boxed, or the reverse, which breaks canonical form and therefore equality |
| Tagged `+ - * /` against an `i128` oracle on random mixed-width pairs and all edge pairs | Fast-path overflow detection disagreeing with the specification; $-2\hat{}60$ / $-1$ not boxing |
| Eq and hash across immediate and boxed integers, and across both implementations | A boxed integer equal to an immediate; hash disagreeing with eq |
| `size_of::<Option<Value>>() == size_of::<Value>()` in both profiles | Losing the niche, which would quietly grow every optional |
| Downcast mismatch: cross-kind in both profiles, within-kind in debug, boxed `Int` as boxed `Nat` in both | A release build reading a word as the wrong kind; the shadow not checking the full type; a descriptor type id that does not distinguish two boxed kinds |
| `match` on a payload-free variant index out of range, and `Some n` read as `n` in release | A within-kind confusion in release becoming a memory error instead of a wrong answer; the gap table above being wrong about any row |
| Nested optionals, generic and monomorphic | The `Some`-box rule collapsing levels that must stay distinct |
| Allocation counts under a counting allocator: immediate division, record of four floats, pointer element from World to arm | Hidden boxing where the design promises none |
| Miri with strict provenance over the module and its tests | Pointers rebuilt from integers, misaligned headers on wasm32, use after free in drop |
| The conformance suite on enum and word, debug and release | Anything the unit tests miss that changes a transcript |
| A wasm32 build of the suite | Assumptions about 64-bit words, 61-bit payloads or eight-byte native alignment |

Each optimisation brings its own test before it lands: immediate floats round-trip every double with the right encoding; reuse and copy-on-write are checked by keeping a second reference, updating, and comparing the old value; persistent maps run the same `for` diff tests as flat maps; fused maps produce the same event stream as unfused ones on the harness; the incremental element list is diffed against the rebuilt one on every occurrence of every suite program.

## What it must not change, and what triggers it

Nothing here may change a transcript, a type, or the meaning of overflow. The trigger is a profile: a suite program, or a real program under a GUI World, whose time per occurrence is dominated by the value module rather than by propagation. The urgent subset can go in whenever someone is in

the module anyway. The full design waits for that profile, because without it the benchmark in step 8 has nothing to compare against and the word would be adopted on faith.

## Parallel runtimes

The runtime is single-threaded: `Rc`, `RefCell`, one entry point, no locks. That is a property of the first implementation and not a claim about the model, and the model has three kinds of parallelism in it, each visible in the structure of the runtime as it stands. None of them is needed to make the language exist. Each is a second runtime variant or a World arrangement, and none touches a program. Each is taken here far enough to say what it costs and what has to be true for it to pay.

### Fork-join per height

Within an occurrence, every node at one height depends only on nodes at lower heights, so every node at one height is independent of every other at that height. Propagation is "pop the lowest height, run every action in it," and the actions in one height can run on as many threads as there are, with a join before the next height. That is the whole idea, and the runtime's action map, a `BTreeMap<usize, Vec<Action>>`, is already the work list it needs.

What has to change for it to be safe:

- Graph types become `Sync`. Subscriber lists are read during propagation and written only at build, so they can be behind a read-write lock that is write-locked only when a `rebuild` or `for` builds mid-occurrence. Node state that an action writes, a fired mark and a cached current-occurrence value, is per node and written by exactly one action per occurrence, which is the at-most-once law doing work for the implementation: no two actions in one height write the same node, so the writes need no lock, only the memory ordering a join gives.
- Buffered commits stay buffered. Holds and switch targets commit at `usize::MAX`, which is one height and runs after the join, so every write that could conflict is already serialised by the design that makes occurrences transactions.
- Construction mid-occurrence is the hard case. A `rebuild` rebuilding at height h builds nodes that schedule at heights above h, and building writes subscriber lists. Two `rebuild`s at the same height building at once would both write the action map. The simple rule: an action that builds takes a global build lock for the length of its build, and the catch-up logic that gives the new nodes the occurrence's earlier firings runs under the same lock. Builds are rare compared with propagation, so the lock is cheap; and a World whose occurrences are all rebuilds is not the World that wanted parallel propagation.
- The scheduling check, a node scheduling at or below the current height is a cycle, stays as is: the current height is a shared atomic read, and the comparison is the same.
- Closures are `send + Sync` in this variant, and values use atomic counts, or the per-object sharing mark described under the representation above, which is the one place the two variants' value modules differ.

The cost is a second runtime crate or a feature flag over the first, with the suite run under both. The payoff exists only when heights are wide: a program with a thousand independent nodes at one height, which is a large `for` over a list whose entries each do a little work. A form with forty fields has heights of width two or three, and a thread pool would make it slower. That is why this is a variant and not the runtime: it has to be measured on the program that wants it.

## Optimistic occurrences

Across occurrences, an occurrence is a transaction with buffered writes. Its read set is every tag read, every hold's value side read, and the element list as it opened; its write set is every hold commit, every switch target and every list change. Two occurrences whose sets do not overlap could run at once and commit in order, and one whose read set overlaps the other's write set has to run after it, which is ordinary optimistic concurrency with the runtime's rollback path as the retry.

The sketch: the World hands the runtime a queue of input sets instead of one. The runtime runs occurrence n on one thread and speculatively starts n+1 on another against the same graph, with n+1's reads recorded. When n commits, its write set is compared with n+1's read set; if disjoint, n+1 commits next; if not, n+1 is rolled back (drop its buffer, clear its fired marks, drop any nodes it built) and rerun against the committed state. Reads during n+1 see the state before n, which is wrong only if n writes something n+1 read, which is exactly the overlap test.

What it needs that the runtime does not have: versioned node state, so that n+1's reads of a hold see the value before n while n's commit is being prepared, which is one extra slot per hold; per-occurrence fired marks instead of one, which is a small array per node; and the rollback path generalised from "the whole occurrence failed" to "this one of two." It also needs the World's queue to be deep, which it is for a server World with many connections and is not for a GUI World that runs one occurrence per input event.

It preserves the transcript. Occurrences commit in the order the World queued them, and a speculative occurrence that conflicted is rerun before it commits, so the sequence of committed states is the sequence a sequential runtime would produce. Determinism is the law that makes optimistic execution invisible, and it is why this variant can exist at all.

## Shards

Across the program, independent subtrees connected only through elements can be separate runtimes, each on its own thread, with a World that routes between them. A server with one runtime per connection, a shell that hosts other programs, a GUI with one runtime per window: in each case the subtrees share nothing but what passes through the element list, and the run call is cheap enough that a World can make many of them.

This one needs no runtime support at all. It needs a World that keeps a runtime per shard, routes each input to the shard whose list has the path, and merges the shards' element lists and request firings for its own components. State elements are the one subtlety: a state element's value is identified by its element value and shared by every slot with that value, and two shards with a `Clock` slot each must see the same clock in the same turn. The World's snapshot rule already says what to do, one snapshot per occurrence answered the same to everyone; a sharded World takes the snapshot once per turn and answers every shard's oracle from it. The equality law then holds across shards because the World made it hold, which is the law's intent: it binds the World, not the runtime.

Communication between shards is the interesting part and it falls out of interception. A program running as a shard that wants to talk to another shard does so through elements: it creates a request element the World routes to the other shard as a response, which is how two programs under a shell talk already. Nothing new. What a shard cannot do is share a dynamic with another shard, because a dynamic is a node and nodes do not cross the run call, and that is the right restriction: the moment two shards share a node they are one runtime.

### What they must not change

The transcript, the laws, the run call. A program cannot tell which variant it runs under, and the suite run under every variant is the test. The language has no threads, no priorities and no phases, and none of these variants adds one.

### The fact it waits on

Fork-join: a profile under a GUI World showing wide heights, which means a large `for`. Optimistic: a server World with a deep input queue and occurrences that rarely conflict, which is a server whose connections are independent, which is most servers, and which is also served by shards with no runtime work at all, so shards first. Shards: the first World that hosts more than one program or serves more than one connection, which is the shell in the examples chapter and a server World after it.

## Performance and constant factors

The runtime chapter's performance notes say what is proportional to what. This section is about the places where the language as specified asks the implementation for work proportional to the size of the program or the screen when the change was small, and about constant factors, which matter for the applications Prism is aimed at: a GUI redraws at the frame rate, a text editor handles every keystroke, a server handles every request. None of the items below changes a transcript, and each has to leave the differential test empty.

*GUI frames.* A container such as `@VBox` captures its children's draw lists, offsets them and re-emits them, and `@Window` emits one `Gui.Scene` whose `draw` field is the whole list. A label change therefore rebuilds every list on its path to the root, about depth times total draws, and the World then diffs or re-rasterizes the window. Under `Gui.Frames` driving an animation this dominates everything else. Draw lists want to be persistent shared values: reference-counted nodes, a group draw that holds a child list by identity, and a cached hash. Re-emitting then costs the changed path, the World's diff takes a pointer-equality fast path to skip unchanged subtrees, and the rasterizer gets damage rectangles from the diff.

*Equal values are not deduplicated.* A derived dynamic whose operands change fires with its new value even when the value is equal to the old one, so `$is_selected` over `$selected` changes in every row when one row is clicked, and every row's `rebuild` over it rebuilds with a new slot and a new path. The semantics are right, since a rebuild is observable. The cost is a rebuild per row per click, and the remedy is `$Distinct` plus a lint warning for a `rebuild` over a derived dynamic that is not wrapped in it.

*Collections.* `Filter` and `MapValues` over a map are linear in the map for every change to it. `for` is incremental per key, but the keyed list widget diffs old and new key sets, which is linear in the keys unless the map can say what changed. Three things fix this together. Maps are persistent balanced trees with structural sharing, since otherwise `Insert` is a copy. Diffing two maps compares shared subtrees by pointer, so a diff costs the changes times log n. And patch-carrying dynamics, in the manner of Reflex's `Incremental` (from memory), let a derived collection say what changed instead of being recomputed.

*Walks that touch the whole program.* The open-time snapshot of every response fan is a walk over every interceptor in the tree, and every GUI container is an interceptor, so a quiet occurrence pays for the whole tree. The snapshot is not needed for correctness: a list's value side already lags to

commit, so reading the behavior at lookup time returns the list as the occurrence opened. The run call's `Outcome` carries the full element list on every call, including every `Defer` occurrence; the patch should be the primary output, with the full list a shared value a World can ask for. The incremental element list belongs in the first implementation, because a GUI meets the naive cost at once.

*Keys.* The supplier's memo and the oracle's cache are keyed by whole element values, and comparing two elements whose fields hold a large text or a draw list costs the size of the field. Paths cost their depth. Interning elements and paths with a cached hash, and comparing by pointer first, makes both constant in the common case.

*Plain compute.* Plain expressions compile to a tree of boxed closures over a two-word value. For tight loops that is probably about an order of magnitude slower than native Rust; the figure is an estimate from similar closure compilers and has not been measured. It matters because layout, hit-testing and text editing are Prism code. The hot prelude functions, the layout arithmetic and the text operations, should be native, and a real backend is an option behind them.

*Replay in the Haskell implementation.* Rollback by replay keeps the log of every committed run call, so memory grows with the length of a run. It is adequate for the differential test, which is the only use the plan gives that implementation.

## The fact it waits on

A profile of the text editor or the thousand-row todo list under the software World. Both exist in the book's examples, and both hit the first four items at once, so the order above is the order of the likely wins: shared draw lists, distinct dynamics, persistent maps, then the fixed per-occurrence walks.

## Chapter 12

# The Rust language implementation

A Prism compiler does not emit code that computes. It emits code that, run once, calls the runtime's constructors in the right order with the right captures, and the product of running that code is a graph, which does the computing for the rest of the program's life. Everything in the `prism_lang` crate follows from taking that sentence seriously: the frontend's only job is to decide what every line means and write the decision down in one tree, the typed AST; a lint checks that the tree is consistent after every pass that touches it; and a closure compiler walks the tree once and hands the runtime an `IrBlueprint`, a value the runtime cannot tell from a blueprint written in Rust by hand. There is no instruction set, no bytecode, no second syntax and no verifier between two forms, because there is only one form. This chapter is that crate, stage by stage, with its data structures, its invariants, its text form, its diagnostics and its tests, written so that a reader who wants to change a decision can see what the decision costs.

## The thesis

You have written a compiler for a language that runs. Its back half was a loop: fetch an instruction, do it, fetch the next, and every feature you added had to find its place inside that loop, with a frame, a stack and a scheduler to keep it company. Then you wrote a reactive framework on top of a language like that, and the framework turned out to be a second runtime inside the first: its own notion of when things run, its own dirty-tracking, its own idea of what a closure may capture, fighting the host's loop for control of time. The afternoon you spent finding out why a handler ran twice was an afternoon spent at the seam between the two.

Prism has no seam because it has no first runtime. A widget block does not run; it is built. Reading it top to bottom is reading a recipe for a node: these children, these holds, these edges between events, these slots. The code that follows the recipe is ordinary code, it runs once per build, and when it returns the node exists and the recipe is finished with. From then on the runtime of the previous chapter owns time, and the only thing left of the program is a graph of nodes with closures in them, which the runtime calls when an occurrence reaches them. The compiler therefore has two kinds of thing to produce, and only two: *construction code*, which runs once per build and makes builder calls, and *leaves*, pure closures the graph calls at occurrence time, which take values and return a value and cannot build, schedule or keep state.

That split is the whole architecture. Suppose a compiler for this language were written the ordinary way, as a bytecode machine with a dispatch loop. Its loop would run exactly once per build and then sit idle while the runtime propagated, because nothing in the program is a loop; the only code that runs repeatedly is the leaves, and the leaves are reached from the runtime's closures, not from any loop the compiler owns. A VM for a language whose programs run once is a VM that spends its life waiting to be called back. The closure compiler is what that VM becomes when the waiting is removed: each node of the typed AST compiled once into a Rust closure, construction nodes into closures that take the builder, leaves into closures that take values, and the runtime holding the

leaves exactly as it holds a hand-written Rust blueprint's.

## The product is a graph

Three consequences fall out of the thesis, and each one is a thing the compiler can stop worrying about.

*The compiler never decides when anything runs.* The occurrence, the height order, the commit, catch-up for a widget built mid-occurrence: all of that is the runtime's, and a compiled widget gets it by being a widget. A `rebuild` block compiled to an `IrBlueprint` is rebuilt by the runtime's `rebuild` at the right height in the right occurrence, and the block's code does not know whether this is its first build or its hundredth. A leaf does not know what height it runs at. The compiler emits no scheduling code because there is nothing to schedule.

*Construction is a value.* A widget function plus its captures is a thing the runtime can build from at any time: at the first build, at every `rebuild` firing, at every `for` key's appearance, at every state supply. The compiler therefore produces blueprints as values, `IrBlueprint { program, block, captures }`, and the same value type serves every one of those places. This is why a `rebuild` can rebuild into a blueprint of a different kind, why a supplier is a function from an element to a blueprint, and why compiled and hand-written blueprints sit side by side in one node tree.

*Construction ties knots.* Prism allows recursive bindings in a widget block and the language is strict, so a knot cannot be tied with laziness the way a Haskell `mdo` ties one. The knot is explicit: open a placeholder, build what refers to it, close it to the real node, and never sample it while it is open. The compiler's job is to place the placeholders, and the resolver's dependency analysis says exactly where: at the delayed edges, which every legal cycle has at least one of. Nothing else in the compiler is about recursion.

## What the compiler does not need

The refusals that shrink the runtime shrink the compiler. What is absent, listed because a model handed this chapter will otherwise reach for each of these out of habit:

*No inference over functors.* The sigils say what every name is. `$count + !delta` is an event because its operands say so, and no unification variable ever stands for "some functor". Hindley–Milner runs on the value layer, where it is principal and well understood, and never sees time. The lifting rule is a table over the operands' outermost sigils, which the elaborator looks up; it is not a search through instances.

*No effect system, no monad transformer machinery.* There is one block type per functor and the set is fixed: widget blocks, `rebuild`, `for`, `match`. A widget block is a `do` block without the `m`, and the compiler knows which `m` it is by the sigil on the block.

*No reconciliation, no lifecycle analysis.* Construction is ordinary code and teardown is ownership. The compiler emits no diffing, no keys for a reconciler to match on (the keys of a `for` are the program's own), and no mount or unmount hooks, because nothing is triggered by a widget being dropped or an element appearing.

*No escape analysis for nodes.* Nodes are not values. A program cannot name a node, store it, pass it, or reach into it, so the compiler never asks whether a node outlives its scope.

*No exception lowering.* There are no exceptions. A program's own failures are values; a runtime error aborts the occurrence as a transaction, and the compiler's only contribution is the span it attaches to every node so that the abort names a line.

*No code generation in the usual sense.* No register allocation, no instruction selection, no calling convention beyond Rust's own. A plain-value expression compiles to a closure over an environment vector; an occurrence-time expression compiles to the closure that the corresponding graph node will call. The hard problems of a code generator exist in this crate only in the way they exist in any Rust program that builds closures.

*No garbage collector in the output.* Nodes own their closures, closures capture by index into the owning node's environment, and the graph's reverse edges are weak. Closure conversion makes this mechanical, and the lint checks it. A compiler that got this wrong would put a reference cycle on the heap that nothing frees; the rule that prevents it is one rule, stated under closure conversion below.

What is left is three jobs and one intermediate form: decide meaning (elaboration), check the decision is consistent (lint), emit the construction (the closure compiler). The stages before elaboration are the ordinary front of any compiler, and the rest of this chapter is in pipeline order.

## The pipeline

```
source text
   │  lex + layout        tokens with spans; indent/dedent; inline colons as plain tokens
   ▼
token stream
   │  parse               paren-less calls, precedence, blocks, patterns, declarations, tests
   ▼
AST (every node spanned)
   │  resolve             scopes, the pronoun, namespaces, the prelude, SCC groups with edge labels
   ▼
resolved AST
   │  elaborate           HM on values; lifting as explicit nodes; traits; marks; causality; exhaustiveness
   ▼
typed AST  ─── print/parse ───  text form (goldens; the Haskell bridge)
   │  convert             function ids, capture lists, placeholders and closes
   ▼
converted typed AST  ──  lint  (after every pass, in every test)
   │  compile             steps, closures, the tag table, boxing
   ▼
IrBlueprint  ──────────────────────  the runtime's DynBlueprint
```

One crate, `prism_lang`, with no dependency on `prism_runtime` until the last stage, which lives in its own module and is the only module that names a runtime type. The frontend up to the text form can be built, tested and frozen without a runtime, and the Haskell frontend of a later chapter reproduces it byte for byte from the same specification.

```
prism_lang/src/
  lex.rs          Token, Lexer, the spacing rule
  layout.rs       the column stack; indent/dedent/separator tokens
  parse.rs        Parser; ast.rs the parse tree
  resolve.rs      scopes, groups, edge labels
  elab/           types.rs (Type, unification), lift.rs (the sigil table),
                  traits.rs (Eq/Ord/Num), causal.rs (the cycle check),
                  exhaust.rs, mod.rs (the walk); typed.rs the typed AST
  typed/text.rs   the text form: one definition, printer and parser
  convert.rs      closure conversion
  lint.rs         the checker
  ir/             program.rs (Program, Step, Function), compile.rs (the walk),
                  widget.rs (IrBlueprint), leaf.rs (the evaluator), boxing.rs
  tags.rs         the tag table, shared by elaboration and ir
  diag.rs         Diagnostic, Span, rendering
```

Every stage after the lexer is a function from one tree to the next, and every tree has a span on every node. Spans are the one thing a compiler cannot retrofit cheaply, so they are in `ast.rs` from the first commit and copied forward without exception; a typed-AST node that lacks a span is a lint error, not a convention.

## Lexer and layout

You have debugged a layout-sensitive language by adding a line and watching an unrelated block close three lines down. The reason was always the same: the lexer and the parser disagreed about who owned the decision, and the error pointed at the parser's symptom rather than the lexer's cause. Here the contract is that every layout decision is the lexer's, every colon decision is the parser's, and the type checker never learns that there was a question. The language chapter states the rules; this section is what a lexer that follows them looks like, down to the token type and the algorithm, so that the two implementations' token streams can be diffed against the layout corpus.

### Tokens

```
pub struct Token { pub kind: TokenKind, pub span: Span, pub prefix: bool }

pub enum TokenKind {
    // names, with any attached sigils as part of the token
    Lower(Sigils, Name),            // $count, !delta, _, $_, x
    Upper(Sigils, Name),            // @Button, !Leftmost, Insert, Vector
    Qualified(Sigils, Vec<Name>),   // Ui.Button, Fs.Contents, Wrap.Add
    // literals
    Nat(u128),                      // integer-spelled; typed later
    Float(f64),                     // decimal-spelled
    Text(Vec<TextPart>),            // "Count: {$count}": literal and interpolated parts
    Char(char),
    // keywords
    Let, Type, Tag, Of, Fn, Match, If, Else, For, Dyn, Intercept, Widget,
    Record, Choice, Map, Array, Unit, Test, Ignore, On, Then, Key, Ix, Jx, Kx, Empty,
    Not, And, Or,                   // the boolean keywords: not a, a and b, a or b
    // operators and punctuation
    Bind, Define,                   // <-  =      (`then`, the hold operator, is the keyword above)
    Lt, Le, Gt, Ge, EqEq, NotEq,    // <  <=  >  >=  ==  !=   (longest match: `<-` and `<=` are single tokens)
    Merge,                          // |   (never the lambda bars)
    Plus, Minus, Star, Slash,       // + - * /     (Plus/Minus take the `prefix` flag)
    PlusInt, MinusInt, StarInt, SlashInt,       // +~ -~ *~ /~
    PlusFloat, MinusFloat, StarFloat, SlashFloat, // +. -. *. /.
    ToNat,                          // +?  (prefix only)
    Arrow, MapArrow, Colon,         // ->  ==>  :   (Colon is the inline colon)
    Comma, Dot, DotDot, Backslash,  // ,  .  ..  \      (Dot takes the `prefix` flag)
    ChainDot,                       // a `.` that is the first token of its line: a chain step
    Semi,                           // ;   separates statements in an inline block
    LambdaOpen, LambdaClose,        // the bars of \|a, b|
    LParen, RParen, LBracket, RBracket, LBrace, RBrace,   // LBracket takes the `prefix` flag
    TupleOpen, TupleClose,          // ((  ))   a tuple, a tuple type, a tuple pattern
    ArrayOpen, ArrayClose,          // [[  ]]   an array literal
    LAngle, RAngle,                 // < >  only inside generic parameter lists
    // free sigils: a sigil not attached to the following token
    FreeSigil(Sigil),
    // layout
    BlockOpen, BlockClose, Sep,     // { } and the layout separator of the corpus notation
    Eof,
```

```
}

pub struct Sigils(SmallVec<[Sigil; 3]>);  // leftmost outermost: !$ is [Event, Dyn]
pub enum Sigil { Event, Dyn, Widget, Opt }

pub enum TextPart { Lit(String), Interp(Vec<Token>) }   // Interp: the tokens between { and the matching }
```

Five things about this type are decisions rather than transcription.

*Sigils are part of a name token.* `$count` is one token, `Lower([Dyn], "count")`, and the parser never has to glue a sigil to a name. The alternative, a sigil token followed by a name token, would make the attachment rule the parser's and would let a later pass ask whether a space had been there; the language says the space is the whole difference between `$ x` and `$x`, so the lexer answers it once. A sigil that is not attached to a name is `FreeSigil`, and the parser treats it as a prefix operator at the precedence the language chapter gives it. That includes the `$` of `$"Increment"`: a text literal has no sigil in its token, so the `$` before the quote is a `FreeSigil` that makes the literal a constant dynamic.

*The merge bar and the lambda bars are different tokens.* `\|a, b| a | b` lexes as `Backslash LambdaOpen a Comma b LambdaClose a Merge b`. The lexer knows which is which by position: a `|` immediately after `\` is `LambdaOpen`, the next `|` at bracket depth zero relative to it is `LambdaClose`, and every other `|` is `Merge`. Inside the bars only patterns appear, so there is no merge to confuse them with.

*`prefix` is a flag on the token, set by the spacing rule.* `Plus`, `Minus`, `Dot`, `LBracket` and `ToNat` can be prefix or binary, and the lexer decides from whitespace: prefix when preceded by whitespace, an open bracket, a comma, a `\`, or the start of a line, and followed immediately by a non-space; binary otherwise. The parser reads the flag and nothing else. `F -1` is `F Minus(prefix) Nat(1)`; `F - 1` is `F Minus(binary) Nat(1)`; `F-1` is `F Minus(binary) Nat(1)`; `\.x` is `Backslash Dot(prefix) x`; `a.x` is `a Dot(binary) x`; `xs[0]` is `xs LBracket(binary) Nat(0) RBracket`, an index; `\[0]` is a prefix `LBracket`, the pronoun's index. The sigils never take the flag because a sigil is never binary, and `*` and `/` never take it because they are never prefix.

*Doubled brackets are one token each, found by matching.* `((a, b))` is a pair and `[[1, 2, 3]]` an array, while `((Size $elems) + 1)` is two groups and `xs[ys[0]]` is two indexings, so the lexer cannot fuse `((` or `]]` by looking at two characters. It fuses by pairs instead: after the character layer, a pass matches every bracket with its partner, and two opening brackets of one kind with no whitespace between them become one `TupleOpen` or `ArrayOpen` exactly when their two partners are also adjacent, with no whitespace, in the mirrored order. The pass runs outermost first, left to right, and a bracket already fused is not fused again. An opening `[` that follows an expression with no whitespace is an index and never starts a pair, so a map keyed by arrays is read with `m[[[1, 2]]]`: the index bracket, then the array `[[1, 2]]`. So `((G b, c))` is `TupleOpen G b Comma c TupleClose`; `(((G b, c), d))` fuses the outer two and leaves the third a group, which is the pair of `G(b, c)` and `d`; `((F (G x)))` fuses the outer two around a group; `[[]]` is `ArrayOpen ArrayClose`, the empty array; and `((Size $elems) + 1)` fuses nothing, because the inner group's partner is followed by `+`. A doubled group with nothing between the two levels, `((x))`, would mean nothing as two groups, so giving the spelling to tuples costs no program anything; the parser rejects a one-element tuple with "a tuple has two or more elements; to group, write `(x)`". In a type, a fused `[[A]]` is read back as the array of arrays it spells, since a type has no array literals.

*A `.` that starts a line is its own token.* When the first token of a line is a `.` followed by an uppercase name, the lexer emits `ChainDot` instead of `Dot`. Layout has already made such a line a continuation (it is indented more than its block), so the token changes nothing about blocks; it tells the parser that the line is a step of a method chain, which closes the call above it. A spaced `.F` in the middle

of a line stays a prefix `Dot`, and the parser gives it the pronoun reading or an error, below. The lexer can decide this from the line structure it already tracks, and the parser then never asks where a line began.

`;` lexes as `Semi` and only as `Semi`. It never becomes a layout `Sep`, because the two end different things: a `Sep` ends a statement of a layout block and with it every inline block open on the line, while a `Semi` separates statements inside the innermost block, inline or layout, and leaves that block open.

A text literal is one token holding a list of parts, and each interpolation part is itself a token stream, lexed with the same lexer in bracket mode (layout suspended) from a `{` up to the matching `}`. Braces inside the interpolation nest, so a map literal or a nested text can sit in it. `\{` and `\}` are literal braces and join the surrounding literal part; a `$` in text is an ordinary character, so `"cost: $5"` has one part and `"$count"` is the six characters it shows. `"Count: {$count}"` is `Text([Lit("Count: "), Interp([Lower([Dyn], "count")])])`. The literal has no sigil of its own: the sigil of the interpolated name is the name token's, and the elaborator, not the lexer, decides that a literal with a dynamic inside is a dynamic.

The corpus prints a text token as `text(` followed by its parts separated by commas: a literal part as the quoted string with `\{` and `\}` replaced by the braces themselves and `\"` kept, and an interpolation as `interp(` followed by the tokens of its expression in the corpus's own notation, then `)`. A text without interpolation has one part, `text("a")`; the empty text is `text("")`; adjacent literal pieces are one part, so `"n={x}, \{ok\}"` is `text("n=", interp(ident(x)), ", {ok}")`, and a leading or trailing interpolation has no empty literal beside it, so `"{x}"` is `text(interp(ident(x)))`. Name tokens print their sigils inside the parentheses, `ident($n)`. This is the one notation for text tokens: the conformance chapter's layout corpus and the Haskell lexer's output use it unchanged.

Names are classified by their first character at lex time, and a name the lexer cannot classify is an error token with the message the language chapter gives, reported against the right span. A lowercase-first name with a capital inside, `isClick`, classifies as a value name and lexes as an ordinary `Lower` token; the lexer attaches a warning (`Level::Warning`, rendered like any diagnostic, below) that says to write `is_click`, and compilation continues. The warning is a lint of the source and the lexer is the only stage that sees the spelling, so it is raised there. A qualified name is one token only when there is no whitespace anywhere in it and every part is uppercase; `Gui.*` is three tokens, `Upper(Gui)`, `Dot(binary)`, `Star`, which the parser recognises as the wildcard pattern.

## The layout algorithm

The lexer runs in two layers: a character layer that produces raw tokens with their line and column, and a layout layer that consumes raw tokens and emits `BlockOpen`, `BlockClose` and `Sep` among them. The layout layer keeps three pieces of state and no more: a stack of block columns, the bracket depth, and whether the last significant token on the current line was a colon.

```
state:
  stack   : [Int]        initially [0]
  depth   : Int          bracket depth, initially 0
  pending : Bool         a line-final ':' is waiting for its block

on each raw line L with first-token column c (blank and comment-only lines are skipped):
  if depth > 0:
      emit L's tokens; continue            # inside brackets nothing below applies
  if pending:
      if c <= top(stack): error "expected an indented block after ':'" at the colon
      push c; emit BlockOpen; pending := false
      emit L's tokens; continue
```

```
  if c == top(stack):
      emit Sep; emit L's tokens; continue  # a new statement of the current block
  if c > top(stack):
      emit L's tokens; continue            # a continuation of the current statement
  # c < top(stack): close blocks until something matches
  while c < top(stack):
      pop; emit BlockClose
  if c != top(stack):
      error "unindent does not match any open block" (c, the open columns)
  emit Sep; emit L's tokens

on each raw token t within a line:
  if t is '(' '[' '{' '((' '[[':                        depth += 1   # fused brackets count once; a text
  part's braces are inside its token
  if t is ')' ']' '}' '))' ']]':                        depth -= 1
  if t is ':' and depth == 0:
      if t is the last token on its line (comments ignored): pending := true   # a layout block
      else: emit Colon                                                         # an inline block
  else if t is '.' and t is the first token of L and the next token is an uppercase name:
      emit ChainDot                                                            # a chain step
  else: emit t                                                                 # ';' is emitted as Semi

at end of file:
  if pending: error "expected an indented block after ':'"
  while stack has more than [0]: pop; emit BlockClose
  emit Eof
```

This is the whole of layout, and each piece of state answers to one rule of the language chapter by its content: the stack is the column rule (a block's column is its first statement's); `pending` is the line-final-colon rule; the three comparisons are the same-column, deeper and shallower cases of the next-line rule; `Colon` is the inline-block rule; `depth` is the brackets-suspend-layout rule; a block in the middle of a statement needs nothing special, because when its `BlockClose` is emitted the next line's column is compared against the enclosing block as usual; `else` is an ordinary token the parser attaches; top level is the initial stack; blank and comment-only lines are skipped before the comparison; end of file closes everything.

Two of the corpus cases are the ones that catch a wrong implementation. A dedent of two levels at once (`@a:` containing `!x <- rebuild $b:` containing `@ unit`, then `@c:` at column 0) must emit two `BlockClose` tokens and then a `Sep`, which the loop does because it pops until the column matches and then falls through to the separator. And a `for`-then-merge (`| !other` at column 6 under a block at 8 inside a block at 4) must be an error naming the open columns, which the loop does because after popping to 4 the column is still not equal. An implementation that treats "less than top" as "close one block and continue the statement" gets the second case wrong and silently parses a program the language rejects.

## The inline colon

A colon with tokens after it on the same line is not a layout decision; the lexer emits it as `Colon` and the parser interprets it. The parser has one routine for every place the grammar wants a block:

```
block(ctx):
  if next is BlockOpen: consume; stmts := statements separated by Sep or Semi until BlockClose; consume
                        return Block(stmts)
  if next is Colon:     consume; stmts := [statement(ctx)]
                        while next is Semi: consume; stmts.push(statement(ctx))
                        expect the inline terminator; return Block(stmts)
  error "expected ':' and a block"
```

```
inline terminator: a Sep, a BlockClose, an Else, a closing bracket at this depth, or Eof
```

A `Semi` belongs to the innermost block open where it appears. `if c: Log x; Log y` is an `if` whose inline block has two statements, and `type Pair<a, b> = record: first: a; second: b` is a record whose inline block has two fields. A field's `name: type` is not a block: in a `record` or `tag` block the parser reads a statement as a field, and the colon in it as the field's separator, so the `;` after `first: a` belongs to the record. A `match` arm's colon does open a block, so `match x: A: 1; B: 2` puts `B: 2` inside the first arm, where it is an error; a match on one line with several arms is written with layout. A layout block accepts `;` too, as a second way to start a statement on the same line, and never needs it.

The same `Colon` token therefore means a one-statement block after `match` arms, `if c`, `rebuild $x`, `for $m`, `@widget`, a `map:` key or a `record:` field, and means a type ascription after a pattern or inside parentheses: `let n: Nat = 0`, `Pair (x: Nat), y`. The parser knows which from what it was parsing when it met the colon, and never from spacing or position. If the inline statement itself ends in a line-final colon, the lexer has already turned that into a `BlockOpen`, and the parser's `statement` consumes the layout block as part of the inline statement, which is how `x: match y: Some v: rebuild $v:` with a block under it works: each `Colon` opens an inline block whose statement ends in the next, the last is a `BlockOpen`, and the `BlockClose` after the layout block closes every inline block above it, because the terminator of an inline block includes a `BlockClose` it did not open.

`else` terminates an inline block and nothing else does besides line structure, so `if c: 3 else: 4` inside parentheses parses as two inline blocks, and `if c: match x:` followed by a layout block of arms and then `else:` at the column of `if` is legal, defined, and in the corpus.

Suppose instead the lexer decided the inline colon too, emitting `BlockOpen` and a synthetic `BlockClose` at the end of the line. Then `(x: Nat)` would need the lexer to know that a colon inside parentheses after a pattern is an ascription, which is parsing, and `if c: 3 else: 4` would need it to know that `else` ends a block, which is parsing. The split chosen puts every decision that needs grammar in the parser and every decision that needs columns in the lexer, and the layout corpus can be checked against the lexer alone.

### What the lexer keeps

The only context the lexer keeps is the column stack, the bracket depth, the bracket pairs it matched for fusing, and whether the previous character was whitespace. The token stream for a program is a pure function of its characters. The lexer has no symbol table, no knowledge of which names are widget functions, and no mode for "inside a widget block"; a lexer that needed any of those would be one whose output a reader could not predict from the text, and the layout corpus, which both implementations keep as a golden, is the test that neither one grew such a mode.

## Parser

The parser has one hard problem, paren-less calls, and the rest is a precedence climber with blocks. The hard problem is hard in the way that a reader's problem is hard: given `Insert items, id, Todo label, False`, who owns the commas? The language answers "the outermost call on the line", and once that rule is in the parser the rest of the grammar is a transcription. This section gives the parse tree and a grammar sketch for every construct, with the places a hand-written recursive-descent parser has to be careful marked as such, so that the Rust and Haskell parsers can be written from the same sketch and the goldens can say whether they agree.

## The parse tree

```
pub struct Module { pub items: Vec<Item> }

pub enum Item {
    Let     { name: SName, params: Vec<TypeParam>, ty: Type, value: Expr, span: Span },   // annotation
required
    Widget  { name: SName, body: Block, span: Span },                                      // @counter: ...
    TypeDecl{ name: Name, params: Vec<Name>, def: TypeDef, span: Span },
    TagDecl { tag: QName, of: Name, fields: Vec<Field>, requests: Option<Type>,
              responds: Option<Type>, state: Option<Type>, span: Span },
    Test    { name: Name, body: Expr, span: Span },                                        // test name: expr
}

pub enum TypeDef {
    Record(Vec<Field>), Choice(Vec<Alt>), Newtype(Type), Unit(Name), Open,              // `choice ..`
}
pub struct Field { pub name: SName, pub ty: Type }
pub struct Alt   { pub name: Name, pub payload: Option<Type> }

pub struct Block { pub stmts: Vec<Stmt>, pub span: Span }

pub enum Stmt {
    Bind   { pat: Pat, widget: Expr, span: Span },      // pat <- widget
    Build  { widget: Expr, span: Span },                // widget          (result discarded, or the block's
result)
    Let    { pat: Pat, value: Expr, span: Span },       // let pat = value
    Hold   { name: SName, init: Expr, event: Expr, span: Span },   // $x <- init then event
    Result { value: Expr, span: Span },                 // @ value   (a free @ on the last statement)
    Expr   { value: Expr, span: Span },                 // in non-widget blocks: the value
}

pub struct SName { pub sigils: Sigils, pub name: Name, pub span: Span }
pub struct QName { pub parts: Vec<Name>, pub span: Span }

pub enum Expr {
    Var(SName), QVar(Sigils, QName),
    Nat(u128), Float(f64), Char(char), Unit, Empty, EmptyMap,
    Text(Vec<TextPartAst>),                                  // "hi {name}"; a dynamic inside makes it a
dynamic, decided by the elaborator
    Array(Vec<Expr>), MapLit(Vec<(Expr, Expr)>), Tuple(Vec<Expr>),   // [[a, b]]  {k: v}  ((a, b))
    Call   { f: Box<Expr>, args: Vec<Expr> },                // F a, b   (F a, b)   x.F a, b   and a chain step
    Named  { ctor: Box<Expr>, args: Vec<(Name, Expr)> },     // Vector(x=1.0, y=2.0), Ui.Button(label="OK")
    RecordBlock { ctor: Box<Expr>, fields: Vec<(Name, Expr)> }, // Vector: x: 0.0 ...
    Binary { op: BinOp, l: Box<Expr>, r: Box<Expr> },        // + - * / +~ ... < <= > >= == != and or |  on
    Prefix { op: PrefixOp, e: Box<Expr> },                   // + - +? not and free sigils
    Field  { e: Box<Expr>, field: SName },                   // e.x  e.!changed
    Index  { e: Box<Expr>, i: Box<Expr> },                   // e[i]
    Lambda { params: Vec<Pat>, body: Box<Expr> },            // \body  (params = [_])  or  \|a, b| body
    Match  { scrutinee: Box<Expr>, arms: Vec<Arm> },
    If     { c: Box<Expr>, t: Box<Expr>, e: Box<Expr> },
    For    { key: Option<Pat>, item: Option<Pat>, coll: Box<Expr>, body: Block },   // for k, $item <- $items;
both None is the shorthand
    Dyn    { dynamic: Box<Expr>, body: Block },
    Intercept { child: Box<Expr>, responses: Box<Expr>, supplier: Box<Expr> },
    WidgetBlock(Block),                                      // @widget: ...
    LetIn  { stmts: Vec<Stmt>, value: Box<Expr> },           // a layout block of lets ending in a value
    Ascribe{ e: Box<Expr>, ty: Type },
}
pub struct Arm { pub pat: Pat, pub guard: Option<Expr>, pub body: Expr, pub span: Span }   // pat if guard:
body
```

```
pub enum Pat {
    Wild,                                  // ignore
    Bind(SName),                           // x, $x, _
    Lit(Lit), Unit,
    Tuple(Vec<Pat>),                       // ((a, b))
    Array  { elems: Vec<Pat>, rest: Option<Box<Pat>> },   // [[]]  [[a, b]]  [[head, ..rest]]; rest is a
binding or `ignore`
    Some(Box<Pat>), Nil,
    Ctor   { name: QName, payload: Option<Box<Pat>> },                // Circle r, Stopped, Clock
    Record { name: QName, fields: Vec<(Name, Pat)>, positional: Vec<Pat>, rest: bool }, // T(f=p, ..)
    Wildcard { ns: QName },                // Gui.*(..)
    Ascribe{ p: Box<Pat>, ty: Type },
}

pub enum Type {
    Named(QName, Vec<Type>),               // Nat, Pair<a, b>, Element
    Var(Name),                             // a lowercase name
    Sigiled(Sigil, Box<Type>),             // !T  $T  @T  ?T
    Fn(Vec<Type>, Box<Type>),              // fn A, B -> C
    Map(Box<Type>, Box<Type>),             // K ==> V
    Array(Box<Type>), Tuple(Vec<Type>), Unit,   // [A]   ((A, B))
}
```

Every variant has a span; the sketch omits the field for the ones where it is obvious. `SName` keeps the sigils the programmer wrote, and the elaborator checks them against the inferred kind rather than trusting them, which is what makes "`count` is bound to a dynamic, but its name has no sigil" a diagnostic rather than a silent reinterpretation.

## Grammar sketch

Written as the parser reads it, top down. `block` is the routine from the lexer section; `expr(p)` is the precedence climber at level `p`.

```
  module      := item*
  item        := 'let' sname tparams? ':' type '=' expr
               | sname ':' block                                  # a widget definition: @counter: ... (the @
               is the name's sigil)
               | 'type' Upper tparams? '=' typedef
               | 'tag' qname 'of' Upper block                     # fields and parts, one per statement
               | 'test' lower ',' expr block                       # test counter, @counter: one turn per
               statement

  typedef     := 'record' block | 'choice' ('..' | alt ('|' alt)* | block)
               | Upper type                                        # newtype: Meters Float
               | lower                                             # unit type: click
  tag_stmt    := lower ':' type                                   # a field
               | 'requests' ':' type | 'responds' ':' type | 'state' ':' type

  block_stmt  := pat '<-' expr 'then' expr                        # hold: pat must be one sigiled name
               | pat '<-' expr                                     # bind
               | 'let' pat '=' expr
               | '@' expr                                          # result, only as the last statement
               | expr                                              # build and discard, or the block's result

  expr(hold)  := expr(chain) ('then' expr(chain))?                # does not associate
  expr(chain) := expr(merge) (ChainDot Upper call_args?)*          # a chain step on a continuation line
  expr(merge) := expr(on) ('|' expr(on))*
```

```
expr(on)    := expr(or) ('on' expr(or))*
expr(or)    := expr(and) ('or' expr(and))*
expr(and)   := expr(not) ('and' expr(not))*
expr(not)   := 'not' expr(not) | expr(cmp)                        # looser than comparison: `not a == b` is
`not (a == b)`
expr(cmp)   := expr(add) (cmpop expr(add))?                       # cmpop: < <= > >= == !=
expr(add)   := expr(mul) (addop expr(mul))*
expr(mul)   := expr(prefix) (mulop expr(prefix))*
expr(prefix):= prefixop expr(prefix) | freesigil expr(postfix) | expr(postfix)
             | Dot(prefix) sname | Dot(prefix) Upper call_args? | '[' expr ']'   # the pronoun forms: _.x,
             _.F a, _[i]; the resolver finds the binder
expr(postfix) := atom ('.' sname | '[' expr ']' | '.' Upper call_args?)*   # field, index, method (all
binary, no space)
atom        := callable call_args?                                # a paren-less call, see below
             | literal | '(' expr(hold) ')' | '((' expr (',' expr)+ '))'
             | '[[' (expr (',' expr)*)? ']]' | '{' map '}'
             | lambda | 'match' expr block | 'if' expr block 'else' block
             | 'for' binders? expr block | 'rebuild' expr block | 'intercept' expr ',' expr ',' expr
             | '@' 'widget' block | ctor_named | ctor_block
callable    := sname | qname
call_args   := expr(merge) (',' expr(merge))*                     # the ',' repeats only when in_args is
clear

lambda      := '\' body | '\' '|' pat (',' pat)* '|' body
             where body := expr(hold) running to the end of the enclosing expression
binders     := (pat ',')? pat '<-'                                # for k, $item <- $items:  or  for $item
<- $items:
ctor_named  := qname '(' (lower '=' expr) (',' lower '=' expr)* ')'
ctor_block  := qname block                                        # Vector: x: 0.0 ...

type        := 'fn' ptype (',' ptype)* '->' ptype                 # fn A, B -> C
             | ptype
ptype       := sigil ptype | qname ('<' type (',' type)* '>')? | lower | 'unit'
             | ptype '==>' ptype | '[' type ']' | '((' type (',' type)+ '))'
             | '(' type ')'                                       # grouping: fn (fn A -> B), [A] -> [B]

arm         := pat ('if' expr)? block                             # a guard: pat if cond: body
pat         := 'ignore' | sname | literal | 'unit' | '((' pat (',' pat)+ '))' | '(' pat ')'
             | '[[' (pat (',' pat)* (',' '..' pat)? | '..' pat)? ']]'     # [[]]  [[a, b]]  [[head, ..rest]]
             | 'Some' pat | 'Nil' | qname pat? | qname '(' fieldpats ')' | qname '.' '*' '(' '..' ')'
             | pat ':' type
fieldpats   := (lower '=' pat | pat | lower) (',' ...)* (',' '..')?
```

The places where a transcription of this sketch goes wrong, each with its answer:

**A call's arguments are whole expressions.** A call is recognised at the `atom` level, when a callable name is followed by a token that begins an expression, and each of its arguments is parsed at the `merge` level, which is the level just below the application's place in the precedence table. That is what makes a call extend to the end of the expression it sits in: `F a + 1` is `F(a + 1)` and `F a | G b` is `F(a | G(b))`, because the first argument absorbs everything up to the next comma or the end. `1 + F a` parses `1`, sees `+`, parses the right operand at the `mul` level, reaches the atom `F` followed by `a`, and makes the call there, so the result is `1 + F(a)`; nothing precedes a call's name that the call could absorb. The practical rule for the implementer: after parsing a callable name, look at the next token; if it begins an expression and is not a binary operator, a closing bracket, a comma, a `Sep`, a `Semi`, a `ChainDot` or a block token, the name is a function and what follows is its first argument.

**Commas belong to the outermost call.** `expr(app)` is entered once per enclosing expression, and inner names followed by arguments parse as one-argument calls because the inner call's `args` loop is

suppressed: the parser passes a flag `in_args` down, and a call parsed with `in_args` set takes exactly one argument. `F a, G b, c` is `F(a, G(b), c)`.

**A group with a call in it is that call, with all its commas.** Single parentheses are grouping, and the parser clears `in_args` inside them, so the outermost call in the group takes every comma in the group: `F a, (G b, c)` gives `G` two arguments, and `(Insert $items, $next, todo) on !add` is one three-argument call sampled at `!add`. A comma inside single parentheses that no call takes, as in `(a, b)`, is the error "a comma in parentheses belongs to a call; a pair is `((a, b))`". A callable followed by `(` with no whitespace between is the same call in bracket form, `G(b, c)`, and `G(x=1, y=2)` with `lower =` items is `Named`; both forms exist because element construction reads best with named fields, and a positional call written the way most languages write one should mean that call.

**Tuples and arrays take their commas.** Inside `TupleOpen ... TupleClose`, `ArrayOpen ... ArrayClose` and `{ }`, the parser sets `in_args`, so commas belong to the bracket and a call inside takes one argument: `((G b, c))` is the pair of `G(b)` and `c`, `[[F a, G b]]` has two elements, and a multi-argument call inside one takes its own single parentheses, `(((G b, c), d))`. Single `[` after an expression with no space is always indexing; a prefix `[` is the pronoun's index (`_[i]`), so `F [1, 2]` is the error "single brackets index; an array is `[[1, 2]]`" (a comma inside a prefix `[`), and `[]` is "an empty array is `[[]]` or `empty`".

**Function types are comma lists.** In a type, `fn` takes comma-separated parameter types up to `->` and one result type after it, `fn A, B -> C`, the same way a call takes its arguments. A parameter or result that is itself a function type is parenthesised, `fn (fn A -> B), [A] -> [B]` and `fn A -> (fn B -> C)`, and an unparenthesised `fn` in either place is the error "a function type as a parameter or result is parenthesised: `(fn A -> B)`". `fn ((A, B)) -> C` takes one pair, and `fn (A, B) -> C` is the error "a comma in parentheses belongs to a call; a pair type is `((A, B))`", so the two readings never share a spelling.

**Method chains end at a continuation line.** `expr(chain)` sits between hold and merge. A `ChainDot` is not an operator any lower level knows, so it ends every paren-less call open on the lines above it, and `expr(chain)` takes the whole expression so far as the receiver of the step:

```
items.Filter IsDone
    .MapValues Label          # MapValues (Filter items, IsDone), Label
    .Sort                   # Sort (MapValues ...)
```

A step's own arguments are parsed as a call's and end at the next `ChainDot`. Inside brackets the rule is the same, because the lexer marks the line start whatever the bracket depth. A one-line spaced chain, `x.F a .G b`, is not a chain: the `.G` is a prefix `Dot`, which is the pronoun where `_` is in scope and an error where it is not ("put each chain step on its own line").

**Trailing lambdas.** A `\` in argument position begins a lambda whose body runs to the end of the enclosing expression, including any commas: `Fold changes, items, \|acc, id, c| Step acc, id, c` gives `Step` three arguments inside the lambda and `Fold` three arguments outside it. The body is parsed with `in_args` cleared and at the `hold` level, and if it ends in a line-final colon the layout block belongs to the body (`\@widget:` and `\match _:` are this). A lambda that is not the last argument is parenthesised, and the parser's error for `F \x, y` where `y` was meant for `F` says so: "a trailing lambda takes the rest of the line; parenthesise it to pass more arguments".

**A free sigil applies to the whole call after it.** `expr(prefix)` on a `FreeSigil` parses its operand at the `postfix` level, where a callable name takes its arguments, so `@ ChecklistUpdate ix, !checked` is `@ (ChecklistUpdate ix, !checked)` and `$ x + 1` is `$ (x + 1)`. An attached sigil is part of the name token and never reaches this rule.

**Boolean operators.** `not` is a prefix keyword that takes an `expr(not)`, so it sits between `and` and comparison: its operand is a whole comparison, and `not a == b` is `not (a == b)`. `and` and `or` are left-associative and sit between `not` and `on`, with `and` tighter, so `x or y and z` is `x or (y and z)`, `a < b and c < d` needs no parentheses, and `$done or $n > limit on !click` is `($done or ($n > limit)) on !click`. `not $done and $n > limit` is `(not $done) and ($n > limit)`, since `not` binds tighter than `and`. The parser builds `Binary` and `Prefix` nodes and stops there; a non-`Bool` operand is the checker's error.

**Hold does not associate.** `expr(hold)` takes at most one `then`; a second is the error "`then` does not associate; a hold has one initial value and one event".

**Method form.** `x.F y, z` is parsed at the postfix level: `x`, then `.` with the binary flag, then an uppercase name, then arguments as for a call, and the tree is `Call { f: F, args: [x, y, z] }`. Method form associates left: `x.F.G` is `G (F x)`.

**The pronoun shorthand applies wherever _ is in scope.** A prefix `.` before a lowercase name is a field of the pronoun, before an uppercase name the method applied to the pronoun, and a prefix `[i]` indexes the pronoun: `\.done`, `\.Len`, `\[0]`, `\not .done`, and the same inside a `for` body or a `rebuild` body or a `match` arm that binds `_`. All are `Field`, `Call` and `Index` nodes over `Var(_)`. The parser does not track which constructs bind `_`: it builds these nodes wherever the lexer set the `prefix` flag, and the resolver, which owns scopes, finds the binder. Where no `_` is in scope the resolver reports "no pronoun here; `.x` means `_.x`, so write `_.x`, or put a chain step on its own line", so a stray space never quietly turns a method call into a pronoun read. A `.Method` that starts a continuation line is a `ChainDot`, never a prefix `Dot`, so indentation separates the two readings.

**Record forms.** `Vector 1.0, 2.0` is an ordinary `Call` and the elaborator learns that `Vector` is a constructor. `Vector(x=1.0, y=2.0)` is `Named`, recognised because a qualified or uppercase name is followed immediately (no whitespace) by `(` and the first thing inside is `lower =`. `Vector:` followed by a block of `field: value` statements is `RecordBlock`. Element construction is the same three forms with a qualified name: `Ui.Button(label="OK")`, `Ui.Button "OK"`, `Ui.Button:` block. A tag with no fields is a bare qualified name, `Clock`, and parses as `QVar`.

**`if` and `else`.** `if c` takes a block, then expects `Else`, then a block. After an inline `if c: a`, the `else` ends the inline block and is the next token; after a layout block, the lexer has emitted `Sep` before `else` at the block's column, and the `if` routine consumes the separator when the token after it is `Else`. An `else` belongs to the nearest `if` that has none yet; a layout `else` finds its `if` by column, because the dedent has already closed every block between them. An `if` with no `else` is the error the language chapter gives.

**`match` guards.** An arm is `pat`, then optionally `if` and an expression, then the block. The guard is parsed at the `hold` level up to the arm's colon, which the lexer has left as `Colon` (inline arm) or turned into `BlockOpen` (layout arm), so `Some n if n > limit: ...` needs no lookahead. The guard is stored on the `Arm`, not wrapped around the body, so that exhaustiveness can see it.

**`for` binders.** After `for`, the parser scans ahead on the line for a `<-` at bracket depth zero before the line-final or inline colon. If there is one, the tokens before it are one pattern or two comma-separated patterns, the key and the item (`for k, $item <- $items:`; with one pattern, `for $item <- $items:`, it is the item). If not, the line is the shorthand `for $items:` and `key` and `_` are bound by the loop. The scan is a token walk with no grammar, and it is sound because a `<-` can appear in an expression only inside a block, which the scan does not enter.

**Array patterns.** `[[` in a pattern is `ArrayOpen`: `[[]]` is empty, `[[a, b]]` is exactly two elements, and `[[head, ..rest]]` is a first element and the remainder. `..` takes a pattern after it, a name or `ignore`,

and may only be last; the parser reports a second `..` or one in the middle ("`..` takes the rest of the array, so it comes last").

**Widget definitions.** `@counter:` at top level is `Item::Widget`; `@Button: fn $Text -> @!Click = \@widget:` is `Item::Let` with a lambda value whose body is a `WidgetBlock`. Both forms exist because the first reads as a declaration and the second as a function, and the parse trees differ only in whether there is a lambda around the block.

**Tag declarations.** `tag Ui.Button of Element:` opens a block whose statements are fields (`label: Text`) and parts (`responds: Click`). The three part names are not keywords; the parser recognises them as the first token of a `tag` statement, and a tag with a field named `state` is rejected with "`state` names a part of the tag; rename the field". Order is free; each part at most once; the `of` type names an extensible variant, which the resolver checks.

`intercept.` Three arguments, comma-separated, parsed as an atom so that the whole call is one expression: `((result, $slots)) <- intercept @child, !down, $ \|e| @Element e`. The third argument begins with a free `$`, which takes the lambda, which takes the rest of the line.

**Tests.** `test counter, @counter:` opens a block with one line per turn and indented `=>` checks under it. The parser keeps it as `Item::Test { body: Expr }` and the elaborator types the body against the conformance suite's `Test` function and step constructors from the prelude; a test is therefore ordinary Prism with one keyword to mark it, and a module's tests are collected by the harness without a second syntax.

**Patterns.** `Type(f, g)` as shorthand for `Type(f=f, g=g)` is recognised when every item inside the parentheses is a bare lowercase name; `Type(p, q)` with any non-name pattern is positional. A mix is an error: "name every field or none". `Ns.*(..)` is `Upper Dot(binary) Star` followed by `( .. )`, and the `..` is mandatory because the arm cannot know the fields. `p: Type` ascribes.

**Layout blocks of `lets`.** A lambda body, a `for` body or a `match` arm that opens a layout block containing `let` statements and ends in an expression is `LetIn`; the parser reads statements until the last, which must be an `Expr`, and a block ending in a `let` is the error "a block of lets must end in a value". Inside a widget block the same statements are the block's.

## Spans

Every node's span runs from its first token's start to its last token's end, and a compound node's span covers its children. The spans the diagnostics need most are the ones a careless parser loses: the span of an operator token inside a `Binary` (for "both are events" pointing at the two operands and the operator), the span of each argument in a `Call` (for "applied to -1"), and the span of each arm's pattern (for exhaustiveness). The parser therefore keeps operator and keyword spans as fields, not only node spans; `Binary` has `op_span`, `Call` has `arg_spans`, `Arm` has `pat.span`. Retrofitting these is the expensive part of a compiler, so they are in the first commit.

## What the parser does not decide

Whether `F` is a function or a constructor, whether `Button` is a widget function, whether `$count` is really a dynamic, whether `0` is a `Nat`, whether `|` is merging events or is a stray token in a type: none of these. The parser produces a tree that says what was written, with sigils as written and types as written, and the resolver and elaborator say what it means. A parser that consulted a symbol table to decide a parse would be one whose output changed when an import changed, and the goldens for the parser, which are parse trees printed, would then depend on the prelude; keeping the parser

context-free is what lets its goldens be small files with no imports at all.

# Resolver

The resolver answers "which thing does this name mean" and "in what order can this block be built", and the second question is the one that distinguishes a Prism resolver from any other. A widget block's statements are mutually recursive and may be written in any order; the program is a set of equations. Someone has to find an order in which the equations can be turned into constructor calls, and that someone is the resolver, with Tarjan's algorithm over the block's dependency graph. Everything else here is the ordinary business of scopes.

## Scopes

```
pub struct Resolved { pub module: Module, pub refs: HashMap<NodeId, Binding>, pub groups: HashMap<BlockId,
Vec<Group>> }

pub enum Binding {
    Local  { block: BlockId, index: usize, kind: Sigils },   // a statement of a widget block
    Param  { lambda: NodeId, index: usize },                 // a lambda argument
    Pronoun{ binder: NodeId },                               // _ in a lambda, rebuild, for (shorthand) or match
    Binder { for_node: NodeId, which: ForVar },              // a for's key or item: named by the loop, or `key`
/ `_` in the shorthand
    Loop   { for_node: NodeId, which: LoopVar },             // ix jx kx: the indices of an update block's loops
    LetSeq { stmt: NodeId },                                 // a let in a sequential block
    Top    { item: ItemId },                                 // a module-level definition
    Prelude{ name: &'static str, intrinsic: Option<Intrinsic> },
    Tag    { id: TagId },
    Ctor   { ty: TypeId, alt: Option<usize> },               // Vector, Circle, Some
}
```

Scopes nest the obvious way: module, then each item, then blocks and lambdas inside it. Three rules are Prism's own.

*A widget block is one scope in which every statement's names are visible everywhere in the block,* including in statements above their definition and in their own definition. A name bound twice in one block is an error, "`$count` is bound twice in this block", because the block is a set of equations and a set has one equation per name. A `let` outside a widget block, in a lambda body or a `for` body, is sequential: it sees names above it and not below, and a reference to a later `let` is "`x` is used before its definition; plain bindings are sequential".

*The pronoun.* `_` is bound by the nearest enclosing lambda (`\body`), `rebuild` block, `for` block or `match` arm pattern that binds it, and its sigiled forms `$_`, `!_`, `@_` name the same binder read at that kind. Inside `for $m:`, `_` is the entry's value as a plain name and `$_` is the entry as a dynamic; which of them the body uses is recorded on the binder, because the lambda rule of the language chapter says a body that mentions `$_` takes a dynamic and the elaborator reads that from the resolver's note. Only one pronoun is in scope at a time; an inner binder shadows an outer one and a program that needs both names one of them. `.x`, `.F` and `[i]` in prefix position are references to the pronoun and resolve the same way, in any of these binders; where none is in scope, they are the error the parser section gives.

*Loop binders.* `for k, $item <- $items:` names the key and the per-key dynamic, and the resolver binds each as `Binder { for_node, which }` with `ForVar` `Key` or `Item`; with one pattern before `<-` only the item is named. The shorthand `for $items:` binds `key`, a keyword, to the key, and `_` and `$_` (the pronoun) to the item. An inner shorthand `for` shadows an outer one's `key` and `_`, as inner pronoun

binders always do; a body that needs both names the binders of one of them. Over an array the key binder is the index. A widget-level `for` has no index variables to count: its variables are the names the program gave it, so an outer binding cannot be captured by accident.

*Loop variables.* `ix`, `jx` and `kx` are keywords, and the resolver binds them by nesting depth in the loops of an `update` block: `ix` is the index of the innermost `for`, `jx` the next out, `kx` the next; the program does not name them. Using `jx` in a single `for` is the error the language chapter gives. They are keywords so that an outer binding cannot capture them, and the resolver's only work is counting.

Top-level names are visible throughout the module; the prelude is an outer scope the module shadows. Shadowing a prelude name is allowed and not warned about, except for the intrinsics below, where shadowing is an error, because a program that rebinds `@Element` has made a program nobody can read.

## Qualified names and namespaces

`Ui.Button` is one token and resolves in the tag table first: every `tag` declaration in the module and the prelude is interned into `tags: Vec<TagInfo>` with its namespace path, and a qualified name that matches a tag resolves to `Binding::Tag`. A qualified name that matches no tag resolves against the prelude's namespaced functions (`Wrap.Add`, `Checked.Mul`), and otherwise is "unknown tag or function `Fs.Content`; did you mean `Fs.Contents`?" with the nearest name by edit distance. In a pattern, `Gui.*` is the three tokens the lexer produces, and the resolver turns them into `Pat::Wildcard { ns: Gui }`, checking that at least one tag is declared under the namespace so that a typo in the namespace does not match nothing silently. A tag's `of` type must resolve to a type declared `choice ..`; declaring a tag of a closed choice is "`Shape` is a closed choice; declare it `choice ..` to add tags".

## Intrinsics

Some prelude names are not functions the compiler calls but nodes the elaborator builds, and the resolver marks them so that the elaborator's walk can dispatch on the mark instead of on a string:

| Name | Node |
|---|---|
| `!Switch` | `switch` over an event of events |
| `$Join` | `join` over a dynamic of dynamics |
| `!Leftmost` | `leftmost` over an array of events |
| `!never` | `never` |
| `@started` | `started` (a `rebuild` over a constant, built as one node) |
| `@HoldFirst` | `hold_first: fn !a -> @$a`, a dynamic seeded by the event's firing in the occurrence it is built |
| `@Element` | `element`, with the tag's kind |
| `!Updated` | `updated`: a dynamic's change event |
| `$Distinct` | `distinct`: a derived dynamic that drops equal-valued changes, equality bound |
| `Kind` | `kind`: a tag-table lookup |
| `intercept` | `intercept` (a keyword, listed here because it is a node) |
| `Some`, `Nil`, `True`, `False`, `click` and every declared constructor | constructor application |

Everything else in the prelude is an ordinary function with an ordinary annotation, compiled from Prism source or bound to a Rust implementation by name (`Len`, `Insert`, `Sort`, `ToText`, `Wrap.Add`), and the resolver treats it as a `Top` binding of a module it did not write.

## Dependency analysis into groups

For each widget block, the resolver builds a graph whose vertices are the block's statements and whose edges are references: statement `s` depends on statement `t` when `s`'s right-hand side mentions a name `t` binds, at any depth, including inside lambdas, nested blocks and the `rebuild` and `for` bodies that `s` contains. References inside a nested *widget* block (a `rebuild` body, a `for` body, a `\@widget:` lambda) count as references of the statement that contains them, because the nested block is built by that statement and captures from its environment. The algorithm is Tarjan's strongly connected components, emitted in reverse topological order so that every group comes after every group it depends on:

```
pub struct Group { pub members: Vec<usize>, pub cyclic: bool, pub edges: Vec<Edge> }
pub struct Edge  { pub from: usize, pub to: usize, pub at: Span, pub label: EdgeLabel }
pub enum EdgeLabel { Prompt, Delayed(Delay) }
pub enum Delay    { Tag, Switch, Intercept }
```

A group with one member and no self-edge is acyclic and is built in one step. A group that is cyclic (two or more members, or one member that mentions itself, as the counter's `$count` does) is where placeholders go, and the edge labels decide where.

The resolver labels edges from syntax alone, which it can do because lifting is read off sigils and sigils are syntax:

- A reference to a dynamic `$x` as an operand of an operation or application that also has exactly one event operand is a *tag* read: `Delayed(Tag)`. `$count + !delta`, `$v on !e`, `(Insert $items, $next, todo) on !add` all read their dynamics this way.
- A reference to a dynamic anywhere in the event expression of a hold `$y <- v then e` is a tag read for the same reason: the event expression is an event, and dynamics inside it are read through tags.
- The argument of `!Switch` is `Delayed(Switch)`: the switch adopts a new inner at commit.
- A reference to the child's element list, the second result of `intercept`, from the responses argument of the same `intercept` is `Delayed(Intercept)`: routing consults the list as the occurrence opened.
- Every other reference is `Prompt`: a dynamic in an operation with no event (a derived dynamic reads its change event), the dynamic of a `rebuild` or a `for` (rebuilt in the occurrence of the change), the argument of `$Join`, the event side of anything, a plain value.

One edge the resolver cannot label is the one from a `for`'s collection to its consolidated event, because its label depends on whether the block fires at birth, which is a property of the block's typed form. The resolver labels it `Prompt` provisionally and the elaborator's causality check, below, relabels it after the block is elaborated. The elaborator also checks every other label against the lift it actually inserted, and a disagreement is an internal error, since both were read from the same sigils.

Why the resolver and not the elaborator does the grouping: the elaborator types a cyclic group monomorphically as one unit and needs to know the groups before it starts, and the groups do not depend on types. Why the labels are computed here even though the check runs later: the labels are also what places the placeholders, and placing them is a property of the resolved tree that the text form prints, so that a golden shows where a knot is cut before any type is mentioned.

### Placing the placeholders

A cyclic group's build order is found by cutting the cycle at its delayed edges. Every legal cycle has at least one delayed edge (the causality check rejects the others), so removing all delayed edges from a group's subgraph leaves it acyclic, and the remaining prompt edges give a topological order. The targets of the removed delayed edges are the names that need a placeholder: each is referenced before its definition is built, and each is referenced only in a position that *subscribes* (a tag subscribes to a behavior and samples it at occurrence time; a switch subscribes to an event of events; routing subscribes to a list) and never in one that *samples at build*. The placeholder is opened before the group, the group is built in the order found, and the placeholder is closed to the real node after its definition's step.

The counter's group is `{ $count }` with one self-edge labelled `Delayed(Tag)`. Cut it: `$count` gets a placeholder; the hold is built with its tag reading the placeholder; the placeholder is closed to the hold. A group `{ !a, !b }` where `!a = F !b` and `!b = !Switch !!c` with `!!c <- rebuild $d: ... !a ...`: the delayed edge is the switch's, so `!b` gets the placeholder, `!a` is built reading it, the `rebuild` is built reading `!a`, the switch is built over the `rebuild`'s event, and the placeholder is closed to the switch. In each case the thing that reads the placeholder at build is a subscription, which is what the runtime's cyclic slots support, and the thing that would need the real value at build (a `rebuild` over the name) is on a prompt edge and therefore comes after the close. That is the property the lint checks under the name "nothing samples a placeholder before its close", and it holds by construction of the cut.

Suppose instead the compiler opened a placeholder for every member of every cyclic group, as the simplest implementation would. Then `rebuild $count` inside the group would read the placeholder at build, the runtime would report "cyclic dynamic never closed", and the program, which is legal, would fail at run time with an error that names the runtime's mechanism rather than the program's cycle. Cutting at the delayed edges is what makes every legal program build.

## Elaboration

You have read a line of a reactive program and not known whether the dynamic in it was sampled or subscribed to, and you found out by running it. Elaboration is the stage that makes sure nobody ever has to find out that way again: it takes the resolved tree and produces a tree in which every lift is a named node, every number has a type, every binding says whether it runs once or at every occurrence, and every cycle has been checked for a delayed edge. Nothing below elaboration decides anything about meaning. The output is the typed AST, and the typed AST is the reference for what a Prism program means; the closure compiler walks it and the text form prints it, and if the two frontends print the same text for a program, they agree about that program entirely.

### The typed AST

```
pub struct TModule { pub items: Vec<TItem>, pub tags: Tags, pub types: Types }

pub enum TItem {
    Let    { name: SName, scheme: Scheme, value: TExpr, span: Span },
    Widget { name: SName, ty: Type, body: TBlock, span: Span },
    Test   { name: Name, body: TExpr, span: Span },
}

pub struct TBlock { pub bindings: Vec<TBinding>, pub result: TExpr, pub groups: Vec<Group>, pub span: Span }
```

```
pub struct TBinding {
    pub name: SName,            // or a generated name for a discarded result
    pub ty: Type,               // with its sigils
    pub mark: Mark,             // Build | Occ
    pub group: GroupId,
    pub def: TDef,
    pub span: Span,
}
pub enum Mark { Build, Occ }

pub enum TDef {
    Bind        { widget: TExpr, pat: TPat },                  // builder.bind
    Hold        { init: TExpr, event: TExpr },                 // Dynamic::hold
    Let         { value: TExpr },                              // a value, plain or lifted
    Placeholder { kind: PlaceholderKind },                     // opened before the group
    Close       { placeholder: BindingId, real: BindingId },   // closed after it
}

pub enum TExpr {
    // the plain layer
    Var(BindingRef, Type), Lit(Lit, Type), Unit,
    App   { f: Box<TExpr>, args: Vec<TExpr>, ty: Type },
    Lambda{ params: Vec<TPat>, body: Box<TExpr>, ty: Type, captures: Option<Vec<BindingRef>> },  // captures
after conversion
    Ctor  { ty: TypeId, alt: Option<usize>, fields: Vec<TExpr> },
    Elem  { tag: TagId, fields: Vec<TExpr>, request: Option<Box<TExpr>> },
    Field { e: Box<TExpr>, index: usize, ty: Type },
    Index { e: Box<TExpr>, i: Box<TExpr>, ty: Type },          // returns ?T
    Match { scrutinee: Box<TExpr>, arms: Vec<TArm>, ty: Type, dispatch: Dispatch },
    If    { c: Box<TExpr>, t: Box<TExpr>, e: Box<TExpr>, ty: Type },
    Let   { pat: TPat, value: Box<TExpr>, body: Box<TExpr> },  // a sequential let in a lambda, for or arm
body
    Array(Vec<TExpr>, Type), MapLit(Vec<(TExpr, TExpr)>, Type), Tuple(Vec<TExpr>),
    TextCat(Vec<TExpr>),                                       // interpolation, all parts Text
    Trait { which: TraitOp, at: Type, args: Vec<TExpr> },      // Num.Add[Int] a, b; Eq[Text] a, b
    Bool  { op: BoolOp, args: Vec<TExpr> },                    // Bool.Not a; Bool.And a, b; Bool.Or a, b
(operands are Bool; And and Or short-circuit)
    Dict  { which: Trait, at: Type },                          // the hidden argument for a bound
    Maybe { over: Vec<TExpr>, body: Box<TExpr> },              // ? lifting at the plain layer
    // the lifts: every one explicit
    Const   { value: Box<TExpr> },                             // $ v : a constant dynamic
    DMap    { over: Vec<TExpr>, body: Box<TExpr> },            // derived dynamic, prompt
    Map     { event: Box<TExpr>, body: Box<TExpr> },           // event map
    Tag     { event: Box<TExpr>, dyns: Vec<TExpr>, body: Box<TExpr> },  // event with dynamics read before
    Leftmost(Vec<TExpr>),
    Never(Type),
    Switch  { events: Box<TExpr> },
    Join    { dyns: Box<TExpr> },
    Updated { dynamic: Box<TExpr> },
    Distinct{ dynamic: Box<TExpr>, eq: Box<TExpr> },
    HoldFirst { event: Box<TExpr> },                           // a hold seeded by the event's build-occurrence
firing
    // construction
    Dyn     { dynamic: Box<TExpr>, block: Box<TLambda> },      // block: \|v| @widget
    For     { coll: Box<TExpr>, block: Box<TLambda>, kind: Consolidation, keyed: bool,
              birth_firing: bool },                            // can the block fire in the occurrence a key
appears?
    Element { elem: Box<TExpr>, kind: ElemKind },              // kind: the tag's parts, or Generic
    Intercept { child: Box<TExpr>, responses: Box<TExpr>, supplier: Box<TExpr> },
```

```
    Started,
    WidgetBlock(Box<TBlock>),
    Kind    { elem: Box<TExpr> },
}
pub enum Consolidation { Plain, Event, Dynamic, Widget }
pub enum ElemKind { Known(TagId), Generic }
pub struct TArm { pub pat: TPat, pub guard: Option<TExpr>, pub body: TExpr, pub span: Span }   // the guard is
a plain Bool over the pattern's bindings
pub enum Dispatch { Choice(TypeId), Tag, Optional, Array, Literal }
```

`Type` is the plain type with its sigils as constructors: `Type::Dyn(Box<Type>)`, `Type::Event(..)`, `Type::Widget(..)`, `Type::Opt(..)` over `Nat`, `Int`, `Float`, `Text`, `Char`, `Unit`, `Fn`, `Map`, `Array`, `Tuple`, `Named`, `Var`. A `Scheme` is a `Type` with quantified variables and their bounds. The lifts are nodes, not annotations: a `Tag` node says "this is an event that fires with `event`, with the dynamics in `dyns` read through tags, and the body applied to the firing and the readings". The closure compiler maps each one to one runtime constructor, and the text form prints each one by name, which is what "every lift explicit" means in practice.

A sequential `let` has a node of its own, `Let { pat, value, body }`, and it appears only inside a lambda body, a `for` body or a `match` arm, never as a binding of a widget block, where a `let` is a `TBinding` with `TDef::Let`. A body `let x = F a` then `let y = G x` then `H x, y` elaborates to `Let { x, F a, Let { y, G x, H x, y } }`, nested right, and prints as `let x = F(a); let y = G(x); H(x, y)`. The leaf evaluator compiles it to a closure that evaluates the value, binds it in the frame's locals, and evaluates the body; there is no inlining, because a `let` of an expensive call used twice is the programmer saying "once".

## Hindley–Milner on the value layer

The checker is Hindley–Milner with levels, over plain types only. The algorithm is the standard one (unification variables with a level, generalisation at the level where a binding is introduced, instantiation with fresh variables at each use) and the choices that the text form exposes are these:

*Where generalisation happens.* At a `let` whose right-hand side is a lambda, and nowhere else. `let Double = \|x\| x + x` is generalised and may be used at `Nat` and at `Float` in one block; `let n = Len xs` is not generalised, which costs nothing since it has no free variables to generalise. A `let` of a non-lambda with a free type variable, which is rare and always a mistake in a strict language, is monomorphic and the variable is fixed by the first use. This is the value restriction in its syntactic form, chosen because the typed AST then has exactly one place where a `Scheme` appears below top level, and a reader of the text form knows that a `\` on the right of a `let` is the only thing that introduces polymorphism.

*Cyclic groups are monomorphic.* A group is typed as one unit with one set of unification variables, and nothing in it is generalised until the whole group is typed, as in ML's `let rec`. Two widget-block statements that refer to each other are the only way to make such a group, and they are events, dynamics and widgets, which are monomorphic by nature; the rule matters only in that the elaborator must not generalise a group member before the group closes.

*Top-level annotations are signatures.* A top-level `let` has a full annotation, and the annotation is the scheme; the body is checked against it, not inferred and compared. A top-level widget definition `@counter:` has no annotation and no parameters, and its type is inferred as `@T` from its result, because its only use is to be bound. This keeps every error local to one item, since the checker never guesses a signature from uses in another item.

*How type variables are named.* Unification variables are numbered during inference and renamed when the typed AST is finalised: in order of first appearance, reading the printed form left to right and top to bottom, as `a`, `b`, `c`, … with the programmer's own names kept where the annotation gave them. Two frontends that infer the same types therefore print the same letters, which the differential test on the text form depends on. The renaming is a pass over the finished item, not a property of inference, so that inference can allocate variables in whatever order its algorithm likes.

*Sigils in annotations.* `$x: $Int <- ...` and `$x: Int <- ...` mean the same thing: an annotation on a sigiled name gives either the plain type under the sigil or the full type. The elaborator normalises to the full type and checks that the sigils agree with the name's; a disagreement is the "sigil does not match" diagnostic.

The elaborator's walk is bidirectional in exactly one place, the kind of a lambda's binder (below), and otherwise is plain inference: `infer(expr) -> (TExpr, Type)` with `check(expr, expected)` as `infer` plus unify, except for lambdas, where an expected type is pushed in.

## The sigil lattice and lifting

Every operation and application is lifted by a rule read off its operands' outermost sigils, and the elaborator applies that rule as a table lookup, producing one node per lift. The boolean operators are operations like `+`: `not $done` is a `DMap` over `[$done]` with the body `\|d| Bool.Not d`, and `$a and !b` is a `Tag`. A lifted `and` or `or` is a lift of the same function: the dynamics are all read and the body is the plain `Bool.And a, b`, which evaluates `b` only when `a` holds, so the lifted node fires and changes by the lifting rule and not by any extra laziness.

| Operands' outer kinds | Result | Node |
|---|---|---|
| all plain | plain | the operation itself |
| some `$`, no `!` | `$` | `DMap { over: the dynamics, body }` |
| some `$` (zero or more), exactly one `!` | `!` | `Tag { event, dyns, body }` or `Map { event, body }` when there are no dynamics |
| two or more `!` | error | "two events in one operation" |
| any `@` | not lifted | an error unless it is an application of a widget function, below |

The body is the plain operation with the lifted operands replaced by lambda parameters: `$count + !delta` becomes `Tag { event: !delta, dyns: [$count], body: \|d, c| Num.Add[Int] c, d }`. The elaborator builds the body as a plain `TExpr` over fresh parameters, types it at the plain layer (where `+` is resolved by the numeric trait), and wraps it. One lifted expression becomes one node: `$a * 2 + 1` is one `DMap` over `[$a]` with the body `\|a| Num.Add[Int] (Num.Mul[Int] a, 2), 1`, not a node per operator, because the elaborator lifts the maximal plain subtree, which is found by walking down until an operand is itself sigiled. `on` needs no rule: it is a binary operator returning its left operand, so `v on !e` is `Map { event: !e, body: \|_| v }` and `$v on !e` is `Tag { event: !e, dyns: [$v], body: \|_, v| v }`, by the table.

`match` lifts like application: `match $x:` with plain arms is a `DMap` over `[$x]` with the match as its body; `match !e:` is a `Map`; dynamics mentioned in the arms, in their guards included, join the `dyns` of a `Tag`. A `match` with a blueprint arm requires a plain scrutinee, and the error points at `rebuild`.

`if` lifts the same way, as a `DMap` or `Map` or `Tag` around a plain `If` body.

`?` lifts at the plain layer, inside the event functor, the dynamic functor and the widget monad, and the node is `Maybe { over, body }`: `a / b + 1` is `Maybe { over: [Num.Div a, b], body: \|q| Num.Add q, 1 }` of type `?Nat`; the `?` comes from `/` at `Nat` and `Int`, which fails on a zero divisor. At `Float` the same `/` is plain and follows IEEE (infinities and NaN, never failing), so `x / y` over two `Float`s has no `Maybe` and `Num.Div[Float] x, y` is an ordinary node. The elaborator finds a `?` operand where a plain one is wanted, wraps, and continues; the `?` propagates outward until a binding names it or a `match` eliminates it. `$a / $b` is `DMap { over: [$a, $b], body: \|a, b| Num.Div a, b }` of type `$?Nat`, since the time functor lifts first and the `?` is in the body's result. `?$x + 1` is a type error: an optional dynamic is matched, not lifted.

**Field access and indexing lift like application.** `$slot.element` is `DMap { over: [$slot], body: \|s| s.element }`; `!!panes.!inbox` is `Map { event: !!panes, body: \|p| p.!inbox }` of type `!!Mail` when `inbox` is `!Mail`; `$items[3]` is `DMap` with an `Index` body, of type `$?Item`. The parser has already written the pronoun shorthand `\.x` and `\[i]` as accesses on `_`, so they lift like any other.

**Interpolation lifts.** A text literal with parts is `TextCat` over its parts, each converted to `Text` by the canonical rendering of its type (`ToText[Int]` for an `Int`, the identity for `Text`, an error for anything else), and the whole is lifted by the table: `"Count: {$count}"` is `DMap { over: [$count], body: \|c| TextCat ["Count: ", ToText[Int] c] }` and `"Got {!line}"` is a `Map`: a literal that interpolates a dynamic is a dynamic, and one that interpolates an event is an event, with no sigil written on the literal. Two interpolations of different functor kinds follow the lifting rules (a dynamic and an event is an event, two events an error). `$"Increment"` is a free `$` applied to a literal with nothing to interpolate, `Const "Increment"`, a constant dynamic; the `$` is the prefix operator, not part of the token, and `$"Got {!line}"` is the sigil error because the literal is an event.

**Lifting over `@`.** A blueprint operand is built, not computed over, and the one case with a meaning is the application of a widget function to a dynamic where the function wants a plain value. Take a button declared over plain text, `@Button: fn Text -> @!Click`. `@Button $label` is then an application of a widget function to a dynamic argument, and its type by the table would be `$@!Click`, a dynamic of blueprints. Binding a dynamic of blueprints with `<-` lifts to `rebuild`, and the elaborator inserts the node:

```
!click <- @Button $label
```

elaborates as the binding `!click: !Click [occ] = Switch { events: Dyn { dynamic: $label, block: \|l| @widget: bind (@Button l) } }`, marked `[occ]` because its root is a graph node, the switch. The inserted block applies `@Button` to the plain value, binds it, and returns its result; the `rebuild`'s event is an event of the results of each build; and the bound name is what the result kind says:

| The widget's result | The bound name | Node |
|---|---|---|
| an event `!R` | `!x: !R` | `Switch (Dyn ...)` |
| a dynamic `$R` | `$x: $R` | `Join (HoldFirst (Dyn ...))` |
| a plain `R` | `$x: $R` | `HoldFirst (Dyn ...)` |
| `unit` | nothing bound | `Dyn ...` as a discarded bind |
| a bundle | each component by its own kind | one `Dyn` for the whole and one projection per component, each lifted by its row above |

A *bundle* is a widget result that is a tuple or record with sigiled components: `intercept`'s `@((a, $(Path`

`==> Slot)))`, `@Focusable`'s `@((a, !Input))`, a panel's `@((!Click, $Text))`. Functor values nest inside plain aggregates, so the result is one plain value with an event and a dynamic inside it, and a pattern bind takes it apart: `((result, $slots)) <- intercept ...` binds the bundle and then projects each component into its own name, which the text form prints as one line for the bundle, named by the pattern, and one line per component. Through the dynamic-of-blueprints lift the projections are what get lifted: `((!click, $text)) <- @Panel $cfg` with `@Panel: fn Text -> @((!Click, $Text))` becomes one `Dyn` over `$cfg` whose event fires with the bundle, then `!click` is `Switch (Map rebuild \|b| b.0)` and `$text` is `Join (HoldFirst (Map rebuild \|b| b.1))`, each component by its own row of the table. The `intercept` dump in the text form section shows the unlifted case.

`HoldFirst` is a hold whose initial value is the event's firing in the occurrence the hold is built, which exists because a `rebuild` fires in its build occurrence and a widget built in n sees all of n; the runtime constructs it as a hold that takes its initial value from the catch-up read of the event it holds. A program reaches the same node by name: the prelude's `@HoldFirst: fn !a -> @$a` is an intrinsic the resolver maps to the `HoldFirst` node, so `$latest <- @HoldFirst !reading` elaborates to the binding `$latest: $T [occ] = hold_first !reading`, the same line the inserted lift prints inside `join (hold_first ...)`. The second row is the canonical cache idiom with the first build as the initial value and no one-occurrence hiccup. The elaborator inserts the `rebuild` when the application's argument is a dynamic and the function's parameter is plain; an argument that is a dynamic where the parameter is `$T` is an ordinary application with no lift, which is how the book's `@Button: fn $Text -> @!Click`, written with a `rebuild` inside, takes `$"Increment"` with no lift at the call site. The rule is the same for either signature: lift only where the parameter wants a plain value and gets a dynamic.

An element whose field is a dynamic is the same rule one level down: `@Element Ui.Button(label=$l)` is `@Element` applied to `Elem { tag: Ui.Button, fields: [$l] }`, the element construction is a plain record build applied to a dynamic argument, so it lifts to a `DMap` giving a `$Element`, and `@Element` applied to a `$Element` is a widget function applied to a dynamic where it wanted a plain value, so it lifts to `rebuild`, which is what "a dynamic where a field goes lifts into a `rebuild`, fresh slot per change" means.

**The kind of a lambda's binder.** A lambda whose body mentions a sigiled pronoun, `$_`, `!_` or `@_`, takes a binder of that kind and is not lifted over it; the resolver has recorded which forms the body uses, and two different sigiled forms of `_` in one body is the error "`_` is read as a dynamic and as an event in one lambda". A named argument has the sigil it is written with. A lambda with no sigiled pronoun and no annotation takes a plain binder and is a plain function, lifted afresh at each application, *unless* it is checked against an expected type whose parameter has a sigil: a lambda passed to a function whose annotation says `fn $Text -> @unit`, or bound by a `let` with an annotation, takes the sigil the annotation gives. The two blocks the language builds from lambdas need no push: a `rebuild`'s block takes the plain value of its dynamic, and `intercept`'s supplier is `$(fn Element -> @$State)`, a plain function under a constant `$`. This is the one place `check` pushes a type into a lambda, and it is bidirectional checking in the smallest form that works.

**Lifting never crosses a sigil boundary.** `$$a + 1` is the error "cannot lift into the inner layer"; the table looks at the outermost sigil of each operand only, and the body it builds is typed at the layer beneath, where `$a` is not a number. `$Join` and `!Switch` are the flatteners, and they are nodes of their own.

## The three traits

`Eq`, `Ord` and `Num` are resolved by inference and nothing else is overloaded. The elaborator keeps a

constraint set alongside unification: `Num t` for every arithmetic operator and every numeric literal, `Eq t` for every use of `==`, `!=` and `$Distinct`, `Ord t` for every comparison operator (`<`, `<=`, `>`, `>=`), every `Sort`, and every map key. Constraints are solved when their type variable is resolved:

- At a concrete type, the constraint is checked against the fixed table (`Num`: `Nat`, `Int`, `Float`; `Eq`: every plain type with no function and no functor, structurally; `Ord`: primitives (`Float` by IEEE totalOrder, so it can key a map), text, chars, tuples and arrays of ordered types, closed choices by declaration order, declared types whose parts are ordered, and extensible-variant values by tag name then fields) and the operation node records the type: `Trait { which: Add, at: Int, args }`. The closure compiler then picks the one implementation for that type, and nothing is looked up at runtime.
- At a type variable that is generalised with a bound, `let Max<a: Ord>: fn a, a -> a`, the constraint becomes a hidden parameter of the function: the scheme is `forall a. Ord a => ...`, the body's `x < y` is `Trait { which: Lt, at: a, args }` whose implementation is the hidden argument `Dict { Ord, a }`, and every call to `Max` at a concrete type passes `Dict { Ord, Text }` first. This is dictionary passing for three dictionaries, and the dictionary is an ordinary function value, erased like everything else.
- At a type variable with no bound in scope, the error names the operation and the variable: "`<` needs `Ord a`; add the bound `<a: Ord>` to `Max`".
- At a type variable that is never resolved and never generalised, which is the case of an integer-spelled literal nothing pins, the variable defaults to `Int` at the end of the item, and the text form prints the literal as `Int` like any other.

Records have equality when all fields do, and ordering when all fields are ordered, lexicographic in field declaration order, as Rust's derived `Ord` is; tuples, arrays, primitives and text are ordered; extensible variants are ordered by tag name, then by plain fields, with any request event excluded as it is from equality, so elements can key sorted maps; element equality is tag plus plain fields, excluding any request event, so that two `Fs.Contents(path="/etc/motd")` are equal whatever events they hold, which is the identity the World's oracle keys on. The elaborator computes these per type once, memoised in `Types`, and the answer for a declared type is derived from its parts.

The negative space: no user-defined traits, no instances, no associated types. The constraint set is three symbols; the solver is a table lookup at concrete types and a hidden parameter at bounded variables. A general trait system would put instance search into the elaborator and a dictionary per instance into generic code; the three traits are enough for sorting, map keys, element identity and arithmetic, and the elaborator's constraint set is written so that a fourth symbol is a table row, not a redesign.

## Literals

An integer-spelled literal is `Nat | Int` by inference: the elaborator gives it a fresh variable with a `Num` constraint and a *spelling restriction* that excludes `Float`; a decimal-spelled literal is `Float` outright; `-1` is the negation of a literal and is `Int` outright (negation excludes `Nat`); prefix `+` is the numeric identity and pins nothing; `+?` converts an `Int` to a `?Nat`. A `Nat` is pinned by an annotation, `let n: Nat = 0`, since `Nat` has no suffixed operator. When inference leaves a literal's variable unresolved at the end of its item, it defaults to `Int`, the type a programmer means by an unadorned counter; the suffixed spellings `+~` (`Int`) and `+.` (`Float`) pin the type at the operator when a program wants to say so: `$count +~ !delta` constrains both operands and the result to `Int`. Defaulting happens once, after the whole item is inferred, so it never overrides a constraint that arrives later in the item.

The counter's numbers in this light: `-1` is `Int`; `+1 on !increment` has the variable of `1` constrained by

`Num` and excluded from `Float`; the merge unifies the two sides, so `1` is `Int`; `$count + !delta` unifies `$count`'s type with `Int`; the hold's `0` unifies with `$count`, so it is `Int`. Nothing needed the default, every literal has one type, and the text form prints each with it: `0:Int`.

#### `for`

`for k, $v <- $m: block` in a widget block is `For { coll: $m, block: \|k, $v\| @widget: ..., kind, keyed }`, where the block's parameters are the loop's binders (the shorthand `for $m:` is the same node with the parameters `key` and `$_`), and `kind` is the consolidation read off the block's result:

| Block result | Consolidation | Type |
|---|---|---|
| plain `R` | `Plain` | `$(K ==> R)`, changing when the key set changes |
| `!R` | `Event` | `!(K ==> R)`, firing with the entries that fired |
| `$R` | `Dynamic` | `$(K ==> R)`, changing when any entry or the key set changes |
| `@R` | `Widget` | an error: a block returns a built widget's result, not an unbuilt blueprint |
| a bundle | one `For` per field kind | the block is split into projections, as for `rebuild` |

Over an array the same node has `keyed: false` and `K = Nat`. Outside a widget block, `for` over a plain collection is a plain mapping and elaborates to an ordinary `App` of the prelude's `MapValues` or `Map`, with the binders (or `key` and `_` for the shorthand) as parameters of the body lambda; over a dynamic collection outside a widget block it is the error the language chapter gives.

### Build-time and occurrence-time marks

Every binding of a widget block gets a `Mark`. The rule is one line: a binding is `Occ` when the root node of its definition is a lift (`DMap`, `Map`, `Tag`, `Leftmost`, `Switch`, `Join`, `Updated`, `Distinct`, `HoldFirst`, or a `For` with `Event` or `Dynamic` consolidation); it is `Build` otherwise (a `Bind`, a `Hold`, a `Placeholder`, a `Let` of a plain value, of a `Const`, of a blueprint, of a function). A `Build` binding's step runs once when the node is built; an `Occ` binding is a graph node the runtime recomputes. A `Build` binding may contain `Occ` subexpressions, as `@Label "Count: {$count}"` contains a `DMap` as its argument; the mark is on the binding and says when the binding's own step runs, and the nested lift is a node the closure compiler constructs on the way.

Why the mark is on the tree and not left for the compiler to recompute: it is a meaning. `let x = ExpensiveF a` runs once per build and `let $y = ExpensiveF $a` runs once per change, and a reader of the text form should see which without re-deriving the lifting rule. A binding that moves between marks is the kind of regression a golden catches and a reviewer reads.

### The causality check

A cycle in a widget block is legal exactly when it passes through a delayed edge, and the elaborator checks it over the resolver's groups with the labels finalised. The delayed edges, stated as things in the typed AST:

- a dynamic in the `dyns` of a `Tag` (read before the occurrence);
- the `events` of a `Switch` (the new inner is adopted at commit);
- a hold's value side, which is what a tag reads, so a hold referenced through a tag is the common case of the first; the hold's change *event*, which a `DMap`, a `rebuild` or a `for` over the hold reads, is prompt;
- the child list consumed by the responses of the same `intercept` (routing against the opened list).

Everything else is prompt, and three things are prompt that a careful programmer might hope were not: a `rebuild`'s event (it fires in the occurrence of the rebuild); a `DMap` over a hold (it reads the change event); and the dynamic of a `rebuild` or a `for` (the rebuild is in the occurrence of the change). The negative golden for a cycle through `rebuild` alone pins the first.

The `for` edge is the one the resolver could not label. The collection's change leads promptly to the consolidated event only through what a newly born block fires at birth, because a block that existed before the occurrence contributes to the merge on its own schedule and a block born in n cannot receive a response in n. So the edge from `coll` to a `For` with `Event` consolidation is `Prompt` if the block's result event is *birth-firing* and `Delayed` otherwise, and birth-firing is a syntactic property of the typed block, computed bottom-up:

- `Started` is birth-firing; a `Dyn`'s event is birth-firing; `Updated` of a hold whose event is birth-firing is birth-firing;
- `Map`, `Tag` and `Leftmost` are birth-firing if any event operand is; `Match` on an event likewise;
- `Switch` is never birth-firing (the creation-occurrence firing is swallowed); an element's response is never (routing against the opened list); a bound widget's result event is birth-firing if the callee's block says so, which the elaborator computes per widget function once and records on the function; a `For`'s consolidated event is birth-firing if its block's is;
- a blueprint whose body is not visible, because it is a parameter `@a`, is assumed birth-firing, and the diagnostic for a cycle through such an edge says "assumed to fire at build; pass a blueprint whose result is held or switched".

Under this rule the ordinary todo list is legal: its rows return events derived from element responses, which are not birth-firing, so the edge from `$items` to `!deletes` is delayed and the cycle `$items -> for -> !deletes -> hold -> $items` has its delayed edge. And `@ Delete on !started` as a row result is a compile error: `Delete on !started` is a `Map` over `Started`, birth-firing, so the edge is prompt, and the cycle through the hold's update event is all prompt. The error lists every binding in the cycle in order with spans and says which edge made it prompt.

The check itself: for each cyclic group, take the subgraph of its members with every edge labelled, remove the delayed edges, and look for a cycle among what remains with a depth-first search. A cycle found is the error; its members are reported in the order the search met them, starting from the member whose span is earliest. Cycles that pass through other widget functions, where a widget binds a child whose result feeds back into something the child was given, are not visible to a per-block check and are reported by the runtime's re-entry check with the spans the compiler attached to every cyclic placeholder, which is the division of labour the specification states.

Suppose there were no static check and every cycle were left to the runtime. The runtime does find them, and names the node; the cost is that the counter's author who writes `$count <- 0 then $count + 1` (no event: a derived dynamic reading itself promptly) learns about it at the first occurrence instead of at compile time, and learns the name of one node instead of the list of bindings. The static check is cheap because the resolver already computed the groups and the elaborator already placed the lifts, and a per-block check is the whole of what the typed AST can see.

## Exhaustiveness and the catch-all

A `match` on a closed `choice` must cover every alternative or end in a catch-all; one on `?T` must have `Some` and `Nil` or a catch-all; one on an array must cover every length (see below); one on a literal type must end in a catch-all; one on an extensible variant must end in a catch-all arm, `ignore:` or a bare name, because the set of tags is open. The elaborator records `Dispatch` on the `Match` node so that the closure compiler knows which test to emit, and checks coverage with the usual matrix algorithm for nested patterns (`Some (Circle r)` under `?Shape`), reporting the first missing constructor by name. Arms after a catch-all are "unreachable arm" errors rather than warnings, because an unreachable arm is a bug the reader will otherwise trust.

A guarded arm does not count toward coverage. The matrix is built from the unguarded arms only, because the elaborator cannot know the guard will hold; the guarded arms are still typed and still checked for reachability against the unguarded arms above them. So `Some n if n > 0:` followed by `Nil:` is a missing-`Some` error, the message names the guarded arm ("this arm has a guard, so `Some _` is still uncovered; add an arm without one"), and a guarded catch-all, `ignore if c:`, is not a catch-all: a `match` on an extensible variant or a literal type must end in an unguarded one.

Array patterns enter the matrix as constructors by length. `[[]]` is length 0, `[[a, b]]` is exactly 2, and `[[head, ..rest]]` is at least 1 (at least $k$ for $k$ elements before the `..`). Coverage is checked at the lengths 0 up to one past the longest pattern and then at "longer than that", where only a `..` pattern can match, so `[[]]` with `[[head, ..rest]]` covers every array and `[[a, b]]` with `[[]]` is missing the lengths 1 and 3 and up, reported as the first missing one. Element patterns inside are specialised in the usual way, and the `Dispatch::Array` the match records tells the closure compiler to emit length tests. Record patterns without `..` must name every field; `..` is required when they do not, and an element pattern `Ns.*(..)` always has it.

## The tag-indexed refinement

`type Slot = record: element: Element; !request: Request` is a plain record, and inside a `match` arm on a tag the types of its fields are refined. The rule is small and lives in the elaborator's environment rather than in the type:

- Inside an arm whose pattern is `T(...)` for a declared tag `T`, the scrutinee's binding, and any `Slot` whose `element` was the scrutinee, have `!request` at `T`'s declared request type (or no `!request` if `T` does not request), and `@Element` applied to an element known to be built from `T` returns exactly what `T` declares: `@!R` for `responds: R`, `@$S` for `state: S`, `@((!R, $S))` for both, `@unit` for neither. The elaborator keeps a map from binding to "known tag" that the arm pushes and pops.
- Inside a catch-all arm, or a `Ns.*(..)` arm, `_.!request` is the generic `!Request`, and `@Element _.element` is the generic `@((!Response, $State))`, with `ElemKind::Generic` on the node so the closure compiler emits the slot creation that checks the per-slot types at runtime. The pattern `((!resp, ignore)) <- @Element _.element` in every forwarding interceptor is this case.
- A supplier lambda `\|e| ...` is checked at `fn Element -> @$State` generically, and inside a `match e:` arm on `T` with `state: S`, the arm's blueprint must return `@$S`; the catch-all arm returns the generic `@$State`, which `@Element e` provides.

The refinement is a checker rule, not a runtime structure: the typed AST records `ElemKind::Known(T)` or `Generic` on each `Element` node, and the runtime's slot creation takes the tag's parts from the table either way. The refinement buys this: an interceptor with one arm per tag it handles is typed at those tags, and one that forwards what it does not know is typed generically and checked at the slot.

`Kind` is a prelude function `fn Element -> Kind` where `type Kind = record: requests: Bool;`

`responds: Bool; state: Bool`, elaborated to a `Kind` node, which the closure compiler turns into a table lookup on the interned tag; a program that wants to know whether it is safe to read a generic slot's state asks `Kind` rather than matching every tag.

### `test` items

A `test counter, @counter: ...` form is typed as a blueprint and a list of turns, each turn a list of inputs (`click`, `send`, `with`, `push`, `build`, `idle`) and a list of checks (`expect`, `expect_top`, `expect_none`, `expect_top_none`, `expect_count`, `returns`, `fired`, `quiet`, `requested`, `unrequested`, `errors`, `settles`), and the patterns inside them are element patterns typed like any `match` pattern. The elaborated test is a `TItem::Test` the harness of the conformance chapter collects; nothing about it is special to elaboration beyond the prelude types, and that is the point of spelling tests in the language.

## Closure conversion

You have shipped a reactive program with a leak you could not find, and when you found it, it was a closure: a handler captured the thing that owned the handler, and the two kept each other alive after the view was gone. Every framework that lets a closure capture a pointer has this bug waiting, and the usual fix is a collector that walks the heap looking for it. Prism's runtime has no collector, and the reason it can do without one is a rule the compiler enforces mechanically: a closure never holds a strong pointer to anything in its own node's graph. It holds *indices* into the node's environment, and the node holds the environment. Closure conversion is the pass that makes every capture an index, and it is the pass whose output is the first form the closure compiler can run.

### Function ids and capture lists

Every `Lambda` in the typed AST becomes a function in a program-wide table, identified by `FunId`, with an explicit list of what it captures:

```
pub struct Function {
    pub id: FunId,
    pub params: Vec<TPat>,
    pub captures: Vec<Capture>,          // in order; the closure's environment is exactly this
    pub body: TExpr,                     // references to captures become Var(Capture(i))
    pub kind: FunKind,                   // Leaf | Widget
    pub span: Span,
}
pub enum Capture { Env(usize), Outer(usize) }   // from the enclosing block's environment, or from the enclosing function's captures
pub enum FunKind { Leaf, Widget }
```

The pass walks each item, assigns ids depth-first, and for each lambda computes its free variables: the names it mentions that are bound outside it. Each free variable becomes a capture, and the body's references to it are rewritten to `Var(Capture(i))`. A free variable that is itself a capture of the enclosing lambda is `Outer(j)`, so that a closure nested two deep reaches the block's environment through its parent's capture list rather than by its own path, which keeps the invariant that every capture is resolvable from the owning node's environment in one step. Top-level names and prelude names are not captures; they are `Global(ItemId)` references resolved from the program table. Constructors and tags are not captures.

`FunKind::Widget` marks a lambda whose body is a widget block: a `\@widget:` function, the block of a `rebuild` or a `for`, a supplier's result. Its captures are what the block's bindings can see from

outside: the enclosing block's environment entries. `FunKind::Leaf` is everything else, and a leaf's body contains no `Bind`, `Hold` or `Element`, which the lint checks.

**Recursive functions refer to each other by id.** Two top-level lambdas that call each other, or a lambda that calls itself, do not capture each other; a call to a function is `App { f: Global(id) }`, and the program table owns every function. A closure record at runtime is therefore `(FunId, captures)`, where the captures are values, and no closure holds another closure except as a captured *value*, which is a plain function value built from a function id and its own captures, pointing into the program table and never back at a node. There is no way to make two closures own each other because neither holds the other; they hold ids.

## Captures through the owning node's environment

A widget block compiles to a node, and the node has an environment: one slot per binding of the block, in group order, holding whatever the binding produced (an event handle, a dynamic handle, a plain value, a blueprint value, a function value). An occurrence-time closure compiled from `$count + !delta` needs `$count`, and it gets it as `Env(3)`, say: at build, the closure compiler resolves the index against the node's environment and copies the handle into the closure. The closure then holds an `Rc` to `$count`'s dynamic, and that is fine: the dynamic's node does not hold the closure. The closure is held by the *event node* that `Tag` constructs, which is subscribed to `!delta` and to `$count`'s behavior through *weak* subscriber entries. The strong edges run from the tag node to its inputs; the inputs reach the tag node only weakly. Dropping the widget's node drops the environment, which drops the tag node, which drops the closure, which drops its `Rc` to `$count`; and `$count`'s hold, which was the thing that could have kept the loop alive, holds only a weak reference to the tag node in its subscriber list.

The rule, then, is about direction: a closure captures things bound *earlier* in group order, which are things built earlier, and the graph's reverse edges are weak. Every strong edge points from later to earlier in build order, so there is no strong cycle. The one exception is a cyclic group, where a later binding is referenced by an earlier one, and that is exactly where the resolver placed a placeholder; a placeholder is a weak forward reference owned by the node as a unit, closed to the real node after the group, so the loop's only strong roots are the node's placeholder slots and the loop dies with the node.

Two more rules keep this true for the constructs that build at occurrence time. A `rebuild` or `for` block compiled as a widget function captures from the enclosing block's environment, and the child node it builds holds handles *up* into its parent's environment, which is the allowed direction, since the parent owns the child. A supplied widget is built in the interceptor's node and captures from the supplier closure, which was built in the interceptor's block; again up. A child never holds a strong handle into a sibling or into its own descendants, because nothing in the language can name those; the resolver's scoping guarantees it.

Suppose instead the compiler let a closure capture a `Dynamic` handle directly from wherever it found it, including a sibling's node under a `for`. The handle is an `Rc`; the sibling's node holds the dynamic strongly; if the sibling's dynamic is ever subscribed to the capturing closure's node strongly, the pair is a cycle nothing frees. The runtime chapter lists the leak test that pins this: a widget whose occurrence-time closure reads a sibling dynamic, dropped, must free the dynamic's node, which it does only if the capture went through the environment and the subscription was weak.

### Placeholders and closes

The resolver found where the knots are cut; the converter writes the cut into the tree. For each cyclic group, every delayed-edge target gets a `TDef::Placeholder { kind }` binding inserted at the group's start, with `kind` `Event` or `Dynamic` by the name's sigil (`Widget` and plain placeholders do not exist, because a cycle through a widget or a plain value has no delayed edge and was rejected). The original definition keeps its name with a suffix the text form prints (`$count.hold` for a hold, `!x.def` otherwise) and the references that were on delayed edges are rewritten to point at the placeholder binding. After the group's last member, a `TDef::Close { placeholder, real }` binding is inserted. The group's members are reordered into the topological order the cut gave.

The output is the same typed AST with three annotations: `captures` filled on every `Lambda`, `Function` entries in the program table, and the placeholder and close bindings in place. The text form prints all three, so a golden of the converted tree shows every capture list and every cut, which is where the hard bugs at the runtime boundary are and where a reader wants to look.

## The lint pass

A compiler with one intermediate form and several passes over it needs one checker, run after every pass and in every test, so that the pass that breaks the form is the pass that gets blamed. The lint is Prism's `-dcore-lint` (GHC's Core-lint flag, from memory). It is a walk over the converted typed AST that re-derives what each pass claimed and compares. Each check is an invariant with a stated consequence, because an invariant without a consequence is a comment, and a checker that enforces comments is one that gets switched off the first time it is inconvenient.

**Types agree.** Every node's type is the type its children and the typing rules give it: a `Tag`'s type is `!` of its body's result; a `DMap`'s is `$` of its body's; a `Hold`'s is `$` of its initial's type and its event is `!` of that; an `App`'s is the function type's result after unifying the arguments; a `Switch`'s input is `!!T` and its output `!T`; a `Join`'s input is `$$T`. *What breaks if it fails:* the closure compiler emits a step whose operand kinds do not match the runtime constructor's, and the runtime's `Typed<T>` downcast panics in the middle of an occurrence. The lint reports the node and both types before anything is built.

**Every lift is explicit.** No `App`, `Trait`, `Field`, `Index`, `TextCat` or `Match` has an operand whose outermost type is `!` or `$` unless the node is itself the body of a lift and the operand is a parameter of that body. *What breaks:* a dynamic reaches the closure compiler as a plain operand, and the compiled leaf tries to add a `Dynamic` handle to an integer. The lint says "unlifted dynamic operand" with the node.

**Marks are right.** A binding marked `Occ` has a lift at its root; one marked `Build` does not; a `Build` binding reads no `Occ` binding's *value* (it may hold its handle). *What breaks:* a `Build` step runs once with the current value of something that was supposed to change, and the program silently shows its first value forever; this is the bug the marks exist to make visible, and the lint is what makes the marks trustworthy.

**Every placeholder has exactly one close.** For each `Placeholder` binding there is one `Close` naming it, after it, in the same block, with a `real` of the same kind and type. *What breaks:* a placeholder never closed panics in the runtime at the first sample ("cyclic dynamic never closed"); closed twice panics at the second close ("set cyclic dynamic twice"); closed at the wrong type panics at the close. The runtime's typed cyclic handles make the third a Rust compile error in hand-written blueprints and the lint makes all three compile errors in compiled ones.

**Nothing samples a placeholder before its close.** Between a placeholder and its close, no binding

has the placeholder in a *sampling* position: the dynamic of a `Dyn` or `For`, the `dyns` of a `Join`, the initial value of a `Hold`, the event of a `HoldFirst`, the child or supplier of an `Intercept`, or any plain position. Subscribing positions (the `dyns` of a `Tag`, the `events` of a `Switch`, the `event` of a `Hold` or `Map`, the responses of an `Intercept`) are allowed. *What breaks:* the build reads a value that does not exist yet; the runtime panics. The resolver's cut makes this hold by construction, and the lint is what catches a resolver that cut in the wrong place.

**Every capture is a free variable.** For each `Function`, its `captures` list equals the set of free variables of its body, in order, with no entry that the body does not mention and no mention that the list lacks; each `Env(i)` names a binding of the enclosing block that is bound *before* the function's binding in group order, or is a placeholder of the same group; each `Outer(j)` names a capture of the enclosing function. *What breaks:* a missing capture is a dangling environment index at build, which the closure compiler would turn into a wrong value or a panic; a capture of a *later* binding is a strong forward pointer, which is the heap cycle the whole design exists to prevent. The negative lint fixtures `lint_rejects_a_capture_of_a_bound_name` and the leak test in the runtime chapter are the two ends of this invariant.

**Leaves are pure.** A `FunKind::Leaf` body contains no `Bind`, `Hold`, `Element`, `Intercept`, `Dyn`, `For`, `Placeholder` or `Close`, and no `WidgetBlock`. *What breaks:* a leaf that built something would build it inside the propagation loop with no node to own it, and the runtime has no path for that; the construction rule has nothing to say about an orphan. The lint also checks the converse: a `Widget` function's body is a `WidgetBlock`.

**Build order respects the groups.** The bindings of each block appear in the order of their groups, every group after every group it depends on, and within a cyclic group in the order the cut gave. *What breaks:* a binding's step runs before an environment entry it needs has been filled, and the index is read as uninitialised. The lint recomputes the dependency order from the references and compares.

**Every node has a span.** *What breaks:* an error at runtime says "in `<unknown>`", which is the worst message a compiler can produce, and the one a careless pass produces by constructing a node with a default.

**Every `Element` node names a declared tag or is `Generic`,** and a `Known` tag's fields match its declaration in number and type. **Every `Match` with `Dispatch::Tag` ends in an unguarded catch-all,** and every guard is a plain `Bool`. **Every `Trait` node is at a concrete type or has a `Dict` in scope.** Each is a restatement of an elaboration rule, and each exists because a later pass could drop the annotation the rule produced.

The lint runs in every test of the compiler and after every pass in `prism dump --lint`; it has negative fixtures of its own, hand-edited converted trees that it must reject, one per check, because a checker nobody has seen fail is a checker nobody knows is on. The checks are cheap: one walk, with a map from binding to type and a stack of placeholder states. A checker that was expensive would be run only sometimes, and a checker run sometimes is where the bug it would have caught waits.

What the lint is not: a second elaborator. It does not infer; it compares what the tree says against what the tree's children say, and when the two agree it says nothing. If a rule of elaboration changes, the lint changes with it, and the two are kept in one crate so that they cannot drift apart unnoticed; a divergence between them fails every golden at once.

## The closure compiler

The last stage is the one with the least to decide and the most to get exactly right, because it is where the compiler meets a runtime that was written without it. The runtime asks for two things from a language: builder calls at build time, and Rust closures the graph calls at occurrence time. A compiled blueprint has to be a blueprint the runtime cannot tell from one written by hand, a compiled closure has to meet the closure bounds the constructors state, and nothing the compiler produces may hold the runtime's objects in a way the ownership rules forbid. The closure compiler is one walk over the converted typed AST that turns each node into one of those two things, and what follows is the walk, node by node, with the Rust types it produces.

### `IrBlueprint` and the program

```
pub struct Program {
    pub functions: Vec<CompiledFunction>,    // by FunId
    pub blocks: Vec<CompiledBlock>,          // by BlockId: the widget blocks
    pub tags: Rc<Tags>,                      // the interned tag table, handed to the runtime at build
    pub constants: Vec<Value>,               // literals and other plain constants, boxed once
}

pub struct CompiledBlock {
    pub steps: Rc<[Step]>,                   // in group order
    pub env_size: usize,
    pub result: Operand,
    pub span: Span,
}

pub struct IrBlueprint {
    pub program: Rc<Program>,
    pub block: BlockId,
    pub captures: Rc<[Value]>,               // the values the block captured from its enclosing node
    pub span: Span,
}

impl DynBlueprint<Value> for IrBlueprint {
    fn build(&self, builder: &mut WidgetBuilder) -> Value {
        let block = &self.program.blocks[self.block];
        let mut env: Vec<Option<Value>> = vec![None; block.env_size];
        for step in block.steps.iter() {
            run_step(step, &self.program, &self.captures, &mut env, builder, self.span);
        }
        read(&env, block.result)
    }
}
```

`IrBlueprint` is one Rust type for every compiled blueprint: the root `@main`, a `\@widget:` function's body once applied, the block of a `rebuild`, the block of a `for`, the blueprint a supplier returns. It implements the runtime's `dyn`-compatible widget trait at `Value`, so a `rebuild` can rebuild into a different block on each rebuild and compiled widgets and Rust widgets sit side by side in one node tree. The runtime builds it by calling `build` with a builder for a fresh node, which is what it does for every widget, and gets back the block's result as a value. The two things that distinguish a compiled widget from a hand-written one, the program table and the environment vector, are inside `build` and invisible outside it.

A blueprint *value* in the language (a `let @b = @Button $"OK"`, the result of applying a widget function,

the thing a supplier returns) is an `IrBlueprint` boxed as `Rc<dyn DynBlueprint<Value>>` and stored as a plain value; binding it twice builds two nodes because `build` is called twice with two builders, which is exactly what "a blueprint is a description, a node is what one bind builds" asks for.

## The environment

A block's environment is a `Vec<Option<Value>>` with one slot per binding of the block, indexed in group order, plus the captures vector for what the block sees from outside. Every operand in a step is an `Operand`:

```
pub enum Operand { Env(usize), Capture(usize), Const(usize), Global(ItemId) }
```

`Env(i)` is a binding of this block, filled by an earlier step (or a placeholder, filled at the group's start); `Capture(i)` is an entry of the `IrBlueprint`'s captures; `Const(i)` is a boxed literal in the program's constant pool; `Global(i)` is a top-level definition, a function value or a plain constant computed once per program. The `Option` is there so that a read of an unfilled slot is a panic with the step's span in debug builds, rather than a wrong value; the lint's build-order check is what makes the panic unreachable, and the `Option` is what makes a lint bug loud instead of quiet.

The values in the environment are the runtime's `Value`: events and dynamics as handles (heap values holding an `Rc` to the runtime object), plain data as plain data, blueprints as boxed `IrBlueprint`s, functions as closure records. Everything a step produces goes into the environment; everything a step consumes comes from it. There is no stack, no register file and no frame for construction code, because construction code runs once and its locals are the node's values.

```
pub enum Step {
    Let       { dst: usize, expr: LeafId },                                  // a plain value, computed once at
build
    Const     { dst: usize, value: Operand },                                // Dynamic::constant
    Bind      { dst: usize, widget: Operand, pat: Option<PatId> },           // builder.bind(&widget)
    Hold      { dst: usize, init: Operand, event: Operand },                 // Dynamic::hold(runtime, init,
event)
    HoldFirst { dst: usize, event: Operand },                                // Dynamic::hold_first(event)
    Map       { dst: usize, event: Operand, f: LeafId, captures: Vec<Operand> },
    Tag       { dst: usize, event: Operand, dyns: Vec<Operand>, f: LeafId, captures: Vec<Operand> },
    DMap      { dst: usize, dyns: Vec<Operand>, f: LeafId, captures: Vec<Operand> },
    Leftmost  { dst: usize, events: Vec<Operand> },
    Never     { dst: usize },
    Switch    { dst: usize, events: Operand },                               // switch_hold
    Join      { dst: usize, dyns: Operand },                                 // Dynamic::join
    Updated   { dst: usize, dynamic: Operand },                              // dynamic.event()
    Distinct  { dst: usize, dynamic: Operand, eq: LeafId },
    Started   { dst: usize },
    Dyn       { dst: usize, dynamic: Operand, block: BlockId, captures: Vec<Operand> },
    For       { dst: usize, coll: Operand, block: BlockId, captures: Vec<Operand>, kind: Consolidation, keyed:
bool },
    Element   { dst: usize, elem: Operand, kind: ElemKind },                 // builder.element(..)
    Intercept { dst: usize, child: Operand, responses: Operand, supplier: Operand },
    Placeholder { dst: usize, kind: PlaceholderKind, name: Span },           // add_cyclic_event / _dynamic
    Close     { placeholder: usize, real: usize },                           // close(handle, real)
    Project   { dst: usize, src: usize, field: usize },                      // a bundle's field after a bind
}
```

One `Step` per typed-AST node kind, each one call on the builder or one constructor on an event or dynamic, with operands resolved to environment indices. The mapping, which is the whole of how the language meets the runtime:

| Typed AST | Step | Runtime call |
|---|---|---|
| `Bind` of a blueprint | `Bind` | `builder.bind(&widget)`; the child's node is pushed onto this node's children; the result goes into `dst` |
| `Hold { init, event }` | `Hold` | `Dynamic::hold(init, event)`: value `init` until the event fires, commits at the end of each occurrence the event fires |
| `HoldFirst` | `HoldFirst` | a hold whose initial value is the event's catch-up value in the build occurrence; from `@HoldFirst` written in a program, or inserted for a dynamic-of-blueprints bind |
| `Map { event, body }` | `Map` | `event.filter_map(closure)` with a closure that always returns `Some` (one node; `filter_map` is the runtime's primitive and `map` its special case) |
| `Tag { event, dyns, body }` | `Tag` | `event.tag_map_n(dyns, closure)`: the closure receives the firing and each dynamic's value *before* the occurrence; n-ary over the dynamics so that `$a + $b + !e` is one node |
| `DMap { over, body }` | `DMap` | `Dynamic::map_n(dyns, closure)`: a derived dynamic, prompt, one node for any arity |
| `Leftmost` | `Leftmost` | `Event::leftmost(vec)` |
| `Never` | `Never` | `Event::never()` |
| `Switch { events }` | `Switch` | `events.switch_hold()`: non-prompt; adopts the new inner at commit |
| `Join { dyns }` | `Join` | `dyns.join()`: prompt on both outer and inner |
| `Updated { dynamic }` | `Updated` | `dynamic.event()`: the change event is a value the dynamic already has; no node is built |

| Typed AST | Step | Runtime call |
|---|---|---|
| `Distinct { dynamic, eq }` | `Distinct` | a derived dynamic whose change event is the source's filtered by `eq old, new` against the value before the occurrence; one node, with the equality closure |
| `Started` | `Started` | the builder's done event filtered to `unit`: fires once at `height + 1` in the build occurrence |
| `Dyn { dynamic, block }` | `Dyn` | `dynamic.map(\|v\| IrBlueprint { block, captures: captures + [v] }).rebuild()`: a dynamic of compiled blueprints handed to the runtime's `rebuild`; its event is the `rebuild`'s result event |
| `For { coll, block, kind }` | `For` | `coll.list_blueprint(\|key, per_key\| IrBlueprint { block, captures: captures + [key, per_key] }, kind)`: one child node per key, consolidated by `kind` |
| `Element { elem, kind }` | `Element` | `builder.element(elem_value, request)`: mints the path, creates the response output if the tag responds, asks the supply context for the state dynamic if it has state, stores the request event if it requests; returns what the tag's parts imply |
| `Intercept` | `Intercept` | `builder.bind(&InterceptBlueprint { child, responses, supplier })`: the child's node is marked captured, its list is the second result |
| `Placeholder` | `Placeholder` | `builder.add_cyclic_dynamic(name)` or `add_cyclic_event(name)`, returning a typed handle; the placeholder's dynamic or event goes into `dst` |
| `Close` | `Close` | `handle.close(real)`: the one write to a placeholder |
| `Kind { elem }` | a `Let` whose leaf reads the tag table | `tags[elem.tag]`'s three flags as a record |

The runtime's target API has three constructors in the table for the compiler's sake: `hold_first`,

`map_n` over any number of dynamics (so that a lift over three dynamics is one node and not a tree of `map2`s that allocate pairs), and `tag_map_n` over a vector of dynamics for the same reason. Each is a small generalisation of what exists and each exists because the compiler would otherwise emit several nodes for one lift, and the text form, which prints one node per lift, would then be a lie about the graph.

**Build-time versus occurrence-time, in the steps.** A `Build` binding is a step that runs once inside `build` and writes its slot: `Let`, `Const`, `Bind`, `Hold`, `Placeholder`, `Close`, `Element`, `Intercept`, and the constructors that make a node without a closure. An `Occ` binding is a step that constructs a graph node holding a closure, and the closure is the thing that runs at occurrence time: `Map`, `Tag`, `DMap`, `Distinct`. The step itself still runs once, at build, to make the node; what runs per occurrence is the closure inside it. The marks on the typed AST and the two kinds of step line up one to one, and the lint's mark check is what guarantees that a `Build` binding never ends up as a closure the graph calls.

## Leaves

An occurrence-time closure, and a plain expression computed once at build, are both *leaves*: pure code over values. The closure compiler compiles each leaf once, when the program is compiled, into a tree of Rust closures, and the step that needs it refers to it by `LeafId`:

```
pub type Leaf = Rc<dyn Fn(&Frame) -> Value>;

pub struct Frame<'a> {
    pub args: &'a [Value],          // the closure's arguments: the firing, the dynamics' readings
    pub captures: &'a [Value],      // resolved at build from the node's environment
    pub program: &'a Program,
    pub depth: Cell<u32>,           // the recursion guard
}
```

Compilation of a leaf is the closure-generation technique in its plainest form: each typed-AST node becomes a closure that calls its children's closures and combines their results, so that evaluation is a tree of calls with no dispatch loop and no instruction stream. `Var(Capture(i))` compiles to `|f| f.captures[i].clone()`; `Var(Param(i))` to `|f| f.args[i].clone()`; `Trait { Add, Int, [a, b] }` to `|f| box_int(unbox_int(a(f)) + unbox_int(b(f)))` with overflow checked and reported as an `Overflow` error; `App { f, args }` to a call through the program table; `Match` to the dispatch the node records, each arm compiled with its pattern as a test-and-bind followed, when the arm has a guard, by the guard evaluated with the pattern's bindings in the frame, so that a false guard falls through to the next arm; `Bool.And` and `Bool.Or` to a closure that calls its second operand's closure only when the first does not decide the result, so an overflow in an operand that was not needed aborts nothing; `If` to a test of the condition that calls only the branch it takes; `Ctor`, `Elem`, `Field`, `Index`, `TextCat`, `Maybe`, `Array`, `MapLit` to the obvious thing over the value representation. Numbers stay unboxed inside a leaf where the Rust compiler can see them, and box at the leaf's boundary. A node whose static type is a type variable moves the one-word value without looking at it, which is erasure, below.

A lambda value inside a leaf is a closure record `(FunId, captures)` built by evaluating the capture operands into a vector; applying it is `program.functions[id]` called with the record's captures as `captures` and the arguments as `args`. The record is a plain value and may be stored in a map, returned from a function or passed to `Fold`. It holds values, never nodes, and so can be copied anywhere a value can.

The closure the runtime receives for a `Map` step is the one closure type for all of Prism:

```
move |firing: Typed<Value>| -> Option<Typed<Value>> {
    let frame = Frame { args: &[firing.into()], captures: &captures, program: &program, depth: Cell::new(0) };
    Some(leaf(&frame).into())
}
```

It holds an `Rc<Program>`, a `LeafId` and an `Rc<[Value]>` of captures, all counted at build and only borrowed per occurrence. With the runtime single-threaded there is no `send + Sync` bound and no `Arc`; a compiled closure captures `Rc`s the way a hand-written blueprint's closure does.

## The recursion depth guard

A plain function that calls itself without end is exhaustion, not an error, and a host stack overflow is not a panic in Rust: it ends the process. The compiler does better by counting. Every call through the program table increments `Frame.depth` and decrements on return, and a depth past the limit (a few thousand, configurable at program load) aborts the occurrence with an `Overflow` error naming the function's span, which is the error chapter's "a conforming implementation does not hang" made concrete. The same counter guards construction: a widget function that binds itself, `@loop: @loop` through a `let`, increments a depth on the builder, and the build past the limit is a `Build` error with the widget's span rather than a stack overflow in `build`. The guard costs one increment per call and is not optional, because the alternative is a process that dies with no span.

## Boxing

One module, `boxing.rs`, is the only place a Rust `u64`, `i64`, `f64`, `bool`, `char`, `Text`, record or closure record becomes a `Value` or comes back. Every leaf's arithmetic goes through `unbox_int`/`box_int` and friends, every element field through `box_plain`, every pattern test through `unbox_ctor`. Today the representation behind `Value` is the runtime's `AnyValue`, which has inline `U32`, `I32` and `Unit` and boxes everything else behind an `Rc`; the values chapter's one-word representation puts 63-bit integers and more inline; and because the module is the only one that knows which, the representation changes behind it with no change to the compiler's walk. A compiled program that reached into `Value`'s representation from anywhere else would be a program that had to be rewritten when the representation changed, and the representation is going to change.

## Attaching spans

Every node the compiler builds gets the span of the binding it came from, through the builder's `with_span`, so that a runtime error names Prism source: a cycle found at a placeholder names the placeholder's binding, a panic in a closure names the lift, an unknown path names the `@Element`, a build failure names the block. The runtime keeps the span on the node and the error chapter's `Error` element reports it in `spans`. A compiler that attached spans only to some nodes would produce errors that say `<unknown>` for the rest, and those would be the errors inside the lifts, which are the ones a programmer needs most.

## The canonical node structure

A hand-written Rust blueprint and a compiled one must produce the same node structure for the same program, because the conformance suite's paths are built from that structure: `0/#1/0` is the first bind's `rebuild`'s second generation's first slot, whichever language wrote the blueprint. The structure the runtime chapter states, and the compiler follows: each `Bind` is one child node, in step order, with the next `Bind(i)` segment; each `Element` is one slot, sharing the counter with child binds;

each `Dyn` is one child node whose generations are `Gen(g)`; each `For` is one child node whose children are `Key(k)`; an `Intercept` is one child node holding the child as its own child; a supplied widget is a child of the interceptor's node under `s/n`. The counter compiled from source and the counter written against the builder by hand give the same list with the same paths, and the milestone that proves the two tracks met is a byte-identical transcript.

### What a compiled blueprint looks like at runtime

Take the counter again and watch `build` run for the block of `@counter` (the `@Button` and `@Label` bodies are their own blocks, built when bound). The steps in group order, with the environment as it fills:

```
step 0  Const   env[0] = Dynamic::constant("Increment")
step 1  Bind    env[1] = builder.bind(&IrBlueprint{block: @Button, captures: [env[0]]})  -> the button's
!Click (a switch)
step 2  Const   env[2] = Dynamic::constant("Decrement")
step 3  Bind    env[3] = builder.bind(&IrBlueprint{block: @Button, captures: [env[2]]})
step 4  Map     env[4] = env[3].filter_map(leaf: \ignore -> -1)
step 5  Map     env[5] = env[1].filter_map(leaf: \ignore -> +1)
step 6  Leftmost env[6] = Event::leftmost([env[4], env[5]])           -> !delta
step 7  Placeholder env[7] = builder.add_cyclic_dynamic("count @15:5") -> $count (open)
step 8  Tag     env[8] = env[6].tag_map_n([env[7]], leaf: \d, c -> c + d)
step 9  Hold    env[9] = Dynamic::hold(0:Int, env[8])                 -> $count.hold
step 10 Close   handle(env[7]).close(env[9])                          -> $count is the hold
step 11 DMap    env[11] = Dynamic::map_n([env[9]], leaf: \c -> "Count: " ++ ToText c)
step 12 Bind    env[12] = builder.bind(&IrBlueprint{block: @Label, captures: [env[11]]})
result  env[12]  (unit)
```

Steps 1, 3 and 12 each build a child node by calling the bound blueprint's own `build` with a sub-builder, which runs that block's steps; inside `@Button`'s block the one step is a `Dyn` over its argument whose block has one `Element` step, and inside `@Label`'s it is the same with no result. Thirteen steps, four graph nodes with closures (two maps, a tag, a derived dynamic), one hold, one placeholder closed once, three child nodes. After `build` returns, the environment is owned by the node and the steps are never run again for this node; the four closures are called by the runtime at occurrences 1, 2, 3, … as clicks arrive. Nothing in the compiled program knows that it has been built, and that is the thesis in one trace.

## The typed-AST text form

You have changed one rule in a type checker and found out a month later, from a user, that it changed the meaning of a program you never tested. The fix every mature compiler arrives at is the same: print the elaborated form of every test program, keep the printout under version control, and make any change to it a diff someone reads. The typed-AST text form is that printout for Prism, and it does three jobs at once: it is what the goldens diff, it is what a person reads to see what the compiler made of a program, and it is the bridge to the Haskell frontend, whose dump of a program must be byte-identical to the Rust frontend's. One binding per line, every lift a named node, every number with its type, the mark, the group, the span.

### The format

Each binding of a widget block is one line:

```
name: type [mark] group = expr @span
```

- `name` is the binding's name with its sigil, exactly as written in the source (`!delta`, `$count`); a

discarded result is `_`; a cyclic definition is `name.hold` for a hold and `name.def` otherwise; a bundle's projection is `name.field`.

- `type` is the full type with its sigils outermost: `!Int`, `$Text`, `@!Click`, `$(Path ==> Slot)`, `fn Element -> @$State`. Inferred type variables are single lowercase letters named by first appearance.
- `mark` is `[build]` or `[occ]`.
- `group` is `g` followed by the group's number in dependency order; placeholders, their definitions and their closes share the group number.
- `expr` is the expression with every lift explicit, in the node grammar below.
- `span` is `@line:col` of the binding's start in the source file; a block's own line takes the block's span.

A block is printed as its header line, its bindings indented by two spaces, and its result on a final line beginning `@`. Nested widget blocks (the block of a `rebuild` or a `for`, a `\@widget:` body) indent again. A placeholder line is `name: type [build] gN = placeholder` and a close line is `close name name.hold` with no type, since it defines nothing. A top-level `let` is `name: scheme = expr @span` with the scheme's bounds in angle brackets where the annotation gave them.

The expression grammar, which is the typed AST's node set in prefix form:

```
expr  := atom
       | 'const' expr                                  # $ v
       | 'dmap' '[' dyns ']' lambda                    # derived dynamic
       | 'map' event lambda                            # event map
       | 'tag' event '[' dyns ']' lambda               # event with dynamics read before
       | 'leftmost' '[' events ']' | 'never'
       | 'switch' expr | 'join' expr | 'updated' expr | 'distinct' expr expr
       | 'hold_first' expr
       | 'rebuild' expr lambda                             # lambda's body is a widget block
       | 'for' expr kind birth lambda                  # kind: plain | event | dynamic; birth: birth |
       no_birth, the can-fire-at-birth flag
       | 'element' (tag '{' fields '}' | expr) parts (expr)?   # parts: the tag's, or 'generic'; then the
       request if any
       | 'intercept' expr expr expr
       | 'started' | 'kind' expr
       | 'bind' expr                                   # in a Bind line: the widget being built
       | 'hold' expr expr                              # in a Hold line: initial, event
       | 'maybe' '[' exprs ']' lambda                  # ? lifting at the plain layer
       | 'match' expr dispatch '{' arm (';' arm)* '}'   # dispatch: choice | tag | optional | array | literal
       | 'if' expr expr expr
       | name '(' exprs ')' | name '{' field '=' expr, ... '}'  # application; constructor or element
       | trait '[' type ']' exprs                      # Num.Add[Int] c, d   Eq[Text] a, b   dict[Ord a]
       | 'Bool.Not' expr | ('Bool.And' | 'Bool.Or') expr ',' expr   # not a, a and b, a or b (And, Or
       short-circuit)
       | 'cat' '[' exprs ']'                           # text interpolation
       | expr '.' field | expr '[' expr ']'
       | lambda
atom  := name | literal ':' type | '"' text '"' | 'unit' | '[[' exprs ']]' | '{' pairs '}' | '((' exprs '))'
lambda:= '\' captures? '|' params '|' expr            # captures: '[' names ']' after conversion
       | '\' captures? '|' params '|' '@widget:' block
arm   := pat ('if' expr)? '->' expr                   # the guard, if any, is part of the arm
pat   := the source patterns in prefix form, array patterns as '[[' pats ('..' pat)? ']]'
```

Application prints with parentheses and commas, `Insert(items, id, todo)`, because the text form has no precedence and no paren-less calls: one syntax per node, read by a parser with no table. Literals print with their type, `0:Int`, `1.5:Float`, `"ok"` (text needs none). A `Trait` node prints the operation, the type in brackets, and the arguments, so that `+` resolved at `Int` and `+` resolved at `Float` are visibly different lines. The boolean operators have no type to resolve and print as `Bool.Not b`,

`Bool.And a, b` and `Bool.Or a, b`. A guard prints after its pattern, so `match n optional {Some(m) if Ord.Gt[Int] m, limit -> m; Nil -> 0:Int}` is a guarded arm and an unguarded one, and `if c Some(x) Nil` is what `if c: x` prints as.

The `for` node prints the elaborator's can-fire-at-birth flag (the `birth_firing` field of the node) between the consolidation kind and the block: `birth` when the row block can fire in the occurrence its key appears, `no_birth` when it cannot, and `no_birth` on a `plain` kind, whose rows build nothing. The flag is a line of the dump and not a note beside it because the Haskell evaluator chooses its `coincidence` from it and the cycle check reads it; a frontend that computed a different flag would otherwise differ only in an invisible field.

## The counter, dumped

The source, with the two library widgets it uses:

```
@Button: fn $Text -> @!Click = \|$label| @widget:
    !!clicks <- rebuild $label:
        @Element Ui.Button(label=_)
    @ !Switch !!clicks

@Label: fn $Text -> @unit = \|$text| @widget:
    rebuild $text:
        @Element Ui.Label(text=_)
    @ unit

@counter:
    !increment <- @Button $"Increment"
    !decrement <- @Button $"Decrement"
    let !delta = -1 on !decrement | +1 on !increment
    $count <- 0 then $count + !delta
    @Label "Count: {$count}"
```

`prism dump counter.prism` after elaboration:

```
  @Button: fn $Text -> @!Click = \|$label| @widget:                                  @1:1
    !!clicks: !!Click [build] g1 = rebuild $label \|label| @widget:                        @2:5
      _: !Click [build] g1 = element Ui.Button{label=label} responds                     @3:9
      @ _
    @ switch !!clicks

  @Label: fn $Text -> @unit = \|$text| @widget:                                       @6:1
    _: unit [build] g1 = rebuild $text \|text| @widget:                                    @7:5
      _: unit [build] g1 = element Ui.Label{text=text}                                   @8:9
      @ unit
    @ unit

  @counter: @unit                                                                      @11:1
    !increment: !Click [build] g1 = bind @Button(const "Increment")                    @12:5
    !decrement: !Click [build] g2 = bind @Button(const "Decrement")                    @13:5
    !delta: !Int [occ] g3 = leftmost [map !decrement \|ignore| -1:Int, map !increment \|ignore| +1:Int]   @14:9
    $count: $Int [build] g4 = placeholder                                              @15:5
    $count.hold: $Int [build] g4 = hold 0:Int (tag !delta [$count] \|d, c| Num.Add[Int] c, d)   @15:5
    close $count $count.hold                                                           @15:5
    _: unit [build] g5 = bind @Label(dmap [$count] \|c| cat ["Count: ", ToText[Int] c])   @16:5
    @ _
```

Read it as the compiler's statement of what it made of the program. `@Button` takes a dynamic and the `rebuild` inside it is the programmer's; its block has one slot, `element Ui.Button{label=label} responds`, whose tag responds and does nothing else, so the binding's type is `!Click`. In `@counter`,

the two binds are build-time and in their own groups. `!delta` is occurrence-time: `on` with a plain left side became a `map` over the event, the two maps are merged leftmost, and `-1` and `+1` are both `Int` because the merge unified them. `$count` is a cyclic group of one, cut at the tag: the placeholder line, the hold line reading `$count` through `tag`, and the close. `+` resolved at `Int` and prints as `Num.Add[Int]`; the hold's `0` prints as `0:Int`. The last line is a discarded bind of `@Label` applied to a derived dynamic, so it is build-time with an occurrence-time node inside. Every line says where in the source it came from.

After closure conversion, the same dump with `--converted` adds the capture lists, which here are all empty except the `rebuild` blocks', which capture nothing either, since their argument is the only thing they use:

```
    $count.hold: $Int [build] g4 = hold 0:Int (tag !delta [$count] \[]|d, c| Num.Add[Int] c, d)   @15:5
```

A program whose lift reads a sibling would show it: `\[$total]|d| Num.Add[Int] d, $total` says the leaf captures `$total` from the environment, and the lint has checked that `$total` is bound earlier in group order.

## The `for` flag, dumped

A `for` is the one node whose line says something the source does not: whether its row block can fire in the occurrence a key appears. Two programs differ only in that, and their dumps differ in one word.

```
@deletes: fn $(Nat ==> Text) -> @!(Nat ==> Click) = \|$items| @widget:
    !rows <- for $items:
        !delete <- @Button $"Delete"
        @ delete
    @ !rows
```

```
  @deletes: fn $(Nat ==> Text) -> @!(Nat ==> Click) = \|$items| @widget:                  @1:1
    !rows: !(Nat ==> Click) [occ] g1 = for $items event no_birth \|_| @widget:            @2:5
      !delete: !Click [build] g1 = bind @Button(const "Delete")                          @3:9
      @ !delete
    @ !rows
```

The row's result is an element's response, which the runtime routes against the list as the occurrence opened, so a new row cannot fire it at birth and the flag is `no_birth`. Replace the `@Button` with a blueprint that is a parameter of `@deletes`, whose body the elaborator cannot see, and the same line reads `for $items event birth \|_| @widget:`; so does a row that returns the event of a `rebuild` or an `@started`-derived event. The flag is also what the cycle check reads for the edge from the collection to the consolidated event.

## The dynamic-of-blueprints lift, dumped

The one place the elaborator inserts a `rebuild` the programmer did not write is worth a golden of its own, because its two forms are easy to get wrong:

```
@panel:
    !click <- @Element Ui.Button(label=$label)
    $text <- @TextInput $initial
```

```
  @panel: @unit                                                                  @1:1
    !click: !Click [occ] g1 = switch (rebuild (dmap [$label] \|l| Ui.Button{label=l}) \|e| @widget:    @2:5
      _: !Click [build] g1 = element e generic
      @ _)
```

```
$text: $Text [occ] g2 = join (hold_first (rebuild $initial \|i| @widget:                @3:5
  _: $Text [build] g1 = bind @TextInput(i)
  @ _))
@ unit
```

The first line is the lift of a dynamic field: the element construction became a `dmap` giving a `$Element`, `@Element` over it became a `rebuild`, and the bound event is the `switch` of the `rebuild`'s event, which is a graph node and so marks the binding `[occ]` even though the `rebuild` inside it is construction. The inserted block's `element` is `generic`, because inside the block `e` is a plain `Element` whose tag the checker cannot see through the `dmap`; the runtime checks the slot's types per tag, as it does for every generic element. The second line is a widget function returning a dynamic, applied to a dynamic: `join` of a `hold_first` of the `rebuild`, with the first build's `$Text` as the hold's initial value. A frontend that printed `hold` with some other initial, or `switch` for the second line, has read the specification differently, and the golden is where the difference shows.

## One definition for printer and parser

The printer and the parser of the text form are generated from one definition, so that the two cannot drift: a table that lists each node kind, its keyword, and its operand kinds in order, from which `print(&TExpr) -> String` and `parse(&str) -> TExpr` are both derived. In Rust the table is a `derive` on the typed AST enum with attributes naming each variant's keyword, and the two functions are the derive's output; in Haskell the same table is a datatype with a pretty-printer and a parser written from one `Grammar` value. The test is a round trip: print every fixture, parse the result, compare to the original tree; parse every golden, print it, compare to the file. A text form whose printer and parser were written separately would have cases the printer emits and the parser rejects, and the first such case would be found by the Haskell frontend's author, who would then not know whether the specification or the parser was wrong.

The parser of the text form exists for two consumers: the round-trip test, and hand-written fixtures. A fixture is a typed AST written by hand in this syntax, before the frontend can produce it, to drive the closure compiler against the runtime: the counter's converted dump, a `rebuild` button, a `match` on a choice, a `@started` request. They are the closure compiler's first tests, and when the frontend arrives the compiled counter has to dump identically to the hand-written one, which is the check that the two were built against one reading.

## What the text form is not

It is not a bytecode and not a serialisation for shipping compiled code. It has no version, no constant pool and no load-time validation, and it refers to tags and functions by name. If compiled code ever has to outlive the process that compiled it (precompiled libraries, a browser bundle without the frontend, programs sent to a World that did not compile them), the format is a serialisation of this same tree with a version and a load-time check, and the loader decodes back into the tree the closure compiler already takes. That is one more printer and one more parser from the same definition, not a second intermediate form.

It is also not pretty. There is no layout, no precedence, no paren-less calls, one syntax per node. A reader who wants the program reads the program; a reader who wants to know what the compiler made of it reads this, and every decision is on its own line.

### Freezing

The form changes while the frontend is being written and that is fine; it stops changing before the Haskell frontend starts, and from then on a change to the form is a change to both frontends and to every golden. The file that defines the table is the one place the form is defined, and a comment at its head records the date it was frozen, so that a divergence between the two frontends is never a question of which version of the form each implemented.

## Elements and tags in the compiler

An element is the one value in the language that crosses to the World, and everything the World knows about it comes from a tag declaration the compiler read. The tag table is therefore the compiler's most consequential data structure after the typed AST: it decides what an element's fields are, what `@Element` returns, what a `match` arm binds, what the runtime checks at the boundary, and what a World is told it may accept. This section is how declarations become the table and how each use of an element compiles against it.

### The tag table

```
pub struct Tags { pub by_id: Vec<TagInfo>, pub by_name: HashMap<Vec<Name>, TagId> }

pub struct TagInfo {
    pub id: TagId,                    // interned: a u32, dense, assigned in declaration order
    pub name: Vec<Name>,              // ["Ui", "Button"]
    pub namespace: Vec<Name>,         // ["Ui"]: the prefix, for Ns.* tests
    pub of: TypeId,                   // the extensible variant it belongs to
    pub fields: Vec<(Name, Type)>,    // concrete types only
    pub requests: Option<Type>,
    pub responds: Option<Type>,
    pub state: Option<Type>,
    pub eq: bool,                     // every field has equality (always true: fields are plain)
    pub span: Span,
}
```

`tag Ui.Button of Element:` with `label: Text` and `responds: Click` becomes one row. The elaborator checks, at declaration: every field type is concrete (no type variable, with the diagnostic naming the field and the variable); every field type has equality, which plain types always do, so that element equality is structural; the three parts are each at most once and their types are plain data (a `requests: !R` is written with its sigil in the declaration and stored as `R`, the event being implied); and no two tags share a name. The table is interned once per program, and the ids are what the runtime's `ElementValue { tag, fields }` holds. The same table, as `Rc<Tags>`, is handed to the runtime at `build`, which consults it for which parts a tag has and the response and state types to check at the boundary, and a World's `World.Tags` state element is made from it by the world core.

There is one tag table per program because there is one program: modules beyond one file plus the prelude are deferred, and until they exist every tag is declared where the compiler can see it. The design does not depend on that. Ids are dense in declaration order so that a later module system can intern across files with an offset, and `namespace` is stored as data rather than expanded into sets, so that a `Ns.*` test compiled before a module added a tag still matches the new tag. The alternative, assigning ids by hashing the name so that separately compiled modules agree, is the thing to do when separate compilation arrives, and nothing in the compiled form would change but the `by_id` vector.

## Construction

`Ui.Button(label="OK")`, `Ui.Button "OK"` and the block form all elaborate to `Elem { tag, fields, request }`: a plain record build with the tag id, fields in declaration order (named arguments reordered, positional checked for count), each field checked against its declared type with lifting over `?` stopped at the field (an element's field is plain, and `Ui.Row(height=xs[i])` is the diagnostic). A `requests` part, if the tag has one, is given as a field named `requests` holding an event, `Cli.Stdout(requests=!line)`, and is stored separately on the node because it is the one part of an element that is not plain data: it is the program's event, which the slot subscribes to, and it is excluded from the element's equality. The compiled leaf builds an `ElementValue { tag, fields: Rc<[Plain]> }` through the boxing module and pairs it with the request event handle.

A tag with no fields is a bare name, `Clock`, and constructs the same way with an empty field vector. A field whose argument is a dynamic lifts the construction into a `DMap`, and `@Element` over the result lifts into a `rebuild`; the compiled element inside the `rebuild`'s block is `Generic`, since the block sees a plain `Element`.

### `@Element`

`@Element e` compiles to `Step::Element { elem, kind }`, which calls `builder.element(value, request)`. The runtime mints the slot's path, creates the response output if the tag responds, asks the supply context for the state dynamic if the tag has state, stores the request event if the tag requests, and returns `ElementResult { response, state }`. What the step writes into the environment depends on `kind`:

- `Known(T)`: the compiler knows the parts and writes exactly what the type says: the response event for `responds`, the state dynamic for `state`, a tuple of both, or `unit`. The types were checked at elaboration against `T`'s declaration and nothing is checked at runtime.
- `Generic`: the element's tag is a runtime value, the type is `@((!Response, $State))`, and the step writes the pair of the response event and the state dynamic as the runtime returns them: a `never` for a tag that does not respond, and for a tag with no state a dynamic that nothing can read, because the generic `$State` is opaque and the only thing a program can do with it is hand it back to a slot or discard it. The runtime checks, per slot, that a response delivered to the path has the tag's response type and that the supplied state has the tag's state type, which is the "checked at runtime per slot" of the elements chapter.

The step also writes the slot's path nowhere, because a program cannot name a path; paths are the runtime's and the World's.

## `match` dispatch

A `match` compiles by the `Dispatch` the elaborator recorded on it, and a `match` on an extensible variant is the `Tag` case:

```
pub enum Dispatch {
    Choice(TypeId),    // a jump table over the alternative index
    Tag,               // see below
    Optional,          // Some / Nil
    Array,             // length tests: empty, exactly n, at least n
    Literal,           // sequential tests
}
```

For `Tag`, the compiled leaf reads the scrutinee's `tag` id and tests arms in order: an arm `T(...)` is `tag`

`== T`; an arm `Ns.*(..)` is `tags[tag].namespace starts with Ns`, a slice comparison on the interned namespace path, or a bitset per `match` precomputed over the table when the table is closed; the catch-all is last and always present, by the elaborator's rule. A tag-specific arm then binds fields by index from `fields`, which the declaration fixed, so there is no name lookup at runtime. The first matching arm is taken, as the language says; a tag that matches no specific arm and no namespace reaches the catch-all, which is where a forwarding interceptor forwards.

Why arms test in order rather than by a hash of the tag: a `match` with two tag arms and a catch-all is the common case, three comparisons, and a hash table per `match` would cost more to build than it saves. The `Choice` dispatch is a table because a closed choice's alternatives are dense indices; tags are dense too, and a `match` with many tag arms can use the same table, which is an optimisation the closure compiler may apply when the arm count passes a threshold and that changes nothing observable.

### `Ns.*`

A namespace wildcard compiles to a prefix test on the interned tag's namespace path. It appears only in patterns, so there is no value of "a namespace" at runtime and no `Ui` that could be passed around; the test is compiled into the `match` that wrote it. Nested namespaces match by prefix: `Ui.*` matches a hypothetical `Ui.Text.Input` because `["Ui"]` is a prefix of `["Ui", "Text"]`.

### `Kind`

`Kind e` is the prelude function that answers which parts `e`'s tag has, as a record of three booleans, and compiles to a leaf that reads `tags[e.tag]`. It exists so that generic code with a `Slot` whose tag it has not matched can decide, without a `match` arm per tag, whether to read the state dynamic `@Element` gave it; a sandbox that denies every request but passes every state element through is written with `Kind` and one `if`. The tag table is known to the compiler, so the function is a lookup, and it is the one place a program can ask a value about its type, which is allowed because a tag is data, not a type.

### `intercept`

`intercept @child, !responses, $supplier` compiles to `Step::Intercept`, which binds an `InterceptBlueprint` holding the compiled child (an `IrBlueprint` value from the environment), the responses event handle and the supplier dynamic handle, and writes the pair `((child's result, child's element list))` into the environment, with the list as a `$(Path ==> Slot)`. The supplier is a `$(fn Element -> @$State)`: a dynamic whose value is a closure record; the runtime, when a state slot is created under this interceptor, reads the closure record from the dynamic, applies it to the element through the program's call entry point, and gets back an `IrBlueprint` (the `@$State` is a blueprint value), which it builds as a child of the interceptor's node. The compiler's part of that is to make sure a supplier lambda compiles to a `Leaf` that returns a blueprint value, which the lint's "leaves are pure" rule allows because returning an unbuilt widget is not building.

The tag-indexed refinement of `Slot` fields and supplier results is a checker rule and has no runtime structure: a `Slot` at runtime is a record of an `ElementValue` and an event handle, whatever arm it is read in.

### Erasure

Generic code is compiled once and runs on values whose types it does not know. A function `Swap<a, b>` moves two one-word values; `Max<a: Ord>` compares two values with the dictionary it was handed; `Fold` applies a closure record. No type descriptor exists at runtime, no monomorphisation pass instantiates `Swap` per use, and no runtime type check happens anywhere in compiled code except at the two boundaries the specification names: a response or state value arriving at a generic slot, checked against the tag's declared type, and a value arriving from a World, checked by the runtime's run call.

The rule that makes erasure sound is the one on tag fields: a tag's fields have concrete types, so a value built in generic code never needs a type it does not have. Suppose that rule were dropped and `tag Test.Box of Element: value: a` were allowed. Then `let Wrap: fn a -> Element = \Test.Box(value=_)` would build an element whose field type depends on `a`, and a `match` arm `Test.Box(value=v)` elsewhere would bind `v` at a type the arm cannot know, and the only ways out are a runtime type descriptor on every value (which is a second type checker in the runtime), descriptors reified where a generic value enters a record (which makes `Wrap` take a hidden argument every caller must supply), or whole-program monomorphisation (which forbids separate compilation and polymorphic recursion). Passing descriptors generally is the hypothetical; the concrete-field rule is the design, and what it buys is that a value is one word and a type is a compile-time thing everywhere except the two boundaries.

The three traits are the one case where generic code needs something at runtime, and it gets a function value, not a descriptor: `Dict { Ord, a }` is a closure record passed as a hidden first argument, resolved at the call site where `a` is concrete. A caller at `Text` passes the text ordering; the callee calls it. Nothing inspects a value; the value's static type at the call site chose the function.

What erasure costs: a generic library cannot wrap its own type parameter in an element, and must take elements already built or be written at a concrete type. The diagnostic names the field and the variable, and the fix is to build the element where the type is known, which is where the World will need it to be known anyway.

## Diagnostics

A language whose syntax leans on spacing, sigils and inference is a language whose errors are part of its design. The mistakes a newcomer makes are predictable, and each one has a right answer the compiler can name. The language chapter fixes the content of each message; this section is the compiler's side: where each is detected, what it needs to have kept to say it well, and the exact rendering, which the negative goldens pin byte for byte. The rendering is one format for every diagnostic:

```
pub struct Diagnostic {
    pub level: Level,                 // Error | Warning
    pub message: String,              // one line, lowercase start, no period
    pub primary: (Span, String),      // the span under the caret, with its label
    pub secondary: Vec<(Span, String)>,
    pub notes: Vec<String>,           // the `=` lines: what the rule is and what the fix is
}
```

The test for every diagnostic is that the message contains the fix. A message that says what is wrong and not what to write is a message the programmer has to look up, and the ones below are the ones that are looked up most.

**Two events in one operation.** Detected in elaboration's lifting table when an operation or application has two operands whose outermost kind is `!`. Needs: the span of each operand and of the operator, kept by the parser on `Binary` and `Call`.

```
error: two events in one operation
  --> counter.prism:4:17
   |
 4 |     let !delta = !increment + !decrement
   |                  ^^^^^^^^^^   ^^^^^^^^^^ both are events
   = an operation may lift over any number of dynamics and at most one event
   = to fire on either, merge them: `!increment | !decrement`
   = to fire on both in one occurrence, hold one and sample it at the other
```

**A cycle with no delayed edge.** Detected by the causality check over a group. Needs: every binding in the cycle in order, the span of each, and the edge that made it prompt.

```
error: these bindings depend on each other in the same occurrence
  --> loop.prism:3:5
   |
 3 |     let $a = $b + 1
   |         ^^ depends on $b
 4 |     let $b = $a * 2
   |         ^^ depends on $a
   = a cycle in a widget block must pass through a hold's value, a tag, or a !Switch
   = a `rebuild` is not a delay: its event fires in the occurrence of the rebuild
```

A cycle through `rebuild` alone gets the same message with the last note as its reason, and a cycle through a `for` whose row fires at birth adds: "`!deletes` fires when a key appears, because its row's result is `Delete on !started`; a row that fires at birth is a prompt path from the collection".

**A negative number where a subtraction was meant.** Detected in elaboration when a `Call` whose head is a plain value fails to unify with a function type and the first argument is a prefix `-` literal. Needs: the parser's `prefix` flag preserved on the literal's sign and the argument's span.

```
error: `a` is not a function
  --> math.prism:7:12
   |
 7 |     let n = a -1
   |             ^ ^^ applied to -1
   = did you mean `a - 1`?
```

**A numeric operator at the wrong type.** There is no "cannot decide the numeric type" error: an integer-spelled literal that nothing pins is an `Int` at the end of its item, so `$n <- 0 then $n + 1 on !tick` is an `Int` counter, and a decimal-spelled literal is a `Float` from the start. What remains is the wrong suffixed operator: `+~` on two `Float`s is "operator `+~` is Int addition, but both operands are Float; use `+.` or `+`"; a bare literal in `Int` arithmetic where the other operand is pinned `Int` resolves silently (the literal unifies), and the message appears only when the literal is a `Nat` by annotation: "`+` on a Nat and an Int; write `+?` to convert, or make the literal an Int: `+1`".

**A sigil that disagrees with the inferred kind.** Detected when a binding's inferred type is compared with its written sigils.

```
error: `count` is bound to a dynamic, but its name has no sigil
  --> counter.prism:5:5
   |
 5 |     let count = $total + 1
   |         ^^^^^ a dynamic must be named with `$`
   = write `$count`
```

The reverse, `let $x = 3`, says "a plain value must be named without a sigil, or lifted: `$ 3`"; a name

with the wrong sigil says which it is and which it should be.

**A state tag with no supplier.** Detected statically when an `intercept`'s supplier argument is a visible lambda, `$ \|e\| match e: ...`, and a child under it creates an element of a state tag that no arm of the lambda handles and the catch-all does not forward with `@Element e`. Needs: the supplier lambda's arms and the set of state tags the child's blocks construct, which the elaborator records per widget function. The runtime reports the same thing at slot creation when the supplier is not visible.

```
error: no supplier for the state element `Fs.Contents` under this interceptor
  --> sandbox.prism:9:30
   |
 9 |     ((r, $slots)) <- intercept @child, !down, $ \|e| match e:
   |                                                  ^^^^^^^^^ handles Clock and denies the rest with `@
   $ Unavailable`
12 |             ignore: @ $ Unavailable
   |                     ^^^^^^^^^^^^^^^^ the catch-all returns `Unavailable`, which is not a `Contents`
   = the child builds `Fs.Contents` at build.prism:4:5; supply it, or forward it: `ignore: @Element e`
```

**A match on an extensible variant without a catch-all**, **a non-exhaustive match on a closed choice**, **a match on a dynamic with a blueprint arm**, **a hold outside a widget block**, **lifting across a sigil boundary**, **an optional where a plain value is required at a boundary**, **a type variable in a tag's field**, **interpolating something without a rendering**, **the layout errors**, **reserved words**, `for` **over a dynamic outside a widget block**, `ix` **outside a** `for`, **an** `if` **without** `else`, **a guarded arm that leaves a case uncovered**, **a** `..` **that is not last in an array pattern**, **a camelCase value name** (the one warning in the list, rendered with `warning:` and the suggested spelling): each renders in the format above with the content the language chapter gives, and each has a negative golden. Two more the compiler adds because its own stages make them possible:

**A trailing lambda that was not last.**

```
error: a trailing lambda takes the rest of the line
  --> fold.prism:3:22
   |
 3 |     let r = Fold xs, \|a, x| a + x, 0
   |                      ^^^^^^^^^^^^^^^^ the lambda's body includes `, 0`
   = parenthesise the lambda to pass more arguments after it: `Fold xs, (\|a, x| a + x), 0`
```

**A capture that would point forward** is a lint error, not a user diagnostic, because the resolver's cut makes it impossible from source; it renders as "internal: capture of a later binding" with the binding and the span, and a program that produces it is a compiler bug to be reported with the dump attached.

Every diagnostic is rendered from `Diagnostic` by one function, so that the format cannot vary between stages, and the negative goldens compare the rendered text, so that a change to the renderer is a diff in every negative golden at once, which is the right size for a change to how every error looks.

## Testing the compiler

Six kinds of test cover the crate, and every one of them is a text file someone can read in a diff.

## Goldens

Each golden is a Prism file and its typed-AST dump after elaboration, and after conversion with `--converted`. Any change to elaboration shows up as a diff a reviewer reads. The set: the counter; each program of the conformance corpus; the GUI library; the worked examples of the first chapter; and one small file per rule of the lexical and expression chapters, including the paren-less call cases (`F a, G b, c`, `F a, (G b, c)`, `G(b, c)`), tuples and arrays (`((G b, c))`, `(((G b, c), d))`, `((Size $elems) + 1)`, `[[]]`), function types (`fn (fn A -> B), [A] -> [B]`, `fn ((A, B)) -> C`), a three-step chain on continuation lines, the boolean levels (`not a == b`, `a < b and c < d`, `x or y and z`), a guarded arm and an exhaustiveness failure caused by one, `for` with named binders and with the shorthand, array patterns of each form, `@HoldFirst` written in a program, `;` in an inline `record:` and an inline `if`, the lambda shorthand (`\.done`, `\[0]`), the spacing cases, the dynamic-of-blueprints lift in both forms, `for` in each consolidation, a cyclic group cut at a switch, an interceptor with a tag-specific arm and a catch-all, a generic function with a bound, and a `test` item. What they catch, by name: a lift that changed kind (a dynamic read promptly where it should go through a tag); a literal or operator that resolved to a different numeric type; a binding that moved between `[build]` and `[occ]`; a group split or cut differently; a `for` consolidating to the wrong kind; a capture list that gained or lost an entry; a `rebuild` inserted or not inserted at a widget application.

The layout corpus of the language chapter is the lexer's golden: a program in, its token stream out, a program's whole stream printed on one line in the conformance chapter's corpus notation, compared exactly, for both lexers.

## Negative goldens

Each is a rejected program and the exact rendered diagnostic, spans included. They catch errors that get worse silently, which nothing else notices: a message that lost its fix line, a span that moved to the wrong token, a diagnostic that stopped firing because a check moved to a stage that runs later. The set is every diagnostic above, with at least one program each, and for the four that need care from the start (two events, a rejected cycle, `a -1`, the numeric operators) several programs each, covering the variants: a cycle through `rebuild` alone, a cycle through a `for` with a birth-firing row, `+~` on floats, `+` on a `Nat` and an `Int`.

## Lint fixtures

Hand-edited converted trees in the text form that the lint must reject, one per check: a placeholder closed twice, a placeholder never closed, a sample before close (`rebuild` over a placeholder inside its group), a capture of a later binding, a capture the body does not use, a leaf containing a `bind`, a `[build]` binding whose root is a `map`, a node with a mismatched type, a `match` on a tag with no catch-all. Each fixture is parsed by the text form's parser, which is why the parser exists for hand-written input, linted, and the lint's message compared. A checker nobody has seen fail is a checker nobody knows is on.

## Round trip

Print every golden's tree, parse the printed text, compare the trees; parse every golden file, print, compare the text. Run on every fixture and every golden in CI. A text form whose printer and parser drift is a bridge the Haskell frontend cannot cross.

### End to end: the counter against the hand-written counter

The test where the frontend and the runtime meet. The runtime chapter has the counter written in Rust against the builder by hand, driven by the harness through three turns (build; click Increment; click Decrement), with its transcript. The compiled counter is built from `counter.prism` through every stage of this chapter into an `IrBlueprint`, handed to the same harness, driven through the same three turns, and its transcript must be byte-identical: the same element list with the same paths (`0/#0/0: Ui.Button(label="Increment")`, `1/#0/0`, `2/#0/0: Ui.Label(text="Count: 0")` after build; `2/#1/0: Ui.Label(text="Count: 1")` after the first click), the same request firings (none), the same results. If the paths differ, the compiler's node structure is not the canonical one; if the label's text differs, a lift or a literal is wrong; if the transcript has an extra turn, something fired at build that should not have. One comparison proves that the two tracks were built against the same reading of the design, and it is the milestone that ends the compiler's first phase.

After the counter, the same harness runs the compiled `@Forward` from the GUI library wrapped around the counter against the hand-written `Forward` fixture, which exercises `intercept`, `for`, `rebuild`, generic `@Element` and the supplier; and then the twelve programs of the conformance corpus under the Test World with their scripts, which is the conformance chapter's test and not this one's, but is the first time most of the compiler's node kinds are exercised with real inputs.

The unit tests the closure compiler keeps beside the transcript tests pin the runtime facts compiled code depends on, each named for the sentence it checks: `a_compiled_hold_reads_its_own_old_value_through_a_tag`, `a_compiled_dyn_fires_its_event_in_the_build_occurrence_and_every_rebuild`, `a_compiled_switch_is_non_prompt`, `a_compiled_closure_captures_by_environment_index_and_not_by_rc` (a widget whose occurrence-time closure reads a sibling dynamic; drop the node; a `Weak` to the dynamic's node fails to upgrade), `compiled_and_hand_written_widgets_sit_in_one_tree` (a Rust `Button` bound inside a compiled widget and the reverse), `a_compiled_for_consolidates_a_birth_firing_promptly`, `the_depth_guard_aborts_a_runaway_recursion_with_its_span`.

### Differential, against the Haskell frontend

The Haskell frontend dumps every golden's program in the same text form, and the dumps are compared byte for byte. A difference is a sentence of the specification the two authors read differently, and the fix is a sentence in this book and then both frontends. The choices this chapter states because the differential test exposes them: where generalisation happens, how type variables are named, that cyclic groups are monomorphic, which edges are delayed, where placeholders are cut, how the dynamic-of-blueprints lift is spelled in each of its forms, and that `Num.Add[Int]` is how a resolved operator prints.

## Deep in: a flat register listing as an alternative lowering

A native backend, if one is ever wanted, would not want a tree. Cranelift's input is SSA with block parameters (from memory); emitted Rust is made of expressions, but a code generator that is also doing register allocation wants its values named and its construction order flat. The lowering such a backend would take from the converted typed AST is worth writing down once, so that its relation to the closure compiler is clear and so that nobody mistakes it for a stage the compiler needs now.

The lowering names every intermediate value in a typed register, flattens each widget block into the build order the groups already give, keeps `match` structured, and keeps closure records as the converter produced them. The counter's knot in that form:

```
%count: $Int  = cyc.rebuild "count @15:5"
%next:  !Int  = ev.tag_map_n %delta, [%count], @add_int
%held:  $Int  = rebuild.hold 0, %next
cyc.close %count, %held
%label: $Text = rebuild.map [%count], @fmt_count
```

Open, use, hold, close, then read. Every line is one runtime call, and the whole widget block is a list of such lines, one per step of `CompiledBlock.steps`. Leaves become functions in the same syntax:

```
fn @add_int(d: Int, c: Int) -> Int:
  %r: Int = int.add c, d
  ret %r
```

Read the two listings against the dump in the text form section: every line of the listing is a line of the dump with the nesting removed and a register name added. The listing adds no information, because the dump already has the lifts explicit, the captures explicit, the placeholders placed and the order fixed; it adds a syntax, and a syntax is a printer, a parser, a fixture set, a verifier and a golden set that every new node kind has to be added to after it has been added to the typed AST. For a backend that emits machine code, the syntax pays, because the backend's own input is flat and typed registers are what its allocator wants. For an evaluator that compiles to closures, the syntax is a second form saying the same thing flat, with its own verifier to keep it honest against the first, and the lint already does that job on the first.

The listing is readable as a work order, can be hand-written as a fixture before any parser exists, is checkable in one pass, and is a short step from an SSA backend. The closure compiler keeps the first three on the typed AST's text form (readable, hand-writable, linted) and gives up the fourth until a backend wants it. When one does, the lowering is a function from the converted typed AST to the listing, the listing's verifier is the lint restated over registers, and the differential test between the two backends is the transcript comparison the Test World already does: the counter compiled through the closure compiler and the counter compiled through the listing must produce one transcript. A bytecode, likewise, is a packed encoding of this listing with a version and a loader, wanted the day compiled code has to outlive the process that compiled it, and not before.

The one thing the listing would change in the compiler proper is nothing: it consumes the converted typed AST, and the converted typed AST is what the closure compiler consumes. That is why the line between elaboration and everything below it sits where it does.

## What is deferred

Each of these is absent on purpose, with what its absence buys now and what adding it would touch.

*Register code and native backends.* A consumer of the converted typed AST; the typed AST does not change.

*`read` and `update` blocks, `&` references, lenses.* The value layer has one operation on a record, construction, so the question of mutation inside a plain value never arises in the compiler: no aliasing analysis, no borrow rules, no in-place update in the leaves. When they come, they come as sugar over construction with an out-of-range write aborting the occurrence, and the leaves gain a few operations.

*User-defined traits.* The constraint set has three symbols and the solver is a table; a fourth symbol is a row. A general trait system would add instance declarations, instance search and coherence, and dictionaries per instance in generic code. The three traits are enough for sorting, map keys, element identity and arithmetic.

*Modules beyond one file plus the prelude.* One tag table, one program table, ids dense in declaration

order. Separate compilation needs stable ids across files and a module scope in the resolver; the compiled form does not change.

*Incremental compilation.* The frontend compiles a program in one pass and the goldens are whole programs. A program large enough to need an incremental frontend is a program with modules, and that line comes first.

*Optimisation.* No inlining, no constant folding beyond what the Rust compiler does inside a leaf, no fusion of adjacent maps into one node, no dead-binding elimination. Per-occurrence cost sits at the runtime boundary where values cross into `Value`, and no evaluator avoids that; a pass that fused `map (map !e f) g` into one node would change the graph a reader sees in the dump, and until a profile says the boundary is cheap and the extra node is not, the dump should be the graph.

*A packed encoding of compiled code.* When something reads compiled code it did not compile: pre-compiled libraries, cached builds, a browser bundle without the frontend, a World receiving programs. A serialisation of the typed AST with a version and a load-time check, decoded into the tree the closure compiler takes; one more printer and parser from the same definition.

*Debug stepping through leaves.* A tree of closures shows nothing in a debugger, and that is a real loss against an interpreter with an instruction pointer. What the closure compiler offers instead is a span on every node and every closure, and the errors a Prism programmer meets (a cycle, a height violation, an unknown path) fire inside the runtime's propagation with no evaluator frame on the stack in either design. A tracing mode that logs each leaf's entry with its span is a flag on the `Frame`, and is where to start if stepping is wanted.

None of these changes the typed AST, which is the point of putting the line there: the frontend can be finished, frozen and matched against a second reading while every one of them is still absent, and each one, when it comes, is a consumer of the tree or a row in a table.

## Native backends

A Prism program ships today as source plus the frontend, and runs through the closure compiler: a `Program { functions, blocks, tags, constants }`, a table of compiled blocks and leaf functions, and an `IrBlueprint { program, block, captures, span }` per widget whose `build` runs its block's steps in order against a builder, each step calling a runtime constructor and writing into an environment vector. That is the right first implementation because it has one intermediate form, the typed AST, and no second syntax. It stops being the right one in two situations. A World has to ship a program without a frontend in it: a browser bundle that should not include a parser, a device that cannot afford one, or a World that receives programs it did not compile and wants to check them before building. Or the closure compiler is measured to be the cost: a leaf that runs per occurrence through a tree walk and an environment lookup, when it should be three machine instructions. This item is what a native backend is, lowered from what, and what the differential test gives once there are two ways to run the same program.

### Register code as a lowering

The typed AST after closure conversion says what every binding means, with every lift explicit, every lambda a function id plus a capture list, every cyclic group with its placeholder and close nodes placed, and every binding marked build-time or occurrence-time. That is the one form the lint checks and the goldens diff. A native backend starts from it and nowhere else.

The tree does not say one thing flat, and a backend needs it: order. A widget function is a work order to the runtime, bind this, hold that, open a placeholder, close it, and the order is meaning. Register code is the tree with the nesting taken out of widget bodies: per function, a flat list of steps, every intermediate value in one named, typed register, builder calls in the order the builder sees them, and `match` kept structured with arms as blocks the way WebAssembly keeps control flow structured. It is a flattening of a compiled block's step list, not of a closure tree: the closure compiler's `CompiledBlock` already has its steps in group order over an environment indexed by position, and register code gives each environment position a name and a type and each leaf a signature over machine values. The counter's knot, the part of its work order where order is everything:

```
%count: $Int  = cyc.rebuild "count"
%next:  !Int  = ev.tag_map %delta, %count, @add_int
%held:  $Int  = rebuild.hold 0, %next
cyc.close %count, %held
%label: $Text = rebuild.map %count, @fmt_count
```

Open, use, hold, close, then read. The tag reads the placeholder's old value, which is what keeps the hold from feeding on itself, and the listing shows it. Each register is named once and has one type, which is an SSA value; a widget function is a basic block; a leaf is a small function over typed registers that keeps `Int`, `Nat` and `Float` as plain machine values until they cross into the runtime as `Value`.

The instruction families are the runtime's combinators one for one: `bind` on a native or compiled blueprint; `ev.map`, `ev.tag_map`, `ev.leftmost`; `rebuild.hold`, `rebuild.map`, `rebuild.map2`, `rebuild.join`; a switch over an event of events; `cyc.rebuild`, `cyc.event`, `cyc.close`; slot creation, the keyed list, `rebuild` and `intercept` as natives by table index; `rec.make` and `rec.match` for elements, with layouts from the tag table. A text form prints and parses from the same instruction enum, so a person can write a fixture before the lowering exists and a golden can diff the lowering's output. A verifier runs after every lowering: register types agree, every open has one close, nothing is sampled while open, every capture names a free register. It is the lint of the closure compiler applied to the flat form, and it is what makes the lowering mechanical in the only sense that matters, that a mistake in it is caught before anything is built.

Lowering is where the closure compiler and the native backend part ways and it is deliberately late. Everything upstream, elaboration, closure conversion, the lint, the typed-AST text form, is shared. A bug found by the differential test is therefore in the lowering or the backend, never in the meaning, because both executors got the same typed AST.

## Cranelift

Cranelift is the native option for a desktop or server World that wants machine code without a C toolchain. Its IR is SSA with block parameters in place of phi nodes (from memory), which is a short step from register code: each register is a Cranelift value, a widget function is one block of calls, and a structured `match` becomes a conditional branch or a jump table on the tag with one block per arm and a join block taking the arm's result as a parameter. The frontend builder crate does the SSA construction (from memory); the emitter never writes a phi.

Every runtime call goes through a shim, one `extern` function per combinator, instantiated at `Value`:

```
pub type Leaf2 = unsafe extern "C-unwind" fn(
    env: *const Value, a: *const Value, b: *const Value, out: *mut Value,
) -> bool;

#[unsafe(no_mangle)]
```

```
pub unsafe extern "C-unwind" fn prism_event_tag_map(
    e: *const Event<Value>, d: *const Dynamic<Value>,
    f: Leaf2, env: *const Rc<[Value]>,
) -> *mut Event<Value> {
    // clone env; call e.tag_map with a closure that calls f
}
```

(The `"C-unwind"` ABI string and the `#[unsafe(no_mangle)]` attribute spelling are from memory; whichever spellings the toolchain in use takes, the requirement is the one below: a Rust panic must be able to cross the emitted frame.)

The closure the shim builds holds a function pointer and a captures array, which is the closure record the typed AST already produces with the function id turned into an address. A compiled blueprint is a `Blueprint` whose `build` calls the emitted build function and whose output is `Value`, so it sits inside a `rebuild` beside closure-compiled and native blueprints and the runtime cannot tell them apart.

The contract on emitted code is two rules, and the shim is the only place they are checked: emitted code borrows values through pointers and never owns one, so reference counts change only in Rust; and it initialises `out` exactly when it returns true. Emitted frames therefore own nothing and need unwind tables but no landing pads. A panic inside a leaf, an `Int` overflow, raises in Rust, unwinds through the emitted frame (which the unwinding ABI permits and the plain C ABI does not, from memory), and is caught by the runtime's `catch_unwind` like any other, which is what keeps the occurrence abort (the errors chapter) working unchanged under a native backend. Build-time handles that emitted code receives belong in an arena the Rust adapter owns for the length of `build`, so a panic mid-build drops them on the Rust side.

Cranelift has no wasm backend (from memory; its targets are the common native architectures). A browser World runs the closure compiler, which it would anyway because a playground that compiles Prism in the page needs the frontend there. Cranelift also offers a JIT, which is what a REPL or the hot reload item later in this chapter would use if the closure compiler's build time were ever the thing in the way; it is not expected to be.

## Emitted Rust

The other native option is to print the register code as Rust: each register a `let`, each structured `match` a Rust `match`, each builder call the same call the closure compiler makes, and each widget function a `build` method on a generated type implementing the runtime's blueprint trait. The output compiles with the runtime into one binary, and `rustc` inlines across the boundary, so a leaf that adds two integers is an add with an overflow check and nothing else. This is the backend for shipping: one `cargo build` produces a World with its program inside it and no frontend.

`rustc` checks the emitter's Rust and nothing else: that the generated code type-checks against the runtime's API. It does not check Prism's meaning. An emitter that writes `wrapping_add` where Prism means checked addition, or swaps the operands of a leftmost merge, produces Rust that compiles and a program that is wrong, and nothing in the Rust toolchain can say so. That is why emitted Rust is a differential backend and not an oracle: its correctness comes from running the suite and matching the closure compiler's transcripts, the same as Cranelift's.

## Other targets

LLVM is one possible target, and the only one that would reach every World including the browser, since the runtime's own wasm build goes through LLVM (from memory, by way of `rustc`). The form

it would take is clear: the register code prints almost directly as LLVM IR, with each widget function a list of calls into the same unwinding shim and each leaf a function over `i64` and `double`. The pin on an LLVM version, the toolchain in the edit loop, and a second unsafe surface are costs that nothing yet justifies, and a program that has to be native on the desktop and also run in a browser is served by Cranelift for one and the closure compiler for the other. If a native backend is ever built, it starts from the register code and picks its target then; the choice of target is the last decision, not the first.

Bytecode is a different question and a smaller one. If compiled code ever has to outlive the process that compiled it, cached builds, precompiled libraries, a World that loads programs, the format is a serialization of the converted typed AST with a version and a load-time lint, decoded back into the tree the closure compiler already takes. Nothing has to be designed for that beyond a version field.

### What the differential test gives

The conformance suite runs each program under the Test World and records its transcript. With two executors of the same typed AST, every program in the suite runs under both and the transcripts are diffed, exactly as they are between the Rust and Haskell implementations. This is the reason a second backend is cheap to trust.

A mismatch between the closure compiler and a native backend is in the lowering, the emitter or the shim, because both executors share everything above the typed AST. A mismatch between the Rust implementation (either executor) and the Haskell one is in a frontend or in the semantics as one of them understood it. Those two diffs localise a bug to one of three places before anyone reads a line of code. And the suite's programs are the specification's programs, so a native backend that passes it has been checked against the laws, not against the closure compiler's accidents. The register code's text form adds a third, cheaper check: goldens on the lowering's output, compared through a normalising printer so that renamed registers do not show as diffs, catch a lowering change as a diff someone reads before any backend has run it.

### What it must not change, and what triggers it

The typed AST is the form everything is checked against, and no backend may need something from the frontend that the typed AST does not say. Transcripts are unchanged. The occurrence abort still works, which is the unwinding requirement on every shim. The runtime's API does not grow a second entry point for native code; the shim is a client of the API the closure compiler uses.

The trigger is one of the two situations the section opened with: a World that has to ship without the frontend, or a profile under a real World where the closure compiler's leaves are the cost after the value representation has been fixed. The first is the likelier, and it points at emitted Rust. Until one of them arrives, register code is a stage with no consumer, and a stage with no consumer is a form that drifts.

## Incremental compilation

You change a label in one widget and wait for the whole program to be re-elaborated. For the suite's programs that wait is unmeasurable. For a program the size of a real application with a GUI library under it, it is the edit loop, and the edit loop is the thing a language is judged on by the people who use it every day. Incremental compilation is making the frontend redo only what the edit touched. It is deferred because the typed AST was designed so that it needs nothing new, and the argument that this is true is the item.

## The unit

The typed AST's text form is one binding per line: name, sigil, type, mark, group, expression with every lift explicit, span. A top-level definition elaborates to a set of such lines and depends on the types of the top-level names it uses and nothing else about them. That makes the top-level definition the unit of incremental work, and its interface the types of its free top-level names.

The pipeline per definition is lex, parse, resolve, elaborate, convert, lint, compile. The first three are per file and cheap. Elaboration is per definition given the types of its dependencies, because Hindley–Milner on the value layer needs the type of each free name and the sigil of each is in the name. Conversion and the lint are per definition. The closure compiler produces one `CompiledBlock` per widget block and one `CompiledFunction` per leaf in the `Program`'s tables, indexed by position, which is per definition too once the tables are appended per definition and the indices are stable across an edit (a recompiled definition reuses its old slots and a new one appends). So the dependency graph of the incremental build is the dependency graph of top-level names, and it is already computed by the resolver for the group analysis.

## The sketch

A query-based build (the `salsa` crate is the implementation choice; that it and `rustc`'s own build work this way is from memory): every stage is a function from inputs to output, memoised by a fingerprint of its inputs, and an edit invalidates exactly the queries whose inputs changed.

- `parse(file)` depends on the file's text.
- `resolve(file)` depends on `parse(file)` and the export lists of imported modules.
- `signature(name)` depends on the definition's annotation, which is why top-level annotations are required: the signature of a definition is readable without elaborating its body, so an edit to a body never invalidates its dependents.
- `elaborate(name)` depends on `parse`, `resolve`, and `signature` of every free name.
- `convert(name)`, `lint(name)`, `compile(name)` depend on the stage before.

An edit to a widget's body re-elaborates that definition and recompiles its blocks and leaves in place, and nothing else, because its signature did not change. An edit to a signature re-elaborates its dependents, which the resolver's graph names. The typed-AST text form is the cache: each definition's elaborated lines are written beside the source with their fingerprint, so a cold start with an unchanged file reads the cache and skips to compile.

The cross-cutting checks are the part that needs care. Exhaustiveness and the cycle check are per definition, since both work on one block's graph. Trait coherence and tag uniqueness are per program, but they are checks over tables that are themselves incremental (each module contributes its instances and tags; the check is over the union), and a violation is reported at the import that caused it. Nothing in the frontend needs whole-program information that cannot be expressed as a union of per-module tables, which is the property that makes the module design above and the incremental design here the same design.

## What it must not change

The output: the typed AST of an incrementally built program is byte-identical to a clean build's, and the goldens check it, since the golden test is exactly "elaborate this program and diff the dump." A test that builds every suite program clean and then incrementally after a no-op edit, a body edit and a signature edit, and diffs the dumps, is the whole correctness test. Transcripts cannot change because the typed AST did not.

## The fact it waits on

An edit loop someone notices. Not before, because the memoisation adds a fingerprint to every stage and a cache to every build, and a frontend that is fast enough clean is simpler without them.

## Chapter 13

# Rust Worlds and the world core

A World is the only part of a Prism system with effects, and it is written in Rust against one synchronous call: plain data in, plain data out, once per occurrence. Everything a World must get right that is not specific to a screen, a terminal or a socket is the same for every World, so it is written once, in a crate called `prism_world`, and a World is that crate plus a handful of components. This chapter gives the crate's design as code: the component trait, the turn loop, the `Defer` bookkeeping, the snapshot and push machinery, slot tracking, `Unsupported`, error reporting, and the harness that lets any event loop drive a runtime. It then builds four Worlds on it, in increasing order of how much of the outside they touch: the Test World, which has no effects and answers to a script; the CLI World, which runs Prism against a real process; a GUI World that puts pixels in a window; and a World that offers nothing but sockets, under which a Prism library speaks X11 and the same GUI programs run anyway. The chapter ends with the tests: the core's conformance tests against a fake component, and the CLI World's golden transcripts.

## Where a World stands

You have had the year where the framework handed you a context object. You stashed it in a static, or threaded it through every constructor, and it worked, and then a test needed to run without a browser and could not, because half the program had learned to reach into that object whenever it wanted something. The context was a door, and once there is a door, everything walks through it.

Prism has no door. A program describes what it wants as elements, plain values in a list; the outside sends plain values back, addressed by path; and the only thing that ever happens between them is one function call. The World calls the runtime; the runtime never calls the World. The World holds no runtime object apart from the runtime itself, and the runtime holds nothing of the World's. This makes the one-way-ness of the language real at the edge: a program cannot reach the World because there is nothing to reach, and a World cannot reach into a program because the run call takes values and returns values.

The run call, as the Rust runtime exposes it, is the whole surface a World sees:

```
pub struct Inputs<'a> {
    /// Responses by path, for slots in the list the World last received.
    pub responses: Vec<(Path, Value)>,
    /// New values for state elements, keyed by element value.
    pub pushes: Vec<(ElementValue, Value)>,
    /// The oracle: the value a state element has as this occurrence opens.
    /// `None` means no component accepts the tag, which aborts the occurrence.
    pub oracle: &'a mut dyn FnMut(&ElementValue) -> Option<Value>,
}

pub struct Outcome {
    /// The element list after the occurrence, in tree order.
    pub list: Vec<(Path, ElementValue)>,
```

```
    /// For each slot with a `requests` field that fired: its firings, in order.
    /// Includes slots torn down in this occurrence, so the World can still act.
    pub requests: Vec<(Path, ElementValue, Vec<Value>)>,
    /// The program's result, when the World reads it.
    pub result: ResultView,
}

impl Runtime {
    pub fn run(&mut self, inputs: Inputs<'_>) -> Result<Outcome, RuntimeError>;
}
```

`Path` is opaque to a World except for equality and ordering; `ElementValue` is an interned tag plus plain fields with structural equality; `Value` is the runtime's value type restricted to plain data, which the compiler guarantees for anything that goes into an element. `ResultView` is the program's result in a form a Test World can compare: a plain value, or an event's firing in this occurrence, or a dynamic's current value. A CLI World never looks at it.

That is all. There is no `subscribe`, no `on_event`, no handle to a node. The first call is occurrence zero, with empty responses and pushes; it builds `@main` and returns the initial list. Everything a World does from then on is a function of what came out of the last call and what the outside has done since.

## What the crate refuses to contain

The shortest way to say what `prism_world` is for is to say what it leaves out, because each omission is a decision with a reason.

*No scheduler.* The crate never decides which of two things to do first inside the runtime; that order is fixed by heights and the runtime owns it. The crate orders only the things outside: inputs into occurrences, and request firings into effects.

*No threads of its own.* The core is single-threaded and blocks nowhere. It runs on whatever thread calls it. Worlds that need threads, for reading stdin or waiting on a socket, create them in their components, and those threads talk to the core through one inbox of plain values. Nothing on another thread ever sees the runtime.

*No callbacks into the runtime.* A component is called by the core; it never calls the runtime, never holds a `Runtime`, never holds a `Path` beyond the turn in which it was told the slot exists, except to address a delivery. The trait makes this structural: nothing in a component's signature mentions the runtime.

*No phases.* A turn is an input occurrence followed by whatever `Defer` occurrences it causes, and nothing else. A turn has no named stages for layout, focus and painting: they would be a second ordering mechanism with its own rules for who runs when, and everything they buy is expressible with `Defer` and ordinary dynamics. A GUI library that wants "layout, then focus" writes two `Defer`s in sequence.

*No budget.* A turn is bounded by the `Defer` rule, not by a counter. There is no runaway report to design, no partial-commit policy, no "yield to the platform and continue next frame," because the `Defer` rule bounds the turn: each `Defer` slot answers at most once per turn, so a turn runs at most one occurrence more than there are `Defer` slots. A `Defer` that a rebuild brings back counts as a new slot, which is the one gap in that argument, and the `Defer` section below works through it.

*No GUI.* The crate has no notion of a window, a draw list or a pointer. A GUI World is a World whose components happen to accept tags under `Gui`, exactly as a CLI World's accept tags under `Cli`.

The core does not know the difference.

What is left is a few hundred lines: a trait, a loop, three maps and a printer. That is the point. A World should be an afternoon's work, so that there will be several, and the way to make it an afternoon is to put every obligation of the specification in one place that every World gets for free.

# The `prism_world` crate

## The types

The core speaks in the runtime's plain-data types and adds a few of its own.

```
pub use prism_runtime::{Path, ElementValue, Value, Tag, RuntimeError, Span};

/// What a tag declares, from the compiled program's tag table.
#[derive(Clone, Copy, PartialEq, Eq)]
pub struct TagKind {
    pub requests: bool,
    pub responds: bool,
    pub state: bool,
}

/// Something that wants to enter the program at the start of an occurrence.
pub enum Delivery {
    /// A response addressed to a slot by path.
    Response { path: Path, value: Value },
    /// A new value for a state element, keyed by element value.
    Push { element: ElementValue, value: Value },
    /// No payload: a thread sends it only to wake the loop, and the harness drops it.
    Wake,
}

/// Which occurrence of which turn, for error reports and transcripts.
#[derive(Clone, Copy, PartialEq, Eq, PartialOrd, Ord)]
pub struct When {
    pub turn: u64,
    pub occurrence: u32,
}
```

`Wake` carries nothing into the program; a component's thread sends it into the inbox so that the loop wakes and asks `has_gatherable`.

The compiled program brings its tag table with it, so the core knows for any tag whether it requests, responds or has state. That table lets the core decide, for an element nobody accepts, which of the three `Unsupported` behaviours applies.

## The Component trait

A component is a thing that accepts some tags and does, in Rust, what those tags mean. Its contract with the core is eight methods, and each one follows from an obligation in the specification.

```
pub trait Component {
    /// Which tags this component accepts. Asked once, at registration.
    fn tags(&self) -> Vec<Tag>;

    /// A slot with one of this component's tags appeared in the list the World
    /// read after a turn. Streams start here; watches start here.
```

```
    fn slot_created(&mut self, path: &Path, element: &ElementValue, cx: &mut Cx);

    /// The slot is gone from the list the World read after a turn. Streams
    /// stop here; in-flight operations are cancelled where the OS allows.
    fn slot_removed(&mut self, path: &Path, element: &ElementValue, cx: &mut Cx);

    /// A slot fired its request. Called once per firing, in occurrence order
    /// and then list order, after the turn. The response, if the tag responds,
    /// goes through `cx` and reaches the program in a later turn.
    fn perform(&mut self, path: &Path, element: &ElementValue, request: Value, cx: &mut Cx);

    /// The value a state element has right now. Called at most once per
    /// element value per turn; the core memoises. Must answer: unavailability
    /// is in the tag's type, never in a failure here.
    fn oracle(&mut self, element: &ElementValue, cx: &mut Cx) -> Value;

    /// Called at the start of every turn. The component reports state changes
    /// since the last turn as pushes through `cx`, and may deliver anything
    /// else it has ready. A clock pushes here, once per turn.
    fn gather(&mut self, cx: &mut Cx);

    /// True when `gather` would produce something that should start a turn on
    /// its own: a watch that saw a change, a timer that expired, held stdin
    /// input. A clock answers false. The default is false.
    fn has_gatherable(&self) -> bool { false }

    /// True when nothing this component does can ever deliver again:
    /// no stream running, no operation in flight, no timer set.
    fn quiescent(&self) -> bool;
}
```

`has_gatherable` lets the core ask, before it opens a turn with an empty batch, whether any component has pending work for `gather` to collect, so a component whose input arrives from a thread (standard input, a file watch) can cause a turn without a delivery of its own.

`Cx` is the component's one way to say anything to the core, and it is write-only:

```
pub struct Cx {
    deliveries: Vec<Delivery>,           // for the next turn
    diagnostics: Vec<Diagnostic>,        // things to report, not errors
    exit: Option<i32>,                   // a request asked the run to end
    inbox: Inbox,                        // a clonable sender for other threads
}

impl Cx {
    pub fn respond(&mut self, path: &Path, value: Value);
    pub fn push(&mut self, element: &ElementValue, value: Value);
    pub fn diagnostic(&mut self, d: Diagnostic);
    pub fn exit(&mut self, code: i32);
    /// A sender that any thread can use to deliver later, plus a waker.
    pub fn inbox(&self) -> Inbox;
}

#[derive(Clone)]
pub struct Inbox {
    tx: std::sync::mpsc::Sender<Delivery>,
    wake: std::sync::Arc<dyn Wake>,
}

pub trait Wake: Send + Sync {
```

```
    /// Called from any thread after a delivery is queued. A CLI World's waker
    /// does nothing (the loop blocks on the channel); a winit World's calls
    /// `EventLoopProxy::send_event`; a tokio World's notifies a `Notify`.
    /// (winit and tokio API names from memory.)
    fn wake(&self);
}
```

Why a trait with these eight methods and not something else?

Suppose a component were an async trait: `async fn perform(...) -> Value`, and the core awaited it. Then the core would be an async function, and the turn would hold the runtime across an await. Every GUI main thread forbids that, and even where it is allowed it means the runtime's state is live while the World is doing something else, which is the door again by another name. The split here is that `perform` returns at once, having started the operation, and the result comes back through the inbox as a `Delivery` for a later turn. A component that wants async machinery, and the tokio World does, spawns a task and hands it the inbox. The core never waits.

Suppose instead the core were table-driven: a component registers, per tag, a closure for requests and a closure for the oracle, and the core calls closures. This is the same trait with the names erased and the state scattered. The trait wins because a component has state that spans the methods (the stdin reader's thread handle is created in `slot_created` and joined in `slot_removed`; the timer wheel is written by `perform` and read by `gather`), and a struct with methods is the Rust way to say that.

`oracle` most needs its contract spelled out. It is called by the core from inside the runtime's run call, because the runtime asks for a state element's value at the instant a slot is created, mid-build. So `oracle` must be synchronous and must not block for long. A file watch component answers `Fs.Contents(path)` by reading the file, which is a blocking syscall, and that is accepted: a read of a local file is the kind of blocking a GUI thread does anyway, and the alternative, answering `Unavailable` and pushing the contents next turn, makes every program's first occurrence see a value it will never see again. The rule a component follows: answer from what it has if it has it, read synchronously if the read is cheap (a local file under a size cap), and answer the type's unavailable case, with its reason, when the real answer cannot be had without waiting on something outside the machine or is past the cap.

## Registering components and the World struct

```
pub struct World<R: Runtime> {
    runtime: R,
    program: Program,                  // compiled widget plus tag table
    components: Vec<Box<dyn Component>>,
    by_tag: HashMap<Tag, usize>,       // which component accepts which tag
    snapshot: Snapshots,
    slots: SlotTable,
    defer: DeferTable,
    when: When,
    inbox_rx: std::sync::mpsc::Receiver<Delivery>,
    cx: Cx,
    report: Box<dyn Report>,           // where errors and diagnostics go
}

impl<R: Runtime> World<R> {
    pub fn new(program: Program, report: Box<dyn Report>, wake: Arc<dyn Wake>) -> Self;
    pub fn add(&mut self, component: impl Component + 'static) -> &mut Self;
    /// Occurrence zero. Returns the initial list.
```

```
    pub fn start(&mut self) -> Result<TurnOutcome, WorldError>;
    /// One turn. The inputs are whatever the caller has gathered.
    pub fn turn(&mut self, inbound: Vec<Delivery>) -> Result<TurnOutcome, WorldError>;
    /// True when no component can deliver again and the inbox is empty.
    pub fn quiescent(&self) -> bool;
    pub fn inbox(&self) -> Inbox;
}
```

Two components claiming one tag is an error at `add`, reported before the program runs. `Defer` is claimed by the core itself and a component that tries to accept it is refused for the same reason.

`TurnOutcome` is what a caller gets back from a turn:

```
pub struct TurnOutcome {
    pub when: When,                              // the turn number and its last occurrence
    pub occurrences: Vec<OccurrenceRecord>,      // one per occurrence of the turn
    pub list: Vec<(Path, ElementValue)>,         // after the last occurrence
    pub exit: Option<i32>,                       // a component asked to end the run
}

pub struct OccurrenceRecord {
    pub when: When,
    pub inputs: Vec<Delivery>,                   // what opened it
    pub requests: Vec<(Path, ElementValue, Vec<Value>)>,
    pub result: ResultView,
}
```

A CLI World looks at `exit` and nothing else. A Test World reads all of it. The records exist so that the transcript printer and the `requested`, `fired` and `errors` steps have a single source, the same one the differential test diffs.

## The turn loop as code

A turn is: take what has arrived, open an occurrence with it, and keep opening occurrences while `Defer` requests are outstanding; then act. Here it is, with the bookkeeping calls named and defined in the sections that follow.

```
impl<R: Runtime> World<R> {
    pub fn turn(&mut self, mut inbound: Vec<Delivery>) -> Result<TurnOutcome, WorldError> {
        self.when.turn += 1;
        self.when.occurrence = 0;
        self.defer.begin_turn();

        // 1. Components report what changed since last turn (clock, watches).
        for c in &mut self.components { c.gather(&mut self.cx); }
        inbound.extend(self.cx.take_deliveries());

        // 2. Sort inbound into this occurrence's responses and pushes.
        //    Responses to paths not in the last list are discarded with a
        //    diagnostic; two responses to one path split into two turns.
        let (mut responses, pushes, held) = self.slots.sort_inbound(inbound, &mut self.cx);
        self.hold_over(held);                          // becomes next turn's inbound
        let mut delivered = self.snapshot.deliverable(&pushes);   // only values someone has asked about

        // 3. The input occurrence, then Defer occurrences until none fire.
        //    An abort stops the loop; the occurrences before it have committed.
        let mut records = Vec::new();
        let mut failure = None;
```

```
        loop {
            self.when.occurrence += 1;
            let first = records.is_empty();
            let outcome = match self.run_occurrence(responses, std::mem::take(&mut delivered)) {
                Ok(outcome) => outcome,
                Err(e) => {
                    if first { self.requeue(&pushes); }       // the pushes never landed
                    failure = Some(e);
                    break;
                }
            };
            if first { self.snapshot.apply_pushes(&pushes); }   // committed: next = prev + pushes
            self.snapshot.swap();                               // prev = next
            records.push(OccurrenceRecord::from(&outcome, self.when));
            self.slots.last_list = outcome.list.clone();
            match self.defer.collect(&outcome, self.when) {
                Ok(deferred) if deferred.is_empty() => break,
                Ok(deferred) => responses = deferred,           // answer the Defers; no pushes
                Err(e) => { failure = Some(e); break; }         // deferring backwards
            }
        }

        // 4. Act on what committed: slot diff, then requests in occurrence order
        //    then list order. Then, if the turn stopped early, report why.
        let (created, removed) = self.slots.diff();
        for (path, el) in &removed { self.route(el).slot_removed(path, el, &mut self.cx); }
        for (path, el) in &created { self.route(el).slot_created(path, el, &mut self.cx); }
        for rec in &records {
            for (path, el, firings) in &rec.requests {
                for v in firings { self.perform(path, el, v.clone()); }
            }
        }
        self.report.diagnostics(self.cx.take_diagnostics());
        if let Some(e) = failure { return Err(e); }
        Ok(TurnOutcome { when: self.when, occurrences: records,
                         list: self.slots.last_list.clone(), exit: self.cx.take_exit() })
    }

    fn run_occurrence(&mut self, responses: Vec<(Path, Value)>, pushes: Vec<(ElementValue, Value)>)
        -> Result<Outcome, WorldError>
    {
        let World { runtime, snapshot, components, by_tag, cx, program, .. } = self;
        let mut oracle = |e: &ElementValue| snapshot.answer(e, |e| {
            let ix = *by_tag.get(&e.tag)?;
            Some(components[ix].oracle(e, cx))
        });
        runtime.run(Inputs { responses, pushes, oracle: &mut oracle })
            .map_err(|e| WorldError::Runtime { error: e, when: self.when })
    }
}
```

Slot removal is told to components before slot creation. A `rebuild` firing tears down a slot at one path and creates another at a fresh path in the same occurrence; if the component were told about the creation first, a stdin reader would briefly have two slots and start a second reader thread before stopping the first. Removals first means a component never sees two of a thing that should be one.

Requests are performed after the slot diff. A request fired by a slot that was torn down in the same occurrence is still in `requests`, with its path, and `perform` is called for it. The component does the operation if it can, because the program asked, and discards the response, because the slot is gone;

`slot_removed` was already called, so the component knows. This is the rule that a write fired in the occurrence that removed the writer still lands.

Pushes land in the first occurrence of a turn only. The `Defer` occurrences that follow are the same turn, and the snapshot did not change between them, so their oracle answers from what is now the previous snapshot and their push list is empty. A clock, which pushes once per turn, is therefore read at the same instant by every occurrence of the turn, which is what makes "a turn is one step" true for state as well as for effects.

The snapshot takes those pushes only after the first occurrence commits. Picture a turn that opens with a click and the clock's push `Clock ↦ Instant 5100`, and the click's occurrence aborts on a cycle. The runtime rolls back, and no `Clock` slot ever saw 5100. Had the snapshot already taken the push, the next turn would open with the snapshot at 5100 and no push pending, so a new `Clock` slot would start at 5100 while every old one still held 5000, with nothing on the way to bring them up. So `deliverable` only picks which pushes to send, `apply_pushes` runs after `run_occurrence` returns `Ok`, and an abort in the first occurrence hands every push back with `requeue`, as inbound for the next turn. The click's response is not re-queued: it was the input to an occurrence that did not happen, and sending it again would run the same cycle again. A push is a fact about the outside, and it is still true next turn.

An abort later in the turn is a different case, because the occurrences before it committed. Picture a turn whose first occurrence fires a `Cli.Stdout` write of `"saved\n"` and a `Defer`, and whose `Defer` occurrence hits a cycle. The write's occurrence happened: its holds committed, and the list the World now holds is the one it produced. If the loop returned the error with `?` from inside, the write would never be performed, and the program would go on believing it had said "saved" to a terminal that never heard it. So the loop collects each occurrence's record as it commits, stops at the abort, performs the firings of everything that committed in the usual order, and only then returns the error. Deferring backwards stops the turn the same way: the occurrence in which a `Defer` fired a second time did commit, so its firings are performed too, and only the answer that would have opened the next occurrence is withheld.

## Defer bookkeeping

`Defer` is the one element the core implements itself:

```
tag Defer of Element:
    requests: !unit
    responds: unit
```

A slot that fires `Defer` in occurrence k of a turn is answered in occurrence k+1 of the same turn. The core's job is to find the firings, deliver the answers, and enforce the one rule that bounds the turn: a slot is answered at most once per turn, and a slot that fires again after being answered is deferring backwards.

```
#[derive(Default)]
pub struct DeferTable {
    answered: BTreeSet<Path>,     // slots answered so far this turn
}

impl DeferTable {
    pub fn begin_turn(&mut self) { self.answered.clear(); }

    /// The responses for the next occurrence: `unit` to every Defer slot that
    /// fired in this one and is still in the list. An error if any of them
```

```
    /// was already answered this turn.
    pub fn collect(&mut self, outcome: &Outcome, when: When)
        -> Result<Vec<(Path, Value)>, WorldError>
    {
        let mut next = Vec::new();
        for (path, el, firings) in &outcome.requests {
            if el.tag != Tag::DEFER || firings.is_empty() { continue; }
            if !outcome.list.iter().any(|(p, _)| p == path) {
                continue;                                  // torn down with the slot: nothing to answer
            }
            if !self.answered.insert(path.clone()) {
                return Err(WorldError::DeferBackwards { path: path.clone(), when });
            }
            next.push((path.clone(), Value::UNIT));
        }
        Ok(next)
    }
}
```

The bound follows from `answered`: every occurrence after the first in a turn is opened by at least one newly answered `Defer` slot, and no slot enters `answered` twice, so a turn has at most one occurrence more than the program has `Defer` slots. There is no counter and no cap, and nothing to tune.

The error is the interesting half. Picture a sandbox that denies a request by firing a `Defer` and answering `Denied` when it comes back, and picture a child that, on `Denied`, immediately asks again. The child's second request fires in the occurrence that delivered the denial; the sandbox's `Defer` fires again in that same occurrence; and the core finds a path already in `answered`. That is deferring backwards, and it is reported, with the path and the occurrence, as an error that stops the turn. The alternative, letting the `Defer` answer again, would be a loop with no bound, and the program that wrote it would hang the World rather than see its mistake on the first turn. A program that wants to retry waits for the next turn, which means waiting for any real input, which is the correct meaning of "later" when the thing being retried was denied by policy.

A `Defer` that fires in the occurrence that tore down its slot is skipped, which is the general rule for request firings from gone slots applied to a request the core itself answers: nothing to answer, nothing to report. The sandbox of chapter 2 does this when its child is removed in the same occurrence that the child asked for something.

The creation rule says why `Defer` is a request element with a response and not, say, a state element the World toggles. A request fires in n and is answered in a later occurrence; that is exactly the gap an interceptor needs, and it is the gap every other request has, so `Defer` costs the language nothing new. The one thing special about it is that every World must provide it, because an interceptor cannot start an occurrence and there has to be some way to get one.

## A `Defer` reborn by a rebuild

The bound above counts slots, and a program can make new `Defer` slots faster than the turn uses them up. On the merits, this part of the core needs more work. Here is the smallest program that shows it:

```
@Pulse: fn Int -> @!unit = \|n| @widget:
    !s <- @started
    !later <- @Element Defer(requests=(unit on !s))
    @ !later
```

```
@main:
    $n <- 0 then $n + 1 on !later
    !later <- @Pulse $n
    @ unit
```

`@Pulse $n` lifts to a `rebuild` over `$n`. The build fires the `Defer` at `1/#0/1`. The next occurrence answers it; `!later` fires; `$n` becomes 1; the `rebuild` builds anew in that same occurrence, and the new `@Pulse` fires its own `Defer` at birth, at `1/#1/1`. That path is not in `answered`, so the core answers it in the next occurrence, which rebuilds again and fires at `1/#2/1`. Every slot is answered once, exactly as the rule asks, and the turn never ends. The program is wrong in the same way the retrying sandbox child is wrong (it answers "later" by asking for "later" again), but `DeferTable` keys by full path, and a full path includes the generation, so the core never sees the repeat.

The candidate fix is to key `answered` by the path with its generation segments stripped, so that a `Defer` reborn at the same tree position counts as the same `Defer`. `1/#0/1` and `1/#1/1` both become `1/#/1`, the second firing is deferring backwards, and the error names both paths. The change is a few lines in `collect`, and it keeps the bound without a counter.

What it might get wrong is a `Defer` that really is a different one. Picture a `rebuild` over the selected tab, where each tab's widget puts a `Defer` at the same bind position: tab A's `Defer` is answered in occurrence 2, its answer switches to tab B, and tab B's `Defer`, born in occurrence 2 for a layout pass of its own, fires at birth. Stripped, the two keys are equal, and a program that ends after one more occurrence is reported as deferring backwards. The fix trades a turn that never ends for an error on a program that was fine, and the tab case is ordinary GUI code. Requiring the element value to match as well as the stripped path would separate tab A's `Defer` from tab B's only if their fields differ, and a `Defer` has no fields.

A `for` key is the other edge. Stripping removes generations and leaves `[k]` segments alone, so a queue whose rows are keyed by a fresh id, where each row's answered `Defer` appends the next row and removes itself, escapes the bound the same way the `rebuild` did: `0/[1]/1`, `0/[2]/1`, and on. Stripping `[k]` too would close that and break something worse: ten rows that each defer once in one occurrence would share one key, and the second of them would be reported as deferring backwards in an occurrence where nothing went backwards.

Suppose instead the core capped a turn at some number of occurrences. That ends every one of these loops and is exactly the budget this crate refuses, with its runaway report and its question of what to do with the occurrences that did commit. The fix that belongs here identifies a reborn `Defer` the way a reader of the program would, by where it sits and what caused it, and neither full paths nor stripped paths capture that.

## Snapshots, the oracle and pushes

The problem the snapshot solves is a race you have met in any framework with subscriptions. A widget mounts, reads the current value of something, and subscribes to its changes; between the read and the subscribe the value changed, and the widget is now showing a value it will never be told is stale. The language's answer is that the read and the subscription are one thing, a dynamic, and the World's answer is that a dynamic's value as an occurrence opens and the changes delivered in that occurrence come from one consistent pair of snapshots.

```
pub struct Snapshots {
    prev: BTreeMap<ElementValue, Value>,   // what the program saw as the occurrence opened
    next: BTreeMap<ElementValue, Value>,   // what it will have seen when it closes
```

```
}

impl Snapshots {
    /// The pushes the turn's first occurrence delivers. A push for a value nobody
    /// has asked about yet is not delivered: there is no slot to deliver it to,
    /// and the first slot to ask will get it from the oracle.
    pub fn deliverable(&self, pushes: &[(ElementValue, Value)]) -> Vec<(ElementValue, Value)> {
        pushes.iter().filter(|(e, _)| self.prev.contains_key(e)).cloned().collect()
    }

    /// Called once the turn's first occurrence has committed, and not before:
    /// every push, delivered or not, is recorded as the value.
    pub fn apply_pushes(&mut self, pushes: &[(ElementValue, Value)]) {
        for (e, v) in pushes { self.next.insert(e.clone(), v.clone()); }
    }

    /// The oracle. Memoised: one answer per element value per occurrence.
    /// For a value never seen, the component is asked, and the answer is recorded
    /// in both snapshots so that no push for it is pending.
    pub fn answer(&mut self, e: &ElementValue,
                  ask: impl FnOnce(&ElementValue) -> Option<Value>) -> Option<Value>
    {
        if let Some(v) = self.prev.get(e) { return Some(v.clone()); }
        let v = ask(e)?;
        self.prev.insert(e.clone(), v.clone());
        self.next.insert(e.clone(), v.clone());
        Some(v)
    }

    pub fn swap(&mut self) { self.prev = self.next.clone(); }
}
```

Walk the rebuild rule through it. In turn n the clock component pushes `Clock ↦ Instant 5100`; `apply_pushes` puts it in `next` and, because `Clock` is in `prev` from an earlier ask, keeps it as a push. Occurrence n opens with that push. A stopwatch that was already running has a `Clock` slot; its dynamic fires with `5100` and its hold commits. A stopwatch built in n, inside a `rebuild` that rebuilt because of a click in the same occurrence, creates a `Clock` slot mid-build; the runtime asks the oracle; `prev` says `5000`; the slot's dynamic starts at `5000`, then receives the push `5100` in the same occurrence, and ends n at `5100` like its older sibling. Both read `5000` through any tag in n, both are `5100` after. One snapshot per occurrence, answered the same to everyone, which is the equality law stated at the boundary.

Now suppose the oracle answered from `next` instead. The new slot would start at `5100`, receive the push `5100`, and look fine; but a tag read in n by anything that depends on the new slot's dynamic would see `5100` while a tag read through the old slot would see `5000`. Two readings of one state element in one occurrence, different. The equality law is the reason the oracle answers from the previous snapshot, and the push is what brings the new slot up to date within the occurrence.

The `clone` in `swap` is the simple version. A World with a large snapshot (a file watcher over a tree) would keep `next` as a delta over `prev` and swap by applying it; the maps are keyed by element value, so a push for one file touches one entry, and the delta is the push list. Nothing in the contract changes.

## Slot tracking and discarding late results

The World sends responses only to paths in the list it last received, and a result for a slot that has gone is discarded. Both are one table:

```
pub struct SlotTable {
    last_list: Vec<(Path, ElementValue)>,          // what the World read after the last turn
    known: BTreeMap<Path, ElementValue>,           // the same, as a map
}

impl SlotTable {
    /// Splits inbound deliveries into this turn's responses and pushes.
    /// A response to an unknown path is discarded with a diagnostic.
    /// A second response to one path in the same batch is held for the next turn:
    /// a slot receives one response per occurrence, and the batch did not say they
    /// were simultaneous in any sense the program could use.
    pub fn sort_inbound(&self, inbound: Vec<Delivery>, cx: &mut Cx)
        -> (Vec<(Path, Value)>, Vec<(ElementValue, Value)>, Vec<Delivery>)
    {
        let mut responses = Vec::new();
        let mut pushes = Vec::new();
        let mut held = Vec::new();
        let mut seen = BTreeSet::new();
        for d in inbound {
            match d {
                Delivery::Push { element, value } => pushes.push((element, value)),
                Delivery::Response { path, value } => {
                    if !self.known.contains_key(&path) {
                        cx.diagnostic(Diagnostic::LateResult { path, value });
                    } else if !seen.insert(path.clone()) {
                        held.push(Delivery::Response { path, value });
                    } else {
                        responses.push((path, value));
                    }
                }
            }
        }
        (responses, pushes, held)
    }

    /// After a turn: which slots appeared and which went, against the list now stored.
    pub fn diff(&mut self) -> (Vec<(Path, ElementValue)>, Vec<(Path, ElementValue)>) {
        let now: BTreeMap<_, _> = self.last_list.iter().cloned().collect();
        let created = now.iter().filter(|(p, _)| !self.known.contains_key(*p))
                         .map(|(p, e)| (p.clone(), e.clone())).collect();
        let removed = self.known.iter().filter(|(p, _)| !now.contains_key(*p))
                         .map(|(p, e)| (p.clone(), e.clone())).collect();
        self.known = now;
        (created, removed)
    }
}
```

A late result is not an error. The World started a read because a slot asked for it; the slot went away before the read finished; the result arrives. Nobody is listening, and that is the correct outcome of the program's decision to stop listening. The core makes it visible: `Diagnostic::LateResult` goes to the report, where a developer can see that a read was wasted, and the World cancels the operation at `slot_removed` time if the OS lets it. The core does not deliver it, because delivering a response to a path the program's list does not have is the one protocol error the runtime would reject with an abort, and the core's job is to never let a World commit it.

Paths are never reused, so a late result cannot be mistaken for a result to a new slot at the same position. A `rebuild` that rebuilds gives the new slot a fresh generation in its path, and a result for the old generation is late by construction.

The "two to one path" case is the other half of routing. A reader thread that delivered two chunks of stdin before the loop woke up has produced two responses for one slot. They cannot share an occurrence: a slot receives one response per occurrence, and the runtime enforces that with an error. The core holds the second for the next turn, which is what a World that ran a turn per chunk would have done anyway. A World that wants two inputs in one occurrence, a the comma, has to address two different slots, and the Test World's the comma step checks exactly this before it delivers.

## Unsupported

A program that was written for a GUI World and run under a CLI World has `Ui.Button` slots at the root, and no component accepts `Ui`. Three things can be in that position, and they get three answers, each chosen so that the program sees a fact rather than silence.

```
impl<R: Runtime> World<R> {
    fn perform(&mut self, path: &Path, el: &ElementValue, request: Value) {
        match self.by_tag.get(&el.tag) {
            Some(&ix) => self.components[ix].perform(path, el, request, &mut self.cx),
            None => {
                let kind = self.program.tags.kind(&el.tag);
                if kind.responds {
                    self.cx.respond(path, Value::unsupported());
                }
                self.cx.diagnostic(Diagnostic::UnsupportedRequest {
                    path: path.clone(), tag: el.tag, chain: self.chain_of(path) });
            }
        }
    }

    fn route(&mut self, el: &ElementValue) -> &mut dyn Component {
        match self.by_tag.get(&el.tag) {
            Some(&ix) => &mut *self.components[ix],
            None => &mut self.nobody,     // a component that records and does nothing
        }
    }
}
```

*A request tag* nobody accepts gets an `Unsupported` response in the next turn, if the tag responds. `Unsupported` is a value of every response type, which the compiler arranges: every `responds: R` declaration is checked as `R | Unsupported`, and a `match` on a response needs an arm for it or a catch-all. A program under the wrong World therefore sees its request answered, with a value that says so, and can show "not available here" rather than wait forever. The response is in the next turn and not the same occurrence, because it is a response, and the creation rule binds the core like anyone else.

*A presence tag* nobody accepts, `Ui.Label` under the CLI World, is reported once per slot at `slot_created`, through the `nobody` component, and otherwise ignored. There is nothing to respond to and nothing to deny. The report names the path, the tag and the chain of interceptors the element came through, so that the developer sees "a `Ui.Label` reached the CLI World from `@vbox` inside `@main`" and not a bare tag name.

*A state tag* nobody accepts cannot be deferred to the next turn, because a state element has a value in the occurrence it is created, and no value is available. The oracle closure in `run_occurrence` returns `None` for it, and the runtime aborts the occurrence with `RuntimeError { kind: ErrorKind::UnsuppliedState, spans, .. }`. The transaction rolls back; the World reports the error with the slot's source span; the responses that opened the occurrence are dropped, and its pushes go back to wait for the next turn. That is harsh and it is right: a program that reads a state

the World does not have cannot run under that World, and the earliest honest moment to say so is the moment the slot would have been born.

The interceptor chain in these reports comes from the path. A path is the chain of bind positions from the root, and the compiled program's binding table maps each position to a source span and the name of the widget function it is in; `chain_of` walks the path and collects the widget names. It is the same walk that gives a cycle error its spans.

## Error reporting

An error from the run call is a value, and the core turns it into text in one format, so that every World's errors read the same and a transcript's `errors` step has one thing to match.

```
pub enum WorldError {
    Runtime { error: RuntimeError, when: When },
    DeferBackwards { path: Path, when: When },
    DuplicateTag { tag: Tag, first: &'static str, second: &'static str },
}

pub enum Diagnostic {
    LateResult { path: Path, value: Value },
    UnsupportedRequest { path: Path, tag: Tag, chain: Vec<Name> },
    UnsupportedPresence { path: Path, tag: Tag, chain: Vec<Name> },
    Component { tag: Tag, message: String },      // a component's own trouble: EPIPE, a watch that failed
}

pub trait Report {
    fn error(&mut self, e: &WorldError, program: &Program);
    fn diagnostics(&mut self, ds: Vec<Diagnostic>);
}
```

The printer, `prism_world::report::Text`, writes an error as a header line, one line per span, and one line saying when:

```
  error[cycle]: an event depends on itself within one occurrence
    --> stopwatch.prism:31:5   `!action`
    --> stopwatch.prism:34:5   `$watch`
    --> stopwatch.prism:31:5   `!action`
     in turn 7, occurrence 2 (answering Defer at 0/3/1)

  error[unsupplied-state]: no component supplies `Gui.Metrics(text="Start", font=Default)`
    --> gui.prism:112:9   `$m <- @Element Gui.Metrics(text=label, font=Default)`
     via @Button, @vbox, @main
     in turn 0, occurrence 1

  error[defer-backwards]: Defer at 0/#0/2 fired again after being answered this turn
    --> sandbox.prism:40:29  `!later <- @Element Defer(requests=(unit on _.!request))`
     in turn 12, occurrence 3
```

Spans come from the compiled program: the runtime names every cyclic placeholder and every slot after its source binding, so a runtime cycle reports Prism source and not runtime internals. The "when" line is what a developer needs to reproduce the error under the Test World, where turn 7 of a script is a line number.

Diagnostics are printed in a shorter form, one line each, to the same sink, prefixed `note:`. A CLI World sends both to stderr; a GUI World could show them in a window; the Test World keeps them in the transcript.

What the World does with an error is the World's policy, and the core leaves it to the caller of `turn`: the `Err` comes back after the firings of every occurrence that committed before the abort have been performed, the aborted occurrence has rolled back, its responses are gone and any pushes it was opened with are re-queued, and the caller decides whether to log and continue, or exit. A CLI World exits nonzero. A GUI World might keep running, because the window is still there and the next click is a fresh occurrence. A Test World records it for the `errors` step and continues the script, so that one test can check both the error and what the program does after it.

## The harness: driving a runtime from any event loop

Every World has a loop, and no two have the same one: a CLI World blocks on a channel, a winit World is called back by the platform, a tokio World selects over futures, a browser World is a callback with no loop at all. The core offers the harness so that each of those can drive the same turn with the same four lines.

```
pub struct Harness<R: Runtime> {
    world: World<R>,
    inbox_rx: std::sync::mpsc::Receiver<Delivery>,
    held: Vec<Delivery>,                    // responses held over from a previous batch
    policy: BatchPolicy,
}

pub enum BatchPolicy {
    /// One delivery per turn. The CLI World's choice.
    OneAtATime,
    /// Everything queued when the turn starts, subject to one response per path.
    /// A GUI World's choice for simultaneous inputs it can truthfully call simultaneous.
    Drain,
}

pub enum Step {
    /// Nothing was queued; the caller should wait for a wake.
    Idle,
    /// A turn ran.
    Ran(TurnOutcome),
    /// A component asked to end the run, after this turn's effects were performed.
    Exit(i32),
    /// Nothing can ever cause another occurrence.
    Quiescent,
    Error(WorldError),
}

impl<R: Runtime> Harness<R> {
    pub fn new(world: World<R>, policy: BatchPolicy) -> Self;
    pub fn inbox(&self) -> Inbox;

    /// Occurrence zero, then the effects of the initial list. Call once.
    pub fn start(&mut self) -> Step;

    /// Run at most one turn with whatever has arrived. Never blocks.
    pub fn step(&mut self) -> Step {
        let mut batch = std::mem::take(&mut self.held);
        match self.policy {
            BatchPolicy::OneAtATime => {
                if batch.is_empty() {
                    match self.inbox_rx.try_recv() { Ok(d) => batch.push(d), Err(_) => {} }
                }
            }
```

```
            BatchPolicy::Drain => {
                while let Ok(d) = self.inbox_rx.try_recv() { batch.push(d); }
            }
        }
        if batch.is_empty() && !self.world.has_gatherable() {
            return if self.world.quiescent() { Step::Quiescent } else { Step::Idle };
        }
        match self.world.turn(batch) {
            Err(e) => Step::Error(e),
            Ok(out) => match out.exit {
                Some(code) => Step::Exit(code),
                None => Step::Ran(out),
            },
        }
    }

    /// Block until something arrives, with a deadline for the next timer.
    pub fn wait(&mut self, until: Option<Instant>) {
        let d = match until { Some(t) => t.saturating_duration_since(Instant::now()),
                              None => Duration::MAX };
        if let Ok(d) = self.inbox_rx.recv_timeout(d) { self.held.push(d); }
    }
}
```

`step` never blocks and never waits, so it can be called from inside a platform callback: a winit `window_event` handler pushes the event into the inbox as a `Delivery` and calls `step` before returning. `wait` is for Worlds that own their thread, and takes a deadline so that a timer component can say when it next needs a turn. The `Wake` trait connects a thread that delivered something to the loop that should call `step`: a CLI World's waker does nothing, because the loop is blocked on the channel and the send wakes it; a winit World's calls the event loop proxy; a tokio World's notifies a `Notify` that the select is waiting on.

The CLI World's whole main is then:

```
fn main() -> ExitCode {
    let program = prism_lang::compile_file(&path)?;
    let mut world = World::new(program, Box::new(report::Text::stderr()), Arc::new(NoWake));
    let inbox = world.inbox();
    world.add(process::Process::new(args, env))
         .add(cli::Stdio::new(inbox.clone()))
         .add(cli::Signals::new(inbox.clone()))
         .add(fs::Files::new(inbox.clone()))
         .add(time::Timers::new(inbox.clone()));
    let mut h = Harness::new(world, BatchPolicy::OneAtATime);
    if let Step::Exit(c) | Step::Error(_) = h.start() { return exit_code(c) }
    loop {
        match h.step() {
            Step::Ran(_) => {}
            Step::Idle => h.wait(h.world().next_deadline()),
            Step::Exit(code) => return ExitCode::from(code as u8),
            Step::Quiescent => return ExitCode::SUCCESS,
            Step::Error(e) => return ExitCode::FAILURE,
        }
    }
}
```

That is the entire difference between the CLI World and any other at the level of the loop. Everything that makes it a CLI World is in the components.

A turn is caused by a delivery, and a push gathered from a component is a delivery the core finds by asking. The clock pushes once per turn, but a clock alone never causes a turn, because a program that only looks at the clock has nothing to react to; the turn that reads the clock is caused by something else. So `has_gatherable` asks components whether they have something that should start a turn on its own (a file watch that saw a change, a timer that expired), and the clock answers no. The distinction keeps a program with a `Clock` slot from spinning.

# The Test World

The Test World is the world core plus one component that answers to a script. It has no effects; it accepts every tag; it is the only source of inputs; its snapshot is whatever the script says. It is also the World the conformance suite runs under, in both implementations, so its transcript format is an interchange format and its steps are a specification of what a World can observe.

## The script, in `lang`

A test is an item in a Prism file, parsed by the `lang` crate like any other:

```
test counter_counts, @counter:
    build
        => expect Ui.Label(text="Count: 0")
    click Ui.Button(label="Increment")
        => settles
    click Ui.Button(label="Increment")
        => expect Ui.Label(text="Count: 2")
    click Ui.Button(label="Increment"), click Ui.Button(label="Decrement")
        => expect Ui.Label(text="Count: 1")
```

The parser produces a `TestItem { name, widget: Expr, turns: Vec<Turn> }` where the blueprint expression is elaborated and compiled as if it were `@main`, and each turn is one line of the script with its `=>` lines:

```
pub struct Turn {
    pub withs: Vec<(Pattern, Expr)>,     // `with p = v`, applied before the inputs
    pub inputs: Vec<Input>,              // `build`, `idle`, or one or more deliveries
    pub checks: Vec<Check>,              // one per `=>` line
}

pub enum Input {
    Build,
    Idle,
    Click(Pattern),
    Send(Pattern, Expr),                 // Expr: a plain value, evaluated at test time
    Push(Pattern, Expr),
}

pub enum Check {
    Expect(Pattern),
    ExpectTop(Pattern),
    ExpectNone(Pattern),
    ExpectTopNone(Pattern),
    ExpectCount(Pattern, usize),
    Returns(Pattern),
    Fired(Pattern),
    Quiet,
    Requested(Pattern, Pattern),
```

```
    Unrequested(Pattern),
    Errors(Pattern),
    Settles,
}
```

A `Pattern` here is an element pattern as the language already has it for `match`: a tag with named fields, each field a value pattern or `..`, or `Ns.*(..)`. The value in a `send` or `with` is an ordinary plain expression, type-checked against the tag's declared response or state type, so `send Ui.Button(..), "hello"` is a compile error in the test file. This is why the script lives in `lang` and not in a separate test language: it reuses the pattern grammar, the type checker, and the error messages, and a test file is a Prism file.

The `requested` step's second argument is a pattern over the request value rather than a value, so that `requested Time.Every(..), ignore` can say "it fired, I do not care with what".

## Execution

```
pub struct ScriptComponent {
    snapshot: BTreeMap<ElementValue, Value>,     // from `with` entries; nothing else supplies state
    accepted: Vec<Tag>,                           // every tag the program declares
}

impl Component for ScriptComponent {
    fn tags(&self) -> Vec<Tag> { self.accepted.clone() }
    fn slot_created(&mut self, _: &Path, _: &ElementValue, _: &mut Cx) {}
    fn slot_removed(&mut self, _: &Path, _: &ElementValue, _: &mut Cx) {}
    fn perform(&mut self, _: &Path, _: &ElementValue, _: Value, _: &mut Cx) {}   // recorded by the core
    fn oracle(&mut self, e: &ElementValue, cx: &mut Cx) -> Value {
        self.snapshot.get(e).cloned().unwrap_or_else(|| {
            cx.diagnostic(Diagnostic::Component {
                tag: e.tag, message: "no State step for this element".into() });
            Value::missing()
        })
    }
    fn gather(&mut self, _: &mut Cx) {}
    fn quiescent(&self) -> bool { true }
}
```

The component is almost empty, and that is the design: a Test World does nothing, so its component does nothing, and everything the steps need is in the `TurnOutcome` the core already returns. `Value::missing()` is a sentinel the core turns into an `UnsuppliedState` error, so a state element with no `with` step fails the test at the slot's creation with the same error a real World gives for a state tag it does not accept. Tag declarations carry no default state value, so there is nothing to fall back on: a program that creates `Clock` or `Gui.Metrics` while it builds needs a `with` step for that element before the build, and a test that forgets one fails with an error that names the element instead of running against a value nobody chose.

The runner holds the harness and the script and walks the turns:

```
pub struct TestRun {
    harness: Harness<RustRuntime>,
    script: ScriptComponent,            // shared with the World through Rc<RefCell>
    last: Option<TurnOutcome>,          // the last turn, for Fired/Quiet/Requested
    last_error: Option<WorldError>,     // for Errors
    transcript: Transcript,
```

```
}

impl TestRun {
    pub fn run(mut self, turns: &[Turn]) -> Result<Transcript, Failure> {
        for (i, t) in turns.iter().enumerate() {
            self.turn_line(t).map_err(|why| Failure { turn: i, why })?;
        }
        Ok(self.transcript)
    }

    fn turn_line(&mut self, t: &Turn) -> Result<(), String> {
        for (p, v) in &t.withs { self.script.set(find_state(p)?, eval(v)); }
        let list = self.last_list();
        let mut batch = Vec::new();
        let mut seen = BTreeSet::new();
        for input in &t.inputs {
            match input {
                Input::Build => self.record(self.harness.start()),      // the build reads this line's withs
                Input::Idle  => {}
                Input::Click(p) | Input::Send(p, _) => {
                    let path = find_one(&list, p)?;
                    if !seen.insert(path.clone()) {
                        return Err(format!("two deliveries to {} on one line", path));
                    }
                    let value = match input { Input::Send(_, v) => eval(v), _ => Value::unit() };
                    batch.push(Delivery::Response { path, value });
                }
                Input::Push(p, v) => {
                    let e = find_state(p)?;
                    self.script.set(e.clone(), eval(v));
                    batch.push(Delivery::Push { element: e, value: eval(v) });
                }
            }
        }
        if !matches!(t.inputs[..], [Input::Build]) { self.turn(batch)?; }
        for c in &t.checks { self.check(c)?; }
        Ok(())
    }

    fn check(&mut self, c: &Check) -> Result<(), String> {
        let list = &self.last_list();
        match c {
            Check::Expect(p)        => (find_all(list, p).len() >= 1).then_some(()).ok_or(no_match(p)),
            Check::ExpectTop(p)     => (list.iter().any(|(_, e)|
            p.matches(e))).then_some(()).ok_or(no_match(p)),
            Check::ExpectNone(p)    => find_all(list, p).is_empty().then_some(()).ok_or(unexpected(p)),
            Check::ExpectTopNone(p) => (!list.iter().any(|(_, e)|
            p.matches(e))).then_some(()).ok_or(unexpected(p)),
            Check::ExpectCount(p, n) => (find_all(list, p).len() == *n).then_some(()).ok_or(count(p, *n)),
            Check::Returns(p)       => p.matches_result(&self.last_result()).then_some(()).ok_or(..),
            Check::Fired(p)         => self.last_turn_fired(p).then_some(()).ok_or(..),
            Check::Quiet            => (!self.last_turn_fired_any()).then_some(()).ok_or(..),
            Check::Requested(p, vp) => self.last_turn_requested(p, vp).then_some(()).ok_or(..),
            Check::Unrequested(p)   => (!self.last_turn_requested_any(p)).then_some(()).ok_or(..),
            Check::Errors(p)        => self.last_error.as_ref().filter(|e| p.matches_error(e)).map(|_|
            ()).ok_or(..),
            Check::Settles          => self.last_error.is_none().then_some(()).ok_or("the turn ended in an
            error".into()),
        }
    }
```

```
    fn turn(&mut self, batch: Vec<Delivery>) -> Result<(), String> {
        self.harness.push_all(batch);
        match self.harness.step() {
            Step::Ran(out) => { self.last_error = None; self.record_turn(out); Ok(()) }
            Step::Error(e) => { self.last_error = Some(e.clone()); self.transcript.error(&e); Ok(()) }
            other => Err(format!("unexpected {:?}", other)),
        }
    }
}
```

The Test World's harness uses `BatchPolicy::Drain`, because a script says exactly what is in a turn and nothing else is queued.

`find_all` is the depth rule. It walks every slot in the list, matches the pattern against the element, and then descends into any field whose value is an element or a collection of elements, because that is the convention a GUI library uses to hold children as data in a container element. The Test World knows the convention as a structural fact about values, not as a tag name: a field of type `Element`, `[Element]` or `k ==> Element` is descended into. `Send` to a nested element delivers to the containing root slot's path with the nested index attached in the response, following the same library convention the other way; chapter 2's vertical box shows the Prism side of it.

A `with` takes effect as its own line's push and oracle: the `with`s on the `build` line run before the build, and the build occurrence reads them; `set` writes the script's snapshot, which the oracle reads for values not yet asked about, and the next turn's `gather` reports the changed values as pushes for values already asked about. `push` is `with` plus a turn with the push as its only delivery.

## How `with`, `push`, `requested` and `errors` are implemented

Each of the four that the walkthroughs in chapter 2 lean on has one source of truth in the core.

`with` and `push` are the snapshot. A `with Clock = Instant 5000` before the first turn means the oracle answers `5000` the first time any `Clock` slot asks, in occurrence zero, so a stopwatch built in the build occurrence has its clock in the build occurrence, which is the rule that state is there in its first occurrence. A `push Clock = Instant 5100` later is a `Delivery::Push`, which `apply_pushes` keeps because `Clock` is in `prev`, and which the runtime fires on the shared state dynamic for that value, reaching every `Clock` slot in the program and any `Clock` slot created in the push's occurrence. The script does not know how many slots there are; it sets one value for one element, which is value identity made visible.

`requested` is the `requests` vector of the last turn's occurrence records. `requested Time.Every(..), ignore` finds a slot in any occurrence of the last turn whose element matches and whose firings contain a value matching the second pattern. It looks at the whole turn because a request fired in a `Defer` occurrence is part of what the World saw after the turn, and a script that could only see the first occurrence would be seeing something no World sees. A request from a slot torn down in the same occurrence is in the records too, with its path, so a test can check that a write fired by a widget on its way out was performed.

`fired` and `quiet` are the `result` view of the same records: `fired` passes if the widget's result event fired in any occurrence of the turn with a matching value, `quiet` if it fired in none. `returns` compares the current value after the turn.

`errors` is the `Err` from `turn`. The runner keeps running after an error, with the runtime unchanged (the occurrence rolled back), so that `errors Cycle(..)` followed by `expect Ui.Label(text="Count:`

`2")` can assert that the label still shows what it showed before the bad occurrence. The pattern matches on the error's kind and, optionally, on the paths and spans in it, so a conformance test for "deferring backwards is an error" is:

```
test defer_backwards, @retry_forever_under_sandbox:
    build
        => settles
    click Ui.Button(label="Read")
        => errors DeferBackwards(..)
```

## The transcript printer

The transcript is what the conformance suite compares and what the differential test diffs between implementations, so its form is fixed and both implementations print it the same way. It is one block per turn:

```
  turn 0
    occurrence 0
      list
        0/#0/0: Ui.Button(label="Increment")
        1/#0/0: Ui.Button(label="Decrement")
        2/#0/0: Ui.Label(text="Count: 0")
      result: unit
  turn 1  response 0/#0/0 click
    occurrence 1
      list
        0/#0/0: Ui.Button(label="Increment")
        1/#0/0: Ui.Button(label="Decrement")
        2/#1/0: Ui.Label(text="Count: 1")
      result: unit
  turn 2  response 0/#0/0 click
    ...
  turn 5  push Clock Instant 5100
    occurrence 7
      requested 3/#0/1: Time.Every(period=100) <- tick
      list
        ...
  turn 6  response 0/2/0 click
    occurrence 8
      requested 0/2/0: Fs.File(path="/etc/shadow", requests=..) <- Read
      requested 0/3/1: Defer <- unit
      list
        ...
    occurrence 9  answering Defer 0/3/1
      list
        ...
  turn 7  response 0/#0/0 click
    error[cycle] at occurrence 10
      --> counter.prism:12:5 `!delta`
```

Paths are printed in the form chapter 2 uses (`bind/#generation/bind`, `[key]` for `for`), which is the one representation the two implementations agree to print even though each stores paths as it likes. Elements print as the language writes them, fields in declaration order, with `requests=..` for an event field. Values print as literals. Lists are in tree order, which is path order.

The printer is `impl Display for Transcript`, fed by `record_turn` from the `TurnOutcome` and nothing else. A transcript therefore contains only what the World saw: no hold values, no node internals, no event names. A test that passes on a transcript passes under any implementation that produces the

same one, and that is what makes the differential test meaningful.

### What the Test World never touches

The Test World reads nothing but the run call's outcome. It has no handle to a node, no table of named triggers it can fire by name, no way to read a dynamic by name, and no delegate that the runtime calls when an event fires. All of those are ways for a test to depend on the structure of a widget rather than on its behaviour, and a test that depends on structure breaks when the structure is refactored without the behaviour changing, which is the worst kind of test. By addressing only elements, which are the only things a real World can address, a test is a World, and a program that passes under it passes under any World that delivers the same inputs.

It also means the runtime's own tests and the Test World are different layers. The runtime's Rust tests build graphs with the combinators directly and check laws about them; they are allowed to look inside, because they test the runtime's structure on purpose. The Test World tests programs.

## The CLI World

The CLI World is the smallest World that is not a test, and the first place a Prism program meets a real process. The problem it solves is the one every language solves on day one, reading input and writing output, done under the constraint that the program cannot do either: the program has elements, and the World has the file descriptors.

### Structure

The main thread runs the harness loop shown above: `step`, and when idle, `wait` with the next timer's deadline. Everything that blocks lives on a thread that is not the main thread, and talks to the main thread through the inbox. There are at most four such threads: one reading stdin, one waiting on signals, a small pool for file operations, and none for timers, because a timer is a deadline the main loop's `wait` already honours. Nothing on any of those threads touches the runtime. A thread's whole interface to the World is `inbox.send(Delivery::Response { path, value })`.

The components, with the tags each accepts:

| Component | Tags | Kinds |
|---|---|---|
| `process::Process` | `Process.Args`, `Process.Env` | state |
| `cli::Stdio` | `Cli.Stdin`, `Cli.Stdout`, `Cli.Stderr`, `Process.Exit` | responds; requests+responds; requests+responds; requests |
| `cli::Signals` | `Cli.Signals` | responds |
| `fs::Files` | `Fs.File`, `Fs.Contents` | requests+responds; state |
| `time::Timers` | `Time.After`, `Time.Every`, `Clock` | requests+responds; responds; state |
| the core | `Defer` | requests+responds |

`Fs`, `Time` and `Clock` are not CLI tags; they are the same components a GUI World uses, in the same crate, and the CLI World is the first World to link them. The tag declarations are the ones chapter 2 uses:

```
type StdinEvent = choice:
    Chunk Text
    End

type Code = Code Int

tag Cli.Stdin of Element:
    responds: StdinEvent

type WriteFailed = choice:
    Closed
    Failed Text

tag Cli.Stdout of Element:
    requests: !Text
    responds: WriteFailed

tag Cli.Stderr of Element:
    requests: !Text
    responds: WriteFailed

tag Process.Exit of Element:
    requests: !Code

tag Process.Args of Element:
    state: [Text]

tag Process.Env of Element:
    state: Text ==> Text

type Signal = choice Interrupt | Terminate | Hangup | Other Int

tag Cli.Signals of Element:
    responds: Signal
```

The kinds are what the language fixes, and the reasoning for each is below with its component; the field names follow the library.

### `Process.Args` and `Process.Env`: state, constant after start

```
pub struct Process { args: Value, env: Value }

impl Component for Process {
    fn tags(&self) -> Vec<Tag> { vec![Tag::PROCESS_ARGS, Tag::PROCESS_ENV] }
    fn oracle(&mut self, e: &ElementValue, _: &mut Cx) -> Value {
        if e.tag == Tag::PROCESS_ARGS { self.args.clone() } else { self.env.clone() }
    }
    fn gather(&mut self, _: &mut Cx) {}            // never pushes
    fn quiescent(&self) -> bool { true }           // an Args slot keeps nothing alive
    fn slot_created(&mut self, ..) {}  fn slot_removed(&mut self, ..) {}  fn perform(&mut self, ..) {}
}
```

Arguments and environment are state elements rather than parameters of `@main` so that an interceptor can give a subtree different ones and a test can set them with `with Process.Args = [["notes.txt"]]`, with no second way to start a program. On a process that cannot observe a change to its arguments the component never pushes, and the `rebuild` a program puts over `$args` builds once. A World that can observe a change (a shell World hosting other programs, where "arguments" is something the

shell decides) pushes, and the same program rebuilds; nothing in the program says which World it is under.

`quiescent` is `true` because a program whose only slot is `Process.Args` is a program that has nothing left to do, and the run should end with code 0 rather than wait forever for a push that never comes.

## `Cli.Stdin`: a reader thread, started and stopped by the slot

Standard input is a stream element: it responds with chunks and then an end, and the program never asks it for anything. The World reads stdin only while a `Cli.Stdin` slot exists, which has a consequence a shell user will recognise: a program that builds its `Cli.Stdin` inside a `rebuild` after reading a file first does not consume stdin until the file has arrived, so a pipeline behind it is not drained early.

```
pub struct Stdio {
    inbox: Inbox,
    stream: Arc<StdinStream>,              // outlives any slot; shared with the reader thread
    slot: Option<Path>,                     // the Cli.Stdin slot that is listening, if any
    reader: Option<JoinHandle<()>>,         // one thread, started by the first slot
    out: BufWriter<std::io::Stdout>,
    err: BufWriter<std::io::Stderr>,
    out_failed: bool,
    chunk: usize,                          // max bytes per chunk; a flag for goldens
}

/// What stdin has produced that no slot has taken. The stream owns it, so a slot
/// that goes away leaves it where the next slot will find it.
struct StdinStream { state: Mutex<StdinState>, changed: Condvar }
struct StdinState {
    listening: bool,                       // a Cli.Stdin slot exists
    held: VecDeque<Held>,                  // read and not yet delivered, in order
    ended: bool,                           // End has been read
}
enum Held { Chunk(String), End }

fn read_loop(stream: Arc<StdinStream>, inbox: Inbox, chunk: usize) {
    let mut stdin = std::io::stdin().lock();
    let mut buf = vec![0u8; chunk];
    let mut pending = Vec::new();                  // bytes of a character the last read split
    loop {
        {   // read only for a listening slot, and only once the last read has been taken
            let mut st = stream.state.lock().unwrap();
            while !(st.listening && st.held.is_empty()) { st = stream.changed.wait(st).unwrap(); }
        }
        let n = stdin.read(&mut buf).unwrap_or(0);  // blocks; cannot be interrupted portably
        let mut st = stream.state.lock().unwrap();
        if n == 0 {
            if !pending.is_empty() { st.held.push_back(Held::Chunk(lossy(&pending))); }
            st.held.push_back(Held::End);
            st.ended = true;
        } else {
            pending.extend_from_slice(&buf[..n]);
            let cut = complete_utf8_prefix(&pending);    // up to the last whole character
            let text = lossy(&pending[..cut]);
            pending.drain(..cut);
            if !text.is_empty() { st.held.push_back(Held::Chunk(text)); }
        }
        let done = n == 0;
        drop(st);
```

```
        inbox.send(Delivery::Wake);               // a turn may be due; the chunk waits in `held`
        if done { return; }
    }
}

impl Component for Stdio {
    fn slot_created(&mut self, path: &Path, el: &ElementValue, cx: &mut Cx) {
        if el.tag != Tag::CLI_STDIN { return; }
        if self.slot.is_some() {
            cx.diagnostic(Diagnostic::Component { tag: el.tag,
                message: "a second Cli.Stdin slot; stdin is one stream and goes to the first".into() });
            return;
        }
        self.slot = Some(path.clone());
        let waiting = {
            let mut st = self.stream.state.lock().unwrap();
            st.listening = true;
            !st.held.is_empty()                   // something was read while no slot listened
        };
        self.stream.changed.notify_all();
        if self.reader.is_none() {
            let (stream, inbox, chunk) = (self.stream.clone(), self.inbox.clone(), self.chunk);
            self.reader = Some(std::thread::spawn(move || read_loop(stream, inbox, chunk)));
        }
        if waiting { self.inbox.send(Delivery::Wake); }
    }

    fn slot_removed(&mut self, path: &Path, el: &ElementValue, _: &mut Cx) {
        if el.tag == Tag::CLI_STDIN && self.slot.as_ref() == Some(path) {
            self.slot = None;
            self.stream.state.lock().unwrap().listening = false;
            // A read already blocked in read(2) still completes. Its chunk goes into `held`
            // and stays there for the next slot; nothing is sent to this path again.
        }
    }

    fn gather(&mut self, cx: &mut Cx) {
        if let Some(path) = &self.slot {
            let mut st = self.stream.state.lock().unwrap();
            if let Some(ev) = st.held.pop_front() {      // one event per turn, to the slot that listens now
                cx.respond(path, ev.into_value());
                self.stream.changed.notify_all();
            }
        }
        // ... flush stdout and stderr, as below
    }

    fn has_gatherable(&self) -> bool {
        self.slot.is_some() && !self.stream.state.lock().unwrap().held.is_empty()
    }

    fn quiescent(&self) -> bool {
        // No slot: nothing will be asked for, and whatever is held is discarded at exit.
        // A slot: alive until End has been read and delivered.
        self.slot.is_none() || {
            let st = self.stream.state.lock().unwrap();
            st.ended && st.held.is_empty()
        }
    }
    ...
```

```
}
```

Two things here are the OS being the OS, and the design accepts both rather than hiding them. A thread blocked in `read(2)` cannot be interrupted portably, so the read in flight when a slot goes will complete. Its chunk is not sent to the dropped slot and not lost. Every read goes into `held`, which belongs to the stream, and `gather` hands one held event per turn to the slot that is listening at that moment; a chunk that arrives after the slot went waits there, and the next `Cli.Stdin` slot, however much later, receives it first, then whatever is read afterwards, in order. A program that drops its `Cli.Stdin` slot and later creates another therefore picks the stream up exactly where the first left it. If no slot ever asks again, the held chunk is discarded when the process ends, the same as the rest of an input nobody read.

Delivering from `gather` and not from the thread makes the drop safe. A thread that sent to the inbox would race the slot's removal: the send could land after `slot_removed`, and the core would discard the chunk as a late result. Here the thread only fills `held` and wakes the loop, and the component, which knows which slot is live, is the only sender. The reader also does not read ahead. It reads only while a slot listens and only when `held` is empty, so a program that drops its slot stops consuming the descriptor after the one read already blocked, and a pipeline behind it is not drained. The second thing is that a `Chunk` is bytes as the OS handed them, which means a line can be split across chunks; the program's `@Lines` library widget keeps the partial line in a hold, as chapter 2's line counter does. The World does not reassemble lines, because a program that wants bytes should get bytes, and lines are a function over chunks.

Text is `from_utf8_lossy`, applied only up to the last whole character: a read that ends in the middle of a multi-byte character keeps those bytes for the next chunk, so `"é"` split across two reads arrives as `"é"` and not as two replacement characters. Without that, the text a program saw would depend on where the OS cut the pipe, which is an accident of buffering and not something the program chose; invalid bytes are still replaced. A `Cli.StdinBytes` tag responding with `Bytes` is the version for programs that want the raw stream, and the only difference is the conversion.

The second `Cli.Stdin` slot is a diagnostic and not an error, because it is the program's business if two widgets both want stdin and only one gets it; the value-identity rule does not apply, since `Cli.Stdin` is a stream element identified by path, and two paths are two conversations on one descriptor, which cannot be made to work. A library that wants to share stdin between two widgets does it in Prism, with one `Cli.Stdin` slot and an interceptor that fans out.

### `Cli.Stdout`, `Cli.Stderr`: buffered per turn, flushed at the end

```
impl Component for Stdio {
    fn perform(&mut self, path: &Path, el: &ElementValue, request: Value, cx: &mut Cx) {
        match el.tag {
            Tag::CLI_STDOUT => {
                if self.out_failed { return; }
                if let Err(e) = self.out.write_all(request.as_text().as_bytes()) {
                    self.out_failed = true;
                    cx.respond(path, write_failed(e));     // reaches the program next turn
                }
            }
            Tag::CLI_STDERR => { let _ = self.err.write_all(request.as_text().as_bytes()); }
            Tag::PROCESS_EXIT => cx.exit(request.as_code()),
            _ => {}
        }
```

```
    }

    fn gather(&mut self, _: &mut Cx) {
        // Called at the start of the next turn, which is after the previous turn's
        // performs: flush what the last turn wrote. Also flushed before exit.
        let _ = self.out.flush();
        let _ = self.err.flush();
    }
    ...
}
```

Writes are performed in occurrence order and then list order, which is the core's order for `perform`, and are flushed once per turn. The reason for per-turn flushing and not per-write is the same as the reason effects happen after the occurrence: a turn is the unit the outside sees, and a program that writes three lines in one occurrence should produce three lines together, not three syscalls. The reason for flushing at all between turns, rather than at exit, is that a CLI program's output is often being read by something, a terminal or the next process in a pipe, and the program's notion of "I said this" is the turn's end.

Two `Cli.Stdout` slots in one program both write to the one descriptor, in list order within an occurrence. That is right: two widgets that both print are two widgets that both print, and the interleaving is the one the program's structure gives. A program that wants a single ordered log puts one `Cli.Stdout` under an interceptor that merges.

A failed write (the reader of a pipe went away, `EPIPE`) sets the flag and comes back to the writing slot as a `WriteFailed` response through `cx.respond`, in a later turn, as the creation rule requires of any response. `SIGPIPE` is left as the Rust runtime leaves it, ignored (from memory; verify), so that the failed write is seen as `EPIPE` and reported, instead of killing the process silently.

### `Process.Exit`: a request, acted on after the turn

Exit is a request element with `requests: !Code`, and the firing is the whole instruction. The component's `perform` records the code with `cx.exit(code)`; the core finishes performing the turn's other requests, which means every write that fired in the same turn lands first, in order; then `Harness::step` returns `Step::Exit(code)` and main flushes and returns it.

Chapter 2 argued from the program's side why exit is a request and not a presence: nothing in the language happens because an element appears, and an exit that happened because a `Process.Exit` element was in the list would be a trigger by existence. From the World's side the argument is about ordering. A request firing has a position in the turn, after some writes and before others; a presence has only a list position, and "exit after this turn's writes but not before the writes of a widget later in the list" would have no meaning. With a request, `Process.Exit` fired in the same occurrence as a `Cli.Stdout` write lands after that write if the stdout slot is earlier in the list, which is the order the program's structure gives, and a program that wants its goodbye message before exit puts the stdout slot first or fires the exit from a later occurrence.

Two `Process.Exit` firings in one turn: the first in occurrence-then-list order wins, and the second gets a diagnostic. Two programs under a shell World both exiting is the shell's business and it intercepts `Process.Exit` before the real World sees it, as chapter 2's shell does.

A code outside 0–255 is clamped and reported, because that is what the OS will do anyway and reporting it is better than letting a program think it exited with 256.

### `Cli.Signals`: the handler follows the slot

```
pub struct Signals { inbox: Inbox, slot: Option<Path>, handle: Option<signal_hook::iterator::Handle> }

impl Component for Signals {
    fn slot_created(&mut self, path: &Path, _: &ElementValue, _: &mut Cx) {
        let mut signals = signal_hook::iterator::Signals::new([SIGINT, SIGTERM, SIGHUP]).unwrap();   //
        signal_hook API from memory
        self.handle = Some(signals.handle());
        let (inbox, p) = (self.inbox.clone(), path.clone());
        std::thread::spawn(move || {
            for sig in signals.forever() {
                inbox.send(Delivery::Response { path: p.clone(), value: signal_value(sig) });
            }
        });
        self.slot = Some(path.clone());
    }
    fn slot_removed(&mut self, _: &Path, _: &ElementValue, _: &mut Cx) {
        if let Some(h) = self.handle.take() { h.close(); }      // restores the OS default (from memory)
        self.slot = None;
    }
    fn quiescent(&self) -> bool { self.slot.is_none() }
    ...
}
```

With no `Cli.Signals` slot, `SIGINT` does what the OS does: kills the process. With one, the signal arrives as a response and the program decides, typically by firing `Process.Exit`. The handler is installed when the slot appears and removed when it goes, so a program that handles `Ctrl-C` only during a critical section has a `Cli.Signals` slot only during it. This is the rule that a stream lives as long as its slot, applied to something that is not a stream of data but a stream of OS events.

A `Cli.Signals` slot keeps the program alive (`quiescent` is false while it exists), because a program waiting for a signal is waiting for something that can happen. That is the difference from `Process.Args`.

### `Fs.File` and `Fs.Contents`: a thread pool and a watcher

`Fs.File(path, requests: !FsRequest, responds: FsResponse)` is the request element for one-shot operations; `Fs.Contents(path, state: Unavailable FsError | Known Text)` is the state element for a file's current contents, where `FsError` is the closed choice `NotFound | Denied | TooLarge | Other Text`. They are one component because they share the path and, on platforms with a watch, the watcher.

```
pub struct Files {
    inbox: Inbox,
    cap: u64,                                          // bytes an Fs.Contents read may return; a flag for
goldens
    pool: threadpool::ThreadPool,                      // for Fs.File operations (crate API from memory)
    inflight: BTreeMap<Path, Arc<AtomicBool>>,          // per slot: a cancel flag
    watched: BTreeMap<ElementValue, Watched>,          // per Fs.Contents value: the watch and the last value
    watcher: notify::RecommendedWatcher,               // notify's API from memory
    changes: std::sync::mpsc::Receiver<notify::Event>,  // from the watcher's thread
}

struct Watched { value: Value, slots: usize }

impl Component for Files {
    fn tags(&self) -> Vec<Tag> { vec![Tag::FS_FILE, Tag::FS_CONTENTS] }
```

```
fn perform(&mut self, path: &Path, el: &ElementValue, request: Value, _: &mut Cx) {
    let file = el.field("path").as_text().to_owned();
    let cancel = self.inflight.entry(path.clone()).or_default().clone();
    let (inbox, p) = (self.inbox.clone(), path.clone());
    self.pool.execute(move || {
        let response = match request.as_fs_request() {
            FsRequest::Read => match std::fs::read_to_string(&file) {
                Ok(t) => fs_contents(t),
                Err(e) if e.kind() == NotFound => fs_missing(),
                Err(e) if e.kind() == PermissionDenied => fs_denied(),
                Err(e) => fs_failed(e),
            },
            FsRequest::Write(t) => match std::fs::write(&file, t) { Ok(()) => fs_written(), Err(e) =>
            fs_failed(e) },
        };
        if !cancel.load(Ordering::Relaxed) {
            inbox.send(Delivery::Response { path: p, value: response });
        }
    });
}

fn slot_removed(&mut self, path: &Path, el: &ElementValue, _: &mut Cx) {
    match el.tag {
        Tag::FS_FILE => { if let Some(c) = self.inflight.remove(path) { c.store(true, Ordering::Relaxed); }
        }
        Tag::FS_CONTENTS => {
            let w = self.watched.get_mut(el).unwrap();
            w.slots -= 1;
            if w.slots == 0 { let _ = self.watcher.unwatch(el.field("path").as_text());
            self.watched.remove(el); }
        }
        _ => {}
    }
}

fn slot_created(&mut self, _: &Path, el: &ElementValue, _: &mut Cx) {
    if el.tag == Tag::FS_CONTENTS {
        // the oracle has already been asked, so `watched` has an entry
        self.watched.get_mut(el).unwrap().slots += 1;
    }
}

fn oracle(&mut self, el: &ElementValue, cx: &mut Cx) -> Value {
    // Fs.Contents only. Read synchronously; start the watch.
    let file = el.field("path").as_text().to_owned();
    let value = contents_of(&file, self.cap);
    if let Err(e) = self.watcher.watch(Path::new(&file), NonRecursive) {
        cx.diagnostic(Diagnostic::Component { tag: el.tag, message: format!("watch {file}: {e}") });
    }
    self.watched.insert(el.clone(), Watched { value: value.clone(), slots: 0 });
    value
}

fn gather(&mut self, cx: &mut Cx) {
    while let Ok(ev) = self.changes.try_recv() {
        for p in ev.paths {
            let el = fs_contents_element(&p);
            if let Some(w) = self.watched.get_mut(&el) {
                let now = contents_of(&p, self.cap);
```

```
                    if now != w.value { w.value = now.clone(); cx.push(&el, now); }
                }
            }
        }
    }

    fn quiescent(&self) -> bool { self.inflight.is_empty() }
    ...
}

/// What an Fs.Contents slot holds for this path: the text, or the reason there is none.
fn contents_of(path: impl AsRef<std::path::Path>, cap: u64) -> Value {
    let mut bytes = Vec::new();
    match std::fs::File::open(path).and_then(|f| f.take(cap + 1).read_to_end(&mut bytes)) {
        Err(e) => unavailable(fs_error(&e)),
        Ok(_) if bytes.len() as u64 > cap => unavailable(FsError::TooLarge),
        Ok(_) => match String::from_utf8(bytes) {
            Ok(t) => known(t),
            Err(e) => unavailable(FsError::Other(e.to_string())),
        },
    }
}

fn fs_error(e: &std::io::Error) -> FsError {
    match e.kind() {
        std::io::ErrorKind::NotFound => FsError::NotFound,
        std::io::ErrorKind::PermissionDenied => FsError::Denied,
        _ => FsError::Other(e.to_string()),
    }
}
```

The synchronous read in `oracle` is the whole reason a file viewer shows the file in its first occurrence instead of flashing "unavailable" for one frame. The watch is started in the same call, so that a change between the read and the first turn is not lost: `notify` delivers it to the channel (from memory), `gather` picks it up at the next turn, re-reads, and pushes if the contents differ. The read stops at `cap + 1` bytes, so a huge file costs one capped read and not its size, and every failure maps to one of the four reasons: a missing path is `NotFound`, `PermissionDenied` is `Denied`, a file past the cap is `TooLarge`, and anything else (a directory, bytes that are not text, a device that errors) is `Other` with the operating system's message. The reason is a value like any other, so a file that is deleted pushes `Unavailable NotFound`, one that is created pushes `Known`, and a program that shows "permission denied" differently from "no such file" matches on the reason. On a platform without a usable watch the component falls back to a stat-and-reread in `gather`, comparing modification times; the program cannot tell, except that changes arrive at the next turn rather than causing one.

Whether a changed file should *cause* a turn is the `has_gatherable` question: yes for a watch, because a program showing a file should update when the file changes even if nothing else is happening, so `Files` reports `true` whenever the change channel has something. The main loop's `wait` is then woken by the inbox, which the watcher thread also sends a no-op `Delivery` into for exactly that purpose; a `Delivery::Wake` variant with no payload is the cleanest spelling and the harness drops it after waking.

`Fs.Contents` value identity shows up as the `slots` count: two widgets viewing the same path share one watch, one read, and one push, and the watch stops when the last of them goes. The core's snapshot already shares the value; the component shares the OS resource.

The cancel flag for `Fs.File` is the "cancel if the World can" obligation. A `std::fs::read_to_string` on a pool thread cannot be interrupted, so cancellation here means only that the result is not delivered;

the core would have discarded it anyway as a late result, and the flag only saves the send. A World on an OS with cancellable IO does better; nothing in the contract changes.

`quiescent` is `false` while an `Fs.File` operation is in flight, so a program that fired a read and is waiting for the answer is not ended. A watch does not keep a program alive on its own, which is a choice: a program whose only live thing is a file watch is like a program whose only live thing is a signal handler, and the two could reasonably go either way. The reasoning for the difference is that a program that wants to run until a file changes is rare, and a program that forgot to exit after reading a file is common, and the second should end with code 0.

### `Time.After`, `Time.Every`, `Clock`: deadlines in the loop's wait

```
pub struct Timers {
    inbox: Inbox,
    after: BTreeMap<(Instant, u64), (Path, u64)>,   // Time.After deadlines, keyed by time then sequence; the
token
    every: BTreeMap<Path, Every>,                    // Time.Every slots: period and next deadline
    seq: u64,
    turn_now: Instant,                              // sampled once per turn
    epoch: Instant,
}

struct Every { period: Duration, next: Instant }

impl Timers {
    /// The main loop asks this before blocking.
    pub fn next_deadline(&self) -> Option<Instant> {
        let a = self.after.keys().next().map(|(t, _)| *t);
        let e = self.every.values().map(|e| e.next).min();
        [a, e].into_iter().flatten().min()
    }
}

impl Component for Timers {
    fn tags(&self) -> Vec<Tag> { vec![Tag::TIME_AFTER, Tag::TIME_EVERY, Tag::CLOCK] }

    fn perform(&mut self, path: &Path, _: &ElementValue, request: Value, _: &mut Cx) {
        // Time.After(requests: !Nat, responds: Nat): milliseconds in, the same Nat echoed back
        self.seq += 1;
        let ms = request.as_nat();
        self.after.insert((self.turn_now + Duration::from_millis(ms), self.seq), (path.clone(), ms));
    }

    fn slot_created(&mut self, path: &Path, el: &ElementValue, _: &mut Cx) {
        if el.tag == Tag::TIME_EVERY {
            let period = Duration::from_millis(el.field("period").as_int() as u64);
            self.every.insert(path.clone(), Every { period, next: self.turn_now + period });
        }
    }
    fn slot_removed(&mut self, path: &Path, el: &ElementValue, _: &mut Cx) {
        match el.tag {
            Tag::TIME_EVERY => { self.every.remove(path); }
            Tag::TIME_AFTER => { self.after.retain(|_, (p, _)| p != path); }
            _ => {}
        }
    }

    fn oracle(&mut self, _: &ElementValue, _: &mut Cx) -> Value { instant(self.turn_now - self.epoch) }
```

```
    fn gather(&mut self, cx: &mut Cx) {
        self.turn_now = Instant::now();
        cx.push(&ElementValue::nullary(Tag::CLOCK), instant(self.turn_now - self.epoch));
        // Every expired Time.After fires; the earliest Time.Every per slot fires, once.
        while let Some((&(t, s), _)) = self.after.first_key_value().filter(|((t, _), _)| *t <= self.turn_now) {
            let (path, ms) = self.after.remove(&(t, s)).unwrap();
            cx.respond(&path, Value::nat(ms));
        }
        for (path, e) in &mut self.every {
            if e.next <= self.turn_now {
                cx.respond(path, tick());
                while e.next <= self.turn_now { e.next += e.period; }    // skip missed ticks, keep phase
            }
        }
    }

    fn quiescent(&self) -> bool { self.after.is_empty() && self.every.is_empty() }
    ...
}
```

There is no timer thread and no timer wheel: timers are deadlines, the main loop's `wait` takes the earliest as its timeout, and `gather` fires everything that has expired as responses for the turn that follows. That is enough for a CLI World, where timer resolution is milliseconds and there are tens of timers, not thousands; a World with thousands of timers replaces the `BTreeMap` with a wheel and nothing else changes.

`Clock` is a state element pushed once per turn at most, and `turn_now` is sampled once at the start of `gather`, so every occurrence of a turn sees the same instant, and every timer that fires in a turn fires against the same `now` the clock shows. A program that reads `$clock` in the occurrence a `Time.Every` tick arrives reads the time the tick was fired at, through a tag, before the push; after the occurrence it holds the pushed value. The stopwatch in chapter 2 depends on exactly this.

Two timers expiring in the same `wait` are two responses in one batch; under `OneAtATime` they are two turns, in deadline order, each with its own clock sample. Under `Drain` they would be one occurrence, which is legal (two different slots) and honest (they did expire together, as far as a millisecond clock knows). The CLI World uses `OneAtATime` because its programs are written as if each input is a turn, and the chunk-and-end reasoning of chapter 2's line counter is easier to trust when it is also true of timers.

A `Time.Every` tick that was missed, because the process was stopped or a turn took longer than the period, is skipped rather than delivered late in a burst. The phase is kept. This is a World policy, and a World that wants to deliver every tick keys them differently; the program sees ticks either way and the language promises nothing about their spacing.

## Exit when nothing can happen

A CLI run ends on `Process.Exit`, or when nothing can ever cause another occurrence. The second is `Step::Quiescent`, and it is a conjunction over components: the inbox is empty, stdin has nothing more to deliver (no `Cli.Stdin` slot, or `End` has been delivered), no signal slot exists, no `Fs.File` operation is in flight, no timer is set. A program with none of those has a graph with no sources left, and a graph with no sources cannot change, so ending it with code 0 is not a policy; it is reading the situation correctly. The alternative, waiting for an explicit `Process.Exit` forever, turns a forgotten exit into a hang, and the forgotten exit is the common bug.

The one judgment call in the conjunction is which slots count as sources, and it was made per component above: stdin, signals, in-flight requests and timers keep a program alive; arguments, environment, the clock and file watches do not. Each is a line in a component's `quiescent`, and changing one is changing one line.

### The line counter under the real OS

Chapter 2 walked the line counter occurrence by occurrence under a hypothetical World. Here is what the CLI World does with it, with `printf 'one\ntwo\nthree\n' | wc_prism` and the reader's chunk size left at the default.

Occurrence 0 builds three slots. `start` performs no requests (nothing fired) and diffs the list against nothing: `slot_created` for `0: Cli.Stdin` starts the reader thread, and the other two are request elements with nothing to start. The loop calls `step`: nothing queued, not quiescent (a slot is listening and `End` has not come), `Idle`; `wait` blocks on the inbox with no deadline.

The reader thread reads 14 bytes in one `read(2)`, because `printf` wrote them in one `write(2)` and the pipe delivered them together, and puts `Chunk "one\ntwo\nthree\n"` in `held`, then wakes the loop. `wait` returns; `step` finds the held chunk gatherable and turn 1 opens with it as one response under `OneAtATime`; occurrence 1 runs; `$lines` becomes `3`. No requests. The list is unchanged; nothing to start or stop. `Idle` again.

The reader reads 0 bytes and holds `End`. Turn 2, occurrence 2: `!end` fires; `Cli.Stdout` fires `"3\n"`; `Process.Exit` fires `Code 0`. The core performs them in list order: `Stdio::perform` writes `"3\n"` to the buffer, then records exit 0. `step` returns `Exit(0)`; main flushes and exits.

Had the input been typed at a terminal, each line would have been its own chunk and the run would have had four turns before `End`, and the output would be the same, which is the point of the chunk-independence a stream program has to have and chapter 2's `@Lines` provides.

## A tokio-based World skeleton

A CLI World can get by with threads because it has four blocking things. A World with a thousand sockets cannot, and the Rust answer to a thousand sockets is tokio. The question is where tokio goes, and the answer is: on the IO side, never on the propagation side.

Suppose the turn loop were an `async fn`, components were `async`, and `perform` awaited the operation and returned the response. Then a turn would be suspended, with the runtime's state live, while a socket was read; a second input arriving in the meantime would either wait (serialising all IO behind the slowest operation) or start a second turn (two turns in flight on one runtime, which the transaction model forbids). And on any platform with a GUI main thread, there is no awaiting inside a callback at all. The CLI World's design avoids both: a turn is synchronous and brief; an IO task is a future that ends by sending a `Delivery` to the inbox; the loop waits on the inbox and on nothing else.

```
pub async fn run_tokio<R: Runtime>(mut harness: Harness<R>,
                                   mut inbox: tokio::sync::mpsc::Receiver<Delivery>,
                                   deadlines: Arc<Deadlines>) -> i32
{
    if let Step::Exit(c) = harness.start() { return c; }
    loop {
        // 1. Wait for the first delivery, or the earliest timer.
        let first = tokio::select! {
```

```
            d = inbox.recv() => match d { Some(d) => Some(d), None => return 0 },
            _ = deadlines.sleep_until_next() => None,
        };
        // 2. Batch what else is ready now. One response per path per occurrence;
        //    a second to the same path stays queued for the next turn.
        let mut batch: Vec<Delivery> = first.into_iter().collect();
        let mut paths = BTreeSet::new();
        for d in &batch { if let Delivery::Response { path, .. } = d { paths.insert(path.clone()); } }
        loop {
            match inbox.try_recv() {
                Ok(Delivery::Response { path, value }) if !paths.insert(path.clone()) => {
                    harness.hold(Delivery::Response { path, value });    // next turn
                    break;                                                // keep order: stop batching
                }
                Ok(d) => batch.push(d),
                Err(_) => break,
            }
        }
        // 3. One turn, synchronous, on this task.
        harness.push_all(batch);
        match harness.step() {
            Step::Ran(_) | Step::Idle => {}
            Step::Exit(code) => return code,
            Step::Quiescent => return 0,
            Step::Error(_) => return 1,
        }
    }
}
```

The components of a tokio World are the same trait with `tokio::spawn` where the CLI World had `std::thread::spawn`:

```
impl Component for Sockets {
    fn slot_created(&mut self, path: &Path, el: &ElementValue, _: &mut Cx) {
        // Net.Socket(address, requests: !Bytes, responds: Received Bytes | Closed,
        //            state: Connecting | Connected | Disconnected Reason)
        let addr = el.field("address").as_text().to_owned();
        let (inbox, p, el) = (self.inbox.clone(), path.clone(), el.clone());
        let (tx, mut rx) = tokio::sync::mpsc::channel::<Bytes>(16);       // outbound, bounded
        self.outbound.insert(path.clone(), tx);
        let handle = tokio::spawn(async move {
            let stream = match TcpStream::connect(&addr).await {
                Ok(s) => { inbox.send(Delivery::Push { element: el.clone(), value: connected() }); s }
                Err(e) => { inbox.send(Delivery::Push { element: el, value: disconnected(e) }); return; }
            };
            let (mut rd, mut wr) = stream.into_split();
            let mut buf = vec![0u8; 4096];
            loop {
                tokio::select! {
                    n = rd.read(&mut buf) => match n {
                        Ok(0) | Err(_) => { inbox.send(Delivery::Response { path: p, value: closed() });
                        return; }
                        Ok(n) => inbox.send(Delivery::Response { path: p.clone(), value: received(&buf[..n])
                        }),
                    },
                    Some(bytes) = rx.recv() => { if wr.write_all(&bytes).await.is_err() { return; } }
                }
            }
        });
        self.tasks.insert(path.clone(), handle);
```

```
    }

    fn perform(&mut self, path: &Path, _: &ElementValue, request: Value, _: &mut Cx) {
        if let Some(tx) = self.outbound.get(path) {
            let _ = tx.try_send(request.as_bytes());      // full: backpressure, below
        }
    }

    fn slot_removed(&mut self, path: &Path, _: &ElementValue, _: &mut Cx) {
        if let Some(h) = self.tasks.remove(path) { h.abort(); }    // cancels the connection
        self.outbound.remove(path);
    }
    ...
}
```

The `inbox.send` here is the tokio channel's `blocking_send` (from memory) or an unbounded send, and that choice is backpressure. If the inbox is bounded and a reader task blocks on sending, the socket is not read until the loop drains, which is the correct behaviour for a program that is slower than its input: the kernel's receive buffer fills, TCP's window closes, and the sender slows down. That is backpressure, and a bounded inbox gives a Prism World it for free. The outbound channel in `perform` is bounded too, and `try_send` failing means the program is writing faster than the socket drains; the component's choice above is to drop with a diagnostic, and a World that wants to tell the program would use a tag whose response includes `Backpressured`, at which point the program's own hold is the buffer and the program decides what to drop. Buffering unboundedly inside the World is the one option not taken, because an unbounded buffer is a memory leak with a delay.

Step 2's batching decides what "simultaneous inputs" means. Two deliveries that were both ready when the loop woke are, as far as the World can know, simultaneous; delivering them in one occurrence tells the program they were, and the program's `|` merges and the comma-style reasoning apply. Delivering them in two turns tells the program one came first, which the World does not know either. Both are honest; the skeleton batches because a server that gets two requests at once should see two requests at once, while the CLI World does not because its programs are written one input at a time. The rule that one slot gets one response per occurrence is enforced either way, and the skeleton stops batching at the first repeat rather than skipping over it, so that the order of deliveries to one path is kept.

The `Deadlines` object is the tokio spelling of the CLI World's `next_deadline`: the `Timers` component writes its earliest deadline into it, and `sleep_until_next` is a `tokio::time::sleep_until` that is recreated whenever the earliest changes. Nothing else about `Timers` changes.

A World that mixes a GUI main thread and tokio IO, which a desktop World does, runs this loop's select on a tokio thread that does nothing but forward: it receives from the inbox and calls the platform's wake (winit's `EventLoopProxy::send_event`, from memory), and the main thread's event handler calls `step`. The turn runs on the main thread, where the platform wants it; tokio never touches the runtime.

## The GUI World

A click at (50, 10) reaches the software rasteriser below as one winit event. The World turns it into one response, `Pointer (Down ((50, 10)), Left)`, addressed to the path of the program's `Gui.Window` slot, runs a turn, and paints the `Gui.Scene` the turn left in the list. It did not look for a button, and it could not have: no button is in the list. The library above it hit-tests the point against the layout

it drew, which is the only place that layout exists. Everything this section builds is the same small job done for a different platform event.

GUI is not a World concept. There is one GUI vocabulary, five tags, and any World that can put pixels on a screen and read a pointer and a keyboard can speak it. Layout, hit-testing, focus traversal, text editing, buttons, lists and scrolling are Prism interceptors over those tags, in a library that is part of the conformance suite, so that they run under the Test World too. The World's part is small on purpose, because everything the World does is something a test cannot see.

## The vocabulary

Chapter 7 declares the vocabulary and the event types, and the Rust side reads them as follows. A window and the one place events for it go, keys included; the picture, as a list of plain `Draw.*` values; the name of the focused region; text measurement; and a tick per frame.

```
tag Gui.Window of Element:
    title: Text
    size: Extent                    # asked for when the window opens
    responds: WindowEvent             # Resized Extent | CloseRequested | Pointer Pointer | Pressed Key

tag Gui.Scene of Element:
    window: Text                    # the title of the window it paints
    draw: [Draw]                     # Draw.Rect | Draw.Text | Draw.Image | Draw.Clip, in paint order

tag Gui.Focusable of Element:     # presence: no responses; keys arrive at Gui.Window as Pressed Key
    name: Text

tag Gui.Metrics of Element:
    text: Text
    font: Font
    state: Extent                   # width and height of the text; answered synchronously

type Frame = record:
    now: Instant

tag Gui.Frames of Element:
    responds: Frame
```

Only `Gui.Metrics` is a state tag, and none of the five has a `requests` field. A World that speaks this vocabulary therefore performs nothing: it paints a list, answers one question, and delivers events. The negative space is deliberate. There is no region for the World to hit-test, so there is no overlap rule, no disabled-region rule and no pointer capture to get wrong; there is no focus request for the World to grant, so the World and the library cannot hold two opinions about who is focused; and a draw command is a value in a list, so it has no path, no slot and no teardown to track. The clipboard is a separate bare tag, `Clipboard`, which a World offers or does not (chapter 7 says why it is not part of `Gui.*`), and installed fonts are not a tag.

A label whose text changes rebuilds the library's scene: `Gui.Scene` is a presence element with a plain `draw` field, so a new draw list is a fresh slot at the next generation, and the World diffs the new list against the one it painted. The `Gui.Window` slot stays put, which is what keeps a pointer event's address stable across frames.

## A software rasteriser first

The first GUI World draws into a pixel buffer on the CPU and blits it to a window. The choice is deliberate: a software rasteriser has no GPU state, no shader pipeline and no driver, so every bug is a Rust bug, and the whole World is a few hundred lines on top of `winit` for the window and input, `softbuffer` for the blit (its API from memory), and a text rasteriser for glyphs. wgpu (from memory) is a drawing backend with no language consequence and can replace the rasteriser when something is measured to be slow.

```
pub struct Screen {
    inbox: Inbox,
    window: Option<Arc<winit::window::Window>>,
    surface: Option<softbuffer::Surface<..>>,
    fonts: FontCache,                                  // glyph layout and rasterising
    window_path: Option<Path>,                          // the Gui.Window slot: where pointer events, keys and
resizes go
    focusable: Option<Path>,                            // the Gui.Focusable slot, if the list has one: where the
focus ring and the candidate window are shown; keys do not go here
    scene: Vec<Draw>,                                  // the draw list as of the last turn
    damage: Vec<Rect>,
    frame_slots: BTreeSet<Path>,
    window_slots: usize,
}

impl Component for Screen {
    fn tags(&self) -> Vec<Tag> {
        vec![Tag::GUI_WINDOW, Tag::GUI_SCENE, Tag::GUI_FOCUSABLE, Tag::GUI_METRICS, Tag::GUI_FRAMES]
    }

    fn oracle(&mut self, el: &ElementValue, _: &mut Cx) -> Value {
        match el.tag {
            Tag::GUI_METRICS => self.fonts.measure(el.field("text").as_text(), el.field("font")),
            _ => unreachable!("only state tags are asked"),
        }
    }

    fn slot_created(&mut self, path: &Path, el: &ElementValue, _: &mut Cx) {
        match el.tag {
            Tag::GUI_WINDOW => {
                self.window_slots += 1;
                self.window_path = Some(path.clone());
                self.ensure_window(el.field("title").as_text(), el.field("size").as_extent());
            }
            Tag::GUI_FOCUSABLE => self.focusable = Some(path.clone()),
            Tag::GUI_FRAMES => { self.frame_slots.insert(path.clone()); self.request_redraw(); }
            _ => {}                                    // a Gui.Scene is read from the list in after_turn
        }
    }

    fn slot_removed(&mut self, path: &Path, el: &ElementValue, _: &mut Cx) {
        match el.tag {
            Tag::GUI_WINDOW => {
                self.window_slots -= 1;
                if self.window_slots == 0 { self.close_window(); self.window_path = None; }
            }
            Tag::GUI_FOCUSABLE => if self.focusable.as_ref() == Some(path) { self.focusable = None; },
            Tag::GUI_FRAMES => { self.frame_slots.remove(path); }
            _ => {}
        }
```

```
    }

    fn perform(&mut self, _: &Path, _: &ElementValue, _: Value, _: &mut Cx) {}   // no tag here has requests
    fn gather(&mut self, _: &mut Cx) {}
    fn quiescent(&self) -> bool { self.window_slots == 0 }
}
```

Metrics are answered from the same `FontCache` the painter uses, so a label is measured with the glyphs it will be drawn with. The oracle cannot fail, and the cache is keyed by the element value, as the core's snapshot is, so every slot asking about `("Start", Default)` shares one glyph layout. The push for it fires only when the installed fonts change, which is almost never.

The World's half of the loop is winit's `ApplicationHandler` (the winit names below are from memory). Each platform event becomes zero or more deliveries and a `step`:

```
impl ApplicationHandler<Delivery> for App {
    fn window_event(&mut self, el: &ActiveEventLoop, _: WindowId, event: WindowEvent) {
        let screen = self.screen.borrow();
        let to_window = |value| screen.window_path.clone()
            .map(|path| Delivery::Response { path, value });
        let mut redraw = false;
        let batch: Vec<Delivery> = match event {
            WindowEvent::CursorMoved { position, .. } => {
                self.pointer = position.into();
                to_window(pointer_moved(self.pointer)).into_iter().collect()
            }
            WindowEvent::MouseInput { state, button, .. } =>
                to_window(pointer_button(state, self.pointer, button)).into_iter().collect(),
            WindowEvent::MouseWheel { delta, .. } =>
                to_window(pointer_scroll(self.pointer, delta)).into_iter().collect(),
            WindowEvent::KeyboardInput { event, .. } =>
                to_window(pressed(key_event(event))).into_iter().collect(),   // the library routes it by focus
name
            WindowEvent::Ime(ime) => to_window(pressed(composition_event(ime))).into_iter().collect(),
            WindowEvent::Resized(size) => to_window(resized(size)).into_iter().collect(),
            WindowEvent::CloseRequested => to_window(close_requested()).into_iter().collect(),
            WindowEvent::RedrawRequested => {
                redraw = true;                                          // Gui.Frames fires before the paint
                screen.frame_slots.iter()
                    .map(|path| Delivery::Response { path: path.clone(), value: frame(self.clock.now()) })
                    .collect()
            }
            _ => vec![],
        };
        drop(screen);
        self.harness.push_all(batch);
        self.run_turns(el);
        if redraw { self.screen.borrow_mut().paint(); }
    }

    fn user_event(&mut self, el: &ActiveEventLoop, _: Delivery) { self.run_turns(el); }   // the waker

    fn run_turns(&mut self, el: &ActiveEventLoop) {
        loop {
            match self.harness.step() {
                Step::Ran(out) => { self.screen.borrow_mut().after_turn(&out.list); }
                Step::Idle => break,
                Step::Exit(_) | Step::Quiescent => { el.exit(); break; }
                Step::Error(e) => { self.screen.borrow_mut().show_error(&e); break; }
```

```
            }
        }
    }
}
```

Three things are missing from the handler on purpose. It does no hit-testing: every pointer event goes to the one window path, with the point in it. It keeps no focus: a key goes to the window path like a pointer event, and the library routes it to the input whose name is the focus, or drops it when no input is focused. The `Gui.Focusable` in the list is only read, to show the platform's focus ring. And `Resized` is a response like any other, not a pushed state; the library holds the size and lays out again in the occurrence the response arrives in.

`after_turn` finds the `Gui.Scene` in the turn's list and compares its draw list with `scene`, command by command in paint order. A command that differs damages both its old and its new bounds, a longer or shorter list damages the tail, and a World that would rather repaint the whole window is free to, because the language cannot see which it did. It then looks for the one `Draw.Text` whose `ime_at` is set, which the library sets only on the focused input's run, turns the offset into a position with the font cache and tells the platform where the candidate window goes (`set_ime_cursor_area`, from memory). It requests a redraw if `damage` is non-empty or a `Gui.Frames` slot exists. `paint` clears the damaged rects and replays the `Draw.Rect`, `Draw.Text`, `Draw.Image` and `Draw.Clip` commands in list order, clipped to the damage; a `Draw.Clip` pushes its rect on a clip stack for its nested list and pops it after. The pointer-move coalescing that every GUI does is the harness's `Drain` policy plus a line in the handler that replaces a queued `Pointer (Moved ..)` with the latest one, which promises the program no less than the platform does; clicks, keys and composition events are never coalesced.

Live resize on platforms that call back synchronously during a drag is one turn per callback, with the paint inside the callback; the harness's `step` never blocks, so this is safe.

The error policy is a GUI World's and not the core's: on `Step::Error`, the window stays open, the error is drawn in it, and the next input is a fresh occurrence against the rolled-back runtime. A CLI World exits; a GUI World has a window to show the error in, and a user who can see what went wrong and keep going is better served than one whose window vanished.

## How a native-toolkit wrapper differs

A World that wraps GTK or AppKit does not speak the `Gui.*` vocabulary; it offers higher tags, and the same Prism program targets it through a different library.

```
tag Gtk.Window of Element:
    title: Text
    children: [Element]

tag Gtk.Button of Element:
    label: Text
    responds: Click

tag Gtk.Entry of Element:
    placeholder: Text
    requests: !EntryRequest        # SetText Text | SelectAll
    responds: EntryEvent            # Changed Text | Activated
    state: Text                     # what the native entry holds right now
```

The whole middle of the World differs. There is no draw list and no rasteriser; instead the component keeps a map from path to native widget, and `after_turn` becomes a reconciliation: a slot created

makes a native widget, a slot removed destroys one, and because fields are fixed for a slot's life, a changed label is a removed slot and a created slot, which the component may recognise as the same tree position and the same tag and turn into a `set_label` on the existing native widget rather than a destroy-and-create, keeping the native widget's own state (cursor, selection, scroll position). That is the continuity-by-tree-position policy the specification leaves to Worlds, and here is where it earns its keep: a native entry has state the program cannot see, and the World keeps it across rebuilds by keying on position and tag.

`Gtk.Entry` shows why the native route is the harder one for the language. Its text is state the native widget owns, so the tag has both a `state: Text` the World pushes on every keystroke and a `requests: !EntryRequest` for the program to set it, and the program has two sources of truth to reconcile, which is exactly the situation the Prism text field of chapter 2 avoids by owning the text itself and letting the World see only a scene and a focusable name. The native wrapper is the right World for a program that wants the platform's own widgets, and it is a different World with a different library, and which is better is a library question. The language has an opinion only about the structure: leaves are elements, containment is interception, and `Gtk.Window(children=[[...]])` holding its children as data is the container convention the Test World's depth rule already understands.

## The X11-over-sockets World

The strongest way to show that GUI is not a World concept is to run the GUI library under a World that has no idea what a window is. This World provides sockets and files and nothing else. The X11 protocol, which is bytes over a socket, is Prism library code, and the GUI programs that run under the software rasteriser run under it unchanged.

```
type Address = choice:
    Tcp Text Nat
    Unix Text

type SocketEvent = choice:
    Received Bytes
    Closed

type SocketState = choice:
    Connecting
    Connected
    Disconnected Text

tag Net.Socket of Element:
    address: Address
    requests: !Bytes
    responds: SocketEvent
    state: SocketState
```

The World is the tokio skeleton of the previous section with the `Sockets` component (extended to connect to a Unix socket when the address says so) and the `Files` component, for reading `~/.Xauthority` (that path, and the socket path `/tmp/.X11-unix/X0` below, from memory). Its entire component list is those two plus the core's `Defer`. It has never heard of `Gui`.

## What the library does

`@X11.Display` is an interceptor. From the inside it offers the `Gui` vocabulary: its child creates `Gui.Window`, `Gui.Scene`, `Gui.Focusable` and `Gui.Metrics` slots and gets what those tags promise. From the outside it makes one `Net.Socket` element and one `Fs.Contents` element, and everything between is Prism.

```
@X11.Display: fn @a -> @a = \|@child| @widget:
    !start <- @started
    $auth <- @Element Fs.Contents(path=HomePath ".Xauthority")
    ((result, $slots)) <- intercept @child, !down, $ supply
    ((!sock, $state)) <- @Element Net.Socket(address=(Unix "/tmp/.X11-unix/X0"), requests=!out)

    # The connection: a hold of the parser's state across chunks.
    $conn <- X11.Initial then X11.Step $conn, $auth, $slots, !sock
    let !events = X11.Events $conn, !sock                  # parsed Expose, KeyPress, ButtonPress, ...

    # Up: the setup request first (the World queues bytes until the socket connects),
    # then CreateWindow for the Gui.Window and a PutImage of the rasterised scene
    # whenever the draw list or the connection changes, then replies to Expose.
    let !out = (X11.Setup $auth) on !start
             | (X11.Render $slots, $conn) on (!events | !later)
             | X11.Reply $conn, !events

    # Down: a pointer event and a key both go to the Gui.Window slot,
    # by the list as the occurrence opened. Nothing is hit-tested here.
    !later <- @Element Defer(requests=(unit on !events))
    let !down = (X11.Route $slots, $conn) on !later
    let supply = \|e| match e:
        Gui.Metrics(text=t, font=f): @ $ (X11.Measure t, f)
        ignore: @Element e               # Window, Scene and Focusable are read from $slots; NonGui forwards the
        rest
    @ result
```

`X11.Step` is a pure function from the connection state, the authority file, the current slot list and a received chunk to the new connection state: it parses the setup reply, tracks the sequence numbers, and keeps the window id. `X11.Render` is a pure function from the slot list to the bytes of a `PutImage` request: it takes the `draw` list of the `Gui.Scene` whose `window` names the `Gui.Window`'s title, rasterises the `Draw.Rect`, `Draw.Text`, `Draw.Image` and `Draw.Clip` values into a pixel buffer in Prism (a rasteriser is a pure function over a list of draw values, and glyphs are a bitmap font held as data), and encodes the request. `X11.Route` turns a parsed `ButtonPress` or `MotionNotify` into a `Pointer` response to the `Gui.Window` slot, a `KeyPress` into a `Pressed` response to the same `Gui.Window` slot, a `ConfigureNotify` into `Resized` and the window manager's close message into `CloseRequested`, each keyed by the child's path, which `for` consolidates into `!(Path ==> Response)`. A key with no focused input is dropped by the library above, as in the rasteriser World, and the interceptor does not look at `Gui.Focusable` for routing. `X11.Measure` is the bitmap font's advance table, the same data `X11.Render` draws with, so a label is measured with the glyphs it will be painted with. This interceptor does not produce `Composition` keys, which would need X input methods, and it does not answer `Gui.Frames`: the slot reaches the World, which has no component for it, and `Unsupported` says so. An animated program under this World needs one more component, a clock.

The `Defer` is there for the reason every interceptor needs one: an event arrives on the socket in occurrence n, and the response to a child is a response, which cannot be in the same occurrence as the event that caused it. The interceptor fires `Defer` and routes when it comes back, in n+1 of the same turn; from the outside World's view, one chunk arrived and one turn happened.

Everything else is encoding and decoding bytes, which is a pure function over `Bytes` and so well within the language. The library is longer than it should be because the prelude's `Bytes` operations are thin today: fixed-width integer packing, slicing and padding are written by hand where a fuller `Bytes` module would give them as one call each. That gap is in the prelude, not a limit of purity. The library is long and the interceptor is short, and that is the right proportion.

### What it proves, and what it does not

It proves that the World boundary is where the design says it is. A World provides effects; the GUI library is a set of interceptors over tags; and a program written against the `Gui` vocabulary runs under the rasteriser World, the Test World and the X11 World without a line changed, because in each case something speaks the vocabulary and the program cannot tell what. It proves that a World with one component is a World, and that `Net.Socket`'s combination of a request event, a response event and a state dynamic in one tag is enough to drive a real protocol. It proves that `Defer` is the only "later" an interceptor needs, including for routing input to the window. And it proves the sandbox argument from the other side: an interceptor that offers a window to its child while making only a socket itself is also an interceptor that could offer a fake window, or a logged one, or a window on a remote display, and the child would not know.

It does not prove that this is a good way to build a GUI. A `PutImage` of the damaged region per keystroke over a Unix socket is fine for a text field and bad for a video; the rasteriser in Prism is slower than one in Rust; and X11 is the protocol chosen because it is a bytes-over-socket protocol with a window in it, not because anyone should target it in 2026. The point is what it demonstrates about the boundary, and a World that speaks Wayland or a GPU is a World with a different component set and the same core.

## Conformance tests for the core

World conformance is tested per World, in Rust, against the world core, and the core itself is tested with a fake component whose every method records what it was called with and does what the test says. The tests are the obligations of the specification, one each, with names that are sentences from it.

```
#[derive(Default)]
struct Fake {
    tags: Vec<Tag>,
    calls: Rc<RefCell<Vec<Call>>>,                // everything the core did to it
    oracle_answers: BTreeMap<ElementValue, Value>,
    to_push: Vec<(ElementValue, Value)>,           // delivered at the next gather
    to_respond: Vec<(Path, Value)>,                // delivered at the next gather
}

#[derive(Debug, PartialEq)]
enum Call {
    Created(Path, ElementValue),
    Removed(Path, ElementValue),
    Perform(Path, ElementValue, Value),
    Oracle(ElementValue),
    Gather,
}

impl Component for Fake { /* records every call; answers from the maps */ }
```

The programs under test are hand-written `IrBlueprint`s or compiled from Prism source where the frontend is available; the tests below assume source. Each test is a `World` with a `Fake`, a `Harness` with `Drain`, and assertions on `calls` and on `TurnOutcome`.

```
#[test]
fn a_defer_fired_in_an_occurrence_is_answered_in_the_next_occurrence_of_the_same_turn() {
    let (mut h, fake) = world_with(r#"
        @main:
            !b <- @Element Ui.Button(label="go")
            !later <- @Element Defer(requests=(unit on !b))
            $text <- "waiting" then "done" on !later
            @Element Ui.Label(text=$text)                # lifts to a rebuild: a fresh label slot per change
            @ unit
    "#);
    h.start();
    send(&mut h, "Ui.Button(label=\"go\")", click());
    let out = ran(h.step());
    assert_eq!(out.occurrences.len(), 2);
    assert!(out.occurrences[0].requests.iter().any(|(_, e, _)| e.tag == Tag::DEFER));
    assert!(out.list.iter().any(|(_, e)| e.matches("Ui.Label(text=\"done\")")));
}

#[test]
fn a_turn_with_two_defers_runs_three_occurrences() { /* two independent Defer chains; occurrences == 3 */ }

#[test]
fn a_defer_that_fires_again_after_being_answered_is_an_error() {
    let (mut h, _) = world_with(r#"
        @main:
            !b <- @Element Ui.Button(label="go")
            !later <- @Element Defer(requests=(unit on !b | unit on !later))
            @ unit
    "#);
    h.start();
    send(&mut h, "Ui.Button(label=\"go\")", click());
    match h.step() {
        Step::Error(WorldError::DeferBackwards { when, .. }) => assert_eq!(when.occurrence, 3),
        other => panic!("{other:?}"),
    }
}

#[test]
fn a_late_result_for_a_gone_slot_is_discarded_with_a_diagnostic() {
    let (mut h, fake) = world_with(r#"
        @main:
            !b <- @Element Ui.Button(label="drop")
            $show <- True then False on !b
            rebuild $show:
                match _:
                    True:
                        !s <- @started
                        !r <- @Element Fs.File(path="x", requests=(Read on !s))
                        @ unit
                    False: @ unit
            @ unit
    "#);
    h.start();
    let read = fake.perform_calls().remove(0);           // Perform(path, Fs.File(..), Read)
    send(&mut h, "Ui.Button(label=\"drop\")", click());  // tears the slot down
    ran(h.step());
```

```
    assert!(fake.calls().contains(&Call::Removed(read.0.clone(), read.1.clone())));
    h.push(Delivery::Response { path: read.0.clone(), value: contents("late") });
    let out = ran(h.step_or_idle());                      // nothing to run but the diagnostic
    assert!(diagnostics(&h).iter().any(|d| matches!(d, Diagnostic::LateResult { path, .. } if *path ==
    read.0)));
    assert_eq!(out.occurrences.len(), 0);
}

#[test]
fn a_slot_created_in_the_push_occurrence_reads_the_previous_snapshot_and_receives_the_push() {
    let (mut h, fake) = world_with(r#"
        @main:
            $t <- @Element Clock
            !b <- @Element Ui.Button(label="new")
            $n <- 0 then $n + 1 on !b
            rebuild $n:
                !s <- @started
                $t2 <- @Element Clock
                $first <- (Instant 0) then $t2 on !s           # the tag law: the value before this occurrence
                @Element Ui.Label(text=$"first=$first now=$t2")
                @ unit
            @ unit
    "#);
    fake.answer(clock(), instant(5000));
    h.start();
    fake.push_next(clock(), instant(5100));
    h.push(Delivery::Response { path: find(&h, "Ui.Button(label=\"new\")"), value: click() });
    let out = ran(h.step());
    assert!(out.list.iter().any(|(_, e)| e.matches("Ui.Label(text=\"first=5000 now=5100\")")));
    assert_eq!(fake.oracle_calls(), vec![clock()]);   // asked once, shared by value
}

#[test]
fn a_request_on_a_tag_no_component_accepts_gets_unsupported_in_the_next_turn() {
    let (mut h, _) = world_with_tags(&[], r#"
        @main:
            !s <- @started
            !r <- @Element Http.Get(url="x", requests=(Go on !s))
            $msg <- "" then Describe !r                       # Describe: Unsupported -> "nope", else "ok"
            @Element Ui.Label(text=$msg)
            @ unit
    "#);
    let out0 = ran(h.start());
    assert!(!out0.list.iter().any(|(_, e)| e.matches("Ui.Label(text=\"nope\")")));
    let out1 = ran(h.step());                                 // the Unsupported response, delivered in the next
    turn
    assert!(out1.list.iter().any(|(_, e)| e.matches("Ui.Label(text=\"nope\")")));
}

#[test]
fn a_state_tag_no_component_accepts_is_an_error_in_the_occurrence_the_slot_is_created() {
    let (mut h, _) = world_with_tags(&[], "@main:\n    $v <- @Element Process.Args\n    @ unit");
    match h.start() {
        Step::Error(WorldError::Runtime { error: RuntimeError { kind: ErrorKind::UnsuppliedState, .. }, .. }) =>
        {}
        other => panic!("{other:?}"),
    }
}

#[test]
```

```
fn request_firings_are_performed_in_occurrence_order_then_list_order() {
    // two Cli.Stdout slots, the second earlier in the list than the first fires in a Defer occurrence
    // ... assert fake.perform_calls() order is [occurrence 1: list order..., occurrence 2: list order...]
}

#[test]
fn a_request_from_a_slot_torn_down_in_the_same_occurrence_is_performed_and_its_response_discarded() {
    // a rebuild child that fires Fs.File Write on the click that removes it:
    // Perform is called; the fake's response is a LateResult diagnostic
}

#[test]
fn removals_are_reported_before_creations() {
    // a rebuild generation change; assert the Removed call precedes the Created call in `calls`
}

#[test]
fn two_responses_to_one_path_in_a_batch_become_two_turns_in_order() {
    // push Chunk "a" and Chunk "b" to one Cli.Stdin path; Drain policy; two Ran steps, "a" first
}

#[test]
fn a_turn_with_nothing_queued_does_not_run_and_a_clock_alone_does_not_cause_one() {
    // a program with only a Clock slot; step() is Quiescent, never Ran
}

#[test]
fn an_abort_in_a_defer_occurrence_performs_the_firings_of_the_occurrences_that_committed() {
    // occurrence 1 fires a Cli.Stdout write and a Defer; occurrence 2 cycles:
    // Perform(write) is in `calls`, and step() is Error
}

#[test]
fn pushes_of_an_aborted_first_occurrence_are_delivered_in_the_next_turn() {
    // a click that cycles, with a Clock push in the same batch: after Error, the
    // next turn's first occurrence delivers the push and every Clock slot agrees
}

#[test]
fn an_aborted_occurrence_leaves_the_list_and_the_snapshot_as_they_were() {
    // a program that cycles on the second click; after Error, a third click sees the state after the first
}
```

The body of each is short enough to read in the failure message. The last one is the rollback obligation seen from the World: the core applies a turn's pushes and swaps the snapshot only after `run_occurrence` returns `Ok`, and `slots.last_list` is likewise updated only on success, so a failed occurrence leaves the World's view of the program exactly where it was, and the next turn addresses the list the program still has.

A second group tests the harness: `OneAtATime` takes one delivery per `step`; `Drain` takes all; `wait` returns at the deadline with nothing queued; `Exit` is returned only after the turn's performs; the waker is called once per delivery from another thread. Each is a few lines.

## CLI golden transcripts

The CLI World's tests run the real binary against the real OS. Each case is a directory:

```
tests/cli/line_counter_many/
    program.prism
    args            # one per line
    env             # KEY=VALUE per line, replaces the environment
    stdin           # bytes
    chunk           # optional: max bytes per stdin chunk, for the --stdin-chunk flag
    fs_cap          # optional: max bytes an Fs.Contents read returns, for the --fs-cap flag
    expected.stdout
    expected.stderr
    expected.code
```

The runner compiles the program, spawns `prism-cli --stdin-chunk=N program.prism args...` with the given environment and stdin piped from the file, collects stdout, stderr and the exit code, and diffs. There is no transcript of turns in a golden, because a CLI program's observable behaviour is its output and exit code, and a test that pinned the number of turns would be pinning how the OS split a pipe. Where chunking matters, the `--stdin-chunk` flag makes it deterministic: with `chunk` set to 4, `"one\ntwo\nthree\n"` arrives as four chunks and the line counter's partial-line hold is exercised against the real reader thread.

The cases, and what each pins:

| Case | Pins |
|---|---|
| `line_counter_{empty,one,many}` | one turn per chunk; output after the turn; exit by `Process.Exit` with code 0 |
| `line_counter_split` (`chunk=4`) | the partial line held across chunks; the count is chunk-independent |
| `falls_off_the_end` | a program that prints and never exits ends with code 0 when stdin ends and nothing is in flight |
| `exit_code` | `Process.Exit` with a computed nonzero code; a code over 255 clamped and reported on stderr |
| `two_exits` | the first firing in occurrence-then-list order wins; the second is a `note:` on stderr |
| `output_order` | two `Cli.Stdout` slots firing in one occurrence, in list order; stdout then stderr across turns, each flushed per turn |
| `stdout_before_exit` | a write and an exit fired in one occurrence: the write lands |
| `args_env` | `Process.Args` and `Process.Env` printed from the build turn, with no args, several args, and an empty environment |
| `file_then_stdin` | `Fs.File` read on `@started`, then `Cli.Stdin` built inside a `rebuild`; stdin is not read until the slot exists (the harness checks the pipe is still full after the file read's turn) |
| `unsupported` | a `Ui.Button` at the root: `note:` on stderr naming the chain, and the `Unsupported` response reaching a `Cli.Stdout` write |
| `unsupplied_state` | a `Gui.Metrics` state element at the root: the error on stderr with span, exit nonzero, nothing on stdout |

| Case | Pins |
|---|---|
| `signal_handled` | a `Cli.Signals` slot; the harness sends `SIGINT` after reading a `ready` line from stdout; the program prints `interrupted` and exits 3 |
| `signal_default` | no `Cli.Signals` slot; `SIGINT` kills the process with the OS's status |
| `closed_pipe` | stdout piped to a reader that closes early; `EPIPE` is a `note:`, the program keeps running and exits by its own `Process.Exit` |
| `stdin_slot_dropped` | a `Cli.Stdin` slot removed after the first line and rebuilt after a file read; the chunk read in between reaches the new slot first and no line is lost or repeated |
| `fs_contents_reasons` | `Fs.Contents` on a missing path, an unreadable one, a directory and a file past `--fs-cap`: `NotFound`, `Denied`, `Other` and `TooLarge` printed from the build turn |
| `fs_contents_watch` | an `Fs.Contents` viewer; the harness rewrites the file after reading a `ready` line; the program prints the new contents and exits |
| `cycle_at_runtime` | a program that forms a cycle on the second line of input; the error on stderr with both spans, exit nonzero, the first line's output present |

Timers are not in the goldens. A test that waits on wall-clock time is a test that fails on a loaded machine, and everything a timer does to a program is tested under the Test World with `Send Time.Every(..), tick` and `push Clock = ...`. What the CLI World's timer component does with the OS, that `wait` returns at the deadline, is one of the harness tests above with a deadline of a few milliseconds and a generous tolerance.

The `ready`-line handshake in the signal and watch cases is the one place the goldens synchronise with the program, and it is a handshake the program is written to perform: print `ready` after the slot exists. The harness reads stdout until it sees the line, then acts. That is deterministic because stdout is flushed per turn, and the turn that created the slot is the turn that printed the line.

Every case in the table is also a Prism program that runs under the Test World with a script, where it runs in milliseconds with no process spawned, and the CLI golden is the one layer that checks that the real reader thread, the real buffered writer and the real signal handler do what the Test World's script assumed. When the two disagree, the CLI World is wrong, because the Test World is the specification's reading of what a World is; a disagreement that turns out to be the specification being unclear is the more valuable find, and it gets fixed in the specification.

## Chapter 14

# The Haskell implementation

A specification with one implementation is a description of that implementation. The Haskell implementation is the second reading of the language chapters, done independently, in a language whose FRP library is where most of those chapters' semantics came from. It has its own frontend, written from the book and sharing nothing with the Rust one but the typed-AST text form; its runtime is Reflex's Spider under a thin evaluator that walks the typed AST and builds the Reflex network each widget block describes; its Worlds are a Test World over the same transcript format as the Rust one and a reflex-dom World that puts Prism programs on a screen before the Rust GUI World exists. It proves that the chapters can be read twice and mean the same thing, that Prism's semantics are Reflex's where they overlap, and that the boundary (slots, paths, the run call, state supply, `Defer`) can be put on an engine that was not built for it without changing the engine. It does not prove anything about the Rust runtime's memory, its closure compiler, or its speed, and running it in production is outside this plan, though nothing in its design rules that out. It is kept passing for the same reason a second witness is kept: so that the specification keeps being one.

## What the second implementation is for

You have written a specification, handed it to a second person, and watched them build something that passed every test you had and still disagreed with your implementation on the first program you had not thought to test. The disagreement was never in the tests; it was in a sentence both of you had read and each of you had read differently. That afternoon is the reason this implementation exists, and it decides everything about how it is built.

The Haskell implementation is a reader, not a port. A port of the Rust runtime would reproduce the Rust runtime's reading of every sentence, bugs included, and the differential test between the two would be a test of the port's fidelity rather than of the specification's precision. So the Haskell implementation does not call the Rust frontend, does not load the Rust runtime, does not read the Rust implementation's internal formats, and the person or model writing it is handed the book and the goldens and not the Rust source. It shares two interchange formats: the typed-AST text form, so that the two frontends can be compared on the same program, and the element-list transcript, so that the two whole implementations can be compared on the same test. Everything between those two formats is written twice.

What the second reading checks, concretely:

- That the language chapters are a specification. Every program in the suite is dumped by both frontends and the dumps are diffed; every divergence is a bug in one frontend or a sentence two careful readers read differently, and the second kind is the find that matters, because it goes back into the book as a rewritten sentence and a new law test.
- That the time chapters are Reflex's semantics where the two overlap. The tag law, the hold law, non-prompt switching, `dyn` firing in the rebuild occurrence, the mid-occurrence construction rule: each of these was taken from Reflex, and running Prism on Spider is a check that nothing was

lost in the taking. If a Prism law cannot be implemented on Spider by calling the combinator the law names, either the law is not what Spider does, which is worth knowing, or the law is Prism's own and the evaluator has to add it, which is also worth knowing.
- That the boundary is separable from the engine. Slots, paths, routing against the opened list, state supply through a supplier, the run call as plain data, `Defer`: these are Prism's additions to Reflex, and the claim that they are additions rather than changes is tested by putting them on Reflex without patching Reflex.
- That two runtimes with different internals produce the same transcripts. The Rust runtime is a height-ordered action map with holds committing at the top height; Spider is a subscriber graph with hold assignments at the end of the frame. Both pass the same scripts and print the same transcripts, which is the strongest available evidence that the transcripts are determined by the chapters and not by a runtime.

What it does not check matters as much. It has no ownership tree, no closure compiler, no lint and no values representation, so it says nothing about any of them; and the plan builds it only two Worlds, so it says nothing about the others either.

## Interpret, do not compile

The Haskell implementation interprets the typed AST at runtime. It does not translate Prism to Haskell source and hand the result to GHC. The decision follows from what the implementation is for: an interpreter checks the specification a second time with no second compiler to be wrong in.

Consider what the alternative involves, because it is attractive and the attraction is real. Prism's constructs line up with Reflex's one to one. The counter,

```
@counter: @unit = @widget:
    !increment <- @Button $"Increment"
    !decrement <- @Button $"Decrement"
    let !delta = -1 on !decrement | +1 on !increment
    $count <- 0 then $count + !delta
    @Label "Count: {$count}"
```

printed as Haskell is the program a Reflex programmer would type:

```
counter = mdo
  increment <- button (constDyn "Increment")
  decrement <- button (constDyn "Decrement")
  let delta = leftmost [(-1) <$ decrement, 1 <$ increment]
  count <- holdDyn 0 (attachWith addInt (current count) delta)
  label (("Count: " <>) . showInt <$> count)
```

Every line of the typed AST has a line here. The cyclic group is `mdo`. The tag is `attachWith` over `current count`. The hold is `holdDyn`. Nothing is closure-converted, because Haskell has closures; nothing is boxed at a crossing, because every Haskell value already has one representation; GHC type-checks the output, so a printer that emits a `Dynamic` where an `Event` was meant is caught by a compiler that has been catching that for decades. The output is readable by anyone who has written Reflex, which makes it a golden format a person can review. And `Strict` on the pure modules gives Prism's eager evaluation for free. All of that is true, and it is why compiling to Haskell is the right second thought and the wrong design.

The first problem is in the sentence "GHC type-checks the output". A printer that emits Haskell has to emit Haskell *types*, and Prism's types are not Haskell's. Polymorphic literals resolved by three

built-in traits, tag-indexed slot fields inside a `match` arm, extensible variants with a catch-all, erasure of type variables in tag fields, records ordered by field declaration order, elements ordered by tag name: each of these has a Haskell spelling, and each spelling is a design decision about how Prism's typing maps onto GHC's, which the elaborator has already made once in Prism's own terms. Make it twice and there are two elaborators, and when GHC rejects the output nobody can say without an afternoon's work whether the Prism elaborator was wrong, the printer was wrong, or GHC's inference reached a different conclusion from a program the printer left underdetermined. The second reading now has a third reader in it, and the third reader is the one whose reasoning is hardest to inspect.

The second problem is GHC in the loop. Compiling to Haskell makes GHC a build dependency of every Prism program the Haskell implementation runs: the conformance suite is a few hundred programs, the differential test runs on every change to either implementation and on every change to the book, and each run is a GHC invocation per program. GHCi takes some of the cost away, since the Prism runtime library compiles once and the generated modules are small, but the cost that remains is the one that matters for a test oracle: the time from "the sentence changed" to "both implementations have been checked against it" has a compiler in it. An interpreter loads a typed AST and runs. The whole suite under the Haskell implementation is one process.

The third problem settles it. A compiler from Prism to Haskell is a compiler, with every place a compiler can be wrong: a lift printed as the wrong combinator, a group placed in the wrong `mdo`, a capture that names the wrong binding after a rename. The Rust implementation already has a compiler, with a lint pass and goldens to keep it honest. The point of the second implementation is to have a reading of the chapters that does *not* share the compiler's structure, so that when the two disagree the disagreement is about the chapters. An evaluator that walks a tree and calls `holdDyn` where the tree says `hold` has one place to be wrong per node kind, and that place is a `case` arm a reviewer can read against the sentence it implements. A compiler has a place to be wrong per node kind times every context the node can appear in.

The typed-AST text form is the bridge that makes this cheap. The Haskell frontend prints it; the Haskell evaluator parses it; and the two are separable, so that a typed AST written by hand, or dumped by the Rust frontend, runs under the Haskell evaluator with no Haskell frontend at all. In the order the plan runs, this matters: the evaluator can be built against hand-written typed ASTs and the Rust frontend's dumps before the Haskell frontend passes the layout corpus, and a Rust frontend bug that produces a wrong dump shows up as a wrong transcript under both evaluators, which localises it to the frontend without anyone reading the frontend.

The interpreter gives up GHC checking the output, and speed, and needs neither: the lint pass on the Rust side and the differential test on both do the first job, and the Haskell implementation runs test scripts, where per-occurrence cost is not a problem. The evaluator itself, the world core, the printer and the Worlds are ordinary typed Haskell, and GHC checks those.

## The frontend

The Haskell frontend is a second reading of the chapters on lexical structure, values and types, expressions, and the language implementation, written from those chapters and from the goldens. Its output is the typed AST in the text form the Rust frontend prints, character for character, and the test of the frontend is `diff`.

A cabal project `prism-hs` with these modules:

- `Prism.Lex`: a hand-written lexer over the token set, with the spacing rule, the suffixed operators

(`+~`, `+.`), the comparison operators `< <= > >=` and the keyword operators `then`, `not`, `and`, `or` beside `==` and `!=`, text literals with `{expr}` interpolation (a `{` inside a text literal opens an expression that the lexer scans to its matching `}`, `\{` and `\}` are literal braces, and `$` is always literal text), `;` as a statement separator, the lambda bars distinct from merge, `Gui.*` as a name, a dot and a star, and a span on every token. Each paren and bracket token records whether it touches its neighbour, so that the parser can tell `((` from `( (` and `]]` from `] ]` without the lexer deciding what a double bracket means.

- `Prism.Layout`: the offside rule as the lexical chapter states it, producing indent and dedent tokens, with inline `:` as a plain token the parser interprets, `;` separating statements inside an inline block (`type Pair<a, b> = record: first: a; second: b`), and `else` terminating an inline block. A continuation line that starts with `.Method` closes the call on the line above and takes it as receiver, so `items.Filter IsDone` followed by an indented `.MapValues Label` is `MapValues (Filter items, IsDone), Label`; on one line a spaced `.F` is not a chain step. The layout corpus is the first test it must pass, before the parser exists.
- `Prism.Parse`: a parser (hand-written recursive descent, or megaparsec; the precedence table is small and the decision does not show in any output). The operator table, loosest to tightest: `on` and `|`, `or`, `and`, `not`, comparison, arithmetic, so `not a == b` parses as `not (a == b)` and `a < b and c < d` needs no parentheses. A `match` arm is `pattern if cond:` or `pattern:`, and the guard goes on the arm node. `for k, $item <- $items:` binds the key and the per-key dynamic by name, and `for $items:` is the shorthand that binds `key` and `$_`; the parser fills in those two names, so no later stage sees the shorthand. Array patterns (`[[]]`, `[[head, ..rest]]`, `[[a, b]]`) are parsed with the same double-bracket rule as array literals. `{expr}` pieces of a text literal are parsed as expressions, and a literal that interpolates a dynamic is a dynamic. Paren-less calls taking the rest of their expression (`F a + 1, b` is `F(a + 1, b)`), the comma rule, a single-paren group whose head is a call taking all its commas (`(G b, c)` is `G(b, c)`), tuples, tuple types and tuple patterns in double parens (`((a, b))`, `((A, B))`, `((x, y)) <- ...`), array literals in double brackets (`[[1, 2, 3]]`, `[[]]`) with single `[i]` always indexing and array types staying `[A]`, function types as `fn A, B -> C` with parens only around a function-typed argument or result, method form (`x.F y` is `F x, y`), a trailing lambda absorbing the rest of the line, sigils, blocks, patterns, tag declarations, `intercept`, and the `test` form with its steps.
- `Prism.Resolve`: names and scopes, the pronoun and its sigiled forms, the pronoun shorthand wherever `_` is in scope (prefix `.x` is `_.x` and prefix `[i]` is `_[i]`, so `\.done` and `\[0]` resolve, as does `F .x` inside a `for` body; with no `_` in scope it is an unbound-pronoun error), tag namespaces, and binding groups: a dependency analysis over each widget block into strongly connected groups in dependency order, with each edge labelled prompt or delayed.
- `Prism.Elab`: Hindley–Milner on the value layer with the three traits, the functor layer read off the sigils by the lifting rule, numeric types by spelling, `for` consolidation by result kind, tag field types from declarations, the build-time or occurrence-time mark on every binding, the cycle check over the labelled groups, and exhaustiveness (a guarded arm does not count toward it; array patterns do, so `[[]]` with `[[head, ..rest]]` is exhaustive). Its output is `Prism.Typed`'s tree.
- `Prism.Typed`: the typed AST as a Haskell data type, one constructor per keyword in chapter 12's grammar table: `dmap`, `tag … [$count]`, `hold_first`, `element … responds`, the trait-resolved operators (`Num.Add[Int]`, `ToText[Int]`), `switch`, `join`, `rebuild`, `for` with its can-fire-at-birth flag, and the rest of that table.
- `Prism.Typed.Text`: the printer and the parser for the text form, from one definition, so that `parse . print` is the identity on every tree and `print . parse` is the identity on every golden. The type-variable naming rule (by first appearance, left to right, top to bottom) and the let-generalisation rule (at `let` of a lambda only) are implemented here and in `Prism.Elab` respectively, and both

are stated in the language implementation chapter because the text form exposes them.
- An executable `prism-hs` with `dump`, `test` and later `run` subcommands.

The frontend's tests are the layout corpus; every golden from the Rust frontend, as a diff between `prism dump` and `prism-hs dump`; and the negative goldens, compared on the error kind and the spans, with the message prose allowed to differ since a message is not a meaning.

The dump is where the second reading pays. Here is what the Haskell frontend prints for the counter in chapter 12's file, which must be byte-identical to what the Rust frontend prints (the spans are that file's, where `@Button` and `@Label` are defined above the counter):

```
@counter: @unit                                                                  @11:1
  !increment: !Click [build] g1 = bind @Button(const "Increment")                @12:5
  !decrement: !Click [build] g2 = bind @Button(const "Decrement")                @13:5
  !delta: !Int [occ] g3 = leftmost [map !decrement \|ignore| -1:Int, map !increment \|ignore| +1:Int]   @14:9
  $count: $Int [build] g4 = placeholder                                          @15:5
  $count.hold: $Int [build] g4 = hold 0:Int (tag !delta [$count] \|d, c| Num.Add[Int] c, d)   @15:5
  close $count $count.hold                                                       @15:5
  _: unit [build] g5 = bind @Label(dmap [$count] \|c| cat ["Count: ", ToText[Int] c])   @16:5
  @ _
```

Every decision the two frontends could make differently is on the page. If the Haskell elaborator read the lifting rule as "a dynamic and an event give a tag" but placed the tag on the other operand, the `$count.hold` line in `g4` differs. If it generalised the `let` in `g3` (it must not; `!delta` is not a lambda), the type differs. If its resolver put `$count` in a group with `!delta`, the group numbers differ. If its inference named a type variable by a different rule, a polymorphic golden differs. None of these are runtime questions, and all of them would otherwise surface as a transcript divergence three layers down, which is why the dumps are diffed first.

The procedure for a dump divergence is fixed and is the same as for a transcript divergence. Either one frontend has a bug, in which case the golden says which and the frontend is fixed; or the sentence the two frontends read is ambiguous, in which case the sentence is rewritten to admit one reading, a golden is added that pins it, and both frontends are brought to it. The Haskell frontend is never brought to the Rust frontend's behaviour without a sentence to point at. Hindley–Milner is where this will happen most: where generalisation occurs, whether a monomorphic group's variables are generalised after the group, how a literal's type is pinned by an operand three lines away. Each of these is a sentence in the language implementation chapter, and a divergence that traces to a choice that chapter does not state is a missing sentence before it is a bug.

# The evaluator

The evaluator is a function from a typed AST and a widget name to a Reflex network. It is the whole of the Haskell runtime. Everything else is Reflex, and the sections below say which Reflex, node kind by node kind.

## Values

The elaborator has already checked every type, so the evaluator does not ask Haskell to check them again. One Haskell type holds any Prism value:

```
data Val t
  = VUnit
  | VBool   !Bool
  | VNat    !Word64
```

```
  | VInt     !Int64
  | VFloat   !Double
  | VText    !Text
  | VOpt     !(Maybe (Val t))
  | VTuple   ![Val t]
  | VArray   !(Seq (Val t))
  | VRecord ![(Name, Val t)]                -- fields in declaration order
  | VCon     !Name !Int ![Val t]            -- closed choices; the Int is the declaration index
  | VMap     !(Map (Val t) (Val t))          -- Ord on Val is hand-written (below)
  | VElem    !ElementValue                   -- tag plus plain fields; Ord by tag name, then fields
  | VFun     ![Val t] -> Val t              -- pure functions are Haskell closures
  | VEv      !(Event t (Val t))
  | VDyn     !(Dynamic t (Val t))
  | VWidget !(PrismM t (Val t))             -- runs at build, in the widget monad
```

This is the Rust runtime's one-word value seen from the other side: there, a `Plain` plus the static type at each downcast; here, a sum whose projections (`asEv`, `asDyn`, `asInt`) fail with "elaborator bug" when the tree lied. A typed alternative would turn the typed AST into a GADT with each expression's type hidden in an existential and recovered at every binding with `eqTypeRep`. That re-proves what the elaborator already proved, in a type system that is not Prism's. The untyped `Val` keeps the evaluator one `case` per node kind and keeps the type questions in the frontend, where the goldens are.

`Int` is `Int64` and `Nat` is `Word64`, so that overflow happens at the same number under both implementations; the bare arithmetic nodes are checked, and overflow raises the occurrence's abort with kind `Overflow` (below); `Checked.*` and `Wrap.*` are prelude functions the evaluator binds to `Int64`/`Word64` operations that return `Maybe` and wrap respectively. `Float` is `Double`, and `/` on it is IEEE division that never fails, while `/` on `Nat` and `Int` returns `VOpt Nothing` for a zero divisor. `Ord (Val t)` is written by hand so that a `VMap` can be keyed by any ordered plain value (text, numbers, tuples, arrays, records, elements), which is what `for` over a `Text ==> Item` needs. Two of the derived instances Haskell would give are wrong for Prism and are not used. Derived `Ord` on `Double` is the partial IEEE comparison, under which a NaN is neither less nor greater than anything and a `Map` keyed by it corrupts itself; `Float` is ordered by IEEE totalOrder instead (negative NaN, negative infinity, the negative numbers, `-0.0`, `0.0`, the positive numbers, positive infinity, positive NaN), implemented on the bit pattern of the `Double`, while `Eq` stays IEEE equality. And derived `Ord` on a constructor name would order a closed `choice` alphabetically; a closed choice is ordered by the declaration order of its alternatives, then by payload, so `VCon` holds each alternative's declaration index and the instance compares the index first. Records compare structurally. A record keeps its fields as a list in declaration order rather than a `Map Name`, because a `Map` would order a record by field *name*; two records of one type have the same names in the same positions, so the derived `Ord` on the list is lexicographic in declaration order, which is the language's record ordering (Rust's derived `Ord`), and the elaborator has already rejected a record key with an unordered field. Equality on elements is tag plus plain fields, with any request event excluded, which is the identity rule for state; `Ord ElementValue` is written by hand to match, comparing tag names first and then the plain fields in declaration order, so that elements can key sorted maps. Events, dynamics, blueprints and functions have no `Eq`, and the elaborator guarantees they are never compared; if a `VFun` reaches `==`, the projection fails as above.

## The monad stack

A widget block runs in `PrismM t`, which is what `@A` corresponds to:

```
type PrismM t = ReaderT (Env t) (PrismBase t)

type PrismBase t =
  DynamicWriterT t (Map Path (Slot t))          -- the element list
    (EventWriterT t (Seq (Path, Val t)))        -- request firings this occurrence
      (PerformEventT t (SpiderHost Global)))    -- the host frame; Adjustable
```

Each layer is there for one law.

The `DynamicWriterT` is the element list. Reflex's `MonadDynamicWriter` documentation says (from memory) it "efficiently collects 'Dynamic' values using 'tellDyn' and combines them monoidally to provide a 'Dynamic' result", and a `Map Path Slot` under union is the monoid, with paths unique so union never has to choose. A slot created in occurrence n is told in n and is in the list after n; a `dyn` that replaces its block replaces the block's told dynamics, through the writer's `Adjustable` instance, so the old block's slots leave the list in the rebuild occurrence. The list is an ordinary `Dynamic`, which is the sentence "the element list of a widget is an ordinary dynamic" implemented by choosing the transformer whose output is one.

The `EventWriterT` is request collection. A slot whose tag has a `requests` field tells its request event at creation, keyed by its path; the run call reads the writer's event in the read phase and groups it by path. The writer's own merging handles the case the mid-occurrence construction rule makes awkward: a block built in n whose request fires in n (`Read on !started` in a newborn row) is told in n, and `EventWriterT`'s `runWithReplace` instance includes a newborn's tells in the frame of its birth (from memory). How it does that is the subject of the `for` section, because `for` needs the same thing and borrows the same mechanism.

The `PerformEventT` is in the stack for one reason: its `Adjustable` instance. `runWithReplace`, which `rebuild` is, and `traverseDMapWithKeyWithAdjust`, which `for` is, are methods of `Adjustable`, and the host frame does not have an instance; `PerformEventT` over the host frame does. The evaluator never calls `performEvent`. A Prism program performs nothing; the World does, after the occurrence commits, from the request firings in the run call's output. The review checklist has a line for this: no `performEvent` anywhere in `Prism.Eval`.

`SpiderHost Global` is the bottom for the host's sake: trigger creation, `fireEventsAndRead`, the read phase. The evaluator is written against `Reflex t` and the transformer classes rather than against `Spider` directly, so that rollback (below) can run a program on a fresh local timeline.

The absences from the stack are as telling. No `PostBuildT`, because `@started` does not use post-build. No `RequesterT` or `QueryT`, because elements are not requests in a transformer and state is not a query; both are slots in the writer. No `StateT` for paths, generation counters or memo tables; those are `IORef`s in the reader, because they are per-node facts that `mfix` must be able to tie through, and a state monad under `mfix` is a knot that bites. And no `IO` reachable from a widget block except through those refs, which is the "there is no `liftIO`" row of the correspondence table made structural.

## The environment

Every slot needs a path, every response needs a fan to select from, every state slot needs a supplier to ask. The environment is where a widget block finds those:

```
data Env t = Env
  { envVars     :: Map Name (Val t)          -- lexical scope
  , envPath     :: Path                      -- prefix for slots made here
  , envBindIx   :: IORef Int                 -- next bind position in this block
```

```
  , envFan      :: EventSelector t (Const2 Path (Val t))  -- responses, by path
  , envOpened   :: Behavior t (Map Path (Slot t))         -- the enclosing list, as opened
  , envSupply   :: Supply t                   -- where a state slot gets its dynamic
  , envPushes   :: Map ElementValue (Val t)  -- this occurrence's pushes (root only)
  , envTags     :: Map Name TagInfo           -- the tag table from the typed AST
  }
```

`envVars` is the scope. Pure functions close over it as Haskell closures; recursive groups extend it through `mfix`. `envPath` and `envBindIx` give every slot its path: a `bind` statement at position i under prefix p makes slots at `p/i`; a `rebuild` adds `#g` for its generation g; a `for` adds `[k]`; a supplied widget runs under `s/n`. The spelling is the transcript's and is the one fact about paths the implementation does not get to choose. `envFan` is the responses event of the nearest enclosing interceptor (the run call's trigger at the root), fanned by path with `fanMap`, so that a slot at path p selects `select envFan (Const2 p)` and nothing is scanned. `envOpened` is the behavior of that interceptor's child list, read only for the unknown-path check. `envSupply` is the nearest supplier context, the heart of state supply. `envPushes` exists only at the root and only during an occurrence, for a cell created mid-occurrence whose value was pushed in that same occurrence.

## Pure expressions

A plain expression evaluates without the monad:

```
evalE :: Reflex t => Env t -> Expr -> Val t
evalE env = \case
  Lit l          -> litVal l
  Var x          -> envVars env ! x
  Lam xs body    -> VFun $ \vs -> evalE (bindAll xs vs env) body
  App f as       -> apply (evalE env f) (map (evalE env) as)
  Op op a b      -> primOp op (evalE env a) (evalE env b)   -- Num.Add[Int] and the other trait operators
  Field e name   -> project name (evalE env e)
  Match s arms   -> matchPlain (evalE env s) arms env
  If c a b       -> if asBool (evalE env c) then evalE env a else evalE env b
  And a b        -> if asBool (evalE env a) then evalE env b else VBool False
  Or a b         -> if asBool (evalE env a) then VBool True else evalE env b
  Text parts     -> VText (foldMap (render . evalE env) parts)
  ...
```

Three absences matter here. It does not type anything; `primOp Num.Add[Int]` is `Int64` addition that aborts with kind `Overflow`, because the elaborator chose `Num.Add[Int]`. It does not decide lifting; `map` and `tag` are explicit nodes, so a `Var` is always a plain value and an `App` a plain application. It is strict: `litVal`, `apply` and `primOp` force their arguments, which is Prism's evaluation order (eager except for functions) as a property of three functions. The exceptions are `If`, `And` and `Or`, which are their own cases so that the operand not needed is never evaluated, and an overflow in it does not abort. `not` is an ordinary strict operator node; its precedence is settled in the parser and the evaluator never sees it.

`matchPlain` tries arms in order. An arm matches when its pattern does and, if it has a guard, the guard evaluates to `True` in the environment the pattern extended; a failed guard falls through to the next arm. Array patterns match on the `Seq`: `[[]]` on empty, `[[a, b]]` on length two, and `[[head, ..rest]]` on a non-empty array, with `rest` the tail. `If c a b` evaluates the branch the condition selects (both branches are always present); the lifted forms over a dynamic condition are `map` nodes over that function like any other. The one laziness the evaluator relies on is the knot of a recursive group, where a placeholder's value is demanded only inside an occurrence-time closure.

## Events and dynamics

The typed AST has every lift explicit, so the evaluator does not lift; it maps the node to the combinator:

```
MapE f e       -> VEv  (apply1 (evalE env f) <$> asEv  (evalE env e))
MapD f d       -> VDyn (apply1 (evalE env f) <$> asDyn (evalE env d))
Map2D f a b    -> VDyn (zipDynWith (apply2 (evalE env f)) (asDyn a') (asDyn b'))
Const v        -> VDyn (constDyn (evalE env v))
Leftmost es    -> VEv  (leftmost (map (asEv . evalE env) es))
Never          -> VEv  never
Tag d e f      -> VEv  (attachWith (apply2 (evalE env f)) (current (asDyn d')) (asEv e'))
Updated d      -> VEv  (updated (asDyn (evalE env d)))
FilterE f e    -> VEv  (fmapMaybe (asOpt . apply1 (evalE env f)) (asEv (evalE env e)))
```

Four rows deserve a sentence.

`Tag` is the only place `current` appears in the evaluator's expression cases. Reflex's `hold` documentation states the law the tag node implements: "if the occurrence that sets the 'Behavior' (or one that is simultaneous with it) is used to sample the 'Behavior', it will see the **old** value of the 'Behavior', not the new one." `attachWith f (current d) e` reads the old value, and that is the tag law. A reviewer greps the evaluator for `current` and expects to find it here and in two other named places (below), and nowhere else.

`Map2D` is the change law. `zipDynWith` produces a dynamic that changes in any occurrence either operand changes, with both new values, and changes once, which is glitch freedom as Reflex provides it. Three or more dynamics are nested `zipDynWith`s, which the elaborator has already arranged since the typed AST has binary nodes.

`Updated` is `updated`, the change event of a dynamic, firing with the new value in every occurrence the dynamic changes and never at build. Nothing more is needed: `!Updated` is a plain prelude function in Prism and a plain projection in Reflex, and the evaluator does not wrap it in a widget.

`Distinct` is `holdUniqDyn`, which is monadic because it needs a hold, so it is a build-time node and appears in the statement cases rather than here. Its definition in Reflex is (from memory) `attachWithMaybe` over `current d` and `updated d`, holding the result, so it fires only when the new value differs from the old; that is exactly `$Distinct`'s law and the `current` inside it is Reflex's, not the evaluator's.

Lifting over `?` is at the plain layer and never reaches the monad: an `App` on optionals is an applicative `Maybe` inside `apply`, and `match` on `?` is a `matchPlain` arm.

## Holds and recursive groups

A widget block is a list of statements in group order, each extending the environment, and the block's result is an expression over the final environment. Groups with a placeholder are knots:

```
evalBlock :: Env t -> Block -> PrismM t (Val t)
evalBlock env0 (Block groups result) = do
  env <- foldM evalGroup env0 groups
  pure (evalE env result)

evalGroup :: Env t -> Group -> PrismM t (Env t)
evalGroup env (Group stmts placeholders) = mdo
  env' <- foldM (evalStmt (bindAll placeholders (lookupAll placeholders env') env)) env stmts
  pure env'
```

The placeholders are the names the resolver found used before their binding, which the cycle check has already passed through a delayed edge. They are bound in the environment to their own eventual values through `mdo`, and nothing else in the block is in the knot. The `close` line in the text form is where the knot's value is demanded; everything between the `placeholder` and the `close` sees the name through the knot.

This works because of where the delayed edges are. `$count` reaches its own `holdDyn` only inside the closure `attachWith` runs on each occurrence, and Spider's `hold` does not look at its event during build; the source says so in a comment above `hold`: "hold cannot examine its event until after the phase is over", and the implementation defers the subscription with `defer $ SomeHoldInit h`. A `!Switch` reaches its held inner event through a behavior read at frame end. A tag reads a behavior inside a push closure. So every legal cycle passes through something Spider resolves lazily, and `mfix` ties the rest. Reflex's own `foldDyn` is the same knot: a `rec` block whose push closure samples the hold it feeds.

If the elaborator were wrong, and a placeholder were read promptly during build, the knot would be forced early and GHC would report `<<loop>>`, which is the counterpart of the Rust runtime's "cyclic dynamic never closed". If the cycle check were wrong, and a prompt cycle were built, Spider throws an `EventLoopException` whose `Show` instance begins "causality loop detected", which is the counterpart of the Rust runtime's re-entry panic. Both are reported as a `Cycle` error with the group's spans, which the typed AST has.

The statements:

```
evalStmt env acc = \case
  Let x e          -> pure (insert x (evalE env e) acc)
  Bind x w         -> do v <- runWidget env (evalE env w); pure (insert x v acc)
  Hold x v e       -> do d <- holdDyn (evalE env v) (asEv (evalE env e))
                         pure (insert x (VDyn d) acc)
  Distinct x d     -> do d' <- holdUniqDyn (asDyn (evalE env d))
                         pure (insert x (VDyn d') acc)
  Dyn x d f         -> evalDyn env x d f acc
  Switch x ee      -> do e <- switchHold never (asEv <$> asEv (evalE env ee))
                         pure (insert x (VEv e) acc)
  Join x dd        -> pure (insert x (VDyn (join (asDyn <$> asDyn (evalE env dd)))) acc)
  For x m f kind   -> evalFor env x m f kind acc
  Slot x el        -> evalSlot env x el acc
  Intercept x c r s -> evalIntercept env x c r s acc
```

`Hold` is `holdDyn v e`: the initial value is the plain value, the change event is the input event itself, so the dynamic's change is prompt in the occurrence its event fires and its value side commits at the frame's end. A hold built in n whose source fired in n adopts that value at commit, because `subscribeAndRead` hands the hold its event's occurrence in n (Spider's `getHoldEventSubscription` reads the `occ` it is given and schedules the assignment). That is the sentence "a hold built in n whose source fired in n adopts that value at commit" implemented by calling `holdDyn` and nothing else.

`Bind` runs a widget. `runWidget` increments the bind position, extends the path, and runs the `VWidget` under the extended environment; the value it returns is whatever the widget's block returned. Binding a widget application whose result is plain gives a `$` holding the first build's value, and binding a dynamic of blueprints is a `rebuild`, but both of those are already in the typed AST as explicit nodes by the time the evaluator sees them.

### `rebuild` and `@started`

`rebuild $x: block` is Reflex's `networkView`, which reflex-dom exports as `dyn` (from memory):

```
networkView :: (NotReady t m, Adjustable t m, MonadHold t m) => Dynamic t (m a) -> m (Event t a)
networkView child = do
  postBuild <- now
  let newChild = leftmost [updated child, tagCheap (current child) postBuild]
  snd <$> runWithReplace notReady newChild
```

Read it against the `rebuild` law. The block is built when `newChild` fires. `now` is the event that fires in the frame it is created, so at the `rebuild`'s own build `newChild` fires at once; if the dynamic also changed in this frame (a `rebuild` built mid-occurrence over a dynamic that changed in that occurrence), `leftmost` prefers `updated child`, so the block is built once, with the new value, and not with the old value and then again. On every later change `updated child` fires and `runWithReplace` replaces the block in that frame. The returned event is the block's result, firing in the frame of every build including the first, delivered to the caller and invisible to the block. Every clause of the `rebuild` law is a line of this function, which is why the evaluator calls it rather than writing its own:

```
evalDyn env x d f acc = do
  gen <- liftIO (newIORef 0)
  let build v = do
        g <- liftIO (atomicModifyIORef' gen (\n -> (n + 1, n)))
        runWidget (env { envPath = envPath env </> Gen g }) (apply1 (evalE env f) v)
  e <- networkView (build <$> asDyn (evalE env d))
  pure (insert x (VEv e) acc)
```

The generation counter gives rebuilt slots their fresh `#g` paths. Holds inside the block are new holds, because the block is a new `m a` action run afresh; the old block's told dynamics leave the writer when `runWithReplace` replaces them; and the old block's nodes keep propagating until the frame ends, because Spider does not tear anything down mid-frame, which is the sentence about the old node being kept alive until commit.

`@started` is a `rebuild` over a constant, and the evaluator spells it that way:

```
started :: PrismM t (Val t)
started = VEv <$> networkView (constDyn (pure VUnit))
```

That is one build, in the frame the widget is built, firing once with `unit`, and then never, because a constant never updates. The reason to write it as the `rebuild` it is defined as, rather than as the `now` it reduces to, is that the `@started` law and the `rebuild` law are then one code path and one law test covers the frame question for both.

The frame question is why `getPostBuild` is not used. The `PostBuild` class documentation says: " 'PostBuild' represents an action that is notified via an 'Event' when it has finished executing. Note that the specific definition of"finished" is determined by the instance of 'PostBuild', but the intent is to allow 'Behavior's and 'Dynamic's to be safely sampled, regardless of where they were created, when the post-build 'Event' fires." Two things in that sentence disqualify it. "Determined by the instance" means the frame in which it fires depends on which transformer answers it, and at the root in a typical host it is a frame of its own, after construction, while inside a `runWithReplace` it is the rebuild frame; a widget using it would behave differently depending on where it is placed. And its stated purpose is to make sampling safe, which is a purpose Prism has no use for, since sampling at build does not exist. `now` has neither problem: it is the current frame, wherever it is asked, and Reflex's own `networkView` uses it for exactly this. In Spider, `now` is an `IORef` holding `Just ()` with a clear scheduled for the frame's end, so a subscriber that arrives mid-frame reads the occurrence

through `subscribeAndRead` like any other late subscriber.

### `!Switch` and `$Join`

`!Switch` is `switchHold never` and nothing else. The documentation of `switchHold` reads: "Switches to the new event whenever it receives one. Only the old event is considered the moment a new one is switched in; the output event will fire at that moment only if the old event does." That is the non-prompt law word for word: at occurrence n the inner event current before n is the one heard, and an inner delivered at n is heard from n+1. The initial `never` is "it starts subscribed to nothing". The consequence the specification draws, that whatever a newly built block fires in its creation occurrence is swallowed at the switch, follows without further code: the new inner is delivered at n, and only the old one is considered at n.

`$Join` is `join` on `Dynamic`, which exists because `Monad (Dynamic t)` is a superclass of `Reflex` (from memory). Its behavior half is the inner's behavior read through the outer's; its event half fires when the outer changes (with the new inner's value after the frame) or the current inner changes. That is the prompt law of `$Join`, and the asymmetry with `!Switch` is Reflex's, kept.

The cache idiom, a hold of `!$B` joined with `$Join`, is two statements in the typed AST and two lines here; Reflex's `widgetHold` and `networkHold` are that idiom with `holdDyn` inside, and there is no primitive for it.

### `for`

`for $m: block` over a `$(K ==> V)` is Reflex's `listWithKey` for the building and the evaluator's own merging for the consolidation. `listWithKey :: (Ord k, Adjustable t m, MonadFix m, MonadHold t m, Eq v) => Dynamic t (Map k v) -> (k -> Dynamic t v -> m a) -> m (Dynamic t (Map k a))` (signature from memory) is `for` almost exactly: one block per key, the value as a dynamic inside the block, a map of results out. The evaluator builds with the lower-level pieces `listWithKey` is made of, because it needs the patch event and not only the held map:

```
evalFor env x m f kind acc = do
  let perKey k dv = runWidget (env { envPath = envPath env </> Key k })
                              (apply2 (evalE env f) k (VDyn dv))   -- f's two parameters are the binders
  (results0, patches) <- listHoldWithKeyAdjust (asDynMap (evalE env m)) perKey
  held <- holdIncremental results0 patches
  pure (insert x (consolidate kind held patches) acc)
```

The block is a two-parameter function because `for k, $item <- $items:` names its binders; the shorthand `for $items:` gets the names `key` and `$_` from the parser, so the evaluator has one case.

`listHoldWithKeyAdjust` is the evaluator's helper over `traverseDMapWithKeyWithAdjust`, the call under `listWithKey`, giving the initial results and an event of patches that fires in the frame a key appears or disappears, with the new keys' results. A key appearing in n is built in n under the mid-occurrence rule; a key disappearing in n has its block replaced at the patch, and its nodes propagate until the frame ends. The per-key dynamic is the map's entry projected with `uniqDyn`-style equality on the value (from memory), so a change under a stable key updates `$_` and rebuilds nothing.

Consolidation is by the result kind the typed AST states:

- plain and `$` results: `joinDynThroughMap (incrementalToDynamic held)`, a `$(K ==> A)` that changes promptly whenever a key appears, disappears or an entry's dynamic changes;
- `!` results: an event of a map with exactly the entries that fired, which is where care is needed.

The event case has two terms. The first is the merge over the members as the occurrence opened: `mergeIncremental` over `held`, whose membership patches Spider applies at the end of the frame. The source is explicit about this: a merge's change subscriber does `defer $ updateMerge subscribed m updateFunc a`, and `runFrame` runs the deferred `_someMergeUpdate_updates` after `clearOccurrences`. So a block born in n joins this merge from n+1, and a block torn down in n is still in it for n, which is right for the torn-down block (it keeps propagating until commit) and wrong for the newborn, because `for` consolidation is prompt: a key that appears in n is built in n and anything its block fires in n is in the consolidated event in n. Only pure consequences of construction can fire then, `@started` and inner `rebuild` events and lifted operations over captured events that fired in n, and the parent must hear them.

The second term supplies the newborn's birth firing:

```
consolidate ConsolidateEvent held patches =
  VEv $ mergeWith Map.union
    [ Map.fromList . DMap.toList' <$> mergeIncremental (asEv <$> held)
    , coincidence (mergeMap . newborns <$> patches)          -- the birth term
    ]
```

`coincidence` subscribes to the inner event in the frame the outer fires and delivers the inner's occurrence in that same frame; its documentation says it will "occur whenever the input event is occurring – and its occurrence value, another 'Event', is also occurring." Here the outer is the patch event, firing in n with the newborn blocks' result events, and the inner is the merge of those events, so the term fires in n with exactly the entries the newborns fired at birth. A key cannot be both an existing member and a newborn in one frame, so the union has nothing to decide. The new block does not see itself, because nothing inside the block is downstream of the block's result.

This is the one place `coincidence` appears in the evaluator, and here is why it is right here and nowhere else. The warning on the prompt combinators is about cyclic timing: `switchHoldPromptly`'s documentation says `switchHold`, "by always forwarding the old event the moment it is switched out, avoids many potential cyclic dependency problems / metastability problems", and a prompt reader placed where a program can make a cycle through it brings the program down. Prism's cycle check removes that possibility for this term: the elaborator rejects a `for` whose block's result depends promptly on the collection's change (`@ Delete on !started` as a row result is a compile error), so the only prompt edges into the birth term are from `@started`, inner `dyn` events and captured events, none of which can be downstream of the consolidation. The term is prompt because the law is prompt, and the law is safe because the checker made it so. Reflex's own `EventWriterT` does the same thing (from memory) in its `runWithReplace` instance, telling `coincidence` over the replacement event so that a newborn child's tells are heard in its birth frame, with a comment that the coincidence case must come first so that it has precedence; the evaluator's request collection inherits that, which is why a newborn row's `Read on !started` is in the run call's output for the occurrence of its birth without any code in the evaluator.

The elaborator's finding, whether a given `for`'s block can fire at birth, is on the `for` node in the typed AST next to the consolidation kind, and the evaluator omits the birth term when the block cannot fire at birth. That is an optimisation with a correctness side: a `for` whose block returns only element responses (the todo list, the interceptor's routing loop) has no birth firings, and leaving the `coincidence` out means Spider never subscribes mid-frame to an inner event that is structurally downstream of the fan, which keeps the interceptor loop's heights static. The law test for a new key's result being seen by the parent in its creation occurrence exercises the term; the law test for the forwarding interceptor exercises its absence.

Arrays are `for` keyed by index and go through the same path with `Nat` keys, which is why an insertion at the front rebuilds every later block, as it does under `simpleList` (from memory), and why the chapter on switching calls the array form second class.

## Elements and the list

`@Element e` makes a slot:

```
evalSlot env x el acc = do
  path <- nextPath env
  let ev   = asElem (evalE env (elemValue el))
      info = envTags env ! elemTag ev
  resp  <- if hasResponds info
             then pure (select (envFan env) (Const2 path))          -- by path
             else pure never
  state <- if hasState info then supplyState (envSupply env) ev else pure Nothing
  let req = fmap (asEv . evalE env) (elemRequests el)
  forM_ req $ \r -> tellEvent ((\v -> Seq.singleton (path, v)) <$> r)
  tellDyn (constDyn (Map.singleton path (Slot ev req)))
  pure (insert x (slotResult info resp state) acc)
```

Four things happen and they are the four parts of a slot. The slot's path is allocated from the environment's prefix and bind position. Its response event, if the tag responds, is a selection from the enclosing interceptor's fan by that path, so that routing is a map lookup per firing and not a scan. Its state dynamic, if the tag has state, comes from the supplier context. Its request event, if the tag requests and the program supplied one, is told to the request writer under its path. And the slot itself is told to the list writer, as a constant dynamic of a one-entry map, because an element never changes after creation: a dynamic where a field goes is a `rebuild` in the typed AST and a fresh slot per change.

The creation rule holds here by construction. A response enters only through the fan, the fan is over an event the World or an interceptor fires, and nothing fires in the frame a slot is created except what was already firing when it was created; a World cannot answer in the request's occurrence because it does not see the request until the read phase. A `state` element has a value in its creation occurrence because `supplyState` returns a `Dynamic` with a value, synchronously, during the build.

Routing against the opened list is in the fan's source, not the slot. The interceptor (and the root) builds its fan from its responses event *checked against the child list as the occurrence opened*:

```
routed = attachWith checkPaths (current childList) responses
  where checkPaths opened rs
          | all (`Map.member` opened) (Map.keys rs) = rs
          | otherwise = throw (UnknownPath (head (Map.keys rs \\ Map.keys opened)))
```

`current childList` is the second sanctioned `current` in the evaluator: a read of the list before the occurrence, which is the routing law ("delivered to the slot at that path in the list as n opened") and a delayed edge, which is what makes the interceptor's loop legal. A response addressed to a slot created in this occurrence is an unknown-path error; a response addressed to a slot torn down in this occurrence is delivered, since the slot was in the opened list and its nodes propagate until the frame ends.

Element equality, needed for state identity, is `Eq ElementValue`: tag and plain fields, requests excluded; the memo and cell maps are keyed by `ElementValue` through the matching `Ord`. `Kind: fn Element -> Kind` is a lookup in `envTags`.

### `intercept`

`intercept @child, !responses, $supplier` builds the child under a fresh writer, so that its list is captured rather than told onward:

```
evalIntercept env x child responses supplier acc = do
  outer <- ask
  memo  <- liftIO (newIORef Map.empty)
  mdo
    let routed   = attachWith checkPaths (current childList) (asEv (evalE env responses))
        childEnv = env { envFan    = fanMap routed
                       , envOpened = current childList
                       , envSupply = Supply (asDyn (evalE env supplier)) memo outer childList
                       , envPath   = envPath env </> Bind ix }
    (result, told) <- lift (runDynamicWriterT (runReaderT (runWidget' child) childEnv))
    let (supplied, childList) = splitDyn (Map.partitionWithKey (isSupplied (envPath env))) told
    tellDyn supplied
    pure (insert x (VTuple [result, VDyn (fmap slotsOf childList)]) acc)
```

The child runs under `runDynamicWriterT`, which peels one writer layer: the child's slots go into a fresh list that comes back as `told`, and nothing the child creates reaches the outer writer unless the interceptor tells it. The child's environment has a fan built from the interceptor's responses event, checked against the child's own list as opened; a supply context holding the supplier dynamic, a memo, the outer environment and the child's list; and the interceptor's path as prefix. The `mdo` is the cycle in the specification's own example: `routed` reads `current childList`, and `childList` is what building the child under `routed` produces. It is legal for the same reason it is legal in Prism, the read is delayed, and Spider resolves it the same way it resolves a hold.

The interceptor returns the child's result and the child's list as a `$(Path ==> Slot)`, which the interceptor's own code loops over with `for` and `rebuild` to re-emit, rewrite or hold back. Forwarding is manual: the evaluator has no table of tags to forward and no mode. `\|e| @Element e` is a Prism lambda the evaluator applies like any other.

## State supply

A `state` element's dynamic has to exist, with a value, in the occurrence the slot is created, and it has to come from outside: the interceptor's supplier, or the World's oracle at the root. The specification's mechanism is that the interceptor's supplier is a dynamic of a function from an element to a blueprint returning the state's dynamic; at slot creation the runtime reads the function, applies it to the element, builds the widget then and there as a child of the interceptor's node, and the widget's result dynamic is the slot's. Once per distinct element value per interceptor; alive as long as some child slot with that value lives; a slot's policy fixed at creation.

```
data Supply t
  = SupplyRoot   { oracle :: ElementValue -> Val t, cells :: IORef (Map ElementValue (Cell t)) }
  | SupplyWidget { supplierDyn :: Dynamic t (Val t)
                 , memo        :: IORef (Map ElementValue (Dynamic t (Val t), Dynamic t (Map Path
(Slot t))))
                 , outerEnv    :: Env t
                 , childList   :: Dynamic t (Map Path (Slot t)) }

supplyState (SupplyWidget sup memo outer childList) ev = do
  live <- liftIO (readIORef memo)
  opened <- sample (current childList)
  case Map.lookup ev live of
```

```
    Just (d, l) | ev `suppliedIn` opened || builtThisFrame ev -> do tellDyn l; pure (Just d)
    _ -> do
      f <- sample (current sup)                                    -- the supplier as opened
      n <- nextSupplied outer
      let suppliedEnv = outer { envPath = envPath outer </> Supplied n }
      (d, l) <- lift (runDynamicWriterT (runReaderT (asWidget (apply1 f (VElem ev))) suppliedEnv))
      liftIO (modifyIORef' memo (Map.insert ev (asDyn d, l)))
      tellDyn l
      pure (Just (asDyn d))
```

Read the miss case first. The supplier function is read with `sample (current sup)`, which is the third and last sanctioned `sample` in the evaluator: the supplier's value as the occurrence opened, which is what "a slot's policy is fixed at creation" means when the supplier changes in the same occurrence. The function is applied to the element and the resulting blueprint is run *in the outer environment*: the interceptor's fan, the interceptor's supply context (so the supplied widget's own state slots ask the next context out, which is how `@Element e` forwards one level), the interceptor's path with an `s/n` segment. It is run under its own fresh writer, so its list comes back as the dynamic `l`, and its result `d` is the slot's dynamic from this instant on. It is run synchronously, during the child's build, in the same frame, which is "then and there, in the same build pass".

Then the two lifetime facts. The supplied widget's list `l` is told into the *child's* writer by every child slot that uses it, under the interceptor's `s/` prefix; the interceptor's partition after `runDynamicWriterT` separates those entries from the child's own and tells them onward to the outer writer, so the supplied widget's elements are in the interceptor's list and not in the child's `$slots`. Because every using slot tells the same dynamic under the same paths, the union has one copy, and the copy persists exactly as long as some telling slot's scope persists: when a `dyn` in the child replaces the last slot with that value, the writer's `Adjustable` instance drops its tells, the supplied elements leave the interceptor's list, and the supplied widget's nodes become garbage. That is "lives as long as some child slot with that value lives", with Reflex's writer doing the reference counting.

The hit case is memoisation by value with a liveness check. A memo entry is live if its paths are in the child's list as the occurrence opened (it was alive at open and nothing torn down in this occurrence has committed yet) or if it was built earlier in this frame (for a second newborn with the same value). A slot created for a value whose supplied widget died in an earlier occurrence gets a fresh build, so a cache's held value does not survive with no slot alive, which is what the Rust runtime's weak memo gives and what the law test for two child slots sharing one supplied widget distinguishes from a global cache. A new child slot whose value already has a live supplied widget joins it regardless of the current supplier dynamic, which the hit case does by not reading `sup` at all.

At the root the supply is the oracle:

```
supplyState (SupplyRoot oracle cells) ev = do
  live <- liftIO (readIORef cells)
  case Map.lookup ev live of
    Just c  -> pure (Just (cellDyn c))
    Nothing -> do
      (pushE, pushRef) <- newEventWithTriggerRef
      pushedNow <- asks (Map.lookup ev . envPushes)
      n <- now
      d <- holdDyn (oracle ev) (leftmost [pushE, maybe never (<$ n) pushedNow])
      liftIO (modifyIORef' cells (Map.insert ev (Cell d pushRef)))
      pure (Just d)
```

A cell is a hold initialised from the oracle's answer, which is the value before this occurrence from

the World's previous snapshot, and fed by a trigger the run call fires with pushes. A cell created mid-occurrence for a value this occurrence's pushes include cannot receive the trigger, since the triggers fired at the frame's start, so the `now` term delivers the push in the creation frame: the hold adopts the pushed value at commit and its change event fires in n, so a slot created in n ends n holding the new snapshot, exactly like a slot that existed before n. Two slots with equal values share one cell and therefore one dynamic, which is the equality law by construction. A state tag the World does not accept is an `UnsuppliedState` error raised here, at creation, because the oracle's type cannot express refusal and the world core checks the tag before the oracle is called.

## The run call

The root is an interceptor the host builds itself: its child is `@main`, its responses event is a trigger, its supply is the oracle, and its captured list is what the run call returns. A program is loaded once:

```
loadProgram :: TypedAst -> Name -> SpiderHost Global (Runtime)
loadProgram ast mainName = do
  (respE, respRef) <- newEventWithTriggerRef
  cells <- newRef Map.empty
  ((listD, reqE, resultV), fire) <- hostPerformEventT $ do
    (((resultV, _), reqE), listD) <- runEventWriterT' (runDynamicWriterT' (runReaderT (rootBuild mainName)
(rootEnv respE cells)))
    pure (listD, reqE, resultV)
  listH   <- subscribeEvent (updated listD)
  reqH    <- subscribeEvent reqE
  resultH <- subscribeEvent (resultEvent resultV)
  pure Runtime { .. }
```

`hostPerformEventT` (from memory, like `newEventWithTriggerRef`, `FireCommand` and `runFireCommand`) runs the stack's bottom and returns the `FireCommand` the run call uses; the evaluator never performs anything through it. Building `@main` inside the host frame is occurrence zero: a frame with no inputs in which every hold has an occurrence to have been built in and every `@started` fires. The three handles are the read side.

Each run call is one frame:

```
run :: Runtime -> RunIn -> IO RunOut
run rt RunIn { responses, pushes, oracle } = runSpiderHost $ do
  setOccurrence rt pushes oracle                          -- envPushes, the oracle for new cells
  pushTriggers <- forM (Map.toList pushes) $ \(ev, v) -> cellTrigger rt ev v
  r <- try $ runFireCommand (fire rt)
         ((respRef rt :=> Identity responses) : pushTriggers) $ do
           list   <- readEvent (listH rt)   >>= sequence
           reqs   <- readEvent (reqH rt)    >>= sequence
           result <- readEvent (resultH rt) >>= sequence
           pure (list, reqs, result)
  pure (toRunOut r)
```

`fireEventsAndRead` (from memory) takes a list of triggers with their values and a read phase, runs one frame in which all the triggers fire together, and returns what the read phase read. That is the run call: responses by path and pushes by element value go in as trigger firings, all simultaneous; the element list, the request firings and the result come out. The read phase reads *events*, never behaviors: Spider's read phase runs inside the frame before the hold assignments, so `sample (current listD)` there would give the list before the occurrence, which is the tag law biting the host. `updated listD` fired in this frame if the list changed, and the host keeps the last list it was given when it did not; the request writer's event fired if any slot requested, grouped by path in tree order; the result's change

event likewise. Pushes for values nobody has asked about are not fired, because there is no cell to fire into; the world core records them in its snapshot and they reach a slot created later through the oracle.

An occurrence is a Spider frame, opened by `fireEventsAndRead` and closed when it returns with holds assigned. Mid-occurrence construction is Spider's `subscribeAndRead`, which every late subscriber goes through and which hands it the event's occurrence in this frame if it has one, so that a `rebuild` built in n sees n. The run call is `runSpiderHost` with `fireEventsAndRead` over trigger refs from `newEventWithTriggerRef`, and those four names appear in the Haskell implementation in this function and in the cell constructor and nowhere else; the Prism program cannot reach them, which is the row that says `newTriggerEvent` is removed.

## Errors and rollback by replay

An occurrence that aborts leaves no trace, in both implementations. The Rust runtime gets that from its transaction: holds commit at the top height, an abort clears the action map, and the committed state is the state before. Spider has no such transaction. Its hold assignments are collected in `eventEnvAssignments` and applied after propagation returns, so an exception thrown mid-frame does leave every hold at its old value; but the frame's occurrence caches, scheduled clears, deferred hold initialisations and merge updates are left half-done, and Spider is not designed to run another frame after that. A cycle is an `EventLoopException` thrown from `checkCycle`; an overflow, an unknown path or an unsupplied state is an exception the evaluator throws from inside a push closure or a build; all of them escape `fireEventsAndRead`, and after any of them the timeline is not to be trusted.

One failure is contained instead of aborting. A `rebuild` whose block fails to build (kind `Build`) does not fire the `rebuild`'s event and is reported through `errors`, the stream element the World offers. In the evaluator that is `evalDyn`'s `build` catching the exception around the block, running the block under its own fresh writer so that the half-built block's slots and holds are dropped, firing nothing for that generation, and telling a `Build` error into the error writer the root reads next to the request writer. The generation counter still advances, so the paths of later generations do not depend on whether an earlier one failed. A cycle, an overflow at occurrence time, an unknown path, a `DeferBackwards` and a `Component` error from the World abort the occurrence as above; the `ErrorKind` closed choice (`Cycle`, `Overflow`, `UnknownPath`, `UnsuppliedState`, `DeferBackwards`, `Component`, `Build`) is the same type in both implementations, and the transcript prints its constructor name.

The Haskell implementation's rollback is replay. A Prism program is deterministic: given the same sequence of input sets and the same snapshots, its sequence of lists and outputs is fully determined, and the determinism law is one of the sentences the suite pins. So the host keeps the log of every committed run call's inputs, which it has anyway since they are plain data, and on an abort it reports the error with the spans the exception names, discards the timeline, creates a fresh local one with `withSpiderTimeline`, rebuilds `@main` and replays the log. The program is then in the state before the aborted occurrence, with every hold, every `rebuild` generation and every supplied widget as it was, and the next run call proceeds. The cost is the length of the history, paid only on the error path, and the benefit is that the transaction law is implemented by the determinism law rather than by surgery on Spider's internals. It is also a test of determinism that runs itself: if the replay produces a different list than the one the host had last received, the Haskell implementation has found a non-determinism, and it says so.

This is why the evaluator is written against `Reflex t` and not against `Global`: a replay needs a fresh `t`, and `SpiderTimeline Global` cannot be reset.

After an error the Haskell implementation does not quarantine. The world core accepts a `quarantine: [Path]` input and ignores it, and the transcript says so, so a script that exercises quarantine is marked as Rust-only in the suite. Everything up to and including the `error:` line of a transcript is compared.

### An occurrence, traced

Take the counter under the Test World, with a click on the increment button in turn 1, and follow it through Spider.

Occurrence zero is `loadProgram`. Inside the host frame the evaluator walks the counter's groups. `g0` and `g1` bind two `@Button`s, which are the GUI library's widgets over `Ui.Button` slots: each allocates a path (`0/#0/0`, `1/#0/0` once the library's own `dyn` over the label has added its generation), selects its response event from the root fan by that path, and tells its slot. `g2` is `leftmost` over two `fmap`s. `g3` is an `mdo: holdDyn (VInt 0) (attachWith add (current count) delta)`, with `count` bound through the knot; Spider defers the hold's subscription, so nothing is demanded early. `g4` binds the label, a `dyn` over the text dynamic, whose `networkView` fires its `now` in this frame and builds the `Ui.Label` slot at `2/#0/0`. The read phase reads `updated listD`, which fired in this frame with the three slots, and the result. The transcript's first turn is printed from that.

Turn 1 is one run call with `responses = {0/#0/0: click}`. The response trigger fires; `attachWith checkPaths (current childList)` reads the list as opened, finds the path, and passes the map to the fan; the fan selects by path, and the first button's response event fires with `click`. The library's widget maps it to `Click`; `g2`'s `leftmost` fires with `VInt 1`; the `attachWith` in `g3` reads `current count`, which Spider's hold still holds at `0` because assignments happen at frame end, and fires with `1`; the hold schedules its assignment. `updated count` fires with `1`, so the label's text dynamic changes, so the `networkView` under the label's `dyn` sees `updated child` and `runWithReplace`s the block: a new `Ui.Label` slot is told at `2/#1/0`, the old one's told dynamic is dropped. The writer's list dynamic changes in this frame with the new map. `runFrame` then runs the assignments: `count` becomes `1`. The read phase reads `updated listD`, and the transcript's second turn shows the new label path and text. Nothing was sampled, nothing was performed, and the whole of what the World learned is the list.

## What must not be used

Reflex offers, next to each combinator the evaluator needs, one that looks the same and means something Prism has removed. The evaluator is small enough that these are review items rather than architecture, and the review is a grep. Each line below has the evidence for it, so that whoever reviews the evaluator can see why the line is there.

**`current` and `sample`, outside three places.** `current` extracts a dynamic's behavior and `sample` reads it, and `MonadSample` is a superclass of nearly every monad a widget runs in, so a model writing the evaluator will reach for `sample (current d)` whenever it needs a value at build time. There is no such need: everything at build time is a build-time binding in the typed AST, and those are plain values; a build-time read of a dynamic does not exist in Prism, and a value sampled at build is a constant that looks like a dynamic, which is the afternoon the whole design is built to prevent. The three places `current` or `sample` legitimately appear are the `Tag` node (the tag law), the fan's `attachWith checkPaths (current childList)` (routing against the opened list) and `sample (current sup)` at supply (the supplier's policy as the occurrence opened). Each is a delayed edge the specification names. A fourth occurrence is a bug until a sentence in the chapters is found that it implements, and then it is a fourth named place.

`switchHoldPromptly`, `switchPromptlyDyn`, `switchHoldPromptOnly`, `coincidence`, **except the birth term.** The evidence is Reflex's own documentation. On `switchHold`: "Because the simultaneous firing case is irrelevant, this function imposes laxer"timing requirements" on the overall circuit, avoiding many potential cyclic dependency / metastability failures. It's also more performant. Use this rather than 'switchHoldPromptly' and 'switchHoldPromptOnly' unless you are absolutely sure you need to act on the new event in the coincidental case." On `switchPromptlyDyn`: "Prefer 'switchDyn' to this where possible. The timing requirements that switching before imposes are likely to bring down your app unless you are very careful." On `coincidence`, after the definition: "You maybe looking for '@switchHold@ @never@' instead." Prism took the advice as a rule, `!Switch` is `switchHold never`, and the evaluator may not spell `!Switch` any other way. The one `coincidence` is in `for`'s birth term, where the cycle check has already made the timing requirement hold, and it is marked with a comment naming the law it implements. A `coincidence` anywhere else, or a `switchHoldPromptly` anywhere, is a reviewer's finding.

`tagPromptlyDyn` **and** `attachPromptlyDyn`**.** The documentation of `tagPromptlyDyn` is the clearest statement of the tag law in the Reflex sources, written as the warning against its own function: "@/tagPromptlyDyn d e/@ differs from @/tag (current d) e/@ in the case that @/e/@ is firing at the same time that @/d/@ is changing. With @/tagPromptlyDyn d e/@, the **new** value of @/d/@ will replace the value of @/e/@, whereas with @/tag (current d) e/@, the **old** value will be used, since the 'Behavior' won't be updated until the end of the frame. Additionally, this means that the output 'Event' may not be used to directly change the input 'Dynamic', because that would mean its value depends on itself. **When creating cyclic data flows, generally @/tag (current d) e/@ is preferred.**" Prism has only the old-value form. The evaluator's `Tag` node is `attachWith` over `current`, and the prompt forms do not appear.

`getPostBuild` **and** `PostBuildT`**.** Its frame is "determined by the instance", and its purpose is safe sampling, which Prism does not have. The stack has no `PostBuildT`, so `getPostBuild` does not type-check in `PrismM`, which is the cheapest kind of review.

`performEvent`, `performEvent_`, `performEventAsync`**.** The documentation says (from memory) "the output 'Event' will generally occur later than the input 'Event', since most 'Performable' actions cannot be performed during 'Event' propagation", which is the creation rule's spirit with the wrong party performing. In Prism the program performs nothing; the World performs request firings after the occurrence commits. `PerformEventT` is in the stack for `Adjustable`, and a `performEvent` call in the evaluator would be the program doing an effect, which no sentence in the chapters allows.

`newTriggerEvent` **and** `TriggerEventT`**.** A trigger is how a frame starts, and only the World starts one. The two triggers that exist are the root's responses trigger and the per-cell push triggers, both made with `newEventWithTriggerRef` in the host and fired only from `run`. A `newTriggerEvent` in a widget would let a program start an occurrence, and `Defer` is the only way a program has to cause a later occurrence, through the World.

`Requester` **and** `MonadQuery`**.** Elements replace both, and the correspondence appendix says what each exchange buys. `Incremental` is used inside `for` and the merges and never surfaces as a `Val`.

How review catches them: `Prism.Eval` is one module of a few hundred lines, the checklist is a comment at its top, and a CI step greps the module for each forbidden name and for `current`, `sample` and `coincidence`, counting the sanctioned sites; the step fails on any new one. That is not a proof, and it is the right amount of machinery for a module whose author is supposed to read the chapters first.

## The Test World and the transcript printer

The Haskell world core is the same few hundred lines as the Rust one, in `Prism.World`: the turn loop, `Defer`, snapshot and push bookkeeping, slot tracking and the discarding of late results, `Unsupported`, error reporting, and the transcript printer. It is written against the run call and nothing else, so it would work over any runtime that had one; there is one, and the Test World is the core plus a component that answers to a script.

The turn loop is the specification's. Gather the turn's inputs into one `RunIn`; call `run`; from the output, collect the `Defer` slots that fired; if there are any, run another occurrence delivering their responses, with no other input, and repeat; a `Defer` answered twice in one turn is a `DeferBackwards` error; when no `Defer` fired, the turn is over and the World acts on the last output. The snapshot is held still across a turn: pushes enter with the first occurrence only, and the oracle answers from the previous snapshot throughout. A failed turn drops its responses and re-queues its pushes. Request firings from a slot that the last list no longer contains are performed if the component can and their responses discarded. All of this is the world core chapter's content and is not different in Haskell; what differs is only that it is twenty lines shorter, because `Map` and `Either` are closer to hand.

The Test World's component accepts every tag. For a `requests` field it records what fired, for `requested`; for a `responds` field it holds nothing, since the script's `send` steps supply the responses; for a `state` field it answers the oracle from the script's `with` steps and turns a later `with` or `push` into a push. It knows no tag's meaning, which is what lets a program with a `Net.Listener` run under it. The script is parsed by `Prism.Parse` as part of the test file, because a `test` is a top-level statement whose steps are an array of plain values, and the harness is a function from a blueprint and an array of steps to a transcript and a verdict:

```
runTest :: TypedAst -> Test -> IO (Transcript, Verdict)
runTest ast (Test widget turns) = do
  rt <- runSpiderHost (loadProgram ast widget)
  w  <- newTestWorld rt
  foldM step (turn0 w) steps
```

Each step is one of the suite's: `expect`, `expect_top`, `expect_none`, `expect_count`, `expect_top_none`, `send`, `click`, `with`, `push`, `returns`, `fired`, `quiet`, `requested`, `unrequested`, `errors`. `expect` reaches inside container elements by the GUI library's convention (`Ui.VBox`'s `children: Path ==> Element`), because the library is part of the suite. `send` finds exactly one matching slot in the last list, delivers the response by that slot's path, and runs a turn; an input line with several deliveries delivers them all in one occurrence and fails if two address one slot. `with` may precede the build, and must for any state tag the program creates at build, since there are no declared defaults. `returns` on an event result is a script error, as the suite says.

Only the transcript printer has to agree with the Rust one character for character, and the suite's format defines it, not either implementation:

```
turn 0
  list:
    0/#0/0: Ui.Button(label="Increment")
    1/#0/0: Ui.Button(label="Decrement")
    2/#0/0: Ui.Label(text="Count: 0")
  result: unit
turn 1  responses: {0/#0/0: click}
  list:
    0/#0/0: Ui.Button(label="Increment")
    1/#0/0: Ui.Button(label="Decrement")
    2/#1/0: Ui.Label(text="Count: 1")
```

```
result: unit
```

A turn header names the turn and its inputs, `responses` by path and `pushes` by element value. The list is one slot per line, `path: element`, in tree order, which is the order the `Map Path` iterates in because `Ord Path` is defined to be tree order: bind positions compare numerically, generations numerically, keys by the key's own ordering, with a supplied segment `s/n` ordering after the binds of its node. A slot that fired its request gets a `requested` line after it with the values in firing order. An occurrence boundary inside a turn, which only `Defer` causes, is `occurrence 2` between the parts. The result is printed by its current value for a plain or dynamic result and as `fired: v` or `quiet` for an event result. An aborted occurrence prints `error: Error(kind=Cycle, spans=[[...]], occurrence=1)` and no list.

Element values print as their constructor with named fields in declaration order, which the tag table gives; plain values print as Prism literals, with `Text` escaped the way the lexer reads it, `Float` in the shortest round-tripping decimal, arrays as `[[a, b]]`, tuples as `((a, b))`, and maps as `{k: v, ..}`. Every spelling decision here is a decision both printers must make the same way, and a `diff` on the counter's transcript is the first test the Haskell printer runs, before any program with a `Defer` or a `for` is attempted.

Paths are where the printers most easily disagree. The Haskell `Path` is a list of segments allocated by the evaluator's environment and the Rust one is an `Rc<[Segment]>`; both print `0/#1/0`, `[k]` and `s/n` because the transcript format says so, and the first `for` test with a space in a key will find out whether both escaped it alike.

## The reflex-dom World

The reflex-dom World is the alternate GUI World and the only place the Haskell implementation does something the Rust implementation cannot yet: it puts a Prism program on a screen. GUI is not a World concept; there is one GUI vocabulary, `Gui.*`, that any World with pixels and a pointer can speak, and the Prism GUI library, with its layout, hit-testing, focus and text editing, runs on top of it unchanged. The reflex-dom World speaks the vocabulary to a browser.

The vocabulary, as the World sees it:

| Tag | Part | What the reflex-dom World does with it |
|---|---|---|
| `Gui.Window` | `title`, `size`; responds `WindowEvent` (`Resized`, `CloseRequested`, `Pointer`, `Pressed`); one per window | the document for the first slot; a popup for further ones. Listeners on the window element turn pointer, key and resize DOM events into `WindowEvent` responses by the slot's path. |

| Tag | Part | What the reflex-dom World does with it |
|---|---|---|
| `Gui.Scene` | presence: `window` (the title it paints) and `draw: [Draw]`, the draw list as data | rendered to the DOM after each turn, one entry of its `draw` field per element: `Draw.Rect` to an absolutely positioned `div` with a background, `Draw.Text` to a `span` at its origin, `Draw.Image` to an `img`, `Draw.Clip` to a `div` with `overflow: hidden` and the clipped entries inside it. |
| `Gui.Focusable` | presence: `name` only, no responses; focus is a library concept | an invisible element with `tabindex` and `aria-label`, so that the browser shows its focus ring, has something to focus for the input method's candidate window and a screen reader has a name to read. Its key DOM events bubble to the window listener and become `Pressed` responses addressed to the `Gui.Window` slot's path, not the focusable's; the library routes them to the input whose name is the focus, and emits at most one focusable, the one it holds as focused. |
| `Gui.Metrics` | `text`, `font`; state `Extent`, answered synchronously | the oracle measures with a hidden `canvas`'s `measureText` for the element's text and font; the push fires when the fonts change, which in a browser is `document.fonts`' `loadingdone`. |
| `Gui.Frames` | responds `Frame(now: Instant)` per frame while a slot exists | `requestAnimationFrame` re-armed after each turn while a `Gui.Frames` slot is in the list; a tick is a turn delivering `Frame(now)` to the slot's path. |

The World is itself a small Reflex application, which is the natural way to write it in this language and a nice symmetry: the Prism runtime is a Reflex network on a local timeline, the renderer is a reflex-dom network on the global one, and the run call is the seam. The renderer's input is a `Dynamic t (Map Path Element)`, fed from the run call's output through a trigger after each turn; `listWithKey` over it draws one DOM subtree per slot, and `reflex-dom`'s own diffing does what a GUI World's draw-list diff does in Rust. DOM events become responses: a `click` on the window element becomes a `Pointer` response addressed to the `Gui.Window` slot's path, a `keydown` becomes a `Pressed` response addressed to the `Gui.Window` slot, a `resize` observer becomes a `Resized` response to the window,

and each is one turn. The World does no geometry: hit-testing is the library's, from the draw list it emitted and the pointer position the window reported, which is why the window responds with pointer events and the draw entries, which are values and not elements, have no events and no paths.

A turn in this World, end to end: the user clicks. The window listener makes `{window_path: Pointer(Down(((x, y)), Left))}`, the world core runs the turn (the run call plus any `Defer` occurrences the library's focus handling needs), the output list has a new `Gui.Scene` with a pressed button drawn differently, the renderer's dynamic is updated and reflex-dom patches two `div`s. Nothing was drawn because an element appeared; the scene is a description, read after the turn.

The `Ui.*` tags never reach this World. `Ui.Button`, `Ui.Label` and `Ui.TextInput` are the library's, handled by its interceptors, which turn them into `Gui.Scene` entries and focusables before the list reaches the root. A native-toolkit World might accept them directly; a browser World could, by mapping `Ui.Button` to a `<button>`, and that is a different World with higher tags, not this one. This one proves the sentence "layout, hit-testing, focus and text editing are Prism interceptors" by running them in a place where the host has its own opinions about all four and is not asked for any of them.

The World runs under GHC's JavaScript or WebAssembly backend through jsaddle, or natively through WebKitGTK with the same code (from memory). The Prism program's typed AST is data the page loads, so there is no compiler on the page and no GHC per program, which is the interpreter decision paying again.

## Differential testing

Everything in the suite runs under both implementations, and the differential test is `diff`. Two commands produce the two interchange formats: `dump` prints the typed AST in the text form and `test --transcript` runs every test in a file under the Test World and prints the transcript of each. The harness runs both commands under `prism` and `prism-hs` over every file in the suite, the layout corpus's token files included, and reports every byte that differs.

An empty diff is a precise claim: that both implementations read the chapters the same way, for every program in the suite, at the level of what the compiler chose and at the level of what a World sees. It is not a claim that either is right. The conformance scripts make that claim, and the differential test adds the second reader.

A divergence is one of two things. One implementation has a bug, which the law test pinning the relevant sentence will usually have caught already and the diff localises to a construct; or two careful readers read a sentence differently, and the sentence is ambiguous. The second kind is the find the whole exercise exists for. Its procedure: rewrite the sentence so that it admits one reading, add a law test pinning that reading, bring both implementations to it. A divergence is never resolved by making one implementation copy the other without a sentence to point at; that turns the second reading back into the first.

Three kinds of divergence have a predictable home. A path printed differently is a reading of "its position in the widget tree" and belongs to the transcript format's definition, not to either runtime. A dump that differs in a lift or a numeric type is a reading of the lifting rule or the inference rule, and the goldens, which both frontends must match, answer it before the transcripts can. A transcript that differs in which occurrence of a turn a request fired is a reading of the `Defer` rule or the creation rule, and the world core tests answer it. Everything else is new, and new is what the test is for.

The Haskell implementation is a particularly good second reader on the sentences a height-ordered runtime finds hardest, because Spider resolves them differently. The mid-occurrence construction rule

is `subscribeAndRead` in Spider and catch-up caching in Rust; the two agreeing on a nested `dyn` over a dynamic that changed in the same occurrence, building once with the new value, is two independent derivations of one transcript. `for`'s prompt consolidation is a `coincidence` term in Haskell and a merge with a prompt birth contribution in Rust. `$Join` is Reflex's `join` in Haskell and a hand-written `Dynamic::join` with three law tests in Rust, and the third test, an outer switching to an inner that does not change, is the kind of case a differential run finds before a person thinks to write it. When the Rust side fails a script, the first question is whether the runtime is wrong or the script is, and the Haskell transcript answers it before anyone reads the runtime.

Random programs are the natural extension once the suite agrees: a generator of well-typed Prism programs, run under both with random scripts, with any transcript difference a finding, the way Rustlantis runs random MIR on several backends (from memory). The harness takes a directory of files, so a generator can feed it without changes.

The differential test runs on every change to either implementation and on every change to the book, since a change to a sentence is a change to what the goldens and transcripts should say. When a sentence changes and the diff stays empty, both implementations have the same bug, and only a law test can find it; that is why there are both.

## What it cannot check, and what it alone can

Some limits are chosen by the design and buy something; others are only where the plan stops.

It cannot check the Rust runtime's ownership. Prism's garbage collection in Rust is the ownership tree: a node is dropped by its parent and its subscriptions die as weak pointers. Haskell has a collector, Spider's nodes are kept alive by whoever references them, and the evaluator never thinks about it. A leak in the Rust runtime, a `trace` that keeps a node alive, an `Rc` cycle through a closure, shows up in no transcript. The Rust runtime's invariant checker and its weak-count tests are the defence, and the Haskell implementation does not help. This buys an evaluator with no lifetime code in it at all, which is the kind of second reading that cannot share an ownership bug with the first.

It cannot check the closure compiler, the lint, or the values representation. It has none of them. A capture list that names the wrong variable, a placeholder closed twice, a `Plain` downcast at the wrong type: the Rust lint and goldens catch these. It does catch a wrong *typed AST*, because it executes the typed AST directly, so a Rust compiler bug that is in the dump is also in the Haskell transcript of that dump, which localises it to the frontend.

It cannot check performance, and does not try. The evaluator allocates a `Map` per environment extension and a `Val` per value, and an occurrence under it costs whatever Spider costs plus that. The suite's programs are small and this is fine; a benchmark is not a test the Haskell implementation runs.

It does not check any World but two, because the plan builds it no others. The CLI World, the software rasteriser, the X11 World and the tokio skeleton are Rust, tested in Rust against the Rust world core; a Haskell CLI World would speak the same plain-data run call and needs nothing the design lacks. The Haskell world core is tested against the same fake-component tests, which is the one place World testing is duplicated, and it is duplicated because the turn loop is the part of a World the chapters specify.

It cannot continue after an error the way a Rust World with quarantine can. Rollback by replay gives it the transaction law; quarantine is a World policy this plan does not give it.

The other side of the ledger has four entries, and nothing else in the plan provides any of them.

It is an executable statement of which Prism laws are Reflex's. Every row of the correspondence table that maps a construct to a Reflex combinator with no evaluator code around it is a law Reflex already had; every row with code around it (`for`'s birth term, state supply, the fan, the run call, replay) is a law Prism added. That list, as code that runs, is more exact than any appendix, and it is what a reader who wants to move a law can consult to see whether moving it means changing an engine or changing a hundred lines.

It is the cheapest place to try a semantic change. A change to what `!Switch` means is one `case` arm in the evaluator against a runtime rewrite and a compiler change on the Rust side; a change to `for`'s consolidation is one function; a change to the run call's inputs is one record. The law tests say what the change did, under both implementations, and a sentence can be tried before it is written.

It is the first Prism on a screen. The reflex-dom World is a few hundred lines over a renderer that already exists, and it runs the GUI library, which means the library's layout and focus and text editing can be seen and poked before the Rust rasteriser exists, and the library's bugs found with a mouse rather than a script.

And it is the oracle when the Rust runtime is wrong in a way no script anticipated. The hard bugs in a height-ordered runtime are the ones around construction in the middle of an occurrence: a nested `rebuild` built anew in the occurrence its parent is rebuilt in, a late subscriber's view of a firing earlier in the same occurrence, a cycle that closes only after a switch adopts a new inner. Each of those is a question Spider has an answer to, and reading Spider's source to find the answer is slow and error-prone. With the Haskell implementation, the reading is a run: write the script, diff the transcripts, and the sentence the Rust runtime got wrong is the one the law test names.

## The smallest version that pays for itself

The whole chapter describes the finished implementation. The smallest one that earns a differential test is a fraction of it, and the order in which the rest arrives is set by which law tests it can run.

*Skip the frontend.* The Rust frontend already prints the typed AST in the text form, and `Prism.Typed.Text` has to parse that form anyway. So the first Haskell binary reads the text form and nothing else: no lexer, no layout, no parser, no elaborator. The differential test still runs, Rust frontend into both evaluators, and every law test exercises the Reflex evaluator from day one. The Haskell frontend is a second project with its own diff, added when the evaluator is believed.

*The evaluator for the counter.* Values are one `Value` sum type, as in the runtime chapter; `Event t Value`, `Dynamic t Value`, `Behavior t Value`. The counter needs five nodes: `hold` is `holdDyn`, `tag` is `attachWith` on `current`, `leftmost` is `leftmost`, lifted application over a dynamic is `fmap`, and `@Element` with a response is a `newTriggerEvent` keyed by path plus a `DynamicWriter` entry for the element list. A widget block is a `RecursiveDo` block, because the counter's hold refers to itself, and the resolver's binding groups say which bindings go in one `rec`. That is enough for the tag law, the leftmost law, the hold-in-build-occurrence law, and the glitch-freedom diamond.

*The Test World.* `runSpiderHost`, one trigger per response path and one per state element, a turn is `fireEventsAndRead` over the turn's triggers, and the transcript printer reads the element list `Dynamic` and the collected request firings after each turn. `Defer` is a loop: after a turn, read the `Defer` firings, fire their triggers, repeat until none. That is the whole World, and it is under two hundred lines.

*Then, in this order, because each unlocks a block of law tests.* `switchHold` for `!Switch` and `join` for

`$Join`, which unlock the switching laws. `networkView` for `dyn`, with the rebuild's element list substituted in the parent's `DynamicWriter`, which unlocks the construction-in-the-middle laws. `listWithKey` over a `Dynamic (Map Key Value)` for `for`, with consolidation as `mergeMap` on the blocks' results, which unlocks the keyed-list laws. `intercept` last, as `runDynamicWriterT` on the child plus a `fanMap` of the responses by path, which unlocks the element laws. Errors and the replay-based rollback come after all of that, because until then a failing test is a bug in the evaluator and not a rollback question.

What this leaves for the day the evaluator is believed: the frontend, `reflex-dom`, and the replay log's memory. None of them is needed to find a disagreement between the two readings of chapters 3 through 7, which is the only reason the implementation exists.

## What the Haskell implementation leaves out

The negative space, in one place, with what each exclusion buys.

No compilation of Prism to Haskell source. Bought: no second compiler to be wrong in, no GHC per program, a typed AST that runs without a Haskell frontend.

No typed representation of the typed AST. `Val` is a sum and projections fail with "elaborator bug". Bought: one `case` arm per node kind, and every type question kept in the frontend where the goldens are.

No `PostBuildT`, `RequesterT`, `QueryT`, `TriggerEventT` in the stack. Bought: `getPostBuild`, `requesting`, `queryDyn` and `newTriggerEvent` do not type-check in a widget block, so the review for them is the compiler's.

No `performEvent`, no `liftIO` reachable from a block. Bought: a program under the Haskell implementation can do exactly what a program under the Rust one can, which is nothing, and the transcript is the whole of what it did.

No prompt switch, no prompt tag, no post-build. Bought: the laws of the switching chapter hold by the choice of combinator, and the warnings in Reflex's own documentation are the argument.

No transaction inside Spider. Bought: no patch to Reflex; rollback by replay, which is the determinism law doing the transaction's work and testing itself.

Real Worlds, quarantine, benchmarks and production use of the Haskell implementation are outside this plan. Nothing in its design rules them out: a Haskell World speaks the same plain-data run call as a Rust one, and the evaluator over Spider is a reasonable place to run real programs. The plan stops at the Test World, which the differential test needs, and the reflex-dom World, which puts the GUI library on a screen early; there is little to say about the rest until someone wants it.

## Chapter 15

# Plan

Eleven milestones in two tracks, each one cut so that a model can be handed this book, the code from the milestone before it, and nothing else, and come back with code whose correctness is decided by named tests and matching transcripts rather than by anyone's opinion. The runtime track makes the Rust graph single-threaded and transactional, gives it the boundary, then interception and `for`. The frontend track takes source to a typed AST with a text form, then to a closure compiler. The two meet when the counter compiled from source runs under the Test World with the same transcript as the counter written by hand in Rust. After that the conformance suite, the world core and the CLI World, the Haskell reading, the GUI Worlds, and errors. Each milestone states what it waits on, the work in files and types and functions, what done means, what finishing it proves about the design, and where it is likely to go wrong. The chapter ends with what is deliberately not on the plan and with how to run each milestone with a model when every line will be read by one person.

## How a milestone is cut

You have had the project where the plan was a list of features and the first week went into deciding which feature to start with, because every feature needed a piece of every other. The cure is to cut along the lines the design already has. Prism's design has three hard lines in it: the run call, which is the only thing a World sees; the typed AST, below which nothing decides meaning; and the transcript, which is the only thing a test compares. Every milestone here ends at one of those lines or at a named test, and the plan is ordered so that each line exists before anything needs to cross it.

A milestone has five parts, and a model should be able to find each one without reading the rest of the chapter:

- *Waits on.* The milestones whose code it starts from. Nothing else. There is no step where someone signs off; a milestone is finished when its tests pass, and the next one begins from its code.
- *The work.* Concrete steps: crates, files, types, functions to add or change, in an order that keeps the existing tests green at every step. Where this chapter names a type or a function it is a target, not a constraint; a model that finds a better name should use it and say so.
- *Done.* The tests that pass and the transcripts that match. A test named here is a sentence about the language, and the sentence is its name. A test that cannot be stated as a sentence about the language is testing the implementation's structure, and the chapter on testing across implementations says what happens to those: they are deleted when the structure changes.
- *What it proves.* Which claim of the design the milestone makes true in code. Every refusal in the design (no scheduler, no collector, no locks, no callbacks across the boundary, no sampling) is a claim that something is unnecessary, and the only proof is a working implementation without it.
- *Risks.* Where the work is likely to bend the design to fit the code, and what to watch for.

Sizes are given as a rough count of files touched and tests added, not days. The runtime milestones are each a few hundred to a couple of thousand lines of Rust against a crate of five thousand (from memory); the frontend milestones are larger; the Haskell milestones are smaller than their Rust

counterparts because Reflex is most of the runtime.

## The two tracks and where they meet

The runtime track (milestones 1 to 3) and the frontend track (4 and 5) share no code and can start on the same day. The runtime track never needs source: its tests are Rust blueprints written by hand against the builder. The frontend track never needs a runtime: its tests are typed-AST dumps diffed against goldens. They meet at milestone 5, when the closure compiler turns the typed AST of the counter into an `IrBlueprint` and the Test World drives it through the run call, and the proof that they met is that the transcript is byte-identical to the one the hand-written Rust counter produced in milestone 2.

| Milestone | Track | Waits on |
|---|---|---|
| 1. Runtime: single-threaded and transactional | runtime | nothing |
| 2. Runtime: the boundary | runtime | 1 |
| 3. Runtime: interception and `for` | runtime | 2 |
| 4. Frontend to typed AST | frontend | nothing |
| 5. Closure compiler | both | 2, 4 (its `intercept` and `for` parts: 3) |
| 6. Conformance suite in full | both | 3, 5, and the core half of 7 |
| 7. World core and CLI World | worlds | core: 3; CLI end to end: 5 |
| 8. Haskell frontend | haskell | 4 (the text form) |
| 9. Haskell evaluator and Test World | haskell | 6, 8 |
| 10. GUI Worlds | worlds | 6, 7; the reflex-dom World also 9 |
| 11. Errors | runtime, worlds | 3; slot it anywhere after |

Three things about the table. The world core is listed under 7 but its turn loop is needed by the Test World of 6, so the core half of 7 is handed out as soon as 3 is done and the CLI half waits for the compiler. The Haskell track starts the moment milestone 4 freezes the typed-AST text form, and a second person or model can take it with no knowledge of the Rust code, which is the point of it. Errors is last in the numbering because nothing above needs it, and first to slot in early if a World wants a better message than “the occurrence was aborted”.

## Milestone 1: the runtime, single-threaded and transactional

**Waits on** nothing. Starts from `prism_runtime` on branch `on-fire-repros`, 38 tests passing.

The runtime today has locks whose only job is to detect re-entry and an error path that resumes a panic across the boundary. None of that is wrong in the sense of computing the wrong answer; it is wrong in the sense that it makes three design claims untestable: that the runtime needs no locks, that an occurrence is a transaction, and that a cycle is a comparison and not a deadlock. This milestone makes the three claims true, adds the one combinator the boundary cannot do without, and pays the small debts that would otherwise be paid three times.

## The work

*Step 1: `Rc` and `RefCell`.* Mechanical, touches every file, done first and alone so that nothing else is in the diff. `Arc` becomes `Rc`, `Weak` stays `Weak` (the `rc` one), `Mutex` becomes `RefCell` or `Cell`. The `Send + Sync` bounds go from `Blueprint`, `EventImpl`, `BehaviorImpl`, `ValueType`, `Action`, `HeapValue` and the closure bounds on every combinator; `'static` stays. `Runtime` becomes `Runtime { inner: RefCell<RuntimeInner> }` and is `Send + !Sync`: the test `runtime_send` is replaced by one named `a_runtime_can_be_moved_to_a_thread_and_driven_there`, which builds a graph, moves the runtime into `std::thread::spawn`, runs an occurrence, and joins. If `Runtime` ends up `!Send` because something inside holds an `Rc` to a thread-local, that is a bug in this step, not a reason to add `unsafe impl Send`.

*Step 2: re-entry is a cycle, reported.* Every place a lock served as a re-entry guard (`SwitchHold::height` re-locking its own mutex is the known one; the `try_lock` panics "event cycle" are the others) becomes a `Cell<bool>` on the node, set on entry and cleared on exit, and a set flag on entry is a cycle. The runtime's `schedule` keeps its height comparison. Both report through one path: `RuntimeInner::fail(RuntimeError)`, which records the first error of the occurrence; propagation stops at the next action boundary. The test `a_cycle_closed_through_a_cyclic_event_is_reported_not_hung` closes an event in terms of itself through `add_cyclic_event`, runs under a timeout, and expects `Err` with `kind == ErrorKind::Cycle`.

*Step 3: the occurrence as a transaction.* `Runtime::propagate` becomes `Runtime::run_occurrence(&mut self) -> Result<(), RuntimeError>` (the public `run` with `Inputs` and `Outcome` is milestone 2; here the inputs are still triggers scheduled by the test). Inside: open (reset per-occurrence state), propagate by height, commit at `usize::MAX`, clear, close. The error path is a rollback, and a rollback needs three things the code does not keep today: a *commit list* (every hold write, switch retarget and child replacement is already deferred to commit time; collect them in one `Vec<Box<dyn Commit>>` on the runtime instead of as actions at `usize::MAX`), a *build journal* (every node built during the occurrence is pushed to `RuntimeInner::built_this_occurrence: Vec<Rc<Widget>>`, so that on error they are dropped and their parents' child lists restored), and the *fired marks* (the per-event `SubscriptionManager.occurrence` cache is already keyed by occurrence number; bumping the number is the clear). On `Err`, drop the commit list, unwind the journal, bump the occurrence number, return. A panic inside an action is caught with `catch_unwind` and becomes a `RuntimeError` of kind `Component` with the panic's message and the node (a panic in native code a World or library put inside a closure is the `Component` kind); nothing resumes it.

Add `Runtime::check_invariants(&self) -> Result<(), String>` and call it at the end of every test, after every occurrence, including failed ones: no event's cached occurrence value is from the current occurrence number, no hold has an uncommitted pending value, no node's re-entry flag is set, the action map is empty, the build journal is empty. This function is the one piece of test scaffolding that is kept forever; the chapter on the runtime calls it the invariant checked after every occurrence.

*Step 4: `Dynamic::join`.* `impl<T: ValueType> Dynamic<Dynamic<T>> { pub fn join(&self) -> Dynamic<T> }` in `dynamic/mod.rs`, with its own file `dynamic/join.rs`. The behavior half samples the outer's current, then that inner's current. The change event is the leftmost of two: the outer's change, mapped to the new inner's value after this occurrence (its change event's value if it fired in this occurrence, else its current), and a non-prompt switch over the current inner's change event, built with the existing `switch_hold` machinery. Its height is dynamic; reuse the height-invalidation path that `SwitchHold` already has.

*Step 5: small debts, each one a test.* `trace` leaks its tracer (make it a weak subscription). The `unreachable!()` stubs in `SubscriptionEvent` impls (implement or delete the trait method). Scheduling at `usize::MAX` while at `usize::MAX` (commit-time actions that schedule

more commit-time actions: with the commit list from step 3 this becomes "append to the list", and the edge disappears). `DWAction` rebuilding at child index 0 (make the child index a field of the action; the general keyed form is milestone 3). `Rebuild` finding its parent through a non-prompt tag read (a set-once `OnceCell<Weak<Widget>>` on the node, set at build; the test is `a_nested_dyn_rebuilt_in_the_same_occurrence_as_its_parent_finds_its_parent`). `DynBlueprintAdapter` replaced by a blanket `impl<W: Blueprint> DynBlueprint<W::Output> for W`, and `widget/runtime.rs` renamed to `widget/polymorphic.rs`. Typed handles for `add_cyclic_dynamic` and `add_cyclic_event`: return `CyclicDynamic<T>` and `CyclicEvent<T>` tokens closed by `close(self, Dynamic<T>)`, so that closing with the wrong type is a compile error; a `compile_fail` doc test pins it.

## Done

All 38 existing tests pass on `Rc`/`RefCell`, with the `Send + Sync` bounds gone. The new tests pass:

- `a_runtime_can_be_moved_to_a_thread_and_driven_there`
- `a_cycle_closed_through_a_cyclic_event_is_reported_not_hung`
- `a_cycle_through_a_switch_is_reported_with_the_nodes_in_it`
- `an_occurrence_that_panics_leaves_the_graph_as_it_was` (build a counter-like graph, run one occurrence, inject a panic in a map closure in the second, expect `Err`, check that the hold's value and every derived dynamic's value equal those after the first occurrence, and that `check_invariants` passes)
- `an_occurrence_that_fails_drops_the_nodes_it_built` (a `rebuild` firing in the failing occurrence; afterwards the old child is still there and the new one is not)
- `join_follows_the_outer_switch_and_reads_the_new_inner` (outer switches to an inner with current value 3; the joined dynamic changes to 3 in that occurrence)
- `join_follows_the_new_inner_when_both_change_in_one_occurrence` (outer switches and the new inner changes in the same occurrence; the join reads the inner's new value, once)
- `join_ignores_the_old_inner_after_a_switch`
- `join_is_prompt` (a dynamic derived from the join changes in the same occurrence as the join)
- `a_nested_dyn_rebuilt_in_the_same_occurrence_as_its_parent_finds_its_parent`
- `a_closure_runs_once_per_occurrence_even_when_subscribed_late` (the existing test, re-stated in the sentence form)
- `trace_does_not_keep_its_tracer_alive`
- the `compile_fail` doc test on typed cyclic handles

`check_invariants` is called after every occurrence in every test, and passes.

## What it proves

That the runtime needs no locks: a graph driven from one thread, movable between threads, with re-entry detected by a comparison. That an occurrence is a transaction: the `Err` path leaves no trace, which is the whole basis of the error story in the chapter on errors and of the optimistic-concurrency note in the runtime chapter. And that `Dynamic::join` is the only join: `rebuild` over dynamics, the cache idiom (a hold of dynamics joined), and the element list under an interceptor all reduce to it.

## Risks

A `RefCell` double borrow is a panic with a worse message than a `Mutex` deadlock is a hang. Every one of them in propagation is a re-entry, and every re-entry is a cycle, so every `borrow_mut` on a node's state during propagation should go through the flag from step 2 and report `Cycle`, never `already`

`borrowed`. Grep the tests for `already borrowed` after the milestone; there should be none.

The rollback is the step most likely to be half done. A model will get the commit list right, because the code already defers writes, and will forget the build journal, because nothing today records what a `rebuild` built. The test `an_occurrence_that_fails_drops_the_nodes_it_built` is there to catch exactly that, and it should be written before the rollback is.

`Dynamic::join`'s "new inner's value after this occurrence" is the one place where the behaviour side and the event side can disagree by one occurrence. Two of the join tests pin the two cases; a third worth adding is `join_where_the_outer_switches_to_an_inner_that_does_not_change`, which should emit the inner's current value in the switch occurrence and nothing afterwards.

## Milestone 2: the runtime boundary

**Waits on** milestone 1.

A World, after this milestone, drives the runtime through one call with plain data in and plain data out, and nothing else. The test backend in the crate today reads behaviours directly with `query_for_tag` and receives firings through a `WidgetDelegate` holding live slots; both are ways of reaching into the graph that no World will have, and keeping them would mean the tests test something no World can see. This milestone adds elements, slots, paths, the element list, the request collector, state supply at the root, and the run call, rewrites the counter by hand as a Rust blueprint against the new builder, and replaces the test backend with a harness that is the first World: a few hundred lines of Rust that drive `run` and print a transcript.

### The work

*Step 1: element values.* A new module `element/`: `TagId(u32)` interned through a `Tags` table on the runtime (`intern(&str) -> TagId`, `name(TagId) -> &str`, `namespace(TagId) -> &str`); `ElementValue { tag: TagId, fields: Vec<(FieldId, Value)> }` with structural `Eq` and `Hash`, fields stored in declaration order (`FieldId` is the declaration index) so equality is bitwise over the vector. `Ord` is the language's element ordering: tag name first, then plain fields in declaration order, with the request event excluded as it is from equality. It compares names, never interned `TagId`s, because intern order depends on load order and a sorted map keyed by elements has to iterate the same way in every run and in both implementations. `Value` here is `AnyValue` restricted to plain data: the compiler guarantees it later, and the runtime asserts it in debug builds (`AnyValue::is_plain`, false for anything holding an event, dynamic or blueprint). A `TagKind { requests: Option<TypeId>, responds: Option<TypeId>, state: Option<TypeId> }` registered per tag when interned; the runtime uses only which parts are present, and the `TypeId`s are for the debug assertion at the slot.

*Step 2: paths.* `Path(Vec<Segment>)`, `Segment = Bind(u32) | Key(ElementValue) | Gen(u64)`, `Ord` derived so that tree order is path order. `WidgetBuilder` mints them: `bind` increments the node's bind counter and the child's path is the parent's plus `Bind(n)`; `Rebuild` adds `Gen(g)` with `g` from a per-node counter that never resets, so a rebuilt slot's path is fresh; `Key` is for milestone 3. `Path` is `Display`ed in the worked-examples chapter's notation (`0/#1/0`, `[k]` for keys) because the transcripts use it.

*Step 3: slots and the element list.* On `Widget`: `slots: Vec<Slot>` where `Slot { path: Path, element: ElementValue, response: Option<EventTrigger<AnyValue>>, state: Option<Dynamic<AnyValue>>, request: Option<Event<AnyValue>> }`. `WidgetBuilder::element(&mut self, e: ElementValue, request: Option<Event<AnyValue>>) -> SlotHandles { response: Option<Event<AnyValue>>, state: Option<Dynamic<AnyValue>> }` creates one, assigning the path and asking the state supplier

(step 5) for the dynamic if the tag has state. The node's element list is a derived dynamic `Dynamic<Rc<BTreeMap<Path, ElementValue>>>`, the union of its own slots and its children's lists; rebuild it whole on any child-list change for now, with a comment that an incremental version is an optimisation and must not change a transcript. The root node's list is what the run call returns.

*Step 4: routing and the request collector.* At open, the runtime snapshots the root list's current value into `RuntimeInner::routing: BTreeMap<Path, EventTrigger<AnyValue>>` (the response triggers of every slot in the list as the occurrence opened). A response to a path not in the snapshot is a `RuntimeError` of kind `UnknownPath` naming the path, and aborts the occurrence before any trigger fires. Every slot with a request event subscribes it, at creation, to a per-occurrence collector on the runtime: `RuntimeInner::requests: Vec<(Path, AnyValue)>`, in firing order. A slot torn down in the occurrence still has its firings collected, with its path; the World discards them if it wants.

*Step 5: state supply at the root.* `RuntimeInner::root_supplier: BTreeMap<ElementValue, (Dynamic<AnyValue>, EventTrigger<AnyValue>)>`. When a slot with state is created and no interceptor encloses it (milestone 3 adds the enclosing case), the runtime looks the element value up; on a miss it calls the oracle (the run call's `&dyn Fn(&ElementValue) -> AnyValue`), builds a hold from the answer whose change event is a fresh external trigger, and memoises. Pushes at the start of an occurrence fire the trigger for each pushed value. The hold's value in the creation occurrence is the oracle's answer, which is the snapshot as the occurrence opened; the push arrives in the same occurrence and commits at its end, so a slot created in n ends n holding the pushed value exactly as an older slot does. An entry is dropped when the last slot with that value is torn down (a count on the entry, decremented at teardown).

*Step 6: the run call.* `Runtime::run(&mut self, inputs: Inputs<'_>) -> Result<Outcome, RuntimeError>` with

```
pub struct Inputs<'a> {
    pub responses: Vec<(Path, Value)>,
    pub pushes: Vec<(ElementValue, Value)>,
    pub oracle: &'a dyn Fn(&ElementValue) -> Value,
}
pub struct Outcome {
    pub list: Vec<(Path, ElementValue)>,
    pub requests: Vec<(Path, Value)>,
    pub result: Value,
}
```

Inside: open (routing snapshot, reset the collector), fire the pushes and then the responses at height 0, propagate, commit, read the root list and the result, close. The first call builds the root: `Runtime::build_root<W: Blueprint>(&mut self, w: &W)` runs occurrence zero with no inputs; make it the same code path as `run` with empty inputs, so there is one occurrence loop and not two.

*Step 7: the counter by hand, and the harness.* In `prism_runtime/src/harness/`: `Harness { runtime, snapshot: BTreeMap<ElementValue, AnyValue>, last_list }` with `turn(&mut self, responses, pushes) -> Transcript` and `set_state(&mut self, e, v)`; a `Transcript` with a `Display` that prints the list one slot per line as `path: Tag(field=value, ..)`, then `requested path: value` lines, then `result: value`, in the format the conformance chapter freezes. The harness is not yet the world core: it runs one occurrence per turn and has no `Defer`, which arrives in milestone 3. Then `harness/counter.rs`: `Button`, `Label` and `Counter` as Rust `Blueprint`s against the new builder, `Button` creating a `Ui.Button` slot inside a `rebuild` over its label dynamic and switching its click events, `Label` the same with no response, `Counter` the five lines of the Prism counter in builder calls. This is the fixture milestone 5 has to match, so write it to look like the compiler's output would: one node per bind, a `rebuild` per lifted widget application, a cyclic dynamic for the hold.

*Step 8: delete.* `backends/`, `WidgetDelegate`, `WidgetDelegateContext`, `backend_data`, `add_public_event`, `add_public_dynamic`, the public-slot views on `Widget`, `query_for_tag`, the `__rebuilt__` event, `schedule_trigger` as a public function. `Slots` becomes an `IndexMap` or goes. The tests that used the backend move to the harness, with their names re-stated as sentences.

## Done

Every milestone 1 test still passes. The harness drives the hand-written counter and prints this, byte for byte:

```
turn 0
  0/#0/0: Ui.Button(label="Increment")
  1/#0/0: Ui.Button(label="Decrement")
  2/#0/0: Ui.Label(text="Count: 0")
  result: unit
turn 1  responses: {0/#0/0: click}
  0/#0/0: Ui.Button(label="Increment")
  1/#0/0: Ui.Button(label="Decrement")
  2/#1/0: Ui.Label(text="Count: 1")
  result: unit
turn 2  responses: {0/#0/0: click, 1/#0/0: click}
  0/#0/0: Ui.Button(label="Increment")
  1/#0/0: Ui.Button(label="Decrement")
  2/#2/0: Ui.Label(text="Count: 0")
  result: unit
```

The transcript is saved as `harness/goldens/counter.txt` and the test is a diff. And these pass:

- `the_first_run_builds_main_and_returns_the_initial_list`
- `a_response_is_delivered_to_the_slot_at_that_path_as_the_occurrence_opened` (the relabelling button of the worked examples: label changes in n, click in n lands on the old label's slot)
- `a_slot_created_in_an_occurrence_is_addressable_from_the_next`
- `a_response_to_an_unknown_path_is_an_error_and_the_occurrence_does_not_happen`
- `a_slot_removed_in_an_occurrence_still_receives_input_in_it_and_is_gone_in_the_next`
- `request_firings_are_reported_per_slot_in_firing_order` (one slot firing three times across three turns; two slots firing in one occurrence, reported in list order)
- `a_request_fired_by_a_slot_torn_down_in_the_same_occurrence_is_reported_with_its_path`
- `a_state_slot_has_the_oracle_value_in_its_creation_occurrence`
- `a_push_reaches_a_slot_created_in_the_push_occurrence`
- `two_slots_with_equal_elements_share_one_state_and_one_oracle_call` (count oracle calls)
- `a_rebuilt_slot_gets_a_fresh_path` (`rebuild` built anew twice; three distinct generations)
- `rebuilding_one_sibling_does_not_change_the_others_paths`
- `one_run_call_is_one_occurrence` (run five turns, count occurrences: five)
- `nothing_crosses_the_boundary_but_plain_data` (a compile-time test: `Inputs` and `Outcome` contain no type from `event/`, `dynamic/` or `widget/`; written as a `static_assertions` or a doc comment and a grep in CI)

## What it proves

That a World needs nothing from the runtime but the run call. The harness is a World, and it is the smallest one there will ever be: it holds a snapshot map and a last list, and it never sees an event. Every later World (Test, CLI, GUI, the Haskell ones) is this harness with components, and anything one of them needs beyond `Inputs` and `Outcome` is a design error to be fixed in the design, not an API to add.

It also proves the routing rule and the diode in code: the snapshot at open is the only routing table, and nothing in the runtime reads the list during an occurrence except to build the next one.

### Risks

Paths. A model will be tempted to make a path a pointer or an index into the node tree, because that is cheaper. It has to be a value, ordered in tree order, never reused, because the World keys its bookkeeping by it across occurrences in which the tree changes under it. The test on rebuilding one sibling pins the never-reused part; add `paths_are_in_tree_order` over the list of a two-level tree.

The element list as a derived dynamic over children. The list's change event must fire in the occurrence a child is built, which is the construction rule applied to the list, and a naive implementation that rebuilds the list at commit time gives a list one occurrence stale. The counter transcript catches this (the label's new generation appears in the same turn), but only because the label rebuilds; a program with no rebuild would not notice.

The harness's transcript format will be frozen in milestone 6 and the Haskell implementation will print it too. Keep the printer in one small file with no dependence on the harness so it can move to the world core.

## Milestone 3: interception and `for`

**Waits on** milestone 2.

With the boundary in place the runtime can be driven from outside but not from inside: there is no way for Prism code to stand between a child and the World. This milestone adds the `InterceptBlueprint`, state supply through an interceptor's supplier, the keyed list widget, and the one World feature the runtime's own tests need to exercise interception, which is `Defer` in the harness turn loop. It is the milestone that makes the claim "a sandbox is ordinary code" testable.

### The work

*Step 1: `InterceptBlueprint`.* In `widget/intercept.rs`: `InterceptBlueprint<W: Blueprint> { child: W, responses: Event<Rc<BTreeMap<Path, AnyValue>>>, supplier: Dynamic<SupplierFn> }` where `SupplierFn = Rc<dyn Fn(&ElementValue) -> Rc<dyn DynBlueprint<Dynamic<AnyValue>>>>`, with `Output = (W::Output, Dynamic<Rc<BTreeMap<Path, ElementValue>>>)`. Its `build` builds the child as a child node marked *captured*: the child's element list does not join the parent's. The captured list is the second half of the result. At open, the runtime snapshots routing tables per interceptor as well as at the root; the interceptor's `responses` event, when it fires, delivers each entry to the child slot at that path in the interceptor's snapshot, and an entry for an unknown path is `UnknownPath` with the interceptor's node named. Delivery is at the responses event's height plus one, which is why routing through an interceptor is a delayed edge and the self-referential `!down` of the rewriting interceptor in the elements chapter is legal.

*Step 2: state supply through the supplier.* `WidgetBuilder::element` for a state tag now asks the nearest enclosing captured node first and the root supplier only if there is none. The interceptor node holds `supplied: BTreeMap<ElementValue, (Rc<Widget>, Dynamic<AnyValue>, count)>`. On a miss: read the supplier dynamic's current value, apply it to the element, build the returned blueprint as a child of the interceptor's node right then, in the same build pass, and memoise its result dynamic. The supplied widget's own slots go into the interceptor's node, so they appear in the interceptor's list and not in the captured child list. The last teardown of a slot with that value drops the entry

and the supplied node. The supplier dynamic's current value is read at the instant of creation and never again for that slot: a changed policy affects slots created after the change.

*Step 3: the keyed list widget.* In `widget/keyed.rs`: `Dynamic<Rc<BTreeMap<K, V>>>::keyed_widget(f: impl Fn(&K, Dynamic<V>) -> W) -> impl Blueprint<Output = ...>` where the output is chosen by a `Consolidate` parameter the compiler states: `Dynamic<Rc<BTreeMap<K, O>>>` for dynamic results, `Event<Rc<BTreeMap<K, O>>>` for event results (the entries that fired, in one occurrence), and `()` for unit. One child node per key, path segment `Key(k as ElementValue)`, with a per-key `Dynamic<V>` projected from the map (a `map` over the map dynamic filtered to the key, with the change event dropped when the value is unchanged, so a change to a neighbour does not reach a child). Keys that appear in occurrence n build in n, deferred to `max(height, current + 1)` like a `rebuild` firing, and their first results are in the consolidated output in n: the consolidation is prompt, which means the consolidated dynamic or event is rebuilt at the point in n after the new children exist. Keys that disappear are torn down at commit. `Rebuild` is generalised at the same time: its rebuild replaces a named child rather than child index 0, and `keyed_widget` and `rebuild` share the deferred-build code.

*Step 4: `Defer` in the harness.* The harness grows a turn loop: run the occurrence; for every entry in `requests` whose slot's tag is `Defer`, record the path; if any, run another occurrence with `Click`-like unit responses to those paths and no pushes; repeat. A path answered twice in one turn is an error of kind `DeferBackwards` naming the path. This is the seed of the world core and should be written to lift out: a function `turn(runtime, inputs, &mut defer_state) -> Vec<Outcome>` with no harness state in it.

*Step 5: three hand-written interceptors.* In `harness/interceptors.rs`, as Rust blueprints: `Forward` (a keyed list over the captured list, each entry re-creating the element with the same request event and returning its response; supplier forwards by creating the same element), `Sandbox` (holds back every `Fs.*`, answers each request with `Denied` through a `Defer`, supplies `Unavailable` for every state tag), and `Rewrite` (forwards with a path prefix on `Fs.*` elements). These are fixtures for milestone 5 as the counter was for milestone 2: the compiled `@Forward` from the GUI library must match the hand-written one's transcript.

## Done

All earlier tests pass. The new tests:

- `an_interceptor_that_forwards_everything_is_observably_the_identity` (the counter under `Forward`, and under `Forward` twice; all three transcripts identical)
- `nothing_the_child_creates_reaches_the_outside_unless_the_interceptor_sends_it` (a child with two slots under an interceptor that re-emits one; the root list has one)
- `a_response_through_an_interceptor_is_routed_against_the_childs_list_as_the_occurrence_opened`
- `a_response_to_a_path_unknown_to_the_interceptor_is_an_error_naming_the_interceptor`
- `a_sandbox_supplies_unavailable_and_the_child_has_it_in_its_creation_occurrence`
- `a_sandbox_denies_a_request_in_a_later_occurrence_of_the_same_turn` (the probe of the worked examples: the turn runs two occurrences, the label reads "(permission denied)" at the end of the build turn, and the request never appears at the root)
- `a_supplied_widgets_own_elements_appear_in_the_interceptors_list`
- `two_child_slots_of_equal_value_share_one_supplied_widget` (count builds)
- `a_supplied_widget_lives_until_the_last_slot_with_its_value_is_torn_down`
- `a_slot_keeps_the_supplier_it_was_created_under_when_the_supplier_changes`
- `a_state_tag_the_root_cannot_answer_is_an_error_in_the_creation_occurrence` (the harness's oracle has no answer for the tag; a `RuntimeError` of kind `UnsuppliedState` with the tag and path

in its report)
- `for_builds_one_child_per_key`
- `for_keeps_a_childs_state_across_an_insert_and_a_neighbours_removal` (each child holds a count; click one, insert before it, remove after it, click again: 2)
- `for_consolidates_a_new_keys_result_in_the_creation_occurrence_and_the_new_child_does_not_see_itself`
- `for_consolidates_events_to_the_entries_that_fired`
- `for_consolidates_dynamics_to_a_map_that_is_prompt`
- `a_child_under_for_does_not_rebuild_when_a_neighbour_changes`
- `defer_is_answered_in_a_later_occurrence_of_the_same_turn`
- `a_turn_with_two_defers_runs_three_occurrences`
- `deferring_backwards_is_an_error`
- `a_cycle_through_an_interceptors_responses_that_is_prompt_is_reported` (an interceptor whose responses depend promptly on its child's list; expect `Cycle`)
- `routing_by_a_sampled_layout_is_not_a_cycle` (a VBox-style interceptor that tags a click with the sampled list; expect no error)

### What it proves

That interception is complete: forwarding, holding back, rewriting, denying and supplying are all written against the two downward supplies and `@Element` upward, with no mode of the primitive for any of them. That state supply by building the supplier widget is the mechanism and needs no chain to walk. That the turn, with `Defer`, is a World concept the runtime never sees: the runtime ran one occurrence per call throughout. And that `for` is a widget like any other, with identity from the data's keys.

### Risks

The supplied widget's node is a child of the interceptor's node but was built in the middle of building a grandchild. The build journal from milestone 1 must record it under the right parent, and teardown must not drop it when the grandchild goes (the count does that). A test that fails the occurrence while a supplied widget is being built is worth adding: `a_failed_occurrence_drops_a_supplied_widget_built_in_it`.

Prompt consolidation in `for` means a derived output that is rebuilt after the children are, within the occurrence, at a height above every child's result. A model will get this right for events and wrong for dynamics, or the other way around; both tests are listed.

`Defer` does not belong in the runtime and the harness must not teach it anything: if the runtime ends up with a `Defer`-specific branch, the diode has been broken. The test `one_run_call_is_one_occurrence` from milestone 2 still holds here.

## Milestone 4: frontend to typed AST

**Waits on** nothing. A new crate, `prism_lang`, with no dependency on `prism_runtime`.

This milestone has no runtime in it, and a model given only the language chapters and the chapter on the Rust language implementation can do all of it. Its output is a program with every meaning decided: every lift explicit, every number typed, every binding marked build-time or occurrence-time, every group placed, every tag named. The deliverable that matters most is the text form of that tree, because it is what the goldens diff, what a reader reads to see what the compiler decided, and what

the Haskell frontend must reproduce byte for byte.

## The work

*Step 1: lexer and layout.* `lex.rs` and `layout.rs`: the token set of the lexical chapter (sigils, `<-`, `then`, `|` as merge distinct from the bars of `\|a, b|`, the suffixed operators `+~ +.` and friends, `==>`, `..`, `;`, `ignore`, the comparison operators `< <= > >=` beside `==` and `!=`, the boolean keywords `not`, `and`, `or`, `if` (which guards a `match` arm as well as opening a conditional), and text literals with `{expr}` interpolation: a text literal lexes as literal runs and `{` expression `}` pieces, `\{` and `\}` are literal braces, and `$` inside text is always literal), indent and dedent tokens by the offside rule, `:` at end of line opening a block, and the spacing rule that tells `F -1` from `F - 1` and a prefix `.x` or `[i]` (the pronoun's field or index, wherever `_` is in scope) from `a.x` and `a[i]`. Every token has a `Span { file, start, end }`. The layout corpus is the first test and is a golden: a program in, its token stream out, one token per line.

*Step 2: parser.* `parse.rs` and `ast.rs`: paren-less calls with comma-separated arguments, single parentheses as grouping (a group headed by a call is that call with all its commas, so `(G b, c)` is `G(b, c)`), double parentheses for tuples, tuple types and tuple patterns (`((a, b))`), double brackets for array literals (`[[1, 2, 3]]`, `[[]]`) with single `[i]` always indexing, function types `fn A, B -> C` with parentheses only around a function-typed argument or result, method chains by indentation (a `.Method` that starts a continuation line closes the call above it), `;` between the statements of an inline block, the precedence table (loosest to tightest: `on` and `|`, `or`, `and`, `not`, comparison, arithmetic, so `not a == b` is `not (a == b)`), guarded `match` arms (`pattern if cond:`), `for` with named binders (`for k, $item <- $items:`, the shorthand `for $items:` binding `key` and `$_`), array patterns (`[[]]`, `[[head, ..rest]]`, `[[a, b]]`), the `{expr}` pieces of a text literal parsed as expressions, sigiled names, blocks (`widget`, `rebuild`, `for`, `match`, `intercept`), patterns including element patterns with `..` and `Ns.*(..)`, `tag T of Element:` declarations with `requests`/`responds`/`state`, `type` declarations, lambdas (`\|e| body` and the bare `\body` with `_`), and `test` blocks (parsed now, used in milestone 6). Every node has a span from the first commit.

*Step 3: resolver.* `resolve.rs`: scopes, the pronoun and its sigiled forms, tag namespaces, the prelude, and binding groups: dependency analysis over each widget block into strongly connected components in dependency order. The resolver labels each dependency edge *prompt* or *delayed* (a read through `on`, a hold's own name on the right of `then`, a `rebuild`'s event, `!Switch`), which the cycle check needs.

*Step 4: elaborator.* `elab/`: Hindley–Milner on the value layer with the three built-in traits (equality, ordering, numeric) resolved by inference and the suffixed spellings when it cannot; the functor layer read off the sigils with the lifting rule (dynamics and no event: dynamic; dynamics and one event: event, dynamics read through tags; two events: error); `?` lifting at the plain layer; text interpolation lifting like application (a literal that interpolates a dynamic is a dynamic, and one that interpolates an event is an event); `match` on `?` as its eliminator; guards, with a guarded arm not counting toward exhaustiveness and array patterns counting toward it; application lifting, including a dynamic of blueprints lifting to `rebuild` with `!Switch` or hold-plus-`$Join` by the result kind; `for` consolidation by result kind; tag field types from declarations; the build-time or occurrence-time mark on every binding; the cycle check over the groups with their edge labels; exhaustiveness for closed choices and the catch-all requirement for extensible ones; the tag-indexed refinement of `Slot.!request` and supplier results inside a `match` arm. The output is `typed.rs`: the typed AST.

*Step 5: the text form.* `typed/text.rs`: one binding per line, `name: type [mark] group = expr @span` (the sigil is part of the type), with every lift an explicit node (`dmap`, `map`, `tag`, `hold`, `hold_first`, `leftmost`, `switch`, `join`, `rebuild`, `for` with its can-fire-at-birth flag, `element`, `intercept`, `placeholder`,

`close`), printed and parsed from the same definition (a derive or a single table), with a round-trip test on every fixture. Inferred type variables are named in order of first appearance, left to right, top to bottom, so that two frontends agree. `prism dump foo.prism` prints it.

*Step 6: goldens and diagnostics.* `tests/goldens/`: the counter, each program of the test corpus, `lib/gui.prism`, and one small file per rule of the lexical and expression chapters, including one each for a guarded arm (the guard printed on the arm), a `for` with named binders, an array pattern match, and `not a == b` against `(not a) == b`. `tests/negative/`: a rejected program and its exact error with spans, for each of: two events in one operation (names both, points at `|` and `!Leftmost`); a rejected prompt cycle in a widget block (every binding in it, in order, with spans); a cycle through `rebuild` alone (pins that `rebuild` is not a delay); `a -1` on a `Nat` ("did you mean `a - 1`?"); a `Nat` and an `Int` in one operation, and `+~` on floats ("use `+.`"); a generic function putting its type parameter into an element field; a `match` on an extensible variant with no catch-all; a sigil disagreeing with the inferred kind; a state tag with no supplier at an interceptor whose supplier is a visible lambda. The first four are the ones whose wording the implementation chapter fixes; the rest need the span and the names right, not the prose.

## Done

The layout corpus passes. `prism dump` of the counter is this, character for character, with `@Button` and `@Label` defined above it as in the language implementation chapter:

```
@Button: fn $Text -> @!Click = \|$label| @widget:
    !!clicks <- rebuild $label:
        @Element Ui.Button(label=_)
    @ !Switch !!clicks

@Label: fn $Text -> @unit = \|$text| @widget:
    rebuild $text:
        @Element Ui.Label(text=_)
    @ unit

@counter:
    !increment <- @Button $"Increment"
    !decrement <- @Button $"Decrement"
    let !delta = -1 on !decrement | +1 on !increment
    $count <- 0 then $count + !delta
    @Label "Count: {$count}"
```

```
  @Button: fn $Text -> @!Click = \|$label| @widget:                                  @1:1
    !!clicks: !!Click [build] g1 = rebuild $label \|label| @widget:                        @2:5
      _: !Click [build] g1 = element Ui.Button{label=label} responds                     @3:9
      @ _
    @ switch !!clicks

  @Label: fn $Text -> @unit = \|$text| @widget:                                      @6:1
    _: unit [build] g1 = rebuild $text \|text| @widget:                                    @7:5
      _: unit [build] g1 = element Ui.Label{text=text}                                   @8:9
      @ unit
    @ unit

  @counter: @unit                                                                    @11:1
    !increment: !Click [build] g1 = bind @Button(const "Increment")                  @12:5
    !decrement: !Click [build] g2 = bind @Button(const "Decrement")                  @13:5
    !delta: !Int [occ] g3 = leftmost [map !decrement \|ignore| -1:Int, map !increment \|ignore| +1:Int]   @14:9
    $count: $Int [build] g4 = placeholder                                            @15:5
    $count.hold: $Int [build] g4 = hold 0:Int (tag !delta [$count] \|d, c| Num.Add[Int] c, d)   @15:5
```

```
close $count $count.hold                                                    @15:5
_: unit [build] g5 = bind @Label(dmap [$count] \|c| cat ["Count: ", ToText[Int] c])   @16:5
@ _
```

The conventions are the text-form section's: sigils inside the type, `[build]` or `[occ]` on every binding, the group number shared by a placeholder, its `.hold` definition and its `close`, application printed with parentheses and commas, literals with their type (`0:Int`), `{expr}` interpolation printed as `cat`, and the span of each binding's start.

Every golden and negative golden is in and passes, the guard, binder, array-pattern and `not` precedence goldens among them; the printer and parser round-trip every fixture; the four diagnostics read as the implementation chapter specifies. The crate has no dependency on `prism_runtime`.

### What it proves

That the sigils remove inference over functors: HM runs on the value layer and never sees time, and the lifting rule is a lookup on operand sigils. That every decision about meaning can be made before anything is built, and written down in a form a person can read. And that the specification is precise enough to produce an identical tree from a second reading, which milestone 8 tests.

### Risks

The text form will change during the milestone and that is fine; it must stop changing before milestone 8 starts, so the last step of this milestone is to go through every golden once more and declare the form frozen in the file that defines it. After that, a change to the form is a change to both frontends.

The cycle check needs the resolver's edge labels to be right, and the labelling of `rebuild`'s event (prompt: it fires in the rebuild occurrence) and `!Switch` (delayed) is where a model will guess. The negative golden for a cycle through `rebuild` alone is there for that.

The dynamic-of-blueprints lift is the one place the elaborator inserts a `rebuild` the programmer did not write. Its golden should show `switch (rebuild ...)` for an event-returning widget and `join (hold_first (rebuild ...))` for a dynamic-returning one, and the second form is easy to get wrong (the hold's initial value is the first build's result, which the `rebuild`'s event provides in occurrence zero).

## Milestone 5: the closure compiler

**Waits on** milestones 2 and 4. Its `intercept` and `for` parts wait on 3, and the counter does not need them.

This is where the tracks meet. The closure compiler walks the converted typed AST once and produces an `IrBlueprint`, a Rust value implementing the runtime's `DynBlueprint`, whose `build` walks a tree of pre-resolved steps in group order calling the runtime's constructors. There is no instruction set and no second syntax; the lint is the verifier, run on one form. Done is a single transcript: the counter compiled from source matches the counter written by hand in milestone 2.

### The work

*Step 1: closure conversion.* `convert.rs` over the typed AST: each lambda becomes a function id and an explicit capture list of environment indices; recursive functions refer to each other by id and never hold each other; each cyclic group's placeholder and close are placed where the resolver's order says.

The output is the typed AST annotated, printed by the same text form with the captures shown.

*Step 2: lint.* `lint.rs`: every node's type agrees with its children's; every placeholder has exactly one close; nothing samples a placeholder before its close; every capture names a free variable of the lambda; build order respects the groups; every occurrence-time expression is reachable only from a graph node's closure; no build-time binding reads an occurrence-time one. Run after every conversion and in every test. It has its own negative fixtures: hand-edited converted trees that it must reject.

*Step 3: `IrBlueprint` and the program.* `ir/`: `Program { functions, blocks, tags, constants }`, `IrBlueprint { program, block, captures, span }` implementing `DynBlueprint<AnyValue>`. Plain-value expressions compile to closures over an environment `Vec<AnyValue>`; occurrence-time expressions compile to the closure the corresponding graph node calls. One module, `boxing.rs`, is the only place a Rust `i64`, `f64`, `bool`, `Text` or record becomes an `AnyValue` or comes back, so the representation can change behind it later.

*Step 4: the steps.* One `Step` per typed-AST node kind, each one a call into the builder: `map`, `tag`, `hold`, `leftmost`, `switch_hold`, `join`, `rebuild` over a dynamic of `IrBlueprint`s, `keyed_widget`, `element` with the tag's kind, `intercept`, `placeholder`/`close` with the typed cyclic handles, `bind` of a nested `IrBlueprint`, and `started` as a `rebuild` over a constant. Generic code is erased: a step whose type is a type variable moves the one-word value.

*Step 5: the driver.* `prism run foo.prism --world harness` builds the root `IrBlueprint`, hands it to the milestone 2 harness, and prints the transcript for a sequence of inputs given on the command line or in a small file. The language-level script language is milestone 6; this driver is enough to diff the counter.

## Done

Lint passes on every golden's conversion. The compiled counter, driven by the harness with the same three turns as milestone 2, prints a transcript byte-identical to `harness/goldens/counter.txt`. The compiled `@Forward` from `lib/gui.prism` wrapped around the counter matches the hand-written `Forward` fixture's transcript, once milestone 3 is in. And:

- `a_compiled_hold_reads_its_own_old_value_through_a_tag`
- `a_compiled_dyn_fires_its_event_in_the_build_occurrence_and_every_rebuild`
- `a_compiled_switch_is_non_prompt` (the creation-occurrence firing of a `rebuild` child is not seen through `!Switch`)
- `a_compiled_closure_captures_by_environment_index_and_not_by_rc` (a widget whose occurrence-time closure reads a sibling dynamic; drop the node; the dynamic's node is freed — a `Weak` upgrade fails)
- `compiled_and_hand_written_widgets_sit_in_one_tree` (a Rust `Button` bound inside a compiled widget and vice versa)
- `lint_rejects_a_placeholder_closed_twice`, `lint_rejects_a_sample_before_close`, `lint_rejects_a_capture_of_a_boun`

## What it proves

That the thing executed is construction: a Prism program compiles to code that, run once, calls constructors in the right order with the right captures, and the graph does the rest. That the typed AST is the only intermediate form the language needs today, because the lint checks it and the compiler walks it. And that the two tracks were built against the same reading of the design, which the byte-identical transcript shows better than any review could.

### Risks

Heap cycles through closures. The runtime has no collector because nodes own their closures and closures capture through the owning node's environment, not by `Rc` to a sibling. A compiler that captures a `Dynamic` directly into a closure that the dynamic's own hold subscribes to has made a cycle the runtime will never free. The leak test is listed; run it under a leak checker in CI if one is handy, and in any case assert `Rc::strong_count` where the test can.

The build-time versus occurrence-time split. A model will compile a plain expression that mentions a dynamic as a build-time read of the dynamic's current value, which is sampling, which the language does not have. The typed AST's marks make this a lint error if the lint checks reachability (step 2's last two rules); make sure it does.

Matching the hand-written counter means the hand-written counter has to have the structure of compiler output, which is why milestone 2 said to write it that way. If the transcripts differ only in generation numbers or bind positions, the hand-written one is at fault as often as the compiler.

## Milestone 6: the conformance suite in full

**Waits on** milestones 3 and 5, and the core half of 7.

Everything so far was tested from Rust. This milestone makes the specification testable from Prism: the script language, the Test World as a real World over the world core, the law tests, the programs, and the GUI library written in Prism and run under the Test World. When it is done, a sentence in the language chapters either has a test or is a sentence someone should look at.

### The work

*Step 1: the script language.* `test name, @widget:` forms parsed by `prism_lang` (the parser from milestone 4 already accepts them), one line per turn with the inputs `build`, `idle`, `click`, `send`, `with`, `push` and the checks `expect`, `expect_top`, `expect_none`, `expect_count`, `expect_top_none`, `returns`, `fired`, `quiet`, `requested`, `unrequested`, `errors`, `settles` on `=>` lines. Patterns are element patterns: a tag with named fields and `..`, or `Ns.*(..)`; a value in a pattern is compared structurally. `prism test foo.prism` runs every test block in the file.

*Step 2: the Test World.* In the worlds crate from milestone 7's core half: `TestWorld` is the world core plus one component, `Script`, that accepts every tag. Its oracle answers from the `with` and `push` lines. Tags declare no defaults, so a state element with no `with` line is an `UnsuppliedState` error at the slot's creation: a script gives a `with` line before the build for every state tag the program creates at build, which a `with` line may do; its responses are the `send` and `click` lines; it records request firings, results and errors for the `requested`, `fired`, `quiet` and `errors` checks. Depth: `expect` and `send` reach into containers by the GUI library's convention (a `children` field holding the keyed element list; a `ChildInput(path, response)` response to route inward), and the Test World knows the convention because the library is part of the suite.

*Step 3: the transcript.* The printer moves from the harness to the world core, prints a turn as its occurrences (each with its inputs, list, request firings and result, and an error if any), and the format is frozen in `transcript.md` beside it with one example per feature. `prism test --transcript` prints it; the suite stores one per test and diffs. Nothing after this milestone changes the format without changing the file and every stored transcript.

*Step 4: the law tests.* One short Prism program and script per sentence the conformance chapter

lists: leftmost merge; the tag law; a hold's initial value and adoption in the build occurrence; derived dynamics prompt and at most once; `rebuild` firing in its build occurrence and every rebuild; `!Switch` non-prompt including the swallowed creation-occurrence firing; `$Join` prompt on both changes; the mid-occurrence construction rule including a nested `rebuild` over a dynamic that changed in the same occurrence building once with the new value; `for` consolidation per result kind, state kept across an insert and a neighbour's removal, a new key's result seen in the creation occurrence; the creation rule and `@started`; routing against the opened list; unknown path is an error; a state element's creation-occurrence value, a push reaching a new slot, two equal slots agreeing; the forwarding identity; the sandbox supplying `Unavailable`; a supplied widget's elements in the interceptor's list; two equal child slots sharing one supplied widget; `Defer` later in the turn, backwards is an error, two `Defer`s are three occurrences. Each file is named after its sentence.

*Step 5: the programs.* The twelve integration programs brought to the current syntax (numbers by inference, `tag` declarations for the `Ui` set, `Click` as a unit type, the `intercept` signature with its three arguments) and three more: `@started` kicking a request answered in a later turn; a `rebuild` child whose creation-occurrence output is swallowed at the boundary; a `for` whose parent sees a new key's output in the creation occurrence. Each with its transcript stored.

*Step 6: the GUI library in Prism.* `lib/gui.prism` as the suite's library: the tag declarations

```
tag Ui.Label of Element:
    text: Text

tag Ui.Button of Element:
    label: Text
    responds: Click

tag Ui.VBox of Element:
    children: Path ==> Element
    responds: ChildInput
```

and `@Button`, `@Label`, `@VBox`, `@Forward`, `@HBox`, with `@VBox` as the interceptor of the elements chapter: capture the children's list, re-emit the `Ui.*` ones inside one `Ui.VBox` element, forward the rest unchanged, route `ChildInput(path, response)` inward. This is the library the GUI Worlds of milestone 10 run, and it is written now, under a World with no screen, so that layout and routing are tested before any pixel exists.

## Done

`prism test` passes on every law test, every program and the library's own tests, under the Rust implementation, and every transcript is stored and frozen. The counter test from the worked examples chapter, including `counter_left_wins` and the sandbox script with its corrected expectations, passes as written there.

## What it proves

That the specification is the thing: a test names elements and values, never a path or a node, and passes under any World that delivers the same inputs. That GUI is a library: `@VBox` is an interceptor and `@Button` is `@Element` plus a filter, and both run under a World that cannot draw. And that the transcript is enough to compare two implementations, which milestone 9 does.

### Risks

Patterns in scripts need matching on element values with structural comparison of fields, including nested element values in a `children` field. This is a small interpreter and a model will put it in the Test World; it belongs in `prism_lang` beside the element-pattern code the compiler already has, so that the Haskell side has one less thing to re-derive (it reads the same script and must match the same way).

The depth convention is the library's and the Test World must not grow a second one. If a test needs to reach into something that is not a `children` field, the library grows a convention and the Test World follows it.

Bringing the twelve programs to current syntax will surface sentences in the language chapters that two readers would read differently. Each one is a fix to the book first and to the program second.

## Milestone 7: the world core and the CLI World

**Waits on** milestone 3 for the core; the CLI World's end-to-end tests wait on 5.

A World is small because the language cannot do anything to it: it receives plain data and sends plain data back, never registers a callback, never holds a runtime object. Everything a World has to get right is the short list of obligations in the Worlds chapter, and that list is the same for every World, so it is written once. The CLI World is the first World that runs Prism against a real operating system and the smallest one that is not a test.

### The work: the core

A crate `prism_world`:

- `World<C: Component>`: holds the runtime, the snapshot (`BTreeMap<ElementValue, AnyValue>`), the last list received, and the component. `turn(&mut self, inputs: TurnInputs) -> TurnOutcome` runs the occurrence, checks for fired `Defer` requests, runs the follow-up occurrences, enforces once-per-slot-per-turn and the deferring-backwards error, and returns every occurrence's outcome. This is the harness's turn loop from milestone 3 lifted out, and the harness is rewritten to use it.
- Snapshot bookkeeping: `set_state(e, v)` records into the next snapshot; before each occurrence the oracle closure answers from the previous snapshot and the pushes are the diff; the swap happens after the occurrence.
- Slot tracking: a response for a path not in the last list is discarded with a diagnostic before the runtime sees it (the runtime would error; the World's obligation is to not send it).
- `Unsupported`: for a request on a tag no component accepts, a response of `Unsupported` in the next turn; for a presence tag, an error report; for a state tag, the unsupplied-state error at creation, surfaced from the runtime.
- The `Component` trait: `tags(&self) -> &[TagId]`; `perform(&mut self, path, element, request: AnyValue)` for a request firing; `poll(&mut self) -> Vec<Response>` for completions since last asked; `answer(&self, element) -> Option<AnyValue>` for the oracle; `changes(&mut self) -> Vec<(ElementValue, AnyValue)>` for pushes; `created(path, element)` and `removed(path, element)` for presence and stream elements. The core calls components; components never see the runtime.
- Error reporting: one `Report` type with the error kind, spans, the path, the tag and the interceptor chain, and one `Display` every World prints.

Tests, with a fake component: `a_turn_is_the_occurrences_before_the_world_acts`; `defer_is_answered_once_per_turn_`

`the_oracle_answers_from_the_previous_snapshot_and_the_pushes_are_the_diff`; `a_slot_created_in_the_push_occurrence`
`a_late_response_for_a_gone_slot_is_discarded_with_a_diagnostic`; `an_unsupported_request_tag_gets_unsupported_in_t`
`an_unsupported_presence_tag_is_reported_and_ignored`; `request_firings_are_performed_in_occurrence_order_then_list`

## The work: the CLI World

A crate `prism_cli` with a binary `prism`:

- The loop: a `std::sync::mpsc` queue of inputs; the main thread blocks on it, runs one turn per input, performs the turn's requests, and repeats. Reader threads (stdin, signals, file IO, timers, sockets) push inputs; `tokio` is not needed for this and should not be pulled in until a component needs it.
- Components, with the kinds the language fixes and the names from the Worlds chapter: `Process.Args` and `Process.Env` (state, constant after start); `Cli.Stdin` (responds with `Chunk Text` then `Eof`; read only while a slot exists); `Cli.Stdout` and `Cli.Stderr` (requests, written after the turn in occurrence order then list order, then flushed; `WriteFailed` as a response); `Process.Exit` (`requests: !Code`; ends the run after the turn's writes); `Cli.Signals` (responds; with no slot, OS defaults); `Fs.File` (requests, responds), `Fs.Contents` (state, stat-and-reread, or a watch where one exists; the oracle reads synchronously up to a size cap and answers `Unavailable(TooLarge)` beyond it, and `Unavailable(reason)` takes its reason from the closed `FsError` choice: `NotFound`, `Denied`, `TooLarge`, `Other(text)`); `Time.After` (requests, responds), `Time.Every` (responds), `Clock` (state, pushed once per turn at most); `Net.Socket` (requests `!Bytes`, responds `Received Bytes | Closed`, state `Connecting | Connected | Disconnected Reason`), which milestone 10's X11 proof needs; `Defer` from the core.
- Exit: on `Process.Exit`'s request, after the turn; and with code 0 when nothing can cause another occurrence (no stdin slot or its `Eof` delivered, no request outstanding, no stream slot that can fire); subscriptions to args and env do not keep a program alive. This is a consequence of the graph having no sources left, and the test for it is a program that prints and falls off the end.
- Golden transcripts: program, args, env and stdin in; stdout, stderr and exit code out, run against the real binary: the line counter on empty input, one line, many lines, lines split across chunks; falling off the end; a computed nonzero exit code; two `Cli.Stdout` slots firing in one occurrence and stdout interleaved with stderr across turns; args and env; a file read with `@started` before stdin is built inside a `rebuild`; a `Ui.Button` at the root getting `Unsupported`; SIGINT with and without a `Cli.Signals` slot. Timers are faked through the Test World, so no golden depends on the clock.

## Done

The core's tests pass with the fake component; the harness runs on the core; the Test World of milestone 6 runs on the core. The line counter from the worked examples chapter passes its golden transcripts against the real OS, and so does every transcript above.

## What it proves

That a World is an afternoon's work once the core exists: the CLI World is components and a queue. That the `Defer` rule bounds a turn without a cap. That the run call's "plain data in both directions" survives contact with threads and a real OS, because the reader threads produce values and the main thread performs requests, and nothing else crosses.

### Risks

Performing requests after the turn, in occurrence order then list order, is the obligation a model will perform during the turn instead, because the firing is right there. The ordering test with two stdout slots catches the list-order half; add one where a request's response would arrive in the same turn if performed early (a `Defer` answered after a write) to catch the timing half.

Exiting when nothing can cause another occurrence needs the core to know which slots can fire. A component declares, per tag, whether a slot of that tag is a live source; the core sums. Getting this wrong hangs a program or exits it early, and both goldens (falling off the end; a program with a `Time.Every` that must not exit) are listed.

## Milestone 8: the Haskell frontend

**Waits on** milestone 4 freezing the typed-AST text form. Nothing else; a second person or model can take it with the language chapters, the Haskell chapter, and the frozen text form.

A specification with one implementation is a description of that implementation. The Haskell frontend reads the language chapters a second time, independently, and the test of whether the chapters are a specification is a `diff` on the dumps.

### The work

A cabal project `prism-hs`, with modules `Prism.Lex`, `Prism.Layout`, `Prism.Parse` (hand-written or megaparsec; the precedence table is small; guarded `match` arms, `for` binders and array patterns are parser work), `Prism.Resolve` (groups and edge labels), `Prism.Elab` (HM on the value layer with the three traits; the sigil lifting rule; marks; the cycle check; exhaustiveness), `Prism.Typed` (the tree), `Prism.Typed.Text` (printer and parser from one definition, with the same type-variable naming rule), and an executable `prism-hs dump`. Its tests: the layout corpus; every golden from milestone 4 and 6 as a diff between `prism dump` and `prism-hs dump`; the negative goldens, compared on the error kind and spans, with the prose allowed to differ.

A script `tools/diff-frontends.sh` that runs both dumps over every `.prism` file in the suite and reports differences, run in CI.

### Done

`diff` on the dumps of every program in the suite is empty. Every negative golden is rejected with the same kind and spans.

### What it proves

That the language chapters are a specification. Every divergence found on the way is either a bug in one frontend or a sentence two careful readers read differently, and the second kind goes back into the book; the milestone is as much a review of the language chapters as it is code.

### Risks

Hindley–Milner has choices in it that the text form exposes: where let-generalisation happens, how a type variable is named, whether a monomorphic group's variables are generalised after the group. The language implementation chapter has to state each one, and if a divergence traces to one it does

not state, the fix is a sentence in that chapter and then both frontends.

A model writing this with the Rust frontend's source in hand will reproduce its bugs. Do not hand it the Rust source. Hand it the book and the goldens.

## Milestone 9: the Haskell evaluator and Test World

**Waits on** milestones 6 and 8.

The Haskell runtime is Reflex plus a thin layer that gives it slots and the run call. The typed AST is interpreted, not compiled, in a monad over Spider; most of the semantics of the functor and time chapters are inherited, and the work is the boundary and the three places Reflex offers the wrong thing next to the right one.

### The work

- `Prism.Eval`: `PrismM t` as `ReaderT Env (DynamicWriterT t [Slot] (PerformEventT ... (SpiderHost)))` (the transformer names from memory) or the equivalent, with the correspondence table from the Haskell chapter as the implementation: `!A` is `Event t A`; `$A` is `Dynamic t A` with `current` used only where the typed AST says `tag`; `|` is `leftmost`; `v on !e` is `tag (current v) e`; hold is `holdDyn`; `rebuild` is `rebuild (f <$> x)`; `!Switch` is `switchHold never`; `$Join` is `join`; `for` is `listWithKey`; `@started` is a `rebuild` over a constant and never `getPostBuild`; the element list is the writer; `intercept` runs the child's writer and passes the supplier and the outer `tell` down in the reader; state supply runs the supplied widget in the outer context at slot creation, memoised per interceptor by element value; the run call is `runSpiderHost` with `fireEventsAndRead` over trigger refs from `newEventWithTriggerRef` (both names from memory).
- `Prism.World`: the world core in Haskell, a few hundred lines: turn loop, `Defer`, snapshot, slot tracking, `Unsupported`, the transcript printer in the frozen format.
- `Prism.World.Test`: the Test World over the script language, parsed by `Prism.Parse`.
- `prism-hs test foo.prism`, and `tools/diff-transcripts.sh` over the whole suite.

A review checklist, applied by reading the evaluator once it exists: no `current` outside the three places the Haskell chapter names (the `tag` node, routing against the opened list, the supplier read); no `switchPromptly` or `switchPromptlyDyn`, and no `coincidence` outside `for`'s birth term; no `getPostBuild`; no `performEvent` that does anything but collect request firings; no `sample` in a build that the typed AST did not mark.

### Done

The conformance suite passes under `prism-hs test`, and `diff` on transcripts between the two implementations is empty for every law test and program.

### What it proves

That the semantics are Reflex's where they overlap, and that the boundary (slots, paths, the run call, state supply, `Defer`) can be put on Reflex without changing it. Also that two runtimes with different internals (a height-ordered action map; Spider's subscriber graph) produce the same transcripts, which is the strongest evidence available that the transcripts are determined by the language chapters and not by an implementation.

### Risks

Reflex's `dyn` and `networkView` fire their event in the rebuild frame and Spider's `subscribeAndRead` gives the mid-occurrence construction rule for free, but `listWithKey`'s consolidation of a new key's result in its creation frame is a thing to check, not assume: the law test for it will say.

The evaluator is an interpreter over a tree, and a model will reach for `current` whenever it needs a value at build time. Everything at build time is a build-time binding in the typed AST, and those are plain values; a build-time read of a dynamic does not exist. The checklist is the defence.

## Milestone 10: GUI Worlds

**Waits on** milestones 6 and 7; the reflex-dom World also waits on 9.

GUI is not a language concept and not a World concept. Some Worlds know how to put pixels on a screen and read a pointer and a keyboard, and offer tags for that; layout, hit-testing, focus, text editing, buttons and lists are Prism interceptors over those tags and run unchanged under any World that offers them. This milestone makes that claim true three times over, with three Worlds that share one Prism library, and the third one has no host GUI code in it at all.

### The vocabulary

Five tags, declared in `lib/gui.prism` beside the widgets of milestone 6. A window, the picture in it, the name of the focused region, a way to measure text, and a tick per frame.

```
type Pointer = choice:
    Moved Point
    Down Point, Button
    Up Point, Button
    Scroll Point, Vector

type Key = choice:
    Char Char
    Named NamedKey                  # Left | Right | Backspace | Tab | Enter | ...
    Composition CompositionEvent    # IME: Start | Update Text, Nat | Commit Text | End

type WindowEvent = choice:
    Resized Extent
    CloseRequested
    Pointer Pointer
    Pressed Key

tag Gui.Window of Element:
    title: Text
    size: Extent                    # the size the program asks for when the window opens
    responds: WindowEvent

tag Gui.Scene of Element:
    window: Text                    # the title of the window it paints
    draw: [Draw]

tag Gui.Focusable of Element:   # presence: the name of the focused region
    name: Text

tag Gui.Metrics of Element:
    text: Text
```

```
    font: Font
    state: Extent                  # width and height of the text; answered synchronously

type Frame = record:
    now: Instant

tag Gui.Frames of Element:
    responds: Frame                # a GUI World fires this before it paints
```

The draw commands are values in the scene's `draw` list, in paint order, and never tags:

```
type Draw.Rect = record:
    rect: Rect
    color: Color

type Draw.Text = record:
    origin: Point
    text: Text
    font: Font
    color: Color
    ime_at: ?Nat                   # an offset into text where an input method's candidate window goes

type Draw.Image = record:
    rect: Rect
    image: ImageId

type Draw.Clip = record:
    rect: Rect
    draw: [Draw]

type Draw = Draw.Rect | Draw.Text | Draw.Image | Draw.Clip
```

`Gui.Metrics` is state because a layout needs the width of a string in the occurrence it lays out. The World does no geometry and no hit-testing: the window responds with every pointer event over it, the library knows what is under the pointer, and focus is by name, a `Text` hold in the library with at most one `Gui.Focusable` emitted at a time, so a World has one place to send keys, the window, and the `Gui.Focusable` is only for the platform's focus ring, input method candidate window and screen reader.

The library over these has its own tags under `Ui` (`Ui.Label`, `Ui.Button` responding `Click`, `Ui.TextInput`, `Ui.VBox`, and so on), which never reach a World: its interceptors claim them. `@Window` is the interceptor at the top. It captures the children's `Ui.*` elements, lays them out using `Gui.Metrics` readings, emits one `Gui.Window` and one `Gui.Scene` holding the whole draw list, hit-tests the window's `Pointer` responses against the layout it holds (read through a tag as the occurrence opened), and routes each hit inward as a `Click`, and each `Pressed` key to the input whose name is the focus. Focus is a hold of the focused name moved by the Tab key and by clicks, and only the focused child's `Gui.Focusable` goes up. `@VBox` and `@HBox` assign positions; `@TextInput` keeps its text and cursor as holds outside any `rebuild`, draws `Draw.Text` runs and a caret `Draw.Rect` placed from a `Gui.Metrics` reading, and receives keystrokes through the library's routing by focus name, from the window's key responses. `@Button` and `@Label` are written over `Draw.Rect` and `Draw.Text`; the milestone 6 versions remain as the Test World's simpler vocabulary. The library's tests run under the Test World first, which sends `Pointer` and `Pressed` responses to the `Gui.Window` slot: a click at a point lands on the right button; Tab moves focus by name; typing moves the cursor; a resize re-lays out.

## The software rasteriser World

A crate `prism_gui_soft`: `winit` for the window and input, `softbuffer` for the pixels, a small rasteriser for rectangles and clipped text with a bitmap or `fontdue` font for `Gui.Metrics`. Components: `Gui.Window` (opens the window, responds with `Resized`, `CloseRequested` and every pointer event as `Pointer` and every key event from `winit` as `Pressed`, a turn per event), `Gui.Scene` (after each turn the draw list is rasterised into the buffer in list order and presented), `Gui.Focusable` (presence; the World notes it to show a focus ring and place the input method's candidate window), `Gui.Frames` (a turn per frame while a slot exists), `Gui.Metrics` (the font's advances, synchronously), plus the CLI World's general components. Which drawing backend is used has no language consequence; `softbuffer` is chosen because it has no GPU dependency and its output can be screenshotted into a golden PNG in a test (from memory).

## The reflex-dom World

In `prism-hs`: the same five tags rendered to the DOM. `Gui.Scene` becomes absolutely positioned `div`s and `span`s, one per `Draw.*` entry, with `Draw.Clip` as a `div` with `overflow: hidden`; `Gui.Window` listens for pointer, key and resize events; `Gui.Focusable` becomes a focusable element that gives the browser a focus ring and a screen reader a name, its key events bubbling to the window listener; `Gui.Metrics` is answered from a hidden `canvas` measurement; a turn per DOM event. This is the cheap way to see Prism programs on a screen, and it is the Haskell implementation doing something the Rust one cannot yet.

## The X11-over-sockets proof

A Prism program, under the CLI World, with no GUI World at all. `lib/x11.prism` is an interceptor `@X11` over the GUI vocabulary: it opens a `Net.Socket` to the X server, speaks the X11 wire protocol (connection setup, `CreateWindow`, `MapWindow`, `PolyFillRectangle`, `ImageText8`, event reads for `ButtonPress`, `KeyPress`, `Expose`, `ConfigureNotify`; request and event names from memory), turns its children's `Gui.Scene` draw lists into request bytes on the socket's `!request` event, and turns `Received` bytes into `Pointer`, `Pressed` and `Resized` responses for the `Gui.Window` slot. It answers its children's `Gui.Metrics` itself through the supplier, with a fixed-width font, so measurement is pure. The counter runs on an X display with the CLI World as the only host code, and the CLI World knows nothing about windows.

## Done

The counter and a text input, from the same `lib/gui.prism`, run on a screen under each of the three, and the library's layout, hit-testing and focus tests pass under the Test World. For the software rasteriser: a golden PNG of the counter after two clicks, compared pixel for pixel. For reflex-dom: a headless-browser test that clicks the button and reads the label. For X11: run under `Xvfb`, take a screenshot with `xwd` (both from memory), compare to the golden; and the transcript of the `Net.Socket` requests for the first three turns is stored and diffed, which pins the protocol bytes without a display.

## What it proves

That one GUI vocabulary serves many Worlds, that layout and focus and text editing are Prism, and, with the X11 interceptor, that "World" and "GUI" are different axes: a World with sockets and nothing else can put pixels on a screen because a Prism program spoke the protocol. The X11 proof is the one that answers the question a sceptic asks first, whether the primitives are really low enough

that the toolkit is in the language.

### Risks

Text. Measurement by a state element is right, but a World whose fonts change (a font loaded later) pushes new metrics for the elements it has answered, and the layout must handle the push by re-laying out, which it does by being a derived dynamic over the measurements. Write the layout that way from the start and the risk is gone.

Hit-testing is the library's and reads its layout as the occurrence opened. A click delivered in the occurrence after a resize lands on the old layout; that is the design (the tag law, applied to the layout), and a test under the Test World pins it so that nobody moves hit-testing into a World.

The X11 interceptor is a few thousand lines of Prism and will find holes in the plain-value library (bytes, integer encoding, slicing). Those are prelude work, not language work, and the milestone should say which prelude functions it added.

## Milestone 11: errors

**Waits on** milestone 3. It can be slotted anywhere after; it is last here because nothing above needs it, and the Test World's `Errors` step is the only thing in the suite that reads it.

The transaction from milestone 1 already makes an occurrence abortable. This milestone decides what the World is told, contains a failed `rebuild` build, and gives the Test World its `Errors` step.

### The work

- `RuntimeError` gains the full report: the kind, the closed `ErrorKind` (`Cycle`, `Overflow`, `UnknownPath`, `UnsuppliedState`, `DeferBackwards`, `Component`, `Build`), the spans of every binding involved (for a cycle, the whole cycle in order, with the Prism spans the compiler attached to each cyclic slot), the path, the tag and the interceptor chain for a protocol error, and which occurrence of which turn. The compiler names every cyclic placeholder after its source span so that a runtime cycle reports Prism source.
- Containment at the build boundary: a failed `rebuild` build does not fire and is reported through `Errors`. A cycle or native failure found while a `rebuild` or `for` builds a block in the middle of an occurrence abandons that build, not the occurrence. The block's construction already has a boundary (the deferred build from milestone 3); wrap it so that an error inside drops the half-built node, leaves the slot holding an `Error` presence element in the block's place, does not fire the `rebuild`'s event, and records a `RuntimeError` of kind `Build` in `Outcome.errors`, which the world core hands to its `Errors` element. The rest of the occurrence continues. The `Error` tag is declared in the prelude.
- World reporting: the world core's `Report` prints the above; the CLI World prints to stderr and exits nonzero; the Test World records it for `Errors pattern`.
- Overflow in `Nat` and `Int` arithmetic aborts the occurrence with kind `Overflow`; `Checked.*` (returning `?`) and `Wrap.*` (wrapping) are prelude functions bound to Rust's `checked_*` and `wrapping_*`; recursion past the closure compiler's depth guard is also `Overflow`; the transaction is the backstop for native panics, which are kind `Component`.
- Quarantine of the owning subtree (replace the lowest common owning widget's node with an `Error` element in the next occurrence) as a World option on the core, off by default, with one test showing it on.

### Done

- `a_cycle_at_runtime_aborts_the_occurrence_and_reports_every_binding_in_it_with_prism_spans`
- `an_aborted_occurrence_leaves_the_program_as_it_was_and_the_world_is_told`
- `a_cycle_found_while_a_dyn_rebuilds_does_not_fire_the_dyn_and_is_reported_through_errors`
- `a_failed_dyn_build_leaves_an_error_element_in_the_list`
- `the_rest_of_the_occurrence_continues_after_a_failed_dyn_build`
- `a_protocol_error_names_the_path_the_tag_and_the_interceptor_chain`
- `overflow_aborts_the_occurrence_with_kind_overflow`
- `checked_functions_return_nothing_on_overflow_and_wrap_functions_wrap`
- `quarantine_replaces_the_owning_subtree_with_an_error_element_when_the_world_asks_for_it`
- the `Errors` step passes in the Test World on each of the above written as Prism tests.

### What it proves

That the transaction was worth having: the error path is cheap, coherent and silent to the program, and the one smaller unit (a failed construction) falls out of the `rebuild` boundary that already exists. And that errors a program is meant to handle are values and nothing else: there is no `catch`, and an interceptor that wants to handle a child's failure handles an `Error` element like anything else a child emits.

### Risks

Whether aborting an occurrence opened by a `Defer` answer aborts the rest of the turn is a World decision; the core should abort the turn and report, because a half-turn is a state no World was written to act on, and a World that wants otherwise can say so. If that reasoning turns out wrong, the change is in the core and not the runtime.

## What is not on the plan

Each exclusion buys something, and the thing it buys is named so that the exclusion can be revisited when the price changes.

*Values representation.* `AnyValue` stays an enum over a boxed trait object and a few inline integers until a profile says otherwise. What this buys: milestones 2 through 10 never wait on a representation decision, and `boxing.rs` from milestone 5 is the one place to change. The urgent subset when the profile arrives is inline `i64` and `f64` and text as one allocation. Constraint: nothing here may change a transcript, and the suite is the test.

*Register code and native backends.* The closure compiler is the only backend. What this buys: one intermediate form, one lint, no verifier between two forms, and no instruction set to keep in step with the runtime's constructors. When something has to ship compiled, register code is a lowering from the converted typed AST, checked against the closure compiler by the differential test, and the target (Cranelift or emitted Rust) is picked then. There is no LLVM.

*`&` references, `read` and `update` blocks.* Chapter 3's, with its own constraints (an `update` block is an expression with a value; no reference escapes it; nothing about occurrences is involved). What this buys: no interaction between references and erasure to design before the language exists. It belongs in the Haskell implementation first, where lens libraries make the experiment cheap.

*User-defined traits and modules beyond one file plus the prelude.* Ordinary language work with no

bearing on time. What this buys: the elaborator of milestone 4 is HM plus three built-in traits, which a model can write in one pass.

*Parallelism.* Single-threaded is a decision about the first implementation. The runtime's structure allows fork-join per height (a second runtime variant with `Sync` graph types), optimistic occurrences (the rollback path of milestone 1 is half of it), and sharded runtimes under one World (no runtime support needed). None is needed to make the language exist.

*A World that wraps a native toolkit.* A different World with higher tags (`Gtk.Button`). What the exclusion buys: the three GUI Worlds of milestone 10 share one library and prove the vocabulary; a toolkit World proves nothing about the language and is a library question.

*Incremental compilation, a language server, a package format.* Each is wanted and none changes the typed AST, which is the line that makes them later work rather than now work.

*A scheduler, dirty tracking, re-entrancy guards beyond the one flag, deferred effects, an unsubscribe story, a collector, locks.* Not deferred: absent by design, and each milestone's "what it proves" is partly the proof that the absence holds.

## Working with a model on a milestone

The author will execute this plan with a model writing most of the code and will read every line. That changes what a milestone brief should contain and what a review should look for, and the plan was cut with it in mind.

### What to hand it

For every milestone: this chapter's section for the milestone, verbatim; the chapters of this book the milestone cites (the runtime milestones cite the time, widgets, elements and runtime chapters; the frontend milestones cite the lexical, expression, functor and language-implementation chapters; the Worlds milestones cite the elements, Worlds and conformance chapters; the Haskell milestones cite the Haskell chapter and the language chapters); the worked-examples chapter, always, because its walkthroughs are the transcripts the milestone has to reproduce; and the code from the milestone it waits on, as a checkout, with its tests passing. Nothing else: no design notes, no chat history, no other implementation's source. For milestone 8 in particular, not the Rust frontend.

Hand the whole set at once. A model that gets the runtime chapter without the elements chapter will invent a boundary.

### What to ask for, in order

Ask for four things, as four replies, and read each before asking for the next.

First, a file plan: every file to be added or changed, with one line each on what goes in it, and the list of tests it intends to write with their names as sentences. This is a page and it is where most of the review's value is: a wrong decomposition is visible here and invisible in two thousand lines of code. Push back on any file that does not map to a step in the milestone, any test name that is not a sentence about the language, and any type in a public signature that the design says does not cross a boundary.

Second, the tests, before the implementation, as failing tests. For a runtime milestone these are Rust tests against the builder; for a frontend milestone they are goldens and negative goldens, with the

golden text written by hand from the book (the model can write the expected dump of the counter from the typed-AST section before the elaborator exists, and that hand-written golden is the check on the elaborator). For a Worlds milestone they are transcripts written by hand from the worked-examples chapter. A test written after the code tests the code; a test written before it tests the book.

Third, the implementation, in the step order the milestone gives, with the suite green after each step. Ask for the steps as separate commits so that each can be read on its own and reverted on its own. A step that touches every file (milestone 1's first step; milestone 2's deletion step) is one commit with nothing else in it.

Fourth, a list of the sentences in the book the model found ambiguous, wrong, or missing while doing the work, each with the choice it made. This is the milestone's second product, and it goes back into the book before the next milestone is handed out, because the next model reads the book and not the previous model's choices.

## How to check its work

Run the suite, and then read, in this order: the public signatures (do `Inputs` and `Outcome` hold only plain data; does the `Component` trait see the runtime; does the evaluator's `PrismM` expose `current`); the invariants (is `check_invariants` called after every occurrence in every test; is every `borrow_mut` during propagation behind the re-entry flag; does the lint run on every conversion); the tests (is each one a sentence; would it fail if the sentence were false; is there a test that tests structure rather than language, to be deleted); and only then the code.

Each milestone has a short list of things a model will do because they are easier and that the design forbids, and the review should grep for them:

- Milestone 1: `Arc`, `Mutex`, `unsafe impl Send`, `resume_unwind`, `already borrowed` in any test output.
- Milestone 2: any public function on `Runtime` other than `run`, `build_root` and `check_invariants`; any `Event` or `Dynamic` in a type the harness uses; a path that is an index or a pointer.
- Milestone 3: a `Defer` branch in the runtime; a supplier table or tag table in `InterceptBlueprint`; a consolidated `for` output that fires one occurrence late.
- Milestone 4: a dependency on `prism_runtime`; any inference over sigils; a `rebuild` edge labelled delayed; a text form with two printers.
- Milestone 5: an `Rc<Dynamic>` captured into a closure; a build-time read of a dynamic's value; an instruction enum.
- Milestone 6: pattern matching in the Test World rather than in `prism_lang`; a second depth convention; a transcript printed two ways.
- Milestone 7: a request performed during a turn; `tokio` before a component needs it; a component holding a runtime object.
- Milestone 8: the Rust frontend's source in the model's context; a text form that differs in type-variable names.
- Milestone 9: `current` outside its three named places; `switchPromptly`; a `coincidence` outside `for`'s birth term; `getPostBuild`; `performEvent` doing work.
- Milestone 10: layout, hit-testing or focus in a World; a text input with a dynamic field; host code in the X11 proof.
- Milestone 11: a `catch`; an error the program can observe other than as an `Error` element.

Then run the differential checks the milestone enables: the counter transcripts (2 against 5), the forwarding fixtures (3 against 5), the dumps (4 against 8), the transcripts (6 against 9), the library

under three Worlds (10). A milestone whose differential check passes has been reviewed by the other track, which is a review no reader can do alone.

### When the model is stuck

A model stuck on a milestone is usually stuck on a sentence, and the fix is to find the sentence. Ask it to quote the part of the book it is implementing and say what it thinks the sentence means; the gap is usually visible in the paraphrase. If the sentence is wrong or missing, fix the book, then re-hand the milestone; do not patch the code around a sentence the book does not have, because the Haskell reading will not have the patch.

A model that proposes a change to the design in order to finish a milestone (a mode on `intercept`, a callback in the run call, a sampled read at build time, a `Defer` the runtime understands) is proposing the thing a milestone exists to prove unnecessary. The answer is the hypothetical-alternative paragraph in the relevant chapter, and if that paragraph does not exist, writing it is the work.

### The order to hand things out

Day one: milestones 1 and 4, to two sessions, because they share nothing. Then 2 and, when 4's text form is frozen, 8 to a third session. Then 3 and the core half of 7. Then 5, which is the first one whose done criterion is a diff between two sessions' work. Then 6 and the CLI half of 7 together, then 9, then 10 and 11 in whichever order a screen or an error message is wanted first. Every milestone after the first two begins with a passing suite from the one before it and ends with a larger passing suite, and nothing in between waits on anything but tests.

## Designs that wait on a fact

Nothing in this chapter may change a transcript. That sentence is the whole discipline of the chapter: every design in that list is either a representation or scheduling choice behind semantics the other chapters fix, or a language extension that adds programs without changing what any existing program means. Each one, in its own chapter, says what it is, the afternoon that makes someone want it, its design as far as it has been taken, the things it must leave alone, and the fact that has to exist before it is worth building. The order is roughly the order in which the pressure to do each one will arrive: a profile first, then a large record, then a binary to ship, then a cycle found in production, then a program too big for one file.

Where each one lives:

- *Values representation* — chapter 11.
- *`&` references, `read` and `update` blocks* — chapter 3.
- *Native backends* — chapter 12.
- *Causality in types* — chapter 9.
- *User-defined traits and modules* — chapter 3.
- *Incremental compilation* — chapter 12.
- *Parallel runtimes* — chapter 11.
- *Hot reload of a widget subtree* — chapter 7.
- *Persistence of holds* — chapter 7.
- *A package system* — chapter 3.
- *Performance and constant factors* — chapter 11.
- *Real time: a tick World and a bounded occurrence* — chapter 7.

## What every one of them has to answer

A reactive language with two implementations and a conformance suite has one asset that nothing here may spend: the transcript. A Prism program under the Test World produces a sequence of element lists, request firings and results, determined by its inputs and nothing else. The Rust runtime and the Haskell evaluator are checked against each other by diffing those transcripts, and the suite is what makes "the spec is the thing" more than a slogan. So the test every one of them has to pass before any other is: does the differential test still come out empty? For the representation and backend items the answer must be yes by construction, since Prism values have no identity and a program cannot tell whether a value was copied, shared, interned or reused. For the language extensions the answer must be yes for every program that compiled before.

The second thing each item has to answer is what triggers it. The plan chapter ends with a list of things that are not on it, and the reason is not that they are unimportant but that each one waits on a fact that does not exist yet: a profile showing the value enum is the cost, a program that cannot be written without `update`, a World that cannot ship with the frontend inside it. Doing any of them before the fact is in hand means designing against a guess, and every one of them is large enough that a wrong guess is expensive. The triggers are written down so that when the fact arrives, the decision is already made.

## What they have in common

Every item above waits on a fact. None of the facts exists yet, and the specification is complete without any of the items, which is the test each had to pass to be in this chapter rather than in the specification chapters.

What none of those subsections says is the order the facts are likely to arrive in and which items block which, and that is worth one paragraph, because an item started out of order is designed against a guess. The world core comes first and is on the plan already; persistence needs it, since `Store.*` is a component the core hosts, and hot reload needs the GUI World built on it. Modules come before packages, because a package is a namespace of modules and has nothing to name without them; traits come beside modules, since coherence is checked at import. The value representation's urgent subset can go in at any time; its full design needs the threading decision made as a feature flag, because the non-atomic count and the per-site caches both assume one thread, and fork-join needs that same flag decided the other way, so the two are one decision with two builds. `update` reuse needs the value representation, since copy-on-write along a path is a property of the heap object's count and layout. Native backends need nothing above them but a consumer, and that consumer is the likeliest to be a World that ships without the frontend, which is a GUI World. Causality in types needs only the GUI library's layout widgets to exist and be used, and is the one item here whose trigger is a bug report. Incremental compilation needs modules, because the unit of invalidation is a definition and a definition's dependencies cross files. Shards need nothing; optimistic occurrences need shards to have been tried first. So the likely sequence is: world core, persistence, modules and traits, the urgent subset, a GUI World, hot reload, and only then anything that needs a profile.

## Chapter 16

# Appendices

Four reference tables, each for a reader who already knows the design and wants one page to check against. Appendix A sets Prism beside Reflex construct by construct, because Reflex is the direct ancestor and almost every choice in Part II is a choice to keep something of Reflex's or to remove it; the notes under the table name the Reflex construct, say what Prism does instead in one worked pair, and point at the chapter that owns the argument. Appendix B lists the prior art, one entry per system, with what each contributed and which claims are quoted from a source. Appendix C is the syntax on one page. Appendix D is the element vocabulary: every tag the design uses anywhere, with its fields, its parts, its identity, and one line of meaning, so that the names stay the same from chapter to chapter and from the specification to the Worlds that implement it.

## A. The Reflex correspondence

You have written the Reflex program where a `Dynamic` was sampled with `current` in one place and read through `updated` in another, and the two disagreed by one frame, and you spent an afternoon finding out which was right. Most of what Prism changes about Reflex is on that afternoon's path. The table gives the mapping; the notes after it take the rows where Prism and Reflex differ, show the two spellings side by side, and name the chapter that argues the choice.

| Prism | Reflex | In one phrase |
|---|---|---|
| `!A` | `Event t a` | the same thing |
| `$A` | `Dynamic t a` | behavior half hidden |
| (no spelling) | `Behavior t a`, `current`, `sample` | removed; only the tag law reads *before* |
| `@A` | `m a` for the widget monad; `PrismM t a` in the Haskell implementation | the same thing, minus the transformer stack |
| widget block, `<-`, `@ v` | `do`, `<-`, `return` | the same thing |
| `?A` | `Maybe a` | a sigil, lifted at the plain layer |
| lifting by sigils | `fmap`, `<$>`, `zipDynWith`, `ffor` | inferred from spelling, not written |
| `v on !e` (dynamic `v`) | `tag (current v) e`, `attachWith` | the one window onto *before* |
| `v on !e` (plain `v`) | `e $> v`, `v <$ e` | the same thing |
| `a \| b`, `!Leftmost [[..]]` | `leftmost [a, b]` | the same thing |
| `!never` | `never` | the same thing |
| `$x <- v then !e` | `holdDyn v e` | a statement, so it has an owner |
| `!Updated $x` | `updated x` | a prelude function; never fires at build |

| Prism | Reflex | In one phrase |
|---|---|---|
| `$Distinct $x` | `holdUniqDyn` | the only deduplication; holds never dedupe |
| `match $x:` with plain arms | `ffor x $ \case ..` | lifted `match` |
| `match !e:` with plain arms | `fforMaybe`, `push` | lifted `match`; `!Filter` is `fmapMaybe` |
| `!Switch !!ee` | `switchHold never ee` | the only switch; non-prompt |
| (none) | `switchHoldPromptly`, `switchPromptlyDyn`, `coincidence` | removed |
| `$Join $$dd` | `join` on `Dynamic` | the same thing, prompt |
| `rebuild $x: ..` | `dyn`, `networkView` | fires in every build occurrence incl. the first |
| hold of `!$B` + `$Join` | `widgetHold`, `networkHold` | the cache idiom; no primitive |
| (none; `rebuild` covers it) | `runWithReplace`, `Adjustable` | the class under `rebuild` |
| `for $m:` | `listWithKey`, `listHoldWithKey` | keyed, prompt consolidation (the Haskell implementation's one `coincidence`) |
| `for $array:` | `simpleList` | index-keyed, discouraged in both |
| `@started` | `getPostBuild` | a `rebuild` over a constant; same occurrence everywhere |
| recursive bindings in a block | `rec` / `mdo`, `MonadFix` | implicit; P8 checks the cycle |
| prompt-cycle error | `EventLoopException` "causality loop detected" | the same failure, reported with spans |
| request element (`requests` + `responds`) | `Requester`, `requesting` | plain data; routed by path |
| state element (`state`) | `MonadQuery`, `queryDyn` | has a value in its first occurrence |
| the element list `$(Path ==> Slot)` | `DynamicWriterT`, `tellDyn` | an ordinary dynamic |
| `intercept` | `runDynamicWriterT` + `runRequesterT` + `runQueryT`, stacked | one primitive, two directions |
| routing responses by path | `fan`, `select`, `fanMap` | the runtime's job |
| the run call | `runSpiderHost`, `fireEventsAndRead`, `newEventWithTriggerRef` | plain data in, plain data out |
| (none) | `performEvent`, `performEventAsync` | removed; the World performs |
| (none) | `newTriggerEvent` | removed; only the World starts an occurrence |
| the occurrence | a Spider frame | the same thing, named as a transaction |
| mid-occurrence construction rule | Spider's `subscribeAndRead` | the same behavior, as a law |
| `Defer` element | (improvised per application) | one element, turn semantics |

| Prism | Reflex | In one phrase |
|---|---|---|
| (none) | `Incremental`, `Patch` | removed; a `for` over a map is the patch. A patch type would return if `for` over a large map proved too slow to diff by key |

## The behavior half is hidden

Reflex exposes `current`, `updated` and `sample` on every `Dynamic`; Prism's `$` has no `current`, and the only way to turn a dynamic into a plain value is `rebuild`, which rebuilds on change. Chapter 4 ("No sampling, and what it buys", "The hidden behavior") owns the argument; `current` survives in exactly one place, the Haskell implementation's evaluator, where a typed-AST tag node is `tag (current d) e`.

The spelling that costs the afternoon, and what replaces it:

```
-- Reflex: a build-time read that looks like a dynamic and is a constant
label0 <- sample (current label)
el "span" $ text label0
```

```
# Prism: the same intent is a rebuild; the label is live, and a change rebuilds
rebuild $label:
    @Label _
```

## Lifting replaces `fmap`, `zipDynWith` and friends

Reflex writes each lifting rule as a combinator (`fmap`, `zipDynWith`, `tag`, `attachWith`, `ffor`, `<*>`); Prism reads the rule off the operands' sigils at every application, and the type-class constraints on a Reflex signature collapse into the `@` on a function's name. Chapter 3 owns lifting; this gives up functions polymorphic over whether their argument is a dynamic, and the rule that a plain `let` lambda lifts afresh at each application gives most of that back (`let F = \|x\| x + 1` serves `F 3`, `F $n` and `F !e`).

```
-- Reflex: three things said, one of them the tag law
count' <- holdDyn 0 $ attachWith (+) (current count) delta
```

```
# Prism: the tag law is the reading of a dynamic at an event
$count <- 0 then $count + !delta
```

## Only one switch, and it is not prompt

Reflex has `switchHold`, `switchHoldPromptly`, `switch` and `coincidence`, with its own documentation recommending the non-prompt form unless you are absolutely sure; Prism's `!Switch` is `switchHold never` and there is no other. Chapter 5 ("`!Switch`, `$Join`, and why they differ") owns the argument, whose core is the element boundary: a block rebuilt in occurrence n cannot have any event addressed to it in n, so a prompt switch could only ever deliver construction noise from it.

```
-- Reflex: the choice is per call site, and the prompt one is the trap
clicks <- switchHoldPromptly never =<< rebuild (button <$> label)
```

```
# Prism: binding a dynamic of widgets is the rebuild plus the one switch
!clicks <- @Button $label
# the click of the occurrence that relabelled the button goes to the old slot, under the old label
```

## There is no PostBuild

`getPostBuild` fires once after the enclosing action has finished building, and "finished" is the instance's to define, so the frame differs between the root and the inside of a `rebuild`; `@started` is a `rebuild` over a constant, so it fires in the build occurrence everywhere. Chapter 5 ("`@started`") owns the definition. `@started` serves the one use of post-build that survives the removal of sampling, firing a request as soon as a widget exists:

```
-- Reflex: in a later frame at the root; in the rebuild frame inside dyn
pb <- getPostBuild
resp <- requesting (Read path <$ pb)
```

```
# Prism: in the build occurrence, wherever the widget is
!started <- @started
!resp <- @Element Fs.File(path=path, requests=(Read on !started))
```

## Elements instead of `Requester` and `MonadQuery`

`Requester`'s method documentation says a response occurs at a later time than its request, which is Prism's creation rule stated by the ancestor; a request element is that idea with the request site a path in the element list rather than a key in a transformer's state. `MonadQuery` is the state half, and its result dynamic has whatever the runner's results dynamic holds when the child is built, which for a query nobody has seen is `mempty`; a state element gets its value from an oracle answered as the occurrence opened. Chapter 6 ("Why state has to be there in its first occurrence") owns the argument.

```
-- Reflex: the first frame has mempty next to fresh captured dynamics
res <- queryDyn (constDyn (Lookup key))
let user = ffor res $ \r -> Map.lookup key (unResult r)   -- Nothing for a frame
```

```
# Prism: a value in occurrence zero, with the outside's unavailability in the type
$user <- @Element Store.Value(key=key)       # $Stored: Absent | Present bytes
```

Reflex can express a query as a group with a `crop`, so the interest set is patched rather than re-sent; Prism cannot: an interceptor that wants the aggregation writes a `for` over the child's slots, groups by target, and forwards one. That is more code than `QueryMorphism`, and it is ordinary code. The aggregation Reflex gets from the query monoid, Prism gets from value identity: two slots whose elements are equal observe one state, supplied once.

## The boundary is plain data

A Reflex application improvises its host: `runSpiderHost`, trigger refs, `fireEventsAndRead`, `performEvent` for effects and `liftIO` anywhere in the widget monad, so two Reflex applications with the same widgets can differ about simultaneity. Prism has one run call, chapter 7 ("The run call") owns it, and the Haskell implementation uses `fireEventsAndRead` and trigger refs to implement that call and nowhere else (chapter 14, "What must not be used").

```
-- Reflex: the host decides what fires together
fireEventsAndRead [tClick :=> Identity (), tTick :=> Identity now] readOutputs
performEvent_ $ liftIO . writeFile path <$> saves
```

```
# Prism: the program fires a request; the World performs it after the turn
@Element Fs.File(path=path, requests=(Write $draft on !save))
```

### `listWithKey` is `for`

`listWithKey :: Dynamic t (Map k v) -> (k -> Dynamic t v -> m a) -> m (Dynamic t (Map k a))` is `for` almost exactly, and `simpleList` is the array form both discourage; `for` adds consolidation by result kind, and the consolidated event is prompt, so the parent sees a new key's first firing in the key's creation occurrence without choosing a switch. Chapter 5 ("`for`") owns the walkthrough. The insert-at-front case, in both:

```
-- Reflex: simpleList rebuilds every row; listWithKey builds one
rows  <- simpleList items row            -- insert at index 0: every row rebuilt
rows' <- listWithKey itemsById row       -- insert key 7: one row built
clicks <- switchHold never $ mergeMap <$> updated rows'   -- which switch decides the first firing
```

```
# Prism: keys are identity, and the consolidated event is the result
!clicks <- for $items_by_id:
    !c <- @Button $"Remove {key}"
    @ key on !c
# insert key 7 in occurrence n: one block built in n; its @started would be in !clicks in n
```

# B. Prior art

Each entry says what the system contributed and sorts its claims into two tiers. *Verified* means the quote is in the book's research notes or in the Reflex sources kept beside them, copied exactly; a quoted sentence under that heading can be relied on. *From memory* means the claim is general knowledge about the system with no quoted source behind it: it is stated as a paraphrase, never in quotation marks, and a URL after it is where a reader who wants to rely on the claim should look. The design leans on the from-memory tier for its intuitions, not for any law, and nothing in Part II depends on a sentence in that tier being exact.

## Elliott, push-pull FRP

**Contribution.** The denotations. A behavior is a value over time, an event is a sequence of timed occurrences, and they are different types; piecewise-constant behaviors, a value plus the discrete changes, are what pushing can implement, and continuous time is what forces pulling. Prism's `$` is Elliott's reactive value restricted to the piecewise-constant case, which is why it can be implemented by propagation and why the language has no notion of a behavior sampled between occurrences. The restriction is on purpose and is a divergence from Elliott's own position, which treats continuous time as essential.

**Verified.** Elliott, "Push-pull functional reactive programming", Haskell Symposium 2009, pp. 25–36 (http://conal.net/papers/push-pull-frp/). Abstract: "Functional reactive programming (FRP) has simple and powerful semantics, but has resisted efficient implementation."; "most past implementations have used demand-driven sampling, which accommodates FRP's continuous time semantics and fits well with the nature of functional programming. Consequently, values are wastefully recomputed even when inputs don't change, and reaction latency can be as high as the sampling period. This paper presents a way to implement FRP that combines data- and demand-driven evaluation, in which values are recomputed only when necessary, and reactions are nearly instantaneous."; the paper introduces "reactive normal form" among its new friends. The Hackage `reactive` package (https://hackage.haskell.org/package/reactive) describes itself as having "a hybrid demand/data-driven implementation" "as described in the paper". Fran (Elliott and Hudak, ICFP 1997), via a published summary: "Behaviors are first-class values, and are built up compositionally"; the design

aims to "clearly distinguish between modeling and presentation".

**From memory.** The paper body's `Reactive a = a 'Stepper' Event a`, its description of a reactive value's meaning as a step function fully defined by an initial value and discrete changes, and its remark that reactive values can be evaluated in push style, leaving pull for time functions. Elliott's position that discrete-only systems are not FRP in his sense, defended on his blog.

## Reflex

**Contribution.** Spider's frames as atomic transactions; `Dynamic` as behavior plus event with the rule that the behavior changes exactly when the event fires; the hold law (a sample in the frame that sets the behavior sees the old value), which is Prism's tag law; non-prompt switching as the recommended form; `dyn` and `networkView` firing in the rebuild frame; `subscribeAndRead` as the mechanism for construction in the middle of a frame; `listWithKey`; `Requester`'s statement that a response is later than its request. Also the negative lessons: exposed `current` and `sample`, `PostBuild` whose frame depends on where it is used, and `MonadQuery` whose results have no first value. Appendix A has the row-by-row account.

**Verified.** From the Reflex sources (github.com/reflex-frp/reflex, develop branch, post-0.9; the version number is not stated in the files read) and reflex-dom's `Basic.hs`:

- `Class.hs`: Behavior, "A container for a value that can change over time. 'Behavior's can be sampled at will, but it is not possible to be notified when they change"; Event, "A stream of occurrences. During any given frame, an 'Event' is either occurring or not occurring"; Dynamic, "Basically a combination of a 'Behavior' and an 'Event', with a rule that the 'Behavior' will change if and only if the 'Event' fires"; `hold`, "if the occurrence that sets the 'Behavior' (or one that is simultaneous with it) is used to sample the 'Behavior', it will see the **old** value of the 'Behavior', not the new one"; `tag`, "Replace each occurrence value of the 'Event' with the value of the 'Behavior' at the time of that occurrence"; `leftmost`, "If multiple occur at the same time the value is the value of the leftmost event"; `coincidence`, "You maybe looking for '@switchHold@ @never@' instead."; `switchHold`, "Only the old event is considered the moment a new one is switched in; the output event will fire at that moment only if the old event does", and "Use this rather than 'switchHoldPromptly' and 'switchHoldPromptOnly' unless you are absolutely sure you need to act on the new event in the coincidental case."
- `Dynamic.hs`: `switchPromptlyDyn`, "The timing requirements that switching before imposes are likely to bring down your app unless you are very careful."; `tagPromptlyDyn`, "**When creating cyclic data flows, generally @/tag (current d) e/@ is preferred.**"
- `Requester/Class.hs`: `requesting :: Event t (Request m a) -> m (Event t (Response m a))`; "Semantically, the response event occurs at a later time than the request event."; module doc, "collect many potential sources of events and send them over a single channel, then distribute the results back out efficiently to their original request sites".
- `Query/Class.hs` and `Query/Base.hs`: `askQueryResult :: m (Dynamic t (QueryResult q))`; `queryDyn q = do tellQueryDyn q; zipDynWith crop q <$> askQueryResult`; `runQueryT :: (MonadHold t m, Commutative q, Group q, Reflex t) => QueryT t q m a -> Dynamic t (QueryResult q) -> m (a, Incremental t (AdditivePatch q))`.
- `PostBuild/Class.hs`: "the intent is to allow 'Behavior's and 'Dynamic's to be safely sampled, regardless of where they were created, when the post-build 'Event' fires. The post-build 'Event' will fire exactly once for an given action."
- `Network.hs`: `networkView :: (NotReady t m, Adjustable t m, MonadHold t m) => Dynamic t (m a) -> m (Event t a)`, "The returned Event of network results fires at post-build time and

when the 'Dynamic' updates"; `networkHold`'s own documentation suggests flattening its result with `switchPromptlyDyn`, the function whose documentation says it is likely to bring down your app. reflex-dom `Basic.hs`: `dyn = networkView`, `widgetHold = networkHold`.

- `Adjustable/Class.hs`: `runWithReplace :: m a -> Event t (m b) -> m (a, Event t b)`; the class is for actions that "adjust itself so that its net effect is as though it had originally been run with the new value", with the documentation itself flagging "persistent side-effects: obviously, IO" as the problem case.
- `Spider/Internal.hs`: `subscribeAndRead :: Event x a -> Subscriber x a -> EventM x (SubscribeResult x a)`, where `SubscribeResult` holds `subscribeOccurrence :: Maybe a` (a reading of the record: the occurrence value if the event already fired in the frame); the comment "hold cannot examine its event until after the phase is over"; `EventLoopException` shown as "causality loop detected"; `coincidence` needs its own machinery in the propagation loop (`SubscriberCoincidenceOuter/Inner`, `SomeResetCoincidence`), which is evidence that prompt switching costs the engine re-subscription and height resets mid-frame.

**From memory.** The `Reflex` class requiring `Monad (Dynamic t)`, which is why `join` on dynamics exists. `listWithKey`'s full constraint list and `simpleList` keyed by index (`Collection.hs`). `fireEventsAndRead` and `newEventWithTriggerRef` in `Host/Class.hs`; `performEvent`'s documentation saying its output generally occurs later than its input because most actions cannot be performed during propagation; `newTriggerEvent :: m (Event t a, a -> IO ())`. `MonadDynamicWriter` collecting dynamics with `tellDyn` and combining them monoidally. The readthedocs description of `RecursiveDo` as the way to make a cyclic graph, and of `networkView`/`networkHold` as the equivalents of `dyn` and `widgetHold`. reflex-dom's `Builder/Immediate.hs` building an input element's value as a hold over the leftmost of a set-value event and a UI event, with an initial value read from the DOM at build (the build-time read that Prism does not have). `switchPromptlyDyn`, `fanMap`, `select`, and `listWithKeyShallowDiff` as described in the correspondence table.

## Esterel, Lustre, Lucid Synchrone

**Contribution.** The instant as the unit in which nothing half-done is observable. A synchronous program is a totally ordered sequence of logical instants; a signal is present or absent in an instant and has one value there; a signal's value may not be used in its own computation in the same instant, and the compiler rejects the attempt as a causality cycle. Prism's occurrence is that instant, its glitch-freedom law is the consistency the instant gives, and P8 is a small causality analysis: a cycle is legal only through a delayed edge, which is Lustre's `pre`. The synchronous languages are designed for controllers with fixed topology; Prism keeps the law and adds `rebuild` and `for`.

**Verified.** Edwards, "The Synchronous Language Esterel", Columbia lecture notes (https://www.cs.columbia.edu/~sedw
02/esterel.pdf): "Esterel's model of time is synchronous like that used in hardware. Threads march in lockstep to a global clock."; "Time divided into sequence of discrete instants"; "Each signal is either present or absent in each cycle"; "Can't take multiple values within a cycle"; "A signal emitted in a cycle is visible immediately"; contradictory programs "are erroneous; the Esterel compiler refuses to compile them."; causality is a "Dynamic concept: whether there's a"guess-free proof" that concludes a signal is absent". Von Hanxleden, Kiel "Synchronous Languages" lecture 2 (https://www.informatik.uni-kiel.de/~rvh/ss20/v-synch/lectures/lecture02.pdf): "Computations considered to take no time (synchrony hypothesis)"; "A system works in perfect synchrony, if all reactions of the system are executed in zero time."; "In practice, 'zero time' means before the next interaction"; "Each signal is only present or absent in a cycle, never both"; "All writers run before any readers do"; `present A else emit A end` is "an erroneous program."; "Fully deterministic behavior". Palshikar, "An Introduction to Esterel", embedded.com (https://www.embedded.com/?p=4024644):

"the underlying machine is infinitely fast and, hence, the reaction of the system to an input event is instantaneous."; the "hypothesis of atomic reactions"; "Esterel has no facility to refer to any of the past or future instants."

"All writers run before any readers do" is the synchronous-language statement of the occurrence as a transaction; `present A else emit A end` is the synchronous form of a prompt cycle, and the compiler's refusal is the static version of the acyclic-prompt-graph rule.

**From memory.** Berry's *Esterel v5 Language Primer* and its phrases for the perfectly synchronous or zero-delay model, for a valued signal having one status and one value at a time, and for the causality cycle (https://www.rw.cdl.uni-saarland.de/people/kaestner/private/es0203/esterel_primer.pdf). The primary papers: Berry and Gonthier, Science of Computer Programming 1992; Halbwachs, Caspi, Raymond and Pilaud on Lustre, Proc. IEEE 1991; Benveniste et al., "The synchronous languages 12 years later", Proc. IEEE 2003. Lustre's `pre` (the previous cycle's value) and `->` operators. Lucid Synchrone's `when` as a sampler between fast and slow processes, its clocks as boolean sequences, and `current` holding a signal between samples with a default used before the first sample (https://www.di.ens.fr/~pouzet/lucid-synchrone/manual_html/manual006.html); that default-before-first-sample is the hold's initial value.

## STM and transactional memory

**Contribution.** Buffered writes, commit at the end, cheap abort, and one snapshot per transaction. The occurrence is a transaction in exactly this sense: holds buffer their new values and commit at the end, an aborted occurrence leaves the graph as it was, and the World answers the oracle from one snapshot per occurrence. Both the error story (abort the occurrence, report, continue) and the parallelism story (optimistic execution across occurrences, later) lean on this.

**From memory.** Haskell's `stm` and `retry`, which abandons the current transaction because it has seen values that mean it should not continue (https://hackage.haskell.org/package/stm/docs/Control-Monad-STM.html); Harris, Marlow, Peyton Jones and Herlihy, "Composable Memory Transactions", PPoPP 2005, as its origin. PostgreSQL's Repeatable Read, which sees a snapshot as of the transaction's first statement and asks applications to be prepared to retry on serialization failure (https://www.postgresql.org/docs/current/transaction-iso.html). Datomic's database value as an immutable point in time, so that several queries against one value are known to see the same data (https://docs.datomic.com/whatis/data-model.html). The design uses only the common idea: a snapshot per transaction, writes visible at commit, abort as a cheap and complete undo.

## Edge and level: epoll, kqueue, interrupts, inotify, ZooKeeper, Kubernetes, Debezium, Kafka, market data

**Contribution.** Every one of these systems rediscovered the same two facts. A subscriber needs a snapshot and the changes since, consistent with each other, keyed by a version so that the seam between them is exact. And "you will get a change notification eventually" is not a value: an edge without the level it refers to is a wake-up, after which the program must read. Prism's state elements are that pattern as a language rule: the oracle is the snapshot, the pushes are the changes, the occurrence is the version, and a slot created in n reads the oracle (before n) and then receives n's push, so it ends n holding what every other slot holds. The creation rule is the other side: a stream element reports changes after it exists and nothing at start, because a first event that said "here is what it is now" would be a query answering a question nobody asked.

**Verified.** Kafka Streams, Confluent documentation (https://docs.confluent.io/platform/current/streams/concepts.html):

"A stream can be considered a changelog of a table"; "A stream is thus a table in disguise"; "A table can be considered a snapshot, at a point in time, of the latest value for each key"; "A table is thus a stream in disguise"; "Essentially, this duality means that a stream can be viewed as a table, and a table can be viewed as a stream."; "A KTable is an abstraction of a changelog stream, where each data record represents an update." Hanlon, "What is tearing?", React 18 working group (https://github.com/reactwg/react-18/discussions/69): "In a user interface, by"tearing" we mean that a UI has shown multiple values for the same state." (the UI-side name for a subscriber whose snapshot and changes disagree).

**From memory**, each with where to check it. The design leans on this entry hardest, so a reader who wants to quote any of it should fetch the source first; the numbers below in particular are approximate.

- epoll (https://man7.org/linux/man-pages/man7/epoll.7.html): the interface behaves as either edge-triggered or level-triggered; in edge-triggered mode an event is delivered only when the descriptor changes, so the man page's pipe example, where more is written than is read, hangs on the next wait despite data still buffered; the recommended edge-triggered use is nonblocking descriptors read until `EAGAIN`; level-triggered is described as a faster `poll`.
- kqueue (https://man.freebsd.org/cgi/man.cgi?query=kqueue): `EV_CLEAR` resets an event's state after it is retrieved, for filters that report transitions rather than current state; a socket read filter also reports how many bytes are available, which is the level beside the edge.
- Interrupts (https://en.wikipedia.org/wiki/Interrupt): an edge-triggered interrupt can be missed while interrupts are masked, unrecoverably unless hardware latches it, and modern hardware has status registers that edge-driven handlers check so nothing is missed. The Linux generic IRQ layer's edge handler loops while the pending bit is set and has a delayed-disable scheme against losing edges while disabled (https://docs.kernel.org/core-api/genericirq.html).
- inotify (https://man7.org/linux/man-pages/man7/inotify.7.html): the event queue can overflow and then events are lost, with a single overflow event always generated; robust applications must handle lost events, possibly by rebuilding their cache; after adding a watch on a new directory, scan it at once; pairing move-from and move-to events is inherently racy.
- ZooKeeper (https://zookeeper.apache.org/doc/current/zookeeperProgrammers.html): a watch is a one-time trigger; a client sees the watch event before the new data, and never sees a change it watched for without first seeing the event; a client cannot reliably see every change; the zxid exposes the total order of all changes.
- Kubernetes (https://kubernetes.io/docs/reference/using-api/api-concepts/): the watch mechanism lets a client fetch current state and then subscribe to changes without missing any; the server keeps a few minutes of history by default; on `410 Gone` the client clears its cache, re-lists, and watches from the returned resource version; the Go client's Reflector is the standard list-then-watch tool. The controllers guide (https://github.com/kubernetes/community/blob/master/contributors/devel/sig-api-machinery/controllers.md): level driven, not edge driven; an object seen with a marker of true gives no assurance of having seen it turn from false; distinct watches update independently. The API conventions: behavior is level-based rather than edge-based, for robustness against missed intermediate states.
- Debezium's PostgreSQL connector (https://debezium.io/documentation/reference/connectors/postgresql): the initial snapshot records the log position, then scans the tables, then streams from the recorded position, so no change made during the snapshot is missed.
- Binance's guide to a local order book (https://github.com/binance/binance-spot-api-docs/blob/master/web-socket-streams.md): buffer the stream, fetch a depth snapshot, discard buffered events at or before the snapshot's update id, restart if the first buffered event is newer

than the snapshot, and restart if a gap appears in update ids; the snapshot has a fixed depth per side, so levels outside it are learned only when they change. CME MDP 3.0 market recovery: a last-sequence-number field on the snapshot against which cached incremental updates are dropped, and packet loss indicated by a sequence gap. Coinbase Exchange: queue messages, fetch the REST snapshot, play back the queue discarding sequence numbers at or before the snapshot's. FIX market data requests: a subscription type of snapshot, or snapshot plus updates, the latter sending the current state and then updates as they occur.

## React

**Contribution.** Three places where React's later additions mark where the model leaked, used in Part I as the comparison: `useSyncExternalStore` is a consistency check with a blocking restart, added after tearing was named; Suspense is a mechanism that interrupts a render when a value is not there yet and retries the subtree from scratch; the compiler is manual memoization admitted to be tedious and easy to get wrong. The retry-from-scratch of a suspended render is also the closest cousin of Prism's rule that a widget built in n behaves as if built before n's inputs: a render that cannot finish consistently is thrown away and rerun, not resumed.

**Verified.** `useSyncExternalStore` (https://react.dev/reference/react/useSyncExternalStore): "While the store has not changed, repeated calls to `getSnapshot` must return the same value."; "The store snapshot returned by `getSnapshot` must be immutable."; during a transition "React will call `getSnapshot` a second time just before applying changes to the DOM. If it returns a different value than when it was called originally, React will restart the update from scratch, this time applying it as a blocking update, to ensure that every component on screen is reflecting the same version of the store." The hook's page does not use the word "tearing"; the React 18 working group's discussion defines it (quoted under the edge-and-level entry), and the React 18 release post says the hook works "by forcing updates to the store to be synchronous". Suspense (https://react.dev/reference/react/Suspense): "`<Suspense>` lets you display a fallback until its children have finished loading."; "React does not preserve any state for renders that got suspended before they were able to mount for the first time. When the component has loaded, React will retry rendering the suspended tree from scratch." Effects (https://react.dev/reference/react/useEffect): "**Notice that you can't "choose" the dependencies of your Effect.** Every reactive value used by your Effect's code must be declared as a dependency."; "By suppressing the linter, you"lie" to React about the values your Effect depends on."; Strict Mode "will **run one extra development-only setup+cleanup cycle** before the first real setup." React Compiler (https://react.dev/learn/react-compiler/introduction): "This manual memoization is tedious, easy to get wrong, and adds extra code to maintain."; the compiler "automatically applies the equivalent of manual memoization, ensuring that only the relevant parts of an app re-render as state changes, which is sometimes referred to as 'fine-grained reactivity'." Batching (https://react.dev/learn/queueing-a-series-of-state-updates): "**React waits until *all* code in the event handlers has run before processing your state updates.**"

**From memory.** The implementation of Suspense as a thrown promise, with `ReactFiberThenable.js`'s `SuspenseException` describing itself as not a real error; the documentation names only `use` and `lazy` as activators and does not promise the mechanism. The "Keeping components pure" page's description of Strict Mode calling each component's function twice in development.

## Solid and signals

**Contribution.** Fine-grained dependency tracking without re-rendering, which is the closest mainstream thing to `$`; and the observation that glitch freedom is a property each implementation engineers

rather than one the model gives, which is the reason Prism states it as a law and orders propagation by the prompt graph. A signal graph has behaviors and no events; Rx has events and no behaviors; FRP has both as types.

**Verified.** Solid documentation (https://docs.solidjs.com/concepts/intro-to-reactivity, https://docs.solidjs.com/advanc concepts/fine-grained-reactivity): "Signals are like mutable variables that can point to a value now and another in the future."; "Effects are functions that are triggered when the signals they depend on point to a different value."; "During this run, if the effect accesses any signals it is then registered as a subscriber to those signals."; "When a signal changes, any corresponding subscribers are immediately updated in an ordered manner."; "`batch` can be used to delay an update so the subscriber runs after each signal has been updated." Neither page uses the words "glitch" or "topological"; consistency is told through `batch` and "ordered manner". TC39 Signals proposal (https://github.com/tc39/proposal-signals): "Computation is glitch-free, meaning no unnecessary calculations are ever performed." (a definition of glitch-free as no wasted work, not as never observing an inconsistent state); "A computed Signal always observes the Signal graph in a consistent state,"; "Computed signals are lazy, i.e., pull-based"; "The Signal API does not include any built-in function like `effect`."; the `Signal.subtle` namespace holds "advanced" features "better to leave for framework authors". The proposal's issue #178 is an open argument between eager tree-ordered evaluation and lazy dirty-marking, which is scheduling left to each implementation. Leptos's description of the Solid-family push-pull scheme (https://book-ru.leptos.dev/appendix_reactive_graph.html): "A naive, push-based reactive implementation would cause this effect to run twice, which would be bad."; node states "`Clean`", "`Check`", "`Dirty`"; "It pushes the `Check` status all the way down the graph, but then"pulls" its way back up."

**From memory.** Svelte 5 runes, Vue's `ref`/`computed`/`watchEffect`, Angular's `signal`/`computed`/`effect` and Preact Signals, as names only; their schedulers are not described anywhere in this book.

## Rx and streams

**Contribution.** Events as values with good combinators, which is half of what is needed, and the clearest example of what a pushed level looks like: a `BehaviorSubject` fires its current value to every new subscriber, which is the form Prism's creation rule forbids. The stream-table duality (Kafka, above) is what happens when there are streams and no behaviors.

**Verified.** RxJS manual (https://reactivex.io/rxjs/manual/overview.html): "Observables are lazy Push collections of multiple values"; "each subscribed Observer owns an independent execution of the Observable"; "A Subject is like an Observable, but can multicast to many Observers"; "Observables are able to deliver values either synchronously or asynchronously."; a `BehaviorSubject` "has a notion of"the current value"", and a new observer"immediately receives the current value" (https://www.learnrxjs.io/learn-rxjs/subjects/behaviorsubject).

**From memory.** The rxjs.dev subject guide's example that a stream of birthdays is a Subject while a person's age is a BehaviorSubject (https://rxjs.dev/guide/subject). Akka Streams, as a name.

## OCaml's extensible variants

**Contribution.** The element typing: `type Element = choice ..` is OCaml's `type t = ..`, with constructors declared across modules, each with concrete field types, and a `match` that must end in a catch-all because the set of alternatives is open. Prism adds the tag's three parts (`requests`, `responds`, `state`) and the namespacing.

**From memory.** The OCaml manual's chapter on extensible variant types (https://ocaml.org/manual/extensiblevariant

the OCaml sources named under the compiler entry are about values and bytecode, not variants.

## Plotkin–Pretnar, OCaml 5 effects, Koka

**Contribution.** Considered for interceptors and not used. An effect handler is a dynamic-scope construct: a `perform` finds the innermost handler on the stack, which gets a delimited continuation and may resume it. An interceptor is a widget in the tree: its scope is lexical in the widget tree, its "handling" is supplying a dynamic or a response by path, and it never resumes anything, because nothing in Prism suspends. The design borrowed the vocabulary of handling and forwarding (a catch-all arm that re-emits is OCaml's `| _ -> None`, read as forwarding the unhandled effects to the outer handler) and rejected the mechanism, because a handler on a stack can be reached from anywhere below it and the sandbox invariant needs the opposite: nothing reaches past an interceptor unless Prism code sends it. The one idea that survives from Koka is that a handler which answers at once needs no captured continuation; an interceptor's state supply is that case, and its request answering goes through `Defer` rather than through a continuation.

**From memory.** Plotkin and Pretnar, "Handling Algebraic Effects", LMCS 9(4:23), 2013 (https://arxiv.org/abs/1312.1399), the journal version of the ESOP 2009 paper, whose handlers generalise exception handlers with timeouts, rollback and stream redirection among the examples. The OCaml 5 manual's effects chapter (https://ocaml.org/manual/5.2/effects.html): the handler receives the delimited continuation representing the suspended computation between `perform` and the handler; `| _ -> None` reads as forwarding to the outer handler; an unhandled effect raises at the `perform`; every captured continuation must be resumed exactly once. Koka's evidence passing, under which a tail-resumptive operation is called directly as a function without capturing the stack; Koka's README describing exceptions, async/await and probabilistic programs as user libraries. Bauer and Pretnar's Eff.

## GHC's core-lint and GHCi

**Contribution.** The lint pass of the Rust compiler: a typed intermediate form that is re-checked after every pass, so that a conversion that breaks a type is caught at the pass that broke it rather than at runtime. The typed AST is the form everything else is checked against, as Core is in GHC. GHCi is the model for an interpreter that runs beside compiled code against one specification.

**From memory.** The GHC user's guide (https://downloads.haskell.org/ghc/latest/docs/users_guide/debugging.html): `-dcore-lint` as intra-pass sanity checking at the Core level, `-ddump-simpl` for simplifier output, `-ddump-bcos` for the byte-code objects. GHCi's documentation: a loaded module is converted to byte-code and interpreted; interpreted code runs alongside compiled code; compiled code is roughly an order of magnitude faster. The recursive-do extension's `mfix` and its description of a `rec` block as telling the compiler where the recursive knot is tied. The STG notes describing STG as a variant of administrative normal form, every `let` as a closure allocation, and the main work of Core-to-STG as computing free variables to construct closures, which is the closure conversion the Prism compiler also has to do.

## Lua 5.0, the WebAssembly rationale, OCaml's bytegen, Cranelift

**Contribution.** Studied for the compiler and set aside. A register VM is a dispatch-cost design, and Prism's per-occurrence cost is at the runtime boundary rather than in dispatch; structured control flow is kept for `match` because it makes verification one pass; OCaml's two backends sharing one Lambda tree is the model for two Prism implementations sharing one typed-AST text form;

Cranelift's SSA with block parameters is where a native backend would start if one is ever built. Two things survived into the plan: the closure compiler over the typed AST, and the principle that running Prism code produces a graph.

**From memory.** Lua 5.0's paper (https://www.lua.org/doc/jucs05.pdf) describing its virtual machine as the first widely used register-based one, register code avoiding the push and pop instructions of stack code, and the new machine as considerably faster than the old. The WebAssembly design rationale (https://github.com/WebAssembly/design/blob/main/Rationale.md): started from an AST and generalised to a structured stack machine; restriction to structured control flow simplifies one-pass verification by avoiding a fixpoint computation. OCaml's `bytecomp/bytegen.mli` (generation of bytecode from lambda terms) and `middle_end/closure/closure.mli` (introduction of closures, uncurrying, recognition of direct calls); Real World OCaml on bytecode as a close mapping to the lambda form, whose advantage is simplicity, portability and compilation speed. Cranelift as a low-level retargetable code generator with several backends, and `cranelift-frontend` as the API that turns mutable variables into SSA (https://github.com/bytecodealliance/wasmtime/tree/main/cranelift). Crafting Interpreters on walking the AST as the wrong execution model, for locality and speed (https://craftinginterpreters.com/chunks-of-bytecode.html). Feeley and Lapalme, "Using Closures for Code Generation", Computer Languages 12(1), 1987, as the technique of compiling each node to a host closure. Rustlantis (https://github.com/cbeuw/rustlantis) treating any discrepancy between backends as a bug in one of them, the model for the differential test. LLVM's developer policy stating that the textual IR is not backwards compatible, one reason LLVM is not a target.

## Squeak

**Contribution.** An implementation written in a subset of the language it implements, debugged inside the language's own environment, as the model for keeping two Prism implementations honest against one specification: the Haskell implementation exists to read Part II a second time, and the differential test is the Interpreter Simulator's job done between two readers.

**From memory.** Ingalls et al., "Back to the Future: The Story of Squeak", OOPSLA 1997 (https://ftp.gwdg.de/pub/languages/smalltalk/st.cs.uiuc.edu/Smalltalk/Squeak/docs/OOPSLA.Squeak.html): the VM written entirely in Smalltalk, in a subset that maps directly onto C; a subclass, the Interpreter Simulator, running the same code inside a Smalltalk environment and serving as the basis for debugging the system into existence.

## Also consulted

Each of these contributed one idea, named here so the reference says why and not only where. All are from memory.

- Jane Street's Incremental (https://github.com/janestreet/incremental): a variable set during stabilization takes effect at the next one, and a node created during a stabilization has its initial value for that stabilization, which is the mid-occurrence construction rule in another system.
- Differential dataflow (https://github.com/TimelyDataflow/differential-dataflow): `advance_to` as a promise to stop changing an input at all times before a given one, the version as the seam.
- Verilog elaboration, Nix instantiation and Bazel's analysis phase: three systems in which evaluating the program produces a graph that something else then runs, the principle behind "building is the only doing".
- winit (https://docs.rs/winit): the event loop must be created on the main thread and only once per application, a constraint every GUI World inherits and the reason the turn loop lives on the platform's main thread.

- AccessKit (https://github.com/AccessKit/accesskit): what an accessibility tree wants from a draw list, which is why `Gui.Scene` is data a World can read rather than a sequence of calls.
- wasi-http (https://github.com/WebAssembly/wasi-http): a request/response boundary with no callbacks, the model for a World whose whole life is one turn.
- Elm's `Cmd`/`Sub`, Cycle.js drivers, Haxl's batching, and the Rust and C# `async` state machines, as the other designs in which a program describes effects and something outside it performs them. ## C. Syntax quick reference

## Sigils

| Sigil | On a value | On a function name | On a type |
|---|---|---|---|
| `!` | an event | returns an event | `!A`, event of `A` |
| `$` | a dynamic | returns a dynamic | `$A`, dynamic of `A` |
| `@` | a blueprint | returns a blueprint | `@A`, blueprint returning `A` |
| `?` | an optional | returns an optional | `?A`, `Nil` or `Some a` |
| none | a plain value | returns a plain value | a plain type |

Sigils nest leftmost-outermost: `?$user`, `!!rendered`, `@!unit`, `$@a`, `$(Key ==> $Item)`. A sigil is attached (part of the token) when followed by a name character, `_`, `"` or another sigil; otherwise it is a prefix operator over the whole expression to the end of the call: `$ x` is the constant dynamic, `@ v` the blueprint returning `v`, `@ ((!a, $b))` a bundle. `$_`, `!_`, `@_` are the pronoun at that kind. `&` is reserved for references (chapter 3); `%` does not exist. Names: `snake_case` for values, `UpperCamelCase` for functions and types; a camelCase value name such as `isClick` is a lint warning (write `is_click`), not a compile error. A dotted name such as `Gui.Button` lexes as name, `.`, name, and `Gui.*` in a pattern as name, `.`, `*`.

## Lexical

Comments `#` to end of line. UTF-8 source. Layout is the offside rule with `:` as the block opener: a `:` ending a line opens a block on the next, more-indented line; a `:` with tokens after it on the same line opens an inline block of that line, and `else` terminates an inline block, so `if c: a else: b` is one line with two inline blocks; inside `( ) (( )) [ ] [[ ]] { }` layout is suspended and `:` is the map key separator. Inline `:` is a plain token the parser interprets, not a layout token. `;` separates statements in an inline block: `type Pair<a, b> = record: first: a; second: b`, `if c: Log x; Log y else: Log z`; a layout block never needs it. `((` and `))` open and close a tuple, `[[` and `]]` an array literal; single `( )` is grouping and single `[ ]` is indexing or an array type. A trailing lambda as the last argument absorbs the rest of the line, so `MapValues !xs, \|x\| x + 1` needs no parentheses around the lambda. Keywords: `let type tag of fn match if else for rebuild intercept widget record choice map array unit test ignore on then not and or key ix jx kx empty`; keywords are reserved everywhere (none is contextual); `read` and `update` are reserved for chapter 3's `update` blocks. `never started main Switch Leftmost Join Filter Updated Distinct Element Kind` are prelude names written with their sigils where they have one: `!never`, `@started`, `@main`, `!Switch: fn !!A -> !A`, `!Leftmost: fn [!A] -> !A`, `$Join: fn $$A -> $A`, `!Filter: fn !?A -> !A`, `!Updated: fn $A -> !A` (fires with the new value in every occurrence the dynamic changes; never at build), `$Distinct: fn $A -> $A` (needs equality; suppresses equal-valued changes, which nothing else does), `@HoldFirst: fn !A -> @$A` (a dynamic that holds its event's value, with the first firing's value, in the build occurrence, as its initial value), `@Element`, `Kind: fn Element -> Kind`, and the optional default

`Or: fn ?A, A -> A` (`Or ?x, default`; a function, unrelated to the keyword operator `or`). There are no prelude `Not`, `And` or boolean `Or` functions: `not`, `and`, `or` are operators. `Wrap.*` and `Checked.*` are the prelude's wrapping and checked arithmetic (`Wrap.Add: fn Nat, Nat -> Nat`, `Checked.Add: fn Nat, Nat -> ?Nat`, and likewise `Sub`, `Mul` and the `Int` forms); the bare operators abort the occurrence with an `Overflow` error.

Literals: an integer-spelled literal is `Nat` or `Int` by inference, and `Int` when nothing constrains it (`$n <- 0 then $n + 1 on !tick` is an `Int` counter); `-1` is an `Int`; a `Nat` is pinned by annotation; a decimal-spelled literal is `Float`. `"text"` with the escapes `\\ \" \n \t \{ \} \u{hex}` and interpolation `{expr}` only (`"hi {name}"`, `"count: {$count}"`); a `$` in text is always a literal `$`, so `"$count"` is plain text, and a literal brace is `\{` or `\}`. Interpolating a dynamic or an event lifts the whole literal (`"Count: {$count}"` is a `$Text`; `"got {!line}"` is a `!Text`). `'c'`, `unit`; `[[1, 2, 3]]` for an array, `[[]]` or `empty` for the empty one; `{k: v}` for a map, `{}` for the empty one; `((a, b))` for a tuple. The spacing rule: a token that can be prefix or binary is prefix when preceded by whitespace and followed immediately by a non-space (`F -1` passes negative one; `F - 1` subtracts). Wherever `_` is in scope (a `\` body, a `for` body, any pronoun-binding construct) a prefix `.x` is `_.x` and a prefix `[i]` is `_[i]`; with no `_` in scope it is an unbound-pronoun error, and a `.Method` that starts a continuation line is a chain step.

## Statements in a widget block

| Form | Meaning |
|---|---|
| `pattern <- blueprint` | build the widget as a child of this widget; bind its result |
| `blueprint` | build it; discard the result |
| `let pattern = expr` | bind a value of any kind; builds nothing |
| `$x <- v then !e` | hold: `$x` is `v` until `!e` fires, then `!e`'s value; owned by this widget; a hold set to the value it already has still changes |
| `!!r <- rebuild $x: block` | rebuild the block on each change of `$x`; `_` is the plain value; event fires in every build occurrence |
| `!!r <- rebuild p <- $x: block` | the same, destructuring `$x`'s value with pattern `p` |
| `r <- for k, $v <- $m: block` | one block per key of `$(K ==> V)`; `k` is the key and `$v` the value as a dynamic; consolidated by result kind, promptly |
| `r <- for $m: block` | the shorthand: `key` is the key and `$_` the value as a dynamic; `for $v <- $m:` names only the value |
| `((r, $slots)) <- intercept @child, !responses, $supplier` | the interceptor primitive (below) |
| last statement | the block's result; `@ v` returns a plain value |

Bindings in a block are mutually recursive; a cycle must pass through a delayed edge (a tag, `!Switch`'s hold, a hold's value side). A block body is `@name:` at top level, `let @w = @widget:` inline, or `\@widget:` as a lambda body. A plain `for k, x <- xs: body` (or `for xs: body`) outside a widget block maps over a plain array or map and is an expression; over an array the key is the index.

A top-level definition is `name: type = expr`, with the annotation required and the name's sigil saying what kind of thing it is: `@Button: fn $Text -> @!Click = \|$label\| @widget: ...`, `IsOne: fn Int -> ?unit = \|n\| ...`, `@main: @unit = @widget: ...`. The shorthand `@name:` opens the widget block

of a definition of type `@unit` that takes nothing. `let` is the binding form inside a block and never appears at top level.

## Operators, loosest to tightest

| Level | Operators | Notes |
|---|---|---|
| 1 | `<-`, `let ... =` | statement level |
| 2 | `then` | hold |
| 3 | paren-less application | extends to the end of the expression: `F a + 1` is `F(a + 1)`, and `(F a) + 1` is the other grouping; commas belong to the outermost call on the line or in the enclosing single-paren group; a trailing lambda absorbs the rest of the line |
| 4 | `\|` | leftmost merge of events; associative; `!never` is its identity |
| 5 | `on` | `v on !e` fires with `!e`, value `v` (dynamic `v` read before the occurrence) |
| 6 | `or` | boolean or; associates left; short-circuits when plain |
| 7 | `and` | boolean and; binds tighter than `or`: `x or y and z` is `x or (y and z)`; short-circuits when plain |
| 8 | prefix `not` | boolean negation; looser than comparison, so `not a == b` is `not (a == b)` and `not a < b and c` is `(not (a < b)) and c` |
| 9 | `< <= > >=`, `== !=` | comparison; `a < b` through the ordering trait, `a == b` through the equality trait; `Bool` result; does not associate |
| 10 | `+ -` and `+~ -~` (Int), `+. -.` (Float) | suffixed spellings only when inference cannot decide between `Int` and `Float`; there are no suffixed `Nat` spellings, a `Nat` is pinned by annotation |
| 11 | `* /` and suffixed | `/` on `Nat` and `Int` returns `?`; on `Float` it follows IEEE (infinities, NaN) and never fails |
| 12 | prefix `+ -`, free sigils `! $ @ ?`, `+?` | prefix `+` is numeric identity; `+?: fn Int -> ?Nat` converts, `Nil` below zero |

| Level | Operators | Notes |
|---|---|---|
| 13 | `.field`, `[index]`, attached sigils, `.F args` method form | `x.F y` is `F x, y`; chains associate left, so `x.F.G` is `G (F x)`; a `.F` that starts a continuation line closes the call above it and takes it as receiver; `a[i]` and `m[k]` return `?` |

`not`, `and` and `or` are keywords over `Bool`, lifted by the sigils like any operator: `$done or $count > limit on !click` is `($done or ($count > limit)) on !click`, and `a < b and c < d` needs no parentheses. Plain `and` and `or` short-circuit, so an overflow in the operand that was not needed aborts nothing; lifted over dynamics or events they are lifts of the same function and fire by the lifting rule.

Parentheses: a single-paren group whose head is a call is that call with all its commas, so `(F a, b)` is `F(a, b)` and `G x, (F a, b)` gives `F` two arguments. Tuples are double parens: `((a, b))` is a pair, `((F a, b))` the pair of `F a` and `b`, and a multi-argument call inside a tuple takes its own single parens, `(((F a, b), c))`. Array literals are double brackets, `[[1, 2, 3]]`; single `[i]` is always indexing. A chain is written one step per line, and indentation decides where each call ends:

```
items.Filter IsDone
    .MapValues Label          # MapValues (Filter items, IsDone), Label
```

On one line, `x.F y` is method form and a spaced `.F` is not a chain step. Comparisons are `<`, `<=`, `>`, `>=`, resolved through the ordering trait, and `==`, `!=`, resolved through the equality trait. Arithmetic, equality and ordering are the three built-in traits; nothing else is overloaded. Records have equality when every field does, and ordering, lexicographic in field declaration order, when every field is ordered; tuples, arrays, primitives and text are ordered; `Float` has a total order (IEEE totalOrder) and can key a map; the alternatives of a closed `choice` are ordered by declaration order; element equality is the tag plus the plain fields, never the request event, and elements are ordered by tag name then plain fields, so they can key sorted maps.

## Lifting

Read off the operands' sigils at every operation and application. All plain: plain. Some `$`, no `!`: a dynamic, prompt. Some `$` and one `!`: an event firing with the `!`, the dynamics read before the occurrence (tag law). Two `!` in one operation: an error naming both. Field access and indexing lift like application: `!!panes.!inbox` and `$users[id]` are fine. `@` operands are not lifted; `@` as a prefix lifts a plain value into a blueprint. `?` lifts at the plain layer like an applicative `Maybe` (`a / b + 1` is `?Nat` for `Nat` operands; `Float` division has no `?`), and `match` on a `?` is its eliminator. Lifting never crosses a sigil boundary; `!Switch` (non-prompt) and `$Join` (prompt) do. `match $x:` and `match !e:` lift like application when all arms are plain; when any arm is a blueprint the scrutinee must be plain. A derived dynamic recomputed to an equal value still changes; `$Distinct` is the one place equal values are suppressed.

Binding a widget application lifts by the result: `!x <- @Button $label` is `rebuild $label:` around the build with `!x` as the `!Switch` of the result; a state-returning element under a `rebuild` binds as a hold of dynamics joined with `$Join`, with the first build's result as the hold's initial value; a widget application whose result is plain binds as a `$` holding the first build's value. An unannotated `let` lambda is a plain function lifted afresh at each application; a lambda mentioning `$_`, `!_` or `@_` takes

that kind and is not lifted over it.

## The prelude's plain functions

The chapters use these without defining them, and chapter 8 assumes them. Each is a plain function at plain types and lifts by the lifting rule, so `Len $xs` is a `$Nat` and `Insert $m, !k, v` fires. Maps: `Insert m, k, v`, `Remove m, k`, `Get m, k` (`?V`), `Member m, k`, `Keys`, `Values`, `Size`, `MapValues m, f`, `Fold m, z, f` (`f` takes the accumulator, the key and the value), `Union a, b` (left wins). Arrays: `Len`, `Append xs, x`, `Prepend x, xs`, `Concat a, b`, `First`, `Last` (`?T`), `Take n`, `Drop n`, `DropLast`, `Reverse`, `Range a, b`, `Fold xs, z, f`, `MapValues xs, f`, `Filter xs, p`, `Count xs, p`. Text: `Len`, `Concat` (or `+`), `Split t, sep`, `JoinText xs, sep`, `Prefix t, p`, `Suffix t, s`, `Upper`, `Lower`, `ToText v` (the transcript spelling of any plain value). Numbers: `Abs`, `Min`, `Max`, `Mod`, `Sqrt` on `Float`, and the `Checked.*` and `Wrap.*` families from the lexical section. Tuples: `First`, `Second`, `Third`. Optionals: `Or ?x, d`, `Some`. Events and dynamics, with their sigils: `!Filter: fn !?A -> !A`, `!Union: fn !(K ==> V), !(K ==> V) -> !(K ==> V)` (both firing in one occurrence merge, left wins per key), `!Leftmost`, `!Switch`, `$Join`, `!Updated`, `$Distinct`, `@HoldFirst`, `!never`, `@started`. The list is the one the Haskell prototype needed to run every program in chapters 2, 4 and 10, so it is complete for the book and not for a library; an implementation that adds to it adds to this list.

## Lambdas and patterns

`\body` takes `_` (or the sigiled pronoun its body uses); `\|a, b| body` names arguments; `\|((x, y)), k| body` destructures a pair argument; `\@widget:` opens a widget block. `ignore` as an argument discards it. Wherever `_` is in scope (a `\` body, a `for` body, any pronoun-binding construct) a prefix `.x` is `_.x` and a prefix `[i]` is `_[i]`, so `\.done` and `\[0]` are lambdas.

Patterns: a literal; a name (binds); `ignore`; `((p, q))` for a tuple; `Some p`, `Nil`; `Type p` for a newtype or payload alternative, `Alt p, q` for an alternative with several payloads, comma-separated as in its construction; `Type(f1, f2=p, ..)` for a record, positional or by name, `..` for the rest; `Ns.Tag(field=p, ..)` for an element by tag; `Ns.*(..)` for any tag in a namespace. `[[]]` matches the empty array, `[[a, b]]` exactly two elements, and `[[head, ..rest]]` a non-empty one (`..` takes the rest and comes last). `match` arms are tried in order; an arm may have a guard, `pattern if cond: body`, and a guarded arm does not count toward exhaustiveness (an extensible variant needs an unguarded catch-all). A closed choice must be exhaustive, arrays included. `if c: a else: b` is `match` on `Bool`; `if` always has an `else`, and there is no `else`-less `if`.

## Types

```
type Vector = record:            # record; constructor is the type name
    x: Float
    y: Float

type Figure = choice:            # choice; a payload alternative is a newtype
    Circle Float
    Square Float
    Line Point, Point            # several payloads: the alternative holds the tuple ((Point, Point))

type Bool = choice True | False  # one-line choice
type Meters = Meters Float       # newtype, unwrapped with .value
type Click = click               # unit type, one value, lowercase
type Pair<A> = record: ...       # generics, erased; no type variables in tag fields
fn A, B -> C                     # function type; fn A -> C for one argument; fn unit -> C for none
```

```
fn ((A, B)) -> C                # one argument, a pair
fn (fn A -> B), [A] -> [B]       # a function-typed argument or result is parenthesised
fn A -> (fn B -> C)
[T]   K ==> V   ((A, B))   ?T    # array, sorted map, tuple, optional
```

Primitives: `Nat Int Float Text Char Bytes unit`; `Int` is 64-bit signed and `Nat` 64-bit unsigned. A record with sigiled fields is a bundle, read with `.!changed`:

```
type TextInput = record:
    !changed: Text
    $value: Text
```

Fields of a record or tag go one per line in a layout block, or separated by `;` in an inline block. The prelude's `Kind` is a record, `type Kind = record: requests: Bool; responds: Bool; state: Bool`, and `Kind e` says which parts the element `e`'s tag has.

## Tag declarations

```
type Element = choice ..                # the extensible variant, once

tag Ns.Name of Element:                   # a tag, in any module
    field: PlainType                    # fields: plain data, concrete types, fixed for the slot's life
    requests: !R                        # optional: an event the program supplies; firings go out
    responds: R                         # optional: the element returns !R; firings come in later
    state: S                            # optional: the element returns $S; a value from creation
```

Construction spells the request event as a named field: `Cli.Stdout(requests=!line)`, `Fs.File(path=p, requests=(Read on !started))`. The request event is excluded from equality, so two slots differing only in their request events are equal elements and share state. `@Element e` returns `@unit` (fields only, or requests only), `@!R` (responds), `@$S` (state, with or without requests), `@((!R, $S))` (responds and state); in a catch-all arm the result is the generic `@((!Response, $State))`, checked per slot at runtime, and inside a tag-specific arm `@Element _.element` has that tag's types. A dynamic where a field goes lifts to a `rebuild` with a fresh slot per change. An event where a field goes is an error unless the field is `requests`. Responses are addressed by path; state is shared by element value. Elements never fire in the occurrence they were created; a state element has a value in it. There is no `default:` line in a tag: a World that cannot answer the oracle for a state tag makes the slot's creation an `UnsuppliedState` error, and a test gives the value with a `with` step before the build.

### `intercept`

```
intercept: fn @a, !(Path ==> Response), $(fn Element -> @$State) -> @((a, $(Path ==> Slot)))

type Slot = record:
    element: Element
    !request: Request        # typed by the tag inside a match arm on element
```

Builds `@a` as a child and captures its element list. Down: `!responses` by path, routed against the list as the occurrence opened; the supplier, read at each state slot's creation, applied to the element, its widget built in the interceptor's widget once per distinct element value, its result the slot's dynamic. A new child slot whose element value already has a live supplied widget joins it, whatever the supplier dynamic holds now. Up: whatever the interceptor makes with `@Element`. Forwarding is `\|e| @Element e`. Answering a request yourself goes through `Defer`. Pure code firing at a path created in the same

occurrence is an `UnknownPath` error.

## Test scripts

A test is a top-level form the suite's harness runs under the Test World, which accepts every tag. One line per turn; inputs on the line, checks on indented `=>` lines under it; no side is ever empty.

```
test name, @widget:
    build
        => expect pattern    # some slot at any depth matches
        => expect_top pattern    # some root-level slot matches
        => expect_none pattern    # no slot matches
        => expect_top_none pattern    # no root-level slot matches
        => expect_count pattern, n    # exactly n match
    click pattern    # exactly one slot matches; deliver a click; run a turn
        => settles    # the turn ran, its Defers were answered, no error
    send pattern, response    # any other response
        => settles
    click pattern, send pattern, response    # several deliveries in one occurrence
        => settles
    with pattern = value, click pattern    # set a snapshot entry for this turn, then deliver
        => settles
    push pattern = value    # a turn whose only input is the snapshot change
        => settles
    idle    # a turn with no inputs
        => returns pattern    # the widget's result matches (a dynamic by current value); on an event result, a
script error (use `fired`/`quiet`)
        => fired pattern    # the result event fired in the last turn with a matching value
        => quiet    # the result event did not fire in the last turn
        => requested pattern, value    # a matching slot fired its request with a matching value
        => unrequested pattern    # no matching slot fired its request in the last turn
        => errors pattern    # the last turn ended in a matching runtime error
```

The build is turn zero. Each input line is one turn including its `Defer` occurrences; independent `Defer`s fired in one occurrence are answered together. `with` may appear before the build and must for every state tag the program creates at build, since the oracle has to answer in occurrence zero. Patterns are element patterns; `errors` patterns are `Error(kind=Cycle, ..)` and the like.

Transcripts and `send` targets spell paths canonically: a slot or child by its index in the parent's child list, a `rebuild` generation as `#n`, a `for` key as `[k]`, and a supplier-built slot as `s/n/...` with the supplied widget's own segments after the prefix. Only slots, `rebuild` generations, `for` keys and intercept children take segments; a plain widget-function application makes a widget but adds no segment, so a `@Button` whose body holds the `rebuild` prints as `0/#0/0`, and the third row of a keyed list's second generation prints as `1/#1/[3]/0`. ## D. The element vocabulary

Every tag the design uses, in one place, so that a chapter, a test, a World and a library agree on the names. The language fixes the kinds (which parts a tag has) and the laws (creation rule, routing by path, identity by value); the names and fields are the vocabulary the book's examples and Worlds speak. A World accepts a subset of these natively and says which through `World.Tags`; everything else under it is Prism, as interceptors over the tags it does accept, or an `Unsupported` response.

Four things apply to every row.

*Fields* are plain data fixed for the slot's life; a dynamic in a field position is a `rebuild` with a fresh slot per change.

*Identity* is by path for responses and by element value for state, and equality of elements is the tag and

the plain fields, never the request event. Two slots with equal elements observe one state, supplied once. A tag with both parts has one slot and two identities: `Net.Socket(address="db:5432")` in two places is two conversations (each slot's sends and receives are addressed to its own path) over one connection (both slots read one `Link` state, because their elements are equal). The rule is the same for every row that has both a `responds` and a `state` column, and the Identity column says "path; value" for it.

*Creation*: nothing fires in the occurrence a slot is created. The runtime enforces half of this, since it has no response to deliver in a slot's creation occurrence and a firing from pure code at a path created in the same occurrence is an `UnknownPath` error; the World is bound to the other half by its obligation never to answer a request in the occurrence it was made. A state element has its value in its creation occurrence, from the snapshot as the occurrence opened; the World is bound to answer the oracle synchronously and consistently, and the runtime raises `UnsuppliedState` on the oracle's refusal.

*Teardown*: a response addressed to a slot that no longer exists is discarded, by the World under its obligation to send only to known paths, and a request fired by a slot torn down in the same occurrence is reported to the World with its path, performed if the World can, and its response discarded. Anything a World started on a slot's behalf and that cannot outlive the slot (a child process, a watch, a listener) is stopped under the World's cancel-on-teardown obligation. No row below is a trigger by presence or absence: appearing is a description, disappearing is the end of delivery, and anything a program wants done is a request it fires.

File paths are `Text` throughout, because the name `Path` is the type of positions in the element tree.

## Value types

The value types the tags refer to are declared once here and reused. A choice alternative with several payloads holds the tuple of them.

```
type Click = click
type Tick = tick
type Code = Code Int
type Instant = Instant Int                # milliseconds since the World's epoch
type Duration = Duration Int               # milliseconds

type Point = record:
    x: Float
    y: Float

type Vector = record:
    x: Float
    y: Float

type Extent = record:                      # a width and height, the only size type: a window's size, a text's
measure (what Gui.Metrics answers)
    w: Float
    h: Float

type Rect = record:
    x: Float
    y: Float
    w: Float
    h: Float

type Color = record:
```

```
    r: Float
    g: Float
    b: Float
    a: Float

type Font = record:                        # named in the value that draws it; the World resolves it
    family: Text
    size: Float

type ImageId = ImageId Nat                 # an opaque handle the World allocates

type Button = choice Primary | Secondary | Middle

type Pointer = choice:
    Moved Point
    Down Point, Button
    Up Point, Button
    Scroll Point, Vector

type NamedKey = choice Left | Right | Up | Down | Backspace | Delete | Tab | Enter | Escape

type CompositionEvent = choice:            # an input method's composition
    Start
    Update Text, Nat
    Commit Text
    End

type Key = choice:
    Char Char
    Named NamedKey
    Composition CompositionEvent

type WindowEvent = choice:                 # what a Gui.Window responds with
    Resized Extent
    CloseRequested
    Pointer Pointer
    Pressed Key

type Draw.Rect = record:                   # draw commands are values, not tags
    rect: Rect
    color: Color

type Draw.Text = record:
    origin: Point
    text: Text
    font: Font
    color: Color
    ime_at: ?Nat                           # offset into text where an input method's candidate window goes

type Draw.Image = record:
    rect: Rect
    image: ImageId

type Draw.Clip = record:                   # children drawn clipped to the rectangle
    rect: Rect
    draw: [Draw]

type Draw = Draw.Rect | Draw.Text | Draw.Image | Draw.Clip
```

```
type Frame = record:                        # one displayed frame
    now: Instant

type ChildInput = record:                   # what a Ui container responds with
    path: Path
    response: Response

type FsError = choice NotFound | Denied | TooLarge | Other Text

type Contents = choice:
    Unavailable FsError                     # the reason
    Known Text

type FsRequest = choice:
    Read
    Write Text
    Append Text
    Delete

type FsResponse = choice:
    Contents Text
    Written
    Unavailable FsError

type Listing = choice:
    Unavailable
    Known [Text]

type Stamp = choice:                        # a file's modification stamp as state
    NoStamp
    Modified Instant

type Link = choice:
    Connecting
    Connected
    Disconnected Text

type SocketEvent = choice:
    Received Text
    Closed

type HttpRequest = record:
    method: Text
    url: Text
    headers: Text ==> Text
    body: Text

type HttpResponse = choice:
    Answer Nat, Text ==> Text, Text         # status, headers, body
    Failed Text

type Accepted = Accepted Nat                # a connection id
type ConnEvent = choice:
    Received Text
    Closed
type ConnRequest = choice:
    Send Text
    Close
```

```
type SpawnRequest = record:
    program: Text
    args: [Text]
    env: Text ==> Text
    cwd: ?Text

type SpawnResponse = choice:
    Spawned Nat                          # the child's id, the World's to allocate
    NotFound
    Denied

type ProcessRequest = choice:
    Input Text
    CloseInput
    Signal Signal

type ProcessEvent = choice:
    Output Text
    Error Text
    Exited Code

type ProcessState = choice:
    Running
    Exited Code
    Gone                                 # no such child; the id is stale

type StdinEvent = choice:
    Chunk Text
    End

type WriteFailed = choice:               # what Cli.Stdout and Cli.Stderr respond with
    Closed
    Failed Text

type Signal = choice Interrupt | Terminate | Hangup | WindowChanged

type Stored = choice:                    # a Store.Value's state
    Absent
    Present Bytes

type PutResult = choice:
    Stored
    Failed Text

type ErrorKind = choice Cycle | Overflow | UnknownPath | UnsuppliedState | DeferBackwards | Component |
Build

type Span = record:
    file: Text
    line: Nat
    column: Nat
    name: Text                           # the binding's name

type Unsupported = unsupported
```

`Bytes` is the byte-string primitive. `Response` and `with` are the generic response and state types an interceptor sees in a catch-all arm; `Path` is the position type. `ErrorKind` is a closed choice on purpose: a program that matches on `Error`'s `kind` is exhaustive, and a new kind of failure is a change to

the language, not to a World. `Error` itself is a tag (the last table below), so one value serves as the presence element a failed build leaves and as the response `errors` delivers; `RuntimeError { kind, spans, occurrence, .. }` is the same record at the Rust boundary, where the runtime's `Outcome` lists the contained build failures.

## Screen: `Gui.*` and `Ui.*`

GUI is not a World concept. `Gui.*` is the primitive vocabulary every GUI World speaks: five tags, and a World that knows about screens accepts these and nothing more. `Ui.*` is the Prism GUI library's own vocabulary, handled by the library's interceptors; the rows are listed because the examples use them, and they reach a World only if that World accepts them (the Test World accepts every tag; a World wrapping a native toolkit may accept some). Under a World that accepts only the primitives, `Ui.Button` is an interceptor's business. The World does no geometry: layout, hit-testing, focus traversal and text editing are Prism over the five primitives. Draw commands (`Draw.Rect`, `Draw.Text`, `Draw.Image`, `Draw.Clip`) are plain values in a `Gui.Scene`'s `draw` list, never tags.

| Tag | Fields | Requests | Responds | State | Identity | Meaning |
|---|---|---|---|---|---|---|
| `Gui.Window` | `title:` `Text`, `size:` `Extent` | | `WindowEvent` | | path | *Primitive.* One top-level window per slot; `size` is what the program asks for when it opens. Every pointer event over the window arrives here, and every key (`Pressed`), as does the size the platform gave (`Resized`) and `CloseRequested`, which the program answers by tearing the window down, or not. The library hit-tests against its own layout, routes pointer hits inward, and routes each key to the input whose name is the focus. |

| Tag | Fields | Requests | Responds | State | Identity | Meaning |
|---|---|---|---|---|---|---|
| `Gui.Scene` | `window: Text`, `draw: [Draw]` | | | | path | *Primitive. Presence.* The draw list for the window of that title, as data in paint order, read by the World from the committed list after each turn. A picture that changes is a fresh slot per change, at the same tree position; nothing is in flight on a picture, so that is right. |

| Tag | Fields | Requests | Responds | State | Identity | Meaning |
|---|---|---|---|---|---|---|
| `Gui.Focusable` | `name: Text` | | | | path | *Primitive. Presence.* The name of the focused region and nothing else; keys do not go here but to `Gui.Window`. It lets a World show the platform focus ring, place an input method candidate window and inform a screen reader. Which name is focused, what Tab does and where the caret is are holds in the library, which emits at most one `Gui.Focusable` at a time. No rectangle: the World does no geometry. |

| Tag | Fields | Requests | Responds | State | Identity | Meaning |
|---|---|---|---|---|---|---|
| `Gui.Metrics` | `text: Text`, `font: Font` | | | `Extent` | value | *Primitive.* The width and height of a text in a font, answered synchronously from the oracle, so a layout has its measurement in the occurrence that asks. Every slot asking about the same text and font shares one answer. |
| `Gui.Frames` | | | `Frame` | | path | *Primitive.* One `Frame(now: Instant)` per displayed frame while the slot exists, for animation. Put the slot under a `rebuild` over whether anything is animating, so that when nothing is, the World stops asking for frames. |

| Tag | Fields | Requests | Responds | State | Identity | Meaning |
|---|---|---|---|---|---|---|
| `Ui.Label` | `text: Text` | | | | path | *Library.* Text at the position layout gives it. |
| `Ui.Button` | `label: Text` | | `Click` | | path | *Library.* A button; responds `click` in the occurrence the library routes the press to it, which is later than the press's own occurrence. |
| `Ui.TextInput` | `name: Text`, `text: Text`, `cursor: Nat` | | `Key` | | path | *Library.* A focusable editing region; the text and cursor are holds in the Prism widget around it, mirrored into the fields for drawing, and there is no second copy in a World. |

| Tag | Fields | Requests | Responds | State | Identity | Meaning |
|---|---|---|---|---|---|---|
| `Ui.VBox` | `children: Path ==> Element` | | `ChildInput` | | path | *Library.* Vertical layout container; routes any child-addressed response inward by path through `ChildInput(path, response)`, one level per container. |
| `Ui.HBox` | `children: Path ==> Element` | | `ChildInput` | | path | *Library.* Horizontal layout container; routes as `Ui.VBox` does. |
| `Ui.Scroll` | `name: Text`, `height: Int`, `children: Path ==> Element` | | `ChildInput` | | path | *Library.* A clipped, scrollable container, a `Draw.Clip` and an offset the library holds. |

A `Gui.Scene` appearing is a description the World draws; a `Ui.Button` disappearing stops nothing but the delivery of clicks to a path that no longer exists.

## Files: `Fs.*`

| Tag | Fields | Requests | Responds | State | Identity | Meaning |
|---|---|---|---|---|---|---|
| `Fs.File` | `path: Text` | `!FsRequest` | `FsResponse` | | path | One file handle's conversation: each request firing is one operation; each response answers one, in a later occurrence, in order. |

| Tag | Fields | Requests | Responds | State | Identity | Meaning |
|---|---|---|---|---|---|---|
| `Fs.Contents` | `path: Text` | | | `Contents` | value | The file's contents as a value over time. `Unavailable(reason)` with a reason from the closed `FsError`: `NotFound` when it does not exist, `Denied` when it cannot be read, `TooLarge` when it is larger than the World's synchronous size cap, `Other(text)` for any other failure; pushed when the World sees a change. Shared by every slot with the same path. |
| `Fs.Dir` | `path: Text` | | | `Listing` | value | A directory's entries, sorted, as state. |

| Tag | Fields | Requests | Responds | State | Identity | Meaning |
|---|---|---|---|---|---|---|
| `Fs.Watch` | `path: Text` | | | `Stamp` | value | The file's modification stamp as state: a real stat answers the oracle at creation (`NoStamp` or `Modified` with the time), so there is no unknown case; pushed when the World sees a change. A program that wants the contents re-reads through `Fs.Contents` or `Fs.File` when the stamp changes, so the World never needs to say it lost changes. |

`Fs.Contents` and `Fs.Watch` are both levels: one is the file, the other is the fact that it changed, and a program that only wants to know *that* something changed watches the stamp rather than holding the contents in memory.

## Time

| Tag | Fields | Requests | Responds | State | Identity | Meaning |
|---|---|---|---|---|---|---|
| `Clock` | | | | `Instant` | value | The current time as state, pushed at most once per turn. Every clock in the program is one clock. Read it through a tag at an event; a derived dynamic over it changes once per push. |

| Tag | Fields | Requests | Responds | State | Identity | Meaning |
|---|---|---|---|---|---|---|
| `Time.After` | | `!Nat` | `Nat` | | path | A request: each firing is a delay in milliseconds and asks for one wake-up that long from now; the response echoes the number back, so a slot with several delays in flight can tell the wake-ups apart by their delays; a program that needs more than that uses one slot per timer in a `for`. |

| Tag | Fields | Requests | Responds | State | Identity | Meaning |
|---|---|---|---|---|---|---|
| `Time.Every` | `period: Duration` | | `Tick` | | path | A periodic stream: one `tick` per period while the slot exists; nothing at start; the phase restarts if the slot is rebuilt. Missed ticks are skipped: at most one `tick` per turn, never a backlog. |

A timer is not a request-with-response-on-creation: `Time.Every` fires its first tick one period after it exists, never in its creation occurrence, and `Time.After` fires nothing until asked.

### Network: `Net.*`

| Tag | Fields | Requests | Responds | State | Identity | Meaning |
|---|---|---|---|---|---|---|
| `Net.Socket` | `address: Text` | `!Text` | `SocketEvent` | `Link` | path; value | One connection per distinct address, shared as state; each slot's sends and receives are its own conversation on it. `Closed` is a response; `Disconnected` is the state afterward. |

| Tag | Fields | Requests | Responds | State | Identity | Meaning |
|---|---|---|---|---|---|---|
| `Net.Http` | | `!HttpRequest` | `HttpResponse` | | path | Outgoing HTTP: each firing is one request, each response answers one, in order. A widget that wants one in flight per key uses a `for`. |
| `Net.Listener` | `port: Nat` | | `Accepted` | | path | Listen while the slot exists; each accepted connection is a response with an id the World allocated. Stopped on teardown under the cancel obligation. |

| Tag | Fields | Requests | Responds | State | Identity | Meaning |
|---|---|---|---|---|---|---|
| `Net.Conn` | `id: Nat` | `![ConnRequest]` | `ConnEvent` | | path | One accepted connection by id; sends and closes are requests, received data and the peer's close are responses. Created in a `for` over the accepted ids, torn down when the id leaves the map. |

## Processes: `Process.*`

The process tags are what every program has, a GUI program included: its own arguments, environment and exit, and the children it starts. They are not under `Cli` because a sandbox wants to give a subtree different ones, and because exit is a process thing whether or not there is a shell.

| Tag | Fields | Requests | Responds | State | Identity | Meaning |
|---|---|---|---|---|---|---|
| `Process.Spawn` | | `!SpawnRequest` | `SpawnResponse` | | path | Spawning is a request: each firing starts one child; the response is `Spawned id`, `NotFound` or `Denied`, in a later occurrence. |

| Tag | Fields | Requests | Responds | State | Identity | Meaning |
|---|---|---|---|---|---|---|
| `Process.Child` | `id: Nat` | `!ProcessRequest` | `!ProcessEvent` | `ProcessState` | path; value | The child's existence is state keyed by the id the World allocated. Input, closing input and signals are requests; output chunks, error chunks and exit are responses; `Running`, `Exited` or `Gone` is the state. The program keys a `for` on the ids it was given; when no slot names an id, the World kills the child under its cancel-on-teardown obligation. There is no `detach` field. |

| Tag | Fields | Requests | Responds | State | Identity | Meaning |
|---|---|---|---|---|---|---|
| `Process.Exit` | | `!Code` | | | path | Exit is a request: the run ends after the turn in which this fires, after the turn's writes. Two firings in one turn: the first in occurrence-then-list order wins. The element being present does nothing. |
| `Process.Args` | | | | `[Text]` | value | The program's arguments as state; one value for the whole program; an interceptor may supply a subtree different ones. |
| `Process.Env` | | | | `Text ==> Text` | value | The environment as state; pushed if the World can see it change. |

A run-to-completion helper (`@Run: fn SpawnRequest -> @$ProcessState`) is a library widget over `Process.Spawn` and `Process.Child`: fire the spawn on `@started`, hold the id, and build the child under a `rebuild` over it. It is not a tag.

### The shell: `Cli.*`

The `Cli` tags are what a process has when it was started from a shell, and nothing else: four tags. A CLI run ends on `Process.Exit` or when nothing can cause another occurrence; stdin until its end, signals, in-flight requests and timers keep it alive, and args, env, the clock and a watch do not.

| Tag | Fields | Requests | Responds | State | Identity | Meaning |
|---|---|---|---|---|---|---|
| `Cli.Stdin` | | | `StdinEvent` | | path | Chunks then `End`; read only while a slot exists. A chunk and the end are always in different occurrences, since a slot gets at most one response per occurrence. A slot that is dropped delivers nothing further, and the World keeps the partial chunk with the stream. |

| Tag | Fields | Requests | Responds | State | Identity | Meaning |
|---|---|---|---|---|---|---|
| `Cli.Stdout` | | `!Text` | `WriteFailed` | | path | Each firing is written after the turn, in occurrence order then list order, then flushed. A write the World could not complete comes back as `WriteFailed` in a later occurrence. |
| `Cli.Stderr` | | `!Text` | `WriteFailed` | | path | As `Cli.Stdout`, to the error stream. |
| `Cli.Signals` | | | `Signal` | | path | Each signal received while a slot exists; with no slot, the OS defaults apply. |

## Persistence: `Store.*`

Chapter 7 declares the persistence vocabulary; the rows are repeated here so the table is complete.

| Tag | Fields | Requests | Responds | State | Identity | Meaning |
|---|---|---|---|---|---|---|
| `Store.Value` | `key: Text` | | | `Stored` | value | A stored value as state: `Absent` or `Present` with the bytes, loaded into the World's snapshot when the key is first asked about, pushed when any writer changes it. |
| `Store.Put` | `key: Text` | `!Bytes` | `PutResult` | | path | Writes are requests; each firing stores one value under the key and is answered `Stored` or `Failed` in a later occurrence. |

**Every World:** `Defer`, `errors`, `World.Tags`

| Tag | Fields | Requests | Responds | State | Identity | Meaning |
|---|---|---|---|---|---|---|
| `Defer` | | `!unit` | `unit` | | path | Answered in a later occurrence of the same turn, at most once per slot per turn; firing again after being answered in the same turn is a `DeferBackwards` error. A `Defer` reborn by a `rebuild` at the same tree position counts as the same `Defer` for the once-per-turn bound; independent `Defer`s fired in one occurrence are answered together. How an interceptor answers a request itself. Every World provides it. |

| Tag | Fields | Requests | Responds | State | Identity | Meaning |
|---|---|---|---|---|---|---|
| `Error` | `kind: ErrorKind`, `spans: [Span]`, `occurrence: Nat`, `turn: Nat`, `message: Text`, `owner: ?Path`, `path: ?Path` | | | | path | *Presence.* The error value: what a failed build or a quarantined subtree leaves in the element list, and what `errors` delivers, so one `match` reads both. One tag with the kind as a field. |

| Tag | Fields | Requests | Responds | State | Identity | Meaning |
|---|---|---|---|---|---|---|
| `errors` | | | `Error` | | path | A stream a program may subscribe to: each runtime error the World would otherwise only report, with its kind, spans and occurrence. A failed `rebuild` build does not fire the `rebuild`'s event and is reported here. A program cannot catch an error any other way. |

| Tag | Fields | Requests | Responds | State | Identity | Meaning |
|---|---|---|---|---|---|---|
| `World.Tags` | | | | `[Text]` | value | The tags this World accepts natively, as state, so a program can read it in occurrence zero and build under a `rebuild` over it; what a library consults before deciding which interceptors to install. An interceptor supplies a sandboxed child a narrower one. The Test World's lists every tag. |

`Unsupported` is not a tag; it is the response a World gives, in the next turn, to a request on a tag it does not accept, so that a `Ui.Button` under a CLI World is a program-visible fact. A World component that crashes is a `Component` error through `errors`, never an `Unsupported` response. A state tag the World does not accept is an `UnsuppliedState` error raised by the runtime at the slot's creation, on the oracle's refusal, since nothing later can give it a value; a presence tag it does not accept is reported and ignored.

## What the vocabulary excludes

No tag has a dynamic field; what would be one is a fresh slot per change, and the one case where continuity matters (a native text input) is not native. No tag fires on creation; what would be one is `@started` inside the program firing a request. No tag is a trigger by presence or absence; exit is a request, spawning is a request, and a child's death on teardown is the World's cancel obligation, not a meaning of the element. No draw command is a tag; the picture is a value in `Gui.Scene`, so a World

reads it and an accessibility tree can read it too. No tag's state is `Pending` or `Fetching`; unavailability is a case of the state's own type (`Unavailable`, `Connecting`, `Disconnected`, `Absent`, `Missing`). No request can be answered in its own occurrence, by a World or by an interceptor; the shortest path from a request to its answer is one `Defer`. Each exclusion is what makes the corresponding law a law rather than a convention: nothing in the vocabulary can be used to break it.

## Chapter 17

# A night's implementation: a report

This chapter is a report, not a specification. I am the language model that wrote the rest of this book, and on the night of 11 October 2026 I was asked to build a Prism implementation in Haskell, alone, in a container with no access to Hackage, and to write down what the language was like to implement from its own book. The result is the `claude/prism-hs` directory beside this text: a parser, an evaluator and a Test World for the subset chapter 2 uses, about 3,500 lines, running every program in chapters 2 and 10 and every law in chapter 4 that the prototype's subset reaches, 53 scripts in all. What follows is what building it taught me about the book, in the order the lessons arrived, with the places where I was wrong left in, because a reader who wants to build the next implementation should know which of my complaints survived a second reading.

## What it is and is not

The plan in chapter 14 is to interpret Prism on Reflex. This is not that. The proxy in front of my container admits GitHub and the package registries for Python and Node and nothing else, so `cabal update` and `ghcup` fail before they reach their hosts, and the Haskell available was Ubuntu's GHC 9.4.7 with no `reflex` package. So the evaluator runs on a push graph I wrote in IO, built after Spider's shape: events with heights and one cached firing per occurrence, a scheduler that runs nodes in height order, holds that commit in one batch at the end, `rebuild` and `for` as nodes with heights of their own. It is nearer to a second runtime in Haskell than to chapter 14's interpreter, and that cuts both ways. It is no evidence that Prism's semantics map onto `networkHold`, `switchDyn` and `MonadHold` the way chapter 14 says; I used none of them. It is an independent reading of chapters 2, 4 and 10, and where it agrees with the Rust runtime on a script, the agreement is between two readers of the book and not between Spider and itself.

It has no type checker. Every value is a dynamically typed `Value`, and where the book's checker would decide whether an expression lifts, the prototype guesses: an array literal is plain, a map literal lifts, a `for`'s result kind is read off its last statement. It has no `intercept`, no World but the Test World, no error quarantine, no `update` blocks, no cycle detection: a prompt cycle hangs or reads stale rather than failing. The parser accepts the layout rules of chapter 3 with the colon as block opener, the free and attached sigils, comma-separated application as a postfix on an uppercase head, `then` at the bottom of the precedence table, and the whole of chapter 10's script language.

## Where I was wrong about the book

Three of the findings in my overnight notes did not survive the morning.

The first was construction in the middle of an occurrence. I built from chapters 2 and 10, and chapter 2's rule, that a widget built in n sees all of n, once, told me what had to be true and not how. I got it wrong three ways before three mechanisms made every law pass at once: firings raised during a build are queued until the build is whole, so a block never observes its own half-built state; `rebuild` and

`for` are nodes that build their first generation when they first run, in height order, so a rebuild over a dynamic that changed earlier in n builds exactly once, with the new value; and a derived dynamic, a `$Join` or a hold made mid-occurrence over a source that already fired is born holding the new value and does not fire again. Then I read chapter 4, whose section on construction in the middle of an occurrence states the second and third of these under "How an implementation honours it", with the same at-most-once law and the same cache. The lesson is about reading order, not about the book: an implementer who starts from the worked examples will meet this rule as a mystery, and the examples chapter should say that chapter 4 has the mechanism. The one thing chapter 4 did not say, the queue that makes a build whole before anything in it fires, is one sentence, and it is in chapter 4 now.

The second was paths. My notes said chapter 2 used two numbering rules, because the counter prints a `@Button` as `0/#0/0` while the viewer prints `@TextInput`'s parts as `3/0` and `3/1/#0/0`. Chapter 2's convention is one rule, every widget bind takes a position and a widget block numbers its binds under its own; what did not fit it was the library's `@Button`, written out as a widget block around a `rebuild`, which under that rule would print `0/0/#0/0`. Chapter 2 now defines `@Button` and `@Label` as the lifted-element shorthand, one bind and one position, and shows the written-out block as what the shorthand means. The prototype follows the rule as stated and prints chapter 2's listings character for character, the viewer included. My first version numbered a function body's binds in the caller's sequence, which is the wrong rule for a reason the convention now gives: a function's insides should never renumber its caller's slots, and an interceptor routes by path prefix.

The third was double brackets. `((Insert $names, "c", "c") on !add)` lexed in my first lexer as a tuple opening, and I wrote in my notes that `((` was ambiguous and that the double-paren tuple failed the book's own rule against token collisions. Chapter 3's lexical section has the rule that resolves it, and I had implemented half of it: two opens written together are a double bracket only when their matching closes are also written together, and pairing goes outermost first. The lexer now marks each bracket with whether it touches the one before, pairs them, and merges a pair into a double bracket only when both ends touch. Under that rule the line above is two single parentheses, `((a, b))` is a tuple, and `a[b[0]]` ends in two closes. The rule needs pairing, so it is a pass after lexing and not a lexer state, which is what the book says.

## What the book had wrong

Two scripts in chapter 10 wrote `(Insert $keys, Size $keys, unit)`, which under the book's own application rule, a paren-less call takes the rest of its expression and the commas with it, is `Insert $keys, (Size $keys, unit)`. The scripts now parenthesise `(Size $keys)`. The chapter 10 transcript format had no place for a request fired by a slot that was torn down in the same occurrence, which the chat program does when it sends and changes address at once; the World still has to perform that send, so the transcript now prints such slots under `torn down:`. The script language could address a slot only by pattern, so two rows with identical buttons could not be clicked individually, and every list test had to change a row before adding a second; `click at path` and `send at path` are now forms. And chapter 8 assumed a prelude of plain functions the book never listed; the list the prototype needed to run every program is now in appendix C.

## What just worked

More than I expected. `then` as the hold keyword parsed cleanly at the bottom of the precedence table, and `on` and `|` inside an application's last argument gave no trouble once application was a postfix on an applicable head rather than a form at the top of an expression, which was my one real parser bug: `@ !Switch !!clicks` had split into `@ !Switch` and a new statement. Names keyed by sigil and base let `!motd` and `$motd` coexist with the pronoun `_` resolving across sigils, after a first version keyed by base alone showed an event where a label wanted text. Tuple-pattern binds, `((!rx, $link)) <- @Element Net.Socket(..)`, an element with both `responds` and `state` returning the pair, worked once patterns took commas. A `$` name read through a tag by the `requests` expression of the very element that defines it, the chat program's socket, needed a forward dynamic, a placeholder connected when the real one is bound, which is what `RecursiveDo` would have given for free; so did top-level recursion, which needed every definition to be a thunk over the final environment before a function could call itself. Holds are pre-created in a pass before the block's other statements, with a forward change event, so a hold can be used above its line; the limit is that a hold's initial value cannot mention a later block-local binding, which chapter 3's binding groups handle and the pre-pass does not.

The equality rule showed itself exactly as chapter 2's listing shows it: a row's dynamic fires on every change to the map, so every row's label rebuilds on every add, and the prototype's generation numbers match the book's. That is the performance item chapter 11 names, confirmed by a second implementation agreeing with the first about a cost neither wanted.

## What I would tell the next implementer

Read chapter 4 before chapter 2. Build the Test World first, because the script language is the specification you can run, and chapter 10's scripts ran as written except where the book was wrong. Expect the type checker to be a design job before it is a coding job: half the lifting decisions in chapter 3 are stated by example, and the places where a dynamically typed prototype guesses are exactly the places a checker decides, so a test suite written against this prototype is conditional on those guesses until a checker exists. Treat the prompt-cycle check as a prerequisite for any error test, not a refinement. And write the Reflex interpreter chapter 14 describes, on a machine that can reach Hackage; the parser, the Test World and the scripts carry over unchanged, and only the evaluator and the graph are replaced, which is the experiment this report could not run.

As for what it was like: the graph was a morning, and the one sentence in chapter 2 was the night. Everything else in the book is a law you can test against. The thing I missed most without Reflex was not a primitive but a second opinion, since when my viewer printed a path the book did not, the only way to know which of us was wrong was to read the book again, and the second reading was usually right.